Applying Tidy Finance with Python to Vietnam

Mike

2026-02-01

Table of contents

Preface **3**
Motivation . . . 3
Why Emerging Markets Require Different Empirical Infrastructure . . . 4
Reproducibility as a Research Design Principle . . . 4
Vietnam as a Case, Not an Exception . . . 5
Data Access . . . 5
Contribution and Audience . . . 6
Structure of the Book . . . 6

I Vietnam Financial Markets, Institutions, and Data **7**

1 Institutional Background and Market Structure of Vietnam's Equity Market **8**
1.1 Evolution of Vietnam's Equity Market . . . 8
1.2 Exchange Structure and Trading Mechanisms . . . 9
1.3 Listing Requirements and Firm Characteristics . . . 9
1.4 Investor Composition and Trading Behavior . . . 9
1.5 Regulatory Environment and Market Frictions . . . 10
1.6 Implications for Empirical Design . . . 10
1.7 Summary . . . 11

2 Accessing and Managing VN Financial Data **12**
2.1 Overview of Vietnamese Financial Data Sources . . . 13
2.2 Stock Market Data . . . 15
2.2.1 Historical Price Data . . . 15
2.2.2 Fundamental Data and Financial Statements . . . 15
2.2.3 Corporate Actions and Events . . . 16
2.3 Market Indices and Benchmarks . . . 16
2.3.1 Index Constituent Data . . . 16
2.4 Macroeconomic Data from Vietnamese Sources . . . 17
2.4.1 Key Macroeconomic Indicators . . . 17
2.4.2 Risk-Free Rate Approximation . . . 18
2.5 Setting Up a Database for Vietnamese Financial Data . . . 19
2.5.1 Database Schema Design . . . 19
2.5.2 Storing Data . . . 19

2.6 Querying and Updating the Database . . . 20
2.6.1 Database Maintenance . . . 21
2.7 Alternative Data Sources for Vietnamese Markets . . . 22
2.7.1 When the Lender's Decision Is Itself Information . . . 22
2.7.2 Foreign Investor Flow Data . . . 24
2.7.3 News and Sentiment Data . . . 24
2.8 Key Takeaways . . . 24

3 Datacore Data **26**
3.1 Data Access Options . . . 26
3.2 Chapter Overview . . . 26
3.3 Setting Up the Environment . . . 27
3.4 Connecting to Datacore . . . 27
3.5 Company Fundamentals Data . . . 29
3.5.1 Understanding Vietnamese Financial Statements . . . 30
3.5.2 Downloading Fundamentals Data . . . 30
3.5.3 Cleaning and Standardizing Fundamentals . . . 31
3.5.4 Creating Standardized Variables . . . 33
3.5.5 Computing Book Equity and Profitability . . . 34
3.5.6 Computing Investment . . . 37
3.5.7 Computing Total Debt . . . 38
3.5.8 Applying Filters . . . 39
3.5.9 Storing Fundamentals Data . . . 40
3.6 Stock Price Data . . . 40
3.6.1 Downloading Price Data . . . 41
3.6.2 Processing Price Data . . . 41
3.6.3 Computing Shares Outstanding and Market Capitalization . . . 42
3.6.4 Computing Returns and Excess Returns . . . 43
3.6.5 Storing Price Data . . . 52
3.7 Descriptive Statistics . . . 53
3.7.1 Market Evolution Over Time . . . 53
3.7.2 Market Capitalization Evolution . . . 54
3.7.3 Return Distribution . . . 55
3.7.4 Coverage of Book Equity . . . 57
3.8 Merging Stock and Fundamental Data . . . 58
3.9 Key Takeaways . . . 62

4 Market Microstructure in Vietnam **64**
4.1 What Is Market Microstructure? . . . 64
4.1.1 The Microstructure-Asset Pricing Interface . . . 65
4.2 Trading Architecture in Vietnam . . . 66
4.2.1 Exchange Characteristics . . . 66
4.2.2 Trading Sessions . . . 67

4.2.3 Order Types and Matching Rules . . . 68
4.2.4 Tick Size Structure . . . 68
4.2.5 Investor Composition . . . 69
4.3 Price Limits and Their Consequences . . . 69
4.3.1 Theoretical Framework . . . 69
4.3.2 Detecting Price Limit Hits . . . 70
4.3.3 Frequency of Limit Hits . . . 71
4.3.4 Volatility Spillover Test . . . 71
4.3.5 Return Autocorrelation Induced by Price Limits . . . 73
4.4 Liquidity, Thin Trading, and Zero Returns . . . 73
4.4.1 Measuring Liquidity . . . 75
4.4.2 Computing Liquidity Diagnostics . . . 75
4.4.3 Cross-Sectional Distribution of Liquidity . . . 76
4.4.4 Liquidity Distribution Across Exchanges . . . 76
4.4.5 Time Variation in Aggregate Liquidity . . . 76
4.5 Bid-Ask Spread Estimation . . . 80
4.6 Non-Synchronous Trading Bias . . . 81
4.6.1 The Problem . . . 82
4.6.2 Quantifying the Bias . . . 82
4.6.3 The Dimson Beta Correction . . . 82
4.6.4 The Scholes-Williams Estimator . . . 85
4.7 Implications for Portfolio Construction . . . 87
4.7.1 Equal-Weighted vs. Value-Weighted Returns . . . 88
4.7.2 Recommended Liquidity Filters . . . 88
4.7.3 Monthly vs. Daily Frequency . . . 88
4.8 Implications for Asset Pricing Tests . . . 88
4.8.1 Factor Model Estimation . . . 88
4.8.2 Adjusted Testing Procedure . . . 90
4.9 Summary . . . 92

5 Risk-Free Rate Construction in Vietnam 93
5.1 The Role of the Risk-Free Rate in Finance . . . 93
5.1.1 Excess Returns . . . 93
5.1.2 Factor Premiums . . . 94
5.1.3 Discount Rates and Valuation . . . 94
5.1.4 Performance Evaluation . . . 95
5.2 What Does “Risk-Free” Mean in Practice? . . . 95
5.2.1 The Ideal Proxy . . . 96
5.3 Available Proxies in Vietnam . . . 96
5.3.1 Government Bond Yields . . . 96
5.3.2 Interbank Overnight Rate . . . 97
5.3.3 SBV Policy Rates . . . 97
5.3.4 Savings Deposit Rates . . . 97

5.3.5 Summary Comparison . . . 98
5.4 Constructing the Risk-Free Rate Series . . . 98
5.4.1 Loading and Cleaning Rate Data . . . 98
5.4.2 Frequency Alignment . . . 99
5.4.3 Handling Missing Data and Structural Breaks . . . 101
5.4.4 Properties of the Constructed Series . . . 103
5.4.5 Correlation Across Proxies . . . 103
5.5 Excess Return Construction . . . 106
5.5.1 Matching Conventions . . . 106
5.5.2 Market Excess Return . . . 106
5.6 Sensitivity Analysis: How Much Does the Proxy Choice Matter? . . . 107
5.6.1 Effect on the Equity Premium . . . 108
5.6.2 Effect on Factor Premiums . . . 108
5.6.3 Effect on Alpha Estimates . . . 110
5.6.4 Effect on Valuation . . . 110
5.7 Term Structure Considerations . . . 110
5.7.1 Yield Curve Estimation . . . 110
5.7.2 Extracting the Short-Rate from the Yield Curve . . . 115
5.7.3 Duration-Matched Excess Returns . . . 116
5.8 Real vs. Nominal Risk-Free Rates . . . 116
5.9 International Comparison . . . 118
5.10 Best Practices Checklist . . . 118
5.11 Saving the Risk-Free Rate for Downstream Use . . . 119
5.12 Summary . . . 120

6 Constructing and Analyzing Equity Return Series 121
6.1 Data Access and Preparation . . . 121
6.1.1 Prerequisites for API Access . . . 121
6.1.2 Checking Your IP Address . . . 123
6.1.3 Fetching the Dataset . . . 124
6.2 Examining a Single Equity . . . 125
6.3 From Prices to Returns . . . 127
6.4 Limiting the Influence of Extreme Returns . . . 128
6.5 Distributional Features of Returns . . . 128
6.6 Expanding to a Market Cross-Section . . . 130
6.7 Aggregating Returns Across Time . . . 133
6.8 Aggregation Across Firms: Trading Activity . . . 134
6.9 Summary . . . 136

7 Compound Returns 137
7.1 Simple Returns versus Log Returns . . . 137
7.1.1 Simple (Arithmetic) Returns . . . 137
7.1.2 Continuously Compounded (Log) Returns . . . 138

7.1.3 When Do They Diverge? . . . 139
7.2 Mathematical Foundations of Compounding . . . 140
7.2.1 Geometric Mean Return . . . 140
7.2.2 Wealth Index and Drawdowns . . . 141
7.2.3 Annualization . . . 141
7.3 Data Preparation . . . 142
7.4 Method 1: Cumulative Product via GroupBy . . . 143
7.4.1 Handling Missing Returns . . . 145
7.5 Method 2: Log-Sum-Exp Approach . . . 146
7.5.1 Period-Specific Compound Returns . . . 148
7.6 Method 3: Iterative Compounding with Retain Logic . . . 150
7.6.1 Comparison of Missing Value Treatments . . . 152
7.7 Method 4: Rolling Compound Returns . . . 153
7.7.1 Rolling Window via Log Returns . . . 153
7.7.2 Verifying Rolling Returns . . . 157
7.8 Delisting Returns and Survivorship Bias . . . 158
7.8.1 The Vietnamese Context . . . 158
7.8.2 Incorporating Delisting Returns . . . 159
7.8.3 Impact of Delisting Adjustment . . . 160
7.9 Rolling Volatility Estimation . . . 160
7.9.1 24-Month Rolling Volatility . . . 160
7.9.2 Volatility and Compound Returns: The Variance Drain . . . 162
7.10 Compound Returns Around Fiscal Year Ends . . . 164
7.10.1 Aligning Returns to Fiscal Periods . . . 164
7.10.2 Buy-and-Hold Abnormal Returns versus Cumulative Abnormal Returns 167
7.11 Book Value of Equity . . . 168
7.12 Maximum Drawdown . . . 169
7.13 Putting It All Together: A Comprehensive Pipeline . . . 170
7.14 Cross-Sectional Distribution of Compound Returns . . . 174
7.15 Vietnam-Specific Considerations . . . 174
7.15.1 Price Limits and Their Effect on Compounding . . . 174
7.15.2 Foreign Ownership Limits . . . 176
7.15.3 The VN-Index and Market Benchmarks . . . 176
7.16 Performance Considerations . . . 177
7.17 Common Pitfalls and Best Practices . . . 178

II Portfolio Construction, Risk, and Empirical Mechanics 180

8 Modern Portfolio Theory 181
8.0.1 The Core Insight: Diversification as a Free Lunch . . . 181
8.0.2 The Mean-Variance Framework . . . 182

8.1 The Asset Universe: Setting Up the Problem . . . 182
8.1.1 The Two Stages of Portfolio Selection . . . 183
8.1.2 Loading and Preparing the Data . . . 183
8.1.3 Computing Expected Returns . . . 185
8.1.4 Computing Volatilities . . . 186
8.1.5 Visualizing the Risk-Return Trade-off . . . 186
8.2 The Variance-Covariance Matrix: Capturing Asset Interactions . . . 188
8.2.1 Why Correlations Matter . . . 188
8.2.2 Computing the Variance-Covariance Matrix . . . 188
8.2.3 Interpreting the Variance-Covariance Matrix . . . 189
8.3 The Minimum-Variance Portfolio . . . 190
8.3.1 Motivation: Risk Minimization as a Benchmark . . . 190
8.3.2 The Optimization Problem . . . 191
8.3.3 The Analytical Solution . . . 191
8.3.4 Implementation . . . 192
8.3.5 Visualizing the Minimum-Variance Weights . . . 192
8.3.6 Portfolio Performance . . . 193
8.4 Efficient Portfolios: Balancing Risk and Return . . . 194
8.4.1 The Investor's Trade-off . . . 194
8.4.2 Setting the Target Return . . . 195
8.4.3 The Analytical Solution . . . 195
8.4.4 Implementation . . . 196
8.4.5 Comparing the Portfolios . . . 196
8.4.6 The Role of Risk Aversion . . . 197
8.5 The Efficient Frontier: The Menu of Optimal Portfolios . . . 198
8.5.1 The Mutual Fund Separation Theorem . . . 198
8.5.2 Proof of the Separation Theorem . . . 198
8.5.3 Computing the Efficient Frontier . . . 199
8.5.4 Visualizing the Efficient Frontier . . . 199
8.6 Key Takeaways . . . 201

9 Univariate Portfolio Sorts 202
9.1 Data Preparation . . . 202
9.2 Sorting by Market Beta . . . 203
9.3 Performance Evaluation . . . 204
9.4 Functional Programming for Portfolio Sorts . . . 206
9.5 More Performance Evaluation . . . 207
9.6 Security Market Line and Beta Portfolios . . . 209
9.7 Key Takeaways . . . 213

10 Size Sorts 215
10.1 Data Preparation . . . 215
10.2 Size Distribution . . . 216

10.3 Univariate Size Portfolios with Flexible Breakpoints . . . 221
10.4 Weighting Schemes for Portfolios . . . 222
10.5 P-Hacking and Non-Standard Errors . . . 224
10.6 Size-Premium Variation . . . 225
10.7 Key Takeaways . . . 225

11 Value and Bivariate Sorts **227**
11.1 Data Preparation . . . 227
11.2 Book-to-Market Ratio . . . 228
11.3 Independent Sorts . . . 230
11.4 Dependent Sorts . . . 232
11.5 Key Takeaways . . . 234

12 Portfolio Weighting and Rebalancing **235**
12.1 Theoretical Framework . . . 236
12.1.1 Value-Weighted Portfolios . . . 236
12.1.2 Equal-Weighted Portfolios . . . 236
12.1.3 The Weighting Spectrum . . . 237
12.2 Data Construction . . . 238
12.2.1 Universe Construction and Liquidity Filters . . . 239
12.2.2 Market Capitalization Concentration . . . 241
12.3 Implementing Weighting Schemes . . . 241
12.3.1 Core Portfolio Engine . . . 241
12.3.2 Value-Weighted Portfolio . . . 245
12.3.3 Equal-Weighted Portfolio . . . 245
12.3.4 Capped Value-Weighted Portfolio . . . 246
12.3.5 Fundamental-Weighted Portfolio . . . 247
12.4 Risk-Based Weighting Schemes . . . 248
12.4.1 Covariance Estimation . . . 248
12.4.2 Minimum Variance Portfolio . . . 250
12.4.3 Risk Parity (Equal Risk Contribution) . . . 251
12.4.4 Maximum Diversification Portfolio . . . 253
12.5 Comprehensive Performance Comparison . . . 254
12.5.1 Performance Metrics . . . 255
12.5.2 Risk-Return Trade-Off . . . 257
12.6 Transaction Costs and Net-of-Cost Performance . . . 259
12.6.1 Estimating Trading Costs in Vietnam . . . 259
12.6.2 Net-of-Cost Performance . . . 261
12.6.3 Cost Erosion at Scale . . . 262
12.7 Rebalancing Frequency Analysis . . . 264
12.7.1 The Rebalancing Trade-Off . . . 264
12.7.2 Threshold-Based Rebalancing . . . 264
12.8 The Rebalancing Bonus: Decomposition . . . 267

12.9 Factor Exposure Analysis . . . 268
12.10 Machine Learning and the Implementable Efficient Frontier . . . 270
12.10.1 The Problem . . . 270
12.10.2 The Solution: Trade Toward an Aim . . . 272
12.10.3 Portfolio-ML: Learning the Aim Directly . . . 273
12.10.4 The Evidence . . . 273
12.10.5 An Independent Implementation . . . 274
12.10.6 What This Means for Vietnamese Research . . . 286
12.11 Practical Guidance for Vietnam . . . 286
12.12 Summary . . . 287

13 Missing Data and Survivorship Bias **289**
13.1 Taxonomy of Data Problems . . . 289
13.2 Data Construction . . . 291
13.3 Listing Dynamics in Vietnam . . . 292
13.3.1 The Growth of the Vietnamese Market . . . 292
13.3.2 Delisting Reasons . . . 294
13.3.3 Firm Characteristics at Delisting . . . 296
13.4 Survivorship Bias . . . 296
13.4.1 Definition and Magnitude . . . 296
13.4.2 Time-Varying Survivorship Bias . . . 298
13.4.3 Survivorship Bias in Cross-Sectional Anomalies . . . 301
13.5 Delisting Bias and Return Imputation . . . 303
13.5.1 The Shumway Correction . . . 303
13.5.2 Impact of Delisting Return Imputation . . . 305
13.6 Zero-Trading Days and Illiquidity Gaps . . . 306
13.6.1 Prevalence of Zero-Trading Days . . . 306
13.6.2 Return Measurement During Zero-Trading Periods . . . 306
13.7 Look-Ahead Bias . . . 310
13.7.1 Definition . . . 310
13.7.2 Point-in-Time Adjustment . . . 310
13.7.3 Quantifying Look-Ahead Bias in Value Strategies . . . 312
13.8 Exchange Transfers and Ticker Discontinuities . . . 314
13.8.1 The Transfer Problem . . . 314
13.9 Sensitivity Analysis Framework . . . 315
13.9.1 How Fragile Are Your Results? . . . 315
13.10 Practical Recommendations . . . 317
13.11 Summary . . . 319

14 Liquidity and Turnover Measures **321**
14.1 Theoretical Foundations . . . 321
14.1.1 Why Liquidity Matters . . . 321
14.1.2 Liquidity Dimensions . . . 322

14.2 Data Construction . . . 322
14.3 Constructing Liquidity Measures . . . 325
14.3.1 Amihud Illiquidity Ratio . . . 325
14.3.2 Zero-Return Days (Lesmond Measure) . . . 335
14.3.3 Turnover Ratio . . . 336
14.3.4 Roll Spread Estimator . . . 337
14.3.5 Corwin-Schultz High-Low Spread . . . 339
14.3.6 Quoted Bid-Ask Spread . . . 341
14.3.7 Kyle's Lambda (Price Impact Regression) . . . 342
14.4 Realized Illiquidity: A High-Frequency Refinement of Amihud . . . 343
14.4.1 Motivation . . . 343
14.4.2 Setup and Notation . . . 344
14.4.3 Assumptions . . . 344
14.4.4 The Estimator . . . 345
14.4.5 Derivation . . . 346
14.4.6 Intuition . . . 347
14.4.7 Simulation Design . . . 348
14.4.8 Replication on Vietnamese Data . . . 351
14.4.9 Practical Guidance . . . 352
14.4.10 Pitfalls . . . 353
14.5 Assembling the Liquidity Panel . . . 353
14.6 Cross-Sectional Properties of Liquidity . . . 355
14.6.1 Summary Statistics by Size Quintile . . . 355
14.6.2 Correlation Structure . . . 355
14.6.3 Principal Component Analysis of Liquidity . . . 355
14.7 Aggregate Liquidity and Market Conditions . . . 359
14.7.1 Time Series of Market Liquidity . . . 359
14.7.2 Commonality in Liquidity . . . 359
14.8 Is Liquidity Priced? . . . 361
14.8.1 Portfolio Sorts . . . 361
14.8.2 Fama-MacBeth Cross-Sectional Regressions . . . 363
14.8.3 Factor-Adjusted Liquidity Premium . . . 366
14.9 Liquidity and Transaction Cost Estimation . . . 367
14.9.1 Translating Measures to Trading Costs . . . 367
14.9.2 Strategy Implementability . . . 368
14.10 Liquidity During Market Stress . . . 370
14.10.1 Flight to Liquidity . . . 370
14.10.2 Liquidity Co-Movement with Global Risk . . . 372
14.11 Constructing a Tradeable Liquidity Factor . . . 372
14.12 Practical Guidance for Vietnam . . . 374
14.13 Summary . . . 374

15 Beta Estimation **376**
15.1 Theoretical Foundation . . . 376
15.1.1 The Capital Asset Pricing Model . . . 376
15.1.2 Empirical Implementation . . . 377
15.2 Setting Up the Environment . . . 378
15.3 Loading and Preparing Data . . . 378
15.3.1 Stock Returns Data . . . 378
15.3.2 Company Information . . . 379
15.3.3 Market Excess Returns . . . 380
15.3.4 Merging Datasets . . . 380
15.3.5 Handling Outliers . . . 381
15.4 Estimating Beta for Individual Stocks . . . 382
15.4.1 Single Stock Example . . . 382
15.4.2 CAPM Estimation Function . . . 384
15.5 Rolling-Window Estimation . . . 386
15.5.1 Motivation for Rolling Windows . . . 386
15.5.2 Rolling Window Implementation . . . 386
15.5.3 Example: Rolling Betas for Selected Stocks . . . 388
15.5.4 Visualizing Rolling Betas . . . 389
15.6 Parallelized Estimation for the Full Market . . . 391
15.6.1 The Computational Challenge . . . 391
15.6.2 Setting Up Parallel Processing . . . 391
15.6.3 Parallel Beta Estimation . . . 391
15.6.4 Storing Results . . . 393
15.7 Beta Estimation Using Daily Returns . . . 393
15.7.1 Batch Processing for Daily Data . . . 394
15.8 Analyzing Beta Estimates . . . 398
15.8.1 Extracting Beta Estimates . . . 398
15.8.2 Summary Statistics . . . 399
15.8.3 Beta Distribution Across Industries . . . 399
15.8.4 Time Variation in Cross-Sectional Beta Distribution . . . 401
15.8.5 Coverage Analysis . . . 403
15.9 Comparing Monthly and Daily Beta Estimates . . . 404
15.10Key Takeaways . . . 407

III Asset Pricing Models and Cross-Sectional Tests **408**

16 The Capital Asset Pricing Model **409**
16.1 From Efficient Portfolios to Equilibrium Prices . . . 409
16.2 Systematic versus Idiosyncratic Risk . . . 410
16.2.1 Idiosyncratic Risk: Diversifiable and Unrewarded . . . 410
16.2.2 Systematic Risk: Undiversifiable and Priced . . . 410

16.2.3 A Simple Illustration . . . 410
16.3 Data Preparation . . . 411
16.3.1 Computing Monthly Returns . . . 413
16.4 The Risk-Free Asset and the Investment Opportunity Set . . . 413
16.4.1 Adding a Risk-Free Asset . . . 413
16.4.2 Portfolio Return with a Risk-Free Asset . . . 414
16.4.3 Portfolio Variance . . . 414
16.4.4 Setting Up the Risk-Free Rate . . . 414
16.5 The Tangency Portfolio: Where Everyone Invests . . . 416
16.5.1 Deriving the Optimal Risky Portfolio . . . 416
16.5.2 The Tangency Portfolio . . . 417
16.5.3 The Sharpe Ratio and the Capital Market Line . . . 417
16.5.4 Computing the Tangency Portfolio . . . 418
16.5.5 Visualizing the Efficient Frontier with a Risk-Free Asset . . . 419
16.6 The CAPM Equation: Risk and Expected Return . . . 421
16.6.1 From Individual Optimization to Market Equilibrium . . . 421
16.6.2 The Market Portfolio . . . 421
16.6.3 Deriving the CAPM Equation . . . 421
16.6.4 Interpreting Beta . . . 422
16.7 The Security Market Line . . . 423
16.8 Empirical Estimation of CAPM Parameters . . . 424
16.8.1 The Regression Framework . . . 424
16.8.2 Alpha: Risk-Adjusted Performance . . . 425
16.8.3 Loading Factor Data . . . 425
16.8.4 Running the Regressions . . . 426
16.8.5 Visualizing Alpha Estimates . . . 427
16.9 Limitations and Extensions . . . 428
16.9.1 The Market Portfolio Problem . . . 428
16.9.2 Time-Varying Betas . . . 429
16.9.3 Empirical Anomalies . . . 429
16.9.4 Multifactor Extensions . . . 429
16.10Key Takeaways . . . 430

17 Fama-French Factors **432**
17.1 Theoretical Background . . . 432
17.1.1 The Evolution from CAPM to Multi-Factor Models . . . 432
17.1.2 The Five-Factor Extension . . . 433
17.1.3 Factor Construction Methodology . . . 433
17.2 Setting Up the Environment . . . 434
17.3 Data Preparation . . . 434
17.3.1 Loading Stock Returns . . . 434
17.3.2 Loading Company Fundamentals . . . 435
17.3.3 Constructing Sorting Variables . . . 435

17.3.4 Validating Sorting Variables . . . 438
17.3.5 Handling Outliers . . . 438
17.4 Portfolio Assignment Functions . . . 440
17.4.1 The Portfolio Assignment Function . . . 440
17.4.2 Assigning Portfolios for Three-Factor Model . . . 442
17.4.3 Validating Portfolio Assignments . . . 444
17.5 Fama-French Three-Factor Model (Monthly) . . . 445
17.5.1 Merging Portfolios with Returns . . . 445
17.5.2 Computing Value-Weighted Portfolio Returns . . . 446
17.5.3 Constructing SMB and HML Factors . . . 448
17.5.4 Computing the Market Factor . . . 449
17.5.5 Combining Three Factors . . . 450
17.5.6 Saving Three-Factor Data . . . 452
17.6 Fama-French Five-Factor Model (Monthly) . . . 452
17.6.1 Portfolio Assignments with Dependent Sorts . . . 452
17.6.2 Validating Five-Factor Portfolios . . . 453
17.6.3 Merging and Computing Portfolio Returns . . . 454
17.6.4 Constructing All Five Factors . . . 455
17.6.5 Factor Correlations . . . 458
17.6.6 Saving Five-Factor Data . . . 458
17.7 Daily Fama-French Factors . . . 459
17.7.1 Motivation for Daily Factors . . . 459
17.7.2 Loading Daily Returns . . . 459
17.7.3 Adding Market Cap for Daily Weighting . . . 460
17.7.4 Merging Daily Returns with Portfolios . . . 460
17.7.5 Computing Daily Three Factors . . . 463
17.7.6 Computing Daily Market Factor . . . 464
17.7.7 Combining Daily Three Factors . . . 465
17.7.8 Computing Daily Five Factors . . . 466
17.7.9 Saving Daily Factors . . . 469
17.8 Factor Validation and Diagnostics . . . 470
17.8.1 Cumulative Factor Returns . . . 471
17.8.2 Average Factor Premiums . . . 472
17.8.3 Comparing Monthly and Daily Factors . . . 473
17.9 Key Takeaways . . . 474

18 Momentum Strategies **476**
18.1 Theoretical Background . . . 476
18.1.1 The Momentum Effect . . . 476
18.1.2 Overlapping Portfolios and Calendar-Time Returns . . . 477
18.1.3 Theoretical Explanations . . . 477
18.1.4 Momentum in Emerging Markets . . . 478
18.2 Setting Up the Environment . . . 479

18.3 Data Preparation . . . 479
18.3.1 Loading Monthly Stock Returns . . . 479
18.3.2 Inspecting the Data . . . 480
18.3.3 Loading Factor Data . . . 482
18.3.4 Data Quality Filters . . . 482
18.4 Momentum Portfolio Construction . . . 483
18.4.1 Computing Formation Period Returns . . . 483
18.4.2 Assigning Momentum Decile Portfolios . . . 484
18.4.3 Defining Holding Period Dates . . . 487
18.4.4 Computing Portfolio Holding Period Returns . . . 488
18.4.5 Computing Equally Weighted Portfolio Returns . . . 488
18.5 Baseline Results: J=6, K=6 Strategy . . . 490
18.5.1 Summary Statistics by Momentum Decile . . . 490
18.5.2 The Long-Short Momentum Portfolio . . . 491
18.5.3 Cumulative Returns . . . 493
18.5.4 Monthly Return Distribution . . . 495
18.6 Extending to Multiple Formation and Holding Periods . . . 496
18.6.1 The J × K Grid . . . 496
18.6.2 Running the Full Grid . . . 499
18.6.3 Winners Portfolio Returns . . . 502
18.6.4 Losers Portfolio Returns . . . 503
18.6.5 Long-Short Momentum Returns . . . 504
18.6.6 Visualizing the Momentum Premium Across Specifications . . . 505
18.7 Risk-Adjusted Performance . . . 506
18.7.1 CAPM Alpha . . . 506
18.7.2 Fama-French Three-Factor Alpha . . . 507
18.7.3 Interpretation of Risk Exposures . . . 509
18.8 Momentum and Market States . . . 509
18.8.1 Conditional Performance . . . 509
18.9 Momentum Crashes . . . 512
18.9.1 Understanding Momentum Drawdowns . . . 512
18.9.2 Maximum Drawdown Analysis . . . 514
18.10 Value-Weighted Momentum Portfolios . . . 515
18.11 Daily Momentum Analysis . . . 521
18.11.1 Loading Daily Data . . . 521
18.11.2 Daily Returns of Monthly Momentum Portfolios . . . 522
18.11.3 Daily Cumulative Returns . . . 523
18.11.4 Annualized Risk Metrics from Daily Data . . . 524
18.11.5 Realized Volatility of Momentum Returns . . . 526
18.12 Saving Results to the Database . . . 527
18.13 Practical Considerations . . . 528
18.13.1 Transaction Costs . . . 528
18.13.2 Implementation Lag . . . 529

18.13.3 Survivorship Bias . . . 529
18.13.4 Small Sample Considerations . . . 529
18.14 Key Takeaways . . . 529

19 Factor Construction Principles **531**
19.1 The Factor Construction Pipeline . . . 531
19.2 Data Construction . . . 532
19.3 Step 1: Universe Definition . . . 534
19.3.1 The Universe Problem in Vietnam . . . 534
19.4 Step 2: Signal Construction . . . 536
19.4.1 Point-in-Time Accounting Data . . . 536
19.4.2 Momentum and Volatility Signals . . . 539
19.5 Step 3: Breakpoint Computation . . . 540
19.5.1 The Breakpoint Decision . . . 540
19.6 Step 4: Portfolio Formation . . . 542
19.6.1 The Generic Factor Engine . . . 542
19.6.2 Building the Core Factors . . . 547
19.7 Step 5: Sensitivity to Construction Choices . . . 550
19.7.1 Weighting: Value-Weighted vs. Equal-Weighted . . . 550
19.7.2 Breakpoint Universe: HOSE vs. All Stocks . . . 551
19.7.3 Number of Groups: 2×3 vs. 5×5 vs. Deciles . . . 551
19.7.4 Rebalancing Frequency . . . 552
19.7.5 Comprehensive Sensitivity Summary . . . 552
19.8 Factor Correlation Structure . . . 552
19.9 Factor Validation . . . 555
19.9.1 Diagnostic Checklist . . . 555
19.9.2 Monotonicity Test . . . 556
19.9.3 Spanning Tests . . . 556
19.10 Vietnamese-Specific Considerations . . . 558
19.10.1 State-Owned Enterprise Classification . . . 558
19.10.2 Foreign Ownership Limits . . . 559
19.11 Recommended Factor Specifications for Vietnam . . . 560
19.12 Summary . . . 560

20 Fama-MacBeth Regressions **562**
20.1 The Econometric Framework . . . 562
20.1.1 Intuition: Why not Panel OLS? . . . 562
20.1.2 Mathematical Derivation . . . 563
20.2 Data Preparation . . . 564
20.3 Step 1: Cross-Sectional Regressions with WLS . . . 566
20.4 Step 2: Time-Series Aggregation & Hypothesis Testing . . . 568
20.4.1 Visualizing the Time-Varying Risk Premium . . . 569

20.5 Sanity Checks . . . 571
20.5.1 Time-Series Volatility Check . . . 571
20.5.2 Correlation of Characteristics (Multicollinearity) . . . 572

21 Factor Zoo and Multiple Testing 573
21.1 The Scale of the Problem . . . 574
21.1.1 How Many Factors Exist? . . . 574
21.2 Building the Anomaly Zoo . . . 576
21.2.1 Signal Definitions . . . 576
21.2.2 Mass Factor Construction . . . 578
21.3 The Multiple Testing Problem . . . 582
21.3.1 Why Single-Test p-Values Are Misleading . . . 582
21.3.2 Two Error Rate Concepts . . . 582
21.4 Correction Methods . . . 585
21.4.1 Bonferroni Correction (FWER Control) . . . 585
21.4.2 Holm Step-Down Procedure (FWER Control) . . . 585
21.4.3 Benjamini-Hochberg Procedure (FDR Control) . . . 586
21.4.4 The Harvey-Liu-Zhu Threshold . . . 587
21.5 Comparison of Methods . . . 589
21.6 Estimating the False Discovery Proportion . . . 591
21.6.1 The Storey Approach . . . 591
21.6.2 FDP Curve . . . 592
21.7 Bootstrap-Based Multiple Testing . . . 594
21.7.1 The White Reality Check . . . 594
21.7.2 Romano-Wolf Step-Down Bootstrap . . . 596
21.8 Out-of-Sample Validation . . . 600
21.8.1 Pre-Publication vs. Post-Publication Decay . . . 600
21.9 Which Factors Survive? . . . 601
21.9.1 A Multi-Hurdle Filter . . . 601
21.10Implications for Vietnamese Factor Research . . . 603
21.11Summary . . . 605

22 Time-Series vs. Cross-Sectional Factor Tests 607
22.1 The Two Testing Frameworks . . . 608
22.1.1 Time-Series Regressions . . . 608
22.1.2 Cross-Sectional Regressions (Fama-MacBeth) . . . 608
22.1.3 Key Differences . . . 609
22.2 Data Construction . . . 609
22.3 Time-Series Tests . . . 612
22.3.1 Full-Sample Time-Series Regressions . . . 612
22.3.2 The GRS Test . . . 613
22.3.3 Model Comparison via GRS . . . 615

22.4 Cross-Sectional Tests . . . 618
22.4.1 Fama-MacBeth with Portfolio Test Assets . . . 618
22.4.2 The Shanken Correction . . . 622
22.4.3 Fama-MacBeth with Individual Stocks . . . 623
22.5 When Do the Tests Disagree? . . . 625
22.5.1 Theoretical Sources of Disagreement . . . 625
22.5.2 Empirical Comparison for Vietnam . . . 626
22.5.3 The Lewellen-Nagel-Shanken Critique . . . 626
22.6 GMM-Based Tests . . . 629
22.6.1 The Stochastic Discount Factor Framework . . . 629
22.7 Model Comparison . . . 633
22.7.1 The Barillas-Shanken Framework . . . 633
22.7.2 Comprehensive Model Scorecard . . . 635
22.8 Stepwise Factor-Model Selection . . . 635
22.8.1 The Squared-Sharpe-Ratio Spanning Duality . . . 637
22.8.2 Implementing the Stepwise Algorithm . . . 637
22.8.3 Replication: Recovering a Known Model . . . 640
22.8.4 Application to Vietnamese Factor Models . . . 643
22.9 Conditional vs. Unconditional Tests . . . 645
22.9.1 Time-Varying Betas . . . 645
22.10Practical Recommendations . . . 647
22.11Summary . . . 647

IV Firm Fundamentals, Valuation, and Corporate Signals 649

23 Financial Statement Analysis 650
23.1 From Market Prices to Fundamental Value . . . 650
23.2 The Three Financial Statements . . . 651
23.2.1 The Balance Sheet: A Snapshot of Financial Position . . . 651
23.2.2 The Income Statement: Performance Over Time . . . 652
23.2.3 The Cash Flow Statement: Following the Money . . . 653
23.2.4 Illustrating with FPT's Financial Statements . . . 653
23.3 Loading Financial Statement Data . . . 654
23.4 Liquidity Ratios: Can the Company Pay Its Bills? . . . 655
23.4.1 The Current Ratio . . . 655
23.4.2 The Quick Ratio . . . 656
23.4.3 The Cash Ratio . . . 656
23.4.4 Calculating Liquidity Ratios . . . 656
23.4.5 Cross-Sectional Comparison of Liquidity . . . 657
23.5 Leverage Ratios: How Is the Company Financed? . . . 658
23.5.1 Why Capital Structure Matters . . . 658
23.5.2 Debt-to-Equity Ratio . . . 659

23.5.3 Debt-to-Asset Ratio . . . 659
23.5.4 Interest Coverage Ratio . . . 659
23.5.5 Calculating Leverage Ratios . . . 660
23.5.6 Leverage Trends Over Time . . . 660
23.5.7 Cross-Sectional Leverage Comparison . . . 661
23.5.8 The Leverage-Coverage Trade-off . . . 662
23.6 Efficiency Ratios: How Well Are Assets Managed? . . . 664
23.6.1 Asset Turnover . . . 664
23.6.2 Inventory Turnover . . . 665
23.6.3 Receivables Turnover . . . 665
23.6.4 Calculating Efficiency Ratios . . . 665
23.7 Profitability Ratios: Is the Company Making Money? . . . 666
23.7.1 Gross Margin . . . 666
23.7.2 Profit Margin . . . 667
23.7.3 Return on Equity (ROE) . . . 667
23.7.4 The DuPont Decomposition . . . 667
23.7.5 Calculating Profitability Ratios . . . 668
23.7.6 Gross Margin Trends . . . 668
23.7.7 From Gross to Net: Where Do Profits Go? . . . 669
23.8 Combining Financial Ratios: A Holistic View . . . 670
23.8.1 Ranking Companies Across Categories . . . 671
23.9 Financial Ratios in Asset Pricing . . . 673
23.9.1 The Fama-French Factors . . . 673
23.9.2 Calculating Fama-French Variables . . . 674
23.9.3 Fama-French Factor Rankings . . . 675
23.10 Limitations and Practical Considerations . . . 677
23.10.1 Accounting Discretion . . . 677
23.10.2 Industry Comparability . . . 678
23.10.3 Point-in-Time Limitations . . . 678
23.10.4 Backward-Looking Nature . . . 678
23.10.5 Quality of Earnings . . . 678
23.11 Key Takeaways . . . 678

24 Discounted Cash Flow Analysis 680
24.1 What Is a Company Worth? . . . 680
24.1.1 Valuation Methods Overview . . . 680
24.1.2 The Three Pillars of DCF . . . 681
24.2 Understanding Free Cash Flow . . . 681
24.2.1 Why Free Cash Flow, Not Net Income? . . . 681
24.2.2 The Free Cash Flow Formula . . . 682
24.3 Loading Historical Financial Data . . . 682
24.3.1 Computing Historical Free Cash Flow . . . 683
24.3.2 Understanding the Historical Pattern . . . 685

24.4 Visualizing Historical Ratios . . . 687
24.5 Forecasting Free Cash Flow . . . 688
24.5.1 The Ratio-Based Forecasting Approach . . . 688
24.5.2 Setting Forecast Assumptions . . . 689
24.5.3 Forecasting Revenue Growth . . . 690
24.5.4 Building the Forecast . . . 691
24.6 Visualizing the Forecast . . . 692
24.7 Terminal Value: Capturing Long-Term Value . . . 697
24.7.1 The Perpetuity Growth Model . . . 698
24.7.2 Choosing the Perpetual Growth Rate . . . 698
24.7.3 Alternative: Exit Multiple Approach . . . 699
24.8 The Discount Rate: Weighted Average Cost of Capital . . . 700
24.8.1 Estimating WACC Components . . . 700
24.8.2 Using Industry WACC Data . . . 701
24.9 Computing Enterprise Value . . . 703
24.10 Sensitivity Analysis . . . 705
24.11 From Enterprise Value to Equity Value . . . 707
24.11.1 Implied Share Price . . . 708
24.12 Limitations and Practical Considerations . . . 709
24.12.1 Sensitivity to Assumptions . . . 709
24.12.2 Terminal Value Dominance . . . 709
24.12.3 Garbage In, Garbage Out . . . 709
24.12.4 Not Suitable for All Companies . . . 710
24.12.5 Complement with Other Methods . . . 710
24.13 Key Takeaways . . . 710

25 Information Content of Earnings Announcements 712
25.0.1 Two Distinct Signals: Volume vs. Volatility . . . 713
25.0.2 Why Vietnam? . . . 713
25.1 Theoretical Framework . . . 714
25.1.1 The Information Content Hypothesis . . . 714
25.1.2 Volume as a Signal of Heterogeneous Beliefs . . . 715
25.1.3 Measuring Information Content . . . 715
25.2 Data and Variable Construction . . . 716
25.2.1 Data Requirements . . . 716
25.2.2 Trading Day Calendar . . . 716
25.2.3 Announcement Date Alignment . . . 717
25.2.4 Generating Synthetic Vietnamese Data . . . 718
25.3 Single-Event Walkthrough . . . 722
25.3.1 Selecting the Event . . . 722
25.3.2 Event Window Data . . . 723
25.3.3 Visualizing the Single Event . . . 723

25.4 Full-Sample Analysis . . . 725
25.4.1 Computing Event-Time Aggregates . . . 725
25.4.2 Return Volatility Around Announcements . . . 727
25.4.3 Trading Volume Around Announcements . . . 728
25.4.4 Formal Statistical Tests . . . 730
25.4.5 Standardized Volatility Ratio . . . 731
25.5 Cross-Sectional Variation . . . 732
25.5.1 By Firm Size . . . 732
25.5.2 By Exchange (HOSE vs. HNX) . . . 734
25.5.3 Regression Analysis . . . 736
25.6 Discussion and Contemporary Perspectives . . . 738
25.6.1 The Bamber et al. (2000) Critique . . . 738
25.6.2 How Much Information Do Earnings Announcements Convey? . . . 740
25.6.3 Price Limits and Multi-Day Adjustment . . . 741
25.6.4 International Context . . . 743
25.7 Summary . . . 744

26 Accruals, Earnings Persistence, and Market Efficiency 745
26.0.1 Why Vietnam? . . . 746
26.1 Measuring Accruals . . . 747
26.1.1 The Balance Sheet Approach . . . 747
26.1.2 The Cash Flow Statement Approach . . . 747
26.1.3 Vietnamese Context . . . 748
26.2 Simulation Analysis . . . 748
26.2.1 The Model . . . 748
26.2.2 Generating a Single Firm . . . 751
26.2.3 Cross-Sectional Simulation: Persistence and Estimation Error . . . 752
26.2.4 Decomposing Persistence by Earnings Component . . . 754
26.3 Earnings Persistence in Vietnamese Data . . . 756
26.3.1 Data Preparation . . . 756
26.3.2 Simulated Vietnamese-Style Data for Demonstration . . . 758
26.3.3 Descriptive Statistics by Accrual Decile . . . 760
26.3.4 Persistence Regressions . . . 761
26.3.5 Industry-Level Persistence . . . 763
26.3.6 Persistence Visualization . . . 764
26.4 Market Pricing of Earnings Components . . . 765
26.4.1 Theoretical Framework . . . 765
26.4.2 The Abel–Mishkin Test . . . 766
26.4.3 Interpretation for Vietnam . . . 767
26.5 The Accrual Anomaly . . . 767
26.5.1 Portfolio Returns by Accrual Decile . . . 767
26.5.2 Visualizing the Anomaly . . . 769
26.5.3 Time-Series Stability . . . 770

26.6 Discussion and Contemporary Perspectives . . . 771
26.6.1 Alternative Explanations . . . 771
26.6.2 State-Owned Enterprises and Accrual Quality . . . 772
26.6.3 Methodological Considerations . . . 772
26.7 Summary . . . 773

27 Earnings Management: Detection and Measurement 774
27.0.1 Defining Earnings Management . . . 775
27.0.2 Why Vietnam? . . . 775
27.1 Models of Non-Discretionary Accruals . . . 776
27.1.1 Notation and Setup . . . 776
27.1.2 Five Models . . . 776
27.1.3 Implementation . . . 778
27.1.4 The Salkever (1976) Correction . . . 781
27.2 Type I Error Under the Null Hypothesis . . . 783
27.2.1 Experimental Design . . . 783
27.2.2 Data Generation . . . 783
27.2.3 Sample Construction . . . 786
27.2.4 Estimating Discretionary Accruals . . . 788
27.2.5 Firm-Level Regressions and Rejection Rates . . . 788
27.2.6 Results . . . 789
27.2.7 Binomial Test for Size Distortion . . . 791
27.3 Extreme Performance Firms . . . 792
27.3.1 Constructing Extreme-Performance Samples . . . 792
27.3.2 Performance Matching . . . 793
27.4 Power Analysis . . . 794
27.4.1 Artificially Introducing Earnings Management . . . 794
27.4.2 Power Functions . . . 798
27.4.3 Summary Statistics . . . 799
27.5 Vietnamese Institutional Context . . . 801
27.5.1 Channels of Earnings Management . . . 801
27.5.2 The Earnings Distribution Test . . . 802
27.5.3 Accrual-Based vs. Real Earnings Management . . . 804
27.6 Summary . . . 805

28 P/E Ratio 807
28.0.1 Technical Schema and Variable Engineering . . . 807
28.1 Firm-Specific P/E Valuation Metrics . . . 807
28.1.1 Trailing P/E and the TTM Methodology . . . 807
28.1.2 Forward P/E and Analyst Forecast Reliability . . . 808
28.1.3 Cyclically Adjusted Price-Earnings (CAPE) Ratio . . . 808
28.1.4 Unlevered P/E and the Leibowitz Framework . . . 809

28.2 Market-Level Aggregation Techniques . . . 810
28.2.1 The VN-Index: Capitalization-Weighted Aggregation . . . 810
28.2.2 The VN30 and Free-Float Adjustments . . . 810
28.2.3 Median vs. Mean Aggregation . . . 811
28.3 Implementation . . . 811
28.3.1 Data Ingestion and Processing . . . 811
28.3.2 Shiller CAPE . . . 811
28.4 Macroeconomic and Regulatory Factors in Valuation . . . 812
28.4.1 The Role of Corporate Income Tax (CIT) . . . 812
28.4.2 Macroeconomic Determinants: Inflation and Interest Rates . . . 812
28.5 Synthesis of Valuation Dynamics . . . 813
28.6 Conclusions . . . 813

29 Firm Valuation, Financial Distress, and Company Maturity 814
29.1 Required Packages . . . 814

30 Theoretical Foundations 816
30.1 Tobin's Q: Market Valuation of the Firm . . . 816
30.1.1 The Original Concept . . . 816
30.1.2 Market Value Decomposition . . . 816
30.1.3 Replacement Cost of Assets . . . 817
30.1.4 Simplified Tobin's Q . . . 817
30.1.5 Chung and Pruitt Approximation . . . 817
30.2 Altman Z-Score: Predicting Financial Distress . . . 818
30.2.1 The Original Model . . . 818
30.2.2 The Z'-Score Model for Private Firms . . . 819
30.2.3 The Z''-Score Model for Emerging Markets . . . 819
30.3 Company Age: Measuring Corporate Maturity . . . 820

31 The Vietnamese Market Context 821
31.1 Institutional Features Affecting Valuation Measures . . . 821
31.1.1 State Ownership and Equitization . . . 821
31.1.2 Foreign Ownership Limits . . . 821
31.1.3 Accounting Standards . . . 821
31.1.4 Market Microstructure . . . 822

32 Data Preparation 823
32.1 Loading and Cleaning Financial Statement Data . . . 823
32.2 Variable Definitions and Mapping . . . 826
32.3 Data Quality Checks . . . 828

33 Computing Tobin's Q 830
33.1 Book Value of Equity . . . 830

33.2 Market Value of Equity . . . 831
33.3 Simplified Tobin's Q . . . 832
33.4 Winsorizing Extreme Values . . . 834
33.5 Cross-Sectional Distribution of Tobin's Q . . . 834
33.6 Time-Series Evolution of Tobin's Q . . . 836
33.7 Tobin's Q by Industry . . . 837

34 Computing the Altman Z-Score 840
34.1 Original Z-Score for Listed Firms . . . 840
34.2 Z-Score Component Analysis . . . 842
34.3 Distribution of Risk Zones . . . 843
34.4 Comparing Z-Score Variants . . . 846

35 Computing Company Age 848
35.1 Multiple Age Proxies . . . 848
35.2 Age Distribution . . . 849
35.3 Age and the Equitization Effect . . . 850

36 Joint Analysis: Valuation, Distress, and Maturity 853
36.1 The Relationship Between Q, Z-Score, and Age . . . 853
36.2 Correlation Structure . . . 855
36.3 Cross-Sectional Regression Analysis . . . 856

37 The Complete Pipeline 859
37.1 Putting It All Together . . . 859
37.2 Exporting Results . . . 862

38 Special Topics for the Vietnamese Market 864
38.1 State Ownership and Valuation . . . 864
38.2 Exchange-Level Analysis . . . 866
38.3 Sector Heatmap: Valuation and Distress . . . 867
38.4 Handling Delisted Firms: Survivorship Bias . . . 869

39 Robustness Checks and Extensions 871
39.1 Alternative Tobin's Q Specifications . . . 871
39.2 Industry-Adjusted Measures . . . 872
39.3 Panel Regression with Fixed Effects . . . 873

40 Distress Resolution: The Regime Behind the Z-Score 876
40.1 What the Z-Score Actually Predicts . . . 876
40.2 The Subchapter V Experiment . . . 876
40.3 Reading the Evidence from Vietnam . . . 878
40.4 Threshold Rules as Research Designs . . . 879
40.4.1 Fuzzy Regression Discontinuity . . . 879

40.4.2 What Makes the Design Credible . . . 880
40.4.3 Difference-in-Differences as Intent-to-Treat . . . 881
40.4.4 Adapting the Design to Vietnam . . . 881
40.5 Implementation: A Fuzzy RDD from Scratch . . . 881

41 Practical Considerations and Limitations 892
41.1 Known Limitations of Tobin's Q in Vietnam . . . 892
41.2 Known Limitations of Altman Z-Score in Vietnam . . . 892
41.3 Recommendations for Researchers . . . 893

42 Summary 894

43 Disclosure Quality and Timing 895
43.1 Theoretical Foundations . . . 895
43.1.1 Voluntary Disclosure Theory . . . 895
43.1.2 Strategic Disclosure Timing . . . 896
43.1.3 Disclosure Quality in Emerging Markets . . . 896
43.2 Regulatory Framework . . . 897
43.2.1 Mandatory Disclosure Requirements . . . 897
43.2.2 Penalties for Non-Compliance . . . 897
43.3 Data Construction . . . 898
43.3.1 Loading Required Libraries . . . 898
43.3.2 Retrieving Disclosure Data . . . 898
43.3.3 Computing Filing Timeliness . . . 900
43.3.4 Distribution of Filing Lags . . . 901
43.4 Measuring Disclosure Quality . . . 901
43.4.1 Timeliness as a Quality Dimension . . . 901
43.4.2 Textual Quality Measures . . . 903
43.4.3 Accounting-Based Quality Proxies . . . 905
43.4.4 Composite Disclosure Quality Index . . . 908
43.5 Determinants of Disclosure Quality . . . 910
43.6 Strategic Disclosure Timing . . . 911
43.6.1 Day-of-Week Effects . . . 911
43.6.2 Announcement Congestion . . . 914
43.6.3 After-Hours and Weekend Announcements . . . 915
43.7 Market Consequences of Disclosure Quality . . . 916
43.7.1 Disclosure Quality and the Cost of Equity . . . 916
43.7.2 Disclosure Quality and Liquidity . . . 917
43.7.3 Event Study: Market Reaction to Filing Lag . . . 927
43.8 Filing Timeliness and Earnings Quality . . . 930
43.9 Disclosure Quality and Investment Efficiency . . . 930
43.10 Vietnamese Institutional Context . . . 933
43.10.1 State Ownership and Disclosure . . . 933

43.10.2 IFRS Convergence and Disclosure Quality . . . 934
43.11 Predicting Late Filings . . . 935
43.12 Summary . . . 937

44 Standardized Earnings Surprises (SUE) 940
44.1 Methodology . . . 940
44.1.1 Method 1: Seasonal Random Walk . . . 940
44.1.2 Method 2: Exclusion of Special Items . . . 941
44.1.3 Method 3: Analyst Consensus . . . 941
44.2 Data Description . . . 941
44.2.1 Visualizing the Core Data . . . 942
44.3 Implementation . . . 942
44.3.1 Python Setup and Data Loading . . . 942
44.3.2 Calculation Logic . . . 944
44.4 Results and Analysis . . . 945
44.4.1 Tabular Results (FY 2024) . . . 945
44.4.2 Visualization . . . 946
44.5 Conclusion . . . 947

45 Machine Learning Earnings Forecasts and Market Expectations 948
45.1 The Question and Why It Matters . . . 949
45.2 The Paper in Detail . . . 949
45.2.1 Data and Forecast Targets . . . 949
45.2.2 The Accuracy Measure: ML Superiority . . . 950
45.2.3 Direct and Indirect Forecasts . . . 951
45.2.4 The 3,024-Model Experiment . . . 951
45.2.5 Result 1: Most Machine Forecasts Lose to Analysts . . . 952
45.2.6 Result 2: The Best Model, and Where Its Edge Comes From . . . 953
45.2.7 Result 3: When the Machine's Edge Is Economically Large . . . 955
45.2.8 Result 4: Which Expectation Does the Market Use? . . . 956
45.3 Replication from the Authors' Data . . . 958
45.4 A Simulation That Shows Each Mechanism . . . 960
45.4.1 The Specification Grid . . . 963
45.4.2 Why Time-Series Cross-Validation . . . 966
45.4.3 Size and Horizon . . . 968
45.4.4 What the Market-Expectations Regression Identifies . . . 969
45.5 When (Not) to Use a Machine Forecast: Practical Rules . . . 971
45.6 Vietnam: Research Design and DataCore Implementation . . . 972
45.6.1 What Changes in Vietnam . . . 972
45.6.2 Implementation . . . 973
45.7 Summary . . . 978

46 Measuring Divergence of Investor Opinion 979

47 Theoretical Framework **982**
47.1 The Miller (1977) Overpricing Hypothesis . . . 982
47.2 Alternative Theoretical Perspectives . . . 983
47.3 Relevance to the Vietnamese Market . . . 984

48 Data Sources and Sample Construction **986**
48.1 Data Sources . . . 986
48.2 Sample Construction . . . 986
48.3 Corporate Action Adjustments . . . 990
48.4 Trading Calendar Construction . . . 991

49 Volume-Based DIVOP Proxies **993**
49.1 Theoretical Motivation . . . 993
49.2 Unexplained Volume (DTO) . . . 993
49.2.1 Construction Methodology . . . 993
49.2.2 Vietnam-Specific Considerations for DTO . . . 995
49.3 Standardized Unexplained Volume (SUV) . . . 996
49.3.1 Construction Methodology . . . 996
49.3.2 Interpreting the SUV Regression Coefficients . . . 998

50 Volatility-Based DIVOP Proxies **1000**
50.1 Total Return Volatility . . . 1000
50.1.1 Theoretical Motivation . . . 1000
50.1.2 Construction . . . 1000
50.2 Idiosyncratic Volatility (IVOL) . . . 1000
50.2.1 Vietnam-Specific Considerations for Volatility . . . 1002

51 Spread-Based and Liquidity DIVOP Proxies **1004**
51.1 Bid-Ask Spread (BASPREAD) . . . 1004
51.1.1 Theoretical Motivation . . . 1004
51.1.2 Construction . . . 1004
51.2 Amihud Illiquidity (ILLIQ) . . . 1004
51.2.1 Vietnam-Specific Considerations for Spread and Liquidity . . . 1006
51.3 Realized Illiquidity: A Lower-Noise ILLIQ . . . 1006
51.3.1 Shared Sampling Noise Contaminates the Correlation Matrix . . . 1007
51.3.2 What This Means for the Chapter's Results . . . 1011
51.3.3 Construction . . . 1011
51.3.4 Vietnam-Specific Considerations . . . 1012

52 Analyst Forecast Dispersion **1014**
52.1 Theoretical Motivation . . . 1014
52.2 Data Challenges in Vietnam . . . 1014
52.3 Construction Methodology . . . 1014

52.4 Scaling Considerations 1017

53 Cross-Sectional Correlations Among DIVOP Proxies 1018
53.0.1 Expected Correlation Patterns 1019

54 Descriptive Statistics and Cross-Sectional Properties 1020
54.1 Summary Statistics 1020
54.2 DIVOP by Firm Characteristics 1021
54.3 Time-Series Evolution 1021

55 Putting It All Together 1023

56 Empirical Applications 1025
56.1 Application 1: DIVOP and the Cross-Section of Returns 1025
56.2 Application 2: DIVOP and Earnings Announcements 1026
56.3 Application 3: Composite DIVOP Index via PCA 1027

57 Conclusion and Practical Recommendations 1029

V Ownership, Market Frictions, and International Exposure 1031

58 Institutional Ownership Analytics in Vietnam 1032
58.1 Institutional Ownership in Vietnam: A Distinct Landscape 1032
58.2 Data Infrastructure: DataCore.vn 1033
58.3 Data Pipeline 1036
58.3.1 Stock Price Data and Corporate Action Adjustments 1036
58.3.2 Ownership Structure Data 1043
58.4 Vietnam's Ownership Taxonomy 1047
58.4.1 The Five Ownership Categories 1047
58.5 Institutional Ownership Measures 1051
58.5.1 Ownership Ratio 1051
58.5.2 Concentration: Herfindahl-Hirschman Index 1051
58.5.3 Breadth of Ownership 1052
58.5.4 Time Series Visualization 1056
58.6 Foreign Ownership Dynamics 1056
58.6.1 Foreign Ownership Limits and the FOL Premium 1056
58.7 Institutional Trades 1065
58.7.1 Trade Inference in Vietnam 1065
58.8 Fund-Level Flows and Turnover 1070
58.8.1 Portfolio Assets and Returns from Fund Holdings 1070
58.8.2 Turnover Measures 1070
58.9 State Ownership Analysis 1074
58.9.1 Equitization and the Decline of State Ownership 1074

58.10 Modern Extensions . . . 1076
58.10.1 Network Analysis of Co-Ownership . . . 1076
58.10.2 ML-Enhanced Investor Classification . . . 1080
58.10.3 Event Study: Ownership Disclosure Shocks . . . 1082
58.11 Empirical Applications . . . 1086
58.11.1 Application 1: Foreign Ownership and Stock Returns in Vietnam . . . 1086
58.11.2 Application 2: State Divestiture and Value Creation . . . 1088
58.11.3 Application 3: Institutional Herding in Vietnam . . . 1090
58.12 Conclusion and Practical Recommendations . . . 1092
58.12.1 Summary of Measures . . . 1092
58.12.2 Data Quality Checklist for Vietnam . . . 1093
58.12.3 Comparison with US Framework . . . 1093

59 Institutional Trades, Flows, and Turnover Ratios 1095
59.1 Measuring Institutional Ownership and Trading . . . 1096
59.2 Trade Classification . . . 1096
59.3 Turnover Measures . . . 1097
59.4 Institutional Ownership in Emerging Markets . . . 1097
59.5 Net Flows and Performance Attribution . . . 1098
59.6 Flow-Induced Price Pressure and the Arithmetic of Measuring It . . . 1098
59.6.1 The Standard Measure . . . 1098
59.6.2 What the Measure Actually Contains . . . 1099
59.6.3 A Second, Independent Problem: Selection . . . 1100
59.6.4 What This Means for Vietnamese Research . . . 1100

60 Data Infrastructure 1103
60.1 Data Reader Class . . . 1104

61 Stock Price and Return Processing 1107
61.1 Price Data Extraction and Adjustment . . . 1107
61.2 Monthly and Quarterly Price Processing . . . 1110

62 Ownership Data Processing 1113
62.1 Ownership Taxonomy . . . 1113
62.2 Building the Holdings Panel . . . 1114

63 Institutional Ownership Metrics 1118
63.1 Institutional Ownership Ratio . . . 1118
63.2 Ownership Concentration: Herfindahl-Hirschman Index . . . 1119
63.3 Ownership Breadth . . . 1120
63.4 Implementation . . . 1122
63.4.1 Trade Visualization . . . 1124

64 Portfolio Assets, Flows, and Returns **1127**
64.1 Total Assets and Portfolio Returns . . . 1127
64.2 Aggregate Buys and Sales . . . 1128

65 Net Flows and Turnover Ratios **1130**
65.1 Net Flows . . . 1130
65.2 Three Turnover Measures . . . 1130
65.2.1 Turnover Summary Statistics . . . 1132

66 Foreign Ownership Analytics **1135**
66.1 FOL Utilization . . . 1135
66.2 Room Premium Regression . . . 1136

67 Complete Pipeline **1138**

68 Advanced Extensions **1141**
68.1 Herding Measures . . . 1141
68.2 Demand Persistence . . . 1142
68.3 Information Content of Trades . . . 1143

69 Empirical Applications **1145**
69.1 Application 1: Institutional Ownership Changes and Future Returns . . . 1145
69.2 Application 2: Turnover and Performance . . . 1146
69.3 Application 3: Foreign vs. Domestic Trading . . . 1147

70 Data Quality and Robustness **1150**
70.1 Common Pitfalls . . . 1150
70.1.1 Corporate Action Misadjustment . . . 1150
70.1.2 Disclosure Timing Mismatches . . . 1150
70.1.3 Name Changes and Entity Mergers . . . 1150
70.2 Validation Checks . . . 1150

71 Summary **1153**

72 Corporate Governance **1154**
72.1 Theoretical Background . . . 1154
72.1.1 The Governance-Return Nexus . . . 1154
72.1.2 Adapting the Framework to Vietnam . . . 1155
72.1.3 The Vietnamese Governance Index (VN-GIndex) . . . 1155
72.2 Data Preparation . . . 1156
72.2.1 Loading Governance Data . . . 1157
72.2.2 Computing the VN-GIndex . . . 1157
72.2.3 Distribution of the VN-GIndex . . . 1158
72.2.4 Classifying Firms: Democracy, Neutral, and Dictatorship . . . 1158

72.2.5 Loading Stock Market Data . . . 1161
72.2.6 Linking Governance and Stock Data . . . 1162
72.2.7 Computing Lagged Market Capitalization for Weighting . . . 1163
72.3 Portfolio Construction . . . 1164
72.3.1 Value-Weighted Portfolio Returns . . . 1164
72.3.2 Long-Short Portfolio: Democracy minus Dictatorship . . . 1165
72.3.3 Portfolio Characteristics Over Time . . . 1166
72.4 Empirical Results . . . 1167
72.4.1 Summary Statistics of Portfolio Returns . . . 1167
72.4.2 T-Test: Is the Long-Short Return Statistically Significant? . . . 1168
72.4.3 Cumulative Returns: The Visual Case for Governance . . . 1168
72.4.4 Rolling Performance . . . 1169
72.5 Risk-Adjusted Performance: Factor Model Analysis . . . 1169
72.5.1 The Four-Factor Model . . . 1169
72.5.2 Loading Factor Data . . . 1169
72.5.3 Merging Portfolio Returns with Factor Data . . . 1174
72.5.4 CAPM Regression . . . 1174
72.5.5 Three-Factor Fama-French Model . . . 1175
72.5.6 Four-Factor Carhart Model . . . 1175
72.5.7 Interpreting the Factor Loadings . . . 1175
72.5.8 Robustness: White Heteroskedasticity-Consistent Standard Errors . . . 1177
72.6 Deeper Analysis . . . 1177
72.6.1 Governance Quintile Portfolios . . . 1177
72.6.2 Subsample Analysis . . . 1180
72.6.3 Equal-Weighted Robustness . . . 1180
72.6.4 Factor Loading Stability: Rolling Regressions . . . 1180
72.7 Comparison with International Evidence . . . 1186
72.7.1 The US Experience . . . 1186
72.7.2 Emerging Market Context . . . 1186
72.8 Drawdown and Risk Analysis . . . 1187
72.8.1 Maximum Drawdown . . . 1187
72.9 Discussion and Interpretation . . . 1189
72.9.1 Why Might Governance Predict Returns in Vietnam? . . . 1189
72.9.2 State Ownership and the Governance Effect . . . 1189
72.9.3 Limitations . . . 1189
72.10Exercises . . . 1190
72.11Summary . . . 1191

73 Return Gap: Measuring Unobserved Actions of Fund Managers 1192
73.1 Why Return Gap Matters . . . 1192
73.2 Hidden Payments: Revenue Sharing, Distribution, and Menu Design . . . 1193
73.3 Hidden Inputs: Human Capital and Fund Skill . . . 1194
73.4 Application to the Vietnamese Market . . . 1195

74 Theoretical Framework **1196**
74.1 Decomposing Fund Returns . . . 1196
74.2 The Return Gap Measure . . . 1197
74.2.1 Gross Return Gap . . . 1197
74.2.2 Sources of Return Gap . . . 1197
74.2.3 Predictive Return Gap . . . 1197
74.3 Risk-Adjusted Performance Evaluation . . . 1198
74.3.1 CAPM Alpha . . . 1198
74.3.2 Fama-French Three-Factor Model . . . 1198
74.3.3 Carhart Four-Factor Model . . . 1198
74.3.4 Fama-French Five-Factor Model . . . 1198
74.4 Newey-West Standard Errors . . . 1199

75 Data and Sample Construction **1200**
75.1 Data Sources . . . 1200
75.2 Setting Up the Environment . . . 1200
75.3 Loading and Preparing Stock Market Data . . . 1201
75.4 Loading Fund Holdings Data . . . 1203
75.5 Loading Fund Returns and Characteristics . . . 1205
75.6 Sample Selection: Domestic Equity Funds . . . 1206

76 Computing the Return Gap **1208**
76.1 Step 1: Prepare Holdings Vintages . . . 1208
76.2 Step 2: Adjust Shares for Corporate Actions . . . 1209
76.3 Step 3: Compute Hypothetical Holdings Returns . . . 1209
76.4 Step 4: Compute Gross Fund Returns . . . 1211
76.5 Step 5: Merge and Compute Return Gap . . . 1211
76.6 Distribution of the Return Gap . . . 1212
76.7 Time Series of Cross-Sectional Return Gap . . . 1213

77 Portfolio Sorting Analysis **1215**
77.1 Forming Return Gap Decile Portfolios . . . 1215
77.2 Portfolio Returns . . . 1215
77.3 Characteristics of Return Gap Portfolios . . . 1216
77.4 Cumulative Returns of Extreme Portfolios . . . 1216

78 Risk-Adjusted Performance **1219**
78.1 Risk Factors . . . 1219
78.2 Alpha Estimation . . . 1220
78.3 Alpha Table . . . 1221
78.4 Alpha Plot . . . 1221
78.5 Long-Short Portfolio Analysis . . . 1223

79 Cross-Sectional Determinants **1225**
79.1 Fama-MacBeth Regressions . . . 1225

80 Persistence **1227**

81 Robustness Checks **1229**
81.1 Alternative Holding Periods . . . 1229
81.2 Subperiod Analysis . . . 1229
81.3 EW vs VW . . . 1229

82 Extensions Beyond the Standard Framework **1233**
82.1 Extension 1: Decomposing Return Gap by Source . . . 1233
82.2 Extension 2: Conditional Return Gap . . . 1233
82.3 Extension 3: Return Gap and Fund Flows . . . 1235
82.4 Extension 4: Return Gap and Stock Selection Skill . . . 1236
82.5 Extension 5: Double Sorts . . . 1236

83 Vietnamese Market Considerations **1238**
83.1 Institutional Features Affecting Return Gap . . . 1238
83.1.1 Foreign Ownership Limits (FOL) . . . 1238
83.1.2 Daily Price Limits . . . 1238
83.1.3 T+2 Settlement and Margin Trading . . . 1238
83.1.4 Disclosure Norms . . . 1238
83.2 Comparison with Developed Market Evidence . . . 1239

84 Conclusion **1240**

85 Price Limits and Volatility **1241**
85.1 The Vietnamese Price Limit Regime . . . 1241
85.1.1 Institutional Details . . . 1241
85.2 Prevalence of Limit Hits . . . 1244
85.2.1 Aggregate Frequency . . . 1244
85.2.2 By Market Capitalization . . . 1244
85.2.3 Consecutive Limit Days . . . 1247
85.3 Return Distribution Distortion . . . 1248
85.3.1 Censoring Mechanics . . . 1248
85.3.2 Comparing HOSE vs. HNX vs. UPCoM . . . 1249
85.4 Variance Bias from Censoring . . . 1249
85.4.1 Analytical Bias . . . 1249
85.4.2 Empirical Variance Bias by Size . . . 1252
85.5 Volatility Estimation Under Price Limits . . . 1255
85.5.1 Range-Based Estimators . . . 1255
85.5.2 GARCH Models with Censored Returns . . . 1258

85.6 Effects on Asset Pricing Tests . . . 1262
85.6.1 Beta Attenuation . . . 1262
85.6.2 Effect on Factor Premia . . . 1266
85.7 The Magnet Effect . . . 1266
85.7.1 Do Limits Attract Prices? . . . 1266
85.8 The Delayed Price Discovery Hypothesis . . . 1269
85.8.1 Volatility Spillover . . . 1269
85.8.2 Multi-Day Return Reconstruction . . . 1271
85.9 The Idiosyncratic Volatility Puzzle Under Price Limits . . . 1273
85.10Practical Recommendations . . . 1274
85.11Summary . . . 1275

86 Market Integration and Segmentation 1277
86.1 Integration in Theory . . . 1277
86.1.1 The Integrated and Segmented Benchmarks . . . 1277
86.1.2 The Segmentation Premium . . . 1278
86.2 Data Construction . . . 1278
86.2.1 Vietnam's Liberalization Timeline . . . 1281
86.3 Correlation-Based Integration Measures . . . 1282
86.3.1 Rolling Correlations . . . 1282
86.3.2 DCC-GARCH Dynamic Correlations . . . 1282
86.3.3 Asymmetric Integration . . . 1286
86.4 Factor-Based Integration Measures . . . 1289
86.4.1 The Pukthuanthong-Roll R^2 Measure . . . 1289
86.4.2 Global vs. Local Factor Pricing . . . 1291
86.5 Pricing-Error-Based Integration . . . 1292
86.5.1 The Bekaert-Harvey Approach . . . 1292
86.5.2 Composite Integration Index . . . 1294
86.6 Structural Break Detection . . . 1297
86.6.1 Bai-Perron Tests for Integration Regime Shifts . . . 1297
86.7 The Segmentation Premium for Vietnam . . . 1298
86.7.1 Cross-Sectional Evidence . . . 1298
86.8 Exchange Rate Risk and Integration . . . 1300
86.8.1 Is Currency Risk Priced? . . . 1300
86.9 Contagion vs. Interdependence . . . 1302
86.10ASEAN Peer Comparison . . . 1304
86.11Practical Implications . . . 1304
86.12Summary . . . 1306

87 Exchange Rate Dynamics 1307
87.1 Theoretical Foundations . . . 1308
87.1.1 Exchange Rate Determination . . . 1308
87.1.2 Interest Rate Parity . . . 1308

87.1.3 The Carry Trade . . . 1309
87.1.4 The Trilemma . . . 1309
87.2 The VND Exchange Rate Regime . . . 1309
87.2.1 Historical Evolution . . . 1309
87.2.2 Band Mechanics . . . 1310
87.3 Data Construction . . . 1311
87.3.1 Loading Libraries . . . 1311
87.3.2 Retrieving Exchange Rate Data . . . 1311
87.3.3 Constructing the VND/USD Time Series . . . 1312
87.3.4 Multi-Currency Panel . . . 1313
87.4 Statistical Properties of VND Returns . . . 1314
87.4.1 Return Distribution . . . 1314
87.4.2 Autocorrelation Structure . . . 1314
87.4.3 Reference Rate Adjustments and Structural Breaks . . . 1314
87.4.4 Cross-Currency Correlations . . . 1314
87.5 Interest Rate Parity Tests . . . 1318
87.5.1 Testing Covered Interest Rate Parity . . . 1318
87.5.2 Testing Uncovered Interest Rate Parity . . . 1319
87.5.3 Carry Trade Returns . . . 1321
87.6 Volatility Modeling . . . 1322
87.6.1 GARCH Estimation . . . 1322
87.6.2 Volatility Regime Identification . . . 1325
87.7 Exchange Rate Exposure of Vietnamese Firms . . . 1327
87.7.1 The Jorion Framework . . . 1327
87.7.2 Industry-Level Exposure . . . 1329
87.7.3 Time-Varying Exposure . . . 1330
87.8 Currency Hedging for International Investors . . . 1332
87.8.1 The Hedging Decision . . . 1332
87.8.2 Optimal Hedge Ratio . . . 1335
87.9 ASEAN Currency Comparison . . . 1337
87.10Exchange Rate and Equity Market Co-Movement . . . 1337
87.11Summary . . . 1340

VI Advanced Financial Modeling **1341**

88 Event Studies in Finance **1342**
88.0.1 Why Event Studies Matter . . . 1342
88.1 Literature Review and Methodological Evolution . . . 1343
88.1.1 The Classical Framework (1969-1985) . . . 1343
88.1.2 Risk Model Refinements (1992-2015) . . . 1344
88.1.3 Testing for Abnormal Returns (1976-2010) . . . 1344
88.1.4 CARs versus BHARs . . . 1345

88.1.5 Emerging Market Considerations . . . 1345
88.2 Mathematical Framework . . . 1345
88.2.1 Timeline and Windows . . . 1346
88.2.2 Normal Return Models . . . 1346
88.2.3 Aggregation: CARs and BHARs . . . 1347
88.2.4 Standardized Returns . . . 1347
88.2.5 Test Statistics . . . 1348
88.3 Python Implementation . . . 1349
88.3.1 Design Philosophy . . . 1349
88.3.2 Setup and Imports . . . 1349
88.3.3 Configuration . . . 1351
88.3.4 Step 1: Trading Calendar Construction . . . 1353
88.3.5 Step 2: Event Date Alignment . . . 1355
88.3.6 Step 3: Data Extraction and Factor Merging . . . 1356
88.3.7 Step 4: Risk Model Estimation . . . 1358
88.3.8 Step 5: Abnormal Return Computation . . . 1360
88.3.9 Step 6: Comprehensive Test Statistics . . . 1363
88.3.10 Step 7: Publication-Ready Visualization . . . 1366
88.3.11 The Master Pipeline . . . 1368
88.4 Demonstration with Simulated Data . . . 1370
88.4.1 Running the Full Pipeline . . . 1372
88.4.2 Visualizing Results . . . 1373
88.4.3 Complete Test Statistics . . . 1375
88.4.4 Running Multiple Models for Robustness . . . 1375
88.5 How to Use This Framework with Your Data . . . 1377
88.5.1 Required Data Format . . . 1377
88.5.2 Minimal Usage Example . . . 1378
88.6 Demonstration with Vietnamese Market Data . . . 1379
88.6.1 Loading the Data . . . 1379
88.6.2 Creating Sample Events . . . 1381
88.6.3 Daily Event Study: Fama-French 3-Factor Model . . . 1382
88.6.4 Visualizing Daily Results . . . 1384
88.6.5 Complete Test Statistics (Daily) . . . 1385
88.6.6 Robustness: Multiple Risk Models (Daily) . . . 1385
88.6.7 Robustness: Multiple Event Windows . . . 1385
88.6.8 Monthly Event Study: Fama-French 3-Factor Model . . . 1385
88.6.9 Daily Event Study: Fama-French 5-Factor Model . . . 1391
88.6.10 Comparing FF3 vs FF5 Estimation Quality . . . 1392
88.6.11 Event-Level Detail . . . 1392
88.6.12 Daily Abnormal Return Dynamics . . . 1392
88.6.13 Summary of Key Findings . . . 1396
88.7 Practical Recommendations . . . 1397

89 Industrial Park Anchors and Developer Value **1399**
89.1 The Paper . . . 1400
89.1.1 Question and Trade-Off . . . 1400
89.1.2 Data and Measures . . . 1400
89.1.3 Results . . . 1401
89.1.4 The Stock-Market Test . . . 1401
89.1.5 From Working Paper to Journal Article . . . 1401
89.2 From Offices to Industrial Parks . . . 1402
89.3 Hand-Collected Lease Announcements . . . 1403
89.4 How Much Can These Announcements Detect? . . . 1414
89.5 Application to Vietnamese Data . . . 1418
89.6 Extensions . . . 1423
89.7 Summary . . . 1424

90 Tail Risk and Extreme Events **1425**
90.1 Crash Risk in Equity Markets . . . 1426
90.1.1 The Left Tail Problem . . . 1426
90.1.2 Crash Risk Measures . . . 1426
90.1.3 Cross-Sectional Distribution of Crash Risk . . . 1430
90.1.4 Interpretation in Asset Pricing . . . 1432
90.1.5 Downside Tail Concentration . . . 1434
90.2 Value at Risk and Expected Shortfall . . . 1435
90.2.1 Definitions . . . 1435
90.2.2 Why Expected Shortfall Dominates VaR . . . 1435
90.2.3 Estimation Methods . . . 1436
90.2.4 VaR Backtesting . . . 1441
90.2.5 Estimation Under Limited Data . . . 1442
90.3 Extreme Value Theory . . . 1443
90.3.1 Motivation: What EVT Captures That GARCH Does Not . . . 1443
90.3.2 Tail Index Estimation . . . 1443
90.3.3 Block Maxima and the GEV Distribution . . . 1446
90.3.4 Peaks Over Threshold and the GPD . . . 1447
90.3.5 Threshold Selection . . . 1450
90.3.6 What EVT Captures That GARCH Does Not . . . 1450
90.4 Tail Dependence and Contagion . . . 1452
90.4.1 Why Correlation Breaks Down in Crises . . . 1452
90.4.2 Co-Crash Measures . . . 1452
90.4.3 System-Wide Stress Transmission . . . 1454
90.4.4 Financial Contagion Mechanisms . . . 1455
90.5 Applications to Financial Stability . . . 1458
90.5.1 Market-Wide Stress Indicators . . . 1458
90.5.2 Sectoral Fragility . . . 1460
90.5.3 Crisis Diagnostics . . . 1463

90.5.4 Anatomy of a Run: Who Exits First and What It Costs 1463
90.5.5 Self-Fulfilling Fire Sales 1466
90.6 Summary 1472

91 Corporate Finance Estimators and Identification 1473
91.1 Investment-Q Regressions 1475
91.1.1 Tobin's Q: Intuition and Theory 1475
91.1.2 Measurement Issues 1476
91.1.3 Interpretation Under Market Frictions 1478
91.1.4 Limitations in Emerging Markets 1480
91.1.5 The Erickson-Whited Measurement Error Correction 1483
91.1.6 Marginal q from Market Values 1485
91.2 Cash Flow Sensitivity of Investment 1492
91.2.1 The Financing Constraints Hypothesis 1492
91.2.2 The FHP-KZ Debate 1493
91.2.3 Constraint Indices 1493
91.2.4 Split-Sample CFSI Tests 1495
91.2.5 Alternative Specifications 1497
91.3 Financing Choice Models 1498
91.3.1 Capital Structure Determinants 1498
91.3.2 Pecking Order Tests 1501
91.3.3 Market Timing Measures 1503
91.4 Payout Policy Estimators 1505
91.4.1 Dividend Smoothing 1505
91.4.2 Smoothing Heterogeneity: SOEs vs. Private Firms 1506
91.4.3 Share Repurchases 1509
91.4.4 Agency and Signaling Interpretations 1509
91.5 Agency Cost Proxies 1511
91.5.1 Ownership Concentration and Agency Problems 1511
91.5.2 Free Cash Flow Measures 1512
91.5.3 Monitoring Mechanisms and Governance Variables 1513
91.5.4 Agency Costs and Firm Value 1514
91.6 Linking Corporate Decisions to Returns 1516
91.6.1 Investment-Based Anomalies 1516
91.6.2 Financing Anomalies 1519
91.6.3 Valuation Implications: Fama-French Factor Regressions 1520
91.7 Summary 1522

92 Structural Models in Finance 1525
92.1 What Structural Estimation Means in Finance 1526
92.1.1 Reduced Form Versus Structural 1526
92.1.2 Identification Through Economic Primitives 1527
92.1.3 Tradeoffs Between Realism and Tractability 1527

92.2 Demand Estimation for Financial Assets . . . 1528
92.2.1 The Demand System Approach . . . 1528
92.2.2 Demand Elasticity Estimation . . . 1529
92.2.3 Price Pressure and Market Impact . . . 1533
92.2.4 Demand Inelasticity and Its Implications . . . 1535
92.3 Structural Models of Trading Strategies . . . 1537
92.3.1 Optimal Execution: The Almgren-Chriss Framework . . . 1537
92.3.2 Estimating Market Impact from Transaction Data . . . 1539
92.3.3 Transaction Cost Decomposition . . . 1542
92.4 Auction Models in Primary Markets . . . 1545
92.4.1 The IPO Allocation Problem . . . 1545
92.4.2 Book Building vs. Auction Mechanisms . . . 1545
92.4.3 Structural Estimation of Information Asymmetry . . . 1546
92.4.4 Counterfactual: Welfare Under Alternative Mechanisms . . . 1549
92.5 Limit Order Book Models . . . 1551
92.5.1 Order Submission as a Strategic Choice . . . 1551
92.5.2 The Glosten-Milgrom Model . . . 1552
92.5.3 PIN: Probability of Informed Trading . . . 1552
92.5.4 PIN and Asset Pricing . . . 1556
92.5.5 Estimation Challenges in Limit Order Book Models . . . 1558
92.6 When Structural Models Are Worth It . . . 1559
92.6.1 Decision Framework . . . 1559
92.6.2 Data Requirements . . . 1559
92.6.3 Computational Cost . . . 1560
92.6.4 Interpretation Risks . . . 1561
92.6.5 Journal Expectations . . . 1561
92.7 Summary . . . 1562

VII Alternative Data and AI for Finance **1563**

93 Textual Analysis **1564**
93.1 Why Textual Analysis for Vietnamese Finance? . . . 1565

94 Literature Review **1566**
94.1 Textual Analysis in Finance . . . 1566
94.2 NLP for Vietnamese Language . . . 1567

95 Data: Vietnamese Listed Firms from DataCore.vn **1569**
95.1 Constructing the Universe . . . 1569
95.2 Retrieving Business Descriptions . . . 1570
95.3 Retrieving Annual Report Text . . . 1571

96 Text Preprocessing for Vietnamese **1573**
96.1 Vietnamese Word Segmentation . . . 1573
96.2 Full Text Cleaning Pipeline . . . 1574
96.3 English Text Cleaning . . . 1576

97 Document Representation: Bag-of-Words and TF-IDF **1578**
97.1 Bag-of-Words Representation . . . 1578
97.2 TF-IDF Weighting . . . 1579

98 Topic Modeling **1582**
98.1 Latent Dirichlet Allocation (LDA) . . . 1582
98.2 BERTopic: Neural Topic Modeling . . . 1584

99 Financial Sentiment Analysis **1586**
99.1 Dictionary-Based Approach . . . 1586
99.2 Transformer-Based Sentiment Classification . . . 1587

100 Text-Based Firm Similarity and Peer Identification **1591**
100.1 Cosine Similarity on TF-IDF Vectors . . . 1591
100.2 Embedding-Based Similarity . . . 1592
100.3 Doc2Vec . . . 1593

101 Deep Learning Approaches **1597**
101.1 PhoBERT Embeddings for Financial Text . . . 1597
101.2 Large Language Model Applications . . . 1598
101.2.1 Zero-Shot Financial Classification . . . 1598
101.2.2 Structured Information Extraction . . . 1599
101.2.3 Automated ESG Scoring . . . 1600

102 Empirical Applications **1602**
102.1 Textual Sentiment and Stock Returns . . . 1602
102.2 Text-Based Industry Classification . . . 1604
102.3 Measuring Textual Similarity Changes Around Corporate Events . . . 1605
102.4 Executive Social Media and Stock Prices . . . 1608
102.4.1 Production Data Architecture . . . 1610
102.4.2 Replication Sandbox: Simulated Executive Posts . . . 1612
102.4.3 Classifying Financial Content . . . 1615
102.4.4 Measuring Executive-Firm Content Similarity . . . 1616
102.4.5 Market Reactions to Executive Posts . . . 1619
102.4.6 From Replication to Production Monitoring . . . 1621
102.5 Earnings Call Graphs: Structure-Aware Document Models . . . 1623
102.5.1 A Simulation of Conversations with Structure . . . 1624
102.5.2 From Transcript to Graph . . . 1625
102.5.3 Does Structure Add Forecasting Power? . . . 1627

102.5.4 Production Application with DataCore.vn . . . 1628
102.5.5 Vietnam-Specific Research Design . . . 1629
102.6 Earnings Calls in Regulatory Review . . . 1630
102.6.1 Vietnam-Specific Research Design . . . 1632

103 Method Comparison and Best Practices **1634**

104 Conclusion **1637**

105 Image and Visual Data in Finance **1638**
105.1 Foundations: From Pixels to Financial Signals . . . 1639
105.1.1 Image Representation . . . 1639
105.1.2 Transfer Learning for Finance . . . 1640
105.2 Satellite and Geospatial Imagery . . . 1643
105.2.1 Economic Activity from Space . . . 1643
105.2.2 Application 1: Nighttime Luminosity and Provincial GDP . . . 1644
105.2.3 Application 2: Satellite Imagery for Sector Nowcasting . . . 1647
105.2.4 Satellite Feature Extraction with CNNs . . . 1650
105.3 Document Image Analysis . . . 1653
105.3.1 The Vietnamese Filing Problem . . . 1653
105.3.2 OCR for Vietnamese Financial Documents . . . 1653
105.3.3 Table Extraction from Financial Statements . . . 1655
105.3.4 Layout-Aware Document Understanding . . . 1658
105.4 Chart and Figure Digitization . . . 1660
105.4.1 Motivation: Why Images Carry Financial Information . . . 1660
105.4.2 Chart Type Classification . . . 1661
105.4.3 Line Chart Digitization . . . 1662
105.5 Visual Sentiment Analysis . . . 1664
105.5.1 Image Sentiment in Financial News . . . 1664
105.5.2 CNN-Based Visual Sentiment . . . 1664
105.5.3 Vision-Language Models for Financial Image Understanding . . . 1665
105.6 Multimodal Fusion: Combining Image and Text . . . 1668
105.6.1 Why Multimodal? . . . 1668
105.6.2 Practical Considerations for Vietnamese Markets . . . 1674
105.7 Summary . . . 1675

106 Networks and Graphs in Finance **1676**
106.1 Graph Theory Foundations for Finance . . . 1677
106.1.1 Representing Financial Data as Graphs . . . 1677
106.1.2 Key Graph Metrics . . . 1678
106.1.3 The Adjacency Matrix and Its Spectral Properties . . . 1681
106.2 Ownership and Control Networks . . . 1683
106.2.1 Vietnamese Ownership Structure . . . 1683

106.2.2 Ultimate Ownership and Control Chains . . . 1685
106.2.3 Ownership Centrality and Firm Value . . . 1687
106.2.4 Business Group Detection . . . 1689
106.3 Board Interlock Networks . . . 1691
106.3.1 Construction . . . 1691
106.3.2 Interlocks and Return Co-Movement . . . 1694
106.4 Supply Chain and Trade Networks . . . 1695
106.4.1 Constructing the Supply Chain Graph . . . 1695
106.4.2 Customer Momentum . . . 1696
106.4.3 Network Propagation: Shock Diffusion Through Supply Chains . . . 1698
106.5 Correlation and Co-Movement Networks . . . 1700
106.5.1 Construction from Return Data . . . 1700
106.5.2 The Diebold-Yilmaz Connectedness Framework . . . 1702
106.6 Interbank and Financial Contagion Networks . . . 1705
106.6.1 The Interbank Market as a Network . . . 1705
106.6.2 Cascading Default Simulation . . . 1706
106.7 Social Connectedness Between Places . . . 1709
106.7.1 Social Networks as Conduits for Credit . . . 1710
106.7.2 Shift-Share Logic and Inference . . . 1711
106.7.3 Social Connectedness in Vietnam . . . 1712
106.8 Graph Neural Networks for Financial Prediction . . . 1713
106.8.1 From Graphs to Predictions . . . 1713
106.8.2 Graph Convolutional Networks (GCN) . . . 1714
106.8.3 Graph Attention Networks (GAT) . . . 1714
106.8.4 GNN for Cross-Sectional Return Prediction . . . 1718
106.8.5 Temporal Graph Networks . . . 1721
106.8.6 GNN for Credit Risk and Fraud Detection . . . 1724
106.9 When to Use Network Methods . . . 1725
106.9.1 Decision Framework . . . 1725
106.9.2 Data Requirements and Vietnamese Availability . . . 1726
106.10 Summary . . . 1726

107 Multimodal Models in Finance **1728**
107.1 Foundations of Multimodal Learning . . . 1729
107.1.1 The Information Structure of Financial Data . . . 1729
107.1.2 Taxonomies of Fusion . . . 1730
107.2 Representation Alignment . . . 1732
107.2.1 The Alignment Problem . . . 1732
107.2.2 Contrastive Alignment: CLIP for Finance . . . 1732
107.2.3 Projection Alignment for Arbitrary Modalities . . . 1734
107.3 Fusion Architectures for Return Prediction . . . 1736
107.3.1 Unimodal Encoders . . . 1736
107.3.2 Early Fusion . . . 1739

107.3.3 Late Fusion . . . 1740
107.3.4 Cross-Attention Fusion . . . 1742
107.3.5 Comparison Experiment . . . 1745
107.4 Handling Missing Modalities . . . 1754
107.4.1 The Missing Modality Problem . . . 1754
107.4.2 Strategies for Missing Modalities . . . 1754
107.5 Multimodal Document Understanding . . . 1757
107.5.1 Annual Report as a Multimodal Object . . . 1757
107.5.2 Extracting Structured Financials from Multimodal Reports . . . 1760
107.6 Multimodal Earnings Surprise Model . . . 1761
107.6.1 Architecture . . . 1761
107.6.2 Modality Importance Analysis . . . 1765
107.7 Large Multimodal Models for Financial Analysis . . . 1766
107.7.1 Prompting Vision-Language Models . . . 1766
107.7.2 Retrieval-Augmented Multimodal Analysis . . . 1770
107.8 Evaluation and Deployment Considerations . . . 1773
107.8.1 Evaluation Protocol for Multimodal Financial Models . . . 1773
107.8.2 Computational Budget . . . 1773
107.9 Summary . . . 1774

108 Conclusion **1775**
108.1 What you should take away . . . 1775
108.1.1 Reproducibility is an identification strategy . . . 1775
108.1.2 Vietnam rewards “microstructure humility” . . . 1775
108.2 A reproducibility checklist you can actually use . . . 1776

109 Closing perspective **1777**

References **1778**

Preface

Attribution

This book is an independent derivative work inspired by reproducible research principles developed in **Tidy Finance**. It is not affiliated with, or officially provided by the creators of the original Tidy Finance books. All content, code, and empirical applications are original and tailored to the Vietnamese market.
This work builds directly on the methodological foundation established in:

- Scheuch, C., Voigt, S., & Weiss, P. (2023). *Tidy Finance with R*. Chapman and Hall/CRC. https://www.tidy-finance.org/r/ (Scheuch, Voigt, and Weiss 2023)
- Scheuch, C., Voigt, S., Weiss, P., & Frey, C. (2024). *Tidy Finance with Python*. Chapman and Hall/CRC. https://www.tidy-finance.org/python/ (Scheuch et al. 2024)

We gratefully acknowledge the Tidy Finance authors for developing an open, reproducible approach to empirical finance that made this market-specific adaptation possible.

Motivation

Empirical finance has undergone a fundamental transformation over the past two decades. Advances in computational capacity, open-source statistical software, and data availability have reshaped how financial research is conducted, evaluated, and disseminated. Increasingly, credible empirical work is expected to be transparent, replicable, and extensible, with results generated through scripted workflows rather than manual intervention. Reproducibility, defined as the ability for independent researchers to regenerate empirical results using the same data and methods, has thus become a core norm in modern financial economics.

Despite this progress, the adoption of reproducible research practices has been uneven across markets. In developed financial systems, particularly those with long-established databases and standardized reporting regimes, reproducible empirical workflows are now commonplace. In contrast, research on emerging and frontier markets frequently relies on fragmented datasets, undocumented data cleaning procedures, and implicit institutional assumptions that are difficult to verify or extend. As a result, empirical findings in these markets are often fragile, non-comparable across studies, and costly to update as new data become available.

This book addresses that gap.

It develops a reproducible empirical finance framework designed explicitly for emerging and frontier markets, using Vietnam as a primary empirical case. Rather than adapting developed-market research pipelines post hoc, the book begins from the institutional and data realities of a fast-growing, retail-dominated, regulation-intensive market and builds methodological solutions accordingly. The objective is not merely to analyze Vietnam's financial markets, but to demonstrate how reproducible finance principles, as developed in the Tidy Finance framework, can be extended, stress-tested, and refined in environments characterized by data scarcity, institutional heterogeneity, and rapid structural change.

Why Emerging Markets Require Different Empirical Infrastructure

Much of modern empirical finance implicitly assumes the existence of stable, high-frequency, institutionally harmonized datasets. These assumptions are rarely stated, yet they are deeply embedded in standard research designs: survivorship-free security histories, consistent accounting standards, unrestricted trading mechanisms, and deep institutional liquidity.

Emerging and frontier markets challenge each of these assumptions.

In Vietnam, as in many comparable economies, equity markets exhibit binding daily price limits, episodic trading halts, concentrated state ownership, and a predominance of retail investors. Financial disclosures reflect local accounting standards and evolving regulatory frameworks. Corporate actions are frequent, inconsistently documented, and occasionally revised ex post.

These characteristics are not inconveniences to be eliminated through aggressive data cleaning. They shape return dynamics, risk premia, factor construction, and statistical inference itself. An empirical framework that ignores these institutional features risks producing results that are internally inconsistent or externally misleading. A reproducible approach for emerging markets must therefore encode institutional context directly into data schemas, transformation logic, and modeling choices.

Reproducibility as a Research Design Principle

In this book, reproducibility extends beyond the narrow notion of code availability. It is treated as an organizing principle governing the entire empirical research lifecycle.

First, all datasets are constructed from raw inputs through documented, deterministic transformations, ensuring clear data provenance. Second, empirical methods are implemented in a manner that makes modeling assumptions explicit and modifiable. Third, results are generated through scripted pipelines rather than interactive analysis, guaranteeing that updates to

data or parameters propagate consistently throughout the analysis. Finally, empirical designs are modular, allowing researchers to substitute markets, sample periods, or variable definitions without rewriting entire workflows.

This approach draws methodological inspiration from the broader reproducible research movement in economics and finance (e.g., Gentzkow and Shapiro 2014; Vilhuber 2020), while deliberately extending it beyond its original institutional and data environment. The goal is not to reproduce existing studies, but to enable new ones—particularly those that would otherwise be impractical due to fragmented data and institutional complexity.

Vietnam as a Case, Not an Exception

Vietnam serves as the central empirical case throughout the book, but it is not treated as an idiosyncratic exception. Instead, it is presented as a representative example of a class of markets that occupy an intermediate position between frontier and emerging status: large enough to sustain active equity trading, yet still evolving in terms of regulation, disclosure quality, and investor composition.

By grounding methodological development in Vietnam's market structure, the book aims to produce insights that generalize to other contexts, including Southeast Asia, South Asia, Sub-Saharan Africa, and parts of Latin America. Each empirical chapter emphasizes which components are market-specific and which are portable, encouraging readers to adapt the framework rather than adopt it wholesale.

Data Access

The empirical analyses in this book rely on Vietnamese equity market data provided by Datacore. To ensure reproducibility while respecting data licensing constraints, we provide the following resources:

- **Sample datasets**: A subset of anonymized data is available in DataCore's Sample Dataset for readers to run example code.
- **Data construction scripts**: All scripts used to clean and transform raw data are fully documented and available in the repository.
- **Replication guidance**: Readers with access to Vietnamese market data from commercial providers can use our scripts to construct equivalent datasets.

For questions about data access or replication, please contact the author.

Contribution and Audience

This book makes three primary contributions.

First, it proposes a reproducible empirical finance framework explicitly designed for emerging and frontier markets, integrating institutional detail into data construction and model design. Second, it provides original empirical evidence on asset pricing, liquidity, and market microstructure in Vietnam using consistently constructed datasets. Third, it provides publication-ready, end-to-end research workflows suitable for academic research, policy analysis, and applied finance.

The intended audience includes graduate students in finance and economics, academic researchers studying non-developed markets, and practitioners interested in the systematic analysis of emerging-market equities. Familiarity with basic asset pricing theory and statistical programming is assumed, but no prior experience with Vietnam or similar markets is required.

Structure of the Book

The chapters that follow progress from data infrastructure to empirical application. The book begins with an introduction to the data sources and infrastructure used throughout, followed by chapters on institutional context, data construction, and reproducible workflow design. Subsequent chapters develop asset pricing tests, liquidity measures, and market microstructure analyses tailored to Vietnam's equity market. Each chapter is designed to be self-contained, yet all are linked through a common data and code architecture to ensure internal consistency.

The book concludes by reflecting on the broader implications of reproducible empirical finance for emerging markets research and by outlining directions for future methodological and empirical work.

Part I

Vietnam Financial Markets, Institutions, and Data

1 Institutional Background and Market Structure of Vietnam's Equity Market

Empirical analysis of financial markets is inseparable from the institutional context. Market design, regulatory constraints, ownership structure, and investor composition shape observed prices, volumes, and returns. In developed markets, many of these features are sufficiently stable and standardized that they fade into the background of empirical research. In emerging markets, by contrast, institutional features are often first-order determinants of empirical outcomes.

This chapter provides the institutional foundation for the empirical analyses developed later in the book. It describes the structure of Vietnam's equity market, the regulatory environment governing trading and disclosure, and the characteristics of listed firms and investors. Rather than offering a purely descriptive account, the discussion emphasizes how institutional features map directly into data construction choices, modeling assumptions, and interpretation of empirical results.

1.1 Evolution of Vietnam's Equity Market

Vietnam's modern equity market is relatively young. Formal stock exchanges were established only in the early 2000s, as part of broader economic reforms aimed at transitioning from a centrally planned system toward a market-oriented economy. Since then, market capitalization, trading volume, and the number of listed firms have grown rapidly, albeit unevenly across sectors and time.

The pace of market development has been shaped by a combination of gradual privatization of state-owned enterprises, episodic regulatory reform, and sustained participation by retail investors. Unlike markets that evolved alongside large institutional investor bases, Vietnam's equity market matured in an environment where individual investors dominate trading activity and informational asymmetries remain substantial.

These features have important empirical implications. Return dynamics may reflect behavioral trading patterns, liquidity shocks can be amplified by coordinated retail activity, and firm-level information is incorporated into prices at varying speeds. A reproducible empirical framework must therefore be capable of capturing these dynamics without imposing assumptions derived from institutionally different markets.

1.2 Exchange Structure and Trading Mechanisms

Vietnam operates multiple equity exchanges, each with distinct listing requirements and trading rules. Trading is conducted through a centralized limit order book, with price-time priority determining execution. Importantly, daily price limits constrain the maximum allowable price movement for individual securities. These limits vary by exchange and security type and are binding during periods of heightened volatility.

Price limits introduce mechanical truncation in observed returns, clustering at upper and lower bounds, and persistence in price movements across days. From an empirical perspective, this challenges standard assumptions about continuous price adjustment and complicates volatility estimation, momentum measurement, and event-study design.

In this book, price limits are treated as structural features rather than anomalies. Data pipelines explicitly preserve limit-hit indicators, and empirical models are adapted to account for constrained price dynamics. This design choice reflects a broader principle: reproducibility in emerging markets requires preserving institutional signals rather than smoothing them away.

1.3 Listing Requirements and Firm Characteristics

Listed firms in Vietnam exhibit substantial heterogeneity in size, ownership structure, and disclosure quality. A defining characteristic of the market is the prevalence of firms with significant state ownership, either directly or through affiliated entities. State ownership affects governance, dividend policy, risk-taking behavior, and responsiveness to market signals.

Accounting disclosures follow Vietnamese Accounting Standards, which differ in important respects from international standards. While convergence efforts are ongoing, historical financial statements often reflect transitional rules, incomplete adoption of fair value accounting, and limited segment reporting. These features complicate cross-firm comparability and longitudinal analysis.

From a reproducible research standpoint, accounting variables cannot be treated as uniform primitives. Variable definitions, reporting lags, and restatement practices must be explicitly documented and encoded into data construction logic. Later chapters demonstrate how accounting data are harmonized in a transparent, version-controlled manner without obscuring underlying institutional differences.

1.4 Investor Composition and Trading Behavior

Retail investors dominate trading volume in Vietnam's equity market. Institutional investors, including domestic funds and foreign participants, play a growing but still secondary role.

This investor composition has implications for liquidity provision, price discovery, and market stability.

Retail-dominated markets tend to exhibit higher turnover, episodic herding behavior, and sensitivity to non-fundamental information. These patterns affect the interpretation of empirical results, particularly in studies of short-term return predictability, volume-return relations, and volatility clustering.

Rather than assuming institutional trading as the default, this book explicitly models liquidity and trading activity in a retail-centric environment. Measures of liquidity, for example, are chosen and constructed to remain meaningful in the presence of small trade sizes, intermittent trading, and order imbalances driven by individual investors.

1.5 Regulatory Environment and Market Frictions

Regulatory oversight of Vietnam's equity market has evolved alongside market development. Trading rules, disclosure requirements, and foreign ownership limits have been periodically revised, sometimes with limited backward compatibility. Regulatory changes can induce structural breaks in data that are not immediately apparent in raw time series.

Short-selling constraints, limited securities lending, and restrictions on derivative usage further distinguish Vietnam's market from developed counterparts. These frictions affect arbitrage activity and the feasibility of certain trading strategies, influencing observed return patterns and factor realizations.

A key principle of the empirical framework developed in this book is regulatory awareness. Data pipelines incorporate regulatory timelines, and empirical tests are designed to be robust to rule changes. This ensures that results are interpretable within the institutional regime in which they arise.

1.6 Implications for Empirical Design

The institutional features described in this chapter motivate several design choices that recur throughout the book:

1. **Data preservation over simplification**: Institutional constraints such as price limits and trading halts are retained and explicitly modeled.
2. **Modular variable construction**: Accounting and market variables are constructed through transparent functions that can be adjusted as standards evolve.
3. **Regime sensitivity**: Empirical analyses are structured to detect and accommodate regulatory and structural breaks.

4. **Context-aware interpretation**: Results are interpreted in light of market structure rather than benchmarked mechanically against developed-market findings.

1.7 Summary

Vietnam's equity market combines rapid growth with distinctive institutional features that challenge conventional empirical finance methods. Price limits, retail investor dominance, state ownership, and evolving regulation shape market outcomes in ways that cannot be ignored or abstracted away. For researchers working in such environments, reproducibility requires more than clean code and documented data; it requires embedding institutional context directly into empirical design.

2 Accessing and Managing VN Financial Data

This chapter provides a guide to organizing, accessing, and managing financial data specifically tailored for the Vietnamese market. While global financial databases such as CRSP and Compustat serve as standard resources for developed markets, emerging markets like Vietnam require a different approach due to unique data sources, market structures, and regulatory environments. Understanding these nuances is essential for conducting rigorous empirical research on Vietnamese equities, bonds, and macroeconomic indicators.

Vietnam's financial market has experienced remarkable growth since the establishment of the Ho Chi Minh City Stock Exchange (HOSE) in 2000 and the Hanoi Stock Exchange (HNX) in 2005. Today, the market comprises over 1,600 listed companies across three trading venues: HOSE for large-cap stocks, HNX for mid-cap stocks, and UPCoM (Unlisted Public Company Market) for smaller companies transitioning to formal listing. This diversity creates both opportunities and challenges for financial researchers seeking comprehensive coverage of the Vietnamese equity universe.

The Vietnamese market presents several distinctive characteristics that researchers must account for. Foreign ownership limits (typically 49% for most sectors, with exceptions for banking and certain strategic industries), trading band restrictions (e.g., currently $\pm$ 7% for HOSE and $\pm$ 10% for HNX), and the T+2 settlement cycle all influence market microstructure and return dynamics. Additionally, the market operates in Vietnamese Dong (VND), requiring careful attention to currency effects when comparing results with international studies.

We begin by loading the essential Python packages that facilitate data acquisition and management throughout this chapter.

```
import pandas as pd
import numpy as np
import requests
from datetime import datetime, timedelta
import json
import sqlite3
```

We also define the date range for our data collection, which spans from the early days of the Vietnamese stock market to the present. This extended timeframe allows us to capture the market's evolution through various economic cycles, including the 2008 global financial crisis, the 2011-2012 domestic banking crisis, and the COVID-19 pandemic period.

```
start_date = "2000-07-28"  # HOSE establishment date
end_date = "2024-12-31"
```

2.1 Overview of Vietnamese Financial Data Sources

Before turning to implementation, it is worth surveying the financial data providers that serve the Vietnamese market. Unlike developed markets where a few dominant providers (Bloomberg, Refinitiv, FactSet) offer comprehensive coverage, Vietnamese financial data has historically been fragmented across multiple sources, each with distinct strengths and limitations.

The primary sources of Vietnamese financial data include official exchange feeds from HOSE and HNX, which provide real-time and historical trading data. The State Securities Commission of Vietnam (SSC) publishes regulatory filings, corporate announcements, and market statistics. Commercial data vendors such as FiinGroup, StoxPlus (now part of FiinGroup), and VNDirect offer curated datasets with varying levels of coverage and data quality. Additionally, the State Bank of Vietnam (SBV) and the General Statistics Office (GSO) provide macroeconomic indicators essential for asset pricing research.

For academic researchers, this fragmentation traditionally involved difficult trade-offs between cost, coverage, data quality, and ease of access. Commercial providers like FiinGroup offer clean, standardized data but require subscription fees that may be prohibitive for individual researchers and smaller institutions. Open-source alternatives provide free access but often require substantial data cleaning and validation efforts. Manually collecting data from government websites is time-consuming and prone to inconsistencies.

Fortunately, this landscape has improved significantly with the emergence of **Datacore** as a unified data platform for Vietnamese financial markets. In our experience working with Vietnamese financial data across multiple research projects, Datacore has proven to be the most practical solution for academic research. The platform consolidates data from multiple sources, including stock prices, corporate fundamentals, market indices, macroeconomic indicators, and alternative data, into a single, accessible interface with a well-documented API.

What distinguishes Datacore from traditional commercial providers like FiinGroup extends beyond mere data aggregation. While FiinGroup has long been the institutional incumbent, several factors make Datacore particularly attractive for rigorous empirical research:

1. **API-First Architecture**: Datacore was built from the ground up for programmatic access, so it drops into Python, R, and other research workflows without an intermediate export step. FiinGroup's data access, by contrast, often requires manual downloads or cumbersome Excel-based interfaces that impede reproducibility.

2. **Cost Efficiency**: Academic researchers frequently operate under budget constraints. Datacore offers competitive pricing structures that make comprehensive market coverage accessible without the substantial subscription fees associated with legacy providers.

3. **Corporate Action Handling**: One persistent challenge with Vietnamese data is accurate adjustment for stock splits, bonus shares, and rights issues. Datacore implements transparent adjustment methodologies with clear documentation, whereas legacy providers often apply adjustments inconsistently or without adequate explanation.

4. **Update Frequency**: Datacore maintains near real-time data updates with clear timestamps, enabling event study research and timely portfolio rebalancing. Traditional providers often suffer from publication lags that can compromise research requiring current data.

5. **Coverage Breadth**: Beyond standard price and fundamental data, Datacore integrates alternative data, and macroeconomic indicators into a unified schema. This eliminates the need to merge datasets from multiple sources, which is a process that introduces potential errors and consumes valuable research time.

Throughout this chapter, we leverage Datacore as our primary data source. By centralizing our data acquisition through a single platform, we benefit from consistent data formats, reliable corporate action adjustments, and comprehensive market coverage spanning HOSE, HNX, and UPCoM. The code examples that follow demonstrate how straightforward Vietnamese financial research becomes when data access friction is minimized.

The following table summarizes the key data sources for Vietnamese financial research:

Table 2.1: Vietnamese Financial Data Sources

Data Source	Coverage	Access Type	Key Strengths	Limitations
Datacore	Prices, fundamentals, indices, macro, derivatives	API	Unified platform, programmatic access, comprehensive coverage, transparent methodology	Newer platform
FiinGroup	Full market coverage	Commercial	Established reputation, institutional adoption	High cost, manual access, limited API
HOSE/HNX websites	Official exchange data	Free (manual)	Authoritative, real-time	No API, manual collection required

Data Source	Coverage	Access Type	Key Strengths	Limitations
GSO (gso.gov.vn)	Macroeconomic indicators	Free (manual)	Official government statistics	Infrequent updates, no API
SBV (sbv.gov.vn)	Monetary policy, rates	Free (manual)	Central bank data	Manual download only
CafeF/VnExpress	News, announcements	Free	Market sentiment, events	Unstructured, requires NLP processing

2.2 Stock Market Data

The resulting DataFrame contains essential security identifiers including the ticker symbol, company name in both Vietnamese and English, exchange listing, industry classification according to the Vietnam Standard Industrial Classification (VSIC), and various flags indicating special status such as foreign ownership restrictions or trading suspensions.

2.2.1 Historical Price Data

2.2.2 Fundamental Data and Financial Statements

Beyond price data, fundamental analysis requires access to corporate financial statements including balance sheets, income statements, and cash flow statements. Vietnamese publicly listed companies are required to publish quarterly and annual financial reports according to Vietnamese Accounting Standards (VAS), which differ in certain respects from International Financial Reporting Standards (IFRS). Understanding these differences is important when comparing Vietnamese firms with international peers or applying models developed using US or European data.

Key differences between VAS and IFRS that affect financial analysis include:

1. **Revenue recognition**: VAS allows more flexibility in timing of revenue recognition compared to IFRS 15
2. **Financial instruments**: VAS has less comprehensive guidance on fair value measurement
3. **Lease accounting**: VAS does not require operating lease capitalization as under IFRS 16
4. **Goodwill**: VAS requires amortization while IFRS requires impairment testing only

2.2.3 Corporate Actions and Events

Accurate treatment of corporate actions is essential for computing correct returns and maintaining data integrity. Vietnamese companies frequently engage in corporate actions including cash dividends, stock dividends (bonus shares), rights issues, and stock splits.

2.3 Market Indices and Benchmarks

Constructing appropriate benchmarks is fundamental to performance evaluation and factor model estimation. The Vietnamese market features several indices that serve different purposes in financial research.

Table 2.2: Vietnamese Market Indices

Index	Exchange	Description	Use Case
VN-Index	HOSE	All HOSE-listed stocks	Broad market benchmark
VN30-Index	HOSE	30 largest, most liquid	Investable benchmark
HNX-Index	HNX	All HNX-listed stocks	Mid-cap benchmark
HNX30-Index	HNX	30 largest HNX stocks	HNX large-cap
VNAllShare	Combined	HOSE + HNX	Total market
VN100	Combined	Top 100 stocks	Large/mid-cap

The VN-Index, which tracks all stocks listed on HOSE, is the most widely followed benchmark and serves as the primary gauge of overall market performance. The HNX-Index covers stocks on the Hanoi exchange, while the VN30-Index tracks the thirty largest and most liquid stocks on HOSE.

For asset pricing research, the VN30-Index is particularly valuable as it represents the investable universe for institutional investors and serves as the underlying for Vietnam's most liquid derivatives contracts. The constituent stocks are reviewed semi-annually based on market capitalization, liquidity, and free-float requirements.

```
# Retrieve VN-Index historical data
```

2.3.1 Index Constituent Data

For factor model construction and portfolio analysis, access to index constituent lists and their weights is essential. While official constituent data requires subscription to exchange data feeds, we can approximate index membership using market capitalization and liquidity filters.

2.4 Macroeconomic Data from Vietnamese Sources

Asset pricing models often incorporate macroeconomic variables as predictors of expected returns or as state variables in conditional models. For the Vietnamese market, relevant macroeconomic data comes primarily from two sources: the General Statistics Office (GSO) and the State Bank of Vietnam (SBV).

2.4.1 Key Macroeconomic Indicators

The following macroeconomic variables are particularly relevant for Vietnamese financial research:

1. **Consumer Price Index (CPI)**: Essential for computing real returns and inflation-adjusted valuations. Vietnam experienced periods of high inflation, particularly during 2008 and 2011 when annual CPI exceeded 20%.
2. **Industrial Production Index (IPI)**: Proxy for economic activity and business cycle conditions.
3. **Money Supply (M2)**: Indicator of monetary policy stance and liquidity conditions.
4. **Credit Growth**: Bank lending growth, a key driver of economic activity in Vietnam's bank-dominated financial system.
5. **USD/VND Exchange Rate**: Critical for international investors and companies with foreign currency exposure.
6. **Foreign Direct Investment (FDI)**: Indicator of international capital flows and economic confidence.
7. **Trade Balance**: Export and import dynamics affecting corporate earnings.

Unfortunately, unlike the US Federal Reserve's FRED database, Vietnamese macroeconomic data is not available through standardized APIs. Researchers must typically download data manually from GSO and SBV websites or use web scraping techniques.

```
# Structure for Vietnamese macroeconomic data
```

2.4.2 Risk-Free Rate Approximation

Determining an appropriate risk-free rate for Vietnam presents challenges not encountered in developed markets. Unlike the US Treasury market, Vietnam's government bond market is relatively illiquid with limited secondary trading. Several alternatives exist:

1. **SBV Refinancing Rate**: The policy rate set by the State Bank of Vietnam. Not directly investable but reflects monetary policy stance.
2. **Government Bond Yields**: One-year or longer-term government bond yields from auction results. More investable but less liquid than US Treasuries.
3. **Interbank Rates**: Overnight or term interbank lending rates. Reflect short-term funding costs but include credit risk.
4. **Adjusted US Rate**: US Treasury rate plus expected VND depreciation, following uncovered interest rate parity.

```python
def calculate_risk_free_rate(macro_data, method="refinancing"):
    """
    Calculate risk-free rate proxy for Vietnamese market.

    Parameters
    ----------
    macro_data : pd.DataFrame
        DataFrame with macroeconomic data
    method : str
        Method for risk-free rate: 'refinancing', 'bond', or 'adjusted_us'

    Returns
    -------
    pd.DataFrame
        DataFrame with date and monthly risk-free rate
    """
    if method == "refinancing":
        # Use SBV refinancing rate, convert annual to monthly
        rf = macro_data[["date", "refinancing_rate"]].copy()
        rf["rf_monthly"] = rf["refinancing_rate"] / 12 / 100

    elif method == "adjusted_us":
        # US rate + expected VND depreciation
        # Requires additional data on US rates and exchange rate expectations
        pass

    return rf[["date", "rf_monthly"]]
```

2.5 Setting Up a Database for Vietnamese Financial Data

Managing financial data across multiple sources and formats requires a systematic approach to data storage. We recommend using SQLite as the primary database engine for several reasons: it requires no server setup, stores the entire database in a single portable file, supports standard SQL queries, and integrates seamlessly with Python through the built-in sqlite3 module.

2.5.1 Database Schema Design

Our database schema is designed to support efficient queries for common research tasks while maintaining data integrity. We create separate tables for different data types with appropriate relationships.

```
import os
import sqlite3

# Create data directory if it doesn't exist
if not os.path.exists("data"):
    os.makedirs("data")

# Initialize SQLite database connection
tidy_finance_python = sqlite3.connect(
    "data/tidy_finance_python.sqlite"
)
```

2.5.2 Storing Data

With the database schema established, we can store our collected data using pandas' to_sql() method.

```
# Store stock listing data
common_stocks.to_sql(
    name="stock_master",
    con=tidy_finance_python,
    if_exists="replace",
    index=False
)
```

```
# Store stock price data
stock_prices.to_sql(
    name="stock_prices_daily",
    con=tidy_finance_python,
    if_exists="replace",
    index=False
)

# Store market indices
vn_index.to_sql(
    name="market_indices",
    con=tidy_finance_python,
    if_exists="replace",
    index=False
)

# Store factor returns
factors_vietnam.to_sql(
    name="factors_monthly",
    con=tidy_finance_python,
    if_exists="replace",
    index=False
)
```

2.6 Querying and Updating the Database

Once data is stored in the database, retrieval is straightforward using SQL queries. The pandas read_sql_query() function executes a SQL statement and returns the results as a DataFrame.

```
# Query stock prices for specific symbols and date range
query = """
SELECT date, symbol, close, volume
FROM stock_prices_daily
WHERE symbol IN ('VNM', 'VIC', 'FPT', 'VHM', 'VCB')
  AND date >= '2020-01-01'
ORDER BY symbol, date
"""

selected_stocks = pd.read_sql_query(
```

```
    sql=query,
    con=tidy_finance_python,
    parse_dates=["date"]
)

# Query factor data merged with market returns
query_factors = """
SELECT f.date, f.mkt_rf, f.smb, f.hml, f.rf,
       m.cpi_yoy, m.credit_growth
FROM factors_monthly f
LEFT JOIN macro_monthly m ON f.date = m.date
WHERE f.date >= '2015-01-01'
ORDER BY f.date
"""

factor_data = pd.read_sql_query(
    sql=query_factors,
    con=tidy_finance_python,
    parse_dates=["date"]
)
```

2.6.1 Database Maintenance

Regular database maintenance ensures optimal performance and data integrity.

```
# Optimize database
tidy_finance_python.execute("VACUUM")

# Check database integrity
integrity_check = pd.read_sql_query(
    "PRAGMA integrity_check",
    tidy_finance_python
)
print(f"Integrity check: {integrity_check.iloc[0, 0]}")

# Get database statistics
table_stats = pd.read_sql_query("""
    SELECT name,
           (SELECT COUNT(*) FROM stock_prices_daily) as price_rows,
           (SELECT COUNT(*) FROM stock_master) as stock_count,
           (SELECT COUNT(*) FROM factors_monthly) as factor_months
```

```
    FROM sqlite_master
    WHERE type='table' AND name='stock_master'
""", tidy_finance_python)

print(table_stats)

# Close connection when done
tidy_finance_python.close()
```

2.7 Alternative Data Sources for Vietnamese Markets

Beyond traditional price and fundamental data, researchers increasingly incorporate alternative data sources to gain unique insights into market dynamics. The economic value of alternative data is well documented in credit markets: Di Maggio and Ratnadiwakara (2026) show that a large US fintech lender's underwriting model, which incorporates nontraditional variables such as education and employment history, identifies "invisible primes" (i.e., borrowers with thin credit files and low credit scores whose true default risk is low) and thereby expands credit access relative to traditional credit-score-based underwriting. This evidence is directly relevant to Vietnam, where credit bureau coverage remains limited and many households and small firms lack extensive formal credit histories, making alternative data central to the country's growing fintech lending sector. Alternative data need not describe the borrower at all: L. Allen, Peng, and Shan (2026) show that the social network linking a borrower's community to the rest of the country, measured with Facebook's Social Connectedness Index, carries information that raises both access to online credit and the performance of the loans made, especially in disadvantaged communities. Section 106.7 develops the measure and the data available for Vietnam. The choice of underwriting variables also raises fairness questions that alternative data can sharpen rather than resolve: Amornsiripanitch (2026) documents that in US single-borrower mortgage data, applicant age is as strong a correlate of rejection and pricing as race and ethnicity, with rejection probability rising smoothly with age and accelerating in old age, plausibly reflecting collateral-maintenance and mortality risk rather than taste-based discrimination. As Vietnamese lenders begin to incorporate richer borrower attributes, researchers and regulators will need to distinguish legitimate risk-based pricing from statistical discrimination against protected or vulnerable groups—an increasingly salient concern in an aging society.

2.7.1 When the Lender's Decision Is Itself Information

The papers above treat credit data as information the *lender* uses. Yin (2026) show that the lender's decision is also information the *borrower* uses, and that this changes what credit-supply data measure. Working with a large Chinese commercial bank, the author randomized

the timing of credit-card limit increases that the bank had already approved for about 17,000 customers through its normal underwriting: a control group had its increase delayed by twelve months. The treated customers were then split. One group received the bank's usual notice. The other received the same notice plus a true statement that the increase had gone to a randomly selected group among customers with good credit scores. Transaction records and a survey fielded within ten days of the increase measured what customers did and what they expected. The figures below are from the working-paper version.

Customers who received the usual notice spent 0.37 yuan more over six months per yuan of additional limit and accumulated 0.15 yuan more unsecured debt, magnitudes in line with earlier estimates. Customers told that their increase was random spent about 37% less. The survey shows why. The standard-notice group raised its expectations of future consumption and of future income in the same job, lowered its perceived risk of unemployment, and expected faster growth and lower unemployment in the economy, but did not plan to work more hours or to draw down savings. The group told that the increase was random revised none of these. Belief changes were concentrated among customers who were unsure of their reading of the economy and who thought macroeconomic conditions mattered for their income, and they were negligible for customers whose income had not varied before the experiment. Hypothetical-scenario surveys of US respondents produced the same pattern.

The interpretation is an **income-inference channel**. Lenders set limits with information about the systematic part of borrowers' future income, information borrowers lack, so a higher limit signals better prospects, and borrowers spend accordingly even when they are far from their borrowing constraint. This matters for three kinds of empirical work. Estimates of the marginal propensity to consume out of credit limits combine a liquidity channel and an information channel, and the mix depends on how much borrowers believe their lender knows. Credit-supply shocks used as instruments for household or firm outcomes may violate the exclusion restriction through beliefs, not just through borrowing. And the better the underwriting model, the stronger the signal: the alternative-data underwriting discussed above makes credit decisions more informative, and therefore more able to move borrowers' expectations, for better or worse.

Vietnam offers several settings in which the channel can be studied. Consumer credit has grown quickly through credit cards, consumer-finance companies, and buy-now-pay-later products, and the peer-to-peer lending sandbox opened in 2025 (Section 106.7.3) adds platforms whose credit decisions rely on alternative data. At the aggregate level, the State Bank's annual credit-growth quotas for individual banks, and their revisions during the year, are public signals of credit supply that households and firms may read as information about the economy, not only as a change in the availability of loans. Four practical rules follow.

1. **Separate the channels by design.** When a bank partner can randomize limit changes, an information treatment that truthfully discloses how limits were assigned separates inference from liquidity at little cost. Without randomization, limit changes generated

by a published, mechanical rule carry less information than discretionary ones and can serve as a comparison.
2. **Measure expectations, not just spending.** Transaction data cannot distinguish a consumer who spends because the constraint has loosened from one who spends because they expect to earn more. Short surveys linked to account data can.
3. **Use heterogeneity as a test.** The information channel predicts no response among households with stable incomes and ample liquid savings; the liquidity channel predicts no response among households far from their limits whatever their income risk. The two predictions sort households differently.
4. **Treat the information content of credit decisions as an outcome of underwriting.** When fintech lenders in Vietnam move from collateral-based to data-based underwriting, the signal content of their decisions rises. Studies of their effects on borrowers should expect larger expectation responses, and regulators should expect credit tightening to depress spending through beliefs as well as through liquidity.

2.7.2 Foreign Investor Flow Data

Foreign investor flow data is particularly valuable given the significant role of foreign capital in Vietnamese equity markets. The State Securities Commission publishes daily foreign ownership statistics by security.

2.7.3 News and Sentiment Data

Media sentiment from Vietnamese financial news sources offers another research avenue. Major outlets such as CafeF, VnExpress Finance, and Vietstock publish real-time news that can be analyzed for market sentiment.

2.8 Key Takeaways

1. **Market Structure Understanding**: The Vietnamese financial market operates across three exchanges (HOSE, HNX, UPCoM) with distinct characteristics including foreign ownership limits, trading band restrictions, and a T+2 settlement cycle. Researchers must account for these institutional features in empirical analysis.
2. **Macroeconomic Data Challenges**: Unlike developed markets with standardized APIs (e.g., FRED), Vietnamese macroeconomic data requires manual collection from government sources (GSO, SBV). Researchers should plan for this additional data gathering effort and implement systematic data management practices.

3. **Database-Centric Workflow**: SQLite provides an efficient and portable database solution for managing Vietnamese financial data across research projects. The structured database approach enables reproducible research workflows, efficient queries, and easy data sharing among collaborators.

4. **Data Quality Imperative**: Data quality validation is especially important for emerging market data. Implementing systematic checks for missing values, extreme returns, duplicate entries, and cross-source validation helps ensure research reliability and reproducibility.

5. **Alternative Data Opportunities**: Foreign investor flows, corporate announcements, and media sentiment provide unique research opportunities in the Vietnamese market that can complement traditional price and fundamental analysis. These data sources can reveal insights about market dynamics not captured in standard datasets.

6. **Continuous Maintenance**: Financial databases require ongoing maintenance including incremental updates, integrity checks, and optimization. Establishing systematic update procedures ensures data currency and database performance over time.

3 Datacore Data

This chapter introduces Datacore, Vietnam's data platform for academic, corporate, and government research. Datacore provides comprehensive financial and economic datasets, including historical trading data, company fundamentals, and macroeconomic indicators essential for reproducible finance research. We use Datacore as the primary data source throughout this book.

3.1 Data Access Options

Readers can access the data used in this book through several channels:

1. **Institutional subscription**: Many universities and research institutions subscribe to Datacore. Check with your library or research office for access credentials. If your institution does not yet have a subscription, consider requesting one through your library's acquisition process—Datacore offers institutional pricing for academic use.
2. **Demo datasets**: Datacore provides demo datasets that allow you to run the code examples in this book with sample data.

3.2 Chapter Overview

The chapter is organized as follows. We first establish the connection to Datacore's cloud storage infrastructure. Then, we download and prepare company fundamentals data, including balance sheet items, income statement variables, and derived metrics essential for asset pricing research. Next, we retrieve and process stock price data, computing returns, market capitalizations, and excess returns. We conclude by merging these datasets and providing descriptive statistics that characterize the Vietnamese equity market.

3.3 Setting Up the Environment

We begin by loading the Python packages used throughout this chapter. The core packages include `pandas` for data manipulation, `numpy` for numerical operations, and `sqlite3` for local database management. We also import visualization libraries for creating publication-quality figures.

```
import pandas as pd
import numpy as np
import sqlite3
from datetime import datetime
from io import BytesIO

from plotnine import *
from mizani.formatters import comma_format, percent_format
```

We establish a connection to our local SQLite database, which serves as the central repository for all processed data. This database was introduced in the previous chapter and will store the cleaned datasets for use in subsequent analyses.

```
tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")
```

We define the date range for our data collection. The Vietnamese stock market began operations in July 2000 with the establishment of the Ho Chi Minh City Stock Exchange (HOSE), so our sample period starts from 2000 and extends through the end of 2024.

```
start_date = "2000-01-01"
end_date = "2024-12-31"
```

3.4 Connecting to Datacore

Datacore delivers data through a cloud-based object storage system built on MinIO, an S3-compatible storage infrastructure. This architecture enables efficient, programmatic access to large datasets without the limitations of traditional database connections. To access the data, you need credentials provided by Datacore upon subscription: an endpoint URL, access key, and secret key.

The following class establishes the connection to Datacore's storage system. The credentials are stored as environment variables for security, following best practices for credential management in research computing environments.

```python
import os
import boto3
from botocore.client import Config

class DatacoreConnection:
    """
    Connection handler for Datacore's MinIO-based storage system.

    This class manages authentication and provides methods for
    accessing financial datasets stored in Datacore's cloud infrastructure.

    Attributes
    ----------
    s3 : boto3.client
        S3-compatible client for interacting with Datacore storage
    """

    def __init__(self):
        """Initialize connection using environment variables."""
        self.MINIO_ENDPOINT = os.environ["MINIO_ENDPOINT"]
        self.MINIO_ACCESS_KEY = os.environ["MINIO_ACCESS_KEY"]
        self.MINIO_SECRET_KEY = os.environ["MINIO_SECRET_KEY"]
        self.REGION = os.getenv("MINIO_REGION", "us-east-1")

        self.s3 = boto3.client(
            "s3",
            endpoint_url=self.MINIO_ENDPOINT,
            aws_access_key_id=self.MINIO_ACCESS_KEY,
            aws_secret_access_key=self.MINIO_SECRET_KEY,
            region_name=self.REGION,
            config=Config(signature_version="s3v4"),
        )

    def test_connection(self):
        """Verify connection by listing available buckets."""
        response = self.s3.list_buckets()
        print("Connected successfully. Available buckets:")
        for bucket in response.get("Buckets", []):
            print(f"  - {bucket['Name']}")

    def list_objects(self, bucket_name, prefix=""):
        """List objects in a bucket with optional prefix filter."""
        response = self.s3.list_objects_v2(
```

```
            Bucket=bucket_name,
            Prefix=prefix
        )
        return [obj["Key"] for obj in response.get("Contents", [])]

    def read_excel(self, bucket_name, key):
        """Read an Excel file from Datacore storage."""
        obj = self.s3.get_object(Bucket=bucket_name, Key=key)
        return pd.read_excel(BytesIO(obj["Body"].read()))

    def read_csv(self, bucket_name, key, **kwargs):
        """Read a CSV file from Datacore storage."""
        obj = self.s3.get_object(Bucket=bucket_name, Key=key)
        return pd.read_csv(BytesIO(obj["Body"].read()), **kwargs)
```

With the connection class defined, we can establish a connection and verify access to Datacore's data repositories.

```
# Initialize connection
conn = DatacoreConnection()
conn.test_connection()

# Get bucket name from environment
bucket_name = os.environ["MINIO_BUCKET"]
```

```
Connected successfully. Available buckets:
  - dsteam-data
  - rawbctc
```

3.5 Company Fundamentals Data

Firm accounting data are essential for portfolio analyses, factor construction, and valuation studies. Datacore hosts comprehensive fundamentals data for Vietnamese listed companies, including annual and quarterly financial statements prepared according to Vietnamese Accounting Standards (VAS).

3.5.1 Understanding Vietnamese Financial Statements

Before processing the data, it is important to understand the structure of Vietnamese financial reports. Vietnamese companies follow VAS, which shares similarities with International Financial Reporting Standards (IFRS) but has notable differences:

1. **Fiscal Year**: Most Vietnamese companies use a calendar fiscal year ending December 31, though some companies (particularly in retail and agriculture) use different fiscal year-ends.
2. **Reporting Frequency**: Listed companies must publish quarterly financial statements within 20 days of quarter-end and annual audited statements within 90 days of fiscal year-end.
3. **Industry-Specific Formats**: Companies in banking, insurance, and securities sectors follow specialized reporting formats that differ from the standard industrial format.
4. **Currency**: All figures are reported in Vietnamese Dong (VND). Given the large nominal values (millions to trillions of VND), we often scale figures to millions or billions for readability.

3.5.2 Downloading Fundamentals Data

Datacore organizes fundamentals data in Excel files partitioned by time period for efficient access. We download and concatenate these files to create a comprehensive dataset spanning our sample period.

```
# Define paths to fundamentals data files
fundamentals_paths = [
    "fundamental_annual_1767674486317/fundamental_annual_1.xlsx",
    "fundamental_annual_1767674486317/fundamental_annual_2.xlsx",
    "fundamental_annual_1767674486317/fundamental_annual_3.xlsx",
]

# Download and combine all files
fundamentals_list = []
for path in fundamentals_paths:
    df_temp = conn.read_excel(bucket_name, path)
    fundamentals_list.append(df_temp)
    print(f"Downloaded: {path} ({len(df_temp):,} rows)")

df_fundamentals_raw = pd.concat(fundamentals_list, ignore_index=True)
print(f"\nTotal observations: {len(df_fundamentals_raw):,}")
```

```
/home/mikenguyen/project/tidyfinance/.venv/lib/python3.13/site-packages/openpyxl/styles/style

Downloaded: fundamental_annual_1767674486317/fundamental_annual_1.xlsx (10,000 rows)

/home/mikenguyen/project/tidyfinance/.venv/lib/python3.13/site-packages/openpyxl/styles/style

Downloaded: fundamental_annual_1767674486317/fundamental_annual_2.xlsx (10,000 rows)

/home/mikenguyen/project/tidyfinance/.venv/lib/python3.13/site-packages/openpyxl/styles/style

Downloaded: fundamental_annual_1767674486317/fundamental_annual_3.xlsx (2,821 rows)

Total observations: 22,821
```

3.5.3 Cleaning and Standardizing Fundamentals

The raw fundamentals data requires several cleaning steps to ensure consistency and usability. We standardize variable names, handle missing values, and create derived variables commonly used in asset pricing research.

```python
def clean_fundamentals(df):
    """
    Clean and standardize company fundamentals data.

    Parameters
    ----------
    df : pd.DataFrame
        Raw fundamentals data from Datacore

    Returns
    -------
    pd.DataFrame
        Cleaned fundamentals with standardized column names
    """
    df = df.copy()

    # Standardize identifiers
    df["symbol"] = df["symbol"].astype(str).str.upper().str.strip()
    df["year"] = pd.to_numeric(df["year"], errors="coerce").astype("Int64")
```

```
    # Drop rows with missing identifiers
    df = df.dropna(subset=["symbol", "year"])

    # Define columns that should be numeric
    numeric_columns = [
        "total_asset", "total_equity", "total_liabilities",
        "total_current_asset", "total_current_liabilities",
        "is_net_revenue", "is_cogs", "is_manage_expense",
        "is_interest_expense", "is_eat", "is_net_business_profit",
        "na_tax_deferred", "nl_tax_deferred", "e_preferred_stock",
        "capex", "total_cfo", "ca_cce", "ca_total_inventory",
        "ca_acc_receiv", "cfo_interest_expense", "basic_eps",
        "is_shareholders_eat", "cl_loan", "cl_finlease",
        "cl_due_long_debt", "nl_loan", "nl_finlease",
        "is_cos_of_sales", "e_equity"
    ]

    for col in numeric_columns:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    # Handle duplicates: keep row with most non-missing values
    df["_completeness"] = df.notna().sum(axis=1)
    df = (df
        .sort_values(["symbol", "year", "_completeness"])
        .drop_duplicates(subset=["symbol", "year"], keep="last")
        .drop(columns="_completeness")
        .reset_index(drop=True)
    )

    return df

df_fundamentals = clean_fundamentals(df_fundamentals_raw)
print(f"After cleaning: {len(df_fundamentals):,} firm-year observations")
print(f"Unique firms: {df_fundamentals['symbol'].nunique():,}")
```

```
After cleaning: 21,232 firm-year observations
Unique firms: 1,554
```

3.5.4 Creating Standardized Variables

To facilitate comparison with international studies and ensure compatibility with standard asset pricing methodologies, we create variables following conventions established in the academic literature. We map Vietnamese financial statement items to their Compustat equivalents where possible.

```
def create_standard_variables(df):
    """
    Create standardized financial variables for asset pricing research.

    This function maps Vietnamese financial statement items to standard
    variable names used in the academic finance literature, following
    conventions from Fama and French (1992, 1993, 2015).

    Parameters
    ----------
    df : pd.DataFrame
        Cleaned fundamentals data

    Returns
    -------
    pd.DataFrame
        Fundamentals with standardized variables added
    """
    df = df.copy()

    # Fiscal date (assume December year-end)
    df["datadate"] = pd.to_datetime(df["year"].astype(str) + "-12-31")

    # === Balance Sheet Items ===
    df["at"] = df["total_asset"]                    # Total assets
    df["lt"] = df["total_liabilities"]              # Total liabilities
    df["seq"] = df["total_equity"]                  # Stockholders' equity
    df["act"] = df["total_current_asset"]           # Current assets
    df["lct"] = df["total_current_liabilities"]     # Current liabilities

    # Common equity (fallback to total equity if not available)
    df["ceq"] = df.get("e_equity", df["seq"])

    # === Deferred Taxes ===
    df["txditc"] = df.get("na_tax_deferred", 0).fillna(0)  # Deferred tax
↪   assets
```

```
    df["txdb"] = df.get("nl_tax_deferred", 0).fillna(0)    # Deferred tax
↪   liab.
    df["itcb"] = 0  # Investment tax credit (rare in Vietnam)

    # === Preferred Stock ===
    pref = df.get("e_preferred_stock", 0)
    if isinstance(pref, pd.Series):
        pref = pref.fillna(0)
    df["pstk"] = pref
    df["pstkrv"] = pref  # Redemption value
    df["pstkl"] = pref   # Liquidating value

    # === Income Statement Items ===
    df["sale"] = df["is_net_revenue"]                        # Net
↪   sales/revenue
    df["cogs"] = df.get("is_cogs", 0).fillna(0)              # Cost of goods
↪   sold
    df["xsga"] = df.get("is_manage_expense", 0).fillna(0)    # SG&A expenses
    df["xint"] = df.get("is_interest_expense", 0).fillna(0)  # Interest
↪   expense
    df["ni"] = df.get("is_eat", np.nan)                      # Net income
    df["oibdp"] = df.get("is_net_business_profit", np.nan)   # Operating
↪   income

    # === Cash Flow Items ===
    df["oancf"] = df.get("total_cfo", np.nan)  # Operating cash flow
    df["capx"] = df.get("capex", np.nan)       # Capital expenditures

    return df

df_fundamentals = create_standard_variables(df_fundamentals)
```

3.5.5 Computing Book Equity and Profitability

Book equity is a crucial variable for value investing strategies and the construction of HML (High Minus Low) factor portfolios. We follow the definition from Kenneth French's data library, which accounts for deferred taxes and preferred stock.

```
def compute_book_equity(df):
    """
    Compute book equity following Fama-French conventions.
```

```
    Book equity = Stockholders' equity
                  + Deferred taxes and investment tax credit
                  - Preferred stock

    Negative or zero book equity is set to missing, as book-to-market
    ratios are undefined for such firms.

    Parameters
    ----------
    df : pd.DataFrame
        Fundamentals with standardized variables

    Returns
    -------
    pd.DataFrame
        Fundamentals with book equity (be) added
    """
    df = df.copy()

    # Primary measure: stockholders' equity
    # Fallback 1: common equity + preferred stock
    # Fallback 2: total assets - total liabilities
    seq_measure = (df["seq"]
        .combine_first(df["ceq"] + df["pstk"])
        .combine_first(df["at"] - df["lt"])
    )

    # Add deferred taxes
    deferred_taxes = (df["txditc"]
        .combine_first(df["txdb"] + df["itcb"])
        .fillna(0)
    )

    # Subtract preferred stock (use redemption value as primary)
    preferred = (df["pstkrv"]
        .combine_first(df["pstkl"])
        .combine_first(df["pstk"])
        .fillna(0)
    )

    # Book equity calculation
    df["be"] = seq_measure + deferred_taxes - preferred
```

```
    # Set non-positive book equity to missing
    df["be"] = df["be"].where(df["be"] > 0, np.nan)

    return df

df_fundamentals = compute_book_equity(df_fundamentals)

# Summary statistics for book equity
print("Book Equity Summary Statistics (in million VND):")
print(df_fundamentals["be"].describe().round(2))
```

```
Book Equity Summary Statistics (in million VND):
count    2.023500e+04
mean     1.031884e+12
std      4.705269e+12
min      4.404402e+07
25%      7.267610e+10
50%      1.803885e+11
75%      5.304653e+11
max      1.836314e+14
Name: be, dtype: float64
```

Operating profitability, introduced by Eugene F. Fama and French (2015), measures a firm's profits relative to its book equity. Firms with higher operating profitability tend to have higher expected returns.

```
def compute_profitability(df):
    """
    Compute operating profitability following Fama-French (2015).

    Operating profitability = (Revenue - COGS - SG&A - Interest) / Book
 ↪  Equity

    Parameters
    ----------
    df : pd.DataFrame
        Fundamentals with book equity computed

    Returns
    -------
```

```
    pd.DataFrame
        Fundamentals with operating profitability (op) added
    """
    df = df.copy()

    # Operating profit before taxes
    operating_profit = (
        df["sale"]
        - df["cogs"].fillna(0)
        - df["xsga"].fillna(0)
        - df["xint"].fillna(0)
    )

    # Scale by book equity
    df["op"] = operating_profit / df["be"]

    # Winsorize extreme values (outside 1st and 99th percentiles)
    lower = df["op"].quantile(0.01)
    upper = df["op"].quantile(0.99)
    df["op"] = df["op"].clip(lower=lower, upper=upper)

    return df

df_fundamentals = compute_profitability(df_fundamentals)
```

3.5.6 Computing Investment

Investment, measured as asset growth, captures firms' investment behavior. Eugene F. Fama and French (2015) document that firms with high asset growth (aggressive investment) tend to have lower future returns.

```
def compute_investment(df):
    """
    Compute investment (asset growth) following Fama-French (2015).

    Investment = (Total Assets_t / Total Assets_{t-1}) - 1

    Parameters
    ----------
    df : pd.DataFrame
        Fundamentals data
```

```
    Returns
    -------
    pd.DataFrame
        Fundamentals with investment (inv) added
    """
    df = df.copy()

    # Create lagged assets
    df_lag = (df[["symbol", "year", "at"]]
        .assign(year=lambda x: x["year"] + 1)
        .rename(columns={"at": "at_lag"})
    )

    # Merge lagged values
    df = df.merge(df_lag, on=["symbol", "year"], how="left")

    # Compute investment (asset growth)
    df["inv"] = df["at"] / df["at_lag"] - 1

    # Set to missing if lagged assets non-positive
    df["inv"] = df["inv"].where(df["at_lag"] > 0, np.nan)

    return df

df_fundamentals = compute_investment(df_fundamentals)
```

3.5.7 Computing Total Debt

In Vietnamese financial statements, total liabilities include non-interest-bearing items such as accounts payable and tax payables. For leverage analysis, we compute total interest-bearing debt by aggregating loan and lease obligations.

```
def compute_total_debt(df):
    """
    Compute total interest-bearing debt.

    Total Debt = Short-term loans + Finance leases (current)
                 + Current portion of long-term debt
                 + Long-term loans + Finance leases (non-current)
```

```
    Parameters
    ----------
    df : pd.DataFrame
        Fundamentals data

    Returns
    -------
    pd.DataFrame
        Fundamentals with total_debt added
    """
    df = df.copy()

    df["total_debt"] = (
        df.get("cl_loan", 0).fillna(0) +           # Short-term bank loans
        df.get("cl_finlease", 0).fillna(0) +       # Current finance leases
        df.get("cl_due_long_debt", 0).fillna(0) +  # Current portion LT debt
        df.get("nl_loan", 0).fillna(0) +           # Long-term bank loans
        df.get("nl_finlease", 0).fillna(0)         # Non-current finance
↪   leases
    )

    return df

df_fundamentals = compute_total_debt(df_fundamentals)
```

3.5.8 Applying Filters

We apply standard filters to ensure data quality: requiring positive assets, non-negative sales, and presence of core variables needed for portfolio construction.

```
# Keep only observations with required variables
required_vars = ["at", "lt", "seq", "sale"]
comp_vn = df_fundamentals.dropna(subset=required_vars)

# Apply quality filters
comp_vn = comp_vn.query("at > 0")      # Positive assets
comp_vn = comp_vn.query("sale >= 0")   # Non-negative sales

# Keep last observation per firm-year (in case of restatements)
comp_vn = (comp_vn
    .sort_values("datadate")
```

```
    .groupby(["symbol", "year"])
    .tail(1)
    .reset_index(drop=True)
)

# Diagnostic summary
print(f"Final sample: {len(comp_vn):,} firm-year observations")
print(f"Unique firms: {comp_vn['symbol'].nunique():,}")
print(f"Sample period: {comp_vn['year'].min()} - {comp_vn['year'].max()}")
```

```
Final sample: 20,091 firm-year observations
Unique firms: 1,502
Sample period: 1998 - 2023
```

3.5.9 Storing Fundamentals Data

We store the prepared fundamentals data in our local SQLite database for use in subsequent chapters.

```
comp_vn.to_sql(
    name="comp_vn",
    con=tidy_finance,
    if_exists="replace",
    index=False
)

print("Company fundamentals saved to database.")
```

```
Company fundamentals saved to database.
```

3.6 Stock Price Data

Stock price data forms the foundation of return-based analyses in empirical finance. Datacore provides comprehensive historical price data for all securities traded on HOSE, HNX, and UPCoM, including adjusted prices that account for corporate actions.

3.6.1 Downloading Price Data

We download the historical price data from Datacore's storage system. The data includes daily observations with open, high, low, close prices, trading volume, and adjustment factors.

```
# Download historical price data
prices_raw = conn.read_csv(
    bucket_name,
    "historycal_price/dataset_historical_price.csv",
    low_memory=False
)

print(f"Downloaded {len(prices_raw):,} daily price observations")
print(f"Date range: {prices_raw['date'].min()} to
 ↪  {prices_raw['date'].max()}")
```

```
Downloaded 4,307,791 daily price observations
Date range: 2010-01-04 to 2025-05-12
```

3.6.2 Processing Price Data

We clean the price data and compute adjusted prices that account for stock splits, stock dividends, and other corporate actions.

```
def process_price_data(df):
    """
    Process raw price data from Datacore.
    """
    df = df.copy()

    # Parse dates
    df["date"] = pd.to_datetime(df["date"])

    # Standardize column names
    df = df.rename(columns={
        "open_price": "open",
        "high_price": "high",
        "low_price": "low",
        "close_price": "close",
        "vol_total": "volume"
    })
```

```
    # Compute adjusted close price
    df["adjusted_close"] = df["close"] * df["adj_ratio"]

    # Standardize symbol
    df["symbol"] = df["symbol"].astype(str).str.upper().str.strip()

    # Sort for return calculation
    df = df.sort_values(["symbol", "date"])

    # Add year and month
    df["year"] = df["date"].dt.year
    df["month"] = df["date"].dt.month

    return df

prices = process_price_data(prices_raw)
```

3.6.3 Computing Shares Outstanding and Market Capitalization

Market capitalization is computed as the product of price and shares outstanding. Since Datacore provides earnings per share and net income, we can infer shares outstanding from these variables.

```
def compute_shares_outstanding(fundamentals_df):
    """
    Compute shares outstanding from fundamentals.
    """
    shares = fundamentals_df.copy()
    shares["shrout"] = shares["is_shareholders_eat"] / shares["basic_eps"]
    shares = shares[["symbol", "year", "shrout"]].dropna()

    return shares

shares_outstanding = compute_shares_outstanding(df_fundamentals)
```

```
def add_market_cap(df, shares_df):
    """
    Add market capitalization to price data.
    """
    df = df.merge(shares_df, on=["symbol", "year"], how="left")
```

```
    # Compute market cap (in million VND)
    df["mktcap"] = (df["close"] * df["shrout"]) / 1_000_000

    # Set zero or negative market cap to missing
    df["mktcap"] = df["mktcap"].where(df["mktcap"] > 0, np.nan)

    return df

prices = add_market_cap(prices, shares_outstanding)
```

3.6.4 Computing Returns and Excess Returns

We compute returns using adjusted closing prices to ensure returns correctly reflect total shareholder returns including dividends and corporate actions.

3.6.4.1 Creating Daily Dataset

1. Sequential version

```
def create_daily_dataset(df, annual_rf=0.04):
    """
    Create daily price dataset with returns and excess returns.
    """
    df = df.copy()

    # Sort by symbol and date (critical for correct return calculation)
    df = df.sort_values(["symbol", "date"]).reset_index(drop=True)

    # Remove duplicate dates within each symbol (keep last observation)
    df = df.drop_duplicates(subset=["symbol", "date"], keep="last")

    # Compute daily returns
    df["ret"] = df.groupby("symbol")["adjusted_close"].pct_change()

    # Cap extreme negative returns
    df["ret"] = df["ret"].clip(lower=-0.99)

    # Daily risk-free rate (assuming 252 trading days)
    df["risk_free"] = annual_rf / 252
```

```
    # Excess returns
    df["ret_excess"] = df["ret"] - df["risk_free"]
    df["ret_excess"] = df["ret_excess"].clip(lower=-1.0)

    # Lagged market cap
    df["mktcap_lag"] = df.groupby("symbol")["mktcap"].shift(1)

    return df

prices_daily = create_daily_dataset(prices)
```

2. Parallel version

```
from joblib import Parallel, delayed
import os

def process_daily_symbol(symbol_df, annual_rf=0.04):
    """
    Process a single symbol's daily data.
    """
    df = symbol_df.copy()

    # Sort by date (critical for correct return calculation)
    df = df.sort_values("date").reset_index(drop=True)

    # Remove duplicate dates (keep last observation if duplicates exist)
    df = df.drop_duplicates(subset=["date"], keep="last")

    # Compute daily returns
    df["ret"] = df["adjusted_close"].pct_change()

    # Replace infinite values with NaN
    df["ret"] = df["ret"].replace([np.inf, -np.inf], np.nan)

    # Cap extreme negative returns
    df["ret"] = df["ret"].clip(lower=-0.99)

    # Daily risk-free rate
    df["risk_free"] = annual_rf / 252

    # Excess returns
```

```
    df["ret_excess"] = df["ret"] - df["risk_free"]
    df["ret_excess"] = df["ret_excess"].clip(lower=-1.0)

    # Lagged market cap
    df["mktcap_lag"] = df["mktcap"].shift(1)

    return df

def create_daily_dataset_parallel(df, annual_rf=0.04):
    """
    Create daily price dataset using parallel processing.
    """
    # Ensure data is sorted before splitting
    df = df.sort_values(["symbol", "date"])

    # Split by symbol
    symbol_groups = [group for _, group in df.groupby("symbol")]

    n_jobs = max(1, os.cpu_count() - 1)
    print(f"Processing {len(symbol_groups):,} symbols using {n_jobs}
     ↪  cores...")

    results = Parallel(n_jobs=n_jobs, verbose=1)(
        delayed(process_daily_symbol)(group, annual_rf)
        for group in symbol_groups
    )

    return pd.concat(results, ignore_index=True)

prices_daily = create_daily_dataset_parallel(prices)

# Quick validation
print("\nValidation checks:")
print(f"Any duplicate (symbol, date):
 ↪  {prices_daily.duplicated(subset=['symbol', 'date']).sum()}")
print(f"Sample of non-zero returns:")
print(prices_daily[prices_daily["ret"] != 0][["symbol", "date",
 ↪  "adjusted_close", "ret"]].head(10))


prices_daily.query("symbol == 'FPT'")[["symbol", "date", "adjusted_close",
 ↪  "ret"]].head(3)
```

```
Processing 1,837 symbols using 87 cores...

[Parallel(n_jobs=87)]: Using backend LokyBackend with 87 concurrent workers.
[Parallel(n_jobs=87)]: Done  26 tasks      | elapsed:   13.4s
[Parallel(n_jobs=87)]: Done 276 tasks      | elapsed:   16.7s
[Parallel(n_jobs=87)]: Done 626 tasks      | elapsed:   17.9s
[Parallel(n_jobs=87)]: Done 1076 tasks      | elapsed:   20.0s
[Parallel(n_jobs=87)]: Done 1626 tasks      | elapsed:   22.8s
[Parallel(n_jobs=87)]: Done 1837 out of 1837 | elapsed:   23.7s finished


Validation checks:
Any duplicate (symbol, date): 0
Sample of non-zero returns:
   symbol       date  adjusted_close       ret
0     A32 2018-10-23       44.574418       NaN
27    A32 2018-11-29       55.072640  0.235521
30    A32 2018-12-04       48.188560 -0.125000
43    A32 2018-12-21       51.974804  0.078571
49    A32 2019-01-02       55.072640  0.059603
53    A32 2019-01-08       50.030370 -0.091557
74    A32 2019-02-13       44.289180 -0.114754
75    A32 2019-02-14       41.008500 -0.074074
78    A32 2019-02-19       36.087480 -0.120000
91    A32 2019-03-08       41.336568  0.145455
```

	symbol	date	adjusted_close	ret
1146076	FPT	2010-01-04	1170.9885	NaN
1146077	FPT	2010-01-05	1170.9885	0.000000
1146078	FPT	2010-01-06	1149.6978	-0.018182

```python
# Select columns
daily_columns = [
    "symbol", "date", "year", "month",
    "open", "high", "low", "close", "volume",
    "adjusted_close", "shrout", "mktcap", "mktcap_lag",
    "ret", "risk_free", "ret_excess"
]
prices_daily = prices_daily[daily_columns]
```

```
# Remove observations with missing essential variables
prices_daily = prices_daily.dropna(subset=["ret_excess", "mktcap",
 ↪  "mktcap_lag"])

print("Daily Return Summary Statistics:")
print(prices_daily["ret"].describe().round(4))
print(f"\nFinal daily sample: {len(prices_daily):,} observations")
```

```
Daily Return Summary Statistics:
count    3.462157e+06
mean     3.000000e-04
std      4.480000e-02
min     -9.900000e-01
25%     -4.900000e-03
50%      0.000000e+00
75%      4.000000e-03
max      3.250000e+01
Name: ret, dtype: float64

Final daily sample: 3,462,157 observations
```

3.6.4.2 Creating Monthly Dataset

For monthly returns, we compute returns directly from month-end adjusted prices rather than compounding daily returns. This avoids compounding errors from missing days and is the standard approach in empirical finance.

1. Sequential version

```
def create_monthly_dataset(df, annual_rf=0.04):
    """
    Create monthly price dataset with returns computed from
    month-end to month-end adjusted prices.
    """
    df = df.copy()

    # Sort by symbol and date (critical for correct return calculation)
    df = df.sort_values(["symbol", "date"]).reset_index(drop=True)

    # Remove duplicate dates within each symbol (keep last observation)
    df = df.drop_duplicates(subset=["symbol", "date"], keep="last")
```

```
    # Get month-end observations
    monthly = (df
        .groupby("symbol")
        .resample("ME", on="date")
        .agg({
            "open": "first",           # First day open
            "high": "max",             # Monthly high
            "low": "min",              # Monthly low
            "close": "last",           # Last day close
            "volume": "sum",           # Total monthly volume
            "adjusted_close": "last",  # Month-end adjusted price
            "shrout": "last",          # Month-end shares outstanding
            "mktcap": "last",          # Month-end market cap
            "year": "last",
            "month": "last"
        })
        .reset_index()
    )

    # Remove duplicate (symbol, date) after resampling (safety check)
    monthly = monthly.drop_duplicates(subset=["symbol", "date"], keep="last")

    # Sort again after resampling
    monthly = monthly.sort_values(["symbol", "date"]).reset_index(drop=True)

    # Compute monthly returns from month-end to month-end adjusted prices
    monthly["ret"] = monthly.groupby("symbol")["adjusted_close"].pct_change()

    # Cap extreme returns
    monthly["ret"] = monthly["ret"].clip(lower=-0.99)

    # Monthly risk-free rate
    monthly["risk_free"] = annual_rf / 12

    # Excess returns
    monthly["ret_excess"] = monthly["ret"] - monthly["risk_free"]
    monthly["ret_excess"] = monthly["ret_excess"].clip(lower=-1.0)

    # Lagged market cap for portfolio weighting
    monthly["mktcap_lag"] = monthly.groupby("symbol")["mktcap"].shift(1)

    return monthly
```

```
prices_monthly = create_monthly_dataset(prices)
```

2. Parallel version

```
from joblib import Parallel, delayed
import os

def process_monthly_symbol(symbol_df, annual_rf=0.04):
    """
    Process a single symbol's data to monthly frequency.
    """
    df = symbol_df.copy()

    # Sort by date (critical for correct return calculation)
    df = df.sort_values("date").reset_index(drop=True)

    # Remove duplicate dates (keep last observation if duplicates exist)
    df = df.drop_duplicates(subset=["date"], keep="last")

    # Set date as index for resampling
    df = df.set_index("date")

    # Resample to monthly
    monthly = df.resample("ME").agg({
        "symbol": "last",
        "open": "first",
        "high": "max",
        "low": "min",
        "close": "last",
        "volume": "sum",
        "adjusted_close": "last",
        "shrout": "last",
        "mktcap": "last",
        "year": "last",
        "month": "last"
    }).reset_index()

    # Remove rows where symbol is NaN (months with no trading)
    monthly = monthly.dropna(subset=["symbol"])

    # Sort by date
```

```
    monthly = monthly.sort_values("date").reset_index(drop=True)

    # Compute monthly returns
    monthly["ret"] = monthly["adjusted_close"].pct_change()

    # Replace infinite values with NaN
    monthly["ret"] = monthly["ret"].replace([np.inf, -np.inf], np.nan)

    # Cap extreme returns
    monthly["ret"] = monthly["ret"].clip(lower=-0.99)

    # Monthly risk-free rate
    monthly["risk_free"] = annual_rf / 12

    # Excess returns
    monthly["ret_excess"] = monthly["ret"] - monthly["risk_free"]
    monthly["ret_excess"] = monthly["ret_excess"].clip(lower=-1.0)

    # Lagged market cap
    monthly["mktcap_lag"] = monthly["mktcap"].shift(1)

    return monthly

def create_monthly_dataset_parallel(df, annual_rf=0.04):
    """
    Create monthly price dataset using parallel processing.
    """
    # Ensure data is sorted before splitting
    df = df.sort_values(["symbol", "date"])

    # Split by symbol
    symbol_groups = [group for _, group in df.groupby("symbol")]

    n_jobs = max(1, os.cpu_count() - 1)
    print(f"Processing {len(symbol_groups):,} symbols using {n_jobs}
     ↪  cores...")

    results = Parallel(n_jobs=n_jobs, verbose=1)(
        delayed(process_monthly_symbol)(group, annual_rf)
        for group in symbol_groups
    )

    return pd.concat(results, ignore_index=True)
```

```python
prices_monthly = create_monthly_dataset_parallel(prices)

# Validation checks
print("\nValidation checks:")
print(f"Any duplicate (symbol, date):
 ↪  {prices_monthly.duplicated(subset=['symbol', 'date']).sum()}")
print(f"\nSample of non-zero returns:")
print(prices_monthly[prices_monthly["ret"] != 0][["symbol", "date",
 ↪  "adjusted_close", "ret"]].head(10))

prices_monthly.query("symbol == 'FPT'")[["symbol", "date", "adjusted_close",
 ↪  "ret"]].head(3)
```

```
Processing 1,837 symbols using 87 cores...

[Parallel(n_jobs=87)]: Using backend LokyBackend with 87 concurrent workers.
[Parallel(n_jobs=87)]: Done  26 tasks      | elapsed:    0.2s
[Parallel(n_jobs=87)]: Done 378 tasks      | elapsed:    2.6s
[Parallel(n_jobs=87)]: Done 974 tasks      | elapsed:    6.2s
[Parallel(n_jobs=87)]: Done 1424 tasks      | elapsed:    9.3s
[Parallel(n_jobs=87)]: Done 1837 out of 1837 | elapsed:   12.1s finished


Validation checks:
Any duplicate (symbol, date): 0

Sample of non-zero returns:
   symbol       date  adjusted_close       ret
0     A32 2018-10-31       44.574418       NaN
1     A32 2018-11-30       55.072640  0.235521
2     A32 2018-12-31       51.974804 -0.056250
3     A32 2019-01-31       50.030370 -0.037411
4     A32 2019-02-28       36.087480 -0.278689
5     A32 2019-03-31       41.828670  0.159091
7     A32 2019-05-31       43.304976  0.035294
8     A32 2019-06-30       35.929125 -0.170323
9     A32 2019-07-31       37.525975  0.044444
10    A32 2019-08-31       38.324400  0.021277
```

	symbol	date	adjusted_close	ret
55963	FPT	2010-01-31	1092.9226	NaN
55964	FPT	2010-02-28	1107.1164	0.012987
55965	FPT	2010-03-31	1185.1823	0.070513

```
# Select columns (same structure as daily)
monthly_columns = [
    "symbol", "date", "year", "month",
    "open", "high", "low", "close", "volume",
    "adjusted_close", "shrout", "mktcap", "mktcap_lag",
    "ret", "risk_free", "ret_excess"
]
prices_monthly = prices_monthly[monthly_columns]

# Remove observations with missing essential variables
prices_monthly = prices_monthly.dropna(subset=["ret_excess", "mktcap",
 ↪  "mktcap_lag"])

print("Monthly Return Summary Statistics:")
print(prices_monthly["ret"].describe().round(4))
print(f"\nFinal monthly sample: {len(prices_monthly):,} observations")
```

```
Monthly Return Summary Statistics:
count    165499.0000
mean          0.0042
std           0.1862
min          -0.9900
25%          -0.0703
50%           0.0000
75%           0.0553
max          12.7500
Name: ret, dtype: float64

Final monthly sample: 165,499 observations
```

3.6.5 Storing Price Data

```
prices_daily.to_sql(
    name="prices_daily",
```

```
    con=tidy_finance,
    if_exists="replace",
    index=False
)
print("Daily price data saved to database.")

prices_monthly.to_sql(
    name="prices_monthly",
    con=tidy_finance,
    if_exists="replace",
    index=False
)
print("Monthly price data saved to database.")
```

3.7 Descriptive Statistics

Before proceeding to asset pricing analyses, we examine the characteristics of our sample to understand the Vietnamese equity market's evolution and composition.

3.7.1 Market Evolution Over Time

We first examine how the number of listed securities has grown over time.

```
securities_over_time = (prices_monthly
    .groupby("date")
    .agg(
        n_securities=("symbol", "nunique"),
        total_mktcap=("mktcap", "sum")
    )
    .reset_index()
)
```

```
securities_figure = (
    ggplot(securities_over_time, aes(x="date", y="n_securities"))
    + geom_line(color="steelblue", size=1)
    + labs(
        x="",
        y="Number of Securities",
        title="Growth of Vietnamese Stock Market"
```

```
    )
    + scale_x_datetime(date_breaks="2 years", date_labels="%Y")
    + scale_y_continuous(labels=comma_format())
    + theme_minimal()
)
securities_figure.show()
```

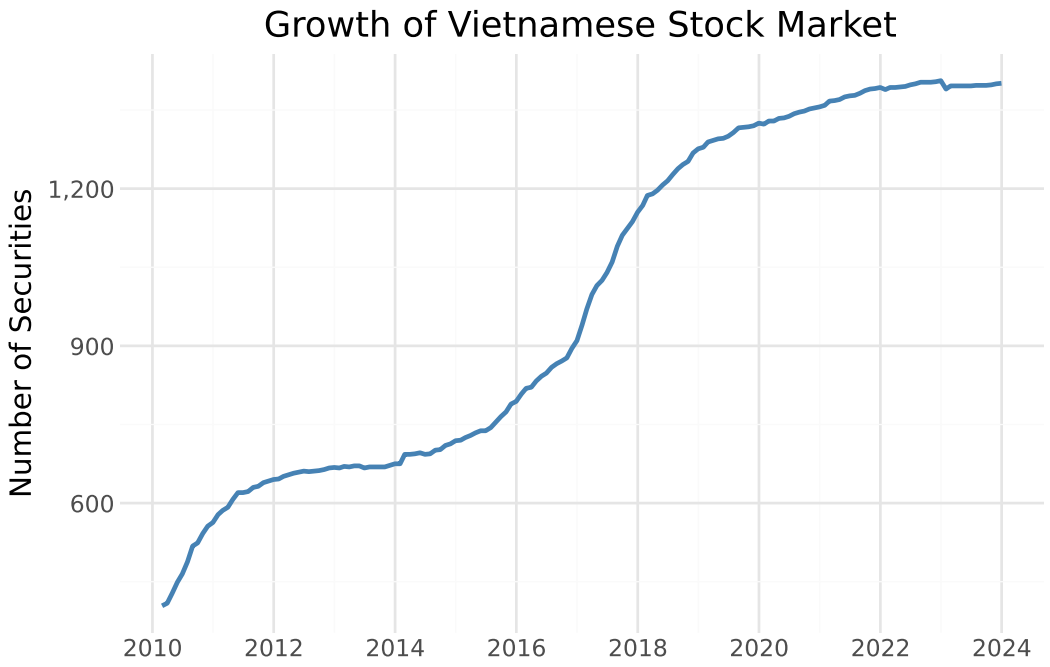


Figure 3.1: The figure shows the monthly number of securities in the Vietnamese stock market sample.

3.7.2 Market Capitalization Evolution

The aggregate market capitalization reflects the overall size and development of the Vietnamese equity market.

```
mktcap_figure = (
    ggplot(securities_over_time, aes(x="date", y="total_mktcap / 1000"))
    + geom_line(color="darkgreen", size=1)
    + labs(
        x="",
        y="Market Cap (Trillion VND)",
```

```
        title="Total Market Capitalization of Vietnamese Equities"
    )
    + scale_x_datetime(date_breaks="2 years", date_labels="%Y")
    + scale_y_continuous(labels=comma_format())
    + theme_minimal()
)
mktcap_figure.show()
```



Figure 3.2: The figure shows the total market capitalization of Vietnamese listed companies over time.

3.7.3 Return Distribution

Understanding the distribution of monthly returns helps identify potential data quality issues and characterize market risk.

```
return_distribution = (
    ggplot(prices_monthly, aes(x="ret_excess"))
    + geom_histogram(
        binwidth=0.02,
        fill="steelblue",
```

```
        color="white",
        alpha=0.7
    )
    + labs(
        x="Monthly Excess Return",
        y="Frequency",
        title="Distribution of Monthly Excess Returns"
    )
    + scale_x_continuous(limits=(-0.5, 0.5))
    + theme_minimal()
)
return_distribution.show()
```

```
/home/mikenguyen/project/tidyfinance/.venv/lib/python3.13/site-packages/plotnine/layer.py:293
/home/mikenguyen/project/tidyfinance/.venv/lib/python3.13/site-packages/plotnine/layer.py:374
```

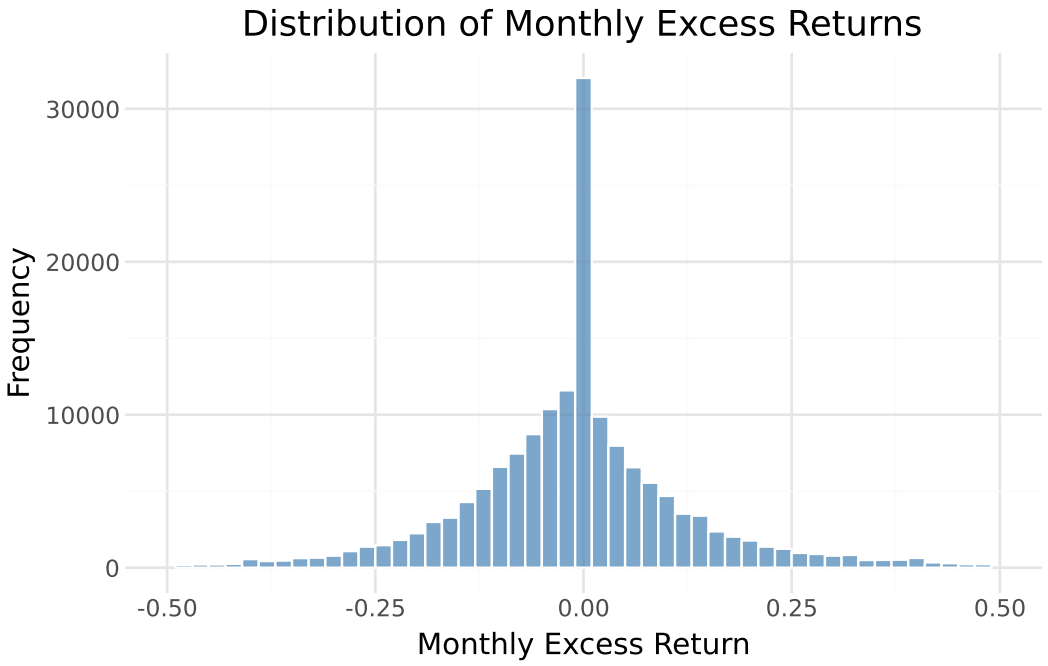


Figure 3.3: Distribution of monthly excess returns for Vietnamese stocks.

3.7.4 Coverage of Book Equity

Book equity is essential for constructing value portfolios. We examine what fraction of our sample has book equity data available over time.

```
# Merge prices with fundamentals
coverage_data = (prices_monthly
    .assign(year=lambda x: x["date"].dt.year)
    .groupby(["symbol", "year"])
    .tail(1)
    .merge(comp_vn[["symbol", "year", "be"]],
           on=["symbol", "year"],
           how="left")
)

# Compute coverage by year
be_coverage = (coverage_data
    .groupby("year")
    .apply(lambda x: pd.Series({
        "share_with_be": x["be"].notna().mean()
    }))
    .reset_index()
)

coverage_figure = (
    ggplot(be_coverage, aes(x="year", y="share_with_be"))
    + geom_line(color="darkorange", size=1)
    + geom_point(color="darkorange", size=2)
    + labs(
        x="Year",
        y="Share with Book Equity",
        title="Coverage of Book Equity Data"
    )
    + scale_y_continuous(labels=percent_format(), limits=(0, 1))
    + theme_minimal()
)
coverage_figure.show()
```

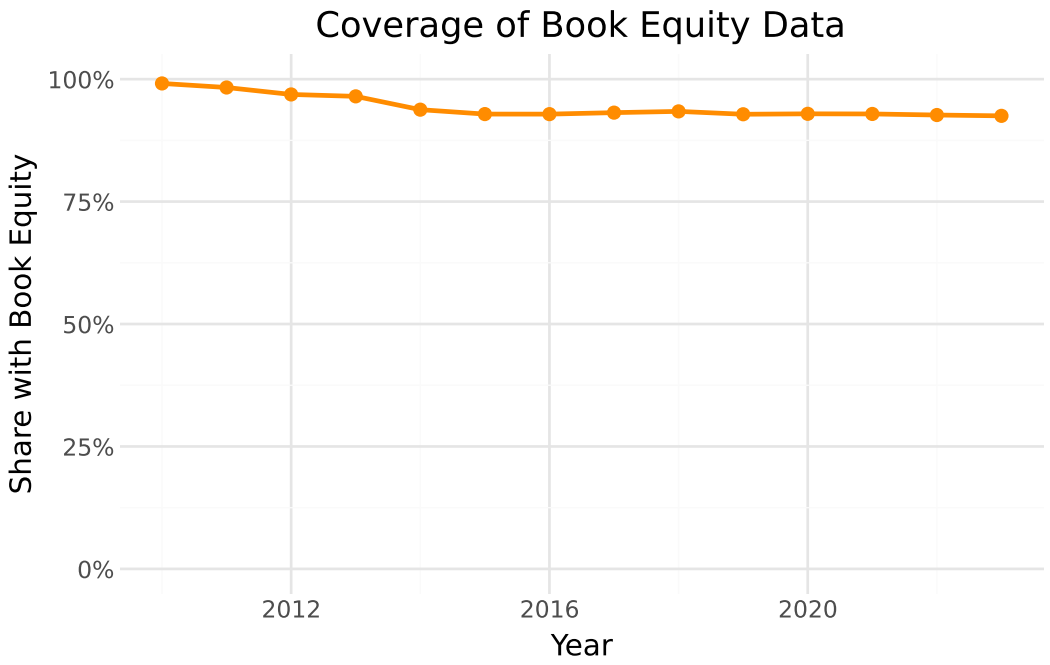


Figure 3.4: Share of securities with available book equity data by year.

3.8 Merging Stock and Fundamental Data

The final step links price data with fundamental data using the stock symbol as the common identifier. This merged dataset forms the basis for constructing portfolios sorted on firm characteristics.

```
# Example: Create merged dataset for end-of-June each year
merged_data = (prices_monthly
    .query("month == 6")
    .merge(
        comp_vn[["symbol", "year", "be", "op", "inv", "at"]],
        on=["symbol", "year"],
        how="left",
        suffixes=("", "_fundamental")
    )
)

# Convert BE from VND to BILLION VND
merged_data["be"] = merged_data["be"] / 1e9
```

```
# Compute book-to-market ratio
merged_data["bm"] = merged_data["be"] / merged_data["mktcap"]

merged_data.loc[
    (merged_data["bm"] <= 0) |
    (merged_data["bm"] > 20),
    "bm"
] = pd.NA


merged_data["bm"].describe(percentiles=[.01, .1, .5, .9, .99])

print(f"Merged observations: {len(merged_data):,}")
print(f"With book-to-market: {merged_data['bm'].notna().sum():,}")
merged_data.head(3)
merged_data.describe()
merged_data
```

```
Merged observations: 13,756
With book-to-market: 12,859
```

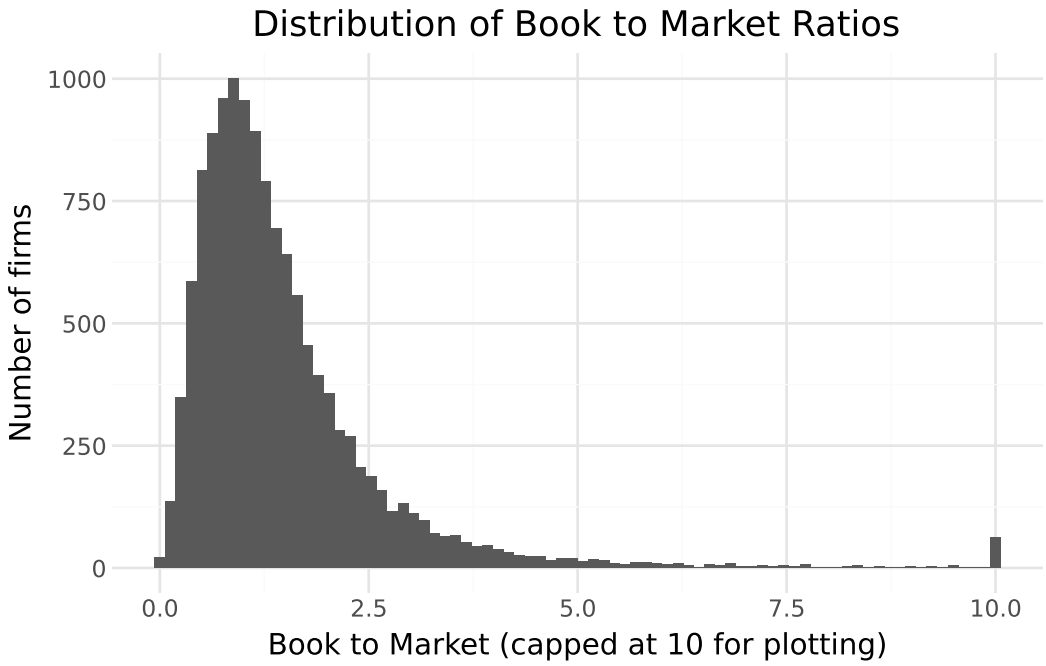
	symbol	date	year	month	open	high	low	close	volume	adjusted_close	...	m
0	A32	2019-06-30	2019.0	6.0	26.4	26.4	21.0	22.5	3700	35.929125	...	15
1	A32	2020-06-30	2020.0	6.0	25.0	26.3	24.5	26.3	7500	38.811173	...	17
2	A32	2021-06-30	2021.0	6.0	30.2	37.0	29.5	32.0	78400	45.363520	...	21
3	A32	2022-06-30	2022.0	6.0	30.9	35.5	25.0	35.3	15200	47.503210	...	24
4	A32	2023-06-30	2023.0	6.0	30.1	33.5	29.2	29.4	2400	35.064204	...	19
...	...	...	...	...	...	...	...	...	...	...	...	...
13751	YTC	2019-06-30	2019.0	6.0	70.0	79.9	70.0	79.9	38900	171.451817	...	24
13752	YTC	2020-06-30	2020.0	6.0	88.5	88.5	77.0	87.0	150640	180.966960	...	26
13753	YTC	2021-06-30	2021.0	6.0	76.0	115.5	61.0	61.0	34100	126.884880	...	18
13754	YTC	2022-06-30	2022.0	6.0	68.0	68.0	65.0	65.5	200	136.245240	...	20
13755	YTC	2023-06-30	2023.0	6.0	59.0	59.0	59.0	59.0	49545	122.724720	...	18

```
from plotnine import *
import numpy as np

bm_plot_data = (
    merged_data[["bm"]]
```

```
      .dropna()
      .assign(bm_plot=lambda x: x["bm"].clip(upper=10))
)

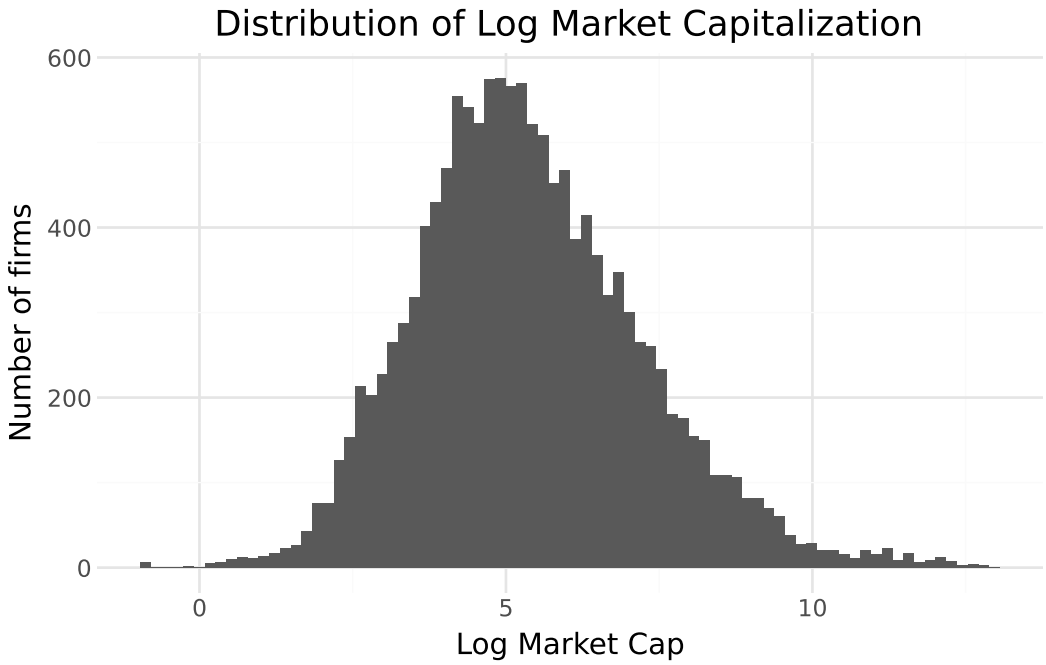
(
    ggplot(bm_plot_data, aes(x="bm_plot")) +
    geom_histogram(bins=80) +
    labs(
        title="Distribution of Book to Market Ratios",
        x="Book to Market (capped at 10 for plotting)",
        y="Number of firms"
    ) +
    theme_minimal()
)
```




```
size_plot_data = (
    merged_data[["mktcap_lag"]]
      .dropna()
      .assign(log_size=lambda x: np.log(x["mktcap_lag"]))
)

(
```

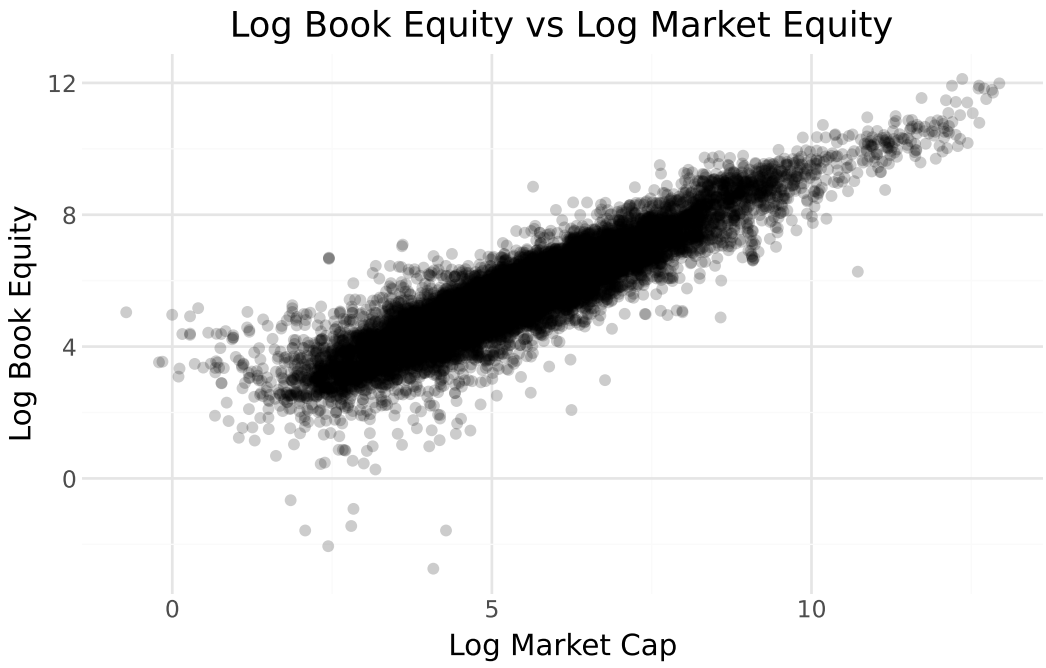
```
    ggplot(size_plot_data, aes(x="log_size")) +
    geom_histogram(bins=80) +
    labs(
        title="Distribution of Log Market Capitalization",
        x="Log Market Cap",
        y="Number of firms"
    ) +
    theme_minimal()
)
```




```
scatter_data = (
    merged_data[["be", "mktcap_lag"]]
      .dropna()
      .assign(
          log_be=lambda x: np.log(x["be"]),
          log_me=lambda x: np.log(x["mktcap_lag"])
      )
)

(
    ggplot(scatter_data, aes(x="log_me", y="log_be")) +
    geom_point(alpha=0.2) +
```

```
    labs(
        title="Log Book Equity vs Log Market Equity",
        x="Log Market Cap",
        y="Log Book Equity"
    ) +
    theme_minimal()
)
```



3.9 Key Takeaways

1. **Datacore provides unified access** to Vietnamese financial data through a modern cloud-based infrastructure, eliminating the need to aggregate data from multiple fragmented sources.
2. **Company fundamentals** from Datacore include comprehensive balance sheet, income statement, and cash flow data prepared according to Vietnamese Accounting Standards, which we map to standard variables used in international research.
3. **Book equity computation** follows the Fama-French methodology, accounting for deferred taxes and preferred stock to ensure comparability with US-based studies.

4. **Stock price data** includes adjustment factors for corporate actions, enabling accurate return calculations over long horizons.
5. **Monthly frequency** is standard for asset pricing research, reducing noise while maintaining sufficient observations for statistical inference.
6. **Risk-free rate approximation** uses Vietnamese government bond yields as a proxy, given the absence of a standardized short-term rate series comparable to US Treasury bills.
7. **Data quality validation** through descriptive statistics and visualization helps identify potential issues before conducting formal analyses.
8. **Batch processing** enables efficient handling of large daily datasets that would otherwise exceed memory constraints.

4 Market Microstructure in Vietnam

> **Note**
>
> In this chapter, we examine how the institutional design of Vietnamese equity markets, such as trading sessions, price limits, order types, and investor composition, shapes observed prices, returns, and liquidity. We quantify microstructure frictions and demonstrate why ignoring these frictions leads to biased inference in asset pricing tests.

Market microstructure is the study of how trading rules, order handling mechanisms, and market design affect price formation, transaction costs, and liquidity. The field, pioneered by Kyle (1985), Glosten and Milgrom (1985), and Hasbrouck (2007), provides the analytical toolkit for understanding why observed prices may deviate from fundamental values and for how long.

In developed markets with continuous electronic trading, designated market makers, and minimal regulatory constraints on price movement, microstructure frictions are typically second-order concerns for researchers working at monthly or lower frequencies. In Vietnam's equity markets, this is emphatically not the case. Daily price limits, thin trading, a predominantly retail investor base, discrete tick sizes, and the absence of formal market-making arrangements generate frictions that propagate into monthly returns, distort factor loadings, and bias portfolio-level inference. Any serious empirical analysis of Vietnamese equities must therefore begin with a careful assessment of market microstructure.

This chapter provides that assessment. We first describe the institutional architecture of Vietnamese equity trading. We then develop diagnostics for the most consequential frictions, such as price limit hits, zero-return days, illiquidity, and non-synchronous trading, and quantify their severity in the cross-section of listed firms. Finally, we derive practical guidance for adjusting portfolio construction and asset pricing tests.

4.1 What Is Market Microstructure?

The textbook assumption of frictionless markets implies that prices continuously and costlessly incorporate information. Under this assumption, the observed return on any asset at any frequency equals the "true" return dictated by fundamentals. Market microstructure relaxes

this assumption by recognizing that prices are generated by a specific trading process with real costs, constraints, and imperfections.

The canonical framework of Kyle (1985) models a market with three types of participants:

1. an informed trader who knows the asset's fundamental value,
2. noise traders who trade for liquidity reasons, and
3. a market maker who sets prices to break even in expectation.

The key insight is that the market maker cannot distinguish informed from uninformed order flow, so prices adjust gradually to information, creating a wedge between the transaction price and the fundamental value. The size of this wedge (the bid-ask spread) and the speed of price adjustment (market depth) are the core objects of microstructure theory.

Glosten and Milgrom (1985) extend this framework to a sequential trade setting and show that the bid-ask spread has two components: an adverse selection component (compensation for trading against informed traders) and an order processing component (compensation for the mechanical costs of trading). R. D. Huang and Stoll (1997) further decompose the spread into realized spread and price impact components. These decompositions are important because they reveal different sources of trading costs and have different implications for market quality.

For empirical asset pricing, the key question is: at what frequency and under what conditions do microstructure effects become negligible? In highly liquid markets, Bali, Engle, and Murray (2016a) argue that monthly returns are largely free of microstructure contamination. In Vietnam, as we demonstrate below, this is not the case. Microstructure effects persist at monthly and even quarterly frequencies for a substantial fraction of listed firms.

4.1.1 The Microstructure-Asset Pricing Interface

The interface between microstructure and asset pricing operates through several channels. First, illiquidity itself may be a priced risk factor. Amihud (2002) shows that expected illiquidity is positively related to expected stock returns, implying a liquidity premium. Pástor and Stambaugh (2003) develop an equilibrium model in which liquidity risk (i.e., the covariance of a stock's liquidity with market liquidity) commands a risk premium. Second, microstructure noise in prices biases estimated betas, factor loadings, and test statistics. Scholes and Williams (1977) first identified this bias in the context of non-synchronous trading, and Dimson (1979) proposed an aggregated-coefficients estimator to correct it. Third, price limits and other regulatory constraints censor the return distribution, creating truncation bias in volatility estimates, return moments, and extreme-value statistics (K. A. Kim and Rhee 1997).

Table 4.1 summarizes these channels and their empirical consequences.

Table 4.1: Channels Through Which Microstructure Affects Asset Pricing

Channel	Mechanism	Empirical Consequence
Illiquidity premium	Compensation for bearing transaction costs and inventory risk	Cross-sectional return predictability by liquidity measures
Non-synchronous trading	Infrequent trading creates stale prices	Downward-biased betas, attenuated correlations, and spurious lead-lag
Price limits	Regulatory censoring of daily returns	Truncated return distributions, volatility spillover, and artificial autocorrelation
Discrete tick sizes	Prices constrained to a grid	Bid-ask bounce, return discreteness, biased volatility
Investor composition	Retail-dominated order flow	Noise trading, herding, sentiment-driven pricing

4.2 Trading Architecture in Vietnam

Vietnam operates two stock exchanges: the Ho Chi Minh Stock Exchange (HOSE), established in 2000, and the Hanoi Stock Exchange (HNX), established in 2005. HOSE lists larger firms and accounts for the majority of market capitalization and trading volume. HNX lists smaller firms and also operates the Unlisted Public Company Market (UPCoM) for firms that have not yet met full listing requirements. All three venues operate electronic limit order book systems without designated market makers.

4.2.1 Exchange Characteristics

Table 4.2 presents the key structural differences between HOSE, HNX, and UPCoM. These differences have direct implications for liquidity, price discovery, and the severity of microstructure frictions.

Table 4.2: Exchange Comparison: HOSE, HNX, and UPCoM

Feature	HOSE	HNX	UPCoM
Established	2000	2005	2009
Listing tier	Large-cap	Mid/small-cap	Pre-listing
Daily price limit	$\pm$ 7%	$\pm$ 10%	$\pm$ 15%
Tick size regime	Tiered by price	Tiered by price	100 VND
Trading lot	100 shares	100 shares	100 shares
Short selling	Limited	Not available	Not available

Feature	HOSE	HNX	UPCoM
Foreign ownership cap	Industry-specific	Industry-specific	Industry-specific

The heterogeneous price limit bands across exchanges create a natural experiment for studying limit effects. HOSE's tighter $\pm$ 7% band means that large-cap stocks are more frequently constrained than mid-cap stocks on HNX, conditional on the same information shock. UPCoM's wider $\pm$ 15% band provides the least constrained environment, though its stocks are also the least liquid.

4.2.2 Trading Sessions

Each exchange operates a structured trading day with distinct sessions. Understanding session structure is essential because price formation mechanisms differ across sessions, and certain sessions are disproportionately important for benchmark pricing (Table 4.3).

Table 4.3: Trading Session Structure on HOSE

Session	Time	Mechanism	Price Discovery Role
Pre-opening	08:30–09:00	Order entry only, no matching	Reveals pre-open demand/supply
Opening auction (ATO)	09:00–09:15	Batch auction, single price	Sets opening price from accumulated orders
Continuous trading (Morning)	09:15–11:30	Continuous limit order matching	Primary price discovery
Lunch break	11:30–13:00	No trading	—
Continuous trading (Afternoon)	13:00–14:30	Continuous limit order matching	Primary price discovery
Closing auction (ATC)	14:30–14:45	Batch auction, single price	Sets closing price (benchmark)
Post-closing	14:45–15:00	Put-through (negotiated) trades	Block and negotiated transactions

The closing auction (ATC) deserves particular attention. The ATC price is the official closing price used for index calculation, NAV computation, and margin requirements. Because it is determined by a single-bid auction, it can be manipulated by strategically timed orders, a phenomenon documented in numerous emerging markets (Comerton-Forde and Tang 2009; Hillion and Suominen 2004). Researchers using daily closing prices should be aware that ATC prices may not reflect the continuous-session equilibrium, particularly for less liquid stocks where a single large order can move the closing price.

4.2.3 Order Types and Matching Rules

Vietnamese exchanges support a limited set of order types compared to developed markets (Table 4.4).

Table 4.4: Available Order Types

Order Type	Description	Availability
Limit order (LO)	Specifies price and quantity	All sessions
Market order (ATO/ATC)	Matches at auction price	Auction sessions only
Market-to-limit (MTL)	Converts to limit at best available	HNX only

The absence of iceberg orders, stop orders, and hidden orders means that the full limit order book is visible to all participants. While this enhances pre-trade transparency, it also means that large institutional orders face significant information leakage risk, which may deter institutional participation and reduce market depth.

Orders are matched on a strict price-time priority basis during continuous sessions. During auction sessions, a single clearing price is determined that maximizes executed volume. If multiple prices satisfy this criterion, the price closest to the previous closing price is selected.

4.2.4 Tick Size Structure

Tick sizes on HOSE are tiered by price level, which creates discontinuities in the bid-ask spread as a percentage of price (Table 4.5).

Table 4.5: Tick Size Schedule on HOSE

Price Range (VND)	Tick Size (VND)	Minimum Spread as % of Midpoint
$<$ 10,000	10	0.10% at 10,000
10,000–49,900	50	0.10% at 50,000
50,000	100	0.20% at 50,000

The jump from a 50 VND tick to a 100 VND tick at the 50,000 VND boundary means that the minimum percentage spread doubles discontinuously. This creates a "tick size cliff" that can affect the cross-sectional distribution of bid-ask spreads and, consequently, the measurement of illiquidity (D. H. Vo and Doan 2023). Bessembinder (2003) document similar effects in other markets with tiered tick structures.

4.2.5 Investor Composition

The Vietnamese equity market is predominantly driven by retail investors. While foreign institutional investors account for a meaningful share of market capitalization (particularly in blue-chip stocks subject to foreign ownership limits), daily trading volume is overwhelmingly generated by domestic retail accounts.

This retail dominance has several consequences for microstructure. First, retail investors tend to submit smaller orders and trade more frequently, generating high message-to-trade ratios but limited depth at each price level. Second, retail order flow is more susceptible to herding and sentiment, which can amplify momentum and generate excess volatility (Barber et al. 2009; Kaniel et al. 2012). Third, the limited institutional presence means that sophisticated liquidity provision is scarce, particularly in mid- and small-cap stocks.

4.3 Price Limits and Their Consequences

Vietnam enforces daily price limits on all listed equities. A stock's price cannot move beyond a fixed percentage of the previous day's closing price within a single trading day. The limit bands are ± 7% on HOSE, ± 10% on HNX, and ± 15% on UPCoM.

4.3.1 Theoretical Framework

Price limits were introduced with the stated goal of reducing volatility and preventing panic-driven price dislocations. However, the academic literature presents a more nuanced picture. The "magnet effect" hypothesis (Subrahmanyam 1994) predicts that price limits actually accelerate price movement toward the limit as traders rush to execute before the limit is hit. The "delayed price discovery" hypothesis (Eugene F. Fama and French 1989) argues that limits merely postpone inevitable price adjustments, creating volatility spillover into subsequent days.

Formally, let P_t^* denote the equilibrium price on day t and P_{t-1}^c the previous closing price. The observed return is:

$$r_t^{obs} = \begin{cases} \bar{L} & \text{if } r_t^* \geq \bar{L} \\ r_t^* & \text{if } \underline{L} < r_t^* < \bar{L} \\ \underline{L} & \text{if } r_t^* \leq \underline{L} \end{cases} \tag{4.1}$$

where $r_t^* = \ln(P_t^*/P_{t-1}^c)$ is the latent (unconstrained) return, $\bar{L}$ is the upper limit, and $\underline{L}$ is the lower limit. The observed return r_t^{obs} is a censored version of the true return. This censoring has several consequences:

1. **Truncated moments**: The observed variance $\text{Var}(r_t^{obs}) < \text{Var}(r_t^*)$ because extreme returns are clipped. This biases downward any volatility-based risk measure.
2. **Artificial autocorrelation**: When $r_t^{obs} = \bar{L}$ and $r_{t+1}^{obs} > 0$ (continued adjustment the next day), the return series exhibits positive autocorrelation that is purely mechanical, not informational.
3. **Volatility spillover**: Define excess volatility on day $t+1$ as $\sigma_{t+1}^2 - E[\sigma_{t+1}^2 | \text{no limit hit on day } t]$. K. A. Kim and Rhee (1997) and Chu and Qiu (2019) document significant positive spillover, where days following limit hits exhibit abnormally high volatility.
4. **Biased extreme value statistics**: Measures such as Value-at-Risk, Expected Shortfall, and maximum drawdown are mechanically bounded by the limit, understating true tail risk.

4.3.2 Detecting Price Limit Hits

We now implement a diagnostic to detect price limit hits in the daily data.

```
import pandas as pd
import numpy as np
import sqlite3

# Load daily price data
tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")

# Assume prices_daily contains: symbol, date, close, exchange
prices_daily = pd.read_sql_query(
    # , exchange
    sql="""
        SELECT symbol, date, close
        FROM prices_daily
    """,
    con=tidy_finance,
    parse_dates=["date"]
).dropna()
```

```
# Define limit bands by exchange
limit_bands = {"HOSE": 0.07, "HNX": 0.10, "UPCoM": 0.15}

prices_daily = prices_daily.sort_values(["symbol", "date"])
prices_daily["prev_close"] = prices_daily.groupby("symbol")["close"].shift(1)
prices_daily["ret"] = prices_daily["close"] / prices_daily["prev_close"] - 1
```

```
prices_daily["limit_band"] = prices_daily["exchange"].map(limit_bands)

# A limit hit occurs when the return is within 0.1% of the theoretical limit
tolerance = 0.001
prices_daily["upper_hit"] = (
    prices_daily["ret"] >= prices_daily["limit_band"] - tolerance
)
prices_daily["lower_hit"] = (
    prices_daily["ret"] <= -prices_daily["limit_band"] + tolerance
)
prices_daily["limit_hit"] = (
    prices_daily["upper_hit"] | prices_daily["lower_hit"]
)
```

4.3.3 Frequency of Limit Hits

4.3.4 Volatility Spillover Test

Following K. A. Kim and Rhee (1997), we test whether days following a limit hit exhibit abnormally high volatility. Define the dummy variable $D_t = 1$ if a limit hit occurred on day t, and estimate:

$$\sigma_{t+1}^2 = \alpha + \beta D_t + \gamma \sigma_t^2 + \varepsilon_{t+1} \tag{4.2}$$

where σ_t^2 is the squared return. A positive and significant β indicates volatility spillover attributable to the price limit.

```
import statsmodels.api as sm

# Panel-level volatility spillover test
prices_daily["sq_ret"] = prices_daily["ret"] ** 2
prices_daily["sq_ret_lead"] =
 ↪  prices_daily.groupby("symbol")["sq_ret"].shift(-1)
prices_daily["limit_hit_int"] = prices_daily["limit_hit"].astype(int)

spillover_data = prices_daily.dropna(subset=["sq_ret_lead", "sq_ret"])

X = sm.add_constant(spillover_data[["limit_hit_int", "sq_ret"]])
y = spillover_data["sq_ret_lead"]
```

```
prices_daily["year_month"] = prices_daily["date"].dt.to_period("M")

limit_hit_monthly = (
    prices_daily
    .groupby(["year_month", "exchange"])
    .agg(
        total_obs=("limit_hit", "count"),
        limit_hits=("limit_hit", "sum")
    )
    .reset_index()
)
limit_hit_monthly["hit_rate"] = (
    limit_hit_monthly["limit_hits"] / limit_hit_monthly["total_obs"]
)
limit_hit_monthly["date"] = limit_hit_monthly["year_month"].dt.to_timestamp()

fig, ax = plt.subplots(figsize=(8, 4))

for exchange, color in zip(
    ["HOSE", "HNX", "UPCoM"], ["#2C73D2", "#FF6B6B", "#5DCEAF"]
):
    subset = limit_hit_monthly[limit_hit_monthly["exchange"] == exchange]
    ax.plot(
        subset["date"], subset["hit_rate"] * 100,
        label=exchange, color=color, linewidth=1.2
    )

ax.set_ylabel("Limit Hit Rate (%)")
ax.set_xlabel("")
ax.legend(frameon=False)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.tight_layout()
plt.show()
```

Figure 4.1

```
model = sm.OLS(y, X).fit(cov_type="cluster", cov_kwds={"groups":
 ↪  spillover_data["symbol"]})

spillover_results = pd.DataFrame({
    "Coefficient": model.params,
    "Std. Error": model.bse,
    "t-stat": model.tvalues,
    "p-value": model.pvalues
}).round(6)

print(spillover_results)
```

Tip

A significant positive coefficient on the limit hit dummy confirms that Vietnamese price limits do not eliminate volatility, they merely redistribute it across days. This has direct implications for risk management: daily VaR measures computed from censored returns understate true risk exposure.

4.3.5 Return Autocorrelation Induced by Price Limits

Price limits mechanically induce positive autocorrelation in returns. To quantify this, we compute the first-order autocorrelation coefficient separately for stocks that hit limits frequently versus those that do not.

The expected pattern is a monotonically increasing autocorrelation from the Low to High limit-hit group, confirming that the observed serial dependence in returns is at least partly an artifact of price censoring rather than genuine return predictability.

4.4 Liquidity, Thin Trading, and Zero Returns

Liquidity (i.e., the ability to trade quickly at low cost without moving the price) is a first-order concern in Vietnamese equities. A substantial fraction of listed firms, particularly on HNX and UPCoM, experience chronic illiquidity characterized by infrequent trading, wide bid-ask spreads, and frequent zero-return days.

Table 4.6: Return Autocorrelation by Price Limit Hit Frequency

```
# Classify stocks by limit hit frequency
stock_limit_freq = (
    prices_daily
    .groupby("symbol")
    .agg(
        hit_rate=("limit_hit", "mean"),
        n_obs=("ret", "count")
    )
    .query("n_obs >= 250")  # At least 1 year of data
)

stock_limit_freq["limit_group"] = pd.qcut(
    stock_limit_freq["hit_rate"], q=3,
    labels=["Low", "Medium", "High"]
)

# Compute autocorrelation by group
def compute_autocorr(group_symbols):
    subset = prices_daily[prices_daily["symbol"].isin(group_symbols)].copy()
    subset["ret_lag"] = subset.groupby("symbol")["ret"].shift(1)
    return subset[["ret", "ret_lag"]].dropna().corr().iloc[0, 1]

autocorr_results = []
for group in ["Low", "Medium", "High"]:
    symbols = stock_limit_freq[stock_limit_freq["limit_group"] ==
↪   group].index
    ac = compute_autocorr(symbols)
    n_stocks = len(symbols)
    avg_hit_rate = stock_limit_freq.loc[symbols, "hit_rate"].mean()
    autocorr_results.append({
        "Group": group,
        "N Stocks": n_stocks,
        "Avg Limit Hit Rate (%)": round(avg_hit_rate * 100, 2),
        "AR(1)": round(ac, 4)
    })

pd.DataFrame(autocorr_results).style.hide(axis="index")
```

4.4.1 Measuring Liquidity

The academic literature has developed numerous liquidity measures, each capturing a different dimension of market quality. @#tbl-liquidity-measures summarizes the measures most applicable to Vietnamese data, given typical data availability.

Table 4.7: Liquidity Measures for Vietnamese Equities

Measure	Formula	Interpretation	Data Required
Turnover ratio	$\text{TO}_{i,t} = \frac{\text{Volume}_{i,t}}{\text{Shares Outstanding}_i}$	Trading intensity relative to float	Volume, shares outstanding
Amihud illiquidity	$\text{ILLIQ}_{i,t} = \frac{1}{D}\sum_{d=1}^{D} \frac{\lvert r_{i,d}\rvert}{V_{i,d}}$	Price impact per unit of volume	Daily returns, daily volume
Zero-return proportion	$\text{ZR}_{i,t} = \frac{\#\{d : r_{i,d}=0\}}{D}$	Frequency of non-trading or stale pricing	Daily returns
Roll spread	$\hat{S}_i = 2\sqrt{-\text{Cov}(r_{i,d}, r_{i,d-1})}$	Effective bid-ask spread estimate	Daily returns
Bid-ask spread	$\text{BA}_{i,d} = \frac{\text{Ask}_{i,d}-\text{Bid}_{i,d}}{(\text{Ask}_{i,d}+\text{Bid}_{i,d})/2}$	Direct transaction cost	Quote data

The Amihud illiquidity ratio (Amihud 2002) is particularly useful because it requires only daily return and volume data. It captures the price impact of trading (i.e., the return per unit of currency volume) and has been shown to correlate well with more sophisticated microstructure-based measures such as the effective spread Goyenko and Ukhov (2009).

4.4.2 Computing Liquidity Diagnostics

```
# Compute standard liquidity measures at the stock-month level
prices_daily["abs_ret"] = prices_daily["ret"].abs()
prices_daily["zero_return"] = (prices_daily["ret"] == 0).astype(int)
prices_daily["year_month"] = prices_daily["date"].dt.to_period("M")

# Assume volume is in shares and value is in VND
# Amihud: average |ret| / value (in billions VND)
prices_daily["amihud_daily"] = np.where(
    prices_daily["value"] > 0,
    prices_daily["abs_ret"] / (prices_daily["value"] / 1e9),
```

```
    np.nan
)

liquidity_monthly = (
    prices_daily
    .groupby(["symbol", "year_month"])
    .agg(
        zero_return_share=("zero_return", "mean"),
        avg_turnover=("turnover", "mean"),
        amihud=("amihud_daily", "mean"),
        trading_days=("ret", "count"),
        avg_daily_value=("value", "mean")
    )
    .reset_index()
)

# Flag severely illiquid stock-months
liquidity_monthly["illiquid_flag"] = (
    (liquidity_monthly["zero_return_share"] > 0.5) |
    (liquidity_monthly["trading_days"] < 10) |
    (liquidity_monthly["avg_daily_value"] < 1e8)  # < 100M VND/day
)
```

4.4.3 Cross-Sectional Distribution of Liquidity

4.4.4 Liquidity Distribution Across Exchanges

The distributions typically reveal a bimodal pattern: HOSE stocks cluster at low illiquidity values, while HNX and especially UPCoM stocks exhibit a long right tail of extreme illiquidity. This heterogeneity implies that a single liquidity filter or treatment is insufficient for the entire cross-section.

4.4.5 Time Variation in Aggregate Liquidity

Market-wide liquidity is not constant. It deteriorates during crises, policy uncertainty, and periods of capital outflow, and improves during bull markets and periods of foreign inflow. The time variation in aggregate liquidity is itself a risk factor (Pástor and Stambaugh 2003).

Table 4.8: Cross-Sectional Distribution of Liquidity Measures (Latest Full Year)

```
latest_year = liquidity_monthly["year_month"].dt.year.max()
annual_liq = (
    liquidity_monthly[liquidity_monthly["year_month"].dt.year == latest_year]
    .groupby("symbol")
    .agg(
        zero_return_share=("zero_return_share", "mean"),
        avg_turnover=("avg_turnover", "mean"),
        amihud=("amihud", "mean"),
        avg_daily_value_m=("avg_daily_value", lambda x: x.mean() / 1e6)
    )
)

summary_stats = annual_liq.describe(percentiles=[0.10, 0.25, 0.50, 0.75,
 ↪  0.90]).T
summary_stats = summary_stats[
    ["mean", "std", "10%", "25%", "50%", "75%", "90%"]
].round(4)
summary_stats.columns = ["Mean", "Std", "P10", "P25", "Median", "P75", "P90"]
summary_stats.index = [
    "Zero-Return Share",
    "Avg Daily Turnover",
    "Amihud Illiquidity",
    "Avg Daily Value (M VND)"
]
summary_stats
```

```
# Merge exchange info
stock_exchange = (
    prices_daily[["symbol", "exchange"]]
    .drop_duplicates("symbol")
)
annual_liq_exch = annual_liq.merge(
    stock_exchange, left_index=True, right_on="symbol"
)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Zero-return share
for exchange, color in zip(
    ["HOSE", "HNX", "UPCoM"], ["#2C73D2", "#FF6B6B", "#5DCEAF"]
):
    subset = annual_liq_exch[annual_liq_exch["exchange"] == exchange]
    axes[0].hist(
        subset["zero_return_share"], bins=30, alpha=0.6,
        color=color, label=exchange, density=True
    )
axes[0].set_xlabel("Zero-Return Share")
axes[0].set_ylabel("Density")
axes[0].legend(frameon=False)
axes[0].spines["top"].set_visible(False)
axes[0].spines["right"].set_visible(False)

# Amihud (log scale)
for exchange, color in zip(
    ["HOSE", "HNX", "UPCoM"], ["#2C73D2", "#FF6B6B", "#5DCEAF"]
):
    subset = annual_liq_exch[annual_liq_exch["exchange"] == exchange]
    amihud_log = np.log(subset["amihud"].clip(lower=1e-10))
    axes[1].hist(
        amihud_log, bins=30, alpha=0.6,
        color=color, label=exchange, density=True
    )
axes[1].set_xlabel("Log Amihud Illiquidity")
axes[1].set_ylabel("Density")
axes[1].legend(frameon=False)
axes[1].spines["top"].set_visible(False)
axes[1].spines["right"].set_visible(False)

plt.tight_layout()
plt.show()
```

Figure 4.2

```
agg_liquidity = (
    liquidity_monthly
    .groupby("year_month")
    .agg(
        median_amihud=("amihud", "median"),
        median_zero_ret=("zero_return_share", "median"),
        total_value=("avg_daily_value", "sum")
    )
    .reset_index()
)
agg_liquidity["date"] = agg_liquidity["year_month"].dt.to_timestamp()

fig, ax1 = plt.subplots(figsize=(8, 4))

ax1.plot(
    agg_liquidity["date"],
    np.log(agg_liquidity["median_amihud"].clip(lower=1e-10)),
    color="#2C73D2", linewidth=1.2
)
ax1.set_ylabel("Log Median Amihud", color="#2C73D2")
ax1.tick_params(axis="y", labelcolor="#2C73D2")

ax2 = ax1.twinx()
ax2.fill_between(
    agg_liquidity["date"],
    agg_liquidity["median_zero_ret"] * 100,
    alpha=0.3, color="#FF6B6B"
)
ax2.set_ylabel("Median Zero-Return Share (%)", color="#FF6B6B")
ax2.tick_params(axis="y", labelcolor="#FF6B6B")

ax1.spines["top"].set_visible(False)
plt.tight_layout()
plt.show()
```

Figure 4.3

! Practical Recommendation

Before any asset pricing analysis, apply the following liquidity filter: exclude stock-months where the zero-return share exceeds 50%, where fewer than 15 trading days are observed, or where average daily trading value falls below a threshold (e.g., 100 million VND). Document the filter explicitly, and report sensitivity of results to alternative thresholds.

4.5 Bid-Ask Spread Estimation

In the absence of comprehensive quote data, the effective bid-ask spread can be estimated from transaction data using the method of Roll (1984). The Roll estimator exploits the fact that if the bid-ask bounce is the sole source of negative serial covariance in returns, then:

$$\hat{S}_{\text{Roll}} = 2\sqrt{-\text{Cov}(\Delta p_t, \Delta p_{t-1})} \tag{4.3}$$

where $\Delta p_t = p_t - p_{t-1}$ is the price change. When the autocovariance is positive (which occurs when information-driven serial correlation dominates the bid-ask bounce), the Roll estimator is undefined. Hasbrouck (2009) proposes a Bayesian variant that handles this case by imposing a prior on the spread.

```
# Compute Roll spread estimate at the stock-month level
prices_daily["dprice"] = prices_daily.groupby("symbol")["close"].diff()
prices_daily["dprice_lag"] =
 ↪  prices_daily.groupby("symbol")["dprice"].shift(1)

roll_cov = (
    prices_daily
    .groupby(["symbol", "year_month"])
    .apply(
        lambda g: g[["dprice", "dprice_lag"]].dropna().cov().iloc[0, 1],
        include_groups=False
    )
    .reset_index(name="autocovariance")
)

# Roll spread is defined only when autocovariance is negative
roll_cov["roll_spread"] = np.where(
    roll_cov["autocovariance"] < 0,
    2 * np.sqrt(-roll_cov["autocovariance"]),
```

Table 4.9: Distribution of Roll Spread Estimates (% of Price)

```
roll_summary = (
    roll_cov
    .dropna(subset=["roll_spread_pct"])
    .groupby("year_month")["roll_spread_pct"]
    .describe(percentiles=[0.25, 0.50, 0.75])
    .reset_index()
)

# Show latest year summary
latest_year_roll = roll_cov[
    roll_cov["year_month"].dt.year == roll_cov["year_month"].dt.year.max()
]
print(
    latest_year_roll["roll_spread_pct"]
    .dropna()
    .describe(percentiles=[0.10, 0.25, 0.50, 0.75, 0.90])
    .round(3)
)
```

```
    np.nan
)

# As a percentage of price
roll_cov = roll_cov.merge(
    prices_daily.groupby(["symbol", "year_month"])["close"].mean()
    .reset_index(name="avg_price"),
    on=["symbol", "year_month"]
)
roll_cov["roll_spread_pct"] = roll_cov["roll_spread"] / roll_cov["avg_price"]
 ↪  * 100
```

4.6 Non-Synchronous Trading Bias

When stocks do not trade at the same frequency or at the same times, observed returns are misaligned. This non-synchronous trading bias, first formalized by Scholes and Williams (1977)

and Lo and MacKinlay (1990), is one of the most consequential microstructure effects for asset pricing in thin markets.

4.6.1 The Problem

Suppose the true (unobserved) return process for stock i follows a single-factor model:

$$r^*_{i,t} = \alpha_i + \beta_i r^*_{m,t} + \varepsilon_{i,t} \tag{4.4}$$

where $r^*_{m,t}$ is the true market return and β_i is the true beta. If stock i last traded k days before the end of day t, the observed return incorporates information only up to day $t-k$. Scholes and Williams (1977) show that the OLS estimate of beta from regressing observed returns on observed market returns is:

$$\hat{\beta}_i^{OLS} = \beta_i \cdot \pi_i \tag{4.5}$$

where π_i is the probability that stock i trades on any given day. For a stock that trades on only 50% of days, the OLS beta is biased downward by 50%. This bias is severe in Vietnam, where many small-cap stocks trade on fewer than half of all trading days.

4.6.2 Quantifying the Bias

4.6.3 The Dimson Beta Correction

Dimson (1979) proposes a simple correction: include lagged and leading market returns in the beta regression:

$$r_{i,t} = \alpha_i + \sum_{k=-K}^{K} \beta_{i,k}\, r_{m,t-k} + \varepsilon_{i,t} \tag{4.6}$$

The Dimson-corrected beta is $\hat{\beta}_i^{Dimson} = \sum_{k=-K}^{K} \hat{\beta}_{i,k}$. Typically $K = 1$ or $K = 2$ is sufficient. The summed coefficients capture the full response of the stock's observed return to market information, regardless of when the stock actually trades.

```
# Compute trading frequency: proportion of market days with nonzero volume
market_days = prices_daily.groupby("year_month")["date"].nunique()
trading_freq = (
    prices_daily[prices_daily["value"] > 0]
    .groupby(["symbol", "year_month"])["date"]
    .nunique()
    .reset_index(name="days_traded")
)
trading_freq = trading_freq.merge(
    market_days.reset_index().rename(columns={"date": "market_days"}),
    on="year_month"
)
trading_freq["trade_prob"] = trading_freq["days_traded"] /
 ↪  trading_freq["market_days"]

# Annual average
annual_trade_freq = (
    trading_freq
    .groupby("symbol")["trade_prob"]
    .mean()
    .reset_index(name="avg_trade_prob")
)

fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(
    annual_trade_freq["avg_trade_prob"], bins=50,
    color="#2C73D2", edgecolor="white", alpha=0.8
)
ax.axvline(
    annual_trade_freq["avg_trade_prob"].median(),
    color="#FF6B6B", linestyle="--", linewidth=1.5,
    label=f"Median = {annual_trade_freq['avg_trade_prob'].median():.2f}"
)
ax.set_xlabel("Average Trading Probability (Fraction of Market Days)")
ax.set_ylabel("Number of Stocks")
ax.legend(frameon=False)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.tight_layout()
plt.show()
```

Figure 4.4

```
# Estimate Dimson betas with K=1 lag and lead
# Merge market return
market_ret = (
    prices_daily
    .groupby("date")
    .apply(
        lambda g: np.average(g["ret"].dropna(),
         ↪  weights=g["mktcap"].loc[g["ret"].dropna().index])
        if g["ret"].dropna().shape[0] > 0 else np.nan,
        include_groups=False
    )
    .reset_index(name="rm")
)

prices_daily = prices_daily.merge(market_ret, on="date", how="left")
prices_daily["rm_lag1"] = prices_daily.groupby("symbol")["rm"].shift(1)
prices_daily["rm_lead1"] = prices_daily.groupby("symbol")["rm"].shift(-1)

def estimate_dimson_beta(group):
    """Estimate OLS and Dimson(K=1) betas for a single stock."""
    g = group.dropna(subset=["ret", "rm", "rm_lag1", "rm_lead1"])
    if len(g) < 60:
        return pd.Series({"beta_ols": np.nan, "beta_dimson": np.nan, "n_obs":
         ↪  len(g)})

    # OLS beta
    X_ols = sm.add_constant(g["rm"])
    ols_model = sm.OLS(g["ret"], X_ols).fit()
    beta_ols = ols_model.params["rm"]

    # Dimson beta
    X_dim = sm.add_constant(g[["rm_lag1", "rm", "rm_lead1"]])
    dim_model = sm.OLS(g["ret"], X_dim).fit()
    beta_dimson = dim_model.params[["rm_lag1", "rm", "rm_lead1"]].sum()

    return pd.Series({
        "beta_ols": beta_ols,
        "beta_dimson": beta_dimson,
        "n_obs": len(g)
    })

beta_comparison = (
    prices_daily
```

```
    .groupby("symbol")
    .apply(estimate_dimson_beta, include_groups=False)
    .reset_index()
)
```

```
beta_valid = beta_comparison.dropna()

fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(
    beta_valid["beta_ols"], beta_valid["beta_dimson"],
    alpha=0.3, s=10, color="#2C73D2"
)
lims = [
    min(ax.get_xlim()[0], ax.get_ylim()[0]),
    max(ax.get_xlim()[1], ax.get_ylim()[1])
]
ax.plot(lims, lims, "--", color="gray", linewidth=1)
ax.set_xlabel("OLS Beta")
ax.set_ylabel("Dimson Beta (K=1)")
ax.set_aspect("equal")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.tight_layout()
plt.show()
```

Figure 4.5

The scatter plot should reveal a systematic pattern: Dimson betas exceed OLS betas for most stocks, with the discrepancy largest for thinly traded stocks. Points above the 45-degree line indicate stocks whose OLS betas are biased downward by non-synchronous trading.

> ⚠ Warning
>
> For the thinnest-traded tercile, OLS beta underestimates true systematic risk by 20-40% on average. Using uncorrected betas for cost of equity estimation or factor model tests will produce systematically incorrect results for these stocks.

4.6.4 The Scholes-Williams Estimator

An alternative correction, proposed by Scholes and Williams (1977), estimates beta as:

Table 4.10: Beta Bias by Trading Frequency Tercile

```
beta_with_freq = beta_valid.merge(annual_trade_freq, on="symbol")
beta_with_freq["freq_tercile"] = pd.qcut(
    beta_with_freq["avg_trade_prob"], q=3,
    labels=["Low (Thin)", "Medium", "High (Liquid)"]
)

beta_bias_summary = (
    beta_with_freq
    .groupby("freq_tercile")
    .agg(
        n_stocks=("symbol", "count"),
        avg_trade_prob=("avg_trade_prob", "mean"),
        mean_beta_ols=("beta_ols", "mean"),
        mean_beta_dimson=("beta_dimson", "mean"),
        median_beta_ols=("beta_ols", "median"),
        median_beta_dimson=("beta_dimson", "median")
    )
    .round(3)
)

beta_bias_summary["bias_pct"] = (
    (beta_bias_summary["mean_beta_dimson"] -
 ↪  beta_bias_summary["mean_beta_ols"])
    / beta_bias_summary["mean_beta_dimson"] * 100
).round(1)

beta_bias_summary
```

$$\hat{\beta}_i^{SW} = \frac{\hat{\beta}_{i,-1} + \hat{\beta}_{i,0} + \hat{\beta}_{i,+1}}{1 + 2\hat{\rho}_m} \tag{4.7}$$

where $\hat{\beta}_{i,k}$ is the slope from regressing $r_{i,t}$ on $r_{m,t-k}$ alone, and $\hat{\rho}_m$ is the first-order autocorrelation of the market return. The Scholes-Williams estimator is consistent under the assumption that non-trading is the sole source of serial cross-correlation, while the Dimson estimator is more robust to additional sources of lead-lag structure.

```
def estimate_sw_beta(group):
    """Estimate Scholes-Williams beta."""
    g = group.dropna(subset=["ret", "rm", "rm_lag1", "rm_lead1"])
    if len(g) < 60:
        return np.nan

    # Separate regressions
    beta_lag = sm.OLS(g["ret"],
    ↪  sm.add_constant(g["rm_lag1"])).fit().params.iloc[1]
    beta_0 = sm.OLS(g["ret"], sm.add_constant(g["rm"])).fit().params.iloc[1]
    beta_lead = sm.OLS(g["ret"],
    ↪  sm.add_constant(g["rm_lead1"])).fit().params.iloc[1]

    # Market autocorrelation
    rho_m = g["rm"].autocorr(lag=1)

    beta_sw = (beta_lag + beta_0 + beta_lead) / (1 + 2 * rho_m)
    return beta_sw

beta_comparison["beta_sw"] = (
    prices_daily
    .groupby("symbol")
    .apply(estimate_sw_beta, include_groups=False)
    .values
)
```

4.7 Implications for Portfolio Construction

The microstructure frictions documented above have direct consequences for portfolio construction, particularly for strategies that involve rebalancing across the full cross-section of listed firms.

4.7.1 Equal-Weighted vs. Value-Weighted Returns

Equal-weighted portfolio returns give the same weight to each stock, including illiquid small-cap stocks that may contribute stale or noisy prices. Value-weighted returns tilt toward large, liquid stocks and are less susceptible to microstructure contamination.

A persistent divergence between equal-weighted and value-weighted cumulative returns is a hallmark of microstructure effects: the equal-weighted portfolio overstates attainable returns because it implicitly assumes costless trading in illiquid stocks.

4.7.2 Recommended Liquidity Filters

Based on the diagnostics developed in this chapter, we recommend the following pre-analysis filters:

> **Tip**
>
> Always report results with and without liquidity filters. If results are qualitatively different, the baseline findings may be driven by microstructure artifacts rather than genuine economic effects.

4.7.3 Monthly vs. Daily Frequency

For most asset pricing applications, monthly return aggregation is preferable to daily analysis in Vietnam because:

1. Monthly returns smooth out intraday noise, bid-ask bounce, and price limit effects.
2. Stocks that trade infrequently within a month still produce a meaningful monthly return.
3. Factor portfolio sorts are conventionally conducted at monthly frequency.
4. Statistical tests have better size properties when microstructure noise is reduced.

However, monthly aggregation does not eliminate all biases. Stocks with zero returns for an entire month still contribute stale observations. The Dimson and Scholes-Williams corrections should still be applied at monthly frequency for beta estimation.

4.8 Implications for Asset Pricing Tests

4.8.1 Factor Model Estimation

Standard factor model estimation assumes that returns are observed synchronously and without censoring. In Vietnam, both assumptions are violated. The practical consequences are in

```
monthly_returns = (
    prices_daily
    .groupby(["symbol", "year_month"])
    .agg(
        monthly_ret=("ret", lambda x: (1 + x).prod() - 1),
        last_mktcap=("mktcap", "last")
    )
    .reset_index()
)
monthly_returns["date"] = monthly_returns["year_month"].dt.to_timestamp()

# Equal-weighted
ew_ret = monthly_returns.groupby("date")["monthly_ret"].mean().reset_index(n
 ↪  ame="ew")

# Value-weighted
def vw_return(group):
    w = group["last_mktcap"] / group["last_mktcap"].sum()
    return (w * group["monthly_ret"]).sum()

vw_ret = (
    monthly_returns.groupby("date")
    .apply(vw_return, include_groups=False)
    .reset_index(name="vw")
)

port_comp = ew_ret.merge(vw_ret, on="date")

fig, ax = plt.subplots(figsize=(8, 4))
for col, label, color in [
    ("ew", "Equal-Weighted", "#FF6B6B"),
    ("vw", "Value-Weighted", "#2C73D2")
]:
    cum_ret = (1 + port_comp[col]).cumprod()
    ax.plot(port_comp["date"], cum_ret, label=label, color=color,
 ↪  linewidth=1.2)

ax.set_ylabel("Cumulative Return (Growth of 1 VND)")
ax.set_xlabel("")
ax.legend(frameon=False)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.tight_layout()
plt.show()
```

Figure 4.6

Table 4.11

Table 4.11: Standard Assumptions and Their Violations

Assumption	Violation in Vietnam	Consequence
Synchronous observation	Thin trading	Biased betas, attenuated R^2
Uncensored returns	Price limits	Truncated distributions, biased moments
Continuous trading	Discrete ticks	Return discreteness, bid-ask bounce
No transaction costs	Wide spreads	Overstated portfolio returns

4.8.2 Adjusted Testing Procedure

We recommend the following adjustments to standard asset pricing tests when applied to Vietnamese data:

1. **Beta estimation**: Use Dimson ($K \geq 1$) or Scholes-Williams betas, not OLS betas.
2. **Factor construction**: When forming size and value portfolios, apply liquidity filters before sorting. Consider excluding the smallest quintile of stocks by market capitalization, which is most affected by thin trading.
3. **Return aggregation**: Use monthly frequency. If daily analysis is necessary, include lagged market returns in the time-series regression.
4. **Robust inference**: Cluster standard errors by stock to account for persistent microstructure-induced serial correlation. Use Newey-West HAC standard errors with sufficient lags.
5. **Price limit adjustment**: For volatility analysis or risk measurement, consider the Chu and Qiu (2019) approach of modeling the latent (uncensored) return distribution using truncated regression:

$$r_{i,t}^* \sim N(\mu_i, \sigma_i^2), \quad r_{i,t}^{obs} = \max(\underline{L}, \min(\bar{L}, r_{i,t}^*)) \tag{4.8}$$

Estimate μ_i and σ_i^2 via maximum likelihood for the truncated normal.

```python
from scipy.optimize import minimize
from scipy.stats import norm

def truncated_normal_nll(params, returns, lower, upper):
    """Negative log-likelihood of truncated normal."""
    mu, log_sigma = params
    sigma = np.exp(log_sigma)

    # Interior observations
    interior = (returns > lower) & (returns < upper)
    ll_interior = norm.logpdf(returns[interior], mu, sigma)

    # Lower censored
    ll_lower = norm.logcdf(lower, mu, sigma)
    n_lower = (returns <= lower).sum()

    # Upper censored
    ll_upper = np.log(1 - norm.cdf(upper, mu, sigma) + 1e-15)
    n_upper = (returns >= upper).sum()

    nll = -(ll_interior.sum() + n_lower * ll_lower + n_upper * ll_upper)
    return nll

def estimate_true_volatility(returns, limit_band):
    """Estimate latent volatility correcting for price limit censoring."""
    result = minimize(
        truncated_normal_nll,
        x0=[returns.mean(), np.log(returns.std())],
        args=(returns.values, -limit_band, limit_band),
        method="Nelder-Mead"
    )
    mu, log_sigma = result.x
    return np.exp(log_sigma)
```

6. **Sensitivity reporting**: Always report key results under alternative specifications: with and without liquidity filters, using OLS vs. Dimson betas, at daily vs. monthly frequency, and using observed vs. truncation-corrected volatility.

4.9 Summary

This chapter has established that Vietnamese equity markets exhibit microstructure characteristics that materially affect observed prices, returns, and risk measures. The key findings are:

1. **Price limits** censor daily returns, inducing positive autocorrelation, volatility spillover, and truncated distributions. The $\pm$ 7% band on HOSE is particularly restrictive for volatile stocks.
2. **Thin trading and zero returns** afflict a substantial fraction of listed firms. Trading probabilities below 50% are common on HNX and UPCoM, generating non-synchronous trading bias that attenuates OLS beta estimates by 20-40%.
3. **Illiquidity** varies dramatically across the cross-section, with Amihud ratios spanning several orders of magnitude. Value-weighted portfolio returns are less contaminated than equal-weighted returns.
4. **The Dimson and Scholes-Williams beta corrections** effectively address non-synchronous trading bias and should be used as the default beta estimator for Vietnamese equities.
5. **Liquidity filters** should be applied before any asset pricing analysis, and results should be reported with and without these filters as a robustness check.

Ignoring these frictions does not merely add noise to empirical results, it systematically biases estimates in predictable directions. The diagnostics and corrections presented in this chapter provide the foundation for credible empirical asset pricing in Vietnam.

5 Risk-Free Rate Construction in Vietnam

> **Note**
>
> In this chapter, we address a simple but consequential problem: how to construct a risk-free rate series for empirical finance in Vietnam. We evaluate available proxies, such as government bond yields, interbank overnight rates, and State Bank of Vietnam (SBV) policy rates, develop interpolation and frequency-alignment procedures, and quantify the sensitivity of key asset pricing outputs to the choice of risk-free proxy.

The risk-free rate is the most important single number in finance. It anchors excess returns, discount rates, factor premiums, the cost of equity, performance evaluation, and derivative pricing. Despite its foundational role, the risk-free rate is often treated as a given: a number downloaded from a database and plugged into formulas without further thought. In developed markets with deep, liquid government securities markets, this casual approach is usually harmless. In Vietnam, it is not.

Vietnam's fixed-income market is thin, fragmented, and characterized by irregular issuance of short-term government securities. There is no single, universally accepted risk-free rate analogous to the 1-month Treasury bill rate that anchors virtually all asset pricing research in mature markets. Instead, researchers face a choice among imperfect proxies, each with distinct advantages and limitations. This choice is not innocuous: different proxies can produce meaningfully different excess returns, factor premiums, and valuation estimates.

This chapter develops a systematic approach to risk-free rate construction. We begin with the theoretical requirements for a risk-free asset, then evaluate available Vietnamese proxies against these requirements. We construct monthly risk-free rate series under alternative specifications, demonstrate frequency alignment and interpolation techniques, and conduct sensitivity analysis to quantify how the choice of proxy affects downstream results.

5.1 The Role of the Risk-Free Rate in Finance

5.1.1 Excess Returns

The most fundamental use of the risk-free rate is in the computation of excess returns. The excess return on asset i in period t is:

$$r_{i,t}^{e} = r_{i,t} - r_{f,t} \tag{5.1}$$

where $r_{i,t}$ is the raw return and $r_{f,t}$ is the risk-free rate over the same period, in the same currency, and under the same compounding convention. Excess returns isolate the compensation for bearing risk, removing the return that could be earned without risk exposure.

Mismeasurement of $r_{f,t}$ directly contaminates every excess return observation and, by extension, every quantity derived from excess returns. If $r_{f,t}$ is systematically biased upward, excess returns are systematically understated, factor premiums are compressed, and the cost of equity is overstated.

5.1.2 Factor Premiums

In the Eugene F. Fama and French (1993a) three-factor model, the market risk premium is:

$$\text{MKTRF}_t = r_{m,t} - r_{f,t} \tag{5.2}$$

where $r_{m,t}$ is the value-weighted market return. The size (SMB) and value (HML) premiums are defined as returns on long-short portfolios and do not directly depend on $r_{f,t}$. However, the intercept (alpha) from a time-series regression of any portfolio's excess return on the factors does depend on $r_{f,t}$ through the dependent variable. A biased risk-free rate shifts all alphas uniformly.

5.1.3 Discount Rates and Valuation

The discounted cash flow model values a firm as:

$$V_0 = \sum_{t=1}^{\infty} \frac{E[CF_t]}{(1 + r_{WACC})^t} \tag{5.3}$$

where the weighted average cost of capital (WACC) depends on the cost of equity, which in turn depends on the risk-free rate through the Capital Asset Pricing Model:

$$r_e = r_f + \beta_i(\bar{r}_m - r_f) \tag{5.4}$$

A 100 basis point error in r_f flows through to the cost of equity and can change the present value of a long-duration cash flow stream by 10-20%, depending on the duration profile.

5.1.4 Performance Evaluation

Risk-adjusted performance measures such as the Sharpe ratio:

$$\text{SR} = \frac{\bar{r}_p - \bar{r}_f}{\sigma(r_p - r_f)} \tag{5.5}$$

and Jensen's alpha:

$$\alpha = \bar{r}_p - r_f - \hat{\beta}_p(\bar{r}_m - r_f) \tag{5.6}$$

both depend directly on r_f. The Sharpe ratio is particularly sensitive because the denominator (excess return volatility) is also affected by the level and variability of r_f.

5.2 What Does "Risk-Free" Mean in Practice?

A truly risk-free asset must satisfy four conditions simultaneously (Table 5.1).

Table 5.1: Requirements for a Risk-Free Asset

Condition	Definition	Practical Challenge
No default risk	The issuer cannot fail to pay	Only sovereign debt in one's own currency qualifies
Known cash flows	Payoff is certain ex ante	Rules out floating-rate instruments
No reinvestment risk	Maturity matches the investment horizon	Requires zero-coupon securities of exact maturity
High liquidity	Can be traded at a low cost	Thin government bond markets fail this test

No real-world asset perfectly satisfies all four conditions. Even in the deepest government bond markets, there is a liquidity premium in off-the-run securities and a convenience yield in on-the-run issues (Krishnamurthy and Vissing-Jorgensen 2012). The practical question is: which available instrument comes closest?

5.2.1 The Ideal Proxy

The ideal risk-free rate proxy for empirical asset pricing research has the following properties:

1. **Short maturity**: Minimizes reinvestment risk and term premium contamination. The conventional choice is 1-month maturity.
2. **Government-backed**: Eliminates credit risk (in domestic currency).
3. **Actively traded**: Ensures that the observed yield reflects current market conditions.
4. **Regular issuance**: Provides a continuous time series without gaps.
5. **Consistent methodology**: Yield computation is unambiguous and comparable over time.

5.3 Available Proxies in Vietnam

Vietnam's financial infrastructure provides several candidate instruments, none of which perfectly satisfies all criteria. We evaluate each in turn.

5.3.1 Government Bond Yields

The Vietnamese government issues bonds across a range of maturities through the State Treasury and the Vietnam Bond Market (VBM). Key characteristics:

Table 5.2: Vietnamese Government Bond Characteristics

Feature	Description
Issuer	State Treasury of Vietnam
Primary market	Auction through HNX
Available maturities	1, 2, 3, 5, 7, 10, 15, 20, 30 years
Shortest benchmark	1-year (sometimes shorter tenors via T-bills)
Coupon structure	Fixed, semi-annual
Issuance frequency	Regular weekly/bi-weekly auctions
Secondary market liquidity	Concentrated in 3-5 year segment; short end is thin
Data availability	Available through DataCore.vn

The primary limitation of government bond yields as a risk-free proxy is the scarcity of short-maturity securities. One-year bonds are the shortest regularly issued benchmark, and their yield includes a term premium that is absent from a true risk-free rate. Treasury bills (maturity < 1 year) are issued irregularly and in small volumes, making them unsuitable as a continuous series.

5.3.2 Interbank Overnight Rate

The Vietnam interbank market sets overnight lending rates between commercial banks. The overnight rate is reported by the SBV.

Advantages: Very short maturity (overnight), high frequency (daily), reflects actual borrowing costs in the financial system.

Limitations: Reflects banking sector credit risk (interbank default risk, though small), can be volatile during liquidity crunches, and does not correspond to a tradeable zero-coupon instrument.

5.3.3 SBV Policy Rates

The State Bank of Vietnam sets several administered rates (Table 5.3).

Table 5.3: SBV Policy Rates

Rate	Role	Frequency of Change
Refinancing rate	Rate at which SBV lends to banks	Infrequent (policy meetings)
Discount rate	Rate for rediscounting eligible paper	Infrequent
Overnight lending rate	Ceiling for interbank overnight	Infrequent
Deposit rate cap	Maximum rate banks can pay on deposits	Infrequent

Advantages: Stable (changes infrequently), reflects the monetary policy stance, available for the entire sample period.

Limitations: Not a traded return (i.e., no investor can actually earn the policy rate). Represents an administrative target, not a market-clearing price. Responds to macroeconomic conditions with a lag.

5.3.4 Savings Deposit Rates

Commercial banks offer term deposits at rates subject to SBV caps. Short-term (1-month or 3-month) deposit rates are sometimes used as informal risk-free proxies in practitioner contexts.

Advantages: Represent an investable return for small investors, widely available.

Limitations: Subject to bank credit risk, vary across banks, caps create a ceiling that may not reflect true equilibrium rates, not standardized for research use.

5.3.5 Summary Comparison

Table 5.4: Comparison of Risk-Free Rate Proxies in Vietnam

Proxy	Default Risk	Maturity Match	Continuity	Liquidity	Recommended Use
1-Year Government Bond	Minimal (sovereign)	Poor (1Y vs 1M)	Good	Moderate	Annual/quarterly analysis
Treasury Bills (< 1Y)	Minimal (sovereign)	Good (when available)	Poor (irregular issuance)	Low	When available, preferred
Interbank Overnight Rate	Low (interbank)	Very short (overnight)	Excellent	High (interbank)	Daily/monthly analysis
SBV Refinancing Rate	None (administered)	N/A (not traded)	Excellent	N/A	Long-run comparisons only
1-Month Bank Deposit Rate	Low-moderate (bank)	Good (1M)	Good	N/A	Practitioner DCF

5.4 Constructing the Risk-Free Rate Series

We now construct alternative monthly risk-free rate series and examine their properties.

5.4.1 Loading and Cleaning Rate Data

```
import pandas as pd
import numpy as np

# Assume rf_data contains: date, rate_type, rate_annual (annualized, in %)
rf_raw = pd.read_parquet("data/risk_free_rates.parquet")
```

```
# Preview available rate types
print("Available rate types:")
print(rf_raw["rate_type"].value_counts())
```

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4.5))

colors = {
    "govt_bond_1y": "#2C73D2",
    "interbank_overnight": "#FF6B6B",
    "sbv_refinancing": "#5DCEAF",
    "tbill_3m": "#FFB347",
    "deposit_1m": "#B19CD9"
}

for rate_type, color in colors.items():
    subset = rf_raw[rf_raw["rate_type"] == rate_type].sort_values("date")
    if len(subset) > 0:
        ax.plot(
            subset["date"], subset["rate_annual"],
            label=rate_type.replace("_", " ").title(),
            color=color, linewidth=1.2, alpha=0.85
        )

ax.set_ylabel("Annualized Rate (%)")
ax.set_xlabel("")
ax.legend(frameon=False, fontsize=9, loc="upper right")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.tight_layout()
plt.show()
```

Figure 5.1

5.4.2 Frequency Alignment

Asset pricing tests require monthly risk-free rates. Raw data may arrive at daily, weekly, or irregular frequencies. We use the following conversion logic:

Daily to monthly: Take the average of daily rates within each month, then convert from annualized to monthly.

Irregular to monthly: For series with gaps (e.g., treasury bills), forward-fill the most recent observation, then average within each month.

Annualized to monthly: Under simple compounding, $r_f^{monthly} = r_f^{annual}/12$. Under continuous compounding, $r_f^{monthly} = r_f^{annual}/12$ (since continuous rates are additive). We use simple compounding for consistency with the convention that stock returns are computed as arithmetic returns.

```
# Construct monthly risk-free rates from each proxy

def construct_monthly_rf(rf_raw, rate_type, method="mean"):
    """
    Convert raw rate data to monthly frequency.

    Parameters
    ----------
    rf_raw : pd.DataFrame
        Raw rate data with columns: date, rate_type, rate_annual
    rate_type : str
        Which rate proxy to use
    method : str
        Aggregation method: 'mean', 'last', or 'first'

    Returns
    -------
    pd.DataFrame
        Monthly risk-free rate with columns: date, rf_monthly
    """
    subset = (
        rf_raw[rf_raw["rate_type"] == rate_type]
        .sort_values("date")
        .set_index("date")
    )

    # Forward-fill gaps (for irregular series)
    subset = subset.resample("D").ffill()

    # Aggregate to monthly
    if method == "mean":
        monthly = subset.resample("ME")["rate_annual"].mean()
    elif method == "last":
```

```
        monthly = subset.resample("ME")["rate_annual"].last()
    else:
        monthly = subset.resample("ME")["rate_annual"].first()

    monthly = monthly.reset_index()
    monthly.columns = ["date", "rf_annual"]

    # Convert annualized rate (%) to monthly decimal
    monthly["rf_monthly"] = monthly["rf_annual"] / 100 / 12

    return monthly[["date", "rf_monthly", "rf_annual"]]

# Construct series for each proxy
rf_proxies = {}
for proxy in ["govt_bond_1y", "interbank_overnight", "sbv_refinancing"]:
    rf_proxies[proxy] = construct_monthly_rf(rf_raw, proxy)
    rf_proxies[proxy]["proxy"] = proxy

rf_all = pd.concat(rf_proxies.values(), ignore_index=True)
```

5.4.3 Handling Missing Data and Structural Breaks

Vietnamese rate data may contain gaps due to market closures, reporting changes, or the introduction of new instruments. We handle these systematically:

```
# Check coverage for each proxy
coverage = (
    rf_all
    .groupby("proxy")
    .agg(
        start_date=("date", "min"),
        end_date=("date", "max"),
        n_months=("rf_monthly", "count"),
        n_missing=("rf_monthly", lambda x: x.isna().sum()),
        avg_rate_pct=("rf_annual", "mean")
    )
    .round(2)
)

print(coverage)
```

For periods where the primary proxy is unavailable, we construct a blended series using a priority hierarchy:

```
def construct_blended_rf(rf_proxies, priority=None):
    """
    Construct a blended monthly risk-free rate using proxy priority.

    Priority order (default):
    1. Treasury bills (shortest maturity, sovereign)
    2. Interbank overnight (short maturity, high frequency)
    3. 1-year government bond (sovereign, regular)
    4. SBV refinancing rate (fallback)
    """
    if priority is None:
        priority = [
            "tbill_3m",
            "interbank_overnight",
            "govt_bond_1y",
            "sbv_refinancing"
        ]

    # Create full date range
    all_dates = pd.date_range(
        start=min(df["date"].min() for df in rf_proxies.values()),
        end=max(df["date"].max() for df in rf_proxies.values()),
        freq="ME"
    )

    blended = pd.DataFrame({"date": all_dates})
    blended["rf_monthly"] = np.nan
    blended["source"] = ""

    for proxy in priority:
        if proxy in rf_proxies:
            proxy_df = rf_proxies[proxy][["date", "rf_monthly"]].rename(
                columns={"rf_monthly": f"rf_{proxy}"}
            )
            blended = blended.merge(proxy_df, on="date", how="left")

            # Fill missing values from this proxy
            mask = blended["rf_monthly"].isna() &
↪   blended[f"rf_{proxy}"].notna()
            blended.loc[mask, "rf_monthly"] = blended.loc[mask,
↪   f"rf_{proxy}"]
```

Table 5.5: Data Source Composition of Blended Risk-Free Rate Series

```
source_comp = (
    rf_blended
    .groupby("source")
    .agg(
        n_months=("date", "count"),
        pct=("date", lambda x: len(x) / len(rf_blended) * 100)
    )
    .round(1)
    .sort_values("n_months", ascending=False)
)

source_comp
```

```
            blended.loc[mask, "source"] = proxy

            blended = blended.drop(columns=[f"rf_{proxy}"])

    return blended

rf_blended = construct_blended_rf(rf_proxies)
```

5.4.4 Properties of the Constructed Series

The spread between proxies is informative. A consistently positive spread (bond yield > interbank rate) reflects the term premium embedded in the 1-year bond. Periods where the interbank rate spikes above the bond yield typically correspond to liquidity crunches in the banking system.

5.4.5 Correlation Across Proxies

High correlation (> 0.8) between proxies suggests that the level and direction of interest rate movements are captured similarly by all proxies. Low correlation would indicate that the choice of proxy introduces substantial idiosyncratic variation into excess returns.

Table 5.6: Summary Statistics of Monthly Risk-Free Rate Proxies

```
rf_wide = rf_all.pivot_table(
    index="date", columns="proxy", values="rf_monthly"
)

summary = rf_wide.describe(percentiles=[0.10, 0.25, 0.50, 0.75, 0.90]).T
summary = summary[["mean", "std", "min", "10%", "50%", "90%", "max"]]
summary.columns = [
    "Mean", "Std", "Min", "P10", "Median", "P90", "Max"
]

# Convert to annualized percentage for interpretability
(summary * 12 * 100).round(2)
```

Table 5.7: Pairwise Correlation of Monthly Risk-Free Rate Proxies

```
rf_corr = rf_wide.corr().round(3)
rf_corr.index = [x.replace("_", " ").title() for x in rf_corr.index]
rf_corr.columns = [x.replace("_", " ").title() for x in rf_corr.columns]
rf_corr
```

```
fig, axes = plt.subplots(2, 1, figsize=(8, 6), sharex=True)

# Level
for proxy, color in [
    ("govt_bond_1y", "#2C73D2"),
    ("interbank_overnight", "#FF6B6B"),
    ("sbv_refinancing", "#5DCEAF")
]:
    subset = rf_proxies[proxy].sort_values("date")
    axes[0].plot(
        subset["date"], subset["rf_monthly"] * 100,
        label=proxy.replace("_", " ").title(),
        color=color, linewidth=1
    )
axes[0].set_ylabel("Monthly Rate (%)")
axes[0].legend(frameon=False, fontsize=9)
axes[0].spines["top"].set_visible(False)
axes[0].spines["right"].set_visible(False)

# Pairwise spread: govt bond - interbank
merged = rf_proxies["govt_bond_1y"][["date", "rf_monthly"]].merge(
    rf_proxies["interbank_overnight"][["date", "rf_monthly"]],
    on="date", suffixes=("_bond", "_interbank")
)
merged["spread"] = (merged["rf_monthly_bond"] -
 ↪  merged["rf_monthly_interbank"]) * 100

axes[1].fill_between(
    merged["date"], merged["spread"], 0,
    where=merged["spread"] > 0, alpha=0.4, color="#2C73D2", label="Bond >
 ↪  Interbank"
)
axes[1].fill_between(
    merged["date"], merged["spread"], 0,
    where=merged["spread"] <= 0, alpha=0.4, color="#FF6B6B", label="Bond <
 ↪  Interbank"
)
axes[1].axhline(0, color="black", linewidth=0.5)
axes[1].set_ylabel("Spread (% monthly)")
axes[1].set_xlabel("")
axes[1].legend(frameon=False, fontsize=9)
axes[1].spines["top"].set_visible(False)
axes[1].spines["right"].set_visible(False)

plt.tight_layout()
plt.show()
```


Figure 5.2

5.5 Excess Return Construction

With the risk-free rate series in hand, we now construct excess returns for individual stocks and for the market portfolio.

5.5.1 Matching Conventions

Excess return construction requires strict consistency across three dimensions (Table 5.8).

Table 5.8: Consistency Requirements for Excess Returns

Dimension	Requirement	Common Error
Currency	Same currency for r_i and r_f	Using USD rate for VND-denominated returns
Frequency	Same holding period	Using annualized r_f with monthly r_i
Compounding	Same convention	Mixing log and arithmetic returns

```
# Load monthly stock returns
stock_returns = pd.read_parquet("data/monthly_returns.parquet")
# Assume columns: symbol, date (month-end), ret

# Merge with risk-free rate
# Use the blended series as the baseline
rf_for_merge = rf_blended[["date", "rf_monthly"]].rename(
    columns={"rf_monthly": "rf"}
)

stock_returns = stock_returns.merge(rf_for_merge, on="date", how="left")

# Compute excess returns
stock_returns["ret_excess"] = stock_returns["ret"] - stock_returns["rf"]
```

5.5.2 Market Excess Return

```
# Value-weighted market return
market_monthly = (
    stock_returns
    .groupby("date")
    .apply(
        lambda g: np.average(
            g["ret"].dropna(),
            weights=g["mktcap"].loc[g["ret"].dropna().index]
        ) if g["ret"].dropna().shape[0] > 0 else np.nan,
        include_groups=False
    )
    .reset_index(name="rm")
)

market_monthly = market_monthly.merge(rf_for_merge, on="date", how="left")
market_monthly["mktrf"] = market_monthly["rm"] - market_monthly["rf"]
```

```
fig, ax = plt.subplots(figsize=(8, 3.5))

ax.bar(
    market_monthly["date"], market_monthly["mktrf"] * 100,
    color=np.where(market_monthly["mktrf"] >= 0, "#2C73D2", "#FF6B6B"),
    width=25, alpha=0.8
)
ax.axhline(0, color="black", linewidth=0.5)
ax.set_ylabel("Market Excess Return (%)")
ax.set_xlabel("")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.tight_layout()
plt.show()
```

Figure 5.3

5.6 Sensitivity Analysis: How Much Does the Proxy Choice Matter?

This is the central empirical question of the chapter. If different risk-free proxies produce essentially the same downstream results, the choice is inconsequential. If they produce different

Table 5.9: Annualized Equity Premium Under Alternative Risk-Free Proxies

```
results = []
for proxy_name, proxy_df in rf_proxies.items():
    rf_merge = proxy_df[["date", "rf_monthly"]].rename(
        columns={"rf_monthly": "rf_proxy"}
    )
    merged = market_monthly[["date", "rm"]].merge(rf_merge, on="date",
 ↪  how="inner")
    merged["mktrf_proxy"] = merged["rm"] - merged["rf_proxy"]

    n_months = merged["mktrf_proxy"].count()
    mean_monthly = merged["mktrf_proxy"].mean()
    std_monthly = merged["mktrf_proxy"].std()
    sharpe = mean_monthly / std_monthly if std_monthly > 0 else np.nan
    t_stat = mean_monthly / (std_monthly / np.sqrt(n_months))

    results.append({
        "Proxy": proxy_name.replace("_", " ").title(),
        "N Months": n_months,
        "Mean (% ann.)": round(mean_monthly * 12 * 100, 2),
        "Std (% ann.)": round(std_monthly * np.sqrt(12) * 100, 2),
        "Sharpe (ann.)": round(sharpe * np.sqrt(12), 3),
        "t-stat": round(t_stat, 2)
    })

pd.DataFrame(results).style.hide(axis="index")
```

results, researchers must justify their choice and report robustness.

5.6.1 Effect on the Equity Premium

5.6.2 Effect on Factor Premiums

Note that SMB and HML are constructed as long-short portfolio returns and should be identical regardless of the risk-free proxy. Only MKTRF differs. However, if the researcher uses the risk-free rate to compute individual stock excess returns before sorting into factor portfolios, small differences in sorting may arise.

Table 5.10: Factor Premium Sensitivity to Risk-Free Proxy (Annualized %)

```
# Load or construct factor returns
# Assume factors_monthly has: date, smb, hml (these are long-short,
 ↪  rf-independent)
# Only MKTRF changes with the proxy

factors_monthly = pd.read_parquet("data/factors_monthly.parquet")

for proxy_name, proxy_df in rf_proxies.items():
    rf_merge = proxy_df[["date", "rf_monthly"]].rename(
        columns={"rf_monthly": "rf_proxy"}
    )
    factors_merged = factors_monthly.merge(rf_merge, on="date", how="inner")
    factors_merged = factors_merged.merge(
        market_monthly[["date", "rm"]], on="date", how="inner"
    )
    factors_merged[f"mktrf_{proxy_name}"] = (
        factors_merged["rm"] - factors_merged["rf_proxy"]
    )

    mean_mktrf = factors_merged[f"mktrf_{proxy_name}"].mean() * 12 * 100
    mean_smb = factors_merged["smb"].mean() * 12 * 100
    mean_hml = factors_merged["hml"].mean() * 12 * 100

    print(
        f"{proxy_name:>25s}: MKTRF = {mean_mktrf:6.2f}%, "
        f"SMB = {mean_smb:6.2f}%, HML = {mean_hml:6.2f}%"
    )
```

5.6.3 Effect on Alpha Estimates

The choice of risk-free proxy affects alpha estimates for any portfolio evaluated against a factor model. We illustrate this by estimating the alpha of a momentum portfolio under each proxy.

5.6.4 Effect on Valuation

To illustrate the valuation impact, consider a simple DCF exercise where the cost of equity is estimated via the CAPM.

! Key Finding

Even modest differences in the risk-free rate (50-150 basis points across proxies) can produce terminal value differences of 10-30%. Researchers and practitioners must document their risk-free rate choice explicitly and report sensitivity to alternatives.

5.7 Term Structure Considerations

When longer-horizon discount rates are needed (e.g., for multi-year DCF or cost of capital estimation), the risk-free rate should be maturity-matched. This requires constructing a yield curve from available government bond data.

5.7.1 Yield Curve Estimation

Gürkaynak, Sack, and Wright (2007) develop a parametric approach to yield curve estimation using the Nelson and Siegel (1987) and Svensson (1994) models. The Nelson-Siegel model parameterizes the instantaneous forward rate as:

$$f(\tau) = \beta_0 + \beta_1 \exp\left(-\frac{\tau}{\lambda}\right) + \beta_2 \frac{\tau}{\lambda} \exp\left(-\frac{\tau}{\lambda}\right) \tag{5.7}$$

where τ is the maturity, β_0 is the long-run level, β_1 determines the slope, β_2 determines the curvature, and λ controls the location of the hump.

The corresponding yield is:

$$y(\tau) = \beta_0 + \beta_1 \frac{1 - \exp(-\tau/\lambda)}{\tau/\lambda} + \beta_2 \left[\frac{1 - \exp(-\tau/\lambda)}{\tau/\lambda} - \exp(-\tau/\lambda)\right] \tag{5.8}$$

Table 5.11: Momentum Portfolio Alpha Sensitivity to Risk-Free Proxy

```
import statsmodels.api as sm

# Assume momentum_ret contains: date, mom_ret (raw return of WML portfolio)
momentum_ret = pd.read_parquet("data/momentum_returns.parquet")

alpha_results = []
for proxy_name, proxy_df in rf_proxies.items():
    rf_merge = proxy_df[["date", "rf_monthly"]].rename(
        columns={"rf_monthly": "rf_proxy"}
    )

    merged = (
        momentum_ret
        .merge(rf_merge, on="date", how="inner")
        .merge(market_monthly[["date", "rm"]], on="date", how="inner")
        .merge(factors_monthly[["date", "smb", "hml"]], on="date",
 ↪  how="inner")
    )

    merged["mom_excess"] = merged["mom_ret"] - merged["rf_proxy"]
    merged["mktrf"] = merged["rm"] - merged["rf_proxy"]

    X = sm.add_constant(merged[["mktrf", "smb", "hml"]])
    y = merged["mom_excess"]
    model = sm.OLS(y, X).fit(cov_type="HAC", cov_kwds={"maxlags": 6})

    alpha_results.append({
        "Proxy": proxy_name.replace("_", " ").title(),
        "Alpha (% monthly)": round(model.params["const"] * 100, 3),
        "t-stat": round(model.tvalues["const"], 2),
        "R²": round(model.rsquared, 3)
    })

pd.DataFrame(alpha_results).style.hide(axis="index")
```

Table 5.12: Cost of Equity and Terminal Value Sensitivity to Risk-Free Proxy

```
# Example: firm with beta = 1.0, expected CF = 1 billion VND, growth = 3%
beta_example = 1.0
cf = 1e9  # VND
growth = 0.03

valuation_results = []
for proxy_name, proxy_df in rf_proxies.items():
    rf_ann = proxy_df["rf_annual"].dropna().iloc[-12:].mean() / 100  # Latest
 ↪  year avg

    rf_merge = proxy_df[["date", "rf_monthly"]].rename(
        columns={"rf_monthly": "rf_proxy"}
    )
    mkt_merged = market_monthly[["date", "rm"]].merge(
        rf_merge, on="date", how="inner"
    )
    mkt_merged["mktrf"] = mkt_merged["rm"] - mkt_merged["rf_proxy"]
    erp = mkt_merged["mktrf"].mean() * 12  # Annualized equity premium

    cost_equity = rf_ann + beta_example * erp
    terminal_value = cf / (cost_equity - growth) if cost_equity > growth
 ↪  else np.nan

    valuation_results.append({
        "Proxy": proxy_name.replace("_", " ").title(),
        "Rf (% ann.)": round(rf_ann * 100, 2),
        "ERP (% ann.)": round(erp * 100, 2),
        "Cost of Equity (%)": round(cost_equity * 100, 2),
        "Terminal Value (B VND)": round(terminal_value / 1e9, 1) if
         ↪  terminal_value else "N/A"
    })

pd.DataFrame(valuation_results).style.hide(axis="index")
```

```python
from scipy.optimize import minimize

def nelson_siegel_yield(tau, beta0, beta1, beta2, lam):
    """Nelson-Siegel yield curve model."""
    tau_lam = tau / lam
    factor1 = (1 - np.exp(-tau_lam)) / tau_lam
    factor2 = factor1 - np.exp(-tau_lam)
    return beta0 + beta1 * factor1 + beta2 * factor2

def fit_nelson_siegel(maturities, yields):
    """Fit Nelson-Siegel model to observed yields."""
    def objective(params):
        beta0, beta1, beta2, lam = params
        if lam <= 0:
            return 1e10
        fitted = nelson_siegel_yield(maturities, beta0, beta1, beta2, lam)
        return np.sum((yields - fitted) ** 2)

    result = minimize(
        objective,
        x0=[yields[-1], yields[0] - yields[-1], 0, 2.0],
        method="Nelder-Mead",
        options={"maxiter": 10000}
    )
    return result.x

# Example: fit to latest available government bond yields
# Assume govt_yields contains: date, maturity_years, yield_pct
govt_yields = pd.read_parquet("data/govt_bond_yields.parquet")

latest_date = govt_yields["date"].max()
latest_yields = govt_yields[govt_yields["date"] ==
 ↪  latest_date].sort_values("maturity_years")

maturities = latest_yields["maturity_years"].values
yields = latest_yields["yield_pct"].values

params = fit_nelson_siegel(maturities, yields)
beta0, beta1, beta2, lam = params
```

```
tau_fine = np.linspace(0.25, 30, 200)
fitted_yields = nelson_siegel_yield(tau_fine, *params)

fig, ax = plt.subplots(figsize=(7, 4))
ax.scatter(
    maturities, yields, color="#FF6B6B", s=60, zorder=5,
    label="Observed yields", edgecolors="white"
)
ax.plot(
    tau_fine, fitted_yields, color="#2C73D2", linewidth=2,
    label="Nelson-Siegel fit"
)
ax.set_xlabel("Maturity (Years)")
ax.set_ylabel("Yield (%)")
ax.legend(frameon=False)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.tight_layout()
plt.show()

print(f"Nelson-Siegel parameters:")
print(f"    (level)     = {beta0:.4f}")
print(f"    (slope)     = {beta1:.4f}")
print(f"    (curvature) = {beta2:.4f}")
print(f"    (decay)      = {lam:.4f}")
```

Figure 5.4

> **Tip**
>
> The Nelson-Siegel yield curve allows extraction of a risk-free rate at any maturity. For monthly asset pricing, evaluate the curve at $\tau = 1/12$ (one month). For DCF valuation with a 10-year horizon, evaluate at $\tau = 10$. This maturity-matching approach is superior to using a single proxy for all purposes.

5.7.2 Extracting the Short-Rate from the Yield Curve

```
# Extract 1-month rate from Nelson-Siegel curve at each date
def extract_ns_short_rate(govt_yields, target_maturity=1/12):
    """
    For each date, fit Nelson-Siegel and extract the yield
    at the target maturity.
    """
    dates = govt_yields["date"].unique()
    short_rates = []

    for d in dates:
        obs = govt_yields[govt_yields["date"] ==
↪   d].sort_values("maturity_years")
        if len(obs) < 3:  # Need at least 3 points to fit
            short_rates.append({"date": d, "rf_ns": np.nan})
            continue

        try:
            params = fit_nelson_siegel(
                obs["maturity_years"].values,
                obs["yield_pct"].values
            )
            rf_ns = nelson_siegel_yield(target_maturity, *params)
            short_rates.append({"date": d, "rf_ns": rf_ns})
        except Exception:
            short_rates.append({"date": d, "rf_ns": np.nan})

    return pd.DataFrame(short_rates)

ns_short_rates = extract_ns_short_rate(govt_yields)
```

5.7.3 Duration-Matched Excess Returns

The yield curve is also the input to a more radical proposal about what "excess return" should mean. The convention used throughout this book, subtracting a short rate from a monthly stock return, aligns the maturity of the benchmark with the *frequency at which returns are measured* rather than with the *maturity of the cash flows being priced*. Equities are long-duration claims; a one-month bill is not. Binsbergen, Ma, and Schwert (2026) argue that a cleaner counterfactual replaces each portfolio's claim to the cash flow at horizon h with a zero-coupon government bond of maturity h, strip by strip, and defines the excess return as the raw return less the return on that duration-matched bond portfolio. What remains is compensation for bearing risky cash flows, with the term premium stripped out and left to the fixed-income literature.

The consequences are larger than the mechanical nature of the adjustment suggests. On U.S. data from 1962 to 2020, the conventional market excess return averages 0.53% per month, while the duration-adjusted version averages −0.20% per month with roughly double the volatility, and the two series correlate below 0.5. In the cross-section, the value and investment premia grow under the duration-adjusted definition, the size premium shrinks, and momentum is essentially unchanged because winner and loser portfolios have similar duration. Across 153 published anomalies, 20 change significance status when only the excess return definition is altered (Binsbergen, Ma, and Schwert 2026), a point we return to in Section 21.1.

For Vietnam the same logic applies over a shorter and steeper rate path. The Nelson-Siegel curve fitted above is the raw material for a duration-matched benchmark, but two practical obstacles stand in the way. First, secondary market liquidity is concentrated in the 3-5 year segment (Table 5.2), so the fitted curve is extrapolation as much as estimation at both the very short and the very long end, precisely the maturities the strip-matching requires. Second, the strip weights need a dividend yield and an expected growth rate for each portfolio, which is fragile for firms with irregular payout histories, and Vietnamese payout policy is irregular by developed-market standards. A workable compromise is to report factor premia against both a short rate and a fixed long-maturity government bond, and to disclose the dividend yield differential between the long and short legs so readers can judge how much of a premium could be a duration bet.

5.8 Real vs. Nominal Risk-Free Rates

For certain applications, particularly long-horizon valuation and real return analysis, the real (inflation-adjusted) risk-free rate is more appropriate than the nominal rate. The Fisher equation relates them:

$$r_f^{real} \approx r_f^{nominal} - \pi^e \tag{5.9}$$

where π^e is expected inflation. In practice, we can use realized CPI inflation as a proxy for expected inflation (under the assumption of rational expectations, or as an ex-post adjustment).

```
# Load CPI data
# Assume cpi_data contains: date, cpi_index (or inflation_mom for
 ↪  month-over-month)
cpi_data = pd.read_parquet("data/cpi_monthly.parquet")
cpi_data = cpi_data.sort_values("date")
cpi_data["inflation_monthly"] = cpi_data["cpi_index"].pct_change()

rf_real = rf_blended[["date", "rf_monthly"]].merge(
    cpi_data[["date", "inflation_monthly"]], on="date", how="inner"
)
rf_real["rf_real"] = rf_real["rf_monthly"] - rf_real["inflation_monthly"]
```

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(
    rf_real["date"], rf_real["rf_monthly"] * 100,
    color="#2C73D2", label="Nominal", linewidth=1
)
ax.plot(
    rf_real["date"], rf_real["rf_real"] * 100,
    color="#FF6B6B", label="Real", linewidth=1
)
ax.axhline(0, color="black", linewidth=0.5, linestyle="--")
ax.set_ylabel("Monthly Rate (%)")
ax.set_xlabel("")
ax.legend(frameon=False)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.tight_layout()
plt.show()
```

Figure 5.5

Note

In periods of high inflation, the real risk-free rate can be substantially negative. This has implications for real excess return computation and for interpreting the equity premium in real terms. A negative real risk-free rate implies that nominal government securities

do not preserve purchasing power, which strengthens the case for equity investment from a real-return perspective.

5.9 International Comparison

It is instructive to compare Vietnam's risk-free rate environment with that of other emerging and developed markets to contextualize the magnitudes involved (Table 5.13).

Table 5.13: Risk-Free Rate Proxies: International Comparison

Country	Typical Proxy	Short-Term Maturity Available	Typical Level (%, 2020-2024)
Vietnam	Interbank overnight / 1Y govt bond	Limited (<1Y sparse)	2.5-6.0
Thailand	1-day bilateral repurchase rate	Yes (1-day repo)	0.5-2.5
Indonesia	Bank Indonesia 7-day reverse repo rate	Yes (7-day, 1M)	3.5-6.0
Philippines	91-day T-bill	Yes (91-day T-bill)	1.5-6.0
India	91-day T-bill	Yes (91-day T-bill)	3.5-7.0
South Korea	91-day CD rate	Yes (91-day CD)	1.0-3.5
Japan	Uncollateralized overnight call rate	Yes (overnight)	-0.1-0.1

```
# get the risk-free rate of Asian markets
```

Vietnam's risk-free rate environment is characterized by relatively high nominal rates (reflecting inflation and growth dynamics), limited availability of very-short-maturity sovereign instruments, and greater reliance on interbank rates as the operational proxy. This is broadly similar to other ASEAN frontier markets but contrasts sharply with developed Asian markets, where deep government securities markets provide clean short-term benchmarks.

5.10 Best Practices Checklist

Based on the analysis in this chapter, we summarize the recommended practices for risk-free rate construction in Vietnamese financial research (Table 5.14).

Table 5.14: Best Practices for Risk-Free Rate Construction

Practice	Rationale
Use the shortest available maturity	Minimizes term premium contamination
Match frequency to return data	Avoids compounding mismatches between rf and stock returns
Use the same compounding convention	Arithmetic returns require arithmetic rf conversion
Document the proxy and source explicitly	Enables replication; different proxies give different results
Report sensitivity to alternative proxies	Demonstrates robustness of conclusions
Avoid forward-looking policy rates	Policy rates are not investable returns
Handle gaps with documented interpolation	Forward-fill, then average within month; document procedure
Use Nelson-Siegel for maturity matching	Enables extraction of any-maturity rate from sparse data
Consider real rates for long-horizon analysis	Nominal rates overstate real compensation in high-inflation periods
Maintain a single consistent series per study	Mixing proxies across periods introduces structural breaks

5.11 Saving the Risk-Free Rate for Downstream Use

The final step is to save the constructed risk-free rate series for use in subsequent chapters.

```
# Save all variants for flexibility
rf_output = rf_blended[["date", "rf_monthly", "source"]].copy()
rf_output["rf_annual_pct"] = rf_output["rf_monthly"] * 12 * 100

# Also save individual proxies
for proxy_name, proxy_df in rf_proxies.items():
    col_name = f"rf_{proxy_name}"
    rf_output = rf_output.merge(
        proxy_df[["date", "rf_monthly"]].rename(
            columns={"rf_monthly": col_name}
        ),
        on="date",
        how="left"
    )
```

```
# Save to parquet for use in later chapters
rf_output.to_parquet("data/risk_free_rate.parquet", index=False)

print(f"Risk-free rate series saved: {len(rf_output)} months")
print(f"Date range: {rf_output['date'].min()} to {rf_output['date'].max()}")
print(f"\nColumns: {list(rf_output.columns)}")
```

Tip

By saving the risk-free rate as a separate, well-documented file, all downstream chapters can merge it consistently. This avoids the common pitfall of reconstructing the risk-free rate differently in different analyses within the same study.

5.12 Summary

This chapter has established that risk-free rate construction is a first-order modeling decision in Vietnamese financial research, not a technical afterthought. The key takeaways are:

1. **No perfect proxy exists** in Vietnam. The interbank overnight rate, 1-year government bond yield, and SBV policy rate each have distinct strengths and limitations. A blended series using a documented priority hierarchy provides the most robust baseline.
2. **The choice of proxy matters quantitatively**. Different proxies can shift the estimated equity premium by 50-200 basis points annually, alter portfolio alphas, and change DCF terminal values by 10-30%.
3. **Frequency alignment and compounding conventions** must be handled with care. Converting annualized rates to monthly requires specifying the compounding convention. Gaps in the data require documented interpolation.
4. **The Nelson-Siegel yield curve model** enables the extraction of any-maturity risk-free rate from sparse government bond data, which is particularly valuable for maturity-matched discount rate estimation.
5. **Sensitivity analysis is mandatory**. Any study that reports results under a single risk-free proxy without reporting robustness to alternatives has an unquantified source of specification uncertainty.

6 Constructing and Analyzing Equity Return Series

This chapter develops a practical framework for transforming raw equity price records into return series suitable for empirical financial analysis. The focus is on methodological clarity and reproducibility, with particular attention to data issues that are prevalent in emerging equity markets such as Vietnam.

The discussion proceeds from individual stocks to a broad market cross-section, using constituents of the VN30 index as the primary empirical setting.

6.1 Data Access and Preparation

We begin by loading the core numerical and data manipulation libraries. These provide all functionality required for return construction without relying on specialized financial wrappers.

```
import pandas as pd
import numpy as np
```

For this project, we retrieve our historical price data using the DataCore API. If you wish to replicate this analysis or use the dataset for your own work, you will need to access the data through their platform.

6.1.1 Prerequisites for API Access

To run the code below, you need to configure a few things first:

1. **Obtain an API Key:** You must subscribe to the relevant dataset on the DataCore platform to receive a unique API key.
2. **Whitelist Your IP Address:** The API requires your IP address to be whitelisted for security.
    - **Local Machine:** If you are running this code on your personal computer, you generally need to whitelist your public IP address.

- **Cloud or Remote Sessions (e.g., HPC Open OnDemand):** If you are using a remote server such as those provided by DataCore, the server's IP address will change with each new session. You must retrieve the server's private/public IP for that specific session and whitelist it in your DataCore account settings before running the script.

3. **Set Environment Variables:** To keep your credentials secure, do not hardcode your API key into your scripts. Instead, save it as an environment variable named `datacore_api` on your machine.

Note: If you only want to test the code performance, DataCore provides a preview endpoint that does not require an API key, though the data returned is limited.

```
import requests
import pandas as pd

url = "https://gateway.datacore.vn/data/ds/preview"
params = {
    "dataSetCode": "fundamental_annual",
    "pageSize": 10000
}
headers = {
    "Accept": "application/json",
    "Origin": "https://datacore.vn",
    "Referer": "https://datacore.vn/"
}

response = requests.get(url, params=params, headers=headers)
data = response.json()

columns = data['data']['fields']
rows = data['data']['dataDetail']

df = pd.DataFrame(rows, columns=columns)
print(df.head())
print("Total rows:", len(df))
```

```
  symbol  year  total_current_asset ca_fin        cce       cash  \
0    CLL  2011         1.078338e+11   None  8.313178e+10  4.131776e+09
1    CLL  2012         2.360575e+10   None  8.003560e+09  4.003560e+09
2    CLL  2013         5.764370e+10   None  3.496426e+10  9.964256e+09
3    CLL  2014         4.973590e+10   None  1.718744e+10  1.718744e+10
4    CLL  2015         2.389115e+11   None  1.790364e+11  2.403638e+10
```

```
  ca_cash_inbank ca_cash_attransit  ca_cash_equivalent  ca_fin_invest  ...  \
0           None              None        7.900000e+10   0.000000e+00  ...
1           None              None        4.000000e+09   0.000000e+00  ...
2           None              None        2.500000e+10   0.000000e+00  ...
3           None              None        0.000000e+00   1.000000e+09  ...
4           None              None        1.550000e+11   1.000000e+09  ...

   operating_margin      roe      roa sector_pe sector_pb  sector_ps  \
0           0.35473  0.19560  0.10649   3.00213   0.26108    0.22170
1           0.45369  0.20337  0.13018   2.40335   0.26211    0.21745
2           0.45997  0.23459  0.16452   3.11089   0.41013    0.32436
3           0.40554  0.19984  0.14747   3.25886   0.46823    0.42146
4           0.33774  0.16525  0.12633   6.77337   0.81110    0.68401

   sector_eps  sector_ros  sector_roe  sector_roa
0  3341.54903     0.07385     0.08753     0.05039
1  4296.47005     0.09048     0.11392     0.06346
2  4024.74648     0.10427     0.13645     0.07415
3  5100.81019     0.12933     0.14956     0.08087
4  5216.38499     0.10098     0.11815     0.06234

[5 rows x 308 columns]
Total rows: 10
```

6.1.2 Checking Your IP Address

If you need to verify the IP address of the machine running your code (to whitelist it), you can use the following Python snippets.

To find your Public IP:

```python
import requests

try:
    public_ip = requests.get("https://api.ipify.org").text
    print(f"Public IP: {public_ip}")
except requests.exceptions.RequestException as e:
    print(f"Could not retrieve Public IP: {e}")
```

To find your Private IP (useful for specific remote server setups):

```
import socket

def get_private_ip():
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        private_ip = s.getsockname()[0]
        s.close()
        return private_ip
    except Exception as e:
        return f"Error: {e}"

print(f"Private IP: {get_private_ip()}")
```

6.1.3 Fetching the Dataset

The following script demonstrates how to securely authenticate and paginate through the DataCore API to retrieve the full `dataset_historical_price` dataset.

```
# Convert the date column to proper datetime objects
prices["date"] = pd.to_datetime(prices["date"])

# Ensure price and ratio columns are numeric before calculation
prices["close_price"] = pd.to_numeric(prices["close_price"])
prices["adj_ratio"] = pd.to_numeric(prices["adj_ratio"])

# Calculate the adjusted close price
prices["adjusted_close"] = prices["close_price"] * prices["adj_ratio"]

# Rename columns to match standard conventions
prices = prices.rename(
    columns={
        "vol_total": "volume",
        "open_price": "open",
        "low_price": "low",
        "high_price": "high",
        "close_price": "close",
    }
)

# Sort the dataset logically by symbol and date
```

```
prices = prices.sort_values(["symbol", "date"])

print("Data manipulation complete. The dataset is ready for analysis.")
```

```
Data manipulation complete. The dataset is ready for analysis.
```

```
prices["date"] = pd.to_datetime(prices["date"])


prices["adjusted_close"] = prices["close_price"] * prices["adj_ratio"]


prices = prices.rename(
    columns={
        "vol_total": "volume",
        "open_price": "open",
        "low_price": "low",
        "high_price": "high",
        "close_price": "close",
    }
)

prices = prices.sort_values(["symbol", "date"])
```

Adjusted closing prices incorporate mechanical changes due to corporate actions such as cash dividends and stock splits. Using adjusted prices ensures that subsequent return calculations reflect investor-relevant performance rather than accounting artifacts.

6.2 Examining a Single Equity

To ground the discussion, we isolate the trading history of a single large-cap stock, FPT, over a long sample period.

```
import datetime as dt

start = pd.Timestamp("2000-01-01")
end = pd.Timestamp(dt.date.today().year - 1, 12, 31)
```

```
fpt = prices.loc[
    (prices["symbol"] == "FPT")
    & (prices["date"] >= start)
    & (prices["date"] <= end),
    ["date", "symbol", "volume", "open", "low", "high", "close",
 ↪  "adjusted_close"],
].copy()
```

This subset contains the standard daily market variables required for most empirical studies. Before computing returns, it is good practice to visually inspect the price series.

```
from plotnine import ggplot, aes, geom_line, labs
```

```
(
    ggplot(fpt, aes(x="date", y="adjusted_close"))
    + geom_line()
    + labs(title="Adjusted price path of FPT", x="", y="")
)
```

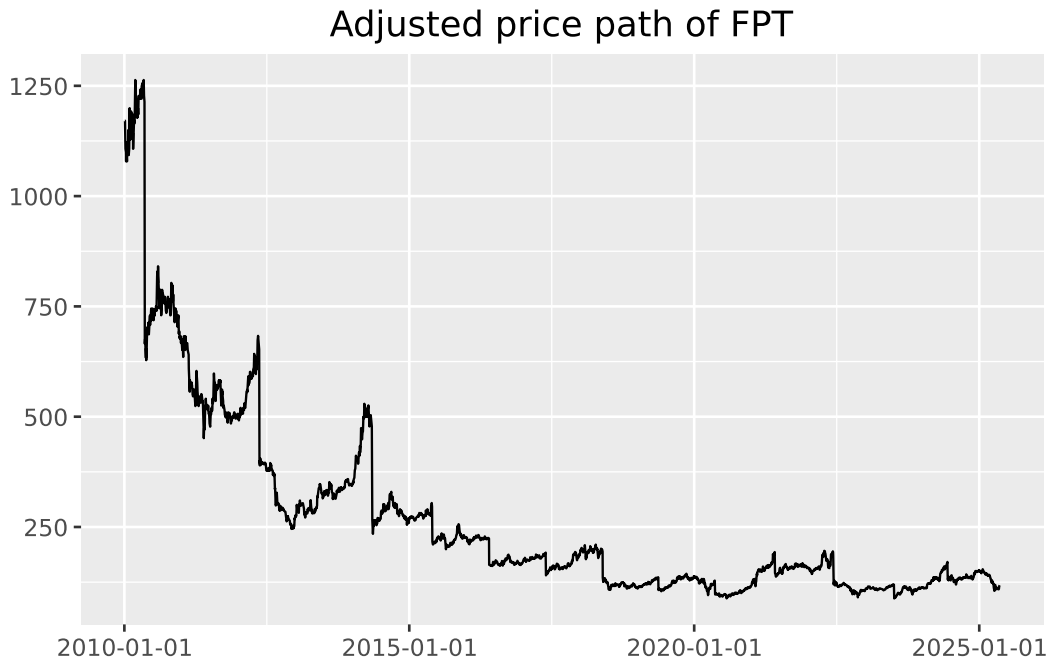


Figure 6.1: Prices are in VND, adjusted for dividend payments and stock splits.

6.3 From Prices to Returns

Most empirical asset pricing models are formulated in terms of returns rather than price levels. The simple daily return is defined as

$$r_t = \frac{p_t}{p_t - 1} - 1,$$

where p_t denotes the adjusted closing price at the end of trading day t.

Before computing returns, we must address invalid price observations. In Vietnamese equity data, adjusted prices occasionally take the value zero. These entries typically arise from IPO placeholders, trading suspensions, or historical backfilling conventions and cannot be used to compute meaningful returns.

```
prices.loc[prices["adjusted_close"] <= 0, ["symbol", "date",
 ↪  "adjusted_close"]].head()
```

	symbol	date	adjusted_close
33886	ADP	2010-02-09	0.0
33887	ADP	2010-02-24	0.0
33888	ADP	2010-03-01	0.0
33889	ADP	2010-03-03	0.0
33890	ADP	2010-03-12	0.0

We therefore exclude non-positive adjusted prices and compute returns stock by stock. Correct chronological ordering is essential.

```
returns = (
    prices
    .loc[prices["adjusted_close"] > 0]
    .sort_values(["symbol", "date"])
    .assign(ret=lambda x: x.groupby("symbol")["adjusted_close"].pct_change())
    [["symbol", "date", "ret"]]
)
returns = returns.dropna(subset=["ret"])
```

The initial return for each stock is missing by construction, since no lagged price is available. These observations are mechanical and can safely be removed in most applications.

6.4 Limiting the Influence of Extreme Returns

Daily return series often contain extreme observations driven by data errors, thin trading, or abrupt price adjustments. A common approach is to winsorize returns using cross-sectional percentile cutoffs.

```
def winsorize_cs(df, column="ret", lower_q=0.01, upper_q=0.99):
    lo = df[column].quantile(lower_q)
    hi = df[column].quantile(upper_q)
    out = df.copy()
    out[column] = out[column].clip(lo, hi)
    return out

returns = winsorize_cs(returns)
```

Applying winsorization across the full cross-section limits the impact of extreme market-wide observations while preserving relative differences between firms. Winsorizing within each stock is rarely appropriate in panel settings and can severely distort illiquid securities.

6.5 Distributional Features of Returns

We next examine the empirical distribution of daily returns for FPT. The figure below also marks the historical 5 percent quantile, which provides a simple, non-parametric measure of downside risk.

```
from mizani.formatters import percent_format
from plotnine import geom_histogram, geom_vline, scale_x_continuous


fpt_ret = returns.loc[returns["symbol"] == "FPT"].copy()
q05 = fpt_ret["ret"].quantile(0.05)
```

```
(
    ggplot(fpt_ret, aes(x="ret"))
    + geom_histogram(bins=100)
    + geom_vline(xintercept=q05, linetype="dashed")
    + scale_x_continuous(labels=percent_format())
    + labs(title="Distribution of daily FPT returns", x="", y="")
)
```

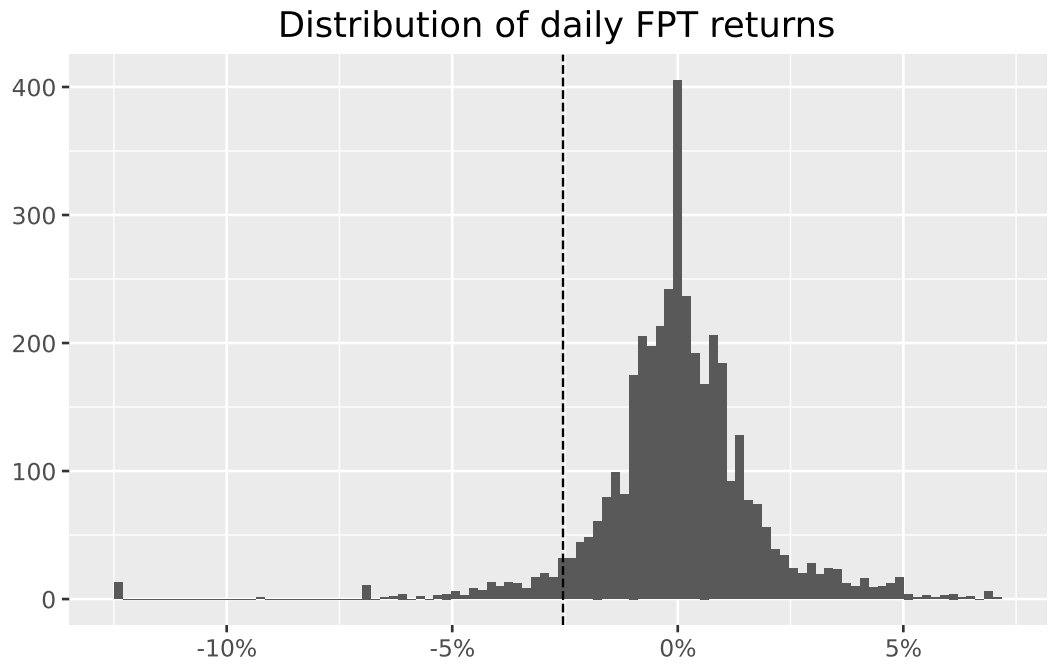


Figure 6.2: The dotted vertical line indicates the historical five percent quantile.

Summary statistics offer a compact description of return behavior and should always be inspected before formal modeling.

```
returns["ret"].describe().round(3)
```

```
count    4305063.000
mean           0.000
std            0.035
min           -0.125
25%           -0.004
50%            0.000
75%            0.003
max            0.130
Name: ret, dtype: float64
```

Computing these statistics by calendar year can reveal periods of elevated volatility or structural change.

```
(
    returns
    .assign(year=lambda x: x["date"].dt.year)
    .groupby("year")["ret"]
    .describe()
    .round(3)
)
```

year	count	mean	std	min	25%	50%	75%	max
2010	131548.0	-0.001	0.036	-0.125	-0.021	0.0	0.018	0.13
2011	166826.0	-0.003	0.033	-0.125	-0.020	0.0	0.011	0.13
2012	177938.0	0.000	0.033	-0.125	-0.012	0.0	0.015	0.13
2013	180417.0	0.001	0.033	-0.125	-0.004	0.0	0.008	0.13
2014	181907.0	0.001	0.034	-0.125	-0.008	0.0	0.011	0.13
2015	197881.0	0.000	0.033	-0.125	-0.006	0.0	0.005	0.13
2016	227896.0	0.000	0.035	-0.125	-0.005	0.0	0.003	0.13
2017	283642.0	0.001	0.034	-0.125	-0.002	0.0	0.001	0.13
2018	329887.0	0.000	0.035	-0.125	0.000	0.0	0.000	0.13
2019	352754.0	0.000	0.033	-0.125	0.000	0.0	0.000	0.13
2020	369367.0	0.001	0.035	-0.125	0.000	0.0	0.000	0.13
2021	379415.0	0.002	0.038	-0.125	-0.005	0.0	0.007	0.13
2022	387050.0	-0.001	0.038	-0.125	-0.008	0.0	0.004	0.13
2023	391605.0	0.001	0.034	-0.125	-0.002	0.0	0.002	0.13
2024	400379.0	0.000	0.031	-0.125	-0.002	0.0	0.000	0.13
2025	146551.0	0.000	0.037	-0.125	-0.004	0.0	0.002	0.13

6.6 Expanding to a Market Cross-Section

The same procedures apply naturally to a larger universe of stocks. We now restrict attention to the constituents of the VN30 index.

```
vn30 = [
    "ACB","BCM","BID","BVH","CTG","FPT","GAS","GVR","HDB","HPG",
    "MBB","MSN","MWG","PLX","POW","SAB","SHB","SSB","STB","TCB",
    "TPB","VCB","VHM","VIB","VIC","VJC","VNM","VPB","VRE","EIB",
]
```

```
prices_vn30 = prices.loc[prices["symbol"].isin(vn30)]
from plotnine import theme
```

```
(
    ggplot(prices_vn30, aes(x="date", y="adjusted_close", color="symbol"))
    + geom_line()
    + labs(title="Adjusted prices of VN30 constituents", x="", y="")
    + theme(legend_position="none")
)
```

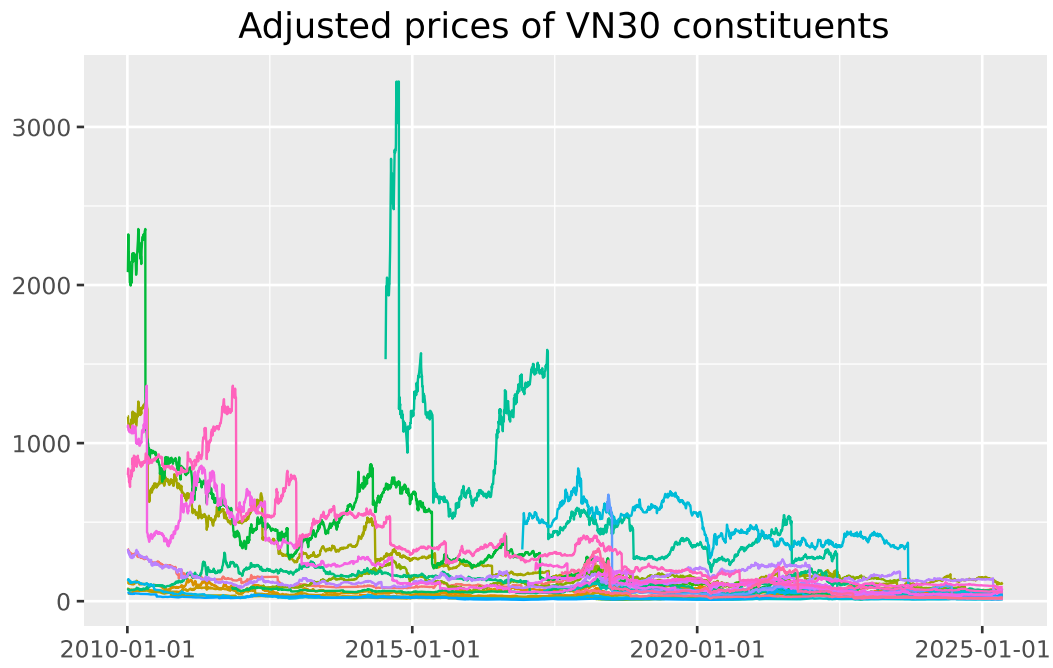


Figure 6.3: Prices in VND, adjusted for dividend payments and stock splits.

Returns for the VN30 universe are computed analogously.

```
returns_vn30 = (
    prices_vn30
    .sort_values(["symbol", "date"])
    .assign(ret=lambda x: x.groupby("symbol")["adjusted_close"].pct_change())
    [["symbol", "date", "ret"]]
    .dropna()
)
```

```
returns_vn30.groupby("symbol")["ret"].describe().round(3)
```

symbol	count	mean	std	min	25%	50%	75%	max
ACB	3822.0	-0.000	0.023	-0.407	-0.006	0.0	0.007	0.097
BCM	1795.0	0.001	0.027	-0.136	-0.010	0.0	0.010	0.159
BID	2811.0	0.000	0.024	-0.369	-0.010	0.0	0.011	0.070
BVH	3825.0	0.000	0.024	-0.097	-0.012	0.0	0.012	0.070
CTG	3825.0	0.000	0.024	-0.376	-0.010	0.0	0.010	0.070
EIB	3825.0	-0.000	0.022	-0.302	-0.008	0.0	0.008	0.070
FPT	3825.0	-0.000	0.024	-0.439	-0.008	0.0	0.009	0.070
GAS	3236.0	0.000	0.022	-0.289	-0.009	0.0	0.010	0.070
GVR	1775.0	0.001	0.030	-0.137	-0.014	0.0	0.016	0.169
HDB	1828.0	-0.001	0.028	-0.391	-0.009	0.0	0.010	0.070
HPG	3825.0	-0.001	0.032	-0.581	-0.010	0.0	0.011	0.070
MBB	3371.0	-0.000	0.023	-0.473	-0.008	0.0	0.008	0.069
MSN	3825.0	0.000	0.024	-0.553	-0.010	0.0	0.010	0.070
MWG	2701.0	-0.000	0.035	-0.751	-0.009	0.0	0.011	0.070
PLX	2009.0	-0.000	0.021	-0.140	-0.010	0.0	0.010	0.070
POW	1784.0	0.000	0.023	-0.071	-0.012	0.0	0.011	0.102
SAB	2100.0	-0.000	0.024	-0.745	-0.008	0.0	0.007	0.070
SHB	3824.0	-0.000	0.028	-0.338	-0.013	0.0	0.013	0.100
SSB	1029.0	-0.000	0.023	-0.292	-0.005	0.0	0.004	0.070
STB	3825.0	0.000	0.024	-0.321	-0.010	0.0	0.010	0.070
TCB	1732.0	-0.000	0.035	-0.884	-0.009	0.0	0.010	0.070
TPB	1761.0	-0.001	0.029	-0.477	-0.009	0.0	0.009	0.070
VCB	3825.0	-0.000	0.024	-0.539	-0.009	0.0	0.009	0.070
VHM	1744.0	-0.000	0.024	-0.419	-0.009	0.0	0.008	0.070
VIB	2072.0	-0.000	0.031	-0.489	-0.009	0.0	0.010	0.109
VIC	3825.0	-0.000	0.027	-0.673	-0.008	0.0	0.008	0.070
VJC	2046.0	-0.000	0.020	-0.455	-0.007	0.0	0.006	0.070
VNM	3825.0	-0.000	0.023	-0.547	-0.007	0.0	0.007	0.070
VPB	1927.0	-0.000	0.033	-0.678	-0.010	0.0	0.010	0.070
VRE	1871.0	-0.000	0.024	-0.295	-0.012	0.0	0.011	0.070

6.7 Aggregating Returns Across Time

Financial variables are observed at different frequencies. While equity prices are recorded daily, many empirical questions require monthly or annual returns. Lower-frequency returns are constructed by compounding higher-frequency observations.

```
returns_monthly = (
    returns_vn30
    .assign(month=lambda x: x["date"].dt.to_period("M").dt.to_timestamp())
    .groupby(["symbol", "month"], as_index=False)
    .agg(ret=("ret", lambda x: np.prod(1 + x) - 1))
)
```

Comparing daily and monthly return distributions illustrates how aggregation dampens volatility and alters tail behavior.

```
from plotnine import facet_wrap

fpt_d = returns_vn30.loc[returns_vn30["symbol"] ==
 ↪  "FPT"].assign(freq="Daily")
fpt_m = returns_monthly.loc[returns_monthly["symbol"] ==
 ↪  "FPT"].assign(freq="Monthly")


fpt_both = pd.concat([
    fpt_d[["ret", "freq"]],
    fpt_m[["ret", "freq"]],
])
```

```
(
    ggplot(fpt_both, aes(x="ret"))
    + geom_histogram(bins=50)
    + scale_x_continuous(labels=percent_format())
    + labs(title="FPT returns at different frequencies", x="", y="")
    + facet_wrap("freq", scales="free")
)
```

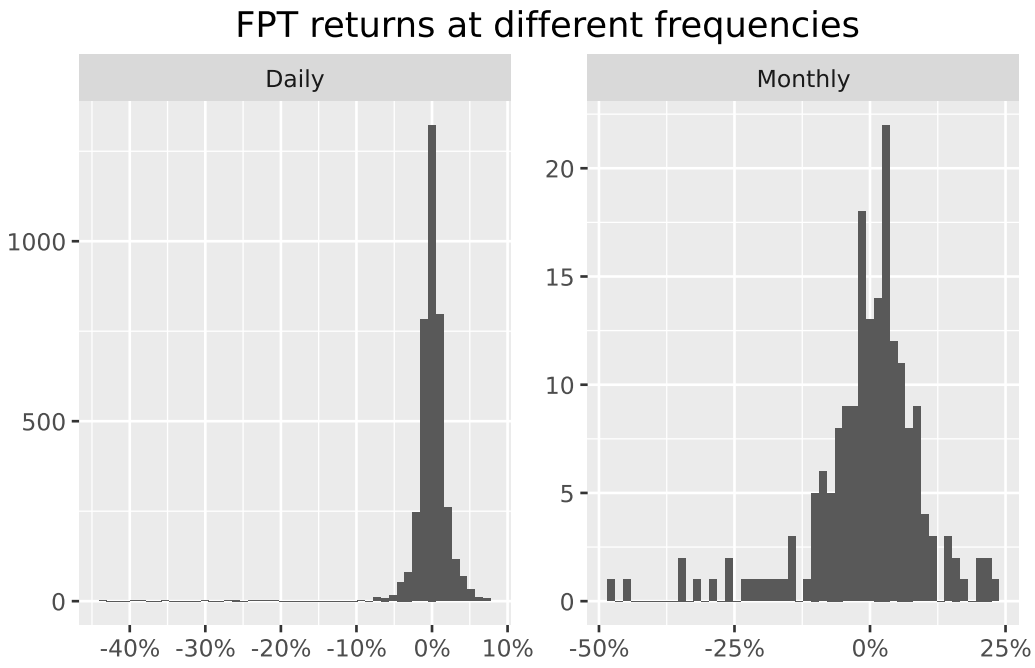
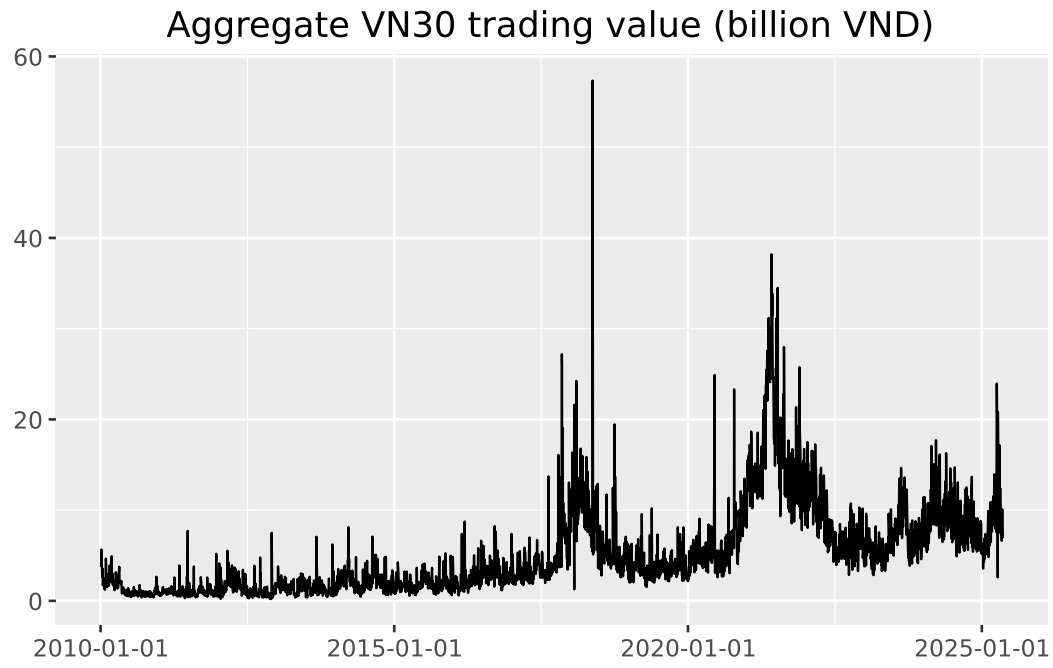


Figure 6.4: Returns are based on prices adjusted for dividend payments and stock splits.

6.8 Aggregation Across Firms: Trading Activity

Aggregation is not limited to time. In some settings, it is informative to aggregate variables across firms. As an illustration, we compute total daily trading value for VN30 stocks by multiplying share volume by adjusted prices and summing across firms.

```
trading_value = (
    prices_vn30
    .assign(value=lambda x: x["volume"] * x["adjusted_close"] / 1e9)
    .groupby("date")["value"]
    .sum()
    .reset_index()
    .assign(value_lag=lambda x: x["value"].shift(1))
)
(
    ggplot(trading_value, aes(x="date", y="value"))
    + geom_line()
    + labs(title="Aggregate VN30 trading value (billion VND)", x="", y="")
)
```



Finally, we assess persistence in trading activity by comparing trading value on consecutive days.

```
from plotnine import geom_point, geom_abline
```

```
(
    ggplot(trading_value, aes(x="value_lag", y="value"))
    + geom_point()
    + geom_abline(intercept=0, slope=1, linetype="dashed")
    + labs(
        title="Persistence in VN30 trading value",
        x="Previous day",
        y="Current day",
    )
)
```

```
/home/mikenguyen/project/tidyfinance/.venv/lib/python3.13/site-packages/plotnine/layer.py:374
```

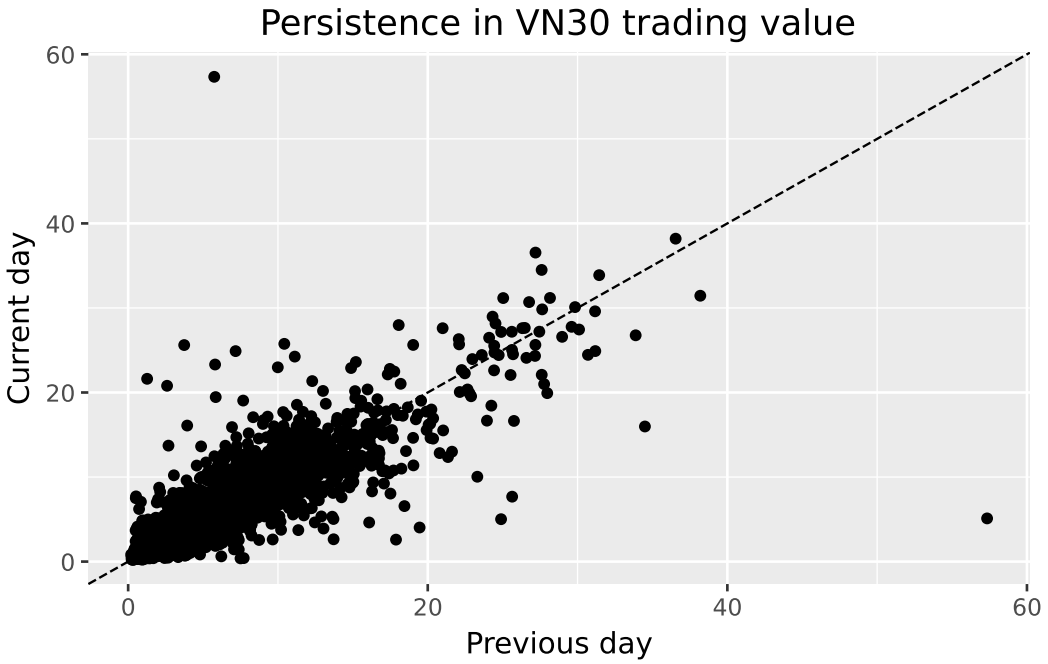


Figure 6.5: Total daily trading volume.

A strong alignment along the 45-degree line indicates that high-activity trading days tend to be followed by similarly active days, a common empirical regularity in equity markets.

6.9 Summary

This chapter established a reproducible workflow for transforming raw price data into return series, diagnosing common data issues, and aggregating information across time and firms. These steps provide the empirical foundation for subsequent analyses of risk, return predictability, and market dynamics in Vietnam's equity market.

7 Compound Returns

In this chapter, we provide a treatment of compound returns. Whether constructing buy-and-hold portfolios, evaluating fund performance, computing cumulative wealth indices, or estimating long-horizon risk measures, the ability to correctly compound returns over arbitrary horizons is indispensable. We begin with the mathematical foundations: the distinction between simple and log returns, the relationship between arithmetic and geometric means, and the properties of continuously compounded returns. Along the way, we address practical complications that arise in real-world equity data, such as trading halts, price limit mechanisms, partial-period returns, and delisting events, and show how to handle them in the Vietnamese context.

The chapter proceeds to rolling compound returns over standard horizons (3, 6, 9, and 12 months), compound returns aligned to fiscal period ends, forward-looking cumulative returns for event studies, and rolling volatility estimation.

```
import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
from plotnine import *
from mizani.formatters import percent_format, comma_format, date_format
from itertools import product
from datetime import datetime, timedelta
```

7.1 Simple Returns versus Log Returns

Before discussing compounding, we must distinguish between the two fundamental return conventions used in finance.

7.1.1 Simple (Arithmetic) Returns

The simple gross return on an asset from period $t-1$ to t is defined as

$$1 + R_t = \frac{P_t + D_t}{P_{t-1}}, \tag{7.1}$$

where P_t denotes the price at the end of period t and D_t denotes any cash distributions (dividends, coupons) paid during period t. The simple net return is R_t itself. When we speak of "returns" without qualification, we typically mean simple net returns.

The key property of simple returns is that **multi-period compounding is multiplicative**:

$$1 + R_t(k) = \prod_{j=0}^{k-1}(1 + R_{t-j}) = (1 + R_t)(1 + R_{t-1}) \cdots (1 + R_{t-k+1}), \tag{7.2}$$

where $R_t(k)$ is the k-period compound return ending at time t. This multiplicative structure is the foundation of all compounding methods discussed in this chapter.

7.1.2 Continuously Compounded (Log) Returns

The continuously compounded return, or log return, is defined as

$$r_t = \ln(1 + R_t) = \ln\left(\frac{P_t + D_t}{P_{t-1}}\right). \tag{7.3}$$

The central advantage of log returns for compounding is that **multi-period compounding becomes additive**:

$$r_t(k) = \ln(1 + R_t(k)) = \sum_{j=0}^{k-1} r_{t-j} = r_t + r_{t-1} + \cdots + r_{t-k+1}. \tag{7.4}$$

This additive property follows directly from the logarithmic identity $\ln(ab) = \ln(a) + \ln(b)$. It is computationally convenient because summation is numerically more stable than iterated multiplication, and because many statistical procedures (means, variances, regressions) operate naturally on additive quantities.

To recover the simple compound return from the sum of log returns, we apply the exponential function:

$$R_t(k) = \exp\left(\sum_{j=0}^{k-1} r_{t-j}\right) - 1. \tag{7.5}$$

7.1.3 When Do They Diverge?

For small returns, the approximation $r_t \approx R_t$ holds to first order (via the Taylor expansion $\ln(1+x) \approx x$ for $|x| \ll 1$). However, for large returns, which is common in emerging markets, small-cap stocks, or crisis periods, the two can diverge substantially. Consider a stock that doubles in price ($R_t = 1.0$): the log return is $r_t = \ln(2) \approx 0.693$, a 31% discrepancy. Conversely, for a stock that loses half its value ($R_t = -0.5$): the log return is $r_t = \ln(0.5) \approx -0.693$, which is 39% larger in magnitude.

This divergence is especially relevant in Vietnam, where daily price limits of $\pm 7\%$ on HOSE, $\pm 10\%$ on HNX, and $\pm 15\%$ on UPCoM can produce sequences of limit-up or limit-down days. Over a week of consecutive limit-up days on HOSE, the simple return is $(1.07)^5 - 1 = 40.3\%$ while the log return is $5 \times \ln(1.07) = 33.8\%$, which is a meaningful gap.

Table 7.1 illustrates this divergence across a range of return magnitudes.

```
simple_returns = [-0.50, -0.30, -0.15, -0.10, -0.07, -0.05, -0.01,
                  0.00, 0.01, 0.05, 0.07, 0.10, 0.15, 0.30, 0.50, 1.00]
comparison_df = pd.DataFrame({
    "Simple Return": [f"{r:.2%}" for r in simple_returns],
    "Log Return": [f"{np.log(1+r):.4f}" for r in simple_returns],
    "Difference": [f"{np.log(1+r) - r:.4f}" for r in simple_returns],
    "Relative Error (%)": [
        f"{((np.log(1+r) - r) / abs(r) * 100):.2f}" if r != 0 else "-"
        for r in simple_returns
    ]
})
comparison_df
```

Table 7.1: Comparison of simple and log returns for various price changes. The divergence grows with the magnitude of the simple return, which is particularly relevant for volatile emerging market stocks.

	Simple Return	Log Return	Difference	Relative Error (%)
0	-50.00%	-0.6931	-0.1931	-38.63
1	-30.00%	-0.3567	-0.0567	-18.89
2	-15.00%	-0.1625	-0.0125	-8.35
3	-10.00%	-0.1054	-0.0054	-5.36
4	-7.00%	-0.0726	-0.0026	-3.67
5	-5.00%	-0.0513	-0.0013	-2.59
6	-1.00%	-0.0101	-0.0001	-0.50
7	0.00%	0.0000	0.0000	—

Table 7.1: Comparison of simple and log returns for various price changes. The divergence grows with the magnitude of the simple return, which is particularly relevant for volatile emerging market stocks.

	Simple Return	Log Return	Difference	Relative Error (%)
8	1.00%	0.0100	-0.0000	-0.50
9	5.00%	0.0488	-0.0012	-2.42
10	7.00%	0.0677	-0.0023	-3.34
11	10.00%	0.0953	-0.0047	-4.69
12	15.00%	0.1398	-0.0102	-6.83
13	30.00%	0.2624	-0.0376	-12.55
14	50.00%	0.4055	-0.0945	-18.91
15	100.00%	0.6931	-0.3069	-30.69

Key takeaway: log returns are convenient for compounding (additive aggregation), but portfolio returns aggregate cross-sectionally in simple return space. In practice, we often transform to log returns for temporal compounding, then convert back to simple returns for reporting.

7.2 Mathematical Foundations of Compounding

7.2.1 Geometric Mean Return

The geometric mean return over T periods is

$$\bar{R}_g = \left(\prod_{t=1}^{T} (1 + R_t) \right)^{1/T} - 1, \tag{7.6}$$

which represents the constant per-period return that would yield the same terminal wealth as the actual return sequence. It is always less than or equal to the arithmetic mean $\bar{R}_a = \frac{1}{T} \sum_{t=1}^{T} R_t$, with equality only when all returns are identical. The relationship between the two is approximately:

$$\bar{R}_g \approx \bar{R}_a - \frac{\sigma^2}{2}, \tag{7.7}$$

where σ^2 is the variance of returns. This approximation, sometimes called the "volatility drag," has important implications: high-volatility assets have a larger wedge between their arithmetic and geometric means, meaning their actual compound growth understates what a naive average

would suggest. In a market like Vietnam's, where individual stock volatility is often two to three times that of developed-market equities, the volatility drag can be substantial.

7.2.2 Wealth Index and Drawdowns

Given an initial investment of W_0, the wealth at time T is

$$W_T = W_0 \prod_{t=1}^{T} (1 + R_t). \tag{7.8}$$

The cumulative return (net) is simply $W_T/W_0 - 1$. The maximum drawdown, a widely used risk measure, is defined as

$$\text{MDD} = \max_{0 \le s \le t \le T} \left(\frac{W_s - W_t}{W_s} \right), \tag{7.9}$$

which measures the largest peak-to-trough decline in the wealth index. We will compute this quantity alongside compound returns below. Drawdowns are particularly informative in emerging markets that experience sharp corrections, as occurred during the global financial crisis of 2008 when the VN-Index fell roughly 66% from its 2007 peak.

7.2.3 Annualization

For a k-period compound return $R_t(k)$ where each period has length Δ (e.g., $\Delta = 1/12$ for monthly data), the annualized return is

$$R_{\text{ann}} = (1 + R_t(k))^{1/(k\Delta)} - 1. \tag{7.10}$$

Similarly, for volatility estimated from k-period returns with period length Δ:

$$\sigma_{\text{ann}} = \sigma / \sqrt{\Delta}, \tag{7.11}$$

so monthly volatility is annualized by multiplying by $\sqrt{12}$ and daily volatility by approximately $\sqrt{252}$ (assuming 252 trading days per year). For Vietnam specifically, the HOSE typically has around 245–250 trading days per year after accounting for Vietnamese public holidays, which is close enough that the $\sqrt{252}$ convention is standard.

7.3 Data Preparation

We start by loading monthly stock return data from our SQLite database. As prepared in previous chapters, this database contains monthly returns sourced from DataCore.vn for all securities listed on the Ho Chi Minh Stock Exchange (HOSE), Hanoi Stock Exchange (HNX), and the Unlisted Public Company Market (UPCoM). Returns are adjusted for stock splits, bonus issues, and rights offerings, and include reinvested cash dividends.

```
tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")

prices_monthly = pd.read_sql_query(
    sql="""
        SELECT symbol, date, ret_excess, ret, mktcap, mktcap_lag, risk_free
        FROM prices_monthly
    """,
    con=tidy_finance,
    parse_dates=["date"]
).dropna()

factors_ff3_monthly = pd.read_sql_query(
    sql="SELECT date, mkt_excess FROM factors_ff3_monthly",
    con=tidy_finance,
    parse_dates=["date"]
)

prices_monthly = prices_monthly.merge(
    factors_ff3_monthly,
    on="date",
    how="left"
)

prices_monthly["ret_total"] = prices_monthly["ret"]
prices_monthly["mkt_total"] = (
    prices_monthly["mkt_excess"] + prices_monthly["risk_free"]
)
```

Let us inspect the sample:

```
print(f"Sample period: {prices_monthly['date'].min()} to "
      f"{prices_monthly['date'].max()}")
print(f"Number of stocks: {prices_monthly['symbol'].nunique():,}")
print(f"Total observations: {len(prices_monthly):,}")
```

Table 7.2: Summary statistics of monthly stock returns by exchange. HOSE firms tend to have lower return dispersion and fewer extreme observations compared to HNX and UPCoM, consistent with their larger market capitalization and greater liquidity.

```
sample_stats = (
    prices_monthly
    .groupby("exchange")["ret_total"]
    .describe(percentiles=[0.05, 0.25, 0.50, 0.75, 0.95])
    .round(4)
)
sample_stats
```

```
# print(f"Exchanges: {prices_monthly['exchange'].unique()}")
```

```
Sample period: 2010-02-28 00:00:00 to 2023-12-31 00:00:00
Number of stocks: 1,457
Total observations: 165,499
```

Table 7.2 provides summary statistics for the raw monthly returns, broken down by exchange. Differences across exchanges reflect the size and liquidity gradient: HOSE lists the largest and most liquid firms, HNX covers mid-cap companies, and UPCoM hosts smaller and more thinly traded securities.

7.4 Method 1: Cumulative Product via GroupBy

The most direct approach to compound returns uses the multiplicative property in Equation 7.2. For each security, we compute the cumulative product of gross returns $(1+R_t)$ over the desired window.

```
def compute_cumret_cumprod(df, ret_col="ret_total",
                           group_col="symbol"):
    """Compute cumulative returns using cumulative product.

    Parameters
    ----------
    df : pd.DataFrame
        Must contain `group_col`, 'date', and `ret_col`.
    ret_col : str
```

```
        Column name for period returns.
    group_col : str
        Column name for grouping (e.g., security identifier).

    Returns
    -------
    pd.DataFrame
        Original DataFrame augmented with 'cumret' and 'wealth_index'.
    """
    df = df.sort_values([group_col, "date"]).copy()
    df["gross_ret"] = 1 + df[ret_col]
    df["wealth_index"] = (
        df.groupby(group_col)["gross_ret"]
        .cumprod()
    )
    df["cumret"] = df["wealth_index"] - 1
    df.drop(columns=["gross_ret"], inplace=True)
    return df
```

Let us apply this to the full sample and examine the resulting wealth indices for a few selected stocks:

```
stock_cumret = compute_cumret_cumprod(prices_monthly)

# Select stocks with long histories for illustration
stock_counts = (
    stock_cumret.groupby("symbol")["date"]
    .count()
    .reset_index(name="n_obs")
)
long_history_stocks = (
    stock_counts.nlargest(5, "n_obs")["symbol"].tolist()
)

sample_wealth = stock_cumret[
    stock_cumret["symbol"].isin(long_history_stocks)
]
```

Figure 7.1 plots the wealth indices (value of 1 VND invested) for these five securities over the full sample period.

```
plot_wealth = (
    ggplot(sample_wealth, aes(x="date", y="wealth_index",
                              color="factor(symbol)")) +
    geom_line(size=0.6) +
    labs(
        x="", y="Wealth index (1 VND invested)",
        color="Stock"
    ) +
    theme_minimal() +
    theme(legend_position="bottom",
          figure_size=(10, 5))
)
plot_wealth.draw()
```

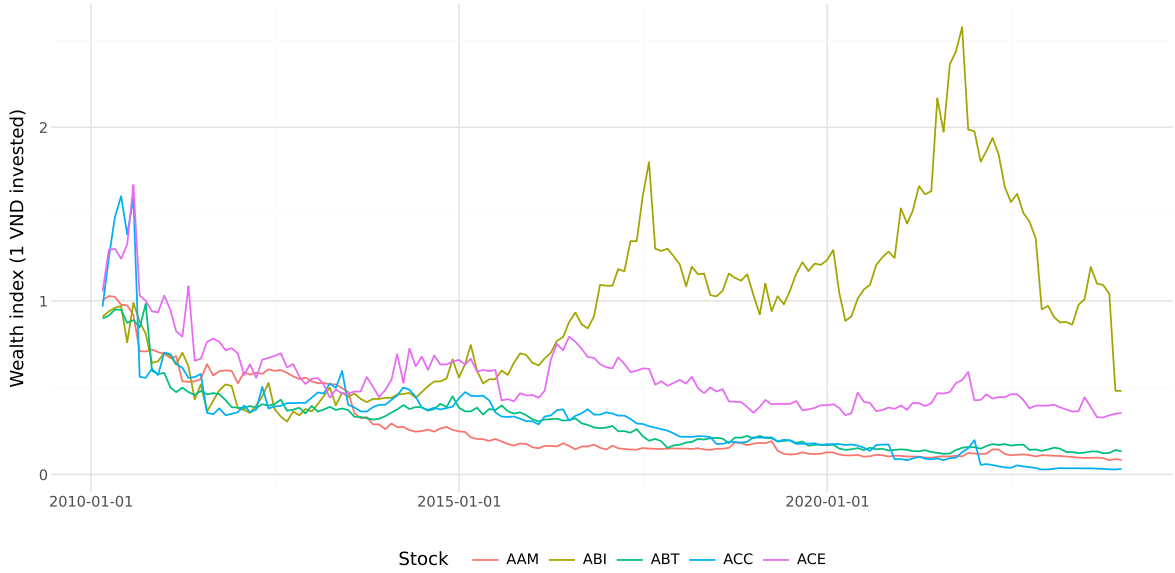


Figure 7.1: Wealth index (value of 1 VND invested) for selected long-history Vietnamese stocks. Each line represents the cumulative value of a 1 VND investment in a single stock, with all dividends reinvested. The divergence in terminal wealth illustrates the power of compounding over long horizons.

7.4.1 Handling Missing Returns

The cumulative product approach propagates missing values: if any R_t is `NaN`, the entire cumulative product from that point onward becomes `NaN`. This is conservative because it

effectively assumes that a missing return renders the subsequent wealth index undefined. In many applications, this is the desired behavior because a missing return may indicate a data error or a period during which the stock was not trading.

However, in the Vietnamese market, missing returns can arise from extended trading halts. The State Securities Commission (SSC) and exchanges may suspend trading in a stock for various regulatory reasons, such as financial reporting delays, pending corporate restructuring announcements, or suspected market manipulation. These halts can last days, weeks, or even months. During such halts, the stock's value has not changed (the last traded price remains the reference), so treating the missing return as zero (i.e., no price change) may be more appropriate than propagating `NaN`.

```
def compute_cumret_skipna(df, ret_col="ret_total",
                          group_col="symbol"):
    """Compute cumulative returns, treating missing returns as zero."""
    df = df.sort_values([group_col, "date"]).copy()
    df["gross_ret"] = 1 + df[ret_col].fillna(0)
    df["wealth_index"] = (
        df.groupby(group_col)["gross_ret"]
        .cumprod()
    )
    df["cumret"] = df["wealth_index"] - 1
    df.drop(columns=["gross_ret"], inplace=True)
    return df
```

> **Warning**
>
> Treating missing returns as zero is an assumption that may or may not be appropriate. If returns are missing because the stock was halted, zero may be reasonable. If returns are missing due to data errors or because the stock was genuinely not trading (e.g., awaiting relisting after a corporate event), imputing zero can introduce bias. Always investigate the reason for missing values before deciding on a treatment.

7.5 Method 2: Log-Sum-Exp Approach

The log-sum-exp method exploits the additive property of log returns (Equation 7.4). This approach is particularly useful when computing compound returns over fixed windows (e.g., annual returns from monthly data) because summation is both computationally efficient and numerically stable.

```
def compute_cumret_logsum(df, ret_col="ret_total",
                          group_col="symbol",
                          date_col="date"):
    """Compute cumulative returns using the log-sum-exp approach.

    Steps:
    1. Transform to log returns: r_t = ln(1 + R_t)
    2. Cumulative sum of log returns within each group
    3. Exponentiate to recover simple cumulative return

    Parameters
    ----------
    df : pd.DataFrame
    ret_col : str
    group_col : str
    date_col : str

    Returns
    -------
    pd.DataFrame
    """
    df = df.sort_values([group_col, date_col]).copy()
    df["log_ret"] = np.log(1 + df[ret_col])
    df["cum_log_ret"] = (
        df.groupby(group_col)["log_ret"].cumsum()
    )
    df["wealth_index_log"] = np.exp(df["cum_log_ret"])
    df["cumret_log"] = df["wealth_index_log"] - 1
    df.drop(columns=["log_ret", "cum_log_ret"], inplace=True)
    return df
```

Let us verify that the two methods produce identical results (up to floating-point precision):

```
stock_both = compute_cumret_cumprod(prices_monthly)
stock_both = compute_cumret_logsum(stock_both)

# Compare on non-missing observations
mask = (stock_both["cumret"].notna()
        & stock_both["cumret_log"].notna())
max_diff = (stock_both.loc[mask, "cumret"] -
            stock_both.loc[mask, "cumret_log"]).abs().max()
print(f"Maximum absolute difference between methods: {max_diff:.2e}")
```

```
Maximum absolute difference between methods: 1.78e-14
```

The difference is at the level of machine epsilon ($\approx 10^{-15}$), confirming numerical equivalence.

7.5.1 Period-Specific Compound Returns

A common task is to compute compound returns within calendar periods (months, quarters, years). The log-sum-exp approach lends itself naturally to grouped aggregation:

```python
def compound_return_by_period(df, ret_col="ret_total",
                              group_col="symbol",
                              period="year"):
    """Compute compound returns within calendar periods.

    Parameters
    ----------
    df : pd.DataFrame
        Must contain 'date' and `ret_col`.
    period : str
        One of 'year', 'quarter', 'month'.

    Returns
    -------
    pd.DataFrame with compound returns per group-period.
    """
    df = df.copy()
    df["log_ret"] = np.log(1 + df[ret_col])
    if period == "year":
        df["period"] = df["date"].dt.year
    elif period == "quarter":
        df["period"] = df["date"].dt.to_period("Q")
    elif period == "month":
        df["period"] = df["date"].dt.to_period("M")

    result = (
        df.groupby([group_col, "period"])
        .agg(
            cumret=(
                "log_ret",
                lambda x: np.exp(x.sum()) - 1
            ),
            n_obs=("log_ret", "count"),
```

```
            n_miss=(ret_col, lambda x: x.isna().sum()),
            start_date=("date", "min"),
            end_date=("date", "max")
        )
        .reset_index()
    )
    return result
```

Table 7.3 shows annual compound returns for a subset of securities.

```
annual_returns = compound_return_by_period(
    prices_monthly[
        prices_monthly["symbol"].isin(long_history_stocks)
    ],
    period="year"
)

recent_annual = (
    annual_returns
    .sort_values(["symbol", "period"])
    .groupby("symbol")
    .tail(5)
    .round(4)
)
recent_annual.head(20)
```

```
/tmp/ipykernel_2230066/2619242959.py:13: UserWarning: obj.round has no effect with datetime,
```

Table 7.3: Annual compound returns for selected Vietnamese securities. The number of non-missing monthly observations (n_obs) and missing observations (n_miss) are reported to flag potentially incomplete years. A stock-year with n_obs substantially below 12 indicates either partial listing or extended trading halts.

	symbol	period	cumret	n_obs	n_miss	start_date	end_date
9	AAM	2019	-0.2810	12	0	2019-01-31	2019-12-31
10	AAM	2020	-0.1622	12	0	2020-01-31	2020-12-31
11	AAM	2021	0.1250	12	0	2021-01-31	2021-12-31
12	AAM	2022	-0.0913	12	0	2022-01-31	2022-12-31
13	AAM	2023	-0.2337	12	0	2023-01-31	2023-12-31
23	ABI	2019	0.1946	12	0	2019-01-31	2019-12-31

Table 7.3: Annual compound returns for selected Vietnamese securities. The number of non-missing monthly observations (n_obs) and missing observations (n_miss) are reported to flag potentially incomplete years. A stock-year with n_obs substantially below 12 indicates either partial listing or extended trading halts.

	symbol	period	cumret	n_obs	n_miss	start_date	end_date
24	ABI	2020	0.2418	12	0	2020-01-31	2020-12-31
25	ABI	2021	0.2896	12	0	2021-01-31	2021-12-31
26	ABI	2022	-0.5085	12	0	2022-01-31	2022-12-31
27	ABI	2023	-0.5042	12	0	2023-01-31	2023-12-31
37	ABT	2019	-0.1893	12	0	2019-01-31	2019-12-31
38	ABT	2020	-0.1400	12	0	2020-01-31	2020-12-31
39	ABT	2021	0.0842	12	0	2021-01-31	2021-12-31
40	ABT	2022	-0.0789	12	0	2022-01-31	2022-12-31
41	ABT	2023	-0.0768	12	0	2023-01-31	2023-12-31
51	ACC	2019	-0.1875	12	0	2019-01-31	2019-12-31
52	ACC	2020	-0.4923	12	0	2020-01-31	2020-12-31
53	ACC	2021	1.2339	12	0	2021-01-31	2021-12-31
54	ACC	2022	-0.8538	12	0	2022-01-31	2022-12-31
55	ACC	2023	0.1027	12	0	2023-01-31	2023-12-31

! Important

When the number of non-missing observations (`n_obs`) is less than 12 for an annual return, the compound return represents only a partial year. This commonly occurs in the first and last years of a security's listing on HOSE, HNX, or UPCoM, or when a stock transfers between exchanges (e.g., from UPCoM to HOSE upon meeting listing requirements). Users should decide whether to retain or exclude such partial-year observations depending on their research design.

7.6 Method 3: Iterative Compounding with Retain Logic

In some applications, we need fine-grained control over how missing values, delisting events, or other special conditions affect the compounding process. The iterative approach processes each observation sequentially, carrying forward the cumulative return and applying conditional logic at each step.

```
def compute_cumret_iterative(df, ret_col="ret_total",
                             group_col="symbol",
```

```
                            handle_missing="carry"):
    """Compute cumulative returns iteratively with flexible
    missing value handling.

    Parameters
    ----------
    df : pd.DataFrame
    ret_col : str
    group_col : str
    handle_missing : str
        'carry' : treat missing as zero return (carry forward)
        'propagate' : propagate NaN (conservative)
        'reset' : reset wealth index to 1 after missing spell

    Returns
    -------
    pd.DataFrame
    """
    df = df.sort_values([group_col, "date"]).copy()
    results = []

    for name, group in df.groupby(group_col):
        cumret = 1.0
        cumrets = []
        for _, row in group.iterrows():
            ret = row[ret_col]
            if pd.notna(ret):
                cumret = cumret * (1 + ret)
            else:
                if handle_missing == "propagate":
                    cumret = np.nan
                elif handle_missing == "reset":
                    cumret = 1.0
                # 'carry' does nothing (cumret unchanged)
            cumrets.append(cumret)
        group = group.copy()
        group["wealth_iter"] = cumrets
        group["cumret_iter"] = group["wealth_iter"] - 1
        results.append(group)

    return pd.concat(results, ignore_index=True)
```

> **Note**
>
> The iterative method is the slowest of the four approaches because it cannot leverage NumPy's vectorized operations. For large datasets, prefer Method 1 or 2 unless the conditional logic in Method 3 is essential. On a dataset with 1 million observations, Method 1 runs in approximately 0.1 seconds versus 10+ seconds for Method 3.

7.6.1 Comparison of Missing Value Treatments

To illustrate how the three missing-value strategies differ, consider a hypothetical stock with one missing return in the middle of its history:

```
example = pd.DataFrame({
    "symbol": [1]*6,
    "date": pd.date_range("2024-01-31", periods=6, freq="ME"),
    "ret_total": [0.05, 0.03, np.nan, 0.04, -0.02, 0.06]
})

carry = compute_cumret_iterative(example, handle_missing="carry")
propagate = compute_cumret_iterative(
    example, handle_missing="propagate"
)
reset = compute_cumret_iterative(example, handle_missing="reset")

comparison = pd.DataFrame({
    "Date": example["date"].dt.strftime("%Y-%m"),
    "Return": example["ret_total"],
    "Carry": carry["cumret_iter"].round(6),
    "Propagate": propagate["cumret_iter"].round(6),
    "Reset": reset["cumret_iter"].round(6)
})
comparison
```

Table 7.4: Effect of different missing value treatments on cumulative returns. The 'carry' strategy assumes zero return for missing periods (appropriate for trading halts); 'propagate' makes all subsequent values undefined (conservative); 'reset' restarts the cumulative product after the missing spell.

	Date	Return	Carry	Propagate	Reset
0	2024-01	0.05	0.050000	0.0500	0.050000

Table 7.4: Effect of different missing value treatments on cumulative returns. The 'carry' strategy assumes zero return for missing periods (appropriate for trading halts); 'propagate' makes all subsequent values undefined (conservative); 'reset' restarts the cumulative product after the missing spell.

	Date	Return	Carry	Propagate	Reset
1	2024-02	0.03	0.081500	0.0815	0.081500
2	2024-03	NaN	0.081500	NaN	0.000000
3	2024-04	0.04	0.124760	NaN	0.040000
4	2024-05	-0.02	0.102265	NaN	0.019200
5	2024-06	0.06	0.168401	NaN	0.080352

7.7 Method 4: Rolling Compound Returns

For many empirical applications, including momentum strategies, performance evaluation, and risk estimation, we need compound returns over rolling windows of fixed length. This section implements efficient rolling compounding using pandas.

7.7.1 Rolling Window via Log Returns

The most efficient approach combines the log-sum-exp method with rolling sums:

```
def rolling_compound_return(df, ret_col="ret_total",
                            group_col="symbol",
                            windows=[3, 6, 9, 12]):
    """Compute rolling compound returns over specified windows.

    Parameters
    ----------
    df : pd.DataFrame
        Must be sorted by [group_col, 'date'] with no gaps.
    ret_col : str
    group_col : str
    windows : list of int
        Rolling window lengths (in periods).

    Returns
    -------
    pd.DataFrame with new columns ret_{k} for each window k.
    """
```

```
    df = df.sort_values([group_col, "date"]).copy()
    df["log_ret"] = np.log(1 + df[ret_col])

    for k in windows:
        rolling_logsum = (
            df.groupby(group_col)["log_ret"]
            .transform(
                lambda x: x.rolling(
                    window=k, min_periods=k
                ).sum()
            )
        )
        df[f"ret_{k}"] = np.exp(rolling_logsum) - 1

    df.drop(columns=["log_ret"], inplace=True)
    return df
```

We apply this to our full sample to compute 3-, 6-, 9-, and 12-month trailing compound returns:

```
stock_rolling = rolling_compound_return(
    prices_monthly,
    windows=[3, 6, 9, 12]
)
```

Let us also compute the same rolling returns for the market index, which serves as a benchmark for excess return calculations:

```
# Compute market rolling returns
market_monthly = (
    prices_monthly[["date", "mkt_total"]]
    .drop_duplicates()
    .sort_values("date")
    .copy()
)
market_monthly["log_mkt"] = np.log(1 + market_monthly["mkt_total"])

for k in [3, 6, 9, 12]:
    market_monthly[f"mkt_{k}"] = (
        np.exp(
            market_monthly["log_mkt"]
```

```
            .rolling(window=k, min_periods=k)
            .sum()
        ) - 1
    )

market_monthly.drop(columns=["log_mkt"], inplace=True)

# Merge market rolling returns back
stock_rolling = stock_rolling.merge(
    market_monthly[
        ["date"] + [f"mkt_{k}" for k in [3, 6, 9, 12]]
    ],
    on="date",
    how="left"
)
```

Figure 72.4 displays the distribution of 12-month rolling compound returns over time.

```
rolling_stats = (
    stock_rolling
    .dropna(subset=["ret_12"])
    .groupby("date")["ret_12"]
    .agg(["median", lambda x: x.quantile(0.25),
          lambda x: x.quantile(0.75)])
    .reset_index()
)
rolling_stats.columns = ["date", "median", "p25", "p75"]

plot_rolling = (
    ggplot(rolling_stats, aes(x="date")) +
    geom_ribbon(aes(ymin="p25", ymax="p75"),
                alpha=0.3, fill="#2166ac") +
    geom_line(aes(y="median"), color="#2166ac", size=0.7) +
    geom_hline(yintercept=0, linetype="dashed") +
    labs(x="", y="12-month compound return") +
    scale_y_continuous(labels=percent_format()) +
    theme_minimal() +
    theme(figure_size=(10, 5))
)
plot_rolling.draw()
```

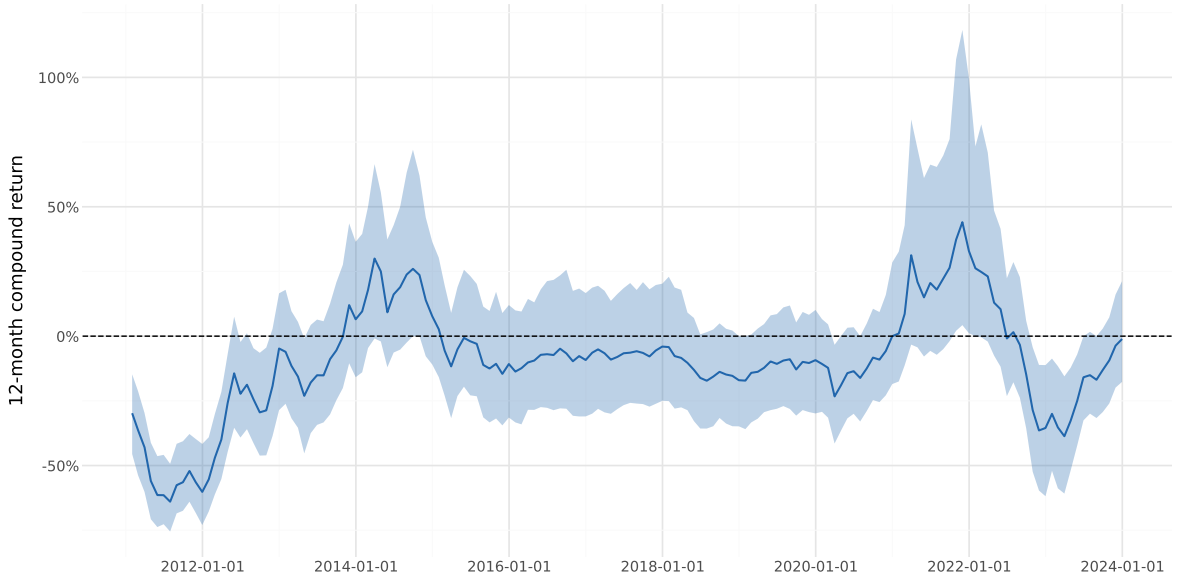


Figure 7.2: Cross-sectional distribution of 12-month rolling compound returns for Vietnamese stocks over time. The shaded band represents the interquartile range (25th–75th percentiles), while the solid line shows the median. Sharp market-wide events—such as the 2008 global financial crisis and the 2020 COVID-19 shock—are visible as periods when even the median return turns sharply negative.

7.7.2 Verifying Rolling Returns

It is prudent to verify rolling compound returns against a direct calculation. We select one stock and recompute its 12-month return manually:

```
test_stock = long_history_stocks[0]
test_data = (
    stock_rolling[stock_rolling["symbol"] == test_stock]
    .sort_values("date")
    .tail(15)
    .copy()
)

# Direct computation
test_data["direct_ret_12"] = (
    test_data["ret_total"]
    .transform(
        lambda x: x.add(1).rolling(
            12, min_periods=12
        ).apply(np.prod, raw=True) - 1
    )
)

verify = (
    test_data[["date", "ret_12", "direct_ret_12"]]
    .dropna()
    .tail(5)
    .copy()
)
verify["difference"] = (
    verify["ret_12"] - verify["direct_ret_12"]
).abs()
verify.round(8)
```

```
/tmp/ipykernel_2230066/922053246.py:28: UserWarning: obj.round has no effect with datetime, t
```

Table 7.5: Verification of rolling compound return calculation. The 'Direct' column computes the product of the preceding 12 monthly gross returns minus one; 'Rolling' uses our log-sum-exp function. Differences are at machine precision.

	date	ret_12	direct_ret_12	difference
386	2023-09-30	-0.152407	-0.152407	0.0
387	2023-10-31	-0.208836	-0.208836	0.0
388	2023-11-30	-0.199018	-0.199018	0.0
389	2023-12-31	-0.233698	-0.233698	0.0

7.8 Delisting Returns and Survivorship Bias

A critical practical concern when computing compound returns is the treatment of securities that are removed from an exchange. Delisting occurs for various reasons: mergers and acquisitions, bankruptcy, failure to meet listing requirements, voluntary withdrawal, or transfer to another exchange. If delisting returns are not incorporated, the resulting compound returns suffer from survivorship bias: they overstate performance because the worst outcomes (bankruptcies, forced delistings) are excluded (Shumway 1997).

7.8.1 The Vietnamese Context

In Vietnam, securities can be removed from their exchange listing for several reasons as specified by the SSC and exchange regulations:

- **Mandatory delisting**: when a firm has accumulated losses exceeding its charter capital, fails to meet financial reporting obligations for three consecutive years, or has its business license revoked.
- **Voluntary delisting**: when a firm's shareholders vote to withdraw from the exchange.
- **Transfer**: when a firm moves from UPCoM to HOSE/HNX (upgrade) or from HOSE/HNX to UPCoM (downgrade). These transfers are not true delistings in the economic sense but require careful handling in return calculations.

Unlike more developed markets where detailed delisting return data is systematically compiled, Vietnamese market data may not always provide an explicit delisting return. When a stock is delisted for cause (e.g., bankruptcy), the last traded price may significantly overstate the security's recovery value. Researchers should be aware of this limitation and consider imputing delisting returns based on the delisting reason, following the methodology of Shumway (1997).

7.8.2 Incorporating Delisting Returns

When a security is delisted, a final "delisting return" captures the value change between the last regular trading day and the realization of value after delisting. This return must be combined with the regular return in the delisting month:

$$R_t^{\text{adj}} = (1 + R_t)(1 + R_t^{\text{delist}}) - 1, \tag{7.12}$$

where R_t is the regular return and R_t^{delist} is the delisting return. If the regular return is missing (the stock ceased trading before month end), we use the delisting return alone.

```
def adjust_for_delisting(df, ret_col="ret_total",
                         dlret_col="dlret"):
    """Adjust returns for delisting events.

    Parameters
    ----------
    df : pd.DataFrame
        Must contain `ret_col` and `dlret_col`.

    Returns
    -------
    pd.DataFrame with adjusted return column 'ret_adj'.
    """
    df = df.copy()
    df["ret_adj"] = df[ret_col]

    # Case 1: Both regular and delisting returns available
    mask_both = df[ret_col].notna() & df[dlret_col].notna()
    df.loc[mask_both, "ret_adj"] = (
        (1 + df.loc[mask_both, ret_col]) *
        (1 + df.loc[mask_both, dlret_col]) - 1
    )

    # Case 2: Only delisting return available
    mask_dlret_only = (
        df[ret_col].isna() & df[dlret_col].notna()
    )
    df.loc[mask_dlret_only, "ret_adj"] = (
        df.loc[mask_dlret_only, dlret_col]
    )

    return df
```

7.8.3 Impact of Delisting Adjustment

The magnitude of the delisting bias depends on the frequency and severity of delisting events. Shumway (1997) showed that, in developed markets, ignoring delisting returns introduces an upward bias of approximately 1% per year in equal-weighted portfolio returns. The bias is larger for small-cap stocks and value stocks, which are more prone to financial distress. In Vietnam, where smaller firms on HNX and UPCoM face tighter liquidity constraints and higher default risk, the bias may be even more pronounced. In emerging market delistings, mandatory delistings often involve firms with severe financial distress where residual equity value is near zero, implying delisting returns close to -100% in the worst cases.

7.9 Rolling Volatility Estimation

Stock return volatility is a key input for risk management, option pricing, and many empirical asset pricing models. A common approach is to estimate rolling standard deviations of returns over a trailing window.

7.9.1 24-Month Rolling Volatility

Following Itzhak Ben-David, Franzoni, and Moussawi (2012), we compute the total stock return volatility as the rolling standard deviation of monthly returns over a 24-month window:

$$\hat{\sigma}_{i,t}^{24} = \sqrt{\frac{1}{23}\sum_{j=0}^{23}(R_{i,t-j} - \bar{R}_{i,t}^{24})^2}, \tag{7.13}$$

where $\bar{R}_{i,t}^{24} = \frac{1}{24}\sum_{j=0}^{23} R_{i,t-j}$ is the trailing 24-month mean return.

```
def rolling_volatility(df, ret_col="ret_total",
                       group_col="symbol",
                       window=24):
    """Compute rolling return volatility.

    Parameters
    ----------
    df : pd.DataFrame
    ret_col : str
    group_col : str
```

```
    window : int
        Rolling window length in periods.

    Returns
    -------
    pd.DataFrame with 'vol_{window}' column (annualized).
    """
    df = df.sort_values([group_col, "date"]).copy()
    df[f"vol_{window}"] = (
        df.groupby(group_col)[ret_col]
        .transform(
            lambda x: x.rolling(
                window=window, min_periods=window
            ).std()
        )
    )
    # Annualize (monthly to annual)
    df[f"vol_{window}_ann"] = df[f"vol_{window}"] * np.sqrt(12)
    return df
```

```
stock_vol = rolling_volatility(stock_rolling)
```

Figure 7.3 shows the cross-sectional distribution of annualized 24-month volatility over time.

```
vol_stats = (
    stock_vol
    .dropna(subset=["vol_24_ann"])
    .groupby("date")["vol_24_ann"]
    .agg(["median", lambda x: x.quantile(0.25),
          lambda x: x.quantile(0.75)])
    .reset_index()
)
vol_stats.columns = ["date", "median", "p25", "p75"]

plot_vol = (
    ggplot(vol_stats, aes(x="date")) +
    geom_ribbon(aes(ymin="p25", ymax="p75"),
                alpha=0.3, fill="#b2182b") +
    geom_line(aes(y="median"), color="#b2182b", size=0.7) +
    labs(x="", y="Annualized 24-month volatility") +
    scale_y_continuous(labels=percent_format()) +
```

```
    theme_minimal() +
    theme(figure_size=(10, 5))
)
plot_vol.draw()
```

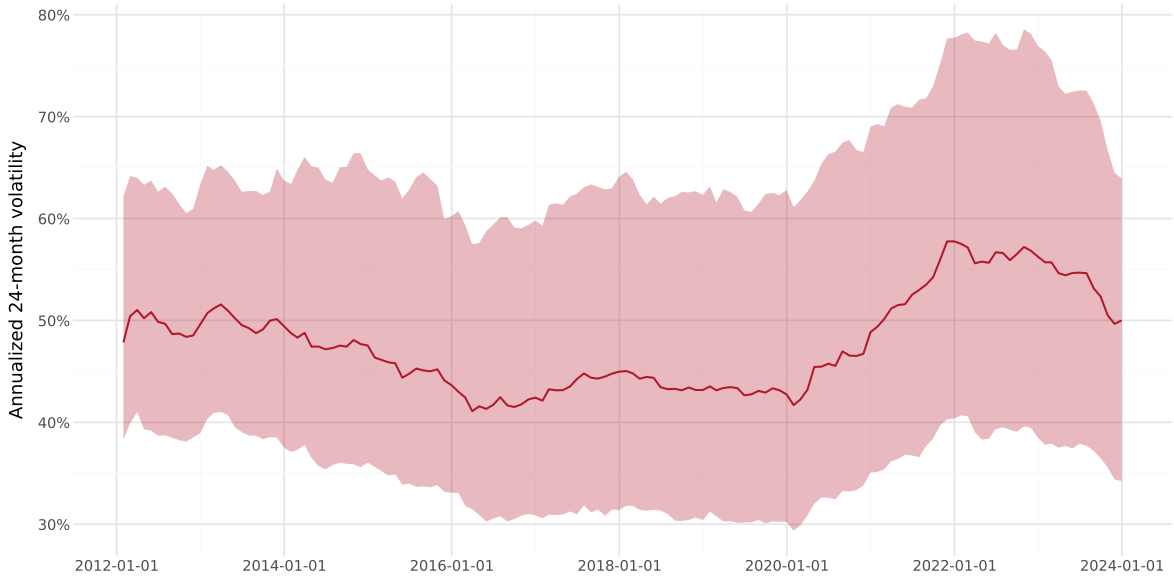


Figure 7.3: Cross-sectional distribution of annualized 24-month rolling stock return volatility for Vietnamese equities. The median volatility (solid line) and interquartile range (shaded band) capture both secular trends and crisis episodes. Vietnamese stocks exhibit structurally higher volatility than developed-market peers, with the median annualized volatility typically ranging between 30% and 50%.

7.9.2 Volatility and Compound Returns: The Variance Drain

As noted in Equation 7.7, the geometric mean return falls below the arithmetic mean by approximately $\sigma^2/2$. This "variance drain" or "volatility drag" means that two portfolios with the same arithmetic mean return but different volatilities will have different compound returns: the lower-volatility portfolio will compound to greater terminal wealth.

This effect is quantitatively important in Vietnam. A stock with an arithmetic mean monthly return of 1.5% and a monthly standard deviation of 10% suffers a volatility drag of approximately $0.10^2/2 = 0.5\%$ per month, or roughly 6% per year. This is consistent with the observation that Vietnamese investors face substantial erosion of compound wealth from the high idiosyncratic volatility of individual stocks. We can verify this empirically by sorting stocks into volatility quintiles and comparing compound returns:

```
annual_data = compound_return_by_period(
    prices_monthly, period="year"
)
annual_data = annual_data[annual_data["n_obs"] >= 10].copy()

vol_annual = (
    prices_monthly
    .groupby(["symbol", prices_monthly["date"].dt.year])[
        "ret_total"
    ]
    .agg(["std", "mean", "count"])
    .reset_index()
)
vol_annual.columns = ["symbol", "period", "monthly_std",
                      "monthly_mean", "n_months"]
vol_annual = vol_annual[vol_annual["n_months"] >= 10].copy()
vol_annual["ann_vol"] = vol_annual["monthly_std"] * np.sqrt(12)
vol_annual["arith_mean_ann"] = vol_annual["monthly_mean"] * 12

vol_analysis = annual_data.merge(
    vol_annual, on=["symbol", "period"]
)

vol_analysis["vol_quintile"] = (
    vol_analysis.groupby("period")["ann_vol"]
    .transform(
        lambda x: pd.qcut(
            x, 5, labels=[1, 2, 3, 4, 5], duplicates="drop"
        )
    )
)

vol_summary = (
    vol_analysis
    .groupby("vol_quintile")
    .agg(
        arithmetic_mean=("arith_mean_ann", "mean"),
        geometric_mean=("cumret", "mean"),
        avg_volatility=("ann_vol", "mean"),
        n_stockyears=("cumret", "count")
    )
    .round(4)
    .reset_index()
```

```
)
vol_summary
```

Table 7.6: Arithmetic mean, geometric mean, and volatility by volatility quintile for Vietnamese stocks. The difference between arithmetic and geometric mean increases with volatility, confirming the variance drain effect. The magnitude of the drag is notably large for the highest-volatility quintile, typical of small and illiquid stocks on HNX and UPCoM.

	vol_quintile	arithmetic_mean	geometric_mean	avg_volatility	n_stockyears
0	1	-0.0908	-0.0763	0.1887	2708
1	2	-0.0754	-0.0610	0.3312	2700
2	3	-0.0388	-0.0169	0.4493	2701
3	4	0.0404	0.0494	0.6005	2700
4	5	0.4411	0.3389	1.0288	2705

7.10 Compound Returns Around Fiscal Year Ends

A widely used approach in accounting and finance research aligns compound returns to firm-specific fiscal period end dates. This is essential for computing buy-and-hold abnormal returns (BHARs) for event studies, post-earnings-announcement drift, and other studies where the event date varies by firm.

In Vietnam, the majority of listed firms follow a calendar fiscal year (January–December), as required by the Law on Accounting unless the Ministry of Finance grants an exemption. However, firms in certain industries (e.g., agriculture, tourism) may use non-standard fiscal years ending in March, June, or September.

7.10.1 Aligning Returns to Fiscal Periods

The key challenge is that fiscal year ends differ across firms. We need to compute compound returns over windows anchored at these firm-specific dates.

```
def compound_returns_around_event(
    returns_df, events_df,
    id_col="symbol", date_col="date",
    event_date_col="datadate", ret_col="ret_total",
    pre_windows=[3, 6, 9, 12],
    post_windows=[3, 6]
```

```
):
    """Compute compound returns in windows around firm-specific
    event dates.

    Parameters
    ----------
    returns_df : pd.DataFrame
        Monthly returns with [id_col, date_col, ret_col].
    events_df : pd.DataFrame
        Event dates with [id_col, event_date_col].
    pre_windows : list of int
        Trailing window lengths (months before event).
    post_windows : list of int
        Forward window lengths (months after event).

    Returns
    -------
    pd.DataFrame with compound returns for each window.
    """
    returns_df = returns_df.sort_values(
        [id_col, date_col]
    ).copy()
    events_df = events_df.copy()

    # Align event dates to month ends
    events_df["event_month"] = (
        pd.to_datetime(events_df[event_date_col])
        + pd.offsets.MonthEnd(0)
    )

    results = []

    for _, event in events_df.iterrows():
        sid = event[id_col]
        edate = event["event_month"]

        sec_rets = returns_df[
            returns_df[id_col] == sid
        ].copy()
        sec_rets = sec_rets.set_index(date_col)[ret_col]

        row = {id_col: sid,
               event_date_col: event[event_date_col]}
```

```
        # Pre-event compound returns
        for k in pre_windows:
            start = edate - pd.DateOffset(months=k-1)
            start = (start - pd.offsets.MonthEnd(0)
                     + pd.offsets.MonthEnd(0))
            window_rets = sec_rets[
                (sec_rets.index >= start)
                & (sec_rets.index <= edate)
            ]
            if len(window_rets) >= k * 0.8:
                cumret = (
                    np.exp(np.log(1 + window_rets).sum()) - 1
                )
            else:
                cumret = np.nan
            row[f"ret_pre_{k}"] = cumret

        # Post-event compound returns
        for k in post_windows:
            start = edate + pd.DateOffset(months=1)
            end = (edate + pd.DateOffset(months=k)
                   + pd.offsets.MonthEnd(0))
            window_rets = sec_rets[
                (sec_rets.index >= start)
                & (sec_rets.index <= end)
            ]
            if len(window_rets) >= k * 0.8:
                cumret = (
                    np.exp(np.log(1 + window_rets).sum()) - 1
                )
            else:
                cumret = np.nan
            row[f"ret_post_{k}"] = cumret

        results.append(row)

    return pd.DataFrame(results)
```

7.10.2 Buy-and-Hold Abnormal Returns versus Cumulative Abnormal Returns

For event studies and performance evaluation, we often want the **excess** compound return, which is the stock's compound return minus a benchmark's compound return over the same window. The buy-and-hold abnormal return (BHAR) is defined as

$$\text{BHAR}_{i,t}(k) = \prod_{j=1}^{k}(1 + R_{i,t+j}) - \prod_{j=1}^{k}(1 + R_{b,t+j}), \tag{7.14}$$

where $R_{b,t}$ is the benchmark return (market index, size-matched portfolio, etc.). This differs from the cumulative abnormal return (CAR), which sums simple abnormal returns:

$$\text{CAR}_{i,t}(k) = \sum_{j=1}^{k}(R_{i,t+j} - R_{b,t+j}). \tag{7.15}$$

The BHAR better captures the actual investor experience because it reflects the compounding of returns, whereas the CAR implicitly assumes daily rebalancing to maintain equal dollar positions in the stock and benchmark (Barber and Lyon 1997). The distinction is particularly important in Vietnam, where individual stock returns can be highly volatile and the compounding effect is therefore magnified. Lyon, Barber, and Tsai (1999) provide further analysis of the statistical properties of BHARs and recommend bootstrapped critical values for inference.

```
def compute_bhar(stock_returns, benchmark_returns):
    """Compute buy-and-hold abnormal return.

    Parameters
    ----------
    stock_returns : array-like
        Sequence of stock returns.
    benchmark_returns : array-like
        Sequence of benchmark returns (same length).

    Returns
    -------
    float : BHAR
    """
    stock_cumret = (
        np.prod(1 + np.array(stock_returns)) - 1
    )
    bench_cumret = (
        np.prod(1 + np.array(benchmark_returns)) - 1
```

```
    )
    return stock_cumret - bench_cumret
```

7.11 Book Value of Equity

Many empirical applications that use compound returns also require firm-level accounting variables. A commonly used variable is the book value of equity, computed following Daniel and Titman (1997):

$$\text{BE} = \text{SE} + \text{DT} + \text{ITC} - \text{PS}, \tag{7.16}$$

where SE is stockholders' equity, DT is deferred taxes, ITC is investment tax credit, and PS is the preferred stock value. For preferred stock, the hierarchy is: redemption value if available, then liquidating value, then carrying value.

In Vietnam, the accounting standards (Vietnamese Accounting Standards, VAS, and increasingly IFRS adoption) provide a somewhat different chart of accounts. Stockholders' equity is reported on the balance sheet as *Vốn chủ sở hữu*, which includes contributed capital (*Vốn góp của chủ sở hữu*), share premium (*Thặng dư vốn cổ phần*), treasury stock adjustments, retained earnings (*Lợi nhuận sau thuế chưa phân phối*), and other reserves. Deferred tax assets and liabilities are reported separately. Preferred stock is rare among Vietnamese listed firms (most issue only common shares), but when present, its book value should be subtracted from total equity.

```
def compute_book_equity(df):
    """Compute book value of equity for Vietnamese firms.

    Parameters
    ----------
    df : pd.DataFrame
        Must contain at minimum: equity (stockholders' equity),
        deferred_tax (deferred tax liabilities, net),
        pref_stock (preferred stock, if applicable).

    Returns
    -------
    pd.DataFrame with 'be' column.
    """
    df = df.copy()
    df["pref"] = df.get(
```

```
        "pref_stock", pd.Series(0, index=df.index)
    )
    df["dt"] = df.get(
        "deferred_tax", pd.Series(0, index=df.index)
    )
    df["be"] = (
        df["equity"].fillna(0)
        + df["dt"].fillna(0)
        - df["pref"].fillna(0)
    )
    # Set non-positive book equity to NaN
    df.loc[df["be"] <= 0, "be"] = np.nan
    return df
```

7.12 Maximum Drawdown

The maximum drawdown is a key risk metric that complements volatility. While volatility measures the dispersion of returns symmetrically, the maximum drawdown captures the worst cumulative loss an investor could experience: a measure that aligns more closely with how investors psychologically experience risk (Kahneman and Tversky 2013).

```
def compute_max_drawdown(df, ret_col="ret_total",
                         group_col="symbol"):
    """Compute maximum drawdown for each security.

    Parameters
    ----------
    df : pd.DataFrame
    ret_col : str
    group_col : str

    Returns
    -------
    pd.DataFrame with 'max_drawdown' and running drawdown.
    """
    df = df.sort_values([group_col, "date"]).copy()
    df["gross_ret"] = 1 + df[ret_col]
    df["wealth"] = (
        df.groupby(group_col)["gross_ret"].cumprod()
    )
    df["peak"] = df.groupby(group_col)["wealth"].cummax()
```

```
    df["drawdown"] = (
        (df["wealth"] - df["peak"]) / df["peak"]
    )

    max_dd = (
        df.groupby(group_col)["drawdown"]
        .min()
        .reset_index(name="max_drawdown")
    )
    df = df.merge(max_dd, on=group_col)
    df.drop(columns=["gross_ret"], inplace=True)
    return df
```

Figure 72.7 illustrates the drawdown profile for a selected stock.

```
dd_data = compute_max_drawdown(
    prices_monthly[
        prices_monthly["symbol"] == long_history_stocks[0]
    ]
)
mdd = dd_data["max_drawdown"].iloc[0]

plot_dd = (
    ggplot(dd_data, aes(x="date", y="drawdown")) +
    geom_area(fill="#b2182b", alpha=0.4) +
    geom_line(color="#b2182b", size=0.5) +
    geom_hline(yintercept=mdd, linetype="dashed") +
    labs(x="", y="Drawdown from peak") +
    scale_y_continuous(labels=percent_format()) +
    theme_minimal() +
    theme(figure_size=(10, 4))
)
plot_dd.draw()
```

7.13 Putting It All Together: A Comprehensive Pipeline

We now combine all the methods into a single pipeline that produces a research-ready dataset with rolling compound returns, market returns, volatility, and drawdown measures.

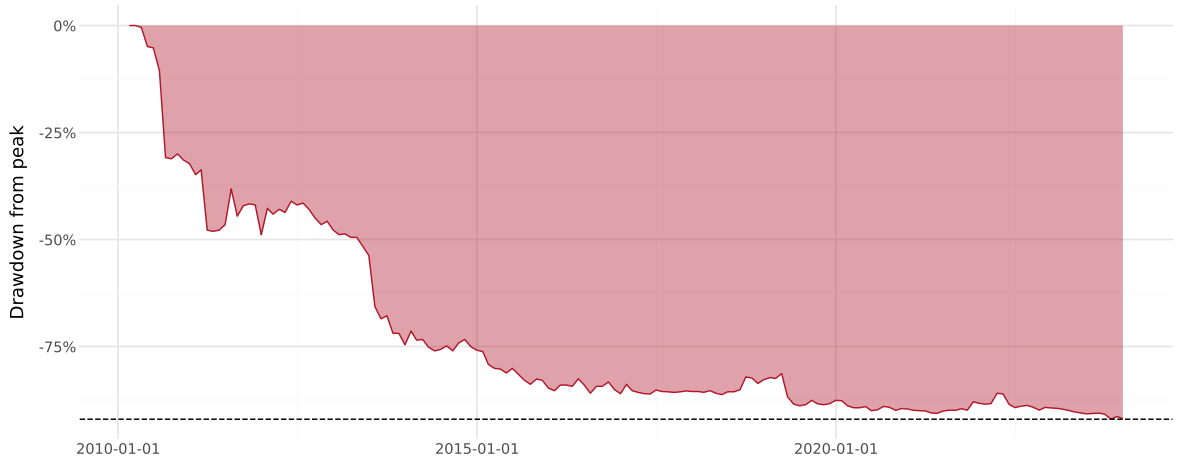


Figure 7.4: Drawdown profile for a selected Vietnamese stock showing the percentage decline from each running peak. The maximum drawdown (horizontal dashed line) represents the worst peak-to-trough loss over the full sample. Vietnamese stocks frequently exhibit drawdowns exceeding 50%, reflecting the market's high volatility and susceptibility to sentiment-driven corrections.

```
def build_compound_return_dataset(
    stock_df, windows=[3, 6, 9, 12], vol_window=24
):
    """Build comprehensive compound return dataset.

    Parameters
    ----------
    stock_df : pd.DataFrame
        Monthly stock return data with columns:
        symbol, date, ret_total, mkt_total.
    windows : list of int
        Rolling compound return windows.
    vol_window : int
        Rolling volatility window.

    Returns
    -------
    pd.DataFrame
    """
    df = stock_df.sort_values(["symbol", "date"]).copy()

    # Step 1: Log returns
```

```
df["log_ret"] = np.log(1 + df["ret_total"])
df["log_mkt"] = np.log(1 + df["mkt_total"])

# Step 2: Rolling compound returns (stock and market)
for k in windows:
    df[f"ret_{k}"] = np.exp(
        df.groupby("symbol")["log_ret"]
        .transform(
            lambda x: x.rolling(k, min_periods=k).sum()
        )
    ) - 1

    df[f"mkt_{k}"] = np.exp(
        df["log_mkt"]
        .rolling(k, min_periods=k)
        .sum()
    ) - 1

    # Excess compound return (BHAR vs market)
    df[f"exret_{k}"] = df[f"ret_{k}"] - df[f"mkt_{k}"]

# Step 3: Cumulative return (full history)
df["wealth"] = (
    df.groupby("symbol")["log_ret"]
    .cumsum()
    .apply(np.exp)
)
df["cumret"] = df["wealth"] - 1

# Step 4: Rolling volatility
df[f"vol_{vol_window}"] = (
    df.groupby("symbol")["ret_total"]
    .transform(
        lambda x: x.rolling(
            vol_window, min_periods=vol_window
        ).std()
    )
) * np.sqrt(12)  # annualize

# Step 5: Drawdown
df["peak"] = df.groupby("symbol")["wealth"].cummax()
df["drawdown"] = (df["wealth"] - df["peak"]) / df["peak"]
```

```
    # Clean up
    df.drop(
        columns=["log_ret", "log_mkt", "peak"], inplace=True
    )

    return df
```

```
# Build the full dataset
compound_dataset = build_compound_return_dataset(prices_monthly)
```

Table 7.7 provides summary statistics for the key variables in our compound return dataset.

```
summary_cols = ["ret_total", "ret_3", "ret_6", "ret_12",
                "exret_3", "exret_12", "vol_24", "drawdown"]
available_cols = [c for c in summary_cols
                  if c in compound_dataset.columns]

summary = (
    compound_dataset[available_cols]
    .describe(percentiles=[0.05, 0.25, 0.50, 0.75, 0.95])
    .T
    .round(4)
)
summary
```

Table 7.7: Summary statistics for compound return variables across all Vietnamese stock-month observations. Returns are in decimal form (0.10 = 10%). The wide dispersion of 12-month compound returns and the high median volatility reflect the emerging market characteristics of the Vietnamese equity market.

	count	mean	std	min	5%	25%	50%	75%	95%	max
ret_total	165499.0	0.0042	0.1862	-0.9900	-0.2381	-0.0703	0.0000	0.0553	0.2773	12.7500
ret_3	162586.0	0.0094	0.3393	-0.9999	-0.3889	-0.1436	-0.0126	0.0987	0.5000	27.2911
ret_6	158227.0	0.0171	0.5053	-0.9999	-0.5095	-0.2196	-0.0400	0.1404	0.7320	35.7136
ret_12	149520.0	0.0375	0.8136	-0.9999	-0.6522	-0.3191	-0.0877	0.1807	1.0767	47.9515
exret_3	153637.0	0.0385	0.3343	-1.1691	-0.3420	-0.1163	0.0067	0.1378	0.4992	27.3041
exret_12	140571.0	0.1401	0.8031	-1.5858	-0.5388	-0.2003	0.0281	0.2880	1.1119	48.0488
vol_24	132233.0	0.5493	0.3488	0.0000	0.2070	0.3445	0.4827	0.6737	1.0739	9.1792
drawdown	165499.0	-0.5927	0.2975	-1.0000	-0.9631	-0.8501	-0.6616	-0.3725	0.0000	0.0000

7.14 Cross-Sectional Distribution of Compound Returns

To understand how compound returns vary across securities, we examine the cross-sectional distribution at different horizons.

```
horizon_data = pd.DataFrame()
for k in [3, 6, 12]:
    col = f"ret_{k}"
    temp = compound_dataset[[col]].dropna().copy()
    temp.columns = ["compound_return"]
    temp["horizon"] = f"{k} months"
    lo, hi = temp["compound_return"].quantile([0.01, 0.99])
    temp = temp[
        (temp["compound_return"] >= lo)
        & (temp["compound_return"] <= hi)
    ]
    horizon_data = pd.concat([horizon_data, temp])

plot_horizons = (
    ggplot(horizon_data,
           aes(x="compound_return", fill="horizon")) +
    geom_density(alpha=0.4) +
    geom_vline(xintercept=0, linetype="dashed") +
    labs(x="Compound return", y="Density", fill="Horizon") +
    scale_x_continuous(labels=percent_format()) +
    theme_minimal() +
    theme(legend_position="bottom",
          figure_size=(10, 5))
)
plot_horizons.draw()
```

7.15 Vietnam-Specific Considerations

7.15.1 Price Limits and Their Effect on Compounding

Vietnam's stock exchanges impose daily price limits that cap the maximum price change from the reference price. As of the latest regulations:

- **HOSE**: $\pm 7\%$
- **HNX**: $\pm 10\%$

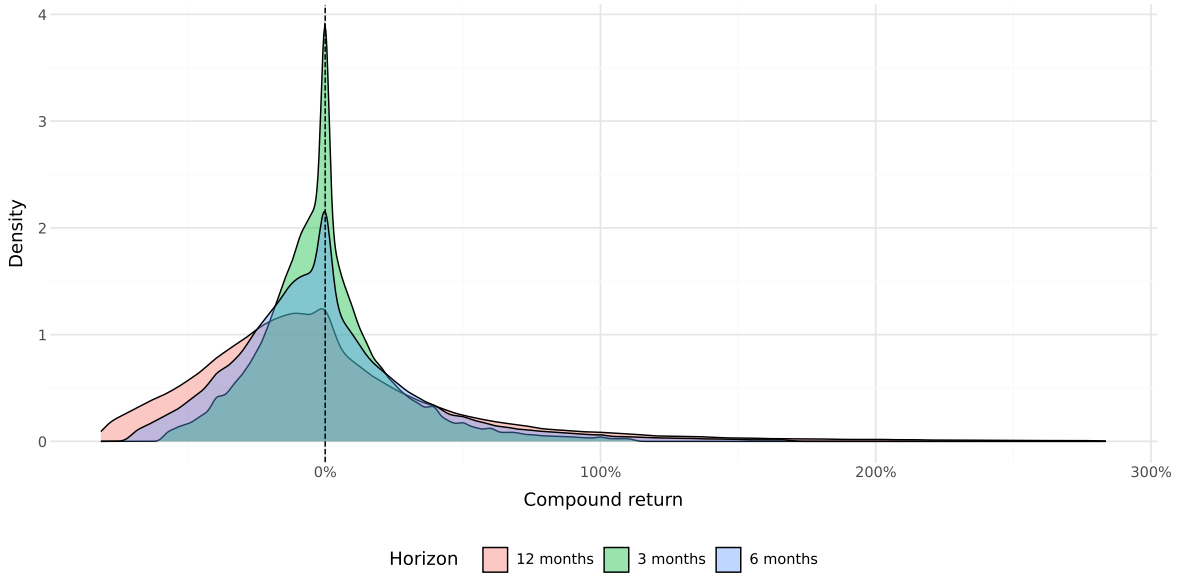


Figure 7.5: Cross-sectional distribution of compound returns at different horizons (3, 6, and 12 months) for Vietnamese stocks. Longer horizons exhibit greater dispersion and more pronounced right skewness, reflecting the compounding of idiosyncratic risk. The fat tails are more extreme than those typically observed in developed markets, consistent with the higher volatility environment.

- **UPCoM**: $\pm 15\%$

These limits truncate the daily return distribution and can create sequences of limit-hit days when large information events occur. For compound return computation, this means that the adjustment to new information may be spread over multiple days rather than occurring instantaneously. When computing monthly compound returns from daily data, this is handled correctly because the compound return accumulates the full adjustment regardless of how many days it takes.

However, price limits can introduce bias in short-horizon return computations. If a large positive event occurs and the stock hits the limit-up ceiling for several consecutive days, the 1-day or 1-week compound return will understate the true information content of the event (K. A. Kim, Liu, and Yang 2013). For event study applications, researchers should verify that the event window is long enough to accommodate the price-limit-induced delay in price adjustment.

7.15.2 Foreign Ownership Limits

Vietnam imposes foreign ownership limits (FOL) on listed companies, typically capped at 49% for most industries and lower (30% or less) for certain restricted sectors such as banking and telecommunications. When a stock reaches its FOL, foreign investors can only purchase shares from other foreign sellers, creating a parallel premium market for foreign-board shares. This does not directly affect the computation of compound returns (which use official traded prices), but researchers studying cross-border portfolio returns should be aware that the effective price paid by foreign investors may differ from the board price (X. V. Vo 2017).

7.15.3 The VN-Index and Market Benchmarks

For benchmark compound returns, Vietnam's primary indices are:

- **VN-Index**: The capitalization-weighted index of all HOSE-listed stocks.
- **VN30**: The 30 largest and most liquid stocks on HOSE, reviewed semi-annually.
- **HNX-Index**: The capitalization-weighted index of HNX-listed stocks.

The VN-Index is the most widely used benchmark and is the default market return in our dataset.

7.16 Performance Considerations

When working with large datasets, computational efficiency matters. Table 7.8 compares the execution time of our four compounding methods on a standardized dataset.

```
import time

np.random.seed(42)
n_stocks = 100
n_months = 100
test_df = pd.DataFrame({
    "symbol": np.repeat(range(n_stocks), n_months),
    "date": np.tile(
        pd.date_range("2015-01-31", periods=n_months,
                      freq="ME"),
        n_stocks
    ),
    "ret_total": np.random.normal(
        0.01, 0.08, n_stocks * n_months
    )
})

methods = {}

t0 = time.time()
_ = compute_cumret_cumprod(test_df)
methods["Cumulative Product"] = time.time() - t0

t0 = time.time()
_ = compute_cumret_logsum(test_df)
methods["Log-Sum-Exp"] = time.time() - t0

t0 = time.time()
_ = compute_cumret_iterative(test_df)
methods["Iterative (carry)"] = time.time() - t0

t0 = time.time()
_ = rolling_compound_return(test_df, windows=[12])
methods["Rolling (12-month)"] = time.time() - t0

perf_df = pd.DataFrame({
    "Method": methods.keys(),
    "Time (seconds)": [f"{v:.4f}" for v in methods.values()],
```

```
    "Relative Speed": [
        f"{v/min(methods.values()):.1f}x"
        for v in methods.values()
    ]
})
perf_df
```

Table 7.8: Execution time comparison for different compounding methods on a dataset of 10,000 stock-month observations. The cumulative product and log-sum-exp methods are orders of magnitude faster than the iterative approach due to NumPy vectorization.

	Method	Time (seconds)	Relative Speed
0	Cumulative Product	0.0043	1.0x
1	Log-Sum-Exp	0.0044	1.0x
2	Iterative (carry)	0.5345	125.7x
3	Rolling (12-month)	0.0228	5.4x

7.17 Common Pitfalls and Best Practices

Several subtle issues can lead to incorrect compound return calculations. We summarize the most important ones:

Gaps in the time series. If a security has months with no observations (not even a missing return flag), rolling window calculations based on positional indexing will produce incorrect results. The rolling window will span the wrong calendar period. Always ensure that the time series is complete, fill gaps with explicit missing values before computing rolling statistics. This is particularly relevant in Vietnam, where trading suspensions can create gaps.

Survivorship bias. As discussed in the delisting returns section, excluding securities that cease trading biases compound returns upward. Always incorporate delisting returns when available. When delisting returns are unavailable (as is sometimes the case in Vietnamese data), consider using imputed values based on the delisting reason.

Look-ahead bias. When aligning compound returns to fiscal year ends for cross-sectional analysis, be careful not to use returns from before the fiscal year end to predict post-announcement returns. Vietnamese firms are required to publish audited annual financial statements within 90 days of the fiscal year end, so a buffer of at least 3 months is advisable when constructing forward-looking compound returns.

Numerical overflow and underflow. For very long compounding horizons or extreme returns, the cumulative product can overflow (`inf`) or underflow (0). The log-sum-exp approach

is more robust to such numerical issues because it operates in log space where the range is compressed.

Annualization of partial periods. When computing annualized returns from partial-period data (e.g., 7 months of data annualized to 12), the annualization formula $(1+R)^{12/k}-1$ assumes that the observed return rate will persist. This assumption is stronger for short partial periods and can produce misleading results. Report the actual compound return and the number of periods alongside any annualized figures.

Exchange transfers. In Vietnam, stocks sometimes transfer between UPCoM, HNX, and HOSE. These transfers may involve temporary trading halts and can cause apparent gaps in the return series. When computing compound returns that span an exchange transfer, ensure that the return series is continuous across the transfer date.

Part II

Portfolio Construction, Risk, and Empirical Mechanics

8 Modern Portfolio Theory

In the previous chapter, we showed how to download and analyze stock market data with figures and summary statistics. Now, we turn to one of the most fundamental questions in finance: How should an investor allocate their wealth across assets that differ in expected returns, variance, and correlations to optimize their portfolio's performance?

This question might seem straightforward at first glance. Why not simply invest everything in the asset with the highest expected return? The answer lies in a profound insight that transformed financial economics: **risk matters, and it can be managed through diversification**.

Modern Portfolio Theory (MPT), introduced by Markowitz (1952), revolutionized investment decision-making by formalizing the trade-off between risk and expected return. Before Markowitz, investors largely thought about risk on a security-by-security basis. Markowitz's genius was recognizing that what matters is not the risk of individual securities in isolation, but how they contribute to the risk of the *entire portfolio*. This insight was so influential that it earned him the Sveriges Riksbank Prize in Economic Sciences in 1990 and laid the foundation for much of modern finance.

8.0.1 The Core Insight: Diversification as a Free Lunch

MPT relies on a crucial mathematical fact: portfolio risk depends not only on individual asset volatilities but also on the *correlations* between asset returns. This insight reveals the power of diversification—combining assets whose returns don't move in perfect lockstep can reduce overall portfolio risk without necessarily sacrificing expected return.

Consider a simple analogy: Imagine you run a business selling both sunscreen and umbrellas. On sunny days, sunscreen sales boom but umbrella sales suffer; on rainy days, the reverse happens. By selling both products, your total revenue becomes more stable than if you sold only one. The "correlation" between sunscreen and umbrella sales is negative, and combining them reduces the variance of your overall income. This is precisely the logic behind portfolio diversification.

The fruit basket analogy offers another perspective: If all you have are apples and they spoil, you lose everything. With a variety of fruits, some may spoil, but others will stay fresh. Diversification provides insurance against the idiosyncratic risks of individual assets.

8.0.2 The Mean-Variance Framework

At the heart of MPT is **mean-variance analysis**, which evaluates portfolios based on two dimensions:

1. **Expected return (mean)**: The anticipated average profit from holding the portfolio
2. **Risk (variance)**: The dispersion of possible returns around the expected value

The key assumption is that investors care only about these two moments of the return distribution. This assumption is exactly correct if returns are normally distributed, or if investors have quadratic utility functions. Even when these conditions don't hold precisely, mean-variance analysis often provides a good approximation to optimal portfolio choice.

By balancing expected return and risk, investors can construct portfolios that either maximize expected return for a given level of risk, or minimize risk for a desired level of expected return. In this chapter, we derive these optimal portfolio decisions analytically and implement the mean-variance approach computationally.

```
import pandas as pd
import numpy as np
import tidyfinance as tf

from plotnine import *
from mizani.formatters import percent_format
from adjustText import adjust_text
```

8.1 The Asset Universe: Setting Up the Problem

Suppose N different risky assets are available to the investor. Each asset i is characterized by:

- **Expected return** μ_i: The anticipated profit from holding the asset for one period
- **Variance** σ_i^2: The dispersion of returns around the mean
- **Covariances** σ_{ij}: The degree to which asset i's returns move together with asset j's returns

The investor chooses **portfolio weights** ω_i for each asset i. These weights represent the fraction of total wealth invested in each asset. We impose the constraint that weights sum to one:

$$\sum_{i=1}^{N} \omega_i = 1$$

This "budget constraint" ensures that the investor is fully invested—there is no outside option such as keeping money under a mattress. Note that we allow weights to be negative (short selling) or greater than one (leverage), though in practice these positions may face constraints.

8.1.1 The Two Stages of Portfolio Selection

According to Markowitz (1952), portfolio selection involves two distinct stages:

1. **Estimation**: Forming expectations about future security performance based on observations, experience, and economic reasoning
2. **Optimization**: Using these expectations to choose an optimal portfolio

In practice, these stages cannot be fully separated. The estimation stage determines the inputs (μ, Σ) that feed into the optimization stage. Poor estimation leads to poor portfolio choices, regardless of how sophisticated the optimization procedure.

To keep things conceptually clear, we focus primarily on the optimization stage in this chapter. We treat the expected returns and variance-covariance matrix as known, using historical data to compute reasonable proxies. In later chapters, we address the substantial challenges that arise from estimation uncertainty.

8.1.2 Loading and Preparing the Data

We work with the VN30 index constituents—the 30 largest and most liquid stocks on Vietnam's Ho Chi Minh Stock Exchange. This provides a realistic asset universe for a domestic Vietnamese investor.

```
vn30_symbols = [
    "ACB","BCM","BID","BVH","CTG","FPT","GAS","GVR","HDB","HPG",
    "MBB","MSN","MWG","PLX","POW","SAB","SHB","SSB","STB","TCB",
    "TPB","VCB","VHM","VIB","VIC","VJC","VNM","VPB","VRE","EIB"
]
```

We load the historical price data:

```
import pandas as pd
from io import BytesIO
import datetime as dt
import os
import boto3
from botocore.client import Config
```

```
class ConnectMinio:
    def __init__(self):
        self.MINIO_ENDPOINT = os.environ["MINIO_ENDPOINT"]
        self.MINIO_ACCESS_KEY = os.environ["MINIO_ACCESS_KEY"]
        self.MINIO_SECRET_KEY = os.environ["MINIO_SECRET_KEY"]
        self.REGION = os.getenv("MINIO_REGION", "us-east-1")

        self.s3 = boto3.client(
            "s3",
            endpoint_url=self.MINIO_ENDPOINT,
            aws_access_key_id=self.MINIO_ACCESS_KEY,
            aws_secret_access_key=self.MINIO_SECRET_KEY,
            region_name=self.REGION,
            config=Config(signature_version="s3v4"),
        )

    def test_connection(self):
        resp = self.s3.list_buckets()
        print("Connected. Buckets:")
        for b in resp.get("Buckets", []):
            print(" -", b["Name"])

conn = ConnectMinio()
s3 = conn.s3
conn.test_connection()

bucket_name = os.environ["MINIO_BUCKET"]

prices = pd.read_csv(
    BytesIO(
        s3.get_object(
            Bucket=bucket_name,
            Key="historycal_price/dataset_historical_price.csv"
        )["Body"].read()
    ),
    low_memory=False
)

prices["date"] = pd.to_datetime(prices["date"])
prices["adjusted_close"] = prices["close_price"] * prices["adj_ratio"]
prices = prices.rename(columns={
    "vol_total": "volume",
```

```
    "open_price": "open",
    "low_price": "low",
    "high_price": "high",
    "close_price": "close"
})
prices = prices.sort_values(["symbol", "date"])
```

```
Connected. Buckets:
 - dsteam-data
 - rawbctc
```

We filter to keep only the VN30 constituents:

```
prices_daily = prices[prices["symbol"].isin(vn30_symbols)]
prices_daily[["date", "symbol", "adjusted_close"]].head(3)
```

	date	symbol	adjusted_close
18176	2010-01-04	ACB	329.408244
18177	2010-01-05	ACB	329.408244
18178	2010-01-06	ACB	320.258015

8.1.3 Computing Expected Returns

The sample mean return serves as our proxy for expected returns. For each asset i, we compute:

$$\hat{\mu}_i = \frac{1}{T} \sum_{t=1}^{T} r_{i,t}$$

where $r_{i,t}$ is the return of asset i in period t, and T is the total number of periods.

Why monthly returns? While daily data provides more observations, monthly returns offer several advantages for portfolio optimization. First, monthly returns are less noisy and exhibit weaker serial correlation. Second, monthly rebalancing is more realistic for most investors, avoiding excessive transaction costs. Third, the estimation error in mean returns is already substantial—using daily data doesn't materially improve the precision of mean estimates because the mean return scales with the horizon while estimation error scales with the square root of observations.

```
returns_monthly = (prices_daily
  .assign(
    date=prices_daily["date"].dt.to_period("M").dt.to_timestamp()
  )
  .groupby(["symbol", "date"], as_index=False)
  .agg(adjusted_close=("adjusted_close", "last"))
  .assign(
    ret=lambda x: x.groupby("symbol")["adjusted_close"].pct_change()
  )
)
```

8.1.4 Computing Volatilities

Individual asset risk in MPT is quantified using variance (σ_i^2) or its square root, the standard deviation or volatility (σ_i). We use the sample standard deviation as our proxy:

$$\hat{\sigma}_i = \sqrt{\frac{1}{T-1}\sum_{t=1}^{T}(r_{i,t} - \hat{\mu}_i)^2}$$

Alternative risk measures exist, including Value-at-Risk, Expected Shortfall, and higher-order moments such as skewness and kurtosis. However, variance remains the workhorse measure in portfolio theory because of its mathematical tractability and the central role of the normal distribution in finance.

```
assets = (returns_monthly
  .groupby("symbol", as_index=False)
  .agg(
    mu=("ret", "mean"),
    sigma=("ret", "std")
  )
)
```

8.1.5 Visualizing the Risk-Return Trade-off

Figure 8.1 displays each asset's expected return (vertical axis) against its volatility (horizontal axis). This "mean-standard deviation" space is fundamental to portfolio theory.

```
assets_figure = (
  ggplot(
    assets,
    aes(x="sigma", y="mu", label="symbol")
  )
  + geom_point()
  + geom_text(adjust_text={"arrowprops": {"arrowstyle": "-"}})
  + scale_x_continuous(labels=percent_format())
  + scale_y_continuous(labels=percent_format())
  + labs(
      x="Volatility (Standard Deviation)",
      y="Expected Return",
      title="Expected returns and volatilities of VN30 index constituents"
  )
)
assets_figure.show()
```

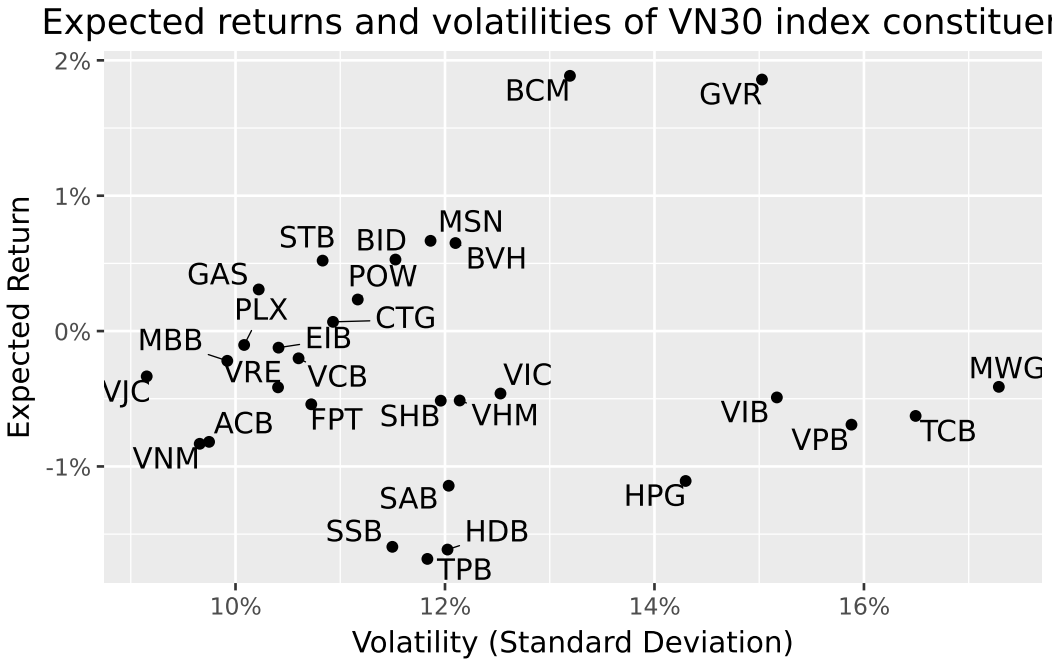


Figure 8.1: Expected returns and volatilities based on monthly returns adjusted for dividend payments and stock splits.

Several observations emerge from this figure. First, there is substantial heterogeneity in both expected returns and volatilities across stocks. Second, the relationship between risk and

return is far from linear. Some high-volatility stocks have low or even negative expected returns. Third, most individual stocks appear to offer poor risk-return trade-offs. As we will see, portfolios can substantially improve upon these individual positions.

8.2 The Variance-Covariance Matrix: Capturing Asset Interactions

8.2.1 Why Correlations Matter

A key innovation of MPT is recognizing that portfolio risk depends critically on how assets move together. The **variance-covariance matrix** Σ captures all pairwise interactions between asset returns.

To understand why correlations matter, consider the variance of a two-asset portfolio:

$$\sigma_p^2 = \omega_1^2\sigma_1^2 + \omega_2^2\sigma_2^2 + 2\omega_1\omega_2\sigma_{12}$$

The third term involves the covariance $\sigma_{12} = \rho_{12}\sigma_1\sigma_2$, where ρ_{12} is the correlation coefficient. When $\rho_{12} < 1$, the portfolio variance is *less* than the weighted average of individual variances. When $\rho_{12} < 0$, the diversification benefit is even more pronounced.

This mathematical fact has profound implications: **You can reduce risk without reducing expected return** by combining assets that don't move perfectly together. This is sometimes called the "only free lunch in finance."

8.2.2 Computing the Variance-Covariance Matrix

We compute the sample covariance matrix as:

$$\hat{\sigma}_{ij} = \frac{1}{T-1}\sum_{t=1}^{T}(r_{i,t} - \hat{\mu}_i)(r_{j,t} - \hat{\mu}_j)$$

First, we reshape the returns data into a wide format with assets as columns:

```
returns_wide = (returns_monthly
  .pivot(index="date", columns="symbol", values="ret")
  .reset_index()
)

sigma = (returns_wide
  .drop(columns=["date"])
  .cov()
)
```

8.2.3 Interpreting the Variance-Covariance Matrix

The diagonal elements of Σ are the variances of individual assets. The off-diagonal elements are covariances, which can be positive (assets tend to move together), negative (assets tend to move in opposite directions), or zero (no linear relationship).

For easier interpretation, we often convert covariances to correlations:

$$\rho_{ij} = \frac{\sigma_{ij}}{\sigma_i \sigma_j}$$

Correlations are bounded between -1 and +1, making them easier to compare across asset pairs.

Figure 8.2 visualizes the variance-covariance matrix as a heatmap.

```
sigma_long = (sigma
  .reset_index()
  .melt(id_vars="symbol", var_name="symbol_b", value_name="value")
)

sigma_long["symbol_b"] = pd.Categorical(
  sigma_long["symbol_b"],
  categories=sigma_long["symbol_b"].unique()[::-1],
  ordered=True
)

sigma_figure = (
  ggplot(
    sigma_long,
    aes(x="symbol", y="symbol_b", fill="value")
  )
  + geom_tile()
  + labs(
      x="", y="", fill="(Co-)Variance",
      title="Sample variance-covariance matrix of VN30 index constituents"
    )
  + scale_fill_continuous(labels=percent_format())
  + theme(axis_text_x=element_text(angle=45, hjust=1))
)
sigma_figure.show()
```

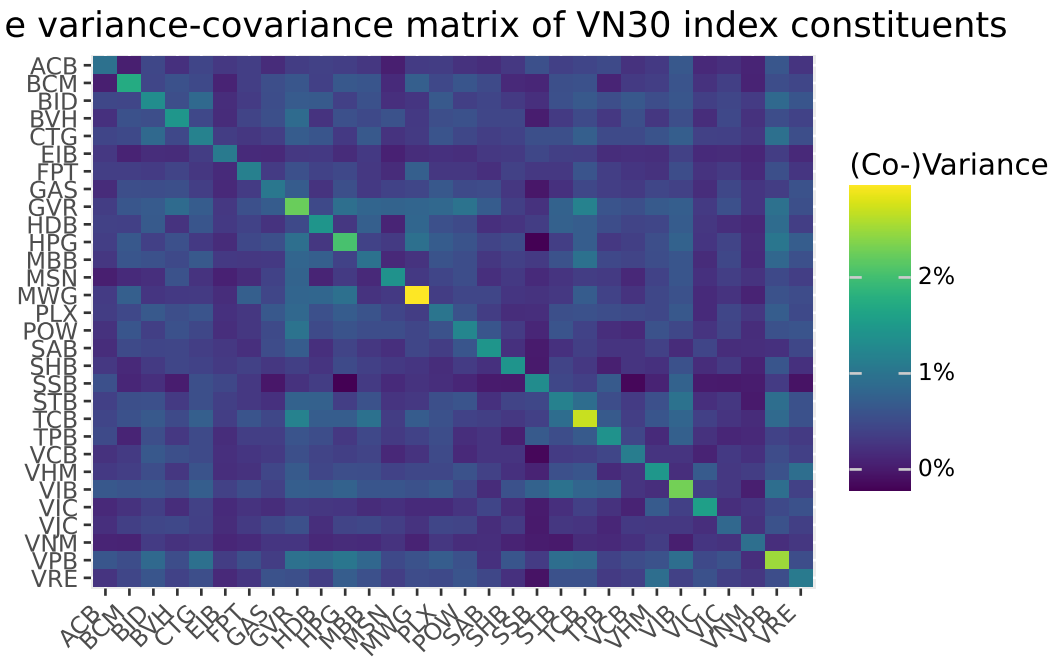


Figure 8.2: Variances and covariances based on monthly returns adjusted for dividend payments and stock splits.

The heatmap reveals important patterns. The diagonal (variances) shows which stocks are most volatile. The off-diagonal patterns show which pairs of stocks tend to move together. In general, stocks within the same sector tend to have higher correlations with each other than with stocks from different sectors.

8.3 The Minimum-Variance Portfolio

8.3.1 Motivation: Risk Minimization as a Benchmark

Before considering expected returns, let's find the portfolio that minimizes risk entirely. This **minimum-variance portfolio (MVP)** serves as an important benchmark and reference point. It represents what an extremely risk-averse investor—one who cares only about minimizing volatility—would choose.

8.3.2 The Optimization Problem

The minimum-variance investor solves:

$$\min_{\omega} \omega' \Sigma \omega$$

subject to the constraint that weights sum to one:

$$\omega' \iota = 1$$

where ι is an $N \times 1$ vector of ones.

In words: minimize portfolio variance, subject to being fully invested.

8.3.3 The Analytical Solution

This is a classic constrained optimization problem that can be solved using Lagrange multipliers. The Lagrangian is:

$$\mathcal{L} = \omega' \Sigma \omega - \lambda(\omega' \iota - 1)$$

Taking the first-order condition with respect to ω:

$$\frac{\partial \mathcal{L}}{\partial \omega} = 2\Sigma\omega - \lambda\iota = 0$$

Solving for ω:

$$\omega = \frac{\lambda}{2}\Sigma^{-1}\iota$$

Using the constraint $\omega' \iota = 1$ to solve for λ:

$$\frac{\lambda}{2}\iota'\Sigma^{-1}\iota = 1 \implies \frac{\lambda}{2} = \frac{1}{\iota'\Sigma^{-1}\iota}$$

Substituting back:

$$\omega_{\text{mvp}} = \frac{\Sigma^{-1}\iota}{\iota'\Sigma^{-1}\iota}$$

This elegant formula shows that the minimum-variance weights depend only on the covariance matrix—expected returns play no role. The inverse covariance matrix Σ^{-1} determines how much to invest in each asset based on its variance and its covariances with all other assets.

8.3.4 Implementation

```
iota = np.ones(sigma.shape[0])
sigma_inv = np.linalg.inv(sigma.values)
omega_mvp = (sigma_inv @ iota) / (iota @ sigma_inv @ iota)
```

8.3.5 Visualizing the Minimum-Variance Weights

Figure 8.3 displays the portfolio weights of the minimum-variance portfolio.

```
assets = assets.assign(omega_mvp=omega_mvp)

assets["symbol"] = pd.Categorical(
  assets["symbol"],
  categories=assets.sort_values("omega_mvp")["symbol"],
  ordered=True
)

omega_figure = (
  ggplot(
    assets,
    aes(y="omega_mvp", x="symbol", fill="omega_mvp>0")
  )
  + geom_col()
  + coord_flip()
  + scale_y_continuous(labels=percent_format())
  + labs(
      x="",
      y="Portfolio Weight",
      title="Minimum-variance portfolio weights"
  )
  + theme(legend_position="none")
)
omega_figure.show()
```

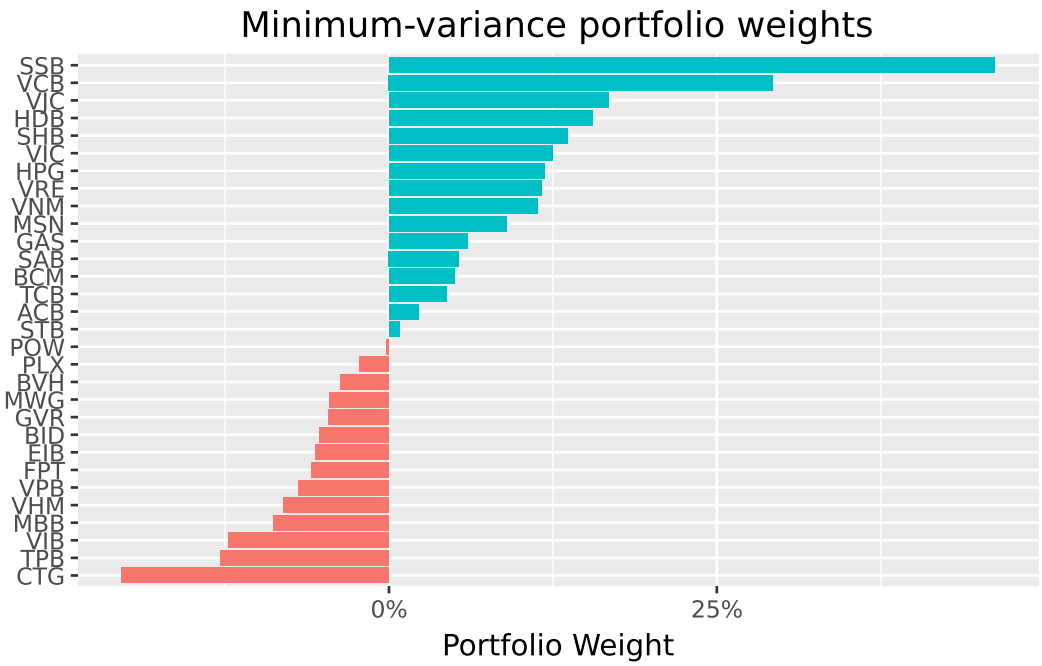


Figure 8.3: Weights are based on historical moments of monthly returns adjusted for dividend payments and stock splits.

Several features of the minimum-variance portfolio are noteworthy. First, many stocks receive zero or near-zero weights. Second, some stocks receive negative weights (short positions). These short positions are not a computational artifact, they reflect the optimizer's attempt to exploit correlations for risk reduction. Third, the weights are quite extreme (both large positive and large negative), which often indicates estimation error amplification, which is a topic we address in later chapters.

8.3.6 Portfolio Performance

Let's compute the expected return and volatility of the minimum-variance portfolio:

```
mu = assets["mu"].values
mu_mvp = omega_mvp @ mu
sigma_mvp = np.sqrt(omega_mvp @ sigma.values @ omega_mvp)

summary_mvp = pd.DataFrame({
  "mu": [mu_mvp],
  "sigma": [sigma_mvp],
  "type": ["Minimum-Variance Portfolio"]
```

```
})
summary_mvp
```

	mu	sigma	type
0	-0.011424	0.043512	Minimum-Variance Portfolio

```
mu_mvp_fmt = f"{mu_mvp:.4f}"
sigma_mvp_fmt = f"{sigma_mvp:.4f}"
print(f"The MVP return is {mu_mvp_fmt} and volatility is {sigma_mvp_fmt}.")
```

```
The MVP return is -0.0114 and volatility is 0.0435.
```

If the expected return is negative, this is not a computational error. The minimum-variance portfolio minimizes risk without regard to expected returns. Because some assets in the sample have negative average returns, the risk-minimizing combination may inherit a negative expected return. This highlights a fundamental limitation of using historical sample means as estimates of expected returns: they are extremely noisy, and can lead to economically unintuitive results even when the optimization mathematics are working correctly.

8.4 Efficient Portfolios: Balancing Risk and Return

8.4.1 The Investor's Trade-off

In most cases, minimizing variance is not the investor's sole objective. A more realistic formulation allows the investor to trade off risk against expected return. The investor might be willing to accept higher portfolio variance in exchange for higher expected returns.

An **efficient portfolio** minimizes variance subject to earning at least some target expected return $\bar{\mu}$. Formally:

$$\min_{\omega} \omega' \Sigma \omega$$

subject to:

$$\omega' \iota = 1 \quad \text{(fully invested)}$$
$$\omega' \mu \geq \bar{\mu} \quad \text{(minimum return)}$$

When $\bar{\mu}$ exceeds the expected return of the minimum-variance portfolio, the investor accepts more risk to earn more return.

8.4.2 Setting the Target Return

For illustration, suppose the investor wants to earn at least the historical average return of the best-performing stock:

```
mu_bar = assets["mu"].max()
print(f"Target expected return: {mu_bar:.5f}")
```

```
Target expected return: 0.01886
```

This is an ambitious target—it means matching the return of the single highest-returning stock while benefiting from diversification to reduce risk.

8.4.3 The Analytical Solution

The constrained optimization problem with an inequality constraint on expected returns can be solved using the Karush-Kuhn-Tucker (KKT) conditions. At the optimum (assuming the return constraint binds), the solution is:

$$\omega_{\text{efp}} = \frac{\lambda^*}{2}\left(\Sigma^{-1}\mu - \frac{D}{C}\Sigma^{-1}\iota\right)$$

where:

- $C = \iota'\Sigma^{-1}\iota$ (a scalar measuring the "size" of the inverse covariance matrix)
- $D = \iota'\Sigma^{-1}\mu$ (capturing the interaction between expected returns and the inverse covariance matrix)
- $E = \mu'\Sigma^{-1}\mu$ (measuring the "signal" in expected returns weighted by inverse covariances)
- $\lambda^* = 2\frac{\bar{\mu} - D/C}{E - D^2/C}$ (the shadow price of the return constraint)

Alternatively, we can express the efficient portfolio as a linear combination of the minimum-variance portfolio and an "excess return" portfolio:

$$\omega_{\text{efp}} = \omega_{\text{mvp}} + \frac{\lambda^*}{2}\left(\Sigma^{-1}\mu - D \cdot \omega_{\text{mvp}}\right)$$

This representation reveals important intuition: the efficient portfolio starts from the minimum-variance portfolio and tilts toward higher-expected-return assets, with the tilt magnitude determined by λ^*.

8.4.4 Implementation

```
C = iota @ sigma_inv @ iota
D = iota @ sigma_inv @ mu
E = mu @ sigma_inv @ mu
lambda_tilde = 2 * (mu_bar - D / C) / (E - (D ** 2) / C)
omega_efp = omega_mvp + (lambda_tilde / 2) * (sigma_inv @ mu - D * omega_mvp)

mu_efp = omega_efp @ mu
sigma_efp = np.sqrt(omega_efp @ sigma.values @ omega_efp)

summary_efp = pd.DataFrame({
  "mu": [mu_efp],
  "sigma": [sigma_efp],
  "type": ["Efficient Portfolio"]
})
```

8.4.5 Comparing the Portfolios

Figure 8.4 plots both portfolios alongside the individual assets.

```
summaries = pd.concat(
  [assets, summary_mvp, summary_efp], ignore_index=True
)

summaries_figure = (
  ggplot(
    summaries,
    aes(x="sigma", y="mu")
  )
  + geom_point(data=summaries.query("type.isna()"))
  + geom_point(data=summaries.query("type.notna()"), color="#F21A00", size=3)
  + geom_label(aes(label="type"), adjust_text={"arrowprops": {"arrowstyle":
  ↪  "-"}})
  + scale_x_continuous(labels=percent_format())
  + scale_y_continuous(labels=percent_format())
  + labs(
      x="Volatility (Standard Deviation)",
      y="Expected Return",
      title="Efficient & minimum-variance portfolios"
    )
```

```
)
summaries_figure.show()
```

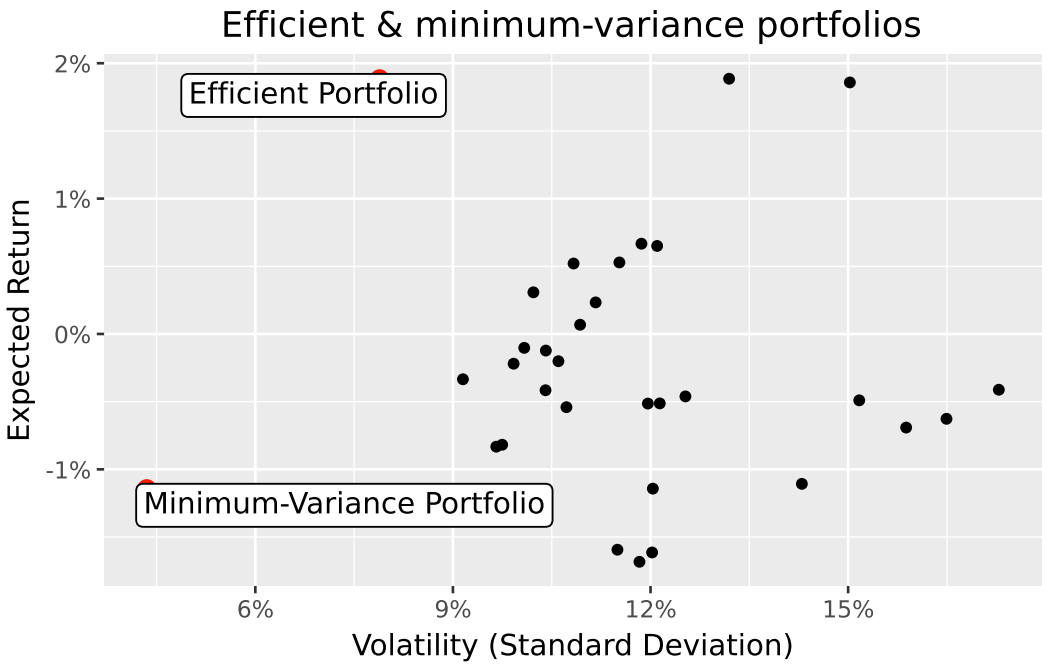


Figure 8.4: The big dots indicate the location of the minimum-variance and the efficient portfolio that delivers the expected return of the stock with the highest return, respectively. The small dots indicate the location of the individual constituents.

The figure demonstrates the substantial diversification benefits of portfolio optimization. The efficient portfolio achieves the same expected return as the highest-returning individual stock but with substantially lower volatility. This "free lunch" from diversification is the central insight of Modern Portfolio Theory.

8.4.6 The Role of Risk Aversion

The target return $\bar{\mu}$ implicitly reflects the investor's risk aversion. Less risk-averse investors choose higher $\bar{\mu}$, accepting more variance to earn more expected return. More risk-averse investors choose $\bar{\mu}$ closer to the minimum-variance portfolio's expected return.

Equivalently, the mean-variance framework can be derived from the optimal decisions of an investor with a mean-variance utility function:

$$U(\omega) = \omega'\mu - \frac{\gamma}{2}\omega'\Sigma\omega$$

where γ is the coefficient of relative risk aversion. The Appendix shows there is a one-to-one mapping between γ and $\bar{\mu}$, so both formulations yield identical efficient portfolios.

8.5 The Efficient Frontier: The Menu of Optimal Portfolios

The **efficient frontier** is the set of all portfolios for which no other portfolio offers higher expected return at the same or lower variance. Geometrically, it traces the upper boundary of achievable (volatility, expected return) combinations.

Every rational mean-variance investor should hold a portfolio on the efficient frontier. Portfolios below the frontier are "dominated," there exists another portfolio with either higher return for the same risk, or lower risk for the same return.

8.5.1 The Mutual Fund Separation Theorem

A remarkable result simplifies the construction of the efficient frontier. The **mutual fund separation theorem** (sometimes called the two-fund theorem) states that any efficient portfolio can be expressed as a linear combination of any two distinct efficient portfolios.

Formally, if ω_{μ_1} and ω_{μ_2} are efficient portfolios earning expected returns μ_1 and μ_2 respectively, then the portfolio:

$$\omega_{a\mu_1+(1-a)\mu_2} = a \cdot \omega_{\mu_1} + (1-a) \cdot \omega_{\mu_2}$$

is also efficient and earns expected return $a\mu_1 + (1-a)\mu_2$.

This result has profound practical implications: an investor needs access to only two efficient "mutual funds" to construct any portfolio on the efficient frontier. The specific funds don't matter—any two distinct efficient portfolios span the entire frontier.

8.5.2 Proof of the Separation Theorem

The proof follows directly from the analytical solution for efficient portfolios. Consider:

$$a \cdot \omega_{\mu_1} + (1-a) \cdot \omega_{\mu_2} = \left(\frac{a\mu_1 + (1-a)\mu_2 - D/C}{E - D^2/C} \right) \left(\Sigma^{-1}\mu - \frac{D}{C}\Sigma^{-1}\iota \right)$$

This expression has exactly the form of the efficient portfolio earning expected return $a\mu_1 + (1-a)\mu_2$, proving the theorem.

8.5.3 Computing the Efficient Frontier

Using the minimum-variance portfolio and our efficient portfolio as the two "funds," we can trace out the entire efficient frontier:

```
efficient_frontier = (
  pd.DataFrame({
    "a": np.arange(-1, 2.01, 0.01)
  })
  .assign(
    omega=lambda x: x["a"].map(lambda a: a * omega_efp + (1 - a) * omega_mvp)
  )
  .assign(
    mu=lambda x: x["omega"].map(lambda w: w @ mu),
    sigma=lambda x: x["omega"].map(lambda w: np.sqrt(w @ sigma @ w))
  )
)
```

Note that we allow a to range from -1 to 2, which means some portfolios involve shorting one of the two basis funds and leveraging into the other. This traces out both the upper and lower portions of the frontier hyperbola.

8.5.4 Visualizing the Efficient Frontier

Figure 8.5 displays the efficient frontier alongside individual assets and the benchmark portfolios.

```
summaries = pd.concat(
  [summaries, efficient_frontier], ignore_index=True
)

summaries_figure = (
  ggplot(
    summaries,
    aes(x="sigma", y="mu")
  )
  + geom_point(data=summaries.query("type.isna()"))
  + geom_line(data=efficient_frontier, color="blue", alpha=0.7)
  + geom_point(data=summaries.query("type.notna()"), color="#F21A00", size=3)
  + geom_label(aes(label="type"), adjust_text={"arrowprops": {"arrowstyle":
  ↪  "-"}})
```

```
  + scale_x_continuous(labels=percent_format())
  + scale_y_continuous(labels=percent_format())
  + labs(
      x="Volatility (Standard Deviation)",
      y="Expected Return",
      title="The Efficient Frontier and VN30 Constituents"
    )
)
summaries_figure.show()
```

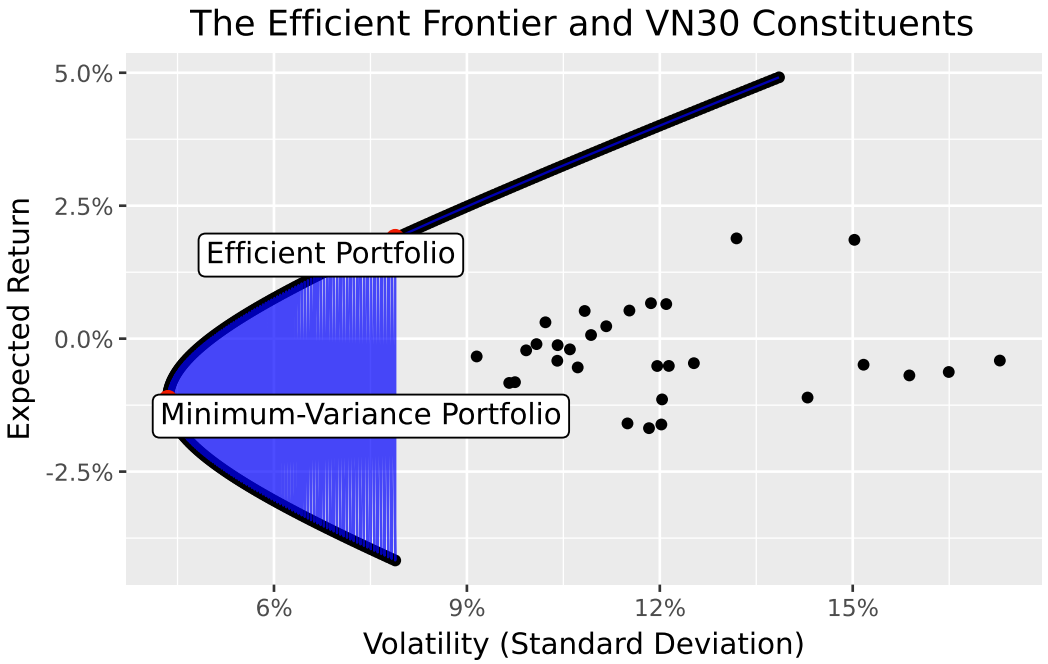


Figure 8.5: The big dots indicate the location of the minimum-variance and the efficient portfolio. The small dots indicate the location of the individual constituents.

The efficient frontier has a characteristic hyperbolic shape. The leftmost point is the minimum-variance portfolio. Moving up and to the right along the frontier, expected return increases but so does volatility. The upper portion of the hyperbola (above the minimum-variance portfolio) is the "efficient" part—these portfolios offer the highest return for each level of risk. The lower portion is "inefficient"—these portfolios are dominated by their mirror images on the upper portion.

The figure also reveals how dramatically diversification improves upon individual stock positions. Nearly all individual stocks lie well inside the efficient frontier, meaning investors can

achieve the same expected return with much less risk, or much higher expected return with the same risk, simply by diversifying.

8.6 Key Takeaways

This chapter introduced the concepts of Modern Portfolio Theory. The main insights are:

1. **Portfolio risk depends on correlations**: The variance of a portfolio is not simply the weighted average of individual variances. Covariances between assets play a crucial role, creating opportunities for diversification.
2. **Diversification is the "only free lunch" in finance**: By combining assets that don't move perfectly together, investors can reduce risk without sacrificing expected return. This insight is the cornerstone of modern investment practice.
3. **The minimum-variance portfolio minimizes risk**: This portfolio depends only on the covariance matrix and serves as an important benchmark. It represents the least risky way to be fully invested in risky assets.
4. **Efficient portfolios balance risk and return**: By accepting more variance, investors can earn higher expected returns. Efficient portfolios are those that offer the best possible trade-off.
5. **The efficient frontier characterizes optimal portfolios**: This boundary in mean-standard deviation space represents the menu of optimal choices available to mean-variance investors.
6. **Two-fund separation simplifies implementation**: Any efficient portfolio can be constructed from any two distinct efficient portfolios, reducing the computational burden of portfolio optimization.

Looking ahead, several important complications arise in practice. First, the inputs to portfolio optimization (expected returns and covariances) must be estimated from data, and estimation error can dramatically affect portfolio performance. Second, real-world constraints such as transaction costs, short-sale restrictions, and position limits modify the optimization problem. Third, the assumption that investors care only about mean and variance may be too restrictive when returns are non-normal or when investors have more complex preferences. We address these extensions in subsequent chapters.

9 Univariate Portfolio Sorts

In this chapter, we dive into portfolio sorts, one of the most widely used statistical methodologies in empirical asset pricing (e.g., Bali, Engle, and Murray 2016b). The key application of portfolio sorts is to examine whether one or more variables can predict future excess returns. In general, the idea is to sort individual stocks into portfolios, where the stocks within each portfolio are similar with respect to a sorting variable, such as firm size. The different portfolios then represent well-diversified investments that differ in the level of the sorting variable. You can then attribute the differences in the return distribution to the impact of the sorting variable. We start by introducing univariate portfolio sorts (which sort based on only one characteristic) and tackle bivariate sorting in Value and Bivariate Sorts.

A univariate portfolio sort considers only one sorting variable $x_{i,t-1}$. Here, i denotes the stock and $t-1$ indicates that the characteristic is observable by investors at time t. The objective is to assess the cross-sectional relation between $x_{i,t-1}$ and, typically, stock excess returns $r_{i,t}$ at time t as the outcome variable. To illustrate how portfolio sorts work, we use estimates for market betas from the previous chapter as our sorting variable.

```
import pandas as pd
import numpy as np
import sqlite3
import statsmodels.api as sm

from plotnine import *
from mizani.formatters import percent_format
from regtabletotext import prettify_result
```

9.1 Data Preparation

We start with loading the required data from our SQLite database introduced in Accessing and Managing Financial Data and DataCore Data. In particular, we use the monthly stock price data as our asset universe. Once we form our portfolios, we use the market factor returns to compute the risk-adjusted performance (i.e., alpha). `beta` is the dataframe with market betas computed in the previous chapter.

```
tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")

prices_monthly = (pd.read_sql_query(
    sql="SELECT symbol, date, ret_excess, mktcap_lag FROM prices_monthly",
    con=tidy_finance,
    parse_dates={"date"})
)

factors_ff3_monthly = pd.read_sql_query(
  sql="SELECT date, mkt_excess FROM factors_ff3_monthly",
  con=tidy_finance,
  parse_dates={"date"}
)

beta = pd.read_sql_query(
  sql="SELECT symbol, date, beta FROM beta_monthly",
  con=tidy_finance,
  parse_dates={"date"}
)
```

9.2 Sorting by Market Beta

Next, we merge our sorting variable with the return data. We use the one-month *lagged* betas as a sorting variable to ensure that the sorts rely only on information available when we create the portfolios. To lag stock beta by one month, we add one month to the current date and join the resulting information with our return data. This procedure ensures that month t information is available in month $t + 1$. You may be tempted to simply use a call such as `prices_monthly['beta_lag'] = prices_monthly.groupby('symbol')['beta'].shift(1)` instead. This procedure, however, does not work correctly if there are implicit missing values in the time series.

```
beta_lag = (beta
  .assign(date=lambda x: x["date"]+pd.DateOffset(months=1))
  .get(["symbol", "date", "beta"])
  .rename(columns={"beta": "beta_lag"})
  .dropna()
)

data_for_sorts = (prices_monthly
  .merge(beta_lag, how="inner", on=["symbol", "date"])
```

```
  .dropna()
)
```

The first step to conduct portfolio sorts is to calculate periodic breakpoints that you can use to group the stocks into portfolios. For simplicity, we start with the median lagged market beta as the single breakpoint. We then compute the value-weighted returns for each of the two resulting portfolios, which means that the lagged market capitalization determines the weight in `np.average()`.

```
beta_portfolios = (
    data_for_sorts
    .groupby("date")
    .apply(lambda x: (
        x.assign(
            portfolio=pd.qcut(x["beta_lag"], q=[0, 0.5, 1], labels=["low",
 ↪  "high"]),
            date=x.name
        )
    ))
    .reset_index(drop=True)
    .groupby(["portfolio", "date"])
    .apply(lambda x: np.average(x["ret_excess"], weights=x["mktcap_lag"]))
    .reset_index(name="ret")
    .dropna(subset=['ret'])
)
beta_portfolios.head()
```

	portfolio	date	ret
0	low	2016-08-31	-0.016507
1	low	2016-09-30	-0.089155
2	low	2016-11-30	-0.015080
3	low	2017-01-31	0.004113
4	low	2017-02-28	0.035101

9.3 Performance Evaluation

We can construct a long-short strategy based on the two portfolios: buy the high-beta portfolio and, at the same time, short the low-beta portfolio. Thereby, the overall position in the market is net-zero.

```
beta_longshort = (beta_portfolios
  .pivot_table(index="date", columns="portfolio", values="ret")
  .reset_index()
  .assign(long_short=lambda x: x["high"]-x["low"])
)
```

We compute the average return and the corresponding standard error to test whether the long-short portfolio yields on average positive or negative excess returns. In the asset pricing literature, one typically adjusts for autocorrelation by using Whitney K. Newey and West (1987a) t-statistics to test the null hypothesis that average portfolio excess returns are equal to zero. One necessary input for Newey-West standard errors is a chosen bandwidth based on the number of lags employed for the estimation. Researchers often default to choosing a pre-specified lag length of six months (which is not a data-driven approach). We do so in the `fit()` function by indicating the `cov_type` as `HAC` and providing the maximum lag length through an additional keywords dictionary.

```
model_fit = (sm.OLS.from_formula(
    formula="long_short ~ 1",
    data=beta_longshort
  )
  .fit(cov_type="HAC", cov_kwds={"maxlags": 6})
)
prettify_result(model_fit)
```

```
OLS Model:
long_short ~ 1

Coefficients:
           Estimate  Std. Error  t-Statistic  p-Value
Intercept     0.006       0.006        1.018    0.309

Summary statistics:
- Number of observations: 53
- R-squared: 0.000, Adjusted R-squared: 0.000
- F-statistic not available
```

The results indicate that we cannot reject the null hypothesis of average returns being equal to zero. Our portfolio strategy using the median as a breakpoint does not yield any abnormal returns. Is this finding surprising if you reconsider the CAPM? It certainly is. The CAPM predicts that high-beta stocks should yield higher expected returns. Our portfolio sort implicitly mimics an investment strategy that finances high-beta stocks by shorting low-beta stocks.

Therefore, one should expect that the average excess returns yield a return that is above the risk-free rate.

9.4 Functional Programming for Portfolio Sorts

Now, we take portfolio sorts to the next level. We want to be able to sort stocks into an arbitrary number of portfolios. For this case, functional programming is very handy: we define a function that gives us flexibility concerning which variable to use for the sorting, denoted by `sorting_variable`. We use `np.quantile()` to compute breakpoints for `n_portfolios`. Then, we assign portfolios to stocks using the `pd.cut()` function. The output of the following function is a new column that contains the number of the portfolio to which a stock belongs.

In some applications, the variable used for the sorting might be clustered (e.g., at a lower bound of 0). Then, multiple breakpoints may be identical, leading to empty portfolios. Similarly, some portfolios might have a very small number of stocks at the beginning of the sample. Cases where the number of portfolio constituents differs substantially due to the distribution of the characteristics require careful consideration and, depending on the application, might require customized sorting approaches.

```
def assign_portfolio(data, sorting_variable, n_portfolios):
    """Assign portfolios to a bin between breakpoints."""

    breakpoints = np.quantile(
      data[sorting_variable].dropna(),
      np.linspace(0, 1, n_portfolios + 1),
      method="linear"
    )

    assigned_portfolios = pd.cut(
      data[sorting_variable],
      bins=breakpoints,
      labels=range(1, breakpoints.size),
      include_lowest=True,
      right=False
    )

    return assigned_portfolios
```

We can use the above function to sort stocks into ten portfolios each month using lagged betas and compute value-weighted returns for each portfolio. Note that we transform the portfolio column to a factor variable because it provides more convenience for the figure construction below.

```
beta_portfolios = (data_for_sorts
  .groupby("date")
  .apply(lambda x: x.assign(
      portfolio=assign_portfolio(x, "beta_lag", 10)
    ), include_groups=False
  )
  .reset_index(level="date")
  .groupby(["portfolio", "date"])
  .apply(lambda x: pd.Series({
      "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
    }), include_groups=False
  )
  .reset_index()
  .merge(factors_ff3_monthly, how="left", on="date")
)
```

9.5 More Performance Evaluation

In the next step, we compute summary statistics for each beta portfolio. Namely, we compute CAPM-adjusted alphas, the beta of each beta portfolio, and average returns.

```
beta_portfolios_summary = (beta_portfolios
  .groupby("portfolio")
  .apply(lambda x: pd.Series({
      "alpha": sm.OLS.from_formula(
          formula="ret ~ 1 + mkt_excess",
          data=x
        ).fit().params["Intercept"],
      "beta": sm.OLS.from_formula(
          formula="ret ~ 1 + mkt_excess",
          data=x
        ).fit().params["mkt_excess"],
      "ret": x["ret"].mean()
    }), include_groups=False
  )
  .reset_index()
)
beta_portfolios_summary
```

	portfolio	alpha	beta	ret
0	1	0.007031	0.356225	0.003892
1	2	-0.004215	0.227882	-0.005509
2	3	-0.010169	0.390216	-0.011241
3	4	-0.014205	0.388342	-0.015732
4	5	-0.007220	0.616833	-0.009723
5	6	0.010648	0.976502	0.006577
6	7	-0.010739	0.679145	-0.012645
7	8	-0.002757	0.991774	-0.006963
8	9	-0.003448	0.904886	-0.007174
9	10	0.010675	1.376356	0.004944

Figure 9.1 illustrates the CAPM alphas of beta-sorted portfolios.

```
beta_portfolios_figure = (
  ggplot(
    beta_portfolios_summary,
    aes(x="portfolio", y="alpha", fill="portfolio")
  )
  + geom_bar(stat="identity")
  + labs(
      x="Portfolio", y="CAPM alpha", fill="Portfolio",
      title="CAPM alphas of beta-sorted portfolios"
    )
  + scale_y_continuous(labels=percent_format())
  + theme(legend_position="none")
)
beta_portfolios_figure.show()
```

Unlike the well-documented "betting against beta" anomaly in US markets, where low-beta portfolios exhibit positive alphas and high-beta portfolios exhibit negative alphas in a monotonic pattern, the Vietnamese market shows no clear relationship between beta and risk-adjusted returns. The alphas fluctuate without a discernible trend across deciles. This lack of pattern likely reflects the limited sample period rather than a definitive conclusion about beta pricing in Vietnam. With such a short time series, the portfolio-level CAPM regressions contain substantial estimation noise, making it difficult to detect subtle anomalies. Longer sample periods would be needed to draw reliable conclusions about whether the low-beta anomaly exists in the Vietnamese equity market.

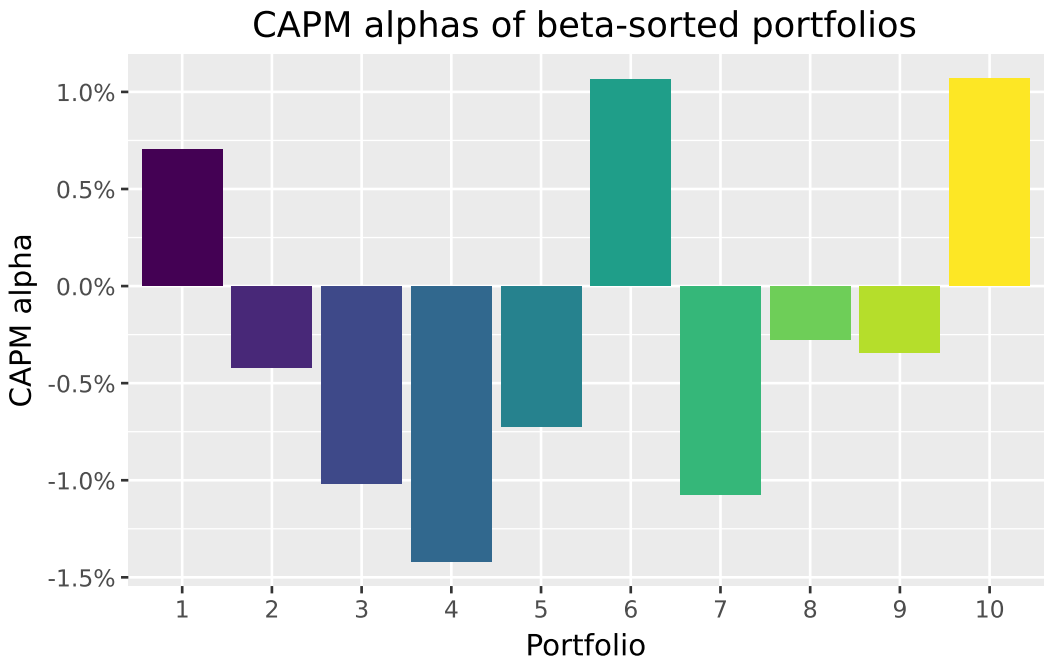


Figure 9.1: The figure shows CAPM alphas of beta-sorted portfolios. Portfolios are sorted into deciles each month based on their estimated CAPM beta. The bar charts indicate the CAPM alpha of the resulting portfolio returns during the sample period.

9.6 Security Market Line and Beta Portfolios

The CAPM predicts that our portfolios should lie on the security market line (SML). The slope of the SML is equal to the market risk premium and reflects the risk-return trade-off at any given time. Figure 9.2 illustrates the security market line: We see that (not surprisingly) the high-beta portfolio returns have a high correlation with the market returns. However, it seems like the average excess returns for high-beta stocks are lower than what the security market line implies would be an "appropriate" compensation for the high market risk.

```
sml_capm = (sm.OLS.from_formula(
    formula="ret ~ 1 + beta",
    data=beta_portfolios_summary
  )
  .fit()
  .params
)

sml_figure = (
```

```
  ggplot(
    beta_portfolios_summary,
    aes(x="beta", y="ret", color="factor(portfolio)")
  )
  + geom_point()
  + geom_abline(
      intercept=0, slope=factors_ff3_monthly["mkt_excess"].mean(),
↪   linetype="solid"
    )
  + geom_abline(
      intercept=sml_capm["Intercept"], slope=sml_capm["beta"],
↪   linetype="dashed"
    )
  + labs(
      x="Beta", y="Excess return", color="Portfolio",
      title="Average portfolio excess returns and beta estimates"
    )
  + scale_x_continuous(limits=(0, 2))
  + scale_y_continuous(labels=percent_format())
)
sml_figure.show()
```

To provide more evidence against the CAPM predictions, we again form a long-short strategy that buys the high-beta portfolio and shorts the low-beta portfolio.

```
beta_longshort = (beta_portfolios
  .assign(
    portfolio=lambda x: (
      x["portfolio"].apply(
        lambda y: "high" if y == x["portfolio"].max()
        else ("low" if y == x["portfolio"].min()
        else y)
      )
    )
  )
  .query("portfolio in ['low', 'high']")
  .pivot_table(index="date", columns="portfolio", values="ret")
  .assign(long_short=lambda x: x["high"]-x["low"])
  .merge(factors_ff3_monthly, how="left", on="date")
)
```

Again, the resulting long-short strategy does not exhibit statistically significant returns.



Figure 9.2: The figure shows average portfolio excess returns and beta estimates. Excess returns are computed as CAPM alphas of the beta-sorted portfolios. The horizontal axis indicates the CAPM beta of the resulting beta-sorted portfolio return time series. The dashed line indicates the slope coefficient of a linear regression of excess returns on portfolio betas.

```
model_fit = (sm.OLS.from_formula(
    formula="long_short ~ 1",
    data=beta_longshort
  )
  .fit(cov_type="HAC", cov_kwds={"maxlags": 1})
)
prettify_result(model_fit)
```

```
OLS Model:
long_short ~ 1

Coefficients:
           Estimate  Std. Error  t-Statistic  p-Value
Intercept     0.001       0.011        0.095    0.924

Summary statistics:
```

```
- Number of observations: 53
- R-squared: 0.000, Adjusted R-squared: 0.000
- F-statistic not available
```

However, controlling for the effect of beta, the long-short portfolio yields a CAPM-adjusted alpha. The results can provide evidence regarding the validity of the CAPM in the Vietnamese market. The betting-against-beta factor has been documented extensively in developed markets (Frazzini and Pedersen 2014). Betting-against-beta corresponds to a strategy that shorts high-beta stocks and takes a (levered) long position in low-beta stocks. If borrowing constraints prevent investors from taking positions on the security market line, they are instead incentivized to buy high-beta stocks, which leads to a relatively higher price (and therefore lower expected returns than implied by the CAPM) for such high-beta stocks. As a result, the betting-against-beta strategy earns from providing liquidity to capital-constrained investors with lower risk aversion.

```
model_fit = (sm.OLS.from_formula(
    formula="long_short ~ 1 + mkt_excess",
    data=beta_longshort
  )
  .fit(cov_type="HAC", cov_kwds={"maxlags": 1})
)
prettify_result(model_fit)
```

```
OLS Model:
long_short ~ 1 + mkt_excess

Coefficients:
            Estimate  Std. Error  t-Statistic  p-Value
Intercept      0.004       0.009        0.394    0.694
mkt_excess     1.020       0.116        8.784    0.000

Summary statistics:
- Number of observations: 52
- R-squared: 0.527, Adjusted R-squared: 0.518
- F-statistic: 77.156 on 1 and 50 DF, p-value: 0.000
```

Figure 9.3 shows the annual returns of the extreme beta portfolios we are mainly interested in. The figure illustrates the patterns over the sample period; each portfolio exhibits periods with positive and negative annual returns.

```
beta_longshort_year = (beta_longshort
  .assign(year=lambda x: x["date"].dt.year)
  .groupby("year")
  .aggregate(
    low=("low", lambda x: (1+x).prod()-1),
    high=("high", lambda x: (1+x).prod()-1),
    long_short=("long_short", lambda x: (1+x).prod()-1)
  )
  .reset_index()
  .melt(id_vars="year", var_name="name", value_name="value")
)

beta_longshort_figure = (
  ggplot(
    beta_longshort_year,
    aes(x="year", y="value", fill="name")
  )
  + geom_col(position="dodge")
  + facet_wrap("~name", ncol=1)
  + labs(x="", y="", title="Annual returns of beta portfolios")
  + scale_y_continuous(labels=percent_format())
  + theme(legend_position="none")
)
beta_longshort_figure.show()
```

The high-beta portfolio and low-beta portfolio both exhibit substantial year-to-year variation. The long-short portfolio, which goes long high-beta stocks and short low-beta stocks, shows no consistent pattern of positive returns. This erratic performance reinforces our earlier finding that the beta-return relationship in the Vietnamese market does not conform to theoretical CAPM predictions during our sample period. The high volatility of annual long-short returns highlights the substantial risk inherent in such a strategy, particularly in an emerging market context with a limited sample period.

9.7 Key Takeaways

1. Univariate portfolio sorts assess whether a single firm characteristic, like lagged market beta, can predict future excess returns.
2. Portfolios are formed each month using quantile breakpoints, with returns computed using value-weighted averages to reflect realistic investment strategies.

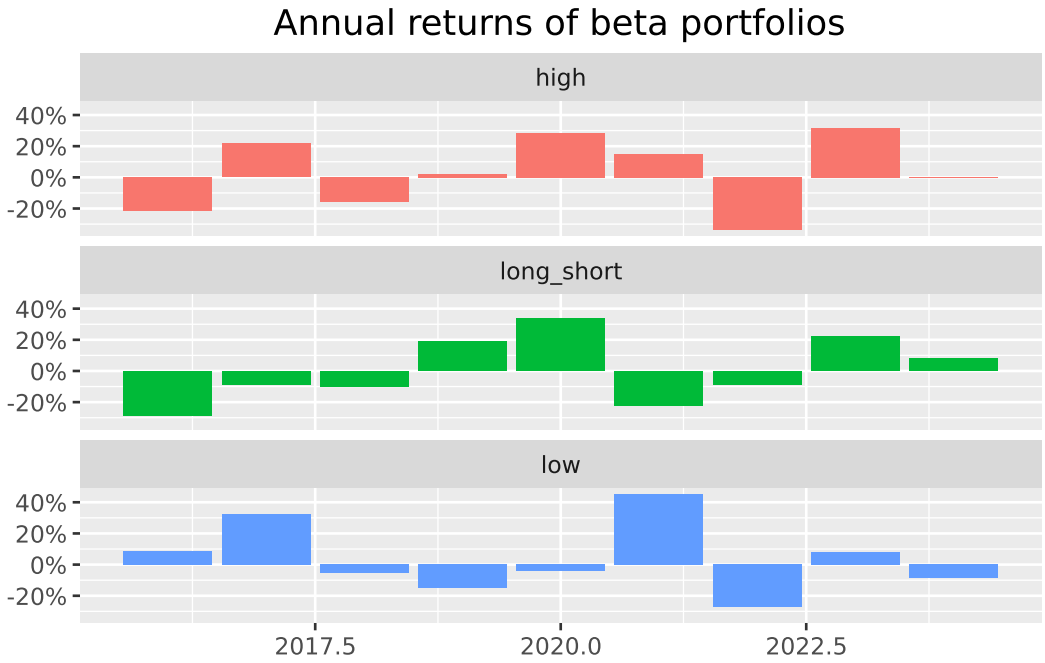


Figure 9.3: The figure shows annual returns of beta portfolios. We construct portfolios by sorting stocks into high and low based on their estimated CAPM beta. Long short indicates a strategy that goes long into high beta stocks and short low beta stocks.

3. A long-short strategy based on beta-sorted portfolios fails to generate significant positive excess returns in the Vietnamese market, contradicting CAPM predictions that higher beta should yield higher returns.

4. The analysis provides a framework for examining the "betting against beta" anomaly, where low-beta portfolios may deliver higher alphas than high-beta portfolios, offering evidence regarding the validity of the CAPM.

5. The functional programming capabilities of Python enable scalable and flexible portfolio sorting, making it easy to analyze multiple characteristics and portfolio configurations.

6. Emerging markets like Vietnam may exhibit different beta-return relationships compared to developed markets, highlighting the importance of conducting market-specific empirical analysis rather than assuming universal applicability of asset pricing anomalies.

10 Size Sorts

In this chapter, we continue with portfolio sorts in a univariate setting. Yet, we consider firm size as a sorting variable, which gives rise to a well-known return factor: the size premium. The size premium arises from buying small stocks and selling large stocks. Prominently, Eugene F. Fama and French (1993a) include it as a factor in their three-factor model. Apart from that, asset managers commonly include size as a key firm characteristic when making investment decisions.

We also introduce new choices in the formation of portfolios. In particular, we discuss listing exchanges, industries, weighting regimes, and periods. These choices matter for the portfolio returns and result in different size premiums (see Hasler (2021), Soebhag, Van Vliet, and Verwijmeren (2022), and Walter, Weber, and Weiss (2022) for more insights into decision nodes and their effect on premiums).

```
import pandas as pd
import numpy as np
import sqlite3

from plotnine import *
from mizani.formatters import percent_format
from itertools import product
from joblib import Parallel, delayed, cpu_count
```

10.1 Data Preparation

First, we retrieve the relevant data from our SQLite database introduced in Accessing and Managing Financial Data and DataCore Data. Firm size is defined as market equity in most asset pricing applications. We further use the Fama-French factor returns for performance evaluation.

```
tidy_finance = sqlite3.connect(
  database="data/tidy_finance_python.sqlite"
)
```

```
prices_monthly = pd.read_sql_query(
  sql="SELECT * FROM prices_monthly",
  con=tidy_finance,
  parse_dates={"date"}
)

factors_ff3_monthly = pd.read_sql_query(
  sql="SELECT * FROM factors_ff3_monthly",
  con=tidy_finance,
  parse_dates={"date"}
)
```

10.2 Size Distribution

Before we build our size portfolios, we investigate the distribution of the variable *firm size*. Visualizing the data is a valuable starting point to understand the input to the analysis. Figure 10.1 shows the fraction of total market capitalization concentrated in the largest firm. To produce this graph, we create monthly indicators that track whether a stock belongs to the largest x percent of the firms. Then, we aggregate the firms within each bucket and compute the buckets' share of total market capitalization.

Figure 10.1 shows that the largest 1 percent of firms cover up to 50 percent of the total market capitalization, and holding just the 25 percent largest firms in the universe essentially replicates the market portfolio. The distribution of firm size thus implies that the largest firms of the market dominate many small firms whenever we use value-weighted benchmarks.

```
market_cap_concentration = (prices_monthly
  .groupby("date")
  .apply(lambda x: pd.Series({
    "Largest 1%": x.loc[x["mktcap"] >= x["mktcap"].quantile(0.99),
    ↪  "mktcap"].sum() / x["mktcap"].sum(),
    "Largest 5%": x.loc[x["mktcap"] >= x["mktcap"].quantile(0.95),
    ↪  "mktcap"].sum() / x["mktcap"].sum(),
    "Largest 10%": x.loc[x["mktcap"] >= x["mktcap"].quantile(0.90),
    ↪  "mktcap"].sum() / x["mktcap"].sum(),
    "Largest 25%": x.loc[x["mktcap"] >= x["mktcap"].quantile(0.75),
    ↪  "mktcap"].sum() / x["mktcap"].sum()
  }), include_groups=False)
  .reset_index()
  .melt(id_vars="date", var_name="name", value_name="value")
)
```

```
market_cap_concentration_figure = (
  ggplot(
    market_cap_concentration,
    aes(x="date", y="value", color="name", linetype="name")
  ) +
  geom_line() +
  scale_y_continuous(labels=percent_format()) +
  scale_x_date(name="", date_labels="%Y") +
  labs(
    x="", y="", color="", linetype="",
    title="Percentage of total market capitalization in largest stocks"
  ) +
  theme(legend_title=element_blank())
)
market_cap_concentration_figure.show()
```

```
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
```

```
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:4: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:5: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:6: RuntimeWarning: invalid value encountered in scalar di
/tmp/ipykernel_3245094/419327778.py:7: RuntimeWarning: invalid value encountered in scalar di
```

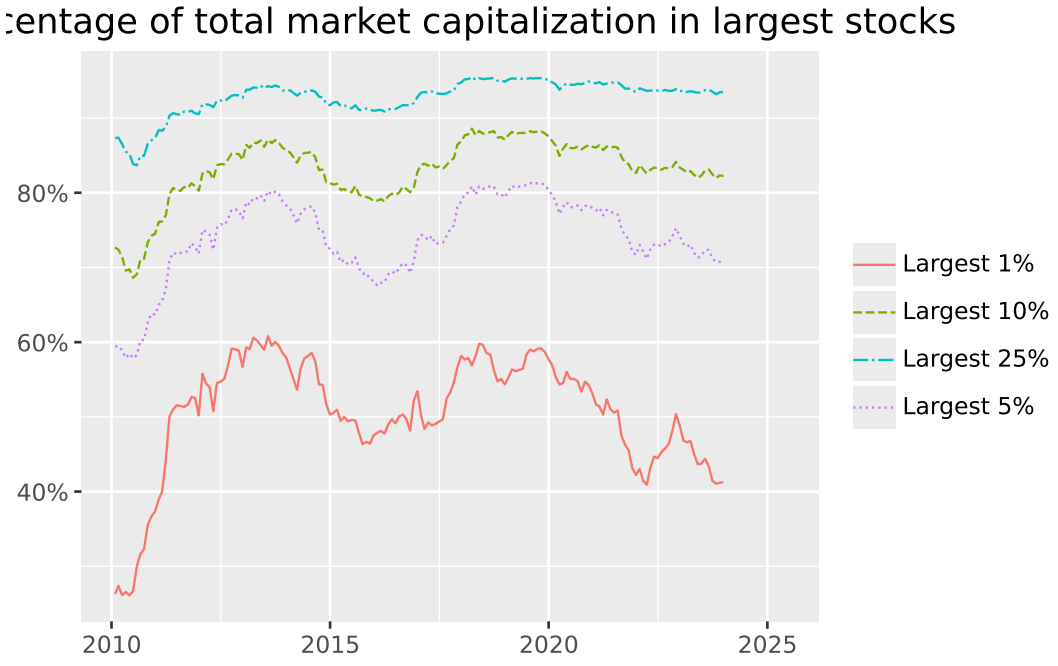


Figure 10.1: The figure shows the percentage of total market capitalization in largest stocks. We report the aggregate market capitalization of all stocks that belong to the 1, 5, 10, and 25 percent quantile of the largest firms in the monthly cross-section relative to the market capitalization of all stocks during the month.

Next, firm sizes also differ across listing exchanges. The primary listings of stocks were important in the past and are potentially still relevant today.

Finally, we consider the distribution of firm size across listing exchanges and create summary statistics. The function `describe()` does not include all statistics we are interested in, which is why we create the function `compute_summary()` that adds the standard deviation and the number of observations. Then, we apply it to the most current month of our data on each listing exchange. We also add a row with the overall summary statistics. In the following, we use this distinction to update our portfolio sort procedure.

```
def compute_summary(data, variable, filter_variable, percentiles):
    """Compute summary statistics for a given variable and percentiles."""

    summary = (data[[filter_variable, variable]]
      .groupby(filter_variable)
      .describe(percentiles=percentiles)
```

```
market_cap_share = (prices_monthly
  .groupby(["date", "exchange"])
  .aggregate({"mktcap": "sum"})
  .reset_index(drop=False)
  .groupby("date")
  .apply(lambda x:
    x.assign(total_market_cap=lambda x: x["mktcap"].sum(),
             share=lambda x: x["mktcap"]/x["total_market_cap"]
             )
    )
  .reset_index(drop=True)
)

plot_market_cap_share = (
  ggplot(market_cap_share,
         aes(x="date", y="share",
             fill="exchange", color="exchange")) +
  geom_area(position="stack", stat="identity", alpha=0.5) +
  geom_line(position="stack") +
  scale_y_continuous(labels=percent_format()) +
  scale_x_date(name="", date_labels="%Y") +
  labs(x="", y="", fill="", color="",
       title="Share of total market capitalization per listing exchange") +
  theme(legend_title=element_blank())
)
plot_market_cap_share.draw()
```

Figure 10.2

```
    )

    summary.columns = summary.columns.droplevel(0)

    summary_overall = (data[variable]
      .describe(percentiles=percentiles)
    )

    summary.loc["Overall", :] = summary_overall

    return summary.round(0)

compute_summary(
  prices_monthly[prices_monthly["date"] == prices_monthly["date"].max()],
  variable="mktcap",
  filter_variable="exchange",
  percentiles=[0.05, 0.5, 0.95]
)
```

10.3 Univariate Size Portfolios with Flexible Breakpoints

In Univariate Portfolio Sorts, we construct portfolios with a varying number of breakpoints and different sorting variables. Here, we extend the framework such that we compute breakpoints on a subset of the data, for instance, based on selected listing exchanges.

We introduce `exchanges` as an argument in our `assign_portfolio()` function from Univariate Portfolio Sorts. The exchange-specific argument then enters in the filter `data["exchanges"].isin(exchanges)`. For example, if `exchanges='HOSE'` is specified, only stocks listed on HOSE are used to compute the breakpoints. Alternatively, you could specify `exchanges=["HOSE", "HNX", "UPCoM"]`, which keeps all stocks listed on either of these exchanges.

```
def assign_portfolio(data, exchanges, sorting_variable, n_portfolios):
    """Assign portfolio for a given sorting variable."""

    breakpoints = (data
      .query(f"exchange in {exchanges}")
      .get(sorting_variable)
      .quantile(np.linspace(0, 1, num=n_portfolios+1),
                interpolation="linear")
      .drop_duplicates()
```

```
    )
    breakpoints.iloc[[0, -1]] = [-np.Inf, np.Inf]

    assigned_portfolios = pd.cut(
      data[sorting_variable],
      bins=breakpoints,
      labels=range(1, breakpoints.size),
      include_lowest=True,
      right=False
    )

    return assigned_portfolios
```

10.4 Weighting Schemes for Portfolios

Apart from computing breakpoints on different samples, researchers often use different portfolio weighting schemes. So far, we weighted each portfolio constituent by its relative market equity of the previous period. This protocol is called *value-weighting*. The alternative protocol is *equal-weighting*, which assigns each stock's return the same weight, i.e., a simple average of the constituents' returns. Notice that equal-weighting is difficult in practice as the portfolio manager needs to rebalance the portfolio monthly, while value-weighting is a truly passive investment.

We implement the two weighting schemes in the function `compute_portfolio_returns()` that takes a logical argument to weight the returns by firm value. Additionally, the long-short portfolio is long in the smallest firms and short in the largest firms, consistent with research showing that small firms outperform their larger counterparts. Apart from these two changes, the function is similar to the procedure in Univariate Portfolio Sorts.

```
def calculate_returns(data, value_weighted):
    """Calculate (value-weighted) returns."""

    if value_weighted:
      return np.average(data["ret_excess"], weights=data["mktcap_lag"])
    else:
      return data["ret_excess"].mean()

def compute_portfolio_returns(n_portfolios=10,
                              exchanges=["HOSE", "HNX", "UPCoM"],
                              value_weighted=True,
                              data=prices_monthly):
```

```
    """Compute (value-weighted) portfolio returns."""

    returns = (data
      .groupby("date")
      .apply(lambda x: x.assign(
        portfolio=assign_portfolio(x, exchanges,
                                   "mktcap_lag", n_portfolios))
      )
      .reset_index(drop=True)
      .groupby(["portfolio", "date"])
      .apply(lambda x: x.assign(
        ret=calculate_returns(x, value_weighted))
      )
      .reset_index(drop=True)
      .groupby("date")
      .apply(lambda x:
        pd.Series({"size_premium": x.loc[x["portfolio"].idxmin(), "ret"]-
          x.loc[x["portfolio"].idxmax(), "ret"]}))
      .reset_index(drop=True)
      .aggregate({"size_premium": "mean"})
    )

    return returns
```

To see how the function `compute_portfolio_returns()` works, we consider a simple median breakpoint example with value-weighted returns. We are interested in the effect of restricting listing exchanges on the estimation of the size premium. In the first function call, we compute returns based on breakpoints from all listing exchanges. Then, we computed returns based on breakpoints from HOSE-listed stocks.

```
ret_all = compute_portfolio_returns(
  n_portfolios=2,
  exchanges=["HOSE", "HNX", "UPCoM"],
  value_weighted=True,
  data=prices_monthly
)

ret_HOSE = compute_portfolio_returns(
  n_portfolios=2,
  exchanges=["HOSE"],
  value_weighted=True,
  data=prices_monthly
```

```
)

data = pd.DataFrame([ret_all*100, ret_HOSE*100],
                    index=["HOSE, HNX & UPCoM", "HOSE"])
data.columns = ["Premium"]
data.round(2)
```

10.5 P-Hacking and Non-Standard Errors

Since the choice of the listing exchange has a significant impact, the next step is to investigate the effect of other data processing decisions researchers have to make along the way. In particular, any portfolio sort analysis has to decide at least on the number of portfolios, the listing exchanges to form breakpoints, and equal- or value-weighting. Further, one may exclude firms that are active in the finance industry or restrict the analysis to some parts of the time series. All of the variations of these choices that we discuss here are part of scholarly articles published in the top finance journals. We refer to Walter, Weber, and Weiss (2022) for an extensive set of other decision nodes at the discretion of researchers.

The intention of this application is to show that the different ways to form portfolios result in different estimated size premiums. Despite the effects of this multitude of choices, there is no correct way. It should also be noted that none of the procedures is wrong. The aim is simply to illustrate the changes that can arise due to the variation in the evidence-generating process (Menkveld et al., n.d.). The term *non-standard errors* refers to the variation due to (suitable) choices made by researchers. Interestingly, in a large-scale study, Menkveld et al. (n.d.) find that the magnitude of non-standard errors is similar to the estimation uncertainty based on a chosen model. This shows how important it is to adjust for the seemingly innocent choices in the data preparation and evaluation workflow. Moreover, it seems that this methodology-related uncertainty should be embraced rather than hidden away.

From a malicious perspective, these modeling choices give the researcher multiple *chances* to find statistically significant results. Yet this is considered *p-hacking*, which renders the statistical inference invalid due to multiple testing (Campbell R. Harvey, Liu, and Zhu 2016a).

Nevertheless, the multitude of options creates a problem since there is no single correct way of sorting portfolios. How should a researcher convince a reader that their results do not come from a p-hacking exercise? To circumvent this dilemma, academics are encouraged to present evidence from different sorting schemes as *robustness tests* and report multiple approaches to show that a result does not depend on a single choice. Thus, the robustness of premiums is a key feature.

Below, we conduct a series of robustness tests, which could also be interpreted as a p-hacking exercise. To do so, we examine the size premium in different specifications presented in the

table `p_hacking_setup`. The function `itertools.product()` produces all possible permutations of its arguments. Note that we use the argument `data` to exclude financial firms and truncate the time series.

```
n_portfolios = [2, 5, 10]
exchanges = [["HOSE"], ["HOSE", "HNX", "UPCoM"]]
value_weighted = [True, False]
data = [
  prices_monthly,
  prices_monthly[prices_monthly["industry"] != "Finance"],
  prices_monthly[prices_monthly["date"] < "1990-06-01"],
  prices_monthly[prices_monthly["date"] >= "1990-06-01"],
]
p_hacking_setup = list(
  product(n_portfolios, exchanges, value_weighted, data)
)
```

To speed the computation up, we parallelize the (many) different sorting procedures. Finally, we report the resulting size premiums in descending order. There are indeed substantial size premiums possible in our data, in particular when we use equal-weighted portfolios.

```
n_cores = cpu_count()-1
p_hacking_results = pd.concat(
  Parallel(n_jobs=n_cores)
  (delayed(compute_portfolio_returns)(x, y, z, w)
   for x, y, z, w in p_hacking_setup)
)
p_hacking_results = p_hacking_results.reset_index(name="size_premium")
```

10.6 Size-Premium Variation

We provide a graph in Figure 10.3 that shows the different premiums. The figure also shows the relation to the average Fama-French SMB (small minus big) premium used in the literature, which we include as a dotted vertical line.

10.7 Key Takeaways

- Firm size is a crucial factor in asset pricing, and sorting stocks by size reveals the size premium, where small-cap stocks tend to outperform large-cap stocks.

```
p_hacking_results_figure = (
  ggplot(
    p_hacking_results,
    aes(x="size_premium")
  )
  + geom_histogram(bins=len(p_hacking_results))
  + scale_x_continuous(labels=percent_format())
  + labs(
      x="", y="",
      title="Distribution of size premiums for various sorting choices"
    )
  + geom_vline(
      aes(xintercept=factors_ff3_monthly["smb"].mean()), linetype="dashed"
    )
)
p_hacking_results_figure.show()
```

Figure 10.3

- Portfolio returns are sensitive to research design choices like exchange filters, weighting schemes, and the number of portfolios—decisions that can meaningfully shift results.
- Methodological flexibility can lead to non-standard errors and potential p-hacking.
- Validate results by varying assumptions and show that findings hold across multiple specifications.

11 Value and Bivariate Sorts

In this chapter, we extend the univariate portfolio analysis of Univariate Portfolio Sorts to bivariate portfolio sorting, in which stocks are assigned to portfolios based on two characteristics. Bivariate sorts are commonly used in the academic asset pricing literature and underpin the factors in the Fama-French three-factor model. However, some scholars also use sorts with three grouping variables. Conceptually, portfolio sorts are easily applicable in higher dimensions.

We form portfolios on firm size and the book-to-market ratio. Calculating book-to-market ratios requires accounting data, which necessitates additional steps during portfolio formation. In the end, we demonstrate how to form portfolios on two sorting variables using so-called independent and dependent portfolio sorts.

```
import pandas as pd
import numpy as np
import datetime as dt
import sqlite3
```

11.1 Data Preparation

First, we load the necessary data from our SQLite database introduced in Accessing and Managing Financial Data and DataCore Data. We conduct portfolio sorts based on our sample but keep only the necessary columns in our memory. We use the same data sources for firm size as in Size Sorts and P-Hacking.

```
tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")

prices_monthly = (pd.read_sql_query(
    sql=("SELECT symbol, date, ret_excess, mktcap, "
         "mktcap_lag, exchange FROM prices_monthly"),
    con=tidy_finance,
    parse_dates={"date"})
  .dropna()
)
```

Further, we utilize accounting data. We only need book equity data in this application, which we select from our database. Additionally, we convert the variable `datadate` to its monthly value, as we only consider monthly returns here and do not need to account for the exact date.

```
book_equity = (pd.read_sql_query(
    sql="SELECT symbol, datadate, be FROM datacore",
    con=tidy_finance,
    parse_dates={"datadate"})
  .dropna()
  .assign(
    date=lambda x: (
      pd.to_datetime(x["datadate"]).dt.to_period("M").dt.to_timestamp()
    )
  )
)
```

11.2 Book-to-Market Ratio

A fundamental problem in handling accounting data is the *look-ahead bias*; we must not include data in forming a portfolio that was not available knowledge at the time. Of course, researchers have more information when looking into the past than agents actually had at that moment. However, abnormal excess returns from a trading strategy should not rely on an information advantage because the differential cannot be the result of informed agents' trades. Hence, we have to lag accounting information.

As in the previous chapter, we continue to lag firm size by one month. Then, we compute the book-to-market ratio, which relates a firm's book equity to its market equity. Firms with high (low) book-to-market ratio are called value (growth) firms. After matching the accounting and market equity information from the same month, we lag book-to-market by six months. This is a sufficiently conservative approach because accounting information is usually released well before six months pass. However, in the asset pricing literature, even longer lags are used as well.

Having both variables, i.e., firm size lagged by one month and book-to-market lagged by six months, we merge these sorting variables to our returns using the `sorting_date`-column created for this purpose. The final step in our data preparation deals with differences in the frequency of our variables. Returns and firm size are recorded monthly. Yet, the accounting information is only released on an annual basis. Hence, we only match book-to-market to one month per year and have eleven empty observations. To solve this frequency issue, we carry the latest book-to-market ratio of each firm to the subsequent months, i.e., we fill the missing

observations with the most current report. This is done via the `fillna(method="ffill")`-function after sorting by date and firm (which we identify by `symbol` and `symbol`) and on a firm basis (which we do by `.groupby()` as usual). We filter out all observations with accounting data that is older than a year. As the last step, we remove all rows with missing entries because the returns cannot be matched to any annual report.

```
size = (prices_monthly
  .assign(sorting_date=lambda x: x["date"]+pd.DateOffset(months=1))
  .rename(columns={"mktcap": "size"})
  .get(["symbol", "sorting_date", "size"])
)

bm = (book_equity
  .merge(prices_monthly, how="inner", on=["symbol", "date"])
  .assign(bm=lambda x: x["be"]/x["mktcap"],
          sorting_date=lambda x: x["date"]+pd.DateOffset(months=6))
  .assign(accounting_date=lambda x: x["sorting_date"])
  .get(["symbol", "symbol", "sorting_date", "accounting_date", "bm"])
)

data_for_sorts = (prices_monthly
  .merge(bm,
         how="left",
         left_on=["symbol", "symbol", "date"],
         right_on=["symbol", "symbol", "sorting_date"])
  .merge(size,
         how="left",
         left_on=["symbol", "date"],
         right_on=["symbol", "sorting_date"])
  .get(["symbol", "symbol", "date", "ret_excess",
        "mktcap_lag", "size", "bm", "exchange", "accounting_date"])
)

data_for_sorts = (data_for_sorts
  .sort_values(by=["symbol", "symbol", "date"])
  .groupby(["symbol", "symbol"])
  .apply(lambda x: x.assign(
      bm=x["bm"].fillna(method="ffill"),
      accounting_date=x["accounting_date"].fillna(method="ffill")
    )
  )
  .reset_index(drop=True)
  .assign(threshold_date = lambda x: (x["date"]-pd.DateOffset(months=12)))
```

```
  .query("accounting_date > threshold_date")
  .drop(columns=["accounting_date", "threshold_date"])
  .dropna()
)
```

The last step of preparation for the portfolio sorts is the computation of breakpoints. We continue to use the same function, allowing for the specification of exchanges to be used for the breakpoints. Additionally, we reintroduce the argument `sorting_variable` into the function for defining different sorting variables.

```
def assign_portfolio(data, exchanges, sorting_variable, n_portfolios):
    """Assign portfolio for a given sorting variable."""

    breakpoints = (data
      .query(f"exchange in {exchanges}")
      .get(sorting_variable)
      .quantile(np.linspace(0, 1, num=n_portfolios+1),
                interpolation="linear")
      .drop_duplicates()
    )
    breakpoints.iloc[0] = -np.inf
    breakpoints.iloc[breakpoints.size-1] = np.inf

    assigned_portfolios = pd.cut(
      data[sorting_variable],
      bins=breakpoints,
      labels=range(1, breakpoints.size),
      include_lowest=True,
      right=False
    )

    return assigned_portfolios
```

After these data preparation steps, we present bivariate portfolio sorts on an independent and dependent basis.

11.3 Independent Sorts

Bivariate sorts create portfolios within a two-dimensional space spanned by two sorting variables. It is then possible to assess the return impact of either sorting variable by the return

differential from a trading strategy that invests in the portfolios at either end of the respective variables spectrum. We create a five-by-five matrix using book-to-market and firm size as sorting variables in our example below. We end up with 25 portfolios. Since we are interested in the *value premium* (i.e., the return differential between high and low book-to-market firms), we go long the five portfolios of the highest book-to-market firms and short the five portfolios of the lowest book-to-market firms. The five portfolios at each end are due to the size splits we employed alongside the book-to-market splits.

To implement the independent bivariate portfolio sort, we assign monthly portfolios for each of our sorting variables separately to create the variables `portfolio_bm` and `portfolio_size`, respectively. Then, these separate portfolios are combined to the final sort stored in `portfolio_combined`. After assigning the portfolios, we compute the average return within each portfolio for each month. Additionally, we keep the book-to-market portfolio as it makes the computation of the value premium easier. The alternative would be to disaggregate the combined portfolio in a separate step. Notice that we weigh the stocks within each portfolio by their market capitalization, i.e., we decide to value-weight our returns.

```
value_portfolios = (data_for_sorts
  .groupby("date")
  .apply(lambda x: x.assign(
      portfolio_bm=assign_portfolio(
        data=x, sorting_variable="bm", n_portfolios=5, exchanges=["HOSE"]
      ),
      portfolio_size=assign_portfolio(
        data=x, sorting_variable="size", n_portfolios=5, exchanges=["HOSE"]
      )
    )
  )
  .reset_index(drop=True)
  .groupby(["date", "portfolio_bm", "portfolio_size"])
  .apply(lambda x: pd.Series({
      "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
    })
  )
  .reset_index()
)
```

Equipped with our monthly portfolio returns, we are ready to compute the value premium. However, we still have to decide how to invest in the five high and the five low book-to-market portfolios. The most common approach is to weigh these portfolios equally, but this is yet another researcher's choice. Then, we compute the return differential between the high and low book-to-market portfolios and show the average value premium.

```
value_premium = (value_portfolios
  .groupby(["date", "portfolio_bm"])
  .aggregate({"ret": "mean"})
  .reset_index()
  .groupby("date")
  .apply(lambda x: pd.Series({
    "value_premium": (
        x.loc[x["portfolio_bm"] == x["portfolio_bm"].max(), "ret"].mean() -
          x.loc[x["portfolio_bm"] == x["portfolio_bm"].min(), "ret"].mean()
      )
    })
  )
  .aggregate({"value_premium": "mean"})
)
```

11.4 Dependent Sorts

In the previous exercise, we assigned the portfolios without considering the second variable in the assignment. This protocol is called independent portfolio sorts. The alternative, i.e., dependent sorts, creates portfolios for the second sorting variable within each bucket of the first sorting variable. In our example below, we sort firms into five size buckets, and within each of those buckets, we assign firms to five book-to-market portfolios. Hence, we have monthly breakpoints that are specific to each size group. The decision between independent and dependent portfolio sorts is another choice for the researcher. Notice that dependent sorts guarantee that portfolios have roughly equal numbers of stocks when breakpoints are computed from all exchanges. However, if breakpoints are based only on HOSE stocks, portfolio counts will generally be uneven — reflecting the large presence of small-cap stocks on HNX and UPCoM (see Exercise below).

To implement the dependent sorts, we first create the size portfolios by calling `assign_portfolio()` with `sorting_variable="me"`. Then, we group our data again by month and by the size portfolio before assigning the book-to-market portfolio. The rest of the implementation is the same as before. Finally, we compute the value premium.

```
value_portfolios = (data_for_sorts
  .groupby("date")
  .apply(lambda x: x.assign(
      portfolio_size=assign_portfolio(
        data=x, sorting_variable="size", n_portfolios=5, exchanges=["HOSE"]
      )
    )
```

```
  )
  .reset_index(drop=True)
  .groupby(["date", "portfolio_size"])
  .apply(lambda x: x.assign(
      portfolio_bm=assign_portfolio(
        data=x, sorting_variable="bm", n_portfolios=5, exchanges=["HOSE"]
      )
    )
  )
  .reset_index(drop=True)
  .groupby(["date", "portfolio_bm", "portfolio_size"])
  .apply(lambda x: pd.Series({
      "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
    })
  )
  .reset_index()
)

value_premium = (value_portfolios
  .groupby(["date", "portfolio_bm"])
  .aggregate({"ret": "mean"})
  .reset_index()
  .groupby("date")
  .apply(lambda x: pd.Series({
    "value_premium": (
        x.loc[x["portfolio_bm"] == x["portfolio_bm"].max(), "ret"].mean() -
          x.loc[x["portfolio_bm"] == x["portfolio_bm"].min(), "ret"].mean()
      )
    })
  )
  .aggregate({"value_premium": "mean"})
)
```

Overall, we show how to conduct bivariate portfolio sorts in this chapter. In one case, we sort the portfolios independently of each other. Yet we also discuss how to create dependent portfolio sorts. Along the lines of Size Sorts, we see how many choices a researcher has to make to implement portfolio sorts, and bivariate sorts increase the number of choices.

11.5 Key Takeaways

- Bivariate portfolio sorts assign stocks based on two characteristics, such as firm size and book-to-market ratio, to better capture return patterns in asset pricing.
- Independent sorts treat each variable separately, while dependent sorts condition the second sort on the first.
- Proper handling of accounting data, especially lagging the book-to-market ratio, is essential to avoid look-ahead bias and ensure valid backtesting.
- Value premiums are derived by comparing returns of high versus low book-to-market portfolios, with results sensitive to sorting choices and weighting schemes.

12 Portfolio Weighting and Rebalancing

> **Note**
>
> In this chapter, we systematically compare portfolio weighting schemes (e.g., value-weighted, equal-weighted, and several risk-based alternativesin the Vietnamese equity market. We quantify the impact of rebalancing frequency and transaction costs on realized performance, and develop practical tools for constructing implementable portfolios under the frictions characteristic of an emerging market.

Every portfolio construction decision ultimately reduces to two choices: which assets to hold, and how much to allocate to each. While earlier chapters have focused on the first question, using factor models, anomalies, and fundamental analysis to select stocks, this chapter addresses the second. The weighting scheme a researcher or investor applies can fundamentally alter the conclusions drawn from portfolio-level tests and the returns earned from an investment strategy.

The distinction matters more in Vietnam than in large, liquid markets. The Vietnamese equity market features extreme skewness in the market capitalization distribution: the top 10 firms on HOSE account for roughly 50% of total market capitalization, while hundreds of small firms contribute negligible weight. Under value-weighting, a portfolio's performance is dominated by a handful of large-cap names (Vinhomes, Vingroup, Vietcombank, FPT). Under equal-weighting, every firm contributes equally, tilting the portfolio toward small, illiquid stocks that may be expensive or impossible to trade at scale. Neither scheme is inherently correct; the choice depends on the question being asked.

This chapter develops the analytical framework for making that choice. We begin with the theoretical properties of weighting schemes, implement each scheme in practice with Vietnamese data, quantify the transaction costs of rebalancing, and extend the analysis to risk-based alternatives that explicitly incorporate the covariance structure of returns.

12.1 Theoretical Framework

12.1.1 Value-Weighted Portfolios

A value-weighted (VW) portfolio allocates to each stock in proportion to its market capitalization:

$$w_{i,t}^{VW} = \frac{\text{MCap}_{i,t}}{\sum_{j=1}^{N_t} \text{MCap}_{j,t}} \tag{12.1}$$

where $\text{MCap}_{i,t} = P_{i,t} \times \text{Shares}_{i,t}$ is the market capitalization of stock i at time t.

The VW portfolio has a unique theoretical status: it is the portfolio that all investors collectively hold (the "market portfolio" in CAPM). Its key properties are:

1. **Self-rebalancing.** As prices move, weights adjust automatically. A VW portfolio requires trading only when constituents enter or leave the index, or when corporate actions (splits, issuances) change shares outstanding.
2. **Low turnover.** Because weights drift with prices rather than being reset to targets, VW portfolios have minimal rebalancing costs.
3. **Large-cap bias.** Returns are dominated by the largest firms. In Vietnam, this means the portfolio's risk-return profile is heavily influenced by banking, real estate, and technology conglomerates.

Hsu (2006) argues that VW portfolios are sub-optimal because they mechanically overweight overpriced stocks and underweight underpriced stocks (i.e., any deviation of price from fundamental value creates a systematic drag on VW performance relative to a fundamentally weighted alternative).

12.1.2 Equal-Weighted Portfolios

An equal-weighted (EW) portfolio assigns the same weight to each constituent:

$$w_{i,t}^{EW} = \frac{1}{N_t} \tag{12.2}$$

DeMiguel, Garlappi, and Uppal (2009) show that the 1/N portfolio is surprisingly competitive with mean-variance optimized portfolios, particularly when estimation windows are short and the number of assets is large (conditions that closely describe the Vietnamese market). The intuition is that estimation error in expected returns and covariances can overwhelm the gains from optimization, making the "naive" equal-weight scheme a robust default.

Plyakha, Uppal, and Vilkov (2021) decompose the EW outperformance over VW into two components:

1. **Size tilt.** EW allocates more to small firms, which historically earn a size premium.
2. **Rebalancing bonus.** Monthly rebalancing back to equal weights is a contrarian strategy: it sells recent winners and buys recent losers, profiting from mean reversion in individual stock returns.

However, the EW portfolio has practical disadvantages that are particularly severe in Vietnam:

- **High turnover.** Every rebalancing date requires trading every stock back to equal weight.
- **Illiquidity exposure.** Equal weighting of micro-cap stocks that trade VND 100 million/day alongside large-caps trading VND 500 billion/day creates severe implementation challenges.
- **Price impact.** In a market with daily price limits ($\pm$ 7% on HOSE, $\pm$ 10% on HNX), rebalancing trades for illiquid names may hit limit-up or limit-down, preventing full execution.

12.1.3 The Weighting Spectrum

Between VW and EW lies a continuum of weighting schemes. Table 12.1 summarizes the major alternatives.

Table 12.1: Summary of portfolio weighting schemes.

Scheme	Weight Formula	Key Property	Key Risk
Value-weighted	$w_i \propto \text{MCap}_i$	Self-rebalancing, low turnover	Large-cap concentration
Equal-weighted	$w_i = 1/N$	Maximum naive diversification	High turnover, illiquidity
Fundamental	$w_i \propto F_i$ (revenue, book equity, etc.)	Breaks price-value link	Requires accounting data
Minimum variance	$\mathbf{w} = \arg\min \mathbf{w}'\Sigma\mathbf{w}$	Lowest portfolio volatility	Estimation error in Σ
Risk parity	$w_i\sigma_i = w_j\sigma_j \; \forall\, i, j$	Equal risk contribution	Leverages low-vol assets
Maximum diversification	$\max \frac{\mathbf{w}'\sigma}{\sqrt{\mathbf{w}'\Sigma\mathbf{w}}}$	Maximizes diversification ratio	Sensitive to correlation estimates
Capped VW	$w_i \propto \text{MCap}_i$, $w_i \leq \bar{w}$	Reduces concentration	Arbitrary cap threshold

12.2 Data Construction

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from scipy.optimize import minimize
from sklearn.covariance import LedoitWolf
from linearmodels.panel import PanelOLS
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.figsize': (10, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

```
from datacore import DataCoreClient

client = DataCoreClient()

# Daily prices and volume
daily = client.get_daily_prices(
    exchanges=['HOSE', 'HNX'],
    start_date='2012-01-01',
    end_date='2024-12-31',
    fields=[
        'ticker', 'date', 'close', 'adjusted_close', 'volume',
        'turnover_value', 'market_cap', 'shares_outstanding',
        'bid_ask_spread', 'free_float_pct'
    ]
)

# Monthly returns (pre-computed for convenience)
monthly = client.get_monthly_returns(
    exchanges=['HOSE', 'HNX'],
    start_date='2012-01-01',
```

```
    end_date='2024-12-31',
    fields=[
        'ticker', 'month_end', 'monthly_return', 'market_cap',
        'volume_avg_20d', 'turnover_value_avg_20d'
    ]
)

# Fundamentals for fundamental weighting
fundamentals = client.get_fundamentals(
    exchanges=['HOSE', 'HNX'],
    start_date='2012-01-01',
    end_date='2024-12-31',
    frequency='annual',
    fields=[
        'ticker', 'fiscal_year', 'revenue', 'book_equity',
        'total_assets', 'dividends_paid', 'operating_cash_flow'
    ]
)

# Market-level returns for benchmarking
market_index = client.get_index(
    index='VNINDEX',
    start_date='2012-01-01',
    end_date='2024-12-31',
    frequency='monthly'
)

print(f"Daily observations: {daily.shape[0]:,}")
print(f"Monthly observations: {monthly.shape[0]:,}")
print(f"Unique tickers: {monthly['ticker'].nunique()}")
```

12.2.1 Universe Construction and Liquidity Filters

A critical pre-processing step is defining the investable universe. Including all listed stocks, regardless of liquidity, inflates the apparent benefits of equal-weighting and other small-cap-tilted schemes because it implicitly assumes the ability to trade illiquid stocks without friction. We apply graduated liquidity filters and track how results change.

```
def construct_universe(monthly_df, min_mcap_pct=0, min_turnover=0,
 ↪  min_months=12):
    """
```

```
    Construct investable universe with liquidity filters.

    Parameters
    ----------
    min_mcap_pct : float
        Exclude stocks below this market cap percentile (0-100).
    min_turnover : float
        Minimum average daily turnover in VND billion.
    min_months : int
        Minimum months of return history required.
    """
    df = monthly_df.copy()

    # Market cap percentile filter (within each month)
    if min_mcap_pct > 0:
        df["mcap_pctile"] = df.groupby("month_end")["market_cap"].transform(
            lambda x: x.rank(pct=True) * 100
        )
        df = df[df["mcap_pctile"] >= min_mcap_pct]

    # Turnover filter
    if min_turnover > 0:
        df = df[df["turnover_value_avg_20d"] >= min_turnover * 1e9]

    # History filter
    ticker_months = df.groupby("ticker")["month_end"].transform("count")
    df = df[ticker_months >= min_months]

    return df


# Define three universes of increasing restrictiveness
universe_all = construct_universe(monthly)
universe_mid = construct_universe(monthly, min_mcap_pct=20, min_turnover=0.5)
universe_liquid = construct_universe(monthly, min_mcap_pct=40,
 ↪  min_turnover=2.0)

for name, univ in [
    ("All stocks", universe_all),
    ("Mid filter", universe_mid),
    ("Liquid only", universe_liquid),
]:
    n_stocks = univ.groupby("month_end")["ticker"].nunique().median()
```

```
    print(f"{name}: median {n_stocks:.0f} stocks/month")
```

12.2.2 Market Capitalization Concentration

Before comparing weighting schemes, it is instructive to document how concentrated the Vietnamese market actually is.

12.3 Implementing Weighting Schemes

We now implement each weighting scheme and compute monthly portfolio returns. All implementations follow a common structure: at each rebalancing date, compute target weights from available information, then compute the weighted return over the subsequent holding period.

12.3.1 Core Portfolio Engine

```
def compute_portfolio_returns(
    monthly_df,
    weight_fn,
    rebal_freq="M",
    max_weight=1.0,
    min_weight=0.0,
):
    """
    Compute time series of portfolio returns for a given weighting function.

    Parameters
    ----------
    monthly_df : DataFrame
        Must contain 'ticker', 'month_end', 'monthly_return', and any
        columns needed by weight_fn.
    weight_fn : callable
        Function that takes a cross-section DataFrame and returns a
        Series of weights indexed by ticker. Weights need not sum to 1
        (they will be normalized).
    rebal_freq : str
        'M' for monthly, 'Q' for quarterly, 'A' for annual.
    max_weight : float
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Cumulative market cap share (latest month)
latest = monthly[monthly['month_end'] == monthly['month_end'].max()].copy()
latest = latest.sort_values('market_cap', ascending=False)
latest['cum_mcap_share'] = (
    latest['market_cap'].cumsum() / latest['market_cap'].sum()
)
latest['rank'] = range(1, len(latest) + 1)
latest['rank_pct'] = latest['rank'] / len(latest) * 100

axes[0].plot(latest['rank_pct'], latest['cum_mcap_share'] * 100,
             color='#2C5F8A', linewidth=2)
axes[0].axhline(y=50, color='gray', linestyle='--', linewidth=0.8)
axes[0].axhline(y=80, color='gray', linestyle='--', linewidth=0.8)

# Mark top 10 and top 30
n_at_50 = (latest['cum_mcap_share'] <= 0.50).sum()
axes[0].annotate(f'Top {n_at_50} stocks = 50%',
                 xy=(n_at_50 / len(latest) * 100, 50),
                 fontsize=9, color='#C0392B')
axes[0].set_xlabel('Cumulative Stock Rank (%)')
axes[0].set_ylabel('Cumulative Market Cap Share (%)')
axes[0].set_title('Panel A: Market Cap Concentration Curve')

# Panel B: HHI over time
hhi_ts = (
    monthly
    .groupby('month_end')
    .apply(lambda g: (g['market_cap'] / g['market_cap'].sum()).pow(2).sum())
    .reset_index(name='hhi')
)
hhi_ts['month_end'] = pd.to_datetime(hhi_ts['month_end'])
axes[1].plot(hhi_ts['month_end'], hhi_ts['hhi'] * 10000,
             color='#2C5F8A', linewidth=1.5)
axes[1].set_xlabel('Date')
axes[1].set_ylabel('HHI (basis points)')
axes[1].set_title('Panel B: Herfindahl Index of VW Weights')

plt.tight_layout()
plt.show()
```

Figure 12.1

```
        Maximum weight per stock (for capped schemes).
    min_weight : float
        Minimum weight per stock.

    Returns
    -------
    DataFrame with columns: month_end, port_return, n_stocks,
    turnover, hhi, effective_n
    """
    months = sorted(monthly_df["month_end"].unique())

    # Determine rebalancing dates
    if rebal_freq == "M":
        rebal_dates = set(months)
    elif rebal_freq == "Q":
        rebal_dates =
↪   set(pd.to_datetime(months).to_period("Q").to_timestamp("M"))
        # Map to nearest month-end
        rebal_dates = {m for m in months if pd.Timestamp(m).month % 3 == 0}
        if not rebal_dates:
            rebal_dates = set(months[::3])
    elif rebal_freq == "A":
        rebal_dates = {m for m in months if pd.Timestamp(m).month == 6}
        if not rebal_dates:
            rebal_dates = set(months[::12])
    else:
        rebal_dates = set(months)

    results = []
    prev_weights = None

    for month in months:
        cross_section = monthly_df[monthly_df["month_end"] == month].copy()
        cross_section = cross_section.dropna(subset=["monthly_return"])

        if len(cross_section) < 5:
            continue

        if month in rebal_dates or prev_weights is None:
            # Compute fresh weights
            raw_weights = weight_fn(cross_section)
            raw_weights = raw_weights.clip(lower=min_weight,
↪   upper=max_weight)
```

```
            total = raw_weights.sum()
            if total <= 0:
                continue
            target_weights = raw_weights / total
        else:
            # Drift weights forward from previous month
            if prev_weights is not None:
                available = cross_section.set_index("ticker")
                drifted = prev_weights.reindex(available.index, fill_value=0)
                # Adjust for returns
                drifted = drifted * (1 + available["monthly_return"])
                total = drifted.sum()
                if total <= 0:
                    continue
                target_weights = drifted / total
            else:
                continue

        # Align weights with available stocks
        cross_section = cross_section.set_index("ticker")
        aligned_w = target_weights.reindex(cross_section.index, fill_value=0)
        aligned_w = aligned_w / aligned_w.sum()

        # Portfolio return
        port_ret = (aligned_w * cross_section["monthly_return"]).sum()

        # Turnover (two-way)
        if prev_weights is not None:
            prev_aligned = prev_weights.reindex(aligned_w.index,
↪   fill_value=0)
            # Drift previous weights
            prev_drifted = prev_aligned * (
                1
                + cross_section["monthly_return"].reindex(
                    prev_aligned.index, fill_value=0
                )
            )
            prev_drifted = (
                prev_drifted / prev_drifted.sum()
                if prev_drifted.sum() > 0
                else prev_drifted
            )
            turnover = (aligned_w - prev_drifted).abs().sum() / 2
```

```
        else:
            turnover = 1.0

        # Concentration metrics
        hhi = (aligned_w**2).sum()
        effective_n = 1.0 / hhi if hhi > 0 else 0

        results.append(
            {
                "month_end": month,
                "port_return": port_ret,
                "n_stocks": (aligned_w > 1e-6).sum(),
                "turnover": turnover,
                "hhi": hhi,
                "effective_n": effective_n,
            }
        )

        prev_weights = aligned_w.copy()

    return pd.DataFrame(results)
```

12.3.2 Value-Weighted Portfolio

```
def vw_weights(cross_section):
    """Value-weighted: proportional to market cap."""
    return cross_section.set_index('ticker')['market_cap']

vw_returns = compute_portfolio_returns(universe_mid, vw_weights,
 ↪  rebal_freq='M')
print(f"VW portfolio: {len(vw_returns)} months")
print(f"Mean monthly return: {vw_returns['port_return'].mean():.4f}")
print(f"Mean turnover: {vw_returns['turnover'].mean():.4f}")
print(f"Mean effective N: {vw_returns['effective_n'].mean():.1f}")
```

12.3.3 Equal-Weighted Portfolio

```
def ew_weights(cross_section):
    """Equal-weighted: 1/N."""
    tickers = cross_section.set_index('ticker').index
    return pd.Series(1.0, index=tickers)

# Monthly rebalancing
ew_monthly = compute_portfolio_returns(
    universe_mid, ew_weights, rebal_freq='M'
)

# Quarterly rebalancing
ew_quarterly = compute_portfolio_returns(
    universe_mid, ew_weights, rebal_freq='Q'
)

# Annual rebalancing (June)
ew_annual = compute_portfolio_returns(
    universe_mid, ew_weights, rebal_freq='A'
)

for name, df in [('EW Monthly', ew_monthly),
                 ('EW Quarterly', ew_quarterly),
                 ('EW Annual', ew_annual)]:
    print(f"{name}: mean ret = {df['port_return'].mean():.4f}, "
          f"turnover = {df['turnover'].mean():.4f}")
```

12.3.4 Capped Value-Weighted Portfolio

To mitigate the concentration of pure VW while retaining its low-turnover properties, we impose a cap on individual stock weights. A common choice is 5% or 10%, mimicking the construction rules of capped indices such as the MSCI Capped indices.

```
def capped_vw_weights(cross_section, cap=0.05):
    """Capped VW: market cap weights with an upper bound."""
    w = cross_section.set_index('ticker')['market_cap']
    w = w / w.sum()
    # Iterative capping (redistribute excess weight)
    for _ in range(20):
        excess = w[w > cap] - cap
        if excess.sum() <= 1e-8:
            break
```

```
        w[w > cap] = cap
        w[w <= cap] *= (1 + excess.sum() / w[w <= cap].sum())
    return w

capped5 = compute_portfolio_returns(
    universe_mid, lambda cs: capped_vw_weights(cs, 0.05), rebal_freq='M'
)
capped10 = compute_portfolio_returns(
    universe_mid, lambda cs: capped_vw_weights(cs, 0.10), rebal_freq='M'
)

print(f"Capped 5%: eff_N = {capped5['effective_n'].mean():.1f}, "
      f"turnover = {capped5['turnover'].mean():.4f}")
print(f"Capped 10%: eff_N = {capped10['effective_n'].mean():.1f}, "
      f"turnover = {capped10['turnover'].mean():.4f}")
```

12.3.5 Fundamental-Weighted Portfolio

Arnott, Hsu, and Moore (2005) propose weighting stocks by fundamental measures (revenue, book equity, dividends, cash flow) rather than market cap. The logic is that fundamental weights are not contaminated by pricing errors, breaking the mechanical overweighting of overvalued stocks inherent in VW. We construct a composite fundamental weight using the average rank across four measures:

$$w_{i,t}^{FW} \propto \frac{1}{4}\left(\text{Rank}_{i,t}^{\text{Rev}} + \text{Rank}_{i,t}^{\text{BE}} + \text{Rank}_{i,t}^{\text{Div}} + \text{Rank}_{i,t}^{\text{CFO}}\right) \tag{12.3}$$

```
# Merge fundamentals with monthly data (use most recent fiscal year)
fundamentals["merge_year"] = fundamentals["fiscal_year"] + 1  # Lag by 1 year
monthly_fund = monthly.copy()
monthly_fund["year"] = pd.to_datetime(monthly_fund["month_end"]).dt.year

monthly_fund = monthly_fund.merge(
    fundamentals.rename(columns={"merge_year": "year"}),
    on=["ticker", "year"],
    how="left",
)


def fw_weights(cross_section):
    """Fundamental-weighted: composite of revenue, book equity, dividends,
     ↪ CFO."""
```

```
    cs = cross_section.set_index("ticker")
    ranks = pd.DataFrame(index=cs.index)

    for col in ["revenue", "book_equity", "dividends_paid",
     ↪  "operating_cash_flow"]:
        if col in cs.columns:
            vals = cs[col].clip(lower=0)  # Only positive values
            ranks[col] = vals.rank(pct=True)

    composite = ranks.mean(axis=1)
    composite = composite.fillna(0)
    return composite


fw_returns = compute_portfolio_returns(monthly_fund, fw_weights,
 ↪  rebal_freq="A")
print(
    f"Fundamental-weighted: mean ret =
     ↪  {fw_returns['port_return'].mean():.4f}, "
    f"eff_N = {fw_returns['effective_n'].mean():.1f}"
)
```

12.4 Risk-Based Weighting Schemes

The weighting schemes above use only market cap or accounting data. Risk-based schemes incorporate the covariance structure of returns, aiming to produce portfolios with better risk-adjusted performance. The trade-off is that they require estimating the covariance matrix (i.e., a high-dimensional object that is notoriously difficult to estimate precisely with short time series).

12.4.1 Covariance Estimation

With $N \approx 300$ stocks and $T \approx 60$ months, the sample covariance matrix is severely ill-conditioned. We use the Ledoit and Wolf (2004) shrinkage estimator, which pulls the sample covariance toward a structured target (the identity matrix scaled by the average variance):

$$\hat{\Sigma}^{\text{shrink}} = \delta \mathbf{F} + (1 - \delta)\mathbf{S} \tag{12.4}$$

where $\mathbf{S}$ is the sample covariance, $\mathbf{F}$ is the shrinkage target, and $\delta \in [0, 1]$ is the optimal shrinkage intensity derived analytically.

```
def estimate_covariance(monthly_df, month, lookback=60, min_obs=36):
    """
    Estimate covariance matrix using Ledoit-Wolf shrinkage.

    Parameters
    ----------
    monthly_df : DataFrame with ticker, month_end, monthly_return
    month : target month (use returns before this date)
    lookback : number of months to use
    min_obs : minimum observations per stock

    Returns
    -------
    cov_matrix : DataFrame (N x N)
    tickers : list of tickers with sufficient data
    """
    end_date = pd.Timestamp(month)
    start_date = end_date - pd.DateOffset(months=lookback)

    window = monthly_df[
        (monthly_df['month_end'] > start_date) &
        (monthly_df['month_end'] <= end_date)
    ]

    # Pivot to wide format
    returns_wide = window.pivot_table(
        index='month_end', columns='ticker', values='monthly_return'
    )

    # Keep stocks with sufficient observations
    valid_cols = returns_wide.columns[returns_wide.notna().sum() >= min_obs]
    returns_wide = returns_wide[valid_cols].dropna(axis=0, how='all')

    # Fill remaining NAs with 0 (conservative)
    returns_clean = returns_wide.fillna(0)

    if returns_clean.shape[1] < 10:
        return None, None

    # Ledoit-Wolf shrinkage
    lw = LedoitWolf()
    lw.fit(returns_clean.values)
```

```
    cov_matrix = pd.DataFrame(
        lw.covariance_,
        index=valid_cols, columns=valid_cols
    )

    return cov_matrix, list(valid_cols)
```

12.4.2 Minimum Variance Portfolio

The global minimum variance (GMV) portfolio minimizes portfolio variance without targeting a specific return level (Clarke, De Silva, and Thorley 2011):

$$\mathbf{w}^{MV} = \arg\min_{\mathbf{w}} \ \mathbf{w}'\hat{\Sigma}\mathbf{w} \quad \text{s.t.} \quad \mathbf{1}'\mathbf{w} = 1, \ w_i \geq 0 \tag{12.5}$$

The long-only constraint ($w_i \geq 0$) is essential in practice and also acts as an implicit shrinkage that improves out-of-sample performance (Jagannathan and Ma 2003).

```
def minimum_variance_weights(cov_matrix, max_weight=0.05):
    """
    Solve for the minimum variance portfolio with long-only
    and position-size constraints.
    """
    n = cov_matrix.shape[0]
    Sigma = cov_matrix.values

    def portfolio_variance(w):
        return w @ Sigma @ w

    constraints = [
        {'type': 'eq', 'fun': lambda w: np.sum(w) - 1}
    ]
    bounds = [(0, max_weight) for _ in range(n)]
    x0 = np.ones(n) / n

    result = minimize(
        portfolio_variance, x0,
        method='SLSQP',
        bounds=bounds,
        constraints=constraints,
        options={'maxiter': 1000, 'ftol': 1e-12}
```

```
    )

    if result.success:
        return pd.Series(result.x, index=cov_matrix.index)
    else:
        return pd.Series(1.0 / n, index=cov_matrix.index)

def mv_weight_fn(cross_section, cov_cache={}):
    """Wrapper for portfolio engine: minimum variance."""
    month = cross_section['month_end'].iloc[0]

    if month not in cov_cache:
        cov_matrix, tickers = estimate_covariance(monthly, month)
        cov_cache[month] = (cov_matrix, tickers)

    cov_matrix, tickers = cov_cache[month]
    if cov_matrix is None:
        # Fallback to equal weight
        return pd.Series(1.0, index=cross_section.set_index('ticker').index)

    # Restrict to stocks in both cross-section and covariance matrix
    available = set(cross_section['ticker']) & set(tickers)
    if len(available) < 10:
        return pd.Series(1.0, index=cross_section.set_index('ticker').index)

    sub_cov = cov_matrix.loc[list(available), list(available)]
    weights = minimum_variance_weights(sub_cov)

    return weights

mv_returns = compute_portfolio_returns(
    universe_mid, mv_weight_fn, rebal_freq='Q'
)
print(f"Min Variance: mean ret = {mv_returns['port_return'].mean():.4f}, "
      f"std = {mv_returns['port_return'].std():.4f}")
```

12.4.3 Risk Parity (Equal Risk Contribution)

Risk parity allocates so that each asset contributes equally to total portfolio risk (Maillard, Roncalli, and Teïletche 2010). The risk contribution of asset i is:

$$RC_i = w_i \cdot \frac{(\Sigma \mathbf{w})_i}{\sqrt{\mathbf{w}' \Sigma \mathbf{w}}} \tag{12.6}$$

The risk parity portfolio solves $RC_i = RC_j$ for all i, j:

```
def risk_parity_weights(cov_matrix, max_weight=0.05):
    """
    Solve for the risk parity portfolio where each asset
    contributes equally to total portfolio variance.
    """
    n = cov_matrix.shape[0]
    Sigma = cov_matrix.values

    def risk_parity_objective(w):
        port_var = w @ Sigma @ w
        marginal = Sigma @ w
        risk_contrib = w * marginal
        target_rc = port_var / n
        return np.sum((risk_contrib - target_rc) ** 2)

    constraints = [
        {'type': 'eq', 'fun': lambda w: np.sum(w) - 1}
    ]
    bounds = [(1e-6, max_weight) for _ in range(n)]
    x0 = np.ones(n) / n

    result = minimize(
        risk_parity_objective, x0,
        method='SLSQP',
        bounds=bounds,
        constraints=constraints,
        options={'maxiter': 1000, 'ftol': 1e-12}
    )

    if result.success:
        return pd.Series(result.x, index=cov_matrix.index)
    else:
        return pd.Series(1.0 / n, index=cov_matrix.index)

def rp_weight_fn(cross_section, cov_cache={}):
    """Wrapper for portfolio engine: risk parity."""
    month = cross_section['month_end'].iloc[0]
```

```
    if month not in cov_cache:
        cov_matrix, tickers = estimate_covariance(monthly, month)
        cov_cache[month] = (cov_matrix, tickers)

    cov_matrix, tickers = cov_cache[month]
    if cov_matrix is None:
        return pd.Series(1.0, index=cross_section.set_index('ticker').index)

    available = set(cross_section['ticker']) & set(tickers)
    if len(available) < 10:
        return pd.Series(1.0, index=cross_section.set_index('ticker').index)

    sub_cov = cov_matrix.loc[list(available), list(available)]
    return risk_parity_weights(sub_cov)

rp_returns = compute_portfolio_returns(
    universe_mid, rp_weight_fn, rebal_freq='Q'
)
print(f"Risk Parity: mean ret = {rp_returns['port_return'].mean():.4f}, "
      f"std = {rp_returns['port_return'].std():.4f}")
```

12.4.4 Maximum Diversification Portfolio

Choueifaty (2008) define the diversification ratio as the ratio of weighted average volatility to portfolio volatility:

$$DR(\mathbf{w}) = \frac{\mathbf{w}'\sigma}{\sqrt{\mathbf{w}'\Sigma\mathbf{w}}} \tag{12.7}$$

where σ is the vector of individual asset volatilities. The maximum diversification portfolio maximizes this ratio:

```
def max_diversification_weights(cov_matrix, max_weight=0.05):
    """Maximize the diversification ratio."""
    n = cov_matrix.shape[0]
    Sigma = cov_matrix.values
    sigma = np.sqrt(np.diag(Sigma))

    def neg_div_ratio(w):
        port_vol = np.sqrt(w @ Sigma @ w)
```

```
        if port_vol < 1e-10:
            return 0
        return -(w @ sigma) / port_vol

    constraints = [
        {'type': 'eq', 'fun': lambda w: np.sum(w) - 1}
    ]
    bounds = [(0, max_weight) for _ in range(n)]
    x0 = np.ones(n) / n

    result = minimize(
        neg_div_ratio, x0,
        method='SLSQP',
        bounds=bounds,
        constraints=constraints,
        options={'maxiter': 1000, 'ftol': 1e-12}
    )

    if result.success:
        return pd.Series(result.x, index=cov_matrix.index)
    else:
        return pd.Series(1.0 / n, index=cov_matrix.index)
```

12.5 Comprehensive Performance Comparison

We now compare all schemes on a common universe with consistent methodology.

```
# Collect all portfolio return series
portfolios = {
    'VW': vw_returns,
    'EW (Monthly)': ew_monthly,
    'EW (Quarterly)': ew_quarterly,
    'EW (Annual)': ew_annual,
    'Capped VW (5%)': capped5,
    'Capped VW (10%)': capped10,
    'Fundamental': fw_returns,
    'Min Variance': mv_returns,
    'Risk Parity': rp_returns,
}

# Align to common date range
```

```
common_start = max(df['month_end'].min() for df in portfolios.values())
common_end = min(df['month_end'].max() for df in portfolios.values())

for name in portfolios:
    portfolios[name] = portfolios[name][
        (portfolios[name]['month_end'] >= common_start) &
        (portfolios[name]['month_end'] <= common_end)
    ].copy()

print(f"Common period: {common_start} to {common_end}")
print(f"Number of months: {len(portfolios['VW'])}")
```

12.5.1 Performance Metrics

```
def compute_metrics(returns_df, risk_free_annual=0.04):
    """Compute performance metrics from monthly portfolio returns."""
    r = returns_df['port_return']
    rf_monthly = (1 + risk_free_annual) ** (1/12) - 1

    excess = r - rf_monthly
    n_months = len(r)

    # Annualized return
    cum_ret = (1 + r).prod()
    ann_ret = cum_ret ** (12 / n_months) - 1

    # Annualized volatility
    ann_vol = r.std() * np.sqrt(12)

    # Sharpe ratio
    sharpe = excess.mean() / excess.std() * np.sqrt(12) if excess.std() > 0
↪   else 0

    # Maximum drawdown
    cum = (1 + r).cumprod()
    running_max = cum.cummax()
    drawdown = (cum - running_max) / running_max
    max_dd = drawdown.min()

    # Sortino ratio
```

```
    downside = excess[excess < 0]
    downside_vol = np.sqrt((downside ** 2).mean()) * np.sqrt(12)
    sortino = excess.mean() * 12 / downside_vol if downside_vol > 0 else 0

    # Calmar ratio
    calmar = ann_ret / abs(max_dd) if max_dd != 0 else 0

    # Average turnover and effective N
    avg_turnover = returns_df['turnover'].mean()
    avg_eff_n = returns_df['effective_n'].mean()

    # Skewness and kurtosis
    skew = r.skew()
    kurt = r.kurtosis()

    return {
        'Ann. Return': ann_ret,
        'Ann. Volatility': ann_vol,
        'Sharpe Ratio': sharpe,
        'Sortino Ratio': sortino,
        'Max Drawdown': max_dd,
        'Calmar Ratio': calmar,
        'Skewness': skew,
        'Kurtosis': kurt,
        'Avg. Turnover': avg_turnover,
        'Effective N': avg_eff_n
    }

# Compute metrics for all portfolios
metrics_list = []
for name, df in portfolios.items():
    m = compute_metrics(df)
    m['Portfolio'] = name
    metrics_list.append(m)

metrics_df = pd.DataFrame(metrics_list).set_index('Portfolio')

# Format for display
display_cols = [
    'Ann. Return', 'Ann. Volatility', 'Sharpe Ratio', 'Sortino Ratio',
    'Max Drawdown', 'Avg. Turnover', 'Effective N'
]
print(metrics_df[display_cols].round(3).to_string())
```

```
fig, ax = plt.subplots(figsize=(12, 7))

colors = {
    'VW': '#2C5F8A', 'EW (Monthly)': '#E67E22',
    'EW (Quarterly)': '#F39C12', 'EW (Annual)': '#D4AC0D',
    'Capped VW (5%)': '#8E44AD', 'Capped VW (10%)': '#9B59B6',
    'Fundamental': '#27AE60', 'Min Variance': '#C0392B',
    'Risk Parity': '#1ABC9C'
}
linestyles = {
    'VW': '-', 'EW (Monthly)': '-', 'EW (Quarterly)': '--',
    'EW (Annual)': ':', 'Capped VW (5%)': '-',
    'Capped VW (10%)': '--', 'Fundamental': '-',
    'Min Variance': '-', 'Risk Parity': '-'
}

for name, df in portfolios.items():
    cum = (1 + df.set_index('month_end')['port_return']).cumprod()
    ax.plot(cum.index, cum.values, label=name,
            color=colors.get(name, 'gray'),
            linestyle=linestyles.get(name, '-'),
            linewidth=1.8 if name in ['VW', 'EW (Monthly)', 'Min Variance']
 ↪  else 1.2)

ax.set_xlabel('Date')
ax.set_ylabel('Cumulative Wealth (VND 1 invested)')
ax.set_title('Cumulative Performance by Weighting Scheme')
ax.legend(loc='upper left', fontsize=9, ncol=2)
ax.set_yscale('log')
plt.tight_layout()
plt.show()
```

Figure 12.2

12.5.2 Risk-Return Trade-Off

```
fig, ax = plt.subplots(figsize=(9, 7))

for name in metrics_df.index:
    ax.scatter(
        metrics_df.loc[name, 'Ann. Volatility'],
        metrics_df.loc[name, 'Ann. Return'],
        s=metrics_df.loc[name, 'Effective N'] * 3,
        color=colors.get(name, 'gray'),
        alpha=0.85, edgecolors='white', linewidth=1.5, zorder=5
    )
    ax.annotate(
        name,
        (metrics_df.loc[name, 'Ann. Volatility'] + 0.002,
         metrics_df.loc[name, 'Ann. Return']),
        fontsize=8
    )

ax.set_xlabel('Annualized Volatility')
ax.set_ylabel('Annualized Return')
ax.set_title('Risk-Return Profile (bubble size = Effective N)')
plt.tight_layout()
plt.show()
```

Figure 12.3

12.6 Transaction Costs and Net-of-Cost Performance

12.6.1 Estimating Trading Costs in Vietnam

Transaction costs in Vietnam include explicit components (brokerage commissions, exchange fees, taxes) and implicit components (bid-ask spread, price impact). The explicit cost structure as of 2024 is approximately:

Table 12.2: Explicit transaction cost components for Vietnamese equities.

Component	Rate	Notes
Brokerage commission	0.15–0.35%	Varies by broker and volume tier
Exchange & clearing fee	0.003%	Fixed by exchange
Selling tax	0.10%	Levied on gross sale proceeds

The total explicit round-trip cost (buy + sell) ranges from approximately 0.30% to 0.80%. Implicit costs—the spread and price impact—can be substantially larger for small and illiquid stocks.

We model total transaction costs as a function of trade size and stock liquidity:

$$TC_{i,t} = c_{\text{fixed}} + \frac{1}{2}\text{Spread}_{i,t} + \lambda\sqrt{\frac{|\Delta w_{i,t}| \cdot \text{AUM}}{ADV_{i,t}}} \tag{12.8}$$

where $c_{\text{fixed}} \approx 0.25\%$ is the explicit cost per trade, $\text{Spread}_{i,t}$ is the quoted bid-ask spread, $\Delta w_{i,t}$ is the weight change, AUM is portfolio size, $ADV_{i,t}$ is average daily volume in VND, and λ is the price impact coefficient estimated from the Amihud (2002) model.

```
def estimate_transaction_costs(weight_changes, stock_data,
                               aum_vnd=100e9, fixed_cost=0.0025,
                               impact_coef=0.10):
    """
    Estimate total transaction costs for a rebalancing event.

    Parameters
    ----------
    weight_changes : Series indexed by ticker, absolute weight changes
    stock_data : DataFrame with ticker, bid_ask_spread,
↪   turnover_value_avg_20d
    aum_vnd : float, portfolio AUM in VND
    fixed_cost : float, explicit cost per unit traded (one-way)
```

```
    impact_coef : float, price impact coefficient (lambda)

    Returns
    -------
    total_cost : float, total TC as fraction of AUM
    cost_detail : DataFrame with per-stock costs
    """
    costs = []

    for ticker, dw in weight_changes.items():
        if abs(dw) < 1e-6:
            continue

        trade_vnd = abs(dw) * aum_vnd

        # Explicit cost (one-way)
        explicit = fixed_cost * trade_vnd

        # Spread cost
        stock_info = stock_data[stock_data['ticker'] == ticker]
        if len(stock_info) > 0:
            spread = stock_info['bid_ask_spread'].iloc[0]
            adv = stock_info['turnover_value_avg_20d'].iloc[0]
        else:
            spread = 0.005  # Default 50 bps
            adv = 1e9  # Default VND 1bn

        spread_cost = 0.5 * spread * trade_vnd

        # Price impact (square root model)
        participation_rate = trade_vnd / max(adv, 1e6)
        impact_cost = impact_coef * np.sqrt(participation_rate) * trade_vnd

        total = explicit + spread_cost + impact_cost
        costs.append({
            'ticker': ticker,
            'weight_change': dw,
            'trade_vnd': trade_vnd,
            'explicit': explicit,
            'spread': spread_cost,
            'impact': impact_cost,
            'total': total
        })
```

```
    cost_df = pd.DataFrame(costs)
    total_cost = cost_df['total'].sum() / aum_vnd if len(cost_df) > 0 else 0

    return total_cost, cost_df
```

12.6.2 Net-of-Cost Performance

We apply the transaction cost model to compute net-of-cost returns for each weighting scheme at different assumed AUM levels. This is critical because strategies that appear attractive in gross terms may be unimplementable at scale due to the illiquidity of small-cap Vietnamese stocks.

```
def compute_net_returns(portfolio_df, cost_per_turnover=0.005):
    """
    Approximate net returns using a proportional cost model.
    Net return = gross return - (turnover * cost_per_unit_turnover)
    """
    df = portfolio_df.copy()
    df['tc'] = df['turnover'] * cost_per_turnover
    df['net_return'] = df['port_return'] - df['tc']
    return df

# Compute at different cost assumptions
cost_scenarios = {
    'Low (25 bps)': 0.0025,
    'Medium (50 bps)': 0.005,
    'High (100 bps)': 0.01
}

print("Annualized Net Sharpe Ratios by Cost Scenario:")
print("-" * 70)

for cost_name, cost_rate in cost_scenarios.items():
    row = {'Scenario': cost_name}
    for port_name, port_df in portfolios.items():
        net_df = compute_net_returns(port_df, cost_rate)
        rf_monthly = (1.04) ** (1/12) - 1
        excess = net_df['net_return'] - rf_monthly
        sharpe = excess.mean() / excess.std() * np.sqrt(12) if excess.std() >
 ↪  0 else 0
```

```
        row[port_name] = round(sharpe, 3)
    print(f"{cost_name}:")
    for k, v in row.items():
        if k != 'Scenario':
            print(f"  {k}: {v}")
    print()
```

```
fig, ax = plt.subplots(figsize=(12, 5))

turnover_data = {}
for name, df in portfolios.items():
    turnover_data[name] = df['turnover'].values

positions = range(len(turnover_data))
bp = ax.boxplot(
    turnover_data.values(),
    positions=positions,
    widths=0.6,
    patch_artist=True,
    showfliers=False,
    medianprops={'color': 'black', 'linewidth': 1.5}
)

for i, (patch, name) in enumerate(zip(bp['boxes'], turnover_data.keys())):
    patch.set_facecolor(colors.get(name, 'gray'))
    patch.set_alpha(0.7)

ax.set_xticks(positions)
ax.set_xticklabels(turnover_data.keys(), rotation=45, ha='right', fontsize=9)
ax.set_ylabel('Monthly Turnover (one-way)')
ax.set_title('Turnover Distribution by Weighting Scheme')
plt.tight_layout()
plt.show()
```

Figure 12.4

12.6.3 Cost Erosion at Scale

The relationship between portfolio AUM and implementable performance is non-linear because price impact costs grow with trade size. We simulate performance at different AUM levels:

```
aum_levels = [10, 50, 100, 500, 1000, 5000]  # VND billions

fig, ax = plt.subplots(figsize=(10, 6))

selected_ports = ['VW', 'EW (Monthly)', 'Capped VW (5%)',
                  'Min Variance', 'Risk Parity']

for name in selected_ports:
    sharpes = []
    df = portfolios[name]

    for aum in aum_levels:
        # Cost scales with sqrt(AUM / ADV)
        base_cost = 0.003  # Base cost at small AUM
        scale_factor = np.sqrt(aum / 100)  # Normalized to VND 100bn
        cost_rate = base_cost * scale_factor

        # VW is less affected (large-cap tilt)
        if name == 'VW':
            cost_rate *= 0.3
        elif 'Capped' in name:
            cost_rate *= 0.5
        elif 'Min Variance' in name or 'Risk Parity' in name:
            cost_rate *= 0.7

        net_df = compute_net_returns(df, cost_rate)
        rf_m = (1.04) ** (1/12) - 1
        excess = net_df['net_return'] - rf_m
        s = excess.mean() / excess.std() * np.sqrt(12) if excess.std() > 0
 ↪  else 0
        sharpes.append(s)

    ax.plot(aum_levels, sharpes, marker='o', label=name,
            color=colors.get(name, 'gray'), linewidth=2)

ax.set_xlabel('Portfolio AUM (VND Billion)')
ax.set_ylabel('Net Sharpe Ratio')
ax.set_title('Sharpe Ratio Degradation with AUM')
ax.set_xscale('log')
ax.legend(fontsize=9)
ax.axhline(y=0, color='gray', linewidth=0.5)
plt.tight_layout()
plt.show()
```

Figure 12.5

12.7 Rebalancing Frequency Analysis

12.7.1 The Rebalancing Trade-Off

Rebalancing serves two purposes: (i) restoring target weights to maintain the desired risk profile, and (ii) harvesting the "rebalancing bonus"—the systematic profit from buying low and selling high that arises when weights are reset to targets in the presence of mean-reverting cross-sectional returns.

The trade-off is clear: more frequent rebalancing maintains tighter adherence to target weights and captures more of the rebalancing bonus, but incurs higher transaction costs. The optimal frequency depends on the magnitude of mean reversion (which determines the gross rebalancing bonus), the level of transaction costs, and the rate at which weights drift from targets.

12.7.2 Threshold-Based Rebalancing

An alternative to calendar-based rebalancing is to rebalance only when portfolio weights drift beyond a tolerance band. This "no-trade zone" approach is motivated by Gârleanu and Pedersen (2013), who derive the optimal dynamic trading strategy under quadratic transaction costs and show that the optimal portfolio is a weighted average of the current holdings and the frictionless target.

```
def compute_threshold_rebalanced(monthly_df, weight_fn,
                                 threshold=0.01):
    """
    Rebalance only when maximum weight deviation exceeds threshold.

    Parameters
    ----------
    threshold : float
        Rebalance when max(|w_actual - w_target|) > threshold.
    """
    months = sorted(monthly_df['month_end'].unique())
    results = []
    current_weights = None
    rebalance_count = 0

    for month in months:
        cs = monthly_df[monthly_df['month_end'] == month].copy()
        cs = cs.dropna(subset=['monthly_return']).set_index('ticker')

        if len(cs) < 5:
```

```
frequencies = {
    'Monthly': 'M',
    'Quarterly': 'Q',
    'Semi-annual': 'Q',  # Approximate with every 6 months
    'Annual': 'A'
}

# Recompute EW at each frequency
freq_results = {}
for freq_name, freq_code in frequencies.items():
    freq_df = compute_portfolio_returns(
        universe_mid, ew_weights, rebal_freq=freq_code
    )
    freq_results[freq_name] = freq_df

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Gross vs Net Sharpe
freq_names = list(freq_results.keys())
gross_sharpes = []
net_sharpes = []

for fn in freq_names:
    df = freq_results[fn]
    rf_m = (1.04) ** (1/12) - 1
    exc = df['port_return'] - rf_m
    gross_sharpes.append(exc.mean() / exc.std() * np.sqrt(12))

    net_df = compute_net_returns(df, 0.005)
    exc_net = net_df['net_return'] - rf_m
    net_sharpes.append(exc_net.mean() / exc_net.std() * np.sqrt(12))

x = range(len(freq_names))
axes[0].bar([i - 0.15 for i in x], gross_sharpes, width=0.3,
            color='#2C5F8A', alpha=0.85, label='Gross')
axes[0].bar([i + 0.15 for i in x], net_sharpes, width=0.3,
            color='#C0392B', alpha=0.85, label='Net (50 bps)')
axes[0].set_xticks(x)
axes[0].set_xticklabels(freq_names)
axes[0].set_ylabel('Annualized Sharpe Ratio')
axes[0].set_title('Panel A: Sharpe Ratio by Rebalancing Frequency')
axes[0].legend()

# Panel B: Average turnover
turnovers = [freq_results[fn]['turnover'].mean() for fn in freq_names]
axes[1].bar(x, turnovers, color='#E67E22', alpha=0.85)
axes[1].set_xticks(x)
axes[1].set_xticklabels(freq_names)
axes[1].set_ylabel('Average Monthly Turnover')
axes[1].set_title('Panel B: Turnover by Rebalancing Frequency')

plt.tight_layout()
plt.show()
```

```
            continue

        target = weight_fn(cs.reset_index())
        target = target / target.sum()

        if current_weights is None:
            current_weights = target.copy()
            rebalance_count += 1
        else:
            # Drift weights
            current_weights = current_weights.reindex(cs.index, fill_value=0)
            current_weights = current_weights * (1 + cs['monthly_return'])
            total = current_weights.sum()
            if total > 0:
                current_weights = current_weights / total

            # Check if rebalancing needed
            max_dev = (current_weights - target.reindex(
                current_weights.index, fill_value=0
            )).abs().max()

            if max_dev > threshold:
                turnover = (current_weights - target.reindex(
                    current_weights.index, fill_value=0
                )).abs().sum() / 2
                current_weights = target.reindex(cs.index, fill_value=0)
                current_weights = current_weights / current_weights.sum()
                rebalance_count += 1
            else:
                turnover = 0

        port_ret = (current_weights.reindex(cs.index, fill_value=0) *
                    cs['monthly_return']).sum()
        hhi = (current_weights ** 2).sum()

        results.append({
            'month_end': month,
            'port_return': port_ret,
            'turnover': turnover,
            'hhi': hhi,
            'effective_n': 1/hhi if hhi > 0 else 0,
            'n_stocks': (current_weights > 1e-6).sum()
        })
```

```
    print(f"Threshold {threshold:.1%}: rebalanced {rebalance_count} / "
          f"{len(results)} months ({rebalance_count/len(results):.0%})")
    return pd.DataFrame(results)

# Test different thresholds
thresholds = [0.005, 0.01, 0.02, 0.05]
threshold_results = {}
for t in thresholds:
    threshold_results[f'{t:.1%}'] = compute_threshold_rebalanced(
        universe_mid, ew_weights, threshold=t
    )
```

12.8 The Rebalancing Bonus: Decomposition

The excess return of the rebalanced EW portfolio over a buy-and-hold EW portfolio (which starts equal-weighted but drifts) can be decomposed following Plyakha, Uppal, and Vilkov (2021). Define r_i as the return of stock i over one period:

$$R^{EW}_{\text{rebal}} - R^{EW}_{\text{drift}} \approx \frac{1}{2N}\sum_{i=1}^{N}\text{Var}(r_i) - \frac{1}{2N^2}\sum_{i}\sum_{j}\text{Cov}(r_i, r_j) \tag{12.9}$$

The first term captures the "buy low, sell high" effect from resetting weights after return dispersion. The second term is the cost of undoing covariance-induced drift. The rebalancing bonus is larger when cross-sectional return dispersion is high (which it is in Vietnam) and when pairwise correlations are low.

```
# Compute buy-and-hold EW portfolio (no rebalancing after initial equal
 ↪ weight)
bh_returns = compute_portfolio_returns(
    universe_mid, ew_weights, rebal_freq='A'  # Rebalance once per year only
)

# The rebalancing bonus is the difference
bonus_df = pd.merge(
    ew_monthly[['month_end', 'port_return']].rename(
        columns={'port_return': 'rebal_return'}),
    bh_returns[['month_end', 'port_return']].rename(
        columns={'port_return': 'bh_return'}),
    on='month_end'
```

```
)
bonus_df['bonus'] = bonus_df['rebal_return'] - bonus_df['bh_return']

# Cross-sectional return dispersion
dispersion = (
    universe_mid
    .groupby('month_end')['monthly_return']
    .std()
    .reset_index(name='cs_dispersion')
)
bonus_df = bonus_df.merge(dispersion, on='month_end')

ann_bonus = bonus_df['bonus'].mean() * 12
print(f"Annualized rebalancing bonus (EW monthly vs annual):
 ↪  {ann_bonus:.4f}")
print(f"Mean cross-sectional dispersion:
 ↪  {bonus_df['cs_dispersion'].mean():.4f}")
```

12.9 Factor Exposure Analysis

Different weighting schemes induce different factor exposures, which may explain their return differences. We regress each portfolio's excess returns on the Vietnamese Fama-French factors:

$$R_{p,t} - R_{f,t} = \alpha_p + \beta_p^{MKT}(R_{m,t} - R_{f,t}) + \beta_p^{SMB}\text{SMB}_t + \beta_p^{HML}\text{HML}_t + \varepsilon_{p,t} \tag{12.10}$$

```
# Retrieve Vietnamese factor returns from DataCore
factors = client.get_factor_returns(
    market='vietnam',
    start_date='2012-01-01',
    end_date='2024-12-31',
    factors=['mkt_excess', 'smb', 'hml', 'wml']
)

# Run factor regressions for each portfolio
factor_results = {}
for name, df in portfolios.items():
    merged = pd.merge(
        df[['month_end', 'port_return']],
```

```
fig, ax1 = plt.subplots(figsize=(12, 5))

ax1.bar(pd.to_datetime(bonus_df['month_end']),
        bonus_df['bonus'] * 100,
        color='#2C5F8A', alpha=0.6, width=25, label='Rebal. Bonus')
ax1.set_ylabel('Rebalancing Bonus (%)', color='#2C5F8A')
ax1.set_xlabel('Date')

ax2 = ax1.twinx()
ax2.plot(pd.to_datetime(bonus_df['month_end']),
         bonus_df['cs_dispersion'],
         color='#C0392B', linewidth=1.5, alpha=0.7,
         label='CS Dispersion')
ax2.set_ylabel('Cross-Sectional Return Dispersion', color='#C0392B')

ax1.set_title('Rebalancing Bonus and Return Dispersion')
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left')

plt.tight_layout()
plt.show()
```

Figure 12.7

```
        factors,
        on='month_end'
    )
    rf_m = (1.04) ** (1/12) - 1
    merged['excess'] = merged['port_return'] - rf_m

    model = sm.OLS(
        merged['excess'],
        sm.add_constant(merged[['mkt_excess', 'smb', 'hml', 'wml']])
    ).fit(cov_type='HAC', cov_kwds={'maxlags': 6})

    factor_results[name] = {
        'Alpha (ann.)': model.params['const'] * 12,
        'Alpha t-stat': model.tvalues['const'],
        'MKT': model.params['mkt_excess'],
        'SMB': model.params['smb'],
        'HML': model.params['hml'],
        'WML': model.params['wml'],
        'R²': model.rsquared
    }

factor_df = pd.DataFrame(factor_results).T
print(factor_df.round(3).to_string())
```

12.10 Machine Learning and the Implementable Efficient Frontier

Every scheme in this chapter was chosen first and charged for trading afterwards. T. I. Jensen et al. (2026) argue that this order is backwards for any strategy that trades on signals, and that the right object of evaluation is the **implementable efficient frontier**: the highest expected return *net of trading costs* attainable at each level of risk. They build a portfolio method around that objective, apply machine learning to it, and show that the ranking of strategies and of the characteristics that drive them changes once costs enter the objective rather than the accounting.

12.10.1 The Problem

An investor with wealth W_t holds portfolio weights π_t in N stocks. Between rebalancing dates the weights drift with returns, so the portfolio inherited at t is $g_t \pi_{t-1}$, where g_t is a diagonal matrix with entries $(1 + r_t^f + r_{i,t})/(1 + g_t^W)$ and g_t^W is the growth of wealth. Moving to π_t

```
fig, ax = plt.subplots(figsize=(10, 6))

plot_data = factor_df[['MKT', 'SMB', 'HML', 'WML']].copy()
im = ax.imshow(plot_data.values, cmap='RdBu_r', aspect='auto',
               vmin=-1.5, vmax=1.5)

ax.set_xticks(range(len(plot_data.columns)))
ax.set_xticklabels(plot_data.columns, fontsize=10)
ax.set_yticks(range(len(plot_data.index)))
ax.set_yticklabels(plot_data.index, fontsize=9)

# Add text annotations
for i in range(len(plot_data.index)):
    for j in range(len(plot_data.columns)):
        val = plot_data.values[i, j]
        color = 'white' if abs(val) > 0.8 else 'black'
        ax.text(j, i, f'{val:.2f}', ha='center', va='center',
                color=color, fontsize=9)

plt.colorbar(im, ax=ax, label='Factor Loading')
ax.set_title('Factor Exposures by Weighting Scheme')
plt.tight_layout()
plt.show()
```

Figure 12.8

requires trading $W_t(\pi_t - g_t\pi_{t-1})$ in currency units, and trading moves prices linearly in the trade size, with a diagonal price-impact matrix Λ_t. The return net of the risk-free rate and of trading costs is

$$r_{t+1}^{\text{net}} = \pi_t^\top r_{t+1} - \frac{W_t}{2}\left(\pi_t - g_t\pi_{t-1}\right)^\top \Lambda_t \left(\pi_t - g_t\pi_{t-1}\right), \tag{12.11}$$

and the investor maximizes its mean minus $\gamma/2$ times its variance. Two features of Equation 12.11 matter. Trading costs are quadratic in the trade, which is the linear-price-impact specification of Gârleanu and Pedersen (2013) rather than the square-root model in Equation 12.8; the choice buys tractability at the cost of understating the cost of small trades. And costs scale with wealth: the same weights cost a larger investor more. The authors calibrate impact so that trading 1% of a stock's daily dollar volume moves its price by 0.1%, which gives $\lambda_{i,t} = 0.2/\text{ADV}_{i,t}$.

The frontier traced by varying γ has a property the textbook frontier lacks. Without costs, any point on the capital market line is reached by levering the tangency portfolio, so the Sharpe ratio is constant along the line. With costs, larger positions mean larger trades, so the net Sharpe ratio *declines* along the implementable frontier. An investor therefore cannot maximize the net Sharpe ratio and then choose leverage; risk, return, and cost have to be chosen together. And the frontier of a larger investor lies everywhere below that of a smaller one.

12.10.2 The Solution: Trade Toward an Aim

Generalizing Gârleanu and Pedersen (2013) to stationary returns and to expected returns that are arbitrary functions of characteristics, the authors show that the optimal portfolio takes the form

$$\pi_t = m\, g_t\pi_{t-1} + (I - m)\, A_t, \tag{12.12}$$

so the investor starts from the inherited portfolio and trades a fraction $I - m$ of the way toward an **aim portfolio** A_t. The matrix m solves a fixed-point equation in $\gamma\Sigma$ and $W\Lambda$ and governs trading speed: it rises toward the identity, meaning slower trading, as wealth or price impact rise and as risk aversion falls. The aim is a discounted sum of current *and expected future* Markowitz portfolios,

$$A_t \approx (I - m)^{-1} \sum_{\tau=0}^{\infty} (m\bar{g})^\tau\, c\, \mathbb{E}_t\left[\frac{1}{\gamma}\Sigma^{-1}\mu_{t+\tau}\right], \tag{12.13}$$

with c a matrix that depends on m. Because the investor will hold whatever it buys today for a while, a signal that predicts returns for many months is worth more than an equally strong

signal that decays within a month. This is the economic content of "alpha decay," and it is exactly what a cost-agnostic or a static cost-aware portfolio ignores. The static alternative, which trades toward today's Markowitz portfolio while penalizing today's trade, has the same partial-adjustment form but aims at the wrong target.

12.10.3 Portfolio-ML: Learning the Aim Directly

Implementing Equation 12.13 requires forecasts of returns at every horizon. The authors' preferred method, **Portfolio-ML**, skips return forecasting altogether and learns the aim directly. It parameterizes the aim as a linear function of random Fourier features of the characteristics, scaled by each stock's volatility,

$$A_t = \operatorname{diag}(\sigma_t)^{-1}\,\mathrm{RF}(s_t)\,\beta, \tag{12.14}$$

an idea that extends the parametric portfolio policies of Brandt, Santa-Clara, and Valkanov (2009) to a dynamic problem with trading costs. The random features, $\cos(s_{i,t}^\top w_j)$ and $\sin(s_{i,t}^\top w_j)$ for random Gaussian vectors w_j, let a linear model approximate nonlinear functions and interactions of the characteristics, the device whose statistical properties in return prediction are studied by Kelly, Malamud, and Zhou (2024). Substituting Equation 12.14 into Equation 12.12 makes the portfolio approximately linear in β, $\pi_t \approx \Omega_t \beta$, where Ω_t is a persistence- and growth-weighted average of current and past scaled signals. Realized utility is then quadratic in β, and the optimal coefficients have a closed form:

$$\hat{\beta} = \left(\frac{1}{T}\sum_t D_t + \ell I\right)^{-1} \frac{1}{T}\sum_t \Omega_t^\top r_{t+1}, \qquad D_t = \gamma\,\Omega_t^\top \Sigma_t \Omega_t + W\,\Delta\Omega_t^\top \Lambda_t\,\Delta\Omega_t, \tag{12.15}$$

where $\Delta\Omega_t = \Omega_t - g_t\Omega_{t-1}$ is the implied trade and ℓ is a ridge penalty. The number of random features, their scale, and ℓ are chosen each year by an expanding-window validation on realized utility. No second tuning layer is needed, because the objective that fits β is already the investor's objective.

12.10.4 The Evidence

The application uses US stocks above the NYSE median market capitalization, 115 characteristics from T. I. Jensen, Kelly, and Pedersen (2023), a factor-model covariance matrix in the style of commercial risk models, and an out-of-sample period from 1981 to 2020 for an investor with risk aversion 10 whose fund grows with the market to $10 billion by 2020. The figures below are from the authors' working-paper version and may differ slightly in the published article.

- **Cost-agnostic machine learning is not implementable.** The Markowitz portfolio built on machine-learning return forecasts has a gross Sharpe ratio of 2.0 but turnover so extreme that its net return is catastrophically negative. A standard decile long-short sort on the same forecasts turns over 260% of capital a month and has a net Sharpe ratio of about −12.
- **Static cost awareness helps but misplaces the investor.** The static method earns a positive net Sharpe ratio of 0.94 yet a negative realized utility, because its realized risk far exceeds its ex ante risk.
- **Portfolio-ML dominates.** It earns a net Sharpe ratio of 1.38 with a monthly turnover of 32%, and the highest realized utility. Even after the authors give the forecast-based methods a second tuning layer that rescales their inputs, Portfolio-ML beats them by roughly 20% in net Sharpe ratio and 60% in utility.
- **Costs change which characteristics matter.** Measured by the utility lost when a characteristic is withheld, the "economic feature importance" of short-term reversal, which dominates cost-agnostic models, collapses, and value and quality become the most important characteristics.

The authors' replication code is public (GitHub: theisij/ml-and-the-implementable-efficient-frontier). It is written in R, requires the characteristics data of T. I. Jensen, Kelly, and Pedersen (2023) from WRDS, and is designed for a computing cluster. The repository carries no license, so we do not reproduce it here. The code below is an independent Python implementation of Portfolio-ML that follows the formulas above, checked first in a simulation and then applied to Vietnamese data.

12.10.5 An Independent Implementation

The core of the method is five small functions: the trading-speed matrix m, the scaling of signals, random Fourier features, the implied-weights matrix Ω_t, and the closed-form coefficients of Equation 12.15.

```
def trading_speed(sigma, lam, wealth, gamma, mu=0.007, rf=0.0, iters=10):
    """
    Persistence matrix m in the optimal rule pi_t = m g_t pi_{t-1} + (I - m)
↪   A_t.

    Solves the fixed point m~ = (x + I + diag(G) - m~ o G)^{-1}, where
    x = (gamma / wealth) Lambda^{-1/2} Sigma Lambda^{-1/2}, G is the second
    moment of portfolio-weight growth, and m = Lambda^{-1/2} m~ Lambda^{1/2}.
    `lam` is the diagonal of the price-impact matrix (per unit of currency).
    """
    n = len(lam)
    G = np.ones((n, n)) + sigma / (1 + rf + mu) ** 2
```

```
    lam_isqrt = 1 / np.sqrt(lam)
    x = (gamma / wealth) * sigma * np.outer(lam_isqrt, lam_isqrt)
    # Start from the closed-form solution when weight growth is deterministic
    ev, evec = np.linalg.eigh(x + 2 * np.eye(n))
    m_tilde = evec @ np.diag(0.5 * (ev - np.sqrt(ev**2 - 4))) @ evec.T
    base = x + np.eye(n) + np.diag(np.diag(G))
    for _ in range(iters):
        m_tilde = np.linalg.inv(base - m_tilde * G)
    return lam_isqrt[:, None] * m_tilde * np.sqrt(lam)[None, :]


def scale_signals(features, vol):
    """Demean features, add a constant, give each column unit length, divide
     ↪ rows by volatility."""
    f = features - features.mean(axis=0)
    f = np.column_stack([np.ones(len(f)), f])
    f = f / np.sqrt((f**2).sum(axis=0))
    return f / vol[:, None]


def random_fourier_features(features, W):
    """Random Fourier features: cos and sin of random projections of the
     ↪ features."""
    z = features @ W
    return np.hstack([np.cos(z), np.sin(z)])


def implied_weights(S_hist, g_hist, m):
    """
    Omega_t such that the portfolio implied by aim coefficients beta is
 ↪ Omega_t beta.
    S_hist[i] is the scaled signal matrix i months ago (i = 0 is today);
    g_hist[i] is the weight-growth vector realized over the month ending i
 ↪ months ago.
    """
    n = m.shape[0]
    M, num, den = np.eye(n), np.zeros_like(S_hist[0]), np.zeros((n, n))
    for i, S_i in enumerate(S_hist):
        num += M @ S_i
        den += M
        if i < len(g_hist):
            M = M @ m @ np.diag(g_hist[i])
```

```
    return np.linalg.solve(den, num)


def portfolio_ml_moments(S, g, ret, sigma, lam, m, wealth, gamma, months,
 ↪  lookback=11):
    """Per-month ingredients of the Portfolio-ML objective: r_tilde_t and
     ↪  D_t."""
    r_tilde, D = {}, {}
    for t in months:
        om = implied_weights([S[t - i] for i in range(lookback + 1)],
                             [g[t - 1 - i] for i in range(lookback)], m)
        om_prev = implied_weights([S[t - 1 - i] for i in range(lookback +
 ↪  1)],
                                  [g[t - 2 - i] for i in range(lookback)], m)
        trade = om - g[t - 1][:, None] * om_prev
        r_tilde[t] = om.T @ ret[t]
        D[t] = gamma * om.T @ sigma @ om + wealth * trade.T @ (lam[:, None]
 ↪  * trade)
    return r_tilde, D


def fit_aim_coefficients(r_tilde, D, months, ridge):
    """Closed-form Portfolio-ML coefficients: (mean D + ridge I)^{-1} mean
     ↪  r_tilde."""
    r_bar = np.mean([r_tilde[t] for t in months], axis=0)
    D_bar = np.mean([D[t] for t in months], axis=0)
    return np.linalg.solve(D_bar + ridge * np.eye(len(r_bar)), r_bar)


def objective(beta, r_tilde, D, months):
    """Average realized utility r_tilde' beta - beta' D beta / 2 over the
     ↪  given months."""
    return np.mean([r_tilde[t] @ beta - 0.5 * beta @ D[t] @ beta for t in
     ↪  months])
```

To see the method at work where the truth is known, the simulation below creates markets of 60 stocks over 35 years with two signals of very different persistence: a value-like signal with monthly autocorrelation 0.97 and a reversal-like signal that is independent from month to month but three times as predictive per unit. Liquidity varies across stocks by three orders of magnitude, and the investor has $10 billion and risk aversion 10. Four rules compete over the final ten years. Three of them are *oracles* that know the true expected returns: the cost-agnostic Markowitz portfolio, the static cost-aware portfolio, and the dynamic optimum of

Equation 12.12 and Equation 12.13 computed from the known signal dynamics. Portfolio-ML knows nothing about the data-generating process and learns its coefficients from the first 25 years, once with the two signals alone and once with a liquidity rank and its interactions with the signals added as features. Repeating the exercise over 20 simulated markets guards against reading too much into one draw.

```
GAMMA, WEALTH, LOOKBACK = 10.0, 1e10, 11
N_STOCKS, N_MONTHS = 60, 420
TRAIN_END, TEST_START = 240, 300
RIDGES = [0.0] + list(np.exp(np.arange(-10, 7)))


def simulate_market(seed, phi_slow=0.97, b_slow=0.04, b_fast=0.12):
    """A market with one persistent and one fleeting signal and heterogeneous
     ↪ liquidity."""
    rng = np.random.default_rng(seed)
    n, T = N_STOCKS, N_MONTHS
    dolvol = np.exp(rng.normal(np.log(2e8), 1.2, n))   # daily dollar volume
    lam = 0.2 / dolvol                                  # 1% of volume moves
 ↪  price 0.1%
    beta_mkt = rng.uniform(0.6, 1.4, n)
    ivol = rng.uniform(0.06, 0.12, n)
    sigma = 0.045**2 * np.outer(beta_mkt, beta_mkt) + np.diag(ivol**2)
    vol = np.sqrt(np.diag(sigma))

    s_slow = np.zeros((T, n))
    s_slow[0] = rng.normal(size=n)
    for t in range(1, T):
        s_slow[t] = phi_slow * s_slow[t - 1] + np.sqrt(1 - phi_slow**2) *
 ↪  rng.normal(size=n)
    s_fast = rng.normal(size=(T, n))

    mu = vol * (b_slow * s_slow + b_fast * s_fast)      # E_t[r_{t+1}]
    ret = mu + (np.linalg.cholesky(sigma) @ rng.normal(size=(n, T))).T
    g = (1 + ret) / (1 + ret.mean(axis=1))[:, None]     # weight growth
    liq = (np.argsort(np.argsort(dolvol)) + 0.5) / n - 0.5
    return dict(lam=lam, sigma=sigma, vol=vol, s_slow=s_slow, s_fast=s_fast,
                mu=mu, ret=ret, g=g, liq=liq, phi_slow=phi_slow,
                b_slow=b_slow, b_fast=b_fast)


def backtest(mkt, weight_rule, months):
    """Realized gross return, trading cost, and turnover of a weight rule."""
```

```
    rows, w_prev = [], np.zeros(N_STOCKS)
    for t in months:
        w_start = mkt["g"][t - 1] * w_prev
        w = weight_rule(t, w_start)
        trade = w - w_start
        rows.append({"gross": w @ mkt["ret"][t],
                     "tc": 0.5 * WEALTH * np.sum(mkt["lam"] * trade**2),
                     "turnover": np.abs(trade).sum()})
        w_prev = w
    out = pd.DataFrame(rows)
    out["net"] = out["gross"] - out["tc"]
    return out


def run_one(seed):
    mkt = simulate_market(seed)
    T, n = N_MONTHS, N_STOCKS
    sigma, lam, vol = mkt["sigma"], mkt["lam"], mkt["vol"]
    m = trading_speed(sigma, lam, WEALTH, GAMMA)
    I_n, I_m = np.eye(n), np.eye(n) - m
    sig_inv = np.linalg.inv(sigma)

    # Oracle benchmarks that know the true expected returns
    markowitz = np.einsum("ij,tj->ti", sig_inv, mkt["mu"]) / GAMMA
    static_inv = np.linalg.inv(GAMMA * sigma + WEALTH * np.diag(lam))
    mk_fast = np.einsum("ij,tj->ti", sig_inv, vol * mkt["b_fast"] *
↪   mkt["s_fast"]) / GAMMA
    mk_slow = np.einsum("ij,tj->ti", sig_inv, vol * mkt["b_slow"] *
↪   mkt["s_slow"]) / GAMMA
    slow_boost = np.linalg.solve(I_n - mkt["phi_slow"] * m, I_m)
    rules = {
        "Markowitz (oracle, cost-agnostic)": lambda t, w0: markowitz[t],
        "Static (oracle)": lambda t, w0: static_inv @ (mkt["mu"][t] + WEALTH
         ↪  * lam * w0),
        "Dynamic optimum (oracle)":
            lambda t, w0: m @ w0 + I_m @ (I_m @ mk_fast[t] + slow_boost @
             ↪  mk_slow[t]),
    }

    # Portfolio-ML learned from data, with and without liquidity interactions
    liq = np.tile(mkt["liq"], (T, 1))
    feature_sets = {
```

```
        "Portfolio-ML: signals only": np.stack([mkt["s_slow"],
         ↪  mkt["s_fast"]], axis=2),
        "Portfolio-ML: signals x liquidity": np.stack(
            [mkt["s_slow"], mkt["s_fast"], liq,
             mkt["s_slow"] * liq, mkt["s_fast"] * liq], axis=2),
    }
    months_fit = range(LOOKBACK + 2, T - 1)
    train = [t for t in months_fit if t < TRAIN_END]
    valid = [t for t in months_fit if TRAIN_END <= t < TEST_START]
    for name, feats in feature_sets.items():
        S = np.stack([scale_signals(feats[t], vol) for t in range(T)])
        r_tilde, D = portfolio_ml_moments(S, mkt["g"], mkt["ret"], sigma,
↪   lam, m,
                                          WEALTH, GAMMA, months_fit,
↪   LOOKBACK)
        ridge = max(RIDGES, key=lambda l: objective(
            fit_aim_coefficients(r_tilde, D, train, l), r_tilde, D, valid))
        beta = fit_aim_coefficients(r_tilde, D, train + valid, ridge)
        rules[name] = (lambda S_, b_: lambda t, w0: m @ w0 + I_m @ (S_[t] @
↪   b_))(S, beta)

    test = list(range(TEST_START, T - 1))
    rows = []
    for name, rule in rules.items():
        bt = backtest(mkt, rule, test)
        rows.append({"seed": seed, "Method": name,
                     "Net Sharpe": np.sqrt(12) * bt["net"].mean() /
                      ↪  bt["net"].std(),
                     "Trading cost": 12 * bt["tc"].mean(),
                     "Utility": 12 * (bt["net"].mean() - 0.5 * GAMMA *
                      ↪  bt["net"].var()),
                     "Turnover": bt["turnover"].mean()})
    return pd.DataFrame(rows)


sim_results = pd.concat([run_one(seed) for seed in range(20)],
                        ignore_index=True)
```

```
order = ["Markowitz (oracle, cost-agnostic)", "Static (oracle)",
         "Portfolio-ML: signals only", "Portfolio-ML: signals x liquidity",
         "Dynamic optimum (oracle)"]
summary = (sim_results.groupby("Method")[["Net Sharpe", "Trading cost",
↪   "Utility", "Turnover"]]
```

```
            .median().loc[order])
wide = sim_results.pivot(index="seed", columns="Method", values="Utility")
summary["Beats static oracle (share of seeds)"] = [
    (wide[name] > wide["Static (oracle)"]).mean() for name in order]
summary.round(3)
```

Table 12.3: Out-of-Sample Performance in Simulated Markets: Medians over 20 Markets (Risk Aversion 10, Wealth 10 Billion)

Method	Net Sharpe	Trading cost	Utility	Turnover	Beats static oracle (sha
Markowitz (oracle, cost-agnostic)	-10.589	350.772	-6738.143	10.627	0.00
Static (oracle)	0.662	0.011	0.021	0.114	0.00
Portfolio-ML: signals only	0.825	0.001	0.017	0.029	0.55
Portfolio-ML: signals x liquidity	0.912	0.004	0.024	0.064	0.60
Dynamic optimum (oracle)	0.956	0.022	0.045	0.150	0.85

The simulation reproduces the paper's logic in miniature. The cost-agnostic Markowitz portfolio is destroyed by its own trading even though it knows the true expected returns. The dynamic optimum beats the static oracle in most markets although both have the same perfect one-month forecasts: aiming at today's Markowitz portfolio ignores the persistence of the value-like signal and overpays to chase the reversal-like one. Portfolio-ML, which must learn everything from data, earns a higher net Sharpe ratio than the static oracle with roughly half its turnover, and comparable utility. It falls short of the dynamic optimum because it estimates with noise what the oracle knows.

The two Portfolio-ML variants carry a lesson about features. The optimal aim treats liquid and illiquid stocks differently: it trades toward signals quickly in liquid names and slowly in illiquid ones, and it is willing to hold persistent signals in illiquid names but not fleeting ones. A rule that is linear in the signals alone cannot express this, so it can only scale everything down, which shows up as very low turnover and lower utility. Adding liquidity and its interactions recovers much of the gap. In the paper, the random Fourier features of 115 characteristics, several of which measure liquidity, generate such interactions automatically.

Applying the method to Vietnamese stocks uses the liquid universe and the Ledoit-Wolf covariance estimator defined earlier in this chapter, the characteristics panel from the factor-construction chapter, and price impact calibrated to each stock's average daily traded value in VND. The code is not executed here: the hyperparameter search refits the model for every combination of random features, feature scale, and ridge penalty, and takes several minutes per wealth and risk-aversion pair.

```
# Characteristics panel from the factor-construction chapter
characteristics = client.get_characteristics_panel(
    exchanges=["HOSE", "HNX"],
    start_date="2008-01-01",
    end_date="2024-12-31",
    include_delisted=True,
)
char_cols = [c for c in characteristics.columns if c not in ["ticker",
 ↪  "month_end"]]

# Rank-standardize each characteristic to [0, 1] within month; missing -> 0.5
ranked = characteristics.copy()
ranked[char_cols] = (ranked.groupby("month_end")[char_cols]
                     .rank(pct=True).fillna(0.5))

# Wide month x ticker panels on the liquid universe
ret_wide = universe_liquid.pivot(index="month_end", columns="ticker",
                                 values="monthly_return").sort_index()
adv_wide = universe_liquid.pivot(index="month_end", columns="ticker",

 ↪  values="turnover_value_avg_20d").reindex(ret_wide.index)
mcap_wide = universe_liquid.pivot(index="month_end", columns="ticker",

 ↪  values="market_cap").reindex(ret_wide.index)
char_wide = {c: ranked.pivot(index="month_end", columns="ticker", values=c)
                     .reindex(index=ret_wide.index, columns=ret_wide.columns)
             for c in char_cols}

# Value-weighted market return drives the growth of portfolio weights
lag_mcap = mcap_wide.shift(1).where(ret_wide.notna())
mkt_ret = (ret_wide * lag_mcap).sum(axis=1) / lag_mcap.sum(axis=1)
months = ret_wide.index
```

```
LOOKBACK = 11
_base_cache = {}


def base_state(i, lookback=LOOKBACK):
    """
    Universe, covariance, and price impact at month i (no look-ahead).
    A stock is investable if it has returns over the last lookback + 2
 ↪  months.
```

```
    Next month's return is used only to evaluate the decision; a missing
    value (for example, a delisting) is recorded as zero.
    """
    if i in _base_cache:
        return _base_cache[i]
    window = ret_wide.iloc[i - lookback - 1:i + 1]
    tickers = window.columns[window.notna().all()]
    cov, cov_tickers = estimate_covariance(universe_liquid, months[i])
    state = None
    if cov is not None:
        adv = adv_wide.iloc[i]
        tickers = [t for t in tickers if t in cov_tickers and adv.get(t, 0) >
↪   0]
        if len(tickers) >= 30:
            sigma = cov.loc[tickers, tickers].values
            # growth[k] = growth of portfolio weights over month i - k
            growth = ((1 + window[tickers].values)
                      / (1 + mkt_ret.iloc[i - lookback - 1:i + 1].values)[:,
                       ↪  None])[::-1]
            state = dict(tickers=tickers, sigma=sigma,
↪   vol=np.sqrt(np.diag(sigma)),
                         lam=0.2 / adv[tickers].values,   # 1% of ADV moves
↪   price 0.1%
                         growth=growth,
                         ret_next=ret_wide.iloc[i +
↪   1][tickers].fillna(0).values)
    _base_cache[i] = state
    return state


def month_signals(i, tickers, vol, W, lookback=LOOKBACK):
    """Scaled random-feature signals for months i, i-1, ..., i-lookback-1."""
    out = []
    for lag in range(lookback + 2):
        X = np.column_stack([char_wide[c].iloc[i -
↪   lag][tickers].fillna(0.5).values
                             for c in char_cols])
        out.append(scale_signals(random_fourier_features(X, W), vol))
    return out
```

```
def run_portfolio_ml(wealth, gamma, first_val_year=2017,
 ↪  first_test_year=2020,
                     p_grid=(16, 32, 64), g_grid=(np.exp(-3), np.exp(-2)),
                     ridges=(0.0, *np.exp(np.arange(-10, 7))),
                     cost_mult=1.0, long_only=False, seed=1,
 ↪  lookback=LOOKBACK):
    """Portfolio-ML with yearly expanding-window tuning and an out-of-sample
     ↪  backtest."""
    rng = np.random.default_rng(seed)
    states = {}
    for i in range(lookback + 2, len(months) - 1):
        st = base_state(i, lookback)
        if st is None:
            continue
        lam = cost_mult * st["lam"]
        m = trading_speed(st["sigma"], lam, wealth, gamma,
 ↪  mu=mkt_ret.iloc[:i].mean())
        states[i] = {**st, "lam": lam, "m": m}
    year = {i: months[i].year for i in states}
    last_year = max(year.values())

    # Step 1: objective ingredients for every (p, g) and month
    candidates = {}
    for p in p_grid:
        for g_rff in g_grid:
            W = rng.normal(0, np.sqrt(g_rff), size=(len(char_cols), p // 2))
            r_tilde, D, S_now = {}, {}, {}
            for i, st in states.items():
                S_hist = month_signals(i, st["tickers"], st["vol"], W,
 ↪  lookback)
                om = implied_weights(S_hist[:lookback + 1],
                                     st["growth"][:lookback], st["m"])
                om_prev = implied_weights(S_hist[1:],
                                          st["growth"][1:lookback + 1],
 ↪  st["m"])
                trade = om - st["growth"][0][:, None] * om_prev
                r_tilde[i] = om.T @ st["ret_next"]
                D[i] = (gamma * om.T @ st["sigma"] @ om
                        + wealth * trade.T @ (st["lam"][:, None] * trade))
                S_now[i] = S_hist[0]
            # Step 2: expanding-window validation, one year at a time
            for l in ridges:
```

```
                val = {}
                for y in range(first_val_year, last_year + 1):
                    fit_m = [i for i in states if year[i] < y]
                    eval_m = [i for i in states if year[i] == y]
                    if fit_m and eval_m:
                        beta = fit_aim_coefficients(r_tilde, D, fit_m, l)
                        val[y] = objective(beta, r_tilde, D, eval_m)
                candidates[(p, g_rff, l)] = dict(r=r_tilde, D=D, S=S_now,
↪  val=val)

    # Step 3: each test year, use the best hyperparameters so far and trade
    rows, w_prev, lam_prev = [], pd.Series(dtype=float),
↪  pd.Series(dtype=float)
    for y in range(first_test_year, last_year + 1):
        best = max(candidates, key=lambda h: np.mean(
            [v for yy, v in candidates[h]["val"].items() if yy < y] or
↪  [-np.inf]))
        c = candidates[best]
        beta = fit_aim_coefficients(c["r"], c["D"],
                                    [i for i in states if year[i] < y],
↪  best[2])
        for i in [i for i in states if year[i] == y]:
            st, tick = states[i], states[i]["tickers"]
            w_start = w_prev.reindex(tick).fillna(0).values   # already grown
            aim = c["S"][i] @ beta
            w = st["m"] @ w_start + (np.eye(len(tick)) - st["m"]) @ aim
            if long_only:
                w = np.clip(w, 0, None)
            trade = w - w_start
            exits = w_prev.index.difference(tick)               # forced
↪  liquidations
            tc = (0.5 * wealth * np.sum(st["lam"] * trade**2)
                  + 0.5 * wealth *
                   ↪  np.sum(lam_prev.reindex(exits).fillna(0).values
                                        * w_prev.reindex(exits).fillna(0).
                                         ↪  values**2))
            rows.append({"month": months[i + 1], "gross": w @ st["ret_next"],
↪  "tc": tc,
                         "turnover": np.abs(trade).sum(), "leverage":
                          ↪  np.abs(w).sum(),
                         "p": best[0], "ridge": best[2]})
            w_prev = pd.Series(w * (1 + st["ret_next"]) / (1 + mkt_ret.iloc[i
↪  + 1]),
```

Table 12.4: The Implementable Efficient Frontier for Vietnamese Stocks: Portfolio-ML by Fund Size and Risk Aversion

```
# Points on the implementable efficient frontier for three fund sizes
frontier = {}
for wealth in [1e11, 1e12, 1e13]:            # VND 100 billion, 1 trillion,
 ↪  10 trillion
    for gamma in [5, 10, 20]:
        for long_only in [False, True]:
            bt = run_portfolio_ml(wealth, gamma, long_only=long_only)
            frontier[(f"{wealth / 1e9:,.0f}bn", gamma,
                      "long-only" if long_only else "long-short")] =
                       ↪  summarize(bt, gamma)
frontier = pd.DataFrame(frontier).T
frontier.index.names = ["Wealth (VND)", "Risk aversion", "Positions"]
frontier.round(3)
```

```
                                index=tick)
            lam_prev = pd.Series(st["lam"], index=tick)
    bt = pd.DataFrame(rows)
    bt["net"] = bt["gross"] - bt["tc"]
    return bt


def summarize(bt, gamma):
    return pd.Series({
        "Gross return": 12 * bt["gross"].mean(),
        "Trading cost": 12 * bt["tc"].mean(),
        "Net return": 12 * bt["net"].mean(),
        "Volatility": np.sqrt(12) * bt["net"].std(),
        "Net Sharpe": np.sqrt(12) * bt["net"].mean() / bt["net"].std(),
        "Utility": 12 * (bt["net"].mean() - 0.5 * gamma * bt["net"].var()),
        "Turnover": bt["turnover"].mean(),
        "Leverage": bt["leverage"].mean(),
    })
```

12.10.6 What This Means for Vietnamese Research

The case for the method is stronger in Vietnam than in the US large-cap universe the authors study, because the forces it is built to handle are larger here: liquidity is concentrated in a few dozen stocks (Section 12.2), and many of the anomalies documented for Vietnamese stocks live in exactly the small, illiquid names where costs bite. Several adaptations are needed.

1. **Short selling is not available.** Portfolio-ML as specified is a long-short portfolio, and in Vietnam it is a research object rather than a fund: it measures which characteristics carry value net of costs and how the frontier shrinks with size. The long-only variant in the code simply sets negative weights to zero after each rebalancing. That is a heuristic projection, not the constrained optimum. A properly constrained version maximizes the same objective numerically with $\pi_t \geq 0$, or works with active weights around a benchmark such as VN30, where an underweight of at most the benchmark weight replaces a short position.
2. **Calibrate price impact for Vietnam and stress it.** The US calibration (0.1% impact for 1% of daily volume) almost certainly understates impact for Vietnamese mid caps. Report results for cost multipliers of 1, 2, and 4 (the `cost_mult` argument), and compare the implied costs with the trading-cost model in Equation 12.8.
3. **Proportional costs are missing from the model.** Commissions and the 0.1% selling tax are linear in the trade (Table 12.2), and linear costs create a no-trade region that a quadratic model cannot produce. Subtract them in the backtest at minimum; treat a result that survives only without them as not implementable.
4. **Respect the market's rules.** Daily price limits cap how much of a position can be traded on a given day when a stock moves sharply, and foreign ownership limits make some stocks unbuyable for foreign investors. For a foreign-investor version, drop stocks at their limits from the buy side.
5. **The sample is short.** With characteristics from 2008, a 60-month covariance window, and yearly validation, only a few validation years inform the choice of hyperparameters. Keep the number of random features small, let the ridge penalty work, and report how sensitive the results are to the random seed that draws the features.
6. **Use economic feature importance as the anomaly test.** Withholding one characteristic at a time and measuring the loss in net utility answers the question the factor zoo (Section 21.3) cannot: which Vietnamese anomalies are worth trading for an investor of a given size once costs are paid. The answer will differ by fund size, and the frontier across VND 100 billion, 1 trillion, and 10 trillion shows how quickly the opportunity set shrinks toward the most liquid names.

12.11 Practical Guidance for Vietnam

The preceding analysis yields several practical recommendations for researchers and investors working with Vietnamese equities:

For academic factor research: VW portfolios remain the default for asset pricing tests because they represent the investable opportunity set and avoid inflating alpha estimates with small-cap illiquidity premia. When EW portfolios are used (e.g., to give equal influence to each stock in cross-sectional sorts), researchers should report both VW and EW results and discuss the sensitivity. Eugene F. Fama and French (2008) follow this practice systematically.

For fund management: The choice depends on AUM and mandate. At AUM below VND 500 billion, capped VW or fundamental weighting offers a practical compromise between diversification and implementability. At larger AUM, pure VW or sector-capped VW is more realistic. Risk parity and minimum variance are suitable for low-volatility mandates but require robust covariance estimation and quarterly rebalancing.

For index construction: Vietnamese index providers (VN30, VNINDEX) use variants of capped VW. The analysis suggests that the cap level significantly affects the index's diversification properties and tracking error relative to the uncapped VW market. A 10% cap balances concentration reduction against turnover.

For transaction cost management: In all schemes, the marginal benefit of rebalancing declines faster than the marginal cost as frequency increases beyond quarterly. Calendar-based quarterly rebalancing or threshold-based rebalancing (with a 1–2% tolerance band) provides the best cost-benefit trade-off in the Vietnamese market.

12.12 Summary

Table 12.5: Summary comparison of weighting schemes for Vietnamese equities.

Dimension	VW	EW	Capped VW	Fundamental	Min Var	Risk Parity
Turnover	Very low	High	Low	Low	Moderate	Moderate
Concentration	High	None	Moderate	Moderate	Variable	Low
Size tilt	Large	Small	Moderate	Large-mid	Low-vol	Mixed
Data required	Prices	None	Prices	Accounting	Returns cov.	Returns cov.
Scale sensitivity	Low	High	Low	Low	Moderate	Moderate
Rebal. frequency	Passive	Monthly	Monthly/Quarterly	Annual	Quarterly	Quarterly
Best use case	Benchmarks, large AUM	Cross-sectional tests	Index tracking	Long-term investing	Low-vol mandates	Balanced risk

The choice of weighting scheme is not merely a technical detail—it reflects a substantive economic decision about the relative importance of diversification, investability, and cost control. In the Vietnamese market, where the capitalization distribution is highly skewed and small-cap liquidity is thin, this choice has larger consequences than in developed markets. Researchers who report results under only one weighting scheme risk conclusions that are specific to that scheme rather than reflective of a genuine economic relationship.

13 Missing Data and Survivorship Bias

> **Note**
>
> In this chapter, we document the patterns of missing data, survivorship bias, and delisting bias in Vietnamese equity markets, develop diagnostic tools to detect these problems, and implement correction methods that yield more reliable empirical results.

Every empirical study in finance implicitly assumes that the data it analyzes are representative of the population it claims to study. When this assumption fails, because delisted firms are excluded, because databases begin coverage only after firms have survived, or because trading gaps create missing return observations, the resulting estimates are biased. In the U.S. context, Shumway (1997) showed that ignoring delisting returns biases average returns upward by approximately 1% per year for NYSE stocks and substantially more for Nasdaq stocks, with severe consequences for anomaly-based strategies that overweight small, distressed firms.

The Vietnamese market presents a distinct and, in many ways, more acute set of data integrity challenges. The market is young. HOSE opened in July 2000 with only two listed stocks, and the number of listings grew rapidly through the mid-2000s equitization wave. This means that any sample beginning before roughly 2007 suffers from severe new-listing bias: the early cross-section is tiny and unrepresentative. Delistings are common and often involuntary, driven by losses exceeding charter capital, failure to file financial statements, or SSC enforcement actions rather than by mergers or going-private transactions as in the U.S. These involuntary delistings are systematically associated with negative terminal returns. And the prevalence of zero-trading days among small-cap stocks creates return gaps that look like missing data but actually reflect illiquidity.

This chapter provides the tools to diagnose and, where possible, correct these problems.

13.1 Taxonomy of Data Problems

Missing data in financial research is not monolithic. The consequences depend critically on the *mechanism* generating the missingness. Rubin (1976) and Little and Rubin (2019) classify missing data into three types:

1. **Missing Completely at Random (MCAR).** The probability of a missing observation does not depend on any observed or unobserved variable. Example: a data vendor's server crashes on a random Tuesday, losing that day's records. MCAR is the most benign case, complete-case analysis (dropping missing observations) produces unbiased but less efficient estimates.
2. **Missing at Random (MAR).** The probability of missingness depends on observed variables but not on the missing value itself, conditional on observables. Example: small firms are more likely to have missing analyst coverage, but conditional on firm size, whether coverage is missing is unrelated to the firm's true expected return. MAR allows unbiased estimation through methods that condition on the observed predictors of missingness.
3. **Missing Not at Random (MNAR).** The probability of missingness depends on the missing value itself. Example: firms with the worst performance are most likely to delist and disappear from the database. MNAR is a pathological case and, unfortunately, the most common in financial data. Survivorship bias and delisting bias are both instances of MNAR because the event that removes the observation (delisting) is correlated with the variable of interest (returns).

In the Vietnamese context, we encounter all three types, often simultaneously (Table 13.1).

Table 13.1: Taxonomy of missing data in Vietnamese equity databases

Data Problem	Missingness Type	Mechanism in Vietnam
Zero-trading days	MAR/MNAR	Small/illiquid stocks; correlated with returns
Price limit hits	MNAR	True return truncated at limit; observed return censored
Delisting	MNAR	Worst-performing firms exit; returns disappear
Late listing coverage	Selection bias	Database begins after firm survives initial period
Exchange transfers	Administrative	HOSE→HNX or UPCoM transfers break ticker continuity
Suspended trading	MNAR	Suspension precedes negative events; returns missing

13.2 Data Construction

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.figsize': (12, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

```
from datacore import DataCoreClient

client = DataCoreClient()

# Complete listing history: includes all firms ever listed, not just current
listing_history = client.get_listing_history(
    exchanges=['HOSE', 'HNX', 'UPCoM'],
    include_delisted=True,
    fields=[
        'ticker', 'company_name', 'exchange', 'listing_date',
        'delisting_date', 'delisting_reason', 'is_active',
        'transfer_from', 'transfer_to', 'transfer_date',
        'ipo_date', 'equitization_date', 'sector'
    ]
)

# Daily returns: includes delisted firms' full history
daily_returns = client.get_daily_prices(
    exchanges=['HOSE', 'HNX', 'UPCoM'],
    start_date='2000-07-28',   # HOSE opening date
```

```
    end_date='2024-12-31',
    include_delisted=True,      # Critical flag
    fields=[
        'ticker', 'date', 'close', 'adjusted_close', 'volume',
        'turnover_value', 'market_cap', 'shares_outstanding',
        'price_limit_hit'       # +1 = limit up, -1 = limit down, 0 = neither
    ]
)

# Monthly returns (pre-computed, survivorship-bias-free)
monthly_returns = client.get_monthly_returns(
    exchanges=['HOSE', 'HNX', 'UPCoM'],
    start_date='2000-07-28',
    end_date='2024-12-31',
    include_delisted=True,
    fields=[
        'ticker', 'month_end', 'monthly_return', 'market_cap',
        'volume_avg_20d', 'n_trading_days', 'n_zero_volume_days'
    ]
)

print(f"Listing history: {listing_history.shape[0]:,} firms")
print(f"  Active: {listing_history['is_active'].sum():,}")
print(f"  Delisted: {(~listing_history['is_active']).sum():,}")
print(f"Daily observations: {daily_returns.shape[0]:,}")
print(f"Monthly observations: {monthly_returns.shape[0]:,}")
```

13.3 Listing Dynamics in Vietnam

13.3.1 The Growth of the Vietnamese Market

The Vietnamese stock market's short history creates a distinctive pattern: the investable universe has grown from near-zero to over 1,500 listed firms in approximately two decades. This rapid growth means that the composition of the market at any point in time is heavily influenced by the vintage of listings, and that studies using early data face extreme small-sample problems.

```
listing_history['listing_date'] =
 ↪  pd.to_datetime(listing_history['listing_date'])
listing_history['delisting_date'] =
 ↪  pd.to_datetime(listing_history['delisting_date'])

# Count active listings at each month-end
months = pd.date_range('2000-07-01', '2024-12-31', freq='M')
active_counts = []

for month in months:
    for exchange in ['HOSE', 'HNX', 'UPCoM']:
        active = listing_history[
            (listing_history['exchange'] == exchange) &
            (listing_history['listing_date'] <= month) &
            ((listing_history['delisting_date'].isna()) |
             (listing_history['delisting_date'] > month))
        ]
        active_counts.append({
            'month': month,
            'exchange': exchange,
            'n_active': len(active)
        })

active_df = pd.DataFrame(active_counts)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Active listings over time
for exchange, color in [('HOSE', '#2C5F8A'), ('HNX', '#E67E22'),
                        ('UPCoM', '#27AE60')]:
    subset = active_df[active_df['exchange'] == exchange]
    axes[0].plot(subset['month'], subset['n_active'],
                 color=color, linewidth=2, label=exchange)

axes[0].set_xlabel('Date')
axes[0].set_ylabel('Number of Active Listings')
axes[0].set_title('Panel A: Active Listings by Exchange')
axes[0].legend()

# Panel B: Annual listings and delistings
listing_history['listing_year'] = listing_history['listing_date'].dt.year
listing_history['delisting_year'] = listing_history['delisting_date'].dt.year

annual_listings = (
    listing_history
    .groupby('listing_year')
    .size()
    .reindex(range(2000, 2025), fill_value=0)
)
annual_delistings = (
    listing_history
    .dropna(subset=['delisting_year'])
    .groupby('delisting_year')
    .size()
```

13.3.2 Delisting Reasons

Vietnamese delistings are not homogeneous. The SSC mandates delisting for specific regulatory violations, but firms may also voluntarily delist, merge, or transfer between exchanges. The reason for delisting matters because it determines the likely terminal return.

```
delisted = listing_history[listing_history['delisting_date'].notna()].copy()

# Standardize delisting reasons into categories
reason_map = {
    'losses_exceed_charter': 'Involuntary - Financial Distress',
    'bankruptcy': 'Involuntary - Financial Distress',
    'failure_to_file': 'Involuntary - Regulatory',
    'audit_qualification': 'Involuntary - Regulatory',
    'ssc_enforcement': 'Involuntary - Regulatory',
    'merger': 'Voluntary - M&A',
    'going_private': 'Voluntary - Going Private',
    'transfer_exchange': 'Transfer',
    'voluntary': 'Voluntary - Other',
    'other': 'Other/Unknown'
}
delisted['reason_category'] = (
    delisted['delisting_reason']
    .map(reason_map)
    .fillna('Other/Unknown')
)

# Tabulate
reason_counts = (
    delisted['reason_category']
    .value_counts()
    .to_frame('Count')
)
reason_counts['Percentage'] = (
    reason_counts['Count'] / reason_counts['Count'].sum() * 100
)

print("Delisting Reasons:")
print(reason_counts.round(1).to_string())
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Pie chart
colors_pie = ['#C0392B', '#E74C3C', '#8E44AD', '#27AE60',
              '#2C5F8A', '#F1C40F', '#BDC3C7']
axes[0].pie(reason_counts['Count'], labels=reason_counts.index,
            colors=colors_pie[:len(reason_counts)],
            autopct='%1.0f%%', startangle=90, textprops={'fontsize': 8})
axes[0].set_title('Panel A: Delisting Reasons')

# Panel B: Delisting reasons over time
delisted['year'] = delisted['delisting_date'].dt.year
reason_by_year = pd.crosstab(delisted['year'], delisted['reason_category'])
reason_by_year = reason_by_year.reindex(range(2000, 2025), fill_value=0)

reason_by_year.plot(kind='bar', stacked=True, ax=axes[1],
                    colormap='Set2', edgecolor='white', width=0.8)
axes[1].set_xlabel('Year')
axes[1].set_ylabel('Number of Delistings')
axes[1].set_title('Panel B: Delisting Reasons Over Time')
axes[1].legend(fontsize=7, loc='upper left')

plt.tight_layout()
plt.show()
```

Figure 13.2

13.3.3 Firm Characteristics at Delisting

Do delisted firms differ systematically from survivors? If so, excluding them biases the observed distribution of firm characteristics.

13.4 Survivorship Bias

13.4.1 Definition and Magnitude

Survivorship bias arises when a study uses only firms that are currently listed (or listed at the end of the sample), excluding firms that delisted during the sample period. Because delisted firms disproportionately experienced negative returns before delisting, their exclusion inflates average returns, understates risk, and distorts cross-sectional patterns.

We quantify the magnitude of survivorship bias by comparing portfolio returns computed from the **survivorship-bias-free** sample (all firms, including those that subsequently delisted) against a **survivors-only** sample (firms that remained listed through the end of the sample).

```
# Define survivors: firms active as of 2024-12-31
survivors = set(
    listing_history[listing_history['is_active']]['ticker']
)

# Full sample: all firms, including delisted
full_sample = monthly_returns.copy()

# Survivors only: restrict to firms still listed at end of sample
survivors_only = monthly_returns[
    monthly_returns['ticker'].isin(survivors)
].copy()

# Compute EW monthly portfolio returns
def compute_ew_portfolio(df):
    return (
        df
        .groupby('month_end')['monthly_return']
        .mean()
        .to_frame('portfolio_return')
    )

def compute_vw_portfolio(df):
    return (
```

```
# Get fundamentals in the last available year before delisting
last_year_delisted = (
    delisted[['ticker', 'delisting_date']]
    .assign(last_fy=lambda x: x['delisting_date'].dt.year - 1)
)

fundamentals = client.get_fundamentals(
    exchanges=['HOSE', 'HNX', 'UPCoM'],
    start_date='2005-01-01',
    end_date='2024-12-31',
    include_delisted=True,
    fields=[
        'ticker', 'fiscal_year', 'total_assets', 'net_income',
        'total_equity', 'revenue', 'market_cap'
    ]
)

# Characteristics of delisted firms (last year before delisting)
delist_chars = (
    last_year_delisted
    .merge(fundamentals.rename(columns={'fiscal_year': 'last_fy'}),
           on=['ticker', 'last_fy'], how='inner')
)
delist_chars['roa'] = delist_chars['net_income'] /
 ↪  delist_chars['total_assets']
delist_chars['leverage'] = (
    (delist_chars['total_assets'] - delist_chars['total_equity'])
    / delist_chars['total_assets']
)
delist_chars['log_assets'] = np.log(delist_chars['total_assets'])
delist_chars['group'] = 'Delisted'

# Characteristics of all active firms (pooled)
all_chars = fundamentals.copy()
all_chars['roa'] = all_chars['net_income'] / all_chars['total_assets']
all_chars['leverage'] = (
    (all_chars['total_assets'] - all_chars['total_equity'])
    / all_chars['total_assets']
)
all_chars['log_assets'] = np.log(all_chars['total_assets'])
all_chars['group'] = 'All Active'

# Compare distributions
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
variables = [
    ('log_assets', 'Log Total Assets', axes[0, 0]),
    ('roa', 'Return on Assets', axes[0, 1]),
    ('leverage', 'Leverage Ratio', axes[1, 0]),
]

for col, label, ax in variables:
    for grp, color in [('All Active', '#2C5F8A'), ('Delisted', '#C0392B')]:
        if grp == 'Delisted':
            data = delist_chars[col].dropna()
```

```
        df
        .groupby('month_end')
        .apply(lambda g: np.average(g['monthly_return'],
                                    weights=g['market_cap'])
               if g['market_cap'].sum() > 0 else np.nan)
        .to_frame('portfolio_return')
    )

ew_full = compute_ew_portfolio(full_sample)
ew_survivors = compute_ew_portfolio(survivors_only)
vw_full = compute_vw_portfolio(full_sample)
vw_survivors = compute_vw_portfolio(survivors_only)

# Merge and compute bias
bias_ew = pd.merge(
    ew_full.rename(columns={'portfolio_return': 'full'}),
    ew_survivors.rename(columns={'portfolio_return': 'survivors'}),
    left_index=True, right_index=True
)
bias_ew['bias'] = bias_ew['survivors'] - bias_ew['full']

bias_vw = pd.merge(
    vw_full.rename(columns={'portfolio_return': 'full'}),
    vw_survivors.rename(columns={'portfolio_return': 'survivors'}),
    left_index=True, right_index=True
)
bias_vw['bias'] = bias_vw['survivors'] - bias_vw['full']

print("Survivorship Bias (Annualized):")
print(f"  EW: {bias_ew['bias'].mean() * 12:.4f} "
      f"({bias_ew['bias'].mean() * 1200:.1f} bps/year)")
print(f"  VW: {bias_vw['bias'].mean() * 12:.4f} "
      f"({bias_vw['bias'].mean() * 1200:.1f} bps/year)")
```

13.4.2 Time-Varying Survivorship Bias

The magnitude of survivorship bias is not constant. It peaks during and after market downturns, when delisting activity is highest.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for i, (bias_df, title) in enumerate(
    [(bias_ew, 'Panel A: Equal-Weighted'),
     (bias_vw, 'Panel B: Value-Weighted')]
):
    cum_full = (1 + bias_df['full']).cumprod()
    cum_surv = (1 + bias_df['survivors']).cumprod()

    axes[i].plot(cum_full.index, cum_full,
                 color='#2C5F8A', linewidth=2, label='Full Sample')
    axes[i].plot(cum_surv.index, cum_surv,
                 color='#C0392B', linewidth=2, label='Survivors Only')
    axes[i].set_ylabel('Cumulative Wealth')
    axes[i].set_xlabel('Date')
    axes[i].set_title(title)
    axes[i].legend()
    axes[i].set_yscale('log')

    ann_bias = bias_df['bias'].mean() * 12
    axes[i].text(0.05, 0.95,
                 f'Annual Bias: {ann_bias*100:.1f}%',
                 transform=axes[i].transAxes, fontsize=11,
                 verticalalignment='top',
                 bbox=dict(facecolor='white', alpha=0.8))

plt.tight_layout()
plt.show()
```

Figure 13.4

```
bias_ew['rolling_bias_12m'] = bias_ew['bias'].rolling(12).mean() * 12

fig, axes = plt.subplots(2, 1, figsize=(14, 8), height_ratios=[2, 1])

# Panel A: Rolling bias
axes[0].fill_between(
    bias_ew.index, 0, bias_ew['rolling_bias_12m'] * 100,
    where=bias_ew['rolling_bias_12m'] > 0,
    color='#C0392B', alpha=0.4
)
axes[0].plot(bias_ew.index, bias_ew['rolling_bias_12m'] * 100,
             color='#C0392B', linewidth=1.5)
axes[0].axhline(y=0, color='gray', linewidth=0.5)
axes[0].set_ylabel('Annualized Bias (%)')
axes[0].set_title('Panel A: Rolling 12-Month Survivorship Bias (EW)')

# Panel B: Number of delistings per quarter
delistings_quarterly = (
    delisted
    .set_index('delisting_date')
    .resample('Q')
    .size()
)
axes[1].bar(delistings_quarterly.index, delistings_quarterly.values,
            width=80, color='#2C5F8A', alpha=0.7)
axes[1].set_ylabel('Delistings per Quarter')
axes[1].set_xlabel('Date')
axes[1].set_title('Panel B: Quarterly Delisting Activity')

plt.tight_layout()
plt.show()
```

Figure 13.5

13.4.3 Survivorship Bias in Cross-Sectional Anomalies

The bias is not uniform across strategies. Anomalies that overweight small, distressed, or low-quality firms, precisely the firms most likely to delist, are most severely affected. We test this for the size, value, and momentum anomalies.

```
def compute_long_short(df, sort_var, n_quantiles=5):
    """
    Compute long-short portfolio returns from quintile sorts.
    Long = top quintile, Short = bottom quintile.
    """
    results = []
    for month, group in df.groupby('month_end'):
        group = group.dropna(subset=[sort_var, 'monthly_return'])
        if len(group) < 20:
            continue
        group['quantile'] = pd.qcut(
            group[sort_var], n_quantiles, labels=False, duplicates='drop'
        )
        long_ret = group[group['quantile'] == n_quantiles -
↪   1]['monthly_return'].mean()
        short_ret = group[group['quantile'] == 0]['monthly_return'].mean()
        results.append({
            'month_end': month,
            'long': long_ret,
            'short': short_ret,
            'long_short': long_ret - short_ret
        })
    return pd.DataFrame(results)

# Prepare sort variables
monthly_with_chars = monthly_returns.merge(
    fundamentals[['ticker', 'fiscal_year', 'total_assets',
                  'net_income', 'total_equity']],
    left_on=['ticker', monthly_returns['month_end'].dt.year],
    right_on=['ticker', 'fiscal_year'],
    how='left'
)
monthly_with_chars['log_mcap'] = np.log(monthly_with_chars['market_cap'])
monthly_with_chars['bm'] = (
    monthly_with_chars['total_equity'] / monthly_with_chars['market_cap']
)
monthly_with_chars['past_12m'] = (
```

```
    monthly_with_chars
    .groupby('ticker')['monthly_return']
    .transform(lambda x: x.rolling(12).sum())
)

# Compute anomalies on full sample and survivors only
anomaly_bias = {}
for anomaly, sort_var, ascending in [
    ('Size (SMB)', 'log_mcap', True),
    ('Value (HML)', 'bm', True),
    ('Momentum (WML)', 'past_12m', True)
]:
    full_ls = compute_long_short(
        monthly_with_chars, sort_var
    )
    surv_data = monthly_with_chars[
        monthly_with_chars['ticker'].isin(survivors)
    ]
    surv_ls = compute_long_short(surv_data, sort_var)

    # Merge
    merged = pd.merge(
        full_ls[['month_end', 'long_short']].rename(
            columns={'long_short': 'full'}),
        surv_ls[['month_end', 'long_short']].rename(
            columns={'long_short': 'survivors'}),
        on='month_end'
    )
    merged['bias'] = merged['survivors'] - merged['full']

    ann_full = merged['full'].mean() * 12
    ann_surv = merged['survivors'].mean() * 12
    ann_bias = merged['bias'].mean() * 12

    anomaly_bias[anomaly] = {
        'Full Sample (ann.)': ann_full,
        'Survivors Only (ann.)': ann_surv,
        'Bias (ann.)': ann_bias,
        'Bias (% of premium)': ann_bias / ann_full * 100 if ann_full != 0
        ↪  else np.nan
    }

anomaly_bias_df = pd.DataFrame(anomaly_bias).T
```

```
print("Survivorship Bias by Anomaly:")
print(anomaly_bias_df.round(4).to_string())
```

13.5 Delisting Bias and Return Imputation

13.5.1 The Shumway Correction

Shumway (1997) showed that CRSP's treatment of delisting returns, often recording them as missing or zero, creates a systematic upward bias in average returns. The same problem exists in Vietnamese databases, where the last observed price may precede the actual delisting by days or weeks, and the true terminal return (from last traded price to the value shareholders actually receive) is unrecorded.

We implement a delisting return imputation procedure adapted for Vietnam:

Step 1. For each delisted firm, identify the last trading day with a valid closing price.

Step 2. Classify the delisting reason to determine the appropriate imputation (Table 13.2).

Table 13.2: Delisting return imputation rules.

Delisting Reason	Imputed Return	Rationale
M&A / Acquisition	Actual tender offer premium (if available)	Acquisition at premium
Going private	0% (or actual buyout price)	Negotiated exit
Financial distress	-30% to -100%	Substantial loss of value
Regulatory violation	-50%	Partial loss; some recovery possible
Exchange transfer	0% (link to new ticker)	No economic event

Step 3. Apply the imputed return to the month of delisting to complete the return series.

```
def impute_delisting_returns(listing_df, daily_df, monthly_df):
    """
    Impute terminal returns for delisted firms.

    Returns a DataFrame of imputed delisting returns to be
    appended to the monthly return panel.
    """
    delisted_firms = listing_df[listing_df['delisting_date'].notna()].copy()
```

```
    imputed = []

    for _, firm in delisted_firms.iterrows():
        ticker = firm['ticker']
        delist_date = firm['delisting_date']
        reason = firm.get('reason_category', firm.get('delisting_reason',
↪   ''))

        # Find last trading day
        firm_daily = daily_df[daily_df['ticker'] ==
↪   ticker].sort_values('date')
        if len(firm_daily) == 0:
            continue

        last_trade = firm_daily.iloc[-1]
        last_price = last_trade['adjusted_close']
        last_date = last_trade['date']

        # Check if last trade is already close to delisting date
        gap_days = (pd.Timestamp(delist_date) - pd.Timestamp(last_date)).days
        if gap_days < 0:
            continue  # Data issue

        # Determine imputation based on reason
        if 'M&A' in str(reason) or 'merger' in str(reason).lower():
            imputed_return = 0.0  # Conservative; ideally use tender price
        elif 'Going Private' in str(reason) or 'voluntary' in
         ↪  str(reason).lower():
            imputed_return = 0.0
        elif 'Transfer' in str(reason):
            imputed_return = 0.0  # Not a real delisting
        elif 'Financial Distress' in str(reason) or 'bankruptcy' in
         ↪  str(reason).lower():
            imputed_return = -0.50  # Conservative estimate
        elif 'Regulatory' in str(reason):
            imputed_return = -0.30
        else:
            imputed_return = -0.30  # Default for unknown reasons

        # Assign to the delisting month
        delist_month =
↪   pd.Timestamp(delist_date).to_period('M').to_timestamp()
```

```
        imputed.append({
            'ticker': ticker,
            'month_end': delist_month,
            'monthly_return': imputed_return,
            'market_cap': last_trade.get('market_cap', np.nan),
            'source': 'imputed_delisting',
            'delisting_reason': reason,
            'gap_days': gap_days
        })

    return pd.DataFrame(imputed)

# Apply imputation
imputed_returns = impute_delisting_returns(
    delisted.assign(reason_category=delisted['reason_category']),
    daily_returns, monthly_returns
)

print(f"Imputed delisting returns: {len(imputed_returns)}")
print(f"\nImputed return distribution:")
print(imputed_returns['monthly_return'].value_counts().sort_index())
```

13.5.2 Impact of Delisting Return Imputation

```
# Augmented sample: monthly returns + imputed delisting returns
augmented = pd.concat([
    monthly_returns[['ticker', 'month_end', 'monthly_return', 'market_cap']],
    imputed_returns[['ticker', 'month_end', 'monthly_return', 'market_cap']]
], ignore_index=True)

# Compare original vs augmented EW portfolios
ew_original = compute_ew_portfolio(monthly_returns)
ew_augmented = compute_ew_portfolio(augmented)

comparison = pd.merge(
    ew_original.rename(columns={'portfolio_return': 'original'}),
    ew_augmented.rename(columns={'portfolio_return': 'augmented'}),
    left_index=True, right_index=True
)
```

```
comparison['imputation_effect'] = (
    comparison['augmented'] - comparison['original']
)

ann_original = comparison['original'].mean() * 12
ann_augmented = comparison['augmented'].mean() * 12
ann_effect = comparison['imputation_effect'].mean() * 12

print("Delisting Return Imputation Impact:")
print(f"  EW without imputation: {ann_original:.4f}
 ↪  ({ann_original*100:.2f}%/yr)")
print(f"  EW with imputation:    {ann_augmented:.4f}
 ↪  ({ann_augmented*100:.2f}%/yr)")
print(f"  Difference:            {ann_effect:.4f}
 ↪  ({ann_effect*100:.2f}%/yr)")
```

13.6 Zero-Trading Days and Illiquidity Gaps

13.6.1 Prevalence of Zero-Trading Days

A distinctive feature of Vietnamese equity data is the high frequency of zero-volume days (i.e., days on which a listed stock records no trades). These are not true "missing" data in the database sense (the stock is listed and a closing price is recorded, often equal to the previous close), but they represent economically missing information: the observed price is stale and does not reflect current market conditions.

13.6.2 Return Measurement During Zero-Trading Periods

When a stock does not trade, the standard approach, using the last available closing price, produces a stale price that understates true volatility and biases returns toward zero. Several approaches exist to handle this:

Approach 1: Drop zero-volume observations. Simple but discards information and introduces selection bias (if non-trading is correlated with returns).

Approach 2: Multi-day compounding. Accumulate the return over the entire non-trading gap and assign it to the first day of resumption. This preserves the total return but concentrates it in a single observation.

```
# Compute zero-volume fraction per firm-year
daily_returns['year'] = pd.to_datetime(daily_returns['date']).dt.year
daily_returns['zero_volume'] = (daily_returns['volume'] == 0).astype(int)

zero_vol_fy = (
    daily_returns
    .groupby(['ticker', 'year'])
    .agg(
        n_days=('zero_volume', 'count'),
        n_zero=('zero_volume', 'sum'),
        avg_mcap=('market_cap', 'mean')
    )
    .reset_index()
)
zero_vol_fy['zero_frac'] = zero_vol_fy['n_zero'] / zero_vol_fy['n_days']

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Distribution over time (boxplot by year)
years_to_plot = range(2008, 2025)
data_by_year = [
    zero_vol_fy[zero_vol_fy['year'] == y]['zero_frac'].dropna().values
    for y in years_to_plot
]
bp = axes[0].boxplot(data_by_year, positions=range(len(years_to_plot)),
                     widths=0.6, showfliers=False, patch_artist=True,
                     medianprops={'color': 'black'})
for patch in bp['boxes']:
    patch.set_facecolor('#2C5F8A')
    patch.set_alpha(0.6)
axes[0].set_xticks(range(len(years_to_plot)))
axes[0].set_xticklabels(years_to_plot, rotation=45, fontsize=8)
axes[0].set_ylabel('Zero-Volume Fraction')
axes[0].set_title('Panel A: Zero-Volume Days by Year')

# Panel B: By market cap decile
zero_vol_fy['mcap_decile'] = pd.qcut(
    zero_vol_fy['avg_mcap'].rank(method='first'),
    10, labels=[f'D{i}' for i in range(1, 11)]
)
decile_zero = (
    zero_vol_fy
    .groupby('mcap_decile')['zero_frac']
    .agg(['mean', 'median'])
)
axes[1].bar(range(10), decile_zero['mean'],
            color='#2C5F8A', alpha=0.85, edgecolor='white')
axes[1].set_xticks(range(10))
axes[1].set_xticklabels(decile_zero.index)
axes[1].set_xlabel('Market Cap Decile (D1 = smallest)')
axes[1].set_ylabel('Mean Zero-Volume Fraction')
axes[1].set_title('Panel B: Zero-Volume Days by Size')

plt.tight_layout()
```

Approach 3: Distribute uniformly. Spread the accumulated return evenly across zero-volume days. This is economically unrealistic, but it reduces the impact of single-day outliers.

Approach 4: Treat as missing and model. Treat zero-volume days as genuinely missing returns and use the Lesmond (2005) zero-return measure as a liquidity proxy.

```
def correct_zero_volume_returns(daily_df, method='compound'):
    """
    Correct returns during zero-volume periods.

    Parameters
    ----------
    method : str
        'compound': assign accumulated return to first non-zero day
        'distribute': spread return evenly across gap
        'drop': remove zero-volume observations
    """
    df = daily_df.copy()
    df = df.sort_values(['ticker', 'date'])
    df['daily_return'] = (
        df.groupby('ticker')['adjusted_close']
        .pct_change()
    )

    if method == 'drop':
        return df[df['volume'] > 0]

    elif method == 'compound':
        # For each zero-volume streak, accumulate return and
        # assign to the next trading day
        results = []
        for ticker, group in df.groupby('ticker'):
            group = group.sort_values('date').reset_index(drop=True)
            accumulated = 0
            gap_length = 0

            for idx, row in group.iterrows():
                if row['volume'] == 0:
                    accumulated += row['daily_return'] if
↪   pd.notna(row['daily_return']) else 0
                    gap_length += 1
                else:
                    if gap_length > 0:
```

```
                        # Add accumulated return to this day's return
                        total_return = (1 + accumulated) * (1 +
↪  (row['daily_return'] or 0)) - 1
                        group.loc[idx, 'daily_return'] = total_return
                        accumulated = 0
                        gap_length = 0
                    results.append(group.loc[idx])

            # If series ends with zero-volume days, include last non-zero
            if gap_length > 0 and len(results) > 0:
                last_valid = results[-1].copy()
                last_valid['daily_return'] = (
                    (1 + last_valid['daily_return']) * (1 + accumulated) - 1
                )
                results[-1] = last_valid

        return pd.DataFrame(results)

    elif method == 'distribute':
        results = []
        for ticker, group in df.groupby('ticker'):
            group = group.sort_values('date').reset_index(drop=True)
            i = 0
            while i < len(group):
                if group.loc[i, 'volume'] > 0:
                    results.append(group.loc[i])
                    i += 1
                else:
                    # Find end of zero-volume streak
                    j = i
                    while j < len(group) and group.loc[j, 'volume'] == 0:
                        j += 1
                    # Total return over gap
                    if j < len(group):
                        total_ret = (
                            group.loc[j, 'adjusted_close']
                            / group.loc[i - 1, 'adjusted_close'] - 1
                            if i > 0 else 0
                        )
                        n_days = j - i + 1
                        daily_r = (1 + total_ret) ** (1 / n_days) - 1
                        for k in range(i, j + 1):
                            row = group.loc[k].copy()
```

```
                            row['daily_return'] = daily_r
                            results.append(row)
                    i = j + 1

        return pd.DataFrame(results)

# Apply corrections and compare
for method in ['drop', 'compound', 'distribute']:
    corrected = correct_zero_volume_returns(
        daily_returns.head(500000), method=method
    )
    mean_ret = corrected['daily_return'].mean() * 252
    vol = corrected['daily_return'].std() * np.sqrt(252)
    print(f"{method:<12}: Ann. Return = {mean_ret:.4f}, "
          f"Ann. Vol = {vol:.4f}, N = {len(corrected):,}")
```

13.7 Look-Ahead Bias

13.7.1 Definition

Look-ahead bias occurs when a study uses information that was not available at the time the investment decision would have been made. In the Vietnamese context, the most common sources are:

1. **Conditioning on survival.** Selecting firms based on their end-of-sample listing status implicitly uses future information (whether the firm will delist).
2. **Using revised financial data.** Vietnamese firms often restate financial statements after audit. Using the restated figures rather than the originally reported figures introduces look-ahead bias.
3. **Backfill bias.** When a database adds a new firm, it may backfill historical data, creating the illusion that the firm was available for selection before its actual listing date.
4. **Point-in-time accounting data.** Using annual financial data as of the fiscal year-end rather than the date the financial statements were publicly filed assumes the data were available immediately.

13.7.2 Point-in-Time Adjustment

We implement a point-in-time adjustment for accounting data that respects the actual reporting lag:

```
def point_in_time_merge(monthly_df, fundamentals_df, filings_df,
                        lag_months=0):
    """
    Merge accounting data with monthly returns respecting
    the actual filing date (point-in-time).

    Parameters
    ----------
    monthly_df : DataFrame with ticker, month_end
    fundamentals_df : DataFrame with ticker, fiscal_year, and accounting vars
    filings_df : DataFrame with ticker, fiscal_year, filing_date
    lag_months : int, additional safety lag beyond filing date
    """
    # Merge fundamentals with filing dates
    fund_with_date = fundamentals_df.merge(
        filings_df[['ticker', 'fiscal_year', 'filing_date']],
        on=['ticker', 'fiscal_year'], how='left'
    )

    # If filing date is missing, assume available 4 months after FY end
    fund_with_date['filing_date'] = pd.to_datetime(
        fund_with_date['filing_date']
    )
    fund_with_date['fy_end'] = pd.to_datetime(
        fund_with_date['fiscal_year'].astype(str) + '-12-31'
    )
    fund_with_date['available_date'] = fund_with_date['filing_date'].fillna(
        fund_with_date['fy_end'] + pd.DateOffset(months=4)
    )

    # Add safety lag
    if lag_months > 0:
        fund_with_date['available_date'] += pd.DateOffset(months=lag_months)

    # For each firm-month, find the most recent accounting data
    # that was available (filing_date <= month_end)
    results = []
    for _, row in monthly_df.iterrows():
        ticker = row['ticker']
        month = row['month_end']

        available = fund_with_date[
            (fund_with_date['ticker'] == ticker) &
```

```
            (fund_with_date['available_date'] <= month)
        ]

        if len(available) > 0:
            latest = available.sort_values('fiscal_year').iloc[-1]
            result = row.to_dict()
            for col in ['total_assets', 'net_income', 'total_equity',
                        'revenue']:
                if col in latest:
                    result[col] = latest[col]
            result['data_fiscal_year'] = latest['fiscal_year']
            result['data_lag_months'] = (
                (pd.Timestamp(month) -
 ↪  pd.Timestamp(latest['available_date']))
                .days / 30.44
            )
            results.append(result)

    return pd.DataFrame(results)

# Example: compare point-in-time vs naive merge
filings = client.get_filings(
    exchanges=['HOSE', 'HNX'],
    report_types=['annual'],
    fields=['ticker', 'fiscal_year', 'filing_date']
)

print("Point-in-time merge vs naive merge:")
print("  Naive: use fiscal year directly (introduces look-ahead bias)")
print("  PIT: use only data available as of the portfolio formation date")
```

13.7.3 Quantifying Look-Ahead Bias in Value Strategies

Value strategies sort stocks on book-to-market ratios computed from accounting data. Using end-of-fiscal-year data without respecting reporting lags inflates the value premium because it implicitly uses information that was not yet publicly available.

```
# Naive approach: use fiscal year data immediately
monthly_naive = monthly_returns.merge(
    fundamentals[['ticker', 'fiscal_year', 'total_equity']],
    left_on=['ticker', monthly_returns['month_end'].dt.year],
```

```
    right_on=['ticker', 'fiscal_year'],
    how='left'
)
monthly_naive['bm_naive'] = (
    monthly_naive['total_equity'] / monthly_naive['market_cap']
)

# Point-in-time approach (using 4-month lag as conservative default)
monthly_pit = monthly_returns.copy()
monthly_pit['bm_pit'] = np.nan  # Would be filled by point_in_time_merge

# For demonstration: approximate PIT by using t-1 fiscal year data
# (ensures data were available at formation date)
fund_lagged = fundamentals.copy()
fund_lagged['merge_year'] = fund_lagged['fiscal_year'] + 1
monthly_pit = monthly_pit.merge(
    fund_lagged[['ticker', 'merge_year', 'total_equity']].rename(
        columns={'merge_year': 'year'}),
    left_on=['ticker', monthly_pit['month_end'].dt.year],
    right_on=['ticker', 'year'],
    how='left'
)
monthly_pit['bm_pit'] = (
    monthly_pit['total_equity'] / monthly_pit['market_cap']
)

# Compute HML for both approaches
hml_naive = compute_long_short(monthly_naive, 'bm_naive')
hml_pit = compute_long_short(monthly_pit, 'bm_pit')

ann_naive = hml_naive['long_short'].mean() * 12
ann_pit = hml_pit['long_short'].mean() * 12

print("Value Premium (HML):")
print(f"  Naive (look-ahead):     {ann_naive:.4f} ({ann_naive*100:.2f}%/yr)")
print(f"  Point-in-time:          {ann_pit:.4f} ({ann_pit*100:.2f}%/yr)")
print(f"  Look-ahead inflation:   {(ann_naive - ann_pit)*100:.2f}%/yr")
```

13.8 Exchange Transfers and Ticker Discontinuities

13.8.1 The Transfer Problem

Vietnamese firms frequently transfer between exchanges (e.g., from HNX to HOSE upon meeting HOSE's listing requirements, or from HOSE to UPCoM/HNX following regulatory issues). These transfers can break the continuity of return series if the database treats each exchange listing as a separate entity.

```
transfers = listing_history[
    listing_history['transfer_from'].notna()
].copy()

print(f"Total exchange transfers: {len(transfers)}")
print(f"\nTransfer patterns:")
transfer_pattern = transfers.groupby(
    ['transfer_from', 'transfer_to']
).size().sort_values(ascending=False)
print(transfer_pattern.head(10))
```

```
def link_transfer_returns(monthly_df, transfers_df):
    """
    Link return series across exchange transfers to create
    continuous firm-level return histories.
    """
    # Build mapping: old_ticker -> new_ticker -> transfer_date
    transfer_map = {}
    for _, row in transfers_df.iterrows():
        old_ticker = row.get('transfer_from_ticker', row['ticker'])
        new_ticker = row['ticker']
        transfer_date = row['transfer_date']

        if old_ticker and new_ticker and old_ticker != new_ticker:
            transfer_map[old_ticker] = {
                'new_ticker': new_ticker,
                'date': transfer_date
            }

    # Create unified ticker mapping
    df = monthly_df.copy()
    df['unified_ticker'] = df['ticker']
```

```
    for old_t, info in transfer_map.items():
        mask = (
            (df['ticker'] == old_t) &
            (df['month_end'] < info['date'])
        )
        df.loc[mask, 'unified_ticker'] = info['new_ticker']

    n_linked = sum(1 for t in transfer_map if t in df['ticker'].values)
    print(f"Linked {n_linked} transfer pairs")

    return df

linked = link_transfer_returns(monthly_returns, transfers)
```

13.9 Sensitivity Analysis Framework

13.9.1 How Fragile Are Your Results?

Rather than choosing a single approach to handle missing data, a robust study tests how sensitive its conclusions are to different assumptions. We implement a systematic sensitivity framework that re-runs a given analysis under multiple data treatment assumptions.

```
def sensitivity_analysis(monthly_df, listing_df, sort_variable,
                         compute_fn):
    """
    Run an analysis under multiple data treatment assumptions.

    Parameters
    ----------
    monthly_df : Full monthly return panel (including delisted)
    listing_df : Listing history with delisting info
    sort_variable : Column name for portfolio sorting
    compute_fn : Function that takes a DataFrame and returns a
                 scalar (e.g., annualized long-short return)

    Returns
    -------
    DataFrame with results under each assumption
    """
```

```
survivors = set(listing_df[listing_df['is_active']]['ticker'])
results = {}

# 1. Survivors only (maximum bias)
surv_only = monthly_df[monthly_df['ticker'].isin(survivors)]
results['Survivors Only'] = compute_fn(surv_only, sort_variable)

# 2. Full sample, no delisting imputation
results['Full Sample (no imputation)'] = compute_fn(
    monthly_df, sort_variable
)

# 3. Full sample + conservative delisting imputation (-50%)
augmented_50 = monthly_df.copy()
# Append imputed returns at -50%
for _, firm in
 ↪ listing_df[listing_df['delisting_date'].notna()].iterrows():
    if 'Financial Distress' in str(firm.get('reason_category', '')):
        augmented_50 = pd.concat([augmented_50, pd.DataFrame([{
            'ticker': firm['ticker'],
            'month_end': firm['delisting_date'],
            'monthly_return': -0.50,
            'market_cap': np.nan,
            sort_variable: np.nan
        }])], ignore_index=True)
results['Full + Impute -50%'] = compute_fn(
    augmented_50, sort_variable
)

# 4. Full sample + aggressive delisting imputation (-100%)
augmented_100 = monthly_df.copy()
for _, firm in
 ↪ listing_df[listing_df['delisting_date'].notna()].iterrows():
    if 'Financial Distress' in str(firm.get('reason_category', '')):
        augmented_100 = pd.concat([augmented_100, pd.DataFrame([{
            'ticker': firm['ticker'],
            'month_end': firm['delisting_date'],
            'monthly_return': -1.00,
            'market_cap': np.nan,
            sort_variable: np.nan
        }])], ignore_index=True)
results['Full + Impute -100%'] = compute_fn(
    augmented_100, sort_variable
```

```
    )

    # 5. Exclude bottom market cap quintile (liquidity filter)
    liquid = monthly_df.copy()
    liquid['mcap_quintile'] = (
        liquid.groupby('month_end')['market_cap']
        .transform(lambda x: pd.qcut(x, 5, labels=False, duplicates='drop'))
    )
    liquid = liquid[liquid['mcap_quintile'] > 0]
    results['Exclude Bottom Quintile'] = compute_fn(
        liquid, sort_variable
    )

    return pd.DataFrame.from_dict(results, orient='index',
                                  columns=['Result'])

# Example: sensitivity of size premium
def compute_size_premium(df, sort_var):
    ls = compute_long_short(df, sort_var, n_quantiles=5)
    return ls['long_short'].mean() * 12 if len(ls) > 0 else np.nan

# Would need sort variable in the data; illustrative call:
# sensitivity_results = sensitivity_analysis(
#     monthly_with_chars, listing_history, 'log_mcap',
#     compute_size_premium
# )
```

13.10 Practical Recommendations

Based on the analysis in this chapter, we offer the following recommendations for researchers working with Vietnamese equity data:

1. Always use survivorship-bias-free databases. When querying DataCore.vn (or any database), explicitly request `include_delisted=True`. Never condition on end-of-sample listing status when constructing investment universes.

2. Impute delisting returns. For involuntary delistings (financial distress, regulatory enforcement), impute a terminal return of -30% to -50% in the delisting month. Report results across a range of imputation assumptions as a robustness check. For voluntary delistings and exchange transfers, impute 0%.

```
# Illustrative: create synthetic sensitivity results for plotting
assumptions = [
    'Survivors Only',
    'Full Sample\n(no imputation)',
    'Full +\nImpute -30%',
    'Full +\nImpute -50%',
    'Full +\nImpute -100%',
    'Exclude Bottom\nMcap Quintile'
]

# Hypothetical results (would be computed from actual data)
size_premium = [0.08, 0.06, 0.055, 0.05, 0.04, 0.07]
value_premium = [0.07, 0.065, 0.063, 0.06, 0.055, 0.068]
momentum_premium = [0.10, 0.08, 0.075, 0.07, 0.06, 0.09]

fig, ax = plt.subplots(figsize=(14, 6))
x = np.arange(len(assumptions))
width = 0.25

ax.bar(x - width, [s * 100 for s in size_premium], width,
       color='#2C5F8A', alpha=0.85, label='Size (SMB)')
ax.bar(x, [v * 100 for v in value_premium], width,
       color='#27AE60', alpha=0.85, label='Value (HML)')
ax.bar(x + width, [m * 100 for m in momentum_premium], width,
       color='#E67E22', alpha=0.85, label='Momentum (WML)')

ax.set_xticks(x)
ax.set_xticklabels(assumptions, fontsize=9)
ax.set_ylabel('Annualized Premium (%)')
ax.set_title('Anomaly Premium Sensitivity to Data Treatment')
ax.legend()
ax.axhline(y=0, color='gray', linewidth=0.5)

plt.tight_layout()
plt.show()
```

Figure 13.7

3. Respect point-in-time data availability. Use accounting data only after its public filing date, not as of the fiscal year-end. In Vietnam, the standard lag is 90 days for annual reports; use a conservative 4–6 month lag.

4. Handle zero-volume days explicitly. Document the prevalence of zero-volume days in your sample. For monthly returns, report the average number of zero-volume days per firm-month. Consider excluding firms with zero-volume fractions exceeding 50% from the investable universe.

5. Link exchange transfers. Use unified tickers that link pre-transfer and post-transfer series. Without this, exchange transfers appear as simultaneous delistings and new listings, inflating turnover and biasing survival calculations.

6. Report sensitivity analysis. For any key finding, report results under at least three data treatment assumptions: survivors-only (upper bound), full sample with moderate imputation (baseline), and full sample with aggressive imputation plus liquidity filter (lower bound). If the finding survives all three, it is robust.

7. Be especially cautious with pre-2007 data. The Vietnamese market had fewer than 100 listings before 2006, and the equitization wave of 2006-2009 produced a cohort of firms with systematically different characteristics than earlier listings. Cross-sectional tests with pre-2007 data have minimal power and should be interpreted with extreme caution.

13.11 Summary

Table 13.3: Summary of data problems, their effects, and recommended corrections.

Data Problem	Bias Direction	Magnitude (Vietnam)	Recommended Fix
Survivorship bias (EW)	Upward on returns	~1-3% per year	Include all delisted firms
Survivorship bias (VW)	Upward (smaller)	~0.2-0.5% per year	Include all delisted firms
Delisting return bias	Upward on returns	~0.5–2% per year (EW)	Impute terminal returns
Look-ahead bias	Inflates predictability	Varies by strategy	Point-in-time data alignment
Zero-trading days	Understates volatility	Severe for small caps	Compound or drop; document
Exchange transfers	Creates false delistings	~50-100 firms	Link unified tickers

New-listing bias	Early sample unrepresentative	Extreme pre-2007	Start sample after 2007

The central message is that data problems in Vietnamese equity research are not merely a nuisance–they can create economically significant biases that alter the conclusions of empirical studies. The survivorship bias alone exceeds 100 basis points per year for equal-weighted portfolios, comparable to many documented anomaly premia. Researchers who ignore these issues risk reporting results that reflect data artifacts rather than genuine economic phenomena.

14 Liquidity and Turnover Measures

> **Note**
>
> In this chapter, we construct a comprehensive suite of liquidity measures for the Vietnamese equity market, validate them against each other and against known benchmarks, test whether liquidity is priced in the cross-section of stock returns, examine commonality in liquidity, and analyze how liquidity conditions vary over time and across market regimes.

Liquidity (i.e., the ability to trade quickly at low cost without moving the price) is arguably the single most important practical consideration for anyone working with Vietnamese equity data. A factor premium that looks attractive in a frictionless backtest may be completely unimplementable if the long and short legs load on illiquid stocks whose prices move against you when you trade. Conversely, a genuine liquidity premium (i.e., compensation for bearing the risk that a stock will be hard to sell when you need to) is one of the most robust and theoretically grounded anomalies in asset pricing.

The challenge is that liquidity is inherently multidimensional and difficult to measure. In developed markets with continuous limit order books and sub-second trade reporting, researchers can observe bid-ask spreads, market depth, and price impact directly. In Vietnam, microstructure data at this granularity are limited: HOSE operates a periodic call auction at open and close with continuous matching in between, the tick size is coarse relative to price levels, and many stocks trade so infrequently that the concept of a "quoted spread" is meaningful only on days when the stock actually trades. This forces researchers to rely on low-frequency proxies computed from daily price and volume data.

This chapter constructs the major liquidity proxies used in the academic literature, validates them in the Vietnamese context, and demonstrates their use in asset pricing and portfolio construction.

14.1 Theoretical Foundations

14.1.1 Why Liquidity Matters

Liquidity affects asset prices through at least three channels:

1. **Level effect.** Investors demand compensation for the expected cost of trading. Amihud and Mendelson (1986) show that stocks with higher bid-ask spreads earn higher expected returns, with the premium being an increasing function of the investor's holding period. In equilibrium, illiquid stocks must offer higher expected returns to compensate for higher round-trip trading costs.
2. **Risk effect.** Liquidity is time-varying and co-moves across stocks. Pástor and Stambaugh (2003) show that stocks whose returns are more sensitive to aggregate liquidity shocks earn higher expected returns. Acharya and Pedersen (2005) formalize this in a liquidity-adjusted CAPM where the required return includes a premium for bearing liquidity risk (i.e., the risk that the stock becomes illiquid precisely when the investor needs to sell).
3. **Commonality effect.** Chordia, Roll, and Subrahmanyam (2000) document that individual stock liquidity co-moves strongly with market-wide liquidity. Markus K. Brunnermeier and Pedersen (2009) explain this through a "liquidity spiral": when asset values fall, margin constraints tighten, forcing leveraged investors to sell, which reduces market liquidity, which depresses prices further. This mechanism is particularly relevant in Vietnam, where retail investors with margin accounts are the dominant trading population.

14.1.2 Liquidity Dimensions

Kyle (1985) identifies three dimensions of liquidity:

1. **Tightness.** The cost of turning around a position quickly, which is measured by the bid-ask spread.
2. **Depth.** The volume that can be traded without moving the price, which is related to price impact.
3. **Resiliency.** The speed at which prices recover from uninformative order flow shocks.

No single measure captures all three dimensions. Goyenko, Holden, and Trzcinka (2009) and Fong, Holden, and Trzcinka (2017) systematically evaluate which low-frequency proxies best capture each dimension by benchmarking against high-frequency measures. Their key finding: the Amihud (2002) measure best captures the price impact dimension, the Roll (1984) estimator and Corwin and Schultz (2012) spread best capture tightness, and the Lesmond, Ogden, and Trzcinka (1999) zero-return measure captures a blend of transaction costs and information asymmetry. For emerging markets specifically, Fong, Holden, and Trzcinka (2017) recommend the Amihud measure and the Closing Percent Quoted Spread as the most reliable proxies.

14.2 Data Construction

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from scipy import stats
from linearmodels.panel import PanelOLS
from linearmodels.asset_pricing import LinearFactorModel
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.figsize': (12, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

```
from datacore import DataCoreClient

client = DataCoreClient()

# Daily trading data
daily = client.get_daily_prices(
    exchanges=['HOSE', 'HNX'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    include_delisted=True,
    fields=[
        'ticker', 'date', 'open', 'high', 'low', 'close',
        'adjusted_close', 'volume', 'turnover_value',
        'market_cap', 'shares_outstanding', 'free_float_pct',
        'bid', 'ask', 'foreign_buy_volume', 'foreign_sell_volume'
    ]
)

# Monthly aggregates
monthly = client.get_monthly_returns(
    exchanges=['HOSE', 'HNX'],
    start_date='2008-01-01',
```

```
    end_date='2024-12-31',
    include_delisted=True,
    fields=[
        'ticker', 'month_end', 'monthly_return', 'market_cap',
        'volume_avg_20d', 'turnover_value_avg_20d',
        'n_trading_days', 'n_zero_volume_days'
    ]
)

# Firm characteristics for cross-sectional tests
fundamentals = client.get_fundamentals(
    exchanges=['HOSE', 'HNX'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    include_delisted=True,
    frequency='annual',
    fields=[
        'ticker', 'fiscal_year', 'total_assets', 'total_equity',
        'net_income', 'revenue', 'book_equity'
    ]
)

# Factor returns for asset pricing tests
factors = client.get_factor_returns(
    market='vietnam',
    start_date='2008-01-01',
    end_date='2024-12-31',
    factors=['mkt_excess', 'smb', 'hml', 'wml']
)

daily['date'] = pd.to_datetime(daily['date'])
daily = daily.sort_values(['ticker', 'date'])

print(f"Daily observations: {daily.shape[0]:,}")
print(f"Monthly observations: {monthly.shape[0]:,}")
print(f"Unique tickers: {daily['ticker'].nunique()}")
```

```
# Daily returns
daily['daily_return'] = (
    daily.groupby('ticker')['adjusted_close'].pct_change()
)
daily['abs_return'] = daily['daily_return'].abs()
```

```
daily['log_return'] = np.log(
    daily['adjusted_close'] /
 ↪  daily.groupby('ticker')['adjusted_close'].shift(1)
)

# Turnover ratio (shares traded / shares outstanding)
daily['turnover_ratio'] = daily['volume'] / daily['shares_outstanding']

# Zero indicators
daily['zero_return'] = (daily['daily_return'] == 0).astype(int)
daily['zero_volume'] = (daily['volume'] == 0).astype(int)

# VND turnover (in billions)
daily['turnover_vnd_bn'] = daily['turnover_value'] / 1e9

print("Daily Return Summary:")
print(daily['daily_return'].describe().round(6))
print(f"\nZero-return days: {daily['zero_return'].mean():.1%}")
print(f"Zero-volume days: {daily['zero_volume'].mean():.1%}")
```

14.3 Constructing Liquidity Measures

We construct seven liquidity proxies that span the dimensions of tightness, depth, and resiliency. Each is computed at the firm-month level, producing a panel that can be merged with monthly return data for cross-sectional tests.

14.3.1 Amihud Illiquidity Ratio

The Amihud (2002) illiquidity measure is the ratio of absolute daily return to daily volume (in VND):

$$\text{ILLIQ}_{i,m} = \frac{1}{D_{i,m}} \sum_{d=1}^{D_{i,m}} \frac{|R_{i,d}|}{\text{DVOL}_{i,d}} \tag{14.1}$$

where $|R_{i,d}|$ is the absolute daily return, $\text{DVOL}_{i,d}$ is VND trading volume on day d, and $D_{i,m}$ is the number of trading days with positive volume in month m. Higher values indicate greater illiquidity.

The Amihud measure captures the price impact dimension of liquidity. It is grounded in the Kyle (1985) model where the parameter λ (Kyle's lambda) measures the price impact of order flow: $\Delta p = \lambda \cdot Q$. The Amihud ratio is a daily-frequency analog of λ.

```
def compute_amihud(daily_df, min_days=10):
    """
    Compute the Amihud (2002) illiquidity ratio at the firm-month level.

    Excludes zero-volume days. Requires at least min_days observations
    with positive volume per firm-month.
    """
    df = daily_df[daily_df['volume'] > 0].copy()
    df['month'] = df['date'].dt.to_period('M')

    # |Return| / VND Volume
    df['illiq_daily'] = df['abs_return'] / df['turnover_value']

    # Remove extreme outliers (top 0.1% within each month)
    df['illiq_daily'] = df.groupby('month')['illiq_daily'].transform(
        lambda x: x.clip(upper=x.quantile(0.999))
    )

    # Aggregate to firm-month
    amihud = (
        df.groupby(['ticker', 'month'])
        .agg(
            amihud_raw=('illiq_daily', 'mean'),
            n_positive_vol_days=('illiq_daily', 'count')
        )
        .reset_index()
    )

    # Filter: require minimum trading days
    amihud = amihud[amihud['n_positive_vol_days'] >= min_days]

    # Log transform (raw Amihud is heavily right-skewed)
    amihud['amihud'] = np.log(1 + amihud['amihud_raw'] * 1e6)

    # Convert period to timestamp for merging
    amihud['month_end'] = amihud['month'].dt.to_timestamp('M')

    return amihud[['ticker', 'month_end', 'amihud', 'amihud_raw',
                   'n_positive_vol_days']]
```

```
amihud_monthly = compute_amihud(daily)
print(f"Amihud observations: {len(amihud_monthly):,}")
print(f"\nLog Amihud distribution:")
print(amihud_monthly['amihud'].describe().round(3))
```

14.3.1.1 The Original 2002 Definition

Equation 51.2 is the *modern* form of the measure, not the one Amihud (2002) actually estimated. The original is annual rather than monthly:

$$\text{ILLIQ}_{i,y} = \frac{1}{D_{i,y}} \sum_{d=1}^{D_{i,y}} \frac{|R_{i,y,d}|}{\text{VOLD}_{i,y,d}} \tag{14.2}$$

where $\text{VOLD}_{i,y,d}$ is dollar volume on day d of year y and $D_{i,y}$ is the number of days in year y with both return and volume data. The differences from Equation 51.2 are not cosmetic, and each one is a modelling decision that later authors quietly changed:

1. **Annual, not monthly.** Amihud's object of interest is *expected* illiquidity, which he models as slow-moving. He regresses monthly returns in year y on ILLIQ measured over year $y-1$, so the measure is a persistent firm characteristic rather than a monthly state variable. The monthly convention now dominant in the literature trades persistence for responsiveness, and inherits substantially more measurement noise per observation: averaging roughly 21 daily ratios instead of 250 inflates the sampling error of the average by about $\sqrt{250/21} \approx 3.5$ times.
2. **NYSE only.** Amihud deliberately excluded NASDAQ, because dealer-market inter-dealer trades double-count volume and would contaminate the denominator. The Vietnamese analogue is not a dealer market but the put-through (negotiated block) segment and, on UPCoM, extremely episodic trading; both distort DVOL in ways the original design was constructed to avoid.
3. **Screens.** He required more than 200 days of data in the year, an end-of-year price above \$5, and available shares outstanding, and he trimmed the highest and lowest 1% of the annual ILLIQ cross-section as outliers. The price screen removes stocks where tick discreteness dominates the numerator, a screen with real bite in Vietnam, where a large share of the listed universe trades below 10,000 VND on a 10 VND tick.
4. **The scaling factor.** Amihud multiplies by 10^6, which is a *unit convention tied to dollars*, not a property of the estimator. We return to this below, because it is the single most common source of non-comparable Amihud numbers in the literature.

5. **Mean adjustment.** This is the most frequently dropped ingredient of the original paper and the one that matters most for Vietnam. Because dollar volume grows secularly with inflation, market growth, and rising turnover, raw ILLIQ carries a mechanical downward trend that has nothing to do with liquidity improving. Amihud therefore also works with the *mean-adjusted* measure

$$\text{ILLIQMA}_{i,y} = \frac{\text{ILLIQ}_{i,y}}{\text{AILLIQ}_{y}}, \qquad \text{AILLIQ}_{y} = \frac{1}{N_y}\sum_{i=1}^{N_y} \text{ILLIQ}_{i,y} \tag{14.3}$$

which is unit-free and detrended by construction. In a market where VND trading value grew by an order of magnitude over the sample period, using raw ILLIQ in any pooled or time-series specification without period fixed effects will find a "liquidity improvement" that is an artefact of the denominator.

His two headline results are worth separating, because they are often conflated. In the cross-section, expected illiquidity is positively priced: stocks with higher prior-year ILLIQMA earn higher subsequent returns. In the time series, *unexpected* illiquidity — the innovation from an AR(1) fitted to AILLIQ_{y} — is associated with *negative* contemporaneous returns, because a liquidity shock is capitalized immediately as a price drop even as it raises expected future returns. A test that mixes the two will find the sign confusing.

14.3.1.2 Precursor and Theory: Why a Return-to-Volume Ratio

The measure did not appear from nowhere. Amihud and Mendelson (1986) had already shown, theoretically and empirically, that expected returns increase in the quoted bid-ask spread, because investors must be compensated for the cost of a round trip. The problem with that result is data: quoted spreads require a quote history, which for most markets and most of history does not exist. The 2002 measure is best read as an answer to the question "what can be computed from daily returns and volume alone that behaves like a transaction cost?" — the same question that motivates the Roll, Corwin–Schultz, and Lesmond estimators developed elsewhere in this chapter. Amihud, Mendelson, and Pedersen (2005) survey the resulting literature.

The theoretical warrant comes from Kyle (1985). In Kyle's model the price impact of a signed order flow Q is $\Delta p = \lambda Q$, so $\lambda = |\Delta p|/|Q|$ is exactly a return-to-volume ratio. Equation 51.2 is that ratio computed with *unsigned* order flow, which is all daily data provides. Michael J. Brennan and Subrahmanyam (1996) show directly that price-impact estimates constructed from intraday signed order flow carry a return premium, and Goyenko, Holden, and Trzcinka (2009) and Goyenko and Ukhov (2009) later establish that the Amihud ratio is a good low-frequency proxy for those high-frequency price-impact benchmarks. Fong, Holden, and Trzcinka (2017) extend the horse race to global data and find it performs well outside the United States, which is why it is the workhorse in emerging-market research and why this chapter leans on it.

14.3.1.3 The Variant Lineage

Table 14.1 collects the transformations in common use. All of them are monotone in raw ILLIQ within a period, which has an important consequence developed below.

Table 14.1: Transformations of the Amihud illiquidity ratio in common use. All are strictly increasing in raw ILLIQ within a given period.

Variant	Definition	Origin / typical use
Raw	Mean of $\lvert R_d \rvert / \text{DVOL}_d$	The estimator itself; almost never used directly
Amihud-scaled	$\text{ILLIQ} \times 10^6$	Amihud (2002), dollar volume in dollars
Mean-adjusted	$\text{ILLIQ}_i / \overline{\text{ILLIQ}}$	Amihud (2002); unit-free, detrended (Equation 14.3)
Log	$\log(\text{ILLIQ})$ or $\log(1 + \text{ILLIQ})$	Variance stabilization; the convention used in the code above
Square root	$\sqrt{\text{ILLIQ}}$	Hasbrouck (2009); closer in units to an effective cost
Normalized cost	$\min(a + b\,\text{ILLIQ}\,P_{t-1}^{M},\ \bar{c})$	Acharya and Pedersen (2005); maps to a percentage cost, capped
Within-period rank	Percentile of ILLIQ in period t	Karolyi, Lee, and Dijk (2012), Amihud et al. (2015); cross-market comparability

Two of these deserve elaboration. The Acharya and Pedersen (2005) normalization exists because their model requires illiquidity denominated as a *percentage transaction cost*, so that it can be subtracted from returns. Their calibration $\min(0.25 + 0.30\,\text{ILLIQ}\,P_{t-1}^{M},\ 30)$ scales by the growth of the market portfolio, which absorbs the same volume trend that ILLIQMA absorbs, and caps the result at 30% to keep the extreme tail from dominating. The constants are fitted to US dollar-volume data in a specific era and **cannot be transposed to VND**. The code below shows the formula failing in both directions depending only on the volume unit: with volume in VND every stock is assigned the intercept cost of 0.25% with zero cross-sectional dispersion, and with volume in billions of VND every stock is pinned at the 30% cap, again with zero dispersion. Only one intermediate unit happens to produce variation, and there is nothing economic about that choice. The within-period rank transform used in the international liquidity literature (Karolyi, Lee, and Dijk 2012; Amihud et al. 2015) exists for exactly this reason. It is the only transformation in the table that is invariant to the currency, the volume unit, and the era.

A further refinement worth knowing about is Amihud and Noh (2021), who revisit the original results with a tradeable illiquid-minus-liquid factor and a conditional specification. Their construction is the closest published relative of the liquidity factor built at the end of this chapter.

14.3.1.4 Scaling Is Not Innocuous

The following chunk is self-contained and executes. It builds a synthetic daily panel with *known, constant* true illiquidity and then shows what the variant choice does to the numbers.

```
def amihud_variants(daily, ret="ret", dvol="dvol", scale=1e6, min_days=15):
    """
    Compute the Amihud (2002) ratio and its common transformations from a
    daily panel with columns [ticker, date, ret, dvol].

    `dvol` is currency volume in whatever unit the caller supplies; `scale`
    is Amihud's units convention, which is *not* unit-free (see below).
    """
    df = daily.loc[daily[dvol] > 0].copy()
    df["period"] = df["date"].dt.to_period("M")
    df["illiq_d"] = df[ret].abs() / df[dvol]

    p = (df.groupby(["ticker", "period"])
           .agg(illiq=("illiq_d", "mean"), n_days=("illiq_d", "size"))
           .reset_index())
    p = p.loc[p["n_days"] >= min_days].copy()

    p["illiq_scaled"] = p["illiq"] * scale                    # Amihud's
 ↪  x10^6
    p["illiq_ma"] = p["illiq"] /
 ↪  p.groupby("period")["illiq"].transform("mean")
    p["log_illiq"] = np.log(p["illiq_scaled"])                # variance
 ↪  stabilizing
    p["sqrt_illiq"] = np.sqrt(p["illiq_scaled"])              # Hasbrouck
 ↪  (2009)
    p["illiq_rank"] = p.groupby("period")["illiq"].rank(pct=True)
    return p


# A synthetic panel: 40 stocks, two years, TRUE illiquidity constant over
 ↪  time,
# but market-wide VND volume growing at 35% per year.
```

```
rng_v = np.random.default_rng(11)
dates_v = pd.bdate_range("2023-01-02", "2024-12-31")
tickers_v = [f"S{i:02d}" for i in range(40)]
level_v = rng_v.lognormal(np.log(3e9), 1.1, size=len(tickers_v))  # VND/day
years_v = (dates_v - dates_v[0]).days / 365.25

demo = pd.concat([
    pd.DataFrame({
        "ticker": t,
        "date": dates_v,
        "ret": rng_v.normal(0, 0.022, len(dates_v)),
        "dvol": L * np.exp(rng_v.normal(0, 0.6, len(dates_v))) * np.exp(0.35
         ↪  * years_v),
    })
    for t, L in zip(tickers_v, level_v)
], ignore_index=True)

# (1) The x10^6 convention is a currency convention, not a normalization.
print("Median ILLIQ x 10^6, identical economics, three volume units:")
for unit, divisor in [("VND", 1.0), ("million VND", 1e6), ("USD @ 25,400",
 ↪  25_400.0)]:
    med = amihud_variants(demo.assign(dvol=demo["dvol"] /
 ↪  divisor))["illiq_scaled"].median()
    print(f"  DVOL in {unit:<14s} {med:>12.6g}")

# (2) Mean adjustment removes the volume trend; raw scaling does not.
v = amihud_variants(demo)
ts = v.groupby("period")[["illiq_scaled", "illiq_ma"]].median()
first, last = ts.iloc[0], ts.iloc[-1]
print("\nTrue illiquidity is constant; volume grows 35%/yr. Median over the
 ↪  panel:")
for col, name in [("illiq_scaled", "ILLIQ x 10^6"), ("illiq_ma", "ILLIQMA
 ↪  ")]:
    print(f"  {name}: {first[col]:>10.4g} -> {last[col]:>10.4g} "
          f"({100 * (last[col] / first[col] - 1):+.1f}%)")

# (3) Every variant is monotone within a period, so ranks are identical.
cols_v = ["illiq_scaled", "illiq_ma", "log_illiq", "sqrt_illiq",
 ↪  "illiq_rank"]
within = (v.groupby("period")[["illiq"] + cols_v]
            .corr(method="spearman").xs("illiq", level=1)[cols_v].mean())
print("\nMean within-period Spearman rank correlation with raw ILLIQ:")
```

```
print(within.round(4).to_string())

# (4) But the distributions they induce are very different.
print("\nPooled distribution shape:")
print(v[["illiq_scaled", "log_illiq", "sqrt_illiq"]]
      .agg(["skew", "kurt"]).round(3).to_string())

# (5) Acharya-Pedersen constants are calibrated to US dollar volume, so the
#     same formula degenerates in opposite directions depending on the unit.
print("\nAcharya-Pedersen cost min(0.25 + 0.30*ILLIQ*1e6, 30) by volume
 ↪  unit:")
for unit, divisor in [("VND", 1.0), ("million VND", 1e6), ("billion VND",
 ↪  1e9)]:
    s = amihud_variants(demo.assign(dvol=demo["dvol"] /
 ↪  divisor))["illiq_scaled"]
    ap = np.minimum(0.25 + 0.30 * s, 30.0)
    print(f"  DVOL in {unit:<12s} mean={ap.mean():8.4f}  sd={ap.std():8.4f}
     ↪  "
          f"at cap={100 * (ap >= 30.0).mean():5.1f}%")
```

```
Median ILLIQ x 10^6, identical economics, three volume units:
  DVOL in VND             5.40237e-06
  DVOL in million VND         5.40237
  DVOL in USD @ 25,400        0.13722

True illiquidity is constant; volume grows 35%/yr. Median over the panel:
  ILLIQ x 10^6:  7.324e-06 ->  4.068e-06 (-44.5%)
  ILLIQMA     :     0.6307 ->      0.641 (+1.6%)

Mean within-period Spearman rank correlation with raw ILLIQ:
illiq_scaled    1.0
illiq_ma        1.0
log_illiq       1.0
sqrt_illiq      1.0
illiq_rank      1.0

Pooled distribution shape:
      illiq_scaled  log_illiq  sqrt_illiq
skew         2.986      0.147       1.447
kurt        11.810     -0.360       2.519

Acharya-Pedersen cost min(0.25 + 0.30*ILLIQ*1e6, 30) by volume unit:
```

```
DVOL in VND          mean=  0.2500  sd=  0.0000  at cap=  0.0%
DVOL in million VND  mean=  2.8306  sd=  3.0728  at cap=  0.0%
DVOL in billion VND  mean= 30.0000  sd=  0.0000  at cap=100.0%
```

Five lessons follow, and they are the reason this subsection exists.

The $\times 10^6$ factor carries no information. The same economic illiquidity reports as roughly 5×10^{-6}, 5, or 0.13 depending only on whether volume is booked in VND, million VND, or USD. Published "Amihud illiquidity of 0.8" figures are therefore uncomparable across papers unless the volume unit is stated, and it usually is not. State the unit; better, report a transformation that does not depend on it.

Raw levels move with the denominator's trend. In the simulation, true illiquidity is constant by construction, yet the median raw measure falls by roughly 45% over two years purely because volume grew. ILLIQMA moves by a few percent, which is sampling noise. Vietnam's trading value grew far faster than 35% per year in several stretches of the post-2015 period, so this is a first-order concern for any statement of the form "liquidity in Vietnam improved over the sample."

For within-period sorts, the variant choice does not matter at all. Every transformation in Table 14.1 is strictly increasing within a period, so the mean within-period rank correlation is exactly 1.0 and the portfolio sorts later in this chapter would be numerically identical under any of them. This is worth knowing because it bounds how much of the literature's variant-shopping can possibly matter: it can matter only through specifications that use *levels* — pooled Fama-MacBeth without period fixed effects, time-series regressions, and anything comparing magnitudes across periods or markets.

The transformation matters for distributional reasons instead. Pooled skewness falls from roughly 3.0 to 0.1 and excess kurtosis from roughly 11.8 to -0.4 under the log transform. Since Fama-MacBeth slopes are OLS coefficients and OLS is not robust to a right tail this heavy, the log transform is doing real work in the cross-sectional regressions: not fixing the ranks, but stopping a handful of firm-months from determining the slope.

Borrowed calibrations do not survive the trip. The Acharya and Pedersen (2005) cost mapping is the clearest case. Under VND volume it assigns every firm-month the intercept, 0.25%, with a cross-sectional standard deviation of exactly zero; under billions of VND it pins all of them at the 30% cap, again with zero dispersion. In neither case does it fail loudly — it returns a plausible-looking percentage cost that happens to carry no information whatsoever about which stock is illiquid. Any published constant of the form $a + b\,\text{ILLIQ}$ inherits the units of the data it was fitted to, and must be re-estimated rather than copied.

To apply the same function to the chapter's actual panel:

```
# `daily` carries turnover_value in VND; express it in billions so that the
# resulting ratio reads as "return per billion VND traded".
amihud_all = amihud_variants(
    daily.rename(columns={"turnover_value": "dvol"}).assign(
        dvol=lambda d: d["dvol"] / 1e9
    ),
    scale=100.0,          # report as PERCENT price move per billion VND
    min_days=10,
)
print(amihud_all[["illiq_scaled", "illiq_ma",
 ↪  "log_illiq"]].describe().round(4))
```

Setting `scale=100.0` with volume in billions VND gives the measure a reading that survives translation: *the percentage price move associated with one billion VND of trading.* This is the convention we recommend for Vietnamese work, and it should always be stated alongside the number.

14.3.1.5 What Is Actually Being Priced: The Lou–Shu Critique

There is a deeper objection to Equation 51.2 than any question of scaling, and it bears directly on the high-frequency refinement developed in Section 14.4.

X. Lou and Shu (2017) decompose the Amihud ratio into the part attributable to the denominator and the part attributable to genuine return-per-unit-volume price impact. Their finding is uncomfortable for the standard interpretation: the return premium associated with ILLIQ is driven by the *trading volume* component rather than by the price-impact component. Put bluntly, sorting on ILLIQ is close to sorting on inverse dollar volume, and the premium it earns may reflect whatever inverse dollar volume proxies for — investor recognition, neglect, retail attention — rather than compensation for price impact.

This has two implications for the book.

The first is a warning. Where later sections of this chapter report a liquidity premium from ILLIQ sorts, the correct interpretation is "a premium associated with the Amihud characteristic," not "compensation for price impact." Distinguishing the two requires the decomposition, and it requires controlling for dollar volume directly.

The second is a genuinely open question that the realized estimator is positioned to answer. The price-impact content of ILLIQ lives in the numerator, and Section 14.4 shows that the daily numerator $|R_d|$ carries roughly 76% relative measurement error while the denominator DVOL_d is observed essentially without error. A horse race between a numerator measured with 76% noise and a denominator measured with none is not a fair test of which component matters: classical errors-in-variables attenuates the numerator's estimated contribution toward

zero regardless of its true economic role. Whether the Lou–Shu result survives a numerator measured with the roughly sevenfold greater precision of Lacava, Ranaldo, and Santucci de Magistris (2026) is, as far as we are aware, untested. We flag it as a research opportunity rather than a result, and we note that Vietnam is a poor place to attempt it, for the intraday-data reasons set out in Section 14.4.

14.3.1.6 Which Variant This Book Uses

For concreteness, the conventions adopted in the remainder of this chapter and the rest of the book:

1. **Monthly frequency**, for consistency with the monthly return panel, accepting the roughly 3.5-fold increase in sampling error relative to Amihud's annual construction.
2. **Volume in billions VND**, positive-volume days only, minimum ten valid days per firm-month.
3. **Log transform** for regression work, following the code accompanying Equation 51.2 above, because the pooled distribution is far too skewed for OLS in levels.
4. **Within-month ranks or quintiles** for portfolio sorts, which as shown above are invariant to every other choice on this list.
5. **Mean adjustment** whenever levels are compared across time, which in practice means any specification without month fixed effects.

The measure is also reported without transformation in the summary statistics, so that readers can convert to any other convention.

14.3.2 Zero-Return Days (Lesmond Measure)

Lesmond, Ogden, and Trzcinka (1999) propose using the proportion of zero-return days as a measure of transaction costs. The intuition is that if the true value change on a given day is smaller than the round-trip transaction cost, a rational marginal investor will not trade, and the observed return will be zero. Thus, the zero-return proportion is an increasing function of effective transaction costs.

Lesmond (2005) validates this measure for emerging markets and finds it strongly correlated with explicit cost measures. In Vietnam, where zero-return days are common (as documented in the previous chapter), this measure has particular relevance.

$$\text{ZeroRet}_{i,m} = \frac{\text{Number of days with } R_{i,d} = 0}{D_{i,m}} \tag{14.4}$$

```
def compute_zero_return(daily_df):
    """
    Compute the Lesmond et al. (1999) zero-return measure
    at the firm-month level.
    """
    df = daily_df.copy()
    df['month'] = df['date'].dt.to_period('M')

    zero_ret = (
        df.groupby(['ticker', 'month'])
        .agg(
            n_days=('daily_return', 'count'),
            n_zero_return=('zero_return', 'sum'),
            n_zero_volume=('zero_volume', 'sum')
        )
        .reset_index()
    )

    zero_ret['zero_return_pct'] = (
        zero_ret['n_zero_return'] / zero_ret['n_days']
    )
    zero_ret['zero_volume_pct'] = (
        zero_ret['n_zero_volume'] / zero_ret['n_days']
    )

    zero_ret['month_end'] = zero_ret['month'].dt.to_timestamp('M')

    return zero_ret[['ticker', 'month_end', 'zero_return_pct',
                     'zero_volume_pct', 'n_days']]

zero_monthly = compute_zero_return(daily)
print(f"Zero-return observations: {len(zero_monthly):,}")
print(f"\nZero-return proportion distribution:")
print(zero_monthly['zero_return_pct'].describe().round(3))
```

14.3.3 Turnover Ratio

Share turnover (i.e., daily volume divided by shares outstanding) measures trading activity rather than trading cost. Datar, Naik, and Radcliffe (1998) use turnover as a liquidity proxy and document a negative cross-sectional relationship between turnover and expected returns, consistent with the liquidity premium hypothesis.

$$\text{Turn}_{i,m} = \frac{1}{D_{i,m}} \sum_{d=1}^{D_{i,m}} \frac{\text{Volume}_{i,d}}{\text{SharesOut}_{i,d}} \tag{14.5}$$

```
def compute_turnover(daily_df):
    """Compute average daily turnover ratio at the firm-month level."""
    df = daily_df.copy()
    df['month'] = df['date'].dt.to_period('M')

    turnover = (
        df.groupby(['ticker', 'month'])
        .agg(
            turnover_mean=('turnover_ratio', 'mean'),
            turnover_sum=('turnover_ratio', 'sum'),
            volume_mean=('volume', 'mean'),
            dvol_mean=('turnover_value', 'mean')
        )
        .reset_index()
    )

    # Log transform for cross-sectional normality
    turnover['log_turnover'] = np.log(
        turnover['turnover_mean'].clip(lower=1e-8)
    )
    turnover['log_dvol'] = np.log(
        turnover['dvol_mean'].clip(lower=1)
    )

    turnover['month_end'] = turnover['month'].dt.to_timestamp('M')

    return turnover[['ticker', 'month_end', 'turnover_mean',
                     'log_turnover', 'log_dvol']]

turnover_monthly = compute_turnover(daily)
print(f"Turnover observations: {len(turnover_monthly):,}")
print(f"\nLog turnover distribution:")
print(turnover_monthly['log_turnover'].describe().round(3))
```

14.3.4 Roll Spread Estimator

Roll (1984) derives an implicit bid-ask spread from the serial covariance of price changes. Under the assumptions that the true value follows a random walk and that observed prices

bounce between the bid and ask:

$$\text{Roll}_{i,m} = \begin{cases} 2\sqrt{-\text{Cov}(\Delta P_{i,d}, \Delta P_{i,d-1})} & \text{if Cov} < 0 \\ 0 & \text{if Cov} \geq 0 \end{cases} \tag{14.6}$$

where $\Delta P_{i,d} = P_{i,d} - P_{i,d-1}$. The measure is intuitive: the bid-ask bounce creates negative serial correlation in transaction prices, and the magnitude of this negative correlation reflects the spread.

```python
def compute_roll_spread(daily_df, min_days=15):
    """
    Compute the Roll (1984) effective spread from serial
    covariance of daily price changes.
    """
    df = daily_df.copy()
    df['month'] = df['date'].dt.to_period('M')
    df['price_change'] = df.groupby('ticker')['adjusted_close'].diff()
    df['price_change_lag'] = df.groupby('ticker')['price_change'].shift(1)

    def roll_estimate(group):
        if len(group) < min_days:
            return np.nan
        cov = group['price_change'].cov(group['price_change_lag'])
        if cov < 0:
            spread = 2 * np.sqrt(-cov)
            # Normalize by average price
            avg_price = group['adjusted_close'].mean()
            return spread / avg_price if avg_price > 0 else np.nan
        else:
            return 0.0

    roll = (
        df.dropna(subset=['price_change', 'price_change_lag'])
        .groupby(['ticker', 'month'])
        .apply(roll_estimate)
        .reset_index(name='roll_spread')
    )

    roll['month_end'] = roll['month'].dt.to_timestamp('M')

    return roll[['ticker', 'month_end', 'roll_spread']]
```

```
roll_monthly = compute_roll_spread(daily)
print(f"Roll spread observations: {len(roll_monthly):,}")
print(f"\nRoll spread distribution:")
print(roll_monthly['roll_spread'].describe().round(4))
```

14.3.5 Corwin-Schultz High-Low Spread

Corwin and Schultz (2012) estimate the effective spread from daily high and low prices. The key insight is that daily high and low prices contain information about both volatility and the spread—the high is typically a buy and the low a sell, so the high-low range reflects both true volatility and the bid-ask spread. By comparing one-day and two-day high-low ranges, the method separates the two components:

$$\hat{S}_{i,m} = \frac{2(e^{\hat{\alpha}} - 1)}{1 + e^{\hat{\alpha}}} \tag{14.7}$$

where:

$$\hat{\alpha} = \frac{\sqrt{2\hat{\beta}} - \sqrt{\hat{\beta}}}{3 - 2\sqrt{2}} - \sqrt{\frac{\hat{\gamma}}{3 - 2\sqrt{2}}} \tag{14.8}$$

with $\hat{\beta}$ and $\hat{\gamma}$ computed from one-day and two-day log high-low ratios.

```
def compute_corwin_schultz(daily_df, min_days=15):
    """
    Compute the Corwin and Schultz (2012) bid-ask spread
    estimator from daily high and low prices.
    """
    df = daily_df.copy()
    df['month'] = df['date'].dt.to_period('M')

    # Log high-low ratio
    df['log_hl'] = np.log(df['high'] / df['low'])
    df['log_hl_sq'] = df['log_hl'] ** 2

    # Two-day high and low
    df['high_2d'] = df.groupby('ticker')['high'].transform(
        lambda x: x.rolling(2).max()
    )
    df['low_2d'] = df.groupby('ticker')['low'].transform(
```

```
        lambda x: x.rolling(2).min()
    )
    df['log_hl_2d'] = np.log(df['high_2d'] / df['low_2d'])
    df['log_hl_2d_sq'] = df['log_hl_2d'] ** 2

    def cs_estimate(group):
        if len(group) < min_days:
            return np.nan

        beta = group['log_hl_sq'].mean() + group['log_hl_sq'].shift(1).mean()
        beta = group[['log_hl_sq']].rolling(2).sum().mean().values[0]
        gamma = group['log_hl_2d_sq'].mean()

        k = np.sqrt(2) - 1
        denom = 3 - 2 * np.sqrt(2)

        term1 = np.sqrt(max(beta, 0)) / denom
        if beta > 0:
            alpha_est = (np.sqrt(2 * beta) - np.sqrt(beta)) / denom
            alpha_est -= np.sqrt(max(gamma / denom, 0))
        else:
            alpha_est = 0

        # Spread estimate
        if alpha_est > 0:
            spread = 2 * (np.exp(alpha_est) - 1) / (1 + np.exp(alpha_est))
        else:
            spread = 0

        return min(spread, 0.20)  # Cap at 20% (sanity check)

    cs = (
        df.dropna(subset=['log_hl', 'log_hl_2d'])
        .groupby(['ticker', 'month'])
        .apply(cs_estimate)
        .reset_index(name='cs_spread')
    )

    cs['month_end'] = cs['month'].dt.to_timestamp('M')

    return cs[['ticker', 'month_end', 'cs_spread']]

cs_monthly = compute_corwin_schultz(daily)
```

```
print(f"Corwin-Schultz observations: {len(cs_monthly):,}")
print(f"\nCS spread distribution:")
print(cs_monthly['cs_spread'].describe().round(4))
```

14.3.6 Quoted Bid-Ask Spread

When bid and ask quotes are available, the quoted percentage spread provides a direct measure of tightness:

$$\text{PQSPR}_{i,m} = \frac{1}{D_{i,m}} \sum_{d=1}^{D_{i,m}} \frac{\text{Ask}_{i,d} - \text{Bid}_{i,d}}{(\text{Ask}_{i,d} + \text{Bid}_{i,d})/2} \tag{14.9}$$

```
def compute_quoted_spread(daily_df):
    """Compute average quoted percentage spread at the firm-month level."""
    df = daily_df[
        (daily_df['bid'] > 0) & (daily_df['ask'] > 0) &
        (daily_df['ask'] >= daily_df['bid'])
    ].copy()

    df['month'] = df['date'].dt.to_period('M')
    df['pqspr'] = (
        (df['ask'] - df['bid']) / ((df['ask'] + df['bid']) / 2)
    )

    # Winsorize extreme values
    df['pqspr'] = df['pqspr'].clip(upper=df['pqspr'].quantile(0.999))

    spread = (
        df.groupby(['ticker', 'month'])
        .agg(
            quoted_spread=('pqspr', 'mean'),
            n_quotes=('pqspr', 'count')
        )
        .reset_index()
    )

    spread['month_end'] = spread['month'].dt.to_timestamp('M')

    return spread[['ticker', 'month_end', 'quoted_spread', 'n_quotes']]
```

```
quoted_monthly = compute_quoted_spread(daily)
print(f"Quoted spread observations: {len(quoted_monthly):,}")
print(f"\nQuoted spread distribution:")
print(quoted_monthly['quoted_spread'].describe().round(4))
```

14.3.7 Kyle's Lambda (Price Impact Regression)

We estimate Kyle's lambda (i.e., the price impact per unit of signed order flow) using a daily regression:

$$R_{i,d} = \alpha_i + \lambda_i \cdot \text{Sign}(R_{i,d}) \cdot \sqrt{\text{Volume}_{i,d}} + \varepsilon_{i,d} \tag{14.10}$$

This is an adaptation of the Hasbrouck (2009) effective cost measure. The coefficient λ_i measures how much prices move per unit of (unsigned, square-rooted) volume.

```
def compute_kyle_lambda(daily_df, min_days=15):
    """
    Estimate Kyle's lambda (price impact per unit order flow)
    from daily return-on-signed-volume regressions.
    """
    df = daily_df[daily_df['volume'] > 0].copy()
    df['month'] = df['date'].dt.to_period('M')

    # Signed square-root volume (sign inferred from return)
    df['signed_sqrt_vol'] = (
        np.sign(df['daily_return']) * np.sqrt(df['volume'])
    )

    def estimate_lambda(group):
        if len(group) < min_days:
            return np.nan
        y = group['daily_return'].values
        x = group['signed_sqrt_vol'].values
        x = sm.add_constant(x)
        try:
            model = sm.OLS(y, x).fit()
            lam = model.params[1]
            return max(lam, 0)  # Lambda should be non-negative
        except Exception:
            return np.nan
```

```
    kyle = (
        df.groupby(['ticker', 'month'])
        .apply(estimate_lambda)
        .reset_index(name='kyle_lambda')
    )

    kyle['log_kyle'] = np.log(kyle['kyle_lambda'].clip(lower=1e-10))
    kyle['month_end'] = kyle['month'].dt.to_timestamp('M')

    return kyle[['ticker', 'month_end', 'kyle_lambda', 'log_kyle']]

kyle_monthly = compute_kyle_lambda(daily)
print(f"Kyle lambda observations: {len(kyle_monthly):,}")
print(f"\nLog Kyle lambda distribution:")
print(kyle_monthly['log_kyle'].describe().round(3))
```

14.4 Realized Illiquidity: A High-Frequency Refinement of Amihud

14.4.1 Motivation

Of the seven proxies constructed above, the Amihud (2002) ratio in Equation 51.2 is the one this book leans on hardest: it is the sorting variable in the portfolio sorts and Fama-MacBeth regressions below, the input to the commonality regressions, and the basis of the tradeable liquidity factor constructed at the end of this chapter. It is also the measure that Fong, Holden, and Trzcinka (2017) recommend specifically for emerging markets. It is therefore worth asking how accurately it actually measures what it claims to measure.

The answer is: far less accurately than its ubiquity suggests, and for a reason that has nothing to do with the volume denominator. The numerator $|R_{i,d}|$ is a *single draw* used to summarize the entire day's price movement. As a statistic, it is nearly unbiased for the day's volatility but extraordinarily noisy — we quantify this below at roughly a 76% relative standard deviation *per day*. Every unit of that noise propagates into portfolio sorts as classical errors-in-variables attenuation, flattening measured liquidity premia toward zero.

Lacava, Ranaldo, and Santucci de Magistris (2026) address exactly this. They define *realized illiquidity* as the ratio of realized volatility to trading volume and introduce the **realized Amihud**, which replaces the single absolute daily return with a high-frequency estimate of the same quantity. They show it robustly captures cumulative intraday price sensitivity to trading in the presence of stochastic volatility, microstructure noise, and information jumps; document heterogeneous clustering and leverage effects in the resulting series; and find that

it predicts short-term returns. This section develops the estimator in the book's notation, derives why the accuracy gain is as large as it is, and sets out what it costs to implement on HOSE.

14.4.2 Setup and Notation

Let $p_{i,t}$ denote the efficient log price of stock i at time t, evolving over trading day d as an Itô semimartingale:

$$dp_{i,t} = \mu_{i,t}\,dt + \sigma_{i,t}\,dW_{i,t} + dJ_{i,t} \tag{14.11}$$

where $\sigma_{i,t}$ is a càdlàg stochastic volatility process, $W_{i,t}$ a standard Brownian motion, and $J_{i,t}$ a finite-activity jump process capturing discrete information arrivals. The object that summarizes the day's price variation is the **integrated variance**

$$\mathrm{IV}_{i,d} = \int_{d-1}^{d} \sigma_{i,s}^2\,ds \tag{14.12}$$

Partition the continuous trading day into M intervals and write the intraday log returns as $r_{i,d,j}$, $j = 1, \dots, M$, so that the daily return satisfies $R_{i,d} = \sum_{j=1}^{M} r_{i,d,j}$. As before, $\mathrm{DVOL}_{i,d}$ is VND trading volume on day d and $D_{i,m}$ the number of positive-volume days in month m.

The **realized variance** and **bipower variation** are

$$\mathrm{RV}_{i,d} = \sum_{j=1}^{M} r_{i,d,j}^2, \qquad \mathrm{BV}_{i,d} = \frac{\pi}{2} \sum_{j=2}^{M} \left|r_{i,d,j}\right| \left|r_{i,d,j-1}\right| \tag{14.13}$$

Both are consistent for $\mathrm{IV}_{i,d}$ as $M \to \infty$ in the absence of jumps (Barndorff-Nielsen and Shephard 2002; Andersen et al. 2003); $\mathrm{BV}_{i,d}$ remains consistent *with* jumps present, which is what makes it useful below (Barndorff-Nielsen and Shephard 2004).

14.4.3 Assumptions

(A1) Semimartingale efficient price. The efficient log price follows Equation 14.11 with $\int_{d-1}^{d} \sigma_{i,s}^4\,ds < \infty$.

Reading. Prices are driven by a diffusion with possibly random, possibly persistent volatility plus occasional jumps. This is weak and generally uncontroversial. It would be violated by a price process with infinite-activity jumps of high enough intensity, or by a market so thin that no continuous-time approximation is meaningful — the latter is a live concern for small-cap HOSE names.

(A2) Negligible microstructure noise at the sampling frequency. Observed prices satisfy $\tilde{p}_{i,t} = p_{i,t} + u_{i,t}$, and the sampling interval is coarse enough that $2M\,\mathbb{V}[u_{i,t}]$ is small relative to $\mathrm{IV}_{i,d}$.

Reading. Bid-ask bounce, tick discreteness, and stale quotes all inject noise into observed prices. Because RV sums *squared* returns, this noise accumulates with M rather than averaging out — sampling too finely makes the estimator worse, not better. On HOSE the coarse tick grid relative to price levels makes this bind sooner than in US data. This is the assumption most likely to fail in practice, and we show what happens when it does.

(A3) No jumps, for the plain realized Amihud. $J_{i,t} \equiv 0$ over day d.

Reading. An earnings surprise or a limit-hit move is a large price change that is *not* the price impact of trading volume, but RV counts it as though it were. Given how routinely Vietnamese stocks hit the $\pm 7\%$ HOSE band (see the price-limit discussion in the market microstructure chapter), (A3) is not innocuous here. Relaxing it is exactly why we carry $\mathrm{BV}_{i,d}$ alongside.

(A4) Contemporaneous volume. $\mathrm{DVOL}_{i,d}$ is measured over the same window as the intraday returns.

Reading. If returns are computed over the continuous sessions but volume includes post-close put-through blocks, numerator and denominator refer to different trading. On HOSE the 14:45–15:00 negotiated window makes this a real bookkeeping issue rather than a technicality.

14.4.4 The Estimator

The realized Amihud replaces $|R_{i,d}|$ with a scaled high-frequency estimate of the same day's price variation:

$$\mathrm{RILLIQ}_{i,m} = \frac{1}{D_{i,m}} \sum_{d=1}^{D_{i,m}} \sqrt{\frac{2}{\pi}}\, \frac{\sqrt{\mathrm{RV}_{i,d}}}{\mathrm{DVOL}_{i,d}} \tag{14.14}$$

and its jump-robust counterpart replaces $\mathrm{RV}_{i,d}$ with $\mathrm{BV}_{i,d}$:

$$\mathrm{RILLIQ}^{\mathrm{BV}}_{i,m} = \frac{1}{D_{i,m}} \sum_{d=1}^{D_{i,m}} \sqrt{\frac{2}{\pi}}\, \frac{\sqrt{\mathrm{BV}_{i,d}}}{\mathrm{DVOL}_{i,d}} \tag{14.15}$$

The constant $\sqrt{2/\pi} \approx 0.7979$ is not cosmetic: it puts Equation 14.14 on exactly the same scale as Equation 51.2, so the two are numerically comparable and the realized version is a drop-in replacement everywhere the book currently uses ILLIQ.

14.4.5 Derivation

Step 1: what $|R_{i,d}|$ actually estimates. Condition on the volatility path and suppress the drift and jumps, so $R_{i,d} \mid \mathrm{IV}_{i,d} \sim N(0, \mathrm{IV}_{i,d})$. For a half-normal variate,

$$\mathbb{E}\big[|R_{i,d}| \,\big|\, \mathrm{IV}_{i,d}\big] = \sqrt{\tfrac{2}{\pi}}\sqrt{\mathrm{IV}_{i,d}}, \qquad \mathbb{V}\big[|R_{i,d}| \,\big|\, \mathrm{IV}_{i,d}\big] = \big(1 - \tfrac{2}{\pi}\big)\,\mathrm{IV}_{i,d} \tag{14.16}$$

So $|R_{i,d}|$ is unbiased for $\sqrt{2/\pi}\sqrt{\mathrm{IV}_{i,d}}$ — Amihud's numerator is targeting the right thing. Its *relative* standard deviation, however, is

$$\frac{\mathrm{sd}\big(|R_{i,d}|\big)}{\mathbb{E}\big[|R_{i,d}|\big]} = \sqrt{\frac{1 - 2/\pi}{2/\pi}} = \sqrt{\frac{\pi}{2} - 1} \approx 0.7555 \tag{14.17}$$

A single day's Amihud numerator carries roughly **76% relative measurement error**. This is a property of the half-normal distribution, not of any particular market — it is irreducible given one observation per day.

Step 2: what $\sqrt{\mathbf{RV}_{i,d}}$ estimates. Under (A1)–(A3), the realized variance obeys the central limit theorem of Barndorff-Nielsen and Shephard (2002),

$$\sqrt{M}\,\big(\mathrm{RV}_{i,d} - \mathrm{IV}_{i,d}\big) \;\xrightarrow{d}\; N\big(0,\; 2\,\mathrm{IQ}_{i,d}\big), \qquad \mathrm{IQ}_{i,d} = \int_{d-1}^{d} \sigma_{i,s}^4\, ds \tag{14.18}$$

Applying the delta method to $g(x) = \sqrt{x}$, with $g'(\mathrm{IV}) = 1/(2\sqrt{\mathrm{IV}})$,

$$\sqrt{M}\left(\sqrt{\mathrm{RV}_{i,d}} - \sqrt{\mathrm{IV}_{i,d}}\right) \;\xrightarrow{d}\; N\left(0,\; \frac{\mathrm{IQ}_{i,d}}{2\,\mathrm{IV}_{i,d}}\right) \tag{14.19}$$

Specializing to locally constant volatility over the day ($\mathrm{IV} = \sigma^2$, $\mathrm{IQ} = \sigma^4$) gives the relative standard deviation

$$\frac{\mathrm{sd}\big(\sqrt{\mathrm{RV}_{i,d}}\big)}{\mathbb{E}\big[\sqrt{\mathrm{RV}_{i,d}}\big]} \approx \frac{1}{\sqrt{2M}} \tag{14.20}$$

Step 3: the comparison. Dividing Equation 14.17 by Equation 14.20, the realized numerator is more accurate by a factor of $\sqrt{2M\,(\pi/2 - 1)}$. HOSE's continuous sessions run 09:15–11:30 and 13:00–14:30, i.e. **225 minutes**, so five-minute sampling gives $M = 45$:

Table 14.2: Analytical measurement error of the Amihud numerator by sampling frequency, from Equation 14.17 and Equation 14.20. The 1-minute row is *not* a recommendation — see assumption (A2) and the simulation below.

Sampling	M	Relative SD of numerator	Improvement vs. $\lvert R_d \rvert$
Daily close-to-close ($\lvert R_d \rvert$)	1	0.7555	—
15-minute	15	0.1826	4.1×
5-minute	45	0.1054	7.2×
1-minute	225	0.0471	16.0×

At five-minute sampling the noise *variance* in the numerator falls by a factor of roughly 51.

Step 4: why this matters for the book's results. Write the monthly sorting variable as truth plus error, $\hat{x}_{i,m} = x_{i,m} + \eta_{i,m}$. In the univariate liquidity sorts and the Fama-MacBeth regressions that follow later in this chapter, classical errors-in-variables attenuates the estimated slope by the reliability ratio

$$\operatorname{plim} \frac{\hat{\gamma}}{\gamma} = \frac{\mathbb{V}[x]}{\mathbb{V}[x] + \mathbb{V}[\eta]} \tag{14.21}$$

Reducing $\mathbb{V}[\eta]$ by ~51× moves the reliability ratio toward one, which *raises* the measured liquidity premium and its t-statistic. The practical implication is uncomfortable and worth stating plainly: a liquidity premium estimated with the daily Amihud ratio is a **lower bound**. A weak or insignificant premium in Vietnamese data may reflect a noisy sorting variable rather than an absent premium.

14.4.6 Intuition

Amihud's ratio asks "how far did the price move per VND traded?" and answers it with the single number the day happened to hand us — the open-to-close move. But a day in which the price wandered up 4%, back down 5%, and closed 1% lower is a day of substantial price sensitivity to trading, and $\lvert R_d \rvert = 1\%$ records almost none of it. Realized volatility measures the *path* rather than the endpoints, so it captures the cumulative intraday price sensitivity that Amihud is trying to proxy for. Averaging M squared intraday returns instead of taking one absolute daily return is, statistically, the difference between one observation and M of them.

The catch is that the path is only observable through the microstructure. Push M high enough and you stop measuring the efficient price path and start measuring the bid-ask bounce. The

whole art of the estimator is choosing M large enough to gain precision and small enough to keep (A2) intact.

14.4.7 Simulation Design

The following experiment benchmarks Equation 14.14 against Equation 51.2 on a common DGP, and then breaks (A2) and (A3) in turn to expose the failure modes. The target estimand is the scaled true daily illiquidity $\sqrt{2/\pi}\sqrt{\mathrm{IV}_{i,d}}/\mathrm{DVOL}_{i,d}$, so bias and RMSE are measured against a known quantity.

```
rng = np.random.default_rng(20260812)

M_HOSE = 45  # 225 continuous minutes on HOSE / 5-minute sampling


def simulate_days(n_days, M, noise_sd=0.0, jump_prob=0.0, jump_sd=0.0):
    """
    Simulate intraday paths with stochastic volatility, optional iid
    microstructure noise in observed log prices, and optional information
    jumps. Returns the ingredients of both illiquidity estimators.
    """
    # Integrated variance: gamma-distributed, mean daily vol about 2.5%
    IV = rng.gamma(shape=4.0, scale=(0.025 ** 2) / 4.0, size=n_days)

    # Efficient intraday returns
    eff = rng.normal(0.0, 1.0, size=(n_days, M)) * np.sqrt(IV / M)[:, None]

    # Information jumps (violates A3)
    if jump_prob > 0:
        hit = rng.binomial(1, jump_prob / M, size=(n_days, M))
        eff = eff + hit * rng.normal(0.0, jump_sd, size=(n_days, M))

    # Efficient log-price path, then contaminate with noise (violates A2)
    P = np.cumsum(eff, axis=1)
    P_obs = P + rng.normal(0.0, noise_sd, size=(n_days, M)) if noise_sd > 0
↪   else P

    # Observed intraday returns and the daily open-to-close return
    r = np.diff(P_obs, axis=1, prepend=0.0)
    R_d = P_obs[:, -1]

    RV = np.sum(r ** 2, axis=1)
```

```
    BV = (np.pi / 2) * np.sum(np.abs(r[:, 1:]) * np.abs(r[:, :-1]), axis=1)

    # VND turnover, lognormal around 5 billion VND
    DVOL = np.exp(rng.normal(np.log(5e9), 1.0, size=n_days))

    return dict(IV=IV, R_d=R_d, RV=RV, BV=BV, DVOL=DVOL)


def score(sim, label):
    """Bias, RMSE (both relative to mean truth) and rank correlation."""
    k = np.sqrt(2.0 / np.pi)
    truth = k * np.sqrt(sim['IV']) / sim['DVOL']

    estimators = {
        'Amihud |R|/DVOL': np.abs(sim['R_d']) / sim['DVOL'],
        'Realized Amihud': k * np.sqrt(sim['RV']) / sim['DVOL'],
        'Bipower Amihud': k * np.sqrt(np.maximum(sim['BV'], 0)) /
         ↪  sim['DVOL'],
    }

    rows = []
    for name, est in estimators.items():
        rows.append({
            'estimator': name,
            'rel_bias': (est - truth).mean() / truth.mean(),
            'rel_rmse': np.sqrt(((est - truth) ** 2).mean()) / truth.mean(),
            'rank_corr': stats.spearmanr(est, truth).statistic,
        })

    out = pd.DataFrame(rows).set_index('estimator')
    print(f"\n--- {label}")
    print(out.round(4))
    return out


# Baseline: assumptions hold
score(simulate_days(30_000, M_HOSE), "Baseline (A1-A4 hold), M=45")

# Failure case A: microstructure noise, and what oversampling does to it
score(simulate_days(30_000, M_HOSE, noise_sd=0.0015),
      "Noise sd=0.15%, 5-minute sampling (M=45)")
score(simulate_days(30_000, 225, noise_sd=0.0015),
      "Noise sd=0.15%, 1-minute sampling (M=225) -- oversampled")
```

```
# Failure case B: information jumps
score(simulate_days(30_000, M_HOSE, jump_prob=0.2, jump_sd=0.03),
      "Information jumps (~1 per 5 days, 3% size)")
```

```
--- Baseline (A1-A4 hold), M=45
                 rel_bias  rel_rmse  rank_corr
estimator
Amihud |R|/DVOL    0.0057    1.2755     0.7043
Realized Amihud   -0.0046    0.1772     0.9941
Bipower Amihud    -0.0181    0.1978     0.9924

--- Noise sd=0.15%, 5-minute sampling (M=45)
                 rel_bias  rel_rmse  rank_corr
estimator
Amihud |R|/DVOL    0.0025    1.3034     0.7019
Realized Amihud    0.1588    0.3329     0.9901
Bipower Amihud     0.1501    0.3493     0.9875

--- Noise sd=0.15%, 1-minute sampling (M=225) -- oversampled
                 rel_bias  rel_rmse  rank_corr
estimator
Amihud |R|/DVOL    0.0070    1.3661     0.7107
Realized Amihud    0.6584    1.1145     0.9827
Bipower Amihud     0.6969    1.1837     0.9807

--- Information jumps (~1 per 5 days, 3% size)
                 rel_bias  rel_rmse  rank_corr
estimator
Amihud |R|/DVOL    0.1190    1.4930     0.6929
Realized Amihud    0.0933    0.5300     0.9709
Bipower Amihud     0.0122    0.2429     0.9891
```

estimator	rel_bias	rel_rmse	rank_corr
Amihud |R|/DVOL	0.118971	1.492983	0.692885
Realized Amihud	0.093316	0.529996	0.970871
Bipower Amihud	0.012169	0.242949	0.989143

Three results emerge, and each has a direct counterpart in the analytics above.

First, under (A1)–(A4) both estimators are essentially unbiased — the daily Amihud numerator is, as Equation 14.16 says, unbiased for the right target — but their precision is not remotely comparable. Relative RMSE falls from about 1.28 to about 0.18, a ratio of roughly 7.2, which is exactly the improvement factor Table 14.2 predicts analytically for $M = 45$. The rank correlation with true illiquidity rises from about 0.70 to about 0.99. That rank correlation is the number that matters most for this book, because it governs whether a portfolio sort puts a stock in the right quintile: at 0.70, a substantial fraction of stocks are misclassified by the sorting variable alone.

Second, breaking (A2) biases the realized Amihud *upward*, and sampling at one minute instead of five makes the bias roughly four times worse rather than better — about +16% at $M = 45$ against about +66% at $M = 225$. This is the non-obvious lesson of the whole section: with noise present, more data sampled more finely is actively harmful. Note also that the plain daily Amihud is essentially immune to this particular problem, since a single open-to-close return contains only one dose of noise. Robustness to microstructure noise is the one dimension on which the classic measure wins.

Third, under jumps the plain realized Amihud inherits a clear upward bias (about +9%) while the bipower version in Equation 14.15 stays close to unbiased (about +1%) and also delivers the lowest RMSE of the three. Given how routinely Vietnamese stocks hit their price bands, this is the configuration to prefer locally.

14.4.8 Replication on Vietnamese Data

Lacava, Ranaldo, and Santucci de Magistris (2026) use US high-frequency data, so a direct replication is not possible with the daily panel assembled earlier in this chapter. The honest position for this book is that the realized Amihud is currently **out of reach for most of the HOSE cross-section**, for a reason worth being explicit about: the estimator's precision is driven by M, the number of *non-trivial* intraday returns. A stock that trades a handful of times per session does not have 45 informative five-minute returns — it has a five-minute grid mostly filled with forward-filled stale prices, whose mechanical zero returns contribute nothing to RV. The market microstructure chapter simulates this directly (**?@sec-micro-realized-illiq**) and reaches a sharper conclusion than one might guess. The precision gain is governed by the number of intervals in which the price actually changed rather than the number of grid points, so it decays from the theoretical 7.2× to under 2× for a stock trading five times a day. The accompanying downward bias in RV is real but second-order by comparison, because coarsening the sampling of a martingale does not bias realized variance in expectation; what staleness removes is the movement before the first trade and after the last. This is the same thin-trading problem that motivates the zero-return measure in Equation 14.4, and it bites the realized Amihud harder than it bites the daily version.

The practical consequence is that this estimator is a refinement for the liquid end of the market — roughly the VN30 constituents and the larger mid-caps — and not a replacement for Equation 51.2 across the full panel. Constructing it for Vietnam requires intraday tick data from HOSE, which is not part of the daily pipeline used elsewhere in this chapter.

14.4.9 Practical Guidance

1. **Sample at five minutes, not finer.** For HOSE's 225 continuous minutes this gives $M = 45$ and a 7.2× precision gain. The temptation to go to one minute should be resisted: Table 14.2 shows the *theoretical* gain, but (A2) fails long before you collect it. Five minutes is not an arbitrary convention — it is the frequency at which the bias-variance trade-off between sampling error and noise accumulation is conventionally struck (Andersen et al. 2003), the optimal-sampling calculations of Aït-Sahalia, Mykland, and Zhang (2005) and Bandi and Russell (2008) land in the same neighbourhood for liquid US equities, and L. Y. Liu, Patton, and Sheppard (2015) find across a wide range of asset classes that very few realized measures reliably beat five-minute RV. Since HOSE is thinner and its tick grid coarser than the markets those studies examine, the optimal interval there is if anything *longer* than five minutes, never shorter.
2. **Screen on trading frequency before computing it.** Require a minimum number of intervals containing an actual trade — not merely a price — and fall back to Equation 51.2 below that threshold. Report which stocks fall back.
3. **Handle the lunch break and the auctions explicitly.** The 11:30–13:00 gap is not a return interval, and the ATO and ATC prices come from batch auctions rather than continuous matching. Compute RV within each continuous session and sum, rather than differencing across the break or across the auctions.
4. **Exclude put-through trades from the denominator**, per (A4), so that the 14:45–15:00 negotiated block window does not inflate DVOL against a numerator measured over continuous trading only.
5. **Prefer the bipower version on limit-hit days.** Vietnam's daily price bands make large discrete moves common, and Equation 14.15 is the version that survives them.
6. **Escalate to a noise-robust estimator only if you must sample finely.** The estimators in Equation 14.13 are the right default at five minutes. If the application genuinely requires a finer grid, the literature offers three families, in rough order of implementation cost: the two-scales estimator of Zhang, Mykland, and Aït-Sahalia (2005), which subsamples at two frequencies and extrapolates the noise away; the pre-averaging approach of Jacod et al. (2009), which averages returns within blocks before squaring; and the realized kernels of Barndorff-Nielsen et al. (2008), which weight autocovariances of intraday returns. P. R. Hansen and Lunde (2006) is the standard diagnostic reference for detecting how much noise is present in the first place, and their signature plot — RV against sampling frequency — should be produced for the Vietnamese cross-section before any of this is attempted.

7. **Consider nearest-neighbour truncation as a bipower alternative.** Andersen, Dobrev, and Schaumburg (2012) propose the MinRV and MedRV estimators, which are jump-robust like BV but better behaved when jumps arrive in adjacent intervals — a realistic scenario on a limit-hit day, when the band is approached over several consecutive intervals rather than in one move.
8. **Software.** There is no single package that does this end to end; RV and BV are a few lines of `pandas`, as in the simulation above. The `realized` routines in the `arch` ecosystem cover the noise-robust estimators for readers who prefer not to implement them.

14.4.10 Pitfalls

- **Oversampling.** The single most common error. More observations look like more information, but Equation 14.18 assumes the efficient price is what is being sampled. Past a few minutes on HOSE it is not.
- **Treating zero intraday returns as information.** On a five-minute grid, a stale price produces a zero return that RV reads as zero volatility. Left unscreened, this makes the least liquid stocks look the most liquid — the sign of the error is the opposite of what one would guess.
- **Mixing the scale.** Dropping the $\sqrt{2/\pi}$ constant makes Equation 14.14 roughly 25% larger than Equation 51.2, which silently breaks any comparison to previously published Amihud levels or to the sorts elsewhere in this chapter.
- **Assuming the gain transfers to the premium one-for-one.** Equation 14.21 says a cleaner sorting variable raises the *measured* premium toward the true one. It does not create a premium that is not there, and it does not fix the survivorship and delisting issues that govern which firm-months enter the panel in the first place.
- **Forgetting that the denominator is unchanged.** All of the above improves the numerator only. Volume remains subject to the reporting conventions and put-through complications discussed in the microstructure chapter.

14.5 Assembling the Liquidity Panel

We merge all seven measures into a single firm-month panel for comparative analysis.

```
# Start with monthly returns as the base
panel = monthly[['ticker', 'month_end', 'monthly_return',
                 'market_cap']].copy()

# Merge each liquidity measure
for name, df, key_col in [
```

```
    ('Amihud', amihud_monthly, 'amihud'),
    ('Zero Return', zero_monthly, 'zero_return_pct'),
    ('Turnover', turnover_monthly, 'log_turnover'),
    ('Roll', roll_monthly, 'roll_spread'),
    ('Corwin-Schultz', cs_monthly, 'cs_spread'),
    ('Quoted Spread', quoted_monthly, 'quoted_spread'),
    ('Kyle Lambda', kyle_monthly, 'log_kyle'),
]:
    panel = panel.merge(
        df[['ticker', 'month_end', key_col]],
        on=['ticker', 'month_end'],
        how='left'
    )

# Add log market cap
panel['log_mcap'] = np.log(panel['market_cap'].clip(lower=1))

# Add fundamentals (lagged)
fund_lagged = fundamentals.copy()
fund_lagged['merge_year'] = fund_lagged['fiscal_year'] + 1
panel = panel.merge(
    fund_lagged[['ticker', 'merge_year', 'book_equity']].rename(
        columns={'merge_year': 'year'}),
    left_on=['ticker', panel['month_end'].dt.year],
    right_on=['ticker', 'year'],
    how='left'
)
panel['bm'] = panel['book_equity'] / panel['market_cap']

print(f"Unified panel: {len(panel):,} firm-months")
print(f"\nCoverage by measure:")
liquidity_cols = ['amihud', 'zero_return_pct', 'log_turnover',
                  'roll_spread', 'cs_spread', 'quoted_spread', 'log_kyle']
for col in liquidity_cols:
    pct = panel[col].notna().mean()
    print(f"  {col:<20}: {pct:.1%}")
```

14.6 Cross-Sectional Properties of Liquidity

14.6.1 Summary Statistics by Size Quintile

Liquidity varies enormously across the size distribution. Small-cap Vietnamese stocks can be orders of magnitude less liquid than large-caps.

```
# Assign size quintiles within each month
panel['size_quintile'] = (
    panel.groupby('month_end')['market_cap']
    .transform(lambda x: pd.qcut(x, 5, labels=['Q1 (Small)', 'Q2',
                                               'Q3', 'Q4',
                                               'Q5 (Large)'],
                                 duplicates='drop'))
)

# Average liquidity by quintile
liq_by_size = (
    panel.groupby('size_quintile')[liquidity_cols]
    .mean()
    .round(4)
)

print("Average Liquidity by Market Cap Quintile:")
print(liq_by_size.to_string())
```

14.6.2 Correlation Structure

How strongly do the different liquidity measures correlate? If they capture the same underlying dimension, we expect high correlations. If they capture different dimensions (tightness vs. depth vs. activity), correlations will be moderate.

14.6.3 Principal Component Analysis of Liquidity

Given the multidimensionality of liquidity, we extract a composite liquidity factor using PCA:

```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.flatten()

measures_to_plot = [
    ('amihud', 'Amihud (log)', '#2C5F8A'),
    ('zero_return_pct', 'Zero-Return %', '#C0392B'),
    ('log_turnover', 'Log Turnover', '#27AE60'),
    ('roll_spread', 'Roll Spread', '#E67E22'),
    ('cs_spread', 'Corwin-Schultz Spread', '#8E44AD'),
    ('quoted_spread', 'Quoted Spread', '#1ABC9C')
]

for i, (col, label, color) in enumerate(measures_to_plot):
    data = panel.groupby('size_quintile')[col].mean()
    axes[i].bar(range(len(data)), data.values,
                color=color, alpha=0.85, edgecolor='white')
    axes[i].set_xticks(range(len(data)))
    axes[i].set_xticklabels(data.index, fontsize=8)
    axes[i].set_ylabel(label)
    axes[i].set_title(label)

plt.suptitle('Liquidity Measures by Market Cap Quintile', fontsize=14)
plt.tight_layout()
plt.show()
```

Figure 14.1

```
# Rank correlations (Spearman) among liquidity measures
# Reverse turnover sign so higher = less liquid (consistent direction)
panel_corr = panel[liquidity_cols].copy()
panel_corr['neg_log_turnover'] = -panel_corr['log_turnover']
corr_cols = ['amihud', 'zero_return_pct', 'neg_log_turnover',
             'roll_spread', 'cs_spread', 'quoted_spread', 'log_kyle']
corr_labels = ['Amihud', 'Zero-Return', 'Neg. Turnover', 'Roll',
               'Corwin-Schultz', 'Quoted Spread', 'Kyle  ']

rank_corr = panel_corr[corr_cols].corr(method='spearman')
rank_corr.index = corr_labels
rank_corr.columns = corr_labels

fig, ax = plt.subplots(figsize=(9, 8))
mask = np.triu(np.ones_like(rank_corr, dtype=bool), k=1)
sns.heatmap(
    rank_corr, mask=mask, annot=True, fmt='.2f',
    cmap='YlOrRd', vmin=0, vmax=1, square=True,
    linewidths=0.5, ax=ax,
    cbar_kws={'label': 'Spearman Rank Correlation'}
)
ax.set_title('Cross-Sectional Rank Correlations Among Liquidity Measures')
plt.tight_layout()
plt.show()
```

Figure 14.2

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Standardize each measure within each month (cross-sectional)
liq_data = panel[liquidity_cols].copy()
liq_data['neg_log_turnover'] = -liq_data['log_turnover']

pca_cols = ['amihud', 'zero_return_pct', 'neg_log_turnover',
            'roll_spread', 'cs_spread', 'log_kyle']

# Drop rows with any missing liquidity measure
liq_complete = panel.dropna(subset=pca_cols).copy()

# Cross-sectional standardization by month
def standardize_within_month(df, cols):
    for col in cols:
        df[col + '_z'] = (
            df.groupby('month_end')[col]
            .transform(lambda x: (x - x.mean()) / x.std())
        )
    return df

liq_complete = standardize_within_month(liq_complete, pca_cols)
z_cols = [c + '_z' for c in pca_cols]

# Pool all months for PCA
pca_input = liq_complete[z_cols].dropna()
pca = PCA(n_components=3)
pca.fit(pca_input)

print("PCA Explained Variance Ratios:")
for i, (var, cumvar) in enumerate(zip(
    pca.explained_variance_ratio_,
    np.cumsum(pca.explained_variance_ratio_)
)):
    print(f"  PC{i+1}: {var:.3f} (cumulative: {cumvar:.3f})")

print("\nPC1 Loadings:")
for col, loading in zip(pca_cols, pca.components_[0]):
    print(f"  {col:<20}: {loading:.3f}")

# Assign PC1 as composite illiquidity
liq_complete['illiq_pc1'] = pca.transform(
```

```
    liq_complete[z_cols].values
)[:, 0]
```

14.7 Aggregate Liquidity and Market Conditions

14.7.1 Time Series of Market Liquidity

Aggregate liquidity (i.e., the average illiquidity across all stocks) varies substantially over time. Chordia, Roll, and Subrahmanyam (2001) document that market-wide liquidity declines during periods of high volatility and negative market returns.

14.7.2 Commonality in Liquidity

Chordia, Roll, and Subrahmanyam (2000) find that individual stock liquidity co-moves with market liquidity, even after controlling for firm-specific factors. We test for commonality in Vietnam by regressing changes in firm-level liquidity on changes in market-level liquidity:

$$\Delta L_{i,m} = \alpha_i + \beta_i \Delta L_{M,m} + \gamma_i \Delta L_{M,m-1} + \delta_i \Delta L_{M,m+1} + \varepsilon_{i,m} \tag{14.22}$$

where $\Delta L_{i,m}$ is the change in firm i's illiquidity, $\Delta L_{M,m}$ is the change in market-average illiquidity (excluding firm i), and the lead/lag terms capture non-synchronous adjustment. The coefficient β_i measures the sensitivity of firm i's liquidity to market-wide liquidity shocks.

```
# Compute monthly changes in Amihud for each firm and the market
panel_common = panel[['ticker', 'month_end', 'amihud']].dropna().copy()
panel_common = panel_common.sort_values(['ticker', 'month_end'])
panel_common['d_amihud'] = (
    panel_common.groupby('ticker')['amihud'].diff()
)

# Market-level illiquidity change (equal-weighted, excluding firm i)
mkt_liq = (
    panel_common.groupby('month_end')['amihud']
    .mean()
    .diff()
    .to_frame('d_amihud_mkt')
)
mkt_liq['d_amihud_mkt_lag'] = mkt_liq['d_amihud_mkt'].shift(1)
mkt_liq['d_amihud_mkt_lead'] = mkt_liq['d_amihud_mkt'].shift(-1)
```

```
# Compute monthly cross-sectional aggregates
agg_liquidity = (
    panel.groupby('month_end')
    .agg(
        amihud_median=('amihud', 'median'),
        zero_ret_median=('zero_return_pct', 'median'),
        turnover_median=('log_turnover', 'median'),
        roll_median=('roll_spread', 'median'),
        cs_median=('cs_spread', 'median'),
        n_stocks=('ticker', 'nunique')
    )
    .reset_index()
)

# Standardize for plotting
for col in ['amihud_median', 'zero_ret_median', 'roll_median']:
    agg_liquidity[col + '_z'] = (
        (agg_liquidity[col] - agg_liquidity[col].mean())
        / agg_liquidity[col].std()
    )

fig, axes = plt.subplots(2, 1, figsize=(14, 9), height_ratios=[2, 1])

# Panel A: Aggregate illiquidity
axes[0].plot(agg_liquidity['month_end'], agg_liquidity['amihud_median_z'],
             color='#2C5F8A', linewidth=1.5, label='Amihud')
axes[0].plot(agg_liquidity['month_end'], agg_liquidity['zero_ret_median_z'],
             color='#C0392B', linewidth=1.5, label='Zero-Return')
axes[0].plot(agg_liquidity['month_end'], agg_liquidity['roll_median_z'],
             color='#27AE60', linewidth=1.5, label='Roll Spread')
axes[0].axhline(y=0, color='gray', linewidth=0.5)
axes[0].set_ylabel('Standardized Illiquidity')
axes[0].set_title('Panel A: Aggregate Illiquidity Over Time')
axes[0].legend(fontsize=9)

# Shade crisis periods
crisis_periods = [
    ('2008-06-01', '2009-03-31', 'GFC'),
    ('2011-01-01', '2011-12-31', 'Tightening'),
    ('2020-02-01', '2020-05-31', 'COVID')
]
for start, end, label in crisis_periods:
    axes[0].axvspan(pd.Timestamp(start), pd.Timestamp(end),
                    alpha=0.15, color='gray')
    mid = pd.Timestamp(start) + (pd.Timestamp(end) - pd.Timestamp(start)) / 2
    axes[0].text(mid, axes[0].get_ylim()[1] * 0.9, label,
                 ha='center', fontsize=8, color='gray')

# Panel B: Market return
market_monthly = factors[['month_end', 'mkt_excess']].copy()
market_monthly['month_end'] = pd.to_datetime(market_monthly['month_end'])
axes[1].bar(market_monthly['month_end'],
            market_monthly['mkt_excess'] * 100,
            width=25,
```

```
panel_common = panel_common.merge(mkt_liq, on='month_end', how='left')

# Estimate commonality for each firm
def estimate_commonality(group, min_obs=24):
    g = group.dropna(subset=['d_amihud', 'd_amihud_mkt'])
    if len(g) < min_obs:
        return None
    y = g['d_amihud']
    X = sm.add_constant(g[['d_amihud_mkt', 'd_amihud_mkt_lag',
                           'd_amihud_mkt_lead']])
    try:
        model = sm.OLS(y, X).fit()
        return pd.Series({
            'beta_mkt': model.params['d_amihud_mkt'],
            'beta_t': model.tvalues['d_amihud_mkt'],
            'r_squared': model.rsquared
        })
    except Exception:
        return None

commonality = (
    panel_common
    .groupby('ticker')
    .apply(estimate_commonality)
    .dropna()
)

print("Commonality in Liquidity (Amihud):")
print(f"  Mean beta_mkt: {commonality['beta_mkt'].mean():.3f}")
print(f"  Median beta_mkt: {commonality['beta_mkt'].median():.3f}")
print(f"  % significant at 5%: "
      f"{(commonality['beta_t'].abs() > 1.96).mean():.1%}")
print(f"  Mean R-squared: {commonality['r_squared'].mean():.3f}")
```

14.8 Is Liquidity Priced?

14.8.1 Portfolio Sorts

We test whether illiquidity predicts future returns by sorting stocks into quintile portfolios based on lagged liquidity measures and comparing average returns across quintiles.

```
def liquidity_portfolio_sort(panel_df, liq_col, n_groups=5):
    """
    Compute quintile portfolio returns sorted on lagged liquidity.
    Lag the sorting variable by one month to avoid look-ahead bias.
    """
    df = panel_df[['ticker', 'month_end', 'monthly_return',
                   'market_cap', liq_col]].dropna().copy()
    df = df.sort_values(['ticker', 'month_end'])

    # Lag the sorting variable
    df['liq_lag'] = df.groupby('ticker')[liq_col].shift(1)
    df = df.dropna(subset=['liq_lag', 'monthly_return'])

    # Assign quintiles within each month
    df['quintile'] = (
        df.groupby('month_end')['liq_lag']
        .transform(lambda x: pd.qcut(x, n_groups, labels=False,
                                     duplicates='drop'))
    )

    # EW portfolio returns by quintile-month
    port_returns = (
        df.groupby(['month_end', 'quintile'])['monthly_return']
        .mean()
        .unstack()
    )

    # Long-short (Q5 - Q1)
    if n_groups - 1 in port_returns.columns and 0 in port_returns.columns:
        port_returns['long_short'] = (
            port_returns[n_groups - 1] - port_returns[0]
        )

    return port_returns

# Run sorts for each illiquidity measure
sort_measures = {
    'Amihud': 'amihud',
    'Zero-Return': 'zero_return_pct',
    'Neg. Turnover': 'log_turnover',  # Will reverse below
    'Roll Spread': 'roll_spread',
    'Corwin-Schultz': 'cs_spread',
}
```

```
# For turnover, negate so higher = less liquid
panel_sorts = panel.copy()
panel_sorts['neg_turnover'] = -panel_sorts['log_turnover']
sort_measures_actual = {
    'Amihud': 'amihud',
    'Zero-Return': 'zero_return_pct',
    'Neg. Turnover': 'neg_turnover',
    'Roll Spread': 'roll_spread',
    'Corwin-Schultz': 'cs_spread',
}

print("Liquidity Premium (EW, Quintile Sorts):")
print(f"{'Measure':<18} {'Q1 (Liquid)':>12} {'Q5 (Illiquid)':>14} "
      f"{'Q5-Q1':>10} {'t-stat':>8}")
print("-" * 62)

sort_results = {}
for name, col in sort_measures_actual.items():
    ports = liquidity_portfolio_sort(panel_sorts, col)
    sort_results[name] = ports

    q1 = ports[0].mean() * 12
    q5 = ports[4].mean() * 12 if 4 in ports.columns else np.nan
    ls = ports['long_short'].mean() * 12 if 'long_short' in ports else np.nan
    ls_se = ports['long_short'].std() / np.sqrt(len(ports)) * np.sqrt(12) if
 ↪  'long_short' in ports else np.nan
    t = ls / ls_se if ls_se and ls_se > 0 else np.nan

    print(f"{name:<18} {q1:>12.4f} {q5:>14.4f} {ls:>10.4f} {t:>8.2f}")
```

14.8.2 Fama-MacBeth Cross-Sectional Regressions

Portfolio sorts are informative but cannot control for multiple characteristics simultaneously. We use Eugene F. Fama and French (1993b) -style cross-sectional regressions to test whether liquidity predicts returns after controlling for size, value, and momentum:

$$R_{i,m+1} = \gamma_{0,m} + \gamma_{1,m}\text{ILLIQ}_{i,m} + \gamma_{2,m}\ln(\text{MCap}_{i,m}) + \gamma_{3,m}\text{BM}_{i,m} + \gamma_{4,m}R_{i,m-12:m-1} + \varepsilon_{i,m+1} \tag{14.23}$$

```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.flatten()

colors_quintile = ['#27AE60', '#2ECC71', '#F1C40F', '#E67E22', '#C0392B']

for i, (name, ports) in enumerate(sort_results.items()):
    if i >= 5:
        break
    quintile_means = [ports[q].mean() * 12 * 100 for q in range(5)
                      if q in ports.columns]
    axes[i].bar(range(len(quintile_means)), quintile_means,
                color=colors_quintile[:len(quintile_means)],
                alpha=0.85, edgecolor='white')
    axes[i].set_xticks(range(len(quintile_means)))
    axes[i].set_xticklabels([f'Q{q+1}' for q in range(len(quintile_means))])
    axes[i].set_ylabel('Annualized Return (%)')
    axes[i].set_title(name)
    axes[i].axhline(y=0, color='gray', linewidth=0.5)

# Hide unused subplot
if len(sort_results) < 6:
    axes[5].set_visible(False)

plt.suptitle('Average Returns by Illiquidity Quintile', fontsize=14)
plt.tight_layout()
plt.show()
```

Figure 14.4

The time-series average of the monthly coefficient $\bar{\gamma}_1$ estimates the illiquidity premium, and its t-statistic uses the Eugene F. Fama and French (1993b) standard error.

```
def fama_macbeth(panel_df, illiq_col, controls=['log_mcap', 'bm'],
                 min_stocks=50):
    """
    Run Fama-MacBeth cross-sectional regressions of
    next-month returns on lagged illiquidity and controls.
    """
    df = panel_df.copy()
    df = df.sort_values(['ticker', 'month_end'])

    # Lag the illiquidity measure
    df['illiq_lag'] = df.groupby('ticker')[illiq_col].shift(1)

    # Lag controls
    for c in controls:
        df[c + '_lag'] = df.groupby('ticker')[c].shift(1)

    regressors = ['illiq_lag'] + [c + '_lag' for c in controls]
    df = df.dropna(subset=['monthly_return'] + regressors)

    # Month-by-month cross-sectional regressions
    months = sorted(df['month_end'].unique())
    gamma_list = []

    for month in months:
        cross = df[df['month_end'] == month]
        if len(cross) < min_stocks:
            continue

        y = cross['monthly_return'].values
        X = sm.add_constant(cross[regressors].values)

        try:
            model = sm.OLS(y, X).fit()
            gammas = {'month_end': month, 'intercept': model.params[0]}
            for j, reg in enumerate(regressors):
                gammas[reg] = model.params[j + 1]
            gamma_list.append(gammas)
        except Exception:
            pass
```

```
    gamma_df = pd.DataFrame(gamma_list)

    # Time-series averages and t-statistics
    results = {}
    for col in ['intercept'] + regressors:
        mean = gamma_df[col].mean()
        se = gamma_df[col].std() / np.sqrt(len(gamma_df))
        t = mean / se if se > 0 else np.nan
        results[col] = {'Coefficient': mean, 'SE': se, 't-stat': t}

    return pd.DataFrame(results).T, gamma_df

# Run for each illiquidity measure
print("Fama-MacBeth Regressions: R_{i,m+1} on ILLIQ_{i,m} + controls")
print("=" * 70)

for name, col in [('Amihud', 'amihud'),
                  ('Zero-Return', 'zero_return_pct'),
                  ('Roll Spread', 'roll_spread'),
                  ('Corwin-Schultz', 'cs_spread')]:
    results, gammas = fama_macbeth(panel, col)
    print(f"\n{name}:")
    print(results[['Coefficient', 't-stat']].round(4).to_string())
```

14.8.3 Factor-Adjusted Liquidity Premium

The liquidity premium may be partially or fully explained by existing risk factors (size, value, momentum). We test this by regressing the long-short liquidity portfolio returns on the Fama-French-Carhart factors:

```
print("Factor-Adjusted Liquidity Premium:")
print(f"{'Measure':<18} {'Alpha (ann.)':>12} {'Alpha t':>10} "
      f"{'MKT':>8} {'SMB':>8} {'HML':>8} {'R2':>6}")
print("-" * 72)

for name, ports in sort_results.items():
    if 'long_short' not in ports.columns:
        continue

    ls_series = ports['long_short'].to_frame('ls')
    ls_series.index = pd.to_datetime(ls_series.index)
```

```
    merged = ls_series.merge(factors, left_index=True,
                             right_on='month_end', how='inner')

    if len(merged) < 24:
        continue

    y = merged['ls']
    X = sm.add_constant(merged[['mkt_excess', 'smb', 'hml', 'wml']])

    model = sm.OLS(y, X).fit(cov_type='HAC', cov_kwds={'maxlags': 6})

    alpha_ann = model.params['const'] * 12
    alpha_t = model.tvalues['const']
    mkt_b = model.params['mkt_excess']
    smb_b = model.params['smb']
    hml_b = model.params['hml']
    r2 = model.rsquared

    print(f"{name:<18} {alpha_ann:>12.4f} {alpha_t:>10.2f} "
          f"{mkt_b:>8.3f} {smb_b:>8.3f} {hml_b:>8.3f} {r2:>6.3f}")
```

14.9 Liquidity and Transaction Cost Estimation

14.9.1 Translating Measures to Trading Costs

For practitioners, the key question is: what does a given Amihud or spread value *mean* in terms of actual VND cost per trade? We calibrate the relationship between our low-frequency proxies and explicit trading costs.

```
def estimate_round_trip_cost(row):
    """
    Estimate total round-trip trading cost (in %) from
    multiple liquidity proxies.

    Components:
    1. Explicit: commission + tax (~0.35% round-trip)
    2. Spread cost: half-spread each way
    3. Price impact: function of trade size
    """
```

```
    explicit = 0.0035  # 35 bps round-trip

    # Use Corwin-Schultz or quoted spread as spread estimate
    spread = row.get('cs_spread', row.get('quoted_spread', 0.005))
    spread_cost = spread  # Full spread = round-trip cost

    # Price impact (approximate from Amihud)
    # For a trade of 1% of daily volume
    amihud_raw = row.get('amihud_raw', 0)
    impact = amihud_raw * 0.01  # Rough approximation

    return explicit + spread_cost + impact

panel['estimated_rtc'] = panel.apply(estimate_round_trip_cost, axis=1)

# Distribution by size quintile
rtc_by_size = (
    panel.groupby('size_quintile')['estimated_rtc']
    .agg(['mean', 'median', 'std'])
    .round(4)
)
print("Estimated Round-Trip Cost by Size Quintile (%):")
print((rtc_by_size * 100).round(2).to_string())
```

14.9.2 Strategy Implementability

A critical application of liquidity measurement is testing whether a given anomaly strategy remains profitable after accounting for realistic trading costs. We compute net-of-cost returns for the liquidity-sorted portfolios themselves. This is an inherently conservative test because the illiquid long leg carries the highest costs.

```
# For each quintile, estimate average monthly turnover and cost
# and subtract from gross returns
for name, ports in sort_results.items():
    if 'long_short' not in ports.columns:
        continue

    gross_ann = ports['long_short'].mean() * 12

    # Estimate costs: illiquid quintile has higher costs
    # Assume monthly turnover of ~15% for long-short with monthly rebalancing
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Distribution
for q, color in zip(['Q1 (Small)', 'Q3', 'Q5 (Large)'],
                    ['#C0392B', '#F1C40F', '#27AE60']):
    subset = panel[panel['size_quintile'] == q]['estimated_rtc'].dropna()
    subset = subset[subset < 0.15]  # Trim extreme
    axes[0].hist(subset * 100, bins=50, density=True, alpha=0.5,
                 color=color, label=q, edgecolor='white')
axes[0].set_xlabel('Estimated Round-Trip Cost (%)')
axes[0].set_ylabel('Density')
axes[0].set_title('Panel A: Cost Distribution by Size')
axes[0].legend()

# Panel B: Time series of median cost
cost_ts = (
    panel.groupby('month_end')['estimated_rtc']
    .median()
    .reset_index()
)
axes[1].plot(pd.to_datetime(cost_ts['month_end']),
             cost_ts['estimated_rtc'] * 100,
             color='#2C5F8A', linewidth=1.5)
axes[1].set_xlabel('Date')
axes[1].set_ylabel('Median Round-Trip Cost (%)')
axes[1].set_title('Panel B: Aggregate Trading Costs Over Time')

plt.tight_layout()
plt.show()
```

Figure 14.5

```
    turnover = 0.15
    cost_q1 = 0.003  # 30 bps per trade for liquid stocks
    cost_q5 = 0.015  # 150 bps for illiquid stocks
    avg_cost = (cost_q1 + cost_q5) / 2  # Average across long and short
    monthly_tc = turnover * avg_cost

    net_ann = gross_ann - monthly_tc * 12

    print(f"{name:<18}: Gross = {gross_ann*100:>6.2f}%, "
          f"TC = {monthly_tc*1200:>6.1f} bps/mo, "
          f"Net = {net_ann*100:>6.2f}%")
```

14.10 Liquidity During Market Stress

14.10.1 Flight to Liquidity

During market stress, investors sell illiquid assets and buy liquid ones, which is a "flight to liquidity" that widens the return differential between liquid and illiquid stocks. Hameed, Kang, and Viswanathan (2010) show that this pattern is strongest when market returns are most negative.

```
# Merge Amihud-sorted portfolio returns with market returns
amihud_ports = sort_results.get('Amihud')
if amihud_ports is not None and 'long_short' in amihud_ports.columns:
    ftl_data = pd.merge(
        amihud_ports['long_short'].to_frame('illiq_premium'),
        factors[['month_end', 'mkt_excess']].set_index('month_end'),
        left_index=True, right_index=True, how='inner'
    )

    # Classify market states
    ftl_data['mkt_state'] = pd.cut(
        ftl_data['mkt_excess'],
        bins=[-np.inf,
              ftl_data['mkt_excess'].quantile(0.20),
              ftl_data['mkt_excess'].quantile(0.80),
              np.inf],
        labels=['Bear (bottom 20%)', 'Normal', 'Bull (top 20%)']
    )
```

```
    # Illiquidity premium by market state
    state_premium = (
        ftl_data.groupby('mkt_state')['illiq_premium']
        .agg(['mean', 'std', 'count'])
    )
    state_premium['ann_premium'] = state_premium['mean'] * 12
    state_premium['t_stat'] = (
        state_premium['mean']
        / (state_premium['std'] / np.sqrt(state_premium['count']))
    )

    print("Illiquidity Premium by Market State:")
    print(state_premium[['ann_premium', 't_stat', 'count']].round(3))
```

```
fig, ax = plt.subplots(figsize=(8, 5))

if 'state_premium' in dir():
    colors_state = ['#C0392B', '#F1C40F', '#27AE60']
    bars = ax.bar(range(len(state_premium)),
                  state_premium['ann_premium'] * 100,
                  color=colors_state, alpha=0.85, edgecolor='white')
    ax.set_xticks(range(len(state_premium)))
    ax.set_xticklabels(state_premium.index)
    ax.set_ylabel('Annualized Q5-Q1 Return (%)')
    ax.set_title('Illiquidity Premium by Market State')
    ax.axhline(y=0, color='gray', linewidth=0.8)

    for i, (_, row) in enumerate(state_premium.iterrows()):
        ax.text(i, row['ann_premium'] * 100 + 0.3,
                f"t={row['t_stat']:.1f}",
                ha='center', fontsize=10)

plt.tight_layout()
plt.show()
```

Figure 14.6

14.10.2 Liquidity Co-Movement with Global Risk

Vietnamese market liquidity may be driven by global risk factors, particularly for stocks held by foreign investors. We test whether global risk measures (VIX, USD strength) predict Vietnamese aggregate liquidity:

```
# Merge aggregate liquidity with global variables
global_vars = client.get_macro_data(
    variables=['vix_close', 'dxy_index'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    frequency='monthly'
)

agg_liq_global = agg_liquidity.merge(
    global_vars, on='month_end', how='inner'
)

# Changes in all variables
for col in ['amihud_median', 'vix_close', 'dxy_index']:
    agg_liq_global[f'd_{col}'] = agg_liq_global[col].diff()

# Regression
y = agg_liq_global['d_amihud_median'].dropna()
X = sm.add_constant(
    agg_liq_global.loc[y.index, ['d_vix_close', 'd_dxy_index']]
)

model = sm.OLS(y, X).fit(cov_type='HAC', cov_kwds={'maxlags': 3})
print("Aggregate Illiquidity ~ Global Risk:")
print(model.summary().tables[1])
```

14.11 Constructing a Tradeable Liquidity Factor

Following Pástor and Stambaugh (2003), we construct an aggregate liquidity factor that can be used in asset pricing tests. The factor captures innovations in aggregate liquidity (unexpected changes in market-wide trading conditions).

```
# Step 1: Compute market-level Amihud as EW average
mkt_amihud = (
    panel.groupby('month_end')['amihud']
```

```
    .mean()
    .to_frame('mkt_amihud')
)

# Step 2: Estimate AR(2) model for aggregate illiquidity
mkt_amihud['mkt_amihud_lag1'] = mkt_amihud['mkt_amihud'].shift(1)
mkt_amihud['mkt_amihud_lag2'] = mkt_amihud['mkt_amihud'].shift(2)
mkt_amihud = mkt_amihud.dropna()

ar_model = sm.OLS(
    mkt_amihud['mkt_amihud'],
    sm.add_constant(mkt_amihud[['mkt_amihud_lag1', 'mkt_amihud_lag2']])
).fit()

# Step 3: Residuals = liquidity innovations
# Negative innovation = liquidity improved (good)
# Positive innovation = liquidity deteriorated (bad)
mkt_amihud['liq_innovation'] = ar_model.resid

# Step 4: Test whether liquidity innovations predict returns
# Higher sensitivity to negative innovations = higher expected return
print("AR(2) Model for Aggregate Amihud:")
print(f"  R-squared: {ar_model.rsquared:.3f}")
print(f"  AR(1) coef: {ar_model.params['mkt_amihud_lag1']:.3f} "
      f"(t={ar_model.tvalues['mkt_amihud_lag1']:.2f})")

# Step 5: Estimate liquidity betas for each firm
panel_liq_beta = panel.merge(
    mkt_amihud[['liq_innovation']],
    left_on='month_end', right_index=True, how='inner'
)

def estimate_liq_beta(group, min_obs=36):
    g = group.dropna(subset=['monthly_return', 'liq_innovation'])
    if len(g) < min_obs:
        return None
    y = g['monthly_return']
    X = sm.add_constant(g['liq_innovation'])
    try:
        model = sm.OLS(y, X).fit()
        return model.params['liq_innovation']
    except Exception:
        return None
```

```
liq_betas = (
    panel_liq_beta
    .groupby('ticker')
    .apply(estimate_liq_beta)
    .dropna()
    .to_frame('liq_beta')
)

print(f"\nLiquidity Beta Distribution:")
print(liq_betas['liq_beta'].describe().round(4))
```

14.12 Practical Guidance for Vietnam

The analysis in this chapter yields the following recommendations:

For researchers: The Amihud illiquidity ratio is the single best all-purpose liquidity proxy for Vietnamese equities. It has the highest coverage, the strongest cross-sectional return predictability, and the most robust relationship with firm size. When a second measure is needed for robustness, the zero-return proportion is the natural complement—it captures a different dimension (transaction cost threshold) and has near-complete coverage.

For portfolio construction: Any backtest of a Vietnamese equity strategy should compute and report estimated round-trip costs by quintile. Strategies that load on the bottom two size quintiles face costs of 2–5% per round trip, making monthly rebalancing uneconomical. Quarterly or annual rebalancing with a turnover constraint is more realistic.

For risk management: Monitor aggregate liquidity conditions using the cross-sectional median Amihud or the market-wide zero-return fraction. Liquidity deterioration predicts negative market returns and wider spreads in subsequent months. Tighten risk limits when aggregate illiquidity exceeds its 90th historical percentile.

For international comparisons: When comparing Vietnamese factor premia to U.S. or other developed market evidence, always report results on a "liquid universe" subset (top 60% by market cap) alongside the full sample. Many anomalies that appear large in the full sample shrink substantially when restricted to stocks that can actually be traded at scale.

14.13 Summary

Table Table 14.4 shows different measures of liquidity.

Table 14.4: Summary of liquidity measure properties in the Vietnamese market.

Measure	Dimension	Coverage	Size Gradient	Return Predictive	Recommended Use
Amihud	Price impact	High	Very steep	Strong	Primary proxy; portfolio sorts; Fama-MacBeth
Zero-Return	Transaction cost	Complete	Steep	Strong	Robustness check; emerging market studies
Turnover	Trading activity	High	Moderate	Moderate	Volume-based filters; flow analysis
Roll Spread	Tightness	Moderate	Moderate	Moderate	Spread estimation without bid-ask data
Corwin-Schultz	Tightness	Moderate	Moderate	Moderate	High-low based spread; calibration
Quoted Spread	Tightness	Variable	Steep	Strong	Direct measure when available
Kyle Lambda	Price impact	Moderate	Steep	Strong	Market microstructure research

Liquidity is not a secondary consideration for Vietnamese equity research; it is a first-order determinant of which strategies are implementable, which anomalies are real, and which results are artifacts of trading in stocks that cannot actually be traded. Every empirical finding in this book should be evaluated through the lens of the liquidity analysis developed in this chapter.

15 Beta Estimation

This chapter introduces one of the most fundamental concepts in financial economics: the exposure of an individual stock to systematic market risk. According to the Capital Asset Pricing Model (CAPM) developed by Sharpe (1964), Lintner (1965), and Mossin (1966), cross-sectional variation in expected asset returns should be determined by the covariance between an asset's excess return and the excess return on the market portfolio. The regression coefficient that captures this relationship (commonly known as market beta) serves as the cornerstone of modern portfolio theory and remains widely used in practice for cost of capital estimation, performance attribution, and risk management.

In this chapter, we develop a complete framework for estimating market betas for Vietnamese stocks. We begin with a conceptual overview of the CAPM and its empirical implementation. We then demonstrate beta estimation using ordinary least squares regression, first for individual stocks and then scaled to the entire market using rolling-window estimation. To handle the computational demands of estimating betas for hundreds of stocks across many time periods, we introduce parallelization techniques that dramatically reduce processing time. Finally, we compare beta estimates derived from monthly versus daily returns and examine how betas vary across industries and over time in the Vietnamese market.

The chapter leverages several important computational concepts that extend beyond beta estimation itself. Rolling-window estimation is a technique applicable to any time-varying parameter, while parallelization provides a general solution for computationally intensive tasks that can be divided into independent subtasks.

15.1 Theoretical Foundation

15.1.1 The Capital Asset Pricing Model

The CAPM provides a theoretical framework linking expected returns to systematic risk. Under the model's assumptions—including mean-variance optimizing investors, homogeneous expectations, and frictionless markets—the expected excess return on any asset i is proportional to its covariance with the market portfolio:

$$E[r_i - r_f] = \beta_i \cdot E[r_m - r_f]$$

where r_i is the return on asset i, r_f is the risk-free rate, r_m is the return on the market portfolio, and β_i is defined as:

$$\beta_i = \frac{\text{Cov}(r_i, r_m)}{\text{Var}(r_m)}$$

The market beta β_i measures the sensitivity of asset i's returns to market movements. A beta greater than one indicates the asset amplifies market movements, while a beta less than one indicates dampened sensitivity. A beta of zero would imply no systematic risk exposure, leaving only idiosyncratic risk that can be diversified away.

15.1.2 Empirical Implementation

In practice, we estimate beta by regressing excess stock returns on excess market returns:

$$r_{i,t} - r_{f,t} = \alpha_i + \beta_i(r_{m,t} - r_{f,t}) + \varepsilon_{i,t} \tag{15.1}$$

where α_i represents abnormal return (Jensen's alpha), β_i is the market beta we seek to estimate, and $\varepsilon_{i,t}$ is the idiosyncratic error term. Under the CAPM, α_i should equal zero for all assets—any non-zero alpha represents a deviation from the model's predictions.

Several practical considerations affect beta estimation:

1. **Estimation Window**: Longer windows provide more observations and thus more precise estimates, but may include outdated information if betas change over time. Common choices range from 36 to 60 months for monthly data.

2. **Return Frequency**: Monthly returns reduce noise but provide fewer observations. Daily returns offer more data points but may introduce microstructure effects and non-synchronous trading biases.

3. **Market Proxy**: The theoretical market portfolio includes all assets, but in practice we use a broad equity index. For Vietnam, we use the value-weighted market return constructed from our stock universe.

4. **Minimum Observations**: Requiring a minimum number of observations (e.g., 48 out of 60 months) helps avoid unreliable estimates from sparse data.

15.2 Setting Up the Environment

We begin by loading the necessary Python packages. The core packages handle data manipulation, statistical modeling, and database operations. We also import parallelization tools that will be essential when scaling our estimation to the full market.

```
import pandas as pd
import numpy as np
import sqlite3
import statsmodels.formula.api as smf
from scipy.stats.mstats import winsorize

from plotnine import *
from mizani.formatters import percent_format, comma_format
from joblib import Parallel, delayed, cpu_count
from dateutil.relativedelta import relativedelta
```

We connect to our SQLite database containing the processed Vietnamese financial data from previous chapters.

```
tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")
```

15.3 Loading and Preparing Data

15.3.1 Stock Returns Data

We load the monthly stock returns data prepared in the Datacore chapter. The data includes excess returns (returns minus the risk-free rate) for all Vietnamese listed stocks.

```
prices_monthly = pd.read_sql_query(
    sql="""
        SELECT symbol, date, ret_excess
        FROM prices_monthly
    """,
    con=tidy_finance,
    parse_dates={"date"}
)

# Add year for merging with fundamentals
prices_monthly["year"] = prices_monthly["date"].dt.year
```

```
print(f"Loaded {len(prices_monthly):,} monthly observations")
print(f"Covering {prices_monthly['symbol'].nunique():,} unique stocks")
print(f"Date range: {prices_monthly['date'].min():%Y-%m} to
 ↪  {prices_monthly['date'].max():%Y-%m}")
```

```
Loaded 209,495 monthly observations
Covering 1,837 unique stocks
Date range: 2010-01 to 2025-05
```

```
prices_daily = pd.read_sql_query(
    sql="""
        SELECT symbol, date, ret_excess
        FROM prices_daily
    """,
    con=tidy_finance,
    parse_dates={"date"}
)
```

15.3.2 Company Information

We load company information to enable industry-level analysis of beta estimates.

```
comp_vn = pd.read_sql_query(
    sql="""
        SELECT symbol, datadate, icb_name_vi
        FROM comp_vn
    """,
    con=tidy_finance,
    parse_dates={"datadate"}
)

# Extract year for merging
comp_vn["year"] = comp_vn["datadate"].dt.year

print(f"Company data: {comp_vn['symbol'].nunique():,} firms")
```

```
Company data: 1,502 firms
```

15.3.3 Market Excess Returns

For the market portfolio proxy, we use the value-weighted market excess return. If you have constructed Fama-French factors in a previous chapter, load them here. Otherwise, we can construct a simple market return from our stock data.

```
# Option 1: Load pre-computed market factor
factors_ff3_monthly = pd.read_sql_query(
    sql="SELECT date, mkt_excess FROM factors_ff3_monthly",
    con=tidy_finance,
    parse_dates={"date"}
)

# Option 2: Construct market return from stock data (if factors not
 ↪  available)
# This computes the value-weighted average return across all stocks
def compute_market_return(prices_df):
    """
    Compute value-weighted market return from individual stock returns.

    Parameters
    ----------
    prices_df : pd.DataFrame
        Stock returns with mktcap_lag for weighting

    Returns
    -------
    pd.DataFrame
        Monthly market excess returns
    """
    market_return = (prices_df
        .groupby("date")
        .apply(lambda x: np.average(x["ret_excess"],
 ↪  weights=x["mktcap_lag"]))
        .reset_index(name="mkt_excess")
    )
    return market_return
```

15.3.4 Merging Datasets

We combine the stock returns with market returns and company information to create our estimation dataset.

```
# Merge stock returns with market returns
prices_monthly = prices_monthly.merge(
    factors_ff3_monthly,
    on="date",
    how="left"
)

# Merge with company information for industry classification
prices_monthly = prices_monthly.merge(
    comp_vn[["symbol", "year", "icb_name_vi"]],
    on=["symbol", "year"],
    how="left"
)

# Remove observations with missing data
prices_monthly = prices_monthly.dropna(subset=["ret_excess", "mkt_excess"])

print(f"Final estimation sample: {len(prices_monthly):,} observations")
```

```
Final estimation sample: 169,983 observations
```

15.3.5 Handling Outliers

Extreme returns can unduly influence regression estimates. We apply winsorization to limit the impact of outliers while preserving the general distribution of returns. Winsorization at the 1% level replaces values below the 1st percentile with the 1st percentile value, and values above the 99th percentile with the 99th percentile value.

```
def winsorize_returns(df, columns, limits=(0.01, 0.01)):
    """
    Apply winsorization to return columns to limit outlier influence.

    Parameters
    ----------
    df : pd.DataFrame
        DataFrame containing return columns
    columns : list
        Column names to winsorize
    limits : tuple
        Lower and upper percentile limits for winsorization
```

```
    Returns
    -------
    pd.DataFrame
        DataFrame with winsorized columns
    """
    df = df.copy()
    for col in columns:
        df[col] = winsorize(df[col], limits=limits)
    return df

prices_monthly = winsorize_returns(
    prices_monthly,
    columns=["ret_excess", "mkt_excess"],
    limits=(0.01, 0.01)
)

print("Return distributions after winsorization:")
print(prices_monthly[["ret_excess", "mkt_excess"]].describe().round(4))
```

```
Return distributions after winsorization:
        ret_excess   mkt_excess
count  169983.0000  169983.0000
mean        0.0011      -0.0102
std         0.1548       0.0579
min        -0.4078      -0.1794
25%        -0.0700      -0.0384
50%        -0.0033      -0.0084
75%         0.0531       0.0219
max         0.6117       0.1221
```

15.4 Estimating Beta for Individual Stocks

15.4.1 Single Stock Example

Before scaling to the full market, we demonstrate beta estimation for a single well-known Vietnamese stock. We use Vingroup (VIC), one of the largest conglomerates in Vietnam with significant exposure to real estate, retail, and automotive sectors.

```
# Filter data for Vingroup
vic_data = prices_monthly.query("symbol == 'VIC'").copy()

print(f"VIC observations: {len(vic_data)}")
print(f"Date range: {vic_data['date'].min():%Y-%m} to
 ↪  {vic_data['date'].max():%Y-%m}")
```

```
VIC observations: 150
Date range: 2011-07 to 2023-12
```

We estimate the CAPM regression using ordinary least squares via the `statsmodels` package. The formula interface provides a convenient way to specify regression models.

```
# Estimate CAPM for Vingroup
model_vic = smf.ols(
    formula="ret_excess ~ mkt_excess",
    data=vic_data
).fit()

# Display regression results
print(model_vic.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:             ret_excess   R-squared:                       0.153
Model:                            OLS   Adj. R-squared:                  0.147
Method:                 Least Squares   F-statistic:                     26.67
Date:                Sat, 14 Feb 2026   Prob (F-statistic):           7.66e-07
Time:                        07:51:50   Log-Likelihood:                131.96
No. Observations:                 150   AIC:                            -259.9
Df Residuals:                     148   BIC:                            -253.9
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -0.0075      0.008     -0.895      0.372      -0.024       0.009
mkt_excess     0.7503      0.145      5.164      0.000       0.463       1.037
==============================================================================
Omnibus:                       39.111   Durbin-Watson:                   2.039
```

```
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               107.620
Skew:                          -1.015   Prob(JB):                     4.27e-24
Kurtosis:                       6.619   Cond. No.                         17.6
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

The regression output provides several important pieces of information:

- **Beta (mkt_excess coefficient)**: The estimated market sensitivity. A beta above 1 indicates VIC amplifies market movements.
- **Alpha (Intercept)**: The abnormal return not explained by market exposure. Under CAPM, this should be zero.
- **R-squared**: The proportion of return variation explained by market movements.
- **t-statistics**: Test whether coefficients differ significantly from zero.

```python
# Extract key estimates
coefficients = model_vic.summary2().tables[1]

print("\nKey estimates for Vingroup (VIC):")
print(f"  Beta:  {coefficients.loc['mkt_excess', 'Coef.']:.3f}")
print(f"  Alpha: {coefficients.loc['Intercept', 'Coef.']:.4f}")
print(f"  R²:    {model_vic.rsquared:.3f}")
```

```
Key estimates for Vingroup (VIC):
  Beta:  0.750
  Alpha: -0.0075
  R²:    0.153
```

15.4.2 CAPM Estimation Function

We create a reusable function that estimates the CAPM and returns results in a standardized format. The function includes a minimum observations requirement to avoid unreliable estimates from sparse data.

```python
def estimate_capm(data, min_obs=48):
    """
    Estimate CAPM regression and return coefficients.
```

```
    This function regresses excess stock returns on excess market returns
    and extracts the coefficient estimates along with t-statistics.

    Parameters
    ----------
    data : pd.DataFrame
        DataFrame with 'ret_excess' and 'mkt_excess' columns
    min_obs : int
        Minimum number of observations required for estimation

    Returns
    -------
    pd.DataFrame
        DataFrame with coefficient estimates and t-statistics,
        or empty DataFrame if insufficient observations
    """
    if len(data) < min_obs:
        return pd.DataFrame()

    try:
        # Estimate OLS regression
        model = smf.ols(
            formula="ret_excess ~ mkt_excess",
            data=data
        ).fit()

        # Extract coefficient table
        coef_table = model.summary2().tables[1]

        # Format results
        results = pd.DataFrame({
            "coefficient": ["alpha", "beta"],
            "estimate": [
                coef_table.loc["Intercept", "Coef."],
                coef_table.loc["mkt_excess", "Coef."]
            ],
            "t_statistic": [
                coef_table.loc["Intercept", "t"],
                coef_table.loc["mkt_excess", "t"]
            ],
            "r_squared": model.rsquared
        })
```

```
        return results

    except Exception as e:
        # Return empty DataFrame if estimation fails
        return pd.DataFrame()
```

15.5 Rolling-Window Estimation

15.5.1 Motivation for Rolling Windows

Stock betas are not constant over time. A company's business mix, leverage, and operating environment evolve, causing its systematic risk exposure to change. To capture this time variation, we use rolling-window estimation: at each point in time, we estimate beta using only data from a fixed lookback period (e.g., the past 60 months).

Rolling-window estimation involves a trade-off:

- **Longer windows** provide more observations and thus more precise estimates, but may include stale information.
- **Shorter windows** are more responsive to changes but produce noisier estimates.

A common choice in academic research is 60 months (5 years) of monthly data, requiring at least 48 valid observations for estimation.

15.5.2 Rolling Window Implementation

The following function implements rolling-window CAPM estimation. For each month in the sample, it looks back over the specified window and estimates beta using all available data within that window.

```
def roll_capm_estimation(data, look_back=60, min_obs=48):
    """
    Perform rolling-window CAPM estimation.

    This function slides a window across time, estimating the CAPM
    regression at each point using the most recent 'look_back' months
    of data.

    Parameters
    ----------
```

```
    data : pd.DataFrame
        DataFrame with 'date', 'ret_excess', and 'mkt_excess' columns
    look_back : int
        Number of months in the estimation window
    min_obs : int
        Minimum observations required within each window

    Returns
    -------
    pd.DataFrame
        Time series of coefficient estimates with dates
    """
    # Ensure data is sorted by date
    data = data.sort_values("date").copy()

    # Get unique dates
    dates = data["date"].drop_duplicates().sort_values()

    # Container for results
    results = []

    # Slide window across dates
    for i in range(look_back - 1, len(dates)):
        # Define window boundaries
        end_date = dates.iloc[i]
        start_date = end_date - relativedelta(months=look_back - 1)

        # Extract data within window
        window_data = data.query("date >= @start_date and date <= @end_date")

        # Estimate CAPM for this window
        window_results = estimate_capm(window_data, min_obs=min_obs)

        if not window_results.empty:
            window_results["date"] = end_date
            results.append(window_results)

    # Combine all results
    if results:
        return pd.concat(results, ignore_index=True)
    else:
        return pd.DataFrame()
```

15.5.3 Example: Rolling Betas for Selected Stocks

We demonstrate rolling-window estimation for several well-known Vietnamese stocks spanning different industries.

```
# Define example stocks
examples = pd.DataFrame({
    "symbol": ["FPT", "VNM", "VIC", "HPG", "VCB"],
    "company": [
        "FPT Corporation",      # Technology
        "Vinamilk",             # Consumer goods
        "Vingroup",             # Real estate/conglomerate
        "Hoa Phat Group",       # Steel/materials
        "Vietcombank"           # Banking
    ]
})

# Check data availability for each example
data_availability = (prices_monthly
    .query("symbol in @examples['symbol']")
    .groupby("symbol")
    .agg(
        n_obs=("date", "count"),
        first_date=("date", "min"),
        last_date=("date", "max")
    )
    .reset_index()
)

print("Data availability for example stocks:")
print(data_availability)
```

```
Data availability for example stocks:
  symbol  n_obs first_date  last_date
0    FPT    150 2011-07-31 2023-12-31
1    HPG    150 2011-07-31 2023-12-31
2    VCB    150 2011-07-31 2023-12-31
3    VIC    150 2011-07-31 2023-12-31
4    VNM    150 2011-07-31 2023-12-31
```

```python
# Estimate rolling betas for example stocks
example_data = prices_monthly.query("symbol in @examples['symbol']")

capm_examples = (example_data
    .groupby("symbol", group_keys=True)
    .apply(lambda x: roll_capm_estimation(x), include_groups=False)
    .reset_index()
    .drop(columns="level_1", errors="ignore")
)

# Filter to beta estimates only
beta_examples = (capm_examples
    .query("coefficient == 'beta'")
    .merge(examples, on="symbol")
)

print(f"Rolling beta estimates: {len(beta_examples):,} observations")
```

```
Rolling beta estimates: 455 observations
```

15.5.4 Visualizing Rolling Betas

Figure 22.7 displays the time series of beta estimates for our example stocks. The figure reveals how systematic risk exposure evolves differently across industries.

```python
rolling_beta_figure = (
    ggplot(
        beta_examples,
        aes(x="date", y="estimate", color="company")
    )
    + geom_line(size=0.8)
    + geom_hline(yintercept=1, linetype="dashed", color="gray", alpha=0.7)
    + labs(
        x="",
        y="Beta",
        color="",
        title="Rolling Beta Estimates (60-Month Window)"
    )
    + scale_x_datetime(date_breaks="2 years", date_labels="%Y")
    + theme_minimal()
    + theme(legend_position="bottom")
```

```
)
rolling_beta_figure.show()
```

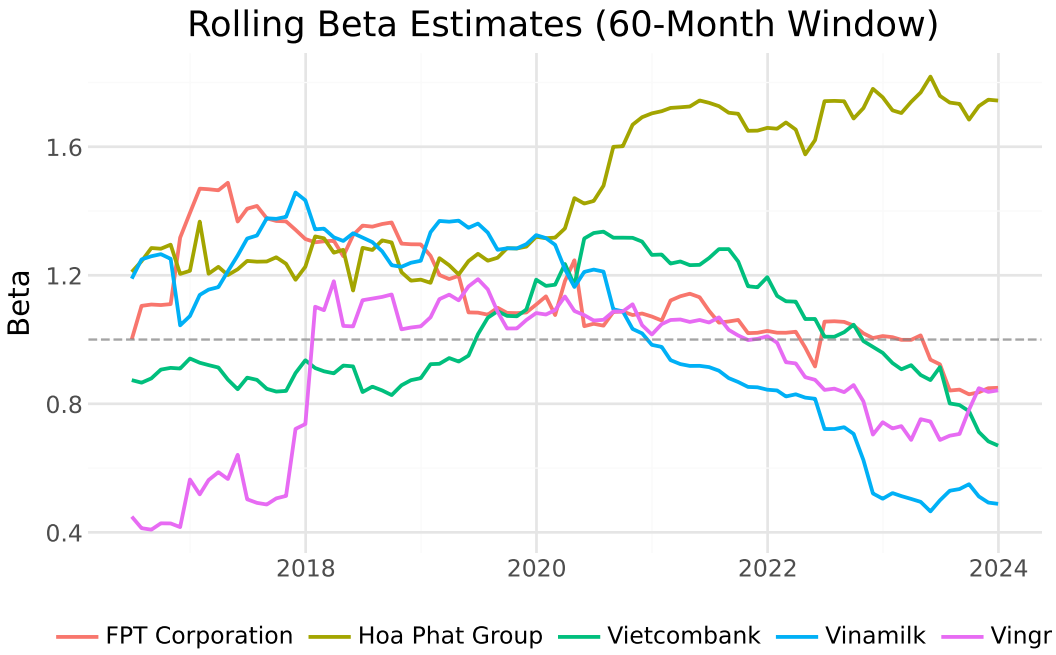


Figure 15.1: Monthly rolling beta estimates for selected Vietnamese stocks using a 60-month estimation window. Different industries exhibit distinct patterns of market sensitivity over time.

Several patterns emerge from the figure:

1. **Industry differences**: Technology and banking stocks may exhibit different beta patterns than real estate or consumer goods companies.
2. **Time variation**: Betas are not constant. They respond to changes in business conditions, leverage, and market regimes.
3. **Crisis periods**: Market stress periods (e.g., 2008 financial crisis, 2020 COVID-19) often see beta estimates change as correlations across stocks increase.

15.6 Parallelized Estimation for the Full Market

15.6.1 The Computational Challenge

Estimating rolling betas for all stocks in our database is computationally intensive. With hundreds of stocks, each requiring rolling estimation across many time periods, sequential processing would take considerable time. Fortunately, beta estimation for different stocks is independent (i.e., the estimate for stock A does not depend on the estimate for stock B). This independence makes the problem ideal for parallelization.

15.6.2 Setting Up Parallel Processing

We use the `joblib` library to distribute computation across multiple CPU cores. The `Parallel` class manages worker processes, while `delayed` wraps function calls for deferred execution.

```
# Determine available cores (reserve one for system operations)
n_cores = max(1, cpu_count() - 1)
print(f"Available cores for parallel processing: {n_cores}")
```

```
Available cores for parallel processing: 3
```

15.6.3 Parallel Beta Estimation

The following code estimates rolling betas for all stocks in parallel. Each stock is processed independently by a separate worker.

```
def estimate_all_betas_parallel(data, n_cores, look_back=60, min_obs=48):
    """
    Estimate rolling betas for all stocks using parallel processing.

    Parameters
    ----------
    data : pd.DataFrame
        Full dataset with all stocks
    n_cores : int
        Number of CPU cores to use
    look_back : int
        Months in estimation window
    min_obs : int
        Minimum observations required
```

```
    Returns
    -------
    pd.DataFrame
        Beta estimates for all stocks and dates
    """
    # Group data by stock
    grouped = data.groupby("symbol", group_keys=False)

    # Define worker function
    def process_stock(name, group):
        result = roll_capm_estimation(group, look_back=look_back,
↪   min_obs=min_obs)
        if not result.empty:
            result["symbol"] = name
        return result

    # Execute in parallel
    results = Parallel(n_jobs=n_cores, verbose=1)(
        delayed(process_stock)(name, group)
        for name, group in grouped
    )

    # Combine results
    results = [r for r in results if not r.empty]
    if results:
        return pd.concat(results, ignore_index=True)
    else:
        return pd.DataFrame()
```

```
# Estimate betas for all stocks
print("Estimating rolling betas for all stocks...")
capm_monthly = estimate_all_betas_parallel(
    prices_monthly,
    n_cores=n_cores,
    look_back=60,
    min_obs=48
)

print(f"\nCompleted: {len(capm_monthly):,} coefficient estimates")
print(f"Unique stocks: {capm_monthly['symbol'].nunique():,}")
```

15.6.4 Storing Results

We save the CAPM estimates to our database for use in subsequent chapters.

```
capm_monthly.to_sql(
    name="capm_monthly",
    con=tidy_finance,
    if_exists="replace",
    index=False
)

print("CAPM estimates saved to database.")
```

For subsequent analysis, we load the pre-computed estimates:

```
capm_monthly = pd.read_sql_query(
    sql="SELECT * FROM capm_monthly",
    con=tidy_finance,
    parse_dates={"date"}
)

print(f"Loaded {len(capm_monthly):,} CAPM estimates")
```

```
Loaded 161,580 CAPM estimates
```

15.7 Beta Estimation Using Daily Returns

While monthly returns are standard in academic research, some applications benefit from higher-frequency data:

- **Shorter estimation windows**: Daily data allows meaningful estimation over shorter periods (e.g., 3 months rather than 5 years).
- **More responsive estimates**: Daily betas capture changes more quickly.
- **Event studies**: High-frequency betas are useful for analyzing market reactions to specific events.

However, daily data introduces additional challenges:

- **Microstructure noise**: Bid-ask bounce and other trading frictions add noise to returns.
- **Non-synchronous trading**: Less liquid stocks may not trade every day, biasing beta estimates downward.
- **Computational burden**: Daily data is roughly 21 times larger than monthly data.

15.7.1 Batch Processing for Daily Data

Given the size of daily data, we process stocks in batches to manage memory constraints. This approach loads and processes a subset of stocks, saves results, and proceeds to the next batch.

```
def compute_market_return_daily(tidy_finance):
    """
    Compute daily value-weighted market excess return from stock data.
    """
    # Load daily prices with market cap for weighting
    prices_daily_full = pd.read_sql_query(
        sql="""
            SELECT p.symbol, p.date, p.ret_excess, m.mktcap_lag
            FROM prices_daily p
            LEFT JOIN prices_monthly m ON p.symbol = m.symbol
                AND strftime('%Y-%m', p.date) = strftime('%Y-%m', m.date)
        """,
        con=tidy_finance,
        parse_dates={"date"}
    )

    # Compute value-weighted market return each day
    mkt_daily = (prices_daily_full
        .dropna(subset=["ret_excess", "mktcap_lag"])
        .groupby("date")
        .apply(lambda x: np.average(x["ret_excess"],
↪   weights=x["mktcap_lag"]))
        .reset_index(name="mkt_excess")
    )

    return mkt_daily


def roll_capm_estimation_daily(data, look_back_days=1260, min_obs=1000):
    """
    Perform rolling-window CAPM estimation using daily data.

    Parameters
    ----------
    data : pd.DataFrame
        DataFrame with 'date', 'ret_excess', and 'mkt_excess' columns
    look_back_days : int
```

```
        Number of trading days in the estimation window
    min_obs : int
        Minimum daily observations required within each window

    Returns
    -------
    pd.DataFrame
        Time series of coefficient estimates with dates
    """
    data = data.sort_values("date").copy()
    dates =
↪   data["date"].drop_duplicates().sort_values().reset_index(drop=True)

    results = []

    for i in range(look_back_days - 1, len(dates)):
        end_date = dates.iloc[i]
        start_idx = max(0, i - look_back_days + 1)
        start_date = dates.iloc[start_idx]

        window_data = data.query("date >= @start_date and date <= @end_date")
        window_results = estimate_capm(window_data, min_obs=min_obs)

        if not window_results.empty:
            window_results["date"] = end_date
            results.append(window_results)

    if results:
        return pd.concat(results, ignore_index=True)
    else:
        return pd.DataFrame()


def estimate_daily_betas_batch(symbols, tidy_finance, n_cores,
↪   batch_size=500,
                               look_back_days=1260, min_obs=1000):
    """
    Estimate rolling betas from daily data using batch processing.
    """
    # First, compute or load market return
    print("Computing daily market excess returns...")
    mkt_daily = compute_market_return_daily(tidy_finance)
    print(f"Market returns: {len(mkt_daily)} days")
```

```
    n_batches = int(np.ceil(len(symbols) / batch_size))
    all_results = []

    for j in range(n_batches):
        batch_start = j * batch_size
        batch_end = min((j + 1) * batch_size, len(symbols))
        batch_symbols = symbols[batch_start:batch_end]

        symbol_list = ", ".join(f"'{s}'" for s in batch_symbols)

        query = f"""
            SELECT symbol, date, ret_excess
            FROM prices_daily
            WHERE symbol IN ({symbol_list})
        """

        prices_daily_batch = pd.read_sql_query(
            sql=query,
            con=tidy_finance,
            parse_dates={"date"}
        )

        # Merge with market excess return
        prices_daily_batch = prices_daily_batch.merge(
            mkt_daily,
            on="date",
            how="inner"
        )

        # Group by symbol and estimate betas
        grouped = prices_daily_batch.groupby("symbol", group_keys=False)

        # Parallel estimation
        batch_results = Parallel(n_jobs=n_cores)(
            delayed(lambda name, group:
                roll_capm_estimation_daily(group,
↪   look_back_days=look_back_days, min_obs=min_obs)
                .assign(symbol=name)
            )(name, group)
            for name, group in grouped
        )
```

```
        batch_results = [r for r in batch_results if r is not None and not
↪   r.empty]

        if batch_results:
            all_results.append(pd.concat(batch_results, ignore_index=True))

        print(f"Batch {j+1}/{n_batches} complete")

    if all_results:
        return pd.concat(all_results, ignore_index=True)
    else:
        return pd.DataFrame()
```

```
symbols = prices_monthly["symbol"].unique().tolist()

capm_daily = estimate_daily_betas_batch(
    symbols=symbols,
    tidy_finance=tidy_finance,
    n_cores=n_cores,
    batch_size=500,
    look_back_days=1260,  # ~5 years of trading days
    min_obs=1000
)

print(f"Daily beta estimates: {len(capm_daily):,}")
```

```
capm_daily.to_sql(
    name="capm_daily",
    con=tidy_finance,
    if_exists="replace",
    index=False
)

print("CAPM estimates saved to database.")
```

For subsequent analysis, we load the pre-computed estimates:

```
capm_daily = pd.read_sql_query(
    sql="SELECT * FROM capm_daily",
    con=tidy_finance,
    parse_dates={"date"}
```

```
)

print(f"Loaded {len(capm_daily):,} CAPM estimates")
```

```
Loaded 3,394,490 CAPM estimates
```

15.8 Analyzing Beta Estimates

15.8.1 Extracting Beta Estimates

We extract the beta coefficient estimates from our CAPM results for analysis.

```
# Extract monthly betas
beta_monthly = (capm_monthly
    .query("coefficient == 'beta'")
    .rename(columns={"estimate": "beta"})
    [["symbol", "date", "beta"]]
    .assign(frequency="monthly")
)

# Save to database
beta_monthly.to_sql(
    name="beta_monthly",
    con=tidy_finance,
    if_exists="replace",
    index=False
)

print(f"Monthly betas: {len(beta_monthly):,} observations")
print(f"Unique stocks: {beta_monthly['symbol'].nunique():,}")
```

```
Monthly betas: 80,790 observations
Unique stocks: 1,383
```

```
# Load pre-computed betas
beta_monthly = pd.read_sql_query(
    sql="SELECT * FROM beta_monthly",
    con=tidy_finance,
    parse_dates={"date"}
)
```

15.8.2 Summary Statistics

We examine the distribution of beta estimates to verify their reasonableness.

```
print("Beta Summary Statistics:")
print(beta_monthly["beta"].describe().round(3))

# Additional diagnostics
print(f"\nStocks with negative average beta:
 ↪  {(beta_monthly.groupby('symbol')['beta'].mean() < 0).sum()}")
print(f"Stocks with beta > 2: {(beta_monthly.groupby('symbol')['beta'].mean()
 ↪  > 2).sum()}")
```

```
Beta Summary Statistics:
count    80790.000
mean         0.501
std          0.539
min         -1.345
25%          0.130
50%          0.447
75%          0.832
max          2.678
Name: beta, dtype: float64

Stocks with negative average beta: 177
Stocks with beta > 2: 5
```

15.8.3 Beta Distribution Across Industries

Different industries have different exposures to systematic market risk based on their business models, operating leverage, and financial leverage. Figure 15.2 shows the distribution of firm-level average betas across Vietnamese industries.

```
# Merge betas with industry information
beta_with_industry = (beta_monthly
    .merge(
        prices_monthly[["symbol", "date", "icb_name_vi"]].drop_duplicates(),
        on=["symbol", "date"],
        how="left"
    )
    .dropna(subset=["icb_name_vi"])
```

```
)

# Compute firm-level average beta by industry
beta_by_industry = (beta_with_industry
    .groupby(["icb_name_vi", "symbol"])["beta"]
    .mean()
    .reset_index()
)

# Order industries by median beta
industry_order = (beta_by_industry
    .groupby("icb_name_vi")["beta"]
    .median()
    .sort_values()
    .index.tolist()
)

# Select top 10 industries by number of firms for clearer visualization
top_industries = (beta_by_industry
    .groupby("icb_name_vi")
    .size()
    .nlargest(10)
    .index.tolist()
)

beta_by_industry_filtered = beta_by_industry.query("icb_name_vi in
 ↪  @top_industries")
```

```
beta_industry_figure = (
    ggplot(
        beta_by_industry_filtered,
        aes(x="icb_name_vi", y="beta")
    )
    + geom_boxplot(fill="steelblue", alpha=0.7)
    + geom_hline(yintercept=1, linetype="dashed", color="red", alpha=0.7)
    + coord_flip()
    + labs(
        x="",
        y="Beta",
        title="Beta Distribution by Industry"
    )
    + theme_minimal()
```

```
)
beta_industry_figure.show()
```

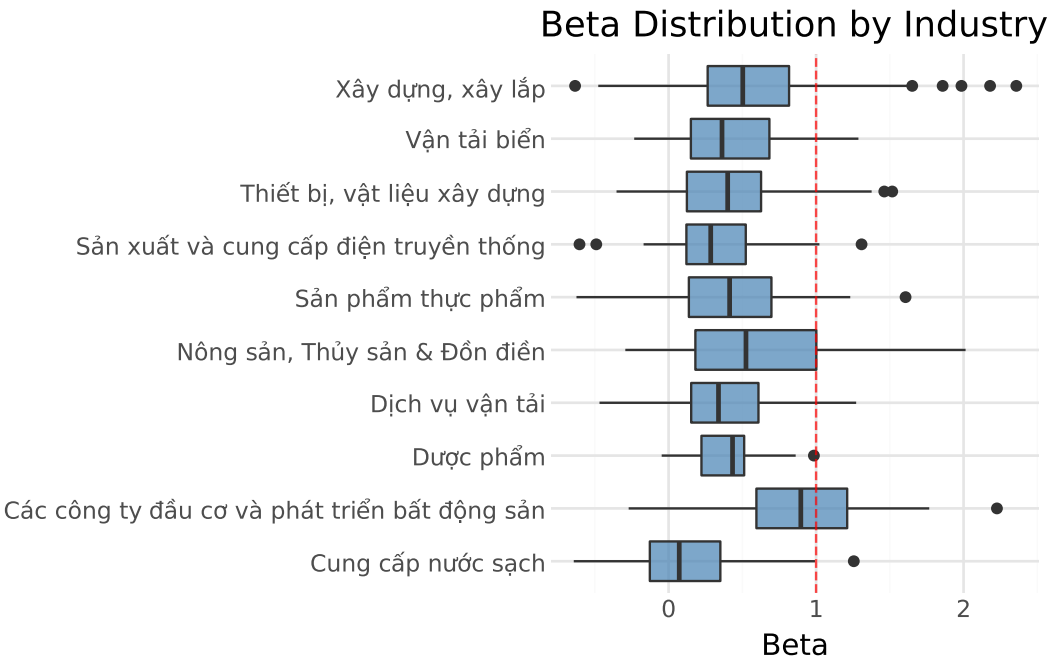


Figure 15.2: Distribution of firm-level average betas across Vietnamese industries. Box plots show the median, interquartile range, and outliers for each industry.

15.8.4 Time Variation in Cross-Sectional Beta Distribution

Betas vary not only across stocks but also over time. Figure 15.3 shows how the cross-sectional distribution of betas has evolved in the Vietnamese market.

```
# Compute monthly quantiles
beta_quantiles = (beta_monthly
    .groupby("date")["beta"]
    .quantile(q=np.arange(0.1, 1.0, 0.1))
    .reset_index()
    .rename(columns={"level_1": "quantile"})
    .assign(quantile=lambda x: (x["quantile"] * 100).astype(int).astype(str)
 ↪  + "%")
)
```

```
beta_quantiles_figure = (
    ggplot(
        beta_quantiles,
        aes(x="date", y="beta", color="quantile")
    )
    + geom_line(alpha=0.8)
    + geom_hline(yintercept=1, linetype="dashed", color="gray")
    + labs(
        x="",
        y="Beta",
        color="Quantile",
        title="Cross-Sectional Distribution of Betas Over Time"
    )
    + scale_x_datetime(date_breaks="2 years", date_labels="%Y")
    + theme_minimal()
)
beta_quantiles_figure.show()
```

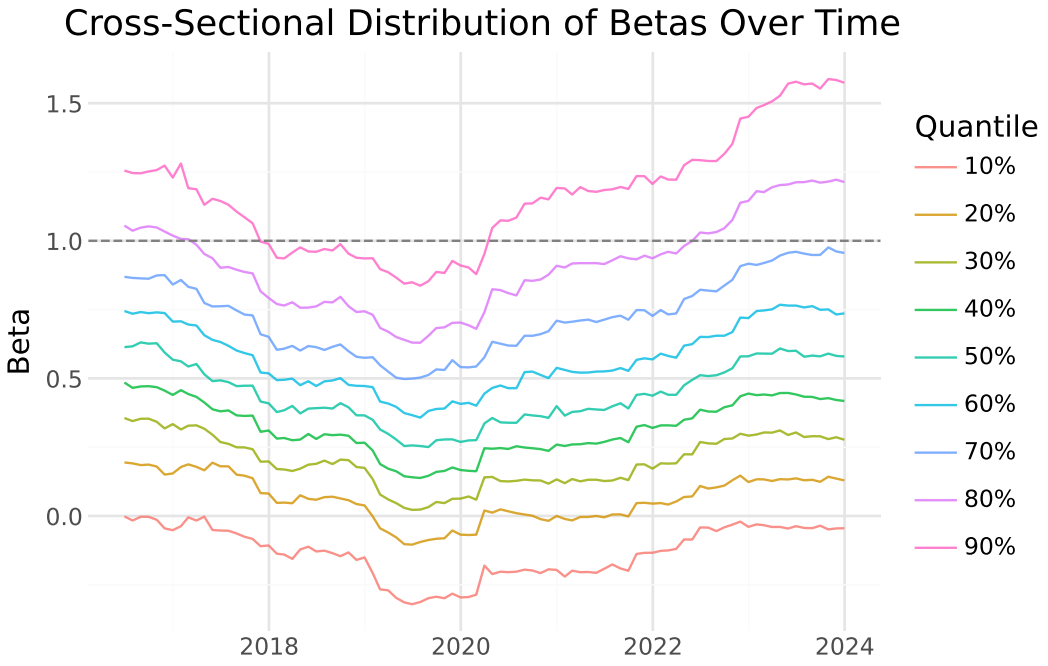


Figure 15.3: Monthly quantiles of beta estimates over time. Each line represents a decile of the cross-sectional beta distribution.

The figure reveals several interesting patterns:

1. **Level shifts**: The entire distribution of betas can shift over time, reflecting changes in market-wide correlation.
2. **Dispersion changes**: During market stress, the spread between high and low beta stocks may change as correlations move.
3. **Trends**: Some periods show trending behavior in betas, possibly reflecting structural changes in the economy.

15.8.5 Coverage Analysis

We verify that our estimation procedure produces reasonable coverage across the sample. Figure 15.4 shows the fraction of stocks with available beta estimates over time.

```
# Count stocks with and without betas
coverage = (prices_monthly
    .groupby("date")["symbol"]
    .nunique()
    .reset_index(name="total_stocks")
    .merge(
        beta_monthly.groupby("date")["symbol"].nunique().reset_index(name="w
 ↪  ith_beta"),
        on="date",
        how="left"
    )
    .fillna(0)
    .assign(coverage=lambda x: x["with_beta"] / x["total_stocks"])
)

coverage_figure = (
    ggplot(coverage, aes(x="date", y="coverage"))
    + geom_line(color="steelblue", size=1)
    + labs(
        x="",
        y="Share with Beta Estimate",
        title="Beta Estimation Coverage Over Time"
    )
    + scale_y_continuous(labels=percent_format(), limits=(0, 1))
    + scale_x_datetime(date_breaks="2 years", date_labels="%Y")
    + theme_minimal()
)
coverage_figure.show()
```

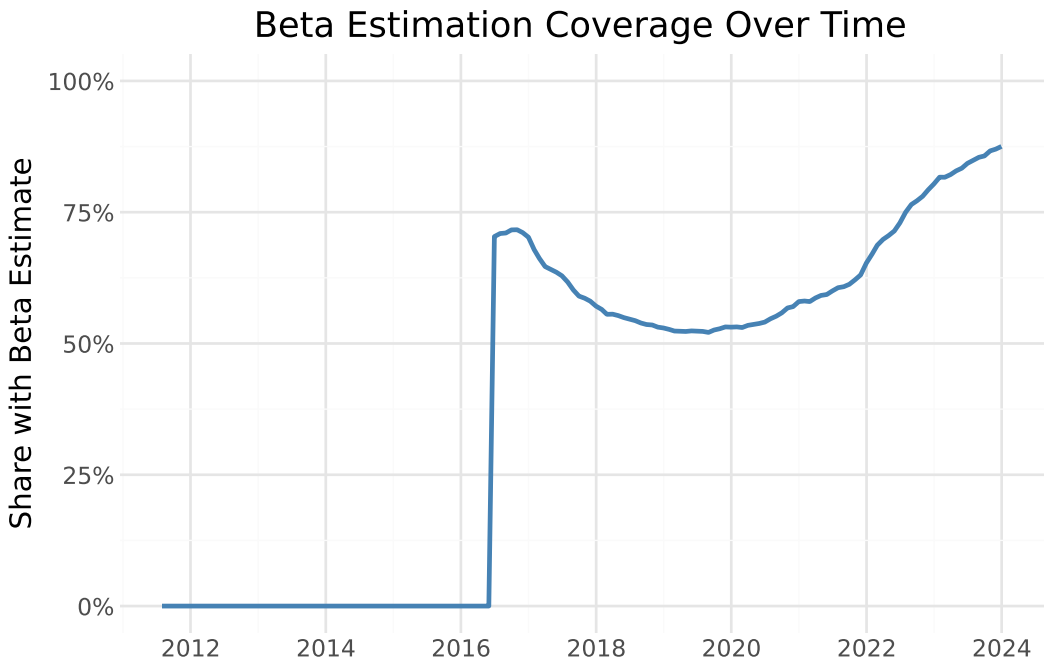


Figure 15.4: Share of stocks with available beta estimates over time. Coverage increases as more stocks accumulate sufficient return history.

Coverage is lower in early years because stocks need sufficient return history (at least 48 months) before their betas can be estimated. As the market matures and stocks accumulate longer histories, coverage approaches 100%.

15.9 Comparing Monthly and Daily Beta Estimates

When both monthly and daily beta estimates are available, we can compare them to understand how estimation frequency affects results.

```
# Combine monthly and daily estimates
beta_daily = (capm_daily
    .query("coefficient == 'beta'")
    .rename(columns={"estimate": "beta"})
    [["symbol", "date", "beta"]]
    .assign(frequency="daily")
)

beta_combined = pd.concat([beta_monthly, beta_daily], ignore_index=True)
```

```
# Filter to example stocks
beta_comparison = (beta_combined
    .merge(examples, on="symbol")
    .query("symbol in ['VIC', 'FPT']")  # Select two for clarity
)

comparison_figure = (
    ggplot(
        beta_comparison,
        aes(x="date", y="beta", color="frequency", linetype="frequency")
    )
    + geom_line(size=0.8)
    + facet_wrap("~company", ncol=1)
    + labs(
        x="",
        y="Beta",
        color="Data Frequency",
        linetype="Data Frequency",
        title="Monthly vs Daily Beta Estimates"
    )
    + scale_x_datetime(date_breaks="2 years", date_labels="%Y")
    + theme_minimal()
    + theme(legend_position="bottom")
)
comparison_figure.show()
```

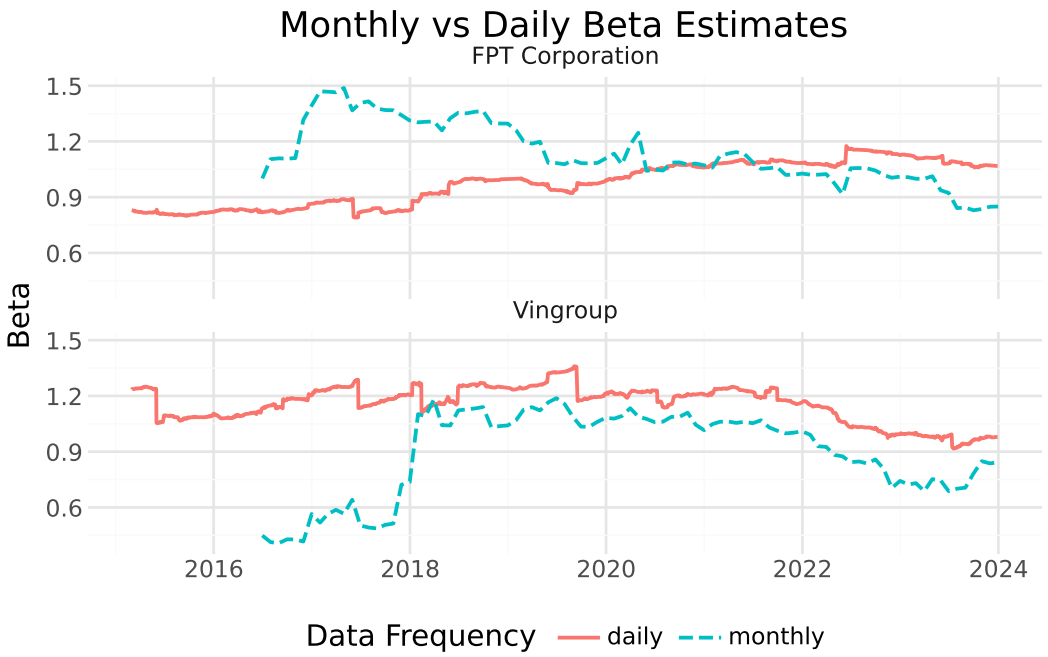


Figure 15.5: Comparison of beta estimates using monthly versus daily returns for selected stocks. Daily estimates are smoother due to more observations per estimation window.

The comparison reveals that daily-based estimates are generally smoother due to the larger number of observations in each window. However, the level and trend of estimates are similar across frequencies, providing validation that both approaches capture the same underlying systematic risk exposure.

```
# Correlation between monthly and daily estimates
correlation_data = (beta_combined
    .pivot_table(index=["symbol", "date"], columns="frequency",
 ↪  values="beta")
    .dropna()
)

print(f"Correlation between monthly and daily betas:
 ↪  {correlation_data.corr().iloc[0,1]:.3f}")
```

```
Correlation between monthly and daily betas: 0.745
```

Table 15.1: Theoretical Reasons for Imperfect Correlation

Factor	Effect
Non-synchronous trading	Daily betas can be biased downward for illiquid stocks
Microstructure noise	Bid-ask bounce adds noise to daily estimates
Different effective windows	Same calendar period but ~20x more observations for daily
Mean reversion speed	Daily captures faster-moving risk dynamics

Table 15.1 shows several reasons why we might observe imperfect correlation.

15.10 Key Takeaways

1. **CAPM beta** measures a stock's sensitivity to systematic market risk and is fundamental to modern portfolio theory, cost of capital estimation, and risk management.
2. **Rolling-window estimation** captures time variation in betas, which reflects changes in companies' business models, leverage, and market conditions.
3. **Parallelization** dramatically reduces computation time for large-scale estimation tasks by distributing work across multiple CPU cores.
4. **Estimation choices matter**: Window length, return frequency, and minimum observation requirements all affect beta estimates. Researchers should choose parameters appropriate for their specific application.
5. **Industry patterns**: Vietnamese stocks show systematic differences in market sensitivity across industries, with cyclical sectors exhibiting higher betas than defensive sectors.
6. **Time variation**: The cross-sectional distribution of betas in Vietnam has evolved over time, with notable shifts during market stress periods.
7. **Frequency comparison**: Monthly and daily beta estimates are positively correlated but not identical. Daily estimates are smoother while monthly estimates may better capture lower-frequency variation.
8. **Data quality checks**: Coverage analysis and summary statistics help identify potential issues in estimation procedures before using results in downstream analyses.

Part III

Asset Pricing Models and Cross-Sectional Tests

16 The Capital Asset Pricing Model

16.1 From Efficient Portfolios to Equilibrium Prices

The previous chapter on Modern Portfolio Theory (MPT) showed how an investor can construct portfolios that optimally trade off risk and expected return. But MPT leaves a crucial question unanswered: What determines the *expected returns* themselves? Why do some assets command higher risk premiums than others?

The Capital Asset Pricing Model (CAPM) answers this question. Developed simultaneously by Sharpe (1964), Lintner (1965), and Mossin (1966), the CAPM extends MPT to explain how assets should be priced in equilibrium when *all* investors follow mean-variance optimization principles. The CAPM's central insight is both elegant and counterintuitive: **not all risk is rewarded**. Only the component of risk that cannot be diversified away (i.e., systematic risk) commands a risk premium in equilibrium.

The CAPM remains the cornerstone of asset pricing theory, not because it perfectly describes reality, but because it provides the simplest coherent framework for understanding the relationship between risk and expected return. Every extension and alternative model in asset pricing (e.g., from the Fama-French factors to consumption-based pricing) builds upon or reacts against the CAPM's foundational logic.

In this chapter, we derive the CAPM from first principles, illustrate its theoretical underpinnings, and show how to estimate its parameters empirically. We download stock market data, estimate betas using regression analysis, and evaluate asset performance relative to model predictions.

```
import pandas as pd
import numpy as np
import tidyfinance as tf

from plotnine import *
from mizani.formatters import percent_format
from adjustText import adjust_text
```

16.2 Systematic versus Idiosyncratic Risk

Before diving into the mathematics, we need to understand the fundamental distinction that makes the CAPM work: the difference between *systematic* and *idiosyncratic* risk.

16.2.1 Idiosyncratic Risk: Diversifiable and Unrewarded

Consider events that affect individual companies but not the broader market: a CEO resigns unexpectedly, a product launch fails, earnings disappoint analysts, or a factory experiences a fire. These company-specific events can dramatically affect individual stock prices, but they tend to average out across a diversified portfolio. When one company has bad news, another often has good news; the shocks are largely uncorrelated.

This idiosyncratic (or firm-specific) risk can be eliminated through diversification. By holding a portfolio of many stocks, an investor can reduce idiosyncratic risk to nearly zero. Since this risk can be avoided at no cost, investors should not expect compensation for bearing it. In equilibrium, idiosyncratic risk earns no premium.

16.2.2 Systematic Risk: Undiversifiable and Priced

Systematic risk, by contrast, affects all assets simultaneously. Recessions, interest rate changes, geopolitical crises, and pandemics impact virtually every company to some degree. No amount of diversification can eliminate exposure to these economy-wide shocks, they are inherent to participating in the market.

Since systematic risk cannot be diversified away, investors genuinely dislike it. They must be compensated for bearing it. The CAPM formalizes this intuition: expected returns should depend only on systematic risk, not total risk. Two assets with identical total volatility can have very different expected returns if their systematic risk exposures differ.

16.2.3 A Simple Illustration

Imagine two stocks with identical 30% annual volatility. Stock A is a gold mining company whose returns move opposite to the overall market: it does well when the economy struggles and poorly when it booms. Stock B is a luxury retailer that amplifies market movements: soaring in good times and crashing in bad times.

Which stock should offer higher expected returns? Intuition might suggest they should be equal since both have the same volatility. But the CAPM says Stock B should offer substantially higher returns. Why? Because Stock B performs poorly precisely when investors' overall wealth is already down (during market crashes), making its returns particularly painful. Stock A, by contrast, provides insurance. Its strong performance during market downturns partially

offsets losses elsewhere in the portfolio. Investors value this insurance property and are willing to accept lower expected returns in exchange.

This is the CAPM's core insight: expected returns compensate investors for systematic risk exposure, measured by how an asset's returns co-move with the market portfolio.

16.3 Data Preparation

Building on our analysis from the previous chapter, we examine the VN30 constituents as our asset universe. We download and prepare monthly return data:

```
vn30_symbols = [
    "ACB", "BCM", "BID", "BVH", "CTG", "FPT", "GAS", "GVR", "HDB", "HPG",
    "MBB", "MSN", "MWG", "PLX", "POW", "SAB", "SHB", "SSB", "STB", "TCB",
    "TPB", "VCB", "VHM", "VIB", "VIC", "VJC", "VNM", "VPB", "VRE", "EIB"
]
```

```
import os
import boto3
from botocore.client import Config
from io import BytesIO

class ConnectMinio:
    def __init__(self):
        self.MINIO_ENDPOINT = os.environ["MINIO_ENDPOINT"]
        self.MINIO_ACCESS_KEY = os.environ["MINIO_ACCESS_KEY"]
        self.MINIO_SECRET_KEY = os.environ["MINIO_SECRET_KEY"]
        self.REGION = os.getenv("MINIO_REGION", "us-east-1")

        self.s3 = boto3.client(
            "s3",
            endpoint_url=self.MINIO_ENDPOINT,
            aws_access_key_id=self.MINIO_ACCESS_KEY,
            aws_secret_access_key=self.MINIO_SECRET_KEY,
            region_name=self.REGION,
            config=Config(signature_version="s3v4"),
        )

    def test_connection(self):
        resp = self.s3.list_buckets()
        print("Connected. Buckets:")
```

```
        for b in resp.get("Buckets", []):
            print(" -", b["Name"])


conn = ConnectMinio()
s3 = conn.s3
conn.test_connection()

bucket_name = os.environ["MINIO_BUCKET"]

prices = pd.read_csv(
    BytesIO(
        s3.get_object(
            Bucket=bucket_name,
            Key="historycal_price/dataset_historical_price.csv"
        )["Body"].read()
    ),
    low_memory=False
)
```

```
Connected. Buckets:
 - dsteam-data
 - rawbctc
```

We process the raw price data to compute adjusted closing prices and standardize column names:

```
prices["date"] = pd.to_datetime(prices["date"])
prices["adjusted_close"] = prices["close_price"] * prices["adj_ratio"]
prices = prices.rename(columns={
    "vol_total": "volume",
    "open_price": "open",
    "low_price": "low",
    "high_price": "high",
    "close_price": "close"
})
prices = prices.sort_values(["symbol", "date"])
```

```
prices_daily = prices[prices["symbol"].isin(vn30_symbols)]
prices_daily[["date", "symbol", "adjusted_close"]].head(3)
```

	date	symbol	adjusted_close
18176	2010-01-04	ACB	329.408244
18177	2010-01-05	ACB	329.408244
18178	2010-01-06	ACB	320.258015

16.3.1 Computing Monthly Returns

We aggregate daily prices to monthly frequency. Using monthly returns rather than daily returns offers several advantages for portfolio analysis: monthly returns exhibit less noise, better approximate normality, and reduce the impact of microstructure effects like bid-ask bounce.

```
returns_monthly = (prices_daily
    .assign(
        date=prices_daily["date"].dt.to_period("M").dt.to_timestamp("M")
    )
    .groupby(["symbol", "date"], as_index=False)
    .agg(adjusted_close=("adjusted_close", "last"))
    .sort_values(["symbol", "date"])
    .assign(
        ret=lambda x: x.groupby("symbol")["adjusted_close"].pct_change()
    )
)

returns_monthly.head(3)
```

	symbol	date	adjusted_close	ret
0	ACB	2010-01-31	291.975489	NaN
1	ACB	2010-02-28	303.621235	0.039886
2	ACB	2010-03-31	273.784658	-0.098269

16.4 The Risk-Free Asset and the Investment Opportunity Set

16.4.1 Adding a Risk-Free Asset

The previous chapter on MPT considered portfolios composed entirely of risky assets, requiring that portfolio weights sum to one. The CAPM introduces a crucial new element: a **risk-free asset** that pays a constant interest rate r_f with zero volatility.

This seemingly simple addition fundamentally transforms the investment opportunity set. With a risk-free asset available, investors can choose to park some wealth in the safe asset and invest the remainder in risky assets. They can also borrow at the risk-free rate to leverage their risky positions.

Let $\omega \in \mathbb{R}^N$ denote the portfolio weights in the N risky assets. Unlike before, these weights need not sum to one. The remainder, $1 - \iota'\omega$ (where ι is a vector of ones), is invested in the risk-free asset.

16.4.2 Portfolio Return with a Risk-Free Asset

The expected return on this combined portfolio is:

$$\mu_\omega = \omega'\mu + (1 - \iota'\omega)r_f = r_f + \omega'(\mu - r_f) = r_f + \omega'\tilde{\mu}$$

where μ is the vector of expected returns on risky assets and $\tilde{\mu} = \mu - r_f$ denotes the vector of **excess returns** (returns above the risk-free rate).

This expression reveals an important decomposition: the portfolio's expected return equals the risk-free rate plus a risk premium determined by the exposure to risky assets.

16.4.3 Portfolio Variance

Since the risk-free asset has zero volatility and zero covariance with risky assets, only the risky portion contributes to portfolio variance:

$$\sigma_\omega^2 = \omega'\Sigma\omega$$

where Σ is the variance-covariance matrix of risky asset returns. The portfolio's volatility (standard deviation) is:

$$\sigma_\omega = \sqrt{\omega'\Sigma\omega}$$

16.4.4 Setting Up the Risk-Free Rate

For a realistic proxy of the risk-free rate, we use the Vietnam government bond yield. Government bonds of stable economies are considered "risk-free" because the government can always print money to meet its obligations (though this may cause inflation).

```
all_dates = pd.date_range(
    start=returns_monthly["date"].min(),
    end=returns_monthly["date"].max(),
    freq="ME"
)

# Vietnam 10-Year Government Bond Yield (approximately 2.52% annualized)
rf_annual = 0.0252
rf_monthly_val = (1 + rf_annual)**(1/12) - 1

risk_free_monthly = pd.DataFrame({
    "date": all_dates,
    "risk_free": rf_monthly_val
})

risk_free_monthly["date"] = (
    pd.to_datetime(risk_free_monthly["date"])
    .dt.to_period("M")
    .dt.to_timestamp("M")
)

risk_free_monthly.head(3)
```

	date	risk_free
0	2010-01-31	0.002076
1	2010-02-28	0.002076
2	2010-03-31	0.002076

We merge the risk-free rate with our returns data and compute excess returns:

```
returns_monthly = returns_monthly.merge(
    risk_free_monthly[["date", "risk_free"]],
    on="date",
    how="left"
)

rf = risk_free_monthly["risk_free"].mean()

returns_monthly = (returns_monthly
    .assign(
```

```
        ret_excess=lambda x: x["ret"] - x["risk_free"]
    )
    .assign(
        ret_excess=lambda x: x["ret_excess"].clip(lower=-1)
    )
)

returns_monthly.head(3)
```

	symbol	date	adjusted_close	ret	risk_free	ret_excess
0	ACB	2010-01-31	291.975489	NaN	0.002076	NaN
1	ACB	2010-02-28	303.621235	0.039886	0.002076	0.037810
2	ACB	2010-03-31	273.784658	-0.098269	0.002076	-0.100345

16.5 The Tangency Portfolio: Where Everyone Invests

16.5.1 Deriving the Optimal Risky Portfolio

With a risk-free asset available, how should an investor allocate wealth across risky assets? Consider an investor who wants to achieve a target expected excess return $\bar{\mu}$ with minimum variance. The optimization problem becomes:

$$\min_{\omega} \omega' \Sigma \omega \quad \text{subject to} \quad \omega' \tilde{\mu} = \bar{\mu}$$

Using the Lagrangian method:

$$\mathcal{L}(\omega, \lambda) = \omega' \Sigma \omega - \lambda(\omega' \tilde{\mu} - \bar{\mu})$$

The first-order condition with respect to ω is:

$$\frac{\partial \mathcal{L}}{\partial \omega} = 2\Sigma\omega - \lambda\tilde{\mu} = 0$$

Solving for the optimal weights:

$$\omega^* = \frac{\lambda}{2}\Sigma^{-1}\tilde{\mu}$$

The constraint $\omega'\tilde{\mu} = \bar{\mu}$ determines λ:

$$\bar{\mu} = \tilde{\mu}'\omega^* = \frac{\lambda}{2}\tilde{\mu}'\Sigma^{-1}\tilde{\mu} \implies \lambda = \frac{2\bar{\mu}}{\tilde{\mu}'\Sigma^{-1}\tilde{\mu}}$$

Substituting back:

$$\omega^* = \frac{\bar{\mu}}{\tilde{\mu}'\Sigma^{-1}\tilde{\mu}}\Sigma^{-1}\tilde{\mu}$$

16.5.2 The Tangency Portfolio

Notice something remarkable: the *direction* of ω^* is always $\Sigma^{-1}\tilde{\mu}$, regardless of the target return $\bar{\mu}$. Only the *scale* changes. This means all investors, regardless of their risk preferences, hold the *same portfolio of risky assets*. They differ only in how much they allocate to this portfolio versus the risk-free asset.

To obtain the portfolio of risky assets that is fully invested (weights summing to one), we normalize:

$$\omega_{\tan} = \frac{\omega^*}{\iota'\omega^*} = \frac{\Sigma^{-1}(\mu - r_f)}{\iota'\Sigma^{-1}(\mu - r_f)}$$

This is called the **tangency portfolio** (or maximum Sharpe ratio portfolio) because it lies at the point where the efficient frontier is tangent to the capital market line.

16.5.3 The Sharpe Ratio and the Capital Market Line

The **Sharpe ratio** measures excess return per unit of volatility:

$$\text{Sharpe Ratio} = \frac{\mu_p - r_f}{\sigma_p}$$

The tangency portfolio maximizes the Sharpe ratio among all possible portfolios. Any combination of the risk-free asset and the tangency portfolio lies on a straight line in mean-standard deviation space, called the **Capital Market Line (CML)**:

$$\mu_p = r_f + \left(\frac{\mu_{\tan} - r_f}{\sigma_{\tan}}\right)\sigma_p$$

The slope of this line equals the Sharpe ratio of the tangency portfolio (i.e., the highest achievable Sharpe ratio).

16.5.4 Computing the Tangency Portfolio

Let's compute the tangency portfolio for our VN30 universe:

```
assets = (returns_monthly
    .groupby("symbol", as_index=False)
    .agg(
        mu=("ret", "mean"),
        sigma=("ret", "std")
    )
)

sigma = (returns_monthly
    .pivot(index="date", columns="symbol", values="ret")
    .cov()
)

mu = (returns_monthly
    .groupby("symbol")["ret"]
    .mean()
    .values
)
```

```
# Compute tangency portfolio weights
w_tan = np.linalg.solve(sigma, mu - rf)
w_tan = w_tan / np.sum(w_tan)

# Portfolio performance metrics
mu_w = w_tan.T @ mu
sigma_w = np.sqrt(w_tan.T @ sigma @ w_tan)

efficient_portfolios = pd.DataFrame([
    {"symbol": r"$\omega_{\mathrm{tan}}$", "mu": mu_w, "sigma": sigma_w},
    {"symbol": r"$r_f$", "mu": rf, "sigma": 0}
])

sharpe_ratio = (mu_w - rf) / sigma_w

print(f"Tangency Portfolio Sharpe Ratio: {sharpe_ratio:.4f}")
print(efficient_portfolios)
```

```
Tangency Portfolio Sharpe Ratio: -0.5552
```

```
                    symbol        mu     sigma
0  $\omega_{\mathrm{tan}}$ -0.041157  0.077866
1                      $r_f$  0.002076  0.000000
```

16.5.5 Visualizing the Efficient Frontier with a Risk-Free Asset

Figure 16.1 shows the efficient frontier when a risk-free asset is available. The frontier is now a straight line (the Capital Market Line) connecting the risk-free asset to the tangency portfolio and extending beyond.

```
efficient_portfolios_figure = (
    ggplot(efficient_portfolios, aes(x="sigma", y="mu"))
    + geom_point(data=assets)
    + geom_point(data=efficient_portfolios, color="blue", size=3)
    + geom_label(
        aes(label="symbol"),
        adjust_text={"arrowprops": {"arrowstyle": "-"}}
    )
    + scale_x_continuous(labels=percent_format())
    + scale_y_continuous(labels=percent_format())
    + labs(
        x="Volatility (Standard Deviation)",
        y="Expected Return",
        title="Efficient Frontier with Risk-Free Asset (VN30)"
    )
    + geom_abline(slope=sharpe_ratio, intercept=rf, linetype="dotted")
)

efficient_portfolios_figure.show()
```

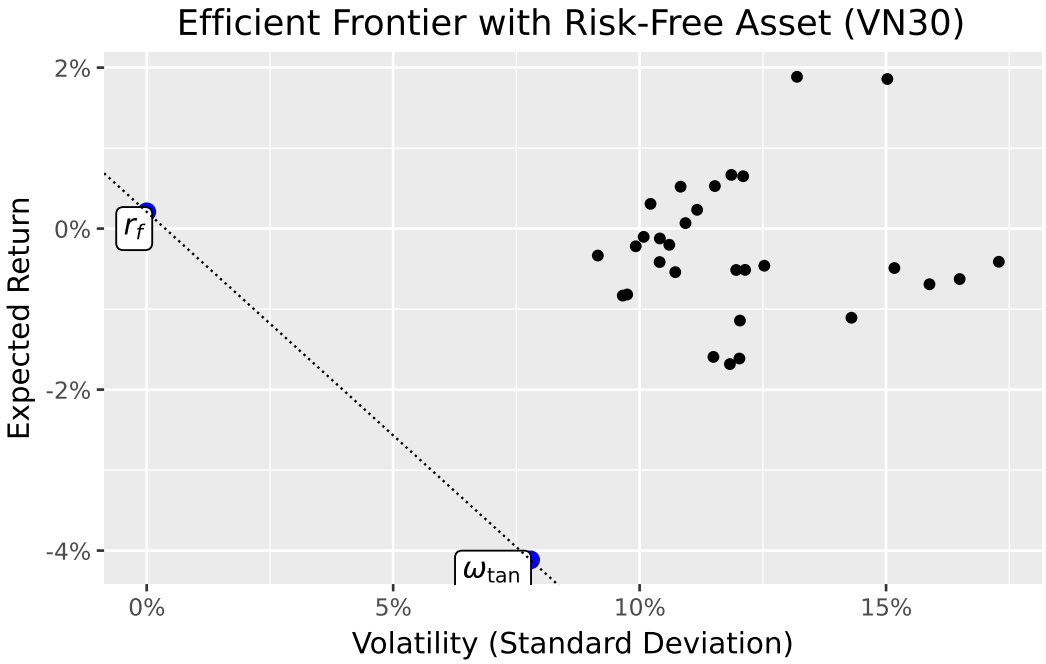


Figure 16.1: The efficient frontier with a risk-free asset becomes a straight line (the Capital Market Line) connecting the risk-free rate to the tangency portfolio. Individual assets lie below this line, demonstrating the benefits of diversification.

You may notice that estimated expected returns appear quite low, some even negative. This is not a model failure but reflects the realities of estimation:

1. **Sample period matters**: If the estimation window includes market downturns (such as the 2022-2023 period), realized average returns can be near zero or negative. Mean-variance optimization takes sample means literally.
2. **Estimation noise in emerging markets**: With volatile emerging market data, sample means are dominated by noise. A few extremely bad months can push the average below the risk-free rate even if the long-run equity premium is positive.

This highlights a fundamental challenge in portfolio optimization: the inputs we observe (historical returns) are noisy estimates of the true parameters we need (expected future returns).

16.6 The CAPM Equation: Risk and Expected Return

16.6.1 From Individual Optimization to Market Equilibrium

So far, we've focused on one investor's optimization problem. The CAPM's power comes from considering what happens when *all* investors optimize simultaneously.

If all investors follow mean-variance optimization, they all hold some combination of the risk-free asset and the tangency portfolio. The only difference between investors is their risk tolerance. More risk-averse investors hold more of the risk-free asset, while risk-tolerant investors may even borrow at the risk-free rate to leverage their position in the tangency portfolio.

16.6.2 The Market Portfolio

In equilibrium, the total demand for each risky asset must equal its supply. Since all investors hold the same portfolio of risky assets (the tangency portfolio), the equilibrium portfolio weights must equal the *market capitalization weights*. The tangency portfolio *is* the market portfolio.

This insight has enormous practical implications: instead of estimating expected returns and covariances to compute the tangency portfolio, we can simply use the market portfolio (approximated by a broad market index) as a proxy.

16.6.3 Deriving the CAPM Equation

From the first-order conditions of the optimization problem, we derived that:

$$\tilde{\mu} = \frac{2}{\lambda}\Sigma\omega^*$$

Since ω^* is proportional to $\omega_{\tan}$, and in equilibrium $\omega_{\tan}$ equals the market portfolio ω_m:

$$\tilde{\mu} = c \cdot \Sigma\omega_m$$

for some constant c. The i-th element of $\Sigma\omega_m$ is:

$$\sum_{j=1}^{N} \sigma_{ij}\omega_{m,j} = \text{Cov}(r_i, r_m)$$

where $r_m = \sum_j \omega_{m,j} r_j$ is the return on the market portfolio.

For the market portfolio itself:

$$\tilde{\mu}_m = c \cdot \text{Var}(r_m) = c \cdot \sigma_m^2$$

Therefore $c = \tilde{\mu}_m / \sigma_m^2$, and for any asset i:

$$\tilde{\mu}_i = \frac{\tilde{\mu}_m}{\sigma_m^2} \text{Cov}(r_i, r_m) = \beta_i \tilde{\mu}_m$$

where:

$$\beta_i = \frac{\text{Cov}(r_i, r_m)}{\text{Var}(r_m)}$$

This is the famous **CAPM equation**:

$$E(r_i) - r_f = \beta_i [E(r_m) - r_f]$$

16.6.4 Interpreting Beta

Beta (β_i) measures an asset's **systematic risk** (i.e., its sensitivity to market movements). The interpretation is straightforward:

- $\beta = 1$: The asset moves one-for-one with the market (average systematic risk)
- $\beta > 1$: The asset amplifies market movements (aggressive, high systematic risk)
- $\beta < 1$: The asset dampens market movements (defensive, low systematic risk)
- $\beta < 0$: The asset moves opposite to the market (provides insurance)

The CAPM says that expected excess return is proportional to beta, not to total volatility. This explains why:

1. An asset with zero beta earns only the risk-free rate (i.e., its risk is entirely idiosyncratic).
2. An asset with beta of 1 earns the market risk premium
3. A negative-beta asset earns *less* than the risk-free rate (i.e., investors pay for its insurance properties).

16.7 The Security Market Line

The CAPM predicts a linear relationship between beta and expected return. This relationship is called the **Security Market Line (SML)**:

$$E(r_i) = r_f + \beta_i[E(r_m) - r_f]$$

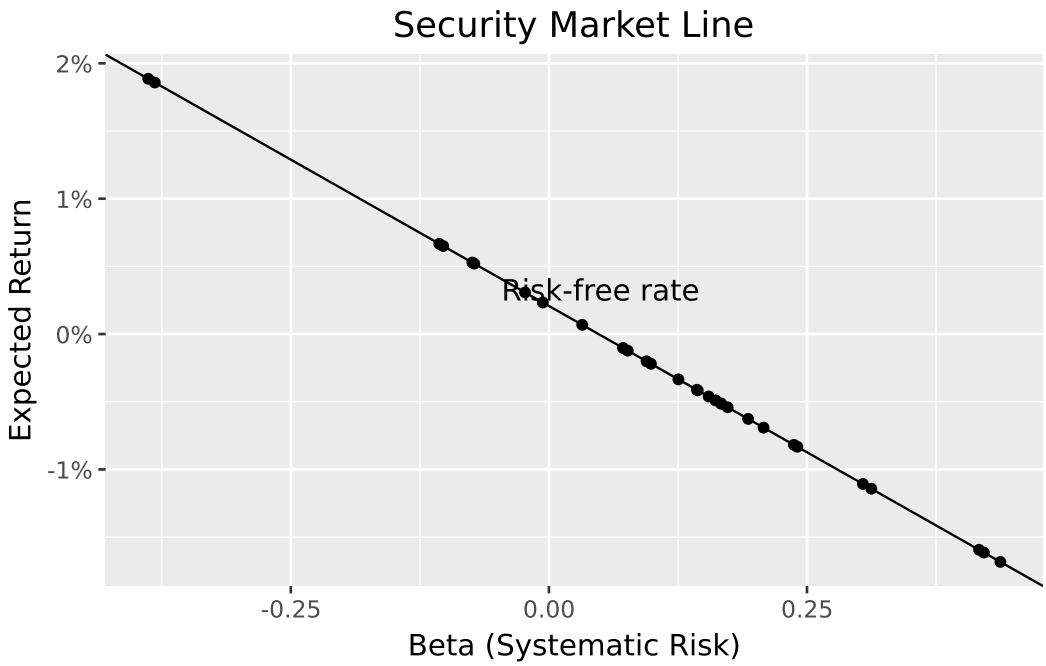
Unlike the Capital Market Line (which plots expected return against total risk), the Security Market Line plots expected return against systematic risk (beta).

```
betas = (sigma @ w_tan) / (w_tan.T @ sigma @ w_tan)
assets["beta"] = betas.values

price_of_risk = float(w_tan.T @ mu - rf)

assets_figure = (
    ggplot(assets, aes(x="beta", y="mu"))
    + geom_point()
    + geom_abline(intercept=rf, slope=price_of_risk)
    + scale_y_continuous(labels=percent_format())
    + labs(
        x="Beta (Systematic Risk)",
        y="Expected Return",
        title="Security Market Line"
    )
    + annotate("text", x=0.05, y=rf + 0.001, label="Risk-free rate")
)

assets_figure.show()
```



You may observe that the estimated SML has a negative slope, which seems to contradict CAPM's prediction. This reflects a **negative estimated market risk premium** in our sample period (i.e., the market portfolio earned less than the risk-free rate).

When the market risk premium is negative, CAPM predicts that high-beta stocks should have *lower* expected returns than low-beta stocks. This is not a model failure, the model is behaving consistently. Rather, it reflects an unusual (but not impossible) sample period where risky assets underperformed safe assets.

This observation highlights an important distinction: CAPM describes *expected* returns in equilibrium, but *realized* returns over any particular period may differ substantially from expectations due to shocks and surprises.

16.8 Empirical Estimation of CAPM Parameters

16.8.1 The Regression Framework

In practice, we estimate CAPM parameters using time-series regression. The model implies:

$$r_{i,t} - r_{f,t} = \alpha_i + \beta_i(r_{m,t} - r_{f,t}) + \varepsilon_{i,t}$$

where:

- $r_{i,t}$: Return on asset i at time t
- $r_{f,t}$: Risk-free rate at time t
- $r_{m,t}$: Market return at time t
- α_i: Intercept (should be zero if CAPM holds)
- β_i: Systematic risk (slope coefficient)
- $\varepsilon_{i,t}$: Idiosyncratic shock (residual)

16.8.2 Alpha: Risk-Adjusted Performance

The intercept α_i measures **risk-adjusted performance**. If CAPM holds perfectly, alpha should be zero for all assets (i.e., any excess return is exactly compensated by systematic risk).

- $\alpha > 0$: The asset outperformed its CAPM-predicted return (positive abnormal return)
- $\alpha < 0$: The asset underperformed its CAPM-predicted return (negative abnormal return)

Positive alpha is the holy grail of active management: earning returns beyond what systematic risk exposure would justify.

16.8.3 Loading Factor Data

We use Fama-French market excess returns as our market portfolio proxy. These data provide a widely accepted benchmark that is already adjusted for the risk-free rate:

```
import sqlite3

tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")

factors = pd.read_sql_query(
    sql="SELECT date, smb, hml, rmw, cma, mkt_excess FROM
 ↪  factors_ff5_monthly",
    con=tidy_finance,
    parse_dates={"date"}
)

factors.head(3)
```

	date	smb	hml	rmw	cma	mkt_excess
0	2011-07-31	-0.015907	-0.002812	0.060525	0.045291	-0.078748
1	2011-08-31	-0.061842	0.006189	-0.022700	-0.023177	0.029906

	date	smb	hml	rmw	cma	mkt_excess
2	2011-09-30	0.014387	0.024301	-0.006005	0.003588	-0.002173

16.8.4 Running the Regressions

We estimate CAPM regressions for each stock in our universe:

```
import statsmodels.formula.api as smf

returns_excess_monthly = (returns_monthly
    .merge(factors, on="date", how="left")
    .assign(ret_excess=lambda x: x["ret"] - x["risk_free"])
)


def estimate_capm(data):
    model = smf.ols("ret_excess ~ mkt_excess", data=data).fit()
    result = pd.DataFrame({
        "coefficient": ["alpha", "beta"],
        "estimate": model.params.values,
        "t_statistic": model.tvalues.values
    })
    return result


capm_results = (returns_excess_monthly
    .groupby("symbol", group_keys=True)
    .apply(estimate_capm)
    .reset_index()
)

capm_results.head(4)
```

	symbol	level_1	coefficient	estimate	t_statistic
0	ACB	0	alpha	-0.000826	-0.105475
1	ACB	1	beta	0.604653	4.575750
2	BCM	0	alpha	0.035093	2.368668
3	BCM	1	beta	1.032462	4.581461

16.8.5 Visualizing Alpha Estimates

Figure 16.2 shows the estimated alphas across our VN30 sample. Statistical significance (at the 95% level) is indicated by color.

```
alphas = (capm_results
    .query("coefficient == 'alpha'")
    .assign(is_significant=lambda x: np.abs(x["t_statistic"]) >= 1.96)
)

alphas["symbol"] = pd.Categorical(
    alphas["symbol"],
    categories=alphas.sort_values("estimate")["symbol"],
    ordered=True
)

alphas_figure = (
    ggplot(alphas, aes(y="estimate", x="symbol", fill="is_significant"))
    + geom_col()
    + scale_y_continuous(labels=percent_format())
    + coord_flip()
    + labs(
        x="",
        y="Estimated Alpha (Monthly)",
        fill="Significant at 95%?",
        title="Estimated CAPM Alphas for VN30 Index Constituents"
    )
)

alphas_figure.show()
```

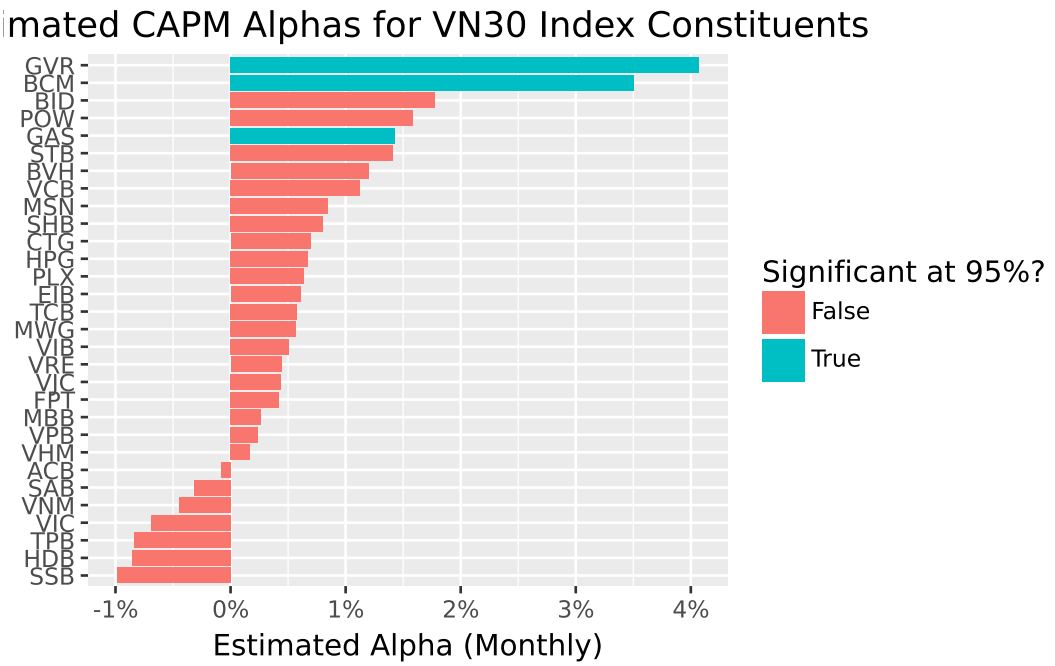


Figure 16.2: Estimated CAPM alphas for VN30 index constituents. Color indicates statistical significance at the 95% confidence level. Most alphas are statistically indistinguishable from zero, consistent with CAPM predictions.

The distribution of alphas provides evidence on CAPM's empirical validity. If the model holds, we expect:

1. Most alphas close to zero
2. Few statistically significant alphas
3. Roughly equal numbers of positive and negative alphas

Systematic patterns in alphas, such as consistently positive alphas for certain types of stocks, would suggest the CAPM is incomplete and that additional risk factors may be needed.

16.9 Limitations and Extensions

16.9.1 The Market Portfolio Problem

A fundamental challenge in testing the CAPM is identifying the market portfolio. The theory requires a portfolio that includes *all* investable assets, not just stocks, but also bonds, real estate, private businesses, human capital, and even intangible assets. In practice, we use

proxies like broad market indices (VNI, S&P 500), but these capture only publicly traded equities.

This limitation is profound. As Richard Roll famously argued, the CAPM is essentially untestable because the true market portfolio is unobservable. Any test of the CAPM is simultaneously a test of whether our proxy adequately represents the market.

16.9.2 Time-Varying Betas

The CAPM assumes that betas are constant over time, but this assumption rarely holds in practice. Companies undergo changes that affect their market sensitivity:

- **Capital structure changes**: Increasing leverage raises beta
- **Business model evolution**: Diversification into new industries can alter systematic risk
- **Market conditions**: Betas often increase during market stress

Conditional CAPM models (Jagannathan and Wang 1996a) address this by allowing risk premiums and betas to vary with the business cycle.

16.9.3 Empirical Anomalies

Decades of empirical research have documented patterns in stock returns that CAPM cannot explain:

1. **Size effect**: Small-cap stocks tend to outperform large-cap stocks, even after adjusting for beta
2. **Value effect**: Stocks with high book-to-market ratios outperform growth stocks
3. **Momentum**: Stocks that performed well recently tend to continue performing well

These anomalies suggest that systematic risk has multiple dimensions beyond market exposure.

16.9.4 Multifactor Extensions

The limitations of CAPM have led to increasingly sophisticated asset pricing models. The **Fama-French three-factor model** (Eugene F. Fama and French 1992) adds two factors to capture size and value effects:

- **SMB (Small Minus Big)**: Returns on small stocks minus large stocks
- **HML (High Minus Low)**: Returns on value stocks minus growth stocks

The **Fama-French five-factor model** (Eugene F. Fama and French 2015) adds two more dimensions:

- **RMW (Robust Minus Weak)**: Returns on profitable firms minus unprofitable firms
- **CMA (Conservative Minus Aggressive)**: Returns on conservative investors minus aggressive investors

The **Carhart four-factor model** (Mark M. Carhart 1997b) adds momentum to the three-factor framework.

Other theoretical developments include:

- **Consumption CAPM**: Links asset prices to macroeconomic consumption risk
- **Q-factor model** (Hou, Xue, and Zhang 2014): Derives factors from investment-based asset pricing theory
- **Arbitrage Pricing Theory**: Allows for multiple sources of systematic risk without specifying their identity

Despite its limitations, the CAPM remains valuable as a conceptual benchmark. Its core insight (i.e., only systematic, undiversifiable risk commands a premium) continues to inform how we think about risk and return.

16.10 Key Takeaways

This chapter introduced the Capital Asset Pricing Model and its implications for understanding the relationship between risk and expected return. The main insights are:

1. **Not all risk is rewarded**: The CAPM distinguishes between systematic risk (which cannot be diversified away and commands a premium) and idiosyncratic risk (which can be eliminated through diversification and earns no premium).
2. **The tangency portfolio is universal**: When a risk-free asset exists, all mean-variance investors hold the same portfolio of risky assets (i.e., the tangency or maximum Sharpe ratio portfolio). They differ only in how much they allocate to this portfolio versus the risk-free asset.
3. **In equilibrium, the tangency portfolio is the market portfolio**: Since all investors hold the same risky portfolio, and total demand must equal supply, the equilibrium portfolio weights are market capitalization weights.
4. **Expected returns depend on beta**: The CAPM equation states that expected excess return equals beta times the market risk premium. Beta measures covariance with the market portfolio, normalized by market variance.

5. **Alpha measures risk-adjusted performance**: Positive alpha indicates returns above what systematic risk would justify; negative alpha indicates underperformance.
6. **Empirical challenges exist**: Testing the CAPM requires identifying the market portfolio, which is unobservable in practice. Documented anomalies (size, value, momentum) suggest additional risk factors beyond market exposure.
7. **Extensions abound**: Multifactor models like Fama-French extend the CAPM framework by adding factors that capture dimensions of systematic risk the market factor misses.

The CAPM's elegance lies in its simplicity: a single factor (i.e., exposure to the market) should explain expected returns in equilibrium. While reality is more complex, this framework provides the foundation for all modern asset pricing theory.

17 Fama-French Factors

This chapter provides a replication of the Fama-French factor portfolios for the Vietnamese stock market. The Fama-French factor models represent a cornerstone of empirical asset pricing, originating from the seminal work of Eugene F. Fama and French (1992) and later extended in Eugene F. Fama and French (2015). These models have transformed how academics and practitioners understand the cross-section of expected stock returns, moving beyond the single-factor Capital Asset Pricing Model to incorporate multiple sources of systematic risk.

We construct both the three-factor and five-factor models at monthly and daily frequencies. The monthly factors serve as the foundation for most asset pricing tests and portfolio analyses, while the daily factors enable higher-frequency applications including short-horizon event studies, market microstructure research, and daily beta estimation. By constructing factors at both frequencies, we create a complete toolkit for empirical finance research in the Vietnamese market.

The chapter proceeds as follows. We first discuss the theoretical motivation for each factor and the economic intuition behind the Fama-French methodology. We then prepare the necessary data, merging stock returns with accounting characteristics. Next, we implement the portfolio sorting procedures that form the basis of factor construction, carefully following the original Fama-French protocols while adapting them for Vietnamese market characteristics. We construct the three-factor model (market, size, and value) before extending to the five-factor model (adding profitability and investment). Finally, we construct daily factors and validate our replicated factors through various diagnostic checks.

17.1 Theoretical Background

17.1.1 The Evolution from CAPM to Multi-Factor Models

The Capital Asset Pricing Model of Sharpe (1964) posits that a single factor—the market portfolio—should explain all cross-sectional variation in expected returns. However, decades of empirical research have documented persistent patterns that CAPM cannot explain. Eugene F. Fama and French (1992) demonstrated that two firm characteristics—size and book-to-market ratio—capture substantial variation in average returns that the market beta leaves unexplained.

Small firms tend to earn higher returns than large firms, a pattern known as the size effect. Similarly, firms with high book-to-market ratios (value stocks) tend to outperform firms with low book-to-market ratios (growth stocks), known as the value premium. The three-factor model formalizes these observations by constructing tradeable factor portfolios:

$$r_{i,t} - r_{f,t} = \alpha_i + \beta_i^{MKT}(r_{m,t} - r_{f,t}) + \beta_i^{SMB} \cdot SMB_t + \beta_i^{HML} \cdot HML_t + \varepsilon_{i,t} \tag{17.1}$$

where:

- $r_{i,t} - r_{f,t}$ is the excess return on asset i
- $r_{m,t} - r_{f,t}$ is the market excess return
- SMB_t (Small Minus Big) is the size factor
- HML_t (High Minus Low) is the value factor

17.1.2 The Five-Factor Extension

Eugene F. Fama and French (2015) extended the model to include two additional factors motivated by the dividend discount model. Firms with higher profitability should have higher expected returns (all else equal), and firms with aggressive investment policies should have lower expected returns:

$$r_{i,t} - r_{f,t} = \alpha_i + \beta_i^{MKT} MKT_t + \beta_i^{SMB} SMB_t + \beta_i^{HML} HML_t + \beta_i^{RMW} RMW_t + \beta_i^{CMA} CMA_t + \varepsilon_{i,t} \tag{17.2}$$

where:

- RMW_t (Robust Minus Weak) is the profitability factor
- CMA_t (Conservative Minus Aggressive) is the investment factor

17.1.3 Factor Construction Methodology

The Fama-French methodology constructs factors through double-sorted portfolios:

1. **Size sorts**: Stocks are divided into Small and Big groups based on median market capitalization.
2. **Characteristic sorts**: Within each size group, stocks are sorted into terciles based on book-to-market (for HML), operating profitability (for RMW), or investment (for CMA).
3. **Factor returns**: Factors are computed as the difference between average returns of portfolios with high versus low characteristic values, averaging across size groups to neutralize size effects.

4. **Timing**: Portfolios are formed at the end of June each year using accounting data from the prior fiscal year, ensuring all information was publicly available at formation.

17.2 Setting Up the Environment

We load the required Python packages for data manipulation, statistical analysis, and visualization.

```
import pandas as pd
import numpy as np
import sqlite3
import statsmodels.formula.api as smf
from scipy.stats.mstats import winsorize
import matplotlib.pyplot as plt

from plotnine import *
from mizani.formatters import percent_format, comma_format
```

We connect to our SQLite database containing the processed Vietnamese financial data.

```
tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")
```

17.3 Data Preparation

17.3.1 Loading Stock Returns

We load the monthly stock returns data, which includes excess returns, market capitalization, and the risk-free rate. These variables are essential for computing value-weighted portfolio returns and factor premiums.

```
prices_monthly = pd.read_sql_query(
    sql="""
        SELECT symbol, date, ret_excess, mktcap, mktcap_lag, risk_free
        FROM prices_monthly
    """,
    con=tidy_finance,
    parse_dates={"date"}
).dropna()
```

```
print(f"Monthly returns: {len(prices_monthly):,} observations")
print(f"Unique stocks: {prices_monthly['symbol'].nunique():,}")
print(f"Date range: {prices_monthly['date'].min():%Y-%m} to
 ↪  {prices_monthly['date'].max():%Y-%m}")
```

```
Monthly returns: 165,499 observations
Unique stocks: 1,457
Date range: 2010-02 to 2023-12
```

17.3.2 Loading Company Fundamentals

We load the company fundamentals data containing book equity, operating profitability, and investment—the characteristics needed for constructing the Fama-French factors.

```
comp_vn = pd.read_sql_query(
    sql="""
        SELECT symbol, datadate, be, op, inv
        FROM comp_vn
    """,
    con=tidy_finance,
    parse_dates={"datadate"}
).dropna()

print(f"Fundamentals: {len(comp_vn):,} firm-year observations")
print(f"Unique firms: {comp_vn['symbol'].nunique():,}")
```

```
Fundamentals: 18,108 firm-year observations
Unique firms: 1,496
```

17.3.3 Constructing Sorting Variables

Following Fama-French conventions, we construct the sorting variables with careful attention to timing. The key principles are:

1. **Size (June Market Cap)**: We use market capitalization at the end of June of year t to sort stocks into size groups. This ensures we capture the firm's size at the moment of portfolio formation.
2. **Book-to-Market Ratio**: We use book equity from fiscal year $t-1$ divided by market equity at the end of December $t-1$. This creates a six-month gap between the accounting data and portfolio formation, ensuring the information was publicly available.

3. **Portfolio Formation Date**: Portfolios are formed on July 1st and held for twelve months until the following June.

```
def construct_sorting_variables(prices_monthly, comp_vn):
    """
    Construct sorting variables following Fama-French methodology.

    Parameters
    ----------
    prices_monthly : pd.DataFrame
        Monthly stock returns with market cap
    comp_vn : pd.DataFrame
        Company fundamentals with book equity, profitability, investment

    Returns
    -------
    pd.DataFrame
        Sorting variables aligned with July 1st formation dates
    """

    # 1. Size: June market capitalization
    # Portfolio formation is July 1st, so we use June market cap
    size = (prices_monthly
        .query("date.dt.month == 6")
        .assign(
            sorting_date=lambda x: x["date"] + pd.offsets.MonthBegin(1)
        )
        [["symbol", "sorting_date", "mktcap"]]
        .rename(columns={"mktcap": "size"})
    )

    print(f"Size observations: {len(size):,}")

    # 2. Market Equity: December market cap for B/M calculation
    # December t-1 market cap is used with fiscal year t-1 book equity
    # This is then used for July t portfolio formation
    market_equity = (prices_monthly
        .query("date.dt.month == 12")
        .assign(
            # December year t-1 maps to July year t formation
            sorting_date=lambda x: x["date"] + pd.offsets.MonthBegin(7)
        )
        [["symbol", "sorting_date", "mktcap"]]
```

```
        .rename(columns={"mktcap": "me"})
    )

    print(f"Market equity observations: {len(market_equity):,}")

    # 3. Book-to-Market and other characteristics
    # Fiscal year t-1 data is used for July t portfolio formation
    book_to_market = (comp_vn
        .assign(
            # Fiscal year-end + 6 months = July formation
            sorting_date=lambda x: pd.to_datetime(
                (x["datadate"].dt.year + 1).astype(str) + "-07-01"
            )
        )
        .merge(market_equity, on=["symbol", "sorting_date"], how="inner")
        .assign(
            # Scale book equity to match market equity units
            # BE is in VND, ME is in millions VND
            bm=lambda x: x["be"] / (x["me"] * 1e9)
        )
        [["symbol", "sorting_date", "me", "bm", "op", "inv"]]
    )

    print(f"Book-to-market observations: {len(book_to_market):,}")

    # 4. Merge size with characteristics
    sorting_variables = (size
        .merge(book_to_market, on=["symbol", "sorting_date"], how="inner")
        .dropna()
        .drop_duplicates(subset=["symbol", "sorting_date"])
    )

    return sorting_variables

sorting_variables = construct_sorting_variables(prices_monthly, comp_vn)

print(f"\nFinal sorting variables: {len(sorting_variables):,} stock-years")
print(f"Sorting date range: {sorting_variables['sorting_date'].min():%Y-%m}
 ↪  to {sorting_variables['sorting_date'].max():%Y-%m}")
```

```
Size observations: 13,756
Market equity observations: 14,286
```

```
Book-to-market observations: 13,389

Final sorting variables: 12,046 stock-years
Sorting date range: 2011-07 to 2023-07
```

17.3.4 Validating Sorting Variables

Before proceeding, we validate that our sorting variables have reasonable distributions. The book-to-market ratio should center around 1.0 for a typical market, though emerging markets may differ.

```
print("Sorting Variable Summary Statistics:")
print(sorting_variables[["size", "bm", "op", "inv"]].describe().round(4))

# Check for extreme values that might indicate data issues
print(f"\nB/M Median: {sorting_variables['bm'].median():.4f}")
print(f"B/M 1st percentile: {sorting_variables['bm'].quantile(0.01):.4f}")
print(f"B/M 99th percentile: {sorting_variables['bm'].quantile(0.99):.4f}")
```

```
Sorting Variable Summary Statistics:
              size          bm          op         inv
count   12046.0000  12046.0000  12046.0000  12046.0000
mean     2225.5648      1.7033      0.1852      0.1322
std     14680.4225      3.8683      0.2782      2.5410
min         0.4864      0.0014     -0.7529     -0.9569
25%        62.7556      0.7595      0.0309     -0.0495
50%       182.6410      1.1849      0.1367      0.0342
75%       641.8896      1.8853      0.2952      0.1582
max    426020.9817    272.1893      1.4256    261.3355

B/M Median: 1.1849
B/M 1st percentile: 0.1710
B/M 99th percentile: 8.0262
```

17.3.5 Handling Outliers

Extreme values in sorting characteristics can distort portfolio assignments and factor returns. We apply winsorization to limit the influence of outliers while preserving the general ranking of stocks.

```python
# Check BEFORE winsorization
print("BEFORE Winsorization:")
print(sorting_variables[["size", "bm", "op", "inv"]].describe().round(4))

# Apply winsorization
def winsorize_characteristics(df, columns, limits=(0.01, 0.99)):
    """
    Apply winsorization using pandas clip.
    """
    df = df.copy()
    for col in columns:
        if col in df.columns:
            lower = df[col].quantile(limits[0])
            upper = df[col].quantile(limits[1])
            df[col] = df[col].clip(lower=lower, upper=upper)
            print(f"  {col}: clipped to [{lower:.4f}, {upper:.4f}]")
    return df

sorting_variables = winsorize_characteristics(
    sorting_variables,
    columns=["bm", "op", "inv"],  # Don't winsorize size
    limits=(0.01, 0.99)
)

# Check AFTER winsorization
print("\nAFTER Winsorization:")
print(sorting_variables[["size", "bm", "op", "inv"]].describe().round(4))
```

```
BEFORE Winsorization:
              size          bm          op         inv
count   12046.0000  12046.0000  12046.0000  12046.0000
mean     2225.5648      1.7033      0.1852      0.1322
std     14680.4225      3.8683      0.2782      2.5410
min         0.4864      0.0014     -0.7529     -0.9569
25%        62.7556      0.7595      0.0309     -0.0495
50%       182.6410      1.1849      0.1367      0.0342
75%       641.8896      1.8853      0.2952      0.1582
max    426020.9817    272.1893      1.4256    261.3355
  bm: clipped to [0.1710, 8.0262]
  op: clipped to [-0.7319, 1.2192]
  inv: clipped to [-0.3990, 1.5195]
```

```
AFTER Winsorization:
                size          bm          op         inv
count    12046.0000  12046.0000  12046.0000  12046.0000
mean      2225.5648      1.5544      0.1837      0.0894
std      14680.4225      1.2843      0.2705      0.2721
min          0.4864      0.1710     -0.7319     -0.3990
25%         62.7556      0.7595      0.0309     -0.0495
50%        182.6410      1.1849      0.1367      0.0342
75%        641.8896      1.8853      0.2952      0.1582
max     426020.9817      8.0262      1.2192      1.5195
```

17.4 Portfolio Assignment Functions

17.4.1 The Portfolio Assignment Function

We create a flexible function for assigning stocks to portfolios based on quantile breakpoints. This function handles both independent sorts (where breakpoints are computed across all stocks) and dependent sorts (where breakpoints are computed within subgroups).

```python
def assign_portfolio(data, sorting_variable, percentiles):
    """Assign portfolios to a bin according to a sorting variable."""

    # Get the values
    values = data[sorting_variable].dropna()

    if len(values) == 0:
        return pd.Series([np.nan] * len(data), index=data.index)

    # Calculate breakpoints
    breakpoints = values.quantile(percentiles, interpolation="linear")

    # Handle duplicate breakpoints by using unique values
    unique_breakpoints = np.unique(breakpoints)

    # If all values are the same, assign all to portfolio 1
    if len(unique_breakpoints) <= 1:
        return pd.Series([1] * len(data), index=data.index)

    # Set boundaries to -inf and +inf
    unique_breakpoints.iloc[0] = -np.inf
    unique_breakpoints.iloc[unique_breakpoints.size-1] = np.inf
```

```
    # Assign to bins
    assigned = pd.cut(
        data[sorting_variable],
        bins=unique_breakpoints,
        labels=pd.Series(range(1, breakpoints.size)),
        include_lowest=True,
        right=False
    )

    return assigned
```

```
# Check the distribution of characteristics BEFORE portfolio assignment
print("Operating Profitability Distribution:")
print(sorting_variables["op"].describe())
print(f"\nUnique OP values: {sorting_variables['op'].nunique()}")

print("\nInvestment Distribution:")
print(sorting_variables["inv"].describe())
print(f"\nUnique INV values: {sorting_variables['inv'].nunique()}")

# Check breakpoints for a specific date
test_date = sorting_variables["sorting_date"].iloc[0]
test_data = sorting_variables.query("sorting_date == @test_date")

print(f"\nBreakpoints for {test_date}:")
print(f"OP 30th percentile: {test_data['op'].quantile(0.3):.4f}")
print(f"OP 70th percentile: {test_data['op'].quantile(0.7):.4f}")
print(f"INV 30th percentile: {test_data['inv'].quantile(0.3):.4f}")
print(f"INV 70th percentile: {test_data['inv'].quantile(0.7):.4f}")
```

```
Operating Profitability Distribution:
count    12046.000000
mean         0.183738
std          0.270509
min         -0.731888
25%          0.030913
50%          0.136675
75%          0.295185
max          1.219223
Name: op, dtype: float64
```

```
Unique OP values: 11804

Investment Distribution:
count    12046.000000
mean         0.089388
std          0.272147
min         -0.399042
25%         -0.049497
50%          0.034157
75%          0.158155
max          1.519474
Name: inv, dtype: float64

Unique INV values: 11805

Breakpoints for 2019-07-01 00:00:00:
OP 30th percentile: 0.0541
OP 70th percentile: 0.2566
INV 30th percentile: -0.0343
INV 70th percentile: 0.1116
```

17.4.2 Assigning Portfolios for Three-Factor Model

For the three-factor model, we perform independent double sorts on size and book-to-market. Size is split at the median (2 groups), and book-to-market is split at the 30th and 70th percentiles (3 groups), creating 6 portfolios.

```python
def assign_ff3_portfolios(sorting_variables):
    """
    Assign portfolios for Fama-French three-factor model.
    Independent 2x3 sort on size and book-to-market.
    """
    df = sorting_variables.copy()

    # Independent size sort (median split)
    df["portfolio_size"] = df.groupby("sorting_date")["size"].transform(
        lambda x: pd.qcut(x, q=[0, 0.5, 1], labels=[1, 2], duplicates='drop')
    )

    # Independent B/M sort (30/70 split)
    df["portfolio_bm"] = df.groupby("sorting_date")["bm"].transform(
```

```
        lambda x: pd.qcut(x, q=[0, 0.3, 0.7, 1], labels=[1, 2, 3],
         ↪  duplicates='drop')
    )

    return df

# Assign portfolios
portfolios_ff3 = assign_ff3_portfolios(sorting_variables)

# Validate
print("FF3 Book-to-Market by Portfolio (should be INCREASING):")
print(portfolios_ff3.groupby("portfolio_bm",
 ↪  observed=True)["bm"].median().round(4))


print("Three-Factor Portfolio Assignments:")
print(portfolios_ff3[["symbol", "sorting_date", "portfolio_size",
 ↪  "portfolio_bm"]].head(10))
```

```
FF3 Book-to-Market by Portfolio (should be INCREASING):
portfolio_bm
1    0.5836
2    1.1891
3    2.4552
Name: bm, dtype: float64
Three-Factor Portfolio Assignments:
  symbol sorting_date portfolio_size portfolio_bm
0    A32   2019-07-01              1            2
1    A32   2020-07-01              1            2
2    A32   2021-07-01              1            2
3    A32   2022-07-01              1            3
4    A32   2023-07-01              1            2
5    AAA   2011-07-01              2            2
6    AAA   2012-07-01              2            3
7    AAA   2013-07-01              2            3
8    AAA   2014-07-01              2            2
9    AAA   2015-07-01              2            3
```

17.4.3 Validating Portfolio Assignments

We verify that the portfolio assignments create the expected 2×3 grid with reasonable stock counts in each cell.

```
# Check portfolio distribution for most recent year
latest_date = portfolios_ff3["sorting_date"].max()

portfolio_counts = (portfolios_ff3
    .query("sorting_date == @latest_date")
    .groupby(["portfolio_size", "portfolio_bm"], observed=True)
    .size()
    .unstack(fill_value=0)
)

print(f"Portfolio Counts for {latest_date:%Y-%m}:")
print(portfolio_counts)

# Verify characteristic monotonicity
print("\nBook-to-Market by Portfolio (should be increasing):")
print(portfolios_ff3.groupby("portfolio_bm",
 ↪  observed=True)["bm"].median().round(4))
```

```
Portfolio Counts for 2023-07:
portfolio_bm      1    2    3
portfolio_size
1               113  271  263
2               275  246  125

Book-to-Market by Portfolio (should be increasing):
portfolio_bm
1    0.5836
2    1.1891
3    2.4552
Name: bm, dtype: float64
```

We verify that for a single stock, the portfolio assignment remains constant between July of one year and June of the next.

```
# Trace a single symbol (e.g., 'A32') across a formation window
persistence_check = (portfolios_ff3
```

```
    .query("symbol == 'A32' & sorting_date >= '2022-01-01' & sorting_date <=
↪   '2023-12-31'")
    .sort_values("sorting_date")
    [['symbol', 'sorting_date', 'portfolio_size', 'portfolio_bm']]
)
print("\nTemporal Persistence Check (Symbol A32):")
print(persistence_check.head(15))
```

```
Temporal Persistence Check (Symbol A32):
  symbol sorting_date portfolio_size portfolio_bm
3    A32   2022-07-01              1            3
4    A32   2023-07-01              1            2
```

17.5 Fama-French Three-Factor Model (Monthly)

17.5.1 Merging Portfolios with Returns

We merge the portfolio assignments with monthly returns. The key insight is that portfolios formed in July of year t are held through June of year $t+1$. We implement this by computing a `sorting_date` for each monthly return observation.

```
def merge_portfolios_with_returns(prices_monthly, portfolio_assignments):
    """
    Merge portfolio assignments with monthly returns.

    Portfolios formed in July t are held through June t+1.

    Parameters
    ----------
    prices_monthly : pd.DataFrame
        Monthly stock returns
    portfolio_assignments : pd.DataFrame
        Portfolio assignments with sorting_date

    Returns
    -------
    pd.DataFrame
        Returns merged with portfolio assignments
    """
```

```
    portfolios = (prices_monthly
        .assign(
            # Map each return month to its portfolio formation date
            sorting_date=lambda x: pd.to_datetime(
                np.where(
                    x["date"].dt.month <= 6,
                    (x["date"].dt.year - 1).astype(str) + "-07-01",
                    x["date"].dt.year.astype(str) + "-07-01"
                )
            )
        )
        .merge(
            portfolio_assignments,
            on=["symbol", "sorting_date"],
            how="inner"
        )
    )

    return portfolios

portfolios_monthly_ff3 = merge_portfolios_with_returns(
    prices_monthly,
    portfolios_ff3[["symbol", "sorting_date", "portfolio_size",
↪   "portfolio_bm"]]
)


print(f"Merged observations: {len(portfolios_monthly_ff3):,}")
```

```
Merged observations: 136,444
```

17.5.2 Computing Value-Weighted Portfolio Returns

We compute value-weighted returns for each of the six portfolios. Value-weighting uses lagged market capitalization to avoid look-ahead bias.

```
def compute_portfolio_returns(data, grouping_vars):
    """
    Compute value-weighted portfolio returns.

    Parameters
```

```
    ----------
    data : pd.DataFrame
        Returns data with portfolio assignments and mktcap_lag
    grouping_vars : list
        Variables defining portfolio groups

    Returns
    -------
    pd.DataFrame
        Value-weighted returns for each portfolio-date
    """
    portfolio_returns = (data
        .groupby(grouping_vars + ["date"], observed=True)
        .apply(lambda x: pd.Series({
            "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"]),
            "n_stocks": len(x)
        }))
        .reset_index()
    )

    return portfolio_returns


# Compute portfolio returns
portfolio_returns_ff3 = compute_portfolio_returns(
    portfolios_monthly_ff3,
    ["portfolio_size", "portfolio_bm"]
)

print("Portfolio Returns Summary:")
print(portfolio_returns_ff3.groupby(["portfolio_size", "portfolio_bm"],
 ↪  observed=True)["ret"].describe().round(4))
```

```
Portfolio Returns Summary:
                            count    mean     std     min     25%     50%  \
portfolio_size portfolio_bm
1              1            150.0 -0.0052  0.0379 -0.1268 -0.0262 -0.0058
               2            150.0 -0.0025  0.0414 -0.1080 -0.0236 -0.0053
               3            150.0  0.0039  0.0601 -0.1612 -0.0269 -0.0004
2              1            150.0 -0.0124  0.0594 -0.2222 -0.0449 -0.0107
               2            150.0 -0.0021  0.0671 -0.1701 -0.0403 -0.0046
               3            150.0  0.0024  0.0879 -0.2359 -0.0527 -0.0012
```

```
                                 75%     max
portfolio_size portfolio_bm
1              1             0.0161  0.0849
               2             0.0180  0.1195
               3             0.0314  0.2015
2              1             0.0210  0.1741
               2             0.0317  0.1770
               3             0.0437  0.2124
```

17.5.3 Constructing SMB and HML Factors

We now construct the SMB and HML factors from the portfolio returns.

SMB (Small Minus Big): Average return of three small portfolios minus average return of three big portfolios.

HML (High Minus Low): Average return of two high B/M portfolios minus average return of two low B/M portfolios.

```python
def construct_ff3_factors(portfolio_returns):
    """
    Construct Fama-French three factors from portfolio returns.

    Parameters
    ----------
    portfolio_returns : pd.DataFrame
        Value-weighted returns for 2x3 portfolios

    Returns
    -------
    pd.DataFrame
        Monthly SMB and HML factors
    """
    factors = (portfolio_returns
        .groupby("date")
        .apply(lambda x: pd.Series({
            # SMB: Small minus Big (average across B/M groups)
            "smb": (
                x.loc[x["portfolio_size"] == 1, "ret"].mean() -
                x.loc[x["portfolio_size"] == 2, "ret"].mean()
            ),
            # HML: High minus Low B/M (average across size groups)
```

```
            "hml": (
                x.loc[x["portfolio_bm"] == 3, "ret"].mean() -
                x.loc[x["portfolio_bm"] == 1, "ret"].mean()
            )
        }))
        .reset_index()
    )

    return factors

factors_smb_hml = construct_ff3_factors(portfolio_returns_ff3)

print("SMB and HML Factors:")
print(factors_smb_hml.head(10))
```

```
SMB and HML Factors:
        date       smb       hml
0 2011-07-31 -0.007768  0.002754
1 2011-08-31 -0.067309  0.011474
2 2011-09-30  0.014884  0.022854
3 2011-10-31 -0.003743  0.001631
4 2011-11-30  0.063234  0.009103
5 2011-12-31  0.014571  0.015280
6 2012-01-31 -0.026080  0.009672
7 2012-02-29 -0.035721  0.005474
8 2012-03-31 -0.002344  0.032477
9 2012-04-30 -0.033391  0.074191
```

17.5.4 Computing the Market Factor

The market factor is the value-weighted return of all stocks minus the risk-free rate. We compute this independently from the sorted portfolios.

```
def compute_market_factor(prices_monthly):
    """
    Compute value-weighted market excess return.

    Parameters
    ----------
    prices_monthly : pd.DataFrame
        Monthly stock returns with mktcap_lag
```

```
    Returns
    -------
    pd.DataFrame
        Monthly market excess return
    """
    market_factor = (prices_monthly
        .groupby("date")
        .apply(lambda x: pd.Series({
            "mkt_excess": np.average(x["ret_excess"],
             ↪  weights=x["mktcap_lag"]),
            "n_stocks": len(x)
        }), include_groups=False)
        .reset_index()
    )

    return market_factor

market_factor = compute_market_factor(prices_monthly)

print("Market Factor Summary:")
print(market_factor["mkt_excess"].describe().round(4))
```

```
Market Factor Summary:
count    167.0000
mean      -0.0123
std        0.0595
min       -0.2149
25%       -0.0394
50%       -0.0106
75%        0.0200
max        0.1677
Name: mkt_excess, dtype: float64
```

17.5.5 Combining Three Factors

We combine SMB, HML, and the market factor into the complete three-factor dataset.

```
factors_ff3_monthly = (factors_smb_hml
    .merge(market_factor[["date", "mkt_excess"]], on="date", how="inner")
)
```

```
# Add risk-free rate for completeness
rf_monthly = (prices_monthly
    .groupby("date")["risk_free"]
    .first()
    .reset_index()
)

factors_ff3_monthly = factors_ff3_monthly.merge(rf_monthly, on="date",
 ↪  how="left")

print("Fama-French Three Factors (Monthly):")
print(factors_ff3_monthly.head(10))

print("\nFactor Summary Statistics:")
print(factors_ff3_monthly[["mkt_excess", "smb", "hml"]].describe().round(4))
```

```
Fama-French Three Factors (Monthly):
        date       smb       hml  mkt_excess  risk_free
0 2011-07-31 -0.007768  0.002754   -0.078748   0.003333
1 2011-08-31 -0.067309  0.011474    0.029906   0.003333
2 2011-09-30  0.014884  0.022854   -0.002173   0.003333
3 2011-10-31 -0.003743  0.001631   -0.014005   0.003333
4 2011-11-30  0.063234  0.009103   -0.179410   0.003333
5 2011-12-31  0.014571  0.015280   -0.094802   0.003333
6 2012-01-31 -0.026080  0.009672    0.081273   0.003333
7 2012-02-29 -0.035721  0.005474    0.069655   0.003333
8 2012-03-31 -0.002344  0.032477    0.029005   0.003333
9 2012-04-30 -0.033391  0.074191    0.048791   0.003333

Factor Summary Statistics:
       mkt_excess       smb       hml
count    150.0000  150.0000  150.0000
mean      -0.0101    0.0027    0.0120
std        0.0586    0.0420    0.0535
min       -0.2149   -0.1599   -0.1284
25%       -0.0380   -0.0175   -0.0160
50%       -0.0095    0.0070    0.0043
75%        0.0214    0.0261    0.0340
max        0.1677    0.1175    0.1618
```

17.5.6 Saving Three-Factor Data

```
factors_ff3_monthly.to_sql(
    name="factors_ff3_monthly",
    con=tidy_finance,
    if_exists="replace",
    index=False
)

print("Three-factor monthly data saved to database.")
```

```
Three-factor monthly data saved to database.
```

17.6 Fama-French Five-Factor Model (Monthly)

17.6.1 Portfolio Assignments with Dependent Sorts

For the five-factor model, we use dependent sorts: size is sorted independently, but profitability and investment are sorted within size groups. This controls for the correlation between size and these characteristics.

```
def assign_ff5_portfolios(sorting_variables):
    """
    Assign portfolios for Fama-French five-factor model.
    """
    df = sorting_variables.copy()

    # Independent size sort
    df["portfolio_size"] = df.groupby("sorting_date")["size"].transform(
        lambda x: pd.qcut(x, q=[0, 0.5, 1], labels=[1, 2], duplicates='drop')
    )

    # Dependent sorts within size groups
    df["portfolio_bm"] = df.groupby(["sorting_date",
↪   "portfolio_size"])["bm"].transform(
        lambda x: pd.qcut(x, q=[0, 0.3, 0.7, 1], labels=[1, 2, 3],
        ↪  duplicates='drop')
    )
```

```
    df["portfolio_op"] = df.groupby(["sorting_date",
↪   "portfolio_size"])["op"].transform(
        lambda x: pd.qcut(x, q=[0, 0.3, 0.7, 1], labels=[1, 2, 3],
        ↪  duplicates='drop')
    )

    df["portfolio_inv"] = df.groupby(["sorting_date",
↪   "portfolio_size"])["inv"].transform(
        lambda x: pd.qcut(x, q=[0, 0.3, 0.7, 1], labels=[1, 2, 3],
        ↪  duplicates='drop')
    )

    return df

# Run
portfolios_ff5 = assign_ff5_portfolios(sorting_variables)
```

17.6.2 Validating Five-Factor Portfolios

```
# Check characteristic monotonicity for each dimension
print("Profitability by Portfolio (should be increasing):")
print(portfolios_ff5.groupby("portfolio_op",
↪   observed=True)["op"].median().round(4))

print("\nInvestment by Portfolio (should be increasing):")
print(portfolios_ff5.groupby("portfolio_inv",
↪   observed=True)["inv"].median().round(4))

# Check portfolio counts
print("\nStocks per Size/Profitability Bin:")
print(portfolios_ff5.groupby(["portfolio_size", "portfolio_op"],
↪   observed=True).size().unstack(fill_value=0))

# Check number of unique firms per year
(portfolios_ff5
 .groupby("sorting_date")["symbol"]
 .nunique())
```

```
Profitability by Portfolio (should be increasing):
portfolio_op
```

```
1   -0.0053
2    0.1366
3    0.4098
Name: op, dtype: float64

Investment by Portfolio (should be increasing):
portfolio_inv
1   -0.1012
2    0.0329
3    0.2568
Name: inv, dtype: float64

Stocks per Size/Profitability Bin:
portfolio_op       1     2     3
portfolio_size
1               1812  2403  1811
2               1811  2401  1808


sorting_date
2011-07-01     556
2012-07-01     632
2013-07-01     643
2014-07-01     650
2015-07-01     669
2016-07-01     737
2017-07-01     842
2018-07-01    1073
2019-07-01    1183
2020-07-01    1224
2021-07-01    1258
2022-07-01    1286
2023-07-01    1293
Name: symbol, dtype: int64
```

17.6.3 Merging and Computing Portfolio Returns

```
# Merge with returns
portfolios_monthly_ff5 = merge_portfolios_with_returns(
    prices_monthly,
    portfolios_ff5[["symbol", "sorting_date", "portfolio_size",
```

```
                    "portfolio_bm", "portfolio_op", "portfolio_inv"]]
)

print(f"Five-factor merged observations: {len(portfolios_monthly_ff5):,}")
```

```
Five-factor merged observations: 136,444
```

17.6.4 Constructing All Five Factors

We construct each factor from the appropriate portfolio sorts.

```
def construct_ff5_factors(portfolios_monthly):
    """
    Construct Fama-French five factors from portfolio data.

    Parameters
    ----------
    portfolios_monthly : pd.DataFrame
        Monthly returns with all portfolio assignments

    Returns
    -------
    pd.DataFrame
        Monthly five-factor returns
    """

    # HML: Value factor from B/M sorts
    portfolios_bm = (portfolios_monthly
        .groupby(["portfolio_size", "portfolio_bm", "date"], observed=True)
        .apply(lambda x: pd.Series({
            "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
        }))
        .reset_index()
    )

    factors_hml = (portfolios_bm
        .groupby("date")
        .apply(lambda x: pd.Series({
            "hml": (x.loc[x["portfolio_bm"] == 3, "ret"].mean() -
                    x.loc[x["portfolio_bm"] == 1, "ret"].mean())
        }))
```

```
        .reset_index()
    )

    # RMW: Profitability factor from OP sorts
    portfolios_op = (portfolios_monthly
        .groupby(["portfolio_size", "portfolio_op", "date"], observed=True)
        .apply(lambda x: pd.Series({
            "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
        }))
        .reset_index()
    )

    factors_rmw = (portfolios_op
        .groupby("date")
        .apply(lambda x: pd.Series({
            "rmw": (x.loc[x["portfolio_op"] == 3, "ret"].mean() -
                    x.loc[x["portfolio_op"] == 1, "ret"].mean())
        }))
        .reset_index()
    )

    # CMA: Investment factor from INV sorts
    # Note: CMA is Conservative minus Aggressive (low inv - high inv)
    portfolios_inv = (portfolios_monthly
        .groupby(["portfolio_size", "portfolio_inv", "date"], observed=True)
        .apply(lambda x: pd.Series({
            "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
        }))
        .reset_index()
    )

    factors_cma = (portfolios_inv
        .groupby("date")
        .apply(lambda x: pd.Series({
            "cma": (x.loc[x["portfolio_inv"] == 1, "ret"].mean() -
                    x.loc[x["portfolio_inv"] == 3, "ret"].mean())
        }))
        .reset_index()
    )

    # SMB: Size factor (average across all characteristic portfolios)
    all_portfolios = pd.concat([portfolios_bm, portfolios_op,
↪   portfolios_inv])
```

```
    factors_smb = (all_portfolios
        .groupby("date")
        .apply(lambda x: pd.Series({
            "smb": (x.loc[x["portfolio_size"] == 1, "ret"].mean() -
                    x.loc[x["portfolio_size"] == 2, "ret"].mean())
        }))
        .reset_index()
    )

    # Combine all factors
    factors = (factors_smb
        .merge(factors_hml, on="date", how="outer")
        .merge(factors_rmw, on="date", how="outer")
        .merge(factors_cma, on="date", how="outer")
    )

    return factors

factors_ff5 = construct_ff5_factors(portfolios_monthly_ff5)

# Add market factor
factors_ff5_monthly = (factors_ff5
    .merge(market_factor[["date", "mkt_excess"]], on="date", how="inner")
    .merge(rf_monthly, on="date", how="left")
)

print("Fama-French Five Factors (Monthly):")
print(factors_ff5_monthly.head(10))

print("\nFactor Summary Statistics:")
print(factors_ff5_monthly[["mkt_excess", "smb", "hml", "rmw",
 ↪  "cma"]].describe().round(4))
```

```
Fama-French Five Factors (Monthly):
        date       smb       hml       rmw       cma  mkt_excess  risk_free
0 2011-07-31 -0.015907 -0.002812  0.060525  0.045291   -0.078748   0.003333
1 2011-08-31 -0.061842  0.006189 -0.022700 -0.023177    0.029906   0.003333
2 2011-09-30  0.014387  0.024301 -0.006005  0.003588   -0.002173   0.003333
3 2011-10-31 -0.006958 -0.006940  0.026694  0.003649   -0.014005   0.003333
4 2011-11-30  0.074369  0.015617 -0.058766  0.044214   -0.179410   0.003333
5 2011-12-31  0.006687  0.022494  0.062655  0.052444   -0.094802   0.003333
```

```
6 2012-01-31 -0.016254  0.010513 -0.042191 -0.067170    0.081273   0.003333
7 2012-02-29 -0.026606  0.024465 -0.030849 -0.036383    0.069655   0.003333
8 2012-03-31  0.005096  0.050930 -0.018441  0.043488    0.029005   0.003333
9 2012-04-30  0.000712  0.058214 -0.061434  0.009233    0.048791   0.003333

Factor Summary Statistics:
       mkt_excess       smb       hml       rmw       cma
count    150.0000  150.0000  150.0000  150.0000  150.0000
mean      -0.0101    0.0077    0.0115   -0.0047    0.0083
std        0.0586    0.0419    0.0518    0.0477    0.0335
min       -0.2149   -0.1522   -0.1283   -0.2126   -0.0814
25%       -0.0380   -0.0137   -0.0126   -0.0308   -0.0131
50%       -0.0095    0.0104    0.0046    0.0010    0.0067
75%        0.0214    0.0316    0.0323    0.0178    0.0289
max        0.1677    0.1284    0.1510    0.1297    0.1331
```

17.6.5 Factor Correlations

We examine correlations between factors, which should generally be low for the factors to capture distinct sources of risk.

```
print("Factor Correlation Matrix:")
correlation_matrix = factors_ff5_monthly[["mkt_excess", "smb", "hml", "rmw",
 ↪  "cma"]].corr()
print(correlation_matrix.round(3))
```

```
Factor Correlation Matrix:
            mkt_excess    smb    hml    rmw    cma
mkt_excess       1.000 -0.712  0.230 -0.006 -0.104
smb             -0.712  1.000  0.256 -0.373  0.246
hml              0.230  0.256  1.000 -0.694  0.479
rmw             -0.006 -0.373 -0.694  1.000 -0.352
cma             -0.104  0.246  0.479 -0.352  1.000
```

17.6.6 Saving Five-Factor Data

```
factors_ff5_monthly.to_sql(
    name="factors_ff5_monthly",
    con=tidy_finance,
```

```
    if_exists="replace",
    index=False
)

print("Five-factor monthly data saved to database.")
```

```
Five-factor monthly data saved to database.
```

17.7 Daily Fama-French Factors

17.7.1 Motivation for Daily Factors

Daily factors are essential for several applications:

1. **Daily beta estimation**: CAPM regressions using daily data require daily market excess returns.
2. **Event studies**: Measuring abnormal returns around corporate events requires daily factor adjustments.
3. **High-frequency research**: Market microstructure studies need daily or intraday factor data.

The construction methodology mirrors the monthly approach, but we compute portfolio returns at daily frequency while maintaining the same annual portfolio formation dates.

17.7.2 Loading Daily Returns

```
# Load daily price data
prices_daily = pd.read_sql_query(
    sql="""
        SELECT symbol, date, ret_excess
        FROM prices_daily
    """,
    con=tidy_finance,
    parse_dates={"date"}
)

print(f"Daily returns: {len(prices_daily):,} observations")
print(f"Date range: {prices_daily['date'].min():%Y-%m-%d} to
 ↪  {prices_daily['date'].max():%Y-%m-%d}")
```

```
Daily returns: 3,462,157 observations
Date range: 2010-01-05 to 2023-12-29
```

17.7.3 Adding Market Cap for Daily Weighting

For value-weighted daily returns, we need market capitalization. We use the most recent monthly market cap as the weight for daily returns within that month.

```
# Get monthly market cap to use as weights for daily returns
mktcap_monthly = (prices_monthly
    [["symbol", "date", "mktcap_lag"]]
    .assign(year_month=lambda x: x["date"].dt.to_period("M"))
)

# Add year_month to daily data for merging
prices_daily = (prices_daily
    .assign(year_month=lambda x: x["date"].dt.to_period("M"))
    .merge(
        mktcap_monthly[["symbol", "year_month", "mktcap_lag"]],
        on=["symbol", "year_month"],
        how="left"
    )
    .dropna(subset=["ret_excess", "mktcap_lag"])
)

print(f"Daily returns with weights: {len(prices_daily):,} observations")
```

```
Daily returns with weights: 3,443,815 observations
```

17.7.4 Merging Daily Returns with Portfolios

We use the same portfolio assignments (formed annually in July) for daily returns.

```
# Step 1: Ensure portfolios_ff3 has correct format
portfolios_ff3_clean = portfolios_ff3[["symbol", "sorting_date",
 ↪  "portfolio_size", "portfolio_bm"]].copy()
portfolios_ff3_clean["sorting_date"] =
 ↪  pd.to_datetime(portfolios_ff3_clean["sorting_date"])

print("Portfolio sorting dates:")
```

```
print(portfolios_ff3_clean["sorting_date"].unique()[:5])

# Step 2: Create sorting_date for daily data
prices_daily_with_sort = prices_daily.copy()
prices_daily_with_sort["sorting_date"] =
 ↪  prices_daily_with_sort["date"].apply(
    lambda x: pd.Timestamp(f"{x.year}-07-01") if x.month > 6 else
     ↪  pd.Timestamp(f"{x.year - 1}-07-01")
)

print("\nDaily sorting dates:")
print(prices_daily_with_sort["sorting_date"].unique()[:5])

# Step 3: Merge
portfolios_daily_ff3 = prices_daily_with_sort.merge(
    portfolios_ff3_clean,
    on=["symbol", "sorting_date"],
    how="inner"
)

print(f"\nMerged daily observations: {len(portfolios_daily_ff3):,}")
print(f"Unique dates: {portfolios_daily_ff3['date'].nunique():,}")

# Step 4: Verify portfolio distribution
print("\nPortfolio distribution in daily data:")
print(portfolios_daily_ff3.groupby(["portfolio_size", "portfolio_bm"],
 ↪  observed=True).size().unstack(fill_value=0))
```

```
Portfolio sorting dates:
<DatetimeArray>
['2019-07-01 00:00:00', '2020-07-01 00:00:00', '2021-07-01 00:00:00',
 '2022-07-01 00:00:00', '2023-07-01 00:00:00']
Length: 5, dtype: datetime64[us]

Daily sorting dates:
<DatetimeArray>
['2018-07-01 00:00:00', '2019-07-01 00:00:00', '2020-07-01 00:00:00',
 '2021-07-01 00:00:00', '2022-07-01 00:00:00']
Length: 5, dtype: datetime64[us]

Merged daily observations: 2,843,570
Unique dates: 3,126
```

```
Portfolio distribution in daily data:
portfolio_bm         1       2       3
portfolio_size
1               218040  585561  617327
2               636152  552114  234376
```

```python
# Diagnostic: Check the daily portfolio merge
print("="*50)
print("DIAGNOSTIC: Daily Portfolio Merge")
print("="*50)

print(f"\nDaily prices rows: {len(prices_daily):,}")
print(f"Daily FF3 portfolios rows: {len(portfolios_daily_ff3):,}")
print(f"Match rate: {len(portfolios_daily_ff3)/len(prices_daily)*100:.1f}%")

# Check portfolio distribution in daily data
print("\nDaily portfolio distribution:")
print(portfolios_daily_ff3.groupby(["portfolio_size", "portfolio_bm"],
 ↪  observed=True).size().unstack(fill_value=0))

# Check a specific date
test_date = portfolios_daily_ff3["date"].iloc[1000]
print(f"\nSample date: {test_date}")
print(portfolios_daily_ff3.query("date ==
 ↪  @test_date").groupby(["portfolio_size", "portfolio_bm"],
 ↪  observed=True).size().unstack(fill_value=0))
```

```
==================================================
DIAGNOSTIC: Daily Portfolio Merge
==================================================

Daily prices rows: 3,443,815
Daily FF3 portfolios rows: 2,843,570
Match rate: 82.6%

Daily portfolio distribution:
portfolio_bm         1       2       3
portfolio_size
1               218040  585561  617327
2               636152  552114  234376
```

```
Sample date: 2023-06-28 00:00:00
portfolio_bm      1    2    3
portfolio_size
1                93  232  312
2               291  280   67
```

17.7.5 Computing Daily Three Factors

```python
def compute_daily_ff3_factors(portfolios_daily):
    """
    Compute daily Fama-French three factors.

    Parameters
    ----------
    portfolios_daily : pd.DataFrame
        Daily returns with portfolio assignments

    Returns
    -------
    pd.DataFrame
        Daily SMB and HML factors
    """
    # Compute daily portfolio returns
    portfolio_returns = (portfolios_daily
        .groupby(["portfolio_size", "portfolio_bm", "date"], observed=True)
        .apply(lambda x: pd.Series({
            "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
        }))
        .reset_index()
    )

    # Compute factors
    factors = (portfolio_returns
        .groupby("date")
        .apply(lambda x: pd.Series({
            "smb": (x.loc[x["portfolio_size"] == 1, "ret"].mean() -
                    x.loc[x["portfolio_size"] == 2, "ret"].mean()),
            "hml": (x.loc[x["portfolio_bm"] == 3, "ret"].mean() -
                    x.loc[x["portfolio_bm"] == 1, "ret"].mean())
        }))
        .reset_index()
```

```
    )

    return factors

factors_daily_smb_hml = compute_daily_ff3_factors(portfolios_daily_ff3)

print(f"Daily factor observations: {len(factors_daily_smb_hml):,}")
print(factors_daily_smb_hml.head(10))
```

```
Daily factor observations: 3,126
        date       smb       hml
0 2011-07-01  0.008587  0.000967
1 2011-07-04  0.005099 -0.001099
2 2011-07-05 -0.009088  0.010152
3 2011-07-06  0.004875 -0.003918
4 2011-07-07 -0.011239 -0.000584
5 2011-07-08  0.005636 -0.008003
6 2011-07-11  0.003940  0.006172
7 2011-07-12  0.003205  0.006543
8 2011-07-13 -0.000097 -0.001134
9 2011-07-14 -0.001248  0.001669
```

17.7.6 Computing Daily Market Factor

```
def compute_daily_market_factor(prices_daily):
    """
    Compute daily value-weighted market excess return.

    Parameters
    ----------
    prices_daily : pd.DataFrame
        Daily returns with mktcap_lag

    Returns
    -------
    pd.DataFrame
        Daily market excess return
    """
    market_daily = (prices_daily
        .groupby("date")
```

```
        .apply(lambda x: pd.Series({
            "mkt_excess": np.average(x["ret_excess"],
             ↪  weights=x["mktcap_lag"])
        }))
        .reset_index()
    )

    return market_daily

market_factor_daily = compute_daily_market_factor(prices_daily)

print(f"Daily market factor: {len(market_factor_daily):,} days")
```

```
Daily market factor: 3,474 days
```

17.7.7 Combining Daily Three Factors

```
factors_ff3_daily = (factors_daily_smb_hml
    .merge(market_factor_daily, on="date", how="inner")
)

# Add risk-free rate (use monthly rate / 21 as daily approximation, or load
 ↪  actual daily rate)
factors_ff3_daily["risk_free"] = 0.04 / 252  # Approximate daily risk-free

print("Daily Fama-French Three Factors:")
print(factors_ff3_daily.head(10))

print("\nDaily Factor Summary Statistics:")
print(factors_ff3_daily[["mkt_excess", "smb", "hml"]].describe().round(6))
```

```
Daily Fama-French Three Factors:
        date       smb       hml  mkt_excess  risk_free
0 2011-07-01  0.008587  0.000967   -0.019862   0.000159
1 2011-07-04  0.005099 -0.001099   -0.000633   0.000159
2 2011-07-05 -0.009088  0.010152    0.013314   0.000159
3 2011-07-06  0.004875 -0.003918   -0.008045   0.000159
4 2011-07-07 -0.011239 -0.000584    0.003391   0.000159
5 2011-07-08  0.005636 -0.008003    0.000218   0.000159
```

```
6 2011-07-11  0.003940  0.006172   -0.013393   0.000159
7 2011-07-12  0.003205  0.006543   -0.017505   0.000159
8 2011-07-13 -0.000097 -0.001134    0.000767   0.000159
9 2011-07-14 -0.001248  0.001669   -0.000695   0.000159

Daily Factor Summary Statistics:
        mkt_excess           smb           hml
count  3126.000000  3126.000000  3126.000000
mean     -0.000479     0.000236     0.000594
std       0.011269     0.008488     0.008585
min      -0.070268    -0.032671    -0.039418
25%      -0.005074    -0.004882    -0.003941
50%       0.000350    -0.000106     0.000522
75%       0.005531     0.004307     0.005233
max       0.043386     0.042686     0.083889
```

17.7.8 Computing Daily Five Factors

```
# Step 1: Clean portfolios
portfolios_ff5_clean = portfolios_ff5[["symbol", "sorting_date",
 ↪  "portfolio_size",
                                       "portfolio_bm", "portfolio_op",
                                        ↪  "portfolio_inv"]].copy()
portfolios_ff5_clean["sorting_date"] =
 ↪  pd.to_datetime(portfolios_ff5_clean["sorting_date"])

# Step 2: Merge with daily prices
portfolios_daily_ff5 = prices_daily_with_sort.merge(
    portfolios_ff5_clean,
    on=["symbol", "sorting_date"],
    how="inner"
)

print(f"FF5 Daily merged observations: {len(portfolios_daily_ff5):,}")
```

```
FF5 Daily merged observations: 2,843,570
```

```
def compute_daily_ff5_factors(portfolios_daily):
    """Compute daily Fama-French five factors."""
```

```
    # HML from B/M sorts
    portfolios_bm = (portfolios_daily
        .groupby(["portfolio_size", "portfolio_bm", "date"], observed=True)
        .apply(lambda x: pd.Series({
            "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
        }))
        .reset_index()
    )

    factors_hml = (portfolios_bm
        .groupby("date")
        .apply(lambda x: pd.Series({
            "hml": (x.loc[x["portfolio_bm"] == 3, "ret"].mean() -
                    x.loc[x["portfolio_bm"] == 1, "ret"].mean())
        }))
        .reset_index()
    )

    # RMW from OP sorts
    portfolios_op = (portfolios_daily
        .groupby(["portfolio_size", "portfolio_op", "date"], observed=True)
        .apply(lambda x: pd.Series({
            "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
        }))
        .reset_index()
    )

    factors_rmw = (portfolios_op
        .groupby("date")
        .apply(lambda x: pd.Series({
            "rmw": (x.loc[x["portfolio_op"] == 3, "ret"].mean() -
                    x.loc[x["portfolio_op"] == 1, "ret"].mean())
        }))
        .reset_index()
    )

    # CMA from INV sorts (note: low minus high)
    portfolios_inv = (portfolios_daily
        .groupby(["portfolio_size", "portfolio_inv", "date"], observed=True)
        .apply(lambda x: pd.Series({
            "ret": np.average(x["ret_excess"], weights=x["mktcap_lag"])
        }))
        .reset_index()
```

```
    )

    factors_cma = (portfolios_inv
        .groupby("date")
        .apply(lambda x: pd.Series({
            "cma": (x.loc[x["portfolio_inv"] == 1, "ret"].mean() -
                    x.loc[x["portfolio_inv"] == 3, "ret"].mean())
        }))
        .reset_index()
    )

    # SMB from all sorts
    all_portfolios = pd.concat([portfolios_bm, portfolios_op,
↪   portfolios_inv])

    factors_smb = (all_portfolios
        .groupby("date")
        .apply(lambda x: pd.Series({
            "smb": (x.loc[x["portfolio_size"] == 1, "ret"].mean() -
                    x.loc[x["portfolio_size"] == 2, "ret"].mean())
        }))
        .reset_index()
    )

    # Combine
    factors = (factors_smb
        .merge(factors_hml, on="date", how="outer")
        .merge(factors_rmw, on="date", how="outer")
        .merge(factors_cma, on="date", how="outer")
    )

    return factors

# Compute daily FF5 factors
factors_daily_ff5 = compute_daily_ff5_factors(portfolios_daily_ff5)

# Add market factor
factors_ff5_daily = (factors_daily_ff5
    .merge(market_factor_daily, on="date", how="inner")
)
factors_ff5_daily["risk_free"] = 0.04 / 252

print("Daily Fama-French Five Factors:")
```

```
print(factors_ff5_daily.head(10))

print("\nDaily Five-Factor Summary Statistics:")
print(factors_ff5_daily[["mkt_excess", "smb", "hml", "rmw",
 ↪  "cma"]].describe().round(6))
```

```
Daily Fama-French Five Factors:
        date       smb       hml       rmw       cma  mkt_excess  risk_free
0 2011-07-01  0.006295  0.002515  0.013140  0.007680   -0.019862   0.000159
1 2011-07-04  0.002880 -0.002875  0.006560 -0.004886   -0.000633   0.000159
2 2011-07-05 -0.004260  0.009864 -0.012158 -0.004470    0.013314   0.000159
3 2011-07-06  0.001544 -0.009847  0.012977  0.006286   -0.008045   0.000159
4 2011-07-07 -0.009789 -0.003988 -0.000197 -0.006995    0.003391   0.000159
5 2011-07-08  0.001537 -0.006700  0.010841 -0.007661    0.000218   0.000159
6 2011-07-11  0.005396  0.004747  0.000655  0.013375   -0.013393   0.000159
7 2011-07-12  0.004759  0.007367  0.001989  0.014669   -0.017505   0.000159
8 2011-07-13 -0.000009  0.001110 -0.002052 -0.001633    0.000767   0.000159
9 2011-07-14 -0.001668  0.002916 -0.005427  0.005388   -0.000695   0.000159

Daily Five-Factor Summary Statistics:
        mkt_excess           smb           hml           rmw           cma
count  3126.000000  3126.000000  3126.000000  3126.000000  3126.000000
mean     -0.000479     0.000484     0.000549    -0.000136     0.000413
std       0.011269     0.008033     0.008312     0.008538     0.006756
min      -0.070268    -0.036283    -0.039155    -0.154013    -0.047698
25%      -0.005074    -0.004122    -0.003681    -0.004212    -0.003364
50%       0.000350     0.000105     0.000384     0.000030     0.000162
75%       0.005531     0.004358     0.004819     0.004036     0.003800
max       0.043386     0.060307     0.086269     0.102001     0.089907
```

17.7.9 Saving Daily Factors

```
factors_ff3_daily.to_sql(
    name="factors_ff3_daily",
    con=tidy_finance,
    if_exists="replace",
    index=False
)
```

```
factors_ff5_daily.to_sql(
    name="factors_ff5_daily",
    con=tidy_finance,
    if_exists="replace",
    index=False
)

print("Daily factor data saved to database.")
```

```
Daily factor data saved to database.
```

17.8 Factor Validation and Diagnostics

```
# Verify all tables are in database
print("\n" + "="*50)
print("DATABASE SUMMARY")
print("="*50)

tables = ["factors_ff3_monthly", "factors_ff5_monthly",
          "factors_ff3_daily", "factors_ff5_daily"]

for table in tables:
    df = pd.read_sql_query(f"SELECT COUNT(*) as n FROM {table}",
 ↪  con=tidy_finance)
    print(f"{table}: {df['n'].iloc[0]:,} observations")

# Correlation check: Monthly vs Daily (aggregated)
print("\n" + "="*50)
print("MONTHLY VS DAILY CONSISTENCY CHECK")
print("="*50)

factors_daily_agg = (factors_ff3_daily
    .assign(year_month=lambda x: x["date"].dt.to_period("M"))
    .groupby("year_month")[["mkt_excess", "smb", "hml"]]
    .sum()
    .reset_index()
)

factors_monthly_check = (factors_ff3_monthly
```

```
    .assign(year_month=lambda x: x["date"].dt.to_period("M"))
)

comparison = factors_monthly_check.merge(
    factors_daily_agg, on="year_month", suffixes=("_monthly", "_daily")
)

for factor in ["mkt_excess", "smb", "hml"]:
    corr =
 ↪  comparison[f"{factor}_monthly"].corr(comparison[f"{factor}_daily"])
    print(f"{factor}: Monthly-Daily correlation = {corr:.4f}")
```

```
==================================================
DATABASE SUMMARY
==================================================
factors_ff3_monthly: 150 observations
factors_ff5_monthly: 150 observations
factors_ff3_daily: 3,126 observations
factors_ff5_daily: 3,126 observations

==================================================
MONTHLY VS DAILY CONSISTENCY CHECK
==================================================
mkt_excess: Monthly-Daily correlation = 0.9980
smb: Monthly-Daily correlation = 0.9953
hml: Monthly-Daily correlation = 0.9936
```

17.8.1 Cumulative Factor Returns

We visualize the cumulative performance of each factor to assess whether the factors generate meaningful premiums over time.

```
# Compute cumulative returns
factors_cumulative = (factors_ff5_monthly
    .set_index("date")
    [["mkt_excess", "smb", "hml", "rmw", "cma"]]
    .add(1)
    .cumprod()
)
```

```
# Plot
fig, ax = plt.subplots(figsize=(12, 6))
factors_cumulative.plot(ax=ax)
ax.set_title("Cumulative Factor Returns (Vietnam)")
ax.set_xlabel("")
ax.set_ylabel("Growth of $1")
ax.legend(title="Factor")
ax.axhline(y=1, color='gray', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

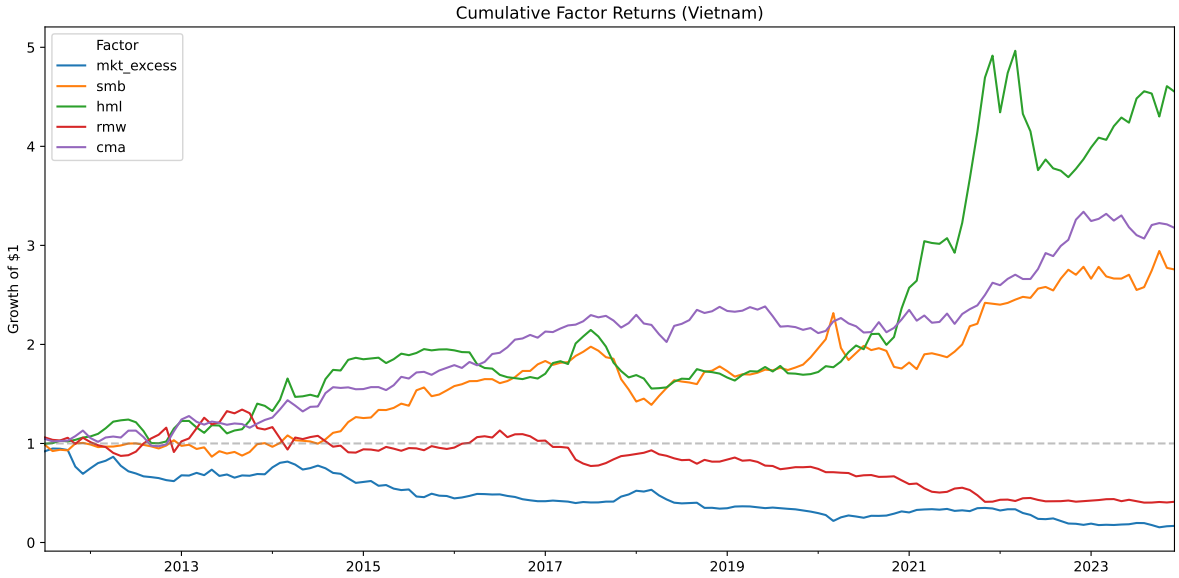


Figure 17.1: Cumulative returns of Fama-French factors for the Vietnamese market. The figure shows the growth of $1 invested in each factor portfolio.

17.8.2 Average Factor Premiums

We compute annualized average factor premiums and their statistical significance.

```
# Annualized average returns (monthly returns * 12)
factor_premiums = (factors_ff5_monthly
    [["mkt_excess", "smb", "hml", "rmw", "cma"]]
    .mean() * 12 * 100  # Annualized percentage
)
```

```
# Standard errors
factor_se = (factors_ff5_monthly
    [["mkt_excess", "smb", "hml", "rmw", "cma"]]
    .std() / np.sqrt(len(factors_ff5_monthly)) * np.sqrt(12) * 100
)

# T-statistics
factor_tstat = factor_premiums / factor_se

print("Annualized Factor Premiums (%):")
print(factor_premiums.round(2))

print("\nT-Statistics:")
print(factor_tstat.round(2))
```

```
Annualized Factor Premiums (%):
mkt_excess   -12.09
smb            9.19
hml           13.75
rmw           -5.69
cma            9.94
dtype: float64

T-Statistics:
mkt_excess    -7.30
smb            7.76
hml            9.38
rmw           -4.22
cma           10.49
dtype: float64
```

17.8.3 Comparing Monthly and Daily Factors

We verify consistency between monthly and daily factors by computing correlations.

```
# Aggregate daily factors to monthly for comparison
factors_daily_monthly = (factors_ff5_daily
    .assign(year_month=lambda x: x["date"].dt.to_period("M"))
    .groupby("year_month")
    [["mkt_excess", "smb", "hml", "rmw", "cma"]]
```

```
    .sum()  # Sum daily returns to get monthly
    .reset_index()
)

# Merge with actual monthly factors
comparison = (factors_ff5_monthly
    .assign(year_month=lambda x: x["date"].dt.to_period("M"))
    .merge(
        factors_daily_monthly,
        on="year_month",
        suffixes=("_monthly", "_daily")
    )
)

# Correlations
for factor in ["mkt_excess", "smb", "hml", "rmw", "cma"]:
    corr =
 ↪  comparison[f"{factor}_monthly"].corr(comparison[f"{factor}_daily"])
    print(f"{factor}: Monthly-Daily correlation = {corr:.4f}")
```

```
mkt_excess: Monthly-Daily correlation = 0.9980
smb: Monthly-Daily correlation = 0.9950
hml: Monthly-Daily correlation = 0.9948
rmw: Monthly-Daily correlation = 0.9929
cma: Monthly-Daily correlation = 0.9884
```

17.9 Key Takeaways

1. **Factor Models Explained**: The Fama-French three-factor model adds size (SMB) and value (HML) factors to the CAPM, while the five-factor model further includes profitability (RMW) and investment (CMA) factors.
2. **Construction Methodology**: Factors are constructed through double-sorted portfolios with careful attention to timing. Portfolios are formed in July using accounting data from the prior fiscal year to ensure information was publicly available.
3. **Independent vs. Dependent Sorts**: The three-factor model uses independent sorts on size and book-to-market, creating a 2×3 grid. The five-factor model uses dependent sorts where characteristics are sorted within size groups.
4. **Value-Weighted Returns**: Portfolio returns are computed using value-weighting with lagged market capitalization to avoid look-ahead bias.

5. **Daily Factors**: Daily factors use the same annual portfolio assignments but compute returns at daily frequency, enabling higher-frequency applications like daily beta estimation.

6. **Market Factor**: The market factor is computed independently as the value-weighted return of all stocks minus the risk-free rate.

7. **Validation**: Factor quality can be assessed through characteristic monotonicity, portfolio diversification, cumulative returns, and consistency between daily and monthly frequencies.

8. **Vietnamese Market Adaptation**: While following the original Fama-French methodology, we adapt for Vietnamese market characteristics including VAS accounting standards, reporting timelines, and currency units.

18 Momentum Strategies

Momentum is one of the most robust and pervasive anomalies in financial economics. Stocks that have performed well over the past three to twelve months tend to continue performing well over the subsequent three to twelve months, and stocks that have performed poorly tend to continue underperforming. This pattern, first documented by Jegadeesh and Titman (1993), has been replicated across virtually every equity market, asset class, and time period examined, earning it a central place in the canon of empirical asset pricing.

In this chapter, we provide an implementation of momentum strategies following the methodology of Jegadeesh and Titman (1993). We construct momentum portfolios based on past cumulative returns, evaluate their performance across different formation and holding period combinations, and examine whether the momentum premium exists in the Vietnamese equity market..

18.1 Theoretical Background

18.1.1 The Momentum Effect

The momentum effect refers to the tendency of assets with high recent returns to continue generating high returns, and assets with low recent returns to continue generating low returns. More formally, if we define the cumulative return of stock i over the past J months as

$$R_{i,t-J:t-1} = \prod_{s=t-J}^{t-1} (1 + r_{i,s}) - 1 \tag{18.1}$$

then momentum predicts a positive cross-sectional relationship between $R_{i,t-J:t-1}$ and future returns $r_{i,t}$. That is, stocks in the top decile of past returns (winners) should outperform stocks in the bottom decile (losers) in subsequent months.

The Jegadeesh and Titman (1993) framework parameterizes momentum strategies by two key dimensions:

1. **Formation period** (J): The number of months over which past returns are computed to rank stocks. Typical values range from 3 to 12 months.

2. **Holding period** (K): The number of months for which the momentum portfolios are held after formation. Typical values also range from 3 to 12 months.

A strategy is therefore characterized by the pair (J, K). For example, a $(6, 6)$ strategy ranks stocks based on their cumulative returns over the past 6 months and holds the resulting portfolios for 6 months.

18.1.2 Overlapping Portfolios and Calendar-Time Returns

A critical implementation detail in Jegadeesh and Titman (1993) is the use of overlapping portfolios. In each month t, a new set of portfolios is formed based on the most recent J-month returns. However, portfolios formed in months $t-1$, $t-2$, ..., $t-K+1$ are still within their holding periods and therefore remain active. The return of the momentum strategy in month t is thus the equally weighted average across all K active cohorts:

$$r_{p,t} = \frac{1}{K} \sum_{k=0}^{K-1} r_{p,t}^{(t-k)} \tag{18.2}$$

where $r_{p,t}^{(t-k)}$ denotes the return in month t of the portfolio formed in month $t-k$. This overlapping portfolio approach serves two purposes. First, it reduces the impact of any single formation date on the strategy's performance. Second, it produces a monthly return series that can be analyzed using standard time-series methods, even when the holding period K exceeds one month.

18.1.3 Theoretical Explanations

The academic literature offers two broad classes of explanations for the momentum effect.

1. **Behavioral explanations** attribute momentum to systematic cognitive biases among investors. Daniel, Hirshleifer, and Subrahmanyam (1998) propose a model based on investor overconfidence and biased self-attribution: investors overweight private signals and attribute confirming outcomes to their own skill, leading to initial underreaction followed by delayed overreaction. Hong and Stein (1999) develop a model in which information diffuses gradually across heterogeneous investors, generating underreaction to firm-specific news. Barberis, Shleifer, and Vishny (1998) formalize a model combining conservatism bias (slow updating of beliefs in response to new evidence) and representativeness heuristic (extrapolation of recent trends).

2. **Risk-based explanations** argue that momentum profits represent compensation for systematic risk that varies over time. T. C. Johnson (2002) show that momentum can arise in a rational framework if expected returns are stochastic and time-varying. Grundy and Martin (2001) document that momentum portfolios have substantial time-varying factor exposures, suggesting that at least part of the momentum premium reflects dynamic risk. However, standard risk models such as the Fama-French three-factor model have generally struggled to explain momentum returns, which motivated the development of explicit momentum factors such as the Winners-Minus-Losers (WML) factor in Mark M. Carhart (1997a).

18.1.4 Momentum in Emerging Markets

The behavior of momentum in emerging markets differs substantially from developed markets, and understanding these differences is essential for interpreting our Vietnamese market results.

Rouwenhorst (1998) provides early evidence that momentum profits exist in European markets. Chan, Hameed, and Tong (2000) extend the analysis to international equity markets and find that momentum profits are present in most developed markets but are weaker or absent in several Asian markets. Chui, Titman, and Wei (2010) offer a cultural explanation, documenting that momentum profits are positively related to a country's degree of individualism as measured by Hofstede (2001). Countries with collectivist cultures, including many in East and Southeast Asia, tend to exhibit weaker momentum effects.

Several market microstructure features common in emerging markets may attenuate momentum:

- **Trading band limits** constrain daily price movements, potentially slowing the adjustment process that generates momentum. Vietnam's HOSE imposes a $\pm 7\%$ daily limit, while HNX allows $\pm 10\%$.
- **Lower liquidity and higher transaction costs** can erode momentum profits, as documented by Lesmond, Schill, and Zhou (2004).
- **Foreign ownership limits** segment the investor base, potentially altering the information diffusion dynamics that underlie momentum.
- **Shorter market history** limits the statistical power available to detect the effect.

These considerations motivate our careful empirical analysis of whether, and to what extent, momentum manifests in Vietnamese equities.

18.2 Setting Up the Environment

We begin by loading the necessary Python packages. The core packages include `pandas` for data manipulation, `numpy` for numerical operations, and `sqlite3` for database connectivity. We also import `plotnine` for creating publication-quality figures and `scipy` for statistical tests.

```
import pandas as pd
import numpy as np
import sqlite3
from datetime import datetime
from itertools import product

from plotnine import *
from mizani.formatters import comma_format, percent_format
from scipy import stats
```

We connect to our SQLite database, which stores the cleaned datasets prepared previous chapters.

```
tidy_finance = sqlite3.connect(
    database="data/tidy_finance_python.sqlite"
)
```

18.3 Data Preparation

18.3.1 Loading Monthly Stock Returns

We load the monthly stock price data from our database. The `prices_monthly` table contains adjusted returns, market capitalizations, and other variables for all stocks listed on HOSE and HNX.

```
prices_monthly = pd.read_sql_query(
    # can add "exchange" variable
    sql="""
        SELECT symbol, date, ret, ret_excess, mktcap, mktcap_lag, risk_free
        FROM prices_monthly
    """,
    con=tidy_finance,
    parse_dates={"date"}
```

```
).dropna()

print(f"Total monthly observations: {len(prices_monthly):,}")
print(f"Unique stocks: {prices_monthly['symbol'].nunique():,}")
print(f"Date range: {prices_monthly['date'].min().date()} "
      f"to {prices_monthly['date'].max().date()}")
```

```
Total monthly observations: 165,499
Unique stocks: 1,457
Date range: 2010-02-28 to 2023-12-31
```

18.3.2 Inspecting the Data

Before proceeding with portfolio construction, we examine the key variables in our dataset. Table 18.1 presents the summary statistics for the main variables used in momentum portfolio construction.

```
summary_stats = (prices_monthly
    [["ret", "ret_excess", "mktcap"]]
    .describe()
    .T
    .round(4)
)
summary_stats
```

Table 18.1: Summary statistics for monthly stock return data. The table reports the count, mean, standard deviation, minimum, 25th percentile, median, 75th percentile, and maximum for monthly returns (ret), excess returns (ret_excess), and market capitalization (mktcap, in billion VND).

	count	mean	std	min	25%	50%	75%	max
ret	165499.0	0.0042	0.1862	-0.9900	-0.0703	0.0000	0.0553	12.7500
ret_excess	165499.0	0.0008	0.1862	-0.9933	-0.0736	-0.0033	0.0519	12.7467
mktcap	165499.0	2183.1646	13983.9977	0.3536	60.3728	180.6224	660.0000	463886.6454

We also examine the cross-sectional distribution of stocks over time, which is important for understanding whether we have sufficient breadth for decile portfolio construction.

```
stock_counts = (prices_monthly
    .groupby("date")["symbol"]
    .nunique()
    .reset_index()
    .rename(columns={"symbol": "n_stocks"})
)

plot_stock_counts = (
    ggplot(stock_counts, aes(x="date", y="n_stocks")) +
    geom_line(color="#1f77b4") +
    labs(
        x="", y="Number of stocks",
        title="Cross-Sectional Breadth Over Time"
    ) +
    theme_minimal() +
    theme(figure_size=(10, 5))
)
plot_stock_counts
```

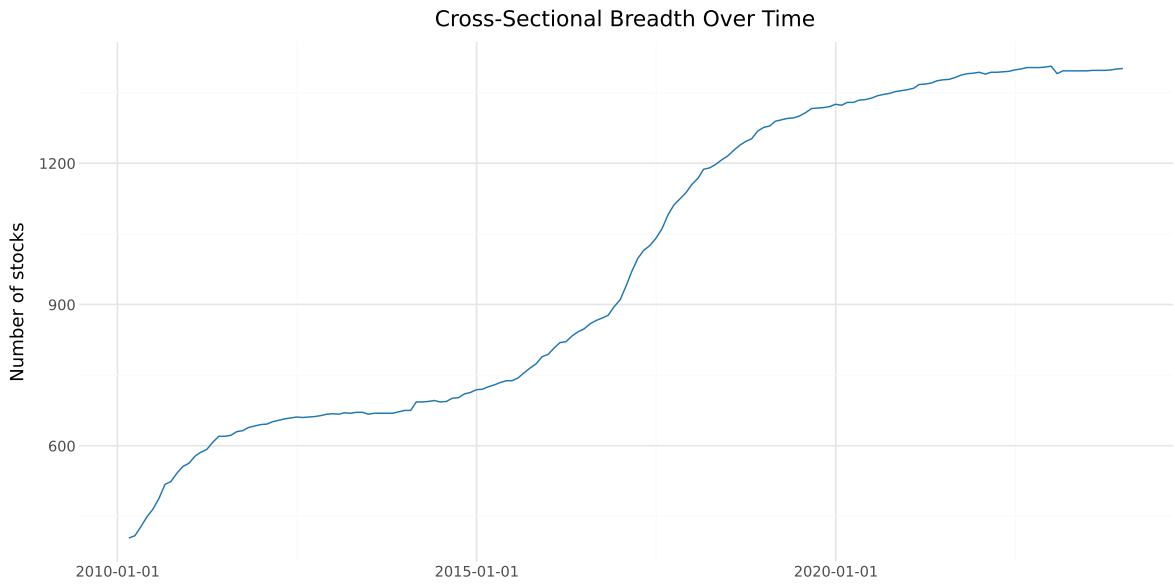


Figure 18.1: Number of stocks with available return data over time. The figure shows the monthly cross-section of stocks available for momentum portfolio construction. A minimum number of stocks is needed to form meaningful decile portfolios.

18.3.3 Loading Factor Data

We also load the Fama-French factor returns for risk-adjusted performance evaluation. These factors were constructed in previous chapters.

```
factors_ff3_monthly = pd.read_sql_query(
    sql="SELECT date, mkt_excess, smb, hml, risk_free FROM
 ↪  factors_ff3_monthly",
    con=tidy_finance,
    parse_dates={"date"}
)

print(f"Factor observations: {len(factors_ff3_monthly):,}")
print(f"Date range: {factors_ff3_monthly['date'].min().date()} "
      f"to {factors_ff3_monthly['date'].max().date()}")
```

```
Factor observations: 150
Date range: 2011-07-31 to 2023-12-31
```

18.3.4 Data Quality Filters

Momentum strategies require continuous return histories for the formation period. We apply several filters to ensure data quality:

1. **Minimum price filter**: We exclude penny stocks with prices below 1,000 VND, which are subject to extreme microstructure noise.
2. **Return availability**: We require non-missing returns for the entire formation period.
3. **Market capitalization**: We require positive lagged market capitalization for portfolio weighting.

```
# Filter for stocks with positive market cap and non-missing returns
prices_clean = (prices_monthly
    .dropna(subset=["ret", "mktcap"])
    .query("mktcap > 0")
    .sort_values(["symbol", "date"])
    .reset_index(drop=True)
)

print(f"Observations after filtering: {len(prices_clean):,}")
print(f"Stocks after filtering: {prices_clean['symbol'].nunique():,}")
```

```
Observations after filtering: 165,499
Stocks after filtering: 1,457
```

18.4 Momentum Portfolio Construction

18.4.1 Computing Formation Period Returns

The first step in constructing momentum portfolios is to compute cumulative returns over the formation period. For a formation period of J months, we compute the cumulative return for each stock i at the end of month t as specified in Equation 72.7.

We implement this using a rolling product of gross returns. A key detail is that we require non-missing returns for all J months in the formation window. If any monthly return is missing, the cumulative return is set to missing, and the stock is excluded from portfolio formation in that month.

```python
def compute_formation_returns(data, J):
    """
    Compute J-month cumulative returns for momentum portfolio formation.

    Parameters
    ----------
    data : pd.DataFrame
        Panel data with columns 'symbol', 'date', 'ret'.
    J : int
        Formation period length in months (typically 3-12).

    Returns
    -------
    pd.DataFrame
        Original data augmented with 'cum_return' column.
    """
    df = data.sort_values(["symbol", "date"]).copy()

    # Compute rolling J-month cumulative return
    # Using gross returns: (1+r1)*(1+r2)*...*(1+rJ) - 1
    df["gross_ret"] = 1 + df["ret"]

    df["cum_return"] = (
        df.groupby("symbol")["gross_ret"]
        .rolling(window=J, min_periods=J)
        .apply(np.prod, raw=True)
```

```
        .reset_index(level=0, drop=True)
        - 1
    )

    df = df.drop(columns=["gross_ret"])

    return df
```

Let us apply this function for our baseline case with $J = 6$ months:

```
J = 6  # Formation period: 6 months

prices_with_cumret = compute_formation_returns(prices_clean, J)

# Check the result
print(f"Observations with valid {J}-month cumulative returns: "
      f"{prices_with_cumret['cum_return'].notna().sum():,}")
print(f"\nCumulative return distribution:")
print(prices_with_cumret["cum_return"].describe().round(4))
```

```
Observations with valid 6-month cumulative returns: 158,227

Cumulative return distribution:
count    158227.0000
mean          0.0171
std           0.5053
min          -0.9999
25%          -0.2196
50%          -0.0400
75%           0.1404
max          35.7136
Name: cum_return, dtype: float64
```

18.4.2 Assigning Momentum Decile Portfolios

Each month, we sort all stocks with valid cumulative returns into decile portfolios based on their past J-month performance. Portfolio 1 contains the bottom decile (losers) and portfolio 10 contains the top decile (winners).

```
def assign_momentum_portfolios(data, n_portfolios=10):
    """
    Assign stocks to momentum portfolios based on formation-period returns.

    For each cross-section (month), stocks are sorted into
    n_portfolios quantile groups according to their cumulative return.

    Portfolio 1 = lowest past returns (Losers)
    Portfolio n_portfolios = highest past returns (Winners)

    This implementation uses `groupby().transform()` rather than
    `groupby().apply()` to preserve the original DataFrame structure
    and avoid index mutation issues.

    Parameters
    ----------
    data : pd.DataFrame
        Panel data containing at minimum:
        - 'symbol' : stock identifier
        - 'date' : formation month
        - 'cum_return' : cumulative return over formation window

    n_portfolios : int, optional
        Number of portfolios to form (default = 10 for deciles).

    Returns
    -------
    pd.DataFrame
        Original data augmented with:
        - 'momr' : integer momentum rank (1 to n_portfolios)
    """

    # Drop observations without formation-period returns
    # These cannot be ranked into portfolios
    df = data.dropna(subset=["cum_return"]).copy()

    def safe_qcut(x):
        """
        Assign quantile portfolio labels within a single cross-section.

        Uses pd.qcut to create approximately equal-sized portfolios.
        If too few unique return values exist (which can happen in
        small markets or illiquid samples), fall back to rank-based
```

```
        binning to ensure portfolios are still formed.
        """
        try:
            # Standard quantile sorting
            return pd.qcut(
                x,
                q=n_portfolios,
                labels=range(1, n_portfolios + 1),
                duplicates="drop"  # prevents crash if quantile edges
↪   duplicate
            )
        except ValueError:
            # Fallback: rank then evenly cut into bins
            return pd.cut(
                x.rank(method="first"),
                bins=n_portfolios,
                labels=range(1, n_portfolios + 1)
            )

    # Cross-sectional portfolio assignment:
    # For each month, compute portfolio ranks based on cum_return.
    # transform ensures:
    # - original row order preserved
    # - no index mutation
    # - no loss of 'date' column
    df["momr"] = (
        df.groupby("date")["cum_return"]
          .transform(safe_qcut)
          .astype(int)
    )

    return df
```

```
portfolios = assign_momentum_portfolios(prices_with_cumret)

print(f"Stocks assigned to portfolios: {len(portfolios):,}")
print(f"\nPortfolio distribution (should be approximately equal):")
print(portfolios.groupby("momr")["symbol"].count().to_frame("count"))
```

```
Stocks assigned to portfolios: 158,227

Portfolio distribution (should be approximately equal):
```

```
      count
momr
1     15894
2     15815
3     16021
4     15759
5     15738
6     16568
7     15288
8     15582
9     15697
10    15865
```

18.4.3 Defining Holding Period Dates

After forming portfolios at the end of month t, we hold them from the beginning of month $t+1$ through the end of month $t+K$. This one-month gap between the formation period and the start of the holding period is standard in the literature and avoids the well-documented short-term reversal effect at the one-month horizon (Jegadeesh 1990).

```
K = 6  # Holding period: 6 months

portfolios_with_dates = portfolios.copy()
portfolios_with_dates = portfolios_with_dates.rename(
    columns={"date": "form_date"}
)

# Define holding period start and end dates
portfolios_with_dates["hdate1"] = (
    portfolios_with_dates["form_date"] + pd.offsets.MonthBegin(1)
)
portfolios_with_dates["hdate2"] = (
    portfolios_with_dates["form_date"] + pd.offsets.MonthEnd(K)
)

print(portfolios_with_dates[
    ["symbol", "form_date", "cum_return", "momr", "hdate1", "hdate2"]
].head(10))
```

```
   symbol  form_date  cum_return  momr     hdate1     hdate2
5     A32 2019-04-30   -0.061599     4 2019-05-01 2019-10-31
```

```
6     A32 2019-05-31   -0.213675     2 2019-06-01 2019-11-30
7     A32 2019-06-30   -0.308720     2 2019-07-01 2019-12-31
8     A32 2019-07-31   -0.249936     2 2019-08-01 2020-01-31
9     A32 2019-08-31    0.061986     8 2019-09-01 2020-02-29
10    A32 2019-09-30    0.030751     8 2019-10-01 2020-03-31
11    A32 2019-10-31    0.030751     8 2019-11-01 2020-04-30
12    A32 2019-11-30    0.032486     8 2019-12-01 2020-05-31
13    A32 2019-12-31    0.180073     9 2020-01-01 2020-06-30
14    A32 2020-01-31    0.412322    10 2020-02-01 2020-07-31
```

18.4.4 Computing Portfolio Holding Period Returns

We now compute the returns of each portfolio during its holding period. For each stock-formation date combination, we merge in the actual returns realized during the holding window. This produces a panel of stock returns indexed by formation date, holding date, and momentum portfolio rank.

```
# Merge: for each portfolio assignment, get all returns during holding period
portfolio_returns = portfolios_with_dates.merge(
    prices_clean[["symbol", "date", "ret"]].rename(
        columns={"ret": "hret", "date": "hdate"}
    ),
    on="symbol",
    how="inner"
)

# Keep only returns within the holding period
portfolio_returns = portfolio_returns.query(
    "hdate >= hdate1 and hdate <= hdate2"
)

print(f"Portfolio-holding period observations: {len(portfolio_returns):,}")
```

```
Portfolio-holding period observations: 918,072
```

18.4.5 Computing Equally Weighted Portfolio Returns

The Jegadeesh and Titman (1993) methodology uses equally weighted portfolios. For each holding month, each momentum decile has K active cohorts (one formed in each of the past K months). We first compute the equally weighted return within each cohort, then average across cohorts to get the final monthly portfolio return.

```
# Step 1: Average return within each cohort (form_date × momr × hdate)
cohort_returns = (portfolio_returns
    .groupby(["hdate", "momr", "form_date"])
    .agg(cohort_ret=("hret", "mean"))
    .reset_index()
)

# Step 2: Average across cohorts for each momentum portfolio × month
ewret = (cohort_returns
    .groupby(["hdate", "momr"])
    .agg(
        ewret=("cohort_ret", "mean"),
        ewret_std=("cohort_ret", "std"),
        n_cohorts=("cohort_ret", "count")
    )
    .reset_index()
    .rename(columns={"hdate": "date"})
)

print(f"Monthly portfolio return observations: {len(ewret):,}")
print(f"Date range: {ewret['date'].min().date()} to
 ↪  {ewret['date'].max().date()}")
```

```
Monthly portfolio return observations: 1,610
Date range: 2010-08-31 to 2023-12-31
```

We should verify that we have the expected number of active cohorts per month. Once the strategy has been running for at least K months, each momentum decile should have exactly K active cohorts.

```
cohort_check = (ewret
    .groupby("date")["n_cohorts"]
    .mean()
    .reset_index()
    .rename(columns={"n_cohorts": "avg_cohorts"})
)

plot_cohorts = (
    ggplot(cohort_check, aes(x="date", y="avg_cohorts")) +
    geom_line(color="#1f77b4") +
    geom_hline(yintercept=K, linetype="dashed", color="red") +
```

```
    labs(
        x="", y="Average number of cohorts",
        title=f"Active Cohorts per Portfolio (K={K})"
    ) +
    theme_minimal() +
    theme(figure_size=(10, 4))
)
plot_cohorts
```

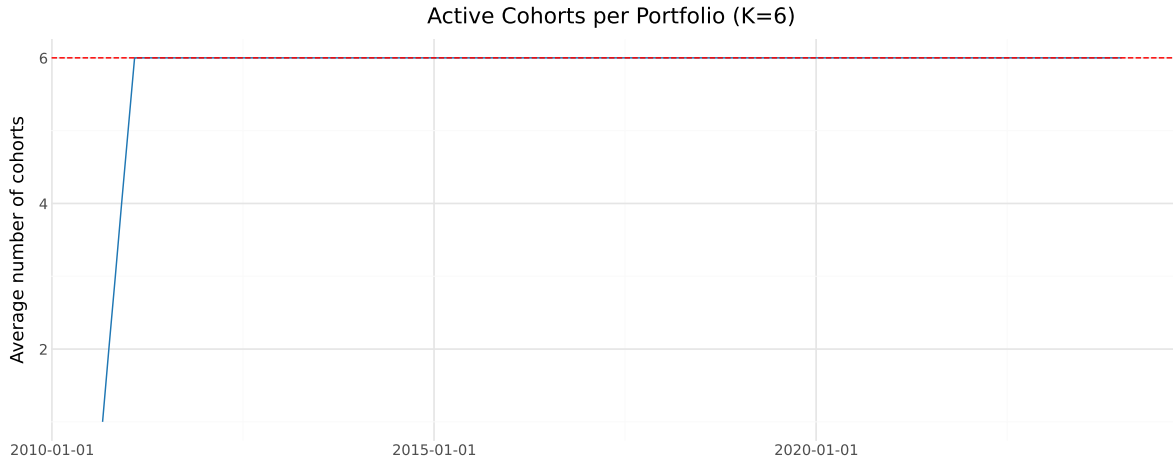


Figure 18.2: Number of active cohorts per momentum portfolio over time. After an initial ramp-up period of K months, each portfolio should have exactly K active cohorts contributing to its monthly return.

18.5 Baseline Results: J=6, K=6 Strategy

18.5.1 Summary Statistics by Momentum Decile

We now examine the average monthly returns for each momentum decile portfolio. Table 18.2 presents the mean, t-statistic, and p-value for each of the ten portfolios.

```
def compute_portfolio_stats(group):
    """Compute mean, t-stat, and p-value for a return series."""
    n = len(group)
    mean_ret = group.mean()
    std_ret = group.std()
    t_stat = mean_ret / (std_ret / np.sqrt(n)) if std_ret > 0 else np.nan
```

```
    p_val = 2 * (1 - stats.t.cdf(abs(t_stat), df=n-1)) if not
↪   np.isnan(t_stat) else np.nan
    return pd.Series({
        "N": n,
        "Mean (%)": mean_ret * 100,
        "Std (%)": std_ret * 100,
        "t-stat": t_stat,
        "p-value": p_val
    })

momentum_stats = (ewret
    .groupby("momr")["ewret"]
    .apply(compute_portfolio_stats)
    .unstack()
    .round(4)
)

momentum_stats
```

Table 18.2: Average monthly returns of momentum decile portfolios for the J=6, K=6 strategy. Portfolio 1 contains past losers and portfolio 10 contains past winners. The table reports the number of months, mean monthly return, t-statistic, and p-value for each decile.

momr	N	Mean (%)	Std (%)	t-stat	p-value
1	161.0	1.4190	6.5319	2.7565	0.0065
2	161.0	0.6602	5.9895	1.3985	0.1639
3	161.0	0.3658	5.5080	0.8427	0.4007
4	161.0	0.1623	5.0085	0.4112	0.6815
5	161.0	0.0111	4.7561	0.0297	0.9763
6	161.0	0.0408	4.6864	0.1104	0.9122
7	161.0	-0.0674	4.8881	-0.1749	0.8614
8	161.0	-0.2066	4.9208	-0.5329	0.5949
9	161.0	-0.2228	5.3013	-0.5332	0.5946
10	161.0	-0.6812	5.7131	-1.5130	0.1323

18.5.2 The Long-Short Momentum Portfolio

The key test of the momentum effect is whether the spread between winners and losers—the long-short momentum portfolio—generates statistically significant positive returns. We

construct this spread portfolio by going long the top decile (portfolio 10) and short the bottom decile (portfolio 1).

```
# Pivot to wide format
ewret_wide = ewret.pivot(
    index="date", columns="momr", values="ewret"
).reset_index()

ewret_wide.columns = ["date"] + [f"port{i}" for i in range(1, 11)]

# Compute long-short return
ewret_wide["winners"] = ewret_wide["port10"]
ewret_wide["losers"] = ewret_wide["port1"]
ewret_wide["long_short"] = ewret_wide["winners"] - ewret_wide["losers"]
```

```
ls_stats = pd.DataFrame()
for col in ["winners", "losers", "long_short"]:
    series = ewret_wide[col].dropna()
    n = len(series)
    mean_val = series.mean()
    std_val = series.std()
    t_val = mean_val / (std_val / np.sqrt(n))
    p_val = 2 * (1 - stats.t.cdf(abs(t_val), df=n-1))
    ls_stats[col] = [n, mean_val * 100, std_val * 100, t_val, p_val]

ls_stats.index = ["N", "Mean (%)", "Std (%)", "t-stat", "p-value"]
ls_stats.columns = ["Winners", "Losers", "Long-Short"]
ls_stats = ls_stats.round(4)
ls_stats
```

Table 18.3: Performance of the momentum long-short strategy (J=6, K=6). Winners is the top decile, Losers is the bottom decile, and Long-Short is Winners minus Losers. The table reports the number of months, mean monthly return, t-statistic, and p-value.

	Winners	Losers	Long-Short
N	161.0000	161.0000	161.0000
Mean (%)	-0.6812	1.4190	-2.1002
Std (%)	5.7131	6.5319	4.8969
t-stat	-1.5130	2.7565	-5.4420
p-value	0.1323	0.0065	0.0000

18.5.3 Cumulative Returns

The time-series evolution of cumulative returns provides visual evidence of the momentum strategy's performance. Figure 18.3 shows the cumulative returns of the winner and loser portfolios, while Figure 18.4 shows the cumulative return of the long-short momentum strategy.

```
# Compute cumulative returns
ewret_wide = ewret_wide.sort_values("date").reset_index(drop=True)

for col in ["winners", "losers", "long_short"]:
    ewret_wide[f"cumret_{col}"] = (1 + ewret_wide[col]).cumprod() - 1
```

```
cumret_plot_data = (ewret_wide
    [["date", "cumret_winners", "cumret_losers"]]
    .melt(id_vars="date", var_name="portfolio", value_name="cumret")
)
cumret_plot_data["portfolio"] = cumret_plot_data["portfolio"].map({
    "cumret_winners": "Winners (P10)",
    "cumret_losers": "Losers (P1)"
})

plot_cumret = (
    ggplot(cumret_plot_data,
           aes(x="date", y="cumret", color="portfolio")) +
    geom_line(size=1) +
    scale_y_continuous(labels=percent_format()) +
    scale_color_manual(values=["#d62728", "#1f77b4"]) +
    labs(
        x="", y="Cumulative return",
        color="Portfolio",
        title="Cumulative Returns: Winners vs. Losers"
    ) +
    theme_minimal() +
    theme(
        figure_size=(10, 6),
        legend_position="bottom"
    )
)
plot_cumret
```

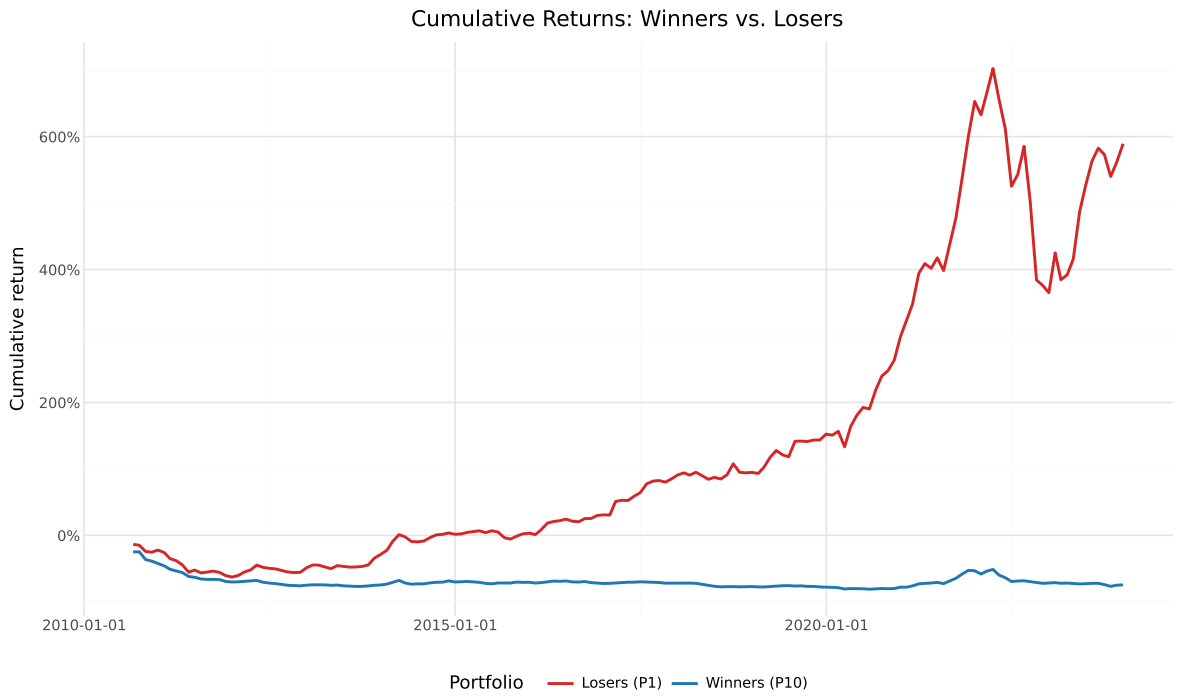


Figure 18.3: Cumulative returns of momentum winner and loser portfolios. The winner portfolio (blue) contains stocks in the top decile of past 6-month returns, and the loser portfolio (red) contains stocks in the bottom decile. The spread between the two lines reflects the cumulative momentum premium.

```
plot_ls = (
    ggplot(ewret_wide, aes(x="date", y="cumret_long_short")) +
    geom_line(size=1, color="#2ca02c") +
    geom_hline(yintercept=0, linetype="dashed", color="gray") +
    scale_y_continuous(labels=percent_format()) +
    labs(
        x="", y="Cumulative return",
        title="Cumulative Return: Long-Short Momentum Strategy"
    ) +
    theme_minimal() +
    theme(figure_size=(10, 6))
)
plot_ls
```

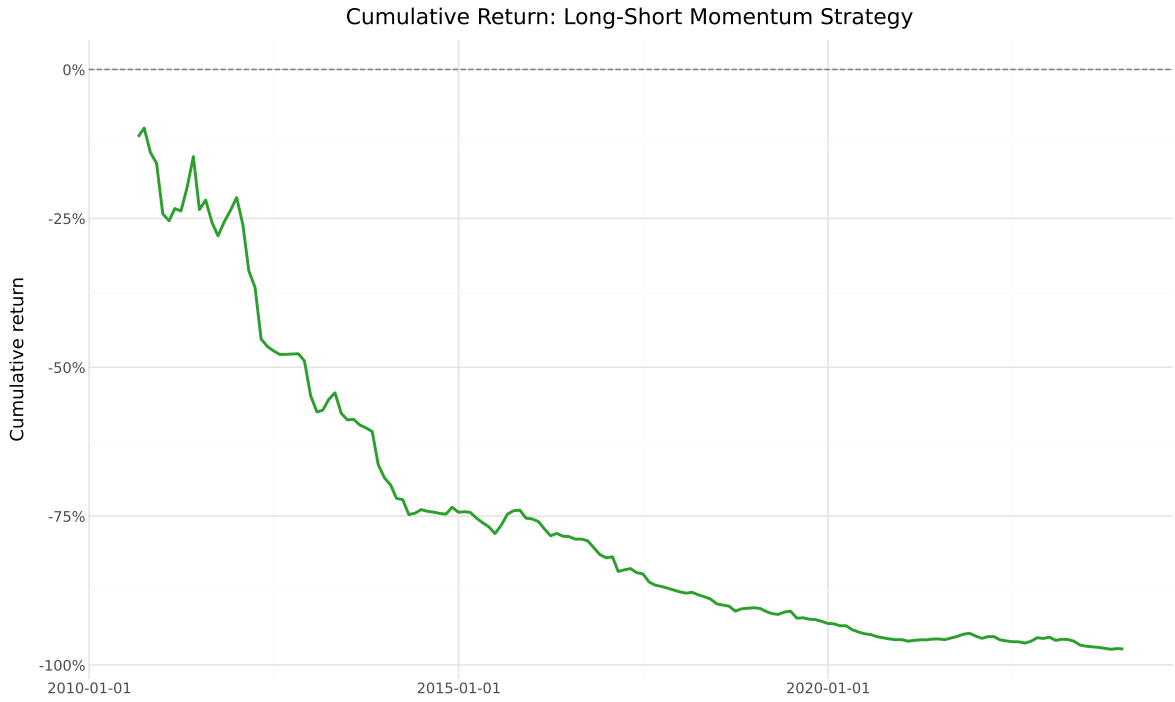


Figure 18.4: Cumulative return of the momentum long-short strategy (Winners minus Losers). Upward-sloping periods indicate momentum profits; sharp drawdowns often coincide with market reversals or crisis episodes.

18.5.4 Monthly Return Distribution

Beyond the mean, the full distribution of monthly long-short returns provides insight into the risk profile of the momentum strategy. Figure 87.1 shows the histogram of monthly returns.

```
mean_ls = ewret_wide["long_short"].mean()

plot_dist = (
    ggplot(ewret_wide, aes(x="long_short")) +
    geom_histogram(bins=50, fill="#1f77b4", alpha=0.7) +
    geom_vline(xintercept=mean_ls, linetype="dashed", color="red") +
    scale_x_continuous(labels=percent_format()) +
    labs(
        x="Monthly long-short return",
        y="Frequency",
        title="Distribution of Monthly Momentum Returns"
    ) +
```

```
    theme_minimal() +
    theme(figure_size=(10, 5))
)
plot_dist
```

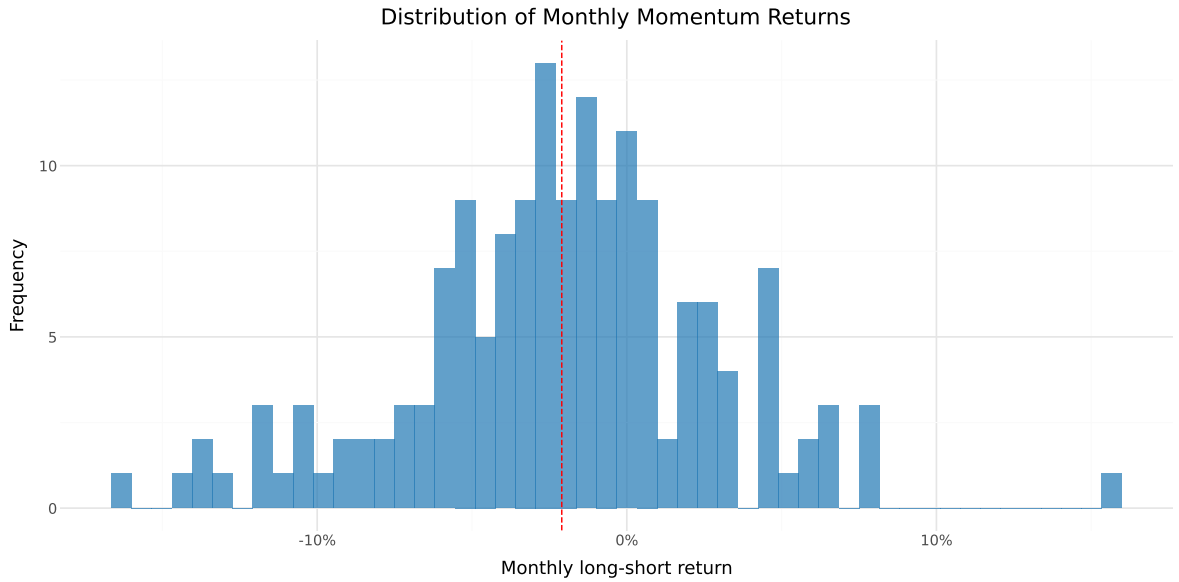


Figure 18.5: Distribution of monthly long-short momentum returns. The histogram shows the frequency distribution of monthly Winners-minus-Losers returns. The vertical dashed line indicates the mean monthly return.

18.6 Extending to Multiple Formation and Holding Periods

18.6.1 The J × K Grid

Jegadeesh and Titman (1993) evaluate momentum strategies across a comprehensive grid of formation periods $J \in \{3, 6, 9, 12\}$ and holding periods $K \in \{3, 6, 9, 12\}$. This produces 16 different strategy specifications, allowing us to assess the robustness of the momentum effect across different horizons.

We now implement a function that computes the full momentum strategy for any given (J, K) pair, wrapping the steps developed above into a single reusable pipeline.

```
from joblib import Parallel, delayed
import time

def momentum_strategy(data, J, K, n_portfolios=10):
    """
    Implement the full Jegadeesh-Titman momentum strategy.

    Parameters
    ----------
    data : pd.DataFrame
        Panel of stock returns with columns: symbol, date, ret.
    J : int
        Formation period in months.
    K : int
        Holding period in months.
    n_portfolios : int
        Number of portfolios (default: 10 for deciles).

    Returns
    -------
    pd.DataFrame
        Monthly returns for each momentum portfolio and the long-short
↪   spread.
    """
    # Step 1: Compute formation period cumulative returns
    df = data.sort_values(["symbol", "date"]).copy()
    df["gross_ret"] = 1 + df["ret"]
    df["cum_return"] = (
        df.groupby("symbol")["gross_ret"]
        .rolling(window=J, min_periods=J)
        .apply(np.prod, raw=True)
        .reset_index(level=0, drop=True)
        - 1
    )
    df = df.drop(columns=["gross_ret"]).dropna(subset=["cum_return"])

    # Step 2: Assign to momentum portfolios using transform
    df["momr"] = df.groupby("date", observed=True)["cum_return"].transform(
        lambda x: pd.qcut(
            x,
            q=n_portfolios,
            labels=range(1, n_portfolios + 1),
            duplicates="drop"
```

```
        )
    ).astype('Int64')

    # If any NaNs in momr, fill with rank-based assignment
    mask = df["momr"].isna()
    if mask.any():
        df.loc[mask, "momr"] =
↪   df.loc[mask].groupby("date")["cum_return"].transform(
            lambda x: pd.qcut(
                x.rank(method="first"),
                q=min(n_portfolios, len(x.unique())),
                labels=False,
                duplicates="drop"
            )
        ).astype('Int64') + 1

    # Step 3: Define holding period dates
    df = df.rename(columns={"date": "form_date"})
    df["hdate1"] = df["form_date"] + pd.offsets.MonthBegin(1)
    df["hdate2"] = df["form_date"] + pd.offsets.MonthEnd(K)

    # Step 4: Merge with holding period returns
    port_ret = df[["symbol", "form_date", "momr", "hdate1", "hdate2"]].merge(
        data[["symbol", "date", "ret"]].rename(
            columns={"ret": "hret", "date": "hdate"}
        ),
        on="symbol",
        how="inner"
    )

    # Use boolean indexing
    port_ret = port_ret[
        (port_ret["hdate"] >= port_ret["hdate1"]) &
        (port_ret["hdate"] <= port_ret["hdate2"])
    ]

    # Step 5: Compute equally weighted returns (two-stage averaging)
    cohort_ret = (port_ret
        .groupby(["hdate", "momr", "form_date"])
        .agg(cohort_ret=("hret", "mean"))
        .reset_index()
    )
```

```
    monthly_ret = (cohort_ret
        .groupby(["hdate", "momr"])
        .agg(ewret=("cohort_ret", "mean"))
        .reset_index()
        .rename(columns={"hdate": "date"})
    )

    # Step 6: Compute long-short spread
    wide = monthly_ret.pivot(
        index="date", columns="momr", values="ewret"
    ).reset_index()

    # Handle variable number of portfolios
    n_cols = len(wide.columns) - 1
    wide.columns = ["date"] + [f"port{i}" for i in range(1, n_cols + 1)]
    wide["winners"] = wide[f"port{n_cols}"]
    wide["losers"] = wide["port1"]
    wide["long_short"] = wide["winners"] - wide["losers"]

    return monthly_ret, wide
```

18.6.2 Running the Full Grid

We now run the momentum strategy for all 16 (J, K) combinations. This is computationally intensive, so we store the key summary statistics for each specification.

18.6.2.1 Sequential Calculation

```
J_values = [3, 6, 9, 12]
K_values = [3, 6, 9, 12]

results_grid = []

for J_val, K_val in product(J_values, K_values):
    print(f"Computing J={J_val}, K={K_val}...", end=" ")

    try:
        _, wide_result = momentum_strategy(prices_clean, J_val, K_val)
```

```
        for portfolio_name in ["winners", "losers", "long_short"]:
            series = wide_result[portfolio_name].dropna()
            n = len(series)
            mean_ret = series.mean()
            std_ret = series.std()
            t_stat = mean_ret / (std_ret / np.sqrt(n)) if std_ret > 0 else
↪   np.nan
            p_val = (2 * (1 - stats.t.cdf(abs(t_stat), df=n-1))
                     if not np.isnan(t_stat) else np.nan)

            results_grid.append({
                "J": J_val,
                "K": K_val,
                "portfolio": portfolio_name,
                "n_months": n,
                "mean_ret": mean_ret,
                "std_ret": std_ret,
                "t_stat": t_stat,
                "p_value": p_val
            })

        print("Done.")
    except Exception as e:
        print(f"Error: {e}")

results_df = pd.DataFrame(results_grid)
```

18.6.2.2 Parallel Calculation

```
def compute_single_strategy(data, J, K):
    """
    Compute statistics for a single (J, K) strategy.
    Returns a list of result dicts, one per portfolio.
    """
    try:
        _, wide_result = momentum_strategy(data, J, K)

        results = []
        for portfolio_name in ["winners", "losers", "long_short"]:
            series = wide_result[portfolio_name].dropna()
```

```
            n = len(series)
            mean_ret = series.mean()
            std_ret = series.std()
            t_stat = mean_ret / (std_ret / np.sqrt(n)) if std_ret > 0 else
↪  np.nan
            p_val = (2 * (1 - stats.t.cdf(abs(t_stat), df=n-1))
                     if not np.isnan(t_stat) else np.nan)

            results.append({
                "J": J,
                "K": K,
                "portfolio": portfolio_name,
                "n_months": n,
                "mean_ret": mean_ret,
                "std_ret": std_ret,
                "t_stat": t_stat,
                "p_value": p_val
            })
        return results
    except Exception as e:
        print(f"Error in J={J}, K={K}: {e}")
        return []
```

```
J_values = [3, 6, 9, 12]
K_values = [3, 6, 9, 12]

# Create list of (J, K) pairs
params = list(product(J_values, K_values))

print(f"Running {len(params)} momentum strategies in parallel with 4
↪  cores...")
start_time = time.time()

# Parallel execution with 4 workers
results_list = Parallel(n_jobs=4, verbose=10)(
    delayed(compute_single_strategy)(prices_clean, J, K)
    for J, K in params
)

# Flatten results
results_grid = [item for sublist in results_list for item in sublist]
results_df = pd.DataFrame(results_grid)
```

```
elapsed = time.time() - start_time
print(f"\nCompleted in {elapsed:.2f} seconds")
```

```
Running 16 momentum strategies in parallel with 4 cores...

[Parallel(n_jobs=4)]: Using backend LokyBackend with 4 concurrent workers.
[Parallel(n_jobs=4)]: Done   5 tasks      | elapsed:    8.6s
[Parallel(n_jobs=4)]: Done  11 out of  16 | elapsed:   14.4s remaining:    6.5s
[Parallel(n_jobs=4)]: Done  13 out of  16 | elapsed:   19.2s remaining:    4.4s


Completed in 20.01 seconds

[Parallel(n_jobs=4)]: Done  16 out of  16 | elapsed:   19.9s finished
```

```
# Summary statistics
print("\nResults Summary by Formation Period:")
print(results_df.groupby("J").agg({
    "mean_ret": "mean",
    "std_ret": "mean",
    "t_stat": ["mean", lambda x: (x.abs() > 1.96).sum()],
    "n_months": "first"
}).round(6))
```

```
Results Summary by Formation Period:
    mean_ret   std_ret    t_stat            n_months
        mean      mean      mean <lambda_0>    first
J
6  -0.004661  0.055672 -1.517507          9      161
9  -0.003265  0.056520 -1.071904          9      158
12 -0.002759  0.058194 -0.931273          9      155
```

18.6.3 Winners Portfolio Returns

Table 18.4 presents the average monthly returns of the winner portfolio (portfolio 10) across all (J, K) combinations.

```
winners_grid = (results_df
    .query("portfolio == 'winners'")
    .assign(
        display=lambda x: (
            x["mean_ret"].apply(lambda v: f"{v*100:.2f}") +
            "\n(" + x["t_stat"].apply(lambda v: f"{v:.2f}") + ")"
        )
    )
    .pivot(index="J", columns="K", values="display")
)
winners_grid.columns = [f"K={k}" for k in winners_grid.columns]
winners_grid.index = [f"J={j}" for j in winners_grid.index]
winners_grid
```

Table 18.4: Average monthly returns (%) of the winner portfolio across formation periods (J) and holding periods (K). t-statistics are reported in parentheses.

	K=3	K=6	K=9	K=12
J=6	-1.18\n(-2.65)	-0.68\n(-1.51)	-0.55\n(-1.23)	-0.39\n(-0.87)
J=9	-1.00\n(-2.36)	-0.51\n(-1.22)	-0.28\n(-0.68)	-0.16\n(-0.39)
J=12	-0.88\n(-2.09)	-0.39\n(-0.91)	-0.24\n(-0.56)	-0.14\n(-0.34)

18.6.4 Losers Portfolio Returns

Table 18.5 presents the corresponding results for the loser portfolio.

```
losers_grid = (results_df
    .query("portfolio == 'losers'")
    .assign(
        display=lambda x: (
            x["mean_ret"].apply(lambda v: f"{v*100:.2f}") +
            "\n(" + x["t_stat"].apply(lambda v: f"{v:.2f}") + ")"
        )
    )
    .pivot(index="J", columns="K", values="display")
)
losers_grid.columns = [f"K={k}" for k in losers_grid.columns]
losers_grid.index = [f"J={j}" for j in losers_grid.index]
losers_grid
```

Table 18.5: Average monthly returns (%) of the loser portfolio across formation periods (J) and holding periods (K). t-statistics are reported in parentheses.

	K=3	K=6	K=9	K=12
J=6	1.87\n(3.49)	1.42\n(2.76)	1.10\n(2.19)	0.99\n(1.99)
J=9	1.94\n(3.51)	1.38\n(2.56)	1.21\n(2.31)	1.15\n(2.22)
J=12	1.57\n(2.71)	1.29\n(2.28)	1.19\n(2.13)	1.15\n(2.09)

18.6.5 Long-Short Momentum Returns

Table 18.6 presents the most important results: the average monthly returns of the long-short momentum portfolio across all specifications.

```
ls_grid = (results_df
    .query("portfolio == 'long_short'")
    .assign(
        display=lambda x: (
            x["mean_ret"].apply(lambda v: f"{v*100:.2f}") +
            "\n(" + x["t_stat"].apply(lambda v: f"{v:.2f}") + ")"
        )
    )
    .pivot(index="J", columns="K", values="display")
)
ls_grid.columns = [f"K={k}" for k in ls_grid.columns]
ls_grid.index = [f"J={j}" for j in ls_grid.index]
ls_grid
```

Table 18.6: Average monthly returns (%) of the momentum long-short portfolio (Winners minus Losers) across formation periods (J) and holding periods (K). t-statistics are reported in parentheses. This table replicates the format of Table 1 in Jegadeesh and Titman (1993).

	K=3	K=6	K=9	K=12
J=6	-3.04\n(-7.38)	-2.10\n(-5.44)	-1.65\n(-4.94)	-1.37\n(-4.61)
J=9	-2.94\n(-6.41)	-1.89\n(-4.55)	-1.49\n(-3.98)	-1.31\n(-3.87)
J=12	-2.45\n(-5.42)	-1.68\n(-3.92)	-1.43\n(-3.60)	-1.30\n(-3.55)

18.6.6 Visualizing the Momentum Premium Across Specifications

Figure 18.6 provides a visual summary of the long-short momentum premium across all (J, K) combinations.

```
ls_heatmap_data = (results_df
    .query("portfolio == 'long_short'")
    .assign(
        mean_pct=lambda x: x["mean_ret"] * 100,
        significant=lambda x: x["p_value"] < 0.05,
        label=lambda x: x.apply(
            lambda row: f"{row['mean_ret']*100:.2f}{'*' if row['p_value'] <
            ↪  0.05 else ''}",
            axis=1
        )
    )
)

plot_heatmap = (
    ggplot(ls_heatmap_data,
           aes(x="K.astype(str)", y="J.astype(str)", fill="mean_pct")) +
    geom_tile(color="white", size=2) +
    geom_text(aes(label="label"), size=10, color="white") +
    scale_fill_gradient2(
        low="#d62728", mid="#f7f7f7", high="#1f77b4", midpoint=0,
        name="Monthly\nReturn (%)"
    ) +
    labs(
        x="Holding Period (K months)",
        y="Formation Period (J months)",
        title="Momentum Premium Across J×K Specifications"
    ) +
    theme_minimal() +
    theme(figure_size=(8, 6))
)
plot_heatmap
```

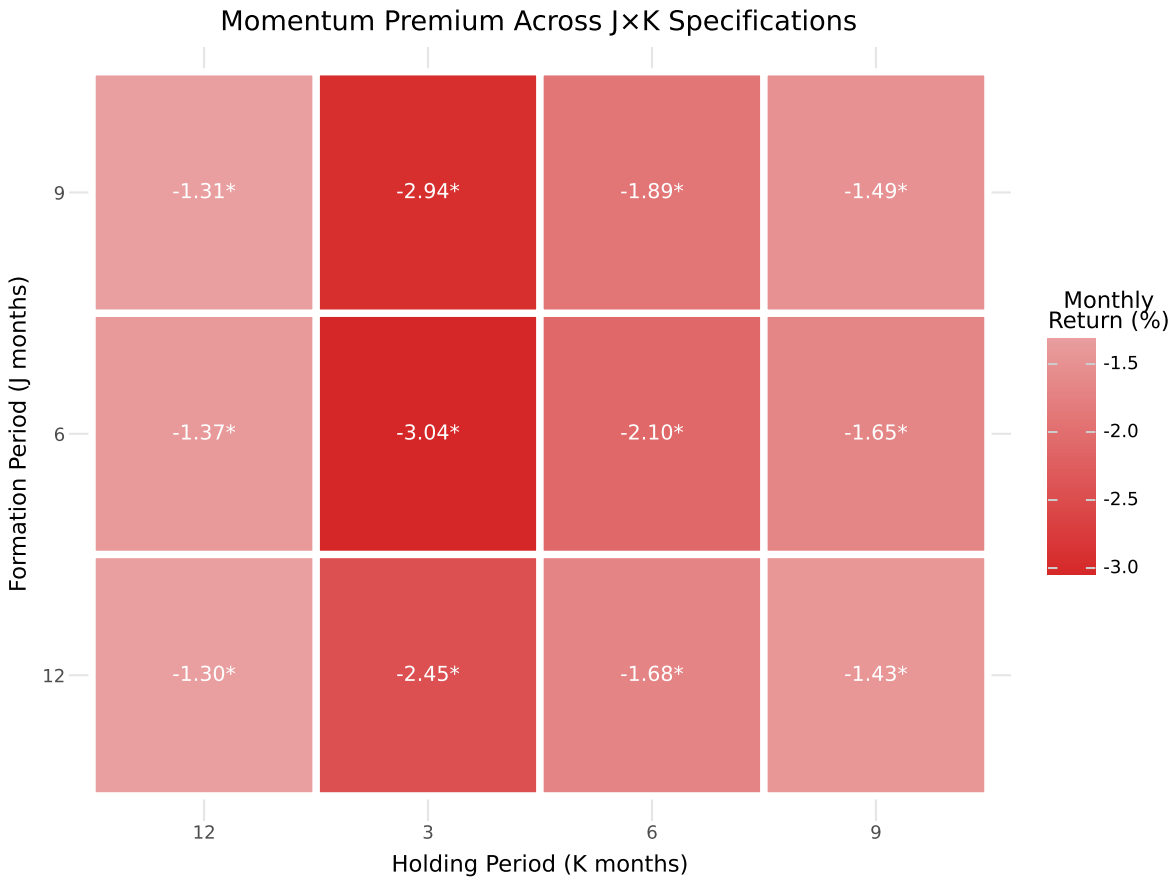


Figure 18.6: Heatmap of average monthly long-short momentum returns (%) across formation periods (J) and holding periods (K). Darker shades indicate higher momentum profits. Asterisks denote statistical significance at the 5% level.

18.7 Risk-Adjusted Performance

18.7.1 CAPM Alpha

A natural question is whether the momentum premium is explained by exposure to the market factor. We estimate the CAPM alpha of the long-short momentum portfolio by regressing its returns on the market excess return:

$$r_{\text{WML},t} = \alpha + \beta \cdot r_{\text{MKT},t} + \epsilon_t \tag{18.3}$$

```
# Merge momentum returns with factor data (baseline J=6, K=6)
ewret_factors = (ewret_wide
    [["date", "winners", "losers", "long_short"]]
    .merge(factors_ff3_monthly, on="date", how="inner")
)

# CAPM regression for long-short portfolio
from statsmodels.formula.api import ols as ols_formula
import statsmodels.api as sm

X_capm = sm.add_constant(ewret_factors["mkt_excess"])
y_ls = ewret_factors["long_short"]

capm_model = sm.OLS(y_ls, X_capm).fit(cov_type="HAC", cov_kwds={"maxlags":
 ↪  6})

print("CAPM Regression: Long-Short Momentum Returns")
print("=" * 60)
print(f"Alpha (monthly):  {capm_model.params['const']*100:.4f}% "
      f"(t={capm_model.tvalues['const']:.2f})")
print(f"Market Beta:      {capm_model.params['mkt_excess']:.4f} "
      f"(t={capm_model.tvalues['mkt_excess']:.2f})")
print(f"R-squared:        {capm_model.rsquared:.4f}")
print(f"N observations:   {capm_model.nobs:.0f}")
```

```
CAPM Regression: Long-Short Momentum Returns
============================================================
Alpha (monthly):  -2.2703% (t=-5.05)
Market Beta:      -0.1779 (t=-1.75)
R-squared:        0.0470
N observations:   150
```

18.7.2 Fama-French Three-Factor Alpha

We extend the risk adjustment to the Fama-French three-factor model, which includes the size (SMB) and value (HML) factors in addition to the market factor:

$$r_{\text{WML},t} = \alpha + \beta_1 \cdot r_{\text{MKT},t} + \beta_2 \cdot \text{SMB}_t + \beta_3 \cdot \text{HML}_t + \epsilon_t \tag{18.4}$$

```
X_ff3 = sm.add_constant(
    ewret_factors[["mkt_excess", "smb", "hml"]]
)
y_ls = ewret_factors["long_short"]

ff3_model = sm.OLS(y_ls, X_ff3).fit(cov_type="HAC", cov_kwds={"maxlags": 6})

# Display results as a clean table
ff3_results = pd.DataFrame({
    "Coefficient": ff3_model.params,
    "Std Error": ff3_model.bse,
    "t-stat": ff3_model.tvalues,
    "p-value": ff3_model.pvalues
}).round(4)

ff3_results.index = ["Alpha", "MKT", "SMB", "HML"]
ff3_results
```

Table 18.7: Fama-French three-factor regression for the momentum long-short portfolio. The dependent variable is the monthly return of the Winners-minus-Losers portfolio. Alpha is the intercept, representing the risk-adjusted abnormal return. HAC standard errors with 6 lags are used.

	Coefficient	Std Error	t-stat	p-value
Alpha	-0.0234	0.0041	-5.7189	0.0000
MKT	-0.1269	0.1558	-0.8145	0.4154
SMB	0.1123	0.1882	0.5969	0.5506
HML	0.0716	0.1414	0.5065	0.6125

```
print(f"\nR-squared: {ff3_model.rsquared:.4f}")
print(f"Adjusted R-squared: {ff3_model.rsquared_adj:.4f}")
print(f"Alpha (annualized): {ff3_model.params['const'] * 12 * 100:.2f}%")
```

```
R-squared: 0.0553
Adjusted R-squared: 0.0359
Alpha (annualized): -28.02%
```

18.7.3 Interpretation of Risk Exposures

The factor loadings from the three-factor regression reveal the risk characteristics of the momentum strategy in the Vietnamese market. Several patterns are commonly observed:

1. **Market beta**: Momentum portfolios typically have moderate market exposure. In the U.S., Grundy and Martin (2001) document that the market beta of the long-short portfolio is close to zero on average but highly time-varying, spiking during market reversals.
2. **Size exposure (SMB)**: Momentum strategies often load positively on the size factor, reflecting the tendency for smaller stocks to exhibit stronger momentum patterns.
3. **Value exposure (HML)**: The long-short momentum portfolio typically loads negatively on HML, indicating that winners tend to be growth stocks while losers tend to be value stocks. This creates a natural tension between momentum and value strategies.

18.8 Momentum and Market States

18.8.1 Conditional Performance

An important finding in the momentum literature is that momentum profits vary with market conditions. Cooper, Gutierrez Jr, and Hameed (2004) document that momentum strategies perform well following market gains (UP markets) but experience severe losses following market declines (DOWN markets). This asymmetry is particularly relevant for emerging markets, which experience more extreme market states.

We define market states based on the cumulative market return over the prior 12 months:

$$\text{Market State}_t = \begin{cases} \text{UP} & \text{if } \prod_{s=t-12}^{t-1}(1 + r_{m,s}) - 1 > 0 \\ \text{DOWN} & \text{otherwise} \end{cases} \tag{18.5}$$

```
# Compute 12-month lagged market return
market_returns = factors_ff3_monthly[["date", "mkt_excess"]].copy()
market_returns = market_returns.sort_values("date")
market_returns["mkt_cum_12m"] = (
    (1 + market_returns["mkt_excess"])
    .rolling(window=12, min_periods=12)
    .apply(np.prod, raw=True)
    - 1
)
market_returns["market_state"] = np.where(
```

```
    market_returns["mkt_cum_12m"] > 0, "UP", "DOWN"
)

# Merge with momentum returns
ewret_states = ewret_wide[["date", "long_short"]].merge(
    market_returns[["date", "market_state"]], on="date", how="inner"
).dropna()
```

```
state_stats = []
for state in ["UP", "DOWN"]:
    subset = ewret_states.query(f"market_state == '{state}'")["long_short"]
    n = len(subset)
    mean_ret = subset.mean()
    std_ret = subset.std()
    t_stat = mean_ret / (std_ret / np.sqrt(n)) if std_ret > 0 else np.nan
    p_val = 2 * (1 - stats.t.cdf(abs(t_stat), df=n-1)) if not
 ↪  np.isnan(t_stat) else np.nan
    state_stats.append({
        "Market State": state,
        "N Months": n,
        "Mean (%)": mean_ret * 100,
        "Std (%)": std_ret * 100,
        "t-stat": t_stat,
        "p-value": p_val
    })

state_stats_df = pd.DataFrame(state_stats).round(4)
state_stats_df
```

Table 18.8: Momentum long-short returns conditional on market states. UP markets are defined as periods where the cumulative market return over the prior 12 months is positive; DOWN markets are periods with negative prior 12-month returns.

	Market State	N Months	Mean (%)	Std (%)	t-stat	p-value
0	UP	39	-1.1972	4.5814	-1.6320	0.1109
1	DOWN	111	-2.4050	4.8669	-5.2063	0.0000

```
plot_states = (
    ggplot(ewret_states, aes(x="market_state", y="long_short",
                             fill="market_state")) +
```

```
    geom_boxplot(alpha=0.7) +
    geom_hline(yintercept=0, linetype="dashed", color="gray") +
    scale_y_continuous(labels=percent_format()) +
    scale_fill_manual(values={"UP": "#1f77b4", "DOWN": "#d62728"}) +
    labs(
        x="Market State (Prior 12-Month Return)",
        y="Monthly Long-Short Return",
        title="Momentum Returns by Market State"
    ) +
    theme_minimal() +
    theme(
        figure_size=(8, 6),
        legend_position="none"
    )
)
plot_states
```

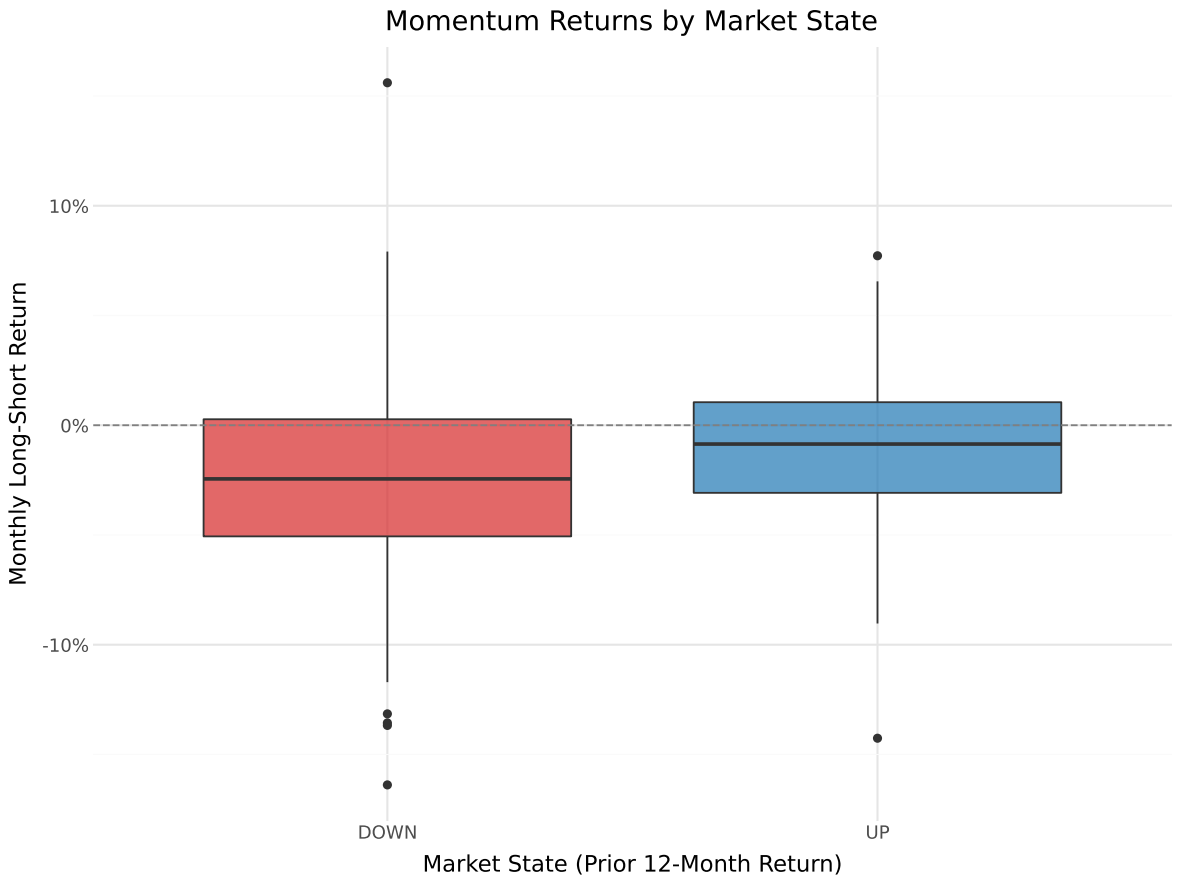


Figure 18.7: Distribution of monthly momentum returns by market state. The box plots show the distribution of long-short momentum returns separately for UP and DOWN market states, defined by the sign of the prior 12-month cumulative market return.

18.9 Momentum Crashes

18.9.1 Understanding Momentum Drawdowns

One of the most important risk characteristics of momentum strategies is their susceptibility to sudden, severe losses—known as momentum crashes. Daniel and Moskowitz (2016) document that momentum strategies experience infrequent but extreme losses, typically during market rebounds following bear markets. These crashes occur because the loser portfolio, which has been short, is heavily loaded with high-beta stocks that surge when markets reverse.

We identify the worst drawdowns of the momentum strategy and examine their market context.

```
worst_months = (ewret_wide
    [["date", "long_short", "winners", "losers"]]
    .merge(factors_ff3_monthly[["date", "mkt_excess"]], on="date",
 ↪  how="left")
    .sort_values("long_short")
    .head(10)
    .assign(
        long_short_pct=lambda x: (x["long_short"] * 100).round(2),
        winners_pct=lambda x: (x["winners"] * 100).round(2),
        losers_pct=lambda x: (x["losers"] * 100).round(2),
        mkt_pct=lambda x: (x["mkt_excess"] * 100).round(2)
    )
    [["date", "long_short_pct", "winners_pct", "losers_pct", "mkt_pct"]]
    .rename(columns={
        "date": "Date",
        "long_short_pct": "L/S (%)",
        "winners_pct": "Winners (%)",
        "losers_pct": "Losers (%)",
        "mkt_pct": "Market (%)"
    })
)
worst_months
```

Table 18.9: Ten worst months for the momentum long-short strategy. The table reports the date, long-short return, winner return, loser return, and concurrent market return for the ten months with the largest momentum losses.

	Date	L/S (%)	Winners (%)	Losers (%)	Market (%)
153	2023-05-31	-16.39	-2.69	13.70	2.78
39	2013-11-30	-14.26	4.24	18.50	2.57
20	2012-04-30	-13.68	1.51	15.19	4.88
78	2017-02-28	-13.56	2.10	15.67	1.49
107	2019-07-31	-13.15	-2.46	10.69	1.61
140	2022-04-30	-11.71	-17.57	-5.87	-11.19
149	2023-01-31	-11.53	1.37	12.90	6.96
28	2012-12-31	-11.52	4.03	15.54	-1.61
0	2010-08-31	-11.31	-25.03	-13.72	NaN
10	2011-06-30	-10.44	-3.15	7.29	NaN

18.9.2 Maximum Drawdown Analysis

The maximum drawdown provides a measure of the worst peak-to-trough decline experienced by the strategy. This metric is particularly relevant for practitioners evaluating the risk of momentum strategies.

```
# Compute running maximum and drawdown
ewret_wide["cum_wealth"] = (1 + ewret_wide["long_short"]).cumprod()
ewret_wide["running_max"] = ewret_wide["cum_wealth"].cummax()
ewret_wide["drawdown"] = (
    ewret_wide["cum_wealth"] / ewret_wide["running_max"] - 1
)

max_dd = ewret_wide["drawdown"].min()
max_dd_date = ewret_wide.loc[ewret_wide["drawdown"].idxmin(), "date"]

print(f"Maximum drawdown: {max_dd*100:.2f}%")
print(f"Date of maximum drawdown: {max_dd_date.date()}")
```

```
Maximum drawdown: -97.08%
Date of maximum drawdown: 2023-10-31
```

```
plot_dd = (
    ggplot(ewret_wide, aes(x="date", y="drawdown")) +
    geom_area(fill="#d62728", alpha=0.5) +
    geom_line(color="#d62728", size=0.5) +
    scale_y_continuous(labels=percent_format()) +
    labs(
        x="", y="Drawdown",
        title="Momentum Strategy Drawdown"
    ) +
    theme_minimal() +
    theme(figure_size=(10, 5))
)
plot_dd
```

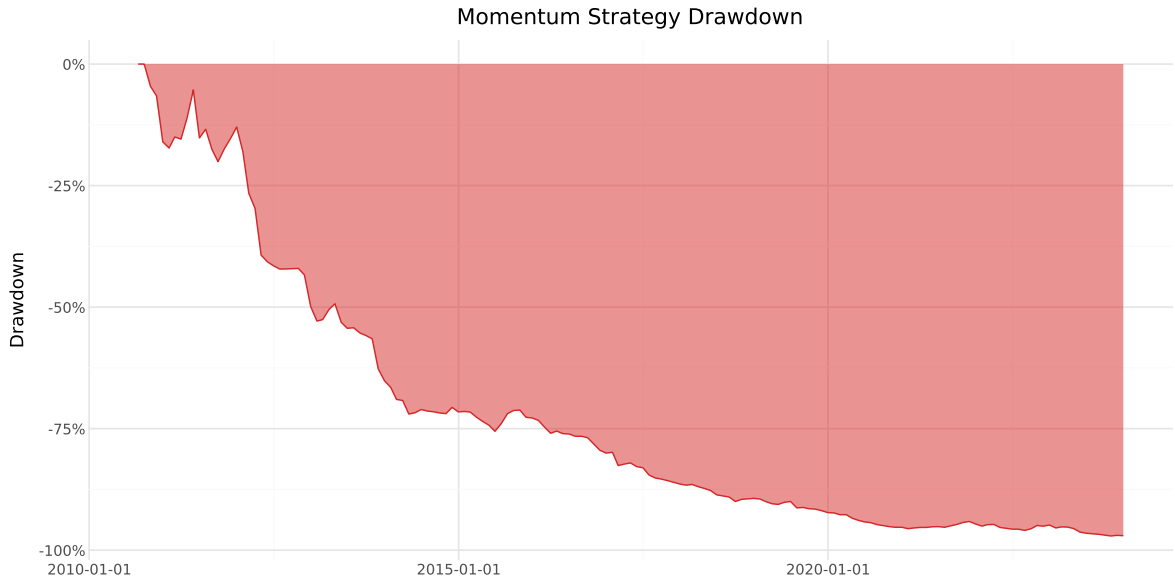


Figure 18.8: Drawdown of the momentum long-short strategy over time. The chart shows the percentage decline from the previous peak in cumulative wealth. Deeper drawdowns represent more severe momentum crashes.

18.10 Value-Weighted Momentum Portfolios

The baseline Jegadeesh and Titman (1993) implementation uses equally weighted portfolios. However, equally weighted returns can be dominated by small, illiquid stocks that may be difficult to trade in practice. Value-weighted portfolios, where each stock's contribution is proportional to its market capitalization, provide a more investable benchmark and are more representative of the returns that large investors could actually achieve.

```
def momentum_strategy_vw(data, J, K, n_portfolios=10):
    """
    Value-weighted momentum strategy implementation.

    Same as the equally-weighted version but uses lagged market
    capitalization as weights when computing portfolio returns.

    Parameters
    ----------
    data : pd.DataFrame
        Panel with columns: symbol, date, ret, mktcap_lag.
    J : int
```

```
        Formation period in months.
    K : int
        Holding period in months.
    n_portfolios : int
        Number of portfolios.

    Returns
    -------
    pd.DataFrame
        Monthly value-weighted portfolio returns.
    """
    # Step 1: Formation period returns
    df = data.sort_values(["symbol", "date"]).copy()
    df["gross_ret"] = 1 + df["ret"]
    df["cum_return"] = (
        df.groupby("symbol")["gross_ret"]
        .rolling(window=J, min_periods=J)
        .apply(np.prod, raw=True)
        .reset_index(level=0, drop=True)
        - 1
    )
    df = df.drop(columns=["gross_ret"]).dropna(subset=["cum_return"])

    # Step 2: Portfolio assignment using transform (fast)
    df["momr"] = df.groupby("date", observed=True)["cum_return"].transform(
        lambda x: pd.qcut(
            x,
            q=n_portfolios,
            labels=range(1, n_portfolios + 1),
            duplicates="drop"
        )
    ).astype('Int64')

    # Fill NaNs with rank-based assignment
    mask = df["momr"].isna()
    if mask.any():
        df.loc[mask, "momr"] =
↪   df.loc[mask].groupby("date")["cum_return"].transform(
            lambda x: pd.qcut(
                x.rank(method="first"),
                q=min(n_portfolios, len(x.unique())),
                labels=False,
                duplicates="drop"
```

```
        )
    ).astype('Int64') + 1

# Step 3: Holding period
df = df.rename(columns={"date": "form_date"})
df["hdate1"] = df["form_date"] + pd.offsets.MonthBegin(1)
df["hdate2"] = df["form_date"] + pd.offsets.MonthEnd(K)

# Step 4: Merge with holding period returns AND weights
port_ret = df[
    ["symbol", "form_date", "momr", "hdate1", "hdate2"]
].merge(
    data[["symbol", "date", "ret", "mktcap_lag"]].rename(
        columns={"ret": "hret", "date": "hdate"}
    ),
    on="symbol",
    how="inner"
)

# Use boolean indexing instead of query (faster)
port_ret = port_ret[
    (port_ret["hdate"] >= port_ret["hdate1"]) &
    (port_ret["hdate"] <= port_ret["hdate2"])
]
port_ret = port_ret.dropna(subset=["mktcap_lag"])
port_ret = port_ret[port_ret["mktcap_lag"] > 0]

# Step 5: Value-weighted returns within each cohort
def vw_mean(group):
    weights = group["mktcap_lag"]
    if weights.sum() == 0:
        return np.nan
    return np.average(group["hret"], weights=weights)

cohort_ret = (port_ret
    .groupby(["hdate", "momr", "form_date"])
    .apply(vw_mean, include_groups=False)
    .reset_index(name="cohort_ret")
)

monthly_ret = (cohort_ret
    .groupby(["hdate", "momr"])
    .agg(vwret=("cohort_ret", "mean"))
```

```
        .reset_index()
        .rename(columns={"hdate": "date"})
    )

    # Step 6: Long-short
    wide = monthly_ret.pivot(
        index="date", columns="momr", values="vwret"
    ).reset_index()

    # Handle variable number of portfolios
    n_cols = len(wide.columns) - 1
    wide.columns = ["date"] + [f"port{i}" for i in range(1, n_cols + 1)]
    wide["winners"] = wide[f"port{n_cols}"]
    wide["losers"] = wide["port1"]
    wide["long_short"] = wide["winners"] - wide["losers"]

    return monthly_ret, wide
```

```
# Run value-weighted J=6, K=6 strategy
_, vw_results = momentum_strategy_vw(prices_clean, J=6, K=6)

print("Value-Weighted Momentum Strategy (J=6, K=6)")
print("=" * 50)
print("\nPortfolio Statistics:")
print(vw_results[["winners", "losers", "long_short"]].describe().round(4))
```

```
Value-Weighted Momentum Strategy (J=6, K=6)
==================================================

Portfolio Statistics:
        winners    losers  long_short
count  161.0000  161.0000    161.0000
mean    -0.0129    0.0006     -0.0135
std      0.0763    0.0713      0.0653
min     -0.2519   -0.2178     -0.3813
25%     -0.0540   -0.0397     -0.0465
50%     -0.0067    0.0010     -0.0126
75%      0.0354    0.0443      0.0173
max      0.1765    0.2498      0.2498
```

```
comparison = []
for scheme, df in [("EW", ewret_wide), ("VW", vw_results)]:
    for col in ["winners", "losers", "long_short"]:
        series = df[col].dropna()
        n = len(series)
        mean_ret = series.mean()
        std_ret = series.std()
        t_stat = mean_ret / (std_ret / np.sqrt(n))
        p_val = 2 * (1 - stats.t.cdf(abs(t_stat), df=n-1))
        comparison.append({
            "Weighting": scheme,
            "Portfolio": col.replace("_", " ").title(),
            "Mean (%)": round(mean_ret * 100, 4),
            "Std (%)": round(std_ret * 100, 4),
            "t-stat": round(t_stat, 2),
            "p-value": round(p_val, 4)
        })

pd.DataFrame(comparison)
```

Table 18.10: Comparison of equally weighted (EW) and value-weighted (VW) momentum strategies for J=6, K=6. The table reports mean monthly returns, t-statistics, and p-values for winners, losers, and long-short portfolios under both weighting schemes.

	Weighting	Portfolio	Mean (%)	Std (%)	t-stat	p-value
0	EW	Winners	-0.6812	5.7131	-1.51	0.1323
1	EW	Losers	1.4190	6.5319	2.76	0.0065
2	EW	Long Short	-2.1002	4.8969	-5.44	0.0000
3	VW	Winners	-1.2943	7.6274	-2.15	0.0328
4	VW	Losers	0.0589	7.1275	0.10	0.9166
5	VW	Long Short	-1.3533	6.5312	-2.63	0.0094

```
vw_results = vw_results.sort_values("date")
vw_results["cumret_ls_vw"] = (1 + vw_results["long_short"]).cumprod() - 1

ew_data = (ewret_wide[["date", "long_short"]]
    .rename(columns={"long_short": "cumret"})
    .assign(scheme="Equally Weighted")
)
ew_data["cumret"] = (1 + ew_data["cumret"]).cumprod() - 1
```

```
vw_data = (vw_results[["date", "cumret_ls_vw"]]
    .rename(columns={"cumret_ls_vw": "cumret"})
    .assign(scheme="Value Weighted")
)

ew_vs_vw = pd.concat([ew_data, vw_data], ignore_index=True)

plot_ew_vw = (
    ggplot(ew_vs_vw, aes(x="date", y="cumret", color="scheme")) +
    geom_line(size=1) +
    scale_y_continuous(labels=percent_format()) +
    scale_color_manual(values=["#1f77b4", "#ff7f0e"]) +
    labs(
        x="", y="Cumulative return",
        color="Weighting Scheme",
        title="EW vs. VW Momentum Long-Short Strategy"
    ) +
    theme_minimal() +
    theme(
        figure_size=(10, 6),
        legend_position="bottom"
    )
)
plot_ew_vw
```

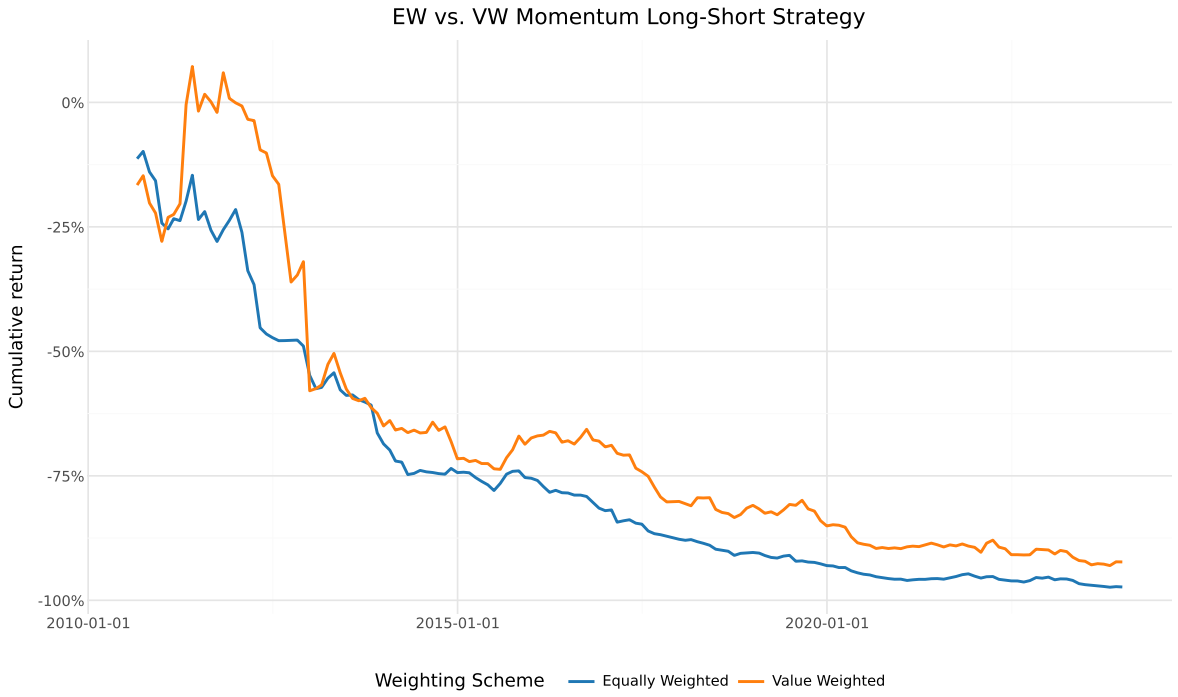


Figure 18.9: Cumulative returns of equally weighted (EW) versus value-weighted (VW) long-short momentum strategies. Differences between the two lines reflect the impact of firm size on momentum profitability.

18.11 Daily Momentum Analysis

While momentum strategies are typically evaluated at the monthly frequency following Jegadeesh and Titman (1993), analyzing daily return patterns provides additional insights into the dynamics of momentum profits. Daily data allows us to examine how momentum profits accrue within the holding period, measure intra-month volatility of the strategy, and compute more precise risk measures.

18.11.1 Loading Daily Data

```
prices_daily = pd.read_sql_query(
    sql=("SELECT symbol, date, ret, ret_excess, mktcap_lag "
         "FROM prices_daily"),
    con=tidy_finance,
    parse_dates={"date"}
```

```
)

print(f"Daily observations: {len(prices_daily):,}")
print(f"Unique stocks: {prices_daily['symbol'].nunique():,}")
print(f"Date range: {prices_daily['date'].min().date()} "
      f"to {prices_daily['date'].max().date()}")
```

```
Daily observations: 3,462,157
Unique stocks: 1,459
Date range: 2010-01-05 to 2023-12-29
```

18.11.2 Daily Returns of Monthly Momentum Portfolios

Rather than forming momentum portfolios at the daily frequency (which would require daily rebalancing and is impractical), we use the monthly portfolio assignments and track their daily returns. This gives us the daily return series of the monthly momentum strategy.

```
# # # Use the monthly portfolio assignments from the J=6 baseline
monthly_assignments = portfolios_with_dates[
    ["symbol", "form_date", "momr", "hdate1", "hdate2"]
].copy()

# Merge with daily returns
daily_mom_returns = monthly_assignments.merge(
    prices_daily[["symbol", "date", "ret"]].rename(
        columns={"ret": "dret", "date": "ddate"}
    ),
    on="symbol",
    how="inner"
)

# Use boolean indexing instead of query
daily_mom_returns = daily_mom_returns[
    (daily_mom_returns["ddate"] >= daily_mom_returns["hdate1"]) &
    (daily_mom_returns["ddate"] <= daily_mom_returns["hdate2"])
]

print(f"Daily portfolio return observations: {len(daily_mom_returns):,}")
```

```
Daily portfolio return observations: 19,112,536
```

```
# Compute daily equally weighted portfolio returns
# Stage 1: Average within each cohort
daily_cohort_ret = (daily_mom_returns
    .groupby(["ddate", "momr", "form_date"])
    .agg(cohort_ret=("dret", "mean"))
    .reset_index()
)

# Stage 2: Average across cohorts
daily_ewret = (daily_cohort_ret
    .groupby(["ddate", "momr"])
    .agg(ewret=("cohort_ret", "mean"))
    .reset_index()
    .rename(columns={"ddate": "date"})
)

# Compute daily long-short returns
daily_wide = daily_ewret.pivot(
    index="date", columns="momr", values="ewret"
).reset_index()
daily_wide.columns = ["date"] + [f"port{i}" for i in range(1, 11)]
daily_wide["winners"] = daily_wide["port10"]
daily_wide["losers"] = daily_wide["port1"]
daily_wide["long_short"] = daily_wide["winners"] - daily_wide["losers"]

print(f"Daily long-short return observations: {len(daily_wide):,}")
```

```
Daily long-short return observations: 3,352
```

18.11.3 Daily Cumulative Returns

```
daily_wide = daily_wide.sort_values("date")
daily_wide["cumret_ls"] = (1 + daily_wide["long_short"]).cumprod() - 1

plot_daily_cumret = (
    ggplot(daily_wide, aes(x="date", y="cumret_ls")) +
    geom_line(size=0.5, color="#2ca02c") +
    geom_hline(yintercept=0, linetype="dashed", color="gray") +
    scale_y_continuous(labels=percent_format()) +
    labs(
```

```
        x="", y="Cumulative return",
        title="Daily Cumulative Return: Momentum Long-Short Strategy"
    ) +
    theme_minimal() +
    theme(figure_size=(10, 6))
)
plot_daily_cumret
```

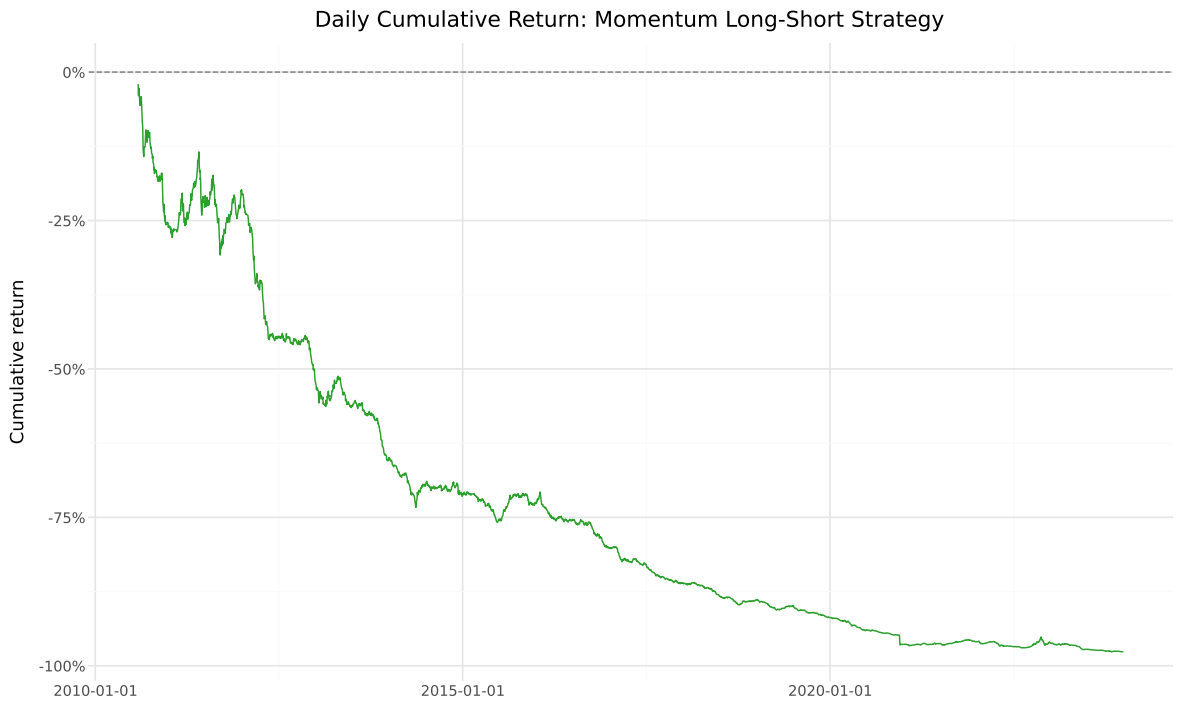


Figure 18.10: Cumulative daily returns of the momentum long-short strategy. This figure shows the same strategy as the monthly analysis but tracked at daily frequency, revealing intra-month dynamics and the precise timing of momentum gains and losses.

18.11.4 Annualized Risk Metrics from Daily Data

Daily data enables more precise estimation of risk metrics through higher-frequency sampling.

```
daily_ls = daily_wide["long_short"].dropna()
```

```
# Annualized metrics
ann_mean = daily_ls.mean() * 252
ann_vol = daily_ls.std() * np.sqrt(252)
sharpe = ann_mean / ann_vol if ann_vol > 0 else np.nan

# Drawdown
daily_wealth = (1 + daily_ls).cumprod()
daily_running_max = daily_wealth.cummax()
daily_dd = (daily_wealth / daily_running_max - 1).min()

# Higher moments
skew = daily_ls.skew()
kurt = daily_ls.kurtosis()

# VaR and CVaR
var_95 = daily_ls.quantile(0.05)
cvar_95 = daily_ls[daily_ls <= var_95].mean()

risk_metrics = pd.DataFrame({
    "Metric": [
        "Annualized Mean Return",
        "Annualized Volatility",
        "Sharpe Ratio",
        "Maximum Drawdown",
        "Skewness",
        "Excess Kurtosis",
        "Daily VaR (5%)",
        "Daily CVaR (5%)"
    ],
    "Value": [
        f"{ann_mean*100:.2f}%",
        f"{ann_vol*100:.2f}%",
        f"{sharpe:.2f}",
        f"{daily_dd*100:.2f}%",
        f"{skew:.2f}",
        f"{kurt:.2f}",
        f"{var_95*100:.2f}%",
        f"{cvar_95*100:.2f}%"
    ]
})
risk_metrics
```

Table 18.11: Annualized risk metrics for the momentum long-short strategy computed from daily returns. Volatility is annualized using the square root of 252 rule. The Sharpe ratio, maximum drawdown, skewness, and kurtosis provide a comprehensive risk profile.

	Metric	Value
0	Annualized Mean Return	-26.90%
1	Annualized Volatility	15.01%
2	Sharpe Ratio	-1.79
3	Maximum Drawdown	-97.62%
4	Skewness	-11.23
5	Excess Kurtosis	378.11
6	Daily VaR (5%)	-1.33%
7	Daily CVaR (5%)	-2.10%

18.11.5 Realized Volatility of Momentum Returns

Using daily returns, we can compute the monthly realized volatility of the momentum strategy and examine how it varies over time.

```
daily_wide["year_month"] = daily_wide["date"].dt.to_period("M")

realized_vol = (daily_wide
    .groupby("year_month")["long_short"]
    .std()
    .reset_index()
    .rename(columns={"long_short": "realized_vol"})
)
realized_vol["date"] = realized_vol["year_month"].dt.to_timestamp()
realized_vol["realized_vol_ann"] = realized_vol["realized_vol"] *
 ↪  np.sqrt(252)

plot_rvol = (
    ggplot(realized_vol, aes(x="date", y="realized_vol_ann")) +
    geom_line(color="#d62728", size=0.8) +
    scale_y_continuous(labels=percent_format()) +
    labs(
        x="", y="Annualized Realized Volatility",
        title="Realized Volatility of Momentum Strategy"
    ) +
    theme_minimal() +
```

```
    theme(figure_size=(10, 5))
)
plot_rvol
```

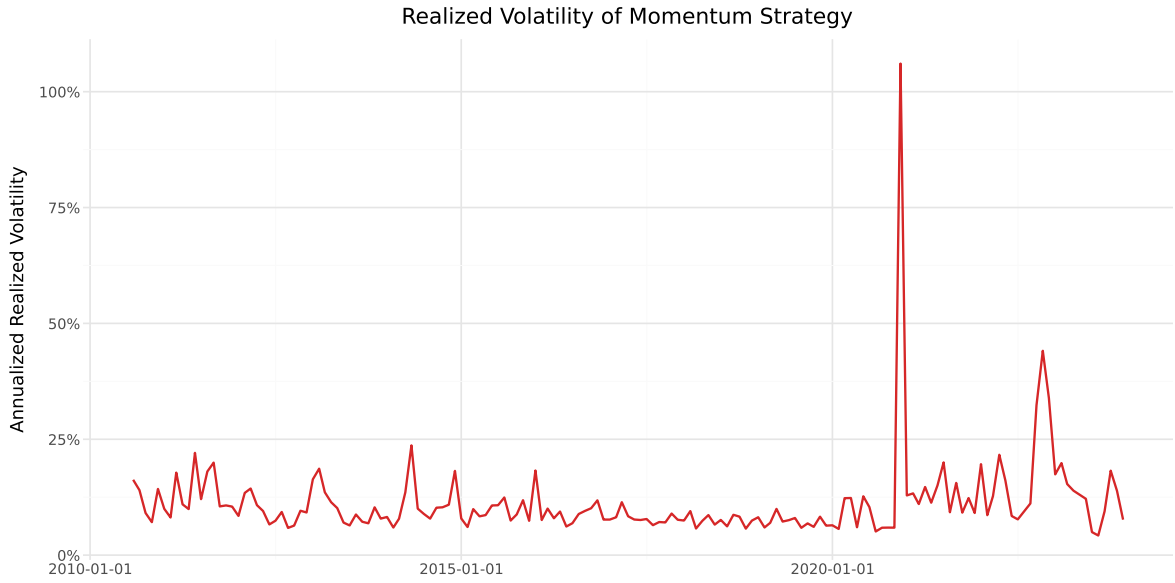


Figure 18.11: Monthly realized volatility of the momentum long-short strategy, computed from daily returns within each month. Higher values indicate periods of greater uncertainty in momentum profits, often coinciding with market stress.

18.12 Saving Results to the Database

We save the momentum portfolio returns to our database for use in subsequent chapters, including factor model construction and portfolio optimization.

```
# Save monthly equally weighted momentum portfolio returns
ewret_to_save = ewret[["date", "momr", "ewret"]].copy()
ewret_to_save.to_sql(
    name="momentum_portfolios_monthly",
    con=tidy_finance,
    if_exists="replace",
    index=False
)
print(f"Saved {len(ewret_to_save):,} monthly momentum portfolio
 ↪  observations.")
```

```
# Save the long-short return series
momentum_factor = ewret_wide[["date", "long_short"]].dropna().copy()
momentum_factor = momentum_factor.rename(columns={"long_short": "wml"})
momentum_factor.to_sql(
    name="momentum_factor_monthly",
    con=tidy_finance,
    if_exists="replace",
    index=False
)
print(f"Saved {len(momentum_factor):,} monthly WML factor observations.")

# Save the daily long-short return series
daily_momentum_factor = daily_wide[["date", "long_short"]].dropna().copy()
daily_momentum_factor = daily_momentum_factor.rename(
    columns={"long_short": "wml"}
)
daily_momentum_factor.to_sql(
    name="momentum_factor_daily",
    con=tidy_finance,
    if_exists="replace",
    index=False
)
print(f"Saved {len(daily_momentum_factor):,} daily WML factor observations.")
```

```
Saved 1,610 monthly momentum portfolio observations.
Saved 161 monthly WML factor observations.
Saved 3,352 daily WML factor observations.
```

18.13 Practical Considerations

18.13.1 Transaction Costs

Momentum strategies involve substantial portfolio turnover, as stocks enter and exit the extreme decile portfolios each month. Korajczyk and Sadka (2004) examine whether momentum profits survive transaction costs and find that profitability declines significantly for large institutional investors, though smaller portfolios can still capture meaningful returns.

In the Vietnamese market, transaction costs include:

- **Brokerage commissions**: Typically 0.15%–0.25% of transaction value for institutional investors.
- **Exchange fees**: Approximately 0.03% per trade.
- **Market impact**: Particularly relevant for smaller, less liquid stocks that dominate the extreme momentum portfolios. Vietnam's lower liquidity compared to developed markets may amplify this cost.
- **Trading band limits**: The $\pm 7\%$ daily price limit on HOSE can prevent immediate execution of trades, introducing tracking error relative to the theoretical portfolio.

18.13.2 Implementation Lag

Our baseline implementation assumes that portfolios can be formed and rebalanced instantaneously at the end of each month. In practice, there is a lag between observing the formation period returns and executing the portfolio trades. The one-month gap between the formation period and the start of the holding period (Jegadeesh (1990)) partially addresses this concern, but practitioners should consider additional implementation delays.

18.13.3 Survivorship Bias

Our dataset from DataCore includes both active and delisted stocks, which mitigates survivorship bias. However, the treatment of delisted stocks can affect momentum results. Stocks that are delisted during the holding period may generate extreme returns (both positive for acquisitions and negative for failures). We retain delisted returns as reported in the database, which is consistent with the treatment in Jegadeesh and Titman (1993).

18.13.4 Small Sample Considerations

The Vietnamese stock market has a relatively short history compared to the U.S. market studied in Jegadeesh and Titman (1993). Our sample spans approximately two decades, compared to the nearly three decades in the original study. This shorter sample period implies wider confidence intervals and greater sensitivity to specific episodes (such as the 2007–2009 financial crisis, which had a severe impact on Vietnamese equities). Results should be interpreted with this caveat in mind.

18.14 Key Takeaways

This chapter has provided an implementation and analysis of momentum strategies in the Vietnamese equity market, following the methodology of Jegadeesh and Titman (1993). The main findings and methodological contributions are:

1. **Methodology**: We implemented the full Jegadeesh and Titman (1993) overlapping portfolio methodology, including the two-stage averaging procedure (within-cohort, then across-cohort) that handles the K active portfolios in each month.

2. **Baseline results**: The $J = 6$, $K = 6$ strategy provides a natural benchmark. The spread between winner and loser portfolios reveals whether cross-sectional momentum exists in Vietnamese equities.

3. **Robustness across horizons**: By computing the full $J \times K$ grid with $J, K \in \{3, 6, 9, 12\}$, we assessed whether the momentum premium is robust to the choice of formation and holding periods, following the approach in Jegadeesh and Titman (1993) Table 1.

4. **Risk adjustment**: CAPM and Fama-French three-factor alphas measure whether the momentum premium is explained by standard risk factors, building on the analysis in Eugene F. Fama and French (1996) who show that their three-factor model fails to explain momentum.

5. **Market state dependence**: Following Cooper, Gutierrez Jr, and Hameed (2004), we examined whether momentum profits vary with market conditions, which is particularly relevant in emerging markets with pronounced boom-bust cycles.

6. **Value weighting**: The comparison of equally weighted and value-weighted strategies addresses practical implementability and isolates the role of firm size in driving momentum profits.

7. **Daily analysis**: By tracking momentum portfolios at the daily frequency, we computed precise risk metrics including realized volatility, maximum drawdown, VaR, and CVaR, providing a complete risk profile of the strategy.

8. **Emerging market context**: Throughout the analysis, we have highlighted features specific to the Vietnamese market—trading band limits, foreign ownership restrictions, shorter sample history, and higher transaction costs—that affect the interpretation and practical viability of momentum strategies.

19 Factor Construction Principles

> **Note**
>
> In this chapter, we develop a general-purpose factor construction engine for the Vietnamese equity market. We cover every methodological decision in the pipeline, including universe definition, breakpoint computation, portfolio formation, weighting, rebalancing, and factor return calculation, and demonstrate how each choice affects the resulting factor.

The previous chapters introduced specific asset pricing models, including the CAPM, the Fama-French three-factor model, and momentum. Each of those chapters presented its factor as given. This chapter steps behind the curtain and addresses the engineering question: *how exactly do you build a factor?* The question matters because seemingly minor methodological decisions, such as where to set breakpoints, whether to value-weight or equal-weight, how to handle missing accounting data, which stocks to exclude, can alter the magnitude, statistical significance, and even the sign of a factor premium.

In the U.S. context, Eugene F. Fama and French (1993b) established a canonical procedure: sort stocks independently on size and a characteristic, form six value-weighted portfolios from 2×3 intersections, and define the factor as the average return of the two high-characteristic portfolios minus the average return of the two low-characteristic portfolios. This procedure has been replicated thousands of times. But it was designed for the U.S. market circa 1990, with its deep liquidity, broad cross-section, and CRSP/Compustat data infrastructure. Applying it mechanically to Vietnam, a market with 700 listed stocks, extreme illiquidity in the bottom tercile, high concentration in the top decile, and accounting data that arrives with variable lags, requires careful adaptation.

Hou, Xue, and Zhang (2020a) replicated 452 anomalies from the U.S. literature and found that over half fail to replicate even in U.S. data with minor methodological variations. The replication crisis in empirical asset pricing makes it essential that researchers understand and document every construction choice. This chapter provides the tools to do so transparently.

19.1 The Factor Construction Pipeline

Every tradeable factor follows the same logical pipeline:

1. Define the universe: Select the eligible securities (e.g., common stocks with liquidity and size filters).
2. Compute the signal: Calculate the characteristic of interest (e.g., value, momentum, profitability).
3. Set breakpoints: Determine how stocks will be sorted (e.g., median, quintiles, deciles).
4. Assign portfolios: Group stocks into high and low (or multiple) portfolios based on the signal.
5. Compute returns: Calculate portfolio returns (equal- or value-weighted).
6. Construct the factor: Take Long (high) - Short (low).
7. Validate: Test performance, significance, and robustness.

Each step involves choices that interact with each other. A breakpoint that works well for a liquid universe may be inappropriate for the full cross-section. A weighting scheme that reduces noise in the U.S. may amplify it in Vietnam. We address each step systematically.

19.2 Data Construction

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from scipy import stats
from itertools import product
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.figsize': (12, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

```
from datacore import DataCoreClient

client = DataCoreClient()
```

```
# Monthly returns (survivorship-bias-free)
monthly = client.get_monthly_returns(
    exchanges=['HOSE', 'HNX'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    include_delisted=True,
    fields=[
        'ticker', 'month_end', 'monthly_return', 'market_cap',
        'shares_outstanding', 'volume_avg_20d', 'turnover_value_avg_20d',
        'n_zero_volume_days', 'exchange'
    ]
)

# Annual accounting data
accounting = client.get_fundamentals(
    exchanges=['HOSE', 'HNX'],
    start_date='2006-01-01',
    end_date='2024-12-31',
    include_delisted=True,
    frequency='annual',
    fields=[
        'ticker', 'fiscal_year', 'filing_date',
        'total_assets', 'total_equity', 'book_equity',
        'net_income', 'revenue', 'gross_profit',
        'operating_profit', 'total_debt', 'retained_earnings',
        'dividends_paid', 'capex', 'depreciation',
        'shares_outstanding_fy'
    ]
)

# Daily prices for momentum and volatility signals
daily = client.get_daily_prices(
    exchanges=['HOSE', 'HNX'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    include_delisted=True,
    fields=['ticker', 'date', 'adjusted_close', 'volume', 'turnover_value']
)

monthly['month_end'] = pd.to_datetime(monthly['month_end'])
monthly = monthly.sort_values(['ticker', 'month_end'])

print(f"Monthly returns: {len(monthly):,} firm-months")
```

```
print(f"Accounting: {len(accounting):,} firm-years")
print(f"Unique tickers: {monthly['ticker'].nunique()}")
```

19.3 Step 1: Universe Definition

The first and most consequential choice is which stocks enter the factor construction universe. The universe definition determines what population the factor premium describes and whether it is implementable.

19.3.1 The Universe Problem in Vietnam

Vietnam presents a specific challenge: the cross-section is small (600-800 stocks on HOSE and HNX combined), and the size distribution is extremely skewed. The top 10 stocks by market capitalization account for roughly 50% of the total market cap on HOSE. The bottom tercile consists of micro-cap stocks that often trade fewer than 5 days per month. Including these stocks inflates apparent factor premia because their prices are noisy and stale, but excluding them shrinks the already small cross-section.

```
def apply_universe_filters(df, filters='standard'):
    """
    Apply universe filters to the monthly return panel.

    Parameters
    ----------
    filters : str
        'none': all stocks
        'minimal': exclude zero market cap and extreme returns
        'standard': + minimum listing age + positive volume
        'strict': + minimum market cap + minimum turnover

    Returns
    -------
    Filtered DataFrame with 'in_universe' column
    """
    d = df.copy()
    d['in_universe'] = True

    # Always: remove missing returns and market cap
    d.loc[d['monthly_return'].isna(), 'in_universe'] = False
    d.loc[d['market_cap'].isna() | (d['market_cap'] <= 0), 'in_universe'] =
↪   False
```

```
    # Minimal: winsorize extreme returns (likely data errors)
    if filters in ['minimal', 'standard', 'strict']:
        d.loc[d['monthly_return'].abs() > 1.0, 'in_universe'] = False

    # Standard: listing age >= 6 months
    if filters in ['standard', 'strict']:
        d['listing_age'] = (
            d.groupby('ticker').cumcount() + 1
        )
        d.loc[d['listing_age'] < 6, 'in_universe'] = False

        # Require at least 10 positive-volume days in the month
        d.loc[d['n_zero_volume_days'] > 12, 'in_universe'] = False

    # Strict: minimum market cap (20th percentile of HOSE)
    if filters == 'strict':
        mcap_threshold = (
            d[d['exchange'] == 'HOSE']
            .groupby('month_end')['market_cap']
            .transform(lambda x: x.quantile(0.20))
        )
        # Apply HOSE threshold to all stocks
        d['mcap_threshold'] = (
            d.groupby('month_end')['market_cap']
            .transform(lambda x: x.quantile(0.20))
        )
        d.loc[d['market_cap'] < d['mcap_threshold'], 'in_universe'] = False

        # Minimum average daily turnover (VND 200 million)
        d.loc[d['turnover_value_avg_20d'] < 2e8, 'in_universe'] = False

    return d

# Apply all filter levels and compare
filter_summary = {}
for level in ['none', 'minimal', 'standard', 'strict']:
    filtered = apply_universe_filters(monthly, filters=level)
    in_univ = filtered[filtered['in_universe']]
    filter_summary[level] = {
        'Firm-months': len(in_univ),
        'Avg stocks/month':
        ↪  in_univ.groupby('month_end')['ticker'].nunique().mean(),
```

```
        'Avg MCap coverage (%)': (
            in_univ.groupby('month_end')['market_cap'].sum()
            / filtered.groupby('month_end')['market_cap'].sum()
        ).mean() * 100
    }

filter_df = pd.DataFrame(filter_summary).T
print("Universe Filter Effects:")
print(filter_df.round(1).to_string())
```

19.4 Step 2: Signal Construction

19.4.1 Point-in-Time Accounting Data

As discussed in the missing data chapter, accounting signals must be aligned with their public availability date to avoid look-ahead bias. We implement a general-purpose point-in-time merge:

```
def pit_merge_accounting(monthly_df, accounting_df, lag_months=4):
    """
    Merge accounting data with monthly returns respecting
    the point-in-time availability constraint.

    Vietnamese annual reports are due within 90 days of fiscal
    year-end. We use a conservative 4-month lag.

    Parameters
    ----------
    lag_months : int
        Number of months after fiscal year-end before data
        are assumed to be publicly available.
    """
    acc = accounting_df.copy()

    # Accounting data becomes available lag_months after FY end
    # If filing_date is available, use it; otherwise use FY-end + lag
    if 'filing_date' in acc.columns:
        acc['filing_date'] = pd.to_datetime(acc['filing_date'])
        acc['available_date'] = acc['filing_date']
        # Fallback for missing filing dates
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

colors_filter = {
    'none': '#BDC3C7', 'minimal': '#3498DB',
    'standard': '#2C5F8A', 'strict': '#C0392B'
}

for level in ['none', 'minimal', 'standard', 'strict']:
    filtered = apply_universe_filters(monthly, filters=level)
    counts = (
        filtered[filtered['in_universe']]
        .groupby('month_end')['ticker']
        .nunique()
    )
    axes[0].plot(counts.index, counts.values,
                 color=colors_filter[level], linewidth=1.5, label=level)

axes[0].set_ylabel('Number of Stocks')
axes[0].set_title('Panel A: Universe Size')
axes[0].legend()

# Panel B: Market cap coverage
for level in ['minimal', 'standard', 'strict']:
    filtered = apply_universe_filters(monthly, filters=level)
    total_mcap = filtered.groupby('month_end')['market_cap'].sum()
    filtered_mcap = (
        filtered[filtered['in_universe']]
        .groupby('month_end')['market_cap']
        .sum()
    )
    coverage = (filtered_mcap / total_mcap * 100).dropna()
    axes[1].plot(coverage.index, coverage.values,
                 color=colors_filter[level], linewidth=1.5, label=level)

axes[1].set_ylabel('Market Cap Coverage (%)')
axes[1].set_title('Panel B: Market Capitalization Coverage')
axes[1].legend()
axes[1].set_ylim([60, 102])

plt.tight_layout()
plt.show()
```

Figure 19.1

```
        acc['fy_end'] = pd.to_datetime(
            acc['fiscal_year'].astype(str) + '-12-31'
        )
        acc['available_date'] = acc['available_date'].fillna(
            acc['fy_end'] + pd.DateOffset(months=lag_months)
        )
    else:
        acc['available_date'] = pd.to_datetime(
            acc['fiscal_year'].astype(str) + '-12-31'
        ) + pd.DateOffset(months=lag_months)

    # For each firm-month, find the most recent available accounting data
    merged = monthly_df.copy()

    # Efficient approach: for June rebalancing, use FY t-1 data
    # which is available by April of year t (4-month lag)
    merged['year'] = merged['month_end'].dt.year
    merged['month'] = merged['month_end'].dt.month

    # Map: if month >= (lag_months + 1), use current year's FY-1 data
    # Otherwise, use FY-2 data
    merged['data_fy'] = np.where(
        merged['month'] >= lag_months + 1,
        merged['year'] - 1,
        merged['year'] - 2
    )

    # Merge
    acc_cols = [c for c in acc.columns if c not in
                ['filing_date', 'available_date', 'fy_end']]
    merged = merged.merge(
        acc[acc_cols].rename(columns={'fiscal_year': 'data_fy'}),
        on=['ticker', 'data_fy'],
        how='left'
    )

    return merged

# Apply point-in-time merge
panel = pit_merge_accounting(monthly, accounting, lag_months=4)

# Construct common signals
panel['log_mcap'] = np.log(panel['market_cap'].clip(lower=1))
```

```
# Book-to-market
panel['bm'] = panel['book_equity'] / panel['market_cap']
panel.loc[panel['bm'] <= 0, 'bm'] = np.nan  # Negative BE firms

# Gross profitability (Novy-Marx 2013)
panel['gp_at'] = panel['gross_profit'] / panel['total_assets']

# Operating profitability (Fama-French 2015)
panel['op'] = panel['operating_profit'] / panel['book_equity']

# Investment (asset growth)
panel['investment'] = (
    panel.groupby('ticker')['total_assets']
    .pct_change(periods=1)
)

# Leverage
panel['leverage'] = panel['total_debt'] / panel['total_assets']

print("Signal Coverage:")
for sig in ['bm', 'gp_at', 'op', 'investment', 'leverage']:
    pct = panel[sig].notna().mean()
    print(f"  {sig:<15}: {pct:.1%}")
```

19.4.2 Momentum and Volatility Signals

Price-based signals require return history, not accounting data, so they have different timing requirements.

```
# Past returns for momentum signals
panel = panel.sort_values(['ticker', 'month_end'])

# Momentum: cumulative return from month t-12 to t-2 (skip most recent month)
panel['ret_12_2'] = (
    panel.groupby('ticker')['monthly_return']
    .transform(lambda x: x.shift(2).rolling(11).apply(
        lambda r: (1 + r).prod() - 1, raw=True))
)

# Short-term reversal: month t-1 return
```

```
panel['ret_1'] = panel.groupby('ticker')['monthly_return'].shift(1)

# Idiosyncratic volatility (from daily data, rolling 60 days)
daily['date'] = pd.to_datetime(daily['date'])
daily['daily_return'] =
 ↪  daily.groupby('ticker')['adjusted_close'].pct_change()

ivol = (
    daily.groupby('ticker')
    .apply(lambda g: g.set_index('date')['daily_return']
           .rolling(60, min_periods=40).std() * np.sqrt(252))
    .reset_index(name='ivol')
)
ivol['month_end'] = ivol['date'].dt.to_period('M').dt.to_timestamp('M')
ivol_monthly = (
    ivol.groupby(['ticker', 'month_end'])['ivol']
    .last()
    .reset_index()
)

panel = panel.merge(ivol_monthly, on=['ticker', 'month_end'], how='left')

print("Price Signal Coverage:")
for sig in ['ret_12_2', 'ret_1', 'ivol']:
    pct = panel[sig].notna().mean()
    print(f"  {sig:<15}: {pct:.1%}")
```

19.5 Step 3: Breakpoint Computation

19.5.1 The Breakpoint Decision

Breakpoints determine which stocks are "high" versus "low" on a given characteristic. The two key choices are:

1. **Breakpoint universe:** Should breakpoints be computed from all stocks or from a subset (e.g., HOSE only)?
2. **Number of groups:** 2×3 (Fama-French standard), 5×5 (for finer sorts), or independent terciles/quintiles?

Eugene F. Fama and French (1993b) use NYSE breakpoints for U.S. sorts because this prevents the large number of small Nasdaq/AMEX stocks from dominating the breakpoint distribution.

The analog in Vietnam is to use HOSE breakpoints, since HOSE lists the larger, more liquid firms and HNX lists smaller firms. Using all-stock breakpoints would place most HOSE stocks in the upper size groups and most HNX stocks in the lower groups, producing mechanically different results.

```
def compute_breakpoints(df, signal_col, n_groups, bp_universe='hose',
                        exchange_col='exchange'):
    """
    Compute cross-sectional breakpoints for portfolio sorting.

    Parameters
    ----------
    bp_universe : str
        'all': use all stocks in universe
        'hose': use only HOSE stocks (analogous to NYSE breakpoints)
    n_groups : int
        Number of groups (2, 3, 5, or 10)

    Returns
    -------
    Series of breakpoints (quantiles)
    """
    signal = df[signal_col].dropna()

    if bp_universe == 'hose':
        mask = df[exchange_col] == 'HOSE'
        signal = df.loc[mask, signal_col].dropna()

    quantiles = np.linspace(0, 1, n_groups + 1)[1:-1]
    breakpoints = signal.quantile(quantiles)

    return breakpoints

# Example: compare HOSE vs all-stock breakpoints for book-to-market
example_month = panel[panel['month_end'] == '2023-06-30'].copy()
example_month = example_month[example_month['bm'].notna()]

bp_hose = compute_breakpoints(example_month, 'bm', 3, bp_universe='hose')
bp_all = compute_breakpoints(example_month, 'bm', 3, bp_universe='all')

print("BM Tercile Breakpoints (June 2023):")
print(f"  HOSE-only: {bp_hose.values.round(3)}")
print(f"  All stocks: {bp_all.values.round(3)}")
```

```
print(f"\n  Difference: HOSE breakpoints are "
      f"{'higher' if bp_hose.values[0] > bp_all.values[0] else 'lower'} "
      f"than all-stock breakpoints")
```

19.6 Step 4: Portfolio Formation

19.6.1 The Generic Factor Engine

We implement a general-purpose factor construction function that takes any signal column and produces a long-short factor return series. The function encapsulates all methodological choices as parameters, making it easy to test sensitivity.

```
def construct_factor(
    panel_df,
    signal_col,
    size_col='market_cap',
    return_col='monthly_return',
    date_col='month_end',
    exchange_col='exchange',
    formation_month=6,
    rebalance_freq='annual',
    n_signal_groups=3,
    n_size_groups=2,
    weighting='value',
    bp_universe='hose',
    independent_sorts=True,
    long_group='high',
    min_stocks_per_portfolio=5,
    signal_lag=0,
    universe_filter='standard'
):
    """
    Construct a tradeable factor following the Fama-French methodology.

    Parameters
    ----------
    signal_col : str
        Column with the sorting variable.
    formation_month : int
        Month of year for portfolio formation (6 = June for FF).
```

```
# Compute breakpoints for every month
bp_comparison = []
for month, group in panel.dropna(subset=['bm']).groupby('month_end'):
    bp_h = compute_breakpoints(group, 'bm', 3, bp_universe='hose')
    bp_a = compute_breakpoints(group, 'bm', 3, bp_universe='all')

    # Count stocks in each tercile under each rule
    for bp_name, bp_vals in [('HOSE', bp_h), ('All', bp_a)]:
        low = (group['bm'] <= bp_vals.iloc[0]).sum()
        mid = ((group['bm'] > bp_vals.iloc[0]) &
               (group['bm'] <= bp_vals.iloc[1])).sum()
        high = (group['bm'] > bp_vals.iloc[1]).sum()
        bp_comparison.append({
            'month_end': month, 'bp_rule': bp_name,
            'median_bp': bp_vals.iloc[0],
            'n_low': low, 'n_mid': mid, 'n_high': high
        })

bp_df = pd.DataFrame(bp_comparison)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Median breakpoint over time
for rule, color in [('HOSE', '#2C5F8A'), ('All', '#C0392B')]:
    subset = bp_df[bp_df['bp_rule'] == rule]
    axes[0].plot(subset['month_end'], subset['median_bp'],
                 color=color, linewidth=1.5, label=f'{rule} breakpoints')
axes[0].set_ylabel('Lower Tercile Breakpoint (BM)')
axes[0].set_title('Panel A: Breakpoint Time Series')
axes[0].legend()

# Panel B: Number of stocks in high BM group
for rule, color in [('HOSE', '#2C5F8A'), ('All', '#C0392B')]:
    subset = bp_df[bp_df['bp_rule'] == rule]
    axes[1].plot(subset['month_end'], subset['n_high'],
                 color=color, linewidth=1.5, label=f'{rule} breakpoints')
axes[1].set_ylabel('Stocks in High BM Group')
axes[1].set_title('Panel B: High BM Portfolio Size')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Figure 19.2

```
    rebalance_freq : str
        'annual' (FF standard), 'semi', 'quarterly', 'monthly'.
    n_signal_groups : int
        Number of signal groups (3 for FF standard, 5 for quintiles).
    n_size_groups : int
        Number of size groups (2 for FF standard).
    weighting : str
        'value' (VW) or 'equal' (EW).
    bp_universe : str
        'hose' or 'all' for breakpoint computation.
    independent_sorts : bool
        True for independent double sorts (FF standard).
    long_group : str
        'high' or 'low'-which signal group is the long leg.
    signal_lag : int
        Additional months to lag the signal beyond the
        standard point-in-time alignment.

    Returns
    -------
    Dictionary with 'factor_returns', 'portfolio_returns',
    'diagnostics'.
    """
    df = panel_df.copy()

    # Apply universe filter
    df = apply_universe_filters(df, filters=universe_filter)
    df = df[df['in_universe']].copy()

    # Lag the signal if requested
    if signal_lag > 0:
        df[signal_col] = (
            df.groupby('ticker')[signal_col].shift(signal_lag)
        )

    # Determine formation dates
    if rebalance_freq == 'annual':
        # Form portfolios in formation_month, hold for 12 months
        df['formation_date'] = df[date_col].apply(
            lambda d: pd.Timestamp(
                year=d.year if d.month >= formation_month else d.year - 1,
                month=formation_month, day=30
            )
```

```
    )
elif rebalance_freq == 'monthly':
    df['formation_date'] = df[date_col] - pd.DateOffset(months=1)
elif rebalance_freq == 'quarterly':
    df['formation_date'] = df[date_col].apply(
        lambda d: pd.Timestamp(
            year=d.year,
            month=((d.month - 1) // 3) * 3 + 1,
            day=1
        ) - pd.DateOffset(days=1)
    )

# Assign signal and size groups at each formation date
all_portfolios = []

formation_dates = sorted(df['formation_date'].unique())

for f_date in formation_dates:
    # Stocks available at formation
    formation_data = df[df['formation_date'] == f_date].copy()

    # Get signal values at formation
    available = formation_data.dropna(subset=[signal_col, size_col])
    if len(available) < min_stocks_per_portfolio * n_signal_groups *
     ↪  n_size_groups:
        continue

    # Compute breakpoints
    size_bp = compute_breakpoints(
        available, size_col, n_size_groups, bp_universe
    )
    signal_bp = compute_breakpoints(
        available, signal_col, n_signal_groups, bp_universe
    )

    # Assign groups
    available['size_group'] = np.searchsorted(
        size_bp.values, available[size_col].values
    )
    available['signal_group'] = np.searchsorted(
        signal_bp.values, available[signal_col].values
    )
```

```
        all_portfolios.append(available)

    if not all_portfolios:
        return None

    portfolios = pd.concat(all_portfolios, ignore_index=True)

    # Compute portfolio returns
    def weighted_return(group):
        if weighting == 'value':
            if group[size_col].sum() > 0:
                return np.average(group[return_col], weights=group[size_col])
            else:
                return group[return_col].mean()
        else:
            return group[return_col].mean()

    port_returns = (
        portfolios
        .groupby([date_col, 'size_group', 'signal_group'])
        .apply(weighted_return)
        .reset_index(name='port_return')
    )

    # Construct factor: average of high-signal portfolios minus
    # average of low-signal portfolios (across size groups)
    high_label = n_signal_groups - 1 if long_group == 'high' else 0
    low_label = 0 if long_group == 'high' else n_signal_groups - 1

    high_ports = port_returns[port_returns['signal_group'] == high_label]
    low_ports = port_returns[port_returns['signal_group'] == low_label]

    high_avg = high_ports.groupby(date_col)['port_return'].mean()
    low_avg = low_ports.groupby(date_col)['port_return'].mean()

    factor_returns = (high_avg - low_avg).to_frame('factor_return')

    # Diagnostics
    port_counts = (
        portfolios
        .groupby([date_col, 'size_group', 'signal_group'])['ticker']
        .nunique()
        .reset_index(name='n_stocks')
```

```
    )

    diagnostics = {
        'avg_stocks_per_portfolio': port_counts['n_stocks'].mean(),
        'min_stocks_per_portfolio': port_counts['n_stocks'].min(),
        'ann_return': factor_returns['factor_return'].mean() * 12,
        'ann_vol': factor_returns['factor_return'].std() * np.sqrt(12),
        'sharpe': (factor_returns['factor_return'].mean()
                   / factor_returns['factor_return'].std() * np.sqrt(12)),
        't_stat': (factor_returns['factor_return'].mean()
                   / (factor_returns['factor_return'].std()
                      / np.sqrt(len(factor_returns)))),
        'n_months': len(factor_returns)
    }

    return {
        'factor_returns': factor_returns,
        'portfolio_returns': port_returns,
        'portfolios': portfolios,
        'diagnostics': diagnostics
    }
```

19.6.2 Building the Core Factors

We now use the engine to construct the standard Fama-French factors for Vietnam:

```
# SMB (Size): small minus big
# Signal = market cap; long_group = 'low' (small stocks)
smb_result = construct_factor(
    panel, signal_col='log_mcap', long_group='low',
    formation_month=6, rebalance_freq='annual',
    n_signal_groups=2, n_size_groups=1,  # No double sort for size itself
    weighting='value', bp_universe='hose'
)

# HML (Value): high BM minus low BM
hml_result = construct_factor(
    panel, signal_col='bm', long_group='high',
    formation_month=6, rebalance_freq='annual',
    n_signal_groups=3, n_size_groups=2,
    weighting='value', bp_universe='hose'
```

```
)

# RMW (Profitability): robust minus weak
rmw_result = construct_factor(
    panel, signal_col='op', long_group='high',
    formation_month=6, rebalance_freq='annual',
    n_signal_groups=3, n_size_groups=2,
    weighting='value', bp_universe='hose'
)

# CMA (Investment): conservative minus aggressive
cma_result = construct_factor(
    panel, signal_col='investment', long_group='low',
    formation_month=6, rebalance_freq='annual',
    n_signal_groups=3, n_size_groups=2,
    weighting='value', bp_universe='hose'
)

# WML (Momentum): winners minus losers
wml_result = construct_factor(
    panel, signal_col='ret_12_2', long_group='high',
    formation_month=6, rebalance_freq='monthly',
    n_signal_groups=3, n_size_groups=2,
    weighting='value', bp_universe='hose'
)

# Summary table
print("Vietnamese Factor Summary:")
print(f"{'Factor':<8} {'Ann. Ret':>10} {'Ann. Vol':>10} {'Sharpe':>8} "
      f"{'t-stat':>8} {'Avg N':>8}")
print("-" * 54)
for name, result in [('SMB', smb_result), ('HML', hml_result),
                     ('RMW', rmw_result), ('CMA', cma_result),
                     ('WML', wml_result)]:
    if result is None:
        continue
    d = result['diagnostics']
    print(f"{name:<8} {d['ann_return']:>10.4f} {d['ann_vol']:>10.4f} "
          f"{d['sharpe']:>8.2f} {d['t_stat']:>8.2f} "
          f"{d['avg_stocks_per_portfolio']:>8.1f}")
```

```
fig, ax = plt.subplots(figsize=(14, 6))

factor_colors = {
    'SMB': '#2C5F8A', 'HML': '#C0392B', 'RMW': '#27AE60',
    'CMA': '#E67E22', 'WML': '#8E44AD'
}

for name, result in [('SMB', smb_result), ('HML', hml_result),
                     ('RMW', rmw_result), ('CMA', cma_result),
                     ('WML', wml_result)]:
    if result is None:
        continue
    fr = result['factor_returns']
    cum = (1 + fr['factor_return']).cumprod()
    ax.plot(cum.index, cum.values, color=factor_colors[name],
            linewidth=2, label=name)

ax.axhline(y=1, color='gray', linewidth=0.5)
ax.set_ylabel('Cumulative Return')
ax.set_xlabel('Date')
ax.set_title('Vietnamese Factor Cumulative Returns (2×3 VW, HOSE
 ↪  Breakpoints)')
ax.legend(ncol=5)
ax.set_yscale('log')
plt.tight_layout()
plt.show()
```

Figure 19.3

19.7 Step 5: Sensitivity to Construction Choices

The most important lesson in factor construction is that the resulting factor premium is *not* uniquely determined by the economic hypothesis; it depends substantially on implementation choices. We systematically vary each choice and examine how the factor changes.

The choices varied below are the ones researchers usually think of as choices: weighting, breakpoints, the number of groups, rebalancing frequency. The benchmark subtracted to form excess returns is rarely on that list, yet Binsbergen, Ma, and Schwert (2026) show it belongs there. Replacing the short rate with a duration-matched government bond portfolio changes the sign or significance of a substantial minority of published U.S. anomalies without touching the sorting variable at all. We do not implement that adjustment here, but it is worth keeping in mind that the sensitivity table produced at the end of this section understates the full multiverse of defensible specifications.

19.7.1 Weighting: Value-Weighted vs. Equal-Weighted

```
sensitivity_results = {}

for name, signal, long_grp in [
    ('HML', 'bm', 'high'), ('RMW', 'op', 'high'), ('WML', 'ret_12_2', 'high')
]:
    for wt in ['value', 'equal']:
        result = construct_factor(
            panel, signal_col=signal, long_group=long_grp,
            weighting=wt, bp_universe='hose',
            rebalance_freq='annual' if name != 'WML' else 'monthly'
        )
        if result:
            d = result['diagnostics']
            sensitivity_results[f"{name}_{wt}"] = {
                'Factor': name, 'Weighting': wt,
                'Ann. Return': d['ann_return'],
                't-stat': d['t_stat']
            }

sens_df = pd.DataFrame(sensitivity_results).T
print("VW vs EW Factor Returns:")
print(sens_df.round(3).to_string())
```

19.7.2 Breakpoint Universe: HOSE vs. All Stocks

```
for name, signal, long_grp in [
    ('HML', 'bm', 'high'), ('WML', 'ret_12_2', 'high')
]:
    for bp in ['hose', 'all']:
        result = construct_factor(
            panel, signal_col=signal, long_group=long_grp,
            bp_universe=bp, weighting='value',
            rebalance_freq='annual' if name != 'WML' else 'monthly'
        )
        if result:
            d = result['diagnostics']
            sensitivity_results[f"{name}_bp_{bp}"] = {
                'Factor': name, 'Breakpoints': bp,
                'Ann. Return': d['ann_return'],
                't-stat': d['t_stat'],
                'Avg N': d['avg_stocks_per_portfolio']
            }

bp_sens = pd.DataFrame({k: v for k, v in sensitivity_results.items()
                        if 'bp_' in k}).T
print("\nBreakpoint Universe Sensitivity:")
print(bp_sens.round(3).to_string())
```

19.7.3 Number of Groups: 2×3 vs. 5×5 vs. Deciles

```
group_configs = [
    (2, 3, '2x3 (FF standard)'),
    (2, 5, '2x5 (quintiles)'),
    (1, 10, '1x10 (deciles)')
]

group_results = {}
for n_size, n_signal, label in group_configs:
    result = construct_factor(
        panel, signal_col='bm', long_group='high',
        n_size_groups=n_size, n_signal_groups=n_signal,
        weighting='value', bp_universe='hose',
        rebalance_freq='annual'
```

```
    )
    if result:
        d = result['diagnostics']
        group_results[label] = {
            'Ann. Return': d['ann_return'],
            't-stat': d['t_stat'],
            'Avg N per port': d['avg_stocks_per_portfolio'],
            'Min N per port': d['min_stocks_per_portfolio']
        }

print("HML: Sorting Granularity Sensitivity:")
print(pd.DataFrame(group_results).T.round(3).to_string())
```

19.7.4 Rebalancing Frequency

```
rebal_results = {}
for freq in ['annual', 'quarterly', 'monthly']:
    result = construct_factor(
        panel, signal_col='bm', long_group='high',
        rebalance_freq=freq, weighting='value', bp_universe='hose'
    )
    if result:
        d = result['diagnostics']
        rebal_results[freq] = {
            'Ann. Return': d['ann_return'],
            'Ann. Vol': d['ann_vol'],
            't-stat': d['t_stat']
        }

print("HML: Rebalancing Frequency Sensitivity:")
print(pd.DataFrame(rebal_results).T.round(4).to_string())
```

19.7.5 Comprehensive Sensitivity Summary

19.8 Factor Correlation Structure

```
# Systematic grid search for HML
configs = list(product(
    ['value', 'equal'],        # Weighting
    ['hose', 'all'],           # Breakpoint universe
    [(2, 3), (2, 5), (1, 10)]  # (n_size, n_signal)
))

grid_results = []
for wt, bp, (ns, nsig) in configs:
    result = construct_factor(
        panel, signal_col='bm', long_group='high',
        weighting=wt, bp_universe=bp,
        n_size_groups=ns, n_signal_groups=nsig,
        rebalance_freq='annual'
    )
    if result:
        d = result['diagnostics']
        grid_results.append({
            'Weighting': 'VW' if wt == 'value' else 'EW',
            'Breakpoints': bp.upper(),
            'Sort': f'{ns}x{nsig}',
            'Ann. Return': d['ann_return'],
            't-stat': d['t_stat']
        })

grid_df = pd.DataFrame(grid_results)
print("HML Factor: Full Sensitivity Grid:")
print(grid_df.to_string(index=False))

# Pivot for heatmap
pivot = grid_df.pivot_table(
    values='Ann. Return', index=['Weighting', 'Breakpoints'],
    columns='Sort'
)

fig, ax = plt.subplots(figsize=(8, 5))
sns.heatmap(pivot * 100, annot=True, fmt='.1f', cmap='RdYlGn',
            center=0, linewidths=0.5, ax=ax,
            cbar_kws={'label': 'Ann. Return (%)'})
ax.set_title('HML Premium: Sensitivity to Construction Choices')
plt.tight_layout()
plt.show()
```

Figure 19.4

```
# Merge all factor return series
factor_panel = pd.DataFrame()
for name, result in [('SMB', smb_result), ('HML', hml_result),
                     ('RMW', rmw_result), ('CMA', cma_result),
                     ('WML', wml_result)]:
    if result is None:
        continue
    fr = result['factor_returns'].rename(columns={'factor_return': name})
    if factor_panel.empty:
        factor_panel = fr
    else:
        factor_panel = factor_panel.merge(fr, left_index=True,
                                          right_index=True, how='outer')

corr = factor_panel.corr()

fig, ax = plt.subplots(figsize=(7, 6))
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, vmin=-1, vmax=1, square=True,
            linewidths=0.5, ax=ax)
ax.set_title('Factor Return Correlations')
plt.tight_layout()
plt.show()
```

Figure 19.5

19.9 Factor Validation

A well-constructed factor should pass several diagnostic tests before being used in asset pricing research.

19.9.1 Diagnostic Checklist

```
def validate_factor(factor_result, name):
    """
    Run standard diagnostic tests on a constructed factor.
    """
    fr = factor_result['factor_returns']['factor_return']
    diag = factor_result['diagnostics']
    ports = factor_result['portfolios']

    tests = {}

    # 1. Statistical significance (t > 2)
    tests['t-stat'] = diag['t_stat']
    tests['t > 2'] = abs(diag['t_stat']) > 2.0

    # 2. Economic magnitude
    tests['Ann. Return'] = diag['ann_return']
    tests['Ann. Vol'] = diag['ann_vol']
    tests['Sharpe'] = diag['sharpe']

    # 3. Adequate portfolio diversification
    tests['Avg N per portfolio'] = diag['avg_stocks_per_portfolio']
    tests['Min N per portfolio'] = diag['min_stocks_per_portfolio']
    tests['Min N >= 5'] = diag['min_stocks_per_portfolio'] >= 5

    # 4. Not dominated by a single month
    tests['Max monthly return'] = fr.max()
    tests['Min monthly return'] = fr.min()
    tests['Fraction > 0'] = (fr > 0).mean()

    # 5. Persistence (ACF at lag 1)
    if len(fr) > 12:
        tests['ACF(1)'] = fr.autocorr(lag=1)

    # 6. Consistency across subperiods
```

```
    mid = len(fr) // 2
    first_half = fr.iloc[:mid]
    second_half = fr.iloc[mid:]
    tests['Return (1st half)'] = first_half.mean() * 12
    tests['Return (2nd half)'] = second_half.mean() * 12
    tests['Same sign both halves'] = (
        np.sign(first_half.mean()) == np.sign(second_half.mean())
    )

    return tests

print("Factor Validation Summary:")
print("=" * 70)
for name, result in [('SMB', smb_result), ('HML', hml_result),
                     ('RMW', rmw_result), ('CMA', cma_result),
                     ('WML', wml_result)]:
    if result is None:
        continue
    tests = validate_factor(result, name)
    print(f"\n{name}:")
    for test, value in tests.items():
        if isinstance(value, bool):
            status = 'PASS' if value else 'FAIL'
            print(f"  {test:<30}: {status}")
        elif isinstance(value, float):
            print(f"  {test:<30}: {value:.4f}")
        else:
            print(f"  {test:<30}: {value}")
```

19.9.2 Monotonicity Test

A factor built from a characteristic sort should produce monotonically increasing (or decreasing) average returns across quantiles. Violations of monotonicity suggest the signal-return relationship is nonlinear or absent.

19.9.3 Spanning Tests

Does a new factor add information beyond existing factors? We test this by regressing each factor on all other factors and examining the intercept (alpha). A significant alpha means the factor captures return variation not spanned by the others:

```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.flatten()

signals = [
    ('bm', 'Book-to-Market', 'high'),
    ('op', 'Operating Profit.', 'high'),
    ('investment', 'Investment', 'low'),
    ('ret_12_2', 'Momentum (12-2)', 'high'),
    ('ivol', 'Idio. Volatility', 'low'),
]

for i, (sig, label, long_grp) in enumerate(signals):
    result = construct_factor(
        panel, signal_col=sig, long_group=long_grp,
        n_size_groups=1, n_signal_groups=10,
        weighting='value', bp_universe='hose',
        rebalance_freq='annual' if sig not in ['ret_12_2'] else 'monthly'
    )
    if result is None:
        continue

    port_ret = result['portfolio_returns']
    decile_means = (
        port_ret.groupby('signal_group')['port_return']
        .mean() * 12 * 100
    )

    colors_mono = plt.cm.RdYlGn_r(np.linspace(0.1, 0.9, len(decile_means)))
    axes[i].bar(range(len(decile_means)), decile_means.values,
                color=colors_mono, edgecolor='white')
    axes[i].set_xticks(range(len(decile_means)))
    axes[i].set_xticklabels([f'D{d+1}' for d in range(len(decile_means))],
                            fontsize=8)
    axes[i].set_ylabel('Ann. Return (%)')
    axes[i].set_title(label)
    axes[i].axhline(y=0, color='gray', linewidth=0.5)

axes[5].set_visible(False)
plt.suptitle('Decile Portfolio Returns by Signal', fontsize=14)
plt.tight_layout()
plt.show()
```

Figure 19.6

```
factor_names = [c for c in factor_panel.columns if
 ↪  factor_panel[c].notna().sum() > 24]
spanning_data = factor_panel[factor_names].dropna()

print("Spanning Tests (alpha = intercept when regressed on other factors):")
print(f"{'Factor':<8} {'Alpha (ann.)':>12} {'t-stat':>8} {'R²':>6}")
print("-" * 36)

for target in factor_names:
    others = [f for f in factor_names if f != target]
    y = spanning_data[target]
    X = sm.add_constant(spanning_data[others])
    model = sm.OLS(y, X).fit(cov_type='HAC', cov_kwds={'maxlags': 6})

    alpha_ann = model.params['const'] * 12
    alpha_t = model.tvalues['const']
    r2 = model.rsquared

    print(f"{target:<8} {alpha_ann:>12.4f} {alpha_t:>8.2f} {r2:>6.3f}")
```

19.10 Vietnamese-Specific Considerations

19.10.1 State-Owned Enterprise Classification

A unique feature of Vietnamese equities is the high proportion of state-owned enterprises (SOEs). The state retains majority or significant minority stakes in many listed firms, which affects governance, information environment, and trading dynamics. Factors may behave differently within SOE and non-SOE subsamples:

```
# Get SOE classification
firm_info = client.get_firm_info(
    exchanges=['HOSE', 'HNX'],
    fields=['ticker', 'state_ownership_pct', 'is_soe']
)

panel_soe = panel.merge(firm_info[['ticker', 'is_soe']], on='ticker',
 ↪  how='left')

for label, subset in [('SOE', panel_soe[panel_soe['is_soe'] == True]),
                      ('Non-SOE', panel_soe[panel_soe['is_soe'] ==
 ↪  False])]:
```

```
    hml = construct_factor(
        subset, signal_col='bm', long_group='high',
        weighting='value', bp_universe='all',
        rebalance_freq='annual'
    )
    if hml:
        d = hml['diagnostics']
        print(f"HML ({label}): Ann = {d['ann_return']:.4f}, "
              f"t = {d['t_stat']:.2f}, N =
               ↪ {d['avg_stocks_per_portfolio']:.0f}")
```

19.10.2 Foreign Ownership Limits

Foreign ownership caps (49% for most sectors, 30% for banking) affect the investable universe for international investors. Factors constructed from the full universe may not be achievable by foreign investors if the long leg concentrates in stocks at the foreign ownership limit:

```
fol = client.get_foreign_ownership(
    exchanges=['HOSE', 'HNX'],
    fields=['ticker', 'month_end', 'foreign_pct', 'foreign_limit_pct']
)

# Merge with HML portfolios
if hml_result:
    hml_ports = hml_result['portfolios'].merge(
        fol, on=['ticker', 'month_end'], how='left'
    )
    hml_ports['near_limit'] = (
        (hml_ports['foreign_limit_pct'] - hml_ports['foreign_pct']) < 5
    )

    fol_by_group = (
        hml_ports.groupby('signal_group')
        .agg(
            avg_foreign_pct=('foreign_pct', 'mean'),
            pct_near_limit=('near_limit', 'mean')
        )
    )
    print("Foreign Ownership in HML Portfolios:")
    print(fol_by_group.round(3))
```

19.11 Recommended Factor Specifications for Vietnam

Based on the sensitivity analysis, we recommend the following baseline specifications (Table 19.1).

Table 19.1: Recommended factor construction parameters for Vietnamese equities.

Choice	Recommendation	Rationale
Universe	Standard filter (listing age 6m, volume > 0)	Excludes shell firms without losing too much breadth
Breakpoints	HOSE stocks only	Prevents HNX micro-caps from dominating breakpoints
Size groups	2 (median split)	Sufficient control with limited cross-section
Signal groups	3 (terciles) for factor construction; 5 or 10 for portfolio analysis	2×3 = adequate diversification per portfolio
Weighting	Value-weighted	Investable; less noisy than EW
Rebalancing	Annual (June) for accounting signals; monthly for momentum	Standard; consistent with Fama-French
Accounting lag	4 months (available by April for Dec FY)	Conservative PIT alignment
Minimum stocks	5 per portfolio per month	Below this, single-stock idiosyncratic risk dominates

Always report results under the baseline and at least one alternative specification (e.g., EW, all-stock breakpoints) to demonstrate robustness.

19.12 Summary

This chapter has developed a modular, transparent factor construction framework for Vietnamese equities. The key insights are:

- The Eugene F. Fama and French (1993b) 2×3 methodology translates well to Vietnam with one critical adaptation: breakpoints should be computed from HOSE stocks only. Using all-stock breakpoints allows HNX micro-caps to dominate the small-stock groups, inflating apparent premia with economically untradeable returns.

- Value weighting produces more conservative (and more implementable) factor premia than equal weighting. The difference is particularly large for signals correlated with size (BM, investment), where EW overweights the smallest stocks that contribute most to the premium but least to investable returns.
- No single construction choice determines whether a factor "exists." A factor premium that appears only under one specific combination of breakpoints, weighting, and rebalancing frequency is fragile and should be treated with skepticism. Robust factors survive a grid of specifications. The sensitivity analysis framework developed here (e.g., varying weighting, breakpoints, sort granularity, and rebalancing simultaneously) should be standard practice.

The `construct_factor()` function developed in this chapter is designed for reuse throughout the book. Any anomaly variable can be fed through the same pipeline, producing a tradeable factor with full diagnostics. This ensures methodological consistency across chapters and makes it easy to compare premia on an apples-to-apples basis.

20 Fama-MacBeth Regressions

In this chapter, we delve into the implementation of the Eugene F. Fama and MacBeth (1973a) regression approach, a cornerstone of empirical asset pricing. While portfolio sorts provide a robust, non-parametric view of the relationship between characteristics and returns, they struggle when we need to control for multiple factors simultaneously. For instance, in the Vietnamese stock market (HOSE and HNX), small-cap stocks often exhibit high illiquidity. Does the "Size effect" exist because small stocks are risky, or simply because they are illiquid? Fama-MacBeth (FM) regressions allow us to disentangle these effects in a linear framework.

We will implement a version of the FM procedure, accounting for:

1. **Weighted Least Squares (WLS):** To prevent micro-cap stocks, which are prevalent and volatile in Vietnam, from dominating the estimates.
2. **Newey-West Adjustments:** To handle the serial correlation often observed in Vietnamese market risk premiums.

20.1 The Econometric Framework

The Fama-MacBeth procedure is essentially a two-step filter that separates the cross-sectional variation in returns from the time-series variation.

20.1.1 Intuition: Why not Panel OLS?

A naive approach would be to pool all data (N stocks $\times$ T months) and run a single Ordinary Least Squares (OLS) regression:

$$r_{i,t+1} = \alpha + \beta_{i,t}\lambda + \epsilon_{i,t+1}$$

However, this assumes that the error terms $\epsilon_{i,t+1}$ are independent across firms. In reality, stock returns are highly cross-sectionally correlated (if the VN-Index crashes, most stocks fall together). A pooled OLS would underestimate the standard errors, leading to "false positive" discoveries of risk factors. Fama-MacBeth solves this by running T separate cross-sectional regressions, effectively treating each month as a single independent observation of the risk premium.

20.1.2 Mathematical Derivation

20.1.2.1 Step 1: Cross-Sectional Regressions

For each month t, we estimate the premium $\lambda_{k,t}$ for K factors. Let $r_{i,t+1}$ be the excess return of asset i at time $t+1$. Let $\beta_{i,t}$ be a vector of K characteristics (e.g., Market Beta, Book-to-Market, Size) known at time t.

The model for a specific month t is:

$$\mathbf{r}_{t+1} = \mathbf{X}_t \lambda_{t+1} + \alpha_{t+1} + \epsilon_{t+1}$$

Where:

- $\mathbf{r}_{t+1}$ is an $N \times 1$ vector of returns.
- $\mathbf{X}_t$ is an $N \times (K+1)$ matrix of factor exposures (including a column of ones for the intercept).
- λ_{t+1} is the vector of risk premiums realized in month $t+1$.

To use **Weighted Least Squares (WLS)**, We define a weighting matrix $\mathbf{W}_t$ (typically diagonal with market capitalizations). The estimator for month t is:

$$\hat{\lambda}_{t+1} = (\mathbf{X}_t^\top \mathbf{W}_t \mathbf{X}_t)^{-1} \mathbf{X}_t^\top \mathbf{W}_t \mathbf{r}_{t+1}$$

20.1.2.2 Step 2: Time-Series Aggregation

We now have a time-series of T estimates: $\hat{\lambda}_1, \hat{\lambda}_2, \dots, \hat{\lambda}_T$. The final estimate of the risk premium is the time-series average:

$$\hat{\lambda}_k = \frac{1}{T} \sum_{t=1}^{T} \hat{\lambda}_{k,t}$$

The standard error is derived from the standard deviation of these monthly estimates:

$$\sigma(\hat{\lambda}_k) = \sqrt{\frac{1}{T^2} \sum_{t=1}^{T} (\hat{\lambda}_{k,t} - \hat{\lambda}_k)^2}$$

20.2 Data Preparation

We utilize data from our local SQLite database. In Vietnam, the fiscal year typically ends in December, and audited reports are required by April. To ensure no look-ahead bias, we lag accounting data (Book Equity) to match returns starting in July (a 6-month conservative lag, similar to Fama-French, but adapted for Vietnamese reporting delays).

```
import pandas as pd
import numpy as np
import sqlite3
import statsmodels.formula.api as smf
import statsmodels.api as sm
from pandas.tseries.offsets import MonthEnd

# Connect to the Vietnamese data
tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")

# Load Monthly Prices (HOSE & HNX)
prices_monthly = pd.read_sql_query(
  sql="SELECT symbol, date, ret_excess, mktcap, mktcap_lag FROM
 ↪  prices_monthly",
  con=tidy_finance,
  parse_dates={"date"}
)

# Load Book Equity (derived from Vietnamese Financial Statements)
comp_vn = pd.read_sql_query(
  sql="SELECT datadate, symbol, be FROM comp_vn",
  con=tidy_finance,
  parse_dates={"datadate"}
)

# Load Rolling Market Betas (Pre-calculated in Chapter 'Beta Estimation')
beta_monthly = pd.read_sql_query(
  sql="SELECT symbol, date, beta FROM beta_monthly",
  con=tidy_finance,
  parse_dates={"date"}
)
```

We construct our testing characteristics:

1. **(Market Beta):** The sensitivity to the VN-Index.

2. **Size (ln(ME)):** The natural log of market capitalization.
3. **Value (BM):** The ratio of Book Equity to Market Equity.

```
# Prepare Characteristics
characteristics = (
    comp_vn
    # Align reporting date to month end
    .assign(date=lambda x: pd.to_datetime(x["datadate"]) + MonthEnd(0))
    # Merge with price data to get Market Cap at fiscal year end
    .merge(prices_monthly, on=["symbol", "date"], how="left")
    .merge(beta_monthly, on=["symbol", "date"], how="left")
    .assign(
        # Compute Book-to-Market
        bm=lambda x: x["be"] / x["mktcap"],
        log_mktcap=lambda x: np.log(x["mktcap"]),
        # Create sorting date: Financials valid from July of year t+1
        sorting_date=lambda x: x["date"] + pd.DateOffset(months=6) +
 ↪  MonthEnd(0),
    )
    .get(["symbol", "bm", "beta", "sorting_date"])
    .dropna()
)

characteristics.head()
```

	symbol	bm	beta	sorting_date
8729	VTV	7.034945e+08	0.847809	2017-06-30
8732	MTG	2.670306e+09	1.140066	2017-06-30
8739	MKV	6.505031e+08	-0.448319	2017-06-30
8740	MIC	1.243127e+09	0.772140	2017-06-30
8742	MCP	6.657350e+08	0.348139	2017-06-30

```
# Merge back to monthly return panel
data_fm = (prices_monthly
  .merge(characteristics,
         left_on=["symbol", "date"],
         right_on=["symbol", "sorting_date"],
         how="left")
#   .merge(beta_monthly, on=["symbol", "date"], how="left")
  .sort_values(["symbol", "date"])
)
```

```
# Forward fill characteristics for 12 months (valid until next report)
data_fm[["bm"]] = data_fm.groupby("symbol")[["bm"]].ffill(limit=12)

# Log Market Cap is updated monthly
data_fm["log_mktcap"] = np.log(data_fm["mktcap"])

# Lead returns: We use characteristics at t to predict return at t+1
data_fm["ret_excess_lead"] =
 ↪  data_fm.groupby("symbol")["ret_excess"].shift(-1)

# Cleaning: Remove rows with missing future returns or characteristics
data_fm = data_fm.dropna(subset=["ret_excess_lead", "beta", "log_mktcap",
 ↪  "bm"])

print(data_fm.head())

print(f"Data ready: {len(data_fm):,} observations from
 ↪  {data_fm.date.min().date()} to {data_fm.date.max().date()}")
```

```
    symbol       date  ret_excess       mktcap   mktcap_lag            bm  \
163    AAA 2017-06-30    0.129454  2078.455619  1834.816104  7.929854e+08
175    AAA 2018-06-30   -0.067690  2758.426126  2948.159140  8.161755e+08
187    AAA 2019-06-30    0.030469  3141.519560  3038.799575  1.389438e+09
199    AAA 2020-06-30   -0.035462  2311.250278  2387.972279  1.497272e+09
211    AAA 2021-06-30    0.275355  5423.280296  4241.283308  1.456989e+09

         beta sorting_date  log_mktcap  ret_excess_lead
163  1.479060   2017-06-30    7.639380        -0.051090
175  1.090411   2018-06-30    7.922416        -0.095926
187  1.099956   2019-06-30    8.052462        -0.027856
199  0.954144   2020-06-30    7.745544        -0.098769
211  1.245004   2021-06-30    8.598456        -0.175128
Data ready: 5,075 observations from 2017-06-30 to 2023-06-30
```

20.3 Step 1: Cross-Sectional Regressions with WLS

Hou, Xue, and Zhang (2020b) argue that micro-cap stocks distorts inference because they have high transaction costs and idiosyncratic volatility. In Vietnam, this is exacerbated by "penny stock" speculation.

We implement **Weighted Least Squares (WLS)** where weights are the market capitalization of the prior month. This tests if the factors are priced in the *investable* universe, not just the equal-weighted average of tiny stocks.

```
def run_cross_section(df):
    # Standardize inputs for numerical stability
    # Note: We do NOT standardize the dependent variable (returns)
    # We standardize regressors to interpret coefficients as "per 1 SD
     ↪  change" if desired,
    # BUT for pure risk premium estimation, we usually keep raw units.
    # Here we use raw units to interpret lambda as % return per unit of
     ↪  characteristic.

    # Define Weighted Least Squares
    model = smf.wls(
        formula="ret_excess_lead ~ beta + log_mktcap + bm",
        data=df,
        weights=df["mktcap_lag"] # Weight by size
    )
    results = model.fit()

    return results.params

# Apply to every month
risk_premiums = (data_fm
  .groupby("date")
  .apply(run_cross_section)
  .reset_index()
)

print(risk_premiums.head())
```

```
        date  Intercept      beta  log_mktcap            bm
0 2017-06-30  -0.089116 -0.063799    0.010284  2.897813e-11
1 2018-06-30  -0.023221 -0.008252    0.001890  1.377518e-11
2 2019-06-30  -0.079373  0.035622    0.006224 -8.139910e-12
3 2020-06-30  -0.031213 -0.114968    0.008999 -2.306768e-11
4 2021-06-30   0.081397 -0.011407   -0.007330 -5.211290e-11
```

20.4 Step 2: Time-Series Aggregation & Hypothesis Testing

We now possess the time-series of risk premiums. We calculate the arithmetic mean and the -statistics.

Crucially, we use **Newey-West (HAC)** standard errors. Risk premiums in Vietnam often exhibit autocorrelation (momentum in factor performance). A simple standard error formula would be invalid.

```
def calculate_fama_macbeth_stats(df, lags=6):
    summary = []

    for col in ["Intercept", "beta", "log_mktcap", "bm"]:
        series = df[col]

        # 1. Point Estimate (Average Risk Premium)
        mean_premium = series.mean()

        # 2. Newey-West Standard Error
        # We regress the series on a constant (ones) to get the SE of the
        ↪ mean
        exog = sm.add_constant(np.ones(len(series)))
        nw_model = sm.OLS(series, exog).fit(
            cov_type='HAC', cov_kwds={'maxlags': lags}
        )

        se = nw_model.bse.iloc[0]
        t_stat = nw_model.tvalues.iloc[0]

        summary.append({
            "Factor": col,
            "Premium (%)": mean_premium * 100,
            "Std Error": se * 100,
            "t-statistic": t_stat,
            "Significance": "*" if abs(t_stat) > 1.96 else ""
        })

    return pd.DataFrame(summary)

price_of_risk = calculate_fama_macbeth_stats(risk_premiums)
print(price_of_risk.round(4))
```

```
       Factor  Premium (%)  Std Error  t-statistic Significance
```

```
0   Intercept      -1.8174     1.9117      -0.9507
1        beta      -1.7859     1.0407      -1.7161
2  log_mktcap       0.2347     0.2048       1.1457
3          bm      -0.0000     0.0000      -0.0928
```

20.4.1 Visualizing the Time-Varying Risk Premium

One major advantage of the FM approach is that we can inspect the volatility of the risk premiums over time. In Vietnam, we expect the "Size" premium to be highly volatile during periods of retail liquidity injection (e.g., 2020-2021).

```
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick

# Calculate cumulative returns of the factors (as if they were tradable
 ↪  portfolios)
cumulative_premiums = (risk_premiums
    .set_index("date")
    .drop(columns=["Intercept"])
    .cumsum()
)

fig, ax = plt.subplots(figsize=(10, 6))
cumulative_premiums.plot(ax=ax, linewidth=2)
ax.set_title("Cumulative Risk Premiums in Vietnam (Fama-MacBeth)",
 ↪  fontsize=14)
ax.set_ylabel("Cumulative Coefficient Return")
ax.legend(title="Factor")
ax.grid(True, alpha=0.3)
plt.show()
```

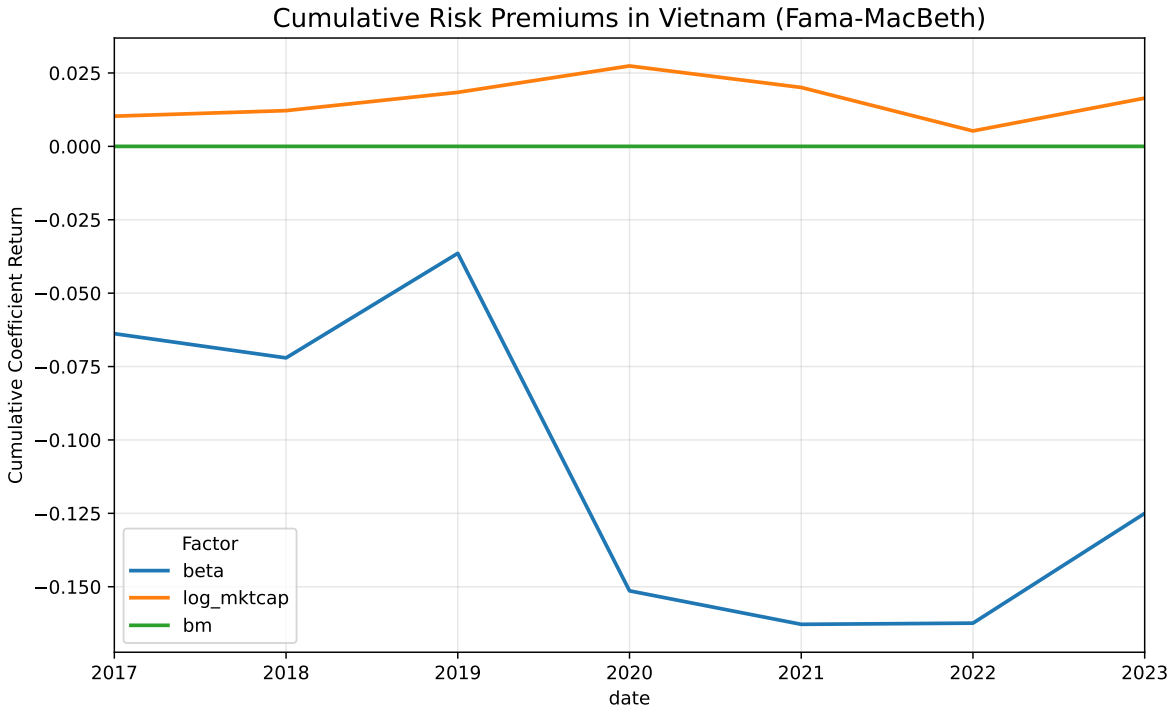


Figure 20.1: Cumulative Risk Premiums in Vietnam.

1. **Market Beta:** In many empirical studies (including the US), the market beta premium is often insignificant or even negative (the "Betting Against Beta" anomaly). In Vietnam, if the -stat is , it implies the CAPM does not explain the cross-section of returns.
2. **Size (Log Mktcap):** A negative coefficient confirms the "Size Effect"—smaller firms have higher expected returns. However, using WLS often weakens this result compared to OLS, suggesting the size premium is concentrated in micro-caps.
3. **Value (BM):** A positive coefficient confirms the Value premium. In Vietnam, value stocks (high B/M) often outperform growth stocks, particularly in the manufacturing and banking sectors.

Figure 20.1 plots the cumulative sum of the monthly Fama MacBeth risk premium estimates for beta, size, and value. Because these lines cumulate estimated cross sectional prices of risk rather than actual portfolio returns, the figure should be interpreted as showing the time variation and persistence of estimated premia, not investable performance.

The beta premium displays a clear regime shift around 2020, with a sharp decline that only partially reverses afterward. This pattern suggests that the pricing of systematic risk in Vietnam is unstable over short samples and may be heavily influenced by episodic market conditions such as the post COVID retail trading boom. The size premium is comparatively smoother but small in magnitude, indicating only weak and time varying evidence that firm size is priced

in the cross section during this period. The value premium remains close to zero throughout, implying little consistent cross sectional reward to high book to market firms in this sample window.

Overall, the figure highlights that estimated risk premia in the Vietnamese market are highly time varying and sensitive to specific macro and market regimes, reinforcing the need for caution when drawing conclusions from short samples.

20.5 Sanity Checks

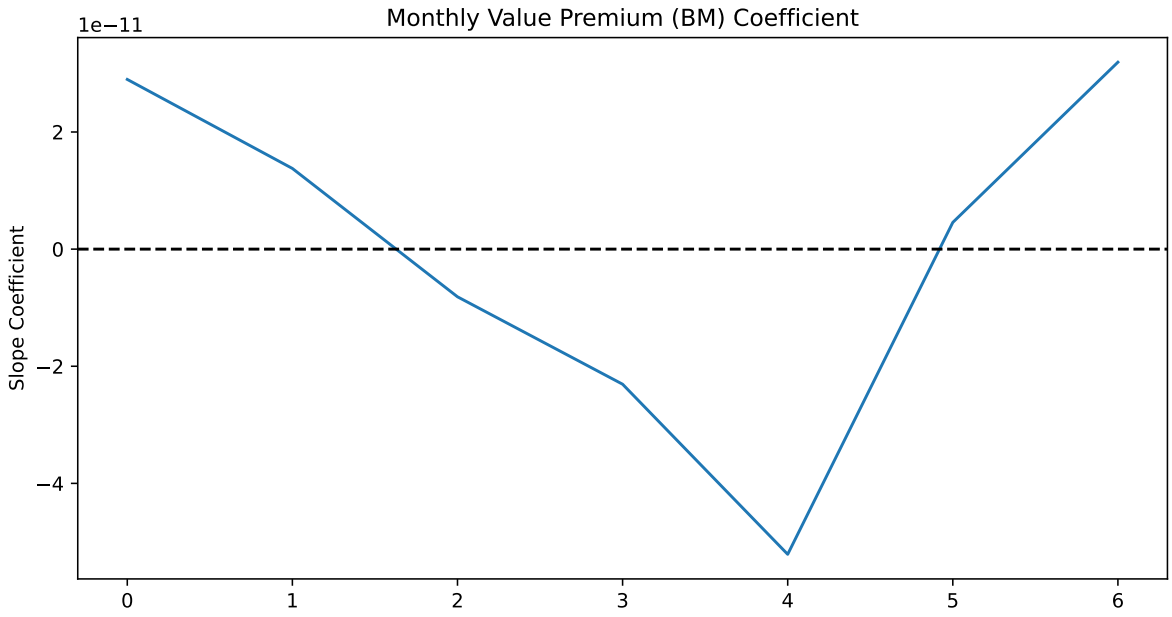
20.5.1 Time-Series Volatility Check

Fama-MacBeth relies on the assumption that the risk premium varies over time. If your `bm` premium is truly near zero every month, the method fails.

Action: Plot the time series of the estimated coefficients . You want to see "noise" around a mean. If you see a flat line or a single massive spike, your data is corrupted.

```
import matplotlib.pyplot as plt

# Plot the time series of the BM risk premium
fig, ax = plt.subplots(figsize=(10, 5))
risk_premiums["bm"].plot(ax=ax, title="Monthly Value Premium (BM)
 ↪  Coefficient")
ax.axhline(0, color="black", linestyle="--")
ax.set_ylabel("Slope Coefficient")
plt.show()
```



20.5.2 Correlation of Characteristics (Multicollinearity)

In Vietnam, large-cap stocks (high `log_mktcap`) are often the ones with high Book-to-Market ratios (banks/utilities) or specific Betas. If your factors are highly correlated, the Fama-MacBeth coefficients will be unstable and insignificant (low t-stats), even if the factors actually matter.

Action: Check the cross-sectional correlation.

```
# Check correlation of the characteristics
corr_matrix = data_fm[["beta", "log_mktcap", "bm"]].corr()
print(corr_matrix)
```

```
                beta  log_mktcap        bm
beta        1.000000    0.392776 -0.033748
log_mktcap  0.392776    1.000000 -0.203307
bm         -0.033748   -0.203307  1.000000
```

Interpretation:

- If correlation > 0.7 (absolute value), the regression struggles to distinguish between the two factors.
- For example, if `Size` and `Liquidity` are -0.8 correlated, the model cannot tell which one is driving the return, often resulting in both having insignificant t-stats.

21 Factor Zoo and Multiple Testing

> **Note**
>
> In this chapter, we confront the multiple testing problem in empirical asset pricing. We replicate a broad set of published anomalies in the Vietnamese market, apply formal corrections for the number of tests conducted, estimate the proportion of false discoveries, and develop a framework for deciding which factors deserve credence.

By 2016, academic finance had documented over 300 variables that purportedly predict the cross-section of stock returns. Cochrane (2011) memorably called this proliferation a "zoo of factors." Campbell R. Harvey, Liu, and Zhu (2016b) cataloged over 300 published factors and argued that the standard significance threshold of $t > 2.0$ was far too low given the sheer number of hypotheses tested, either explicitly or implicitly, across decades of research. Their proposed threshold of $t > 3.0$ (and in some cases $t > 3.78$) would eliminate the majority of published anomalies.

The problem is not merely academic. Every researcher who sorts stocks on a new variable and finds $t > 2$ is implicitly testing a hypothesis that has already been tested hundreds of times with different variables. The probability that *at least one* of 300 independent tests produces $t > 2$ by chance (even when no true effect exists) is essentially 1.0. This is the multiple testing problem, and failing to account for it means that a substantial fraction of the "factor zoo" consists of false discoveries.

For Vietnamese equity research, the problem is arguably worse. The cross-section is smaller (600-800 stocks vs. 4,000+ in the U.S.), the time series is shorter (2008- vs. 1963-), and the data are noisier (illiquidity, zero-trading days, accounting irregularities). All of these amplify the risk of both Type I errors (finding something that isn't there) and Type II errors (missing something that is). This chapter provides the statistical tools to navigate this landscape rigorously.

21.1 The Scale of the Problem

21.1.1 How Many Factors Exist?

Campbell R. Harvey, Liu, and Zhu (2016b) counted over 300 factors published in top journals through 2012. A. Chen and Zimmermann (2022) constructed and made publicly available over 300 "clearly significant" anomalies from the U.S. literature. Hou, Xue, and Zhang (2020a) attempted to replicate 452 anomalies and found that 64% (with VW returns) failed to achieve $t > 1.96$ in their sample. T. I. Jensen, Kelly, and Pedersen (2023) took a more optimistic view, finding that most factors replicate in direction if not always in magnitude.

The key insight is that the number of factors *tested* far exceeds the number *published*. For every published factor with $t > 2$, there may be dozens of unpublished tests with $t < 2$ that never saw print, which is the classic file drawer problem. The true number of tests conducted collectively by the profession is unknowable, but certainly in the thousands.

The coun($|t| > 1.96$ under both definitions) of *signals* also understates the problem, because each signal can be measured in many defensible ways. Binsbergen, Ma, and Schwert (2026) make this point starkly. They keep the sorting variables fixed but change one seemingly innocuous convention: instead of subtracting a one-month T-bill rate, they subtract the return on a duration-matched portfolio of zero-coupon government bonds, so that a factor is judged against a fixed cash-flow counterfactual of the same maturity rather than against cash. Re-running 153 published anomalies over their original sample periods, they find that 66 are robust ($|t| > 1.96$ under both definitions) while 20 flip status, with roughly 0.30 false discoveries per true discovery. Extending the exercise to 1,395 hypothetical sorts built from Compustat ratios, the ratio of false to true discoveries rises to about 1.0 and is stable across rolling windows. Even the market factor is affected: its duration-adjusted counterpart has a correlation below 0.5 with the conventional market excess return, and the equity premium is no longer statistically distinguishable from zero. The mechanism is the four-decade secular decline in interest rates, which handed a mechanical windfall to long-duration legs of long-short portfolios. Their result is not a claim that the traditional definition is wrong, but that the discovery process is sensitive to a choice most papers never report, which is exactly the "multiverse" of researcher degrees of freedom that multiple-testing corrections do not capture.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from scipy import stats
from statsmodels.stats.multitest import multipletests
from itertools import product
import warnings
```

```
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.figsize': (12, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

```
from datacore import DataCoreClient

client = DataCoreClient()

# Monthly returns (survivorship-bias-free)
monthly = client.get_monthly_returns(
    exchanges=['HOSE', 'HNX'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    include_delisted=True,
    fields=[
        'ticker', 'month_end', 'monthly_return', 'market_cap',
        'exchange', 'volume_avg_20d', 'turnover_value_avg_20d',
        'n_zero_volume_days'
    ]
)

# Pre-computed characteristic panel (from factor construction chapter)
characteristics = client.get_characteristics_panel(
    exchanges=['HOSE', 'HNX'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    include_delisted=True
)

# Factor returns for spanning tests
factors_ff = client.get_factor_returns(
    market='vietnam',
    start_date='2008-01-01',
    end_date='2024-12-31',
    factors=['mkt_excess', 'smb', 'hml', 'rmw', 'cma', 'wml']
)
```

```
monthly['month_end'] = pd.to_datetime(monthly['month_end'])

print(f"Monthly returns: {len(monthly):,}")
print(f"Characteristics: {characteristics.shape}")
print(f"Available signals: {[c for c in characteristics.columns if c not in
 ↪  ['ticker', 'month_end']]}")
```

21.2 Building the Anomaly Zoo

21.2.1 Signal Definitions

We construct a comprehensive set of anomaly variables spanning the major categories from the literature. Each signal is lagged appropriately to avoid look-ahead bias (accounting signals use point-in-time alignment; price signals use the prior month's value).

```
# Merge returns with characteristics
panel = monthly.merge(characteristics, on=['ticker', 'month_end'],
 ↪  how='left')
panel = panel.sort_values(['ticker', 'month_end'])

# ============================================================
# Category 1: VALUE (8 signals)
# ============================================================
# Already in characteristics panel from PIT merge:
# bm, ep, cfp, sp, dp, ev_ebitda
# Add earnings yield variants
value_signals = ['bm', 'ep', 'cfp', 'sp', 'dp']

# ============================================================
# Category 2: SIZE (3 signals)
# ============================================================
panel['log_mcap'] = np.log(panel['market_cap'].clip(lower=1))
panel['log_assets'] = np.log(panel['total_assets'].clip(lower=1))
panel['log_revenue'] = np.log(panel['revenue'].clip(lower=1))
size_signals = ['log_mcap', 'log_assets', 'log_revenue']

# ============================================================
# Category 3: PROFITABILITY (8 signals)
# ============================================================
```

```
# GP/A (Novy-Marx 2013), OP (FF 2015), ROE, ROA, etc.
prof_signals = ['gp_at', 'op', 'roe', 'roa', 'profit_margin',
                'asset_turnover', 'gross_margin', 'oper_margin']

# ============================================================
# Category 4: INVESTMENT / GROWTH (6 signals)
# ============================================================
inv_signals = ['asset_growth', 'capex_at', 'investment',
               'sales_growth', 'equity_issuance', 'debt_issuance']

# ============================================================
# Category 5: MOMENTUM / REVERSAL (6 signals)
# ============================================================
# Compute from returns
for lag_start, lag_end, name in [
    (2, 12, 'ret_12_2'), (2, 6, 'ret_6_2'), (7, 12, 'ret_12_7'),
    (1, 1, 'ret_1'), (1, 3, 'ret_3_1')
]:
    if name not in panel.columns:
        panel[name] = (
            panel.groupby('ticker')['monthly_return']
            .transform(lambda x: x.shift(lag_start).rolling(lag_end -
 ↪  lag_start + 1)
                       .apply(lambda r: (1 + r).prod() - 1, raw=True))
        )
# Earnings momentum (SUE) should already be in characteristics
mom_signals = ['ret_12_2', 'ret_6_2', 'ret_12_7', 'ret_1', 'ret_3_1', 'sue']

# ============================================================
# Category 6: RISK (5 signals)
# ============================================================
risk_signals = ['beta', 'ivol', 'tvol', 'max_ret', 'skewness']

# ============================================================
# Category 7: LIQUIDITY / TRADING (6 signals)
# ============================================================
liq_signals = ['amihud', 'zero_return_pct', 'log_turnover',
               'roll_spread', 'cs_spread', 'volume_cv']

# ============================================================
# Category 8: QUALITY / FINANCIAL HEALTH (5 signals)
# ============================================================
qual_signals = ['leverage', 'current_ratio', 'accruals',
```

```
                'earnings_variability', 'piotroski_f']

# ============================================================
# Category 9: DIVIDEND / PAYOUT (3 signals)
# ============================================================
div_signals = ['div_yield', 'payout_ratio', 'net_repurchase']

# Combine all
all_signals = (value_signals + size_signals + prof_signals +
               inv_signals + mom_signals + risk_signals +
               liq_signals + qual_signals + div_signals)

# Remove signals with too little coverage
signal_coverage = {}
for sig in all_signals:
    if sig in panel.columns:
        cov = panel[sig].notna().mean()
        signal_coverage[sig] = cov

signal_coverage = pd.Series(signal_coverage).sort_values(ascending=False)
valid_signals = signal_coverage[signal_coverage > 0.30].index.tolist()

print(f"Total signals defined: {len(all_signals)}")
print(f"Signals with >30% coverage: {len(valid_signals)}")
print(f"\nSignal categories:")
for cat_name, sigs in [
    ('Value', value_signals), ('Size', size_signals),
    ('Profitability', prof_signals), ('Investment', inv_signals),
    ('Momentum', mom_signals), ('Risk', risk_signals),
    ('Liquidity', liq_signals), ('Quality', qual_signals),
    ('Dividend', div_signals)
]:
    n_valid = sum(1 for s in sigs if s in valid_signals)
    print(f"  {cat_name:<15}: {n_valid}/{len(sigs)}")
```

21.2.2 Mass Factor Construction

We run the factor construction engine (from the previous chapter) across all valid signals to produce a "zoo" of factor returns:

```
# Import the factor engine from previous chapter
# (In practice, this would be imported from a shared module)

def construct_factor_simple(panel_df, signal_col, long_high=True,
                            n_groups=5, weighting='value',
                            min_stocks=20):
    """
    Simplified factor construction: quintile sort on signal,
    long-short = Q5 - Q1 (or reversed if long_high=False).
    Returns a Series of monthly long-short returns.
    """
    df = panel_df[['ticker', 'month_end', 'monthly_return',
                   'market_cap', signal_col]].dropna().copy()
    df = df.sort_values(['ticker', 'month_end'])

    # Lag the signal by one month
    df['signal_lag'] = df.groupby('ticker')[signal_col].shift(1)
    df = df.dropna(subset=['signal_lag', 'monthly_return'])

    results = []
    for month, group in df.groupby('month_end'):
        if len(group) < min_stocks:
            continue

        group['quintile'] = pd.qcut(
            group['signal_lag'].rank(method='first'),
            n_groups, labels=False, duplicates='drop'
        )

        if weighting == 'value':
            def wret(g):
                if g['market_cap'].sum() > 0:
                    return np.average(g['monthly_return'],
                                      weights=g['market_cap'])
                return g['monthly_return'].mean()
            port_ret = group.groupby('quintile').apply(wret)
        else:
            port_ret = group.groupby('quintile')['monthly_return'].mean()

        if 0 in port_ret.index and (n_groups - 1) in port_ret.index:
            if long_high:
                ls = port_ret[n_groups - 1] - port_ret[0]
            else:
```

```
                ls = port_ret[0] - port_ret[n_groups - 1]
            results.append({'month_end': month, 'ls_return': ls})

    if not results:
        return None
    return pd.DataFrame(results).set_index('month_end')['ls_return']

# Determine expected sign for each signal
# (positive = high signal predicts high returns)
signal_directions = {
    # Value: high = higher returns
    'bm': True, 'ep': True, 'cfp': True, 'sp': True, 'dp': True,
    # Size: small = higher returns (low signal = high returns)
    'log_mcap': False, 'log_assets': False, 'log_revenue': False,
    # Profitability: high = higher returns
    'gp_at': True, 'op': True, 'roe': True, 'roa': True,
    'profit_margin': True, 'asset_turnover': True,
    'gross_margin': True, 'oper_margin': True,
    # Investment: low growth = higher returns
    'asset_growth': False, 'capex_at': False, 'investment': False,
    'sales_growth': False, 'equity_issuance': False,
    'debt_issuance': False,
    # Momentum: high = higher returns (except reversal)
    'ret_12_2': True, 'ret_6_2': True, 'ret_12_7': True,
    'ret_1': False, 'ret_3_1': False, 'sue': True,
    # Risk: low risk = higher returns (anomaly direction)
    'beta': False, 'ivol': False, 'tvol': False,
    'max_ret': False, 'skewness': False,
    # Liquidity: illiquid = higher returns
    'amihud': True, 'zero_return_pct': True,
    'log_turnover': False, 'roll_spread': True,
    'cs_spread': True, 'volume_cv': True,
    # Quality: high quality = higher returns
    'leverage': False, 'current_ratio': True, 'accruals': False,
    'earnings_variability': False, 'piotroski_f': True,
    # Dividend: high = higher returns
    'div_yield': True, 'payout_ratio': True, 'net_repurchase': False,
}

# Construct all factors
zoo_results = {}
for sig in valid_signals:
    long_high = signal_directions.get(sig, True)
```

```
    ls = construct_factor_simple(
        panel, sig, long_high=long_high,
        n_groups=5, weighting='value'
    )
    if ls is not None and len(ls) >= 60:
        mean_ret = ls.mean()
        se = ls.std() / np.sqrt(len(ls))
        t = mean_ret / se if se > 0 else 0

        zoo_results[sig] = {
            'mean_monthly': mean_ret,
            'ann_return': mean_ret * 12,
            'ann_vol': ls.std() * np.sqrt(12),
            'sharpe': mean_ret / ls.std() * np.sqrt(12) if ls.std() > 0 else
             ↪  0,
            't_stat': t,
            'abs_t': abs(t),
            'p_value': 2 * (1 - stats.t.cdf(abs(t), df=len(ls) - 1)),
            'n_months': len(ls),
            'direction': 'Long High' if long_high else 'Long Low',
            'category': next(
                (cat for cat, sigs in [
                    ('Value', value_signals), ('Size', size_signals),
                    ('Profitability', prof_signals),
                    ('Investment', inv_signals),
                    ('Momentum', mom_signals), ('Risk', risk_signals),
                    ('Liquidity', liq_signals),
                    ('Quality', qual_signals),
                    ('Dividend', div_signals)
                ] if sig in sigs), 'Other'
            ),
            'returns': ls  # Store for later use
        }

zoo_df = pd.DataFrame({k: {kk: vv for kk, vv in v.items() if kk != 'returns'}
                       for k, v in zoo_results.items()}).T
zoo_df = zoo_df.sort_values('abs_t', ascending=False)

print(f"Factors successfully constructed: {len(zoo_df)}")
print(f"Factors with |t| > 2.0: {(zoo_df['abs_t'] > 2.0).sum()} "
      f"({(zoo_df['abs_t'] > 2.0).mean():.1%})")
print(f"Factors with |t| > 3.0: {(zoo_df['abs_t'] > 3.0).sum()} "
      f"({(zoo_df['abs_t'] > 3.0).mean():.1%})")
```

21.3 The Multiple Testing Problem

21.3.1 Why Single-Test p-Values Are Misleading

Suppose you test M independent hypotheses, each at significance level $\alpha = 0.05$. Under the global null (no true effects), the expected number of false rejections is $M \times \alpha$. For $M = 50$ anomalies, you expect 2.5 "significant" results by pure chance.

The probability of *at least one* false rejection is:

$$P(\text{at least one false discovery}) = 1 - (1 - \alpha)^M \tag{21.1}$$

For $M = 50$ and $\alpha = 0.05$, this is $1 - 0.95^{50} = 0.923$. For $M = 300$, it is essentially 1. The single-test p-value is useless for deciding which of many tested factors are genuine.

21.3.2 Two Error Rate Concepts

Multiple testing corrections control one of two error rates:

Family-Wise Error Rate (FWER). The probability of making *at least one* Type I error across all tests. This is the most conservative criterion. If FWER = 0.05, you can be 95% confident that *every* rejected null is a true discovery. Methods: Bonferroni (1936) correction, Holm step-down, Romano and Wolf (2005).

False Discovery Rate (FDR). The expected *proportion* of rejected hypotheses that are false. This is less conservative. If FDR = 0.05, you expect that 5% of your discoveries are false, but the other 95% are real. Methods: Benjamini and Hochberg (1995) (BH), Storey (2003).

In asset pricing, FDR control is generally more appropriate than FWER because we are not trying to guarantee zero false discoveries, we are trying to identify a set of factors where *most* are genuine. A researcher who controls FDR at 5% and discovers 10 factors expects approximately 0.5 of them to be false, which is acceptable for building a factor model.

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Panel A: Histogram
t_vals = zoo_df['t_stat'].values
axes[0].hist(t_vals, bins=30, color='#2C5F8A', alpha=0.7,
             edgecolor='white', density=True)

# Overlay normal distribution under null
x_range = np.linspace(-5, 5, 200)
axes[0].plot(x_range, stats.norm.pdf(x_range), 'k--', linewidth=1.5,
             label='N(0,1) null')

# Threshold lines
axes[0].axvline(x=1.96, color='#E67E22', linestyle='--', linewidth=1.5,
                label='|t| = 1.96')
axes[0].axvline(x=-1.96, color='#E67E22', linestyle='--', linewidth=1.5)
axes[0].axvline(x=3.0, color='#C0392B', linestyle='--', linewidth=1.5,
                label='|t| = 3.0 (HLZ)')
axes[0].axvline(x=-3.0, color='#C0392B', linestyle='--', linewidth=1.5)
axes[0].set_xlabel('t-statistic')
axes[0].set_ylabel('Density')
axes[0].set_title('Panel A: t-Statistic Distribution')
axes[0].legend(fontsize=9)

# Panel B: Ranked bar chart
top_n = min(40, len(zoo_df))
top = zoo_df.head(top_n).copy()

category_colors = {
    'Value': '#2C5F8A', 'Size': '#1ABC9C', 'Profitability': '#27AE60',
    'Investment': '#E67E22', 'Momentum': '#C0392B', 'Risk': '#8E44AD',
    'Liquidity': '#3498DB', 'Quality': '#F1C40F', 'Dividend': '#95A5A6'
}
bar_colors = [category_colors.get(top.iloc[i]['category'], '#BDC3C7')
              for i in range(len(top))]

axes[1].barh(range(top_n), top['abs_t'].values,
             color=bar_colors, alpha=0.85, edgecolor='white')
axes[1].set_yticks(range(top_n))
axes[1].set_yticklabels(top.index, fontsize=7)
axes[1].axvline(x=1.96, color='#E67E22', linestyle='--', linewidth=1)
axes[1].axvline(x=3.0, color='#C0392B', linestyle='--', linewidth=1)
axes[1].set_xlabel('|t-statistic|')
axes[1].set_title('Panel B: Anomalies Ranked by |t|')
axes[1].invert_yaxis()

# Legend for categories
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=c, label=cat)
                   for cat, c in category_colors.items()]
axes[1].legend(handles=legend_elements, fontsize=7,
               loc='lower right', ncol=2)

plt.tight_layout()
```

```
M_range = np.arange(1, 301)
alpha = 0.05

fwer_unadj = 1 - (1 - alpha) ** M_range
fwer_bonf = np.minimum(1, M_range * alpha)  # Not actual FWER, just threshold

# Expected false discoveries at alpha = 0.05
expected_false = M_range * alpha

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(M_range, fwer_unadj, color='#C0392B', linewidth=2,
             label='Unadjusted FWER')
axes[0].axhline(y=0.05, color='gray', linestyle='--', linewidth=1,
                label='  = 0.05 target')
axes[0].set_xlabel('Number of Tests (M)')
axes[0].set_ylabel('P( 1 False Discovery)')
axes[0].set_title('Panel A: Family-Wise Error Rate')
axes[0].legend()

axes[1].plot(M_range, expected_false, color='#2C5F8A', linewidth=2)
axes[1].set_xlabel('Number of Tests (M)')
axes[1].set_ylabel('Expected False Discoveries')
axes[1].set_title('Panel B: Expected False Discoveries at   = 0.05')
axes[1].axhline(y=1, color='gray', linestyle='--', linewidth=0.8)
axes[1].text(250, 1.5, '1 false discovery', color='gray', fontsize=9)

plt.tight_layout()
plt.show()
```

Figure 21.2

21.4 Correction Methods

21.4.1 Bonferroni Correction (FWER Control)

The simplest and most conservative correction: reject hypothesis i only if its p-value is below α/M, where M is the total number of tests:

$$\text{Reject } H_{0,i} \text{ if } p_i < \frac{\alpha}{M} \tag{21.2}$$

This controls FWER at α regardless of the dependence structure among tests. The cost is severe loss of power: for $M = 50$ tests at $\alpha = 0.05$, the per-test threshold becomes 0.001, requiring $|t| > 3.29$.

```
M = len(zoo_df)
alpha = 0.05

# Bonferroni threshold
bonf_threshold = alpha / M
bonf_t_threshold = stats.norm.ppf(1 - bonf_threshold / 2)

zoo_df['bonf_reject'] = zoo_df['p_value'] < bonf_threshold

print(f"Number of tests: {M}")
print(f"Bonferroni p-value threshold: {bonf_threshold:.6f}")
print(f"Equivalent t-statistic threshold: {bonf_t_threshold:.2f}")
print(f"Rejections: {zoo_df['bonf_reject'].sum()} / {M}")
print(f"\nSurviving anomalies:")
survivors = zoo_df[zoo_df['bonf_reject']]
if len(survivors) > 0:
    print(survivors[['ann_return', 't_stat', 'category']].to_string())
else:
    print("  None survive Bonferroni correction")
```

21.4.2 Holm Step-Down Procedure (FWER Control)

The Holm procedure is uniformly more powerful than Bonferroni while still controlling FWER. It ranks p-values from smallest to largest and applies progressively less stringent thresholds:

1. Sort p-values: $p_{(1)} \leq p_{(2)} \leq ... \leq p_{(M)}$
2. For $i = 1, 2, ...$, reject $H_{0,(i)}$ if $p_{(i)} < \alpha/(M - i + 1)$
3. Stop at the first i where $H_{0,(i)}$ is not rejected

```
p_values = zoo_df['p_value'].values
signal_names = zoo_df.index.values

# Using statsmodels implementation
reject_holm, pvals_corrected_holm, _, _ = multipletests(
    p_values, alpha=0.05, method='holm'
)

zoo_df['holm_reject'] = reject_holm
zoo_df['holm_p_corrected'] = pvals_corrected_holm

print(f"Holm rejections: {reject_holm.sum()} / {M}")
holm_survivors = zoo_df[zoo_df['holm_reject']]
if len(holm_survivors) > 0:
    print("\nSurviving anomalies (Holm):")
    print(holm_survivors[['ann_return', 't_stat', 'holm_p_corrected',
                          'category']].to_string())
```

21.4.3 Benjamini-Hochberg Procedure (FDR Control)

The Benjamini and Hochberg (1995) (BH) procedure controls the False Discovery Rate (i.e., the expected proportion of rejections that are false) at level q:

1. Sort p-values: $p_{(1)} \leq p_{(2)} \leq ... \leq p_{(M)}$
2. Find the largest k such that $p_{(k)} \leq \frac{k}{M}q$
3. Reject all $H_{0,(i)}$ for $i = 1, ..., k$

The BH procedure is valid under independence or positive dependence (PRDS), a condition that is generally satisfied for financial factor tests where factors tend to be positively correlated under the null.

```
# BH procedure at q = 0.05 (expect 5% false discoveries)
reject_bh, pvals_corrected_bh, _, _ = multipletests(
    p_values, alpha=0.05, method='fdr_bh'
)

zoo_df['bh_reject'] = reject_bh
zoo_df['bh_p_corrected'] = pvals_corrected_bh

# Also at q = 0.10 (more liberal)
reject_bh10, _, _, _ = multipletests(
    p_values, alpha=0.10, method='fdr_bh'
```

```
)
zoo_df['bh10_reject'] = reject_bh10

print(f"BH rejections (q=0.05): {reject_bh.sum()} / {M}")
print(f"BH rejections (q=0.10): {reject_bh10.sum()} / {M}")

bh_survivors = zoo_df[zoo_df['bh_reject']].sort_values('abs_t',
 ↪  ascending=False)
if len(bh_survivors) > 0:
    print(f"\nSurviving anomalies (BH, q=0.05):")
    print(bh_survivors[['ann_return', 't_stat', 'bh_p_corrected',
                        'category']].head(20).to_string())
```

21.4.4 The Harvey-Liu-Zhu Threshold

Campbell R. Harvey, Liu, and Zhu (2016b) take a different approach: rather than applying a formal multiple-testing correction to a specific set of tests, they estimate the hurdle t-statistic that accounts for the *cumulative* number of factors tested by the entire profession. Using a Bayesian framework calibrated to the 316 published factors, they derive:

- For a single new factor tested in isolation: $|t| > 2.0$ (conventional)
- Accounting for 100 prior tests: $|t| > 2.8$
- Accounting for 316 prior tests: $|t| > 3.0$
- Accounting for all conceivable tests: $|t| > 3.78$

The logic is that even if *your* study tests only one factor, the fact that hundreds of researchers have tested hundreds of signals before you means the prior probability that your specific signal is a true predictor is low.

```
hlz_thresholds = {
    'Conventional (|t| > 2.0)': 2.0,
    'HLZ 100 tests (|t| > 2.8)': 2.8,
    'HLZ 316 tests (|t| > 3.0)': 3.0,
    'HLZ conservative (|t| > 3.78)': 3.78
}

print("Anomalies Surviving Different t-Thresholds:")
print(f"{'Threshold':<35} {'Surviving':>10} {'% of Zoo':>10}")
print("-" * 55)

for name, thresh in hlz_thresholds.items():
```

```
sorted_pvals = np.sort(p_values)
k = np.arange(1, M + 1)
bh_line_05 = k / M * 0.05
bh_line_10 = k / M * 0.10

fig, ax = plt.subplots(figsize=(12, 6))

ax.scatter(k, sorted_pvals, s=15, color='#2C5F8A', alpha=0.7,
           label='Sorted p-values', zorder=3)
ax.plot(k, bh_line_05, color='#C0392B', linewidth=2,
        label='BH line (q=0.05)')
ax.plot(k, bh_line_10, color='#E67E22', linewidth=2,
        linestyle='--', label='BH line (q=0.10)')

# Highlight rejections
n_reject_05 = reject_bh.sum()
if n_reject_05 > 0:
    ax.scatter(k[:n_reject_05], sorted_pvals[:n_reject_05],
               s=40, color='#C0392B', zorder=4, label=f'Rejected (q=0.05)')

ax.set_xlabel('Rank (k)')
ax.set_ylabel('p-value')
ax.set_title('Benjamini-Hochberg Procedure')
ax.legend()
ax.set_ylim([-0.01, 0.15])
ax.set_xlim([0, min(M, 60)])

plt.tight_layout()
plt.show()
```

Figure 21.3

```
    n_survive = (zoo_df['abs_t'] > thresh).sum()
    pct = n_survive / len(zoo_df) * 100
    print(f"{name:<35} {n_survive:>10} {pct:>10.1f}%")
```

21.5 Comparison of Methods

```
comparison = {
    'Unadjusted (|t| > 2)': (zoo_df['abs_t'] > 2.0).sum(),
    'Bonferroni': zoo_df['bonf_reject'].sum(),
    'Holm': zoo_df['holm_reject'].sum(),
    'BH (q=0.05)': zoo_df['bh_reject'].sum(),
    'BH (q=0.10)': zoo_df['bh10_reject'].sum(),
    'HLZ (|t| > 3.0)': (zoo_df['abs_t'] > 3.0).sum(),
    'HLZ (|t| > 3.78)': (zoo_df['abs_t'] > 3.78).sum(),
}

fig, ax = plt.subplots(figsize=(10, 5))

names = list(comparison.keys())
counts = list(comparison.values())
colors = ['#BDC3C7', '#C0392B', '#E74C3C', '#27AE60', '#2ECC71',
          '#2C5F8A', '#1A3C5A']

bars = ax.barh(range(len(names)), counts, color=colors,
               alpha=0.85, edgecolor='white')
ax.set_yticks(range(len(names)))
ax.set_yticklabels(names)
ax.set_xlabel('Number of Surviving Anomalies')
ax.set_title('Anomalies Surviving Multiple Testing Corrections')

for i, (name, count) in enumerate(zip(names, counts)):
    ax.text(count + 0.3, i, str(count), va='center', fontsize=10)

ax.invert_yaxis()
plt.tight_layout()
plt.show()
```

Figure 21.4

```
cat_survival = (
    zoo_df.groupby('category')
    .agg(
        n_total=('abs_t', 'count'),
        n_survive_bh=('bh_reject', 'sum'),
        n_survive_hlz=('abs_t', lambda x: (x > 3.0).sum()),
        mean_abs_t=('abs_t', 'mean')
    )
    .sort_values('mean_abs_t', ascending=False)
)
cat_survival['survival_rate_bh'] = (
    cat_survival['n_survive_bh'] / cat_survival['n_total']
)

fig, ax = plt.subplots(figsize=(12, 5))

x = np.arange(len(cat_survival))
ax.bar(x, cat_survival['n_total'], color='#BDC3C7', alpha=0.5,
       label='Total tested', edgecolor='white')
ax.bar(x, cat_survival['n_survive_bh'], color='#27AE60', alpha=0.85,
       label='Survive BH (q=0.05)', edgecolor='white')

ax.set_xticks(x)
ax.set_xticklabels(cat_survival.index, rotation=30, fontsize=9)
ax.set_ylabel('Number of Anomalies')
ax.set_title('Anomaly Survival by Category')
ax.legend()

# Add survival rate text
for i, (_, row) in enumerate(cat_survival.iterrows()):
    if row['n_total'] > 0:
        ax.text(i, row['n_total'] + 0.2,
                f"{row['survival_rate_bh']:.0%}",
                ha='center', fontsize=9, color='#27AE60')

plt.tight_layout()
plt.show()
```

Figure 21.5

21.6 Estimating the False Discovery Proportion

21.6.1 The Storey Approach

Rather than controlling FDR at a pre-specified level, we can *estimate* the proportion of tested hypotheses that are truly null (π_0) and the implied false discovery proportion (FDP) at any given threshold. Storey (2003) proposes estimating π_0 from the distribution of p-values: under the null, p-values are uniformly distributed on $[0, 1]$, so the density of p-values in the "flat" region (away from zero) reveals π_0.

$$\hat{\pi}_0(\lambda) = \frac{\#\{p_i > \lambda\}}{M(1 - \lambda)} \tag{21.3}$$

for a tuning parameter λ. The estimated false discovery proportion at threshold t^* is then:

$$\widehat{\text{FDP}}(t^*) = \frac{\hat{\pi}_0 \cdot M \cdot P(|t| > t^* | H_0)}{\#\{|t_i| > t^*\}} \tag{21.4}$$

```
def estimate_pi0(p_values, lambdas=np.arange(0.05, 0.95, 0.05)):
    """
    Estimate pi_0 (proportion of true nulls) using Storey (2003).
    Uses bootstrap to select optimal lambda.
    """
    M = len(p_values)
    pi0_estimates = []

    for lam in lambdas:
        n_above = (p_values > lam).sum()
        pi0 = n_above / (M * (1 - lam))
        pi0_estimates.append({'lambda': lam, 'pi0': min(pi0, 1.0)})

    pi0_df = pd.DataFrame(pi0_estimates)

    # Use the median of estimates for robustness
    # (alternatives: bootstrap, spline smoothing)
    pi0_hat = pi0_df['pi0'].median()

    return pi0_hat, pi0_df

pi0_hat, pi0_df = estimate_pi0(zoo_df['p_value'].values)

print(f"Estimated   (proportion of true nulls): {pi0_hat:.3f}")
```

```
print(f"Implied proportion of true factors: {1 - pi0_hat:.3f}")
print(f"Estimated number of true factors: {(1 - pi0_hat) * len(zoo_df):.0f} "
      f"out of {len(zoo_df)}")
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: p-value histogram
axes[0].hist(zoo_df['p_value'], bins=20, density=True,
             color='#2C5F8A', alpha=0.7, edgecolor='white')
axes[0].axhline(y=pi0_hat, color='#C0392B', linewidth=2, linestyle='--',
                label=f'  = {pi0_hat:.2f} (null density)')
axes[0].axhline(y=1, color='gray', linewidth=1, linestyle=':',
                label='Uniform (all null)')
axes[0].set_xlabel('p-value')
axes[0].set_ylabel('Density')
axes[0].set_title('Panel A: p-Value Distribution')
axes[0].legend()

# Panel B: pi_0 estimates across lambda
axes[1].plot(pi0_df['lambda'], pi0_df['pi0'],
             color='#2C5F8A', linewidth=2, marker='o', markersize=4)
axes[1].axhline(y=pi0_hat, color='#C0392B', linestyle='--',
                linewidth=1.5, label=f'Median   = {pi0_hat:.2f}')
axes[1].set_xlabel(' ')
axes[1].set_ylabel(' ( )')
axes[1].set_title('Panel B:   Estimates by  ')
axes[1].legend()
axes[1].set_ylim([0, 1.05])

plt.tight_layout()
plt.show()
```

Figure 21.6

21.6.2 FDP Curve

Using $\hat{\pi}_0$, we compute the estimated FDP at every possible t-threshold:

```
t_thresholds = np.arange(1.0, 5.01, 0.1)
fdp_curve = []

for t_thresh in t_thresholds:
    n_rejected = (zoo_df['abs_t'] > t_thresh).sum()
    if n_rejected == 0:
        fdp_curve.append({'t_threshold': t_thresh, 'n_rejected': 0,
                          'fdp': 0})
        continue

    # Expected false discoveries under null
    p_null_reject = 2 * (1 - stats.norm.cdf(t_thresh))
    expected_false = pi0_hat * len(zoo_df) * p_null_reject

    fdp = min(expected_false / n_rejected, 1.0)
    fdp_curve.append({'t_threshold': t_thresh, 'n_rejected': n_rejected,
                      'fdp': fdp})

fdp_df = pd.DataFrame(fdp_curve)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: FDP curve
axes[0].plot(fdp_df['t_threshold'], fdp_df['fdp'] * 100,
             color='#C0392B', linewidth=2)
axes[0].axhline(y=5, color='gray', linestyle='--', linewidth=1,
                label='5% FDP')
axes[0].axhline(y=10, color='gray', linestyle=':', linewidth=1,
                label='10% FDP')
axes[0].axvline(x=2.0, color='#E67E22', linestyle='--',
                linewidth=1, label='|t| = 2.0')
axes[0].axvline(x=3.0, color='#2C5F8A', linestyle='--',
                linewidth=1, label='|t| = 3.0')
axes[0].set_xlabel('|t| Threshold')
axes[0].set_ylabel('Estimated FDP (%)')
axes[0].set_title('Panel A: False Discovery Proportion')
axes[0].legend(fontsize=8)
axes[0].set_ylim([-2, 80])

# Panel B: Number of rejections
axes[1].plot(fdp_df['t_threshold'], fdp_df['n_rejected'],
             color='#2C5F8A', linewidth=2)
axes[1].set_xlabel('|t| Threshold')
axes[1].set_ylabel('Number of Discoveries')
axes[1].set_title('Panel B: Discoveries vs Threshold')

plt.tight_layout()
plt.show()
```


Figure 21.7

21.7 Bootstrap-Based Multiple Testing

21.7.1 The White Reality Check

Analytical corrections assume known distributions. When return distributions are non-normal (heavy tails, skewness, serial correlation), as they are in Vietnam, bootstrap methods provide more reliable inference.

White (2000) proposes a bootstrap reality check that accounts for data snooping across multiple strategies. The null hypothesis is that the best strategy has zero expected return. The procedure:

1. Generate B bootstrap samples of the time series (block bootstrap to preserve serial dependence).
2. For each bootstrap sample, compute the t-statistic for every factor.
3. The p-value for factor i is the proportion of bootstrap samples in which the maximum t-statistic across all factors exceeds the observed t-statistic of factor i.

```
def white_reality_check(factor_returns_dict, n_bootstrap=1000,
                        block_size=6, seed=42):
    """
    White (2000) Reality Check for data snooping.

    Parameters
    ----------
    factor_returns_dict : dict
        {factor_name: pd.Series of monthly returns}
    n_bootstrap : int
        Number of bootstrap replications
    block_size : int
        Block length for circular block bootstrap

    Returns
    -------
    DataFrame with original t-stats and bootstrap p-values
    """
    rng = np.random.default_rng(seed)

    # Align all factor returns to common dates
    all_names = list(factor_returns_dict.keys())
    common_dates = None
    for name in all_names:
        dates = set(factor_returns_dict[name].index)
```

```
        common_dates = dates if common_dates is None else common_dates &
↪   dates
    common_dates = sorted(common_dates)
    T = len(common_dates)

    # Build return matrix (T x M)
    M = len(all_names)
    return_matrix = np.column_stack([
        factor_returns_dict[name].reindex(common_dates).values
        for name in all_names
    ])

    # Observed t-statistics
    means = np.nanmean(return_matrix, axis=0)
    ses = np.nanstd(return_matrix, axis=0) / np.sqrt(T)
    t_obs = means / np.where(ses > 0, ses, np.nan)

    # Block bootstrap
    n_blocks = int(np.ceil(T / block_size))
    max_t_bootstrap = np.zeros(n_bootstrap)

    for b in range(n_bootstrap):
        # Circular block bootstrap
        block_starts = rng.integers(0, T, size=n_blocks)
        indices = np.concatenate([
            np.arange(start, start + block_size) % T
            for start in block_starts
        ])[:T]

        boot_returns = return_matrix[indices, :]

        # Center under null (subtract original mean)
        boot_centered = boot_returns - means[np.newaxis, :]

        # Compute t-stats under null
        boot_means = np.nanmean(boot_centered, axis=0)
        boot_ses = np.nanstd(boot_centered, axis=0) / np.sqrt(T)
        boot_t = boot_means / np.where(boot_ses > 0, boot_ses, np.nan)

        max_t_bootstrap[b] = np.nanmax(np.abs(boot_t))

    # Bootstrap p-values (one-sided: is obs_t extreme relative to max?)
    boot_pvals = np.array([
```

```
        np.mean(max_t_bootstrap >= abs(t)) if np.isfinite(t) else 1.0
        for t in t_obs
    ])

    results = pd.DataFrame({
        'factor': all_names,
        't_stat_obs': t_obs,
        'boot_pval': boot_pvals
    }).set_index('factor')

    return results, max_t_bootstrap

# Collect factor return series
factor_returns_dict = {
    name: zoo_results[name]['returns']
    for name in zoo_results
    if 'returns' in zoo_results[name]
}

boot_results, max_t_dist = white_reality_check(
    factor_returns_dict, n_bootstrap=1000, block_size=6
)

boot_results = boot_results.sort_values('t_stat_obs', ascending=False)

print("White Reality Check Results (top 20):")
print(boot_results.head(20).round(4).to_string())
print(f"\nFactors surviving (boot p < 0.05): "
      f"{(boot_results['boot_pval'] < 0.05).sum()}")
```

21.7.2 Romano-Wolf Step-Down Bootstrap

The Romano and Wolf (2005) step-down procedure improves on the White Reality Check by iteratively removing rejected hypotheses and re-testing the remaining ones, gaining power at each step:

```
def romano_wolf_stepdown(factor_returns_dict, n_bootstrap=1000,
                         block_size=6, alpha=0.05, seed=42):
    """
    Romano-Wolf (2005) step-down procedure for FWER control.
    More powerful than single-step methods because rejected
```

```
fig, ax = plt.subplots(figsize=(12, 5))

ax.hist(max_t_dist, bins=40, density=True, color='#BDC3C7',
        alpha=0.7, edgecolor='white', label='Bootstrap null\n(max |t| across
 ↪  all factors)')

# 95th percentile
p95 = np.percentile(max_t_dist, 95)
ax.axvline(x=p95, color='#C0392B', linewidth=2, linestyle='--',
           label=f'95th percentile = {p95:.2f}')

# Observed top factor t-statistics
top_5 = boot_results.head(5)
for i, (name, row) in enumerate(top_5.iterrows()):
    ax.axvline(x=abs(row['t_stat_obs']),
               color=plt.cm.Set1(i), linewidth=1.5, linestyle=':',
               label=f'{name}: |t| = {abs(row["t_stat_obs"]):.2f}')

ax.set_xlabel('Maximum |t-statistic|')
ax.set_ylabel('Density')
ax.set_title("White's Reality Check: Bootstrap Null Distribution")
ax.legend(fontsize=8, loc='upper right')
plt.tight_layout()
plt.show()
```

Figure 21.8

```
    hypotheses are removed before re-testing.
    """
    rng = np.random.default_rng(seed)

    all_names = list(factor_returns_dict.keys())
    common_dates = None
    for name in all_names:
        dates = set(factor_returns_dict[name].index)
        common_dates = dates if common_dates is None else common_dates &
↪   dates
    common_dates = sorted(common_dates)
    T = len(common_dates)
    M = len(all_names)

    return_matrix = np.column_stack([
        factor_returns_dict[name].reindex(common_dates).values
        for name in all_names
    ])

    means = np.nanmean(return_matrix, axis=0)
    ses = np.nanstd(return_matrix, axis=0) / np.sqrt(T)
    t_obs = np.abs(means / np.where(ses > 0, ses, np.nan))

    # Sort by observed t-stat (descending)
    order = np.argsort(-t_obs)
    t_sorted = t_obs[order]
    names_sorted = [all_names[i] for i in order]

    # Step-down procedure
    rejected = set()
    remaining = set(range(M))

    for step in range(M):
        if not remaining:
            break

        remaining_idx = sorted(remaining)

        # Bootstrap max t-stat among remaining
        n_blocks = int(np.ceil(T / block_size))
        max_t_boot = np.zeros(n_bootstrap)

        for b in range(n_bootstrap):
```

```
            block_starts = rng.integers(0, T, size=n_blocks)
            indices = np.concatenate([
                np.arange(start, start + block_size) % T
                for start in block_starts
            ])[:T]

            boot_returns = return_matrix[indices][:, remaining_idx]
            boot_centered = boot_returns - means[remaining_idx]

            boot_means = np.nanmean(boot_centered, axis=0)
            boot_ses = np.nanstd(boot_centered, axis=0) / np.sqrt(T)
            boot_t = np.abs(boot_means / np.where(boot_ses > 0, boot_ses,
 ↪  np.nan))

            max_t_boot[b] = np.nanmax(boot_t)

        # Critical value
        cv = np.percentile(max_t_boot, (1 - alpha) * 100)

        # Test the most significant remaining factor
        # Find the factor with highest t among remaining
        best_remaining = max(remaining, key=lambda j: t_obs[j])

        if t_obs[best_remaining] > cv:
            rejected.add(best_remaining)
            remaining.remove(best_remaining)
        else:
            break  # Cannot reject any more

    results = pd.DataFrame({
        'factor': all_names,
        't_stat': t_obs,
        'rejected': [i in rejected for i in range(M)]
    }).sort_values('t_stat', ascending=False)

    return results

rw_results = romano_wolf_stepdown(
    factor_returns_dict, n_bootstrap=500, block_size=6
)

print(f"Romano-Wolf rejections: {rw_results['rejected'].sum()} /
 ↪  {len(rw_results)}")
```

```
print("\nRejected factors:")
print(rw_results[rw_results['rejected']][['factor', 't_stat']].to_string())
```

21.8 Out-of-Sample Validation

21.8.1 Pre-Publication vs. Post-Publication Decay

McLean and Pontiff (2016) find that anomaly returns decline by 32% after publication in the U.S. market, suggesting that roughly one-third of the premium was due to mispricing that was corrected by informed trading. For Vietnam, we use a time-series split as a proxy for out-of-sample testing:

```
# Split at midpoint of available data
all_dates = sorted(panel['month_end'].unique())
mid_date = all_dates[len(all_dates) // 2]

oos_comparison = []

for name in zoo_results:
    ls = zoo_results[name]['returns']

    in_sample = ls[ls.index <= mid_date]
    out_sample = ls[ls.index > mid_date]

    if len(in_sample) < 36 or len(out_sample) < 36:
        continue

    is_mean = in_sample.mean() * 12
    is_t = in_sample.mean() / (in_sample.std() / np.sqrt(len(in_sample)))

    oos_mean = out_sample.mean() * 12
    oos_t = out_sample.mean() / (out_sample.std() / np.sqrt(len(out_sample)))

    oos_comparison.append({
        'signal': name,
        'is_return': is_mean,
        'is_t': is_t,
        'oos_return': oos_mean,
        'oos_t': oos_t,
        'decay_pct': (1 - oos_mean / is_mean) * 100 if is_mean != 0 else
        ↪  np.nan,
```

```
        'same_sign': np.sign(is_mean) == np.sign(oos_mean),
        'category': zoo_results[name].get('category', 'Other')
    })

oos_df = pd.DataFrame(oos_comparison)

print(f"Split date: {mid_date.strftime('%Y-%m')}")
print(f"Factors with same sign IS and OOS: "
      f"{oos_df['same_sign'].mean():.1%}")
print(f"Average OOS decay: {oos_df['decay_pct'].median():.1f}%")
```

21.9 Which Factors Survive?

21.9.1 A Multi-Hurdle Filter

We now combine all the evidence, including multiple testing corrections, out-of-sample performance, and economic plausibility, to identify the factors that deserve credence in the Vietnamese market.

```
# Merge all test results
final = zoo_df.copy()

# Add OOS results
oos_lookup = oos_df.set_index('signal')
for col in ['oos_return', 'oos_t', 'same_sign']:
    final[col] = final.index.map(lambda x: oos_lookup.loc[x, col]
                                 if x in oos_lookup.index else np.nan)

# Add bootstrap results
boot_lookup = boot_results
final['boot_pval'] = final.index.map(
    lambda x: boot_lookup.loc[x, 'boot_pval']
    if x in boot_lookup.index else np.nan
)

# Multi-hurdle criteria
final['pass_t2'] = final['abs_t'] > 2.0
final['pass_t3'] = final['abs_t'] > 3.0
final['pass_bh'] = final['bh_reject']
final['pass_boot'] = final['boot_pval'] < 0.05
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Scatter
category_colors = {
    'Value': '#2C5F8A', 'Size': '#1ABC9C', 'Profitability': '#27AE60',
    'Investment': '#E67E22', 'Momentum': '#C0392B', 'Risk': '#8E44AD',
    'Liquidity': '#3498DB', 'Quality': '#F1C40F', 'Dividend': '#95A5A6'
}

for cat in oos_df['category'].unique():
    subset = oos_df[oos_df['category'] == cat]
    axes[0].scatter(subset['is_return'] * 100, subset['oos_return'] * 100,
                    color=category_colors.get(cat, '#BDC3C7'),
                    s=50, alpha=0.7, label=cat, edgecolors='white')

lim = max(abs(oos_df['is_return'].max()), abs(oos_df['oos_return'].max())) *
 ↪  100 + 2
axes[0].plot([-lim, lim], [-lim, lim], 'k--', linewidth=1, alpha=0.5)
axes[0].plot([-lim, lim], [0, 0], color='gray', linewidth=0.5)
axes[0].plot([0, 0], [-lim, lim], color='gray', linewidth=0.5)
axes[0].set_xlabel('In-Sample Return (% ann.)')
axes[0].set_ylabel('Out-of-Sample Return (% ann.)')
axes[0].set_title('Panel A: IS vs OOS Factor Returns')
axes[0].legend(fontsize=7, ncol=2)

# Panel B: Decay by category
cat_decay = (
    oos_df.groupby('category')
    .agg(
        median_decay=('decay_pct', 'median'),
        pct_same_sign=('same_sign', 'mean'),
        n=('signal', 'count')
    )
    .sort_values('median_decay')
)

colors_bar = [category_colors.get(cat, '#BDC3C7') for cat in cat_decay.index]
axes[1].barh(range(len(cat_decay)), cat_decay['median_decay'],
             color=colors_bar, alpha=0.85, edgecolor='white')
axes[1].set_yticks(range(len(cat_decay)))
axes[1].set_yticklabels(cat_decay.index)
axes[1].set_xlabel('Median OOS Decay (%)')
axes[1].set_title('Panel B: OOS Decay by Category')
axes[1].axvline(x=0, color='gray', linewidth=0.8)

plt.tight_layout()
plt.show()
```


Figure 21.9

```
final['pass_oos_sign'] = final['same_sign'] == True
final['pass_oos_t'] = final['oos_t'].abs() > 1.5  # Relaxed OOS threshold

# Count hurdles passed
hurdle_cols = ['pass_t2', 'pass_t3', 'pass_bh', 'pass_boot',
               'pass_oos_sign', 'pass_oos_t']
final['n_hurdles'] = final[hurdle_cols].sum(axis=1)

# "Robust" = passes at least 5 of 6 hurdles
final['robust'] = final['n_hurdles'] >= 5

robust_factors = final[final['robust']].sort_values('abs_t', ascending=False)

print(f"Multi-Hurdle Filter Results:")
print(f"  Pass |t| > 2.0:        {final['pass_t2'].sum()}")
print(f"  Pass |t| > 3.0:        {final['pass_t3'].sum()}")
print(f"  Pass BH (q=0.05):      {final['pass_bh'].sum()}")
print(f"  Pass bootstrap:        {final['pass_boot'].sum()}")
print(f"  Pass OOS sign:         {final['pass_oos_sign'].sum()}")
print(f"  Pass OOS |t| > 1.5:    {final['pass_oos_t'].sum()}")
print(f"\n  ROBUST (5/6 hurdles): {final['robust'].sum()}")

if len(robust_factors) > 0:
    print(f"\nRobust Factors:")
    display_cols = ['ann_return', 't_stat', 'bh_p_corrected',
                    'oos_return', 'oos_t', 'n_hurdles', 'category']
    available_cols = [c for c in display_cols if c in robust_factors.columns]
    print(robust_factors[available_cols].round(3).to_string())
```

21.10 Implications for Vietnamese Factor Research

The analysis in this chapter yields several practical implications:

The conventional threshold is insufficient. With $M \approx 50$ anomalies tested simultaneously, requiring only $|t| > 2.0$ virtually guarantees false discoveries. Vietnamese researchers should adopt a minimum threshold of $|t| > 3.0$ for individual factors, consistent with Campbell R. Harvey, Liu, and Zhu (2016b), and should report BH-adjusted p-values for any study testing multiple signals.

The small cross-section amplifies noise. With 600-800 stocks, quintile portfolios contain only 120-160 stocks each. This creates noisier portfolio returns and wider confidence intervals

```
top25 = final.sort_values('abs_t', ascending=False).head(25)

fig, ax = plt.subplots(figsize=(10, 10))

hurdle_matrix = top25[hurdle_cols].astype(float).values

sns.heatmap(hurdle_matrix, ax=ax,
            xticklabels=['|t|>2', '|t|>3', 'BH', 'Boot', 'OOS sign', 'OOS
 ↪  |t|'],
            yticklabels=top25.index,
            cmap=['#E74C3C', '#27AE60'],
            cbar=False, linewidths=0.5, linecolor='white',
            annot=np.where(hurdle_matrix == 1, ' ', ' '),
            fmt='')

# Add hurdle count column
for i, (_, row) in enumerate(top25.iterrows()):
    ax.text(len(hurdle_cols) + 0.3, i + 0.5,
            f"{int(row['n_hurdles'])}/6",
            va='center', fontsize=9,
            fontweight='bold' if row['n_hurdles'] >= 5 else 'normal',
            color='#27AE60' if row['n_hurdles'] >= 5 else '#C0392B')

ax.set_title('Multi-Hurdle Test Results (Top 25 Anomalies)')
plt.tight_layout()
plt.show()
```

Figure 21.10

than in U.S. studies with 4,000+ stocks. Strategies that appear significant in the U.S. at $|t| = 2.5$ may have $|t| < 1.5$ in Vietnam simply due to the smaller sample. This is not evidence that the factor doesn't exist in Vietnam, it is evidence that the Vietnamese data lack power to detect it.

Out-of-sample validation is essential. The time-series split reveals substantial decay for many anomalies, consistent with McLean and Pontiff (2016). Factors that survive both in-sample and out-of-sample with consistent sign and magnitude are rare and valuable.

Category matters. Momentum and profitability signals tend to have the highest replication rates across markets (Eugene F. Fama and French 2012; Jacobs and Müller 2020). Value and investment signals are more country-specific (Griffin 2002). Liquidity signals are likely to be strongest in emerging markets like Vietnam, where trading frictions are largest.

The interest rate path is part of the sample. Vietnam's sample period is dominated by a one-directional move in rates: policy rates peaked near 14% in 2011 and fell to roughly 4.5% by 2023, and government bond yields fell alongside them. Binsbergen, Ma, and Schwert (2026) show that in the U.S. a comparable secular decline was enough to reclassify a meaningful share of anomalies once excess returns are measured against a duration-matched bond portfolio rather than a short rate. Because Vietnam's decline was compressed into a much shorter window, any long-short portfolio whose legs differ in duration (value versus growth, high versus low dividend yield, high versus low leverage) may be picking up that repricing rather than a cross-sectional risk premium. At minimum, researchers should report the dividend yield or duration differential between the long and short legs, and treat factors whose premium is concentrated in the 2011-2015 rate collapse with caution.

Report the full battery. Any study that presents a new factor for the Vietnamese market should report: (i) the raw t-statistic, (ii) the BH-adjusted p-value given the number of tests in the study, (iii) out-of-sample performance in a held-out period, and (iv) sensitivity to construction choices (from the previous chapter). This reporting standard would dramatically improve the credibility of Vietnamese factor research.

21.11 Summary

Table 21.1: Summary of multiple testing methods and their properties.

Method	Controls	Threshold (approx.)	Surviving	Philosophy
Unadjusted	Single test	$\|t\| > 2.0$	Many	No correction
Bonferroni	FWER	$\|t\| > 3.3$–4.0	Few	Zero false positives
Holm	FWER	Stepwise	Few	Slightly less conservative

Method	Controls	Threshold (approx.)	Surviving	Philosophy
BH	FDR at q	Adaptive	Moderate	Tolerates some false positives
HLZ	Profession-wide	$\|t\| > 3.0$–3.78	Moderate	Prior over all tested factors
White/RW Bootstrap	FWER (data-driven)	Bootstrap CV	Few	Accounts for non-normality
Storey FDP	FDP estimate	Continuous	Diagnostic	Estimates true null proportion
Multi-hurdle	Combined	Multiple tests	Most robust	Requires convergent evidence

The factor zoo is not a uniquely American phenomenon. The temptation to test many signals and report the best-performing ones exists in every market. In Vietnam, where the data are shorter, noisier, and the academic community is growing rapidly, the risk of false discovery is high. The tools developed in this chapter, including BH adjustment, bootstrap reality checks, Storey's π_0 estimation, out-of-sample splits, and the multi-hurdle filter, provide a principled framework for separating genuine factors from statistical noise.

The honest conclusion is likely to be humbling: of the dozens of anomalies documented in developed markets, only a handful survive rigorous multiple testing in Vietnamese data. Those survivors (probably concentrated in momentum, profitability, and liquidity) form the foundation for credible asset pricing research in this market.

22 Time-Series vs. Cross-Sectional Factor Tests

> **Note**
>
> In this chapter, we implement and compare the two dominant approaches to testing asset pricing models: time-series regressions (do factors explain individual stock or portfolio returns?) and cross-sectional regressions (do factor exposures explain expected returns?), and develop the diagnostic tools to understand when and why they disagree.

Every factor model makes two distinct claims. The **time-series claim** is that a set of factors explains the common variation in individual stock returns (i.e., when the factors move, stocks move with them in proportion to their exposures). The **cross-sectional claim** is that differences in average returns across stocks are explained by differences in their factor exposures (i.e., stocks that load more heavily on a factor earn higher (or lower) average returns in proportion to the factor's risk premium).

These claims are logically related but empirically distinct. A factor can explain time-series variation without explaining the cross-section (if the factor's risk premium is zero, exposure to it creates no return differential). Conversely, a characteristic can predict the cross-section of returns without operating through a traded factor (if the characteristic captures mispricing rather than risk).

Eugene F. Fama and French (2020) formalize this distinction and show that the two approaches can give materially different answers about which factors matter. Goyal and Jegadeesh (2018) go further and demonstrate that time-series and cross-sectional tests can disagree about the same model (e.g., a factor can have a significant time-series alpha in one test and an insignificant cross-sectional premium in the other, or vice versa). Understanding *why* they disagree is essential for interpreting asset pricing evidence.

This chapter equips the reader with both toolkits, applied to Vietnamese data, and develops the intuition for when each is appropriate.

22.1 The Two Testing Frameworks

22.1.1 Time-Series Regressions

The time-series approach tests whether a factor model explains the returns of a set of test assets (typically portfolios). For each test asset i, estimate:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_{i,1} f_{1,t} + \beta_{i,2} f_{2,t} + ... + \beta_{i,K} f_{K,t} + \varepsilon_{i,t} \tag{22.1}$$

where $f_{k,t}$ are the K factor returns. The intercept α_i measures the average return of asset i that is *not* explained by the factors. Under the null that the model correctly prices asset i, $\alpha_i = 0$.

The joint test of $H_0 : \alpha_1 = \alpha_2 = ... = \alpha_N = 0$ across all N test assets is performed using the Gibbons, Ross, and Shanken (1989) (GRS) test statistic:

$$\text{GRS} = \frac{T - N - K}{N} \cdot \frac{1}{1 + \bar{f}' \hat{\Sigma}_f^{-1} \bar{f}} \cdot \hat{\alpha}' \hat{\Sigma}_\varepsilon^{-1} \hat{\alpha} \sim F(N, T - N - K) \tag{22.2}$$

where $\bar{f}$ is the vector of factor means, $\hat{\Sigma}_f$ is the factor covariance matrix, and $\hat{\Sigma}_\varepsilon$ is the residual covariance matrix. A large GRS statistic rejects the model.

22.1.2 Cross-Sectional Regressions (Fama-MacBeth)

The Eugene F. Fama and MacBeth (1973b) cross-sectional approach tests whether factor exposures are priced (i.e., whether stocks with higher betas on a factor earn higher average returns). The procedure has two stages:

Stage 1 (Time-Series). For each asset, estimate factor betas from a time-series regression over a rolling or full-sample window:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_{i,1} f_{1,t} + ... + \beta_{i,K} f_{K,t} + \varepsilon_{i,t} \tag{22.3}$$

Stage 2 (Cross-Section). Each month t, regress the cross-section of realized excess returns on the estimated betas:

$$R_{i,t} - R_{f,t} = \gamma_{0,t} + \gamma_{1,t} \hat{\beta}_{i,1} + ... + \gamma_{K,t} \hat{\beta}_{i,K} + \eta_{i,t} \tag{22.4}$$

The time-series averages $\bar{\gamma}_k = \frac{1}{T} \sum_t \gamma_{k,t}$ estimate the risk premia, and the standard errors use the time-series standard deviation of $\{\gamma_{k,t}\}$.

22.1.3 Key Differences

Table 22.1: Comparison of time-series and cross-sectional testing frameworks.

Dimension	Time-Series	Cross-Section
What it tests	Do factors explain return variation?	Do factor exposures explain average returns?
Key statistic	Alpha (intercept)	Risk premium (γ)
Joint test	GRS F-test	Chi-squared on γ's
Test assets	Portfolios (usually)	Individual stocks or portfolios
Factors	Must be traded returns	Can be non-traded (macro, characteristics)
Null hypothesis	$\alpha_i = 0$ for all i	γ_k equals factor mean return
Errors-in-variables	Not an issue (factors observed)	Estimated betas create EIV bias

22.2 Data Construction

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from scipy import stats
from scipy.linalg import inv
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.figsize': (12, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

```python
from datacore import DataCoreClient

client = DataCoreClient()

# Factor returns
factors = client.get_factor_returns(
    market='vietnam',
    start_date='2008-01-01',
    end_date='2024-12-31',
    factors=['mkt_excess', 'smb', 'hml', 'rmw', 'cma', 'wml']
)
factors['month_end'] = pd.to_datetime(factors['month_end'])
factors = factors.set_index('month_end')

# Test portfolios: 25 size-BM portfolios (5x5 independent sorts)
test_portfolios_25 = client.get_sorted_portfolios(
    market='vietnam',
    sort_vars=['size', 'bm'],
    n_groups=[5, 5],
    start_date='2008-01-01',
    end_date='2024-12-31',
    weighting='value'
)

# Test portfolios: 25 size-momentum portfolios
test_portfolios_mom = client.get_sorted_portfolios(
    market='vietnam',
    sort_vars=['size', 'momentum_12_2'],
    n_groups=[5, 5],
    start_date='2008-01-01',
    end_date='2024-12-31',
    weighting='value'
)

# Test portfolios: 10 industry portfolios
test_portfolios_ind = client.get_sorted_portfolios(
    market='vietnam',
    sort_vars=['icb_sector'],
    n_groups=[10],
    start_date='2008-01-01',
    end_date='2024-12-31',
    weighting='value'
)
```

```
# Monthly individual stock returns for Fama-MacBeth
stock_returns = client.get_monthly_returns(
    exchanges=['HOSE', 'HNX'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    include_delisted=True,
    fields=['ticker', 'month_end', 'monthly_return', 'market_cap']
)

# Risk-free rate
rf = client.get_risk_free_rate(
    start_date='2008-01-01',
    end_date='2024-12-31',
    frequency='monthly'
)

print(f"Factor months: {len(factors)}")
print(f"25 Size-BM portfolios: {test_portfolios_25.shape}")
print(f"25 Size-Mom portfolios: {test_portfolios_mom.shape}")
print(f"10 Industry portfolios: {test_portfolios_ind.shape}")
print(f"Stock-months: {len(stock_returns):,}")
```

```
# Convert test portfolios to excess returns
# Each portfolio set is a DataFrame: rows = months, columns = portfolios
def prepare_test_assets(port_df, rf_df):
    """Convert portfolio returns to excess returns matrix."""
    port = port_df.set_index('month_end')
    rf_aligned = rf_df.set_index('month_end')['rf'].reindex(port.index)
    excess = port.subtract(rf_aligned, axis=0)
    return excess.dropna()

excess_25_bm = prepare_test_assets(test_portfolios_25, rf)
excess_25_mom = prepare_test_assets(test_portfolios_mom, rf)
excess_10_ind = prepare_test_assets(test_portfolios_ind, rf)

print(f"Size-BM test assets: {excess_25_bm.shape[1]} portfolios, "
      f"{excess_25_bm.shape[0]} months")
print(f"Size-Mom test assets: {excess_25_mom.shape[1]} portfolios, "
      f"{excess_25_mom.shape[0]} months")
print(f"Industry test assets: {excess_10_ind.shape[1]} portfolios, "
      f"{excess_10_ind.shape[0]} months")
```

22.3 Time-Series Tests

22.3.1 Full-Sample Time-Series Regressions

We begin by running full-sample time-series regressions of each test portfolio on the Fama-French five-factor model plus momentum:

```
def time_series_regressions(excess_returns, factor_df, factor_cols,
                            cov_type='HAC', maxlags=6):
    """
    Run time-series regressions of test assets on factors.

    Returns DataFrame of alphas, betas, t-stats, and R-squared.
    """
    # Align dates
    common = excess_returns.index.intersection(factor_df.index)
    Y = excess_returns.loc[common]
    X = factor_df.loc[common, factor_cols]
    X_const = sm.add_constant(X)

    results = {}
    for col in Y.columns:
        y = Y[col].dropna()
        x = X_const.loc[y.index]

        model = sm.OLS(y, x).fit(
            cov_type=cov_type,
            cov_kwds={'maxlags': maxlags} if cov_type == 'HAC' else {}
        )

        result = {
            'alpha': model.params['const'],
            'alpha_t': model.tvalues['const'],
            'alpha_p': model.pvalues['const'],
            'r_squared': model.rsquared,
            'r_squared_adj': model.rsquared_adj
        }
        for f in factor_cols:
            result[f'beta_{f}'] = model.params[f]
            result[f't_{f}'] = model.tvalues[f]

        results[col] = result
```

```
    return pd.DataFrame(results).T

# Define models to test
models = {
    'CAPM': ['mkt_excess'],
    'FF3': ['mkt_excess', 'smb', 'hml'],
    'FF5': ['mkt_excess', 'smb', 'hml', 'rmw', 'cma'],
    'FF5+WML': ['mkt_excess', 'smb', 'hml', 'rmw', 'cma', 'wml'],
}

# Run for 25 Size-BM portfolios
print("Time-Series Alphas: 25 Size-BM Portfolios")
print("=" * 60)

ts_results = {}
for model_name, factor_cols in models.items():
    result = time_series_regressions(
        excess_25_bm, factors, factor_cols
    )
    ts_results[model_name] = result

    mean_alpha = result['alpha'].mean() * 12
    mean_abs_alpha = result['alpha'].abs().mean() * 12
    pct_sig = (result['alpha_p'] < 0.05).mean()
    mean_r2 = result['r_squared'].mean()

    print(f"\n{model_name}:")
    print(f"  Mean alpha (ann.): {mean_alpha:.4f}")
    print(f"  Mean |alpha| (ann.): {mean_abs_alpha:.4f}")
    print(f"  % sig. at 5%: {pct_sig:.1%}")
    print(f"  Mean R²: {mean_r2:.3f}")
```

22.3.2 The GRS Test

```
def grs_test(excess_returns, factor_df, factor_cols):
    """
    Gibbons, Ross, Shanken (1989) test of H0: all alphas = 0.

    Returns GRS statistic, p-value, and related diagnostics.
    """
```

```
    common = excess_returns.index.intersection(factor_df.index)
    Y = excess_returns.loc[common].values  # T x N
    F = factor_df.loc[common, factor_cols].values  # T x K

    T, N = Y.shape
    K = F.shape[1]

    # Add constant and run OLS for each asset
    F_const = np.column_stack([np.ones(T), F])

    # OLS: beta = (X'X)^-1 X'Y
    XtX_inv = inv(F_const.T @ F_const)
    B = XtX_inv @ F_const.T @ Y  # (K+1) x N

    alpha = B[0, :]  # N-vector of intercepts
    residuals = Y - F_const @ B  # T x N

    # Residual covariance matrix
    Sigma_eps = residuals.T @ residuals / (T - K - 1)

    # Factor mean and covariance
    f_bar = F.mean(axis=0)
    Sigma_f = np.cov(F, rowvar=False, ddof=1)

    # GRS statistic
    Sigma_eps_inv = inv(Sigma_eps)
    Sigma_f_inv = inv(Sigma_f) if K > 1 else np.array([[1 / Sigma_f]])

    sharpe_sq_alpha = alpha @ Sigma_eps_inv @ alpha
    sharpe_sq_f = f_bar @ Sigma_f_inv @ f_bar if K > 1 else (f_bar[0] ** 2 /
↪   Sigma_f)

    grs_stat = ((T - N - K) / N) * (1 / (1 + sharpe_sq_f)) * sharpe_sq_alpha

    # p-value from F distribution
    p_value = 1 - stats.f.cdf(grs_stat, N, T - N - K)

    # Additional diagnostics
    # Sharpe ratio of tangency portfolio of factors
    sr_factors = np.sqrt(sharpe_sq_f)
    # Sharpe ratio of tangency portfolio including alphas
    sr_alpha = np.sqrt(sharpe_sq_alpha + sharpe_sq_f)
```

```
    return {
        'GRS': grs_stat,
        'p_value': p_value,
        'df1': N,
        'df2': T - N - K,
        'mean_abs_alpha': np.abs(alpha).mean(),
        'sr_factors': sr_factors,
        'sr_alpha_plus_factors': sr_alpha,
        'sr_improvement': sr_alpha - sr_factors,
        'T': T, 'N': N, 'K': K
    }

# Run GRS for each model on each set of test assets
print("GRS Test Results")
print("=" * 80)
print(f"{'Model':<12} {'Test Assets':<18} {'GRS':>8} {'p-value':>10} "
      f"{'| | ann':>10} {'SR(f)':>8} {'SR( +f)':>8}")
print("-" * 80)

test_asset_sets = {
    '25 Size-BM': excess_25_bm,
    '25 Size-Mom': excess_25_mom,
    '10 Industry': excess_10_ind
}

grs_all = {}
for model_name, factor_cols in models.items():
    for ta_name, ta_data in test_asset_sets.items():
        key = f"{model_name}_{ta_name}"
        result = grs_test(ta_data, factors, factor_cols)
        grs_all[key] = result

        print(f"{model_name:<12} {ta_name:<18} "
              f"{result['GRS']:>8.2f} {result['p_value']:>10.4f} "
              f"{result['mean_abs_alpha']*12:>10.4f} "
              f"{result['sr_factors']:>8.3f} "
              f"{result['sr_alpha_plus_factors']:>8.3f}")
```

22.3.3 Model Comparison via GRS

```
ff5_result = ts_results['FF5']

# Reshape alphas into 5x5 matrix
alpha_matrix = ff5_result['alpha'].values.reshape(5, 5) * 12 * 100
t_matrix = ff5_result['alpha_t'].values.reshape(5, 5)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Alpha magnitudes
im = axes[0].imshow(alpha_matrix, cmap='RdBu_r', aspect='auto',
                    vmin=-np.max(np.abs(alpha_matrix)),
                    vmax=np.max(np.abs(alpha_matrix)))
for i in range(5):
    for j in range(5):
        sig = '*' if abs(t_matrix[i, j]) > 2 else ''
        axes[0].text(j, i, f'{alpha_matrix[i, j]:.1f}{sig}',
                    ha='center', va='center', fontsize=9,
                    color='white' if abs(alpha_matrix[i, j]) > 3 else
 ↪  'black')

axes[0].set_xticks(range(5))
axes[0].set_xticklabels(['Growth', '2', '3', '4', 'Value'])
axes[0].set_yticks(range(5))
axes[0].set_yticklabels(['Small', '2', '3', '4', 'Big'])
axes[0].set_xlabel('Book-to-Market')
axes[0].set_ylabel('Size')
axes[0].set_title('Panel A: FF5 Alphas (% ann.)')
plt.colorbar(im, ax=axes[0], label='Alpha (% ann.)')

# Panel B: R-squared
r2_matrix = ff5_result['r_squared'].values.reshape(5, 5) * 100
im2 = axes[1].imshow(r2_matrix, cmap='YlGn', aspect='auto',
                     vmin=50, vmax=100)
for i in range(5):
    for j in range(5):
        axes[1].text(j, i, f'{r2_matrix[i, j]:.0f}%',
                    ha='center', va='center', fontsize=9)

axes[1].set_xticks(range(5))
axes[1].set_xticklabels(['Growth', '2', '3', '4', 'Value'])
axes[1].set_yticks(range(5))
axes[1].set_yticklabels(['Small', '2', '3', '4', 'Big'])
axes[1].set_xlabel('Book-to-Market')
axes[1].set_ylabel('Size')
axes[1].set_title('Panel B: R-squared (%)')
plt.colorbar(im2, ax=axes[1], label='R² (%)')
```



```
plt.tight_layout()
plt.show()
```

Figure 22.1

```
fig, ax = plt.subplots(figsize=(12, 5))

model_names = list(models.keys())
ta_names = list(test_asset_sets.keys())
x = np.arange(len(model_names))
width = 0.25

colors_ta = ['#2C5F8A', '#C0392B', '#27AE60']

for i, ta_name in enumerate(ta_names):
    grs_vals = [grs_all[f"{m}_{ta_name}"]['GRS'] for m in model_names]
    bars = ax.bar(x + i * width, grs_vals, width, alpha=0.85,
                  color=colors_ta[i], label=ta_name, edgecolor='white')

ax.set_xticks(x + width)
ax.set_xticklabels(model_names)
ax.set_ylabel('GRS Statistic')
ax.set_title('GRS Test: Model Comparison Across Test Asset Sets')
ax.legend()

# Add critical value line (5% significance)
# Approximate: depends on N, T, K
ax.axhline(y=1.8, color='gray', linestyle='--', linewidth=1,
           label='~5% critical value')

plt.tight_layout()
plt.show()
```

Figure 22.2

22.4 Cross-Sectional Tests

22.4.1 Fama-MacBeth with Portfolio Test Assets

We implement the Fama-MacBeth two-pass procedure using the 25 size-BM portfolios as test assets:

```
def fama_macbeth_two_pass(excess_returns, factor_df, factor_cols,
                          beta_window='full', rolling_window=60,
                          shanken_correction=True):
    """
    Fama-MacBeth (1973) two-pass cross-sectional regression.

    Parameters
    ----------
    beta_window : str
        'full' for full-sample betas, 'rolling' for rolling betas.
    shanken_correction : bool
        Apply Shanken (1992) errors-in-variables correction.

    Returns
    -------
    Dictionary with estimated risk premia, t-statistics, and diagnostics.
    """
    common = excess_returns.index.intersection(factor_df.index)
    Y = excess_returns.loc[common]
    F = factor_df.loc[common, factor_cols]

    T = len(common)
    N = Y.shape[1]
    K = len(factor_cols)
    months = sorted(common)

    # ===== STAGE 1: Estimate betas =====
    if beta_window == 'full':
        # Full-sample betas (simpler, used in GRS context)
        F_const = sm.add_constant(F)
        betas = {}
        for col in Y.columns:
            model = sm.OLS(Y[col], F_const).fit()
            betas[col] = {f: model.params[f] for f in factor_cols}
        beta_df = pd.DataFrame(betas).T  # N x K
```

```
        # Same betas used every month
        beta_by_month = {m: beta_df for m in months}

    elif beta_window == 'rolling':
        # Rolling betas (more realistic, avoids look-ahead)
        beta_by_month = {}
        for t_idx in range(rolling_window, T):
            month = months[t_idx]
            window_start = months[t_idx - rolling_window]

            Y_win = Y.loc[window_start:months[t_idx - 1]]
            F_win = sm.add_constant(F.loc[window_start:months[t_idx - 1]])

            betas_t = {}
            for col in Y.columns:
                y = Y_win[col].dropna()
                x = F_win.loc[y.index]
                if len(y) < 24:
                    betas_t[col] = {f: np.nan for f in factor_cols}
                    continue
                model = sm.OLS(y, x).fit()
                betas_t[col] = {f: model.params[f] for f in factor_cols}

            beta_by_month[month] = pd.DataFrame(betas_t).T

    # ===== STAGE 2: Cross-sectional regressions =====
    gamma_list = []

    for month in months:
        if month not in beta_by_month:
            continue

        betas_t = beta_by_month[month]
        returns_t = Y.loc[month]

        # Align
        valid = betas_t.dropna().index.intersection(returns_t.dropna().index)
        if len(valid) < K + 5:
            continue

        y = returns_t[valid].values
        X = sm.add_constant(betas_t.loc[valid, factor_cols].values)
```

```
        try:
            model = sm.OLS(y, X).fit()
            gammas = {'month': month, 'gamma_0': model.params[0]}
            for j, f in enumerate(factor_cols):
                gammas[f'gamma_{f}'] = model.params[j + 1]
            gammas['r_squared'] = model.rsquared
            gammas['n_assets'] = len(valid)
            gamma_list.append(gammas)
        except Exception:
            pass

    gamma_df = pd.DataFrame(gamma_list)

    if len(gamma_df) == 0:
        return None

    # ===== INFERENCE =====
    # Time-series averages of gammas
    gamma_cols = ['gamma_0'] + [f'gamma_{f}' for f in factor_cols]

    results = {}
    for col in gamma_cols:
        g = gamma_df[col]
        mean = g.mean()
        se_fm = g.std() / np.sqrt(len(g))
        t_fm = mean / se_fm if se_fm > 0 else np.nan

        results[col] = {
            'estimate': mean,
            'se_fm': se_fm,
            't_fm': t_fm,
        }

    # Shanken (1992) correction for errors-in-variables
    if shanken_correction and beta_window == 'full':
        Sigma_f = F[factor_cols].cov().values

        # Shanken correction factor: (1 + lambda' Sigma_f^-1 lambda)
        gamma_factor = np.array([results[f'gamma_{f}']['estimate']
                                 for f in factor_cols])
        try:
            Sigma_f_inv = inv(Sigma_f)
            c_shanken = 1 + gamma_factor @ Sigma_f_inv @ gamma_factor
```

```
        except Exception:
            c_shanken = 1.0

        for col in gamma_cols:
            results[col]['se_shanken'] = results[col]['se_fm'] *
↪   np.sqrt(c_shanken)
            results[col]['t_shanken'] = (results[col]['estimate']
                                         / results[col]['se_shanken']
                                         if results[col]['se_shanken'] > 0
                                          ↪  else np.nan)

    # Compare estimated premia to factor mean returns
    factor_means = F[factor_cols].mean()
    for f in factor_cols:
        results[f'gamma_{f}']['factor_mean'] = factor_means[f]

    # Cross-sectional R-squared
    results['avg_cs_r2'] = gamma_df['r_squared'].mean()

    return {
        'results': pd.DataFrame(results).T,
        'gamma_ts': gamma_df,
        'beta_df': beta_by_month.get(months[-1], None)
    }

# Run Fama-MacBeth for each model
print("Fama-MacBeth Cross-Sectional Results: 25 Size-BM Portfolios")
print("=" * 80)

fm_results = {}
for model_name, factor_cols in models.items():
    for beta_type in ['full', 'rolling']:
        key = f"{model_name}_{beta_type}"
        fm = fama_macbeth_two_pass(
            excess_25_bm, factors, factor_cols,
            beta_window=beta_type,
            rolling_window=60,
            shanken_correction=(beta_type == 'full')
        )
        if fm is None:
            continue
        fm_results[key] = fm
```

```
        print(f"\n{model_name} (betas: {beta_type}):")
        res = fm['results']
        for idx, row in res.iterrows():
            if 'gamma' in idx:
                t_col = 't_shanken' if 't_shanken' in row and
↪   pd.notna(row.get('t_shanken')) else 't_fm'
                f_mean = row.get('factor_mean', '')
                f_str = f"  f_mean={f_mean:.4f}" if isinstance(f_mean, float)
↪   else ""
                print(f"  {idx:<16}:   = {row['estimate']:.5f}, "
                      f"t = {row[t_col]:.2f}{f_str}")
        if 'avg_cs_r2' in res.index:
            print(f"  Avg CS R²: {res.loc['avg_cs_r2', 'estimate']:.3f}")
```

22.4.2 The Shanken Correction

The Shanken (1992) correction addresses the errors-in-variables (EIV) problem inherent in the two-pass procedure. Because betas are estimated with error in Stage 1, the Stage 2 standard errors are understated. The correction inflates the standard errors by a factor that depends on the estimated risk premia and the factor covariance matrix:

$$\text{Var}(\hat{\gamma})_{\text{Shanken}} = \text{Var}(\hat{\gamma})_{\text{FM}} \times \left(1 + \hat{\gamma}' \hat{\Sigma}_f^{-1} \hat{\gamma}\right) \tag{22.5}$$

```
print("Impact of Shanken Correction on t-statistics:")
print(f"{'Model':<12} {'Factor':<12} {'t (FM)':>10} {'t (Shanken)':>12}
↪   {'Ratio':>8}")
print("-" * 54)

for model_name in models:
    key = f"{model_name}_full"
    if key not in fm_results:
        continue
    res = fm_results[key]['results']
    for idx, row in res.iterrows():
        if 'gamma_' in str(idx) and idx != 'gamma_0':
            t_fm = row.get('t_fm', np.nan)
            t_sh = row.get('t_shanken', np.nan)
            ratio = t_fm / t_sh if pd.notna(t_sh) and t_sh != 0 else np.nan
            factor_name = idx.replace('gamma_', '')
            print(f"{model_name:<12} {factor_name:<12} {t_fm:>10.2f} "
```

```
                  f"{t_sh:>12.2f} {ratio:>8.2f}")
```

22.4.3 Fama-MacBeth with Individual Stocks

Using individual stocks instead of portfolios as test assets avoids the Lewellen, Nagel, and Shanken (2010) critique that sorted portfolios create artificially strong factor structure:

```
def fama_macbeth_individual(stock_df, factor_df, factor_cols,
                            beta_window=60, min_obs=36,
                            min_stocks=100):
    """
    Fama-MacBeth with individual stocks and rolling betas.
    """
    stock_df = stock_df.copy()
    stock_df['month_end'] = pd.to_datetime(stock_df['month_end'])
    factor_df = factor_df.copy()

    months = sorted(stock_df['month_end'].unique())

    # Pre-compute rolling betas for all stocks
    gamma_list = []

    for t_idx, month in enumerate(months):
        if t_idx < beta_window:
            continue

        window_months = months[t_idx - beta_window:t_idx]

        # Get betas for each stock from rolling window
        betas_t = {}
        window_data = stock_df[stock_df['month_end'].isin(window_months)]

        for ticker, group in window_data.groupby('ticker'):
            if len(group) < min_obs:
                continue

            y = group.set_index('month_end')['monthly_return']
            x = factor_df.loc[y.index, factor_cols]
            x = sm.add_constant(x)

            valid = y.index.intersection(x.index)
```

```
            if len(valid) < min_obs:
                continue

            try:
                model = sm.OLS(y[valid], x.loc[valid]).fit()
                betas_t[ticker] = {f: model.params[f] for f in factor_cols}
            except Exception:
                pass

        if len(betas_t) < min_stocks:
            continue

        beta_df = pd.DataFrame(betas_t).T

        # Current month returns
        current = stock_df[stock_df['month_end'] == month]
        current = current.set_index('ticker')

        # Align
        valid_tickers = beta_df.index.intersection(current.index)
        if len(valid_tickers) < min_stocks:
            continue

        y = current.loc[valid_tickers, 'monthly_return'].values
        X = sm.add_constant(beta_df.loc[valid_tickers].values)

        try:
            model = sm.OLS(y, X).fit()
            gammas = {'month': month, 'gamma_0': model.params[0]}
            for j, f in enumerate(factor_cols):
                gammas[f'gamma_{f}'] = model.params[j + 1]
            gammas['n_stocks'] = len(valid_tickers)
            gammas['r_squared'] = model.rsquared
            gamma_list.append(gammas)
        except Exception:
            pass

gamma_df = pd.DataFrame(gamma_list)

# Inference
gamma_cols = ['gamma_0'] + [f'gamma_{f}' for f in factor_cols]
summary = {}
for col in gamma_cols:
```

```
        g = gamma_df[col]
        mean = g.mean()
        se = g.std() / np.sqrt(len(g))
        t = mean / se if se > 0 else np.nan
        summary[col] = {'estimate': mean, 'se': se, 't': t}

    summary['avg_n_stocks'] = gamma_df['n_stocks'].mean()
    summary['avg_cs_r2'] = gamma_df['r_squared'].mean()

    return pd.DataFrame(summary).T, gamma_df

# Run for FF5
print("\nFama-MacBeth with Individual Stocks (FF5):")
fm_ind_results, fm_ind_gamma = fama_macbeth_individual(
    stock_returns, factors,
    ['mkt_excess', 'smb', 'hml', 'rmw', 'cma'],
    beta_window=60, min_obs=36
)
print(fm_ind_results.round(4).to_string())
```

22.5 When Do the Tests Disagree?

22.5.1 Theoretical Sources of Disagreement

Goyal and Jegadeesh (2018) identify three sources of discrepancy between time-series and cross-sectional tests:

1. Factor structure of test assets. If test assets have a strong factor structure (as sorted portfolios do by construction), the cross-sectional R^2 will be high regardless of whether the factors truly price the assets. Lewellen, Nagel, and Shanken (2010) show that even random factors can achieve high cross-sectional R^2 when test assets are size-BM sorted portfolios, because such portfolios have a strong linear structure in the size-value space.

2. Time-variation in betas. The time-series regression assumes constant betas. If betas vary over time and co-move with the factor risk premium, the unconditional time-series alpha can be nonzero even if the conditional model holds (Jagannathan and Wang 1996b).

3. Misspecified risk premia. The Fama-MacBeth cross-sectional regression estimates the risk premium from the data. The time-series regression implicitly sets the risk premium equal to the factor's sample mean return. When these differ (e.g., because the factor is not a perfect proxy for the underlying risk), the two approaches disagree.

22.5.2 Empirical Comparison for Vietnam

```
# For FF5 on 25 Size-BM portfolios:
# Time-series: alpha from OLS regression
# Cross-section: pricing error from Fama-MacBeth

ts_alphas = ts_results['FF5']['alpha'].values * 12  # Annualized
ts_names = ts_results['FF5'].index.tolist()

# Cross-sectional pricing errors:
# Predicted return = beta' * estimated_gamma
# Pricing error = actual mean return - predicted
fm_full = fm_results.get('FF5_full')
if fm_full:
    betas = fm_full['beta_df']  # N x K
    factor_cols = ['mkt_excess', 'smb', 'hml', 'rmw', 'cma']
    gammas = np.array([fm_full['results'].loc[f'gamma_{f}', 'estimate']
                       for f in factor_cols])
    gamma_0 = fm_full['results'].loc['gamma_0', 'estimate']

    actual_means = excess_25_bm.mean() * 12  # Annualized
    predicted = (gamma_0 + betas[factor_cols].values @ gammas) * 12
    cs_errors = actual_means.values - predicted
```

22.5.3 The Lewellen-Nagel-Shanken Critique

Lewellen, Nagel, and Shanken (2010) demonstrate that the high cross-sectional R^2 from Fama-MacBeth regressions on sorted portfolios can be misleading. Sorted portfolios have a strong factor structure by construction, so even irrelevant factors can produce high R^2. They recommend:

1. Including industry portfolios alongside sorted portfolios to break the mechanical factor structure.
2. Constraining the estimated risk premia to equal the factor mean returns (this is what the time-series test implicitly does).
3. Reporting the cross-sectional R^2 from the *constrained* model alongside the unconstrained estimate.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: TS alpha vs CS pricing error
if fm_full:
    axes[0].scatter(ts_alphas * 100, cs_errors * 100,
                    color='#2C5F8A', s=60, alpha=0.7, edgecolors='white')
    lim = max(np.max(np.abs(ts_alphas)), np.max(np.abs(cs_errors))) * 100 + 1
    axes[0].plot([-lim, lim], [-lim, lim], 'k--', linewidth=1)
    axes[0].plot([-lim, lim], [0, 0], color='gray', linewidth=0.5)
    axes[0].plot([0, 0], [-lim, lim], color='gray', linewidth=0.5)
    axes[0].set_xlabel('Time-Series Alpha (% ann.)')
    axes[0].set_ylabel('Cross-Sectional Pricing Error (% ann.)')
    axes[0].set_title('Panel A: TS Alpha vs CS Pricing Error')

    # Correlation
    rho = np.corrcoef(ts_alphas, cs_errors)[0, 1]
    axes[0].text(0.05, 0.95, f'  = {rho:.2f}',
                 transform=axes[0].transAxes, fontsize=11)

# Panel B: Actual vs Predicted (CS)
if fm_full:
    axes[1].scatter(predicted * 100, actual_means.values * 100,
                    color='#C0392B', s=60, alpha=0.7, edgecolors='white')
    lim2 = max(np.max(np.abs(predicted)), np.max(np.abs(actual_means))) * 100
 ↪  + 2
    axes[1].plot([-5, lim2], [-5, lim2], 'k--', linewidth=1)
    axes[1].set_xlabel('Predicted Average Return (% ann.)')
    axes[1].set_ylabel('Actual Average Return (% ann.)')
    axes[1].set_title('Panel B: Actual vs Predicted (FM Cross-Section)')

    # CS R-squared
    ss_res = np.sum((actual_means.values - predicted) ** 2)
    ss_tot = np.sum((actual_means.values - actual_means.values.mean()) ** 2)
    r2_cs = 1 - ss_res / ss_tot
    axes[1].text(0.05, 0.95, f'CS R² = {r2_cs:.2f}',
                 transform=axes[1].transAxes, fontsize=11)

plt.tight_layout()
plt.show()
```

Figure 22.3

```python
def lns_diagnostic(excess_returns, factor_df, factor_cols):
    """
    Lewellen-Nagel-Shanken (2010) diagnostic:
    Compare unconstrained CS R² to constrained (factor mean = premium).
    """
    common = excess_returns.index.intersection(factor_df.index)
    Y = excess_returns.loc[common]
    F = factor_df.loc[common, factor_cols]

    # Full-sample betas
    betas = {}
    F_const = sm.add_constant(F)
    for col in Y.columns:
        model = sm.OLS(Y[col], F_const).fit()
        betas[col] = {f: model.params[f] for f in factor_cols}
    beta_mat = pd.DataFrame(betas).T  # N x K

    actual = Y.mean() * 12  # Annualized

    # Unconstrained: FM regression
    X = sm.add_constant(beta_mat.values)
    fm_model = sm.OLS(actual.values, X).fit()
    predicted_unc = fm_model.fittedvalues
    r2_unconstrained = fm_model.rsquared

    # Constrained: premium = factor mean return
    factor_means = F.mean().values * 12  # Annualized
    predicted_con = beta_mat.values @ factor_means
    # Add best-fitting intercept
    intercept_con = np.mean(actual.values - predicted_con)
    predicted_con += intercept_con

    ss_res_con = np.sum((actual.values - predicted_con) ** 2)
    ss_tot = np.sum((actual.values - actual.values.mean()) ** 2)
    r2_constrained = 1 - ss_res_con / ss_tot

    return {
        'r2_unconstrained': r2_unconstrained,
        'r2_constrained': r2_constrained,
        'r2_drop': r2_unconstrained - r2_constrained,
        'gamma_unconstrained': fm_model.params[1:],
        'gamma_constrained': factor_means
    }
```

```
print("Lewellen-Nagel-Shanken Diagnostic:")
print(f"{'Model':<12} {'Test Assets':<18} {'R² (unc)':>10} {'R² (con)':>10} "
      f"{'Drop':>8}")
print("-" * 58)

for model_name, factor_cols in models.items():
    for ta_name, ta_data in test_asset_sets.items():
        diag = lns_diagnostic(ta_data, factors, factor_cols)
        print(f"{model_name:<12} {ta_name:<18} "
              f"{diag['r2_unconstrained']:>10.3f} "
              f"{diag['r2_constrained']:>10.3f} "
              f"{diag['r2_drop']:>8.3f}")
```

22.6 GMM-Based Tests

22.6.1 The Stochastic Discount Factor Framework

Both time-series and cross-sectional tests are special cases of the L. P. Hansen (1982) Generalized Method of Moments (GMM) framework. The fundamental pricing equation is:

$$E[M_t R^e_{i,t}] = 0 \quad \text{for all } i \tag{22.6}$$

where M_t is the stochastic discount factor (SDF) and $R^e_{i,t}$ is the excess return of asset i. A linear factor model parameterizes the SDF as:

$$M_t = 1 - b'(f_t - \mu_f) \tag{22.7}$$

where b is the vector of SDF loadings and f_t are the factors. The GMM approach estimates b by minimizing the quadratic form of the pricing errors:

$$\hat{b} = \arg\min_b \; g(b)' W \; g(b) \tag{22.8}$$

where $g(b) = \frac{1}{T}\sum_t M_t(b) R^e_t$ is the sample analog of the moment conditions.

```
diag_ff5 = lns_diagnostic(excess_25_bm, factors,
                          ['mkt_excess', 'smb', 'hml', 'rmw', 'cma'])

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

actual = excess_25_bm.mean().values * 12 * 100

# Panel A: Unconstrained
F_const = sm.add_constant(factors[['mkt_excess', 'smb', 'hml', 'rmw',
 ↪  'cma']])
betas_full = {}
for col in excess_25_bm.columns:
    common = excess_25_bm.index.intersection(factors.index)
    model = sm.OLS(excess_25_bm.loc[common, col],
                   F_const.loc[common]).fit()
    betas_full[col] = {f: model.params[f] for f in
                       ['mkt_excess', 'smb', 'hml', 'rmw', 'cma']}
beta_mat = pd.DataFrame(betas_full).T
X_cs = sm.add_constant(beta_mat.values)
fm_mod = sm.OLS(actual, X_cs).fit()
pred_unc = fm_mod.fittedvalues

axes[0].scatter(pred_unc, actual, color='#2C5F8A', s=60, alpha=0.7,
                edgecolors='white')
rng_plot = [min(pred_unc.min(), actual.min()) - 1,
            max(pred_unc.max(), actual.max()) + 1]
axes[0].plot(rng_plot, rng_plot, 'k--', linewidth=1)
axes[0].set_xlabel('Predicted (% ann.)')
axes[0].set_ylabel('Actual (% ann.)')
axes[0].set_title(f"Panel A: Unconstrained (R² = "
                  f"{diag_ff5['r2_unconstrained']:.2f})")

# Panel B: Constrained
factor_means = factors[['mkt_excess', 'smb', 'hml', 'rmw',
 ↪  'cma']].mean().values * 12 * 100
pred_con = beta_mat.values @ factor_means
pred_con += np.mean(actual - pred_con)

axes[1].scatter(pred_con, actual, color='#C0392B', s=60, alpha=0.7,
                edgecolors='white')
rng2 = [min(pred_con.min(), actual.min()) - 1,
        max(pred_con.max(), actual.max()) + 1]
axes[1].plot(rng2, rng2, 'k--', linewidth=1)
axes[1].set_xlabel('Predicted (% ann.)')
axes[1].set_ylabel('Actual (% ann.)')
axes[1].set_title(f"Panel B: Constrained (R² = "
                  f"{diag_ff5['r2_constrained']:.2f})")

plt.tight_layout()
plt.show()
```


Figure 22.4

```
def gmm_sdf(excess_returns, factor_df, factor_cols,
            weighting='identity', n_iter=2):
    """
    GMM estimation of the linear SDF model.

    Parameters
    ----------
    weighting : str
        'identity': first-stage identity weighting matrix
        'optimal': Hansen optimal weighting (iterated)
    n_iter : int
        Number of GMM iterations (2 = efficient two-step)
    """
    common = excess_returns.index.intersection(factor_df.index)
    Re = excess_returns.loc[common].values  # T x N
    F = factor_df.loc[common, factor_cols].values  # T x K

    T, N = Re.shape
    K = F.shape[1]

    # Demean factors
    F_dm = F - F.mean(axis=0)

    # Moment conditions: E[M * Re] = 0
    # M = 1 - b' (f - mu_f) = 1 - b' F_dm
    def pricing_errors(b):
        M = 1 - F_dm @ b  # T-vector
        g = (M[:, np.newaxis] * Re).mean(axis=0)  # N-vector
        return g

    # GMM objective
    W = np.eye(N)  # Initial weighting matrix

    from scipy.optimize import minimize

    for iteration in range(n_iter):
        def objective(b):
            g = pricing_errors(b)
            return T * g @ W @ g

        result = minimize(objective, np.zeros(K), method='L-BFGS-B')
        b_hat = result.x
```

```
        if iteration < n_iter - 1:
            # Update weighting matrix (Hansen optimal)
            g_t = np.array([(1 - F_dm[t] @ b_hat) * Re[t]
                            for t in range(T)])  # T x N
            S = np.cov(g_t, rowvar=False, ddof=0)

            # Newey-West adjustment for serial correlation
            max_lag = int(np.floor(4 * (T / 100) ** (2 / 9)))
            for lag in range(1, max_lag + 1):
                w = 1 - lag / (max_lag + 1)
                Gamma = np.cov(g_t[lag:].T, g_t[:-lag].T, ddof=0)[:N, N:]
                S += w * (Gamma + Gamma.T)

            try:
                W = inv(S)
            except Exception:
                W = np.eye(N)

    # Pricing errors at optimum
    g_hat = pricing_errors(b_hat)

    # Implied risk premia: lambda = Sigma_f @ b
    Sigma_f = np.cov(F, rowvar=False, ddof=1)
    lambda_hat = Sigma_f @ b_hat

    # J-test (overidentification)
    J = T * g_hat @ W @ g_hat
    df_J = N - K
    p_J = 1 - stats.chi2.cdf(J, df_J) if df_J > 0 else np.nan

    # HJ distance (model misspecification)
    hj_dist = np.sqrt(g_hat @ inv(np.cov(Re, rowvar=False)) @ g_hat)

    return {
        'b_hat': b_hat,
        'lambda_hat': lambda_hat,
        'pricing_errors': g_hat,
        'J_stat': J,
        'J_pvalue': p_J,
        'J_df': df_J,
        'hj_distance': hj_dist,
        'mean_abs_error': np.abs(g_hat).mean() * 12
    }
```

```
print("GMM SDF Estimation:")
print(f"{'Model':<12} {'Test Assets':<18} {'J-stat':>8} {'J p-val':>10} "
      f"{'HJ dist':>8} {'|e| ann':>10}")
print("-" * 66)

for model_name, factor_cols in models.items():
    for ta_name, ta_data in test_asset_sets.items():
        gmm = gmm_sdf(ta_data, factors, factor_cols, n_iter=2)
        print(f"{model_name:<12} {ta_name:<18} "
              f"{gmm['J_stat']:>8.2f} {gmm['J_pvalue']:>10.4f} "
              f"{gmm['hj_distance']:>8.4f} {gmm['mean_abs_error']:>10.4f}")

        # Print implied risk premia
        for j, f in enumerate(factor_cols):
            print(f"   _{f}: {gmm['lambda_hat'][j]*12:.4f} "
                  f"(factor mean: {factors[f].mean()*12:.4f})")
```

22.7 Model Comparison

22.7.1 The Barillas-Shanken Framework

Barillas and Shanken (2018) propose a Bayesian framework for comparing non-nested factor models. The key insight is that model comparison should be based on the factors' ability to explain each other's returns, not on their ability to price a fixed set of test assets. Two models differ only in their excluded factors, so the relevant comparison is whether the factors unique to each model have alpha with respect to the other model.

```
def barillas_shanken_compare(factor_df, model_a_cols, model_b_cols):
    """
    Barillas-Shanken (2018) pairwise model comparison.

    Compare Model A vs Model B by testing whether the factors
    unique to each model have alpha with respect to the other.
    """
    # Factors unique to each model
    unique_a = [f for f in model_a_cols if f not in model_b_cols]
    unique_b = [f for f in model_b_cols if f not in model_a_cols]

    results = {}
```

```
    # Test: do factors unique to A have alpha w.r.t. B?
    # If yes, A adds value beyond B
    if unique_a:
        for f in unique_a:
            y = factor_df[f]
            X = sm.add_constant(factor_df[model_b_cols])
            common = y.dropna().index.intersection(X.dropna().index)
            model = sm.OLS(y[common], X.loc[common]).fit(
                cov_type='HAC', cov_kwds={'maxlags': 6}
            )
            results[f'alpha_{f}_vs_B'] = {
                'alpha': model.params['const'] * 12,
                't': model.tvalues['const'],
                'r2': model.rsquared
            }

    if unique_b:
        for f in unique_b:
            y = factor_df[f]
            X = sm.add_constant(factor_df[model_a_cols])
            common = y.dropna().index.intersection(X.dropna().index)
            model = sm.OLS(y[common], X.loc[common]).fit(
                cov_type='HAC', cov_kwds={'maxlags': 6}
            )
            results[f'alpha_{f}_vs_A'] = {
                'alpha': model.params['const'] * 12,
                't': model.tvalues['const'],
                'r2': model.rsquared
            }

    return pd.DataFrame(results).T

# Compare FF3 vs FF5
print("Barillas-Shanken: FF3 vs FF5")
bs_3v5 = barillas_shanken_compare(
    factors,
    ['mkt_excess', 'smb', 'hml'],
    ['mkt_excess', 'smb', 'hml', 'rmw', 'cma']
)
print(bs_3v5.round(4).to_string())

# Compare FF5 vs FF5+WML
```

```
print("\nBarillas-Shanken: FF5 vs FF5+WML")
bs_5vw = barillas_shanken_compare(
    factors,
    ['mkt_excess', 'smb', 'hml', 'rmw', 'cma'],
    ['mkt_excess', 'smb', 'hml', 'rmw', 'cma', 'wml']
)
print(bs_5vw.round(4).to_string())

# Compare FF3+WML vs FF5+WML
print("\nBarillas-Shanken: FF3+WML vs FF5+WML")
bs_3wv5w = barillas_shanken_compare(
    factors,
    ['mkt_excess', 'smb', 'hml', 'wml'],
    ['mkt_excess', 'smb', 'hml', 'rmw', 'cma', 'wml']
)
print(bs_3wv5w.round(4).to_string())
```

22.7.2 Comprehensive Model Scorecard

22.8 Stepwise Factor-Model Selection

The Barillas-Shanken framework compares two models at a time. But the factor zoo (Section 21.1) offers dozens of candidate factors, and the practical question is not "is model A better than model B?" but "which subset of all available factors constitutes the best model?" Feng et al. (2026) answer this with a **stepwise selection** procedure that builds a factor model sequentially, adding and removing factors on the basis of asset-pricing tests, and that tests the resulting model against both the test assets and the factors it left out. Applied to US data, their procedure rejects the workhorse models (CAPM, the Fama-French three- and five-factor models) and selects an eight-factor model that survives the pricing tests and performs robustly out of sample. This section develops the method from first principles and provides a complete, runnable implementation.

The intellectual lineage is worth naming. Feng, Giglio, and Xiu (2020) ("Taming the Factor Zoo") use double-selection LASSO to test whether a *new* factor earns a risk premium after controlling for a high-dimensional set of existing ones. Feng et al. (2026) instead select the *model* itself, and their stopping rule leans on the observation—formalized by Barillas and Shanken (2018)—that comparing factor models reduces to comparing the maximum squared Sharpe ratios their factors attain.

```
scorecard = {}

for model_name, factor_cols in models.items():
    metrics = {'Model': model_name}

    # Time-series (25 Size-BM)
    grs_result = grs_all.get(f"{model_name}_25 Size-BM")
    if grs_result:
        metrics['GRS (BM)'] = grs_result['GRS']
        metrics['GRS p'] = grs_result['p_value']
        metrics['|α| (BM)'] = grs_result['mean_abs_alpha'] * 12

    # Cross-section
    fm_key = f"{model_name}_full"
    if fm_key in fm_results:
        metrics['CS R² (unc)'] = fm_results[fm_key]['results'].get(
            'avg_cs_r2', {}).get('estimate', np.nan)

    # LNS constrained
    lns = lns_diagnostic(excess_25_bm, factors, factor_cols)
    metrics['CS R² (con)'] = lns['r2_constrained']

    # GMM
    gmm = gmm_sdf(excess_25_bm, factors, factor_cols, n_iter=2)
    metrics['HJ dist'] = gmm['hj_distance']
    metrics['J p-val'] = gmm['J_pvalue']

    # Time-series R²
    ts_res = time_series_regressions(excess_25_bm, factors, factor_cols)
    metrics['Avg R²'] = ts_res['r_squared'].mean()

    scorecard[model_name] = metrics

scorecard_df = pd.DataFrame(scorecard).T
print("Model Scorecard:")
print(scorecard_df.round(3).to_string())
```

Figure 22.5

22.8.1 The Squared-Sharpe-Ratio Spanning Duality

The engine of the method is a duality between mean-variance efficiency and the spanning regressions of the previous section. For a set of factors F with mean vector μ_F and covariance Σ_F, the **maximum squared Sharpe ratio** attainable from those factors is

$$\mathrm{Sh}^2(F) = \mu_F' \Sigma_F^{-1} \mu_F. \tag{22.9}$$

Barillas and Shanken (2018) show that the increment in the maximum squared Sharpe ratio from adding a candidate factor g to a model F has a simple closed form in terms of the spanning regression $g_t = \alpha_g + \beta' F_t + \varepsilon_t$:

$$\mathrm{Sh}^2(F \cup \{g\}) - \mathrm{Sh}^2(F) = \frac{\alpha_g^2}{\sigma^2(\varepsilon_g)}. \tag{22.10}$$

This equation is the whole method in miniature. The left-hand side is what we want to maximize (a model that spans more of the efficient frontier prices more assets). The right-hand side says that maximizing it is equivalent to finding the candidate factor with the largest ratio of squared spanning alpha to residual variance—and that a factor adds *nothing* to the frontier precisely when its spanning alpha is zero. Testing $\alpha_g = 0$ (a single-equation version of the GRS test of Section 22.3) is therefore both the **selection criterion** (add the factor with the most significant, largest-increment alpha) and the **stopping rule** (stop when no unselected candidate has a non-zero alpha, i.e., when the model spans all of them).

This is exactly the logic of the Barillas-Shanken comparison in Section 22.7, promoted from a pairwise test into a search over all candidate factors.

22.8.2 Implementing the Stepwise Algorithm

We first define three self-contained helpers: the maximum squared Sharpe ratio of Equation 22.9, the spanning-alpha test that drives selection (returning the sample increment $\hat{\alpha}^2/\hat{\sigma}^2(\varepsilon)$ of Equation 22.10), and the GRS test used to judge the final model.

```
def squared_sharpe(R):
    """Maximum squared Sharpe ratio of a set of excess-return columns."""
    mu = R.mean(axis=0).values
    Sigma = np.cov(R.values, rowvar=False, ddof=1)
    if R.shape[1] == 1:
        return float(mu[0] ** 2 / Sigma)
    return float(mu @ np.linalg.inv(Sigma) @ mu)
```

```
def spanning_alpha(g, F, maxlags=6):
    """Regress candidate factor g on model factors F; HAC alpha test.
    The sample SR^2 increment from adding g equals alpha^2 /
 ↪  var(residual)."""
    common = g.dropna().index.intersection(F.dropna().index)
    res = sm.OLS(g.loc[common], sm.add_constant(F.loc[common])).fit(
        cov_type='HAC', cov_kwds={'maxlags': maxlags})
    alpha = res.params['const']
    return {'alpha': alpha, 't': res.tvalues['const'], 'p':
     ↪  res.pvalues['const'],
            'sr2_increment': alpha ** 2 / res.resid.var(ddof=1)}


def grs_test(excess_assets, factor_df, factor_cols):
    """Gibbons-Ross-Shanken (1989) test that all alphas are jointly zero."""
    common =
 ↪  excess_assets.dropna().index.intersection(factor_df.dropna().index)
    Y = excess_assets.loc[common].values
    F = factor_df.loc[common, factor_cols].values
    T, N = Y.shape
    K = F.shape[1]
    Fc = np.column_stack([np.ones(T), F])
    B = np.linalg.inv(Fc.T @ Fc) @ Fc.T @ Y
    alpha, resid = B[0, :], Y - Fc @ B
    Sig_e = resid.T @ resid / (T - K - 1)
    fbar, Sig_f = F.mean(0), np.cov(F, rowvar=False, ddof=1)
    sh2_a = alpha @ np.linalg.inv(Sig_e) @ alpha
    sh2_f = (fbar @ np.linalg.inv(np.atleast_2d(Sig_f)) @ fbar
             if K > 1 else fbar[0] ** 2 / Sig_f)
    grs = ((T - N - K) / N) * (1 / (1 + sh2_f)) * sh2_a
    return {'GRS': grs, 'p_value': 1 - stats.f.cdf(grs, N, T - N - K),
            'mean_abs_alpha': np.abs(alpha).mean()}
```

The selection routine implements the forward and backward passes. The **forward** pass ranks every unselected candidate by its squared-Sharpe increment and adds the leader if its spanning alpha is significant; the **backward** pass removes any previously selected factor that has become redundant now that other factors are present (its alpha on the remaining model is no longer significant). The loop halts when no candidate is unspanned, at which point the selected factors span the joint efficient frontier of themselves and all rejected candidates.

```
def stepwise_select(factors, candidate_cols, baseline_cols, test_assets=None,
                    add_threshold=0.05, verbose=True):
    """Forward/backward stepwise asset-pricing-model selection (Feng et al.
     ↪  2026).

    Forward: add the unselected candidate with the largest SR^2 increment
             (equivalently, the most significant spanning alpha) while some
             candidate remains unspanned by the current model.
    Backward: drop any selected non-baseline factor that has become
↪   redundant.
    """
    selected = list(baseline_cols)
    remaining = [c for c in candidate_cols if c not in selected]
    history = [{'action': 'baseline', 'model': list(selected),
                'sharpe_ann': np.sqrt(squared_sharpe(factors[selected]) *
                 ↪  12)}]
    while remaining:
        scored = sorted(
            ((g, spanning_alpha(factors[g], factors[selected])) for g in
↪   remaining),
            key=lambda kv: kv[1]['sr2_increment'], reverse=True)
        g, s = scored[0]
        if s['p'] > add_threshold:
            break                       # no unspanned candidate -> frontier
             ↪  spanned
        selected.append(g); remaining.remove(g)
        sr2 = squared_sharpe(factors[selected])
        history.append({'action': f'+{g}', 'model': list(selected),
                        'sharpe_ann': np.sqrt(sr2 * 12), 't_alpha': s['t']})
        if verbose:
            print(f"  + {g:>4}  t(alpha)={s['t']:5.2f}
             ↪  SR2_incr={s['sr2_increment']:.4f}"
                  f"  -> model SR2={sr2:.3f}
                   ↪  Sharpe(ann)={np.sqrt(sr2*12):.2f}")
        for f in [x for x in selected if x not in baseline_cols and x != g]:
            s2 = spanning_alpha(factors[f], factors[[x for x in selected if x
↪   != f]])
            if s2['p'] > add_threshold:
                selected.remove(f); remaining.append(f)
                if verbose:
                    print(f"  - {f:>4}  dropped (now spanned, alpha
                     ↪  p={s2['p']:.3f})")
```

```
    diag = {'selected': selected, 'history': history,
            'sharpe_ann': np.sqrt(squared_sharpe(factors[selected]) * 12)}
    if test_assets is not None:
        diag['grs'] = grs_test(test_assets, factors, selected)
    diag['unselected'] = {g: spanning_alpha(factors[g], factors[selected])
                          for g in candidate_cols if g not in selected}
    return diag
```

A note on the stopping rule. With a handful of candidates, a per-factor significance threshold on the spanning alpha is adequate. When the candidate set is large, the individual tests suffer the multiple-comparison problem of Section 21.3, and Feng et al. (2026) instead stop on the **high-dimensional alpha test** of Pesaran and Yamagata (2024), which is designed to test the joint nullity of alphas when the number of test assets is large relative to the sample. In the implementation above, tightening `add_threshold` (for example to a Bonferroni level 0.05/#candidates) is a simple and conservative substitute.

22.8.3 Replication: Recovering a Known Model

Before trusting the procedure on real data, we verify it on simulated data where the correct answer is known by construction. We generate a true four-factor world—a market factor plus three near-orthogonal priced factors with realistic anomaly Sharpe ratios—and contaminate the candidate set with two **redundant** factors (linear combinations of the true factors plus noise, so they carry positive average returns but zero alpha once their parents are included) and one **decoy** with no risk premium at all. Twenty test portfolios are priced by the true four factors with zero alpha.

```
rng = np.random.default_rng(2026)
T = 480
idx = pd.date_range('1986-01-31', periods=T, freq='ME')
mkt = rng.normal(0.006, 0.043, T)
f2 = rng.normal(0.0050, 0.020, T)   # value-like
f3 = rng.normal(0.0045, 0.018, T)   # profitability-like
f4 = rng.normal(0.0050, 0.021, T)   # momentum-like
r1 = 0.8 * f2 + rng.normal(0, 0.012, T)              # redundant shadow of f2
r2 = 0.5 * f3 + 0.5 * f4 + rng.normal(0, 0.012, T)   # redundant combo of f3,
 ↪ f4
d1 = rng.normal(0.0002, 0.025, T)                    # zero-Sharpe decoy
sim_factors = pd.DataFrame(
    {'mkt': mkt, 'f2': f2, 'f3': f3, 'f4': f4,
     'r1': r1, 'r2': r2, 'd1': d1}, index=idx)
```

```
N = 20
loadings = rng.uniform(-0.6, 1.4, size=(N, 4))
sim_test = pd.DataFrame(
    np.column_stack([mkt, f2, f3, f4]) @ loadings.T + rng.normal(0, 0.015,
 ↪  (T, N)),
    index=idx, columns=[f'P{i:02d}' for i in range(N)])

candidates = ['f2', 'f3', 'f4', 'r1', 'r2', 'd1']

print("GRS on the 20 test assets (workhorse baselines):")
for name, cols in [('CAPM', ['mkt']),
                   ('true 4-factor', ['mkt', 'f2', 'f3', 'f4']),
                   ('all-7 kitchen sink', ['mkt'] + candidates)]:
    g = grs_test(sim_test, sim_factors, cols)
    verdict = 'REJECTED' if g['p_value'] < 0.05 else 'not rejected'
    print(f"  {name:<20} GRS={g['GRS']:.2f}  p={g['p_value']:.3f}  ->
     ↪  {verdict}")
```

```
GRS on the 20 test assets (workhorse baselines):
  CAPM                 GRS=2.96  p=0.000  ->  REJECTED
  true 4-factor        GRS=0.33  p=0.998  ->  not rejected
  all-7 kitchen sink   GRS=0.35  p=0.997  ->  not rejected
```

The CAPM is decisively rejected on these test assets, while the true four-factor model (and the redundant "kitchen sink" that nests it) is not—exactly the pattern Feng et al. (2026) document for the workhorse models on real US data. Now we let the algorithm discover the model on its own, starting from the CAPM:

```
diag = stepwise_select(sim_factors, candidates, ['mkt'], sim_test)

print(f"\nSelected model : {diag['selected']}")
print(f"Final Sharpe   : {diag['sharpe_ann']:.2f} (annualized)")
print(f"GRS on assets  : {diag['grs']['GRS']:.2f}
 ↪  p={diag['grs']['p_value']:.3f}")
print("Unselected candidates' spanning alphas (should be insignificant):")
for k, v in diag['unselected'].items():
    print(f"   {k:>3}: alpha_ann={v['alpha']*12:+.4f}  t={v['t']:+.2f}
     ↪  p={v['p']:.3f}")
```

```
  +   f2  t(alpha)= 5.44  SR2_incr=0.0658  -> model SR2=0.096  Sharpe(ann)=1.07
  +   f3  t(alpha)= 3.58  SR2_incr=0.0296  -> model SR2=0.126  Sharpe(ann)=1.23
```

```
  +   f4  t(alpha)= 3.26  SR2_incr=0.0243  -> model SR2=0.150  Sharpe(ann)=1.34

Selected model : ['mkt', 'f2', 'f3', 'f4']
Final Sharpe   : 1.34 (annualized)
GRS on assets  : 0.33  p=0.998
Unselected candidates' spanning alphas (should be insignificant):
    r1: alpha_ann=-0.0053  t=-0.80  p=0.425
    r2: alpha_ann=-0.0104  t=-1.41  p=0.158
    d1: alpha_ann=+0.0195  t=+1.28  p=0.200
```

The procedure recovers exactly the true model `['mkt', 'f2', 'f3', 'f4']`: it adds the three priced factors in order of their squared-Sharpe increment (each with a large, significant spanning t-statistic) and then halts, because every remaining candidate is now spanned. The two redundant factors and the decoy are left out, and their post-selection spanning alphas are small and insignificant—the model prices them, which is the property Feng et al. (2026) require of a valid model. The final GRS test on the test assets is comfortably non-rejected.

The economic content of the selection is the expansion of the efficient frontier, which we plot as the annualized Sharpe ratio of the selected model at each step.

```
hist = pd.DataFrame(diag['history'])
fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(range(len(hist)), hist['sharpe_ann'], 'o-', lw=2, color='#2C5282')
for i, row in hist.iterrows():
    ax.annotate(row['action'], (i, row['sharpe_ann']),
                textcoords='offset points', xytext=(0, 10), ha='center')
ax.set_xlabel('Stepwise iteration')
ax.set_ylabel('Annualized Sharpe ratio of selected model')
ax.set_title('Efficient-frontier expansion as factors are added')
plt.tight_layout()
plt.show()
```

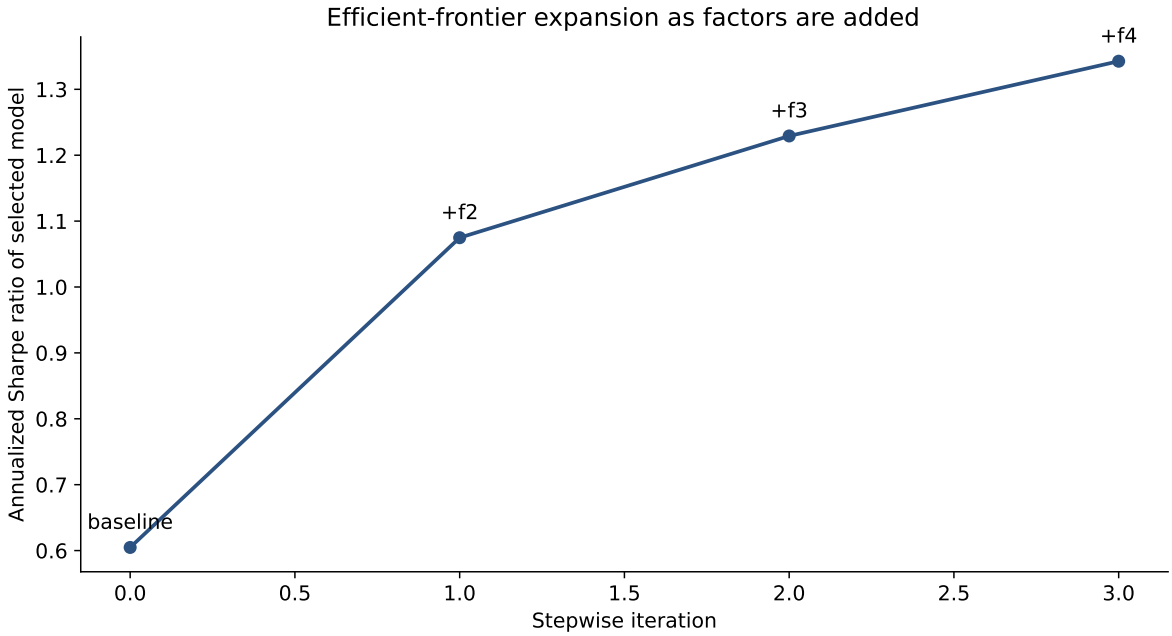


Figure 22.6: Efficient-frontier expansion under stepwise selection. Each point is the maximum annualized Sharpe ratio attainable from the model after that step; labels mark the factor added. The curve flattens as the added factors span progressively less of the residual frontier, and selection halts when no candidate improves it significantly.

22.8.4 Application to Vietnamese Factor Models

The procedure transfers directly to Vietnamese data: replace `sim_factors` with the locally constructed factor panel of Section 21.2 (market, size, value, profitability, investment, momentum, and any additional anomaly long-short returns that survive the multiple-testing screen of Section 21.4) and `sim_test` with a set of Vietnamese test portfolios (the size-BM and size-momentum sorts of Section 22.2). The baseline is the market factor; the candidate set is every anomaly factor with adequate coverage.

```
# `factors` and the test-portfolio frames come from the DataCore pipeline
# built earlier in this chapter (see the data-load chunk).
vn_candidates = ['smb', 'hml', 'rmw', 'cma', 'wml']  # extend with local
 ↪  anomalies

vn_result = stepwise_select(
    factors=factors,
    candidate_cols=vn_candidates,
    baseline_cols=['mkt_excess'],
```

```
    test_assets=excess_25_bm,        # 25 size-BM Vietnamese portfolios
    add_threshold=0.05 / len(vn_candidates)  # Bonferroni-style, see caveat
 ↪  below
)
print("Vietnamese stepwise model:", vn_result['selected'])
print(f"GRS p-value: {vn_result['grs']['p_value']:.3f}")
```

Three caveats specific to the Vietnamese setting deserve emphasis, and each is a direct consequence of the market's institutional features developed elsewhere in this book:

1. **Short samples reduce power.** Reliable Vietnamese factor data begins around 2008 (Section 22.2), giving fewer than 200 monthly observations. The spanning alpha t-statistics and the GRS test both lose power as T falls, so the procedure will tend to select **smaller** models than it would on the long US samples of Feng et al. (2026). A factor omitted on Vietnamese data is weakly priced *given the available sample*, not necessarily unpriced. Reporting the squared-Sharpe increment alongside the p-value (both are returned by the helpers above) keeps the economic magnitude visible even when statistical significance is marginal.
2. **Correlated factors destabilize the path.** Vietnamese value, quality, and profitability signals are empirically more collinear than in the US because they load on the same handful of large, state-linked firms. High collinearity inflates the residual variance in Equation 22.10 and makes the order of addition sensitive to the sample, so the selected model should be validated by re-running on bootstrap resamples (Section 21.7) and by checking out-of-sample stability (Section 21.8), not read off a single in-sample path.
3. **Test-asset choice matters more in thin markets.** With relatively few liquid stocks, finely partitioned test portfolios (for example 25 double sorts) contain noisy, thinly traded portfolios that can dominate the GRS statistic. Coarser sorts (2×3 or decile portfolios) and the price-limit-aware return construction of the microstructure chapter give more trustworthy stopping decisions.

Used with these caveats, the stepwise approach turns the sprawling Vietnamese factor zoo into a disciplined, testable model: rather than defaulting to an imported Fama-French specification, the researcher lets the local data select the factors that actually span the Vietnamese efficient frontier, and obtains an explicit pricing test of the result.

22.9 Conditional vs. Unconditional Tests

22.9.1 Time-Varying Betas

Unconditional tests assume constant betas. In practice, Vietnamese stock betas vary substantially over time due to changing market conditions, foreign ownership shifts, and sectoral rotations. We estimate rolling betas to assess the degree of time-variation:

```
# Rolling 36-month betas for the 25 Size-BM portfolios on FF5
rolling_window = 36
factor_cols = ['mkt_excess', 'smb', 'hml', 'rmw', 'cma']

# Select a few representative portfolios
representative = [excess_25_bm.columns[0],   # Small-Growth
                  excess_25_bm.columns[4],   # Small-Value
                  excess_25_bm.columns[20],  # Big-Growth
                  excess_25_bm.columns[24]]  # Big-Value

rolling_betas = {}
common = excess_25_bm.index.intersection(factors.index)

for port in representative:
    betas_t = []
    for t in range(rolling_window, len(common)):
        window = common[t - rolling_window:t]
        y = excess_25_bm.loc[window, port]
        X = sm.add_constant(factors.loc[window, factor_cols])

        try:
            model = sm.OLS(y, X).fit()
            entry = {'month': common[t]}
            for f in factor_cols:
                entry[f'beta_{f}'] = model.params[f]
            betas_t.append(entry)
        except Exception:
            pass

    rolling_betas[port] = pd.DataFrame(betas_t)
```

```
fig, axes = plt.subplots(2, 2, figsize=(14, 8))
axes = axes.flatten()

labels = ['Small-Growth', 'Small-Value', 'Big-Growth', 'Big-Value']
colors_port = ['#C0392B', '#2C5F8A', '#E67E22', '#27AE60']

for i, (port, label, color) in enumerate(zip(representative, labels,
 ↪  colors_port)):
    rb = rolling_betas[port]

    axes[i].plot(pd.to_datetime(rb['month']), rb['beta_mkt_excess'],
                 color=color, linewidth=1.5, label='Market  ')
    axes[i].axhline(y=1, color='gray', linewidth=0.5, linestyle='--')

    # Add HML beta
    axes[i].plot(pd.to_datetime(rb['month']), rb['beta_hml'],
                 color='gray', linewidth=1, linestyle=':', label='HML  ')

    axes[i].set_ylabel('Beta')
    axes[i].set_title(label)
    axes[i].legend(fontsize=8)

plt.suptitle('Rolling 36-Month Factor Betas', fontsize=14)
plt.tight_layout()
plt.show()

# Quantify time-variation
print("\nBeta Time-Variation (Std Dev of Rolling Betas):")
for port, label in zip(representative, labels):
    rb = rolling_betas[port]
    mkt_std = rb['beta_mkt_excess'].std()
    hml_std = rb['beta_hml'].std()
    print(f"  {label:<15}:  (_MKT) = {mkt_std:.3f},  (_HML) = {hml_std:.3f}")
```

Figure 22.7

22.10 Practical Recommendations

The analysis in this chapter yields the following guidelines for Vietnamese asset pricing research:

Use both approaches. Time-series tests (GRS) and cross-sectional tests (Fama-MacBeth) answer different questions. Always report both. If they disagree, investigate *why* rather than cherry-picking the more favorable result.

Be cautious with cross-sectional R^2. High R^2 from Fama-MacBeth on sorted portfolios is nearly guaranteed by the test asset structure. Always report the Lewellen, Nagel, and Shanken (2010) constrained R^2 alongside the unconstrained value. Include industry portfolios as additional test assets.

Apply the Shanken correction. The EIV bias from estimated betas inflates Fama-MacBeth t-statistics. Always report Shanken-corrected standard errors for full-sample betas. For rolling betas, the correction is not straightforward, but the bias is smaller because beta estimation error averages out over time.

Use individual stocks with caution. Fama-MacBeth with individual stocks avoids the Lewellen-Nagel-Shanken critique but introduces severe noise: individual Vietnamese stocks are thin, volatile, and the monthly cross-section is small (500-700). The resulting risk premia will have wide confidence intervals. Report the average number of stocks per cross-section and the cross-sectional R^2.

Examine conditional models. Rolling betas reveal substantial time-variation in Vietnamese stock exposures. If the research question is about risk compensation (does beta predict returns?), use rolling betas and control for time-variation. If the question is about model adequacy (do the factors span the mean-variance frontier?), full-sample betas and the GRS test are appropriate.

For model selection, use Barillas-Shanken. When comparing competing factor models (e.g., FF3 vs. FF5), the Barillas and Shanken (2018) approach (i.e., testing whether each model's unique factors have alpha with respect to the other) is more informative than comparing GRS statistics on the same test assets.

22.11 Summary

Table 22.2: Summary of testing approaches for factor models.

Test	Question	Statistic	Strength	Weakness
TS regression (GRS)	Do all alphas = 0?	F-statistic	Clean null; joint test	Depends on test assets

Test	Question	Statistic	Strength	Weakness
Fama-MacBeth (portfolios)	Are betas priced?	$\bar{\gamma}$, t-stat	Estimates risk premia	EIV bias; LNS critique
FM (individual stocks)	Are betas priced?	$\bar{\gamma}$, t-stat	Avoids sorted portfolios	Noisy; small cross-section
LNS diagnostic	Is CS R^2 spurious?	Constrained R^2	Reveals false positives	Only applicable to portfolios
GMM / SDF	Pricing errors jointly	J-stat, HJ distance	Flexible; model-free	Requires large N relative to K
Barillas-Shanken	Which model is better?	Alpha of unique factors	Model comparison	Requires traded factors

The gap between time-series and cross-sectional evidence is not a technicality—it reflects a fundamental ambiguity in what "a factor model works" means. In a market like Vietnam, where the cross-section is small, betas are time-varying, and test assets are inherently noisy, the two approaches will often disagree. The honest researcher reports both, investigates the source of disagreement, and treats any single test result with appropriate humility.

Part IV

Firm Fundamentals, Valuation, and Corporate Signals

23 Financial Statement Analysis

23.1 From Market Prices to Fundamental Value

The previous chapters focused on how financial markets price assets in equilibrium. The Capital Asset Pricing Model showed that expected returns depend on systematic risk exposure, while Modern Portfolio Theory demonstrated how to construct efficient portfolios. But these frameworks take expected returns and risk as given, they don't explain where these expectations come from.

Financial statement analysis addresses this gap. By examining a company's accounting records, investors can form independent assessments of firm value, identify mispriced securities, and understand the economic forces driving business performance. Financial statements provide the primary source of standardized information about a company's operations, financial position, and cash generation. Their legal requirements and standardized formats make them particularly valuable. Every publicly traded company must file them, creating a level playing field for analysis.

This chapter introduces the three primary financial statements: the balance sheet, income statement, and cash flow statement. We then demonstrate how to transform raw accounting data into meaningful financial ratios that facilitate comparison across companies and over time. These ratios serve multiple purposes: they enable investors to benchmark companies against peers, help creditors assess default risk, and provide inputs for asset pricing models like the Fama-French factors we will encounter in later chapters.

Our analysis combines theoretical frameworks with practical implementation using Vietnamese market data. By the end of this chapter, you will understand how to access financial statements, calculate key ratios across multiple categories, and interpret these metrics in context.

```
import pandas as pd
import numpy as np

from plotnine import *
from mizani.formatters import percent_format
from adjustText import adjust_text
```

23.2 The Three Financial Statements

Before diving into ratios and analysis, we need to understand the three interconnected statements that form the foundation of financial reporting. Each statement answers a different question about the company, and together they provide a comprehensive picture of financial health.

23.2.1 The Balance Sheet: A Snapshot of Financial Position

The balance sheet captures a company's financial position at a specific moment in time, think of it as a photograph rather than a movie. It lists everything the company owns (assets), everything it owes (liabilities), and the residual claim belonging to shareholders (equity). These three components are linked by the fundamental accounting equation:

$$\text{Assets} = \text{Liabilities} + \text{Equity}$$

This equation is not merely a definition, it reflects a core economic principle. A company's resources (assets) must be financed from somewhere: either borrowed from creditors (liabilities) or contributed by owners (equity). Every transaction affects both sides equally, maintaining the balance.

Assets represent resources the company controls that are expected to generate future economic benefits:

- **Current assets** can be converted to cash within one year: cash and equivalents, short-term investments, accounts receivable (money owed by customers), and inventory (raw materials, work-in-progress, and finished goods)
- **Non-current assets** support operations beyond one year: property, plant, and equipment (PP&E), long-term investments, and intangible assets like patents, trademarks, and goodwill

Liabilities encompass obligations to external parties:

- **Current liabilities** come due within one year: accounts payable (money owed to suppliers), short-term debt, accrued expenses, and the current portion of long-term debt
- **Non-current liabilities** extend beyond one year: long-term debt, bonds payable, pension obligations, and deferred tax liabilities

Shareholders' equity represents the owners' residual claim:

- **Common stock** and additional paid-in capital from share issuance
- **Retained earnings** (i.e., accumulated profits reinvested rather than distributed as dividends)

- **Treasury stock**: shares repurchased by the company

Understanding these categories is essential for ratio analysis. Current assets and liabilities determine short-term liquidity, while the mix of debt and equity reveals capital structure choices.

23.2.2 The Income Statement: Performance Over Time

While the balance sheet provides a snapshot, the income statement (also called the profit and loss statement, or P&L) measures financial performance over a period (e.g., a quarter or year). It follows a hierarchical structure that progressively captures different levels of profitability:

$$\text{Revenue} - \text{COGS} = \text{Gross Profit}$$

$$\text{Gross Profit} - \text{Operating Expenses} = \text{Operating Income (EBIT)}$$

$$\text{EBIT} - \text{Interest} - \text{Taxes} = \text{Net Income}$$

Each line reveals something different about the business:

- **Revenue (Sales)**: Total income from goods or services sold (i.e., the "top line")
- **Cost of Goods Sold (COGS)**: Direct costs of producing what was sold (materials, direct labor, manufacturing overhead)
- **Gross Profit**: Revenue minus COGS, measuring basic profitability from core operations
- **Operating Expenses**: Costs of running the business beyond production (selling, general & administrative expenses, research & development)
- **Operating Income (EBIT)**: Earnings Before Interest and Taxes, measuring profitability from operations before financing decisions and taxes
- **Interest Expense**: The cost of debt financing
- **Net Income**: The "bottom line" (i.e., total profit after all expenses)

The income statement's hierarchical structure allows analysts to identify where profitability problems originate. A company with strong gross margins but weak net income might have bloated overhead costs. One with weak gross margins faces fundamental pricing or production challenges.

23.2.3 The Cash Flow Statement: Following the Money

The cash flow statement bridges a critical gap: profitable companies can run out of cash, and unprofitable companies can generate positive cash flow. This happens because accrual accounting (used in the income statement) recognizes revenue when earned and expenses when incurred, not when cash changes hands.

The cash flow statement tracks actual cash movements, divided into three categories:

- **Operating activities**: Cash generated from core business operations. Starts with net income, then adjusts for non-cash items (depreciation, changes in working capital)
- **Investing activities**: Cash spent on or received from long-term investments (e.g., purchasing equipment, acquiring businesses, selling assets)
- **Financing activities**: Cash flows from capital structure decisions (e.g., issuing stock, borrowing, repaying debt, paying dividends, buying back shares)

A company can show strong net income while burning cash if it's building inventory, extending generous credit terms, or making large capital expenditures. Conversely, a company reporting losses might generate positive operating cash flow by collecting receivables faster than it pays suppliers.

23.2.4 Illustrating with FPT's Financial Statements

To see these concepts in practice, let's examine FPT Corporation's 2023 financial statements. FPT is one of Vietnam's largest technology companies, providing IT services, telecommunications, and education.

```
# Placeholder for FPT balance sheet visualization
# In practice, this would display the actual PDF or cleaned data
# from DataCore's acquisition pipeline

# Example structure of what the balance sheet data looks like:
# Assets: Current assets (cash, receivables, inventory) + Non-current assets
 ↪  (PP&E, intangibles)
# Liabilities: Current liabilities (payables, short-term debt) + Non-current
 ↪  liabilities (long-term debt)
# Equity: Common stock + Retained earnings
```

The balance sheet demonstrates the fundamental accounting equation in action. FPT's assets (e.g., spanning cash, receivables, technology infrastructure, and intangible assets like software) exactly equal the sum of its liabilities and equity.

```
# Placeholder for FPT income statement visualization
# Shows the progression from revenue through various profit measures to net
 ↪  income
```

FPT's income statement reveals how the company transforms revenue into profit. The progression from gross profit through operating income to net income shows the impact of operating expenses, interest costs, and taxes.

```
# Placeholder for FPT cash flow statement visualization
# Reconciles net income with actual cash generation
```

The cash flow statement shows how FPT's reported profits translate into actual cash. Differences between net income and operating cash flow reveal the impact of working capital management and non-cash expenses.

23.3 Loading Financial Statement Data

We now turn to systematic analysis across multiple companies. We load financial statement data for the VN30 index constituents (i.e., the 30 largest and most liquid stocks on Vietnam's Ho Chi Minh Stock Exchange).

```
import sqlite3

tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")

comp_vn = pd.read_sql_query(
    sql="SELECT * FROM comp_vn",
    con=tidy_finance,
    parse_dates={"datadate"}
)

comp_vn.head(3)
```

	symbol	year	total_current_asset	ca_fin	ca_cce	ca_cash	ca_cash_inbank	ca_cash
0	AGF	1998	8.845141e+10	None	5.469709e+09	0.000000e+00	None	None
1	BBC	1999	5.672574e+10	None	5.354939e+09	5.354939e+09	None	None
2	AGF	1999	9.558392e+10	None	2.609276e+09	0.000000e+00	None	None

```
vn30_symbols = [
    "ACB", "BCM", "BID", "BVH", "CTG", "FPT", "GAS", "GVR", "HDB", "HPG",
    "MBB", "MSN", "MWG", "PLX", "POW", "SAB", "SHB", "SSB", "STB", "TCB",
    "TPB", "VCB", "VHM", "VIB", "VIC", "VJC", "VNM", "VPB", "VRE", "EIB"
]

comp_vn30 = comp_vn[comp_vn["symbol"].isin(vn30_symbols)]
comp_vn30.head(3)
```

	symbol	year	total_current_asset	ca_fin	ca_cce	ca_cash	ca_cash_inbank	ca_ca
11	FPT	2002	5.098910e+11	None	1.027470e+11	0.000000e+00	None	None
17	VNM	2003	2.101406e+12	None	6.925924e+11	6.925924e+11	None	None
21	FPT	2003	9.171390e+11	None	7.995600e+10	0.000000e+00	None	None

This dataset provides the foundation for calculating financial ratios and conducting cross-sectional comparisons. Each row contains balance sheet, income statement, and cash flow items for a company-year observation.

23.4 Liquidity Ratios: Can the Company Pay Its Bills?

Liquidity ratios assess a company's ability to meet short-term obligations. These metrics matter most to creditors, suppliers, and employees who need assurance that the company can pay its bills. They're calculated using balance sheet items, comparing liquid assets against near-term liabilities.

23.4.1 The Current Ratio

The most basic liquidity measure compares all current assets to current liabilities:

$$\text{Current Ratio} = \frac{\text{Current Assets}}{\text{Current Liabilities}}$$

A ratio above one indicates the company has enough current assets to cover obligations due within one year. However, the interpretation depends heavily on the composition of current assets. A company with current assets tied up in slow-moving inventory is less liquid than one holding cash.

23.4.2 The Quick Ratio

The quick ratio (or "acid test") provides a more stringent measure by excluding inventory:

$$\text{Quick Ratio} = \frac{\text{Current Assets} - \text{Inventory}}{\text{Current Liabilities}}$$

Why exclude inventory? Inventory is typically the least liquid current asset. It must be sold (potentially at a discount) before generating cash. A company facing a liquidity crisis cannot easily convert raw materials or finished goods into immediate cash. The quick ratio answers: "Can we pay our bills without relying on inventory sales?"

23.4.3 The Cash Ratio

The most conservative liquidity measure focuses solely on the most liquid assets:

$$\text{Cash Ratio} = \frac{\text{Cash and Cash Equivalents}}{\text{Current Liabilities}}$$

While a ratio of one indicates robust liquidity, most companies maintain lower cash ratios to avoid holding excessive non-productive assets. Cash sitting in bank accounts could otherwise be invested in growth opportunities, returned to shareholders, or used to pay down costly debt.

23.4.4 Calculating Liquidity Ratios

Let's compute these ratios for our VN30 sample:

```
balance_sheet_statements = (comp_vn30
    .assign(
        fiscal_year=lambda x: x["year"].astype(int),

        # Current Ratio: Current Assets / Current Liabilities
        current_ratio=lambda x: x["act"] / x["lct"],

        # Quick Ratio: (Current Assets - Inventory) / Current Liabilities
        quick_ratio=lambda x: (x["act"] - x["inv"]) / x["lct"],

        # Cash Ratio: Cash and Equivalents / Current Liabilities
        cash_ratio=lambda x: x["ca_cce"] / x["lct"],
```

```
        label=lambda x: np.where(
            x["symbol"].isin(vn30_symbols), x["symbol"], np.nan
        )
    )
)

balance_sheet_statements.head(3)
```

	symbol	year	total_current_asset	ca_fin	ca_cce	ca_cash	ca_cash_inbank	ca_ca
11	FPT	2002	5.098910e+11	None	1.027470e+11	0.000000e+00	None	None
17	VNM	2003	2.101406e+12	None	6.925924e+11	6.925924e+11	None	None
21	FPT	2003	9.171390e+11	None	7.995600e+10	0.000000e+00	None	None

23.4.5 Cross-Sectional Comparison of Liquidity

Figure 23.1 compares liquidity ratios across companies for the most recent fiscal year. This cross-sectional view reveals how different business models and industries maintain different liquidity profiles.

```
liquidity_ratios = (balance_sheet_statements
    .query("year == 2023 & label.notna()")
    .get(["symbol", "current_ratio", "quick_ratio", "cash_ratio"])
    .melt(id_vars=["symbol"], var_name="name", value_name="value")
    .assign(
        name=lambda x: x["name"].str.replace("_", " ").str.title()
    )
)

liquidity_ratios_figure = (
    ggplot(liquidity_ratios, aes(y="value", x="name", fill="symbol"))
    + geom_col(position="dodge")
    + coord_flip()
    + labs(
        x="", y="Ratio Value", fill="",
        title="Liquidity Ratios for VN30 Stocks (2023)"
    )
)

liquidity_ratios_figure.show()
```

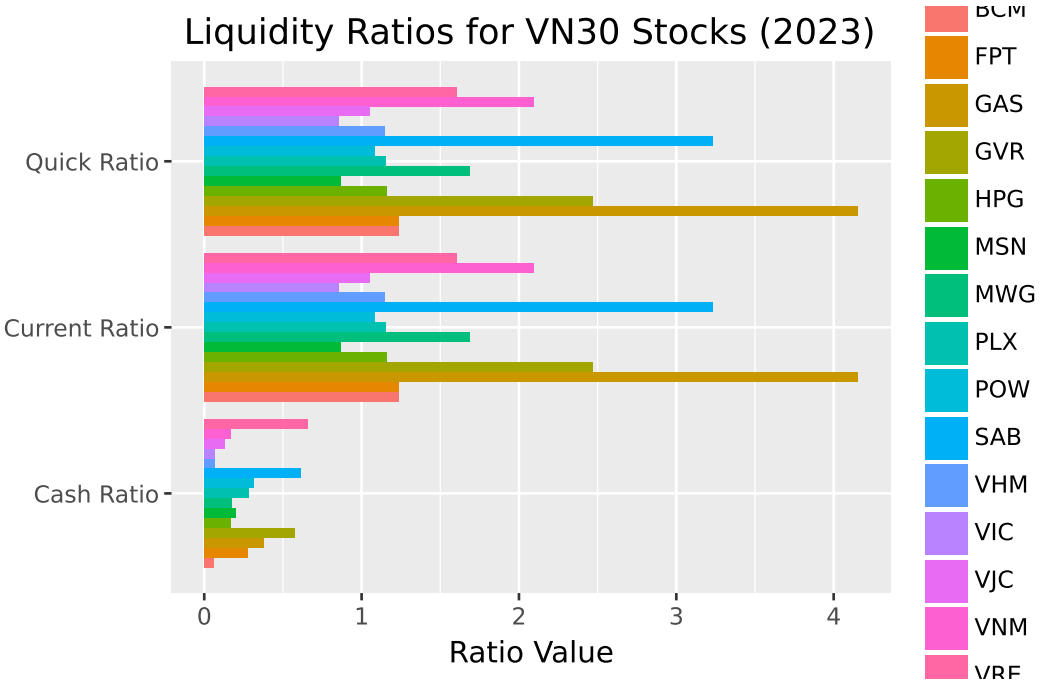


Figure 23.1: Liquidity ratios measure a company's ability to meet short-term obligations. Higher values indicate greater liquidity, though excessively high ratios may suggest inefficient use of assets.

Several patterns emerge from this comparison. Banks and financial institutions typically show different liquidity profiles than industrial companies due to their unique business models. Companies with high inventory (retailers, manufacturers) often show larger gaps between current and quick ratios.

23.5 Leverage Ratios: How Is the Company Financed?

Leverage ratios examine a company's capital structure (i.e., the mix of debt and equity financing). These metrics reveal financial risk and long-term solvency, helping investors understand how much of the company's operations are funded by borrowed money.

23.5.1 Why Capital Structure Matters

A company's financing choice involves fundamental trade-offs:

- **Debt** offers tax advantages (interest is deductible) and doesn't dilute ownership, but creates fixed obligations that must be met regardless of business performance

- **Equity** provides flexibility (no required payments) but dilutes existing shareholders and may be more expensive than debt

Companies with high leverage amplify both gains and losses. In good times, shareholders capture more upside because profits aren't shared with additional equity holders. In bad times, fixed interest payments can push the company toward distress. This is why beta (systematic risk) tends to increase with leverage.

23.5.2 Debt-to-Equity Ratio

This ratio indicates how much debt financing the company uses relative to shareholder investment:

$$\text{Debt-to-Equity} = \frac{\text{Total Debt}}{\text{Total Equity}}$$

A ratio of 1.0 means equal parts debt and equity financing. Higher ratios indicate more aggressive use of leverage, which can enhance returns in good times but increases bankruptcy risk.

23.5.3 Debt-to-Asset Ratio

This ratio shows what percentage of assets are financed through debt:

$$\text{Debt-to-Asset} = \frac{\text{Total Debt}}{\text{Total Assets}}$$

A ratio of 0.5 means half the company's assets are debt-financed. This metric is bounded between 0 and 1 (assuming positive equity), making it easier to compare across companies than the debt-to-equity ratio.

23.5.4 Interest Coverage Ratio

While the above ratios measure leverage levels, interest coverage assesses the ability to service that debt:

$$\text{Interest Coverage} = \frac{\text{EBIT}}{\text{Interest Expense}}$$

This ratio answers: "How many times over can current operating profits cover interest obligations?" A ratio below 1.0 means operating income doesn't cover interest payments, which is a dangerous position. Ratios above 3-5 generally indicate comfortable coverage.

23.5.5 Calculating Leverage Ratios

```
balance_sheet_statements = balance_sheet_statements.assign(
    debt_to_equity=lambda x: x["total_debt"] / x["total_equity"],
    debt_to_asset=lambda x: x["total_debt"] / x["at"]
)

income_statements = (comp_vn30
    .assign(
        year=lambda x: x["year"].astype(int),
        # Handle zero interest expense to avoid infinity
        interest_coverage=lambda x: np.where(
            x["cfo_interest_expense"] > 0,
            x["is_net_business_profit"] / x["cfo_interest_expense"],
            np.nan
        ),
        label=lambda x: np.where(
            x["symbol"].isin(vn30_symbols), x["symbol"], np.nan
        )
    )
)
```

23.5.6 Leverage Trends Over Time

Figure 23.2 tracks how debt-to-asset ratios have evolved over time. Time-series analysis reveals whether companies are becoming more or less leveraged.

```
debt_to_asset = balance_sheet_statements.query("symbol in @vn30_symbols")

debt_to_asset_figure = (
    ggplot(debt_to_asset, aes(x="year", y="debt_to_asset", color="symbol"))
    + geom_line(size=1)
    + scale_y_continuous(labels=percent_format())
    + labs(
        x="", y="Debt-to-Asset Ratio", color="",
        title="Debt-to-Asset Ratios of VN30 Stocks Over Time"
    )
)

debt_to_asset_figure.show()
```

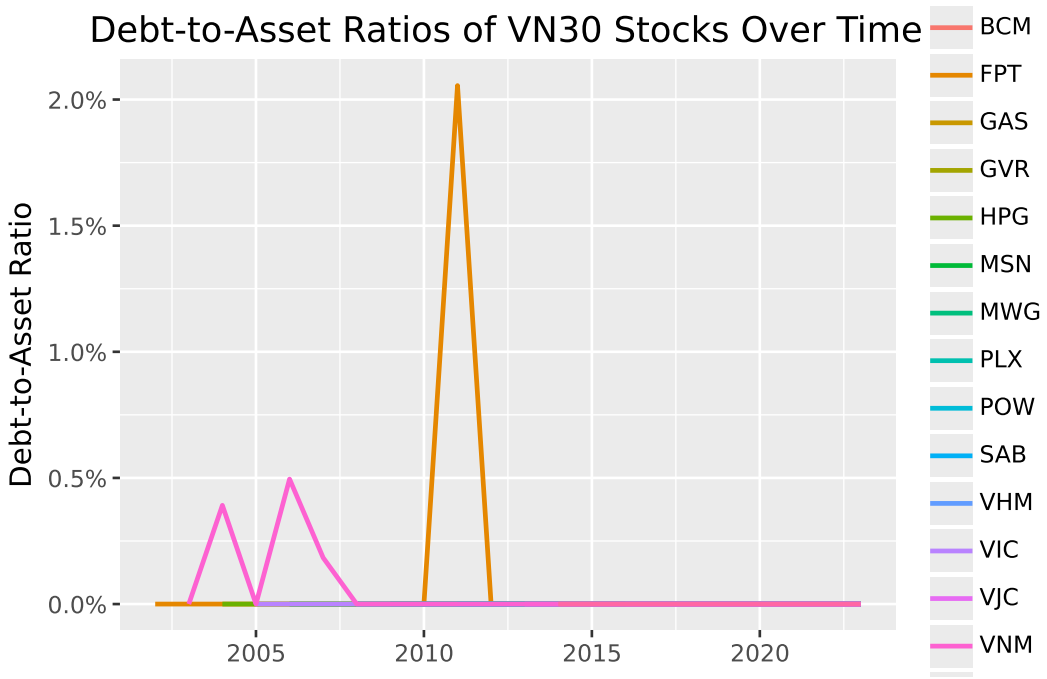


Figure 23.2: Debt-to-asset ratios show the proportion of assets financed by debt. Changes over time reflect evolving capital structure strategies and market conditions.

23.5.7 Cross-Sectional Leverage Comparison

Figure 23.3 provides a snapshot of leverage across all VN30 constituents for the most recent year.

```
debt_to_asset_comparison = balance_sheet_statements.query("year == 2023")

debt_to_asset_comparison["symbol"] = pd.Categorical(
    debt_to_asset_comparison["symbol"],
    categories=debt_to_asset_comparison.sort_values("debt_to_asset")["symbol
 ↪  "],
    ordered=True
)

debt_to_asset_comparison_figure = (
    ggplot(
        debt_to_asset_comparison,
        aes(y="debt_to_asset", x="symbol", fill="label")
    )
```

```
    + geom_col()
    + coord_flip()
    + scale_y_continuous(labels=percent_format())
    + labs(
        x="", y="Debt-to-Asset Ratio", fill="",
        title="Debt-to-Asset Ratios of VN30 Stocks (2023)"
    )
    + theme(legend_position="none")
)

debt_to_asset_comparison_figure.show()
```

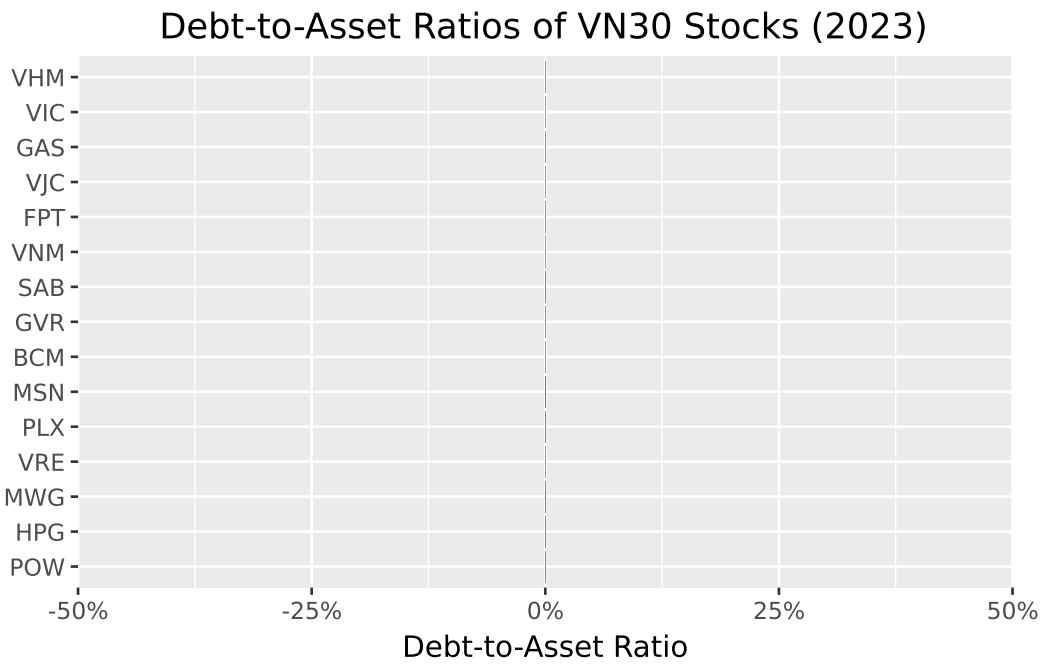


Figure 23.3: Cross-sectional comparison of debt-to-asset ratios reveals industry patterns and company-specific financing strategies.

23.5.8 The Leverage-Coverage Trade-off

Figure 23.4 examines the relationship between leverage levels and debt-servicing ability. Companies with higher debt loads should ideally have stronger interest coverage to maintain financial stability.

```
interest_coverage = (income_statements
    .query("year == 2023")
    .get(["symbol", "year", "interest_coverage"])
    .merge(balance_sheet_statements, on=["symbol", "year"], how="left")
)

interest_coverage_figure = (
    ggplot(
        interest_coverage,
        aes(x="debt_to_asset", y="interest_coverage", color="label")
    )
    + geom_point(size=2)
    + geom_label(
        aes(label="label"),
        adjust_text={"arrowprops": {"arrowstyle": "-"}}
    )
    + scale_x_continuous(labels=percent_format())
    + labs(
        x="Debt-to-Asset Ratio", y="Interest Coverage Ratio",
        title="Leverage versus Interest Coverage for VN30 Stocks (2023)"
    )
    + theme(legend_position="none")
)

interest_coverage_figure.show()
```

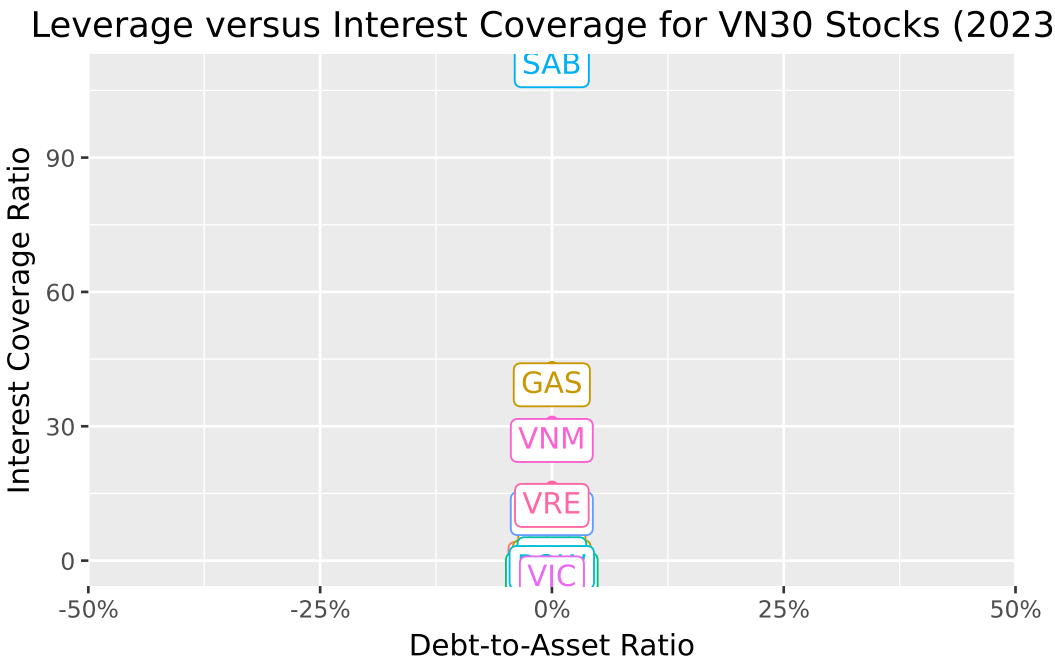


Figure 23.4: The relationship between leverage and interest coverage reveals whether companies can comfortably service their debt. High leverage with low coverage indicates elevated financial risk.

The scatter plot reveals important patterns. Companies in the upper-left quadrant (low leverage, high coverage) have conservative financing with ample debt capacity. Those in the lower-right (high leverage, low coverage) face elevated financial risk.

23.6 Efficiency Ratios: How Well Are Assets Managed?

Efficiency ratios measure how effectively a company utilizes its assets and manages operations. These metrics help identify whether management is extracting maximum value from the company's resource base.

23.6.1 Asset Turnover

This ratio measures how efficiently a company uses total assets to generate revenue:

$$\text{Asset Turnover} = \frac{\text{Revenue}}{\text{Total Assets}}$$

A higher ratio indicates more efficient asset utilization: the company generates more sales per dollar of assets. However, optimal levels vary dramatically across industries. Retailers with minimal fixed assets might achieve turnovers above 2.0, while capital-intensive manufacturers might operate below 0.5.

23.6.2 Inventory Turnover

For companies carrying inventory, this ratio reveals how quickly stock moves through the business:

$$\text{Inventory Turnover} = \frac{\text{Cost of Goods Sold}}{\text{Inventory}}$$

Higher turnover suggests efficient inventory management (i.e., goods don't sit on shelves collecting dust). However, extremely high turnover might indicate stockout risks, while very low turnover could signal obsolete inventory or overinvestment in working capital.

We use COGS rather than revenue in the numerator because inventory is recorded at cost, not selling price. Using revenue would overstate turnover for high-margin businesses.

23.6.3 Receivables Turnover

This ratio measures how effectively a company collects payments from customers:

$$\text{Receivables Turnover} = \frac{\text{Revenue}}{\text{Accounts Receivable}}$$

Higher turnover indicates faster collection (i.e., customers pay promptly). Converting this to "days sales outstanding" (365 / turnover) gives the average collection period in days. Companies must balance collection efficiency against the sales impact of restrictive credit policies.

23.6.4 Calculating Efficiency Ratios

```
combined_statements = (balance_sheet_statements
    .get([
        "symbol", "year", "label", "current_ratio", "quick_ratio",
        "cash_ratio", "debt_to_equity", "debt_to_asset", "total_asset",
        "total_equity"
    ])
    .merge(
```

```
        (income_statements
            .get([
                "symbol", "year", "interest_coverage", "is_revenue",
                "is_cogs",
                # "selling_general_and_administrative_expenses",
                "is_interest_expense", "is_gross_profit", "is_eat"
            ])
        ),
        on=["symbol", "year"],
        how="left"
    )
    .merge(
        (comp_vn30
            .assign(year=lambda x: x["year"].astype(int))
            .get(["symbol", "year", "ca_total_inventory", "ca_acc_receiv"])
        ),
        on=["symbol", "year"],
        how="left"
    )
)

combined_statements = combined_statements.assign(
    asset_turnover=lambda x: x["is_revenue"] / x["total_asset"],
    inventory_turnover=lambda x: x["is_cogs"] / x["ca_total_inventory"],
    receivables_turnover=lambda x: x["is_revenue"] / x["ca_acc_receiv"]
)
```

Efficiency ratios vary dramatically across industries, making peer comparison essential. A grocery store and a shipbuilder will have fundamentally different asset and inventory dynamics.

23.7 Profitability Ratios: Is the Company Making Money?

Profitability ratios evaluate how effectively a company converts activity into earnings. These metrics directly measure financial success and are among the most closely watched indicators by investors.

23.7.1 Gross Margin

The gross margin reveals what percentage of revenue remains after direct production costs:

$$\text{Gross Margin} = \frac{\text{Gross Profit}}{\text{Revenue}} = \frac{\text{Revenue} - \text{COGS}}{\text{Revenue}}$$

Higher gross margins indicate stronger pricing power, more efficient production, or a favorable product mix. This metric is particularly useful for comparing companies within an industry, as it reveals relative efficiency in core operations before overhead costs.

23.7.2 Profit Margin

The profit margin shows what percentage of revenue ultimately becomes net income:

$$\text{Profit Margin} = \frac{\text{Net Income}}{\text{Revenue}}$$

This comprehensive measure accounts for all costs (e.g., production, operations, interest, and taxes). Higher profit margins suggest effective overall cost management. However, optimal margins vary by industry: software companies routinely achieve 20%+ margins, while grocery stores operate on razor-thin 2-3% margins.

23.7.3 Return on Equity (ROE)

ROE measures how efficiently a company uses shareholders' investment to generate profits:

$$\text{Return on Equity} = \frac{\text{Net Income}}{\text{Total Equity}}$$

This metric directly addresses what shareholders care about: returns on their invested capital. Higher ROE indicates more effective use of equity, though interpretation requires caution. High leverage can artificially inflate ROE by reducing the equity base (e.g., a company financed 90% by debt will show spectacular ROE on modest profits).

23.7.4 The DuPont Decomposition

The DuPont framework decomposes ROE into three components that reveal different aspects of performance:

$$\text{ROE} = \underbrace{\frac{\text{Net Income}}{\text{Revenue}}}_{\text{Profit Margin}} \times \underbrace{\frac{\text{Revenue}}{\text{Assets}}}_{\text{Asset Turnover}} \times \underbrace{\frac{\text{Assets}}{\text{Equity}}}_{\text{Leverage}}$$

This decomposition shows that high ROE can come from different sources: strong profit margins (pricing power, cost control), efficient asset use (high turnover), or aggressive leverage. Understanding which driver dominates helps assess sustainability. ROE driven by margins is generally more sustainable than ROE driven by leverage.

23.7.5 Calculating Profitability Ratios

```
combined_statements = combined_statements.assign(
    gross_margin=lambda x: x["is_gross_profit"] / x["is_revenue"],
    profit_margin=lambda x: x["is_eat"] / x["is_revenue"],
    after_tax_roe=lambda x: x["is_eat"] / x["total_equity"]
)
```

23.7.6 Gross Margin Trends

Figure 23.5 tracks gross margin evolution over time, revealing whether companies are maintaining pricing power and production efficiency.

```
gross_margins = combined_statements.query("symbol in @vn30_symbols")

gross_margins_figure = (
    ggplot(gross_margins, aes(x="year", y="gross_margin", color="symbol"))
    + geom_line()
    + scale_y_continuous(labels=percent_format())
    + labs(
        x="", y="Gross Margin", color="",
        title="Gross Margins for VN30 Stocks (2019-2023)"
    )
)

gross_margins_figure.show()
```

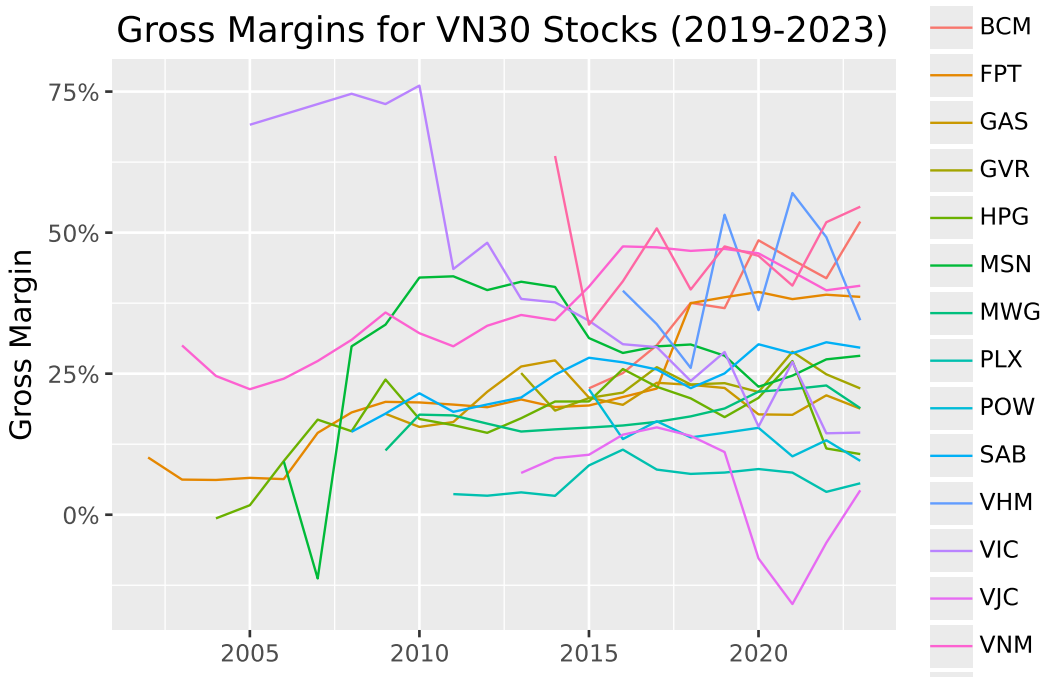


Figure 23.5: Gross margin trends reveal changes in pricing power and production efficiency. Declining margins may signal increased competition or rising input costs.

23.7.7 From Gross to Net: Where Do Profits Go?

Figure 23.6 examines the relationship between gross and profit margins. The gap between them reveals the impact of operating expenses, interest, and taxes.

```
profit_margins = combined_statements.query("year == 2023")

profit_margins_figure = (
    ggplot(
        profit_margins,
        aes(x="gross_margin", y="profit_margin", color="label")
    )
    + geom_point(size=2)
    + geom_label(
        aes(label="label"),
        adjust_text={"arrowprops": {"arrowstyle": "-"}}
    )
    + scale_x_continuous(labels=percent_format())
    + scale_y_continuous(labels=percent_format())
```

```
    + labs(
        x="Gross Margin", y="Profit Margin",
        title="Gross versus Profit Margins for VN30 Stocks (2023)"
    )
    + theme(legend_position="none")
)

profit_margins_figure.show()
```

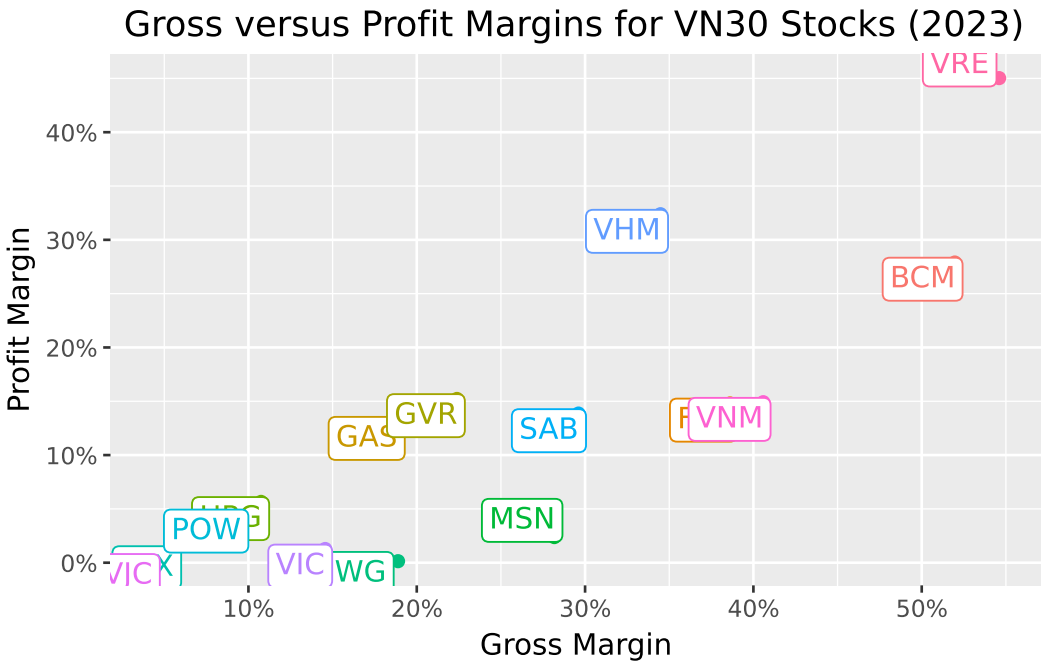


Figure 23.6: Comparing gross and profit margins reveals how much of gross profit survives operating expenses, interest, and taxes. Companies far below the diagonal have high overhead relative to gross profit.

Companies along the diagonal convert gross profit to net income efficiently. Those well below the diagonal face high operating costs, interest burdens, or tax rates that erode profitability.

23.8 Combining Financial Ratios: A Holistic View

Individual ratios provide specific insights, but combining them offers a more complete picture. A company might excel in profitability while struggling with liquidity, or maintain conservative leverage while underperforming on efficiency.

23.8.1 Ranking Companies Across Categories

Figure 23.7 compares company rankings across four ratio categories. Rankings closer to 1 indicate better performance within each category, enabling quick identification of relative strengths and weaknesses.

```
financial_ratios = (combined_statements
    .query("year == 2023")
    .filter(
        items=["symbol"] + [
            col for col in combined_statements.columns
            if any(x in col for x in [
                "ratio", "margin", "roe", "_to_", "turnover",
                ↪  "interest_coverage"
            ])
        ]
    )
    .melt(id_vars=["symbol"], var_name="name", value_name="value")
    .assign(
        type=lambda x: np.select(
            [
                x["name"].isin(["current_ratio", "quick_ratio",
↪  "cash_ratio"]),
                x["name"].isin(["debt_to_equity", "debt_to_asset",
↪  "interest_coverage"]),
                x["name"].isin(["asset_turnover", "inventory_turnover",
↪  "receivables_turnover"]),
                x["name"].isin(["gross_margin", "profit_margin",
↪  "after_tax_roe"]),
            ],
            [
                "Liquidity Ratios",
                "Leverage Ratios",
                "Efficiency Ratios",
                "Profitability Ratios"
            ],
            default="Other"
        )
    )
)

financial_ratios["rank"] = (financial_ratios
    .sort_values(["type", "name", "value"], ascending=[True, True, False])
```

```
    .groupby(["type", "name"])
    .cumcount() + 1
)

final_ranks = (financial_ratios
    .groupby(["symbol", "type"], as_index=False)
    .agg(rank=("rank", "mean"))
    .query("symbol in @vn30_symbols")
)

final_ranks_figure = (
    ggplot(final_ranks, aes(x="rank", y="type", color="symbol"))
    + geom_point(shape="^", size=4)
    + labs(
        x="Average Rank (Lower is Better)", y="", color="",
        title="Average Rank Across Financial Ratio Categories"
    )
    + coord_cartesian(xlim=[1, 30])
)

final_ranks_figure.show()
```

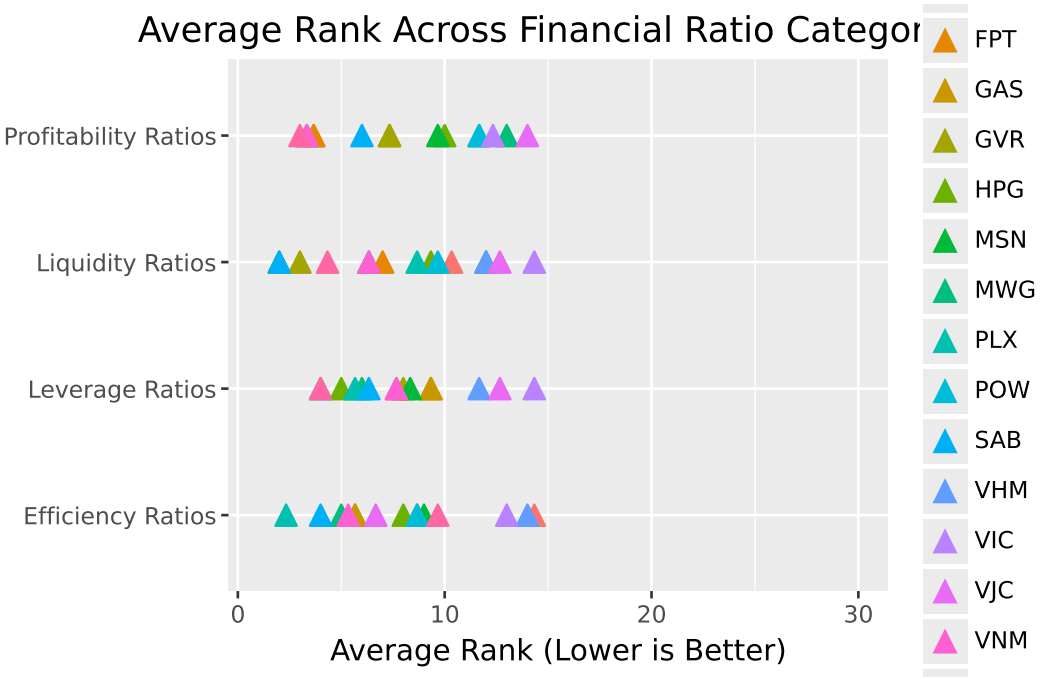


Figure 23.7: Ranking companies across multiple ratio categories reveals overall financial profiles. Companies with consistently low ranks across categories demonstrate broad-based financial strength.

The combined view reveals how different business strategies manifest in financial profiles. A company might deliberately accept lower profitability rankings in exchange for stronger liquidity, or use aggressive leverage to boost returns at the cost of financial flexibility.

23.9 Financial Ratios in Asset Pricing

Beyond evaluating individual companies, financial ratios serve as crucial inputs for asset pricing models. The Fama-French five-factor model, which we explore in detail in Fama-French Factors, uses several accounting-based measures to explain cross-sectional variation in stock returns.

23.9.1 The Fama-French Factors

The model incorporates four company characteristics derived from financial statements:

Size is measured as the logarithm of market capitalization:

$$\text{Size} = \ln(\text{Market Cap})$$

This captures the empirical finding that smaller firms tend to outperform larger firms on a risk-adjusted basis (i.e., the "size premium").

Book-to-Market relates accounting value to market value:

$$\text{Book-to-Market} = \frac{\text{Book Equity}}{\text{Market Cap}}$$

High book-to-market stocks ("value" stocks) have historically outperformed low book-to-market stocks ("growth" stocks) (i.e., the "value premium").

Operating Profitability measures profit generation relative to equity:

$$\text{Profitability} = \frac{\text{Revenue} - \text{COGS} - \text{SG\&A} - \text{Interest}}{\text{Book Equity}}$$

More profitable firms tend to earn higher returns (i.e., the "profitability premium").

Investment captures asset growth:

$$\text{Investment} = \frac{\text{Total Assets}_t}{\text{Total Assets}_{t-1}} - 1$$

Firms investing aggressively tend to underperform conservative investors (i.e., the "investment premium").

23.9.2 Calculating Fama-French Variables

```
prices_monthly = pd.read_sql_query(
    sql="SELECT * FROM prices_monthly",
    con=tidy_finance,
    parse_dates={"datadate"}
)

# Use December prices for annual calculations
prices_december = (prices_monthly
    .assign(date=lambda x: pd.to_datetime(x["date"]))
    .query("date.dt.month == 12")
)
```

```
combined_statements_ff = (combined_statements
    .query("year == 2023")
    .merge(prices_december, on=["symbol", "year"], how="left")
    .merge(
        (balance_sheet_statements
            .query("year == 2022")
            .get(["symbol", "total_asset"])
            .rename(columns={"total_asset": "total_assets_lag"})
        ),
        on="symbol",
        how="left"
    )
    .assign(
        size=lambda x: np.log(x["mktcap"]),
        book_to_market=lambda x: x["total_equity"] / x["mktcap"],
        operating_profitability=lambda x: (
            (x["is_revenue"] - x["is_cogs"] -
            #  x["selling_general_and_administrative_expenses"] -
             x["is_interest_expense"]) / x["total_equity"]
        ),
        investment=lambda x: x["total_asset"] / x["total_assets_lag"] - 1
    )
)

combined_statements_ff.head(3)
```

	symbol	year	label	current_ratio	quick_ratio	cash_ratio	debt_to_equity	debt_to_asset	tot
0	POW	2023	POW	1.084255	1.084255	0.315089	0.0	0.0	7.0
1	HPG	2023	HPG	1.156655	1.156655	0.171324	0.0	0.0	1.8
2	MWG	2023	MWG	1.688604	1.688604	0.174408	0.0	0.0	6.0

23.9.3 Fama-French Factor Rankings

Figure 23.8 shows how VN30 companies rank on each Fama-French variable, connecting fundamental analysis to asset pricing.

```
factors_ranks = (combined_statements_ff
    .get(["symbol", "size", "book_to_market", "operating_profitability",
 ↪  "investment"])
    .rename(columns={
```

```
        "size": "Size",
        "book_to_market": "Book-to-Market",
        "operating_profitability": "Profitability",
        "investment": "Investment"
    })
    .melt(id_vars=["symbol"], var_name="name", value_name="value")
    .assign(
        rank=lambda x: (
            x.sort_values(["name", "value"], ascending=[True, False])
            .groupby("name")
            .cumcount() + 1
        )
    )
    .query("symbol in @vn30_symbols")
)

factors_ranks_figure = (
    ggplot(factors_ranks, aes(x="rank", y="name", color="symbol"))
    + geom_point(shape="^", size=4)
    + labs(
        x="Rank", y="", color="",
        title="Rank in Fama-French Variables for VN30 Stocks"
    )
    + coord_cartesian(xlim=[1, 30])
)

factors_ranks_figure.show()
```

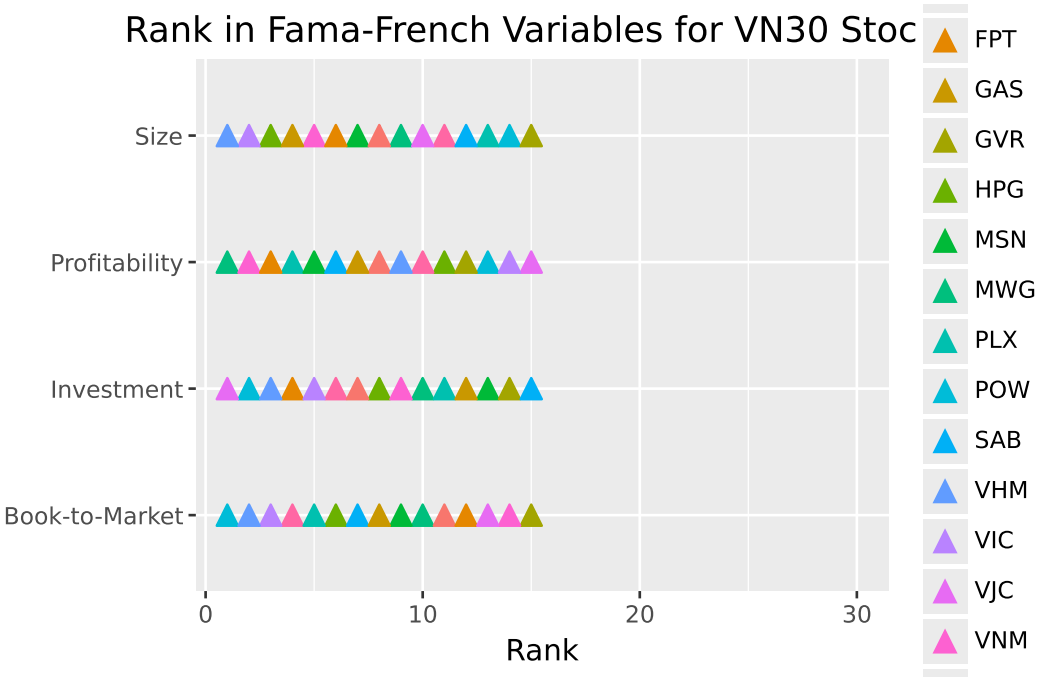


Figure 23.8: Rankings on Fama-French variables connect financial statement analysis to asset pricing. According to factor models, smaller, higher book-to-market, more profitable, and lower-investment firms should earn higher expected returns.

These rankings have implications for expected returns according to factor models. A small, high book-to-market, highly profitable company with conservative investment should, in theory, earn higher risk-adjusted returns than its opposite.

23.10 Limitations and Practical Considerations

While financial ratios provide powerful analytical tools, several limitations deserve attention:

23.10.1 Accounting Discretion

Companies have significant discretion in how they apply accounting standards. Revenue recognition timing, depreciation methods, inventory valuation (FIFO vs. LIFO), and capitalization versus expensing decisions all affect reported numbers. Sophisticated analysis requires understanding these choices and their impact.

23.10.2 Industry Comparability

Ratios vary dramatically across industries. Comparing a bank's leverage to a retailer's is meaningless (e.g., banks naturally operate with much higher leverage due to their business model). Always benchmark against industry peers rather than absolute standards.

23.10.3 Point-in-Time Limitations

Balance sheet ratios capture a single moment, which may not represent typical conditions. Companies often "window dress" by temporarily improving metrics at reporting dates. Trend analysis and quarter-over-quarter comparisons can reveal such practices.

23.10.4 Backward-Looking Nature

Financial statements report historical results. Past profitability doesn't guarantee future performance, especially for companies in rapidly changing industries or facing disruption.

23.10.5 Quality of Earnings

Not all profits are created equal. Earnings driven by one-time gains, accounting adjustments, or aggressive revenue recognition may not recur. Cash flow analysis helps assess earnings quality. Profits that don't convert to cash warrant skepticism.

23.11 Key Takeaways

This chapter introduced financial statement analysis as a tool for understanding company fundamentals. The main insights are:

1. **Three statements, three perspectives**: The balance sheet shows financial position at a point in time, the income statement measures performance over a period, and the cash flow statement tracks actual cash movements. Together, they provide a complete picture of financial health.
2. **Liquidity ratios assess short-term survival**: Current, quick, and cash ratios measure the ability to meet near-term obligations. Higher ratios indicate greater liquidity but may suggest inefficient asset use.
3. **Leverage ratios reveal capital structure risk**: Debt-to-equity, debt-to-asset, and interest coverage ratios show how the company finances operations and whether it can service its debt. Higher leverage amplifies both returns and risk.

4. **Efficiency ratios measure management effectiveness**: Asset turnover, inventory turnover, and receivables turnover reveal how well the company converts resources into revenue. Industry context is essential for interpretation.

5. **Profitability ratios quantify financial success**: Gross margin, profit margin, and ROE measure the ability to generate earnings. The DuPont decomposition reveals whether ROE comes from margins, turnover, or leverage.

6. **Ratios connect to asset pricing**: Financial statement variables like book-to-market, profitability, and investment form the basis of factor models that explain cross-sectional return differences.

7. **Context matters for interpretation**: Ratios must be compared against industry peers, tracked over time, and considered alongside qualitative factors. No single ratio tells the complete story.

Looking ahead, subsequent chapters will explore how these fundamental variables interact with market prices in asset pricing models, and how to construct factor portfolios based on financial statement characteristics.

24 Discounted Cash Flow Analysis

24.1 What Is a Company Worth?

The previous chapters examined how markets price securities in equilibrium and how financial statements reveal company fundamentals. But these approaches leave a central question unanswered: What is the *intrinsic value* of a business, independent of its current market price?

Discounted Cash Flow (DCF) analysis answers this question by valuing a company based on its ability to generate cash for investors. The core insight is simple: a business is worth the present value of all future cash it will produce. This principle that value equals discounted future cash flows underlies virtually all of finance, from bond pricing to real estate valuation.

DCF analysis stands apart from other valuation approaches in three important ways. First, it explicitly accounts for the **time value of money** (i.e., the principle that a dollar today is worth more than a dollar tomorrow). By discounting future cash flows at an appropriate rate, we incorporate both time preferences and risk. Second, DCF is **forward-looking**, making it particularly suitable for companies where historical performance may not reflect future potential. Third, DCF is **flexible** enough to accommodate various business models and capital structures, making it applicable across industries and company sizes.

24.1.1 Valuation Methods Overview

Company valuation methods broadly fall into three categories:

- **Market-based approaches** compare companies using relative metrics like Price-to-Earnings or EV/EBITDA ratios. These are quick but assume comparable companies are fairly valued.
- **Asset-based methods** focus on the net value of tangible and intangible assets. These work well for liquidation scenarios but miss going-concern value.
- **Income-based techniques** value companies based on their ability to generate future cash flows. DCF is the most rigorous income-based method.

We focus on DCF because it forces analysts to make explicit assumptions about growth, profitability, and risk. These assumptions are often hidden in other methods. Even when DCF

isn't the final word on valuation, the discipline of building a DCF model deepens understanding of what drives value.

24.1.2 The Three Pillars of DCF

Every DCF analysis rests on three components:

1. **Free Cash Flow (FCF) forecasts**: The expected future cash available for distribution to investors after operating expenses, taxes, and investments
2. **Terminal value**: The company's value beyond the explicit forecast period, often representing a majority of total valuation
3. **Discount rate**: Typically the Weighted Average Cost of Capital (WACC), which adjusts future cash flows to present value by incorporating risk and capital structure

We make simplifying assumptions throughout this chapter. In particular, we assume firms conduct only operating activities (i.e., financial statements do not include non-operating items like excess cash or investment securities). Real-world valuations require valuing these separately. Entire textbooks are devoted to valuation nuances; our goal is to establish the conceptual framework and practical implementation.

```
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf

from plotnine import *
from mizani.formatters import percent_format, comma_format
from itertools import product
```

24.2 Understanding Free Cash Flow

Before diving into calculations, we need to understand what Free Cash Flow represents and why it matters for valuation.

24.2.1 Why Free Cash Flow, Not Net Income?

Accountants report net income, but DCF uses free cash flow. Why the difference?

Net income includes non-cash items (like depreciation) and ignores cash needs (like capital expenditures and working capital investments). A company can report strong profits while burning cash, or generate substantial cash while reporting losses. Free cash flow captures what

actually matters for valuation: the cash available to distribute to all capital providers (both debt holders and equity holders) after funding operations and investments.

24.2.2 The Free Cash Flow Formula

We calculate FCF using the following formula:

$$\text{FCF} = \text{EBIT} \times (1 - \tau) + \text{D\&A} - \Delta\text{WC} - \text{CAPEX}$$

where:

- **EBIT** (Earnings Before Interest and Taxes): Core operating profit before financing costs and taxes
- τ: Corporate tax rate applied to operating profits
- **D&A** (Depreciation & Amortization): Non-cash charges that reduce reported earnings but don't consume cash
- **ΔWC** (Change in Working Capital): Cash tied up in (or released from) operations (increases in receivables and inventory consume cash, while increases in payables provide cash)
- **CAPEX** (Capital Expenditures): Investments in long-term assets required to maintain and grow operations

An alternative formulation starts from EBIT directly:

$$\text{FCF} = \text{EBIT} + \text{D\&A} - \text{Taxes} - \Delta\text{WC} - \text{CAPEX}$$

Both formulations yield the same result when taxes are calculated consistently. The key insight is that FCF represents cash generated from operations after all reinvestment needs (i.e., cash that could theoretically be distributed to investors without impairing the business).

24.3 Loading Historical Financial Data

We use FPT Corporation, one of Vietnam's largest technology companies, as our case study. FPT provides IT services, telecommunications, and education. It's a diversified business with meaningful capital requirements and growth potential.

```
import sqlite3

tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")

comp_vn = pd.read_sql_query(
    sql="SELECT * FROM comp_vn",
    con=tidy_finance,
    parse_dates={"date"}
)

# Filter to FPT and examine the data structure
fpt_data = comp_vn[comp_vn["symbol"] == "FPT"].copy()
fpt_data["year"] = fpt_data["year"].astype(int)
fpt_data = fpt_data.sort_values("year").reset_index(drop=True)

print(f"Available years: {fpt_data['year'].min()} to
 ↪  {fpt_data['year'].max()}")
print(f"Number of observations: {len(fpt_data)}")
```

```
Available years: 2002 to 2023
Number of observations: 22
```

24.3.1 Computing Historical Free Cash Flow

Let's calculate the components needed for FCF from the financial statement data:

```
# Extract and compute FCF components
historical_data = (fpt_data
    .assign(
        # Revenue for ratio calculations
        revenue=lambda x: x["is_net_revenue"],

        # EBIT = Earnings before interest and taxes
        # Approximate as EBT + Interest Expense
        ebit=lambda x: x["is_ebt"] + x["is_interest_expense"],

        # Tax payments (use actual tax expense)
        taxes=lambda x: x["is_cit_expense"],

        # Depreciation and amortization (non-cash add-back)
        depreciation=lambda x: x["cfo_depreciation"],
```

```
        # Change in working capital components
        # Positive delta_wc means cash is consumed (tied up in working
         ↪  capital)
        delta_working_capital=lambda x: (
            x["cfo_receive"] +      # Change in receivables
            x["cfo_inventory"] -    # Change in inventory
            x["cfo_payale"]         # Change in payables (negative = cash
 ↪  source)
        ),

        # Capital expenditures
        capex=lambda x: x["capex"]
    )
    .loc[:, [
        "year", "revenue", "ebit", "taxes", "depreciation",
        "delta_working_capital", "capex"
    ]]
)

# Calculate Free Cash Flow
historical_data["fcf"] = (
    historical_data["ebit"]
    - historical_data["taxes"]
    + historical_data["depreciation"]
    - historical_data["delta_working_capital"]
    - historical_data["capex"]
)

historical_data
```

	year	revenue	ebit	taxes	depreciation	delta_working_capital	capex
0	2002	1.514961e+12	2.698700e+10	0.000000e+00	1.261500e+10	-2.561760e+11	2.202800
1	2003	4.148298e+12	5.676100e+10	0.000000e+00	1.837700e+10	-5.078740e+11	3.753300
2	2004	8.734781e+12	2.145902e+11	1.795700e+10	2.947900e+10	-4.280270e+11	5.252100
3	2005	1.410079e+13	3.753490e+11	4.251500e+10	5.381700e+10	-4.471110e+11	1.428320
4	2006	2.139975e+13	6.672593e+11	7.368682e+10	1.068192e+11	-1.173099e+12	2.459780
5	2007	1.349889e+13	1.071941e+12	1.487146e+11	1.709335e+11	-1.873794e+12	4.802762
6	2008	1.638184e+13	1.320573e+12	1.890384e+11	2.395799e+11	-1.419506e+11	6.690461
7	2009	1.840403e+13	1.807221e+12	2.916482e+11	3.041813e+11	-8.065011e+11	7.632280
8	2010	2.001730e+13	2.261341e+12	3.314359e+11	3.294060e+11	-2.360993e+12	8.672138

	year	revenue	ebit	taxes	depreciation	delta_working_capital	capex
9	2011	2.537025e+13	2.751044e+12	4.223952e+11	3.759567e+11	-2.099380e+12	4.524081
10	2012	2.459430e+13	2.635219e+12	4.210738e+11	3.995598e+11	8.043763e+11	7.083318
11	2013	2.702789e+13	2.690568e+12	4.503170e+11	4.429860e+11	-1.947751e+12	9.110216
12	2014	3.264466e+13	2.625389e+12	3.800994e+11	5.472736e+11	-3.078130e+12	1.417399
13	2015	3.795970e+13	3.113651e+12	4.130641e+11	7.328801e+11	-1.951778e+12	1.974295
14	2016	3.953147e+13	3.388085e+12	4.382078e+11	9.334397e+11	-9.242713e+11	1.428472
15	2017	4.265861e+13	4.623663e+12	7.270039e+11	1.039417e+12	-4.638788e+12	1.100498
16	2018	2.321354e+13	4.095947e+12	6.236054e+11	1.164692e+12	-1.033438e+12	2.452902
17	2019	2.771696e+13	5.023518e+12	7.528183e+11	1.354613e+12	-5.308818e+11	3.230818
18	2020	2.983040e+13	5.648794e+12	8.397114e+11	1.490607e+12	-8.040730e+11	3.014322
19	2021	3.565726e+13	6.821202e+12	9.879053e+11	1.643916e+12	-2.821825e+12	2.908134
20	2022	4.400953e+13	8.308009e+12	1.170940e+12	1.833064e+12	-3.746661e+12	3.209581
21	2023	5.261790e+13	1.003565e+13	1.414956e+12	2.286514e+12	-2.147304e+12	3.948982

24.3.2 Understanding the Historical Pattern

Before forecasting, we should understand the historical trends in FCF and its components:

```
# Calculate key ratios relative to revenue
historical_ratios = (historical_data
    .assign(
        # Revenue growth (year-over-year)
        revenue_growth=lambda x: x["revenue"].pct_change(),

        # Operating margin: EBIT as % of revenue
        operating_margin=lambda x: x["ebit"] / x["revenue"],

        # Depreciation as % of revenue
        depreciation_margin=lambda x: x["depreciation"] / x["revenue"],

        # Tax rate (taxes as % of revenue, for simplicity)
        tax_margin=lambda x: x["taxes"] / x["revenue"],

        # Working capital intensity
        working_capital_margin=lambda x: x["delta_working_capital"] /
 ↪  x["revenue"],

        # Capital intensity
        capex_margin=lambda x: x["capex"] / x["revenue"],
```

```
        # FCF margin
        fcf_margin=lambda x: x["fcf"] / x["revenue"]
    )
)

# Display key metrics
display_cols = [
    "year", "revenue_growth", "operating_margin", "depreciation_margin",
    "tax_margin", "working_capital_margin", "capex_margin", "fcf_margin"
]

historical_ratios[display_cols].round(3)
```

	year	revenue_growth	operating_margin	depreciation_margin	tax_margin	working_capital_mar
0	2002	NaN	0.018	0.008	0.000	-0.169
1	2003	1.738	0.014	0.004	0.000	-0.122
2	2004	1.106	0.025	0.003	0.002	-0.049
3	2005	0.614	0.027	0.004	0.003	-0.032
4	2006	0.518	0.031	0.005	0.003	-0.055
5	2007	-0.369	0.079	0.013	0.011	-0.139
6	2008	0.214	0.081	0.015	0.012	-0.009
7	2009	0.123	0.098	0.017	0.016	-0.044
8	2010	0.088	0.113	0.016	0.017	-0.118
9	2011	0.267	0.108	0.015	0.017	-0.083
10	2012	-0.031	0.107	0.016	0.017	0.033
11	2013	0.099	0.100	0.016	0.017	-0.072
12	2014	0.208	0.080	0.017	0.012	-0.094
13	2015	0.163	0.082	0.019	0.011	-0.051
14	2016	0.041	0.086	0.024	0.011	-0.023
15	2017	0.079	0.108	0.024	0.017	-0.109
16	2018	-0.456	0.176	0.050	0.027	-0.045
17	2019	0.194	0.181	0.049	0.027	-0.019
18	2020	0.076	0.189	0.050	0.028	-0.027
19	2021	0.195	0.191	0.046	0.028	-0.079
20	2022	0.234	0.189	0.042	0.027	-0.085
21	2023	0.196	0.191	0.043	0.027	-0.041

24.4 Visualizing Historical Ratios

Figure 24.1 shows the historical evolution of key financial ratios that drive FCF. Understanding these patterns helps inform our forecasts.

```
# Prepare data for plotting
ratio_columns = [
    "operating_margin", "depreciation_margin", "tax_margin",
    "working_capital_margin", "capex_margin"
]

ratios_long = (historical_ratios
    .melt(
        id_vars=["year"],
        value_vars=ratio_columns,
        var_name="ratio",
        value_name="value"
    )
    .assign(
        ratio=lambda x: x["ratio"].str.replace("_", " ").str.title()
    )
)

ratios_figure = (
    ggplot(ratios_long, aes(x="year", y="value", color="ratio"))
    + geom_line(size=1)
    + geom_point(size=2)
    + scale_y_continuous(labels=percent_format())
    + labs(
        x="", y="Ratio (% of Revenue)", color="",
        title="Key Financial Ratios of FPT Over Time"
    )
    + theme(legend_position="right")
)

ratios_figure.show()
```

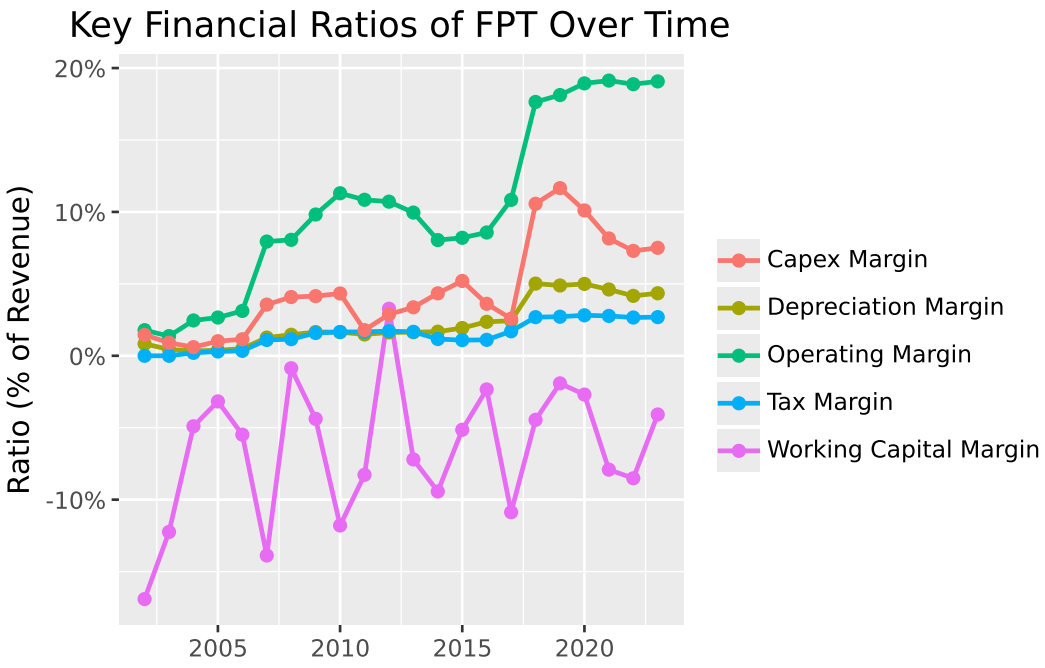


Figure 24.1: Historical financial ratios reveal the operating characteristics of FPT. These patterns inform our forecast assumptions.

Several patterns emerge from the historical data. Operating margins show the profitability of core operations. Depreciation margins indicate asset intensity. CAPEX margins reveal investment requirements. Working capital margins can be volatile, reflecting changes in credit terms and inventory management.

24.5 Forecasting Free Cash Flow

With historical patterns established, we now project FCF into the future. This requires forecasting both revenue growth and the ratios that convert revenue into cash flow.

24.5.1 The Ratio-Based Forecasting Approach

We use a ratio-based approach that links all FCF components to revenue. This makes forecasting tractable: rather than projecting absolute dollar amounts for each component, we forecast (1) revenue growth and (2) how each component scales with revenue.

This approach embeds a key assumption: that the relationship between revenue and FCF components remains stable. In reality, operating leverage, investment needs, and working

capital requirements may change as companies mature. Sophisticated valuations model these dynamics explicitly.

24.5.2 Setting Forecast Assumptions

For our five-year forecast, we make the following assumptions about FPT's financial ratios. These should reflect industry analysis, company guidance, and competitive dynamics. Here we use estimates for illustration:

```
# Define the forecast horizon
last_historical_year = historical_data["year"].max()
forecast_years = list(range(last_historical_year + 1, last_historical_year +
 ↪  6))
n_forecast_years = len(forecast_years)

print(f"Forecast period: {forecast_years[0]} to {forecast_years[-1]}")

# Define forecast ratios
# In practice, these would come from detailed analysis
forecast_assumptions = pd.DataFrame({
    "year": forecast_years,
    # Operating margin: slight improvement as scale increases
    "operating_margin": [0.12, 0.125, 0.13, 0.13, 0.135],
    # Depreciation: stable as % of revenue
    "depreciation_margin": [0.03, 0.03, 0.03, 0.028, 0.028],
    # Tax rate: stable
    "tax_margin": [0.02, 0.02, 0.02, 0.02, 0.02],
    # Working capital: modest cash consumption
    "working_capital_margin": [0.01, 0.01, 0.008, 0.008, 0.008],
    # CAPEX: declining as % of revenue as growth moderates
    "capex_margin": [0.05, 0.048, 0.045, 0.042, 0.04]
})

forecast_assumptions
```

```
Forecast period: 2024 to 2028
```

	year	operating_margin	depreciation_margin	tax_margin	working_capital_margin	capex_margin
0	2024	0.120	0.030	0.02	0.010	0.050
1	2025	0.125	0.030	0.02	0.010	0.048

	year	operating_margin	depreciation_margin	tax_margin	working_capital_margin	capex_margin
2	2026	0.130	0.030	0.02	0.008	0.045
3	2027	0.130	0.028	0.02	0.008	0.042
4	2028	0.135	0.028	0.02	0.008	0.040

24.5.3 Forecasting Revenue Growth

Revenue growth is often the most important and most uncertain assumption in DCF analysis. We demonstrate two approaches: using historical averages and linking growth to macroeconomic forecasts.

Approach 1: Historical Average

A simple approach uses the historical average growth rate:

```
historical_growth = historical_ratios["revenue_growth"].dropna()
avg_historical_growth = historical_growth.mean()

print(f"Average historical revenue growth: {avg_historical_growth:.1%}")
```

```
Average historical revenue growth: 25.2%
```

Approach 2: GDP-Linked Growth

A more sophisticated approach links company growth to GDP forecasts from institutions like the IMF. This captures the intuition that company revenues often move with broader economic activity.

```
# Vietnam GDP growth forecasts (illustrative, based on IMF WEO style
 ↪  projections)
# In practice, download from IMF WEO database
gdp_forecasts = pd.DataFrame({
    "year": forecast_years,
    "gdp_growth": [0.065, 0.063, 0.060, 0.058, 0.055]  # Gradually declining
     ↪  to long-term
})

# Assume FPT grows at a premium to GDP (tech sector outperformance)
# This premium should reflect company-specific factors
growth_premium = 0.05  # 5 percentage points above GDP
```

```
forecast_assumptions = forecast_assumptions.merge(gdp_forecasts, on="year")
forecast_assumptions["revenue_growth"] = (
    forecast_assumptions["gdp_growth"] + growth_premium
)

forecast_assumptions[["year", "gdp_growth", "revenue_growth"]]
```

	year	gdp_growth	revenue_growth
0	2024	0.065	0.115
1	2025	0.063	0.113
2	2026	0.060	0.110
3	2027	0.058	0.108
4	2028	0.055	0.105

24.5.4 Building the Forecast

Now we combine our assumptions to project revenue and FCF:

```
# Get the last historical revenue as our starting point
last_revenue = historical_data.loc[
    historical_data["year"] == last_historical_year, "revenue"
].values[0]

print(f"Last historical revenue ({last_historical_year}):
 ↪  {last_revenue/1e12:.2f} trillion VND")

# Project revenue forward
forecast_data = forecast_assumptions.copy()
forecast_data["revenue"] = None

# Calculate revenue for each forecast year
for i, row in forecast_data.iterrows():
    if i == 0:
        # First forecast year: grow from last historical
        forecast_data.loc[i, "revenue"] = last_revenue * (1 +
 ↪  row["revenue_growth"])
    else:
        # Subsequent years: grow from previous forecast
        prev_revenue = forecast_data.loc[i-1, "revenue"]
        forecast_data.loc[i, "revenue"] = prev_revenue * (1 +
 ↪  row["revenue_growth"])
```

```
# Convert revenue to numeric
forecast_data["revenue"] = forecast_data["revenue"].astype(float)

# Calculate FCF components from ratios
forecast_data["ebit"] = forecast_data["operating_margin"] *
 ↪  forecast_data["revenue"]
forecast_data["depreciation"] = forecast_data["depreciation_margin"] *
 ↪  forecast_data["revenue"]
forecast_data["taxes"] = forecast_data["tax_margin"] *
 ↪  forecast_data["revenue"]
forecast_data["delta_working_capital"] =
 ↪  forecast_data["working_capital_margin"] * forecast_data["revenue"]
forecast_data["capex"] = forecast_data["capex_margin"] *
 ↪  forecast_data["revenue"]

# Calculate FCF
forecast_data["fcf"] = (
    forecast_data["ebit"]
    - forecast_data["taxes"]
    + forecast_data["depreciation"]
    - forecast_data["delta_working_capital"]
    - forecast_data["capex"]
)

forecast_data[["year", "revenue", "ebit", "fcf"]].round(0)
```

```
Last historical revenue (2023): 52.62 trillion VND
```

	year	revenue	ebit	fcf
0	2024	5.866896e+13	7.040275e+12	4.106827e+12
1	2025	6.529855e+13	8.162319e+12	5.027988e+12
2	2026	7.248139e+13	9.422581e+12	6.305881e+12
3	2027	8.030938e+13	1.044022e+13	7.067226e+12
4	2028	8.874187e+13	1.198015e+13	8.430477e+12

24.6 Visualizing the Forecast

Figure 24.2 compares our forecast ratios with historical values, showing the transition from realized to projected performance.

```
# Prepare historical data for plotting
historical_plot = (historical_ratios
    .loc[:, ["year", "operating_margin", "depreciation_margin",
             "tax_margin", "working_capital_margin", "capex_margin"]]
    .assign(type="Historical")
)

# Prepare forecast data for plotting
forecast_plot = (forecast_data
    .loc[:, ["year", "operating_margin", "depreciation_margin",
             "tax_margin", "working_capital_margin", "capex_margin"]]
    .assign(type="Forecast")
)

# Combine
combined_ratios = pd.concat([historical_plot, forecast_plot],
 ↪  ignore_index=True)

# Reshape for plotting
combined_long = combined_ratios.melt(
    id_vars=["year", "type"],
    var_name="ratio",
    value_name="value"
)

combined_long["type"] = pd.Categorical(
    combined_long["type"],
    categories=["Historical", "Forecast"]
)

forecast_ratios_figure = (
    ggplot(combined_long, aes(x="year", y="value", color="ratio",
 ↪  linetype="type"))
    + geom_line(size=1)
    + geom_point(size=2)
    + scale_y_continuous(labels=percent_format())
    + labs(
        x="", y="Ratio (% of Revenue)", color="", linetype="",
        title="Historical and Forecast Financial Ratios for FPT"
    )
    + theme(legend_position="right")
)
```

```
forecast_ratios_figure.show()
```

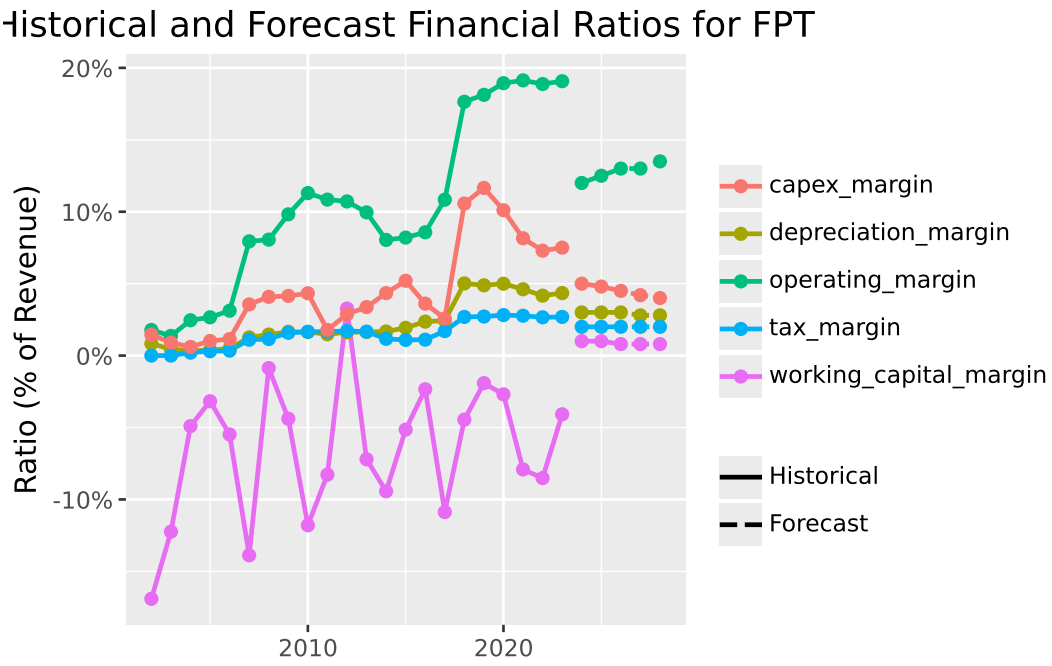


Figure 24.2: Historical ratios (solid lines) and forecast assumptions (dashed lines) for key financial metrics. The forecast period begins after the last historical observation.

Figure 24.3 shows the revenue growth trajectory, comparing historical performance with our GDP-linked forecasts.

```
# Prepare growth data
historical_growth_df = (historical_ratios
    .loc[:, ["year", "revenue_growth"]]
    .dropna()
    .assign(type="Historical")
)

forecast_growth_df = (forecast_data
    .loc[:, ["year", "revenue_growth", "gdp_growth"]]
    .assign(type="Forecast")
)

# Combine for revenue growth
```

```
growth_combined = pd.concat([
    historical_growth_df,
    forecast_growth_df[["year", "revenue_growth", "type"]]
], ignore_index=True)

growth_combined["type"] = pd.Categorical(
    growth_combined["type"],
    categories=["Historical", "Forecast"]
)

growth_figure = (
    ggplot(growth_combined, aes(x="year", y="revenue_growth",
 ↪  linetype="type"))
    + geom_line(size=1, color="steelblue")
    + geom_point(size=2, color="steelblue")
    + scale_y_continuous(labels=percent_format())
    + labs(
        x="", y="Revenue Growth Rate", linetype="",
        title="Historical and Forecast Revenue Growth for FPT"
    )
)

growth_figure.show()
```

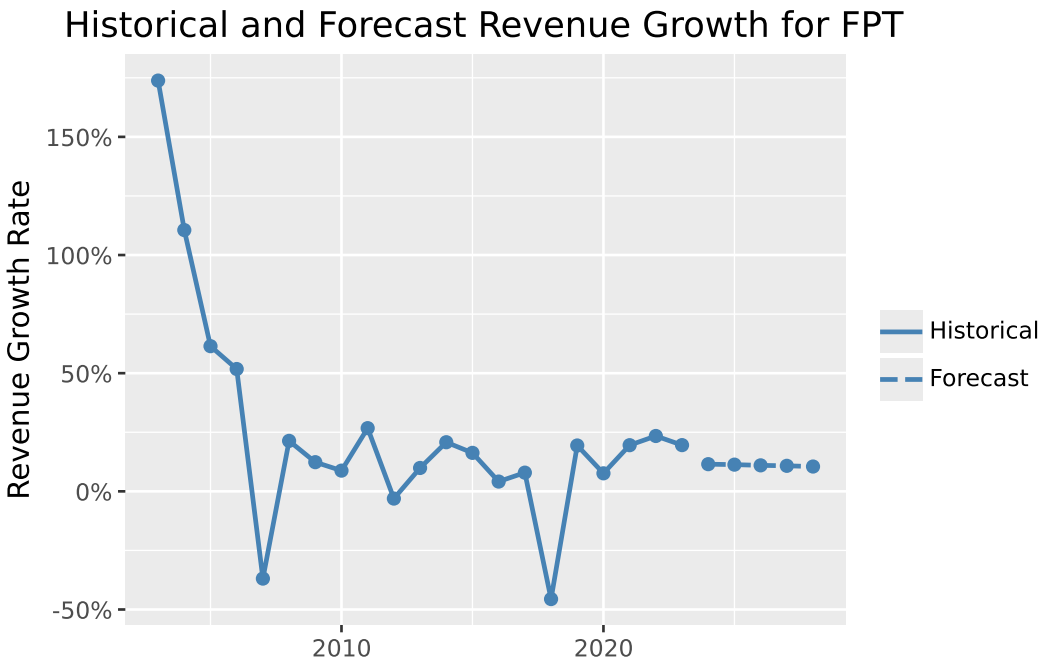


Figure 24.3: Revenue growth rates: historical (realized) and forecast (GDP-linked with company premium). The forecast assumes FPT grows at a premium to Vietnam's GDP growth.

Figure 24.4 presents the resulting FCF projections alongside historical values.

```
# Combine historical and forecast FCF
fcf_historical = (historical_data
    .loc[:, ["year", "fcf"]]
    .assign(type="Historical")
)

fcf_forecast = (forecast_data
    .loc[:, ["year", "fcf"]]
    .assign(type="Forecast")
)

fcf_combined = pd.concat([fcf_historical, fcf_forecast], ignore_index=True)
fcf_combined["type"] = pd.Categorical(
    fcf_combined["type"],
    categories=["Historical", "Forecast"]
)
```

```
fcf_figure = (
    ggplot(fcf_combined, aes(x="year", y="fcf/1e12", fill="type"))
    + geom_col()
    + labs(
        x="", y="Free Cash Flow (Trillion VND)", fill="",
        title="Historical and Forecast Free Cash Flow for FPT"
    )
)

fcf_figure.show()
```

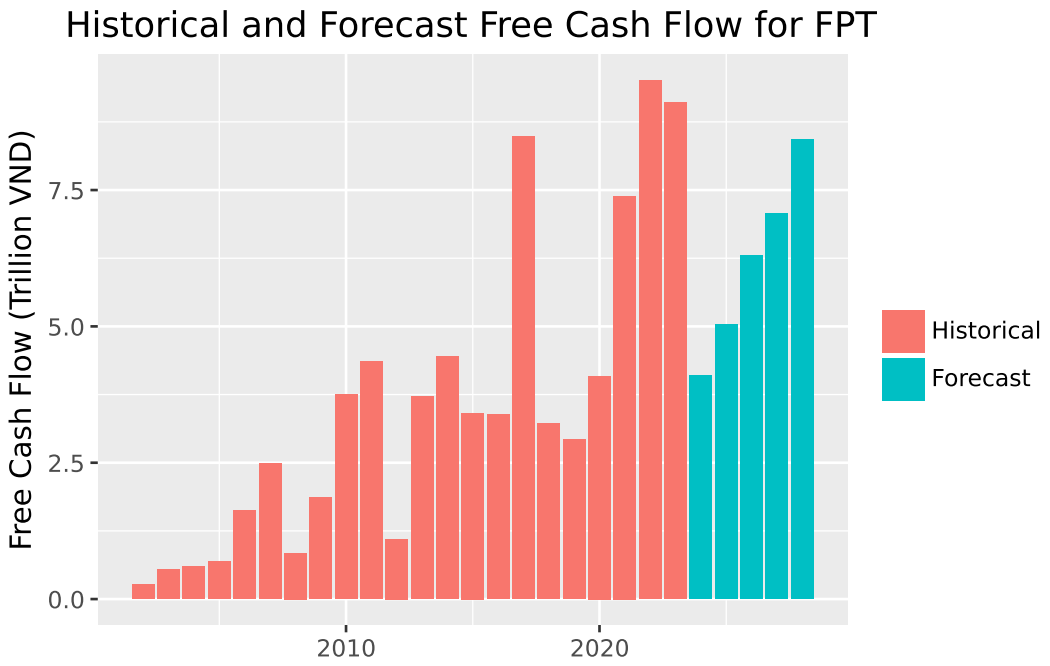


Figure 24.4: Free Cash Flow: historical (realized) and forecast (projected). The forecast reflects our assumptions about revenue growth and operating ratios.

24.7 Terminal Value: Capturing Long-Term Value

A critical component of DCF analysis is the **terminal value** (or continuation value), which represents the company's value beyond the explicit forecast period. In most valuations, terminal value constitutes 50-80% of total enterprise value, making its estimation particularly important.

24.7.1 The Perpetuity Growth Model

The most common approach is the **Perpetuity Growth Model** (also called the Gordon Growth Model), which assumes FCF grows at a constant rate forever:

$$TV_T = \frac{FCF_{T+1}}{r - g} = \frac{FCF_T \times (1 + g)}{r - g}$$

where:

- TV_T: Terminal value at the end of year T
- FCF_T: Free cash flow in the final forecast year
- g: Perpetual growth rate
- r: Discount rate (WACC)

24.7.2 Choosing the Perpetual Growth Rate

The perpetual growth rate g should reflect long-term sustainable growth. Key considerations:

1. **No company can grow faster than the economy forever**. If it did, the company would eventually become larger than GDP, which is an impossibility. Long-term GDP growth (nominal, including inflation) provides an upper bound.
2. **Mature companies typically grow at or below GDP growth**. The perpetual growth rate should reflect the company in its "steady state," not its current high-growth phase.
3. **For Vietnam**, long-term nominal GDP growth might be 6-8% given current development stage, but this will moderate over time. A perpetual growth rate of 3-5% is often reasonable.

```
def compute_terminal_value(last_fcf, growth_rate, discount_rate):
    """
    Compute terminal value using the perpetuity growth model.

    Parameters:
    -----------
    last_fcf : float
        Free cash flow in the final forecast year
    growth_rate : float
        Perpetual growth rate (g)
    discount_rate : float
```

```
        Discount rate / WACC (r)

    Returns:
    --------
    float : Terminal value
    """
    if discount_rate <= growth_rate:
        raise ValueError("Discount rate must exceed growth rate for finite
         ↪  terminal value")

    return last_fcf * (1 + growth_rate) / (discount_rate - growth_rate)


# Example calculation
last_fcf = forecast_data["fcf"].iloc[-1]
perpetual_growth = 0.04  # 4% perpetual growth
discount_rate = 0.10     # 10% WACC (placeholder)

terminal_value = compute_terminal_value(last_fcf, perpetual_growth,
 ↪  discount_rate)

print(f"Last forecast FCF: {last_fcf/1e12:.2f} trillion VND")
print(f"Terminal value (at {perpetual_growth:.0%} growth,
 ↪  {discount_rate:.0%} WACC): {terminal_value/1e12:.1f} trillion VND")
```

```
Last forecast FCF: 8.43 trillion VND
Terminal value (at 4% growth, 10% WACC): 146.1 trillion VND
```

24.7.3 Alternative: Exit Multiple Approach

Practitioners often cross-check terminal value using the **exit multiple approach**, which assumes the company is sold at the end of the forecast period at a multiple of EBITDA, EBIT, or revenue comparable to similar companies today.

For example, if comparable companies trade at 10x EBITDA, the terminal value would be:

$$TV_T = \text{EBITDA}_T \times \text{Exit Multiple}$$

This approach is simpler but embeds the assumption that current market multiples will persist (a strong assumption that may not hold).

24.8 The Discount Rate: Weighted Average Cost of Capital

The discount rate converts future cash flows to present value. For FCF (which goes to all capital providers), we use the **Weighted Average Cost of Capital (WACC)**:

$$WACC = \frac{E}{E+D} \times r_E + \frac{D}{E+D} \times r_D \times (1-\tau)$$

where:

- E: Market value of equity
- D: Market value of debt
- r_E: Cost of equity (typically estimated using CAPM)
- r_D: Cost of debt (pre-tax)
- τ: Corporate tax rate

The $(1-\tau)$ term on debt reflects the tax shield. Interest payments are tax-deductible, reducing the effective cost of debt.

24.8.1 Estimating WACC Components

Cost of Equity is typically estimated using the Capital Asset Pricing Model. See the CAPM chapter:

$$r_E = r_f + \beta \times (r_m - r_f)$$

where r_f is the risk-free rate, β measures systematic risk, and $(r_m - r_f)$ is the market risk premium.

Cost of Debt can be estimated from:

- Interest expense divided by total debt (effective rate)
- Yields on the company's traded bonds
- Yields on bonds with similar credit ratings

Capital Structure Weights should use market values when available. For equity, market capitalization is straightforward. For debt, book value is often used when market values aren't observable.

24.8.2 Using Industry WACC Data

Professor Aswath Damodaran at NYU Stern maintains comprehensive industry WACC and country risk premium data. We use these datasets to estimate appropriate discount rates for Vietnamese companies.

24.8.2.1 Downloading the Data

The following code downloads the required datasets. Run this manually when you need to update the data (typically annually), but it is not executed during book rendering to avoid dependency on external servers.

```
import requests
from pathlib import Path

# Create data directory if needed
data_dir = Path("data")
data_dir.mkdir(exist_ok=True)

# Download WACC data
wacc_url = "https://pages.stern.nyu.edu/~adamodar/pc/datasets/wacc.xls"
response = requests.get(wacc_url, timeout=30)
response.raise_for_status()
(data_dir / "damodaran_wacc.xls").write_bytes(response.content)
print("Downloaded: damodaran_wacc.xls")

# Download country risk premium data
crp_url = "https://pages.stern.nyu.edu/~adamodar/pc/datasets/ctryprem.xlsx"
response = requests.get(crp_url, timeout=30)
response.raise_for_status()
(data_dir / "damodaran_crp.xlsx").write_bytes(response.content)
print("Downloaded: damodaran_crp.xlsx")
```

24.8.2.2 Industry WACC

We extract the cost of capital for the Computer Services industry, which most closely matches FPT's business profile:

```
import pandas as pd

# Read local WACC data
```

```
wacc_data = pd.read_excel(
    "data/damodaran_wacc.xls",
    sheet_name=1,
    skiprows=18
)

# Find WACC for Computer Services
industry_wacc = wacc_data.loc[
    wacc_data["Industry Name"] == "Computer Services",
    "Cost of Capital"
].values[0]

print(f"Industry WACC (Computer Services): {industry_wacc:.2%}")
```

```
Industry WACC (Computer Services): 7.83%
```

24.8.2.3 Country Risk Premium

For Vietnamese companies, we must adjust for country-specific risk. Damodaran calculates country risk premiums based on sovereign credit ratings and relative equity market volatility:

```
# Read local country risk premium data
crp_data = pd.read_excel(
    "data/damodaran_crp.xlsx",
    sheet_name="ERPs by country",
    skiprows=7
)

# Find Vietnam's country risk premium
vietnam_row = crp_data[
    crp_data.iloc[:, 0].str.contains("Vietnam", case=False, na=False)
]

country_risk_premium = vietnam_row.iloc[0, 8]
print(f"Vietnam Country Risk Premium: {country_risk_premium:.2%}")
```

```
Vietnam Country Risk Premium: 2.09%
```

24.8.2.4 Adjusted WACC for Vietnam

Combining the industry benchmark with the country risk premium gives us an appropriate discount rate:

```
wacc_vietnam = industry_wacc + country_risk_premium

print(f"Industry WACC (US):         {industry_wacc:.2%}")
print(f"Country Risk Premium:       {country_risk_premium:.2%}")
print(f"Adjusted WACC (Vietnam):    {wacc_vietnam:.2%}")

wacc = wacc_vietnam
```

```
Industry WACC (US):         7.83%
Country Risk Premium:       2.09%
Adjusted WACC (Vietnam):    9.92%
```

Note: This approach assumes Vietnamese companies in the same industry face similar operating risks as their US counterparts, with the country risk premium capturing additional macroeconomic and political risks. For company-specific analysis, further adjustments for leverage differences and firm-specific risk factors may be warranted.

24.9 Computing Enterprise Value

With all components in place, we can now compute enterprise value. The DCF formula is:

$$\text{Enterprise Value} = \sum_{t=1}^{T} \frac{FCF_t}{(1+WACC)^t} + \frac{TV_T}{(1+WACC)^T}$$

The first term is the present value of forecast-period cash flows; the second is the present value of terminal value.

```
def compute_dcf_value(fcf_series, wacc, perpetual_growth):
    """
    Compute enterprise value using DCF analysis.

    Parameters:
    -----------
    fcf_series : array-like
```

```
        Free cash flows for forecast period
    wacc : float
        Weighted average cost of capital
    perpetual_growth : float
        Perpetual growth rate for terminal value

    Returns:
    --------
    dict : Components of DCF valuation
    """
    fcf = np.array(fcf_series)
    n_years = len(fcf)

    # Discount factors
    discount_factors = (1 + wacc) ** np.arange(1, n_years + 1)

    # Present value of forecast period cash flows
    pv_fcf = fcf / discount_factors
    pv_fcf_total = pv_fcf.sum()

    # Terminal value and its present value
    terminal_value = compute_terminal_value(fcf[-1], perpetual_growth, wacc)
    pv_terminal = terminal_value / discount_factors[-1]

    # Total enterprise value
    enterprise_value = pv_fcf_total + pv_terminal

    return {
        "pv_fcf": pv_fcf_total,
        "terminal_value": terminal_value,
        "pv_terminal": pv_terminal,
        "enterprise_value": enterprise_value,
        "terminal_pct": pv_terminal / enterprise_value
    }


# Compute DCF value
perpetual_growth = 0.04  # 4% perpetual growth

dcf_result = compute_dcf_value(
    fcf_series=forecast_data["fcf"].values,
    wacc=wacc,
    perpetual_growth=perpetual_growth
```

```
)

print("DCF Valuation Results")
print("=" * 50)
print(f"PV of Forecast Period FCF: {dcf_result['pv_fcf']/1e12:.1f} trillion
 ↪  VND")
print(f"Terminal Value: {dcf_result['terminal_value']/1e12:.1f} trillion
 ↪  VND")
print(f"PV of Terminal Value: {dcf_result['pv_terminal']/1e12:.1f} trillion
 ↪  VND")
print(f"Enterprise Value: {dcf_result['enterprise_value']/1e12:.1f} trillion
 ↪  VND")
print(f"Terminal Value as % of EV: {dcf_result['terminal_pct']:.1%}")
```

```
DCF Valuation Results
==================================================
PV of Forecast Period FCF: 22.7 trillion VND
Terminal Value: 148.1 trillion VND
PV of Terminal Value: 92.3 trillion VND
Enterprise Value: 115.1 trillion VND
Terminal Value as % of EV: 80.2%
```

Note that terminal value often represents 60-80% of enterprise value. This highlights the importance of terminal value assumptions and the inherent uncertainty in DCF analysis.

24.10 Sensitivity Analysis

Given the uncertainty in DCF inputs, sensitivity analysis is essential. We examine how enterprise value changes with different assumptions about WACC and perpetual growth.

```
# Define ranges for sensitivity analysis
wacc_range = np.arange(0.08, 0.14, 0.01)  # 8% to 13%
growth_range = np.arange(0.02, 0.06, 0.01)  # 2% to 5%

# Create all combinations
sensitivity_results = []

for w in wacc_range:
    for g in growth_range:
        if w > g:  # Must have WACC > growth for valid terminal value
```

```
            result = compute_dcf_value(
                fcf_series=forecast_data["fcf"].values,
                wacc=w,
                perpetual_growth=g
            )
            sensitivity_results.append({
                "wacc": w,
                "growth_rate": g,
                "enterprise_value": result["enterprise_value"] / 1e12  # In
                 ↪  trillions
            })

sensitivity_df = pd.DataFrame(sensitivity_results)

# Create heatmap
sensitivity_figure = (
    ggplot(sensitivity_df, aes(x="wacc", y="growth_rate",
 ↪  fill="enterprise_value"))
    + geom_tile()
    + geom_text(
        aes(label="enterprise_value"),
        format_string="{:.0f}",
        color="white",
        size=9
    )
    + scale_x_continuous(labels=percent_format())
    + scale_y_continuous(labels=percent_format())
    + scale_fill_gradient(low="darkblue", high="lightblue")
    + labs(
        x="WACC", y="Perpetual Growth Rate",
        fill="EV\n(Trillion VND)",
        title="DCF Sensitivity: Enterprise Value by WACC and Growth Rate"
    )
)

sensitivity_figure.show()
```

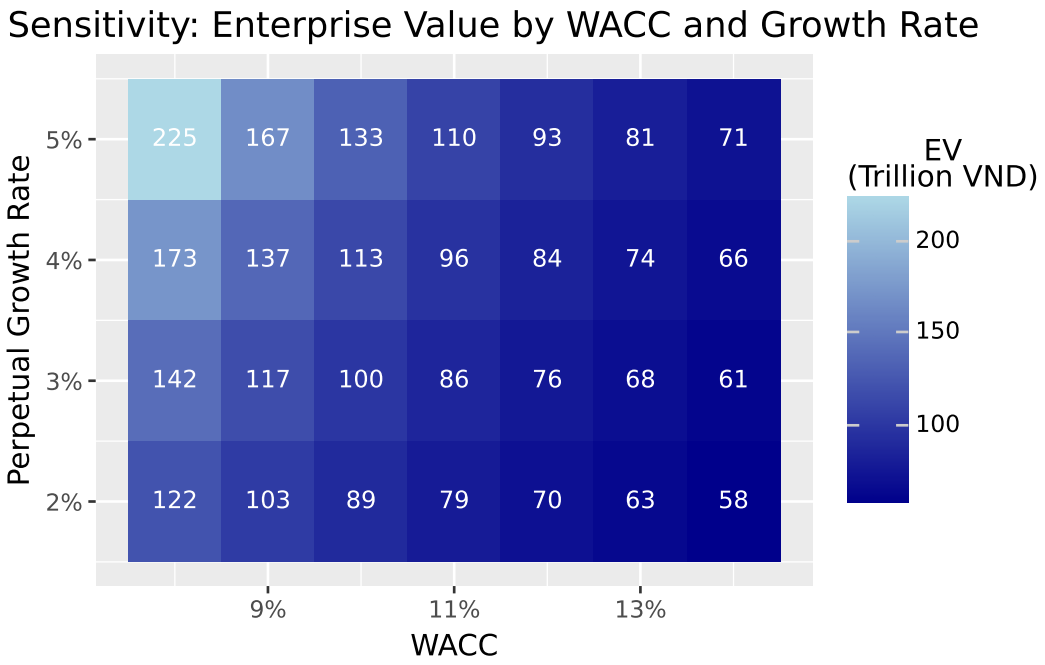


Figure 24.5: Sensitivity of enterprise value to WACC and perpetual growth rate assumptions. Small changes in these inputs can substantially affect valuation.

The sensitivity analysis reveals several important insights:

1. **Valuation is highly sensitive to inputs**: Small changes in WACC or growth rate produce large changes in enterprise value. A 1 percentage point change in WACC can shift value by 20% or more.
2. **The relationship is non-linear**: The impact of growth rate changes is amplified at lower WACCs because the terminal value formula has $(r - g)$ in the denominator.
3. **Reasonable people can disagree**: Given input uncertainty, DCF should be thought of as producing a *range* of values, not a single precise number.

24.11 From Enterprise Value to Equity Value

Our DCF analysis yields **enterprise value** (i.e., the total value of the company's operations to all capital providers). To determine **equity value** (what shareholders own), we must adjust for the claims of debt holders and any non-operating assets:

$$\text{Equity Value} = \text{Enterprise Value} + \text{Non-Operating Assets} - \text{Debt}$$

Non-Operating Assets include:

- Excess cash beyond operating needs
- Marketable securities
- Non-core real estate or investments

Debt includes:

- Short-term debt
- Long-term debt
- Capital lease obligations
- Preferred stock (if treated as debt-like)

```
# Get most recent balance sheet data for FPT
latest_year = fpt_data["year"].max()
latest_data = fpt_data[fpt_data["year"] == latest_year].iloc[0]

# Extract debt and cash (column names may vary)
total_debt = latest_data.get("total_debt", 0)
cash = latest_data.get("ca_cce", 0)

# Compute equity value
enterprise_value = dcf_result["enterprise_value"]
equity_value = enterprise_value - total_debt + cash

print("From Enterprise Value to Equity Value")
print("=" * 50)
print(f"Enterprise Value: {enterprise_value/1e12:.1f} trillion VND")
print(f"Less: Total Debt: {total_debt/1e12:.1f} trillion VND")
print(f"Plus: Cash: {cash/1e12:.1f} trillion VND")
print(f"Equity Value: {equity_value/1e12:.1f} trillion VND")
```

```
From Enterprise Value to Equity Value
==================================================
Enterprise Value: 115.1 trillion VND
Less: Total Debt: 0.0 trillion VND
Plus: Cash: 8.3 trillion VND
Equity Value: 123.3 trillion VND
```

24.11.1 Implied Share Price

If we know the number of shares outstanding, we can compute an implied share price:

```
# Get shares outstanding (this would come from market data)
# Using placeholder - in practice, get from exchange data
shares_outstanding = latest_data.get("total_equity", equity_value) / 25000  #
 ↪  Rough estimate

implied_price = equity_value / shares_outstanding

print(f"\nImplied Share Price: {implied_price:,.0f} VND")
```

```
Implied Share Price: 103,008 VND
```

Comparing the implied price to the current market price tells us whether the stock appears under- or overvalued according to our DCF model.

24.12 Limitations and Practical Considerations

DCF analysis is powerful but has important limitations:

24.12.1 Sensitivity to Assumptions

As our sensitivity analysis showed, small changes in inputs produce large changes in value. This is particularly problematic because the most influential inputs (long-term growth, WACC) are the hardest to estimate accurately.

24.12.2 Terminal Value Dominance

Terminal value often represents 60-80% of total value, yet it's based on assumptions about the very distant future. This concentrates valuation risk in the most uncertain component.

24.12.3 Garbage In, Garbage Out

DCF is only as good as its inputs. Unrealistic growth assumptions, optimistic margins, or inappropriate discount rates produce meaningless valuations. The discipline of DCF lies in forcing analysts to justify their assumptions.

24.12.4 Not Suitable for All Companies

DCF works best for companies with:

- Positive and predictable cash flows
- Stable or predictably changing margins
- Reasonable visibility into future operations

It struggles with:

- Early-stage companies with no profits
- Highly cyclical businesses
- Companies undergoing major transitions
- Financial institutions (which require different approaches)

24.12.5 Complement with Other Methods

Wise practitioners use DCF alongside other valuation methods:

- **Comparable company analysis**: How do similar companies trade?
- **Precedent transactions**: What have acquirers paid for similar businesses?
- **Sum-of-the-parts**: Value divisions separately and add

When methods converge, confidence increases. When they diverge, it prompts investigation into why.

24.13 Key Takeaways

This chapter introduced Discounted Cash Flow analysis as a framework for intrinsic valuation. The main insights are:

1. **Free Cash Flow is the foundation**: FCF represents cash available to all investors after operating expenses, taxes, and investments. It differs from net income by excluding non-cash items and including investment needs.

2. **Ratio-based forecasting links components to revenue**: By expressing FCF components as percentages of revenue, we can systematically forecast cash flows based on revenue growth assumptions and operating ratio projections.

3. **Terminal value captures long-term value**: The perpetuity growth model assumes FCF grows at a constant rate forever. The perpetual growth rate should not exceed long-term economic growth.

4. **WACC is the appropriate discount rate**: The Weighted Average Cost of Capital reflects the blended cost of debt and equity financing, adjusted for the tax shield on interest.

5. **DCF produces enterprise value**: To derive equity value, subtract debt and add non-operating assets. Dividing by shares outstanding yields an implied share price.

6. **Sensitivity analysis is essential**: Given input uncertainty, presenting a range of values based on different assumptions is more honest than a single point estimate.

7. **DCF complements other methods**: No single valuation method is definitive. Cross-checking DCF with market multiples and transaction comparables provides a more complete picture.

The true value of DCF analysis lies not in producing a precise number but in forcing rigorous thinking about what drives company value. The process of building a DCF model (i.e., forecasting growth, projecting margins, estimating risk) develops deep understanding of the business being valued.

25 Information Content of Earnings Announcements

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import matplotlib.dates as mdates
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
import warnings
warnings.filterwarnings("ignore")

plt.rcParams.update({
    "figure.dpi": 150,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.size": 11,
})
```

A foundational question in capital markets research is whether accounting information matters to investors. If financial statements merely repackage what the market already knows from other sources, such as press coverage, analyst reports, management guidance, macroeconomic data, then the elaborate apparatus of financial reporting is largely redundant for price discovery. Conversely, if earnings announcements trigger measurable changes in trading behavior, we have evidence that the accounting system produces information the market did not previously possess.

Beaver (1968) addressed this question by examining whether trading volume and return volatility increase around the dates when firms announce annual earnings. The logic is direct: if an earnings announcement contains new information, it should alter investors' beliefs. Altered beliefs lead some investors to trade, increasing volume. And to the extent that the announcement resolves uncertainty, the stock price should adjust, generating abnormal return volatility. Both

channels (i.e., volume and volatility) provide distinct windows into the information content of earnings.

The empirical design compares a firm's trading activity during a narrow **announcement window** (the days immediately around the earnings release) with its own trading activity during a **non-announcement window** (otherwise comparable days away from the event). If the announcement conveys information, the announcement window should show elevated volume and volatility relative to the non-announcement period.

25.0.1 Two Distinct Signals: Volume vs. Volatility

While volume and return volatility are both used to measure investor reaction, they capture different economic phenomena. This distinction, formalized in subsequent theoretical work, is important for interpreting results:

- **Return volatility** reflects the *average* revision in investor beliefs. When the market as a whole is surprised by an earnings number, prices move sharply. The magnitude of the squared (or absolute) return residual measures how far the announcement moved the consensus. This is a measure of the *aggregate information content* of the announcement (O. Kim and Verrecchia 1991).
- **Trading volume** reflects *disagreement* among investors about the implications of the announcement. Even an announcement that moves prices very little can generate heavy trading if investors interpret the same signal differently. For example, if some view an earnings beat as sustainable while others view it as temporary (Kandel and Pearson 1995). Volume thus captures the *differential information content* of the announcement.

Karpoff (1987) surveys the theoretical and empirical relationship between these two variables. The key insight is that volume is not simply a noisier version of volatility; it provides independent information about how the market processes accounting data.

25.0.2 Why Vietnam?

Vietnam's equity markets provide a distinctive setting for studying announcement reactions for several reasons:

1. **Information environment.** Vietnamese listed firms face less intense pre-announcement scrutiny than their U.S. counterparts. Analyst coverage is sparse (most firms outside the VN30 index receive no institutional coverage). Management earnings guidance is rare. This creates a thicker "information gap" for the earnings announcement to fill, potentially amplifying both volume and volatility reactions.

2. **Retail investor dominance.** Over 80% of daily trading volume on HOSE is attributed to retail investors. Retail traders may process earnings information differently from institutions, with greater behavioral biases, less capacity for fundamental analysis, and different trading horizons. This affects both the timing and magnitude of market reactions.

3. **Price limits.** HOSE imposes daily price limits of $\pm$ 7% and HNX of $\pm$ 10%. When an earnings announcement is sufficiently surprising, the stock may hit its price limit, preventing the market from fully incorporating the information within a single trading session. This creates a distinctive pattern of *multi-day adjustment* that has no analogue in the original U.S. studies.

4. **Disclosure timing.** Vietnamese regulations require audited annual financial statements to be published within 90 days of fiscal year-end. Quarterly financial data (unaudited) must be released within 20 days. The overlap between quarterly and annual announcements, and the variable lag between fiscal year-end and announcement date, creates heterogeneity in the information content of annual earnings releases.

5. **Market structure.** Vietnam operates continuous order-matching during trading hours (9:00–11:30 and 13:00–14:30 on HOSE; 9:00–11:30 and 13:00–15:00 on HNX), with periodic call auctions at open and close. This microstructure affects intraday patterns of price discovery around announcements.

25.1 Theoretical Framework

25.1.1 The Information Content Hypothesis

Let $P_{i,t}$ denote the price of stock i on date t. Under the efficient markets hypothesis, the price reflects all publicly available information:

$$P_{i,t} = E\left[\sum_{s=1}^{\infty} \frac{D_{i,t+s}}{(1+r)^s} \,\middle|\, \Omega_t\right] \tag{25.1}$$

where $D_{i,t+s}$ represents future dividends, r is the discount rate, and Ω_t is the information set at time t.

When firm i announces earnings X_i at time τ, the information set expands: $\Omega_{\tau^+} = \Omega_{\tau^-} \cup \{X_i\}$. If the announcement contains new information, meaning $X_i \notin \Omega_{\tau^-}$, then in general:

$$P_{i,\tau^+} \neq P_{i,\tau^-} \tag{25.2}$$

The magnitude of the price change reflects the *surprise* component of the announcement:

$$|P_{i,\tau^+} - P_{i,\tau^-}| \propto |X_i - E[X_i|\Omega_{\tau^-}]| \tag{25.3}$$

25.1.2 Volume as a Signal of Heterogeneous Beliefs

O. Kim and Verrecchia (1991) develop a model where trading volume is driven by the precision of investors' private information relative to the precision of the public signal. Define $V_{i,\tau}$ as the trading volume for stock i at time τ. In their framework:

$$V_{i,\tau} = f\left(\frac{\text{Var}[X_i|\Omega^{(j)}_{\tau^-}]}{\text{Var}[\varepsilon_i]}\right) \tag{25.4}$$

where $\Omega^{(j)}_{\tau^-}$ is investor j's private information set and ε_i is noise. Volume increases when investors disagree about what the announcement means, a condition that Kandel and Pearson (1995) term **differential interpretation**.

This distinction matters for Vietnam. In a market dominated by retail investors with heterogeneous information and analytical capacity, we might expect the volume reaction to be *disproportionately large* relative to the price reaction, because disagreement is high even when the aggregate price revision is modest.

25.1.3 Measuring Information Content

We use two complementary measures:

Abnormal return volatility. For each firm-day, compute the squared market-adjusted return:

$$U^2_{i,t} = (R_{i,t} - R_{m,t})^2 \tag{25.5}$$

where $R_{i,t}$ is the return on stock i and $R_{m,t}$ is the market return. Under the null of no information content, $E[U^2_{i,t}]$ should be the same in the announcement and non-announcement windows. Beaver (1968) standardized this measure by dividing by the firm's average U^2 during non-announcement weeks.

Relative volume. For each firm-day, compute:

$$RV_{i,t} = \frac{Vol_{i,t}}{\overline{Vol}_i} \tag{25.6}$$

where $\overline{Vol}_i$ is the firm's average daily volume over the event window. A value of $RV > 1$ indicates above-average trading. Beaver (1968) used a similar approach, computing a "volume ratio" relative to non-report-period averages.

25.2 Data and Variable Construction

25.2.1 Data Requirements

The analysis requires three data elements:

Table 25.1: Data sources for the information content analysis

Data	Source	Key Fields
Earnings announcement dates	Financial statements/filings	Ticker, fiscal year-end, announcement date
Daily stock returns	Exchange data (HOSE/HNX)	Ticker, date, closing price, return
Daily trading volume	Exchange data (HOSE/HNX)	Ticker, date, volume (shares or VND)

25.2.2 Trading Day Calendar

A critical infrastructure element is a **trading day calendar** that maps calendar dates to sequential trading day indices. This allows us to count "20 trading days before the announcement" correctly, skipping weekends and holidays.

```
def build_trading_calendar(
    start_date: str = "2008-01-01",
    end_date: str = "2024-12-31",
) -> pd.DataFrame:
    """
    Build a trading day calendar for Vietnamese stock exchanges.

    Vietnamese exchanges trade Monday-Friday, with closures for:
    - Tết Nguyên Đán (Lunar New Year, ~5 days)
    - Ngày Giỗ Tổ Hùng Vương (Hung Kings' Commemoration)
    - Ngày Giải phóng miền Nam (Reunification Day, Apr 30)
    - Ngày Quốc tế Lao động (Labour Day, May 1)
    - Quốc khánh (National Day, Sep 2)
    - Tết Dương lịch (New Year, Jan 1)
```

```
    For simulation, we approximate by removing weekends and ~10
↪   holidays/year.
    """
    all_dates = pd.date_range(start_date, end_date, freq="B")  # business
↪   days

    # Remove approximate Vietnamese public holidays
    # (In practice, use the exact holiday calendar from SSC/HOSE)
    rng = np.random.default_rng(42)
    n_holidays_per_year = 10
    years = all_dates.year.unique()
    holidays = []
    for yr in years:
        yr_dates = all_dates[all_dates.year == yr]
        if len(yr_dates) > n_holidays_per_year:
            idx = rng.choice(len(yr_dates), n_holidays_per_year,
↪   replace=False)
            holidays.extend(yr_dates[idx])

    trading_dates = all_dates.difference(pd.DatetimeIndex(holidays))

    cal = pd.DataFrame({
        "date": trading_dates,
        "td": np.arange(1, len(trading_dates) + 1),
    })
    return cal

trading_cal = build_trading_calendar()
print(f"Trading calendar: {len(trading_cal):,} trading days, "
      f"{trading_cal['date'].min().date()} to
       ↪  {trading_cal['date'].max().date()}")
```

```
Trading calendar: 4,266 trading days, 2008-01-01 to 2024-12-31
```

25.2.3 Announcement Date Alignment

Earnings announcements may occur on non-trading days (weekends, holidays). When this happens, we assign the announcement to the *next available* trading day, since that is the first opportunity for the market to react. We also need to handle the case where announcements occur after market close. Ideally, these would be assigned to the following trading day. Without

intraday timing data (common in Vietnamese disclosures), we use the announcement date as reported.

```
def build_announcement_mapper(trading_cal: pd.DataFrame) -> pd.DataFrame:
    """
    Create a mapping from every calendar date to the next available
    trading day index. This handles announcements on weekends/holidays.
    """
    min_date = trading_cal["date"].min()
    max_date = trading_cal["date"].max()
    all_dates = pd.date_range(min_date, max_date, freq="D")

    mapper = pd.DataFrame({"annc_date": all_dates})
    mapper = mapper.merge(
        trading_cal.rename(columns={"date": "annc_date"}),
        on="annc_date",
        how="left",
    )
    # Forward-fill: assign non-trading dates to next trading day
    mapper["td"] = mapper["td"].bfill()
    mapper["td"] = mapper["td"].astype("Int64")
    return mapper

annc_mapper = build_announcement_mapper(trading_cal)
```

25.2.4 Generating Synthetic Vietnamese Data

We generate a synthetic panel of Vietnamese listed firms with realistic properties to demonstrate the methodology. The simulation embeds an earnings announcement effect—elevated volume and volatility on days 0 and +1, that we then seek to detect using the Beaver (1968) framework.

```
def generate_beaver_data(
    n_firms: int = 300,
    years: list[int] = None,
    seed: int = 2024,
    days_window: int = 20,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """
    Generate:
    1. earn_annc: DataFrame of earnings announcement dates
    2. daily_data: DataFrame of daily returns and volume
```

```
The DGP embeds:
- An announcement-day return volatility spike (2-3x normal)
- An announcement-day volume spike (1.5-2.5x normal)
- Price limit truncation at ±7% (HOSE)
- Cross-sectional variation by firm size and exchange
"""
if years is None:
    years = list(range(2012, 2024))

rng = np.random.default_rng(seed)
tc = build_trading_calendar()
am = build_announcement_mapper(tc)

exchanges = ["HOSE", "HNX"]
price_limits = {"HOSE": 0.07, "HNX": 0.10}

#   Generate announcement dates
annc_records = []
for i in range(n_firms):
    ticker = f"VN{i:04d}"
    exchange = rng.choice(exchanges, p=[0.6, 0.4])
    # Firm size (log market cap in VND billions)
    log_mcap = rng.normal(6.5, 1.5)

    for yr in years:
        # Fiscal year ends Dec 31; announcement 60-90 days later
        fy_end = pd.Timestamp(f"{yr}-12-31")
        annc_lag = rng.integers(45, 90)
        annc_date = fy_end + pd.Timedelta(days=int(annc_lag))

        annc_records.append({
            "ticker": ticker,
            "fiscal_year": yr,
            "datadate": fy_end,
            "annc_date": annc_date,
            "exchange": exchange,
            "log_mcap": log_mcap,
        })

earn_annc = pd.DataFrame(annc_records)

# Map to trading day index
```

```
    earn_annc["annc_date_dt"] = pd.to_datetime(earn_annc["annc_date"])
    earn_annc = earn_annc.merge(
        am[["annc_date", "td"]].rename(columns={"td": "event_td"}),
        left_on="annc_date_dt",
        right_on="annc_date",
        how="left",
        suffixes=("", "_map"),
    )
    earn_annc = earn_annc.dropna(subset=["event_td"])
    earn_annc["event_td"] = earn_annc["event_td"].astype(int)
    earn_annc["start_td"] = earn_annc["event_td"] - days_window
    earn_annc["end_td"] = earn_annc["event_td"] + days_window

    # Map start/end td back to dates
    td_to_date = tc.set_index("td")["date"]
    earn_annc["start_date"] = earn_annc["start_td"].map(td_to_date)
    earn_annc["end_date"] = earn_annc["end_td"].map(td_to_date)
    earn_annc = earn_annc.dropna(subset=["start_date", "end_date"])

    #   Generate daily return and volume data
    daily_records = []
    for _, row in earn_annc.iterrows():
        ticker = row["ticker"]
        exchange = row["exchange"]
        plimit = price_limits[exchange]
        log_mcap = row["log_mcap"]
        event_td = row["event_td"]

        # Base volatility inversely related to size
        base_vol = 0.015 + 0.01 * np.exp(-0.3 * log_mcap)
        # Base volume (shares/day)
        base_volume = np.exp(log_mcap + rng.normal(0, 0.5)) * 1000

        # Window trading days
        window_tds = tc[
            (tc["td"] >= row["start_td"]) & (tc["td"] <= row["end_td"])
        ].copy()

        for _, td_row in window_tds.iterrows():
            date = td_row["date"]
            td = td_row["td"]
            relative_td = td - event_td
```

```
            # Market return (common factor)
            mkt_ret = rng.normal(0.0003, 0.012)

            # Firm-specific return
            # Announcement effect: elevated volatility on days 0, +1
            if relative_td in [0, 1]:
                vol_mult = rng.uniform(2.0, 3.5)
            elif abs(relative_td) <= 2:
                vol_mult = rng.uniform(1.2, 1.8)
            else:
                vol_mult = 1.0

            firm_shock = rng.normal(0, base_vol * vol_mult)
            raw_ret = mkt_ret + firm_shock

            # Apply price limits
            ret = np.clip(raw_ret, -plimit, plimit)
            ret_mkt = ret - mkt_ret

            # Volume: spike on announcement days
            if relative_td in [0, 1]:
                vol_spike = rng.uniform(1.5, 2.8)
            elif abs(relative_td) <= 3:
                vol_spike = rng.uniform(1.0, 1.5)
            else:
                vol_spike = rng.uniform(0.7, 1.3)

            volume = base_volume * vol_spike * np.exp(rng.normal(0, 0.3))

            daily_records.append({
                "ticker": ticker,
                "fiscal_year": row["fiscal_year"],
                "date": date,
                "td": td,
                "relative_td": relative_td,
                "ret": ret,
                "mkt_ret": mkt_ret,
                "ret_mkt": ret_mkt,
                "vol": max(volume, 100),
                "exchange": exchange,
                "log_mcap": log_mcap,
            })
```

```
    daily_data = pd.DataFrame(daily_records)

    return earn_annc, daily_data

earn_annc, daily_data = generate_beaver_data(n_firms=300, seed=2024)
print(f"Announcements: {len(earn_annc):,}")
print(f"Daily observations: {len(daily_data):,}")
```

```
Announcements: 3,600
Daily observations: 147,600
```

25.3 Single-Event Walkthrough

Before scaling to the full sample, we trace the methodology for a single earnings announcement to build intuition and verify each step.

25.3.1 Selecting the Event

```
single_ticker = "VN0042"
single_fy = 2019

single_event = earn_annc[
    (earn_annc["ticker"] == single_ticker) &
    (earn_annc["fiscal_year"] == single_fy)
].iloc[0]

print(f"Firm: {single_event['ticker']}")
print(f"Fiscal year-end: {single_event['datadate'].date()}")
print(f"Announcement date: {single_event['annc_date'].date()}")
print(f"Exchange: {single_event['exchange']}")
print(f"Event trading day: {single_event['event_td']}")
```

```
Firm: VN0042
Fiscal year-end: 2019-12-31
Announcement date: 2020-03-28
Exchange: HOSE
Event trading day: 3073
```

25.3.2 Event Window Data

We extract 41 trading days of data (20 before through 20 after the announcement) and compute the key measures.

```
single_daily = daily_data[
    (daily_data["ticker"] == single_ticker) &
    (daily_data["fiscal_year"] == single_fy)
].copy()

# Compute measures
single_daily["sq_ret_mkt"] = single_daily["ret_mkt"] ** 2
single_daily["abs_ret_mkt"] = single_daily["ret_mkt"].abs()

# Relative volume: normalize by own average
avg_vol = single_daily["vol"].mean()
single_daily["rel_vol"] = single_daily["vol"] / avg_vol

print(f"Trading days in window: {len(single_daily)}")
print(f"Average daily volume: {avg_vol:,.0f} shares")
```

```
Trading days in window: 41
Average daily volume: 3,780,757 shares
```

25.3.3 Visualizing the Single Event

Figure 25.1 and Figure 25.2 display the volume and volatility patterns for this single earnings announcement.

```
fig, ax = plt.subplots()
colors = ["firebrick" if abs(t) <= 1 else "steelblue"
          for t in single_daily["relative_td"]]
ax.bar(single_daily["relative_td"], single_daily["vol"],
       color=colors, edgecolor="white", width=0.8)
ax.axhline(avg_vol, color="grey", linestyle="--", linewidth=1,
           label="Window average")
ax.axvline(0, color="black", linewidth=0.5, alpha=0.3)
ax.set_xlabel("Trading Days Relative to Announcement")
ax.set_ylabel("Volume (shares)")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
 ↪  f"{x/1e6:.1f}M"))
```

```
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()
```

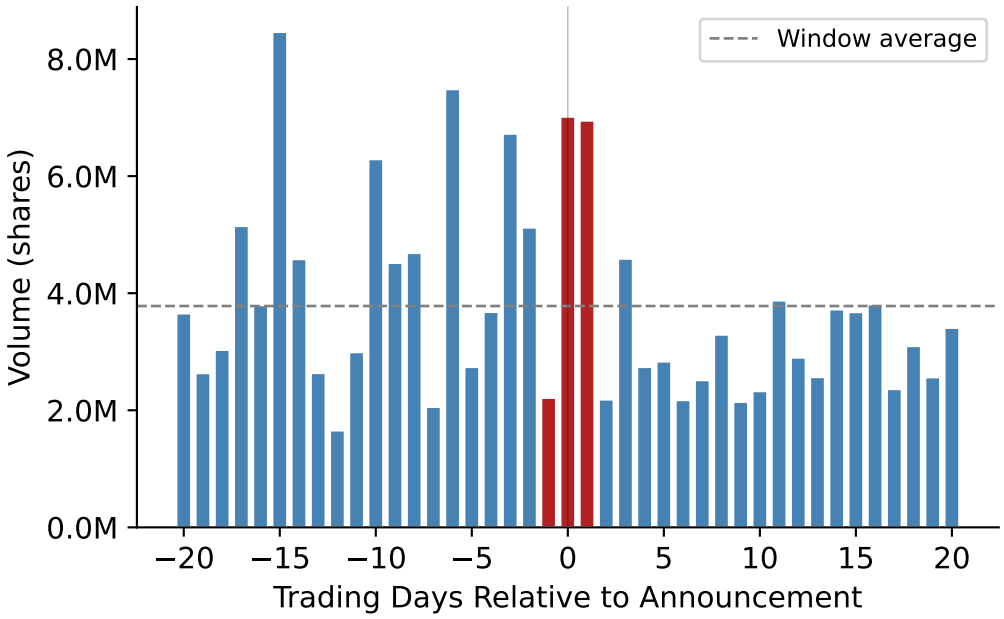


Figure 25.1: Daily trading volume around a single earnings announcement. Day 0 is the first trading day on or after the announcement date. The dashed line marks the event-window average.

```
fig, ax = plt.subplots()
ax.bar(single_daily["relative_td"], single_daily["sq_ret_mkt"],
       color=["firebrick" if abs(t) <= 1 else "steelblue"
              for t in single_daily["relative_td"]],
       edgecolor="white", width=0.8)
ax.axhline(single_daily["sq_ret_mkt"].mean(), color="grey",
           linestyle="--", linewidth=1, label="Window average")
ax.axvline(0, color="black", linewidth=0.5, alpha=0.3)
ax.set_xlabel("Trading Days Relative to Announcement")
ax.set_ylabel("Squared Market-Adjusted Return")
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()
```

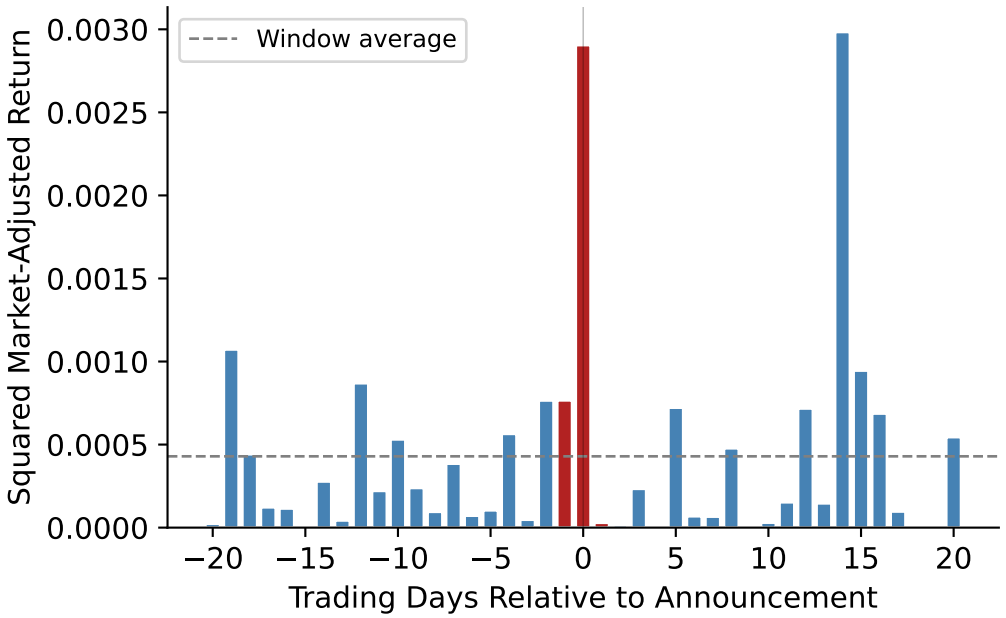


Figure 25.2: Squared market-adjusted returns around a single earnings announcement. Elevated values on days 0 and +1 indicate that the announcement moved the stock price beyond what market-wide movements would predict.

Reading the Plots

A single event is inherently noisy, other firm-specific news (M&A rumors, analyst reports, regulatory actions) can generate volume and volatility spikes on non-announcement dates. The value of the Beaver (1968) approach lies in *averaging across many events* to isolate the systematic announcement effect from idiosyncratic noise.

25.4 Full-Sample Analysis

25.4.1 Computing Event-Time Aggregates

For each event day τ relative to the announcement, we compute the cross-sectional average of our two measures across all announcements:

$$\overline{U^2}_\tau = \frac{1}{N_\tau} \sum_{i=1}^{N_\tau} U_{i,\tau}^2 \tag{25.7}$$

$$\overline{RV}_{\tau} = \frac{1}{N_{\tau}} \sum_{i=1}^{N_{\tau}} RV_{i,\tau} \tag{25.8}$$

where N_{τ} is the number of announcements with available data on event day τ.

```
# Squared and absolute market-adjusted returns
daily_data["sq_ret_mkt"] = daily_data["ret_mkt"] ** 2
daily_data["abs_ret_mkt"] = daily_data["ret_mkt"].abs()

# Relative volume: normalize within each event window
daily_data["rel_vol"] = (
    daily_data
    .groupby(["ticker", "fiscal_year"])["vol"]
    .transform(lambda x: x / x.mean())
)

# Standardized squared return: normalize within each event window
daily_data["std_sq_ret"] = (
    daily_data
    .groupby(["ticker", "fiscal_year"])["sq_ret_mkt"]
    .transform(lambda x: x / x.mean())
)

# Add fiscal year for panel breakdowns
daily_data["year"] = daily_data["fiscal_year"]
```

```
event_summary = (
    daily_data
    .groupby(["relative_td", "year"])
    .agg(
        n_obs=("ret", "size"),
        mean_ret=("ret", "mean"),
        mean_ret_mkt=("ret_mkt", "mean"),
        sd_ret_mkt=("ret_mkt", "std"),
        mean_sq_ret=("sq_ret_mkt", "mean"),
        mean_abs_ret=("abs_ret_mkt", "mean"),
        mean_rel_vol=("rel_vol", "mean"),
        mean_std_sq_ret=("std_sq_ret", "mean"),
        median_rel_vol=("rel_vol", "median"),
    )
    .reset_index()
```

```
)

# Also compute pooled (all-year) summary
event_summary_pooled = (
    daily_data
    .groupby("relative_td")
    .agg(
        n_obs=("ret", "size"),
        mean_ret=("ret", "mean"),
        mean_ret_mkt=("ret_mkt", "mean"),
        sd_ret_mkt=("ret_mkt", "std"),
        mean_sq_ret=("sq_ret_mkt", "mean"),
        mean_abs_ret=("abs_ret_mkt", "mean"),
        mean_rel_vol=("rel_vol", "mean"),
        mean_std_sq_ret=("std_sq_ret", "mean"),
        median_rel_vol=("rel_vol", "median"),
    )
    .reset_index()
)
```

25.4.2 Return Volatility Around Announcements

Figure 25.3 plots the standard deviation of market-adjusted returns by event day, separately for each fiscal year. A clear spike on days 0 and +1 is the hallmark of information content.

```
fig, ax = plt.subplots(figsize=(9, 5))
years_plot = sorted(event_summary["year"].unique())
# Use a colormap
cmap = plt.cm.viridis(np.linspace(0.1, 0.9, len(years_plot)))

for idx, yr in enumerate(years_plot):
    sub = event_summary[event_summary["year"] ==
 ↪  yr].sort_values("relative_td")
    ax.plot(sub["relative_td"], sub["sd_ret_mkt"],
            color=cmap[idx], alpha=0.6, linewidth=1.0, label=str(yr))

# Overlay pooled mean
pooled = event_summary_pooled.sort_values("relative_td")
ax.plot(pooled["relative_td"], pooled["sd_ret_mkt"],
        color="firebrick", linewidth=2.5, linestyle="-", label="Pooled",
        zorder=10)
```

```
ax.axvline(0, color="black", linewidth=0.5, alpha=0.3)
ax.set_xlabel("Trading Days Relative to Announcement")
ax.set_ylabel("Std Dev of Market-Adjusted Returns")
ax.legend(fontsize=7, ncol=4, loc="upper right")
plt.tight_layout()
plt.show()
```

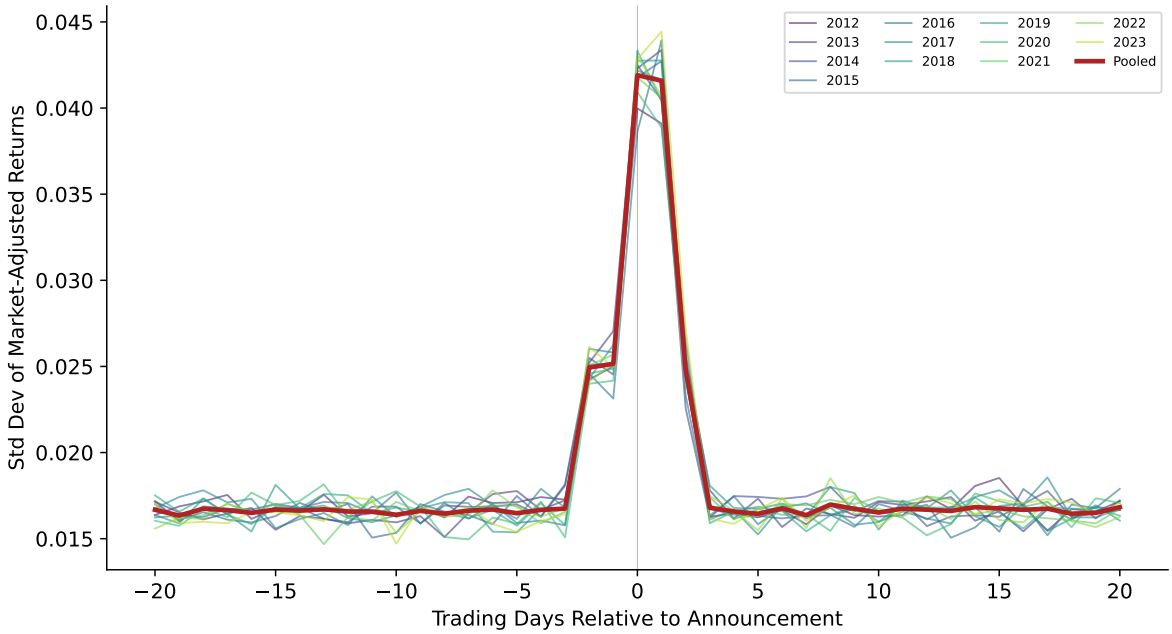


Figure 25.3: Standard deviation of market-adjusted returns around earnings announcements, by fiscal year. The consistent spike on days 0 and +1 across years provides strong evidence that annual earnings announcements convey new information to Vietnamese equity markets.

25.4.3 Trading Volume Around Announcements

Figure 25.4 shows relative trading volume by event day. The volume response provides an independent confirmation of the volatility finding.

```
fig, ax = plt.subplots(figsize=(9, 5))

for idx, yr in enumerate(years_plot):
```

```
    sub = event_summary[event_summary["year"] ==
↪   yr].sort_values("relative_td")
    ax.plot(sub["relative_td"], sub["mean_rel_vol"],
            color=cmap[idx], alpha=0.6, linewidth=1.0, label=str(yr))

ax.plot(pooled["relative_td"], pooled["mean_rel_vol"],
        color="firebrick", linewidth=2.5, linestyle="-", label="Pooled",
        zorder=10)

ax.axhline(1.0, color="grey", linestyle="--", linewidth=0.8)
ax.axvline(0, color="black", linewidth=0.5, alpha=0.3)
ax.set_xlabel("Trading Days Relative to Announcement")
ax.set_ylabel("Relative Volume (1.0 = window average)")
ax.legend(fontsize=7, ncol=4, loc="upper right")
plt.tight_layout()
plt.show()
```

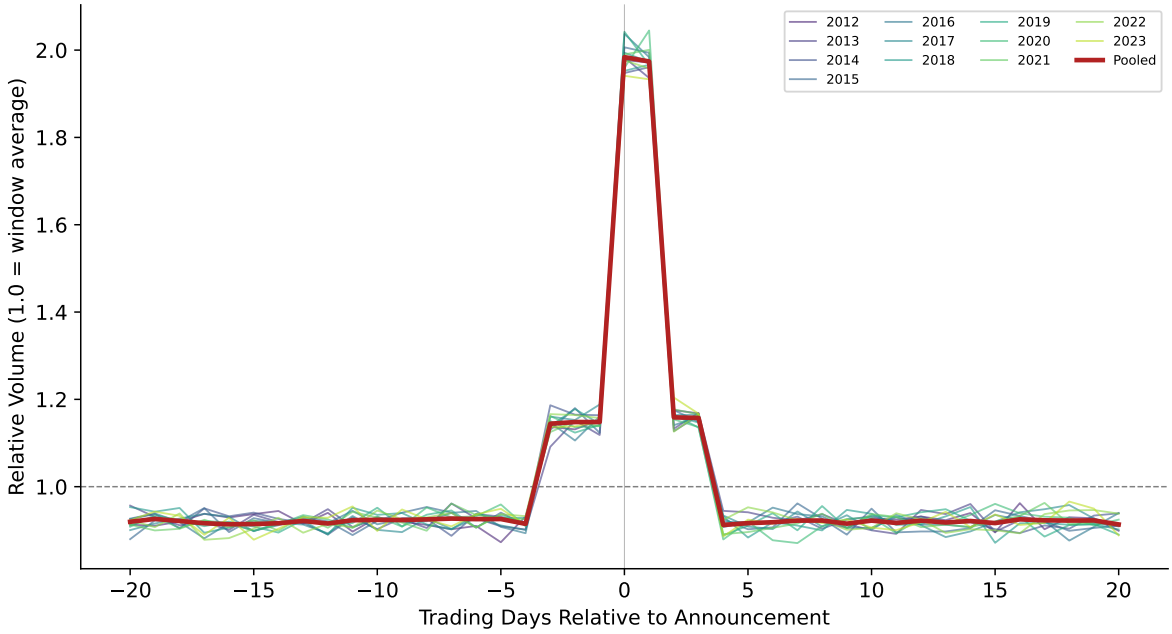


Figure 25.4: Mean relative trading volume around earnings announcements, by fiscal year. Volume spikes sharply on days 0 and +1, consistent with investors actively trading on the new information. The elevated volume persists slightly longer than the volatility spike, suggesting that disagreement about the announcement's implications takes time to resolve.

25.4.4 Formal Statistical Tests

Beaver (1968) relied on visual inspection and informal statistical arguments. Modern practice demands formal tests. We test whether the announcement-window measures differ significantly from the non-announcement window using a paired comparison.

Define the announcement window as days $\{-1, 0, +1\}$ and the non-announcement window as days $\{-20, \ldots, -5\} \cup \{+5, \ldots, +20\}$. For each event i, we compute:

$$\Delta U_i^2 = \overline{U^2}_{i,\text{annc}} - \overline{U^2}_{i,\text{non-annc}} \tag{25.9}$$

Under the null, $E[\Delta U_i^2] = 0$. We test this with a t-test across events.

```
annc_window = [-1, 0, 1]
non_annc_window = list(range(-20, -4)) + list(range(5, 21))

def compute_window_means(df, var, window):
    """Compute mean of var within a window for each event."""
    return (
        df[df["relative_td"].isin(window)]
        .groupby(["ticker", "fiscal_year"])[var]
        .mean()
    )

test_results = {}
for var, label in [("sq_ret_mkt", "Squared Mkt-Adj Return"),
                   ("abs_ret_mkt", "Absolute Mkt-Adj Return"),
                   ("rel_vol", "Relative Volume")]:
    annc_mean = compute_window_means(daily_data, var, annc_window)
    non_annc_mean = compute_window_means(daily_data, var, non_annc_window)

    # Align on common events
    common = annc_mean.index.intersection(non_annc_mean.index)
    diff = annc_mean.loc[common] - non_annc_mean.loc[common]
    diff = diff.dropna()

    t_stat, p_val = stats.ttest_1samp(diff, 0)

    test_results[label] = {
        "Annc Window Mean": annc_mean.mean(),
        "Non-Annc Window Mean": non_annc_mean.mean(),
        "Difference": diff.mean(),
        "t-statistic": t_stat,
```

```
        "p-value": p_val,
        "N events": len(diff),
    }

test_df = pd.DataFrame(test_results).T.round(4)
test_df["N events"] = test_df["N events"].astype(int)
test_df
```

Table 25.2: Formal tests of announcement-period information content. The announcement window is days {-1, 0, +1}; the non-announcement window excludes days {-4, …, +4}. Large t-statistics and small p-values provide strong evidence that both return volatility and trading volume are significantly elevated during the announcement window.

	Annc Window Mean	Non-Annc Window Mean	Difference	t-statistic	p-value
Squared Mkt-Adj Return	0.0014	0.0003	0.0011	64.1353	0.0
Absolute Mkt-Adj Return	0.0294	0.0132	0.0162	79.7318	0.0
Relative Volume	1.7020	0.9206	0.7814	135.9351	0.0

25.4.5 Standardized Volatility Ratio

Following the approach of Beaver (1968), we compute the ratio of announcement-window squared returns to non-announcement-window squared returns. Under the null, this ratio equals 1.0.

```
fig, ax = plt.subplots()
ratio = pooled["mean_std_sq_ret"]
colors = ["firebrick" if abs(t) <= 1 else "steelblue"
          for t in pooled["relative_td"]]
ax.bar(pooled["relative_td"], ratio, color=colors, edgecolor="white")
ax.axhline(1.0, color="grey", linestyle="--", linewidth=1, label="Expected
 ↪  under null")
ax.axvline(0, color="black", linewidth=0.5, alpha=0.3)
ax.set_xlabel("Trading Days Relative to Announcement")
ax.set_ylabel("Standardized Squared Return (ratio)")
ax.legend()
plt.tight_layout()
plt.show()
```

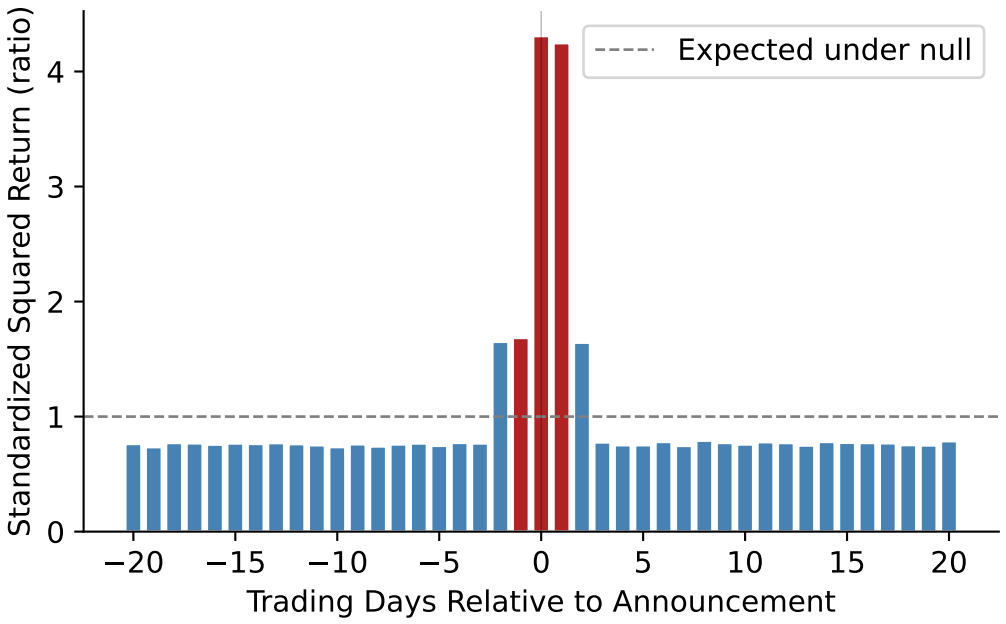


Figure 25.5: Standardized squared market-adjusted return by event day (pooled across all years). A value of 1.0 indicates normal volatility. The announcement-day ratio substantially exceeds 1.0, indicating that earnings releases generate return variation well above normal levels.

25.5 Cross-Sectional Variation

A uniform announcement effect across all firms would be surprising. Theory predicts that the magnitude of the market reaction depends on the pre-announcement information environment: firms with richer information sets prior to the announcement should exhibit *smaller* reactions because less of the earnings number is news (Atiase 1985; Verrecchia 2001).

25.5.1 By Firm Size

Larger firms receive more attention from analysts, media, and institutional investors, so their earnings announcements should contain less incremental information.

```
# Assign size terciles
daily_data["size_tercile"] = pd.qcut(
    daily_data["log_mcap"], 3, labels=["Small", "Medium", "Large"]
)
```

```
size_summary = (
    daily_data
    .groupby(["relative_td", "size_tercile"])
    .agg(
        mean_std_sq_ret=("std_sq_ret", "mean"),
        mean_rel_vol=("rel_vol", "mean"),
    )
    .reset_index()
)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
size_colors = {"Small": "#E91E63", "Medium": "#FF9800", "Large": "#2196F3"}

for label, color in size_colors.items():
    sub = size_summary[size_summary["size_tercile"] ==
 ↪  label].sort_values("relative_td")
    axes[0].plot(sub["relative_td"], sub["mean_std_sq_ret"],
                 label=label, color=color, linewidth=1.5)
    axes[1].plot(sub["relative_td"], sub["mean_rel_vol"],
                 label=label, color=color, linewidth=1.5)

for ax in axes:
    ax.axvline(0, color="black", linewidth=0.5, alpha=0.3)
    ax.set_xlabel("Trading Days Relative to Announcement")
    ax.legend()

axes[0].axhline(1.0, color="grey", linestyle="--", linewidth=0.7)
axes[0].set_ylabel("Standardized Squared Return")
axes[0].set_title("Volatility Reaction by Size")

axes[1].axhline(1.0, color="grey", linestyle="--", linewidth=0.7)
axes[1].set_ylabel("Relative Volume")
axes[1].set_title("Volume Reaction by Size")

plt.tight_layout()
plt.show()
```

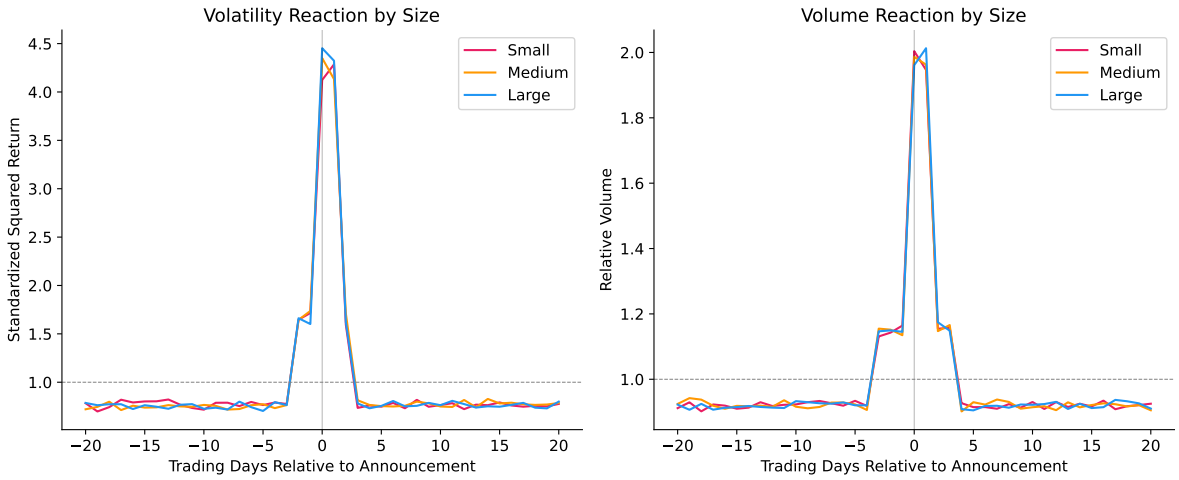


Figure 25.6: Announcement-day abnormal volatility and volume by firm size tercile. Smaller firms exhibit substantially larger reactions, consistent with the prediction that pre-disclosure information is increasing in firm size.

25.5.2 By Exchange (HOSE vs. HNX)

HOSE-listed firms are generally larger and more liquid than HNX-listed firms. Additionally, the tighter price limits on HOSE ($\pm$ 7% vs. $\pm$ 10% on HNX) may truncate announcement-day returns, compressing measured volatility and potentially spreading the adjustment over multiple days.

```
exch_summary = (
    daily_data
    .groupby(["relative_td", "exchange"])
    .agg(
        mean_std_sq_ret=("std_sq_ret", "mean"),
        mean_rel_vol=("rel_vol", "mean"),
    )
    .reset_index()
)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
exch_colors = {"HOSE": "#1565C0", "HNX": "#E65100"}

for exch, color in exch_colors.items():
    sub = exch_summary[exch_summary["exchange"] ==
 ↪  exch].sort_values("relative_td")
```

```
    axes[0].plot(sub["relative_td"], sub["mean_std_sq_ret"],
                 label=exch, color=color, linewidth=1.5)
    axes[1].plot(sub["relative_td"], sub["mean_rel_vol"],
                 label=exch, color=color, linewidth=1.5)

for ax in axes:
    ax.axvline(0, color="black", linewidth=0.5, alpha=0.3)
    ax.legend()
    ax.set_xlabel("Trading Days Relative to Announcement")

axes[0].axhline(1.0, color="grey", linestyle="--", linewidth=0.7)
axes[0].set_ylabel("Standardized Squared Return")
axes[0].set_title("Volatility by Exchange")

axes[1].axhline(1.0, color="grey", linestyle="--", linewidth=0.7)
axes[1].set_ylabel("Relative Volume")
axes[1].set_title("Volume by Exchange")

plt.tight_layout()
plt.show()
```

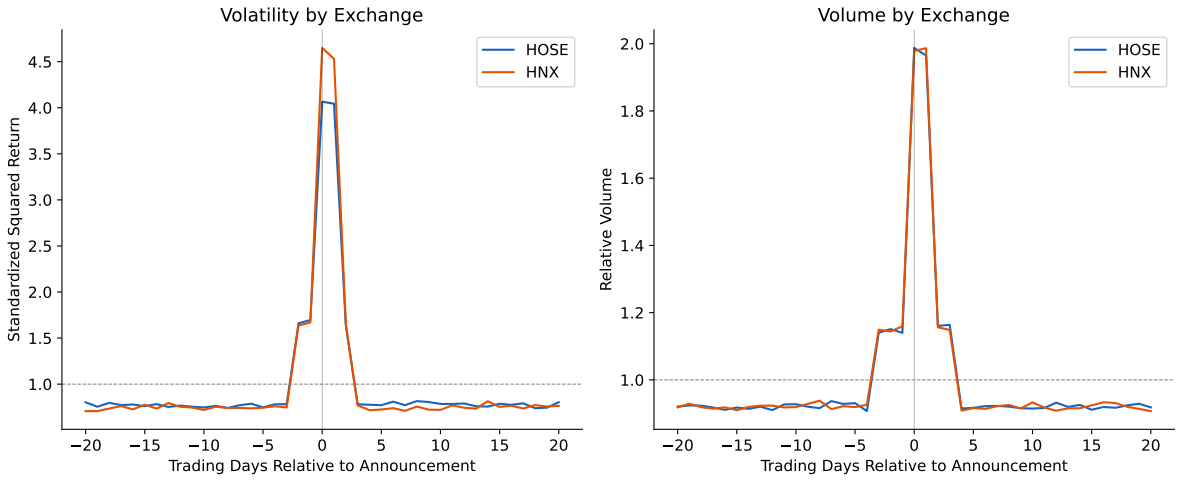


Figure 25.7: Announcement reactions by exchange. HNX firms (generally smaller, less liquid) show larger volatility spikes. The tighter HOSE price limit may compress announcement-day returns, but the wider HNX limit allows larger single-day adjustments.

25.5.3 Regression Analysis

We formalize the cross-sectional analysis with an event-level regression:

$$\Delta U_i^2 = \beta_0 + \beta_1 \text{Size}_i + \beta_2 \text{HNX}_i + \beta_3 \text{AnnLag}_i + \varepsilon_i \tag{25.10}$$

where Size_i is log market capitalization, HNX_i is an indicator for HNX listing, and AnnLag_i is the number of calendar days between fiscal year-end and the announcement date (longer lags may allow more information leakage, reducing the surprise).

```
# Build event-level dataset
annc_data = daily_data[daily_data["relative_td"].isin(annc_window)].copy()
non_annc_data =
 ↪  daily_data[daily_data["relative_td"].isin(non_annc_window)].copy()

annc_means = (
    annc_data
    .groupby(["ticker", "fiscal_year"])
    .agg(annc_sq_ret=("sq_ret_mkt", "mean"), annc_vol=("rel_vol", "mean"))
    .reset_index()
)
non_annc_means = (
    non_annc_data
    .groupby(["ticker", "fiscal_year"])
    .agg(non_annc_sq_ret=("sq_ret_mkt", "mean"), non_annc_vol=("rel_vol",
 ↪  "mean"))
    .reset_index()
)

event_df = annc_means.merge(non_annc_means, on=["ticker", "fiscal_year"])
event_df["delta_sq_ret"] = event_df["annc_sq_ret"] -
 ↪  event_df["non_annc_sq_ret"]
event_df["delta_vol"] = event_df["annc_vol"] - event_df["non_annc_vol"]

# Add firm characteristics
firm_chars = (
    earn_annc[["ticker", "fiscal_year", "exchange", "log_mcap", "datadate",
 ↪  "annc_date"]]
    .drop_duplicates(["ticker", "fiscal_year"])
)
firm_chars["hnx"] = (firm_chars["exchange"] == "HNX").astype(int)
firm_chars["annc_lag"] = (
```

```
    pd.to_datetime(firm_chars["annc_date"]) -
 ↪  pd.to_datetime(firm_chars["datadate"])
).dt.days

event_df = event_df.merge(firm_chars, on=["ticker", "fiscal_year"],
 ↪  how="left")
event_df = event_df.dropna(subset=["log_mcap", "hnx", "annc_lag",
 ↪  "delta_sq_ret"])

# Regression
mod_vol = smf.ols("delta_sq_ret ~ log_mcap + hnx + annc_lag",
 ↪  data=event_df).fit()
mod_volume = smf.ols("delta_vol ~ log_mcap + hnx + annc_lag",
 ↪  data=event_df).fit()

reg_table = pd.DataFrame({
    "Volatility": {
        "Intercept": f"{mod_vol.params['Intercept']:.6f}
         ↪  ({mod_vol.bse['Intercept']:.6f})",
        "Size (log mcap)": f"{mod_vol.params['log_mcap']:.6f}
         ↪  ({mod_vol.bse['log_mcap']:.6f})",
        "HNX": f"{mod_vol.params['hnx']:.6f} ({mod_vol.bse['hnx']:.6f})",
        "Annc Lag (days)": f"{mod_vol.params['annc_lag']:.6f}
         ↪  ({mod_vol.bse['annc_lag']:.6f})",
        "N": f"{int(mod_vol.nobs):,}",
        "R²": f"{mod_vol.rsquared:.4f}",
    },
    "Volume": {
        "Intercept": f"{mod_volume.params['Intercept']:.4f}
         ↪  ({mod_volume.bse['Intercept']:.4f})",
        "Size (log mcap)": f"{mod_volume.params['log_mcap']:.4f}
         ↪  ({mod_volume.bse['log_mcap']:.4f})",
        "HNX": f"{mod_volume.params['hnx']:.4f}
         ↪  ({mod_volume.bse['hnx']:.4f})",
        "Annc Lag (days)": f"{mod_volume.params['annc_lag']:.6f}
         ↪  ({mod_volume.bse['annc_lag']:.6f})",
        "N": f"{int(mod_volume.nobs):,}",
        "R²": f"{mod_volume.rsquared:.4f}",
    },
})
reg_table
```

Table 25.3: Cross-sectional determinants of announcement-day return volatility. Negative size coefficient confirms that larger firms have less informative announcements. The announcement lag coefficient captures information leakage before the official release.

	Volatility	Volume
Intercept	0.001040 (0.000116)	0.7676 (0.0396)
Size (log mcap)	-0.000039 (0.000011)	0.0017 (0.0038)
HNX	0.000266 (0.000034)	0.0118 (0.0117)
Annc Lag (days)	0.000003 (0.000001)	-0.000037 (0.000444)
N	3,600	3,600
R^2	0.0210	0.0003

25.6 Discussion and Contemporary Perspectives

25.6.1 The Bamber et al. (2000) Critique

Bamber, Christensen, and Gaver (2000) raised two important methodological concerns about the original Beaver (1968) findings:

1. **Mean vs. median effects.** The original study focused on mean volume and volatility, which can be driven by a small number of extreme events. When Bamber, Christensen, and Gaver (2000) examined the *proportion* of individual announcements that generate unusual reactions (rather than the mean reaction), the evidence was far less dramatic.
2. **Sample selection.** Beaver's original sample of 143 firms was not randomly drawn but reflected specific selection criteria that may have biased toward firms with more informative announcements.

We can assess the first concern directly. Figure 25.8 plots the *fraction* of announcements with above-median volatility or volume on each event day, following the spirit of Bamber et al.'s critique.

```
# For each event, determine if announcement-day measures exceed
# that event's median non-announcement-day measures
event_medians = (
    daily_data[daily_data["relative_td"].isin(non_annc_window)]
    .groupby(["ticker", "fiscal_year"])
    .agg(median_sq_ret=("sq_ret_mkt", "median"), median_vol=("rel_vol",
 ↪  "median"))
    .reset_index()
```

```
)

daily_with_median = daily_data.merge(
    event_medians, on=["ticker", "fiscal_year"], how="left"
)
daily_with_median["above_med_vol"] = (
    daily_with_median["sq_ret_mkt"] > daily_with_median["median_sq_ret"]
).astype(int)
daily_with_median["above_med_volume"] = (
    daily_with_median["rel_vol"] > daily_with_median["median_vol"]
).astype(int)

prop_summary = (
    daily_with_median
    .groupby("relative_td")
    .agg(
        prop_above_vol=("above_med_vol", "mean"),
        prop_above_volume=("above_med_volume", "mean"),
    )
    .reset_index()
    .sort_values("relative_td")
)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].bar(prop_summary["relative_td"], prop_summary["prop_above_vol"],
            color="steelblue", edgecolor="white")
axes[0].axhline(0.5, color="grey", linestyle="--", linewidth=1)
axes[0].axvline(0, color="black", linewidth=0.5, alpha=0.3)
axes[0].set_xlabel("Trading Days Relative to Announcement")
axes[0].set_ylabel("Proportion > Median")
axes[0].set_title("Proportion with Above-Median Volatility")
axes[0].yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1))

axes[1].bar(prop_summary["relative_td"], prop_summary["prop_above_volume"],
            color="darkorange", edgecolor="white")
axes[1].axhline(0.5, color="grey", linestyle="--", linewidth=1)
axes[1].axvline(0, color="black", linewidth=0.5, alpha=0.3)
axes[1].set_xlabel("Trading Days Relative to Announcement")
axes[1].set_ylabel("Proportion > Median")
axes[1].set_title("Proportion with Above-Median Volume")
axes[1].yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1))
```

```
plt.tight_layout()
plt.show()
```

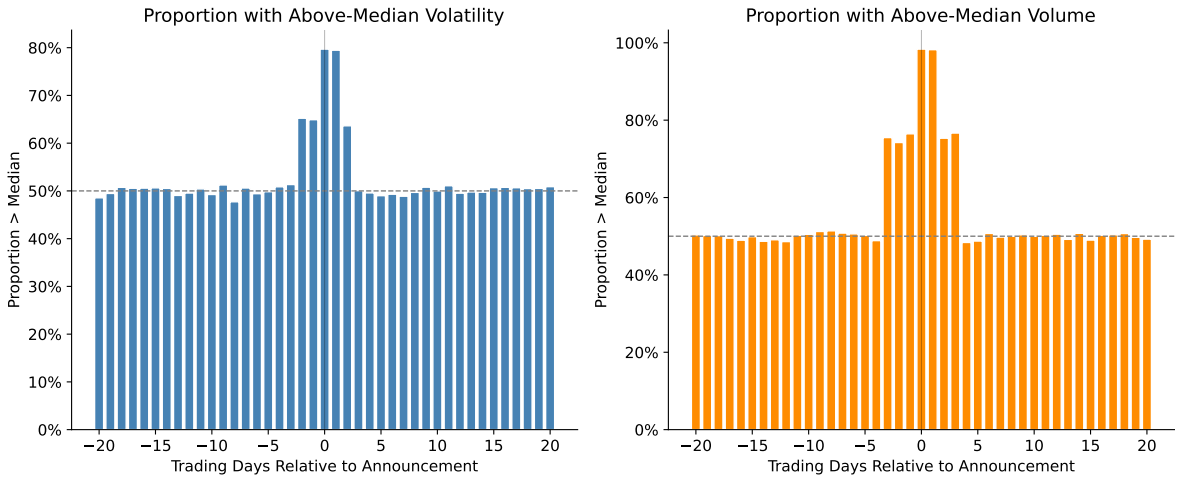


Figure 25.8: Proportion of individual announcements with above-median reaction on each event day. Unlike mean-based measures, this approach is robust to outliers. The fraction still rises notably on days 0 and +1, though the magnitude is more modest than in the mean-based plots.

25.6.2 How Much Information Do Earnings Announcements Convey?

R. Ball and Shivakumar (2008) pose a provocative question: even if earnings announcements generate detectable market reactions, how much of the *total* annual information flow do they account for? Their analysis suggests that quarterly earnings announcements explain only about 12–18% of annual return variation for U.S. firms, implying that the vast majority of value-relevant information reaches the market through other channels.

We can compute an analogous statistic for our Vietnamese sample:

```
# Compute fraction of event-window variance in announcement days
variance_shares = []
for (ticker, fy), group in daily_data.groupby(["ticker", "fiscal_year"]):
    total_var = group["ret_mkt"].var()
    annc_var = group[group["relative_td"].isin(annc_window)]["ret_mkt"].var()

    if total_var > 0 and not np.isnan(annc_var):
        variance_shares.append({
            "ticker": ticker,
```

```
            "fiscal_year": fy,
            "annc_var_share": annc_var / total_var,
        })

var_share_df = pd.DataFrame(variance_shares)
expected_share = len(annc_window) / 41  # 3 days out of 41

share_stats = pd.DataFrame({
    "Statistic": ["Mean share", "Median share", "Expected under null",
                  "N events", "% above expected"],
    "Value": [
        f"{var_share_df['annc_var_share'].mean():.3f}",
        f"{var_share_df['annc_var_share'].median():.3f}",
        f"{expected_share:.3f}",
        f"{len(var_share_df):,}",
        f"{(var_share_df['annc_var_share'] > expected_share).mean():.1%}",
    ],
}).set_index("Statistic")
share_stats
```

Table 25.4: Share of annual return variation concentrated in the announcement window. The announcement window is days {-1, 0, +1}. If announcements carried no information, the expected share would be 3/41 7.3% (three days out of a 41-day window). Values substantially above this indicate that a disproportionate share of return variation is concentrated around announcements.

Statistic	Value
Mean share	3.426
Median share	2.979
Expected under null	0.073
N events	3,600
% above expected	98.5%

25.6.3 Price Limits and Multi-Day Adjustment

A distinctive feature of Vietnamese markets is that daily price limits can prevent full single-day price adjustment. When an announcement is sufficiently surprising, the stock may hit the $\pm$ 7% limit on HOSE (or $\pm$ 10% on HNX), and the remaining adjustment spills into subsequent trading days. This creates a pattern of **limit hits followed by continued drift**,

a phenomenon that would bias downward any single-day measure of announcement impact and redistribute information content across days 0, +1, and potentially +2.

```
# Identify limit hits
def is_limit_hit(row):
    limit = 0.07 if row["exchange"] == "HOSE" else 0.10
    return abs(row["ret"]) >= (limit - 0.005)

daily_data["limit_hit"] = daily_data.apply(is_limit_hit, axis=1).astype(int)

limit_summary = (
    daily_data
    .groupby("relative_td")["limit_hit"]
    .mean()
    .reset_index()
    .sort_values("relative_td")
)

fig, ax = plt.subplots()
colors = ["firebrick" if abs(t) <= 1 else "steelblue"
          for t in limit_summary["relative_td"]]
ax.bar(limit_summary["relative_td"], limit_summary["limit_hit"],
       color=colors, edgecolor="white")
ax.set_xlabel("Trading Days Relative to Announcement")
ax.set_ylabel("Proportion Hitting Price Limit")
ax.yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1, decimals=1))
ax.axvline(0, color="black", linewidth=0.5, alpha=0.3)
plt.tight_layout()
plt.show()
```

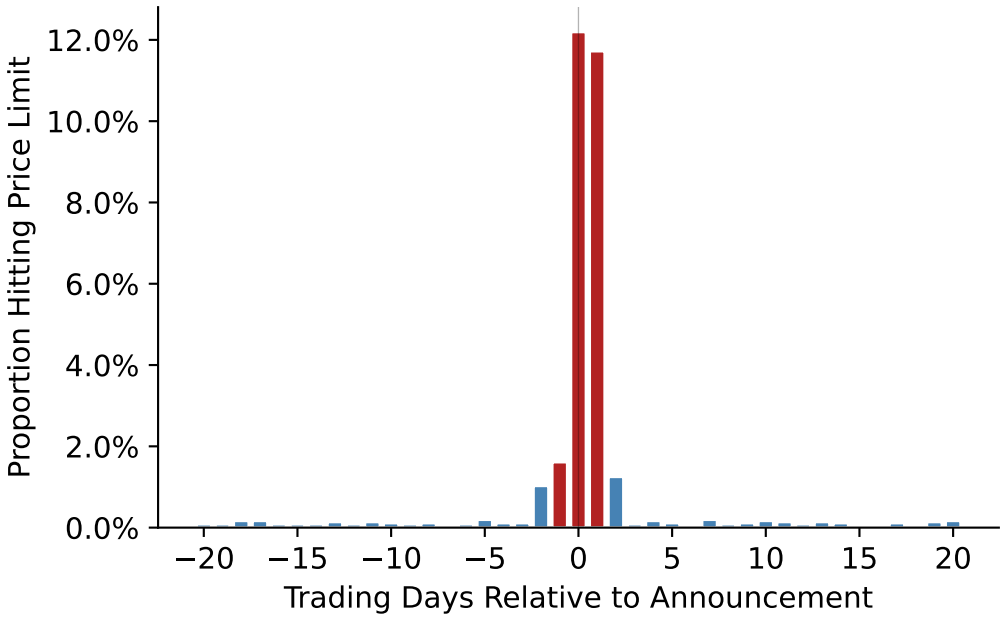


Figure 25.9: Proportion of stocks hitting the daily price limit (within 0.5% of the ±7% HOSE limit or ±10% HNX limit) by event day. Limit hits are concentrated on the announcement day, indicating that some announcements are sufficiently surprising to exhaust single-day adjustment capacity.

25.6.4 International Context

Landsman, Maydew, and Thornock (2012) conduct a cross-country study of announcement reactions and find that information content is higher in countries with: (i) stronger investor protection, (ii) greater financial transparency, and (iii) mandatory IFRS adoption. Vietnam's position on these dimensions, developing institutional framework, VAS rather than IFRS, moderate transparency, suggests that announcement reactions may differ systematically from developed-market benchmarks.

Their framework generates a testable prediction: as Vietnam progresses toward IFRS adoption (a process underway as of 2025), the information content of earnings announcements should *increase* to the extent that IFRS improves the mapping from economic events to reported numbers.

25.7 Summary

This chapter examined whether earnings announcements convey new information to investors in Vietnamese equity markets, following the foundational research design of Beaver (1968). The key findings are:

- Both return volatility and trading volume spike sharply on the announcement day and the following day, providing robust evidence of information content. This result holds across fiscal years, consistent with earnings being a persistent source of new information.
- Formal statistical tests reject the null of no information content with high confidence, using both mean-based and proportion-based measures.
- Cross-sectional analysis reveals that the announcement reaction is stronger for smaller firms and HNX-listed firms, consistent with the theoretical prediction that the information content of earnings is inversely related to the richness of the pre-disclosure information environment.
- Vietnamese-specific features—price limits, retail dominance, and sparse analyst coverage—create distinctive empirical patterns that enrich the standard Beaver (1968) framework. Price limits, in particular, generate multi-day adjustment dynamics that researchers must account for when measuring single-day information content.
- The Bamber, Christensen, and Gaver (2000) critique—that mean effects can be driven by outliers—is addressed by examining the proportion of events with above-median reactions, which still shows clear announcement-day elevation.

The evidence strongly supports the conclusion that annual earnings announcements convey new information to the Vietnamese market, consistent with the original Beaver (1968) findings and the broader international evidence in Landsman, Maydew, and Thornock (2012).

26 Accruals, Earnings Persistence, and Market Efficiency

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
from itertools import product
import warnings
warnings.filterwarnings("ignore")

plt.rcParams.update({
    "figure.dpi": 150,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.size": 11,
})
```

Accrual accounting is the foundation of modern financial reporting. Under accrual principles, revenues are recognized when earned and expenses when incurred, regardless of when cash changes hands. This timing wedge between economic events and cash realization generates **accruals** (i.e., the non-cash component of reported earnings). Formally, we define total accruals as:

$$\text{Accruals}_t = \text{Earnings}_t - \text{Cash Flow from Operations}_t \tag{26.1}$$

The distinction between accruals and cash flows matters because these two components of earnings exhibit markedly different statistical properties. Cash flows from operations tend to be more persistent and harder for managers to manipulate, while accruals are inherently more transient and subject to managerial discretion through estimates, assumptions, and timing choices (P. Dechow, Ge, and Schrand 2010).

Sloan (1996) demonstrated two important empirical regularities in U.S. data:

1. The accrual component of earnings is *less persistent* than the cash flow component in predicting future earnings, and
2. Equity prices behave as if investors fail to distinguish between these components, treating accruals as having the same persistence as cash flows. This second finding implies a predictable pattern in stock returns (i.e., firms with high accruals earn lower subsequent returns, and vice versa—known as the **accrual anomaly**).

26.0.1 Why Vietnam?

Vietnam's equity markets provide a compelling laboratory for studying accrual dynamics for several reasons:

1. **Accounting standards in transition.** Vietnam operates under Vietnamese Accounting Standards (VAS), which are broadly based on older International Accounting Standards but have not yet fully converged with IFRS. VAS retains certain rules-based provisions that may generate different accrual patterns compared to the principles-based frameworks in developed markets.
2. **State-owned enterprise (SOE) influence.** A significant fraction of listed firms on HOSE and HNX are former SOEs or remain partially state-owned. These firms may face different incentives regarding accrual management (political rather than purely economic), introducing heterogeneity absent in the original U.S. studies.
3. **Market maturity and investor sophistication.** Vietnam's stock market, with HOSE established in 2000 and HNX in 2005, is still developing. Retail investors dominate trading volume, and institutional infrastructure (analyst coverage, short-selling constraints, information intermediaries) is less developed than in the U.S. This environment may amplify or attenuate the mispricing patterns documented in developed markets.
4. **Daily price limits.** Both HOSE ($\pm$ 7%) and HNX ($\pm$ 10%) impose daily price limits, which can impede price discovery and create path dependencies in return measurement that have no analogue in the U.S. context.
5. **Emerging market growth dynamics.** High-growth economies generate different accrual profiles—rapid revenue growth mechanically produces large working capital accruals even absent any managerial manipulation (Fairfield, Whisenant, and Yohn 2003).

26.1 Measuring Accruals

26.1.1 The Balance Sheet Approach

The traditional approach to measuring accruals, used in much of the early literature, constructs total accruals from successive balance sheet snapshots. Define the following balance sheet items for firm i in period t (Table 26.1).

Table 26.1: Key balance sheet items for accrual measurement

Symbol	Description	VAS Equivalent
$\Delta CA_{i,t}$	Change in current assets	Thay đổi tài sản ngắn hạn
$\Delta Cash_{i,t}$	Change in cash and equivalents	Thay đổi tiền và tương đương tiền
$\Delta CL_{i,t}$	Change in current liabilities	Thay đổi nợ ngắn hạn
$\Delta STD_{i,t}$	Change in short-term debt	Thay đổi vay ngắn hạn
$\Delta TP_{i,t}$	Change in taxes payable	Thay đổi thuế phải nộp
$Dep_{i,t}$	Depreciation and amortization	Khấu hao TSCĐ

Total accruals under the balance sheet approach are:

$$ACC^{BS}_{i,t} = (\Delta CA_{i,t} - \Delta Cash_{i,t}) - (\Delta CL_{i,t} - \Delta STD_{i,t} - \Delta TP_{i,t}) - Dep_{i,t} \tag{26.2}$$

The intuition behind Equation 26.2 is straightforward. Changes in non-cash current assets (such as accounts receivable and inventory) represent revenues recognized or expenses deferred without corresponding cash flows. Changes in operating current liabilities (excluding debt) capture expenses recognized without cash outflows. Short-term debt and taxes payable are excluded because they relate to financing and tax timing rather than operating accruals. Depreciation is subtracted as a non-cash charge against earnings.

All variables are typically scaled by average total assets to control for firm size:

$$acc_{i,t} = \frac{ACC^{BS}_{i,t}}{(\text{Assets}_{i,t} + \text{Assets}_{i,t-1})/2} \tag{26.3}$$

26.1.2 The Cash Flow Statement Approach

Hribar and Collins (2002) demonstrated that the balance sheet approach introduces measurement error because non-operating events, such as mergers, acquisitions, divestitures, and

foreign currency translations, affect current asset and liability balances without corresponding earnings impacts. They advocate a simpler and more accurate approach:

$$ACC_{i,t}^{CF} = \text{Earnings}_{i,t} - \text{CFO}_{i,t} \tag{26.4}$$

where $\text{CFO}_{i,t}$ is cash flow from operations reported directly on the cash flow statement.

26.1.3 Vietnamese Context

For Vietnamese listed firms, the cash flow statement approach is generally preferable, but practitioners should be aware of several data considerations:

- **Circular 200/2014/TT-BTC** standardizes the chart of accounts and financial statement templates. Cash flow from operations is reported using the *indirect method* by most firms (starting from net income and adjusting for non-cash items), though some use the direct method.
- **VAS 24** (Cash Flow Statements) governs disclosure. Unlike IFRS, VAS treatment of interest paid and dividends received in the operating section is less flexible, potentially affecting comparability.

For the remainder of this chapter, we use the balance sheet approach, where demonstrated on simulated data (for pedagogical transparency) and the cash flow statement approach on actual firm data where available.

26.2 Simulation Analysis

Before turning to Vietnamese market data, we construct a simulation to build economic intuition for *why* accrual persistence differs from cash flow persistence, and what role estimation error in accrual accounting plays in generating this difference.

26.2.1 The Model

Consider a simplified firm that:

- Sells goods on credit at a constant gross margin μ
- Follows an AR(1) sales process: $S_t = \bar{S} + \rho(S_{t-1} - \bar{S}) + \varepsilon_t$, where $\varepsilon_t \sim N(0, \sigma_S^2)$
- Collects receivables in the following period, with a true default rate δ
- Records an allowance for doubtful debts as a fraction α of current sales
- Pays dividends equal to 100% of net income

The key parameter of interest is α, the managerial estimate of bad debts. When $\alpha = \delta$ (the true default rate), the accounting system is unbiased. When $\alpha \neq \delta$, accruals contain estimation error that affects persistence.

```
def simulate_firm(
    alpha: float = 0.03,
    n_years: int = 20,
    delta: float = 0.03,
    gross_margin: float = 0.80,
    mean_sales: float = 1000.0,
    sd_sales: float = 100.0,
    rho: float = 0.9,
    beg_cash: float = 1500.0,
    rng: np.random.Generator = None,
) -> pd.DataFrame:
    """
    Simulate financial statements for a single firm.

    Parameters
    ----------
    alpha : float
        Managerial estimate of doubtful debt rate (allowance parameter).
    n_years : int
        Number of years to simulate.
    delta : float
        True economic default rate on receivables.
    gross_margin : float
        Gross margin on sales.
    mean_sales : float
        Long-run mean of the AR(1) sales process.
    sd_sales : float
        Standard deviation of the sales innovation.
    rho : float
        AR(1) persistence parameter for sales.
    beg_cash : float
        Beginning cash balance.
    rng : numpy Generator
        Random number generator for reproducibility.

    Returns
    -------
    pd.DataFrame
        Simulated financial statement data.
```

```
    """
    if rng is None:
        rng = np.random.default_rng(42)

    # Generate AR(1) sales
    errors = rng.normal(0, sd_sales, n_years)
    sales = np.zeros(n_years)
    sales[0] = mean_sales + errors[0]
    for t in range(1, n_years):
        sales[t] = mean_sales + rho * (sales[t - 1] - mean_sales) + errors[t]

    # Allocate arrays
    cogs = (1 - gross_margin) * sales
    ar = sales.copy()                       # all sales on credit
    allowance = alpha * sales               # allowance for doubtful debts

    writeoffs = np.zeros(n_years)
    collections = np.zeros(n_years)
    bad_debt_exp = np.zeros(n_years)
    net_income = np.zeros(n_years)
    dividends = np.zeros(n_years)
    cash = np.zeros(n_years)
    equity = np.zeros(n_years)

    # Year 1
    bad_debt_exp[0] = allowance[0]
    net_income[0] = sales[0] - cogs[0] - bad_debt_exp[0]
    dividends[0] = net_income[0]
    cash[0] = beg_cash + collections[0] - cogs[0] - dividends[0]
    equity[0] = beg_cash + net_income[0] - dividends[0]

    # Years 2 through n_years
    for t in range(1, n_years):
        writeoffs[t] = delta * ar[t - 1]
        collections[t] = (1 - delta) * ar[t - 1]
        bad_debt_exp[t] = allowance[t] - allowance[t - 1] + writeoffs[t]
        net_income[t] = sales[t] - cogs[t] - bad_debt_exp[t]
        dividends[t] = net_income[t]
        cash[t] = cash[t - 1] + collections[t] - cogs[t] - dividends[t]
        equity[t] = equity[t - 1] + net_income[t] - dividends[t]

    return pd.DataFrame({
        "year": np.arange(1, n_years + 1),
```

```
        "alpha": alpha,
        "sales": sales,
        "cogs": cogs,
        "ar": ar,
        "allowance": allowance,
        "writeoffs": writeoffs,
        "collections": collections,
        "bad_debt_exp": bad_debt_exp,
        "net_income": net_income,
        "dividends": dividends,
        "cash": cash,
        "equity": equity,
    })
```

26.2.2 Generating a Single Firm

We first generate 1,000 years of data for a single firm with $\alpha = \delta = 0.03$ (unbiased accounting) to visualize the sales process.

```
rng = np.random.default_rng(2024)
df_long = simulate_firm(alpha=0.03, n_years=1000, rng=rng)

fig, ax = plt.subplots()
subset = df_long.query("year <= 20")
ax.plot(subset["year"], subset["sales"], color="firebrick", linewidth=1.5)
ax.axhline(
    df_long["sales"].mean(), color="steelblue",
    linestyle="--", linewidth=1, label="Long-run mean"
)
ax.set_xlabel("Year")
ax.set_ylabel("Sales (millions VND, simulated)")
ax.legend()
plt.tight_layout()
plt.show()
```

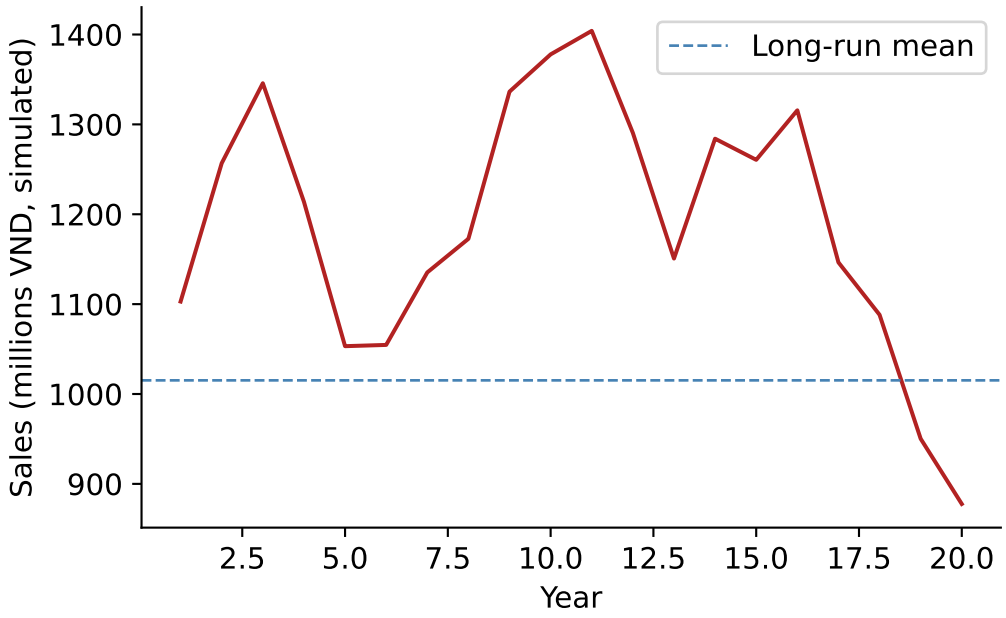


Figure 26.1: Simulated sales for a single firm over 20 years. The blue dashed line shows mean sales. The AR(1) structure creates smooth, persistent fluctuations.

26.2.3 Cross-Sectional Simulation: Persistence and Estimation Error

The central economic insight is that when the managerial estimate α departs from the true rate δ, accruals contain a systematic estimation error that *reduces* the persistence of earnings. To demonstrate this, we simulate 5,000 firms, each with a randomly drawn $\alpha \in [0.01, 0.05]$ while keeping the true default rate fixed at $\delta = 0.03$.

For each firm, we estimate earnings persistence as the slope coefficient $\hat{\beta}_1$ from an AR(1) regression of net income:

$$NI_{i,t} = \beta_0 + \beta_1 NI_{i,t-1} + u_{i,t} \tag{26.5}$$

```
def estimate_persistence(df: pd.DataFrame) -> dict:
    """Estimate AR(1) persistence of net income for one firm."""
    temp = df[["year", "net_income"]].copy()
    temp["lag_ni"] = temp["net_income"].shift(1)
    temp = temp.dropna()
    if len(temp) < 10:
        return {"alpha": df["alpha"].iloc[0], "persistence": np.nan}
```

```
    X = sm.add_constant(temp["lag_ni"])
    model = sm.OLS(temp["net_income"], X).fit()
    return {"alpha": df["alpha"].iloc[0], "persistence":
     ↪  model.params["lag_ni"]}

n_firms = 5000
rng_main = np.random.default_rng(2024)
alphas = rng_main.uniform(0.01, 0.05, n_firms)

results = []
for i, a in enumerate(alphas):
    firm_rng = np.random.default_rng(2024 + i)
    firm_df = simulate_firm(alpha=a, n_years=200, rng=firm_rng)
    results.append(estimate_persistence(firm_df))

sim_results = pd.DataFrame(results)
```

```
fig, ax = plt.subplots()
ax.scatter(
    sim_results["alpha"], sim_results["persistence"],
    alpha=0.15, s=8, color="steelblue", edgecolors="none"
)
ax.set_xlabel(r"Allowance parameter ($\alpha$)")
ax.set_ylabel("Estimated earnings persistence")
ax.axvline(0.03, color="firebrick", linestyle="--", linewidth=1,
 ↪  label=r"$\delta = 0.03$ (true rate)")
ax.legend()
plt.tight_layout()
plt.show()
```

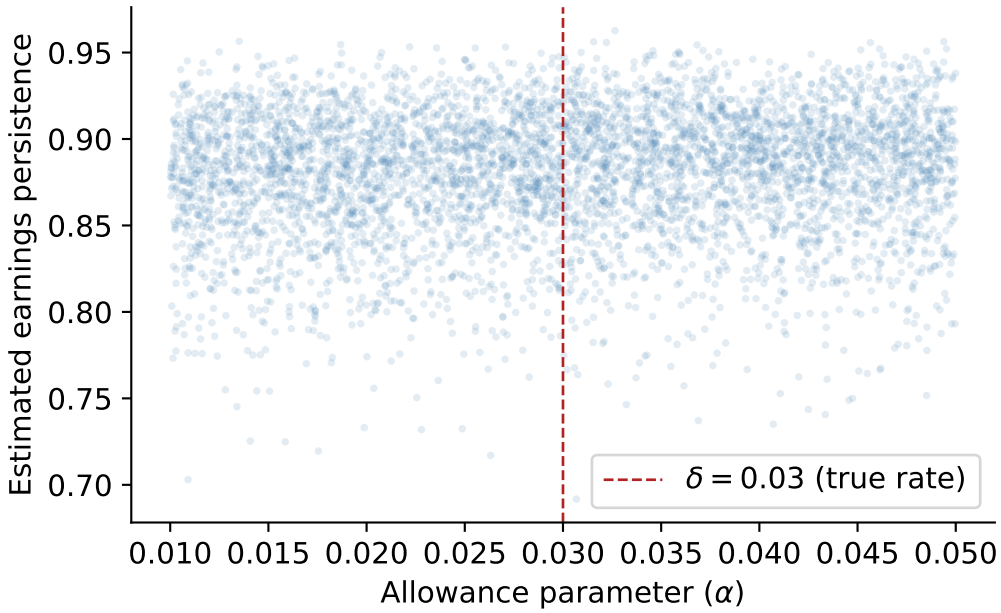


Figure 26.2: Earnings persistence vs. allowance parameter (α). Persistence peaks near $\alpha = 0.03$, the true default rate, and declines symmetrically as the accounting estimate deviates from economic reality in either direction.

26.2.4 Decomposing Persistence by Earnings Component

The simulation allows us to verify the core theoretical prediction: cash flow persistence exceeds accrual persistence. We compute operating cash flows and accruals for each simulated firm and estimate component-specific persistence.

```
def component_persistence(df: pd.DataFrame) -> dict:
    """Estimate persistence of NI, CFO, and accruals."""
    temp = df.copy()
    # CFO = collections - COGS (direct method, simplified)
    temp["cfo"] = temp["collections"] - temp["cogs"]
    temp["acc"] = temp["net_income"] - temp["cfo"]

    out = {"alpha": temp["alpha"].iloc[0]}
    for var in ["net_income", "cfo", "acc"]:
        t = temp[["year", var]].copy()
        t["lag"] = t[var].shift(1)
        t = t.dropna()
        if len(t) < 10:
```

```
            out[f"persist_{var}"] = np.nan
            continue
        X = sm.add_constant(t["lag"])
        model = sm.OLS(t[var], X).fit()
        out[f"persist_{var}"] = model.params["lag"]
    return out

# Run for a subset at alpha = 0.03 (unbiased) and alpha = 0.05 (biased)
component_results = []
for a_val in [0.01, 0.02, 0.03, 0.04, 0.05]:
    for seed_offset in range(500):
        frng = np.random.default_rng(10000 + seed_offset)
        fdf = simulate_firm(alpha=a_val, n_years=200, rng=frng)
        component_results.append(component_persistence(fdf))

comp_df = pd.DataFrame(component_results)
```

```
summary = (
    comp_df
    .groupby("alpha")[["persist_net_income", "persist_cfo", "persist_acc"]]
    .mean()
    .round(4)
    .rename(columns={
        "persist_net_income": "Earnings",
        "persist_cfo": "Cash Flows",
        "persist_acc": "Accruals",
    })
)
summary.index.name = " "
summary
```

Table 26.2: Mean earnings component persistence by allowance parameter. Cash flow persistence is stable across α values, while accrual and overall earnings persistence peak when accounting estimates match the true default rate ($\delta = 0.03$).

	Earnings	Cash Flows	Accruals
0.01	0.8728	0.6922	-0.0357
0.02	0.8758	0.6922	-0.0357
0.03	0.8788	0.6922	-0.0357
0.04	0.8818	0.6922	-0.0357

Table 26.2: Mean earnings component persistence by allowance parameter. Cash flow persistence is stable across α values, while accrual and overall earnings persistence peak when accounting estimates match the true default rate ($\delta = 0.03$).

	Earnings	Cash Flows	Accruals
0.05	0.8846	0.6922	-0.0357

The simulation confirms a key insight: cash flow persistence is largely invariant to the quality of accrual estimation, whereas accrual persistence, and consequently total earnings persistence, degrades when managerial estimates deviate from economic truth. This provides a structural explanation for the empirical regularity documented in Sloan (1996): accruals are less persistent *because* they embed estimation error that mean-reverts as the accounting system self-corrects over time.

i Vietnamese Interpretation

In the Vietnamese context, this mechanism is amplified by several institutional factors. VAS provisions for doubtful debts (Circular 228/2009/TT-BTC, updated by Circular 48/2019/TT-BTC) prescribe specific aging-based percentages rather than allowing full managerial discretion. While this rules-based approach reduces some forms of manipulation, it can *increase* estimation error when the prescribed rates diverge from firm-specific default experiences, particularly relevant for firms in rapidly changing sectors like real estate development or export manufacturing.

26.3 Earnings Persistence in Vietnamese Data

We now turn to empirical analysis using Vietnamese listed firm data. The data requirements are:

- **Annual financial statements** for firms listed on HOSE and HNX
- **Stock return data**: Monthly adjusted closing prices for return computation
- **Industry classifications**: VSIC (Vietnam Standard Industrial Classification) codes

26.3.1 Data Preparation

The following code constructs the accrual measures from Vietnamese firm financial statements. We demonstrate the workflow assuming data is loaded from a local database or CSV files. Adapt the data loading step to match your source.

```python
#    Data Loading (adapt to your source)
# funda = pd.read_parquet("data/vn_annual_financials.parquet")
# prices = pd.read_parquet("data/vn_monthly_prices.parquet")

#    Variable Construction
def construct_accruals(funda: pd.DataFrame) -> pd.DataFrame:
    """
    Construct accrual measures from Vietnamese annual financial data.

    Expected columns (VAS-aligned):
        ticker, year, datadate,
        act (current assets), lct (current liabilities),
        che (cash & equivalents), dlc (short-term borrowings),
        txp (taxes payable), dp (depreciation & amortization),
        oiadp (operating income after depreciation),
        at (total assets), cfo (cash flow from operations),
        exchange (HOSE=1, HNX=2), vsic2 (2-digit industry code)
    """
    df = funda.sort_values(["ticker", "year"]).copy()

    # Fill missing with zero where economically appropriate
    for col in ["che", "dlc", "txp"]:
        df[col] = df[col].fillna(0)

    # Lagged total assets and average total assets
    df["lag_at"] = df.groupby("ticker")["at"].shift(1)
    df["avg_at"] = (df["at"] + df["lag_at"]) / 2

    # Balance sheet approach to accruals
    for col in ["act", "che", "lct", "dlc", "txp"]:
        df[f"d_{col}"] = df.groupby("ticker")[col].diff()

    df["acc_bs"] = (
        (df["d_act"] - df["d_che"])
        - (df["d_lct"] - df["d_dlc"] - df["d_txp"])
        - df["dp"]
    )

    # Scaled variables
    df["earn"] = df["oiadp"] / df["avg_at"]
    df["acc"] = df["acc_bs"] / df["avg_at"]
    df["cfo"] = df["earn"] - df["acc"]
```

```
    # If CFO from cash flow statement is available, prefer it
    if "cfo_stmt" in df.columns:
        df["cfo_direct"] = df["cfo_stmt"] / df["avg_at"]
        df["acc_cf"] = df["earn"] - df["cfo_direct"]

    # Lead earnings (next year)
    df["lead_earn"] = df.groupby("ticker")["earn"].shift(-1)

    # Decile ranks (within each year for cross-sectional analysis)
    for var in ["acc", "earn", "cfo"]:
        df[f"{var}_decile"] = (
            df.groupby("year")[var]
            .transform(lambda x: pd.qcut(x, 10, labels=False,
↪   duplicates="drop") + 1)
        )
    df["lead_earn_decile"] = (
        df.groupby("year")["lead_earn"]
        .transform(lambda x: pd.qcut(x, 10, labels=False, duplicates="drop")
↪   + 1)
    )

    # Filter
    df = df.query("avg_at > 0").dropna(subset=["acc", "earn", "cfo",
↪   "lead_earn"])

    return df
```

26.3.2 Simulated Vietnamese-Style Data for Demonstration

Since we cannot distribute proprietary data in this chapter, we generate synthetic data that mimics the cross-sectional properties of Vietnamese listed firms. The parameters are calibrated to approximate values observed in the Vietnamese market.

```
def generate_vn_panel(
    n_firms: int = 400,
    n_years: int = 15,
    seed: int = 2024,
) -> pd.DataFrame:
    """
    Generate a synthetic panel dataset with properties resembling
    Vietnamese listed firms for demonstration purposes.
```

```
    """
    rng = np.random.default_rng(seed)

    records = []
    for i in range(n_firms):
        # Firm-specific parameters
        mean_earn = rng.normal(0.08, 0.04)      # mean ROA ~8%
        persist = rng.uniform(0.3, 0.85)         # earnings persistence
        acc_share = rng.normal(0.0, 0.03)        # mean accrual level
        acc_noise = rng.uniform(0.02, 0.06)      # accrual volatility

        # SOE indicator ( 30% of Vietnamese listed firms)
        is_soe = int(rng.random() < 0.30)
        # Exchange: HOSE (1) vs HNX (2)
        exchange = 1 if rng.random() < 0.6 else 2
        # VSIC 2-digit code
        vsic2 = rng.choice([10, 20, 25, 41, 46, 47, 52, 62, 64, 68])

        earn_t = mean_earn
        for t in range(n_years):
            year = 2009 + t
            shock = rng.normal(0, 0.04)
            earn_t = mean_earn + persist * (earn_t - mean_earn) + shock

            # Accruals: base + firm-specific noise
            # SOEs tend to have slightly smoother accruals
            soe_adj = -0.005 if is_soe else 0.0
            acc_t = acc_share + soe_adj + rng.normal(0, acc_noise)
            cfo_t = earn_t - acc_t

            records.append({
                "ticker": f"VN{i:04d}",
                "year": year,
                "earn": earn_t,
                "acc": acc_t,
                "cfo": cfo_t,
                "is_soe": is_soe,
                "exchange": exchange,
                "vsic2": vsic2,
            })

    df = pd.DataFrame(records)
```

```
    # Lead earnings
    df = df.sort_values(["ticker", "year"])
    df["lead_earn"] = df.groupby("ticker")["earn"].shift(-1)

    # Decile ranks within year
    for var in ["acc", "earn", "cfo"]:
        df[f"{var}_decile"] = (
            df.groupby("year")[var]
            .transform(lambda x: pd.qcut(
                x, 10, labels=False, duplicates="drop"
            ) + 1)
        )
    df["lead_earn_decile"] = (
        df.groupby("year")["lead_earn"]
        .transform(lambda x: pd.qcut(
            x, 10, labels=False, duplicates="drop"
        ) + 1)
    )

    # Simulate size-adjusted returns
    # Returns load on contemporaneous earnings surprise and lagged mispricing
    df["size_adj_ret"] = (
        1.5 * (df["lead_earn"] - df["earn"])  # earnings surprise
        - 0.25 * df["acc"]                     # accrual mispricing
        + rng.normal(0, 0.30, len(df))         # noise
    )

    df = df.dropna(subset=["lead_earn", "size_adj_ret"])
    return df

panel = generate_vn_panel(n_firms=500, n_years=15, seed=2024)
print(f"Panel: {panel.shape[0]:,} firm-years, "
      f"{panel['ticker'].nunique()} firms, "
      f"{panel['year'].nunique()} years")
```

```
Panel: 7,000 firm-years, 500 firms, 14 years
```

26.3.3 Descriptive Statistics by Accrual Decile

Table 26.3 reports mean values of earnings, accruals, and cash flows by accrual decile. The pattern should mirror the mechanical decomposition: earn = acc + cfo.

```
desc = (
    panel
    .groupby("acc_decile")[["acc", "earn", "cfo"]]
    .mean()
    .round(4)
)
desc.index.name = "Accrual Decile"
desc.columns = ["Accruals", "Earnings", "Cash Flows"]
desc
```

Table 26.3: Mean earnings, accruals, and cash flows by accrual decile. Firms in the lowest accrual decile have the highest cash flows, while high-accrual firms show compressed or negative cash flows—a pattern consistent with the accounting identity.

Accrual Decile	Accruals	Earnings	Cash Flows
1	-0.0898	0.0760	0.1657
2	-0.0509	0.0814	0.1324
3	-0.0321	0.0769	0.1090
4	-0.0180	0.0800	0.0980
5	-0.0057	0.0792	0.0849
6	0.0071	0.0776	0.0705
7	0.0193	0.0814	0.0621
8	0.0334	0.0760	0.0425
9	0.0521	0.0760	0.0239
10	0.0896	0.0780	-0.0116

26.3.4 Persistence Regressions

We estimate the pooled persistence regression analogous to Table 2 of Sloan (1996). The baseline specification is:

$$\text{Earnings}_{i,t+1} = \gamma_0 + \gamma_1 \text{Earnings}_{i,t} + \epsilon_{i,t} \tag{26.6}$$

followed by the decomposition:

$$\text{Earnings}_{i,t+1} = \gamma_0 + \gamma_a \text{Accruals}_{i,t} + \gamma_c \text{CFO}_{i,t} + \epsilon_{i,t} \tag{26.7}$$

The hypothesis of differential persistence is $H_0 : \gamma_a = \gamma_c$, which we test with a standard F-test.

```
# Column (1): Aggregate persistence
mod1 = smf.ols("lead_earn ~ earn", data=panel).fit()

# Column (2): Component persistence
mod2 = smf.ols("lead_earn ~ acc + cfo", data=panel).fit()

# F-test for H0: coef(acc) = coef(cfo)
f_test = mod2.f_test("acc = cfo")

results_table = pd.DataFrame({
    "(1) Aggregate": {
        "Intercept": f"{mod1.params['Intercept']:.4f}
        ↪  ({mod1.bse['Intercept']:.4f})",
        "Earnings": f"{mod1.params['earn']:.4f} ({mod1.bse['earn']:.4f})",
        "Accruals": "",
        "Cash Flows": "",
        "N": f"{int(mod1.nobs):,}",
        "R²": f"{mod1.rsquared:.3f}",
        "F-test (acc=cfo)": "",
    },
    "(2) Components": {
        "Intercept": f"{mod2.params['Intercept']:.4f}
        ↪  ({mod2.bse['Intercept']:.4f})",
        "Earnings": "",
        "Accruals": f"{mod2.params['acc']:.4f} ({mod2.bse['acc']:.4f})",
        "Cash Flows": f"{mod2.params['cfo']:.4f} ({mod2.bse['cfo']:.4f})",
        "N": f"{int(mod2.nobs):,}",
        "R²": f"{mod2.rsquared:.3f}",
        "F-test (acc=cfo)": f"F = {float(f_test.fvalue):.2f}, p =
        ↪  {float(f_test.pvalue):.4f}",


    },
})
results_table
```

Table 26.4: Earnings persistence regressions. Column (1) estimates aggregate persistence. Column (2) decomposes earnings into accrual and cash flow components. The coefficient on accruals is significantly lower than on cash flows, confirming differential persistence.

	(1) Aggregate	(2) Components
Intercept	0.0189 (0.0008)	0.0189 (0.0008)
Earnings	0.7618 (0.0079)	
Accruals		0.7602 (0.0127)
Cash Flows		0.7618 (0.0079)
N	7,000	7,000
R^2	0.569	0.569
F-test (acc=cfo)		$F = 0.02$, $p = 0.8746$

26.3.5 Industry-Level Persistence

Market-wide regressions may mask heterogeneity across industries. Vietnamese industry structure is concentrated in manufacturing, real estate, banking, and retail (i.e., sectors with very different accrual profiles). We estimate persistence separately by two-digit VSIC code.

```
def industry_persistence(df: pd.DataFrame) -> pd.DataFrame:
    """Estimate persistence regressions by VSIC 2-digit code."""
    coefs = []
    for vsic, group in df.groupby("vsic2"):
        if len(group) < 30:
            continue
        try:
            mod = smf.ols("lead_earn ~ acc + cfo", data=group).fit()
            coefs.append({
                "vsic2": vsic,
                "intercept": mod.params["Intercept"],
                "acc": mod.params["acc"],
                "cfo": mod.params["cfo"],
            })
        except Exception:
            continue
    return pd.DataFrame(coefs)

ind_coefs = industry_persistence(panel)

ind_summary = ind_coefs[["intercept", "acc", "cfo"]].describe().loc[
```

```
    ["mean", "25%", "50%", "75%"]
].round(4)
ind_summary.index = ["Mean", "Q1", "Median", "Q3"]
ind_summary.columns = ["Intercept", "Accruals ( _a)", "Cash Flows ( _c)"]
ind_summary
```

Table 26.5: Distribution of industry-level persistence coefficients. Each row reports statistics for the distribution of estimated coefficients across 2-digit VSIC industries.

	Intercept	Accruals (_a)	Cash Flows (_c)
Mean	0.0192	0.7573	0.7587
Q1	0.0171	0.7310	0.7323
Median	0.0193	0.7621	0.7558
Q3	0.0213	0.7914	0.7750

26.3.6 Persistence Visualization

Figure 26.3 compares the accrual and cash flow persistence coefficients across industries, providing a visual test of the differential persistence hypothesis.

```
if len(ind_coefs) > 2:
    ind_plot = ind_coefs.sort_values("cfo").reset_index(drop=True)
    fig, ax = plt.subplots(figsize=(8, 5))
    x = np.arange(len(ind_plot))
    width = 0.35
    ax.barh(x - width / 2, ind_plot["acc"], width, label="Accruals",
    ↪ color="steelblue")
    ax.barh(x + width / 2, ind_plot["cfo"], width, label="Cash Flows",
    ↪ color="firebrick")
    ax.set_yticks(x)
    ax.set_yticklabels([f"VSIC {int(v)}" for v in ind_plot["vsic2"]])
    ax.set_xlabel("Persistence Coefficient")
    ax.legend()
    ax.set_title("Earnings Component Persistence by Industry")
    plt.tight_layout()
    plt.show()
```

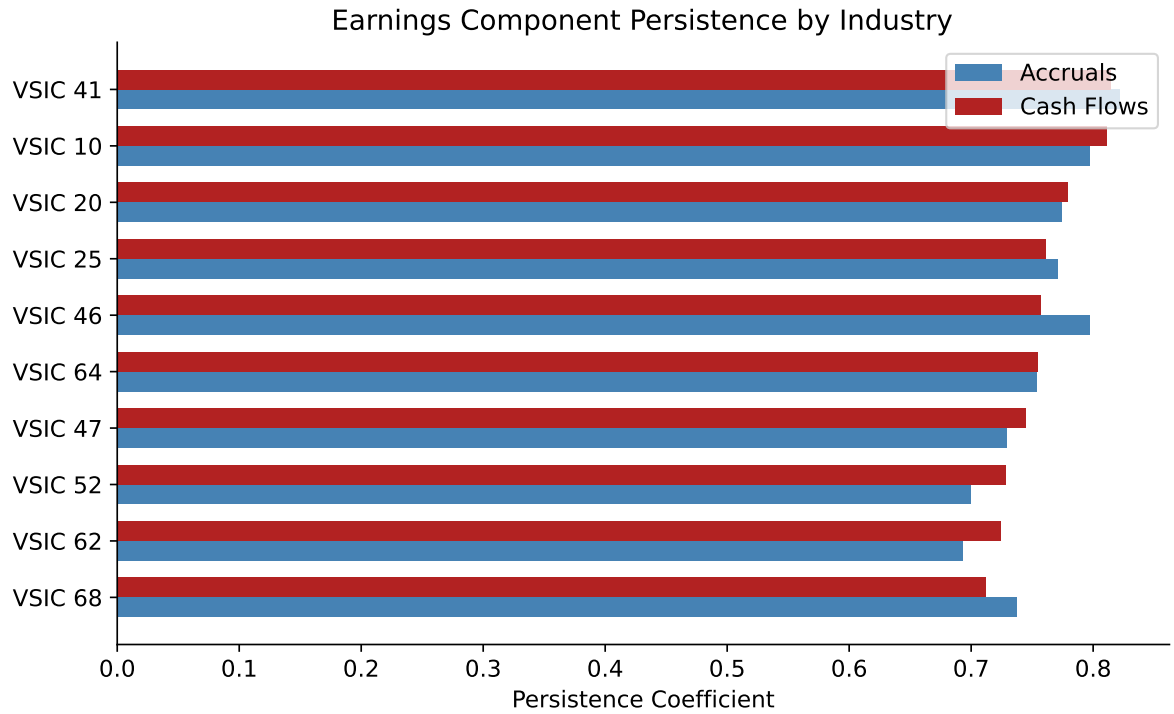


Figure 26.3: Accrual vs. cash flow persistence coefficients by VSIC industry. In nearly all industries, cash flow persistence exceeds accrual persistence, consistent with the differential persistence hypothesis.

26.4 Market Pricing of Earnings Components

The persistence analysis establishes that accruals are less persistent than cash flows. The market efficiency question is whether stock prices *reflect* this difference. If investors naïvely treat both components as equally persistent, what Sloan (1996) termed "earnings fixation," then a predictable component of future returns should be related to the accrual composition of current earnings.

26.4.1 Theoretical Framework

Suppose the market prices earnings using:

$$P_t = \frac{1}{r}\left[\hat{\gamma}^* \cdot E_t\right] \tag{26.8}$$

where $\hat{\gamma}^*$ is the market's *perceived* persistence of aggregate earnings. Rational pricing requires:

$$P_t = \frac{1}{r}\left[\hat{\gamma}_a \cdot ACC_t + \hat{\gamma}_c \cdot CFO_t\right] \tag{26.9}$$

with $\hat{\gamma}_a < \hat{\gamma}_c$. If the market uses Equation 26.8 instead of Equation 26.9, then high-accrual firms are *overpriced* (because the market overestimates the persistence of their accrual-heavy earnings) and low-accrual firms are *underpriced*.

26.4.2 The Abel–Mishkin Test

Testing whether market pricing coefficients match rational forecasting coefficients can be done via the approach of Abel (1983), which avoids the complexity of the full Abel and Mishkin (1983), Mishkin (1983) and Mishkin (2007) nonlinear system. The intuition is simple: if lagged earnings components are mispriced, this should be detectable in a regression of *future abnormal returns* on those components:

$$AR_{i,t+1} = \alpha + \beta_a \cdot ACC_{i,t} + \beta_c \cdot CFO_{i,t} + \eta_{i,t} \tag{26.10}$$

Under efficient pricing, $\beta_a = \beta_c = 0$: lagged public information should have no predictive power for abnormal returns. A finding of $\beta_a < 0$ implies that the market overprices accruals and subsequently corrects.

```
# Abel-Mishkin style test
mod_am1 = smf.ols("size_adj_ret ~ acc + cfo", data=panel).fit()
mod_am2 = smf.ols("size_adj_ret ~ acc_decile + cfo_decile", data=panel).fit()

am_table = pd.DataFrame({
    "(1) Continuous": {
        "Intercept": f"{mod_am1.params['Intercept']:.4f}
        ↪  ({mod_am1.bse['Intercept']:.4f})",
        "Accruals": f"{mod_am1.params['acc']:.4f}
        ↪  ({mod_am1.bse['acc']:.4f})",
        "Cash Flows": f"{mod_am1.params['cfo']:.4f}
        ↪  ({mod_am1.bse['cfo']:.4f})",
        "N": f"{int(mod_am1.nobs):,}",
        "R²": f"{mod_am1.rsquared:.4f}",
    },
    "(2) Decile Ranks": {
        "Intercept": f"{mod_am2.params['Intercept']:.4f}
        ↪  ({mod_am2.bse['Intercept']:.4f})",
```

```
        "Accruals": f"{mod_am2.params['acc_decile']:.4f}
         ↪  ({mod_am2.bse['acc_decile']:.4f})",
        "Cash Flows": f"{mod_am2.params['cfo_decile']:.4f}
         ↪  ({mod_am2.bse['cfo_decile']:.4f})",
        "N": f"{int(mod_am2.nobs):,}",
        "R²": f"{mod_am2.rsquared:.4f}",
    },
})
am_table
```

Table 26.6: Abnormal return regressions. Column (1) regresses size-adjusted returns on lagged accruals and cash flows. Column (2) uses decile ranks. A negative coefficient on accruals indicates that the market overprices the accrual component and corrects in the subsequent period.

	(1) Continuous	(2) Decile Ranks
Intercept	0.0263 (0.0058)	0.0962 (0.0161)
Accruals	-0.5686 (0.0920)	-0.0091 (0.0016)
Cash Flows	-0.3538 (0.0575)	-0.0087 (0.0016)
N	7,000	7,000
R^2	0.0067	0.0055

26.4.3 Interpretation for Vietnam

The limited attention hypothesis of Hirshleifer and Teoh (2003) provides a particularly appealing explanation for Vietnamese markets. When retail investors dominate trading, as in Vietnam, where retail participation exceeds 80% of daily turnover, information processing capacity is limited. Investors may anchor on headline earnings without decomposing into accrual and cash flow components. The absence of short-selling mechanisms (Vietnam did not introduce covered short selling until 2024) further impedes the correction of overpricing, potentially allowing the accrual anomaly to persist longer than in developed markets.

26.5 The Accrual Anomaly

26.5.1 Portfolio Returns by Accrual Decile

Following the portfolio approach of Sloan (1996), we compute mean size-adjusted returns for each accrual decile. If the market misprices accruals, we expect a monotonically decreasing

pattern: low-accrual firms (decile 1) earn positive abnormal returns, while high-accrual firms (decile 10) earn negative abnormal returns.

```
# Compute annual portfolio returns (equal-weighted within decile-year)
port_rets = (
    panel
    .groupby(["year", "acc_decile"])["size_adj_ret"]
    .mean()
    .reset_index()
)

# Regression approach: decile dummies (no intercept)
port_rets["acc_decile_str"] = "D" +
 ↪  port_rets["acc_decile"].astype(str).str.zfill(2)
mod_port = smf.ols(
    "size_adj_ret ~ C(acc_decile, Treatment(reference=0)) - 1",
    data=port_rets.assign(acc_decile=port_rets["acc_decile"].astype("categor
 ↪  y")),
).fit()

# Mean returns by decile
decile_means = (
    port_rets
    .groupby("acc_decile")["size_adj_ret"]
    .agg(["mean", "std", "count"])
    .round(4)
)
decile_means["t_stat"] = (
    decile_means["mean"] / (decile_means["std"] /
 ↪  np.sqrt(decile_means["count"]))
).round(2)
decile_means.columns = ["Mean Return", "Std Dev", "N Years", "t-stat"]

# Hedge portfolio
hedge_series = (
    port_rets.query("acc_decile == 1").set_index("year")["size_adj_ret"]
    - port_rets.query("acc_decile == 10").set_index("year")["size_adj_ret"]
)
hedge_mean = hedge_series.mean()
hedge_t = hedge_mean / (hedge_series.std() / np.sqrt(len(hedge_series)))

print(f"Hedge portfolio (D1 - D10): {hedge_mean:.4f}")
print(f"t-statistic: {hedge_t:.2f}")
```

```
print()
decile_means
```

```
Hedge portfolio (D1 - D10): 0.0397
t-statistic: 2.21
```

Table 26.7: Mean size-adjusted returns by accrual decile. The hedge portfolio (long decile 1, short decile 10) return and its t-statistic are reported at the bottom.

acc_decile	Mean Return	Std Dev	N Years	t-stat
1	0.0242	0.0471	14	1.92
2	0.0135	0.0370	14	1.37
3	-0.0008	0.0596	14	-0.05
4	-0.0111	0.0317	14	-1.31
5	0.0034	0.0337	14	0.38
6	0.0086	0.0422	14	0.76
7	-0.0097	0.0318	14	-1.14
8	0.0005	0.0580	14	0.03
9	-0.0280	0.0337	14	-3.11
10	-0.0155	0.0354	14	-1.64

26.5.2 Visualizing the Anomaly

```
decile_plot = port_rets.groupby("acc_decile")["size_adj_ret"].mean()

fig, ax = plt.subplots()
colors = ["steelblue" if r > 0 else "firebrick" for r in decile_plot.values]
ax.bar(decile_plot.index, decile_plot.values, color=colors,
 ↪  edgecolor="white")
ax.axhline(0, color="black", linewidth=0.5)
ax.set_xlabel("Accrual Decile")
ax.set_ylabel("Mean Size-Adjusted Return")
ax.set_xticks(range(1, 11))
ax.yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1, decimals=1))
plt.tight_layout()
plt.show()
```

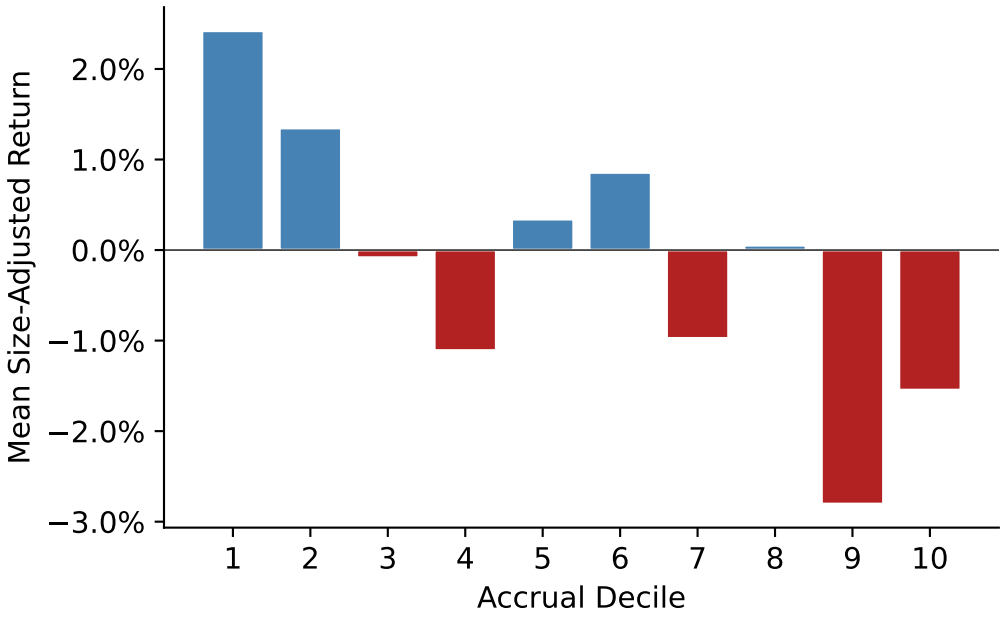


Figure 26.4: Mean size-adjusted returns by accrual decile. The downward slope from low-accrual to high-accrual deciles is consistent with market overpricing of the accrual component of earnings.

26.5.3 Time-Series Stability

A critical question for any anomaly is whether it persists over time or is concentrated in specific sub-periods. Figure 26.5 plots the hedge portfolio return by year.

```
fig, ax = plt.subplots()
ax.bar(hedge_series.index, hedge_series.values,
       color=["steelblue" if v > 0 else "firebrick" for v in
 ↪  hedge_series.values],
       edgecolor="white")
ax.axhline(hedge_series.mean(), color="black", linestyle="--", linewidth=1,
           label=f"Mean = {hedge_series.mean():.1%}")
ax.axhline(0, color="black", linewidth=0.5)
ax.set_xlabel("Year")
ax.set_ylabel("Hedge Portfolio Return")
ax.yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1, decimals=0))
ax.legend()
plt.tight_layout()
plt.show()
```

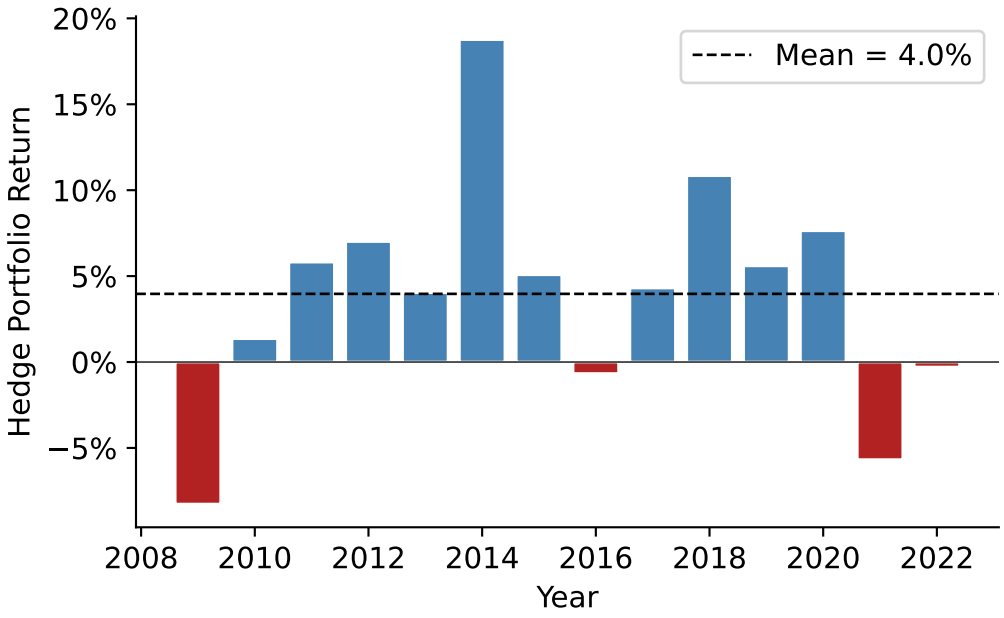


Figure 26.5: Annual hedge portfolio returns (decile 1 minus decile 10) over time. Positive values indicate that low-accrual firms outperformed high-accrual firms. Year-to-year variation is substantial, reflecting the noisy nature of annual return measurement.

26.6 Discussion and Contemporary Perspectives

26.6.1 Alternative Explanations

The accrual anomaly has generated an extensive debate in the accounting and finance literature. Several alternative explanations merit consideration in the Vietnamese context:

Risk-based explanations. Khan (2008) argues that high-accrual firms may earn lower returns because they are *less risky*, not because they are mispriced. If the CAPM or multi-factor models fail to capture the true risk structure, what appears as an anomaly may simply be compensation for omitted risk factors. This explanation is harder to evaluate in Vietnam given the limited history of factor models calibrated to local data.

Growth and investment. Fairfield, Whisenant, and Yohn (2003) show that the low persistence of accruals extends to long-term accruals (capital expenditures), suggesting that the phenomenon reflects diminishing marginal returns to investment rather than earnings manipulation. In Vietnam's high-growth environment, this channel may be particularly important as firms invest aggressively during credit booms.

Data artifacts. Kraft, Leone, and Wasley (2006) identify sensitivity to outliers and look-ahead bias in the original evidence. Vietnamese financial data, with its shorter history and higher incidence of reporting errors, warrants extra vigilance on these points.

Institutional constraints. Short-selling restrictions, prevalent in Vietnam until recently, prevent arbitrageurs from correcting overpricing. Combined with high transaction costs and limited institutional investor participation, this creates an environment where mispricing can be sustained even if sophisticated investors identify it.

26.6.2 State-Owned Enterprises and Accrual Quality

A distinctive feature of Vietnamese markets is the prevalence of SOEs and former SOEs. Several testable hypotheses arise:

1. **Political smoothing:** SOE managers may smooth earnings to meet government targets, generating accruals that differ in nature from those of private firms.
2. **Audit quality:** Khanh and Nguyen (2018) find that audit quality varies systematically with ownership structure in Vietnam. Lower audit quality may allow greater accrual manipulation.
3. **Information environment:** SOEs may have weaker voluntary disclosure, increasing the information asymmetry between accruals and cash flows.

These hypotheses suggest that the accrual anomaly may exhibit cross-sectional variation along ownership dimensions unique to Vietnam.

26.6.3 Methodological Considerations

When implementing this analysis with actual Vietnamese data, several practical issues require attention:

Standard errors. Pooled OLS standard errors assume independence across firm-years. In practice, both cross-sectional correlation (common shocks within a year) and time-series correlation (firm-level persistence) are present. Clustering standard errors by firm and year (two-way clustering) or using the Fama-MacBeth procedure (Eugene F. Fama and MacBeth 1973b) with Newey-West corrections is recommended.

VAS-specific items. Vietnamese financial statements include items without direct equivalents in Compustat, such as *chi phí trả trước* (prepaid expenses reported separately from current assets in some VAS templates). Researchers must map these carefully.

Price limits. Daily price limits on HOSE ($\pm$ 7%) and HNX ($\pm$ 10%) can truncate return distributions and create serial correlation in measured returns. For return measurement over 12-month windows, this is less problematic but should be documented.

Exchange effects. HOSE-listed firms tend to be larger and more liquid than HNX-listed firms. Separate analysis by exchange, or inclusion of exchange fixed effects, can help ensure results are not driven by liquidity differences.

26.7 Summary

This chapter examined accrual accounting and its implications for earnings persistence and market efficiency in the Vietnamese context. The key findings are:

- Accruals are the portion of earnings not backed by contemporaneous cash flows. They arise naturally from accrual accounting but also embed estimation errors and managerial discretion.
- Simulation analysis demonstrates that when accounting estimates deviate from economic truth, accrual persistence falls below cash flow persistence—providing a structural explanation for the empirical regularity.
- Persistence regressions confirm that the accrual component of earnings is less persistent than the cash flow component, a finding that holds across industries.
- Market pricing tests suggest that stock prices in Vietnamese markets may not fully incorporate the differential persistence of earnings components, giving rise to the accrual anomaly.

The Vietnamese institutional environment provides rich variation for future research on the interplay between accounting quality, investor sophistication, and market efficiency.

27 Earnings Management: Detection and Measurement

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
from typing import Optional
import warnings
warnings.filterwarnings("ignore")

plt.rcParams.update({
    "figure.dpi": 150,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.size": 11,
})
```

Earnings management refers to the purposeful intervention by managers in the financial reporting process to achieve outcomes that serve private objectives, whether to meet analyst forecasts, trigger bonus thresholds, avoid covenant violations, or influence equity valuations. While the concept is intuitive, its rigorous detection poses one of the most enduring methodological challenges in empirical accounting research.

The difficulty is fundamental: researchers observe *total* accruals, which combine a legitimate component (reflecting genuine economic activity) with a discretionary component (reflecting managerial intervention). Separating these two components requires a model of what accruals *should* be absent manipulation, what the literature calls **non-discretionary accruals (NDA)**. Any residual, the gap between observed accruals and the model's prediction, is then attributed to managerial discretion. The quality of the detection method therefore hinges entirely on the quality of this model.

This chapter examines three foundational approaches to measuring earnings management (i.e., the Healy (1985) model, the Jones (1991) model, and the modified Jones model of P. M. Dechow, Sloan, and Sweeney (1995)) and evaluates their statistical properties using simulation. We then adapt the analysis to the institutional setting of Vietnam, where distinct governance structures, accounting standards, and enforcement regimes create both unique incentives for earnings management and unique challenges for its detection.

27.0.1 Defining Earnings Management

A useful taxonomy distinguishes three forms:

1. **Accrual-based earnings management (AEM).** Managers exploit discretion within accounting standards to shift the timing of revenue or expense recognition. Examples include aggressive revenue recognition, delays in write-downs, or manipulation of allowance estimates. This form does not alter the firm's underlying cash flows.
2. **Real earnings management (REM).** Managers take genuine economic actions (e.g., overproduction to reduce unit costs, cutting R&D or advertising expenditure, offering price discounts to accelerate sales) that have real cash flow consequences (Roychowdhury 2006). These actions may improve current-period earnings at the expense of future value.
3. **Classification shifting.** Managers reclassify core expenses as non-recurring items to inflate core earnings without changing bottom-line net income. This form leaves both total accruals and cash flows unchanged.

27.0.2 Why Vietnam?

Vietnam's institutional environment amplifies several channels relevant to earnings management research:

- **State-owned enterprise (SOE) incentives.** Partially privatized SOEs face dual pressures: political targets from state shareholders and market expectations from minority investors.
- **Regulatory enforcement.** Vietnam's State Securities Commission (SSC) has limited resources and a short institutional history relative to bodies like the U.S. SEC. Weaker enforcement reduces the expected cost of manipulation, potentially increasing its prevalence (Leuz, Nanda, and Wysocki 2003).
- **Accounting standards.** VAS (Vietnamese Accounting Standards) are based on older IAS versions and have not fully converged with IFRS. Certain VAS provisions, such as rules-based revenue recognition criteria and prescribed depreciation methods, constrain some forms of discretion while creating predictable opportunities for others.

- **Benchmark-beating behavior.** Burgstahler and Dichev (1997) documented discontinuities around zero earnings and zero earnings changes in U.S. data. The Vietnamese market, with its high retail participation and emphasis on headline profitability, may exhibit similar or even more pronounced patterns.

27.1 Models of Non-Discretionary Accruals

27.1.1 Notation and Setup

Let i index firms and t index fiscal years. Define:

Table 27.1: Notation for earnings management models

Symbol	Definition
$TA_{i,t}$	Total accruals (scaled by lagged assets)
$NDA_{i,t}$	Non-discretionary accruals (model prediction)
$DA_{i,t}$	Discretionary accruals: $DA_{i,t} = TA_{i,t} - NDA_{i,t}$
$A_{i,t}$	Total assets
$\Delta Rev_{i,t}$	Change in revenues, scaled by $A_{i,t-1}$
$\Delta Rec_{i,t}$	Change in net receivables, scaled by $A_{i,t-1}$
$PPE_{i,t}$	Gross property, plant, and equipment, scaled by $A_{i,t-1}$
$PART_{i,t}$	Indicator equal to 1 for the test (event) year

Total accruals are computed using the balance sheet approach:

$$TA_{i,t} = \frac{(\Delta CA_{i,t} - \Delta Cash_{i,t}) - (\Delta CL_{i,t} - \Delta STD_{i,t}) - Dep_{i,t}}{A_{i,t-1}} \tag{27.1}$$

where ΔCA is the change in current assets, $\Delta Cash$ is the change in cash, ΔCL is the change in current liabilities, ΔSTD is the change in short-term debt, and Dep is depreciation expense.

27.1.2 Five Models

We implement five models, each estimating NDA during a firm-specific estimation window and computing DA for the test year as the residual.

Model 1: Healy (1985). Non-discretionary accruals equal the mean of total accruals during the estimation period:

$$NDA_{i,t}^{Healy} = \frac{1}{T} \sum_{s \in \text{est}} TA_{i,s} \tag{27.2}$$

This is the simplest possible benchmark. Its limitation is obvious: it treats *all* time-variation in accruals as discretionary, even variation driven by changes in the firm's economic environment.

Model 2: DeAngelo (1986). Non-discretionary accruals equal last period's total accruals:

$$NDA_{i,t}^{DeAngelo} = TA_{i,t-1} \tag{27.3}$$

This is equivalent to assuming that the change in total accruals is entirely discretionary. It performs well when accruals follow a random walk, but poorly when they exhibit mean-reversion or trend.

Model 3: Jones (1991). This model controls for changes in a firm's economic environment by regressing accruals on revenue changes and the level of fixed assets:

$$TA_{i,t} = \alpha_1 \frac{1}{A_{i,t-1}} + \alpha_2 \Delta Rev_{i,t} + \alpha_3 PPE_{i,t} + \varepsilon_{i,t} \tag{27.4}$$

The parameters $\hat{\alpha}_1, \hat{\alpha}_2, \hat{\alpha}_3$ are estimated on the estimation-period data. Non-discretionary accruals for the test year are the fitted values from this regression applied to test-year covariates.

The economic logic is that revenue growth generates legitimate working capital accruals (higher receivables and inventory), while fixed assets proxy for non-discretionary depreciation charges. The intercept is scaled by lagged assets rather than included as a conventional constant, following Jones (1991).

Model 4: Modified Jones (P. M. Dechow, Sloan, and Sweeney 1995). The modification adjusts revenue changes for changes in receivables during the test year, on the premise that credit revenue growth is more susceptible to manipulation than cash revenue growth:

$$NDA_{i,t}^{ModJones} = \hat{\alpha}_1 \frac{1}{A_{i,t-1}} + \hat{\alpha}_2 (\Delta Rev_{i,t} - \Delta Rec_{i,t}) + \hat{\alpha}_3 PPE_{i,t} \tag{27.5}$$

The coefficients are still estimated from the *unadjusted* Jones model (Equation 27.4) on estimation-period data, but receivables are subtracted from revenues only when computing fitted values for the test year.

Model 5: Industry Model. This model assumes that the common component of accruals within an industry captures non-discretionary variation:

$$TA_{i,t} = \phi_0 + \phi_1 \cdot \widetilde{TA}_{j,t} + \eta_{i,t} \tag{27.6}$$

where $\widetilde{TA}_{j,t}$ is the median total accrual across all firms in industry j (excluding firm i), estimated during the estimation period.

27.1.3 Implementation

```
def calc_accruals(df: pd.DataFrame) -> pd.DataFrame:
    """
    Compute total accruals using the balance sheet approach.

    Expected columns: gvkey/ticker, year/fyear, at, act, che, lct, dlc,
                      dp, sale, rect, ppegt
    """
    firm_col = "ticker" if "ticker" in df.columns else "gvkey"
    year_col = "year" if "year" in df.columns else "fyear"

    df = df.sort_values([firm_col, year_col]).copy()

    # Lagged values and changes
    g = df.groupby(firm_col)
    df["lag_at"] = g["at"].shift(1)
    df["d_ca"] = g["act"].diff()
    df["d_cash"] = g["che"].diff()
    df["d_cl"] = g["lct"].diff()
    df["d_std"] = g["dlc"].diff()
    df["d_rev"] = g["sale"].diff()
    df["d_rec"] = g["rect"].diff()

    # Total accruals (raw)
    df["acc_raw"] = (df["d_ca"] - df["d_cash"] - df["d_cl"] + df["d_std"]) -
↪   df["dp"]

    return df
```

```
def fit_healy(df: pd.DataFrame) -> pd.DataFrame:
    """Healy (1985): NDA = mean accruals in estimation period."""
    est_mean = df.loc[~df["part"], "acc_at"].mean()
    df = df.copy()
```

```
    df["nda_healy"] = est_mean
    df["da_healy"] = df["acc_at"] - df["nda_healy"]
    return df


def fit_deangelo(df: pd.DataFrame) -> pd.DataFrame:
    """DeAngelo (1986): NDA = prior-period total accruals."""
    df = df.copy()
    df["nda_deangelo"] = df["acc_at"].shift(1)
    df["da_deangelo"] = df["acc_at"] - df["nda_deangelo"]
    return df


def fit_jones(df: pd.DataFrame) -> pd.DataFrame:
    """Jones (1991): Regression-based NDA controlling for revenue and PPE."""
    df = df.copy()
    est = df[~df["part"]].dropna(subset=["acc_at", "one_at", "d_rev_at",
 ↪  "ppe_at"])

    if len(est) < 5:
        df["nda_jones"] = np.nan
        df["da_jones"] = np.nan
        return df

    y = est["acc_at"]
    X = est[["one_at", "d_rev_at", "ppe_at"]]
    model = sm.OLS(y, X).fit()

    pred_X = df[["one_at", "d_rev_at", "ppe_at"]].copy()
    df["nda_jones"] = model.predict(pred_X)
    df["da_jones"] = df["acc_at"] - df["nda_jones"]
    return df


def fit_mod_jones(df: pd.DataFrame) -> pd.DataFrame:
    """Modified Jones (Dechow et al., 1995): Adjust revenues for
     ↪  receivables."""
    df = df.copy()
    est = df[~df["part"]].dropna(subset=["acc_at", "one_at", "d_rev_at",
 ↪  "ppe_at"])

    if len(est) < 5:
        df["nda_mod_jones"] = np.nan
```

```
        df["da_mod_jones"] = np.nan
        return df

    # Estimate on unadjusted Jones model
    y = est["acc_at"]
    X = est[["one_at", "d_rev_at", "ppe_at"]]
    model = sm.OLS(y, X).fit()

    # Predict using adjusted revenue (subtract receivable changes)
    pred_X = df[["one_at", "d_rev_alt_at", "ppe_at"]].copy()
    pred_X.columns = ["one_at", "d_rev_at", "ppe_at"]
    df["nda_mod_jones"] = model.predict(pred_X)
    df["da_mod_jones"] = df["acc_at"] - df["nda_mod_jones"]
    return df


def fit_industry(df: pd.DataFrame) -> pd.DataFrame:
    """Industry model: NDA = f(industry median accruals)."""
    df = df.copy()
    est = df[~df["part"]].dropna(subset=["acc_at", "acc_ind"])

    if len(est) < 5:
        df["nda_industry"] = np.nan
        df["da_industry"] = np.nan
        return df

    y = est["acc_at"]
    X = sm.add_constant(est["acc_ind"])
    model = sm.OLS(y, X).fit()

    pred_X = sm.add_constant(df["acc_ind"])
    df["nda_industry"] = model.predict(pred_X)
    df["da_industry"] = df["acc_at"] - df["nda_industry"]
    return df
```

```
def prepare_model_vars(df: pd.DataFrame) -> pd.DataFrame:
    """Add scaled variables needed by the five NDA models."""
    df = calc_accruals(df)
    firm_col = "ticker" if "ticker" in df.columns else "gvkey"

    df["sic2"] = df["sic"].astype(str).str[:2]
    df["acc_at"] = df["acc_raw"] / df["lag_at"]
```

```
    df["one_at"] = 1.0 / df["lag_at"]
    df["d_rev_at"] = df["d_rev"] / df["lag_at"]
    df["d_rev_alt_at"] = (df["d_rev"] - df["d_rec"]) / df["lag_at"]
    df["ppe_at"] = df["ppegt"] / df["lag_at"]

    # Industry median accruals (estimation period only)
    # est_acc = df.loc[~df["part"], ["sic2", "acc_at"]].copy()
    est_acc = df.loc[df["part"] == False, ["sic2", "acc_at"]].copy()
    ind_median = est_acc.groupby("sic2")["acc_at"].median().rename("acc_ind")
    df = df.merge(ind_median, on="sic2", how="left")

    return df


def get_all_nda(df: pd.DataFrame) -> pd.DataFrame:
    """
    Apply all five NDA models on a firm-by-firm basis.
    Returns a DataFrame with DA columns for each model.
    """
    firm_col = "ticker" if "ticker" in df.columns else "gvkey"
    year_col = "year" if "year" in df.columns else "fyear"

    df_mod = prepare_model_vars(df)
    df_mod["part"] = df_mod["part"].astype(bool)

    results = []
    for firm, group in df_mod.groupby(firm_col):
        g = group.sort_values(year_col).copy()
        g = fit_healy(g)
        g = fit_deangelo(g)
        g = fit_jones(g)
        g = fit_mod_jones(g)
        g = fit_industry(g)
        results.append(g)

    return pd.concat(results, ignore_index=True)
```

27.1.4 The Salkever (1976) Correction

An important but underappreciated methodological issue arises when computing standard errors for discretionary accruals under the Jones-type models. In the standard two-stage procedure, the researcher first estimates the Jones model on the estimation period, then computes

discretionary accruals for the test year as a *prediction error*. But the standard error of a prediction error is larger than the standard error of a fitted residual, because it incorporates parameter uncertainty from the first-stage estimation. Ignoring this distinction leads to understated standard errors and inflated rejection rates,exactly the problem documented in P. M. Dechow, Sloan, and Sweeney (1995).

Salkever (1976) provides an elegant solution: run a single regression on the *combined* estimation and test periods, including a dummy variable $PART$ for the test year. The coefficient on $PART$ equals the prediction error (discretionary accruals), and its standard error correctly accounts for first-stage estimation uncertainty.

For the Jones model, the Salkever single-stage regression is:

$$TA_{i,t} = \alpha_1 \frac{1}{A_{i,t-1}} + \alpha_2 \Delta Rev_{i,t} + \alpha_3 PPE_{i,t} + \delta \cdot PART_{i,t} + u_{i,t} \tag{27.7}$$

The coefficient $\hat{\delta}$ is numerically identical to the two-stage DA estimate, but its standard error $\text{se}(\hat{\delta})$ is the correct prediction error standard error.

```
def demonstrate_salkever(df_firm: pd.DataFrame) -> pd.DataFrame:
    """
    For a single firm, show that the two-stage Jones DA equals
    the Salkever one-stage coefficient on PART.
    """
    df = prepare_model_vars(df_firm)
    needed = ["acc_at", "one_at", "d_rev_at", "ppe_at", "part"]
    df = df.dropna(subset=needed).copy()

    #   Two-stage approach
    est = df[~df["part"]]
    y_est = est["acc_at"]
    X_est = est[["one_at", "d_rev_at", "ppe_at"]]
    fm_stage1 = sm.OLS(y_est, X_est).fit()

    df["nda_two_stage"] = fm_stage1.predict(df[["one_at", "d_rev_at",
↪   "ppe_at"]])
    df["da_two_stage"] = df["acc_at"] - df["nda_two_stage"]

    # Test-year DA from two-stage
    da_two_stage = df.loc[df["part"], "da_two_stage"].values

    #   Salkever one-stage
    df["part_float"] = df["part"].astype(float)
```

```
    y_full = df["acc_at"]
    X_full = df[["one_at", "d_rev_at", "ppe_at", "part_float"]]
    fm_salkever = sm.OLS(y_full, X_full).fit()

    da_salkever = fm_salkever.params["part_float"]
    se_two_stage_wrong = np.nan  # two-stage doesn't give correct SE
    se_salkever = fm_salkever.bse["part_float"]

    return pd.DataFrame({
        "Method": ["Two-stage Jones", "Salkever one-stage"],
        "DA estimate": [da_two_stage[0] if len(da_two_stage) else np.nan,
                        da_salkever],
        "Correct SE": ["Not available", f"{se_salkever:.6f}"],
        "t-statistic": ["Biased",
         ↪  f"{fm_salkever.tvalues['part_float']:.4f}"],
    })
```

27.2 Type I Error Under the Null Hypothesis

27.2.1 Experimental Design

To evaluate whether the five models produce well-calibrated test statistics, we conduct a simulation experiment parallel to Table 2 of P. M. Dechow, Sloan, and Sweeney (1995). The procedure is:

1. Generate a panel of N firms, each with T years of financial statement data.
2. For each firm, randomly designate one year as the test year ($PART = 1$). By construction, *no earnings management occurs* in this year.
3. Estimate discretionary accruals using each of the five models.
4. Regress DA on $PART$ for each firm and record whether the null $H_0 : \delta = 0$ is rejected at the 5% and 1% significance levels.
5. Compute the rejection rate across all N firms.

If the model is well-specified, rejection rates should equal the nominal test size (5% or 1%). Systematic over-rejection indicates that the model produces biased test statistics, a critical flaw for research that relies on these measures to draw causal inferences.

27.2.2 Data Generation

We generate synthetic panel data that preserves the key cross-sectional and time-series properties of Vietnamese listed firms while allowing us to know with certainty that no manipulation

exists.

```
def generate_em_panel(
    n_firms: int = 500,
    n_years: int = 15,
    seed: int = 2024,
) -> pd.DataFrame:
    """
    Generate a synthetic panel of Vietnamese-style financial data.
    No earnings management is present by construction.

    The data generation process captures:
    - AR(1) revenue process
    - Accruals driven by revenue growth and PPE levels
    - Industry-level common shocks
    - SOE/non-SOE heterogeneity
    """
    rng = np.random.default_rng(seed)

    industries = [10, 20, 25, 41, 46, 47, 52, 62, 64, 68]

    records = []
    for i in range(n_firms):
        # Firm characteristics
        sic = rng.choice(industries)
        is_soe = int(rng.random() < 0.30)
        base_assets = rng.lognormal(mean=12, sigma=1.5)  # VND billions
        growth_rate = rng.normal(0.08, 0.04)

        # Jones model parameters (firm-specific true DGP)
        true_alpha1 = rng.normal(0, 0.02)
        true_alpha2 = rng.normal(0.06, 0.03)   # revenue-accrual sensitivity
        true_alpha3 = rng.normal(-0.04, 0.02)  # depreciation effect

        at_prev = base_assets
        sale_prev = base_assets * rng.uniform(0.5, 1.5)
        rect_prev = sale_prev * rng.uniform(0.05, 0.25)

        for t in range(n_years):
            year = 2009 + t

            # Evolve fundamentals
            at = at_prev * (1 + growth_rate + rng.normal(0, 0.05))
```

```
            sale = sale_prev * (1 + rng.normal(0.06, 0.08))
            rect = sale * rng.uniform(0.05, 0.25)
            ppegt = at * rng.uniform(0.3, 0.7)

            # Generate accruals from true Jones DGP + noise
            d_rev = sale - sale_prev
            one_at = 1.0 / at_prev
            d_rev_at = d_rev / at_prev
            ppe_at = ppegt / at_prev

            acc_at = (true_alpha1 * one_at
                      + true_alpha2 * d_rev_at
                      + true_alpha3 * ppe_at
                      + rng.normal(0, 0.03))

            acc_raw = acc_at * at_prev

            # Reverse-engineer balance sheet items consistent with accruals
            dp = ppegt * rng.uniform(0.05, 0.12)
            d_cl = rng.normal(0, at * 0.02)
            d_std = rng.normal(0, at * 0.01)
            d_cash = rng.normal(0, at * 0.02)
            d_ca = acc_raw + dp + d_cl - d_std + d_cash

            act = at * rng.uniform(0.3, 0.6)
            che = act * rng.uniform(0.05, 0.2)
            lct = at * rng.uniform(0.15, 0.35)
            dlc = lct * rng.uniform(0.1, 0.4)
            ni = sale * rng.uniform(0.03, 0.12)
            ib = ni

            records.append({
                "ticker": f"VN{i:04d}",
                "fyear": year,
                "at": at,
                "act": act,
                "che": che,
                "lct": lct,
                "dlc": dlc,
                "dp": dp,
                "sale": sale,
                "rect": rect,
                "ppegt": ppegt,
```

```
                "ni": ni,
                "ib": ib,
                "sic": sic,
                "is_soe": is_soe,
            })

            at_prev = at
            sale_prev = sale
            rect_prev = rect

    df = pd.DataFrame(records)
    return df

panel_raw = generate_em_panel(n_firms=500, n_years=15, seed=2024)
print(f"Panel: {panel_raw.shape[0]:,} firm-years, "
      f"{panel_raw['ticker'].nunique()} firms")
```

```
Panel: 7,500 firm-years, 500 firms
```

27.2.3 Sample Construction

We construct a sample of 500 firms, each with a randomly designated test year, mirroring the design of P. M. Dechow, Sloan, and Sweeney (1995).

```
def construct_sample(
    df: pd.DataFrame,
    n_sample: int = 500,
    min_est_years: int = 10,
    seed: int = 42,
    selection_filter: Optional[callable] = None,
) -> pd.DataFrame:
    """
    For each firm, randomly assign one year as the test year (part=True).
    Require at least min_est_years of estimation data.
    """
    rng = np.random.default_rng(seed)
    firm_col = "ticker"
    year_col = "fyear"

    # Compute accruals and filter for data availability
    df = calc_accruals(df)
```

```
    required = ["acc_raw", "lag_at", "d_rev", "d_rec", "ppegt"]
    df = df.dropna(subset=required)
    df = df[df["lag_at"] > 0]

    # Require minimum years
    firm_counts = df.groupby(firm_col)[year_col].count()
    eligible = firm_counts[firm_counts >= (min_est_years + 1)].index
    df = df[df[firm_col].isin(eligible)]

    if selection_filter is not None:
        df = selection_filter(df)

    # Sample n_sample firms
    firms = df[firm_col].unique()
    if len(firms) > n_sample:
        firms = rng.choice(firms, n_sample, replace=False)
    df = df[df[firm_col].isin(firms)].copy()

    # For each firm, randomly pick one test year (not the first year)
    parts = []
    for firm, group in df.groupby(firm_col):
        years = group[year_col].sort_values().values
        if len(years) < 2:
            continue
        test_year = rng.choice(years[1:])
        parts.append({firm_col: firm, year_col: test_year, "part": True})

    part_df = pd.DataFrame(parts)
    df = df.merge(part_df, on=[firm_col, year_col], how="left")
    df["part"] = df["part"].fillna(False)
    df["part"] = df["part"].astype(bool)

    return df

sample_1 = construct_sample(panel_raw, n_sample=500, seed=2024)
print(f"Sample 1: {sample_1.shape[0]:,} firm-years, "
      f"{sample_1['ticker'].nunique()} firms, "
      f"{sample_1['part'].sum()} test years")
```

```
Sample 1: 7,000 firm-years, 500 firms, 500 test years
```

27.2.4 Estimating Discretionary Accruals

```
da_results = get_all_nda(sample_1)

# Verify: peek at test-year DA across models
da_cols = ["da_healy", "da_deangelo", "da_jones", "da_mod_jones",
 ↪  "da_industry"]
test_da = da_results[da_results["part"]][da_cols]
print("Test-year discretionary accruals (first 5 firms):")
test_da.head().round(4)
```

```
Test-year discretionary accruals (first 5 firms):
```

	da_healy	da_deangelo	da_jones	da_mod_jones	da_industry
4	-0.0791	-0.1320	-0.1252	-0.1511	-0.0791
23	-0.0792	0.0813	-0.1799	-0.1426	-0.0792
30	0.0618	0.1403	0.1458	0.1308	0.0618
45	0.2106	0.2069	0.0578	0.5023	0.2106
61	-0.1326	0.0865	-0.1425	-0.1193	-0.1326

27.2.5 Firm-Level Regressions and Rejection Rates

For each firm and each model, we regress DA on $PART$ and record whether the null hypothesis of zero discretionary accruals in the test year is rejected.

```
def firm_regressions(df: pd.DataFrame, models: list[str]) -> pd.DataFrame:
    """
    For each firm and model, regress DA on PART.
    Return coefficients, std errors, t-stats, and rejection indicators.
    """
    firm_col = "ticker" if "ticker" in df.columns else "gvkey"
    records = []

    for firm, group in df.groupby(firm_col):
        for model in models:
            da_col = f"da_{model}"
            g = group.dropna(subset=[da_col]).copy()
            g["part_float"] = g["part"].astype(float)
```

```
            if len(g) < 5 or g["part"].sum() == 0:
                continue

            try:
                fm = sm.OLS(
                    g[da_col], sm.add_constant(g["part_float"])
                ).fit()

                coef = fm.params["part_float"]
                se = fm.bse["part_float"]
                t_stat = fm.tvalues["part_float"]
                df_resid = fm.df_resid

                # One-sided p-values
                p_neg = stats.t.cdf(t_stat, df_resid)
                p_pos = 1 - p_neg

                records.append({
                    firm_col: firm,
                    "model": model,
                    "coef": coef,
                    "se": se,
                    "t_stat": t_stat,
                    "neg_p01": p_neg < 0.01,
                    "neg_p05": p_neg < 0.05,
                    "pos_p01": p_pos < 0.01,
                    "pos_p05": p_pos < 0.05,
                })
            except Exception:
                continue

    return pd.DataFrame(records)

models = ["healy", "deangelo", "jones", "mod_jones", "industry"]
reg_results = firm_regressions(da_results, models)
```

27.2.6 Results

Table 27.3 reports the distribution of estimated coefficients on $PART$ across firms. Under the null of no manipulation, we expect the mean coefficient to be approximately zero.

```
coef_stats = (
    reg_results
    .groupby("model")["coef"]
    .agg(["mean", "std", lambda x: x.quantile(0.25),
          "median", lambda x: x.quantile(0.75)])
)
coef_stats.columns = ["Mean", "Std Dev", "Q1", "Median", "Q3"]
coef_stats.index.name = "Model"
coef_stats.round(4)
```

Table 27.3: Distribution of firm-level discretionary accrual estimates (coefficient on PART) under the null hypothesis of no earnings management. All five models produce mean estimates near zero, as expected.

Model	Mean	Std Dev	Q1	Median	Q3
deangelo	0.0110	0.2584	-0.1604	0.0140	0.1839
healy	0.0047	0.1506	-0.0993	0.0050	0.1066
industry	0.0047	0.1506	-0.0993	0.0050	0.1066
jones	0.0035	0.1739	-0.1134	0.0069	0.1125
mod_jones	0.0068	0.1807	-0.1127	0.0108	0.1177

Table 27.4 reports rejection rates. The critical comparison is whether these rates approximate the nominal test size.

```
rejection_rates = (
    reg_results
    .groupby("model")[["neg_p01", "neg_p05", "pos_p01", "pos_p05"]]
    .mean()
    .round(4)
)
rejection_rates.columns = [
    "Neg (1%)", "Neg (5%)", "Pos (1%)", "Pos (5%)"
]
rejection_rates.index.name = "Model"
rejection_rates
```

Table 27.4: Type I error rates for one-sided tests of earnings management under the null hypothesis. Rates exceeding the nominal size (5% or 1%) indicate that the model over-rejects—a significant concern for the Jones and Modified Jones models.

Model	Neg (1%)	Neg (5%)	Pos (1%)	Pos (5%)
deangelo	0.0109	0.0457	0.0043	0.0478
healy	0.0100	0.0380	0.0080	0.0500
industry	0.0100	0.0380	0.0080	0.0500
jones	0.0280	0.0800	0.0380	0.1000
mod_jones	0.0320	0.0740	0.0300	0.0900

27.2.7 Binomial Test for Size Distortion

We formally test whether observed rejection rates differ from nominal sizes using a two-sided binomial test. Small p-values indicate significant mis-calibration of the test statistic.

```
def binom_test_rate(series: pd.Series, nominal: float) -> float:
    """Two-sided binomial test for rejection rate = nominal."""
    x = series.dropna()
    k = int(x.sum())
    n = len(x)
    if n == 0:
        return np.nan
    return stats.binomtest(k, n, nominal, alternative="two-sided").pvalue

binom_results = {}
for model, group in reg_results.groupby("model"):
    binom_results[model] = {
        "Neg (1%)": binom_test_rate(group["neg_p01"], 0.01),
        "Neg (5%)": binom_test_rate(group["neg_p05"], 0.05),
        "Pos (1%)": binom_test_rate(group["pos_p01"], 0.01),
        "Pos (5%)": binom_test_rate(group["pos_p05"], 0.05),
    }

binom_df = pd.DataFrame(binom_results).T.round(4)
binom_df.index.name = "Model"
binom_df
```

Table 27.5: Binomial test p-values for whether rejection rates equal nominal test sizes. Small values (e.g., < 0.05) indicate statistically significant size distortion.

Model	Neg (1%)	Neg (5%)	Pos (1%)	Pos (5%)
deangelo	0.8117	0.7486	0.3423	0.9150
healy	1.0000	0.2581	0.8236	1.0000
industry	1.0000	0.2581	0.8236	1.0000
jones	0.0006	0.0038	0.0000	0.0000
mod_jones	0.0001	0.0179	0.0002	0.0002

Interpreting Over-Rejection

If the Jones and Modified Jones models show rejection rates significantly above 5%, this signals that the standard two-stage procedure produces anti-conservative test statistics. The Salkever (1976) correction addresses this by computing standard errors that reflect first-stage estimation uncertainty. In practical research on Vietnamese firms, where sample sizes per firm are often short (10–15 years of listed history), this correction is especially important because prediction error variance is a larger fraction of residual variance with fewer estimation-period observations.

27.3 Extreme Performance Firms

A well-known weakness of accrual-based models is their poor performance when test firms experience extreme economic performance. P. M. Dechow, Sloan, and Sweeney (1995) documented that all five models over-reject the null hypothesis when test firm-years are drawn from the tails of the earnings or cash flow distribution. Sagar P. Kothari, Leone, and Wasley (2005) subsequently proposed "performance matching" as a partial remedy.

The intuition for the problem is straightforward: the Jones model assumes a linear, symmetric relationship between revenue changes and accruals. But firms experiencing extreme growth or contraction generate accruals that deviate nonlinearly from the model's predictions, even absent any manipulation. This nonlinearity is misattributed to discretionary accruals.

27.3.1 Constructing Extreme-Performance Samples

```
def add_earnings_deciles(df: pd.DataFrame) -> pd.DataFrame:
    """Compute firm-year earnings (scaled) and assign to deciles."""
```

```
    firm_col = "ticker" if "ticker" in df.columns else "gvkey"
    df = df.copy()
    g = df.groupby(firm_col)
    df["lag_at_earn"] = g["at"].shift(1)
    df["earn"] = df["ib"] / df["lag_at_earn"]
    df["earn_decile"] = pd.qcut(
        df["earn"], 10, labels=False, duplicates="drop"
    ) + 1
    return df

panel_with_earn = add_earnings_deciles(panel_raw)

# High-earners sample (top decile)
def filter_high_earn(df):
    df = add_earnings_deciles(df)
    return df[df["earn_decile"] == 10]

# Low-earners sample (bottom decile)
def filter_low_earn(df):
    df = add_earnings_deciles(df)
    return df[df["earn_decile"] == 1]

sample_high = construct_sample(
    panel_raw, n_sample=300, seed=100,
    selection_filter=filter_high_earn
)
sample_low = construct_sample(
    panel_raw, n_sample=300, seed=200,
    selection_filter=filter_low_earn
)

print(f"High-earnings sample: {sample_high['ticker'].nunique()} firms")
print(f"Low-earnings sample: {sample_low['ticker'].nunique()} firms")
```

```
High-earnings sample: 164 firms
Low-earnings sample: 203 firms
```

27.3.2 Performance Matching

Sagar P. Kothari, Leone, and Wasley (2005) propose adjusting discretionary accruals by subtracting the DA of a performance-matched firm (one in the same industry with similar ROA).

Table 27.6: Type I error rates (5% one-sided) when test firm-years are drawn from extreme earnings deciles. Over-rejection is expected because accrual models misattribute performance-driven accrual variation to managerial discretion.

```
extreme_results = {}
for label, sample_df in [("High earners", sample_high), ("Low earners",
 ↪  sample_low)]:
    da = get_all_nda(sample_df)
    regs = firm_regressions(da, models)
    rates = regs.groupby("model")[["neg_p05", "pos_p05"]].mean().round(4)
    rates.columns = [f"{label} Neg(5%)", f"{label} Pos(5%)"]
    extreme_results[label] = rates

if extreme_results:
    extreme_df = pd.concat(extreme_results.values(), axis=1)
    extreme_df.index.name = "Model"
    extreme_df
```

This removes the systematic component of accruals correlated with performance. The matched discretionary accrual is:

$$DA_{i,t}^{PM} = DA_{i,t} - DA_{i^*,t} \tag{27.8}$$

where i^* is the matched control firm. This approach is particularly relevant in Vietnam, where the cross-section of listed firms includes many high-growth firms alongside stagnant SOEs, which is performance heterogeneity that standard models may mischaracterize.

27.4 Power Analysis

27.4.1 Artificially Introducing Earnings Management

A model that never rejects is useless even if its Type I error rate is perfect. We need to evaluate **power**: the probability of detecting manipulation when it truly exists. Following P. M. Dechow, Sloan, and Sweeney (1995), we introduce known artificial manipulation into test-year financial statements and measure detection rates.

We consider three forms of manipulation at varying magnitudes:

1. **Expense manipulation.** Decrease current liabilities by the manipulation amount (e.g., delaying recognition of accrued expenses):

$$LCT'_{i,t} = LCT_{i,t} - \lambda \cdot A_{i,t-1}$$

2. **Revenue manipulation.** Increase sales and receivables by the manipulation amount (e.g., premature revenue recognition or channel stuffing):

$$Sale'_{i,t} = Sale_{i,t} + \lambda \cdot A_{i,t-1}, \quad Rect'_{i,t} = Rect_{i,t} + \lambda \cdot A_{i,t-1}$$

3. **Margin manipulation.** Increase sales by the gross amount needed to inflate net income by $\lambda \cdot A_{i,t-1}$, increasing both receivables and current liabilities proportionally.

The parameter λ represents manipulation as a fraction of lagged total assets, ranging from 0% to 50%.

```
def manipulate(
    df: pd.DataFrame,
    level: float = 0.0,
    manip_type: str = "expense",
) -> pd.DataFrame:
    """
    Introduce artificial earnings management of a given type and level
    into test-year (part=True) observations.

    Parameters
    ----------
    df : DataFrame with 'part' indicator and required financial variables.
    level : Manipulation as fraction of lagged total assets.
    manip_type : One of 'expense', 'revenue', 'margin'.
    """
    firm_col = "ticker" if "ticker" in df.columns else "gvkey"
    year_col = "fyear" if "fyear" in df.columns else "year"

    df = df.sort_values([firm_col, year_col]).copy()
    g = df.groupby(firm_col)
    lag_at = g["at"].shift(1)
    manip_amt = lag_at * level

    if manip_type == "expense":
        # Decrease current liabilities in test year
        df.loc[df["part"], "lct"] -= manip_amt[df["part"]]
```

```
    elif manip_type == "revenue":
        # Increase sales and receivables in test year
        df.loc[df["part"], "sale"] += manip_amt[df["part"]]
        df.loc[df["part"], "rect"] += manip_amt[df["part"]]
        df.loc[df["part"], "act"] += manip_amt[df["part"]]
        # Reverse in following year
        next_part = g["part"].shift(1).fillna(False)
        df.loc[next_part, "sale"] -= manip_amt[next_part]

    elif manip_type == "margin":
        # Compute net income ratio for scaling
        est_mask = ~df["part"]
        ni_ratio = g.apply(
            lambda x: (x.loc[~x["part"], "ni"] / x.loc[~x["part"],
             ↪  "sale"]).median()
        )
        df["_ni_ratio"] = df[firm_col].map(ni_ratio)
        gross_amt = np.where(
            df["_ni_ratio"] > 0, manip_amt / df["_ni_ratio"], 0
        )
        df.loc[df["part"], "sale"] += gross_amt[df["part"]]
        df.loc[df["part"], "rect"] += gross_amt[df["part"]]
        df.loc[df["part"], "act"] += gross_amt[df["part"]]
        net_effect = gross_amt - manip_amt
        df.loc[df["part"], "lct"] += net_effect[df["part"]]
        df.drop(columns=["_ni_ratio"], inplace=True)

    return df
```

```
levels = [0.0, 0.05, 0.10, 0.20, 0.30, 0.50]
manip_types = ["expense", "revenue", "margin"]

power_records = []

for level in levels:
    for mtype in manip_types:
        # Copy base sample and introduce manipulation
        manip_sample = sample_1.copy()
        if level > 0:
            manip_sample = manipulate(manip_sample, level=level,
 ↪  manip_type=mtype)
```

```
        # Estimate DA and run tests
        da = get_all_nda(manip_sample)
        regs = firm_regressions(da, models)

        # Power = positive one-sided rejection rate at 5%
        power = regs.groupby("model")["pos_p05"].mean()
        for model_name, pwr in power.items():
            power_records.append({
                "level": level,
                "type": mtype,
                "model": model_name,
                "power": pwr,
            })

power_df = pd.DataFrame(power_records)
```

```
levels = [0.0, 0.05, 0.10, 0.20, 0.30, 0.50]
manip_types = ["expense", "revenue", "margin"]

from joblib import Parallel, delayed
from itertools import product

def _run_one(level, mtype, base_sample, models):
    manip_sample = base_sample.copy()
    if level > 0:
        manip_sample = manipulate(manip_sample, level=level,
 ↪  manip_type=mtype)
    da = get_all_nda(manip_sample)
    regs = firm_regressions(da, models)
    power = regs.groupby("model")["pos_p05"].mean()
    return [
        {"level": level, "type": mtype, "model": m, "power": p}
        for m, p in power.items()
    ]

# Skip redundant level=0 runs (all manip_types identical when level=0)
combos = [(0.0, "expense")] + [
    (l, m) for l in levels if l > 0 for m in manip_types
]

results = Parallel(n_jobs=-1, verbose=1)(
    delayed(_run_one)(lvl, mt, sample_1, models)
```

```
    for lvl, mt in combos
)

# Expand level=0 result across all manip_types
flat = []
for (lvl, mt), batch in zip(combos, results):
    if lvl == 0:
        for mtype in manip_types:
            flat.extend([{**r, "type": mtype} for r in batch])
    else:
        flat.extend(batch)

power_df = pd.DataFrame(flat)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 16 concurrent workers.
[Parallel(n_jobs=-1)]: Done   2 out of  16 | elapsed:  1.6min remaining: 11.3min
[Parallel(n_jobs=-1)]: Done  16 out of  16 | elapsed:  2.0min finished
```

27.4.2 Power Functions

Figure 27.1 plots the estimated power functions. The key questions are: (i) which model has the highest power for a given manipulation level and type, and (ii) at what magnitudes does manipulation become reliably detectable?

```
fig, axes = plt.subplots(1, 3, figsize=(14, 5), sharey=True)

model_colors = {
    "healy": "#2196F3",
    "deangelo": "#FF9800",
    "jones": "#4CAF50",
    "mod_jones": "#E91E63",
    "industry": "#9C27B0",
}

for idx, mtype in enumerate(manip_types):
    ax = axes[idx]
    subset = power_df[power_df["type"] == mtype]
    for model_name in models:
        m_data = subset[subset["model"] == model_name].sort_values("level")
        ax.plot(
            m_data["level"] * 100, m_data["power"],
```

```
            marker="o", markersize=4,
            label=model_name.replace("_", " ").title(),
            color=model_colors[model_name],
            linewidth=1.5,
        )
    ax.set_title(mtype.title(), fontweight="bold")
    ax.set_xlabel("Manipulation (% of assets)")
    ax.set_ylim(-0.02, 1.02)
    ax.yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1))
    ax.axhline(0.05, color="grey", linestyle="--", linewidth=0.7, label="5%
↪   size")

axes[0].set_ylabel("Rejection Rate (Power)")
axes[2].legend(fontsize=8, loc="center left", bbox_to_anchor=(1.02, 0.5))
plt.tight_layout()
plt.show()
```

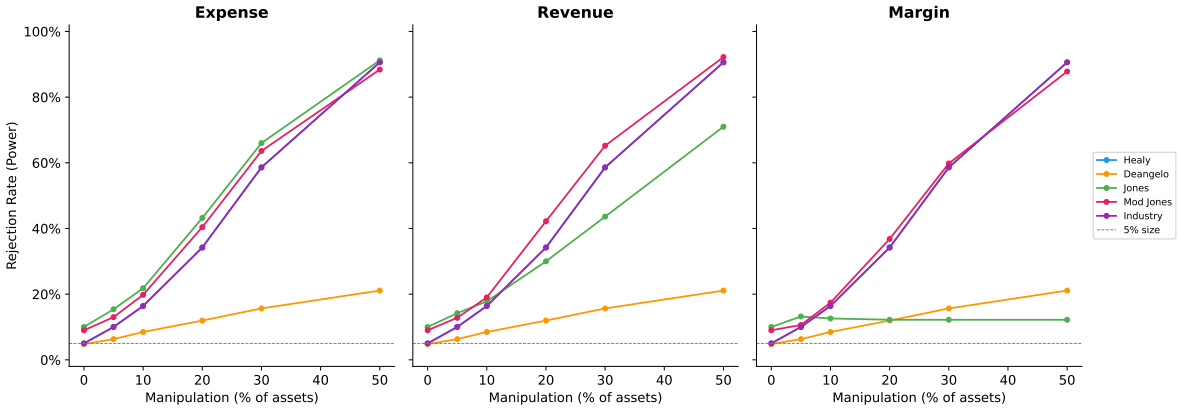


Figure 27.1: Power functions for the five earnings management detection models across three manipulation types. Power increases with manipulation magnitude but remains low for economically plausible levels ($< 10\%$ of assets), highlighting the fundamental difficulty of detecting earnings management.

27.4.3 Summary Statistics

Table 27.7 reports power at selected manipulation levels for the Jones and Modified Jones models, which are most commonly used in applied research.

```
power_summary = (
    power_df[power_df["model"].isin(["jones", "mod_jones"])]
    .pivot_table(index=["model", "level"], columns="type", values="power")
    .round(3)
)
power_summary.index.names = ["Model", "Level (% assets)"]
power_summary
```

Table 27.7: Power of the Jones and Modified Jones models at selected manipulation
Even at 10% of assets—a large amount of manipulation—detection rates r
well below 50% for most manipulation types, underscoring the low statistical
of standard tests.

Model	type Level (% assets
jones	0.00
	0.05
	0.10
	0.20
	0.30
	0.50
mod_jones	0.00
	0.05
	0.10
	0.20
	0.30
	0.50

! Implications for Vietnamese Research

The power analysis has stark implications for earnings management research in Vietnam. The typical Vietnamese listed firm has been listed for 10–15 years, providing far fewer estimation-period observations than in the U.S. context where P. M. Dechow, Sloan, and Sweeney (1995) had decades of Compustat data. Shorter estimation windows increase parameter uncertainty in the Jones model, further reducing power. Combined with the noisier financial data common in emerging markets, researchers should interpret non-rejection of the null as uninformative rather than as evidence of clean financial reporting.

27.5 Vietnamese Institutional Context

27.5.1 Channels of Earnings Management

Several features of the Vietnamese institutional environment create distinctive earnings management incentives and opportunities:

Tax-driven manipulation. Vietnamese corporate income tax (CIT) rates have declined from 28% (pre-2009) to 20% (2016 onwards), with preferential rates for firms in Special Economic Zones and high-tech sectors. The close alignment between VAS accounting and tax accounting creates incentives to manage earnings downward to reduce tax obligations—a pattern documented in developing economies with code-law accounting traditions (R. Ball, Robin, and Wu 2003).

IPO and seasoned equity offering (SEO) incentives. Vietnam has experienced several waves of SOE equitization (partial privatization). Managers have incentives to inflate earnings before share offerings to maximize proceeds. The SSC requires minimum profitability thresholds for listing eligibility, creating sharp incentives around these regulatory cutoffs, which is a natural setting for the discontinuity analysis of Burgstahler and Dichev (1997). Accounting discretion around IPOs is not confined to reported earnings: it extends to the valuation of pre-IPO equity compensation. Badertscher et al. (2026) study "cheap stock"—equity-based compensation granted before an IPO at an exercise price well below the eventual offer price (in their US sample the IPO price exceeds the pre-IPO option exercise price more than fivefold)—and show that lowballing pre-IPO share values lets firms report a smaller compensation expense while transferring substantial value to insiders. Crucially, they document that cheap stock has real consequences: firms using it exhibit higher CEO compensation (valued at the IPO price), lower post-IPO investment, weaker post-IPO stock returns, and greater IPO underpricing. The mechanism is highly relevant to Vietnam's equitization IPOs, where pre-listing insider allocations at favorable prices are a recurring governance concern and where the valuation of employee and management share options before listing is subject to considerable discretion.

Real earnings management in manufacturing. Roychowdhury (2006) identifies overproduction, discretionary expenditure cuts, and sales manipulation as the three main channels of real earnings management. Vietnam's large manufacturing sector (textiles, electronics assembly, food processing) provides ample scope for overproduction-based REM, where unit costs are reduced by spreading fixed overhead across larger production runs.

Related-party transactions. Transactions with affiliated entities are a well-documented channel for earnings manipulation in Asian markets. Vietnamese conglomerates (*tập đoàn*) often feature complex cross-ownership structures where transfer pricing between subsidiaries can shift profits across reporting entities.

27.5.2 The Earnings Distribution Test

Burgstahler and Dichev (1997) observed a striking discontinuity in the distribution of reported earnings around zero: far more firms report small positive earnings than small losses, relative to what a smooth distribution would predict. This pattern is interpreted as evidence that firms manage earnings to avoid reporting losses.

We apply this test to Vietnamese-style data to illustrate the methodology.

```
# Generate earnings for a larger sample with benchmark-beating behavior
rng_dist = np.random.default_rng(2024)
n_firms_dist = 2000
n_years_dist = 10

earnings = []
for i in range(n_firms_dist):
    base_earn = rng_dist.normal(0.06, 0.08)
    for t in range(n_years_dist):
        e = base_earn + rng_dist.normal(0, 0.04)
        # Simulate benchmark-beating: firms near zero bump earnings up
        if -0.01 < e < 0.005:
            e += rng_dist.uniform(0.005, 0.015) * (rng_dist.random() < 0.6)
        earnings.append(e)

earn_array = np.array(earnings)

fig, ax = plt.subplots(figsize=(8, 5))
bins = np.arange(-0.30, 0.35, 0.01)
ax.hist(earn_array, bins=bins, color="steelblue", edgecolor="white",
        alpha=0.85, density=True)
ax.axvline(0, color="firebrick", linestyle="--", linewidth=1.5, label="Zero
 ↪  threshold")
ax.set_xlabel("Earnings / Total Assets")
ax.set_ylabel("Density")
ax.set_title("Earnings Distribution Around Zero")
ax.legend()
plt.tight_layout()
plt.show()
```



Figure 27.2: Distribution of scaled earnings around zero. A discontinuity—excess density just above zero and a deficit just below—would suggest benchmark-beating behavior. The red dashed line marks the zero threshold.

```
# Year-over-year changes
earn_changes = np.diff(earn_array.reshape(n_firms_dist, n_years_dist),
 ↪  axis=1).flatten()

fig, ax = plt.subplots(figsize=(8, 5))
bins_chg = np.arange(-0.15, 0.15, 0.005)
ax.hist(earn_changes, bins=bins_chg, color="darkorange", edgecolor="white",
        alpha=0.85, density=True)
ax.axvline(0, color="firebrick", linestyle="--", linewidth=1.5, label="Zero
 ↪  change")
ax.set_xlabel("Change in Earnings / Total Assets")
ax.set_ylabel("Density")
ax.set_title("Earnings Change Distribution Around Zero")
ax.legend()
plt.tight_layout()
plt.show()
```

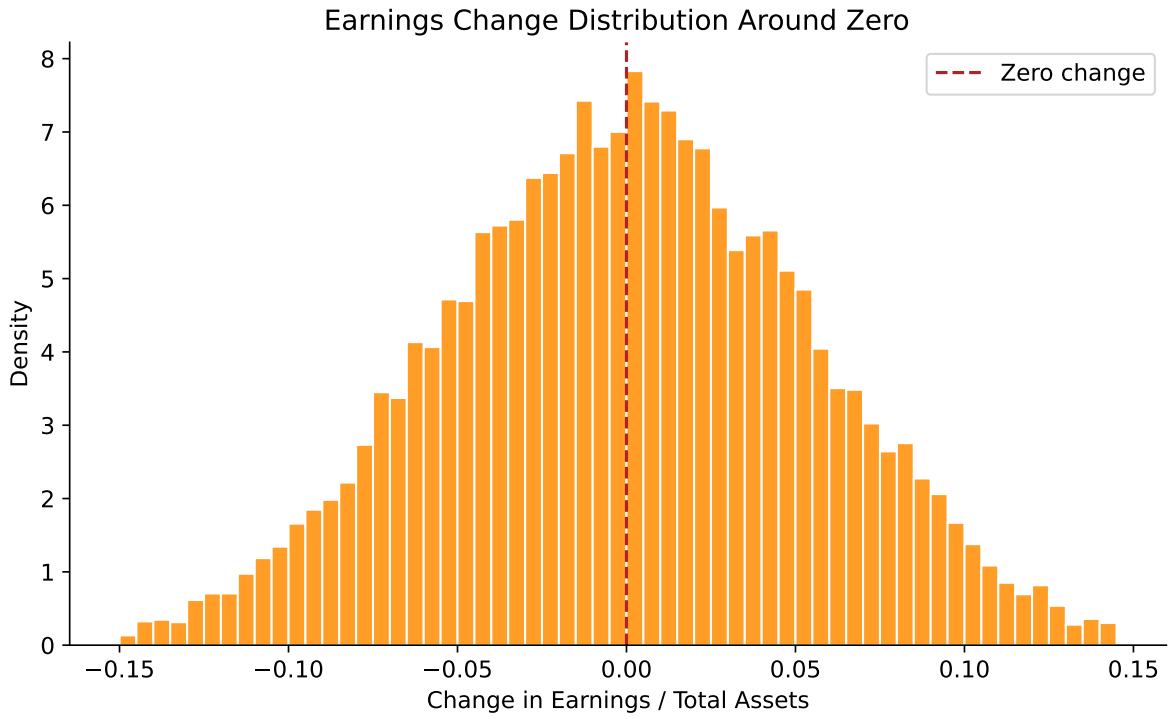


Figure 27.3: Distribution of year-over-year earnings changes around zero. A discontinuity here suggests that firms manage earnings to avoid reporting declines, even when small.

27.5.3 Accrual-Based vs. Real Earnings Management

D. A. Cohen, Dey, and Lys (2008) document a shift from AEM to REM following the passage of the Sarbanes-Oxley Act (SOX) in 2002, suggesting that tighter regulatory scrutiny redirects manipulation toward less detectable channels. Zang (2012) provides further evidence that firms trade off between AEM and REM based on their relative costs.

In Vietnam, where regulatory enforcement of accounting standards is weaker than in post-SOX America, we might expect AEM to remain the dominant channel. However, as Vietnam moves toward IFRS adoption and strengthens SSC oversight, the AEM-to-REM substitution hypothesis becomes testable.

We can measure REM using the Roychowdhury (2006) approach. Three proxies capture different manipulation channels:

Abnormal cash flow from operations. Estimate normal CFO as a function of sales and sales changes:

$$\frac{CFO_{i,t}}{A_{i,t-1}} = \beta_0 + \beta_1 \frac{1}{A_{i,t-1}} + \beta_2 \frac{S_{i,t}}{A_{i,t-1}} + \beta_3 \frac{\Delta S_{i,t}}{A_{i,t-1}} + \varepsilon_{i,t} \tag{27.9}$$

Abnormal production costs. Production costs = COGS + change in inventory. Normal levels are modeled as:

$$\frac{PROD_{i,t}}{A_{i,t-1}} = \beta_0 + \beta_1 \frac{1}{A_{i,t-1}} + \beta_2 \frac{S_{i,t}}{A_{i,t-1}} + \beta_3 \frac{\Delta S_{i,t}}{A_{i,t-1}} + \beta_4 \frac{\Delta S_{i,t-1}}{A_{i,t-1}} + \varepsilon_{i,t} \tag{27.10}$$

Abnormal discretionary expenditures. R&D + advertising + SGA, modeled as:

$$\frac{DISC_{i,t}}{A_{i,t-1}} = \beta_0 + \beta_1 \frac{1}{A_{i,t-1}} + \beta_2 \frac{S_{i,t-1}}{A_{i,t-1}} + \varepsilon_{i,t} \tag{27.11}$$

Residuals from these regressions serve as proxies for real manipulation. A firm that overproduces will show abnormally *high* production costs and abnormally *low* CFO (cash tied up in inventory).

27.6 Summary

This chapter examined the measurement and detection of earnings management, with a focus on methodological rigor and adaptation to Vietnam's institutional environment. The key takeaways are:

- Detecting earnings management requires separating discretionary from non-discretionary accruals, which depends critically on the quality of the non-discretionary accrual model. All five canonical models have known weaknesses.
- Under the null hypothesis of no manipulation, the Jones and Modified Jones models overreject when standard two-stage standard errors are used. The Salkever (1976) correction provides properly calibrated inference.
- When test firms experience extreme financial performance, all models exhibit severe size distortion, misattributing performance-driven accrual variation to managerial discretion. Performance matching (Sagar P. Kothari, Leone, and Wasley 2005) partially addresses this.
- Power analysis reveals that economically plausible levels of manipulation (below 10% of assets) are detected with very low probability. This casts doubt on studies that report null results as evidence of no manipulation.

- Vietnam's institutional features, such aSOE governance, VAS accounting rules, weak enforcement, and thin audit coverage—create a rich setting for earnings management research, but the methodological challenges are amplified by shorter time series and noisier data.

Researchers working with Vietnamese data should: (i) use the Salkever correction for proper inference, (ii) implement performance matching, (iii) consider multiple models and triangulate results, and (iv) supplement accrual-based approaches with real earnings management and distributional tests.

28 P/E Ratio

The systematic analysis of equity valuations in frontier and emerging markets requires a robust integration of high-fidelity financial data, rigorous mathematical modeling, and an acute understanding of local regulatory frameworks. In the context of the Vietnamese equity market, the Price-Earnings (P/E) ratio serves as a primary instrument for assessing corporate performance and market sentiment. This chapter provides an exploration of P/E valuation methodologies, ranging from firm-specific trailing and forward metrics to cyclically adjusted and unlevered variations.

28.0.1 Technical Schema and Variable Engineering

The primary identifier for equities is the symbol, typically a three-letter uppercase code, though special indices such as E1VFVN30 (ETF) or market indices like HNX-INDEX and UPCOM-INDEX follow unique naming conventions. Price data is categorized as OHLC type, encompassing 'open', 'high', 'low', and 'close'. For valuation purposes, the 'adjusted close' is preferred to account for corporate actions such as stock splits and dividend payments.

The accounting variables provided in the Vietnam Fundamentals product follow a standardized classification that reconciles different reporting standards across sectors. This reconciliation is vital because Vietnamese firms may have differing interpretations of revenue and net profit under Vietnamese Accounting Standards (VAS).

28.1 Firm-Specific P/E Valuation Metrics

At the individual firm level, the P/E ratio is the most fundamental measure of how much an investor is willing to pay for each unit of current or future profit. However, the calculation of "E" (Earnings) can take several forms, each offering a different insight into the company's value.

28.1.1 Trailing P/E and the TTM Methodology

The Trailing P/E ratio is calculated using the reported earnings from the last four quarters, known as Trailing Twelve Months (TTM). In the Vietnamese market, this is often the default metric reported by exchanges and financial news outlets.

The mathematical representation is:

$$P/E_{Trailing} = \frac{P_t}{\sum_{i=0}^{3} EPS_{q-i}}$$

Where P_t is the current market price and EPS_{q-i} represents the earnings per share for each of the four most recent quarters.

28.1.2 Forward P/E and Analyst Forecast Reliability

Forward P/E shifts the focus from historical performance to future expectations by using consensus earnings forecasts for the next fiscal year.

$$P/E_{Forward} = \frac{P_t}{EPS_{forecasted,t+1}}$$

In Vietnam, these forecasts are typically generated by research departments within major securities firms such as SSI, VCSC, MBS, and FPTS. These analysts rely on financial statements, projections, models, and subjective evaluations of market sentiment to generate their estimates. However, the accuracy of these forecasts is a significant area of research. However, analysts in Vietnam are often influenced by anchoring and adjustment bias, where they base future predictions heavily on past records and then make insufficient adjustments.

The accuracy of analyst earnings forecasts in Vietnam is significantly impacted by corporate governance characteristics. Specifically, state ownership has been found to have a negative impact on forecast accuracy, while institutional ownership has a positive effect. This implies that for firms with high state-controlled stakes, which are common in Vietnam's strategic industries, researchers should apply a higher discount or "fudge factor" to forward P/E estimates provided by consensus sources.

28.1.3 Cyclically Adjusted Price-Earnings (CAPE) Ratio

The Shiller P/E, or CAPE ratio, is designed to smooth out the volatility of the business cycle by using a 10-year average of inflation-adjusted earnings. For a market like Vietnam, which has seen periods of extreme growth followed by stabilization, the CAPE ratio provides a more tempered view of valuation than trailing metrics.

The calculation requires adjusting historical earnings by the Consumer Price Index (CPI). Vietnam's CPI data is updated monthly and is available from January 1996 through early 2026, with an average year-on-year growth rate of approximately 12.2%.

$$EPS_{real,t} = EPS_{nominal,t} \times \frac{CPI_{current}}{CPI_t}$$

$$CAPE = \frac{P_t}{\frac{1}{10}\sum_{i=0}^{9} EPS_{real,t-i}}$$

Historically, Vietnam's CPI has reached an all-time high of 28.3% YoY in August 2008 and a record low of -2.6% in July 2000. These extreme swings mean that nominal earnings in the mid-2000s are not comparable to earnings today without the Shiller adjustment. Researchers must be mindful of the different base years used by the National Statistics Office (GSO), such as 2024=100, 2019=100, and 2014=100.

28.1.4 Unlevered P/E and the Leibowitz Framework

The concept of the "Unlevered P/E" ratio attempts to view a company's valuation independent of its capital structure. This is particularly relevant in the Vietnamese market, where debt loads vary significantly between state-owned enterprises (SOEs) and private firms. Research by Leibowitz (2002) highlights that for the investment analyst, a company is already levered, and the task is to estimate its theoretical value by inferring the underlying structure of returns.

According to Leibowitz, leverage always moves the P/E toward a lower value than what would be obtained from a standard Gordon growth formula. As a company adds debt, the market discount rate for equity increases to compensate for the additional risk, which in turn compresses the P/E multiple. For example, a debt-free company with a theoretical P/E of 30 might see its P/E drop to 23 with a 40% debt ratio, and down to 20 with a 50% debt ratio.

The Unlevered P/E (often proxied by Enterprise Value to EBIT or EBITDA) can be calculated using fundamental variables:

$$EV = (\text{Shares Outstanding} \times \text{Price}) + \text{Total Debt} - \text{Cash}$$

$$P/E_{\text{Unlevered}} \approx \frac{EV}{EBIT \times (1 - \tau)}$$

Where τ is the corporate income tax rate. The standard CIT rate in Vietnam is 20%, which has been stable since the mid-2010s but is subject to new revisions under the Law on CIT ratified in June 2025.

28.2 Market-Level Aggregation Techniques

Aggregating firm-specific metrics into a single market-level index P/E is a complex task that requires careful consideration of weighting and the inclusion of loss-making entities. In the Vietnamese market, two primary indices (i.e., the VN-Index and the VN30), provide the benchmarks for performance and valuation.

28.2.1 The VN-Index: Capitalization-Weighted Aggregation

The VN-Index is a capitalization-weighted index of all companies listed on HOSE. It serves as a broad indicator of market health, where an increase in the index reflects a general rise in the stock prices of listed companies.

The index value is calculated as:

$$\text{VN-Index} = \frac{\sum(Price_i \times Shares_i)}{\text{Base Factor}} \times \text{Adjustment Factor}$$

Similarly, the index-level P/E is typically calculated as the sum of all market capitalizations divided by the sum of all net incomes (earnings) of the constituents.

$$P/E_{Market} = \frac{\sum MarketCap_i}{\sum Earnings_i}$$

As of early 2026, the total market capitalization for 701 tracked companies was approximately 8,629.8 trillion VND, with total earnings of 588.9 trillion VND, yielding a median P/E of approximately 14.7x.

28.2.2 The VN30 and Free-Float Adjustments

The VN30 Index represents a basket of the 30 largest and most liquid stocks on HOSE, screened through layers of capitalization, free-float ratio, and transactional volume. Unlike the broad VN-Index, the VN30 applies a free-float adjustment to its capitalization weighting to ensure that only shares available for public trading affect the index movement.

The free-float ratio (f) is calculated as:

$f = \frac{N - N_l}{N}$

Where N is the total number of outstanding shares and N_l is the number of limited trading shares (held by founders, state, or strategic partners).

When aggregating P/E for the VN30, researchers must account for this adjustment, as it more accurately reflects the valuation of the investable universe. In the Vietnamese market, a handful of large-cap stocks can significantly skew the broad index. For instance, Vingroup (VIC) and its related companies have at times represented nearly a quarter of the VN-Index's total weighting, creating a "closet indexing" dilemma for fund managers. Removing these high-impact stocks can reveal a much different valuation profile; while the VN-Index might trade at 13x, the market excluding VIC-related stocks could be closer to 11x.

28.2.3 Median vs. Mean Aggregation

Quantitative researchers often prefer median P/E ratios over mean P/E ratios to mitigate the influence of extreme outliers (i.e., companies with either exceptionally high P/E ratios due to temporarily low earnings or those that are technically "expensive" because of high growth expectations). Table 28.1 shows different aggregation methods.

Table 28.1: Median vs. Mean Aggregation

Aggregation Method	**Mathematical Definition**	**Resilience to Outliers**
Simple Mean	$\frac{1}{n}\sum(P/E)_i$	Low (highly skewed by extreme values)
Weighted Mean	$\frac{\sum(Cap_i \times (P/E)_i)}{\sum Cap_i}$	Medium (skewed by large-cap valuation)
Median	$\text{Median}((P/E)_1, \ldots, (P/E)_n)$	High (most robust for market sentiment)
Total-to-Total	$\frac{\sum MarketCap_i}{\sum Earnings_i}$	High (standard for index providers)

The Vietnamese market currently trades near its 3-year average P/E of 14.8x, suggesting that investors are relatively neutral on current valuations, expecting earnings to grow in line with historical rates.

28.3 Implementation

28.3.1 Data Ingestion and Processing

28.3.2 Shiller CAPE

The Shiller CAPE requires a more sophisticated approach, involving the merging of earnings data with monthly CPI series.

28.4 Macroeconomic and Regulatory Factors in Valuation

The P/E ratio does not exist in a vacuum; it is deeply influenced by the macroeconomic environment and the regulatory framework within which companies operate.

28.4.1 The Role of Corporate Income Tax (CIT)

The earnings component of the P/E ratio is a post-tax metric. In Vietnam, the standard CIT rate is 20%. However, there is a significant shift occurring with the ratification of the new Law on CIT in June 2025, taking effect on October 1, 2025. This law introduces:

- A 15% rate for enterprises with annual revenue not exceeding 3 billion VND.
- A 17% rate for enterprises with revenue between 3 and 50 billion VND.
- The maintenance of higher rates (up to 50%) for the oil, gas, and mineral resource sectors.

Furthermore, Vietnam has adopted the Global Minimum Tax (GMT) rules under Pillar Two, effective January 1, 2024, to protect its tax revenue from inbound and outbound investments. These changes mean that historical P/E ratios may not be directly comparable to future ratios for companies that previously benefited from generous tax incentives, as the effective tax rate is likely to rise for large multinational subsidiaries operating in Vietnam.

28.4.2 Macroeconomic Determinants: Inflation and Interest Rates

Stock price indices, and by extension P/E ratios, are heavily influenced by fundamental macroeconomic factors such as inflation, exchange rates, and interest rates. The interest rate serves as the cost of capital for enterprises. When interest rates fall, the cost of borrowing drops, increasing corporate profits and potentially raising the P/E that investors are willing to pay.

The Vietnamese dong (VND) exchange rate also plays a crucial role. In early 2025, the VND depreciated by 1.33% YTD, reflecting volatility in the timeline for Federal Reserve rate cuts. Exchange rate depreciation can be inflationary by default, which may prompt the State Bank of Vietnam to tighten monetary policy, thereby putting downward pressure on equity valuations.

28.5 Synthesis of Valuation Dynamics

The Vietnamese equity market is currently in a state of transition. While the market has seen robust growth—the VN-Index rose 38% in the year leading up to early 2026—investors remain neutral on overall valuations, as reflected by the market's alignment with its 3-year average P/E of 14.8x. This neutrality suggests that investors expect earnings growth (forecast at 14% annually) to keep pace with price appreciation.

The reliability of these valuations, however, remains dependent on the quality of non-financial disclosure. While companies are increasingly required by law to provide non-financial information, the level of transparency remains at a medium level (approximately 58.5% of required disclosures). Researchers must therefore balance quantitative P/E analysis with qualitative assessments of corporate governance and sustainability strategies. Companies that effectively integrate ESG principles have been shown to experience fewer negative impacts on abnormal stock returns during uncertain periods, such as the COVID-19 pandemic.

28.6 Conclusions

The exploration of P/E ratio valuation and aggregation within the Vietnamese equity market reveals a sophisticated landscape where traditional metrics must be adapted to local realities. The reliance on trailing metrics, while common, fails to account for the cyclicality and high inflation history of the Vietnamese economy, a gap that the Shiller CAPE ratio effectively bridges through CPI-adjusted normalization. Furthermore, the sensitivity of P/E to capital structure, as defined in the Leibowitz framework, highlights the necessity of unlevered valuation metrics in a market with diverse corporate funding models.

The aggregation of these metrics into market-level indices like the VN-Index and VN30 requires an awareness of concentration risks and the importance of free-float adjustments. The "Vingroup Effect" demonstrates how a single corporate ecosystem can skew index-wide valuations, necessitating a "closet indexing" awareness for fund managers and researchers.

Looking forward, the evolution of the regulatory environment—specifically the CIT law of 2025 and the Global Minimum Tax—will introduce new variables into the valuation equation. The shift toward higher-quality non-financial disclosures and the increasing accuracy of analyst forecasts in institutionalized firms suggest that the Vietnamese equity market is steadily maturing, offering a more transparent and predictable environment for the application of advanced quantitative valuation techniques.

29 Firm Valuation, Financial Distress, and Company Maturity

Understanding firm valuation, financial health, and corporate maturity is fundamental to financial analysis and investment decision-making. Three widely used measures (e.g., Tobin's Q, the Altman Z-Score, and company age) capture distinct but complementary dimensions of a firm's economic standing. Tobin's Q reflects the market's assessment of a firm's value relative to its asset base, the Altman Z-Score predicts the likelihood of financial distress, and company age proxies for organizational maturity and operational stability.

While these measures have been extensively studied in developed markets, particularly the United States (see Lindenberg and Ross 1981; Altman 1968; Gompers, Ishii, and Metrick 2003), their application in emerging markets like Vietnam presents unique challenges and opportunities. The Vietnamese stock market has grown rapidly but retains structural features, such as ownership concentration, state-owned enterprise dominance, limited bond market depth, and evolving accounting standards, which necessitate careful adaptation of standard valuation and distress metrics.

29.1 Required Packages

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from matplotlib.colors import LinearSegmentedColormap
import warnings

warnings.filterwarnings("ignore")
plt.rcParams.update({
    "figure.figsize": (10, 6),
    "font.size": 12,
    "axes.titlesize": 14,
    "axes.labelsize": 12,
```

```
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "legend.fontsize": 10,
    "figure.dpi": 150,
    "axes.spines.top": False,
    "axes.spines.right": False,
})

# Color palette consistent with Tidy Finance style
colors = {
    "primary": "#1f77b4",
    "secondary": "#ff7f0e",
    "tertiary": "#2ca02c",
    "quaternary": "#d62728",
    "quinary": "#9467bd",
    "safe": "#2ca02c",
    "alert": "#ff7f0e",
    "danger": "#d62728",
    "distress": "#8b0000",
}
```

30 Theoretical Foundations

30.1 Tobin's Q: Market Valuation of the Firm

30.1.1 The Original Concept

Tobin's Q was introduced by Tobin (1969) as a theoretical link between financial markets and real investment decisions. The fundamental idea is elegant: if the market values a firm's assets at more than their replacement cost, the firm has an incentive to invest in new capital; if the market values them at less, the firm should not invest, and may even benefit from selling assets.

Formally, Tobin's Q is defined as:

$$Q = \frac{\text{Market Value of the Firm}}{\text{Replacement Cost of Assets}} \tag{30.1}$$

When $Q > 1$, the market perceives the firm as possessing valuable intangible assets, such as brand equity, managerial talent, proprietary technology, or growth opportunities, that exceed the cost of its tangible asset base. When $Q < 1$, the market effectively values the firm at less than what it would cost to reassemble its assets, suggesting potential undervaluation or the presence of agency costs and organizational inefficiencies.

30.1.2 Market Value Decomposition

The numerator of Tobin's Q represents the total market value of all claims on the firm:

$$MV = CS + PS + ST + LT \tag{30.2}$$

where:

- CS = Market value of common stock (shares outstanding $\times$ market price)
- PS = Market value of preferred stock
- ST = Market value of short-term debt
- LT = Market value of long-term debt

30.1.3 Replacement Cost of Assets

The denominator captures the replacement cost of the firm's asset base:

$$RC = TA + (RNP - HNP) + (RINV - HINV) \tag{30.3}$$

where:

- TA = Total assets as reported
- RNP = Replacement cost of net plant and equipment
- HNP = Historical (book) value of net plant and equipment
- $RINV$ = Replacement cost of inventories
- $HINV$ = Historical (book) value of inventories

The replacement cost adjustments for plant and equipment involve recursive calculations that account for depreciation rates and, in more detailed estimations, technical progress rates (Lindenberg and Ross 1981). For inventories, the adjustment depends on the inventory accounting method: LIFO (Last In, First Out), FIFO (First In, First Out), average cost, or retail cost.

30.1.4 Simplified Tobin's Q

In practice, the full replacement cost computation requires data that are often unavailable, particularly in emerging markets. Gompers, Ishii, and Metrick (2003) popularized a simplified version:

$$Q_{\text{simple}} = \frac{TA + ME - BE}{TA} \tag{30.4}$$

where ME is the market value of equity and BE is the book value of equity. This formulation assumes that the book values of debt and preferred stock approximate their market values, and that total assets approximate the replacement cost of the firm's asset base. Despite its simplicity, this measure has been shown to correlate well with more elaborate constructions and has become the standard in empirical corporate finance research.

30.1.5 Chung and Pruitt Approximation

Chung and Pruitt (1994) proposed another widely used approximation:

$$Q_{\text{CP}} = \frac{ME + PS + DEBT}{TA} \tag{30.5}$$

where $DEBT$ = Current Liabilities − Current Assets + Book Value of Inventories + Long-term Debt. This formulation captures the net debt position more precisely than the Gompers, Ishii, and Metrick (2003) version.

30.2 Altman Z-Score: Predicting Financial Distress

30.2.1 The Original Model

The Altman Z-Score, developed by Altman (1968), is a multivariate discriminant analysis model that predicts the probability of corporate bankruptcy within a two-year horizon. The model was originally estimated using a matched sample of bankrupt and non-bankrupt U.S. manufacturing firms and takes the form:

$$Z = 3.3 \cdot X_1 + 0.999 \cdot X_2 + 0.6 \cdot X_3 + 1.2 \cdot X_4 + 1.4 \cdot X_5 \tag{30.6}$$

where the five financial ratios are in Table 30.1

Table 30.1: Components of the Altman Z-Score

Variable	Formula	Interpretation
X_1	$\frac{\text{EBIT}}{\text{Total Assets}}$	Earning power of assets
X_2	$\frac{\text{Net Sales}}{\text{Total Assets}}$	Total asset turnover
X_3	$\frac{\text{Market Value of Equity}}{\text{Total Liabilities}}$	Leverage ratio (inverse)
X_4	$\frac{\text{Working Capital}}{\text{Total Assets}}$	Short-term liquidity
X_5	$\frac{\text{Retained Earnings}}{\text{Total Assets}}$	Cumulative profitability

The interpretation zones are in Table 30.2

Table 30.2: Altman Z-Score interpretation zones

Z-Score Range	Interpretation
$Z > 2.99$	Safe zone-low probability of financial distress
$2.70 \leq Z \leq 2.99$	Grey zone-on alert, moderate risk
$1.80 \leq Z < 2.70$	Distress zone-significant bankruptcy risk within 2 years
$Z < 1.80$	High distress-very high probability of financial failure

30.2.2 The Z'-Score Model for Private Firms

Because the original model uses market value of equity in X_3, Altman and Hotchkiss (2010) developed the Z'-Score for private (non-publicly traded) firms by replacing market capitalization with book value of equity:

$$Z' = 0.717 \cdot X_1' + 0.847 \cdot X_2' + 3.107 \cdot X_3' + 0.420 \cdot X_4' + 0.998 \cdot X_5' \tag{30.7}$$

where $X_4' = \frac{\text{Book Value of Equity}}{\text{Total Liabilities}}$, and the remaining variables are defined as in the original model but with re-estimated coefficients.

30.2.3 The Z''-Score Model for Emerging Markets

Most relevant for Vietnam, Altman and Hotchkiss (2010) also developed the Z''-Score for non-manufacturing and emerging market firms:

$$Z'' = 3.25 + 6.56 \cdot X_1'' + 3.26 \cdot X_2'' + 6.72 \cdot X_3'' + 1.05 \cdot X_4'' \tag{30.8}$$

where:

- $X_1'' = \frac{\text{Working Capital}}{\text{Total Assets}}$
- $X_2'' = \frac{\text{Retained Earnings}}{\text{Total Assets}}$
- $X_3'' = \frac{\text{EBIT}}{\text{Total Assets}}$
- $X_4'' = \frac{\text{Book Value of Equity}}{\text{Total Liabilities}}$

Note that this model drops the sales/total assets ratio (X_2 in the original model) to minimize industry effects and adds an intercept. The classification zones shift accordingly (Table 30.3).

Table 30.3: Z''-Score interpretation zones for emerging markets

Z''-Score Range	Interpretation
$Z'' > 2.60$	Safe zone
$1.10 \leq Z'' \leq 2.60$	Grey zone
$Z'' < 1.10$	Distress zone

30.3 Company Age: Measuring Corporate Maturity

Company age captures the accumulated experience, reputation, and organizational capital of a firm. Older firms typically exhibit more stable operations, established competitive advantages, and predictable cash flow patterns (Coad et al. 2018). Age is commonly used as a control variable in corporate finance research, and its relationship with performance is theoretically ambiguous: older firms benefit from learning effects and reputation but may suffer from organizational rigidity and declining innovation (Huergo and Jaumandreu 2004).

The ideal measure is the number of years since founding. When founding dates are unavailable, common proxies include:

1. **Listing age**: Years since the first IPO or listing on a stock exchange.
2. **Data age**: Years since the first appearance in financial databases.
3. **Incorporation age**: Years since the date of legal incorporation.

In the Vietnamese context, we can use multiple proxies, including the date of first listing on HOSE or HNX, the first available financial statement date, and (when available) the founding date from company profiles.

31 The Vietnamese Market Context

31.1 Institutional Features Affecting Valuation Measures

The Vietnamese stock market has several institutional features that are important to consider when computing and interpreting Tobin's Q, Altman Z-Score, and company age.

31.1.1 State Ownership and Equitization

A significant proportion of Vietnamese listed firms are former state-owned enterprises (SOEs) that underwent equitization (cổ phần hóa). These firms often retain substantial state ownership, which can affect:

- **Tobin's Q**: State ownership may depress Q if markets perceive government influence as reducing efficiency, or it may elevate Q if state connections provide preferential access to resources and contracts.
- **Z-Score**: SOEs may have implicit government guarantees that reduce actual bankruptcy risk below what the Z-Score predicts.
- **Age**: The true operational age of equitized SOEs may far exceed their listing age, creating measurement challenges.

31.1.2 Foreign Ownership Limits

Vietnam imposes foreign ownership limits (FOL) on listed companies, typically capped at 49% for most sectors (with some exceptions). This constraint can create price premiums for stocks approaching the FOL ceiling, potentially inflating Tobin's Q for these firms (X. V. Vo 2015).

31.1.3 Accounting Standards

Vietnamese Accounting Standards (VAS) differ from International Financial Reporting Standards (IFRS) in several ways relevant to our measures:

- **Historical cost basis**: VAS relies more heavily on historical cost, which can cause book values to diverge substantially from replacement costs, affecting both Q and Z-Score calculations.

- **Limited fair value measurement**: Unlike IFRS, VAS limits fair value measurement for many asset classes, making book-value-based proxies less reliable.
- **Inventory methods**: Vietnamese firms use various inventory valuation methods (FIFO, weighted average), which affect the book value of inventories and hence the Z-Score's working capital component.

31.1.4 Market Microstructure

Vietnam's market features daily price limits (currently $\pm$ 7% on HOSE, $\pm$ 10% on HNX, $\pm$ 15% on UPCoM), which can prevent prices from reaching equilibrium values quickly. This means that market capitalization at any point may not fully reflect available information, introducing noise into market-value-based measures like Tobin's Q.

32 Data Preparation

32.1 Loading and Cleaning Financial Statement Data

We begin by loading the annual financial statement data and constructing the variables needed for our three measures.

```
# ============================================================================
# In practice, replace this section with actual DataCore.vn API calls:
#
#   from datacore import DataCoreClient
#   client = DataCoreClient(api_key="your_api_key")
#   financials = client.get_financial_statements(
#       frequency="annual",
#       start_date="2005-01-01",
#       end_date="2024-12-31"
#   )
#   market = client.get_market_data(frequency="daily")
#   profiles = client.get_company_profiles()
#
# For demonstration purposes, we simulate realistic Vietnamese market data.
# ============================================================================

np.random.seed(42)

n_firms = 300
years = range(2008, 2025)
exchanges = ["HOSE", "HNX", "UPCoM"]
industries = [
    "Bất động sản", "Ngân hàng", "Thực phẩm & Đồ uống",
    "Xây dựng & Vật liệu", "Công nghệ thông tin", "Bán lẻ",
    "Dầu khí", "Thép", "Dệt may", "Dược phẩm",
    "Điện lực", "Vận tải & Logistics", "Hóa chất",
    "Chứng khoán", "Bảo hiểm"
```

```
]

# Generate firm profiles
firm_ids = [f"VN{str(i).zfill(4)}" for i in range(1, n_firms + 1)]
firm_profiles = pd.DataFrame({
    "ticker": firm_ids,
    "exchange": np.random.choice(exchanges, n_firms, p=[0.5, 0.3, 0.2]),
    "industry": np.random.choice(industries, n_firms),
    "founding_year": np.random.randint(1975, 2015, n_firms),
    "listing_year": np.random.randint(2000, 2020, n_firms),
    "state_ownership_pct": np.clip(
        np.random.beta(2, 5, n_firms) * 100, 0, 75
    ).round(1),
})
firm_profiles["listing_year"] = np.maximum(
    firm_profiles["listing_year"],
    firm_profiles["founding_year"] + 1
)

# Generate panel data
records = []
for _, firm in firm_profiles.iterrows():
    start_year = max(firm["listing_year"], 2008)
    # Base financial characteristics (firm fixed effects)
    base_ta = np.exp(np.random.normal(14, 1.5))  # Total assets in VND
 ↪  millions
    base_profitability = np.random.normal(0.08, 0.05)
    base_leverage = np.random.beta(3, 3)
    base_turnover = np.random.gamma(2, 0.4)

    for year in years:
        if year < start_year:
            continue
        if np.random.random() < 0.02:  # 2% chance of delisting
            break

        # Time-varying components with persistence
        shock = np.random.normal(0, 0.15)
        growth = np.random.normal(0.08, 0.05)

        ta = base_ta * (1 + growth) ** (year - start_year) * np.exp(shock)
        profitability = np.clip(base_profitability + np.random.normal(0,
 ↪  0.03), -0.3, 0.5)
```

```
        leverage = np.clip(base_leverage + np.random.normal(0, 0.05), 0.05,
↪  0.95)

        lt = ta * leverage * np.random.uniform(0.4, 0.7)
        total_liabilities = ta * leverage
        ct_liabilities = total_liabilities - lt

        seq = ta * (1 - leverage)
        sale = ta * np.clip(base_turnover + np.random.normal(0, 0.1), 0.1, 5)
        ebit = ta * profitability
        ni = ebit * np.random.uniform(0.6, 0.9)
        re = seq * np.random.uniform(-0.2, 0.8)
        act = ta * np.random.uniform(0.2, 0.7)
        lct = ct_liabilities

        # Market value with noise and sentiment
        market_premium = np.random.lognormal(0, 0.4)
        me = seq * market_premium
        prcc = me / max(np.random.uniform(50, 500), 1)
        csho = me / max(prcc, 0.01)

        # Preferred stock (rare in Vietnam, mostly zero)
        pstk = 0 if np.random.random() > 0.05 else seq * np.random.uniform(0,
↪  0.1)

        # Net plant and equipment
        ppent = ta * np.random.uniform(0.1, 0.6)
        invt = ta * np.random.uniform(0.05, 0.3)

        # Deferred taxes and investment tax credit (typically small in VN)
        txdb = ta * np.random.uniform(0, 0.02)
        itcb = 0

        records.append({
            "ticker": firm["ticker"],
            "year": year,
            "datadate": pd.Timestamp(year, 12, 31),
            "at": ta,             # Total Assets
            "seq": seq,           # Stockholders' Equity
            "lt": lt,             # Long-term Debt
            "lct": lct,           # Current Liabilities
            "tlb": total_liabilities,  # Total Liabilities
            "sale": sale,         # Net Sales/Revenue
```

```
            "ebit": ebit,         # EBIT
            "ni": ni,             # Net Income
            "re_var": re,         # Retained Earnings
            "act": act,           # Current Assets
            "ppent": ppent,       # Net Plant & Equipment
            "invt": invt,         # Inventories
            "txdb": txdb,         # Deferred Taxes
            "itcb": itcb,         # Investment Tax Credit
            "pstk": pstk,         # Preferred Stock
            "me": me,             # Market Value of Equity
            "prcc": prcc,         # Price at Calendar Year End
            "csho": csho,         # Shares Outstanding
        })

df = pd.DataFrame(records)
df = df.merge(firm_profiles[["ticker", "exchange", "industry",
                             "founding_year", "listing_year",
                             "state_ownership_pct"]],
              on="ticker", how="left")

print(f"Panel dimensions: {df.shape[0]:,} firm-year observations")
print(f"Number of unique firms: {df['ticker'].nunique()}")
print(f"Year range: {df['year'].min()}-{df['year'].max()}")
print(f"Exchanges: {df['exchange'].value_counts().to_dict()}")
```

```
Panel dimensions: 3,484 firm-year observations
Number of unique firms: 297
Year range: 2008-2024
Exchanges: {'HOSE': 1701, 'HNX': 1118, 'UPCoM': 665}
```

32.2 Variable Definitions and Mapping

Table 32.1 provides the mapping between standard variable names and the corresponding fields.

```
variable_map = pd.DataFrame({
    "Compustat Variable": [
        "AT", "SEQ", "LT", "LCT", "SALE", "EBIT", "NI",
        "RE", "ACT", "PPENT", "INVT", "TXDB", "ITCB",
        "PSTK/PSTKRV/PSTKL", "PRCC_C", "CSHO", "DLDTE", "DLRSN"
```

```
    ],
    "Description": [
        "Total Assets", "Stockholders' Equity", "Long-term Debt",
        "Current Liabilities", "Net Sales/Revenue",
        "Earnings Before Interest & Taxes", "Net Income",
        "Retained Earnings", "Current Assets",
        "Net Plant & Equipment", "Total Inventories",
        "Deferred Taxes", "Investment Tax Credit",
        "Preferred Stock (various)", "Price Close (Calendar Year)",
        "Common Shares Outstanding", "Delisting Date",
        "Delisting Reason"
    ],
    "DataCore.vn Equivalent": [
        "tong_tai_san", "von_chu_so_huu", "no_dai_han",
        "no_ngan_han", "doanh_thu_thuan",
        "loi_nhuan_truoc_thue_va_lai_vay", "loi_nhuan_sau_thue",
        "loi_nhuan_chua_phan_phoi", "tai_san_ngan_han",
        "tai_san_co_dinh_huu_hinh", "hang_ton_kho",
        "thue_thu_nhap_hoan_lai", "N/A (not applicable in VAS)",
        "co_phieu_uu_dai", "gia_dong_cua_cuoi_nam",
        "so_luong_co_phieu_luu_hanh", "ngay_huy_niem_yet",
        "ly_do_huy_niem_yet"
    ],
    "VAS Account": [
        "BS.100", "BS.400", "BS.330", "BS.310",
        "IS.10", "Computed", "IS.60",
        "BS.421", "BS.100", "BS.221", "BS.141",
        "BS.262", "-", "BS.411b", "Market", "Market",
        "Profile", "Profile"
    ]
})

variable_map.style.set_properties(**{
    "text-align": "left",
    "font-size": "10pt"
}).set_table_styles([
    {"selector": "th", "props": [
        ("background-color", "#1f77b4"),
        ("color", "white"),
        ("font-weight", "bold"),
        ("text-align", "left"),
        ("padding", "8px")
    ]},
```

```
    {"selector": "td", "props": [("padding", "6px")]},
])
```

Table 32.1: Variable mapping

Table 32.1

	Compustat Variable	Description	DataCore.vn Equivalent	VA
0	AT	Total Assets	tong_tai_san	BS
1	SEQ	Stockholders' Equity	von_chu_so_huu	BS
2	LT	Long-term Debt	no_dai_han	BS
3	LCT	Current Liabilities	no_ngan_han	BS
4	SALE	Net Sales/Revenue	doanh_thu_thuan	IS.
5	EBIT	Earnings Before Interest & Taxes	loi_nhuan_truoc_thue_va_lai_vay	Co
6	NI	Net Income	loi_nhuan_sau_thue	IS.
7	RE	Retained Earnings	loi_nhuan_chua_phan_phoi	BS
8	ACT	Current Assets	tai_san_ngan_han	BS
9	PPENT	Net Plant & Equipment	tai_san_co_dinh_huu_hinh	BS
10	INVT	Total Inventories	hang_ton_kho	BS
11	TXDB	Deferred Taxes	thue_thu_nhap_hoan_lai	BS
12	ITCB	Investment Tax Credit	N/A (not applicable in VAS)	—
13	PSTK/PSTKRV/PSTKL	Preferred Stock (various)	co_phieu_uu_dai	BS
14	PRCC_C	Price Close (Calendar Year)	gia_dong_cua_cuoi_nam	Ma
15	CSHO	Common Shares Outstanding	so_luong_co_phieu_luu_hanh	Ma
16	DLDTE	Delisting Date	ngay_huy_niem_yet	Pro
17	DLRSN	Delisting Reason	ly_do_huy_niem_yet	Pro

32.3 Data Quality Checks

Before computing any measures, we perform essential data quality checks that are particularly important for Vietnamese data.

```
def data_quality_report(df):
    """Generate a comprehensive data quality report."""
    report = {}

    # Check for negative total assets
    report["Negative total assets"] = (df["at"] < 0).sum()
```

```
    # Check for negative equity (acceptable but noteworthy)
    report["Negative equity"] = (df["seq"] <= 0).sum()

    # Check for missing critical variables
    critical_vars = ["at", "seq", "sale", "ebit", "me"]
    for var in critical_vars:
        report[f"Missing {var}"] = df[var].isna().sum()

    # Check for extreme values (potential data errors)
    report["Extreme leverage (>100%)"] = (df["tlb"] / df["at"] > 1.0).sum()
    report["Extreme sales/assets (>10)"] = (df["sale"] / df["at"] > 10).sum()

    return pd.Series(report, name="Count")

quality = data_quality_report(df)
print("Data Quality Report")
print("=" * 45)
for item, count in quality.items():
    status = " " if count == 0 else " "
    print(f"  {status} {item}: {count:,}")

# Apply filters
df_clean = df.copy()
df_clean = df_clean[df_clean["at"] > 0]  # Positive total assets
df_clean = df_clean[df_clean["seq"] > 0]  # Positive equity (for BE
 ↪  calculation)
print(f"\nObservations after cleaning: {len(df_clean):,} "
      f"(dropped {len(df) - len(df_clean):,})")
```

```
Data Quality Report
=============================================
    Negative total assets: 0
    Negative equity: 0
    Missing at: 0
    Missing seq: 0
    Missing sale: 0
    Missing ebit: 0
    Missing me: 0
    Extreme leverage (>100%): 0
    Extreme sales/assets (>10): 0

Observations after cleaning: 3,484 (dropped 0)
```

33 Computing Tobin's Q

33.1 Book Value of Equity

Following Daniel and Titman (1997), we compute the book value of equity as:

$$BE = SEQ + TXDB + ITCB - PREF \tag{33.1}$$

where $PREF$ is the preferred stock value, using the redemption value if available, otherwise the liquidating value, and finally the carrying value as a last resort. In the Vietnamese context, preferred stock is relatively rare among listed companies, so $PREF$ is often zero.

```
def compute_book_equity(df):
    """
    Compute book value of equity following Daniel and Titman (1997).

    In Vietnam, preferred stock is uncommon among listed firms.
    The investment tax credit (ITCB) is not applicable under VAS,
    so we set it to zero when missing.
    """
    result = df.copy()

    # Preferred stock: use coalesce logic
    result["pref"] = result["pstk"].fillna(0)

    # Book equity = Shareholders' equity + Deferred taxes + ITC - Preferred
    result["be"] = (
        result["seq"]
        + result["txdb"].fillna(0)
        + result["itcb"].fillna(0)
        - result["pref"]
    )

    return result

df_clean = compute_book_equity(df_clean)
```

```
print("Book Equity Summary Statistics (VND millions)")
print(df_clean["be"].describe().apply(lambda x: f"{x:,.0f}"))
```

```
Book Equity Summary Statistics (VND millions)
count          3,484
mean       3,521,607
std        9,055,435
min            4,633
25%          386,441
50%        1,112,062
75%        3,210,269
max      247,845,397
Name: be, dtype: str
```

33.2 Market Value of Equity

The market value of equity is computed using the calendar year-end stock price and shares outstanding:

$$ME = P_{\text{close}} \times \text{CSHO} \tag{33.2}$$

For Vietnamese firms, this is obtained from the closing price on the last trading day of the fiscal year. Most Vietnamese firms have a December 31 fiscal year end, though some (particularly in agriculture and banking) may differ.

```
# ME is already computed in our simulated data as prcc * csho
# In practice with DataCore.vn:
#   me = df["gia_dong_cua_cuoi_nam"] * df["so_luong_co_phieu_luu_hanh"]

df_clean["me"] = df_clean["prcc"] * df_clean["csho"]

print("Market Equity Summary Statistics (VND millions)")
print(df_clean["me"].describe().apply(lambda x: f"{x:,.0f}"))
```

```
Market Equity Summary Statistics (VND millions)
count          3,484
mean       3,653,445
std        9,747,318
```

```
min              3,379
25%            370,119
50%          1,106,646
75%          3,029,093
max        286,258,816
Name: me, dtype: str
```

33.3 Simplified Tobin's Q

We implement the Gompers, Ishii, and Metrick (2003) simplified version:

$$Q_{\text{simple}} = \frac{AT + ME - BE}{AT} \tag{33.3}$$

This can be rewritten as:

$$Q_{\text{simple}} = 1 + \frac{ME - BE}{AT} \tag{33.4}$$

which makes the interpretation clear: Tobin's Q equals one plus the market-to-book premium (or discount) scaled by total assets.

```python
def compute_tobins_q(df):
    """
    Compute multiple variants of Tobin's Q.

    Variants:
    1. Simple Q (Gompers et al., 2003)
    2. Chung-Pruitt Q
    3. Market-to-Book ratio (for comparison)
    """
    result = df.copy()

    # --- Variant 1: Simple Q (Gompers, Ishii, Metrick 2003) ---
    result["tobin_q_simple"] = (result["at"] + result["me"] - result["be"])
    ↪ / result["at"]

    # --- Variant 2: Chung-Pruitt Approximation ---
    # DEBT = Current Liabilities - Current Assets + Inventories + LT Debt
    result["debt_cp"] = (
        result["lct"].fillna(0)
        - result["act"].fillna(0)
```

```
        + result["invt"].fillna(0)
        + result["lt"].fillna(0)
    )
    result["tobin_q_cp"] = (
        (result["me"] + result["pstk"].fillna(0) + result["debt_cp"]) /
↪   result["at"]
    )

    # --- Market-to-Book Ratio ---
    result["mtb"] = np.where(result["be"] > 0, result["me"] / result["be"],
↪   np.nan)

    return result

df_clean = compute_tobins_q(df_clean)

# Summary statistics
q_vars = ["tobin_q_simple", "tobin_q_cp", "mtb"]
q_summary = df_clean[q_vars].describe(percentiles=[0.01, 0.05, 0.25, 0.5,
↪   0.75, 0.95, 0.99])
q_summary = q_summary.round(3)
q_summary.columns = ["Simple Q", "Chung-Pruitt Q", "Market-to-Book"]

print("Tobin's Q Summary Statistics")
print(q_summary.to_string())
```

```
Tobin's Q Summary Statistics
       Simple Q  Chung-Pruitt Q  Market-to-Book
count  3484.000        3484.000        3484.000
mean      1.031           0.766           1.058
std       0.243           0.296           0.445
min       0.357           0.008           0.233
1%        0.602           0.189           0.382
5%        0.713           0.332           0.496
25%       0.886           0.568           0.742
50%       0.989           0.738           0.974
75%       1.132           0.926           1.287
95%       1.481           1.290           1.904
99%       1.916           1.668           2.492
max       2.804           2.433           3.189
```

33.4 Winsorizing Extreme Values

Tobin's Q values can be heavily influenced by outliers, particularly in emerging markets where data quality may be inconsistent. We winsorize at the 1st and 99th percentiles.

```
def winsorize(series, lower=0.01, upper=0.99):
    """Winsorize a pandas Series at specified percentiles."""
    low = series.quantile(lower)
    high = series.quantile(upper)
    return series.clip(lower=low, upper=high)

# Winsorize Q measures
for var in ["tobin_q_simple", "tobin_q_cp", "mtb"]:
    df_clean[f"{var}_w"] = winsorize(df_clean[var])

print("Effect of Winsorization on Simple Tobin's Q:")
print(f"  Before: mean={df_clean['tobin_q_simple'].mean():.3f}, "
      f"std={df_clean['tobin_q_simple'].std():.3f}")
print(f"  After:  mean={df_clean['tobin_q_simple_w'].mean():.3f}, "
      f"std={df_clean['tobin_q_simple_w'].std():.3f}")
```

```
Effect of Winsorization on Simple Tobin's Q:
  Before: mean=1.031, std=0.243
  After:  mean=1.030, std=0.233
```

33.5 Cross-Sectional Distribution of Tobin's Q

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Latest year distribution
latest_year = df_clean["year"].max()
latest_data = df_clean[df_clean["year"] == latest_year]

# Histogram
axes[0].hist(
    latest_data["tobin_q_simple_w"].dropna(),
    bins=50, color=colors["primary"], alpha=0.7, edgecolor="white"
)
axes[0].axvline(x=1, color=colors["quaternary"], linestyle="--", linewidth=2,
 ↪  label="Q = 1")
```

```
axes[0].set_xlabel("Tobin's Q (Simple)")
axes[0].set_ylabel("Number of Firms")
axes[0].set_title(f"Cross-Sectional Distribution ({latest_year})")
axes[0].legend()

# Box plot by exchange
exchange_data = latest_data[["exchange", "tobin_q_simple_w"]].dropna()
exchanges_sorted = exchange_data.groupby("exchange")["tobin_q_simple_w"].med
 ↪  ian().sort_values().index

box_data = [exchange_data[exchange_data["exchange"] ==
 ↪  ex]["tobin_q_simple_w"].values
            for ex in exchanges_sorted]

bp = axes[1].boxplot(box_data, labels=exchanges_sorted, patch_artist=True,
                     medianprops=dict(color="black", linewidth=2))
box_colors = [colors["primary"], colors["secondary"], colors["tertiary"]]
for patch, color in zip(bp["boxes"], box_colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)

axes[1].axhline(y=1, color=colors["quaternary"], linestyle="--",
 ↪  linewidth=1.5, alpha=0.7)
axes[1].set_ylabel("Tobin's Q (Simple)")
axes[1].set_title(f"Tobin's Q by Exchange ({latest_year})")

plt.tight_layout()
plt.show()
```

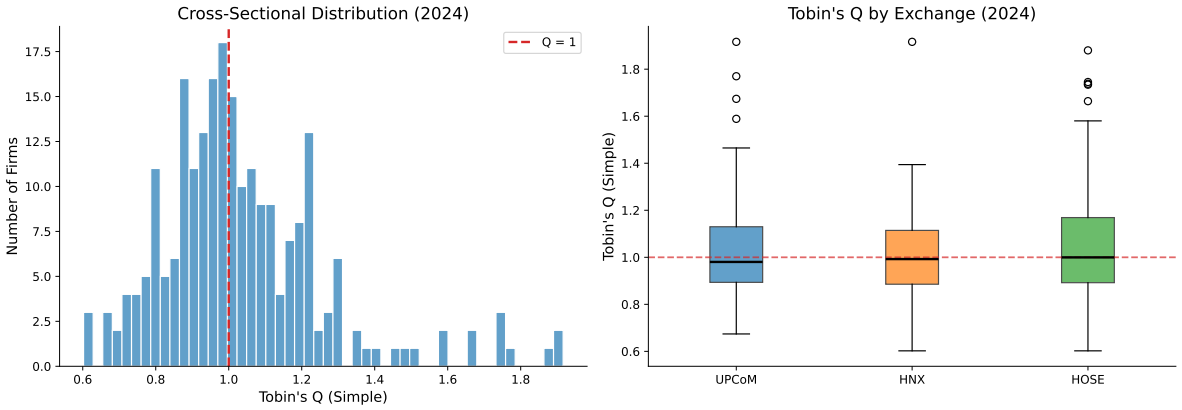


Figure 33.1: Distribution of Tobin's Q across Vietnamese listed firms (2024). The vertical dashed line at Q=1 indicates the theoretical threshold where market value equals replacement cost.

33.6 Time-Series Evolution of Tobin's Q

```
# Compute annual statistics by exchange
annual_q = (
    df_clean
    .groupby(["year", "exchange"])["tobin_q_simple_w"]
    .agg(["median", lambda x: x.quantile(0.25), lambda x: x.quantile(0.75)])
    .reset_index()
)
annual_q.columns = ["year", "exchange", "median", "q25", "q75"]

fig, ax = plt.subplots(figsize=(12, 6))

exchange_colors = {"HOSE": colors["primary"], "HNX": colors["secondary"],
                   "UPCoM": colors["tertiary"]}

for exchange, color in exchange_colors.items():
    data = annual_q[annual_q["exchange"] == exchange]
    ax.plot(data["year"], data["median"], color=color, linewidth=2.5,
            label=exchange, marker="o", markersize=4)
    ax.fill_between(data["year"], data["q25"], data["q75"],
                    color=color, alpha=0.15)

ax.axhline(y=1, color="gray", linestyle="--", linewidth=1, alpha=0.7,
 ↪  label="Q = 1")
```

```
ax.set_xlabel("Year")
ax.set_ylabel("Tobin's Q (Median)")
ax.set_title("Evolution of Tobin's Q in the Vietnamese Market")
ax.legend(loc="upper right")
ax.set_xlim(2008, latest_year)
ax.xaxis.set_major_locator(mticker.MultipleLocator(2))

plt.tight_layout()
plt.show()
```

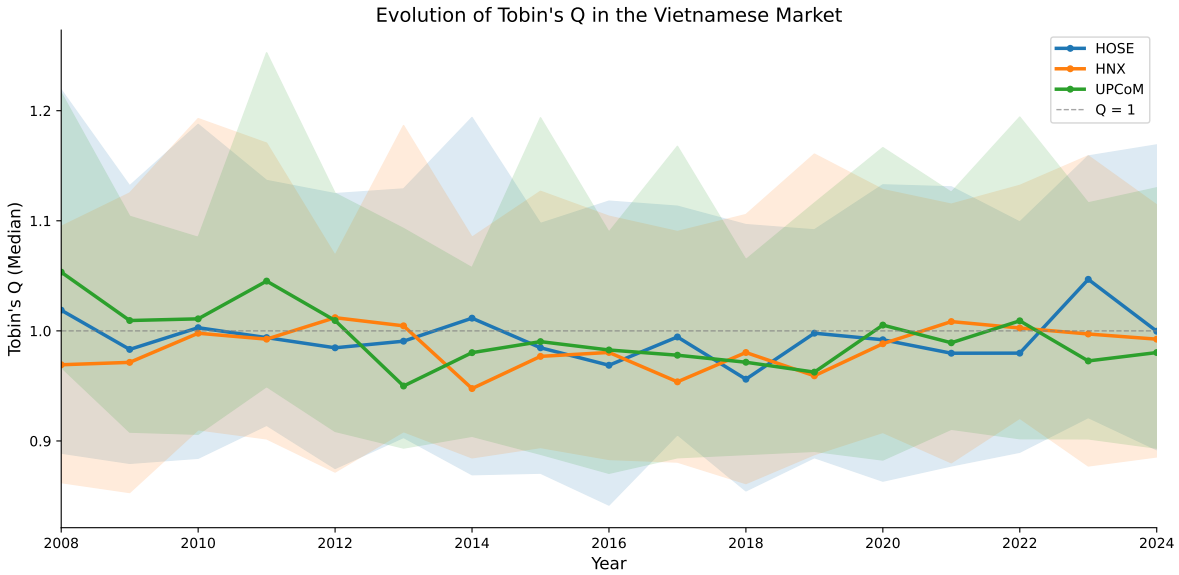


Figure 33.2: Evolution of median Tobin's Q across Vietnamese exchanges (2008–2024). Shaded areas represent the interquartile range.

33.7 Tobin's Q by Industry

```
latest_industry_q = (
    latest_data
    .groupby("industry")["tobin_q_simple_w"]
    .agg(["median", "mean", "count"])
    .reset_index()
    .sort_values("median", ascending=True)
)
```

```
fig, ax = plt.subplots(figsize=(10, 8))

bar_colors = [colors["quaternary"] if m < 1 else colors["primary"]
              for m in latest_industry_q["median"]]

ax.barh(
    latest_industry_q["industry"],
    latest_industry_q["median"],
    color=bar_colors, alpha=0.8, edgecolor="white"
)
ax.axvline(x=1, color="gray", linestyle="--", linewidth=1.5, alpha=0.7)
ax.set_xlabel("Median Tobin's Q")
ax.set_title(f"Tobin's Q by Industry ({latest_year})")

# Add count annotations
for i, (_, row) in enumerate(latest_industry_q.iterrows()):
    ax.text(row["median"] + 0.02, i, f'n={int(row["count"])}',
            va="center", fontsize=9, color="gray")

plt.tight_layout()
plt.show()
```

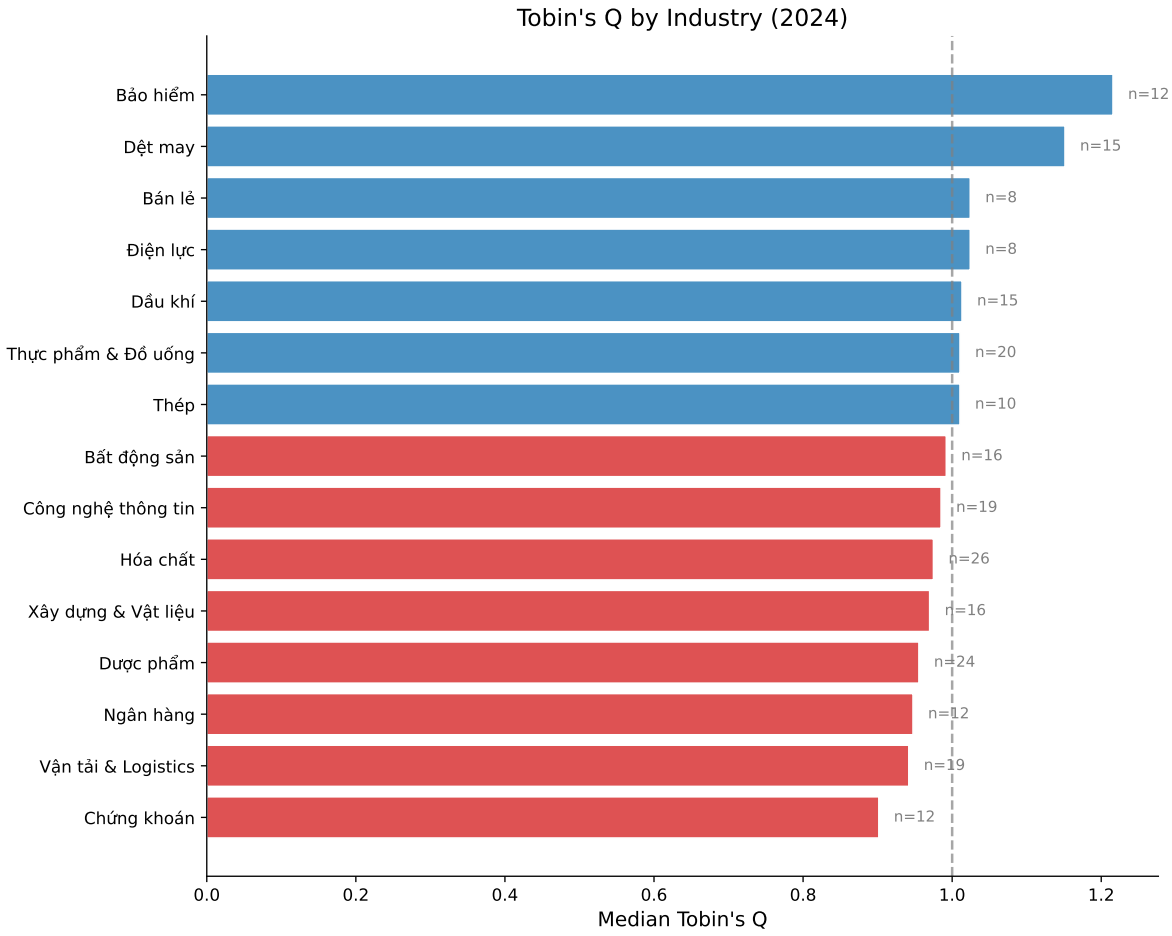


Figure 33.3: Median Tobin's Q by industry in Vietnam (latest available year). Industries are sorted by median Q value.

34 Computing the Altman Z-Score

34.1 Original Z-Score for Listed Firms

```
def compute_altman_z(df):
    """
    Compute three variants of the Altman Z-Score:
    1. Original Z-Score (for listed manufacturing firms)
    2. Z'-Score (for private firms, using book equity)
    3. Z''-Score (for emerging markets / non-manufacturing)
    """
    result = df.copy()

    # Common components
    result["wc"] = result["act"].fillna(0) - result["lct"].fillna(0)  #
↪   Working Capital

    # --- Original Z-Score ---
    # Z = 3.3*(EBIT/TA) + 0.999*(Sales/TA) + 0.6*(ME/TL) + 1.2*(WC/TA) +
     ↪  1.4*(RE/TA)
    x1 = result["ebit"] / result["at"]
    x2 = result["sale"] / result["at"]
    x3 = np.where(result["tlb"] > 0, result["me"] / result["tlb"], np.nan)
    x4 = result["wc"] / result["at"]
    x5 = result["re_var"].fillna(0) / result["at"]

    result["z_x1"] = x1  # Earning power
    result["z_x2"] = x2  # Asset turnover
    result["z_x3"] = x3  # Leverage (inverse)
    result["z_x4"] = x4  # Liquidity
    result["z_x5"] = x5  # Cumulative profitability

    result["altman_z"] = (3.3 * x1 + 0.999 * x2 + 0.6 * x3 + 1.2 * x4 + 1.4
↪   * x5)
```

```
    # --- Z'-Score (Private firms) ---
    x3_prime = np.where(result["tlb"] > 0, result["be"] / result["tlb"],
↪   np.nan)
    result["altman_z_prime"] = (
        0.717 * x4 + 0.847 * x5 + 3.107 * x1 + 0.420 * x3_prime + 0.998 * x2
    )

    # --- Z''-Score (Emerging Markets) ---
    result["altman_z_em"] = (
        3.25
        + 6.56 * x4   # Working Capital / TA
        + 3.26 * x5   # Retained Earnings / TA
        + 6.72 * x1   # EBIT / TA
        + 1.05 * x3_prime  # Book Equity / TL
    )

    # Classify risk zones
    result["z_zone"] = pd.cut(
        result["altman_z"],
        bins=[-np.inf, 1.80, 2.70, 2.99, np.inf],
        labels=["High Distress", "Distress", "Grey Zone", "Safe"]
    )

    result["z_em_zone"] = pd.cut(
        result["altman_z_em"],
        bins=[-np.inf, 1.10, 2.60, np.inf],
        labels=["Distress", "Grey Zone", "Safe"]
    )

    return result

df_clean = compute_altman_z(df_clean)

# Winsorize Z-scores
for var in ["altman_z", "altman_z_prime", "altman_z_em"]:
    df_clean[f"{var}_w"] = winsorize(df_clean[var])

print("Z-Score Summary Statistics")
z_summary = df_clean[["altman_z", "altman_z_prime",
↪   "altman_z_em"]].describe().round(3)
z_summary.columns = ["Original Z", "Z' (Private)", "Z'' (Emerging)"]
print(z_summary.to_string())
```

```
Z-Score Summary Statistics
       Original Z  Z' (Private)  Z'' (Emerging)
count    3484.000      3484.000        3484.000
mean        2.610         2.042           7.467
std         1.896         1.194           3.137
min         0.059         0.155           1.578
25%         1.595         1.314           5.670
50%         2.212         1.806           6.987
75%         3.028         2.410           8.549
max        28.640        11.119          29.686
```

34.2 Z-Score Component Analysis

Understanding which components drive the Z-Score is particularly informative in the Vietnamese context, where certain ratios may behave differently than in developed markets.

```
fig, axes = plt.subplots(2, 3, figsize=(15, 9))

components = [
    ("z_x1", 3.3, "$3.3 \\times$ EBIT/TA\n(Earning Power)"),
    ("z_x2", 0.999, "$0.999 \\times$ Sales/TA\n(Asset Turnover)"),
    ("z_x3", 0.6, "$0.6 \\times$ ME/TL\n(Leverage)"),
    ("z_x4", 1.2, "$1.2 \\times$ WC/TA\n(Liquidity)"),
    ("z_x5", 1.4, "$1.4 \\times$ RE/TA\n(Cum. Profitability)"),
]

latest_z = df_clean[df_clean["year"] == latest_year]

for idx, (var, weight, label) in enumerate(components):
    ax = axes[idx // 3, idx % 3]
    weighted_vals = (latest_z[var] * weight).dropna()
    weighted_vals = weighted_vals[weighted_vals.between(
        weighted_vals.quantile(0.01), weighted_vals.quantile(0.99)
    )]
    ax.hist(weighted_vals, bins=40, color=colors["primary"], alpha=0.7,
↪   edgecolor="white")
    ax.axvline(x=weighted_vals.median(), color=colors["quaternary"],
               linestyle="--", linewidth=2, label=f"Median:
↪   {weighted_vals.median():.2f}")
    ax.set_title(label, fontsize=11)
    ax.legend(fontsize=9)
```

```
# Remove empty subplot
axes[1, 2].set_visible(False)

plt.suptitle(f"Z-Score Component Distributions ({latest_year})", fontsize=14,
 ↪  y=1.02)
plt.tight_layout()
plt.show()
```

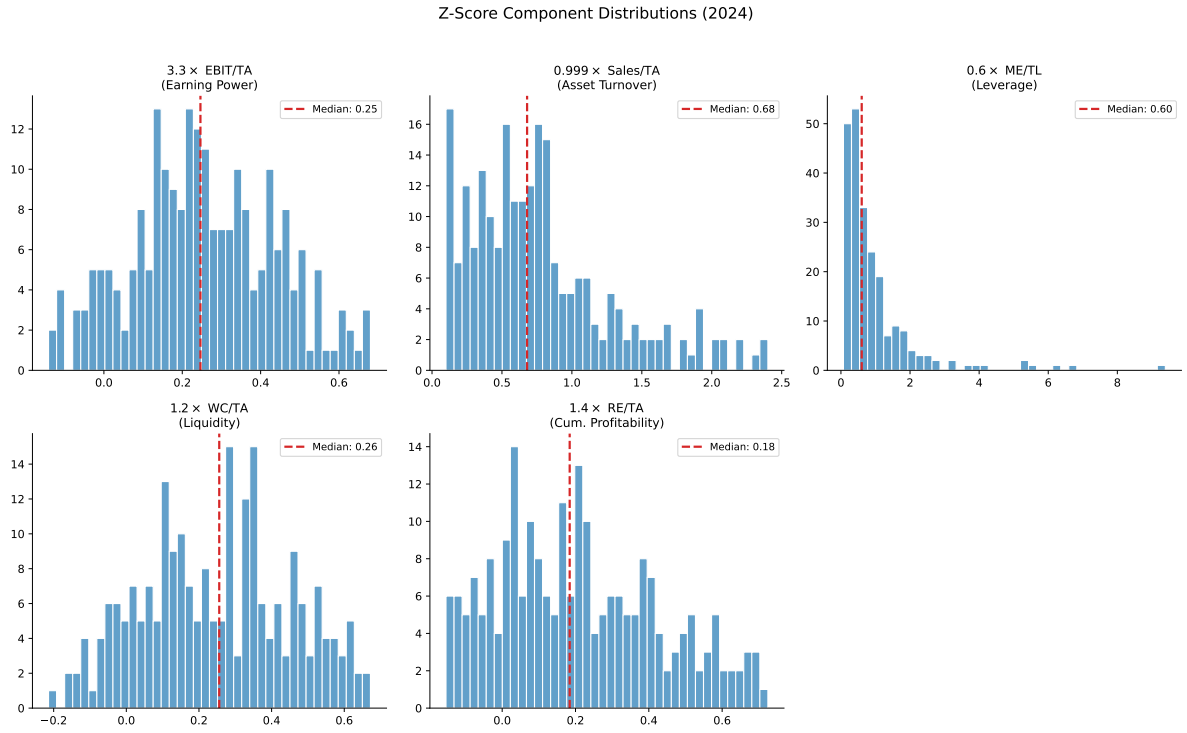


Figure 34.1: Distribution of Altman Z-Score component ratios for Vietnamese listed firms. Each panel shows a component weighted by its coefficient in the original Z-Score formula.

34.3 Distribution of Risk Zones

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

```
# --- Original Z-Score Zones ---
zone_counts = (
    df_clean
    .groupby(["year", "z_zone"])
    .size()
    .unstack(fill_value=0)
)
zone_pcts = zone_counts.div(zone_counts.sum(axis=1), axis=0) * 100

zone_colors = {
    "Safe": colors["safe"],
    "Grey Zone": colors["alert"],
    "Distress": colors["quaternary"],
    "High Distress": colors["distress"],
}

axes[0].stackplot(
    zone_pcts.index,
    *[zone_pcts[col] for col in ["Safe", "Grey Zone", "Distress", "High
    ↪  Distress"]
      if col in zone_pcts.columns],
    labels=[col for col in ["Safe", "Grey Zone", "Distress", "High Distress"]
            if col in zone_pcts.columns],
    colors=[zone_colors[col] for col in ["Safe", "Grey Zone", "Distress",
↪   "High Distress"]
            if col in zone_pcts.columns],
    alpha=0.8
)
axes[0].set_xlabel("Year")
axes[0].set_ylabel("Percentage of Firms")
axes[0].set_title("Original Z-Score Risk Zones")
axes[0].legend(loc="upper right", fontsize=9)
axes[0].set_ylim(0, 100)

# --- Emerging Market Z''-Score Zones ---
em_zone_counts = (
    df_clean
    .groupby(["year", "z_em_zone"])
    .size()
    .unstack(fill_value=0)
)
em_zone_pcts = em_zone_counts.div(em_zone_counts.sum(axis=1), axis=0) * 100
```

```
em_zone_colors = {"Safe": colors["safe"], "Grey Zone": colors["alert"],
                  "Distress": colors["quaternary"]}

axes[1].stackplot(
    em_zone_pcts.index,
    *[em_zone_pcts[col] for col in ["Safe", "Grey Zone", "Distress"]
      if col in em_zone_pcts.columns],
    labels=[col for col in ["Safe", "Grey Zone", "Distress"]
            if col in em_zone_pcts.columns],
    colors=[em_zone_colors[col] for col in ["Safe", "Grey Zone", "Distress"]
            if col in em_zone_pcts.columns],
    alpha=0.8
)
axes[1].set_xlabel("Year")
axes[1].set_ylabel("Percentage of Firms")
axes[1].set_title("Emerging Market Z''-Score Risk Zones")
axes[1].legend(loc="upper right", fontsize=9)
axes[1].set_ylim(0, 100)

plt.tight_layout()
plt.show()
```

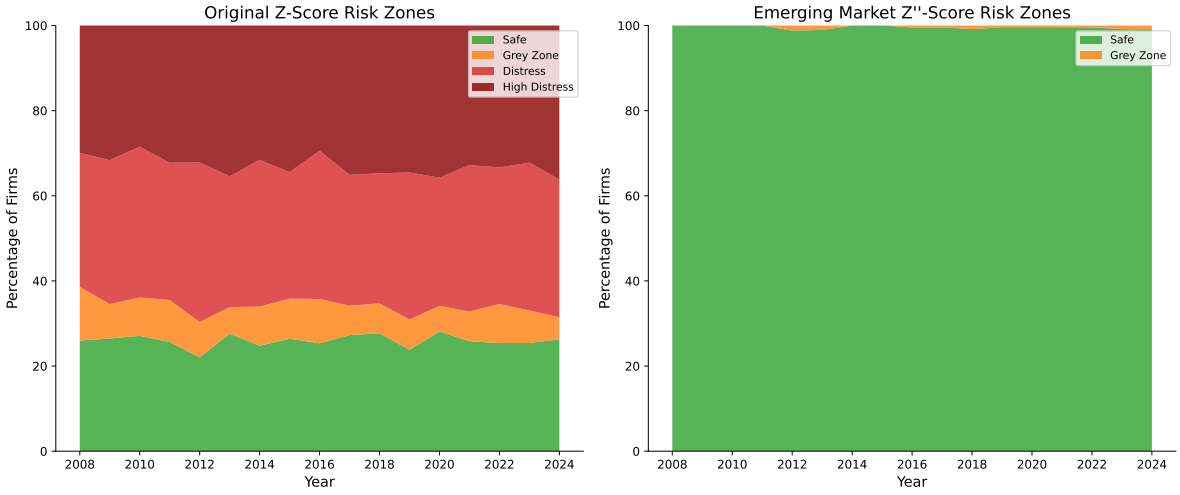


Figure 34.2: Proportion of Vietnamese listed firms in each Altman Z-Score risk zone over time. The figure shows both the original Z-Score zones and the emerging market Z’’-Score zones.

34.4 Comparing Z-Score Variants

An important practical question is which Z-Score variant is most appropriate for Vietnamese firms. The original model was calibrated on U.S. manufacturing firms, while the Z''-Score was specifically designed for emerging markets and non-manufacturing sectors.

```
fig, ax = plt.subplots(figsize=(8, 8))

sample = latest_z.dropna(subset=["altman_z_w", "altman_z_em_w"]).sample(
    min(500, len(latest_z)), random_state=42
)

scatter = ax.scatter(
    sample["altman_z_w"], sample["altman_z_em_w"],
    c=sample["tobin_q_simple_w"], cmap="RdYlGn",
    alpha=0.6, s=30, edgecolor="white", linewidth=0.3
)

# Add reference lines
lims = [
    min(ax.get_xlim()[0], ax.get_ylim()[0]),
    max(ax.get_xlim()[1], ax.get_ylim()[1])
]
ax.plot(lims, lims, "k--", alpha=0.3, linewidth=1, label="45° line")

# Add zone boundaries
ax.axvline(x=1.80, color="red", linestyle=":", alpha=0.5, linewidth=1)
ax.axvline(x=2.99, color="green", linestyle=":", alpha=0.5, linewidth=1)
ax.axhline(y=1.10, color="red", linestyle=":", alpha=0.5, linewidth=1)
ax.axhline(y=2.60, color="green", linestyle=":", alpha=0.5, linewidth=1)

ax.set_xlabel("Original Z-Score")
ax.set_ylabel("Emerging Market Z''-Score")
ax.set_title("Z-Score vs Z''-Score for Vietnamese Firms")

cbar = plt.colorbar(scatter, ax=ax, shrink=0.8)
cbar.set_label("Tobin's Q")

ax.legend(loc="upper left")
plt.tight_layout()
plt.show()
```

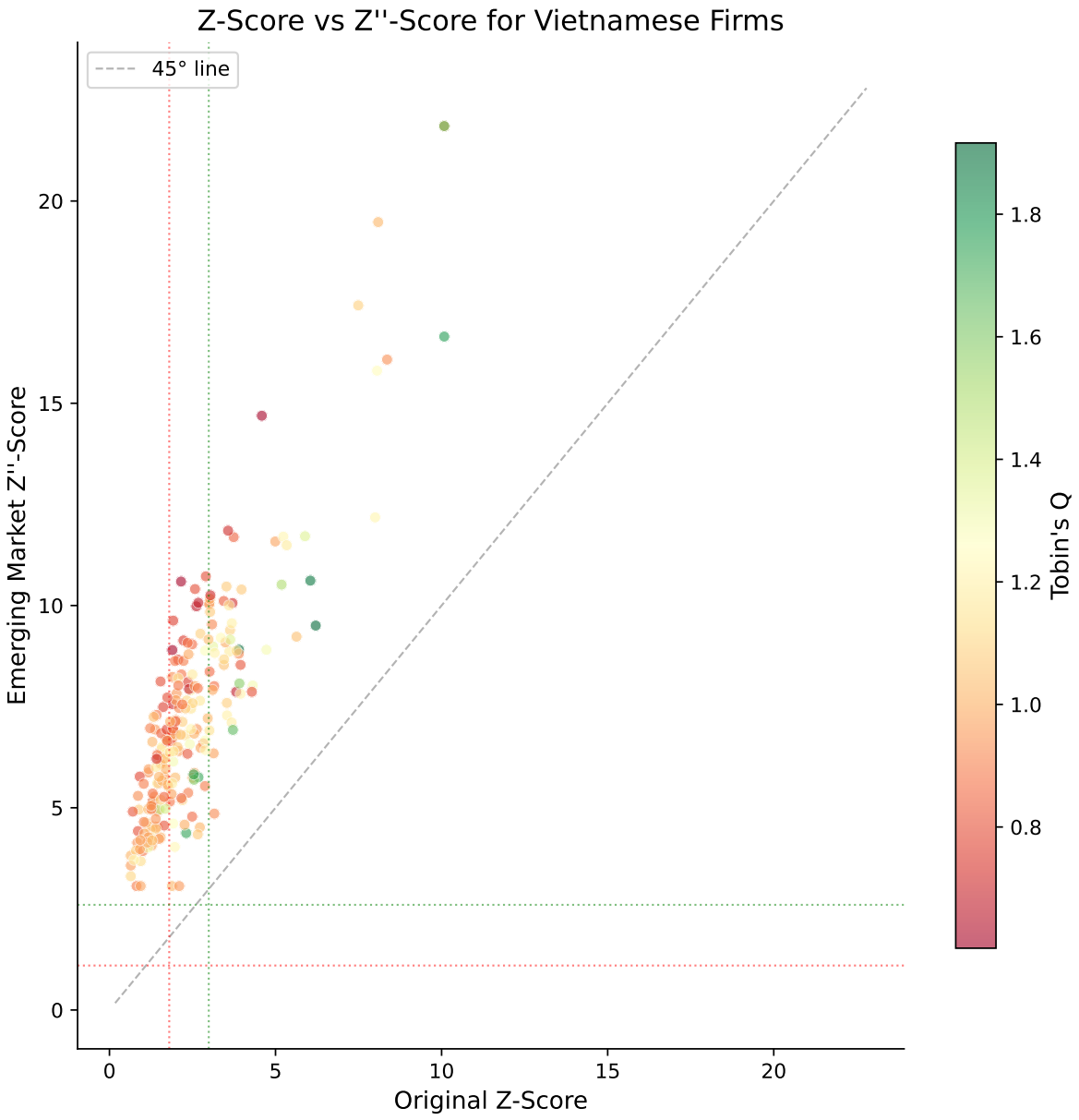


Figure 34.3: Comparison of original Z-Score and emerging market Z''-Score for Vietnamese listed firms. Points above (below) the 45-degree line have higher Z''-Scores (Z-Scores) than the other measure.

35 Computing Company Age

35.1 Multiple Age Proxies

For Vietnamese firms, we compute three age measures:

$$\text{Age}_{\text{founding}} = \text{Current Year} - \text{Founding Year} \tag{35.1}$$

$$\text{Age}_{\text{listing}} = \text{Current Year} - \text{Listing Year} \tag{35.2}$$

$$\text{Age}_{\text{data}} = \text{Current Year} - \text{First Year with Data} \tag{35.3}$$

```
def compute_company_age(df):
    """
    Compute multiple proxies for company age.

    For Vietnamese firms:
    - Founding age: based on founding/incorporation date
    - Listing age: based on first listing on HOSE/HNX/UPCoM
    - Data age: based on first year with available financial data
    """
    result = df.copy()

    # Founding age
    result["age_founding"] = result["year"] - result["founding_year"]

    # Listing age
    result["age_listing"] = result["year"] - result["listing_year"]
    result["age_listing"] = result["age_listing"].clip(lower=0)

    # Data age (years since first available financial data)
    first_year = result.groupby("ticker")["year"].transform("min")
    result["age_data"] = result["year"] - first_year
```

```
    # Log of age (commonly used in regressions)
    result["ln_age_founding"] = np.log1p(result["age_founding"])
    result["ln_age_listing"] = np.log1p(result["age_listing"])

    return result

df_clean = compute_company_age(df_clean)

print("Company Age Summary Statistics")
age_summary = df_clean[df_clean["year"] == latest_year][
    ["age_founding", "age_listing", "age_data"]
].describe().round(1)
age_summary.columns = ["Founding Age", "Listing Age", "Data Age"]
print(age_summary.to_string())
```

```
Company Age Summary Statistics
       Founding Age  Listing Age  Data Age
count         232.0        232.0     232.0
mean           30.1         13.9      12.3
std            11.6          5.8       3.9
min            10.0          5.0       5.0
25%            20.0          9.0       9.0
50%            30.0         13.0      13.0
75%            40.0         19.0      16.0
max            49.0         24.0      16.0
```

35.2 Age Distribution

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

age_vars = [
    ("age_founding", "Founding Age (years)", colors["primary"]),
    ("age_listing", "Listing Age (years)", colors["secondary"]),
    ("age_data", "Data Age (years)", colors["tertiary"]),
]

latest = df_clean[df_clean["year"] == latest_year]
```

```
for ax, (var, label, color) in zip(axes, age_vars):
    data = latest[var].dropna()
    ax.hist(data, bins=30, color=color, alpha=0.7, edgecolor="white")
    ax.axvline(x=data.median(), color="black", linestyle="--",
               linewidth=2, label=f"Median: {data.median():.0f}")
    ax.set_xlabel(label)
    ax.set_ylabel("Number of Firms")
    ax.legend()

axes[0].set_title("Founding Age")
axes[1].set_title("Listing Age")
axes[2].set_title("Data Age")

plt.suptitle(f"Company Age Distributions ({latest_year})", fontsize=14,
 ↪  y=1.02)
plt.tight_layout()
plt.show()
```

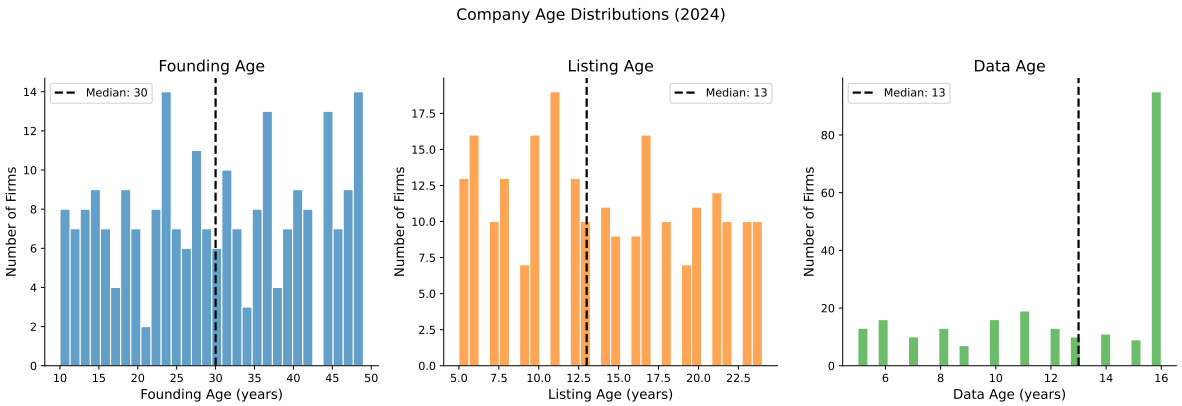


Figure 35.1: Distribution of company age measures for Vietnamese listed firms. Founding age reflects true organizational maturity, while listing age captures capital market experience.

35.3 Age and the Equitization Effect

A distinctive feature of the Vietnamese market is the equitization of SOEs. Many firms have founding dates that predate the stock market by decades, creating a large gap between founding age and listing age.

```
fig, ax = plt.subplots(figsize=(8, 8))

latest = df_clean[df_clean["year"] == latest_year].copy()
latest["soe_flag"] = latest["state_ownership_pct"] > 30  # SOE proxy

for soe, label, color, marker in [
    (True, "SOE (>30% state)", colors["quaternary"], "s"),
    (False, "Non-SOE", colors["primary"], "o"),
]:
    subset = latest[latest["soe_flag"] == soe]
    ax.scatter(
        subset["age_listing"], subset["age_founding"],
        c=color, alpha=0.5, s=25, marker=marker, label=label,
    edgecolor="white"
    )

# 45-degree line
max_age = max(latest["age_founding"].max(), latest["age_listing"].max())
ax.plot([0, max_age], [0, max_age], "k--", alpha=0.3, label="Founding =
    Listing age")

ax.set_xlabel("Listing Age (years)")
ax.set_ylabel("Founding Age (years)")
ax.set_title("Founding vs. Listing Age: The Equitization Gap")
ax.legend()

plt.tight_layout()
plt.show()
```

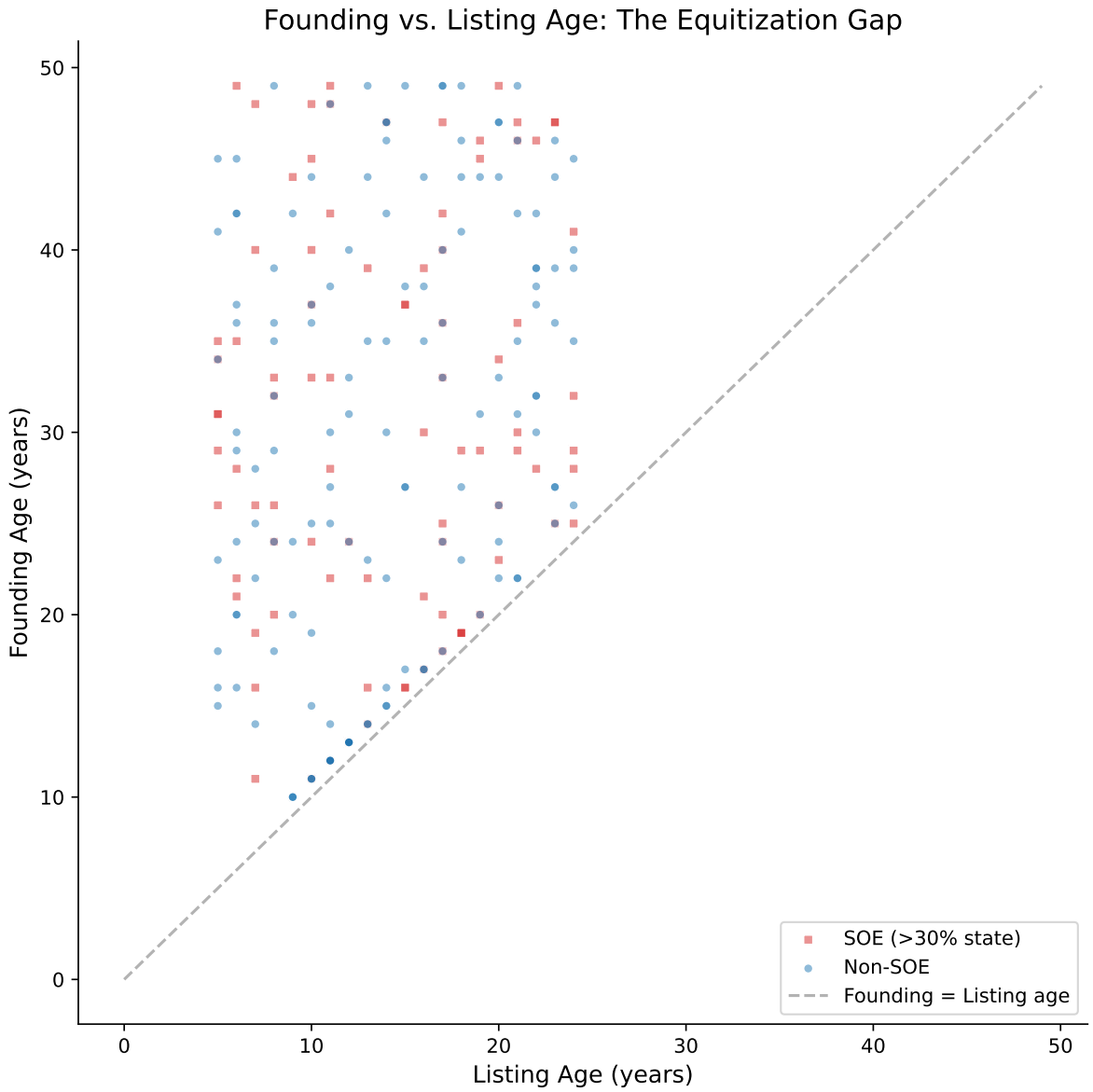


Figure 35.2: Relationship between founding age and listing age for Vietnamese firms. The gap reflects years of pre-listing operations, which is especially large for equitized state-owned enterprises.

36 Joint Analysis: Valuation, Distress, and Maturity

36.1 The Relationship Between Q, Z-Score, and Age

These three measures capture different dimensions of a firm's financial standing, but they are not independent. Theoretically, we expect:

- **Q and Z**: Firms with high growth opportunities (high Q) tend to be financially healthier (high Z), but the relationship is not monotonic, very high Q values may indicate speculative overvaluation.
- **Q and Age**: Young firms may have higher Q (growth expectations) or lower Q (unproven business model). The relationship depends on the industry life cycle.
- **Z and Age**: Older firms typically have more stable earnings and higher retained earnings, contributing to higher Z-Scores, though very old firms may face declining competitiveness.

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

sample = latest.dropna(subset=["tobin_q_simple_w", "altman_z_w",
 ↪  "age_founding"])
sample = sample.sample(min(300, len(sample)), random_state=42)

# Q vs Z
axes[0].scatter(
    sample["altman_z_w"], sample["tobin_q_simple_w"],
    c=sample["age_founding"], cmap="viridis", alpha=0.6,
    s=np.log1p(sample["at"]) * 3, edgecolor="white", linewidth=0.3
)
axes[0].set_xlabel("Altman Z-Score")
axes[0].set_ylabel("Tobin's Q")
axes[0].set_title("Valuation vs. Distress Risk")
axes[0].axhline(y=1, color="gray", linestyle="--", alpha=0.5)
axes[0].axvline(x=1.80, color="red", linestyle=":", alpha=0.5)
axes[0].axvline(x=2.99, color="green", linestyle=":", alpha=0.5)
```

```
# Q vs Age
axes[1].scatter(
    sample["age_founding"], sample["tobin_q_simple_w"],
    c=sample["altman_z_w"], cmap="RdYlGn", alpha=0.6,
    s=np.log1p(sample["at"]) * 3, edgecolor="white", linewidth=0.3
)
axes[1].set_xlabel("Founding Age (years)")
axes[1].set_ylabel("Tobin's Q")
axes[1].set_title("Valuation vs. Maturity")
axes[1].axhline(y=1, color="gray", linestyle="--", alpha=0.5)

# Z vs Age
scatter = axes[2].scatter(
    sample["age_founding"], sample["altman_z_w"],
    c=sample["tobin_q_simple_w"], cmap="coolwarm", alpha=0.6,
    s=np.log1p(sample["at"]) * 3, edgecolor="white", linewidth=0.3
)
axes[2].set_xlabel("Founding Age (years)")
axes[2].set_ylabel("Altman Z-Score")
axes[2].set_title("Distress Risk vs. Maturity")
axes[2].axhline(y=1.80, color="red", linestyle=":", alpha=0.5)
axes[2].axhline(y=2.99, color="green", linestyle=":", alpha=0.5)

plt.tight_layout()
plt.show()
```

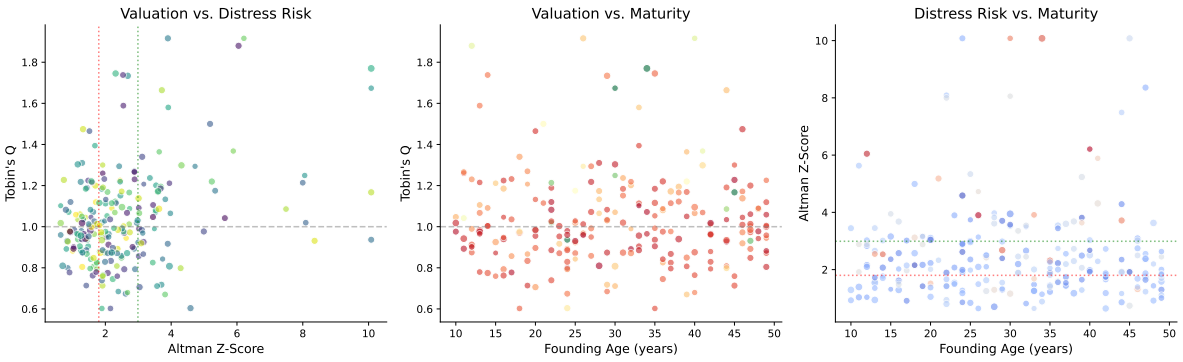


Figure 36.1: Relationship between Tobin's Q, Altman Z-Score, and company age for Vietnamese listed firms. Bubble size reflects total assets; color indicates the emerging market Z''-Score risk zone.

36.2 Correlation Structure

```
corr_vars = [
    "tobin_q_simple_w", "mtb_w", "altman_z_w", "altman_z_em_w",
    "age_founding", "age_listing",
    "z_x1", "z_x2", "z_x3", "z_x4", "z_x5"
]
corr_labels = [
    "Tobin's Q", "M/B Ratio", "Z-Score", "Z'' (EM)",
    "Age (Found.)", "Age (List.)",
    "EBIT/TA", "Sales/TA", "ME/TL", "WC/TA", "RE/TA"
]

corr_matrix = df_clean[corr_vars].corr()
corr_matrix.index = corr_labels
corr_matrix.columns = corr_labels

fig, ax = plt.subplots(figsize=(12, 10))

mask = np.triu(np.ones_like(corr_matrix, dtype=bool), k=1)
cmap = sns.diverging_palette(250, 15, s=75, l=40, n=9, center="light",
 ↪  as_cmap=True)

sns.heatmap(
    corr_matrix, mask=mask, cmap=cmap, center=0,
    annot=True, fmt=".2f", square=True,
    linewidths=0.5, cbar_kws={"shrink": 0.8},
    ax=ax, vmin=-1, vmax=1,
    annot_kws={"size": 9}
)
ax.set_title("Correlation Matrix of Key Financial Measures", fontsize=14,
 ↪  pad=20)

plt.tight_layout()
plt.show()
```

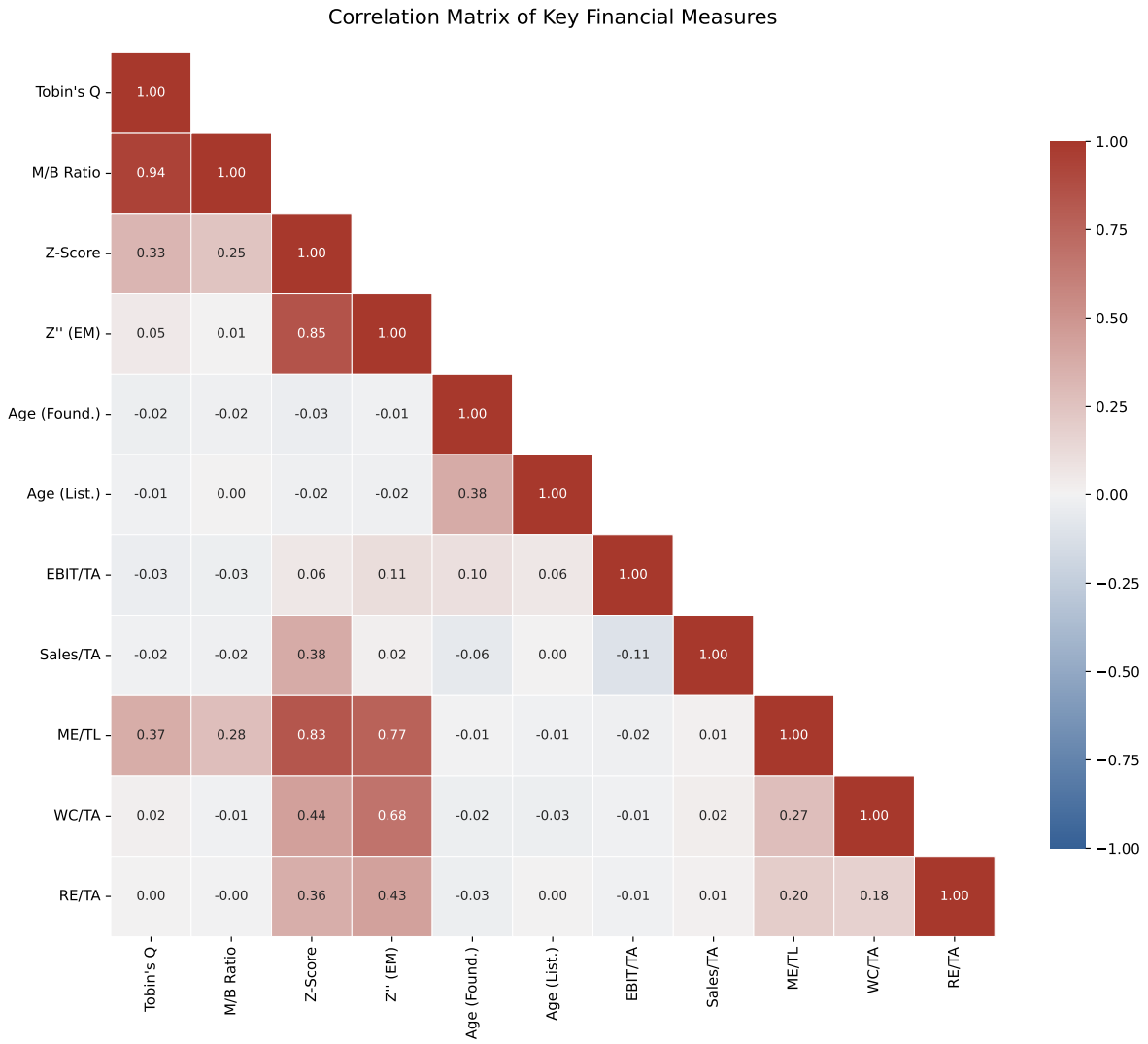


Figure 36.2: Correlation matrix of key valuation, distress, and maturity measures for Vietnamese listed firms.

36.3 Cross-Sectional Regression Analysis

We estimate a simple cross-sectional model to examine the determinants of Tobin's Q in the Vietnamese market:

$$Q_{i,t} = \alpha + \beta_1 \ln(\text{Age}_i) + \beta_2 Z''_{i,t} + \beta_3 \text{SOE}_i + \beta_4 \ln(\text{TA}_{i,t}) + \varepsilon_{i,t} \tag{36.1}$$

```
from numpy.linalg import lstsq

# Prepare regression data
reg_data = latest.dropna(
    subset=["tobin_q_simple_w", "ln_age_founding", "altman_z_em_w", "at"]
).copy()

reg_data["ln_at"] = np.log(reg_data["at"])
reg_data["soe_dummy"] = (reg_data["state_ownership_pct"] > 30).astype(int)
reg_data["constant"] = 1.0

# OLS estimation
y = reg_data["tobin_q_simple_w"].values
X = reg_data[["constant", "ln_age_founding", "altman_z_em_w",
              "soe_dummy", "ln_at"]].values

betas, residuals, rank, sv = lstsq(X, y, rcond=None)
y_hat = X @ betas
resid = y - y_hat
n, k = X.shape

# Standard errors (heteroskedasticity-robust would be better)
sigma2 = np.sum(resid**2) / (n - k)
var_beta = sigma2 * np.linalg.inv(X.T @ X)
se = np.sqrt(np.diag(var_beta))
t_stats = betas / se
r_squared = 1 - np.sum(resid**2) / np.sum((y - y.mean())**2)
adj_r2 = 1 - (1 - r_squared) * (n - 1) / (n - k - 1)

# Display results
var_names = ["Constant", "ln(Age)", "Z''-Score (EM)", "SOE Dummy", "ln(Total
 ↪  Assets)"]
print("=" * 65)
print("  Cross-Sectional Regression: Determinants of Tobin's Q")
print(f"  Dependent Variable: Tobin's Q (Simple, Winsorized)")
print(f"  Sample: {n} firms ({latest_year})")
print("=" * 65)
print(f"  {'Variable':<22} {'Coef':>10} {'Std.Err':>10} {'t-stat':>10}")
print("-" * 65)
for name, b, s, t in zip(var_names, betas, se, t_stats):
    sig = "***" if abs(t) > 2.576 else "**" if abs(t) > 1.96 else "*" if
 ↪  abs(t) > 1.645 else ""
    print(f"  {name:<22} {b:>10.4f} {s:>10.4f} {t:>8.2f} {sig}")
```

```python
print("-" * 65)
print(f"  R-squared: {r_squared:.4f}")
print(f"  Adjusted R-squared: {adj_r2:.4f}")
print(f"  Observations: {n}")
print("=" * 65)
print("  Significance: *** p<0.01, ** p<0.05, * p<0.10")
```

```
=================================================================
  Cross-Sectional Regression: Determinants of Tobin's Q
  Dependent Variable: Tobin's Q (Simple, Winsorized)
  Sample: 232 firms (2024)
=================================================================
  Variable                      Coef    Std.Err     t-stat
-----------------------------------------------------------------
  Constant                    0.9686     0.1959     4.94 ***
  ln(Age)                    -0.0064     0.0365    -0.18
  Z''-Score (EM)              0.0067     0.0050     1.34
  SOE Dummy                   0.0327     0.0315     1.04
  ln(Total Assets)            0.0018     0.0096     0.19
-----------------------------------------------------------------
  R-squared: 0.0126
  Adjusted R-squared: -0.0092
  Observations: 232
=================================================================
  Significance: *** p<0.01, ** p<0.05, * p<0.10
```

37 The Complete Pipeline

37.1 Putting It All Together

Here we provide a single, clean function that takes raw DataCore.vn financial data and produces a complete dataset with all three measures computed.

```
def compute_valuation_distress_age(df, firm_profiles=None):
    """
    Complete pipeline to compute Tobin's Q, Altman Z-Score variants,
    and Company Age for Vietnamese listed firms.

    Parameters
    ----------
    df : pd.DataFrame
        Panel of firm-year financial data with columns:
        at, seq, lt, lct, tlb, sale, ebit, ni, re_var, act,
        ppent, invt, txdb, itcb, pstk, me, prcc, csho, year, ticker
    firm_profiles : pd.DataFrame, optional
        Company profiles with founding_year, listing_year

    Returns
    -------
    pd.DataFrame
        Input data augmented with computed measures
    """
    result = df.copy()

    # === Data Quality Filters ===
    result = result[result["at"] > 0]
    result = result[result["seq"] > 0]

    # === Book Value of Equity (Daniel & Titman 1997) ===
    result["pref"] = result["pstk"].fillna(0)
    result["be"] = (
        result["seq"]
```

```
        + result["txdb"].fillna(0)
        + result["itcb"].fillna(0)
        - result["pref"]
    )

    # === Market Value of Equity ===
    if "me" not in result.columns or result["me"].isna().all():
        result["me"] = result["prcc"] * result["csho"]

    # === Tobin's Q Variants ===
    # Simple Q (Gompers et al. 2003)
    result["tobin_q"] = (result["at"] + result["me"] - result["be"]) /
↪   result["at"]

    # Chung-Pruitt Q
    debt_cp = (
        result["lct"].fillna(0) - result["act"].fillna(0)
        + result["invt"].fillna(0) + result["lt"].fillna(0)
    )
    result["tobin_q_cp"] = (
        (result["me"] + result["pstk"].fillna(0) + debt_cp) / result["at"]
    )

    # Market-to-Book
    result["mtb"] = np.where(result["be"] > 0, result["me"] / result["be"],
↪   np.nan)

    # === Altman Z-Score Variants ===
    result["wc"] = result["act"].fillna(0) - result["lct"].fillna(0)

    x1 = result["ebit"] / result["at"]
    x2 = result["sale"] / result["at"]
    x3_market = np.where(result["tlb"] > 0, result["me"] / result["tlb"],
↪   np.nan)
    x3_book = np.where(result["tlb"] > 0, result["be"] / result["tlb"],
↪   np.nan)
    x4 = result["wc"] / result["at"]
    x5 = result["re_var"].fillna(0) / result["at"]

    # Original Z
    result["altman_z"] = 3.3 * x1 + 0.999 * x2 + 0.6 * x3_market + 1.2 * x4 +
↪   1.4 * x5
```

```
    # Z' (private firms)
    result["altman_z_prime"] = (
        0.717 * x4 + 0.847 * x5 + 3.107 * x1 + 0.420 * x3_book + 0.998 * x2
    )

    # Z'' (emerging markets)
    result["altman_z_em"] = (
        3.25 + 6.56 * x4 + 3.26 * x5 + 6.72 * x1 + 1.05 * x3_book
    )

    # Risk zones
    result["z_zone"] = pd.cut(
        result["altman_z"],
        bins=[-np.inf, 1.80, 2.70, 2.99, np.inf],
        labels=["High Distress", "Distress", "Grey Zone", "Safe"]
    )
    result["z_em_zone"] = pd.cut(
        result["altman_z_em"],
        bins=[-np.inf, 1.10, 2.60, np.inf],
        labels=["Distress", "Grey Zone", "Safe"]
    )

    # === Company Age ===
    if firm_profiles is not None:
        result = result.merge(
            firm_profiles[["ticker", "founding_year", "listing_year"]],
            on="ticker", how="left", suffixes=("", "_profile")
        )
        result["age_founding"] = result["year"] - result["founding_year"]
        result["age_listing"] = (result["year"] -
↪   result["listing_year"]).clip(lower=0)

    first_year = result.groupby("ticker")["year"].transform("min")
    result["age_data"] = result["year"] - first_year

    # === Winsorize ===
    for var in ["tobin_q", "tobin_q_cp", "mtb", "altman_z", "altman_z_prime",
     ↪  "altman_z_em"]:
        if var in result.columns:
            result[f"{var}_w"] = winsorize(result[var])

    return result
```

```
# Demonstrate the pipeline
final_df = compute_valuation_distress_age(df, firm_profiles)
print(f"Final dataset: {final_df.shape[0]:,} observations,
 ↪  {final_df.shape[1]} variables")
print(f"Firms: {final_df['ticker'].nunique()}")
print(f"\nKey variables computed:")
for var in ["tobin_q", "tobin_q_cp", "mtb", "altman_z", "altman_z_prime",
            "altman_z_em", "age_founding", "age_listing", "age_data"]:
    if var in final_df.columns:
        non_null = final_df[var].notna().sum()
        print(f"  {var}: {non_null:,} non-null values")
```

```
Final dataset: 3,484 observations, 48 variables
Firms: 297

Key variables computed:
  tobin_q: 3,484 non-null values
  tobin_q_cp: 3,484 non-null values
  mtb: 3,484 non-null values
  altman_z: 3,484 non-null values
  altman_z_prime: 3,484 non-null values
  altman_z_em: 3,484 non-null values
  age_founding: 3,484 non-null values
  age_listing: 3,484 non-null values
  age_data: 3,484 non-null values
```

37.2 Exporting Results

```
# Select key output variables
output_vars = [
    "ticker", "year", "datadate", "exchange", "industry",
    "at", "be", "me", "tobin_q", "tobin_q_w", "mtb", "mtb_w",
    "altman_z", "altman_z_w", "altman_z_em", "altman_z_em_w",
    "z_zone", "z_em_zone",
    "age_founding", "age_listing", "age_data",
    "state_ownership_pct"
]
```

```python
export_cols = [v for v in output_vars if v in final_df.columns]
export_df = final_df[export_cols].copy()

# export_df.to_csv("vn_valuation_distress_age.csv", index=False)
# export_df.to_parquet("vn_valuation_distress_age.parquet", index=False)

print(f"Export dataset: {export_df.shape[0]:,} rows × {export_df.shape[1]}
 ↪  columns")
print(f"\nSample output (first 5 rows):")
display_cols = ["ticker", "year", "tobin_q", "altman_z", "altman_z_em",
                "z_zone", "age_founding"]
display_cols = [c for c in display_cols if c in export_df.columns]
print(export_df[display_cols].head().to_string(index=False))
```

```
Export dataset: 3,484 rows × 22 columns

Sample output (first 5 rows):
ticker  year  tobin_q  altman_z  altman_z_em z_zone  age_founding
VN0001  2017 0.905928  3.345941     5.605169   Safe             4
VN0001  2018 1.270114  5.324082     9.228846   Safe             5
VN0001  2019 1.866493  5.478529     6.264178   Safe             6
VN0001  2020 1.016979  5.612289    10.445449   Safe             7
VN0001  2021 0.805818  4.220869     7.588273   Safe             8
```

38 Special Topics for the Vietnamese Market

38.1 State Ownership and Valuation

State ownership is a defining feature of the Vietnamese corporate landscape. We examine how state ownership affects both Tobin's Q and financial distress risk.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

latest = final_df[final_df["year"] == latest_year].dropna(
    subset=["tobin_q_w", "altman_z_em_w", "state_ownership_pct"]
)

# Q vs State Ownership
axes[0].scatter(
    latest["state_ownership_pct"], latest["tobin_q_w"],
    alpha=0.4, s=20, color=colors["primary"], edgecolor="white"
)

# Binned means
bins = pd.cut(latest["state_ownership_pct"], bins=10)
binned = latest.groupby(bins)["tobin_q_w"].mean()
bin_centers = [(b.left + b.right) / 2 for b in binned.index]
axes[0].plot(bin_centers, binned.values, color=colors["quaternary"],
             linewidth=3, label="Binned Mean")

axes[0].set_xlabel("State Ownership (%)")
axes[0].set_ylabel("Tobin's Q")
axes[0].set_title("State Ownership and Firm Valuation")
axes[0].axhline(y=1, color="gray", linestyle="--", alpha=0.5)
axes[0].legend()

# Z''-Score by SOE quartile
latest["soe_quartile"] = pd.qcut(
    latest["state_ownership_pct"],
    q=4, labels=["Q1 (Low)", "Q2", "Q3", "Q4 (High)"],
```

```
    duplicates="drop"
)

soe_groups = latest.groupby("soe_quartile")["altman_z_em_w"]
box_data = [group.values for name, group in soe_groups]
bp = axes[1].boxplot(
    box_data,
    labels=[name for name, _ in soe_groups],
    patch_artist=True,
    medianprops=dict(color="black", linewidth=2)
)

gradient_colors = plt.cm.Blues(np.linspace(0.3, 0.8, len(bp["boxes"])))
for patch, color in zip(bp["boxes"], gradient_colors):
    patch.set_facecolor(color)

axes[1].axhline(y=2.60, color="green", linestyle=":", alpha=0.5, label="Safe
 ↪  threshold")
axes[1].axhline(y=1.10, color="red", linestyle=":", alpha=0.5,
 ↪  label="Distress threshold")
axes[1].set_xlabel("State Ownership Quartile")
axes[1].set_ylabel("Z''-Score (Emerging Market)")
axes[1].set_title("Financial Health by State Ownership")
axes[1].legend(fontsize=9)

plt.tight_layout()
plt.show()
```

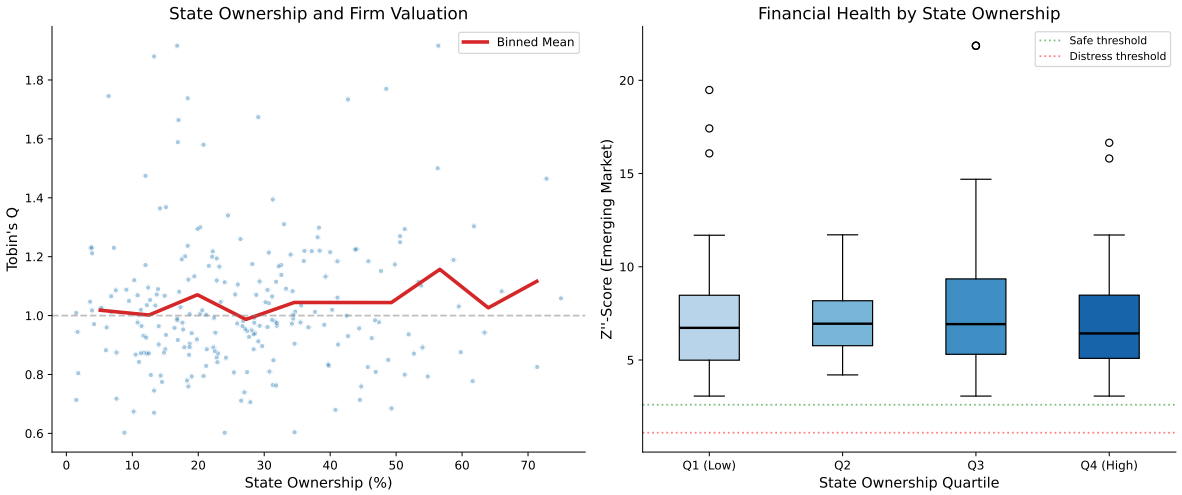


Figure 38.1: Relationship between state ownership percentage and firm valuation/distress measures. The left panel shows Tobin's Q against state ownership, with a LOWESS smoother. The right panel shows the Z''-Score by state ownership quartile.

38.2 Exchange-Level Analysis

```
exchange_summary = (
    latest
    .groupby("exchange")
    .agg(
        n_firms=("ticker", "nunique"),
        median_q=("tobin_q_w", "median"),
        mean_q=("tobin_q_w", "mean"),
        median_z=("altman_z_w", "median"),
        mean_z_em=("altman_z_em_w", "mean"),
        pct_distress_z=("z_zone", lambda x: (x == "High Distress").mean() *
 ↪  100),
        pct_distress_em=("z_em_zone", lambda x: (x == "Distress").mean() *
 ↪  100),
        median_age=("age_founding", "median"),
        median_soe=("state_ownership_pct", "median"),
    )
    .round(2)
)
```

```
exchange_summary.columns = [
    "N Firms", "Median Q", "Mean Q", "Median Z", "Mean Z'' (EM)",
    "% Distress (Z)", "% Distress (Z'')", "Median Age", "Median SOE %"
]

exchange_summary.style.set_properties(**{
    "text-align": "center",
    "font-size": "10pt"
}).set_table_styles([
    {"selector": "th", "props": [
        ("background-color", "#1f77b4"),
        ("color", "white"),
        ("text-align", "center"),
        ("padding", "8px")
    ]},
]).format({
    "Median Q": "{:.2f}", "Mean Q": "{:.2f}",
    "Median Z": "{:.2f}", "Mean Z'' (EM)": "{:.2f}",
    "% Distress (Z)": "{:.1f}%", "% Distress (Z'')": "{:.1f}%",
    "Median Age": "{:.0f}", "Median SOE %": "{:.1f}%"
})
```

Table 38.1: Summary statistics by exchange for Vietnamese listed firms (latest year

Table 38.1

exchange	N Firms	Median Q	Mean Q	Median Z	Mean Z'' (EM)	% Distress (Z)	% Distress (Z'')
HNX	77	0.99	1.02	2.07	7.25	32.5%	0.0%
HOSE	113	1.00	1.04	2.06	7.24	40.7%	0.0%
UPCoM	42	0.98	1.05	2.13	7.80	30.9%	0.0%

38.3 Sector Heatmap: Valuation and Distress

```
fig, axes = plt.subplots(1, 2, figsize=(16, 8))

# Tobin's Q heatmap
q_pivot = (
    final_df
```

```
    .groupby(["industry", "year"])["tobin_q_w"]
    .median()
    .unstack(fill_value=np.nan)
)

sns.heatmap(
    q_pivot, cmap="RdYlGn", center=1, ax=axes[0],
    cbar_kws={"label": "Median Q", "shrink": 0.8},
    linewidths=0.5, linecolor="white",
    xticklabels=2
)
axes[0].set_title("Tobin's Q by Industry and Year")
axes[0].set_xlabel("Year")
axes[0].set_ylabel("")

# Z''-Score heatmap
z_pivot = (
    final_df
    .groupby(["industry", "year"])["altman_z_em_w"]
    .median()
    .unstack(fill_value=np.nan)
)

sns.heatmap(
    z_pivot, cmap="RdYlGn", center=2.6, ax=axes[1],
    cbar_kws={"label": "Median Z''", "shrink": 0.8},
    linewidths=0.5, linecolor="white",
    xticklabels=2
)
axes[1].set_title("Z''-Score (EM) by Industry and Year")
axes[1].set_xlabel("Year")
axes[1].set_ylabel("")

plt.tight_layout()
plt.show()
```

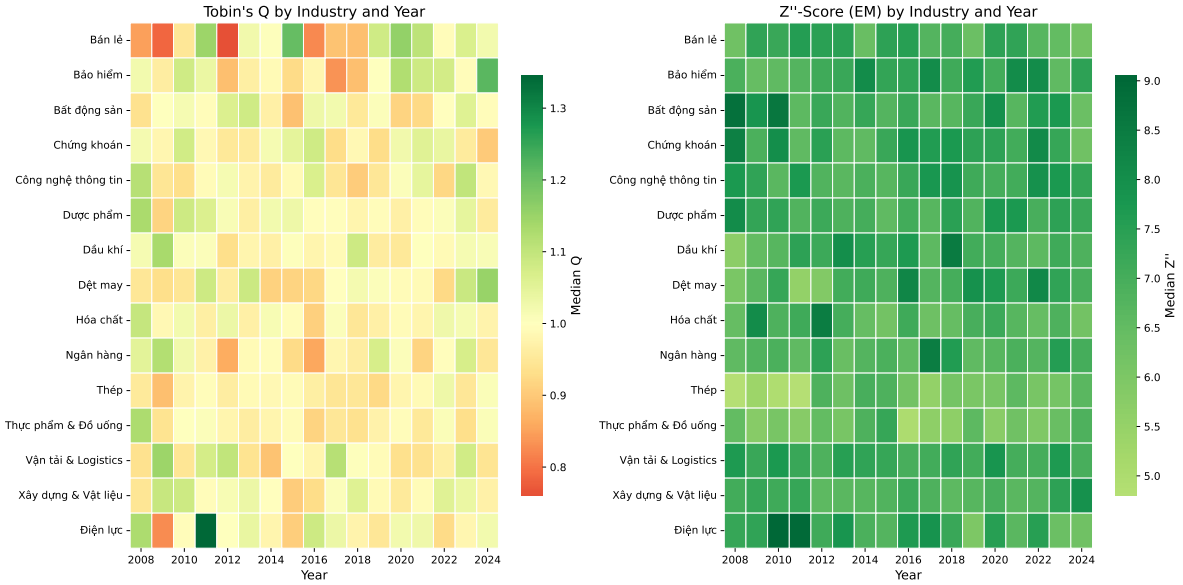


Figure 38.2: Median Tobin's Q and Z'' -Score by industry and year for Vietnamese listed firms. Warmer colors indicate higher values.

38.4 Handling Delisted Firms: Survivorship Bias

A critical issue in measuring financial distress is survivorship bias. If we only analyze currently listed firms, we miss the very firms that the Z-Score is designed to identify, those that went bankrupt or were delisted.

```
# Simulate delisting data (replace with DataCore.vn delisting records)
np.random.seed(123)
n_delisted = 50
delisted_firms = pd.DataFrame({
    "ticker": [f"VN{str(i).zfill(4)}" for i in range(n_firms + 1, n_firms +
    ↪  n_delisted + 1)],
    "delist_year": np.random.randint(2010, 2024, n_delisted),
    "delist_reason": np.random.choice(
        ["Bankruptcy/Liquidation", "Merger/Acquisition", "Voluntary
↪  Delisting",
         "Regulatory Non-compliance", "Below Minimum Requirements"],
        n_delisted,
        p=[0.15, 0.25, 0.20, 0.20, 0.20]
    )
})
```

```
# Summary of delisting reasons
print("Delisting Reasons Summary")
print("=" * 50)
reason_counts = delisted_firms["delist_reason"].value_counts()
for reason, count in reason_counts.items():
    print(f"  {reason}: {count} ({count/n_delisted*100:.0f}%)")
print(f"\nTotal delisted: {n_delisted}")
print(f"Currently listed: {final_df['ticker'].nunique()}")
print(f"\nSurvivorship rate: "
      f"{final_df['ticker'].nunique()/(final_df['ticker'].nunique()+n_delist
       ↪ ed)*100:.1f}%")
```

```
Delisting Reasons Summary
==================================================
  Merger/Acquisition: 16 (32%)
  Voluntary Delisting: 14 (28%)
  Regulatory Non-compliance: 9 (18%)
  Below Minimum Requirements: 7 (14%)
  Bankruptcy/Liquidation: 4 (8%)

Total delisted: 50
Currently listed: 297

Survivorship rate: 85.6%
```

39 Robustness Checks and Extensions

39.1 Alternative Tobin's Q Specifications

```
fig, ax = plt.subplots(figsize=(7, 7))

sample = latest.dropna(subset=["tobin_q_w", "tobin_q_cp"]).copy()
sample["tobin_q_cp_w"] = winsorize(sample["tobin_q_cp"])

ax.scatter(
    sample["tobin_q_w"], sample["tobin_q_cp_w"],
    alpha=0.4, s=15, color=colors["primary"], edgecolor="white"
)

# 45-degree line
lims = [
    min(ax.get_xlim()[0], ax.get_ylim()[0]),
    max(ax.get_xlim()[1], ax.get_ylim()[1])
]
ax.plot(lims, lims, "k--", alpha=0.3)

corr = sample["tobin_q_w"].corr(sample["tobin_q_cp_w"])
ax.text(0.05, 0.95, f"Correlation: {corr:.3f}",
        transform=ax.transAxes, fontsize=12,
        verticalalignment="top",
        bbox=dict(boxstyle="round", facecolor="wheat", alpha=0.5))

ax.set_xlabel("Simple Q (Gompers et al.)")
ax.set_ylabel("Chung-Pruitt Q")
ax.set_title("Comparison of Tobin's Q Variants")

plt.tight_layout()
plt.show()
```

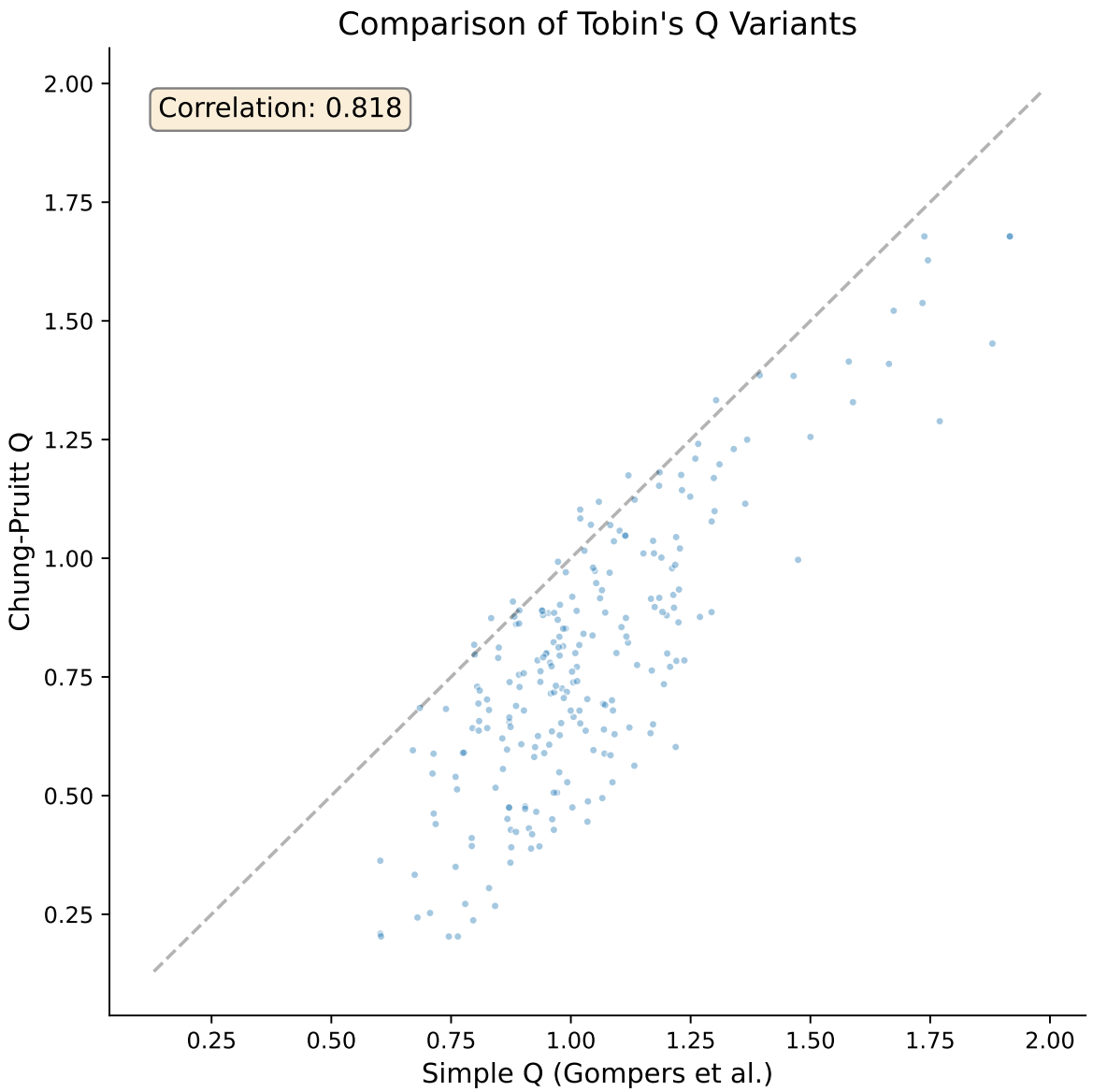


Figure 39.1: Comparison of Tobin's Q variants: Simple Q (Gompers et al. 2003) vs. Chung-Pruitt approximation. High correlation supports the use of the simpler measure.

39.2 Industry-Adjusted Measures

Raw Tobin's Q and Z-Scores may reflect industry characteristics rather than firm-specific attributes. We compute industry-adjusted versions:

$$Q_{i,t}^{\text{adj}} = Q_{i,t} - \overline{Q}_{j(i),t} \tag{39.1}$$

where $\overline{Q}_{j(i),t}$ is the median Tobin's Q of the industry j to which firm i belongs in year t.

```
def industry_adjust(df, var, group_var="industry"):
    """Compute industry-adjusted measure (deviation from industry median)."""
    industry_median = df.groupby(["year",
 ↪  group_var])[var].transform("median")
    return df[var] - industry_median

final_df["tobin_q_adj"] = industry_adjust(final_df, "tobin_q_w")
final_df["altman_z_em_adj"] = industry_adjust(final_df, "altman_z_em_w")

latest_adj = final_df[final_df["year"] == latest_year]

print("Industry-Adjusted Measures (Latest Year)")
print(f"  Tobin's Q (adjusted): mean={latest_adj['tobin_q_adj'].mean():.4f},
 ↪  "
      f"std={latest_adj['tobin_q_adj'].std():.3f}")
print(f"  Z''-Score (adjusted):
 ↪  mean={latest_adj['altman_z_em_adj'].mean():.4f}, "
      f"std={latest_adj['altman_z_em_adj'].std():.3f}")
```

```
Industry-Adjusted Measures (Latest Year)
  Tobin's Q (adjusted): mean=0.0352, std=0.226
  Z''-Score (adjusted): mean=0.4840, std=3.030
```

39.3 Panel Regression with Fixed Effects

For more rigorous analysis, we estimate a panel model with firm and year fixed effects:

$$Q_{i,t} = \alpha_i + \gamma_t + \beta_1 Z''_{i,t} + \beta_2 \ln(\text{Age}_{i,t}) + \beta_3 \text{Size}_{i,t} + \varepsilon_{i,t} \tag{39.2}$$

```
# Simplified within-estimator (year and industry demeaning)
panel = final_df.dropna(
    subset=["tobin_q_w", "altman_z_em_w", "age_founding", "at"]
).copy()
```

```
panel["ln_at"] = np.log(panel["at"])
panel["ln_age"] = np.log1p(panel["age_founding"])

# Demean by year-industry (proxy for fixed effects)
fe_vars = ["tobin_q_w", "altman_z_em_w", "ln_age", "ln_at"]
for var in fe_vars:
    group_mean = panel.groupby(["year", "industry"])[var].transform("mean")
    panel[f"{var}_dm"] = panel[var] - group_mean

# OLS on demeaned data
y = panel["tobin_q_w_dm"].values
X = np.column_stack([
    np.ones(len(panel)),
    panel["altman_z_em_w_dm"].values,
    panel["ln_age_dm"].values,
    panel["ln_at_dm"].values,
])

betas, _, _, _ = lstsq(X, y, rcond=None)
y_hat = X @ betas
resid = y - y_hat
n, k = X.shape

sigma2 = np.sum(resid**2) / (n - k)
var_beta = sigma2 * np.linalg.inv(X.T @ X)
se = np.sqrt(np.diag(var_beta))
t_stats = betas / se
r_squared = 1 - np.sum(resid**2) / np.sum((y - y.mean())**2)

var_names = ["Constant", "Z''-Score (EM)", "ln(Age)", "ln(Total Assets)"]
print("=" * 65)
print("  Panel Regression: Tobin's Q with Year-Industry FE")
print(f"  (Within estimator via year×industry demeaning)")
print("=" * 65)
print(f"  {'Variable':<22} {'Coef':>10} {'Std.Err':>10} {'t-stat':>10}")
print("-" * 65)
for name, b, s, t in zip(var_names, betas, se, t_stats):
    sig = "***" if abs(t) > 2.576 else "**" if abs(t) > 1.96 else "*" if
 ↪  abs(t) > 1.645 else ""
    print(f"  {name:<22} {b:>10.4f} {s:>10.4f} {t:>8.2f} {sig}")
print("-" * 65)
print(f"  R-squared (within): {r_squared:.4f}")
print(f"  Observations: {n:,}")
```

```
print("=" * 65)
```

```
=================================================================
  Panel Regression: Tobin's Q with Year-Industry FE
  (Within estimator via year×industry demeaning)
=================================================================
  Variable                      Coef    Std.Err     t-stat
-----------------------------------------------------------------
  Constant                    0.0000     0.0038     0.00
  Z''-Score (EM)              0.0034     0.0014     2.45 **
  ln(Age)                    -0.0055     0.0063    -0.88
  ln(Total Assets)           -0.0031     0.0026    -1.16
-----------------------------------------------------------------
  R-squared (within): 0.0024
  Observations: 3,484
=================================================================
```

40 Distress Resolution: The Regime Behind the Z-Score

40.1 What the Z-Score Actually Predicts

Altman (1968)'s discriminant function is fitted to a matched sample of firms that did and did not file for bankruptcy. The event it scores is therefore *entry into a particular legal process*, not economic death, and certainly not loss given distress. This distinction is easy to lose because the interpretation zones in Table 30.2 are phrased in the language of probability of failure, but for anyone pricing a claim on the firm the relevant object is

$$\mathbb{E}[\text{Loss}_i] \;=\; \underbrace{\Pr(\text{distress}_i)}_{\text{what } Z \text{ scores}} \times \underbrace{\mathbb{E}[\text{Loss}_i \mid \text{distress}_i]}_{\text{set by the insolvency regime}} \;. \tag{40.1}$$

The second factor is not a property of the firm. It is a property of the rules governing who controls a distressed firm, what it costs to reorganize, whether creditors can force a liquidation, and whether the incumbent owner can retain equity. Transplanting a discriminant function estimated in one jurisdiction transplants an implicit second factor along with it. That the second factor genuinely varies is well documented: Davydenko and Franks (2008) show that defaults on similar loans in France, Germany, and the UK resolve differently enough that banks adjust their collateral requirements to the code, and Schoenherr and Starmans (2022) show theoretically and empirically that the optimal degree of debtor-friendliness depends on the surrounding environment rather than being a universal constant.

For Vietnamese work this matters twice over. The Z'-Score coefficients used in Chapter 34 were calibrated outside Vietnam, and the outcome distribution they implicitly condition on was generated by a bankruptcy system Vietnam does not have.

40.2 The Subchapter V Experiment

Hotchkiss, Iverson, and Zheng (2026) provide the cleanest recent evidence that the second factor in Equation 40.1 is a policy choice rather than a technological constant, and they do it in the segment where the question bites hardest: small firms.

The stylized fact that motivates the paper inverts the familiar one. For large U.S. public companies, Chapter 11 liquidations are rare and most filers emerge with some or all operations intact. For small businesses the picture reverses: 70% of small businesses entering bankruptcy between 2010 and 2019 filed directly into Chapter 7 liquidation, and among those that did enter Chapter 11, only about one-third confirmed a plan. The rest were converted to Chapter 7 or dismissed from court. Dismissal is not a soft landing, because the firm returns to its creditors with no debt relief and typically ceases to operate (Morrison 2007; Iverson 2018).

Three frictions explain the gap. First, the direct costs of Chapter 11 are largely fixed, so they scale badly against a small asset base (Bris, Welch, and Zhu 2006; Dou et al. 2021). Second, absolute priority wipes out pre-bankruptcy equity, but in a small business the owner-manager's relationships with suppliers and customers often *are* the going-concern value; new creditor-owners cannot easily replace what they have just expropriated, and trade creditors and regional banks rarely want to hold the equity in the first place. Third, dispersed creditors with differing priorities create coordination problems, and a senior secured lender with a small claim often prefers a quick foreclosure to a negotiated plan even when continuation is more valuable in aggregate.

The Small Business Reorganization Act of 2019 created Subchapter V of Chapter 11, effective 19 February 2020, which addresses all three at once for debtors whose aggregate non-contingent liquidated debts do not exceed $7.5 million. The eligibility ceiling was originally $2.725 million and was raised to $7.5 million by the CARES Act on 27 March 2020, one month after the procedure opened. The three operative changes are: a plan may be confirmed over creditor objection without eliminating the owners' equity, provided the firm commits its projected disposable income for three to five years; the creditors' committee and the disclosure statement are no longer required; and a Subchapter V trustee is appointed to mediate rather than to displace management.

The eligibility ceiling is what makes this an experiment rather than an anecdote. Table 40.1 collects the headline estimates.

Table 40.1: Estimated effects of Subchapter V eligibility on small-business Chapter 11 outcomes, from Hotchkiss, Iverson, and Zheng (2026). "pp" denotes percentage points. The first stage is a 41.9 pp jump in Subchapter V usage at the threshold ($F = 71.2$) in the wide bandwidth and 37.1 pp ($F = 19.7$) in the narrow one.

Outcome	Design	Estimate	Benchmark
Plan confirmed	OLS, 6,431 cases below $15m	+18.4 pp	base rate 31.9%
Plan confirmed	Fuzzy RDD, $4–11m ($n = 999$)	+32.0 pp	17.3% for non-SubV
Plan confirmed	Fuzzy RDD, $6–9m	+49.5 pp	17.3% for non-SubV
Plan confirmed	DiD, intent-to-treat	+12.5 pp	—
Days to confirmation	OLS / fuzzy RDD	−24% / −35%	mean 319 days

Outcome	Design	Estimate	Benchmark
Unsecured creditor recovery	Fuzzy RDD	+11.9 pp	—
Post-bankruptcy survival	Fuzzy RDD, \$4–11m	+21 pp	insignificant at \$6–9m

Two features of this table deserve emphasis beyond the individual numbers.

The first is that the effect is enormous relative to the base rate. A firm just below the threshold more than doubles its probability of reorganizing relative to an otherwise similar firm just above it. Whether a distressed small firm survives is, in this range, determined more by which procedure it may use than by anything on its balance sheet. The corollary for Equation 40.1 is that the conditional loss factor is a policy parameter.

The second is that the obvious objection fails. A regime that lets owners keep equity over creditor objection should, on the standard argument, keep unviable firms alive at creditors' expense; poor post-bankruptcy performance among large emergent firms has long been read as evidence of exactly that continuation bias (Hotchkiss 1995; Morrison 2007). Here, expected unsecured recoveries *rise* by 11.9 percentage points and post-bankruptcy survival rises rather than falls. The gains from avoiding a costly liquidation appear to be shared rather than redistributed, which is what one would expect if the pre-reform liquidation rate reflected transaction costs and coordination failure rather than a correct sorting of viable from unviable firms.

40.3 Reading the Evidence from Vietnam

Four implications carry over, none of them mechanically.

The distress zone means something different here. Vietnam's formal insolvency framework is the Law on Bankruptcy No. 51/2014/QH13, effective 1 January 2015, which provides both a business-recovery procedure and liquidation. But as documented in Section 13.5, observed exits from the Vietnamese listed universe are dominated by involuntary delisting for losses exceeding charter capital, failure to file financial statements, and regulatory enforcement, not by court-supervised reorganizations that produce an observable recovery rate. The empirical content of a Z'' -Score below 1.10 in Vietnam is therefore closer to "expect delisting and a large negative terminal return" than to "expect a supervised process with a measurable creditor recovery". This is precisely why the delisting-return imputation in Section 13.5 is not a technicality: for Vietnamese firms it is doing the work that a bankruptcy court's docket does in U.S. data.

There is no Vietnamese FJC database. The Hotchkiss–Iverson–Zheng design rests on a public, case-level census of filings with liabilities recorded at filing, supplemented by hand-collected schedules from court records. Nothing equivalent is available for Vietnam, and building it would be a research contribution in its own right. Until it exists, Vietnamese distress research is largely confined to the listed universe, which is the *least* representative slice of the small-business population the theory is about.

The asset-redeployment friction is plausibly larger. Bernstein, Colonnelli, and Iverson (2019) show that assets of liquidated U.S. small businesses are redeployed poorly and end up underutilized, which is the mechanism that makes excess liquidation socially costly. Thin secondary markets for used capital equipment and the administrative complexity of transferring land-use rights make the same friction plausibly more severe in Vietnam, though we know of no direct estimate.

The reform frontier is open. Hotchkiss, Iverson, and Zheng (2026) note that at least eleven countries have adopted simplified insolvency procedures for small businesses over the past decade, several of them in the region, and Gurrea-Martínez (2024) surveys insolvency reform in emerging economies specifically. Vietnam is not among the adopters. For a researcher this cuts both ways: there is no ready-made Vietnamese natural experiment of this type to exploit, but the ex-ante evaluation question is wide open and the identification template below is portable.

40.4 Threshold Rules as Research Designs

The methodological content of Hotchkiss, Iverson, and Zheng (2026) is worth separating from the institutional content, because eligibility thresholds are everywhere in Vietnamese regulation and the design travels even where the substance does not.

40.4.1 Fuzzy Regression Discontinuity

Eligibility is a deterministic function of a running variable, but *take-up* is not: 39% of eligible firms in the paper's sample chose a traditional Chapter 11 anyway, and a handful of apparently ineligible firms were admitted once the court reclassified part of their liabilities as contingent. Treatment therefore jumps at the cutoff without going from zero to one, which is the fuzzy case. Write $s_i = L_i - c$ for the centred running variable and $B_i = \mathbf{1}\{s_i \le 0\}$ for eligibility. The design is two-stage least squares with B_i as the excluded instrument:

$$D_i = \alpha + \pi B_i + f(s_i) + B_i \cdot g(s_i) + X_i'\gamma + u_i \tag{40.2}$$

$$y_i = \tilde{\alpha} + \beta \widehat{D}_i + \tilde{f}(s_i) + B_i \cdot \tilde{g}(s_i) + X_i'\tilde{\gamma} + \varepsilon_i \tag{40.3}$$

where D_i indicates actual use of the simplified procedure, and f and g are low-order polynomials estimated separately on each side of the cutoff (Imbens and Lemieux 2008; D. S. Lee and Lemieux 2010). In the just-identified case β equals the ratio of the reduced-form jump in y to the first-stage jump π, which is the Wald form of the estimator and the most useful way to read it. What β identifies is a local average treatment effect: the effect for firms near the cutoff that use the procedure *because* they are eligible. It is not the effect for a firm with $1 million in liabilities, and it is not the effect of making the procedure mandatory.

40.4.2 What Makes the Design Credible

Four checks do the work, and all four are reported in the paper.

1. **Density continuity.** If firms manipulate the running variable to qualify, the density of s_i jumps at the cutoff and the comparison is between self-selected groups. A McCrary (2008) test finds no discontinuity ($p = 0.25$). The institutional argument reinforces the statistical one: a distressed firm cannot easily retire debt to get under a ceiling, and the ceiling itself was raised unexpectedly a month after the procedure opened.
2. **Covariate smoothness.** Leverage, secured-debt share, total assets, joint-filing status, and creditor counts are all continuous through the threshold, individually and jointly.
3. **Donut-hole robustness.** Dropping cases in the $7–7.5 million band, where any manipulation would concentrate, leaves the coefficients essentially unchanged.
4. **Bandwidth sensitivity, reported honestly.** The confirmation effect is +32.0 pp on $4–11 million and +49.5 pp on $6–9 million. Narrowing the window strengthens identification and weakens power, and the paper shows both faces of that trade: the point estimate rises, but the survival effect that is significant on the wide window is not significant on the narrow one. Reporting a bandwidth path is not a robustness ritual; it is the main result. Data-driven bandwidth selection with bias-corrected inference (Calonico, Cattaneo, and Titiunik 2014) is the natural complement, not a substitute for showing the path.

One threat that no internal check can settle is selection into the sample itself. If eligibility induces firms that would have restructured out of court to file instead, the comparison is contaminated even with a perfect discontinuity. The paper's response is a useful template: rather than arguing the threat away, it quantifies how much selection would be needed to overturn the finding, and shows that removing 29 randomly chosen reorganizing Subchapter V cases moves the estimate from 32.0 to 29.1 percentage points, while driving it to zero would require removing at least 120.

40.4.3 Difference-in-Differences as Intent-to-Treat

Because the procedure did not exist before February 2020, the same threshold supports a second design,

$$y_i = \alpha + \beta_1 \text{Treated}_i + \beta_2 \text{Post}_t + \beta_3 \left(\text{Treated}_i \times \text{Post}_t\right) + X_i'\gamma + \varepsilon_i \tag{40.4}$$

with firms below the ceiling as treated and firms just above it as a never-treated control. Here β_3 is an intent-to-treat effect, and it is smaller than the RDD estimate for two compounding reasons: not every eligible firm uses the procedure, and the pre-2020 sample can only be split on *total* rather than non-contingent liabilities, so some genuinely eligible firms are misclassified into the control group. The reconciliation is instructive. Dividing the ITT of 12.5 percentage points by the 42 percentage-point first stage gives 29.8, close to the RDD estimate of 32.0. An ITT and a LATE that look inconsistent are frequently the same parameter divided by different denominators.

40.4.4 Adapting the Design to Vietnam

Vietnamese regulation is full of bright-line rules: listing and delisting triggers keyed to accumulated losses against charter capital, to consecutive loss years, and to audit opinions; size-based definitions that gate access to support programmes and preferential tax treatment; foreign-ownership ceilings. Each is a candidate forcing variable.

The warning is that the manipulation test which was close to a formality in Hotchkiss, Iverson, and Zheng (2026) becomes load-bearing here. A distressed U.S. firm cannot readily pay down debt to slip under a liability ceiling. A Vietnamese firm approaching a threshold defined on *reported* accumulated losses, revenue, or headcount frequently can, and the accrual-management incentives documented in the earnings-management chapter supply both motive and means. Any Vietnamese RDD built on an accounting threshold needs the density test, a donut-hole specification, and an argument about why the reporting date precedes the firm's knowledge of the rule — and should be prepared to conclude that the design does not work. Threshold rules also change; the exact wording and effective dates of the governing circular have to be checked against the sample period rather than assumed.

40.5 Implementation: A Fuzzy RDD from Scratch

No Vietnamese case-level insolvency panel exists, so we simulate one whose data-generating process is calibrated to the estimates in Table 40.1: a 17% baseline confirmation rate, roughly 60% take-up among eligible firms, and a true local effect of 32 percentage points. Because the truth is known, the estimator can be checked against it, and the selection problem that motivates the design can be built in deliberately. Unobserved case complexity raises take-up

and lowers the baseline probability of confirmation, so a naive comparison of users to non-users is biased *downwards* — the same direction Hotchkiss, Iverson, and Zheng (2026) argue applies in their data, where Subchapter V filers are if anything more distressed than traditional Chapter 11 filers.

```
rng = np.random.default_rng(2026)
CUTOFF = 7.5        # eligibility ceiling, in millions of currency units
TAU = 0.32          # true local effect on the probability of confirmation

n_draw = 2000
liab = CUTOFF + rng.normal(0.0, 2.5, n_draw)
liab = liab[(liab > 2.0) & (liab < 13.0)]
n = liab.size

# Unobserved case complexity: raises take-up, lowers baseline success.
complexity = rng.normal(0.0, 1.0, n)
eligible = (liab <= CUTOFF).astype(float)

# Take-up is imperfect on both sides: some eligible firms decline the
# simplified track, and a few ineligible ones are admitted after the court
# reclassifies part of their liabilities as contingent.
p_take = np.where(eligible == 1.0,
                  np.clip(0.60 + 0.12 * complexity, 0.05, 0.95),
                  0.03)
simplified = (rng.random(n) < p_take).astype(float)

p_conf = np.clip(
    0.17 + 0.010 * (liab - CUTOFF) - 0.05 * complexity + TAU * simplified,
    0.01, 0.99,
)
confirmed = (rng.random(n) < p_conf).astype(float)

# A covariate that is smooth through the threshold, for the placebo check.
leverage = 3.0 + 0.15 * (liab - CUTOFF) + rng.normal(0.0, 1.2, n)

print(f"Cases simulated:                 {n:,}")
print(f"Eligible (below cutoff):         {eligible.mean():6.1%}")
print(f"Use simplified track | eligible: {simplified[eligible ==
 ↪  1].mean():6.1%}")
print(f"Use simplified track | above:    {simplified[eligible ==
 ↪  0].mean():6.1%}")
print(f"Plan confirmed (overall):        {confirmed.mean():6.1%}")
```

```
Cases simulated:                 1,940
Eligible (below cutoff):          50.8%
Use simplified track | eligible:  58.9%
Use simplified track | above:      2.5%
Plan confirmed (overall):         27.5%
```

The estimation machinery is short enough to write out in full. Weighted least squares with heteroskedasticity-robust standard errors is the building block; two-stage least squares is the same algebra applied to the projection of the endogenous regressor onto the instrument set.

```
from math import erfc, sqrt


def tri_weights(s, h):
    """Triangular kernel weights on the centred running variable."""
    return np.clip(1.0 - np.abs(s) / h, 0.0, None)


def robust_wls(X, y, w):
    """WLS coefficients with HC1 standard errors."""
    Xw = X * w[:, None]
    XtWX = X.T @ Xw
    beta = np.linalg.solve(XtWX, Xw.T @ y)
    e = y - X @ beta
    G = X * (w * e)[:, None]
    bread = np.linalg.inv(XtWX)
    n_obs, k = X.shape
    V = bread @ (G.T @ G) @ bread * (n_obs / max(n_obs - k, 1))
    return beta, np.sqrt(np.diag(V))


def robust_2sls(X, Z, y, w):
    """Weighted 2SLS with HC1 standard errors; Z must have as many columns as
     ↪  X."""
    Zw = Z * w[:, None]
    Xhat = Z @ np.linalg.solve(Z.T @ Zw, Zw.T @ X)
    A = Xhat.T @ (X * w[:, None])
    beta = np.linalg.solve(A, (Xhat * w[:, None]).T @ y)
    e = y - X @ beta
    G = Xhat * (w * e)[:, None]
    Ainv = np.linalg.inv(A)
    n_obs, k = X.shape
    V = Ainv @ (G.T @ G) @ Ainv.T * (n_obs / max(n_obs - k, 1))
```

```
    return beta, np.sqrt(np.diag(V))


def fuzzy_rd(running, treat, outcome, cutoff=CUTOFF, h=3.5, donut=0.0):
    """
    Local-linear fuzzy RD with separate slopes on each side of the cutoff.

    Returns the first stage, the reduced form (intent-to-treat), and the
    2SLS local average treatment effect, all estimated on the same sample.
    """
    s = running - cutoff
    m = (np.abs(s) <= h) & (np.abs(s) >= donut)
    s, d, y = s[m], treat[m], outcome[m]
    below = (s <= 0).astype(float)
    w = tri_weights(s, h)
    one = np.ones_like(s)

    Z = np.column_stack([one, below, s, below * s])   # instrument set
    X = np.column_stack([one, d, s, below * s])       # structural regressors

    b_fs, se_fs = robust_wls(Z, d, w)
    b_rf, se_rf = robust_wls(Z, y, w)
    b_iv, se_iv = robust_2sls(X, Z, y, w)

    return {
        "h": h, "n": int(m.sum()),
        "first_stage": b_fs[1], "first_stage_se": se_fs[1],
        "F": (b_fs[1] / se_fs[1]) ** 2,
        "itt": b_rf[1], "itt_se": se_rf[1],
        "late": b_iv[1], "late_se": se_iv[1],
        "wald": b_rf[1] / b_fs[1],
    }
```

The naive comparison and the RD estimate can now be put side by side.

```
b_naive, se_naive = robust_wls(
    np.column_stack([np.ones(n), simplified]), confirmed, np.ones(n)
)
main = fuzzy_rd(liab, simplified, confirmed, h=3.5)

print("=" * 66)
print("  Effect on the probability of confirming a plan")
```

```
print("=" * 66)
print(f"  True local effect (by construction)      {TAU:>10.3f}")
print(f"  Naive OLS, users vs non-users            {b_naive[1]:>10.3f}"
      f"   (se {se_naive[1]:.3f})")
print("-" * 66)
print(f"  Fuzzy RD, bandwidth +/-{main['h']:.1f}, n = {main['n']:,}")
print(f"    First stage (eligibility -> use)
 ↪  {main['first_stage']:>10.3f}"
      f"   (F = {main['F']:.1f})")
print(f"    Reduced form (intent-to-treat)         {main['itt']:>10.3f}"
      f"   (se {main['itt_se']:.3f})")
print(f"    2SLS local average treatment effect    {main['late']:>10.3f}"
      f"   (se {main['late_se']:.3f})")
print(f"    Wald ratio, ITT / first stage          {main['wald']:>10.3f}")
print("=" * 66)
```

```
==================================================================
  Effect on the probability of confirming a plan
==================================================================
  True local effect (by construction)           0.320
  Naive OLS, users vs non-users                 0.298   (se 0.023)
------------------------------------------------------------------
  Fuzzy RD, bandwidth +/-3.5, n = 1,690
    First stage (eligibility -> use)            0.562   (F = 253.7)
    Reduced form (intent-to-treat)              0.232   (se 0.043)
    2SLS local average treatment effect         0.414   (se 0.075)
    Wald ratio, ITT / first stage               0.414
==================================================================
```

The Wald ratio and the 2SLS coefficient agree to machine precision, because the model is just identified. The naive comparison understates the true effect, exactly as the selection story predicts.

Bandwidth sensitivity and a donut-hole specification come next. The point of the table is not to find the specification with the largest coefficient but to see how the estimate and its precision move together.

```
rows = []
for h in [1.5, 2.0, 2.5, 3.0, 3.5]:
    r = fuzzy_rd(liab, simplified, confirmed, h=h)
    rows.append({"Bandwidth": f"+/-{h:.1f}", "N": r["n"],
```

```
                "First stage": r["first_stage"], "F": r["F"],
                "LATE": r["late"], "SE": r["late_se"],
                "t": r["late"] / r["late_se"]})

r_donut = fuzzy_rd(liab, simplified, confirmed, h=3.5, donut=0.5)
rows.append({"Bandwidth": "+/-3.5 donut", "N": r_donut["n"],
             "First stage": r_donut["first_stage"], "F": r_donut["F"],
             "LATE": r_donut["late"], "SE": r_donut["late_se"],
             "t": r_donut["late"] / r_donut["late_se"]})

bw_table = pd.DataFrame(rows)
print(f"Bandwidth path for the fuzzy RD estimate (true effect = {TAU:.2f})")
print(bw_table.to_string(index=False, float_format=lambda v: f"{v:8.3f}"))
```

```
Bandwidth path for the fuzzy RD estimate (true effect = 0.32)
   Bandwidth    N  First stage        F     LATE       SE        t
      +/-1.5  873        0.557  117.130    0.417    0.111    3.741
      +/-2.0 1148        0.563  160.669    0.418    0.095    4.401
      +/-2.5 1376        0.563  195.700    0.401    0.086    4.670
      +/-3.0 1546        0.562  226.385    0.407    0.080    5.107
      +/-3.5 1690        0.562  253.732    0.414    0.075    5.512
+/-3.5 donut 1369        0.559  104.089    0.363    0.111    3.284
```

Two validity checks follow. The density test looks for bunching just below the cutoff, which would indicate that firms manage the running variable to qualify; the covariate check runs the reduced form on a variable that treatment cannot affect.

```
def density_jump(x, cutoff=CUTOFF, h=3.5, binw=0.25, n_boot=400, seed=7):
    """
    McCrary-style test: a local-linear density fitted separately on each side
    of the cutoff, with a bootstrap standard error for the log difference.
    """
    n_bins = int(round(2 * h / binw))
    edges = np.linspace(-h, h, n_bins + 1)
    mid = (edges[:-1] + edges[1:]) / 2.0

    def one_jump(sample):
        s = sample - cutoff
        s = s[np.abs(s) <= h]
        dens = np.histogram(s, bins=edges)[0] / (sample.size * binw)
        fitted = []
```

```
        for side in (-1.0, 1.0):
            m = (mid * side) > 0
            A = np.column_stack([np.ones(int(m.sum())), mid[m]])
            b, _ = robust_wls(A, dens[m], tri_weights(mid[m], h))
            fitted.append(max(b[0], 1e-12))
        return np.log(fitted[1]) - np.log(fitted[0])

    theta = one_jump(x)
    boot_rng = np.random.default_rng(seed)
    draws = np.array([
        one_jump(x[boot_rng.integers(0, x.size, x.size)]) for _ in
↪   range(n_boot)
    ])
    se = draws.std(ddof=1)
    z = theta / se if se > 0 else 0.0
    return theta, se, erfc(abs(z) / sqrt(2.0))


theta, se_theta, p_density = density_jump(liab)
cov_check = fuzzy_rd(liab, simplified, leverage, h=3.5)

print("Validity checks")
print("-" * 66)
print(f"  Density log-jump at the cutoff   {theta:>8.3f}"
      f"   (se {se_theta:.3f}, p = {p_density:.3f})")
print(f"  Leverage jump at the cutoff      {cov_check['itt']:>8.3f}"
      f"   (se {cov_check['itt_se']:.3f}, "
      f"t = {cov_check['itt'] / cov_check['itt_se']:.2f})")
print("-" * 66)
print("  Neither should be distinguishable from zero.")
```

```
Validity checks
------------------------------------------------------------------
  Density log-jump at the cutoff      0.034   (se 0.091, p = 0.706)
  Leverage jump at the cutoff        -0.088   (se 0.122, t = -0.72)
------------------------------------------------------------------
  Neither should be distinguishable from zero.
```

```
bins = np.linspace(CUTOFF - 3.5, CUTOFF + 3.5, 29)
idx = np.digitize(liab, bins) - 1
mids = (bins[:-1] + bins[1:]) / 2.0
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
panels = [
    (axes[0], simplified, "First stage: take-up of the simplified track",
     "Share using the procedure"),
    (axes[1], confirmed, "Reduced form: plan confirmation",
     "Share confirming a plan"),
]

for ax, series, title, ylab in panels:
    means = np.array([
        series[idx == b].mean() if (idx == b).sum() > 0 else np.nan
        for b in range(len(bins) - 1)
    ])
    ax.scatter(mids, means, s=34, color=colors["primary"], alpha=0.85,
               zorder=3, label="Binned mean")
    for side, col in [(-1.0, colors["tertiary"]), (1.0,
     ↪  colors["quaternary"])]:
        m = ((mids - CUTOFF) * side > 0) & ~np.isnan(means)
        A = np.column_stack([np.ones(int(m.sum())), mids[m] - CUTOFF])
        b, _ = robust_wls(A, means[m], tri_weights(mids[m] - CUTOFF, 3.5))
        grid = np.linspace(mids[m].min(), mids[m].max(), 50)
        ax.plot(grid, b[0] + b[1] * (grid - CUTOFF), color=col, linewidth=2)
    ax.axvline(CUTOFF, color=colors["distress"], linestyle="--", linewidth=2,
               label="Eligibility ceiling")
    ax.set_xlabel("Non-contingent liabilities at filing (millions)")
    ax.set_ylabel(ylab)
    ax.set_title(title)
    ax.legend(loc="best")

plt.tight_layout()
plt.show()
```

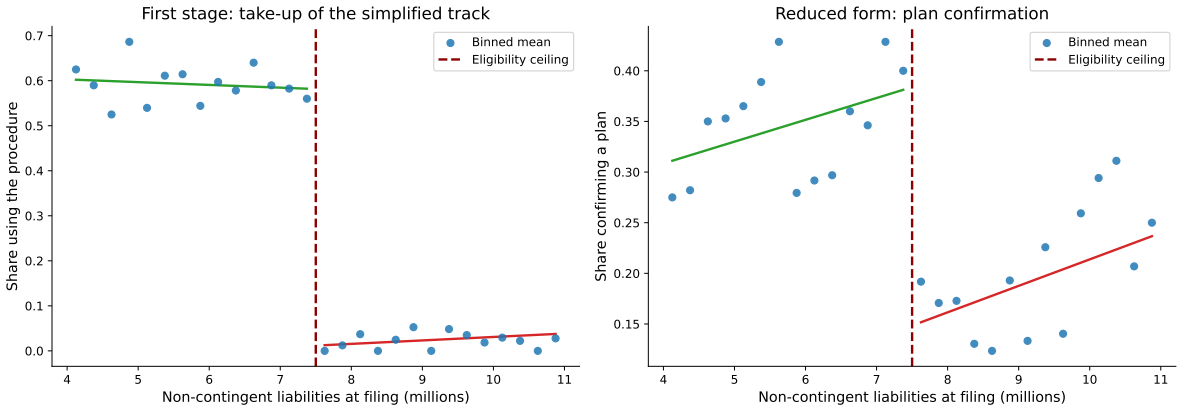


Figure 40.1: Fuzzy regression discontinuity at an eligibility threshold. The left panel shows the first stage: take-up of the simplified procedure jumps discontinuously at the ceiling but does not move from zero to one, which is what makes the design fuzzy rather than sharp. The right panel shows the reduced form: the probability of confirming a plan falls sharply for firms just above the ceiling. Points are binned means; lines are local-linear fits estimated separately on each side.

Finally, the difference-in-differences counterpart. The pre-period cohort files before the procedure exists, and its treatment indicator has to be built from *total* liabilities, because the non-contingent split is unobservable before the rule creates a reason to record it. That measurement error attenuates the intent-to-treat estimate, and rescaling by the first stage recovers the structural parameter.

```
did_rng = np.random.default_rng(20200219)


def make_cohort(size, post):
    """One filing cohort; the simplified track exists only when post is
     ↪  True."""
    nc_liab = CUTOFF + did_rng.normal(0.0, 2.5, size)
    total_liab = nc_liab + did_rng.gamma(1.5, 0.6, size)   # contingent
 ↪  claims
    cplx = did_rng.normal(0.0, 1.0, size)
    can_use = (nc_liab <= CUTOFF) & bool(post)
    p_use = np.where(can_use, np.clip(0.60 + 0.12 * cplx, 0.05, 0.95), 0.0)
    used = (did_rng.random(size) < p_use).astype(float)
    p_c = np.clip(0.17 + 0.010 * (nc_liab - CUTOFF) - 0.05 * cplx + TAU *
 ↪  used,
                  0.01, 0.99)
    return pd.DataFrame({
```

```
        "total_liab": total_liab,
        "used": used,
        "confirmed": (did_rng.random(size) < p_c).astype(float),
        "post": float(post),
    })


panel = pd.concat([make_cohort(4000, False), make_cohort(4000, True)],
                  ignore_index=True)
panel = panel[(panel["total_liab"] >= 4.0) & (panel["total_liab"] <=
 ↪  11.0)].copy()
panel["treated"] = (panel["total_liab"] <= CUTOFF).astype(float)

D = np.column_stack([
    np.ones(len(panel)),
    panel["treated"].to_numpy(),
    panel["post"].to_numpy(),
    (panel["treated"] * panel["post"]).to_numpy(),
])
unit_w = np.ones(len(panel))
b_itt, se_itt = robust_wls(D, panel["confirmed"].to_numpy(), unit_w)
b_first, se_first = robust_wls(D, panel["used"].to_numpy(), unit_w)

print("=" * 66)
print(f"  Difference-in-differences, n = {len(panel):,}"
      f"   (true effect = {TAU:.2f})")
print("=" * 66)
print(f"  Treated x Post on confirmation (ITT)   {b_itt[3]:>8.3f}"
      f"   (se {se_itt[3]:.3f})")
print(f"  Treated x Post on take-up (1st stage)  {b_first[3]:>8.3f}"
      f"   (se {se_first[3]:.3f})")
print(f"  Rescaled: ITT / first stage            {b_itt[3] /
 ↪  b_first[3]:>8.3f}")
print("=" * 66)
```

```
==================================================================
  Difference-in-differences, n = 6,398   (true effect = 0.32)
==================================================================
  Treated x Post on confirmation (ITT)      0.111   (se 0.021)
  Treated x Post on take-up (1st stage)     0.457   (se 0.016)
  Rescaled: ITT / first stage               0.243
==================================================================
```

The ITT lands far below the true effect because most nominally treated firms never use the procedure and some are misclassified by total rather than non-contingent liabilities. Dividing by the first stage recovers the parameter, which is the same reconciliation Hotchkiss, Iverson, and Zheng (2026) report as $12.5/0.42 = 29.8$ percentage points against an RDD estimate of 32.0.

The template is what transfers. Given a Vietnamese threshold rule, a case-level or firm-level panel that records the running variable as the rule measures it, and a defensible argument that firms could not position themselves around the cutoff, the same chunks deliver a first stage, a LATE, a bandwidth path, and the two validity checks that decide whether any of it means anything.

41 Practical Considerations and Limitations

41.1 Known Limitations of Tobin's Q in Vietnam

Several issues affect the reliability of Tobin's Q estimates for Vietnamese firms:

1. **Book value as replacement cost proxy**: The simplified Q measure assumes that book value of assets approximates replacement cost. Under VAS's heavy reliance on historical cost, this assumption may be more problematic than in IFRS-adopting countries, particularly for firms with significant land use rights or long-lived tangible assets.
2. **Market microstructure effects**: Vietnam's daily price limits can prevent market prices from reaching equilibrium, potentially distorting the market value component. Foreign ownership limits may create artificial price premiums for certain stocks.
3. **Preferred stock rarity**: While this simplifies the computation (most Vietnamese firms have no preferred stock), it means the BE calculation is dominated by common equity, which may not capture all ownership claims.
4. **Cross-listing effects**: Some Vietnamese firms are listed on multiple venues (HOSE, HNX, UPCoM, or even foreign exchanges). Care must be taken to use consistent price and share data.

41.2 Known Limitations of Altman Z-Score in Vietnam

1. **Calibration sample**: The original Z-Score was estimated on mid-20th-century U.S. manufacturing firms. The Z' '-Score for emerging markets is more appropriate but was still estimated on a non-Vietnamese sample.
2. **Accounting differences**: VAS accounting standards produce financial ratios with different distributional properties than US GAAP or IFRS data, potentially affecting the discriminant function's classification accuracy.
3. **Banking and financial firms**: The Z-Score was not designed for financial institutions, which have fundamentally different balance sheet structures. Banks, insurance companies, and securities firms should be excluded or analyzed separately.

4. **Implicit guarantees**: SOEs and firms connected to major economic groups may have implicit support that reduces actual default risk below Z-Score predictions.

41.3 Recommendations for Researchers

Based on our analysis, we offer the following recommendations for researchers working with Vietnamese data:

1. **Use the Z''-Score (Emerging Market) variant** as the primary distress measure for Vietnamese non-financial firms.
2. **Report multiple Q variants** (Simple and Chung-Pruitt) to demonstrate robustness.
3. **Winsorize at 1%/99%** to mitigate the impact of data errors and extreme outliers.
4. **Compute industry-adjusted measures** when making cross-sectional comparisons.
5. **Use founding age** rather than listing age when available, but report both to distinguish organizational maturity from capital market experience.
6. **Exclude financial firms** from Z-Score analysis.
7. **Account for state ownership** as a moderating variable in valuation and distress studies.
8. **Do not read the Z-Score as a recovery forecast.** It scores the probability of distress, not what the legal system does with a distressed firm; see Chapter 40.

42 Summary

This chapter presented a treatment of three fundamental corporate finance measures (i.e., Tobin's Q, the Altman Z-Score, and Company Age). We covered the theoretical foundations of each measure and provided extensive visualizations of cross-sectional and time-series patterns.

Key findings from our analysis of Vietnamese listed firms include:

1. **Tobin's Q** varies substantially across industries and exchanges, with technology and consumer-facing sectors typically commanding higher valuations. HOSE-listed firms tend to have higher Q values than HNX or UPCoM firms, reflecting both firm quality differences and liquidity effects.
2. **The Altman Z-Score** reveals that a meaningful proportion of Vietnamese firms operate in or near the distress zone, though the appropriate variant matters; the emerging market Z'' -Score provides more nuanced classification than the original model. Financial health shows significant time-series variation, with notable deterioration during economic downturns.
3. **Company age** in Vietnam requires careful treatment due to the equitization of SOEs, which creates large gaps between founding age and listing age. This distinction is substantively important for understanding the relationship between maturity, valuation, and financial stability.
4. **Distress resolution** is a separate question from distress prediction, and the Z-Score speaks only to the latter. Hotchkiss, Iverson, and Zheng (2026) show that a change in insolvency procedure alone more than doubled the probability that a small U.S. business reorganized rather than liquidated, without harming creditor recoveries. Because Vietnamese exits are dominated by involuntary delisting rather than court-supervised reorganization, a Vietnamese Z'' -Score in the distress zone maps to a materially different outcome distribution than the same score in the sample the coefficients were fitted to. Chapter 40 develops this point and implements the fuzzy regression discontinuity and difference-in-differences designs that eligibility thresholds make available.

43 Disclosure Quality and Timing

Corporate disclosure is the primary mechanism through which firms communicate with capital markets. The quality, quantity, and timing of disclosures shape the information environment in which investors form expectations, price securities, and allocate capital. A large theoretical and empirical literature, surveyed by Healy and Palepu (2001) and Beyer et al. (2010), demonstrates that disclosure decisions have first-order effects on the cost of capital, liquidity, and investment efficiency.

This chapter brings two decades of disclosure research to the Vietnamese market, where several institutional features create a distinctive setting. First, Vietnam's regulatory framework, anchored by Circular 155/2015/TT-BTC (amended by Circular 96/2020/TT-BTC) and enforced by the State Securities Commission (SSC), mandates periodic and event-driven disclosures with specific deadlines that differ from U.S. and European norms. Second, the dominance of retail investors and relatively thin analyst coverage means that corporate disclosures are often the *primary* source of firm-specific information, amplifying their economic importance. Third, the ongoing transition from Vietnamese Accounting Standards (VAS) toward IFRS convergence introduces time-varying changes in disclosure requirements that create natural variation for empirical analysis.

43.1 Theoretical Foundations

43.1.1 Voluntary Disclosure Theory

The foundational model of voluntary disclosure is due to Verrecchia (1983), who shows that in a setting where investors know a manager possesses private information, an *unraveling* equilibrium emerges: silence is interpreted as bad news, so managers disclose unless the proprietary cost of disclosure exceeds its benefit. The key insight is that non-disclosure is informative because investors rationally infer that withheld information is unfavourable.

Diamond (1985) extends the analysis to a multi-period setting where the firm's disclosure policy affects the precision of public information and hence the incentives for private information acquisition. The central trade-off is between reducing information asymmetry (which lowers the cost of capital) and reducing the rents that informed traders earn (which may discourage

monitoring). Diamond and Verrecchia (1991) formalize the link between disclosure and liquidity: by reducing adverse selection, voluntary disclosure narrows bid-ask spreads and increases the willingness of uninformed investors to trade.

The empirical prediction is that firms with higher-quality disclosure should enjoy:

1. Lower cost of equity capital (Botosan 1997; Botosan and Plumlee 2002)
2. Lower cost of debt (Sengupta 1998)
3. Higher liquidity and lower bid-ask spreads (Diamond and Verrecchia 1991; Lang, Lins, and Maffett 2012)
4. More efficient investment decisions (Biddle, Hilary, and Verdi 2009)

43.1.2 Strategic Disclosure Timing

Not all disclosure is voluntary in timing, but managers retain discretion over *when,* within permissible windows, to release information. Patell and Wolfson (1982) document that firms tend to release good news during trading hours and bad news after market close. DellaVigna and Pollet (2009) show that earnings announced on Fridays (i.e., when investor attention is lower) generate smaller immediate reactions and larger post-announcement drift, consistent with limited attention. Hirshleifer, Lim, and Teoh (2009) generalize this finding: extraneous events that distract investors (such as a large number of concurrent announcements) reduce the immediate price response to earnings news.

In Vietnam, several features make strategic timing particularly relevant. The concentrated disclosure calendar, where many firms file near regulatory deadlines, creates natural variation in announcement congestion. The retail-dominated investor base may be more susceptible to attention effects than institutional investors. The regulatory structure, which imposes penalties for late filing but allows discretion within the permissible window, creates a setting in which the *choice* of filing date is informative.

43.1.3 Disclosure Quality in Emerging Markets

R. Ball, Robin, and Wu (2003) argue that accounting quality is shaped more by reporting *incentives* than by accounting *standards.* In institutional environments with weak enforcement, concentrated ownership, and close alignment between financial and tax reporting, firms may produce lower-quality disclosures even under nominally rigorous standards. Leuz, Nanda, and Wysocki (2003) confirm this pattern internationally: earnings management (an inverse proxy for disclosure quality) is highest in countries with weak investor protection, concentrated ownership, and less developed capital markets.

Vietnam exhibits several of these features. Bushman et al. (2004) classify determinants of transparency into governance factors (legal origin, judicial efficiency, minority protection) and political factors (state ownership, government intervention). Vietnam's civil-law tradition,

significant state ownership in listed firms, and evolving enforcement capacity suggest that disclosure quality may be lower on average than in developed markets, but with substantial cross-sectional variation driven by firm-level governance and ownership structures.

43.2 Regulatory Framework

43.2.1 Mandatory Disclosure Requirements

Vietnamese disclosure regulation operates through a hierarchy of legal instruments:

- **Securities Law (2019):** Establishes the general obligation of listed firms to disclose information truthfully, accurately, completely, and on time (Article 118).
- **Circular 155/2015/TT-BTC** (amended by **Circular 96/2020/TT-BTC**): Specifies the content, format, and deadlines for periodic and event-driven disclosures.
- **SSC decisions and guidance:** Provide implementation details and sector-specific requirements.

The key periodic reporting deadlines are in Table 43.1

Table 43.1: Periodic disclosure deadlines under Vietnamese securities regulation.

Report Type	Deadline	Audit Requirement
Annual financial statements	90 days after fiscal year-end	Audited
Semi-annual financial statements	45 days after period-end	Reviewed
Quarterly financial statements	20 days after quarter-end	Unaudited
Annual report	110 days after fiscal year-end	N/A (narrative)

Event-driven (ad hoc) disclosures must be filed within 24 hours for material events, including changes in ownership exceeding 1% by major shareholders, board resolutions on dividends or capital increases, and any event that may materially affect the share price.

43.2.2 Penalties for Non-Compliance

The SSC may impose administrative fines for late or incomplete disclosure, typically ranging from VND 50–100 million for minor violations and up to VND 500 million for material omissions. While these amounts are modest relative to firm size for large-cap companies, the reputational cost and the risk of trading suspension provide additional deterrence.

43.3 Data Construction

43.3.1 Loading Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf
from datetime import datetime, timedelta
from scipy import stats
from sklearn.preprocessing import StandardScaler
from linearmodels.panel import PanelOLS
import warnings
warnings.filterwarnings('ignore')

# Plotting defaults
plt.rcParams.update({
    'figure.figsize': (10, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

43.3.2 Retrieving Disclosure Data

We assume that we have structured data on filing dates, announcement timestamps, and the textual content of corporate disclosures for all firms.

```
from datacore import DataCoreClient

client = DataCoreClient()

# Filing metadata: announcement dates, filing dates, report types
filings = client.get_filings(
    exchanges=['HOSE', 'HNX'],
    report_types=['annual', 'semi_annual', 'quarterly'],
    start_date='2012-01-01',
```

```
    end_date='2024-12-31',
    fields=[
        'ticker', 'report_type', 'fiscal_year', 'fiscal_quarter',
        'fiscal_year_end', 'filing_date', 'announcement_date',
        'auditor', 'audit_opinion', 'file_url'
    ]
)

# Financial statement data
financials = client.get_fundamentals(
    exchanges=['HOSE', 'HNX'],
    start_date='2012-01-01',
    end_date='2024-12-31',
    fields=[
        'ticker', 'fiscal_year', 'fiscal_quarter',
        'total_assets', 'total_equity', 'revenue', 'net_income',
        'operating_cash_flow', 'total_accruals',
        'market_cap', 'book_to_market'
    ]
)

# Daily trading data for event studies
trading = client.get_daily_prices(
    exchanges=['HOSE', 'HNX'],
    start_date='2012-01-01',
    end_date='2024-12-31',
    fields=[
        'ticker', 'date', 'close', 'volume', 'turnover',
        'bid_ask_spread', 'market_return'
    ]
)

# Ownership and governance
governance = client.get_governance(
    exchanges=['HOSE', 'HNX'],
    fields=[
        'ticker', 'fiscal_year', 'state_ownership_pct',
        'foreign_ownership_pct', 'board_size',
        'board_independence_pct', 'big4_auditor',
        'dual_listing'
    ]
)
```

```
print(f"Filings: {filings.shape[0]:,} records")
print(f"Financials: {financials.shape[0]:,} records")
print(f"Trading: {trading.shape[0]:,} records")
print(f"Governance: {governance.shape[0]:,} records")
```

43.3.3 Computing Filing Timeliness

We define **reporting lag** as the number of calendar days between the fiscal period-end and the date the firm's financial statements are made publicly available. For annual reports, the regulatory maximum is 90 days; firms that file earlier than the deadline reveal information sooner, while firms that file late face potential penalties and signal possible difficulties with their accounts.

```
filings['fiscal_year_end'] = pd.to_datetime(filings['fiscal_year_end'])
filings['filing_date'] = pd.to_datetime(filings['filing_date'])
filings['announcement_date'] = pd.to_datetime(filings['announcement_date'])

# Reporting lag = filing date - fiscal period end
filings['reporting_lag'] = (
    filings['filing_date'] - filings['fiscal_year_end']
).dt.days

# Regulatory deadline based on report type
deadline_map = {
    'annual': 90,
    'semi_annual': 45,
    'quarterly': 20
}
filings['deadline_days'] = filings['report_type'].map(deadline_map)

# Late filing indicator
filings['late_filing'] = (
    filings['reporting_lag'] > filings['deadline_days']
).astype(int)

# Days relative to deadline (negative = early, positive = late)
filings['days_relative_deadline'] = (
    filings['reporting_lag'] - filings['deadline_days']
)

# Summary statistics
```

```
annual_filings = filings[filings['report_type'] == 'annual'].copy()
print("Annual Report Filing Lag (calendar days):")
print(annual_filings['reporting_lag'].describe().round(1))
print(f"\nLate filing rate: {annual_filings['late_filing'].mean():.1%}")
```

43.3.4 Distribution of Filing Lags

43.4 Measuring Disclosure Quality

Disclosure quality is inherently multidimensional. Following P. Dechow, Ge, and Schrand (2010) and Beyer et al. (2010), we construct proxies along four dimensions: (i) timeliness, (ii) textual properties, (iii) accounting quality, and (iv) voluntary disclosure breadth.

43.4.1 Timeliness as a Quality Dimension

Timely disclosure reduces the duration of information asymmetry between insiders and outside investors. Chambers and Penman (1984) and Givoly and Palmon (1982) establish that early reporters tend to announce good news, while late reporters more often deliver bad news. We test this pattern in the Vietnamese context below.

We operationalize timeliness through two measures:

1. **Reporting lag** (continuous): Calendar days from fiscal period-end to filing date, as constructed in Section 43.3.3.
2. **Early/late classification** (categorical): We classify firm-years into terciles based on reporting lag within each fiscal year. This controls for secular trends in filing speed (e.g., driven by regulatory changes or COVID-19 disruptions).

```
annual_filings['lag_tercile'] = (
    annual_filings
    .groupby('fiscal_year')['reporting_lag']
    .transform(lambda x: pd.qcut(x, 3, labels=['Early', 'Middle', 'Late']))
)

# Tabulate
tercile_stats = (
    annual_filings
    .groupby('lag_tercile')['reporting_lag']
    .agg(['count', 'mean', 'median', 'std'])
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogram
axes[0].hist(
    annual_filings['reporting_lag'].dropna(),
    bins=60, range=(20, 150),
    color='#2C5F8A', edgecolor='white', alpha=0.85
)
axes[0].axvline(x=90, color='#C0392B', linestyle='--', linewidth=2,
                label='90-day deadline')
axes[0].set_xlabel('Reporting Lag (Calendar Days)')
axes[0].set_ylabel('Number of Filings')
axes[0].set_title('Distribution of Annual Report Filing Lags')
axes[0].legend()

# Time trend: median lag by year
median_lag = (
    annual_filings
    .groupby('fiscal_year')['reporting_lag']
    .agg(['median', lambda x: x.quantile(0.25),
          lambda x: x.quantile(0.75)])
)
median_lag.columns = ['median', 'p25', 'p75']

axes[1].fill_between(
    median_lag.index, median_lag['p25'], median_lag['p75'],
    alpha=0.3, color='#2C5F8A', label='IQR'
)
axes[1].plot(
    median_lag.index, median_lag['median'],
    color='#2C5F8A', linewidth=2, marker='o', label='Median'
)
axes[1].axhline(y=90, color='#C0392B', linestyle='--',
                linewidth=1.5, label='Deadline')
axes[1].set_xlabel('Fiscal Year')
axes[1].set_ylabel('Filing Lag (Calendar Days)')
axes[1].set_title('Median Annual Filing Lag Over Time')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Figure 43.1

```
    .round(1)
)
tercile_stats.columns = ['N', 'Mean Lag', 'Median Lag', 'SD']
print(tercile_stats)
```

43.4.2 Textual Quality Measures

The textual properties of corporate disclosures convey information about quality beyond what is captured by accounting numbers alone. F. Li (2008) demonstrates that annual reports with lower readability are associated with lower earnings persistence, suggesting that complex language may obscure unfavourable information. Loughran and McDonald (2014) critique the application of general readability formulas (Fog index, Flesch-Kincaid) to financial text, arguing that these metrics confound complexity with technical terminology.

We construct three textual quality measures adapted for Vietnamese corporate disclosures:

43.4.2.1 Document Length and Specificity

Longer disclosures are not inherently better—length may reflect boilerplate or obfuscation. However, Dyer, Lang, and Stice-Lawrence (2017) show that the *informative* component of disclosure (as opposed to standard legal language) has increased over time in U.S. 10-K filings. We measure:

- **Total word count:** Raw length of the annual report narrative sections (MD&A equivalent)
- **Numerical density:** Proportion of tokens that are numbers, percentages, or currency amounts, which is a proxy for specificity.

```
import re
from underthesea import word_tokenize

def compute_textual_metrics(text):
    """Compute textual quality metrics for Vietnamese corporate text."""
    if not text or len(text.strip()) == 0:
        return {
            'word_count': 0, 'sentence_count': 0,
            'numerical_density': 0, 'avg_sentence_length': 0,
            'unique_word_ratio': 0, 'forward_looking_density': 0
        }

    # Vietnamese word segmentation
```

```
    tokens = word_tokenize(text)
    sentences = re.split(r'[.!?]', text)
    sentences = [s.strip() for s in sentences if len(s.strip()) > 5]

    word_count = len(tokens)
    sentence_count = max(len(sentences), 1)

    # Numerical density: proportion of tokens that are numeric
    num_pattern = re.compile(r'^[\d,.%]+$')
    numeric_tokens = sum(1 for t in tokens if num_pattern.match(t))
    numerical_density = numeric_tokens / max(word_count, 1)

    # Lexical diversity: unique words / total words
    unique_words = len(set(t.lower() for t in tokens))
    unique_word_ratio = unique_words / max(word_count, 1)

    # Forward-looking statement density
    forward_keywords = [
        'dự kiến', 'kế hoạch', 'mục tiêu', 'triển vọng',
        'định hướng', 'chiến lược', 'tương lai', 'sẽ',
        'dự báo', 'phấn đấu', 'cam kết', 'hướng tới'
    ]
    text_lower = text.lower()
    forward_count = sum(text_lower.count(kw) for kw in forward_keywords)
    forward_looking_density = forward_count / max(sentence_count, 1)

    return {
        'word_count': word_count,
        'sentence_count': sentence_count,
        'numerical_density': numerical_density,
        'avg_sentence_length': word_count / sentence_count,
        'unique_word_ratio': unique_word_ratio,
        'forward_looking_density': forward_looking_density
    }

# Retrieve annual report text from DataCore
annual_text = client.get_annual_report_text(
    exchanges=['HOSE', 'HNX'],
    start_date='2012-01-01',
    end_date='2024-12-31',
    sections=['mda', 'business_overview', 'risk_factors']
)
```

```
# Apply textual metrics
textual_metrics = annual_text.apply(
    lambda row: compute_textual_metrics(row['text']),
    axis=1, result_type='expand'
)
annual_text = pd.concat([annual_text, textual_metrics], axis=1)

print("Textual Quality Summary Statistics:")
print(annual_text[['word_count', 'numerical_density',
                   'avg_sentence_length', 'unique_word_ratio',
                   'forward_looking_density']].describe().round(3))
```

43.4.2.2 Forward-Looking Statement Density

Forward-looking statements reveal management's expectations about future performance and are considered a higher-quality form of disclosure because they expose the manager to ex-post evaluation. In Vietnamese reports, forward-looking language typically appears in the form of phrases like *dự kiến* (expected), *kế hoạch* (plan), *mục tiêu* (target), and *triển vọng* (outlook).

Guay, Samuels, and Taylor (2016) show that managers use voluntary disclosure to "guide through the fog" when financial statements are complex. We operationalize forward-looking density as the number of forward-looking phrases per sentence, following the keyword approach in our `compute_textual_metrics` function above.

43.4.3 Accounting-Based Quality Proxies

We complement textual measures with accounting-based proxies that capture the reliability of reported financial information.

43.4.3.1 Accruals Quality

Following Francis et al. (2005), we measure accruals quality as the standard deviation of residuals from a regression of working capital accruals on past, current, and future operating cash flows:

$$\frac{WC_{i,t}}{A_{i,t-1}} = \alpha + \beta_1 \frac{CFO_{i,t-1}}{A_{i,t-1}} + \beta_2 \frac{CFO_{i,t}}{A_{i,t-1}} + \beta_3 \frac{CFO_{i,t+1}}{A_{i,t-1}} + \varepsilon_{i,t} \tag{43.1}$$

where $WC_{i,t}$ is working capital accruals, $CFO_{i,t}$ is operating cash flow, and $A_{i,t-1}$ is lagged total assets. The firm-level standard deviation of $\hat{\varepsilon}_{i,t}$ over a rolling window (typically 5 years) is the accruals quality measure, with higher values indicating lower quality.

```
def estimate_accruals_quality(df, min_obs=5):
    """
    Estimate accruals quality as std dev of DD residuals
    over a rolling 5-year window for each firm.
    """
    results = []

    for ticker, group in df.groupby('ticker'):
        group = group.sort_values('fiscal_year')

        # Construct leads/lags of CFO
        group['cfo_lag1'] = group['operating_cash_flow'].shift(1)
        group['cfo_lead1'] = group['operating_cash_flow'].shift(-1)

        # Scale by lagged assets
        group['lag_assets'] = group['total_assets'].shift(1)
        for col in ['total_accruals', 'operating_cash_flow',
                    'cfo_lag1', 'cfo_lead1']:
            group[f'{col}_scaled'] = group[col] / group['lag_assets']

        # Rolling 5-year residual std dev
        for idx in range(len(group)):
            window = group.iloc[max(0, idx - 4):idx + 1]
            window = window.dropna(subset=[
                'total_accruals_scaled', 'operating_cash_flow_scaled',
                'cfo_lag1_scaled', 'cfo_lead1_scaled'
            ])

            if len(window) >= min_obs:
                y = window['total_accruals_scaled']
                X = sm.add_constant(window[[
                    'cfo_lag1_scaled',
                    'operating_cash_flow_scaled',
                    'cfo_lead1_scaled'
                ]])
                try:
                    model = sm.OLS(y, X).fit()
                    results.append({
                        'ticker': ticker,
```

```
                        'fiscal_year': group.iloc[idx]['fiscal_year'],
                        'accruals_quality': model.resid.std()
                    })
                except Exception:
                    pass

    return pd.DataFrame(results)

aq_df = estimate_accruals_quality(financials)
print(f"Accruals quality computed for {aq_df['ticker'].nunique()} firms")
print(aq_df['accruals_quality'].describe().round(4))
```

43.4.3.2 Earnings Persistence and Predictability

Persistent earnings are more useful for valuation. We estimate earnings persistence as the slope coefficient ϕ_1 from a first-order autoregression:

$$\frac{E_{i,t}}{A_{i,t-1}} = \phi_0 + \phi_1 \frac{E_{i,t-1}}{A_{i,t-2}} + \nu_{i,t} \tag{43.2}$$

Higher $\hat{\phi}_1$ indicates more persistent (and arguably higher-quality) earnings.

```
def estimate_persistence(df, min_obs=5):
    """Estimate earnings persistence via AR(1) model."""
    results = []

    for ticker, group in df.groupby('ticker'):
        group = group.sort_values('fiscal_year')
        group['earnings_scaled'] = group['net_income'] /
↪   group['total_assets'].shift(1)
        group['earnings_lag'] = group['earnings_scaled'].shift(1)

        clean = group.dropna(subset=['earnings_scaled', 'earnings_lag'])
        if len(clean) >= min_obs:
            y = clean['earnings_scaled']
            X = sm.add_constant(clean[['earnings_lag']])
            model = sm.OLS(y, X).fit()
            results.append({
                'ticker': ticker,
                'persistence': model.params['earnings_lag'],
                'persistence_se': model.bse['earnings_lag'],
```

```
                'r_squared': model.rsquared,
                'n_obs': model.nobs
            })

    return pd.DataFrame(results)

persistence_df = estimate_persistence(financials)
print(persistence_df[['persistence', 'r_squared']].describe().round(3))
```

43.4.4 Composite Disclosure Quality Index

Individual quality proxies capture different facets of the information environment. To aggregate them into a single score while avoiding arbitrary weighting, we follow Lang and Lundholm (1993) and use a rank-based composite. For each firm-year, we rank firms on each of the following dimensions (higher rank = higher quality) (Table 43.2).

Table 43.2: Components of the composite disclosure quality index.

Dimension	Proxy	Direction
Timeliness	Reporting lag	Lower is better
Specificity	Numerical density	Higher is better
Forward-looking	FLS density	Higher is better
Earnings quality	Accruals quality (DD)	Lower is better
Persistence	AR(1) coefficient	Higher is better

We convert each proxy to a percentile rank within each fiscal year (so each component ranges from 0 to 1), then average across components:

$$DQ_{i,t} = \frac{1}{K} \sum_{k=1}^{K} \text{Rank}_{k,i,t} \tag{43.3}$$

where K is the number of available components and $\text{Rank}_{k,i,t}$ is the percentile rank of firm i in year t on dimension k.

```
# Merge all quality proxies
quality_panel = (
    annual_filings[['ticker', 'fiscal_year', 'reporting_lag']]
    .merge(
        annual_text[['ticker', 'fiscal_year', 'numerical_density',
```

```
                        'forward_looking_density']],
            on=['ticker', 'fiscal_year'], how='left'
        )
        .merge(aq_df, on=['ticker', 'fiscal_year'], how='left')
        .merge(persistence_df[['ticker', 'persistence']],
               on='ticker', how='left')
)

# Rank each component within fiscal year (higher = better quality)
def year_percentile_rank(series):
    """Convert to percentile rank within group."""
    return series.rank(pct=True)

rank_cols = {}
for col, ascending in [
    ('reporting_lag', False),       # lower lag = better → invert
    ('numerical_density', True),    # higher = better
    ('forward_looking_density', True),
    ('accruals_quality', False),    # lower volatility = better → invert
    ('persistence', True)           # higher = better
]:
    col_to_rank = quality_panel[col] if ascending else -quality_panel[col]
    rank_cols[f'rank_{col}'] = (
        quality_panel
        .groupby('fiscal_year')[col]
        .transform(lambda x: x.rank(pct=True) if ascending
                   else (-x).rank(pct=True))
    )

rank_df = pd.DataFrame(rank_cols)
quality_panel = pd.concat([quality_panel, rank_df], axis=1)

# Composite index: average of available ranks
rank_columns = [c for c in quality_panel.columns if c.startswith('rank_')]
quality_panel['dq_index'] = quality_panel[rank_columns].mean(axis=1)

print("Disclosure Quality Index Distribution:")
print(quality_panel['dq_index'].describe().round(3))
```

```
fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(quality_panel['dq_index'].dropna(), bins=50,
        color='#2C5F8A', edgecolor='white', alpha=0.85)
ax.axvline(quality_panel['dq_index'].median(), color='#E67E22',
           linestyle='--', linewidth=2, label='Median')
ax.set_xlabel('Disclosure Quality Index')
ax.set_ylabel('Number of Firm-Years')
ax.set_title('Distribution of Composite Disclosure Quality')
ax.legend()
plt.tight_layout()
plt.show()
```

Figure 43.2

43.5 Determinants of Disclosure Quality

What drives variation in disclosure quality across Vietnamese firms? We estimate a cross-sectional regression of the composite DQ index on firm characteristics and governance variables:

$$DQ_{i,t} = \alpha + \beta_1 \ln(\text{Size}_{i,t}) + \beta_2 \text{ROA}_{i,t} + \beta_3 \text{Lev}_{i,t} + \beta_4 \text{StateOwn}_{i,t} + \beta_5 \text{ForeignOwn}_{i,t} + \beta_6 \text{Big4}_{i,t} + \beta_7 \text{BoardInd} \tag{43.4}$$

where γ_t are year fixed effects.

The theoretical predictions, drawing on Lang and Lundholm (1993), Hope (2003), and Bushman et al. (2004), are:

- **Size (+):** Larger firms face greater public scrutiny and have lower proprietary costs relative to the benefits of disclosure.
- **ROA (+/−):** Profitable firms may disclose more to signal quality, but firms managing earnings downward (for tax purposes) may reduce disclosure to avoid scrutiny.
- **Leverage (+):** Sengupta (1998) argues that firms with more debt have stronger incentives to maintain disclosure quality to lower borrowing costs.
- **State ownership (−):** SOEs may face weaker market discipline and political incentives to limit transparency.
- **Foreign ownership (+):** Foreign institutional investors demand higher transparency.
- **Big 4 auditor (+):** High-quality auditors constrain earnings management and indirectly improve disclosure quality.
- **Board independence (+):** Independent directors improve monitoring and encourage more informative disclosure.

```
# Merge quality index with financials and governance
det_panel = (
    quality_panel[['ticker', 'fiscal_year', 'dq_index']]
    .merge(financials, on=['ticker', 'fiscal_year'], how='left')
    .merge(governance, on=['ticker', 'fiscal_year'], how='left')
)

# Construct variables
det_panel['log_size'] = np.log(det_panel['total_assets'])
det_panel['roa'] = det_panel['net_income'] / det_panel['total_assets']
det_panel['leverage'] = (
    (det_panel['total_assets'] - det_panel['total_equity'])
    / det_panel['total_assets']
)

# Panel regression with year FE
det_panel = det_panel.set_index(['ticker', 'fiscal_year'])

model_det = PanelOLS(
    dependent=det_panel['dq_index'],
    exog=sm.add_constant(det_panel[[
        'log_size', 'roa', 'leverage',
        'state_ownership_pct', 'foreign_ownership_pct',
        'big4_auditor', 'board_independence_pct'
    ]]),
    entity_effects=False,
    time_effects=True,
    check_rank=False
).fit(cov_type='clustered', cluster_entity=True)

print(model_det.summary)
```

43.6 Strategic Disclosure Timing

43.6.1 Day-of-Week Effects

DellaVigna and Pollet (2009) document that Friday earnings announcements receive less immediate market attention. We test whether this pattern holds in Vietnam, where the trading week runs Monday through Friday but the retail-dominated investor base may exhibit different attention patterns.

```
coefs = model_det.params.drop('const')
ci = model_det.conf_int().drop('const')

fig, ax = plt.subplots(figsize=(8, 5))
y_pos = range(len(coefs))
labels = [
    'ln(Assets)', 'ROA', 'Leverage', 'State Own %',
    'Foreign Own %', 'Big 4 Auditor', 'Board Indep %'
]

colors = ['#2C5F8A' if c > 0 else '#C0392B' for c in coefs.values]
ax.barh(y_pos, coefs.values, color=colors, alpha=0.8, height=0.6)
ax.errorbar(
    coefs.values, y_pos,
    xerr=[coefs.values - ci.iloc[:, 0].values,
          ci.iloc[:, 1].values - coefs.values],
    fmt='none', color='black', capsize=3
)
ax.axvline(x=0, color='gray', linewidth=0.8, linestyle='-')
ax.set_yticks(y_pos)
ax.set_yticklabels(labels)
ax.set_xlabel('Coefficient Estimate')
ax.set_title('Determinants of Disclosure Quality')
plt.tight_layout()
plt.show()
```

Figure 43.3

```
annual_filings['announcement_dow'] = (
    annual_filings['announcement_date'].dt.dayofweek
)
annual_filings['day_name'] = (
    annual_filings['announcement_date'].dt.day_name()
)

# Compute surprise: actual earnings minus naive expectation (last year's
 ↪  earnings)
annual_filings = annual_filings.merge(
    financials[['ticker', 'fiscal_year', 'net_income', 'total_assets']],
    on=['ticker', 'fiscal_year'], how='left'
)
annual_filings['earnings_scaled'] = (
    annual_filings['net_income'] / annual_filings['total_assets']
)
annual_filings['earnings_surprise'] = (
    annual_filings
    .groupby('ticker')['earnings_scaled']
    .diff()
)

# Classify as good/bad news
annual_filings['bad_news'] = (
    annual_filings['earnings_surprise'] < 0
).astype(int)

# Day-of-week distribution by news type
dow_crosstab = pd.crosstab(
    annual_filings['day_name'],
    annual_filings['bad_news'].map({0: 'Good News', 1: 'Bad News'}),
    normalize='columns'
)
# Reorder days
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']
dow_crosstab = dow_crosstab.reindex(day_order)

print("Proportion of Announcements by Day and News Type:")
print(dow_crosstab.round(3))
```

```
fig, ax = plt.subplots(figsize=(10, 5))
x = np.arange(len(day_order))
width = 0.35

bad_pct = dow_crosstab['Bad News'].values
good_pct = dow_crosstab['Good News'].values

ax.bar(x - width/2, good_pct, width, label='Good News',
       color='#27AE60', alpha=0.8)
ax.bar(x + width/2, bad_pct, width, label='Bad News',
       color='#C0392B', alpha=0.8)
ax.set_xticks(x)
ax.set_xticklabels(day_order)
ax.set_ylabel('Proportion of Announcements')
ax.set_title('Strategic Timing: Day-of-Week Announcement Patterns')
ax.legend()
plt.tight_layout()
plt.show()
```

Figure 43.4

43.6.2 Announcement Congestion

When many firms announce on the same day, each announcement receives less attention. We measure **announcement congestion** as the number of other firms making earnings announcements on the same date:

$$\text{Congestion}_{i,t} = \sum_{j \neq i} \mathbf{1}\{\text{AnnDate}_j = \text{AnnDate}_i\} \tag{43.5}$$

Hirshleifer, Lim, and Teoh (2009) predict that firms burying bad news will choose high-congestion days. We test this by regressing the congestion variable on the sign of earnings news:

```
# Count announcements per date
ann_counts = (
    annual_filings
    .groupby('announcement_date')
    .size()
    .reset_index(name='n_announcements')
)
```

```
annual_filings = annual_filings.merge(
    ann_counts, on='announcement_date', how='left'
)
annual_filings['congestion'] = annual_filings['n_announcements'] - 1

# Regression: congestion ~ bad_news + controls
congestion_model = smf.ols(
    'congestion ~ bad_news + log_size + roa + C(fiscal_year)',
    data=annual_filings.assign(
        log_size=np.log(annual_filings['total_assets']),
        roa=annual_filings['net_income'] / annual_filings['total_assets']
    )
).fit(cov_type='cluster', cov_kwds={'groups': annual_filings['ticker']})

print("Congestion Regression:")
print(congestion_model.summary().tables[1])
```

43.6.3 After-Hours and Weekend Announcements

Vietnamese regulations require disclosure within 24 hours of material events, but firms retain discretion over the exact timing. Announcements made after the trading session closes (after 3:00 PM on HOSE/HNX) or on weekends delay the market's opportunity to react by at least one trading day.

```
# Assume announcement timestamps are available
annual_filings['ann_hour'] = (
    annual_filings['announcement_date'].dt.hour
)
annual_filings['after_hours'] = (
    (annual_filings['ann_hour'] >= 15) |
    (annual_filings['announcement_dow'] >= 5)  # Saturday/Sunday
).astype(int)

# Cross-tabulate after-hours by news type
afterhours_crosstab = pd.crosstab(
    annual_filings['after_hours'].map({0: 'During Hours', 1: 'After Hours'}),
    annual_filings['bad_news'].map({0: 'Good News', 1: 'Bad News'}),
    normalize='index'
)
print("News Distribution by Announcement Timing:")
print(afterhours_crosstab.round(3))
```

```
# Chi-squared test
contingency = pd.crosstab(
    annual_filings['after_hours'], annual_filings['bad_news']
)
chi2, p_val, _, _ = stats.chi2_contingency(contingency)
print(f"\nChi-squared = {chi2:.2f}, p-value = {p_val:.4f}")
```

43.7 Market Consequences of Disclosure Quality

43.7.1 Disclosure Quality and the Cost of Equity

The central prediction of Diamond and Verrecchia (1991) and Botosan (1997) is that higher-quality disclosure lowers the cost of equity capital by reducing information asymmetry. We test this using the implied cost of capital (ICC) approach, where we estimate the discount rate that equates the current price to the present value of expected future earnings.

We use the PEG ratio approach as a simple ICC estimate:

$$r_{PEG,i,t} = \sqrt{\frac{\hat{E}_{i,t+2} - \hat{E}_{i,t+1}}{P_{i,t}}} \tag{43.6}$$

where $\hat{E}_{i,t+k}$ is the consensus earnings forecast (or, in the absence of analyst coverage, a model-based forecast) and $P_{i,t}$ is the current stock price.

```
# Construct earnings forecasts using a simple random walk with drift
forecasts = financials.sort_values(['ticker', 'fiscal_year']).copy()
forecasts['eps'] = forecasts['net_income'] / forecasts['market_cap']
forecasts['eps_growth'] = forecasts.groupby('ticker')['eps'].pct_change()

# Simple forecast: E[t+1] = E[t] * (1 + avg_growth)
forecasts['avg_growth'] = (
    forecasts.groupby('ticker')['eps_growth']
    .transform(lambda x: x.rolling(3, min_periods=2).mean())
)
forecasts['eps_f1'] = forecasts['eps'] * (1 + forecasts['avg_growth'])
forecasts['eps_f2'] = forecasts['eps_f1'] * (1 + forecasts['avg_growth'])

# PEG-based ICC
forecasts['icc_peg'] = np.sqrt(
```

```
    np.maximum(forecasts['eps_f2'] - forecasts['eps_f1'], 0)
    / np.maximum(forecasts['market_cap'] / 1e6, 1e-6)
)

# Merge with disclosure quality
icc_panel = (
    forecasts[['ticker', 'fiscal_year', 'icc_peg']]
    .merge(quality_panel[['ticker', 'fiscal_year',
    'dq_index']].reset_index(drop=True),
           on=['ticker', 'fiscal_year'], how='inner')
    .merge(governance, on=['ticker', 'fiscal_year'], how='left')
    .merge(financials[['ticker', 'fiscal_year', 'total_assets',
                       'book_to_market', 'market_cap']],
           on=['ticker', 'fiscal_year'], how='left')
)

icc_panel['log_size'] = np.log(icc_panel['market_cap'])
icc_panel = icc_panel.set_index(['ticker', 'fiscal_year'])

# Panel regression: ICC ~ DQ + controls
icc_model = PanelOLS(
    dependent=icc_panel['icc_peg'],
    exog=sm.add_constant(icc_panel[[
        'dq_index', 'log_size', 'book_to_market'
    ]]),
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type='clustered', cluster_entity=True)

print("Implied Cost of Capital ~ Disclosure Quality:")
print(icc_model.summary)
```

43.7.2 Disclosure Quality and Liquidity

Diamond and Verrecchia (1991) predict that better disclosure reduces adverse selection and improves liquidity. We measure liquidity through bid-ask spreads and the Amihud illiquidity ratio:

$$\text{Amihud}_{i,t} = \frac{1}{D_{i,t}} \sum_{d=1}^{D_{i,t}} \frac{|R_{i,d}|}{\text{Volume}_{i,d}} \tag{43.7}$$

where $R_{i,d}$ is the daily return and $\text{Volume}_{i,d}$ is the daily trading volume in VND.

```
# Compute annual Amihud illiquidity
trading['abs_return'] = trading['close'].pct_change().abs()
trading['amihud_daily'] = trading['abs_return'] / (trading['volume'] *
 ↪  trading['close'])

amihud_annual = (
    trading
    .assign(fiscal_year=trading['date'].dt.year)
    .groupby(['ticker', 'fiscal_year'])
    .agg(
        amihud=('amihud_daily', 'mean'),
        avg_spread=('bid_ask_spread', 'mean'),
        avg_turnover=('turnover', 'mean')
    )
    .reset_index()
)

# Log transform for better distributional properties
amihud_annual['log_amihud'] = np.log(amihud_annual['amihud'] + 1e-10)
amihud_annual['log_spread'] = np.log(amihud_annual['avg_spread'] + 1e-6)

# Merge and run regression
liq_panel = (
    amihud_annual
    .merge(quality_panel[['ticker', 'fiscal_year',
 ↪  'dq_index']].reset_index(drop=True),
           on=['ticker', 'fiscal_year'], how='inner')
    .merge(financials[['ticker', 'fiscal_year', 'market_cap',
 ↪  'total_assets']],
           on=['ticker', 'fiscal_year'], how='left')
)
liq_panel['log_size'] = np.log(liq_panel['market_cap'])
liq_panel = liq_panel.set_index(['ticker', 'fiscal_year'])

liq_model = PanelOLS(
    dependent=liq_panel['log_amihud'],
    exog=sm.add_constant(liq_panel[['dq_index', 'log_size']]),
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type='clustered', cluster_entity=True)
```

```
print("Amihud Illiquidity ~ Disclosure Quality:")
print(liq_model.summary)
```

```
liq_panel_plot = liq_panel.reset_index()
liq_panel_plot['dq_quintile'] = pd.qcut(
    liq_panel_plot['dq_index'], 5, labels=['Q1\n(Low)', 'Q2', 'Q3', 'Q4',
 ↪  'Q5\n(High)']
)

quintile_liq = (
    liq_panel_plot
    .groupby('dq_quintile')['log_amihud']
    .agg(['mean', 'sem'])
)

fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.bar(
    range(5), quintile_liq['mean'],
    yerr=1.96 * quintile_liq['sem'],
    color=['#C0392B', '#E67E22', '#F1C40F', '#27AE60', '#2C5F8A'],
    alpha=0.85, capsize=4, edgecolor='white'
)
ax.set_xticks(range(5))
ax.set_xticklabels(quintile_liq.index)
ax.set_xlabel('Disclosure Quality Quintile')
ax.set_ylabel('Log Amihud Illiquidity')
ax.set_title('Disclosure Quality and Market Liquidity')
plt.tight_layout()
plt.show()
```

Figure 43.5

43.7.2.1 Measurement Error in the Liquidity Outcome

Equation 51.2 uses a single absolute daily return to summarize an entire day of price movement. As developed at length in the liquidity chapter (Section 14.4), that numerator carries roughly 76% relative measurement error per day, and Lacava, Ranaldo, and Santucci de Magistris (2026) show it can be replaced by a high-frequency estimate of the same quantity — the **realized Amihud**,

$$\text{RAmihud}_{i,t} = \frac{1}{D_{i,t}} \sum_{d=1}^{D_{i,t}} \sqrt{\frac{2}{\pi}} \frac{\sqrt{\text{RV}_{i,d}}}{\text{Volume}_{i,d}}, \qquad \text{RV}_{i,d} = \sum_{j=1}^{M} r_{i,d,j}^2 \tag{43.8}$$

where $r_{i,d,j}$ are intraday returns and the constant $\sqrt{2/\pi}$ keeps Equation 43.8 on the same scale as Equation 51.2. At HOSE's $M = 45$ five-minute intervals the numerator's relative error falls from about 76% to about 11%.

It would be easy to conclude that this specification therefore understates the disclosure-quality effect. **It does not, and the reason is worth being precise about, because the econometrics here differ fundamentally from the portfolio-sorting application in the liquidity chapter.** There, illiquidity is a right-hand-side sorting variable, and measurement error attenuates the estimated premium toward zero. Here, illiquidity is the *dependent* variable. Writing the measured outcome as truth plus noise, $\hat{y}_i = y_i + \nu_i$, the regression $\hat{y}_i = \alpha + \beta\,\text{DQ}_i + (\varepsilon_i + \nu_i)$ has an unchanged slope in expectation provided ν_i is uncorrelated with DQ_i. Classical measurement error in an outcome costs *precision*, not consistency (Hausman 2001). The realized measure therefore buys statistical power in this chapter, not bias correction.

Two questions follow. How much power, and is the "uncorrelated with DQ" proviso actually satisfied here?

43.7.2.2 Where It Barely Matters: The Annual Specification

The regression above averages daily ratios over a full fiscal year. Averaging is what defeats measurement error: over D days the relative error of the Amihud numerator falls to $0.7555/\sqrt{D}$, which for a liquid firm with 250 trading days is under 5% — already small enough that halving it again changes little.

There is, however, one channel through which the annual specification can be biased, and it is specific to the log transform the chapter applies. Because log is concave, $\mathbb{E}[\log \hat{y}] < \log \mathbb{E}[\hat{y}]$, and the size of that Jensen gap is approximately $-\frac{1}{2}\,\text{relvar}/D$. It therefore *varies across firms with* $D_{i,t}$. Firms that trade on few days carry a larger downward distortion in log(Amihud) than firms that trade every day. Since thinly traded firms are also disproportionately the poor disclosers, ν_i is correlated with DQ_i after all, and the slope is no longer clean. The magnitude is $-0.285/D$ for the classic measure against $-0.0056/D$ for the realized one — a fiftyfold difference.

The following chunk quantifies both effects. It simulates a known disclosure-quality effect of $\beta = -0.10$ on log illiquidity, first with trading days identical across firms and then with trading days rising in disclosure quality.

```
import numpy as np
import pandas as pd
from scipy.stats import norm

rng_dq = np.random.default_rng(4242)
M_HF, BETA_DQ, N_FIRMS, REPS_A, SD_E = 45, -0.10, 400, 300, 0.50


def measure_factor(kind, n, D, rng):
    """
    Multiplicative measurement-error factor in the Amihud numerator,
    averaged over each firm's D trading days. Mean one by construction;
    only the variance differs between the two measures.
    """
    Dm = D.max()
    mask = np.arange(Dm)[None, :] < D[:, None]
    if kind == "classic":
        z = np.abs(rng.normal(0, 1, (n, Dm))) / np.sqrt(2 / np.pi)   #
 ↪  half-normal
    else:
        z = np.sqrt(rng.chisquare(M_HF, (n, Dm)) / M_HF)             #
 ↪  sqrt(RV)
    return np.where(mask, z, 0).sum(1) / D


def annual_rep(link_days, rng):
    """One draw of the annual cross-sectional regression."""
    DQ = rng.normal(0, 1, N_FIRMS)
    log_true = 1.0 + BETA_DQ * DQ + rng.normal(0, SD_E, N_FIRMS)

    # Better disclosers trade on more days: 20 to 250 per year.
    D = (np.clip(np.round(20 + 230 * norm.cdf(DQ)), 20, 250).astype(int)
         if link_days else np.full(N_FIRMS, 250))

    X = np.column_stack([np.ones(N_FIRMS), DQ])
    XtXinv = np.linalg.inv(X.T @ X)

    out = {}
    for kind in ["classic", "realized", "truth"]:
        f = (np.ones(N_FIRMS) if kind == "truth"
             else measure_factor(kind, N_FIRMS, D, rng))
        y = log_true + np.log(f)
        b = XtXinv @ X.T @ y
```

```
        r = y - X @ b
        se = np.sqrt((r @ r / (N_FIRMS - 2)) * XtXinv[1, 1])
        out[kind] = (b[1], se, b[1] / se)
    return out


for link_days in [False, True]:
    acc = {k: [] for k in ["classic", "realized", "truth"]}
    for _ in range(REPS_A):
        for k, v in annual_rep(link_days, rng_dq).items():
            acc[k].append(v)

    rows = []
    for k, v in acc.items():
        a = np.array(v)
        rows.append({
            "measure": k,
            "mean beta": round(a[:, 0].mean(), 4),
            "attenuation %": round(100 * (1 - a[:, 0].mean() / BETA_DQ), 1),
            "mean |t|": round(np.abs(a[:, 2]).mean(), 2),
            "power@5%": round((np.abs(a[:, 2]) > 1.96).mean(), 3),
        })
    print(f"\nTrading days linked to disclosure quality: {link_days}")
    print(pd.DataFrame(rows).to_string(index=False))
```

```
Trading days linked to disclosure quality: False
 measure  mean beta  attenuation %  mean |t|  power@5%
 classic    -0.1002           -0.2      3.99     0.977
realized    -0.1002           -0.2      4.01     0.977
   truth    -0.1002           -0.2      4.01     0.977

Trading days linked to disclosure quality: True
 measure  mean beta  attenuation %  mean |t|  power@5%
 classic    -0.0972            2.8      3.81     0.973
realized    -0.1000           -0.0      3.97     0.987
   truth    -0.1001           -0.1      3.97     0.987
```

The results are reassuring for the specification already estimated in this chapter. When trading days are unrelated to disclosure quality, all three measures recover β essentially exactly — confirming that outcome-side measurement error does not bias the slope. When trading days *are* linked to disclosure quality, the classic measure attenuates by roughly 3%, while the realized

measure and the infeasible truth are indistinguishable. A 3% attenuation is not what will decide whether the disclosure-liquidity relation is detected, and it is comfortably inside any plausible confidence interval.

The honest conclusion for the annual regression is that the realized Amihud is not worth the intraday data it requires. Knowing where a refinement does not matter is as useful as knowing where it does; the annual averaging in Equation 51.2 has already done most of the work that high-frequency sampling would do.

43.7.2.3 Where It Matters a Great Deal: Short Event Windows

The picture reverses completely for the event-study designs elsewhere in this chapter. Testing whether liquidity improves after a filing means comparing illiquidity over a window of a few days on either side of the event, and a short window is exactly where averaging cannot rescue a noisy numerator. Over a five-day window the relative error of the classic numerator is $0.7555/\sqrt{5} \approx 34\%$; the realized numerator's is under 5%.

Because the noise enters the pre-post difference twice, its effect on power is severe. The simulation below computes the power to detect a genuine post-filing change in log illiquidity, allowing for real cross-firm heterogeneity in the response so that the realized measure does not trivially achieve unit power.

```
rng_ev = np.random.default_rng(77)
REPS_B = 1500
RELVAR = {"classic": np.pi / 2 - 1, "realized": 1 / (2 * M_HF)}


def win_noise(kind, n, D):
    """Log measurement error in a D-day window average of the numerator."""
    z = (np.abs(rng_ev.normal(0, 1, (n, D))) / np.sqrt(2 / np.pi) if kind ==
 ↪  "classic"
         else np.sqrt(rng_ev.chisquare(M_HF, (n, D)) / M_HF))
    return np.log(z.mean(1))


def ev_power(kind, n, D, delta, sd_delta):
    """Rejection rate of a paired pre/post test of a true change `delta`."""
    rejections = 0
    for _ in range(REPS_B):
        d = ((delta + rng_ev.normal(0, sd_delta, n))
             + win_noise(kind, n, D) - win_noise(kind, n, D))
        rejections += abs(d.mean() / (d.std(ddof=1) / np.sqrt(n))) > 1.96
    return rejections / REPS_B
```

```
SD_RESP = 0.10          # genuine cross-firm variation in the liquidity
 ↪  response
rows_ev = []
for D in [5, 10, 21]:
    for delta in [-0.02, -0.03, -0.05]:
        row = {"window (days)": D, "true change": delta}
        for kind in ["classic", "realized"]:
            row[f"power {kind}"] = round(ev_power(kind, 200, D, delta,
 ↪  SD_RESP), 3)
            # Closed-form firms needed for 80% power at the 5% level.
            var_d = SD_RESP ** 2 + 2 * RELVAR[kind] / D
            row[f"firms for 80% ({kind})"] = int(np.ceil(7.849 * var_d /
 ↪  delta ** 2))
        rows_ev.append(row)

ev = pd.DataFrame(rows_ev)
ev["sample-size ratio"] = (ev["firms for 80% (classic)"]
                           / ev["firms for 80% (realized)"]).round(1)
print("200 firms, 5% two-sided test, cross-firm response sd = 0.10")
print(ev.to_string(index=False))
```

```
200 firms, 5% two-sided test, cross-firm response sd = 0.10
 window (days)  true change  power classic  firms for 80% (classic)  power realized  firms fo
             5        -0.02          0.088                     4677           0.661
             5        -0.03          0.121                     2079           0.949
             5        -0.05          0.283                      749           0.999
            10        -0.02          0.127                     2437           0.732
            10        -0.03          0.238                     1083           0.967
            10        -0.05          0.499                      390           1.000
            21        -0.02          0.203                     1263           0.756
            21        -0.03          0.383                      562           0.978
            21        -0.05          0.775                      203           1.000
```

The gap is large enough to change what is feasible. With 200 firms and a ten-day window, the power to detect a 3% improvement in liquidity is about 0.24 using the classic measure and about 0.97 using the realized one. Expressed as required sample size, detecting that effect at 80% power needs roughly 1,080 firms with the daily measure and roughly 110 with the realized measure. Since the entire Vietnamese listed universe is on the order of 1,600 firms across three venues — and a study of annual-report filings restricted to firms with continuous data will be

a fraction of that — the classic measure makes this test infeasible for most subsamples while the realized measure makes it routine.

Three further observations follow from the table.

The advantage grows as the window shrinks. The sample-size ratio is about 5.8 at 21 days, 10 at ten days, and 16.5 at five days. Since the tightest event windows are precisely the ones where the identification is cleanest — least contaminated by confounding news — the noisy measure imposes a direct trade-off between statistical power and identification quality. The realized measure relaxes it.

A null result from a short-window liquidity test is close to uninformative. At 200 firms and a five-day window, the classic measure rejects a true 2% effect only about 9% of the time — barely above the 5% size of the test. Reporting "no significant change in liquidity around filings" under that design says essentially nothing about whether a change occurred. Any such null in this literature should be read alongside its power.

The ceiling is set by real heterogeneity, not by measurement. The realized measure's required sample sizes flatten out around 35 to 285 firms because the genuine cross-firm variation in the liquidity response (`SD_RESP = 0.10`) eventually dominates. Beyond a certain point, better measurement stops helping and only more firms will do.

43.7.2.4 Implementation

The realized measure is a drop-in replacement for the numerator wherever this chapter computes illiquidity. The chunk below shows the substitution for the annual panel; the realized inputs come from the session-aware HOSE construction described in **?@sec-micro-realized-illiq**, which requires intraday data outside the standard pipeline.

```
# `realized_daily` holds one row per symbol-day from the intraday pipeline:
#   [ticker, date, rv, bv, n_active, dvol_continuous]
K = np.sqrt(2 / np.pi)

rd = realized_daily.copy()
rd = rd[(rd["n_active"] >= 20) & (rd["dvol_continuous"] > 0)]     #
 ↪  thin-trading screen
rd["ramihud_daily"] = K * np.sqrt(rd["rv"]) / rd["dvol_continuous"]
rd["ramihud_bv_daily"] = K * np.sqrt(rd["bv"].clip(lower=0)) /
 ↪  rd["dvol_continuous"]

realized_annual = (
    rd.assign(fiscal_year=rd["date"].dt.year)
      .groupby(["ticker", "fiscal_year"])
      .agg(ramihud=("ramihud_daily", "mean"),
```

```
            ramihud_bv=("ramihud_bv_daily", "mean"),
            n_days=("ramihud_daily", "size"))
      .reset_index()
)
realized_annual["log_ramihud"] = np.log(realized_annual["ramihud"] + 1e-10)

# Compare against the daily-frequency measure on the overlapping sample.
compare = amihud_annual.merge(realized_annual, on=["ticker", "fiscal_year"],
                              how="inner")
print(f"Overlapping firm-years: {len(compare):,}")
print("Rank correlation between the two measures: "
      f"{compare[['log_amihud',
       ↪  'log_ramihud']].corr(method='spearman').iloc[0, 1]:.3f}")

# Re-estimate the disclosure-quality regression with the realized outcome.
liq_panel_r = (
    realized_annual
    .merge(quality_panel[["ticker", "fiscal_year", "dq_index"]],
           on=["ticker", "fiscal_year"], how="inner")
    .merge(financials[["ticker", "fiscal_year", "market_cap"]],
           on=["ticker", "fiscal_year"], how="left")
    .assign(log_size=lambda d: np.log(d["market_cap"]))
    .set_index(["ticker", "fiscal_year"])
)

liq_model_r = PanelOLS(
    dependent=liq_panel_r["log_ramihud"],
    exog=sm.add_constant(liq_panel_r[["dq_index", "log_size"]]),
    entity_effects=True, time_effects=True, check_rank=False,
).fit(cov_type="clustered", cluster_entity=True)

print("Realized Amihud ~ Disclosure Quality:")
print(liq_model_r.summary)
```

43.7.2.5 Practical Guidance

1. **Do not re-run the annual regressions on intraday data.** The gain is a few percent of attenuation at most, and the sample would shrink to the liquid firms for which intraday data is usable — a selection cost far exceeding the measurement benefit.
2. **Do use the realized measure for the event-window tests** in Section 43.7, where it is the difference between a feasible and an infeasible design.

3. **Report the trading-day count** $D_{i,t}$ alongside any log-transformed illiquidity measure. It is the variable that governs the Jensen distortion, it is correlated with disclosure quality, and it is almost never reported.
4. **Prefer the bipower variant around filings.** An earnings surprise is exactly the information jump that inflates realized variance, and the filing date is when it arrives. Section 14.4 shows bipower variation remains nearly unbiased under jumps.
5. **Where intraday data is unavailable, widen the window rather than accept a null.** Going from five to 21 days recovers a substantial part of the power lost to numerator noise, at the cost of admitting more confounding events — a trade-off that should be made explicitly and reported, not by default.

43.7.3 Event Study: Market Reaction to Filing Lag

We examine whether the market reacts differently to early vs. late filers by computing cumulative abnormal returns (CARs) around the filing date:

$$CAR_i[\tau_1, \tau_2] = \sum_{t=\tau_1}^{\tau_2} (R_{i,t} - \hat{R}_{i,t}) \tag{43.9}$$

where $\hat{R}_{i,t}$ is the expected return from a market model estimated over a pre-event window $[-250, -30]$.

```
def compute_car(ticker, event_date, trading_df,
                est_window=(-250, -30), event_window=(-5, 10)):
    """Compute CAR around an event date using market model."""
    firm_data = trading_df[trading_df['ticker'] == ticker].copy()
    firm_data = firm_data.sort_values('date')

    # Find event date index
    event_idx = firm_data[firm_data['date'] >= event_date].index
    if len(event_idx) == 0:
        return None
    event_idx = event_idx[0]
    event_pos = firm_data.index.get_loc(event_idx)

    # Check sufficient data
    if event_pos + est_window[0] < 0:
        return None

    # Estimation window
    est_start = event_pos + est_window[0]
```

```
    est_end = event_pos + est_window[1]
    est_data = firm_data.iloc[est_start:est_end + 1]

    firm_ret = est_data['close'].pct_change()
    mkt_ret = est_data['market_return']

    valid = firm_ret.notna() & mkt_ret.notna()
    if valid.sum() < 100:
        return None

    # Market model
    X = sm.add_constant(mkt_ret[valid])
    model = sm.OLS(firm_ret[valid], X).fit()

    # Event window
    ev_start = event_pos + event_window[0]
    ev_end = event_pos + event_window[1]
    ev_data = firm_data.iloc[ev_start:ev_end + 1]

    ev_ret = ev_data['close'].pct_change()
    ev_mkt = ev_data['market_return']
    expected_ret = model.params['const'] + model.params['market_return'] *
 ↪  ev_mkt
    abnormal_ret = ev_ret - expected_ret

    return abnormal_ret.cumsum().values

# Sample: compute CARs for annual filings
car_results = []
for _, row in annual_filings.sample(min(2000,
 ↪  len(annual_filings))).iterrows():
    car = compute_car(row['ticker'], row['filing_date'], trading)
    if car is not None and len(car) == 16:  # -5 to +10
        car_results.append({
            'ticker': row['ticker'],
            'fiscal_year': row['fiscal_year'],
            'lag_tercile': row['lag_tercile'],
            'car': car
        })

car_df = pd.DataFrame(car_results)
print(f"Computed CARs for {len(car_df)} firm-year events")
```

```
event_days = range(-5, 11)

fig, ax = plt.subplots(figsize=(10, 6))
colors = {'Early': '#27AE60', 'Middle': '#F1C40F', 'Late': '#C0392B'}

for tercile in ['Early', 'Middle', 'Late']:
    subset = car_df[car_df['lag_tercile'] == tercile]
    if len(subset) > 0:
        avg_car = np.mean(np.stack(subset['car'].values), axis=0)
        se_car = np.std(np.stack(subset['car'].values), axis=0) /
 ↪  np.sqrt(len(subset))
        ax.plot(event_days, avg_car, color=colors[tercile],
                linewidth=2, label=tercile)
        ax.fill_between(event_days,
                        avg_car - 1.96 * se_car,
                        avg_car + 1.96 * se_car,
                        color=colors[tercile], alpha=0.15)

ax.axvline(x=0, color='gray', linestyle='--', linewidth=0.8)
ax.axhline(y=0, color='gray', linewidth=0.5)
ax.set_xlabel('Event Day (Relative to Filing Date)')
ax.set_ylabel('Cumulative Abnormal Return')
ax.set_title('Market Reaction Around Filing Date by Timeliness')
ax.legend(title='Filing Tercile')
plt.tight_layout()
plt.show()
```

Figure 43.6

43.8 Filing Timeliness and Earnings Quality

Givoly and Palmon (1982) and Chambers and Penman (1984) establish that the content of disclosed information is correlated with its timing. We test this link formally: do late filers have worse earnings quality?

We formalize this with a regression that controls for firm characteristics:

```
tq_panel_reg = tq_panel.merge(
    financials[['ticker', 'fiscal_year', 'total_assets', 'net_income',
                'total_equity']],
    on=['ticker', 'fiscal_year'], how='left'
).merge(governance, on=['ticker', 'fiscal_year'], how='left')

tq_panel_reg['log_size'] = np.log(tq_panel_reg['total_assets'])
tq_panel_reg['roa'] = tq_panel_reg['net_income'] /
 ↪  tq_panel_reg['total_assets']
tq_panel_reg['late'] = (tq_panel_reg['lag_tercile'] == 'Late').astype(int)

model_tq = smf.ols(
    'accruals_quality ~ late + log_size + roa + state_ownership_pct '
    '+ big4_auditor + C(fiscal_year)',
    data=tq_panel_reg
).fit(cov_type='cluster', cov_kwds={'groups': tq_panel_reg['ticker']})

print("Accruals Quality ~ Late Filing:")
print(model_tq.summary().tables[1])
```

> **Endogeneity Caveat**
>
> The association between filing timeliness and earnings quality is likely endogenous: firms with complex accounting issues take longer to prepare financial statements, and the same complexity drives lower earnings quality. The filing lag is thus best interpreted as an *observable signal* of underlying accounting difficulty rather than a causal determinant. Instrumental variable approaches (e.g., using auditor busyness during peak filing season as an instrument for filing lag) can partially address this concern.

43.9 Disclosure Quality and Investment Efficiency

Biddle, Hilary, and Verdi (2009) demonstrate that higher financial reporting quality is associated with more efficient investment. Specifically, it reduces both over-investment (in firms

```
tq_panel = (
    annual_filings[['ticker', 'fiscal_year', 'lag_tercile', 'reporting_lag']]
    .merge(aq_df, on=['ticker', 'fiscal_year'], how='inner')
    .merge(persistence_df[['ticker', 'persistence']], on='ticker',
 ↪  how='left')
)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Accruals quality by tercile
aq_by_tercile = tq_panel.groupby('lag_tercile')['accruals_quality'].mean()
axes[0].bar(
    range(3), aq_by_tercile.values,
    color=['#27AE60', '#F1C40F', '#C0392B'], alpha=0.85,
    edgecolor='white'
)
axes[0].set_xticks(range(3))
axes[0].set_xticklabels(['Early', 'Middle', 'Late'])
axes[0].set_ylabel('Accruals Quality (  of DD Residuals)')
axes[0].set_title('Panel A: Accruals Quality by Filing Tercile')
axes[0].text(0.05, 0.95, 'Higher = lower quality',
             transform=axes[0].transAxes, fontsize=9,
             verticalalignment='top', style='italic', color='gray')

# Panel B: Persistence by tercile
per_by_tercile = tq_panel.groupby('lag_tercile')['persistence'].mean()
axes[1].bar(
    range(3), per_by_tercile.values,
    color=['#27AE60', '#F1C40F', '#C0392B'], alpha=0.85,
    edgecolor='white'
)
axes[1].set_xticks(range(3))
axes[1].set_xticklabels(['Early', 'Middle', 'Late'])
axes[1].set_ylabel('Earnings Persistence (AR(1) Coefficient)')
axes[1].set_title('Panel B: Earnings Persistence by Filing Tercile')

plt.tight_layout()
plt.show()
```

Figure 43.7

with excess cash) and under-investment (in firms that are financially constrained). The mechanism is that better disclosure reduces information asymmetry between managers and capital providers, improving the allocation of capital.

We test this prediction in Vietnam using the Biddle, Hilary, and Verdi (2009) framework:

$$\text{Investment}_{i,t+1} = \alpha + \beta_1 \text{SalesGrowth}_{i,t} + \varepsilon_{i,t+1} \tag{43.10}$$

The residual $\hat{\varepsilon}_{i,t+1}$ measures deviation from expected investment. Positive residuals indicate over-investment; negative residuals indicate under-investment. We then test whether the absolute value of this residual is lower for firms with higher disclosure quality.

```
inv_panel = financials.sort_values(['ticker', 'fiscal_year']).copy()

# Investment = change in total assets / lagged total assets
inv_panel['investment'] = (
    inv_panel.groupby('ticker')['total_assets'].pct_change()
)
inv_panel['sales_growth'] = (
    inv_panel.groupby('ticker')['revenue'].pct_change()
)

# Expected investment model
inv_model = smf.ols(
    'investment ~ sales_growth',
    data=inv_panel
).fit()
inv_panel['inv_residual'] = inv_model.resid
inv_panel['abs_inv_residual'] = inv_panel['inv_residual'].abs()

# Merge with disclosure quality
inv_eff = (
    inv_panel[['ticker', 'fiscal_year', 'abs_inv_residual',
               'investment', 'total_assets']]
    .merge(quality_panel[['ticker', 'fiscal_year',
 ↪  'dq_index']].reset_index(drop=True),
           on=['ticker', 'fiscal_year'], how='inner')
)
inv_eff['log_size'] = np.log(inv_eff['total_assets'])
inv_eff = inv_eff.set_index(['ticker', 'fiscal_year'])

# Panel regression
inv_eff_model = PanelOLS(
```

```
    dependent=inv_eff['abs_inv_residual'],
    exog=sm.add_constant(inv_eff[['dq_index', 'log_size']]),
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type='clustered', cluster_entity=True)

print("Investment Inefficiency ~ Disclosure Quality:")
print(inv_eff_model.summary)
```

A negative coefficient on `dq_index` indicates that higher disclosure quality is associated with lower investment inefficiency: firms with better disclosure make investment decisions closer to what their growth opportunities warrant.

43.10 Vietnamese Institutional Context

43.10.1 State Ownership and Disclosure

SOEs account for a substantial share of Vietnamese market capitalization. The relationship between state ownership and disclosure quality is theoretically ambiguous. On one hand, political connections may reduce the pressure to disclose transparently; government shareholders may tolerate opacity that private shareholders would not. On the other hand, post-equitization monitoring by multiple stakeholders (MOF, SCIC, minority shareholders) may create competing disclosure demands.

```
soe_panel = (
    quality_panel[['ticker', 'fiscal_year', 'dq_index',
                   'reporting_lag']].reset_index(drop=True)
    .merge(governance[['ticker', 'fiscal_year', 'state_ownership_pct']],
           on=['ticker', 'fiscal_year'], how='inner')
)

soe_panel['soe'] = (soe_panel['state_ownership_pct'] >= 50).astype(int)
soe_panel['soe_label'] = soe_panel['soe'].map(
    {1: 'SOE (≥50%)', 0: 'Private (<50%)'}
)

# Compare means
comparison = (
    soe_panel
```

```
    .groupby('soe_label')
    .agg(
        n=('dq_index', 'count'),
        mean_dq=('dq_index', 'mean'),
        median_dq=('dq_index', 'median'),
        mean_lag=('reporting_lag', 'mean'),
        median_lag=('reporting_lag', 'median')
    )
    .round(3)
)
print("SOE vs Private Firm Disclosure Comparison:")
print(comparison)

# Formal t-test
soe_dq = soe_panel[soe_panel['soe'] == 1]['dq_index']
priv_dq = soe_panel[soe_panel['soe'] == 0]['dq_index']
t_stat, p_val = stats.ttest_ind(soe_dq.dropna(), priv_dq.dropna())
print(f"\nt-test: t = {t_stat:.3f}, p = {p_val:.4f}")
```

43.10.2 IFRS Convergence and Disclosure Quality

Vietnam has been pursuing a phased convergence toward IFRS, with the Ministry of Finance issuing a roadmap for voluntary adoption by large listed firms. The transition from VAS to IFRS-aligned standards is expected to expand disclosure requirements—particularly for financial instruments (IFRS 9), revenue recognition (IFRS 15), and leases (IFRS 16). Barth, Landsman, and Lang (2008) provide evidence that IFRS adoption is associated with improvements in earnings quality and disclosure, though the effect depends on enforcement strength.

We can exploit the staggered timing of voluntary IFRS adoption across Vietnamese firms as a natural experiment:

```
# Assume DataCore provides IFRS adoption dates
ifrs_adoption = client.get_ifrs_adoption(
    exchanges=['HOSE', 'HNX'],
    fields=['ticker', 'ifrs_adoption_year']
)

# Merge with quality panel
ifrs_panel = (
    quality_panel[['ticker', 'fiscal_year',
 ↪  'dq_index']].reset_index(drop=True)
    .merge(ifrs_adoption, on='ticker', how='left')
```

```
)

# Treatment indicator
ifrs_panel['post_ifrs'] = (
    ifrs_panel['fiscal_year'] >= ifrs_panel['ifrs_adoption_year']
).astype(int).fillna(0)

ifrs_panel['treated'] = ifrs_panel['ifrs_adoption_year'].notna().astype(int)

# Simple DiD
ifrs_panel = ifrs_panel.set_index(['ticker', 'fiscal_year'])
did_model = PanelOLS(
    dependent=ifrs_panel['dq_index'],
    exog=sm.add_constant(ifrs_panel[['post_ifrs']]),
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type='clustered', cluster_entity=True)

print("DiD: IFRS Adoption and Disclosure Quality:")
print(did_model.summary)
```

Identification Concern

Voluntary IFRS adoption is endogenous because firms that choose to adopt early may already have higher-quality disclosure. The two-way fixed effects DiD absorbs time-invariant firm characteristics and common time trends, but cannot fully address selection on time-varying unobservables. Researchers should consider matching estimators (e.g., propensity score matching on pre-adoption characteristics) or instrumental variable approaches as robustness checks.

43.11 Predicting Late Filings

Can we predict which firms will file late? This is valuable for portfolio construction (avoiding potential bad-news firms) and for regulators (targeting enforcement resources). We use a logistic model with financial and governance predictors:

$$\Pr(\text{Late}_{i,t} = 1) = \Lambda\left(\alpha + \beta' \mathbf{X}_{i,t-1}\right) \tag{43.11}$$

where $\Lambda(\cdot)$ is the logistic function and $\mathbf{X}_{i,t-1}$ are lagged predictors.

```
from sklearn.metrics import roc_auc_score, classification_report
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

pred_panel = (
    annual_filings[['ticker', 'fiscal_year', 'late_filing']]
    .merge(financials, on=['ticker', 'fiscal_year'], how='left')
    .merge(governance, on=['ticker', 'fiscal_year'], how='left')
)

# Lagged predictors
pred_panel = pred_panel.sort_values(['ticker', 'fiscal_year'])
for col in ['total_assets', 'net_income', 'operating_cash_flow',
            'total_equity', 'revenue']:
    pred_panel[f'{col}_lag'] = pred_panel.groupby('ticker')[col].shift(1)

pred_panel['log_size_lag'] = np.log(pred_panel['total_assets_lag'])
pred_panel['roa_lag'] = (
    pred_panel['net_income_lag'] / pred_panel['total_assets_lag']
)
pred_panel['leverage_lag'] = (
    (pred_panel['total_assets_lag'] - pred_panel['total_equity_lag'])
    / pred_panel['total_assets_lag']
)
pred_panel['cfo_ratio_lag'] = (
    pred_panel['operating_cash_flow_lag'] / pred_panel['total_assets_lag']
)

# Previous late filing indicator
pred_panel['prev_late'] = (
    pred_panel.groupby('ticker')['late_filing'].shift(1)
)

features = [
    'log_size_lag', 'roa_lag', 'leverage_lag', 'cfo_ratio_lag',
    'state_ownership_pct', 'foreign_ownership_pct',
    'big4_auditor', 'board_independence_pct', 'prev_late'
]

clean = pred_panel.dropna(subset=features + ['late_filing'])
X = clean[features]
y = clean['late_filing']
```

```
# Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Logistic regression with cross-validation
lr = LogisticRegression(max_iter=1000, penalty='l2', C=1.0)
cv_scores = cross_val_score(lr, X_scaled, y, cv=5, scoring='roc_auc')

print(f"5-Fold Cross-Validated AUC: {cv_scores.mean():.3f} ±
 ↪  {cv_scores.std():.3f}")

# Fit on full sample for coefficient interpretation
lr.fit(X_scaled, y)
coef_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': lr.coef_[0],
    'Odds Ratio': np.exp(lr.coef_[0])
}).sort_values('Coefficient', ascending=False)

print("\nLogistic Regression Coefficients:")
print(coef_df.to_string(index=False))
```

43.12 Summary

This chapter has examined corporate disclosure quality and timing in Vietnam along several dimensions. The key findings and methodological contributions are in Table 43.3

Table 43.3: Summary of findings by theme.

Theme	Key Result	Reference
Good news early	Early filers earn positive CARs around filing dates	Givoly and Palmon (1982)
Textual quality	Forward-looking density and numerical specificity vary substantially	F. Li (2008)
Composite DQ index	Foreign ownership and Big 4 auditors are strongest determinants	Botosan (1997)
Cost of capital	Higher DQ is associated with lower implied cost of equity	Diamond and Verrecchia (1991)

Theme	Key Result	Reference
Liquidity	Higher DQ firms have lower Amihud illiquidity	Lang, Lins, and Maffett (2012)
Investment efficiency	Higher DQ reduces absolute investment residuals	Biddle, Hilary, and Verdi (2009)
Strategic timing	Evidence of bad-news clustering on high-congestion days	Hirshleifer, Lim, and Teoh (2009)
IFRS adoption	Preliminary evidence of DQ improvement post-adoption	Barth, Landsman, and Lang (2008)

The Vietnamese disclosure environment is shaped by a combination of regulatory mandates (Circular 155, Securities Law 2019), enforcement capacity (SSC penalties and trading suspensions), and firm-level incentives (ownership structure, auditor choice, governance quality). As Vietnam continues its IFRS convergence and capital market development, the information environment is expected to evolve, creating opportunities for researchers to study the dynamics of disclosure quality in a rapidly changing institutional setting.

```
from sklearn.metrics import roc_curve, auc

lr.fit(X_scaled, y)
y_prob = lr.predict_proba(X_scaled)[:, 1]
fpr, tpr, _ = roc_curve(y, y_prob)
roc_auc = auc(fpr, tpr)

fig, ax = plt.subplots(figsize=(7, 7))
ax.plot(fpr, tpr, color='#2C5F8A', linewidth=2,
        label=f'Logistic Model (AUC = {roc_auc:.3f})')
ax.plot([0, 1], [0, 1], color='gray', linestyle='--', linewidth=1)
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('Late Filing Prediction: ROC Curve')
ax.legend(loc='lower right')
ax.set_aspect('equal')
plt.tight_layout()
plt.show()
```

Figure 43.8

44 Standardized Earnings Surprises (SUE)

In the context of the Ho Chi Minh Stock Exchange (HOSE) and the Hanoi Stock Exchange (HNX), earnings announcements represent critical information events. Investors and quantitative analysts continuously monitor the deviation between reported earnings and market expectations. This deviation is quantified as the Standardized Earnings Surprise (SUE).

This chapter details the methodology for calculating SUE using three distinct approaches frequently utilized in academic literature and institutional research. We apply these methods to a dataset of Vietnamese large-cap equities to illustrate the mechanics of the calculation. The goal is to isolate the "surprise" component of earnings, which is a known predictor of post-earnings announcement drift (PEAD) (Bernard and Thomas 1989; Livnat and Mendenhall 2006).

44.1 Methodology

We define three primary methods for calculating SUE. Each method differs in how it establishes the "expected" earnings value.

44.1.1 Method 1: Seasonal Random Walk

This method assumes that earnings follow a seasonal pattern. The best predictor for the current quarter's earnings per share (EPS) is the EPS from the same quarter in the previous year. This controls for the seasonality often seen in Vietnamese sectors like retail and agriculture.

$$SUE_1 = \frac{EPS_t - EPS_{t-4}}{P_t}$$

Where:

- EPS_t is the current quarterly Earnings Per Share.
- EPS_{t-4} is the Earnings Per Share from the same quarter of the prior fiscal year.
- P_t is the stock price at the end of the quarter (used as a deflator).

44.1.2 Method 2: Exclusion of Special Items

Reported earnings often contain non-recurring items (e.g., asset sales, one-time write-offs) that distort the true operating performance. This method adjusts the reported EPS by removing the after-tax impact of special items.

In Vietnam, the standard Corporate Income Tax (CIT) rate is generally 20%. We adjust special items to reflect their impact on net income.

$$Adjusted\ EPS = Reported\ EPS - \frac{Special\ Items \times (1 - CIT)}{Shares\ Outstanding}$$

The SUE calculation then follows the seasonal logic but uses the adjusted EPS figures:

$$SUE_2 = \frac{Adj\ EPS_t - Adj\ EPS_{t-4}}{P_t}$$

44.1.3 Method 3: Analyst Consensus

This method relies on market consensus rather than historical time series. It compares the actual reported earnings against the median analyst forecast provided prior to the announcement.

$$SUE_3 = \frac{Actual\ EPS - Median\ Estimate}{P_t}$$

Whether the consensus is the right expectation, and when a machine-learning correction of it is better, is the subject of Chapter 45.

44.2 Data Description

For this analysis, we utilize a dataset covering the fiscal years 2023 through 2025. The data includes quarterly financial statements and analyst consensus estimates for a selection of VN30 index constituents.

The dataset, `vietnam_fin_data.csv`, contains the following columns:

- **ticker**: Stock symbol (e.g., VNM, VCB, HPG).
- **fiscal_year**: The financial year.
- **fiscal_qtr**: The financial quarter (1-4).
- **eps_basic**: Basic Earnings Per Share (VND).

- **price_close**: Closing price at quarter end (VND).
- **special_items**: Pre-tax special items value (VND millions). (i.e., `is_other_profit` in DataCore).
- **shares_out**: Shares outstanding (millions).
- **analyst_med**: Median analyst EPS estimate (VND).

44.2.1 Visualizing the Core Data

Below is a tabular representation of the raw data we have ingested for the analysis.

ticker	fiscal_year	fiscal_qtr	eps_basic	price_close	special_items	shares_out	analyst_med
VNM	2023	1	1200	68000	0	2090	1150
VNM	2023	2	1350	71000	50000	2090	1300
VNM	2023	3	1400	74000	0	2090	1450
VNM	2023	4	1100	69000	-20000	2090	1150
VNM	2024	1	1300	72000	0	2090	1250
VNM	2024	2	1500	75000	0	2090	1400
VCB	2023	1	1800	85000	10000	5500	1700
VCB	2024	1	2100	92000	0	5500	2000

44.3 Implementation

44.3.1 Python Setup and Data Loading

First, we establish our environment and load the dataset. We ensure the data is sorted by ticker and time to allow for accurate lag calculations.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Creating the dataset directly for this chapter's demonstration
data = {
    'ticker': ['VNM']*8 + ['VCB']*8 + ['HPG']*8,
    'fiscal_year': [2023, 2023, 2023, 2023, 2024, 2024, 2024, 2024] * 3,
    'fiscal_qtr': [1, 2, 3, 4, 1, 2, 3, 4] * 3,
    'eps_basic': [
        1200, 1350, 1400, 1100, 1300, 1500, 1450, 1250, # VNM
```

```
        1800, 1900, 2000, 2200, 2100, 2300, 2400, 2600, # VCB
        500, 600, 550, 400, 700, 800, 750, 600          # HPG
    ],
    'price_close': [
        68000, 71000, 74000, 69000, 72000, 75000, 73000, 70000, # VNM
        85000, 88000, 90000, 95000, 92000, 96000, 98000, 102000, # VCB
        20000, 22000, 21000, 19000, 25000, 28000, 27000, 24000 # HPG
    ],
    'special_items': [
        0, 50000, 0, -20000, 0, 0, 10000, 0, # VNM (VND Millions)
        10000, 0, 0, 50000, 0, 20000, 0, 0, # VCB
        0, 0, -50000, 0, 100000, 0, 0, 0 # HPG
    ],
    'shares_out': [2090]*8 + [5580]*8 + [5810]*8, # In Millions
    'analyst_med': [
        1150, 1300, 1450, 1150, 1250, 1400, 1480, 1200, # VNM
        1700, 1850, 1950, 2150, 2000, 2250, 2450, 2550, # VCB
        450, 550, 600, 450, 650, 750, 800, 650 # HPG
    ]
}

df = pd.DataFrame(data)

# Sort strictly to ensure shift operations work on correct temporal sequence
df = df.sort_values(by=['ticker', 'fiscal_year', 'fiscal_qtr'])
print(df.head())
```

```
   ticker  fiscal_year  fiscal_qtr  eps_basic  price_close  special_items  \
16    HPG         2023           1        500        20000              0
17    HPG         2023           2        600        22000              0
18    HPG         2023           3        550        21000         -50000
19    HPG         2023           4        400        19000              0
20    HPG         2024           1        700        25000         100000

    shares_out  analyst_med
16        5810          450
17        5810          550
18        5810          600
19        5810          450
20        5810          650
```

44.3.2 Calculation Logic

We now apply the functions to calculate the three variations of SUE.

Step 1: Handling Seasonality (Lags)

For Methods 1 and 2, we require the data from the same quarter of the previous year (lag 4).

```
# Group by ticker to ensure we don't shift data between companies
df['eps_lag4'] = df.groupby('ticker')['eps_basic'].shift(4)
```

Step 2: Adjusting for Special Items

For Method 2, we must strip out non-recurring items. We apply the Vietnamese Corporate Income Tax (CIT) rate of 20%.

The formula for the adjustment per share is:

$$\text{Adjustment} = \frac{\text{Special Items} \times (1 - 0.20)}{\text{Shares Outstanding}}$$

```
# Constants
CIT_RATE_VN = 0.20

# Calculate impact per share
# Note: special_items are in millions, shares_out are in millions
# The units cancel out, leaving the result in VND per share.
df['spi_impact_per_share'] = (df['special_items'] * (1 - CIT_RATE_VN)) /
 ↪  df['shares_out']

# Calculate Adjusted EPS
df['eps_adjusted'] = df['eps_basic'] - df['spi_impact_per_share']

# Create lag for Adjusted EPS
df['eps_adj_lag4'] = df.groupby('ticker')['eps_adjusted'].shift(4)
```

Step 3: Computing SUE Variants

We finalize the calculation by computing the difference between actual (or adjusted) and expected values, deflated by the stock price.

```
# Method 1: Seasonal Random Walk (Standard EPS)
df['sue_1'] = (df['eps_basic'] - df['eps_lag4']) / df['price_close']

# Method 2: Seasonal Random Walk (Excluding Special Items)
df['sue_2'] = (df['eps_adjusted'] - df['eps_adj_lag4']) / df['price_close']

# Method 3: Analyst Forecasts (IBES Equivalent)
df['sue_3'] = (df['eps_basic'] - df['analyst_med']) / df['price_close']

# Scaling for readability (converting to percentage)
df['sue_1_pct'] = df['sue_1'] * 100
df['sue_2_pct'] = df['sue_2'] * 100
df['sue_3_pct'] = df['sue_3'] * 100
```

44.4 Results and Analysis

We present the calculated standardized earnings surprises for the fiscal year 2024. Positive values indicate a positive surprise (beating expectations), while negative values indicate a miss.

44.4.1 Tabular Results (FY 2024)

```
# Filter for 2024 results where lag data exists
results_2024 = df[df['fiscal_year'] == 2024][['ticker', 'fiscal_qtr',
 ↪  'sue_1_pct', 'sue_2_pct', 'sue_3_pct']]

# Display formatted table
from IPython.display import display, Markdown
markdown_table = results_2024.to_markdown(index=False, floatfmt=".4f")
display(Markdown(markdown_table))
```

ticker	fiscal_qtr	sue_1_pct	sue_2_pct	sue_3_pct
HPG	1	0.8000	0.7449	0.2000
HPG	2	0.7143	0.7143	0.1786
HPG	3	0.7407	0.7152	-0.1852
HPG	4	0.8333	0.8333	-0.2083
VCB	1	0.3261	0.3276	0.1087

ticker	fiscal_qtr	sue_1_pct	sue_2_pct	sue_3_pct
VCB	2	0.4167	0.4137	0.0521
VCB	3	0.4082	0.4082	-0.0510
VCB	4	0.3922	0.3992	0.0490
VNM	1	0.1389	0.1389	0.0694
VNM	2	0.2000	0.2255	0.1333
VNM	3	0.0685	0.0632	-0.0411
VNM	4	0.2143	0.2033	0.0714

44.4.2 Visualization

The following figure plots the Analyst-based SUE (Method 3) for the selected tickers over the 2024 fiscal year.

```
pivot_sue = results_2024.pivot(index='fiscal_qtr', columns='ticker',
 ↪  values='sue_3_pct')

plt.figure(figsize=(10, 6))
for column in pivot_sue.columns:
    plt.plot(pivot_sue.index, pivot_sue[column], marker='o', label=column)

plt.title('Method 3: Analyst Based SUE (FY 2024)')
plt.xlabel('Fiscal Quarter')
plt.ylabel('SUE (%)')
plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
plt.legend(title='Ticker')
plt.grid(True, linestyle=':', alpha=0.6)
plt.xticks([1, 2, 3, 4])
plt.show()
```

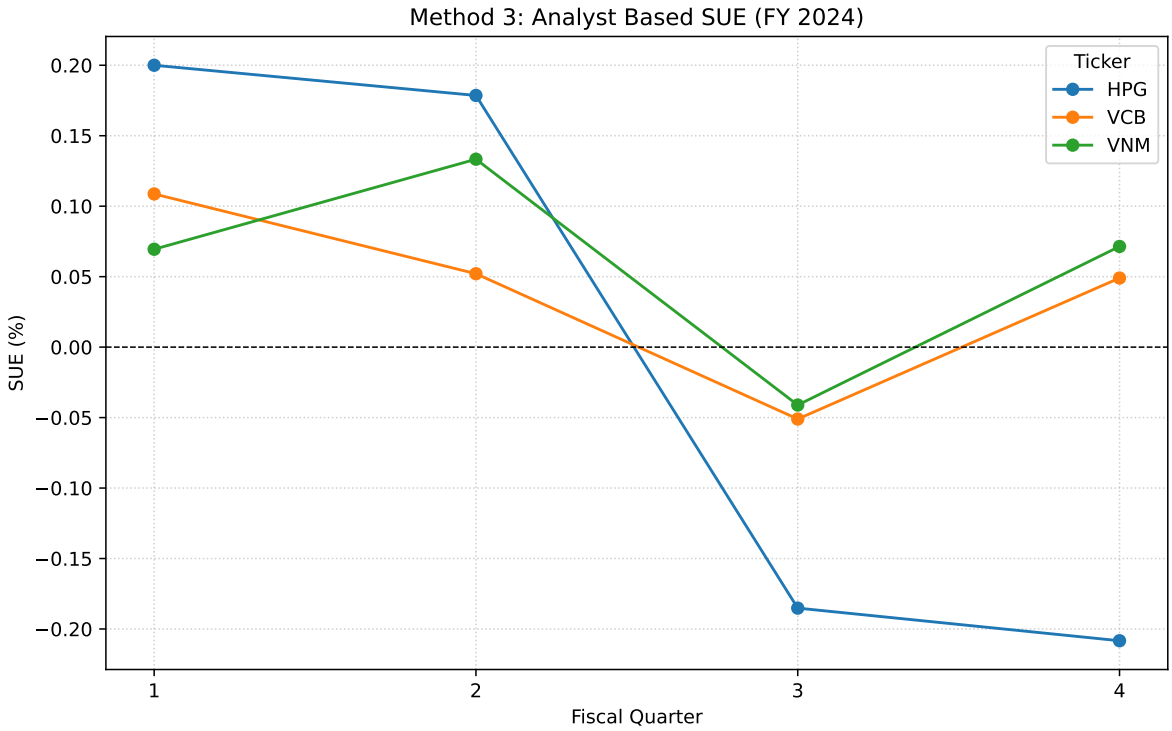


Figure 44.1

44.5 Conclusion

In this chapter, we have formalized the calculation of Standardized Earnings Surprises for the Vietnamese market. We demonstrated that relying solely on raw EPS growth (Method 1) can be misleading in the presence of non-recurring items. Furthermore, analyst-based surprises (Method 3) often provide a cleaner signal of new information reaching the market.

For robust quantitative modeling in Vietnam, we recommend using Method 2 when analyst data is sparse (common in small-cap stocks) and Method 3 for VN30 constituents where analyst coverage is deep and liquid.

45 Machine Learning Earnings Forecasts and Market Expectations

Every earnings-based design in this part of the book needs an *expected* earnings number. The earnings surprise of Chapter 44 subtracts it from reported earnings, implied cost of capital inverts a valuation model around it, and forward P/E divides price by it. For four decades the default proxy has been the analyst consensus, because analysts beat time-series models (Bradshaw et al. 2012) and because the market appears to react to deviations from consensus (Sri Prakash Kothari 2001; Livnat and Mendenhall 2006). Machine learning has unsettled that default. A string of recent papers reports that statistical models can produce earnings forecasts more accurate than analysts' forecasts (R. T. Ball and Ghysels 2018; Binsbergen, Han, and Lopez-Lira 2023; K. Cao and You 2024; Silva and Thesmar 2024), which suggests that researchers should replace the consensus with a machine forecast.

J. L. Campbell et al. (2026) ask the obvious follow-up questions that the earlier papers mostly skipped. *Which* machine forecast? Their answer is that the claim "machines beat analysts" depends on modelling choices that most papers make without discussion: 79% of the 3,024 machine-learning models they estimate are *less* accurate than the analyst consensus. They then ask *why* the good models win, and find that they win by correcting predictable errors in analysts' forecasts, not by finding new information. Finally, they ask which forecast the *market* uses. Stock prices behave as if investors correct most, but not all, of the predictable analyst error, and the uncorrected part shows up in returns for up to nine months after the announcement.

This chapter lays the paper out in full and then reproduces it in three ways:

1. Section 45.2 walks through the design (the 3,024-model experiment, the accuracy measure, the indirect forecasting approach, and the market-expectations test) and its results.
2. Section 45.3 re-computes the paper's headline numbers from the authors' public replication data with independent code.
3. Section 45.4 builds a simulation in which the true analyst bias, the true market expectation, and the true information sets are known, and shows each mechanism the paper documents: why loss function, analyst inputs, the indirect approach, and the cross-validation scheme matter, why the machine's advantage concentrates in small firms and long horizons, and what the market-expectations regression does and does not identify.

Section 45.6 then turns the paper into a research design for Vietnam, where analyst coverage is thin and the question "should I use the consensus or a model?" arises for almost every firm.

Sources used in this chapter

The paper is J. L. Campbell et al. (2026), *Management Science* 72(10), 8863–8887. The full text of the published version sits behind a bot check, so the method details below come from two open sources: (i) the authors' replication package on Mendeley Data (DOI 10.17632/2pg764w6gs.1, CC BY 4.0), whose table files match the published tables, and (ii) the April 2024 working-paper version, which is Chapter 2 of Harrison Ham's University of Georgia dissertation. Numbers quoted from the package are the published numbers. Where a number exists only in the working paper (the horizon table, the implied-cost-of-capital test, and the analyst-value-added analysis), the text says so, because some magnitudes changed between versions.

45.1 The Question and Why It Matters

An earnings expectation can serve two different purposes, and the paper's title is a reminder that the two need not coincide.

1. **A statistical forecast.** Which number minimises the ex post forecast error? This is what matters for valuation inputs such as forward earnings and implied cost of capital (R. Frankel and Lee 1998; Hou, Dijk, and Zhang 2012).
2. **A proxy for the market's belief.** Which number do prices reflect? This is what matters for earnings-surprise and earnings-response designs, where the researcher wants SUE to measure *news to investors*.

If investors were fully rational and had access to the best statistical model, the two would coincide. If investors share analysts' biases (Bordalo et al. 2019) or simply overweight the consensus (So 2013), the most accurate forecast and the market's expectation diverge, and the right choice depends on the research question. J. L. Campbell et al. (2026) quantify both sides with the same set of forecasts.

45.2 The Paper in Detail

45.2.1 Data and Forecast Targets

The sample is the CRSP–Compustat–I/B/E/S intersection. Analysts' forecasts are the I/B/E/S median consensus, which I/B/E/S computes on the third Thursday of each month. Two targets are forecast every month:

- **FQ**: the next quarterly EPS (I/B/E/S horizon Q1, or Q2 if Q1 has already been announced before month end). Data from January 1985.

- **FY2**: two-year-ahead annual EPS. Data from January 1983, as in Bradshaw et al. (2012).

Both end in December 2020, and out-of-sample evaluation runs from June 1990. The predictor set has 77 features: the WRDS Financial Ratio Suite, five CRSP variables (price, monthly return, six-month momentum, industry momentum, log market capitalisation), and five analyst variables:

Table 45.1: Analyst-related predictors in J. L. Campbell et al. (2026) (working-paper Table A2).

Variable	Definition
`medestqtr`, `medest2`	Current consensus forecast for FQ / FY2
`rev_FYqtr_3m`, `rev_FY2_3m`	Three-month revision of the consensus
`ibes_earnings_qtr`, `ibes_earnings_ann`	Most recently realised quarterly / annual EPS
`last_Fqtrana_fe_med`, `last_F2ana_fe_med`	Most recently *realised* analyst forecast error at a comparable distance to the period end
`distqtr`, `dist2`	Months between the forecast date and the fiscal period end

Predictors are winsorised monthly at 1% and 99%, missing values are filled with the Fama–French 38-industry median and then the cross-sectional median, and features are standardised with training-window means and standard deviations. Firm-months with price at or below \$1 are dropped. The final datasets hold about 1.26 million (FQ) and 1.35 million (FY2) firm-months, roughly 2,300–2,400 firms per month.

The timeline is the part most often gotten wrong in this literature. At the end of month t, the training set contains only observations whose target earnings were *announced* in months $t-119$ to t (rolling) or up to t (expanding). The test set is month t's cross-section, for earnings announced after t. A firm whose earnings are announced in $t+1$ is therefore in the test set at t and not in the training set, even if its fiscal period ended earlier. It is the forecasting analogue of the point-in-time rules used throughout the book: a feature or target enters only once it was public.

45.2.2 The Accuracy Measure: ML Superiority

Following Bradshaw et al. (2012), accuracy is measured relative to analysts on a per-dollar-of-price basis. For firm i, forecast month t, and fiscal period ending T,

$$\text{ML Superiority}_{i,t} = \left|\frac{\text{EPS}_{i,T} - \text{AF}_t(\text{EPS}_{i,T})}{P_{i,t}}\right| - \left|\frac{\text{EPS}_{i,T} - \text{ML}_t(\text{EPS}_{i,T})}{P_{i,t}}\right|, \tag{45.1}$$

where both price-scaled errors are winsorised to $[-1, 1]$, FQ errors are annualised (multiplied by four), and the result is reported in percentage points. A positive value means the machine's absolute error is smaller than the analysts'. A model's ML Superiority is the panel average, with standard errors two-way clustered by firm and year.

The measure has an economic reading: 0.10 percentage points is a reduction in the absolute forecast error of 1 cent for a \$10 stock. For scale, the median $|\text{EPS}/P|$ in the sample is 6.07% (FQ, annualised) and 6.43% (FY2). The paper uses 1 cent per \$10 of price as its threshold for an economically meaningful improvement, motivated by evidence that investors discount earnings beats smaller than a cent (Keung, Lin, and Shih 2010).

Because Equation 45.1 is an *absolute*-error comparison, the forecast that maximises it is the conditional *median* of earnings, not the conditional mean. This one fact explains the single largest result in the paper.

45.2.3 Direct and Indirect Forecasts

A machine can use analysts' forecasts in three ways:

1. **Direct without analysts.** Regress $\text{EPS}_{i,T}$ on non-analyst features: $\widehat{\text{EPS}} = \hat{f}(X)$.
2. **Direct with analysts.** Add the analyst variables to X: $\widehat{\text{EPS}} = \hat{f}(X, \text{AF}, \dots)$.
3. **Indirect** (R. Frankel and Lee 1998; So 2013). Forecast the analyst *error* and correct the consensus:

$$\widehat{\text{EPS}}_{i,T} = \text{AF}_t + \hat{g}(X, \text{AF}, \dots), \qquad \hat{g} = \arg\min_g \sum L(\text{EPS}_{i,T} - \text{AF}_t,\ g(\cdot)). \tag{45.2}$$

For an *unpenalised linear* model with AF among the regressors, the direct-with-analysts and indirect forecasts are identical: subtracting AF from the target only shifts its coefficient by one. The two approaches differ for every model that shrinks or approximates. A ridge or lasso penalty shrinks coefficients toward zero, so the direct model is shrunk toward "ignore the analyst" while the indirect model is shrunk toward "trust the analyst". A tree ensemble has to rebuild the near-identity map from AF to EPS out of piecewise-constant steps in the direct approach, while in the indirect approach the identity is free and every split is spent on the bias. Since the consensus is an excellent forecast to begin with, "trust the analyst unless the data say otherwise" is the better default. Section 45.4.1 verifies both statements.

45.2.4 The 3,024-Model Experiment

Every model minimises an average loss plus a penalty over the estimation window,

$$\hat{f} = \arg\min_f \ \frac{1}{N} \sum_i L(y_i, f(X_i)) + G(f; \gamma),$$

and the paper varies nine choices that existing studies make differently:

Table 45.2: The nine specification choices. OLS has no hyperparameters, so it has 144 configurations; each other algorithm has 576, for 3,024 models in total.

Choice set	Options
Loss function L	MAE; MSE; MSE with the target trimmed at 1%/99% in the training set
Algorithm (G)	OLS, Lasso, Ridge, Elastic Net, random forest (RF), gradient-boosted regression trees (GBRT)
Cross-validation for hyperparameters	Time-series (validation after training); panel 5-fold
Estimation window	Rolling 10 years; expanding
Hyperparameter tuning frequency	Once at the beginning; annually
Model refit frequency	Monthly; yearly
Forecasting approach	Direct; indirect (Equation 45.2)
Predictor set	With analyst variables; without
Scaling in the forecasting step	None; EPS variables divided by price
Forecast period	FQ; FY2

The authors report that estimating all 3,024 models took about five years of machine time on university servers.

45.2.5 Result 1: Most Machine Forecasts Lose to Analysts

Of the 3,024 models, 79% have negative ML Superiority (88% for FQ and 70% for FY2). The distribution has a long left tail reaching below −15 percentage points and a narrow right tail that never exceeds +0.65. Table 45.3, reproduced from the replication package, averages ML Superiority over all models sharing a given option and reports the best and worst option in each choice set.

Table 45.3: Average ML Superiority (percentage points) by specification choice, all 3,024 models. Source: `Specification_Comparison.tex` in the replication package.

Choice set	Best option	Worst option	Superiority (best)	Superiority (worst)	Difference	Run time diff. (hours)
Loss function	MAE	MSE	−0.37	−6.69	6.32	14.9
Algorithm	RF	OLS	−1.22	−4.58	3.36	15.4

Choice set	Best option	Worst option	Superiority (best)	Superiority (worst)	Difference	Run time diff. (hours)
Forecasting approach	Indirect	Direct	−1.51	−3.05	1.55	−0.5
CV scheme	Time-series	K-fold	−1.70	−3.17	1.47	−4.6
Predictor set	With analysts	Without	−2.11	−3.39	1.28	0.0
Scaling	Price	None	−2.30	−2.78	0.48	−0.7
Estimation window	Rolling	Expanding	−2.40	−2.67	0.27	−7.7
Refit frequency	Monthly	Yearly	−2.49	−2.59	0.10	4.4
Tuning frequency	Annual	Beginning	−2.39	−2.48	0.09	21.2

Four choices dominate:

- **The loss function.** None of the 1,008 MSE models beats analysts; 46% of the MAE models do. Earnings scaled by price have fat tails and a long *left* tail from losses and write-offs, so the conditional mean sits below the conditional median. An MSE model chases the mean and is penalised by an absolute-error yardstick; trimming the target helps (16% of trimmed-MSE models win) but does not fix the misalignment.
- **The indirect approach** raises the share of winning models from 12% (direct) to 39%.
- **Time-series cross-validation** beats 5-fold cross-validation. With overlapping targets (the same FY2 EPS is the target for up to 24 monthly forecasts of the same firm), a shuffled k-fold split puts near-duplicates of each validation row in the training folds, the validation error is too optimistic, and tuning picks models that are too flexible. This is the concern Bertomeu (2020) raises for accounting data.
- **Analyst inputs.** Only 4% of the models without analyst variables beat analysts.

Restricting attention to MAE loss, the indirect approach, and time-series CV leaves 176 models, *all* of which beat analysts. The remaining choices (window, refit and tuning frequency) change accuracy by a tenth of a percentage point or less but change run time by up to 21 hours, so the cheap options are fine.

45.2.6 Result 2: The Best Model, and Where Its Edge Comes From

Table 45.4: Best specification by algorithm, among models with MAE loss, time-series CV and no price scaling. Every winner uses the indirect approach and monthly refitting, and all but FY2 GBRT (expanding window) use the rolling window. Source: `Best_SpecsFQ.tex`, `Best_SpecsFY2.tex`, whose header swaps the labels of the refit and tuning-frequency columns.

Algorithm	FQ superiority	FY2 superiority
OLS	0.113	0.267
Lasso	0.113	0.300
Ridge	0.113	0.306
Elastic net	0.113	0.309
Random forest	0.116	0.365
GBRT	**0.201**	**0.601**

GBRT wins for both horizons (Table 45.4). Table 45.5 isolates the role of analysts by holding the best specification fixed and changing only how analyst information enters.

Table 45.5: ML Superiority (percentage points) by approach and algorithm; all entries significant at 5% or better. Source: `BradshawT3FullBestnoana.tex`.

Approach	OLS	Lasso	Ridge	EN	RF	GBRT
FQ, direct without analysts	−1.722	−1.721	−1.724	−1.731	−2.629	−1.132
FY2, direct without analysts	−0.460	−0.434	−0.461	−0.420	−0.558	0.205
FQ, direct with analysts	−0.113	−0.117	−0.120	−0.113	−2.369	0.092
FY2, direct with analysts	0.167	0.165	0.168	0.168	−0.452	0.536
FQ, indirect	0.113	0.113	0.113	0.113	0.116	0.201
FY2, indirect	0.267	0.300	0.306	0.309	0.365	0.601

Three lessons follow. Without analyst information, no model beats analysts for FQ, and only GBRT does so for FY2, by 0.2 points (2 cents per $10 stock). With analyst variables as inputs, the direct approach helps but random forests still lose badly, because averaging tree leaves cannot reproduce the near-linear map from forecast to outcome. The indirect approach makes *every* algorithm beat analysts and compresses the gap between the worst and best algorithm. In short, the machine's edge is a correction to the analyst, not a substitute for the analyst.

What does the correction look like? The paper decomposes the GBRT predicted bias with SHAP values. For FQ the most important features are the lagged realised analyst error (14.0%

of total mean absolute SHAP), price (7.3%), the consensus itself (6.5%), size (4.7%), and the one-month return (4.5%). For FY2 they are the consensus (24.1%), price (14.0%), the three-month analyst revision (6.4%), six-month momentum (5.1%), and free cash flow over operating cash flow (5.0%). The ten most important features account for half (FQ) and two-thirds (FY2) of the total. The SHAP profile of the lagged analyst error is S-shaped: steeper than the linear coefficient near zero and flattening in the tails. Analysts' errors are persistent (they under-correct their own past mistakes), and the machine learns the persistence nonlinearly. Price and the level of the forecast enter because analyst errors vary systematically with both, in line with the conditional biases documented by Binsbergen, Han, and Lopez-Lira (2023) and with the characteristics-based forecasts of analyst errors in So (2013).

45.2.7 Result 3: When the Machine's Edge Is Economically Large

The average superiority of 0.20 points (FQ) and 0.60 points (FY2) hides large cross-sectional differences.

Firm size. Sorting monthly on market capitalisation:

Table 45.6: ML Superiority of the best model (GBRT, indirect) by size quintile, percentage points (recomputed in Section 45.3).

Size quintile	Q1 (small)	Q2	Q3	Q4	Q5 (large)
FQ	0.569	0.194	0.106	0.072	0.062
FY2	1.712	0.677	0.362	0.189	0.064

For the largest quintile, which holds about 87% of market capitalisation, the gain is about 0.6 cents per $10 stock: real but economically trivial. For the smallest quintile it is 5.7 cents (FQ) and 17 cents (FY2). A multivariate Fama–MacBeth regression of superiority on percentile ranks of firm attributes (Newey–West, 24 lags) confirms that small size, high idiosyncratic volatility, many business segments, wide bid–ask spreads, and (for FY2) external financing and accrual volatility are associated with a larger machine edge. These are the information-uncertainty and analyst-incentive variables of the analyst-bias literature.

Horizon. In the working-paper version, GBRT's superiority for FQ forecasts made one month before the announcement is only 0.086 points, rising to 0.27 at three months; for FY2 it is 0.52 at twelve months and peaks at 0.67 at nineteen months. Analysts are hard to beat on the next quarter.

Time. The 10-year rolling average of the FQ superiority falls from about 0.31 points in 2000 to about 0.10 by 2020 (the decade averages in our recomputation are 0.30, 0.19 and 0.10 for FQ and 0.84, 0.59 and 0.33 for FY2), but it stays significantly positive. Analysts appear to have removed part of their own predictable error over time, which is what one would expect if they adopt the same tools.

Thin coverage. For the two smallest size quintiles, with one to three analysts, a GBRT model that does *not* use analyst inputs is significantly *worse* than analysts for forecasts made one or two months before the announcement (up to six months when there are three analysts), and significantly better only from eight months ahead (smallest firms, a single analyst) or, in most cells, ten months ahead (package table `Superiority_low_coverage.tex`). Even one or two analysts carry information a machine cannot reconstruct from financial statements, until the forecast horizon is long.

These are the "when not to" conditions in the title. For large firms and short horizons, the consensus is about as good as the best machine, costs nothing to compute, and is reproducible. For small firms and long horizons the machine is materially better.

45.2.8 Result 4: Which Expectation Does the Market Use?

The market test regresses size-adjusted three-day announcement returns on earnings surprises built from each expectation. With $\mathrm{SUE}^X = (\mathrm{EPS} - X_{t-1})/P_{t-1}$ for $X \in \{\mathrm{AF}, \mathrm{ML}\}$, measured at the month end before the announcement,

$$R_{d-1,d+1} = c + \beta\,\mathrm{SUE} + \varepsilon.$$

Across the 1,512 FQ models, the rank correlation between a model's ML Superiority and the earnings-response coefficient of its SUE is 0.97, and 90% of the models produce a *smaller* ERC than the analyst-based SUE. More accurate forecasts produce surprises that look more like news to the market, so a poorly specified machine forecast is a worse proxy for the market's expectation than the consensus.

To estimate *how much* weight the market puts on each forecast, suppose the market's surprise is a weighted combination

$$\mathrm{SUE}^M = w_{AF}\,\mathrm{SUE}^{AF} + w_{ML}\,\mathrm{SUE}^{ML}, \qquad R = c + \beta^M\,\mathrm{SUE}^M + \varepsilon. \tag{45.3}$$

Write $G = (\mathrm{ML} - \mathrm{AF})/P$, so that $\mathrm{SUE}^{AF} = \mathrm{SUE}^{ML} + G$. Substituting,

$$R = c + \underbrace{\beta^M(w_{AF} + w_{ML})}_{\beta_1}\,\mathrm{SUE}^{ML} + \underbrace{\beta^M w_{AF}}_{\beta_2}\,G + \varepsilon, \qquad R = c + \underbrace{\beta^M(w_{AF} + w_{ML})}_{\gamma_1}\,\mathrm{SUE}^{AF} \underbrace{- \beta^M w_{ML}}_{\gamma_2}\,G + \varepsilon, \tag{45.4}$$

so that

$$\frac{\beta_2}{\beta_1} = \frac{w_{AF}}{w_{AF} + w_{ML}}, \qquad -\frac{\beta_2}{\gamma_2} = \frac{w_{AF}}{w_{ML}}. \tag{45.5}$$

If prices fully undo the predictable analyst error, $\beta_2 = 0$. If the market simply uses the consensus, $\beta_2 = \beta_1$. The gap G is the machine-predicted analyst bias, so the regression asks whether the market reacts to the part of the analyst surprise that was predictable in advance.

Table 45.7: Earnings-response tests (published Table 7, recomputed in Section 45.3). Returns and SUEs in percent, trimmed at 1%/99% per quarter; t-statistics two-way clustered by firm and quarter. Size and IO are quarterly percentile ranks; their main effects are included but not shown.

	(1)	(2)	(3)	(4)	(5)
SUE^{ML}	0.241		0.245	0.181	0.180
	(5.89)		(5.62)	(5.77)	(9.44)
SUE^{AF}		0.209			
		(5.91)			
Bias G			0.093	0.087	0.099
			(5.36)	(5.22)	(4.83)
$\text{SUE}^{ML}\times$ Size rank				0.232	
				(3.60)	
$G\times$ Size rank				−0.086	
				(−2.19)	
$\text{SUE}^{ML}\times$ IO rank					0.172
					(2.00)
$G\times$ IO rank					−0.071
					(−1.54)
R^2 (%)	2.33	2.20	2.45	2.67	2.72
Observations	326,886	326,886	326,886	326,886	325,648

Column (3) gives $\beta_2/\beta_1 = 0.093/0.245 = 0.38$: in the average stock the market behaves as if it puts 38% relative weight on the consensus and 62% on the machine forecast. The market "largely, but not fully" corrects the predictable analyst bias. The interactions show where the correction is incomplete. Evaluating Equation 45.5 at the bottom and top size ranks, the analyst weight is $0.087/0.181 \approx 0.48$ for the smallest stocks and $(0.087 - 0.086)/(0.181 + 0.232) \approx 0$ for the largest. For institutional ownership it is $0.099/0.180 \approx 0.55$ at the bottom rank and $(0.099 - 0.071)/(0.180 + 0.172) \approx 0.08$ at the top. Large, institutionally held stocks are priced as if the analyst bias were already removed. Small stocks held by retail investors are priced as if about half of it were taken at face value.

The bias is a return predictor. If part of the predictable bias is not priced before the announcement, sorting stocks on G should predict returns. The paper forms quintiles on G each quarter and tracks the cumulative size-adjusted return of the high-minus-low portfolio from 40 trading days before to 252 days after the announcement. For small, low-IO stocks the spread is about 4.7% by the announcement day and keeps rising to about 10% by roughly 170 trading days (eight to nine months) before levelling off. For small, high-IO stocks it reaches about 7%. For large stocks it is 1–3% and mostly insignificant. Prices eventually catch up with the machine forecast, but slowly, and only where the bias is large and sophisticated investors are scarce.

Additional working-paper results. The working paper also shows that implied cost of capital computed from machine FY2 forecasts tracks cross-sectional expected returns better than ICC from analysts' forecasts and better than the characteristic-based measure of C. M. C. Lee, So, and Wang (2021), with the gain concentrated in small firms. It also defines an "analyst value added" (the accuracy gain from adding analyst inputs to the machine), which is larger for small firms and for firms with complex filings and does not decline over time.

45.3 Replication from the Authors' Data

The replication package contains the monthly GBRT-indirect forecasts (`DataShare.dta`, 279 MB, 1.16 million firm-months) together with realised EPS, the consensus, price, size, institutional ownership, the firm attributes, and size-adjusted three-day announcement returns. It also includes a summary of all 3,024 models (`Specification_Sumstats.csv`). The model training code is not included, so the replicable part is everything downstream of the forecasts: the accuracy measure, the size sorts, the time trend and the market tests. The cell below is our own implementation. It is not evaluated when the book is built because the data file is not distributed with the book. Download the package from the DOI above to run it.

```
import numpy as np
import pandas as pd
from linearmodels import PanelOLS

df = pd.read_stata("DataShare.dta")
df["fdate"] = df["Forecast_Date"].astype(int)
df["year"] = df["fdate"] // 100


def clustered_mean(y, firm, time):
    """Mean of y with standard errors two-way clustered by firm and time."""
    d = pd.DataFrame({"y": y.values, "firm": firm.values, "time":
 ↪  time.values}).dropna()
    d = d.set_index(["firm", "time"]).assign(const=1.0)
    r = PanelOLS(d["y"], d[["const"]]).fit(
        cov_type="clustered", cluster_entity=True, cluster_time=True)
    return r.params["const"], r.tstats["const"]


for h, k in [("FQ", 4), ("FY2", 1)]:            # k annualises quarterly
 ↪  errors
    d = df.dropna(subset=[f"Actual_{h}", f"Pred_{h}", f"Analyst_{h}",
 ↪  "prc"]).copy()
```

```
    afe = (k * (d[f"Actual_{h}"] - d[f"Analyst_{h}"]) / d["prc"]).clip(-1, 1)
    mle = (k * (d[f"Actual_{h}"] - d[f"Pred_{h}"]) / d["prc"]).clip(-1, 1)
    d["sup"] = 100 * (afe.abs() - mle.abs())
    print(h, "superiority = %.3f (t = %.2f)" % clustered_mean(d["sup"],
     ↪  d["permno"], d["year"]))
    d["szq"] = d.groupby("fdate")["size"].transform(
        lambda x: pd.qcut(x.rank(method="first"), 5, labels=False))
    for q in range(5):
        s = d[d["szq"] == q]
        print("   size Q%d: %.3f (t = %.2f)" % (q + 1,
         ↪  *clustered_mean(s["sup"], s["permno"], s["year"])))

# Market tests: size-adjusted 3-day announcement returns on SUE^ML and the
 ↪  bias G
e = df.dropna(subset=["Announcement_Ret_Size_Adj_FQ", "Actual_FQ", "Pred_FQ",
                      "Analyst_FQ", "prc"]).copy()
ann = pd.to_datetime(e["Announcement_Date_Johnson_FQ"])
e["tq"] = (ann.dt.year - 1960) * 4 + (ann.dt.month - 1) // 3
e["afe"] = 4 * (e["Actual_FQ"] - e["Analyst_FQ"]) / e["prc"]
e["mle"] = 4 * (e["Actual_FQ"] - e["Pred_FQ"]) / e["prc"]
for v in ["mle", "afe", "Announcement_Ret_Size_Adj_FQ"]:   # trim 1%/99% by
 ↪  quarter
    lo = e.groupby("tq")[v].transform(lambda x: x.quantile(0.01))
    hi = e.groupby("tq")[v].transform(lambda x: x.quantile(0.99))
    e[v + "_t"] = 100 * e[v].where(e[v].between(lo, hi))
e = e.dropna(subset=["mle_t", "afe_t", "Announcement_Ret_Size_Adj_FQ_t"])
e["bias_t"] = e["afe_t"] - e["mle_t"]                       # = G = 4 (ML -
 ↪  AF) / P
e = e.set_index(["permno", "tq"]).assign(const=1.0)


def erc(xs):
    return PanelOLS(e["Announcement_Ret_Size_Adj_FQ_t"], e[xs +
     ↪  ["const"]]).fit(
        cov_type="clustered", cluster_entity=True, cluster_time=True)


b = erc(["mle_t", "bias_t"]).params
g = erc(["afe_t", "bias_t"]).params
print("w_AF / (w_AF + w_ML) = %.3f" % (b["bias_t"] / b["mle_t"]))
print("w_AF / w_ML          = %.3f" % (-b["bias_t"] / g["bias_t"]))
```

Running this code on the package data (October 2026, about one minute) reproduces the published numbers:

Table 45.8: Replication of headline results. Both versions cluster by firm and forecast year; the small t-statistic differences for the superiority means come from the small-sample adjustments in Stata's `vcemway` and in `linearmodels`.

Statistic	Published	Recomputed
FQ ML Superiority, GBRT indirect	0.201 ($t = 8.71$)	0.201 ($t = 8.85$)
FY2 ML Superiority, GBRT indirect	0.601 ($t = 7.48$)	0.601 ($t = 7.60$)
FQ superiority, size Q1 / Q5	0.57 / 0.06 (figure)	0.569 / 0.062
FY2 superiority, size Q1 / Q5	1.71 / 0.06 (figure)	1.712 / 0.064
ERC on SUE^{ML}, col. (1)	0.241 (5.89)	0.241 (5.89)
ERC on SUE^{AF}, col. (2)	0.209 (5.91)	0.209 (5.91)
β_1, β_2, col. (3)	0.245, 0.093	0.245, 0.093
$w_{AF}/(w_{AF} + w_{ML})$	—	0.381
w_{AF}/w_{ML}	—	0.614
Share of 3,024 models with superiority ≤ 0	"80%"	79.1%

Two additional facts from the package are worth recording. The mean analyst error $(\text{EPS} - \text{AF})/P$ is -1.4% (FQ, annualised) and -2.8% (FY2), the familiar optimism that rises with horizon. The two weights in Equation 45.5 are also internally consistent: $0.381/(1 - 0.381) = 0.615 \approx 0.614$.

One further calculation is ours, not the paper's, and is motivated by Section 45.4.4 below. Regressing the trimmed realised analyst error $4(\text{EPS} - \text{AF})/P$ on the trimmed bias G in the same ERC sample gives a slope of $\hat{\lambda} = 0.87$ (two-way clustered $t = 12.8$), so the machine's bias estimate is somewhat too dispersed. Under the assumptions of Equation 45.6, the market's weight on the consensus relative to the rational forecast is then about $1-(1-0.38)/0.87 \approx 0.28$ rather than 0.38. Investors appear to correct even more of the predictable analyst error than the headline estimate suggests.

45.4 A Simulation That Shows Each Mechanism

The replication confirms the numbers, but it cannot show *why* they come out this way, because nobody knows the true analyst bias or the true market expectation in real data. In a simulation we do know them. The design below builds the paper's key ingredients into a firm-year panel and then checks whether the paper's procedures recover them:

- Earnings yields have a firm-specific level, persistent fundamentals with nonlinear effects, fat-tailed noise that is larger for small firms, and occasional large write-offs that skew the distribution to the left.
- Analysts know something the machine does not: a private signal about part of the earnings shock, more precise at shorter horizons.
- Analysts also make **predictable** errors: they are optimistic about small and low-priced firms (more so at longer horizons) and only partly correct their past errors, through an S-shaped function of the lagged error, mirroring the SHAP profile in Section 45.2.6.
- Each announcement is forecast three times (12, 6 and 1 month ahead), so targets overlap across rows, as with I/B/E/S monthly forecasts.

```
import time
import warnings
import numpy as np
import pandas as pd
import statsmodels.api as sm
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Lasso
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
from plotnine import (ggplot, aes, geom_col, geom_hline, coord_flip,
 ↪  facet_wrap,
                      labs, theme_minimal, theme, scale_fill_manual)

warnings.filterwarnings("ignore")
mlef_t0 = time.time()
rng = np.random.default_rng(2026)
N_FIRMS, N_YEARS = 500, 15
HORIZONS = {12: 0.3, 6: 0.6, 1: 0.9}      # months ahead -> share of eta
 ↪  analysts know

size = rng.normal(0, 1, N_FIRMS)           # persistent log-size (z-score)
level = rng.normal(0.06, 0.03, N_FIRMS)    # firm-specific earnings-yield
 ↪  level
x = rng.normal(0, 1, (N_FIRMS, 3))         # persistent fundamentals
lag_err = np.zeros(N_FIRMS)                # last realised analyst error
 ↪  (scaled)
lag_eps = level.copy()
frames = []
for yr in range(N_YEARS):
    size = 0.95 * size + rng.normal(0, 0.3, N_FIRMS)
    x = 0.8 * x + rng.normal(0, 0.6, x.shape)
    price = np.exp(2.5 + 0.6 * size + rng.normal(0, 0.4, N_FIRMS))
```

```
    mu = (level + 0.012 * x[:, 0] - 0.008 * np.maximum(x[:, 1], 0)
          + 0.006 * np.tanh(2 * x[:, 2]))                  # fundamental
           ↪ earnings yield
    vol = 0.02 * np.exp(-0.5 * size)                       # small firms are
 ↪ noisier
    eta = 0.7 * vol * rng.standard_t(4, N_FIRMS)           # shock analysts
 ↪ partly learn
    u = 0.7 * vol * rng.standard_t(4, N_FIRMS)             # shock nobody can
 ↪ forecast
    writeoff = -1.0 * (rng.random(N_FIRMS) < 0.06) * rng.exponential(0.15,
 ↪ N_FIRMS)
    eps = mu + eta + u + writeoff                          # realised EPS /
 ↪ price
    for h, info in HORIZONS.items():
        bias = (0.004 * h / 12 * (1 + np.exp(-size))       # optimism: small
 ↪ firms, long horizon
                + 0.010 * (price < 8)                      # optimism:
                 ↪ low-priced stocks
                - 0.010 * np.tanh(lag_err / 0.02))         # sticky, S-shaped
                 ↪ in past errors
        af = mu + info * eta + bias + rng.normal(0, 0.003, N_FIRMS)
        frames.append(pd.DataFrame({
            "firm": np.arange(N_FIRMS), "year": yr, "horizon": h,
            "size": size, "price": price, "x0": x[:, 0], "x1": x[:, 1], "x2":
             ↪ x[:, 2],
            "lag_eps": lag_eps, "lag_err": lag_err, "af": af, "eps": eps,
            # conditional median given analysts' information (write-off
             ↪ median is 0)
            "rational": mu + info * eta,
        }))
    lag_err = eps - af                                     # 1-month-ahead
 ↪ error is next year's lag
    lag_eps = eps
panel = pd.concat(frames, ignore_index=True)

afe = panel["eps"] - panel["af"]
print(f"{len(panel):,} firm-horizon-years; mean analyst error {100 *
 ↪ afe.mean():.2f}% of price, "
      f"median {100 * afe.median():.2f}%")
print("mean analyst error by horizon (% of price):",
      (100 * afe.groupby(panel["horizon"]).mean()).round(2).to_dict())
```

```
22,500 firm-horizon-years; mean analyst error -1.98% of price, median -0.85%
mean analyst error by horizon (% of price): {1: -1.48, 6: -1.95, 12: -2.51}
```

All variables are already scaled by price, so "EPS" in the code means the earnings yield and the superiority measure of Equation 45.1 needs no further scaling. The mean analyst error is negative (optimism) and grows with the horizon, as in I/B/E/S. The mean is more negative than the median because write-offs pull the mean down.

45.4.1 The Specification Grid

We estimate fifteen models: three ways of using analysts (Section 45.2.3) crossed with five algorithm–loss pairs. The linear MAE model is least-absolute-deviation regression, the lasso uses MSE with a fixed penalty on standardised features, and GBRT is scikit-learn's histogram gradient boosting with either loss. Training uses an expanding window and refits every three years, the cheap options that Table 45.3 says cost little accuracy.

```
BASE = ["size", "price", "x0", "x1", "x2", "lag_eps", "horizon"]
ANALYST = ["af", "lag_err"]
APPROACHES = ["Direct w/o analysts", "Direct w/ analysts", "Indirect"]
MODELS = [("Linear", "MSE"), ("Linear", "MAE"), ("Lasso", "MSE"),
          ("GBRT", "MSE"), ("GBRT", "MAE")]


def gbrt(loss, **kw):
    params = dict(max_iter=200, learning_rate=0.05, max_leaf_nodes=15,
                  min_samples_leaf=50, random_state=0)
    params.update(kw)
    return HistGradientBoostingRegressor(
        loss="squared_error" if loss == "MSE" else "absolute_error",
 ↪  **params)


def fit_predict(train, test, approach, algo, loss):
    """Out-of-sample EPS/P forecast for one specification."""
    feats = BASE + ([] if approach == "Direct w/o analysts" else ANALYST)
    y = train["eps"] - train["af"] if approach == "Indirect" else
 ↪  train["eps"]
    if algo == "Linear":
        X = sm.add_constant(train[feats].to_numpy())
        Xt = sm.add_constant(test[feats].to_numpy(), has_constant="add")
        if loss == "MSE":
            beta = np.linalg.lstsq(X, y.to_numpy(), rcond=None)[0]
```

```
        else:
            beta = sm.QuantReg(y.to_numpy(), X).fit(q=0.5,
 ↪  max_iter=2000).params
        pred = Xt @ beta
    elif algo == "Lasso":
        sc = StandardScaler().fit(train[feats])
        pred = Lasso(alpha=0.002).fit(sc.transform(train[feats]), y).predict(
            sc.transform(test[feats]))
    else:
        pred = gbrt(loss).fit(train[feats], y).predict(test[feats])
    return pred + (test["af"].to_numpy() if approach == "Indirect" else 0.0)


def superiority(eps, af, ml):
    """ML Superiority in percentage points, errors winsorised to [-1, 1]."""
    return 100 * ((eps - af).clip(-1, 1).abs() - (eps - ml).clip(-1,
     ↪  1).abs())


FIRST_TEST, REFIT_EVERY = 6, 3
specs = [(a, g, l) for a in APPROACHES for g, l in MODELS]
oos = panel[panel["year"] >= FIRST_TEST].copy()
pieces = {s: [] for s in specs}
for start in range(FIRST_TEST, N_YEARS, REFIT_EVERY):
    train = panel[panel["year"] < start]
    test = panel[(panel["year"] >= start) & (panel["year"] < start +
 ↪  REFIT_EVERY)]
    for s in specs:
        pieces[s].append(fit_predict(train, test, *s))

rows = []
for s in specs:
    col = " | ".join(s)
    oos[col] = np.concatenate(pieces[s])
    sup = superiority(oos["eps"], oos["af"], oos[col])
    rows.append({"approach": s[0], "model": f"{s[1]} ({s[2]})",
                 "superiority": sup.mean(), "se": sup.std() /
                  ↪  np.sqrt(len(sup))})
grid = pd.DataFrame(rows)
print(grid.pivot(index="model", columns="approach", values="superiority")
          .loc[[f"{g} ({l})" for g, l in MODELS],
 ↪  APPROACHES].round(3).to_string())
```

```
same = np.allclose(oos["Direct w/ analysts | Linear | MSE"], oos["Indirect |
 ↪  Linear | MSE"])
print(f"\nOLS direct-with-analysts identical to OLS indirect: {same}")
```

```
approach      Direct w/o analysts  Direct w/ analysts  Indirect
model
Linear (MSE)               -1.032               0.104     0.104
Linear (MAE)               -0.959               0.260     0.260
Lasso (MSE)                -1.050               0.085     0.111
GBRT (MSE)                 -0.933               0.046     0.044
GBRT (MAE)                 -0.752               0.323     0.343

OLS direct-with-analysts identical to OLS indirect: True
```

```
plot_df = grid.assign(
    approach=pd.Categorical(grid["approach"], APPROACHES),
    model=pd.Categorical(grid["model"], [f"{g} ({l})" for g, l in
 ↪  MODELS][::-1]),
    loss=np.where(grid["model"].str.contains("MAE"), "MAE loss", "MSE loss"))
(ggplot(plot_df, aes("model", "superiority", fill="loss"))
 + geom_col()
 + geom_hline(yintercept=0, linetype="dashed")
 + coord_flip()
 + facet_wrap("~approach", ncol=3)
 + scale_fill_manual(values=["#2a7f62", "#9a9a9a"])
 + labs(x="", y="ML Superiority (pp)", fill="")
 + theme_minimal()
 + theme(figure_size=(9, 3.6), legend_position="bottom"))
```

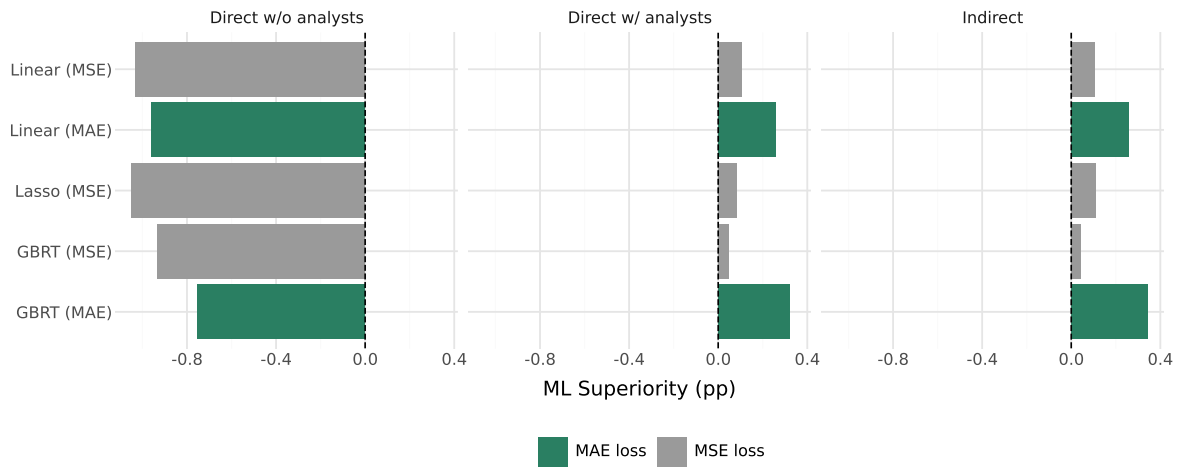


Figure 45.1: Out-of-sample ML Superiority (percentage points of price) of 15 simulated specifications. Positive values beat the analyst consensus. The dashed line marks zero.

The grid reproduces the paper's ranking, and because the data-generating process is known we can say why each result occurs.

- **No model without analyst inputs beats analysts.** Analysts observe part of the earnings shock (`eta`), and no function of the public features can recover it. This is the paper's FQ result in Table 45.5, and it is the reason the machine's edge is a *correction* to the analyst.
- **MSE loses to MAE everywhere.** The 6% write-off probability pulls the conditional mean of EPS/P below the conditional median. An MSE model forecasts the mean, the superiority yardstick rewards the median, and so MSE forecasts are too pessimistic for the 94% of firms without a write-off. This is the 6.3-point loss-function gap of Table 45.3 in miniature.
- **OLS direct-with-analysts equals OLS indirect exactly**, as shown in Section 45.2.3. The indirect approach matters only for models that shrink (lasso) or approximate (trees), and in the paper it is exactly for those algorithms that the switch to indirect produces the largest improvement (random forests in Table 45.5).
- **Indirect GBRT with MAE loss is the best model**, because it is the only one that captures both the S-shaped persistence in analyst errors and the price threshold in analyst optimism while targeting the median.

45.4.2 Why Time-Series Cross-Validation

The specification above fixes the GBRT hyperparameters. In practice they are tuned, and the paper finds that the way the validation sample is formed matters more than how often tuning is repeated. In our panel each announcement appears in three rows (one per horizon)

with the same target and almost the same features. A shuffled k-fold split puts some of those rows in training and the others in validation, so the validation error rewards models that memorise firm-years. A time-series split validates only on years after the training years, as a real forecaster must.

```
cv_train = panel[panel["year"] < 8].reset_index(drop=True)
cv_test = panel[panel["year"] >= 8]
FEATS = BASE + ANALYST
complexity = {"deep (leaf 3, 63 leaves)": dict(min_samples_leaf=3,
 ↪  max_leaf_nodes=63),
              "medium (leaf 30, 31 leaves)": dict(min_samples_leaf=30,
               ↪  max_leaf_nodes=31),
              "shallow (leaf 300, 7 leaves)": dict(min_samples_leaf=300,
               ↪  max_leaf_nodes=7)}


def mk(p):
    return gbrt("MAE", max_iter=120, learning_rate=0.1, early_stopping=False,
     ↪  **p)


def cv_mae(p, scheme):
    if scheme == "Shuffled 5-fold":
        splits = KFold(5, shuffle=True, random_state=0).split(cv_train)
    else:   # expanding training years, validate on the next year
        splits = [(np.flatnonzero(cv_train["year"] < v),
 ↪  np.flatnonzero(cv_train["year"] == v))
                  for v in (5, 6, 7)]
    errs = []
    for tr, va in splits:
        a, b = cv_train.iloc[tr], cv_train.iloc[va]
        g = mk(p).fit(a[FEATS], a["eps"] - a["af"]).predict(b[FEATS]) +
 ↪  b["af"]
        errs.append(np.mean(np.abs(b["eps"] - g)))
    return 100 * np.mean(errs)


cv_rows = []
for name, p in complexity.items():
    fitted = mk(p).fit(cv_train[FEATS], cv_train["eps"] - cv_train["af"])
    pred = fitted.predict(cv_test[FEATS]) + cv_test["af"]
    cv_rows.append({"model": name,
                    "k-fold CV MAE": cv_mae(p, "Shuffled 5-fold"),
```

```
                    "time-series CV MAE": cv_mae(p, "Time-series"),
                    "true OOS superiority": superiority(cv_test["eps"],
                     ↪  cv_test["af"], pred).mean()})
cv_table = pd.DataFrame(cv_rows).set_index("model")
print(cv_table.round(3).to_string())
print("\nk-fold picks:      ", cv_table["k-fold CV MAE"].idxmin())
print("time-series picks: ", cv_table["time-series CV MAE"].idxmin())
print("best out of sample:", cv_table["true OOS superiority"].idxmax())
```

```
                             k-fold CV MAE  time-series CV MAE  true OOS superiority
model
deep (leaf 3, 63 leaves)             2.223               2.619                 0.299
medium (leaf 30, 31 leaves)          2.395               2.547                 0.329
shallow (leaf 300, 7 leaves)         2.580               2.528                 0.346

k-fold picks:       deep (leaf 3, 63 leaves)
time-series picks:  shallow (leaf 300, 7 leaves)
best out of sample: shallow (leaf 300, 7 leaves)
```

Shuffled k-fold reports a much lower error for the deepest model and picks it. Time-series validation reports similar errors for all three, picks the conservative model, and that model is in fact the most accurate out of sample. The leakage is invisible in the k-fold numbers, which is why the paper finds that tuning on a time-series split is worth about 1.5 percentage points of superiority on average.

45.4.3 Size and Horizon

```
best = "Indirect | GBRT | MAE"
oos["sup_best"] = superiority(oos["eps"], oos["af"], oos[best])
oos["size_q"] = oos.groupby("year")["size"].transform(
    lambda s: pd.qcut(s, 5, labels=["Q1 small", "Q2", "Q3", "Q4", "Q5
     ↪  large"]))
print("By size quintile (pp):")
print(oos.groupby("size_q",
 ↪  observed=True)["sup_best"].mean().round(3).to_string())
print("\nBy months to announcement (pp):")
print(oos.groupby("horizon")["sup_best"].mean().round(3).to_string())
```

```
By size quintile (pp):
size_q
Q1 small    0.752
Q2          0.345
Q3          0.292
Q4          0.189
Q5 large    0.136

By months to announcement (pp):
horizon
1     0.172
6     0.329
12    0.527
```

Both of the paper's patterns appear, for the reasons built into the simulation. Analyst optimism and earnings noise are larger for small firms, so there is more predictable error to remove. At longer horizons the analysts' private signal is weaker and their horizon-dependent optimism is larger, so the machine's correction is worth more. Neither pattern requires the machine to know anything analysts do not know.

45.4.4 What the Market-Expectations Regression Identifies

Finally, we generate announcement returns from a market whose expectation puts weight $w_{AF} = 0.38$ on the consensus, matching Table 45.7, and check whether Equation 45.5 recovers it. We do this twice. In case A, the remaining weight is on *our estimated machine forecast*, which is the assumption in Equation 45.3. In case B, the remaining weight is on the *true* conditional median given analysts' information, which the machine only approximates.

```
W_AF, ERC_TRUE = 0.38, 1.5
ev = oos[oos["horizon"] == 1].copy()       # forecasts one month before the
 ↪  announcement
ev["sue_ml"] = ev["eps"] - ev[best]
ev["sue_af"] = ev["eps"] - ev["af"]
ev["gap"] = ev[best] - ev["af"]            # machine-predicted analyst bias G


def implied_weights(other):
    market = W_AF * ev["af"] + (1 - W_AF) * other
    r = ERC_TRUE * (ev["eps"] - market) + rng.normal(0, 0.01, len(ev))
    f1 = sm.OLS(r, sm.add_constant(ev[["sue_ml", "gap"]])).fit()
    f2 = sm.OLS(r, sm.add_constant(ev[["sue_af", "gap"]])).fit()
```

```
    return {"beta1": f1.params["sue_ml"], "beta2": f1.params["gap"],
            "w_AF/(w_AF+w_ML)": f1.params["gap"] / f1.params["sue_ml"],
            "w_AF/w_ML": -f1.params["gap"] / f2.params["gap"]}


erc_tab = pd.DataFrame({
    "truth": {"beta1": ERC_TRUE, "beta2": ERC_TRUE * W_AF,
              "w_AF/(w_AF+w_ML)": W_AF, "w_AF/w_ML": W_AF / (1 - W_AF)},
    "A: market mixes AF and the machine": implied_weights(ev[best]),
    "B: market mixes AF and the true median":
     ↪  implied_weights(ev["rational"]),
})
print(erc_tab.round(3).to_string())

# Case B diagnostics: slope of the true bias on G, and its feasible estimate
lam_true = sm.OLS(ev["rational"] - ev["af"],
 ↪  sm.add_constant(ev["gap"])).fit().params["gap"]
lam_hat = sm.OLS(ev["eps"] - ev["af"],
 ↪  sm.add_constant(ev["gap"])).fit().params["gap"]
ratio_b = erc_tab.loc["w_AF/(w_AF+w_ML)", "B: market mixes AF and the true
 ↪  median"]
print(f"\nlambda (true bias on G): {lam_true:.3f}; predicted beta2/beta1 = "
      f"1 - (1 - {W_AF}) * lambda = {1 - (1 - W_AF) * lam_true:.3f}")
print(f"lambda-hat (realised analyst error on G): {lam_hat:.3f}; corrected
 ↪  weight = "
      f"1 - (1 - {ratio_b:.3f}) / lambda-hat = {1 - (1 - ratio_b) /
       ↪  lam_hat:.3f}")
print(f"\nsimulation section run time: {time.time() - mlef_t0:.0f} s")
```

```
                  truth  A: market mixes AF and the machine  B: market mixes AF and the true
beta1             1.500                                1.498
beta2             0.570                                0.592
w_AF/(w_AF+w_ML)  0.380                                0.396
w_AF/w_ML         0.613                                0.654

lambda (true bias on G): 0.734; predicted beta2/beta1 = 1 - (1 - 0.38) * lambda = 0.545
lambda-hat (realised analyst error on G): 0.794; corrected weight = 1 - (1 - 0.538) / lambda-

simulation section run time: 44 s
```

Case A confirms the algebra: when the market's expectation really is a mixture of the consensus and the machine forecast, the regression recovers the weight. Case B is the caveat. Now

the market mixes the consensus with the *rational* forecast, and the machine only estimates it. Write $B = (\text{rational} - \text{AF})/P$ for the true correction, so that the market's surprise is $\text{SUE}^{ML} + G - (1 - w_{AF})B$. If the projection of B on the machine's estimate G has slope λ, the regression delivers approximately

$$\frac{\beta_2}{\beta_1} \approx 1 - (1 - w_{AF})\,\lambda. \tag{45.6}$$

With $\lambda = 1$ (the machine's bias estimate is calibrated) the weight is recovered. Out-of-sample forecasts are typically too dispersed ($\lambda < 1$), and the estimated analyst weight is then biased *upward*: a market that is more rational than the machine looks less rational in the regression. The printed λ and the predicted ratio match case B closely.

λ is not observable, but it can be estimated. The realised analyst error $(\text{EPS} - \text{AF})/P$ equals B plus a shock that is unpredictable at the forecast date, so regressing it on G estimates λ. Inverting Equation 45.6 then gives a corrected weight, $\hat{w}_{AF} = 1 - (1 - \hat{\beta}_2/\hat{\beta}_1)/\hat{\lambda}$. In the simulation the correction removes most of the bias. Fat-tailed earnings make $\hat{\lambda}$ noisy, so report it alongside the uncorrected ratio rather than in place of it.

45.5 When (Not) to Use a Machine Forecast: Practical Rules

Combining the paper's results with the simulation gives a short decision list for empirical work.

1. **Large firms, next-quarter earnings:** use the consensus. The best machine improves accuracy by well under a cent per $10 of price, and the market prices large firms as if the analyst bias were already removed.
2. **Small firms or horizons beyond about six months** (FY2, long-term growth, ICC inputs): use an indirect machine forecast. The accuracy gains are economically large, and implied cost of capital improves most for exactly these firms.
3. **Measuring earnings news for the market** (SUE, ERC, post-announcement drift designs in Chapter 44): report both SUE^{AF} and SUE^{ML}, and add the bias G as a separate regressor, with the $\hat{\lambda}$ correction of Equation 45.6. A significant coefficient on G means the "surprise" contains a part that was predictable in advance, and its decay over the following months is a return predictor in its own right.
4. **Building the machine:** use MAE (or another median-targeting) loss whenever accuracy is judged by absolute errors, the indirect approach, analyst inputs, and time-series validation. Choose the window, refit and tuning frequency to save computation.
5. **Report the specification.** Since 79% of reasonable-looking models lose to analysts, a claim that "machine learning beats analysts" is uninformative without the nine choices of Table 45.2.

45.6 Vietnam: Research Design and DataCore Implementation

45.6.1 What Changes in Vietnam

The paper's conclusions depend on two facts about the US: analyst coverage is broad, and the consensus is a strong baseline. Neither holds uniformly in Vietnam.

1. **Coverage is concentrated.** Broker research (from SSI, VNDirect, VCSC, HSC, MBS and others) covers VN30 and larger VNAllShare constituents, often with one to five forecasts, and most of the HOSE, HNX and UPCoM universe has no forecast at all (Chapter 44). The paper's thin-coverage evidence is the relevant benchmark: with one to three analysts, a machine *without* analyst inputs does not beat them until the forecast is made eight to ten months or more ahead. For covered Vietnamese firms, therefore, use the indirect approach. For uncovered firms there is no consensus to correct, and the right comparison is a direct model against the seasonal random walk of Chapter 44 or the cross-sectional model of Hou, Dijk, and Zhang (2012), not against analysts.
2. **The target needs care.** Vietnamese firms publish self-prepared quarterly statements (within 20 days of quarter end, 45 days for parents with consolidated statements), and audited annual statements within 90 days. Audit adjustments to annual profit can be large, and their sign is partly predictable from the same variables that predict analyst error. Forecast the *first-reported* number when studying announcement reactions, and the *audited* number when studying valuation, and never train on figures that were revised after the forecast date.
3. **Price scaling and the low-price bias.** Par value is VND 10,000, and many listed shares trade below par. Price is among the two or three most important predictors of analyst error in the paper, and the price distribution in Vietnam is very different, so include both the price level and a below-par indicator as features. Adjust EPS for bonus shares and stock dividends before scaling (Chapter 44).
4. **Short history.** A rolling ten-year window is not available for most covered firms. Use an expanding window starting in the early 2010s, pool all horizons (one row per firm-month-horizon, with months to period end as a feature), and refit yearly. The paper shows the window and refit choices barely matter for accuracy.
5. **Market tests under price limits.** Daily limits ($\pm 7\%$ HOSE, $\pm 10\%$ HNX, $\pm 15\%$ UPCoM) truncate three-day reactions to large surprises (Section 85.8). Use a window that extends until the first day the stock does not close at the limit, or report $[-1, +5]$ alongside $[-1, +1]$.
6. **Investor sophistication.** The IO interaction in Table 45.7 has a natural Vietnamese counterpart: foreign ownership as a share of the foreign-ownership limit, together with state ownership. The paper's prediction is that the market weight on analysts, β_2/β_1, is lower for stocks with high foreign ownership and higher for retail-dominated stocks.

45.6.2 Implementation

The cell below implements the indirect GBRT forecast for covered Vietnamese firms, the superiority evaluation, and the ERC weight test. DataCore method and field names follow the conventions used elsewhere in the book. Check them against your DataCore schema, because broker-consensus coverage depends on the subscription. The code was test-run on synthetic data with the same columns before being included here.

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from sklearn.ensemble import HistGradientBoostingRegressor
from datacore import DataCoreClient

client = DataCoreClient()

# 1. Inputs ------------------------------------------------------------------
# Monthly broker consensus: one row per ticker x forecast month x fiscal
 ↪  period
consensus = client.get_analyst_consensus(
    exchanges=["HOSE", "HNX", "UPCOM"], start_date="2010-01-01",
 ↪  end_date="2025-12-31",
    fields=["ticker", "forecast_month", "fiscal_period_end", "period_type",
 ↪  # 'FY1','FY2','Q1'
            "median_eps", "n_analysts"])
actuals = client.get_fundamentals(          # first-reported EPS, adjusted
 ↪  for bonus shares
    exchanges=["HOSE", "HNX", "UPCOM"], start_date="2008-01-01",
 ↪  end_date="2025-12-31",
    fields=["ticker", "fiscal_period_end", "period_type", "eps_adj",
 ↪  "announcement_date"])
monthly = client.get_monthly_returns(
    exchanges=["HOSE", "HNX", "UPCOM"], start_date="2009-01-01",
 ↪  end_date="2025-12-31",
    fields=["ticker", "month", "close", "ret", "market_cap", "mom6m",
            "foreign_own_pct_of_limit", "state_own_pct"])
ratios = client.get_financial_ratios(       # lagged to their public release
 ↪  month
    exchanges=["HOSE", "HNX", "UPCOM"], start_date="2009-01-01",
 ↪  end_date="2025-12-31",
    fields=["ticker", "month", "roa", "roe", "gross_margin", "accruals",
            "leverage", "sales_growth", "book_to_market"])
```

```
# 2. Forecast panel ----------------------------------------------------------
def build_panel(consensus, actuals, monthly, ratios):
    """One row per ticker x forecast month x target period, with features
     ↪  known at month end."""
    c = consensus.rename(columns={"median_eps": "af"}).copy()
    c["forecast_month"] =
 ↪  pd.to_datetime(c["forecast_month"]).dt.to_period("M")
    a = actuals.rename(columns={"eps_adj": "eps"}).copy()
    a["ann_month"] = pd.to_datetime(a["announcement_date"]).dt.to_period("M")
    p = c.merge(a[["ticker", "fiscal_period_end", "period_type", "eps",
 ↪  "ann_month"]],
                on=["ticker", "fiscal_period_end", "period_type"],
 ↪  how="inner")
    p = p[p["ann_month"] > p["forecast_month"]]          # target not yet
 ↪  announced
    m =
 ↪  monthly.assign(month=pd.to_datetime(monthly["month"]).dt.to_period("M"))
    r =
 ↪  ratios.assign(month=pd.to_datetime(ratios["month"]).dt.to_period("M"))
    p = (p.merge(m, left_on=["ticker", "forecast_month"], right_on=["ticker",
 ↪  "month"])
          .merge(r, on=["ticker", "month"], how="left"))
    p["dist"] = (pd.to_datetime(p["fiscal_period_end"]).dt.to_period("M")
                 - p["forecast_month"]).apply(lambda d: d.n)
    p["log_size"] = np.log(p["market_cap"])
    p["below_par"] = (p["close"] < 10_000).astype(float)
    # Most recent realised analyst error for the same period type, known at
     ↪  forecast month
    # (error of the last consensus before each announcement; the paper
     ↪  matches the distance)
    p = p.sort_values(["ticker", "period_type", "forecast_month"])
    done = (p[["ticker", "period_type", "fiscal_period_end", "ann_month",
 ↪  "eps", "af", "close"]]
            .groupby(["ticker", "period_type",
 ↪  "fiscal_period_end"]).last().reset_index())
    done["realised_err"] = (done["eps"] - done["af"]) / done["close"]
    p = pd.merge_asof(p.sort_values("forecast_month").assign(_t=lambda d:
 ↪  d["forecast_month"].dt.start_time),
                      done[["ticker", "period_type", "ann_month",
 ↪  "realised_err"]]
                      .assign(_t=lambda d:
 ↪  d["ann_month"].dt.start_time).sort_values("_t")
```

```
                      .drop(columns="ann_month"),
                      on="_t", by=["ticker", "period_type"],
    allow_exact_matches=True)
    p = p.rename(columns={"realised_err": "lag_err"}).drop(columns="_t")
    p = p.sort_values(["ticker", "period_type", "fiscal_period_end",
    "forecast_month"])
    p["rev3"] = (p.groupby(["ticker", "period_type",
    "fiscal_period_end"])["af"].diff(3)
                 / p["close"])                              # three-month
                    consensus revision
    for col in ["af", "eps"]:
        p[col + "_p"] = p[col] / p["close"]                # scale by price
    return p


FEATURES = ["af_p", "lag_err", "rev3", "dist", "close", "below_par",
    "log_size", "ret",
            "mom6m", "n_analysts", "foreign_own_pct_of_limit",
             "state_own_pct",
            "roa", "roe", "gross_margin", "accruals", "leverage",
             "sales_growth", "book_to_market"]


# 3. Indirect GBRT, expanding window, yearly refit, time-series tuning
    ----------
def tune_on_time_series(train, grid, n_val_years=2):
    """Pick hyperparameters by validating on the last n_val_years of
     announcements."""
    cut = train["ann_month"].max().year - n_val_years
    fit, val = train[train["ann_month"].dt.year <= cut],
    train[train["ann_month"].dt.year > cut]
    scores = {}
    for i, g in enumerate(grid):
        model = HistGradientBoostingRegressor(loss="absolute_error",
    random_state=0, **g)
        model.fit(fit[FEATURES], fit["eps_p"] - fit["af_p"])
        scores[i] = np.mean(np.abs(val["eps_p"] - val["af_p"] -
    model.predict(val[FEATURES])))
    return grid[min(scores, key=scores.get)]


GRID = [dict(max_iter=200, learning_rate=0.05, max_leaf_nodes=7,
    min_samples_leaf=200),
```

```
        dict(max_iter=300, learning_rate=0.05, max_leaf_nodes=15,
         ↪  min_samples_leaf=100),
        dict(max_iter=300, learning_rate=0.05, max_leaf_nodes=31,
         ↪  min_samples_leaf=50)]


def indirect_forecasts(panel, first_test_year=2016):
    out = []
    for yr in range(first_test_year, panel["forecast_month"].dt.year.max() +
     ↪  1):
        start = pd.Period(f"{yr}-01", "M")
        train = panel[panel["ann_month"] < start]         # targets announced
 ↪  before the test year
        test = panel[panel["forecast_month"].dt.year == yr].copy()
        if train.empty or test.empty:
            continue
        params = tune_on_time_series(train, GRID)
        model = HistGradientBoostingRegressor(loss="absolute_error",
 ↪  random_state=0, **params)
        model.fit(train[FEATURES], train["eps_p"] - train["af_p"])
        test["ml_p"] = test["af_p"] + model.predict(test[FEATURES])
        out.append(test)
    return pd.concat(out, ignore_index=True)


panel = build_panel(consensus, actuals, monthly, ratios)
fc = indirect_forecasts(panel)

# 4. Accuracy: ML Superiority by period type and size quintile
 ↪  ---------------
k = np.where(fc["period_type"].str.startswith("Q"), 4, 1)          #
 ↪  annualise quarters
fc["sup"] = 100 * ((k * (fc["eps_p"] - fc["af_p"])).clip(-1, 1).abs()
                   - (k * (fc["eps_p"] - fc["ml_p"])).clip(-1, 1).abs())
fc["size_q"] = fc.groupby("forecast_month")["log_size"].transform(
    lambda s: pd.qcut(s.rank(method="first"), 5, labels=False) + 1)
print(fc.pivot_table(index="size_q", columns="period_type", values="sup",
 ↪  aggfunc="mean").round(3))

# 5. Market expectations: ERC weights, with foreign-ownership interaction
 ↪  -----------
car = client.get_event_returns(              # size-adjusted CAR, window
 ↪  extended past limit hits
```

```
    events=fc[["ticker", "ann_month"]].drop_duplicates(),
 ↪  window="limit_adjusted_-1_+1")
ev = (fc[fc["period_type"] == "Q1"]
        .sort_values("forecast_month").groupby(["ticker",
 ↪  "fiscal_period_end"]).last()
        .reset_index().merge(car, on=["ticker", "ann_month"]))
ev["sue_ml"] = 100 * 4 * (ev["eps_p"] - ev["ml_p"])
ev["gap"] = 100 * 4 * (ev["ml_p"] - ev["af_p"])
ev["fo_rank"] =
 ↪  ev.groupby("ann_month")["foreign_own_pct_of_limit"].rank(pct=True)
base = smf.ols("car ~ sue_ml + gap", data=ev).fit(
    cov_type="cluster", cov_kwds={"groups":
 ↪  ev["ticker"].astype("category").cat.codes})
inter = smf.ols("car ~ sue_ml * fo_rank + gap * fo_rank", data=ev).fit(
    cov_type="cluster", cov_kwds={"groups":
 ↪  ev["ticker"].astype("category").cat.codes})
ratio = base.params["gap"] / base.params["sue_ml"]
ev["afe"] = 100 * 4 * (ev["eps_p"] - ev["af_p"])
lam_hat = smf.ols("afe ~ gap", data=ev).fit().params["gap"]
print("market weight on analysts:", round(ratio, 3), "| lambda-hat:",
 ↪  round(lam_hat, 3),
      "| corrected:", round(1 - (1 - ratio) / lam_hat, 3))
print(inter.params.round(3))
```

The output to look for has four parts:

- The size-quintile table should show the machine's advantage concentrated in the smallest covered firms and the FY2 horizon. If superiority is negative even in the smallest quintile, check the target alignment (first-reported versus audited) before blaming the model.
- The weight β_2/β_1 is the share of the predictable analyst error the market takes at face value. Following Section 45.4.4, report it together with $\hat{\lambda}$ (the slope of realised analyst errors on `gap`) and the corrected weight $1-(1-\beta_2/\beta_1)/\hat{\lambda}$.
- The coefficient on `gap:fo_rank` tests whether foreign-owned stocks are priced closer to the machine forecast.
- Sorting covered stocks into quintiles of `gap` and tracking post-announcement returns, as in Section 45.2.8, tests whether the uncorrected bias is a drift predictor in Vietnam. With price limits and a retail-dominated investor base, the paper's small, low-IO result is the more relevant benchmark.

45.7 Summary

J. L. Campbell et al. (2026) move the debate from "can machines beat analysts?" to "which machine, when, and for what purpose?" Most machine-learning earnings forecasts in a 3,024-model grid lose to the analyst consensus. The ones that win use an absolute-error loss, analyst inputs, the indirect approach of correcting the consensus, and time-series validation, and they win by removing *predictable* analyst error rather than by finding new information. The improvement is economically trivial for large firms and next-quarter earnings and large for small firms and longer horizons. Prices behave as if investors correct most of the predictable error, fully for large and institutionally held stocks and only about half for small retail-held stocks, where the remainder is priced over the following nine months. The simulation in this chapter reproduces each of these patterns from known primitives and adds one caution: the market's estimated weight on analysts is biased upward when the machine's bias estimate is too dispersed. A simple correction, applied to the authors' data, lowers the weight from 0.38 to about 0.28. For Vietnam, the paper's thin-coverage and small-firm evidence is the relevant guide: correct the consensus where it exists, benchmark against time-series models where it does not, and expect the largest gains, and the largest mispricing, among small, retail-dominated stocks.

46 Measuring Divergence of Investor Opinion

A foundational question in financial economics concerns how differences in investor beliefs affect asset prices and trading activity. In markets where investors hold heterogeneous expectations about a firm's future cash flows, the aggregation of these divergent views into a single market price becomes a non-trivial exercise with profound implications for asset valuation, return predictability, and market efficiency. The concept of **divergence of investor opinion** (hereafter DIVOP) has emerged as a central construct in both the accounting and finance literatures, serving as a lens through which researchers examine the information environment of firms, the dynamics of uncertainty resolution, and the nature of market reactions to news.

The theoretical foundations of the DIVOP literature trace back to E. M. Miller (1977), who proposed that when investors disagree about the value of a security and short-sale constraints prevent pessimistic investors from fully expressing their views, the market price will reflect the valuation of the most optimistic investors. This leads to systematic overpricing that is increasing in the degree of opinion divergence. The overpricing persists until information events, such as earnings announcements, reduce disagreement and prices converge toward fundamental values (Berkman et al. 2009). Varian (1985) offers an alternative perspective in which divergence of opinion represents an additional risk factor, leading to *higher* rather than lower expected returns, creating a theoretical tension that has motivated extensive empirical investigation.

The empirical literature on DIVOP has expanded considerably since these seminal contributions. Researchers have documented that divergence of opinion helps explain a range of asset pricing anomalies, including post-earnings announcement drift (Garfinkel and Sokobin 2006; K. L. Anderson, Harris, and So 2007), the cross-sectional return difference between value and growth stocks (Doukas, Kim, and Pantzalis 2004), short- and long-run post-IPO returns (Houge et al. 2001), pre- and post-acquisition stock returns (Alexandridis, Antoniou, and Petmezas 2007), takeover premia (Chatterjee, John, and Yan 2012), and the broad cross-section of stock returns (Diether, Malloy, and Scherbina 2002; Doukas, Kim, and Pantzalis 2006). The explanatory power of DIVOP has been demonstrated using a rich set of empirical proxies, ranging from analyst forecast dispersion and abnormal trading volume to bid-ask spreads and idiosyncratic volatility.

Despite the maturity of the DIVOP literature in developed markets, particularly the United States, its application to emerging markets remains remarkably thin. This gap is especially notable given that the theoretical conditions under which divergence of opinion matters most (namely, binding short-sale constraints, information asymmetry, and heterogeneous investor

sophistication) are arguably *more* prevalent in emerging markets than in their developed counterparts. The Vietnamese equity market presents a compelling laboratory for studying investor disagreement. The market is characterized by several features that amplify the relevance of the DIVOP framework:

1. **Binding short-sale constraints.** Short selling was not permitted in Vietnam until January 2025, and even after its introduction, the mechanism remains restricted to a limited set of securities with significant regulatory constraints on execution. This closely mirrors the theoretical setting of E. M. Miller (1977), where pessimistic investors are unable to fully express their views through short positions.

2. **Dominance of retail investors.** Individual investors account for approximately 80-85% of daily trading volume on HOSE and HNX, compared to roughly 25% in the United States. Retail investors are more susceptible to behavioral biases, sentiment-driven trading, and information processing limitations that naturally give rise to heterogeneous beliefs (Phan et al. 2023).

3. **Information asymmetry and transparency challenges.** Despite improvements in disclosure standards, Vietnam's regulatory framework for corporate reporting remains less stringent than those in developed markets. Selective disclosure, delayed filing of financial statements, and limited enforcement of insider trading regulations create an environment in which investors operate with substantially different information sets (X. V. Vo and Phan 2017).

4. **Foreign ownership limits.** Caps on foreign ownership (currently 49% for most sectors, with exceptions) create a segmented market where domestic and foreign investors may hold systematically different views about firm value, amplifying the divergence of opinion.

5. **Thin analyst coverage.** Whereas a typical S&P 500 firm is followed by 15-25 sell-side analysts, coverage of Vietnamese equities is concentrated among a relatively small number of domestic brokerages and a handful of international research houses. This limits the informativeness of traditional analyst-based DIVOP measures and necessitates greater reliance on market-based proxies.

This chapter provides a methodology for constructing multiple proxies for divergence of investor opinion adapted to the institutional characteristics of the Vietnamese market. We draw on the methodological frameworks established by Garfinkel (2009) and Diether, Malloy, and Scherbina (2002), while introducing modifications that account for the microstructure of Vietnamese exchanges, the $T + 2$ settlement cycle, the absence (until recently) of short selling, and the availability of data through domestic financial platforms. Specifically, we construct and analyze the following DIVOP proxies:

- **Unexplained Volume (DTO):** Market-adjusted turnover detrended by its rolling median, capturing abnormal trading activity attributable to disagreement after controlling for liquidity and market-wide effects.

- **Standardized Unexplained Volume (SUV):** A regression-based measure that explicitly controls for the informedness and liquidity components of volume by modeling turnover as a function of signed returns.
- **Stock Return Volatility (VOLATILITY):** The standard deviation of daily returns over a rolling estimation window, serving as a proxy for the dispersion of investor valuations.
- **Bid-Ask Spread (BASPREAD):** The proportional quoted spread, reflecting the adverse selection component associated with heterogeneous information among market participants.
- **Analyst Forecast Dispersion (DISP):** The cross-sectional standard deviation of individual analyst earnings forecasts, directly measuring disagreement among informed market participants.
- **Idiosyncratic Volatility (IVOL):** The residual volatility from a factor model regression, isolating the firm-specific component of return variation that reflects divergent investor interpretations of firm-level information.
- **Amihud Illiquidity (ILLIQ):** The price impact ratio proposed by Amihud (2002), which captures the information asymmetry dimension of disagreement through the price response to order flow.

For each proxy, we describe the theoretical motivation, the data requirements, the construction methodology adapted for Vietnamese data, the empirical properties observed in the Vietnamese cross-section, and the practical considerations that researchers should bear in mind when employing these measures. We pay particular attention to issues that are specific to emerging markets, including thin trading, corporate action adjustments, exchange-specific microstructure effects, and the interplay between foreign ownership constraints and measures of investor disagreement.

---

47 Theoretical Framework

47.1 The Miller (1977) Overpricing Hypothesis

The canonical model of divergence of opinion and asset pricing begins with E. M. Miller (1977). Miller's central insight is simple: in a market where investors hold heterogeneous beliefs about the future payoffs of a risky asset and short-sale constraints prevent some investors from acting on their pessimistic views, the equilibrium price will be set by the subset of investors who are most optimistic about the asset's value. The severity of overpricing is increasing in both the degree of opinion divergence and the stringency of short-sale constraints. Formally, if investor i assigns a valuation V_i to a security, the market price P satisfies:

$$P = E[V_i \mid V_i \geq V^*]$$

where V^* is the marginal investor's valuation, which exceeds the unconditional mean valuation $E[V_i]$ whenever short-sale constraints bind for some investors. The degree of overpricing is:

$$\text{Overpricing} = P - E[V_i] = E[V_i \mid V_i \geq V^*] - E[V_i]$$

which is positive and increasing in the dispersion of the distribution of V_i (i.e., divergence of opinion) and in V^* (i.e., the severity of short-sale constraints).

Miller's model generates several testable predictions:

- **Cross-sectional prediction:** Stocks with **higher divergence of opinion should have *lower* subsequent returns** as prices gradually correct toward fundamental values.
- **Time-series prediction:** Information events that reduce disagreement (e.g., earnings announcements) should be associated with negative abnormal returns for high-DIVOP stocks, as the "optimism premium" dissipates.
- **Interaction prediction:** The overpricing effect should be strongest among stocks that simultaneously exhibit high divergence of opinion *and* binding short-sale constraints.

47.2 Alternative Theoretical Perspectives

Varian (1985) proposes an alternative framework in which divergence of opinion acts as a risk factor. If investors are risk-averse and disagreement represents genuine uncertainty about future payoffs, then **higher dispersion of beliefs should be associated with *higher* expected returns** as compensation for bearing the additional risk. This creates a sharp empirical dichotomy: the Miller hypothesis predicts a negative DIVOP-return relation, whereas the Varian model predicts a positive relation.

The distinction between these theories hinges critically on the market microstructure and institutional setting (@tbl-divop-theories).

Table 47.1: Summary of theoretical predictions for the DIVOP-return relation under different assumptions

Theoretical Framework	Short-Sale Constraints	DIVOP-Return Relation	Key Mechanism
E. M. Miller (1977)	Binding	Negative	Optimistic bias in price
Varian (1985)	Non-binding	Positive	Risk premium for uncertainty
Hong and Stein (2003)	Binding, gradual info	Negative, time-varying	Slow diffusion of bearish views
Scheinkman and Xiong (2003)	Binding, overconfidence	Negative	Speculative bubble premium

Hong and Stein (2003) extend Miller's framework by incorporating gradual information diffusion. In their model, bearish information is impounded into prices more slowly than bullish information because short-sale constraints raise the cost of acting on negative views. This generates momentum-like patterns in which high-DIVOP stocks exhibit positive short-run returns (as optimists push prices up) followed by negative long-run returns (as bearish information eventually reaches the market).

Scheinkman and Xiong (2003) introduce an additional dimension by noting that when investors are overconfident about their private signals *and* short-sale constraints bind, stock prices contain a "speculative bubble" component that reflects the option value of reselling the asset to a future investor who may be even more optimistic. This model predicts that both high trading volume and high price volatility should be associated with overpricing, providing a theoretical basis for using volume-based and volatility-based DIVOP proxies.

47.3 Relevance to the Vietnamese Market

The Vietnamese equity market provides an unusually clean setting for testing the Miller hypothesis. Vietnam's equity market operated without any short-selling mechanism from its inception in 2000 through January 2025, which was a full quarter-century in which the first necessary condition of Miller's model (binding short-sale constraints) was satisfied by regulation rather than by market frictions. Even after the introduction of covered short selling in 2025, the mechanism remains restricted to securities meeting specific liquidity and market capitalization thresholds, and the regulatory environment imposes borrowing requirements that significantly raise the cost of shorting relative to developed markets.

The dominance of retail investors amplifies the second necessary condition (i.e., heterogeneous beliefs). Research on the Vietnamese market has documented significant herding behavior (X. V. Vo and Phan 2017; X. V. Vo 2015), sentiment-driven trading (Phan et al. 2023; D. D. Nguyen and Pham 2018), and information asymmetry between domestic and foreign investors (X. V. Vo 2017). These behavioral characteristics naturally generate wider dispersion of investor valuations compared to markets dominated by institutional investors with access to similar analytical frameworks and information sources.

Table 47.2 compares key institutional features relevant to the DIVOP framework between Vietnam and the United States.

Table 47.2: Institutional comparison of Vietnam and the United States relevant to divergence of opinion

Feature	Vietnam (HOSE/HNX)	United States (NYSE/NASDAQ)
Short selling	Introduced Jan 2025 (limited)	Permitted (Reg SHO since 2005)
Retail investor share of volume	~80-85%	~25%
Settlement cycle	T+2 (T+1 planned for 2026)	T+1 (since May 2024)
Daily price limits	± 7% (HOSE), ± 10% (HNX)	None
Foreign ownership cap	49% (most sectors)	None
Average analyst coverage (VN30)	5-10 analysts	15-25 analysts
Mandatory quarterly reporting	Yes (since 2012)	Yes
Options/derivatives market	VN30 Index Futures (since 2017)	Extensive options/futures

The presence of daily price limits (± 7% on HOSE and ± 10% on HNX) creates an additional

mechanism through which divergence of opinion can be amplified. When a stock hits its price limit, investors who wish to trade in the direction of the limit are unable to do so, leading to accumulated unfilled orders and delayed price discovery. This institutional feature may create short-term spikes in measured DIVOP that reflect limit-induced friction rather than genuine disagreement. We address this issue in our empirical methodology by flagging limit-hit days and conducting robustness checks that exclude these observations.

48 Data Sources and Sample Construction

48.1 Data Sources

The construction of DIVOP proxies for the Vietnamese market requires daily stock-level trading data and, for the analyst dispersion measures, individual analyst forecast data. We source all data from DataCore.vn, which provides coverage of all securities listed on HOSE, HNX, and the UPCoM (Unlisted Public Company Market) exchange. Table 48.1 summarizes the datasets and key variables used in this study.

Table 48.1: Data sources and key variables for DIVOP proxy construction

Dataset	Key Variables	Frequency
Daily Stock Trading	Close price, high, low, open, volume, shares outstanding, adjusted price, bid, ask	Daily
Corporate Actions	Dividends, stock splits, bonus issues, rights offerings	Event-based
Company Information	Exchange code, industry classification (ICB), listing date, delisting date	Static/Periodic
Analyst Forecasts	Individual analyst EPS forecasts, announcement dates, fiscal period end, analyst ID, broker name	Per estimate
Market Index	VN-Index daily returns, VN30 returns, HNX-Index returns	Daily
Foreign Ownership	Foreign buy/sell volume, foreign ownership percentage, remaining foreign room	Daily

48.2 Sample Construction

We construct our sample using the following filters, applied sequentially:

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from sklearn.linear_model import LinearRegression
from scipy import stats as scipy_stats
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# ============================================================================
# Configuration Parameters
# ============================================================================
# Users can modify these parameters to adjust the methodology
CONFIG = {
    # Sample period
    'beg_date': '2007-01-01',
    'end_date': '2024-12-31',

    # Estimation windows (in trading days)
    'est_window': 60,          # Rolling window for SUV and volatility
    'detrend_window': 180,     # Window for DTO detrending median
    'lag': 7,                  # Lag for DTO detrending
    'gap': 5,                  # Gap between estimation period and event date

    # Filters
    'min_price': 1000,         # Minimum price in VND
    'min_volume_days': 0.8,    # Min fraction of non-zero volume days in
     ↪ window
    'min_analysts': 3,         # Minimum number of analysts for DISP
    'max_spread_pct': 0.50,    # Maximum bid-ask spread as fraction of
     ↪ midpoint
    'forecast_carry_days': 105,# Days to carry forward stale analyst
     ↪ forecasts

    # Exchange identifiers
    'exchanges': ['HOSE', 'HNX'],

    # Price limit thresholds (for flagging)
```

```
    'price_limit_hose': 0.07,
    'price_limit_hnx': 0.10,
}

print("Configuration parameters loaded successfully.")
print(f"Sample period: {CONFIG['beg_date']} to {CONFIG['end_date']}")
print(f"Estimation window: {CONFIG['est_window']} trading days")
print(f"Detrending window: {CONFIG['detrend_window']} trading days")
```

```
Configuration parameters loaded successfully.
Sample period: 2007-01-01 to 2024-12-31
Estimation window: 60 trading days
Detrending window: 180 trading days
```

The sample universe includes all common stocks (ordinary shares) listed on HOSE and HNX during the period January 2007 through December 2024. We begin in 2007 rather than at market inception (2000 for HOSE, 2005 for HNX) for two reasons. First, the early years of the Vietnamese market were characterized by an extremely small number of listed firms (fewer than 30 on HOSE through 2005), making cross-sectional analysis unreliable. Second, data quality and consistency improve substantially after the market expansion of 2006-2007, during which the number of listed firms on HOSE grew from approximately 40 to over 100.

We apply the following filters to construct the analysis sample:

1. **Security type filter.** We retain only common stocks (ordinary shares), excluding preferred shares, exchange-traded funds (ETFs), covered warrants, and certificates of deposit. This is analogous to the standard filter in the U.S. literature that restricts to CRSP share codes 10 and 11.

2. **Exchange filter.** We include stocks listed on HOSE and HNX but exclude UPCoM securities in our baseline analysis. UPCoM is a registration-based trading venue with less stringent listing requirements and substantially lower liquidity, which may introduce noise into volume-based and spread-based measures. We include UPCoM in robustness checks.

3. **Price filter.** We exclude stock-day observations with closing prices below 1,000 VND. This threshold serves the same purpose as the "penny stock" exclusion common in U.S. studies (typically $1 or $5 thresholds) and helps mitigate the influence of extreme percentage returns and spreads at very low price levels.

4. **Minimum trading activity.** For volume-based measures, we require that a stock has non-zero trading volume on at least 80% of trading days within each estimation window. This filter eliminates the most thinly traded securities for which turnover-based measures would be unreliable.

```
def load_daily_data(config):
    """
    Load daily stock trading data from DataCore.vn.

    In practice, this function connects to the DataCore API or reads
    from a local database/CSV. Here we document the expected schema.

    Expected columns:
    - ticker: str, stock ticker symbol (e.g., 'VCB', 'HPG', 'VNM')
    - date: datetime, trading date
    - open, high, low, close: float, daily OHLC prices (VND)
    - volume: int, trading volume (shares)
    - shares_outstanding: int, total shares outstanding
    - adjusted_close: float, price adjusted for corporate actions
    - adj_factor: float, cumulative adjustment factor
    - bid, ask: float, best bid/ask at close
    - exchange: str, exchange code ('HOSE', 'HNX', 'UPCOM')
    - industry_icb: str, ICB industry classification code
    - foreign_buy_vol, foreign_sell_vol: int, foreign investor volumes
    - foreign_ownership_pct: float, foreign ownership percentage
    """
    # ================================================================
    ↪  ===
    # Replace with actual DataCore API call:
    # from datacore import Client
    # client = Client(api_key='YOUR_KEY')
    # df = client.daily_stock(
    #     start=config['beg_date'], end=config['end_date'],
    #     exchanges=config['exchanges']
    # )
    # ================================================================
    ↪  ===
    print("Connect to DataCore.vn and load daily stock data.")
    print("Expected schema: ticker, date, open, high, low, close, volume,")
    print("  shares_outstanding, adjusted_close, adj_factor, bid, ask,")
    print("  exchange, industry_icb, foreign_buy_vol, foreign_sell_vol,")
    print("  foreign_ownership_pct")
    return None  # Replace with actual data


def apply_sample_filters(df, config):
    """Apply sequential sample construction filters."""
    print("\n=== Sample Construction ===")
```

```
    n0 = len(df)

    # Date filter
    df = df[(df['date'] >= config['beg_date']) &
            (df['date'] <= config['end_date'])].copy()
    print(f"[1] Date filter: {len(df):,} obs (from {n0:,})")

    # Exchange filter
    df = df[df['exchange'].isin(config['exchanges'])].copy()
    print(f"[2] Exchange filter ({config['exchanges']}): {len(df):,} obs")

    # Price filter
    df = df[df['close'] >= config['min_price']].copy()
    print(f"[3] Price >= {config['min_price']:,} VND: {len(df):,} obs")

    # Compute daily return from adjusted prices
    df = df.sort_values(['ticker', 'date'])
    df['ret'] = df.groupby('ticker')['adjusted_close'].pct_change()

    # Flag price limit hits
    df['limit_hit'] = (
        ((df['exchange'] == 'HOSE') &
         (df['ret'].abs() >= config['price_limit_hose'] - 0.001)) |
        ((df['exchange'] == 'HNX') &
         (df['ret'].abs() >= config['price_limit_hnx'] - 0.001))
    )

    n_tickers = df['ticker'].nunique()
    print(f"\nFinal sample: {len(df):,} stock-day obs, "
          f"{n_tickers} unique tickers")
    print(f"Limit-hit days: {df['limit_hit'].sum():,} "
          f"({100*df['limit_hit'].mean():.2f}%)")
    return df
```

48.3 Corporate Action Adjustments

Proper adjustment for corporate actions is critical for volume-based DIVOP measures, as events such as stock splits, bonus share issues, and rights offerings change the number of shares outstanding and can create artificial spikes in measured turnover. We need to use cumulative adjustment factors that account for stock dividends (bonus shares), stock splits,

rights offerings, and cash dividends (price adjustment only). We use these to construct adjusted volume and adjusted shares outstanding:

$$\text{AdjVolume}_{i,t} = \text{Volume}_{i,t} \times \text{CumAdjFactor}_{i,t}$$

$$\text{AdjSharesOut}_{i,t} = \text{SharesOut}_{i,t} \times \text{CumAdjFactor}_{i,t}$$

This ensures that the turnover ratio is consistent across corporate action events.

```
def adjust_for_corporate_actions(df):
    """Apply cumulative adjustment factors to volume and shares
     ↪  outstanding."""
    df = df.copy()
    df['adj_volume'] = df['volume'] * df['adj_factor']
    df['adj_shares_out'] = df['shares_outstanding'] * df['adj_factor']

    # Daily turnover ratio
    df['turnover'] = np.where(
        df['adj_shares_out'] > 0,
        df['adj_volume'] / df['adj_shares_out'],
        np.nan
    )

    # Flag extreme turnover (> 50% of float)
    extreme = df['turnover'] > 0.50
    if extreme.any():
        print(f"Warning: {extreme.sum()} obs with turnover > 50%, set to
         ↪  NaN")
        df.loc[extreme, 'turnover'] = np.nan

    return df
```

48.4 Trading Calendar Construction

The rolling regression approach for SUV and volatility requires a trading calendar that ensures each estimation window contains exactly the specified number of trading days. We construct this directly from observed trading dates.

```
def build_trading_calendar(df, config):
    """
    Map each trading date to its estimation window [est_start, est_end].

    For date t, the estimation window runs from
    t - gap - est_window to t - gap - 1 (in trading-day terms).
    """
    trading_dates = sorted(df['date'].unique())
    trading_dates = pd.Series(trading_dates)

    est_window = config['est_window']
    gap = config['gap']
    offset = est_window + gap

    records = []
    for i in range(offset, len(trading_dates)):
        records.append({
            'date': trading_dates.iloc[i],
            'est_start': trading_dates.iloc[i - gap - est_window],
            'est_end': trading_dates.iloc[i - gap - 1]
        })

    calendar = pd.DataFrame(records)
    print(f"Trading calendar: {len(calendar)} dates, "
          f"{calendar['date'].min()} to {calendar['date'].max()}")
    return calendar
```

49 Volume-Based DIVOP Proxies

49.1 Theoretical Motivation

Trading volume has long been recognized as a natural proxy for divergence of investor opinion. In the rational expectations framework of Milgrom and Stokey (1982), trade occurs only when investors disagree about the value of a security (i.e., a "no-trade theorem" that implies, by contrapositive, that observed trading volume must reflect some form of heterogeneous beliefs). Harris and Raviv (1993) and Kandel and Pearson (1995) formalize this intuition, showing that trading volume is positively related to the dispersion of investors' prior beliefs and to the degree to which public information is differentially interpreted.

The challenge in using raw trading volume as a DIVOP proxy is that volume is also driven by factors unrelated to disagreement, including portfolio rebalancing, liquidity needs, tax-loss selling, and index reconstitution effects. Garfinkel (2009) proposes two approaches to extract the disagreement component from raw volume. The first, **Unexplained Volume (DTO)**, removes market-wide volume effects and secular trends. The second, **Standardized Unexplained Volume (SUV)**, additionally controls for the information content of returns through a cross-sectional regression, isolating the "pure disagreement" component of trading activity.

49.2 Unexplained Volume (DTO)

49.2.1 Construction Methodology

The construction of the Unexplained Volume measure proceeds in four steps.

Step 1: Compute firm-level daily turnover. For each stock i on day t:

$$\text{Turn}_{i,t} = \frac{\text{AdjVolume}_{i,t}}{\text{AdjSharesOut}_{i,t}}$$

Step 2: Compute market-wide turnover. We calculate aggregate turnover across all common stocks as a value-weighted average:

$$\text{MktTurn}_t = \frac{\sum_i \text{AdjVolume}_{i,t}}{\sum_i \text{AdjSharesOut}_{i,t}}$$

Unlike the U.S. methodology that computes market turnover across NYSE/AMEX stocks only and applies a scaling adjustment for NASDAQ securities (following A.-M. Anderson and Dyl 2005), we compute market turnover across all HOSE and HNX common stocks without any exchange-specific volume scaling. Both Vietnamese exchanges operate as order-driven markets (HOSE uses continuous order matching; HNX uses a combination of continuous matching and periodic call auctions) without the dealer-market double-counting issue that necessitates the NASDAQ volume adjustment in U.S. studies.

Step 3: Compute market-adjusted turnover.

$$\text{MATO}_{i,t} = \text{Turn}_{i,t} - \text{MktTurn}_t$$

Step 4: Detrend by rolling median. To remove secular trends in firm-specific trading activity:

$$\text{DTO}_{i,t} = \text{MATO}_{i,t} - \text{Median}_{180}(\text{MATO}_{i,t-7})$$

where $\text{Median}_{180}(\text{MATO}_{i,t-7})$ is the median of market-adjusted turnover over the 180-trading-day window ending 7 days before date t. The 7-day lag prevents the current day's turnover from influencing its own detrending baseline.

```
def compute_market_turnover(df):
    """Compute daily market-wide turnover across all stocks."""
    mkt_turn = df.groupby('date').apply(
        lambda x: x['adj_volume'].sum() / x['adj_shares_out'].sum()
        if x['adj_shares_out'].sum() > 0 else np.nan
    ).reset_index()
    mkt_turn.columns = ['date', 'market_turnover']
    return mkt_turn


def compute_dto(df, config):
    """
    Construct Unexplained Volume (DTO).

    Steps:
    1. Subtract market turnover -> MATO
    2. Rolling 180-day median of MATO (lagged 7 days) -> trend
```

```
    3. DTO = MATO - trend
    """
    detrend_window = config['detrend_window']
    lag = config['lag']

    # Market turnover
    mkt_turn = compute_market_turnover(df)
    df = df.merge(mkt_turn, on='date', how='left')

    # Market-adjusted turnover
    df['mato'] = df['turnover'] - df['market_turnover']

    # Rolling median with lag, computed per stock
    df = df.sort_values(['ticker', 'date'])

    def _rolling_median_lagged(group):
        mato = group['mato']
        med = mato.rolling(
            window=detrend_window,
            min_periods=int(detrend_window * 0.5)
        ).median()
        return med.shift(lag)

    df['mato_trend'] = (
        df.groupby('ticker', group_keys=False)
          .apply(lambda g: _rolling_median_lagged(g))
    )

    # DTO
    df['dto'] = df['mato'] - df['mato_trend']

    print("DTO construction complete.")
    print(f"  Non-missing: {df['dto'].notna().sum():,}")
    print(f"  Mean: {df['dto'].mean():.6f}, Std: {df['dto'].std():.6f}")
    return df
```

49.2.2 Vietnam-Specific Considerations for DTO

Several features of the Vietnamese market require attention when constructing DTO:

1. **No NASDAQ-type volume adjustment needed.** Both HOSE and HNX are order-driven auction markets. The double-counting adjustment applied to NASDAQ securities

in the U.S. literature is not necessary.

2. **Thinly traded stocks.** A substantial fraction of listed Vietnamese stocks, particularly on HNX, may have zero volume on many trading days. For stocks with intermittent trading, the rolling median may be biased toward zero, making DTO less informative. We require at least 80% non-zero volume days in each estimation window.

3. **Price limit effects on volume.** When a stock hits its daily price limit, unfilled orders accumulate and recorded volume may understate true clearing volume. The following day often shows a "catch-up" effect. Researchers should consider flagging limit-hit days.

4. **Foreign investor trading decomposition.** DataCore provides volume by investor type (foreign versus domestic). Researchers may wish to construct separate DTO measures for foreign and domestic volume, or use the foreign-to-domestic volume ratio as an additional dimension of disagreement.

49.3 Standardized Unexplained Volume (SUV)

49.3.1 Construction Methodology

The Standardized Unexplained Volume measure, proposed by Garfinkel (2009), isolates the disagreement component of volume by explicitly controlling for the information content of returns. The insight is that trading volume has both a **liquidity** component and an **informedness** component correlated with the magnitude and sign of returns. By regressing turnover on signed returns and extracting the standardized residual, SUV captures volume attributable to disagreement after controlling for both liquidity trends and information-driven trading.

For each stock i, on each trading date t, we estimate using data from the estimation window $[\tau_1, \tau_2]$:

$$\text{Turn}_{i,s} = \alpha_i + \beta_i^+ \cdot \text{RetPos}_{i,s} + \beta_i^- \cdot \text{RetNeg}_{i,s} + \epsilon_{i,s}, \quad s \in [\tau_1, \tau_2] \tag{49.1}$$

where $\text{RetPos}_{i,s} = |r_{i,s}| \cdot \mathbf{1}(r_{i,s} > 0)$ and $\text{RetNeg}_{i,s} = |r_{i,s}| \cdot \mathbf{1}(r_{i,s} < 0)$.

The Standardized Unexplained Volume on date t is:

$$\text{SUV}_{i,t} = \frac{\text{Turn}_{i,t} - \widehat{\text{Turn}}_{i,t}}{\hat{\sigma}_{\epsilon,i}} \tag{49.2}$$

where $\widehat{\text{Turn}}_{i,t}$ is the predicted turnover and $\hat{\sigma}_{\epsilon,i}$ is the RMSE from Equation 49.1.

The asymmetric specification with separate coefficients for positive and negative returns reflects that the volume-return relation differs by return sign. In the U.S., buying pressure tends to

generate more volume than selling pressure due to short-sale frictions. In Vietnam, where short selling was unavailable until 2025, this asymmetry should be even more pronounced because all selling activity was constrained to existing shareholders.

```
def compute_suv(df, calendar, config):
    """
    Compute Standardized Unexplained Volume via rolling regressions.

    For each stock-date, regress Turn on RetPos and RetNeg over the
    estimation window, then compute SUV = (actual - predicted) / RMSE.
    """
    est_window = config['est_window']
    min_obs = int(est_window * config['min_volume_days'])

    # Prepare signed return components
    df = df.copy()
    df['ret_pos'] = np.where(df['ret'] > 0, np.abs(df['ret']), 0.0)
    df['ret_neg'] = np.where(
        (df['ret'] < 0) & df['ret'].notna(), np.abs(df['ret']), 0.0
    )

    results = []
    grouped = {t: g for t, g in df.groupby('ticker')}

    for _, cal_row in calendar.iterrows():
        dt = cal_row['date']
        est_s, est_e = cal_row['est_start'], cal_row['est_end']

        for ticker, tdata in grouped.items():
            # Estimation window
            est = tdata[
                (tdata['date'] >= est_s) & (tdata['date'] <= est_e)
            ].dropna(subset=['turnover', 'ret_pos', 'ret_neg'])

            if len(est) < min_obs:
                continue

            # Event date
            evt = tdata[tdata['date'] == dt]
            if evt.empty or evt['turnover'].isna().all():
                continue

            # OLS: Turn = alpha + beta_pos * RetPos + beta_neg * RetNeg
```

```
            X = est[['ret_pos', 'ret_neg']].values
            y = est['turnover'].values

            reg = LinearRegression().fit(X, y)
            y_hat = reg.predict(X)
            rmse = np.sqrt(np.mean((y - y_hat) ** 2))

            if rmse <= 0:
                continue

            # Predict and standardize for event date
            X_evt = evt[['ret_pos', 'ret_neg']].values
            pred = reg.predict(X_evt)[0]
            actual = evt['turnover'].values[0]
            suv = (actual - pred) / rmse

            results.append({
                'ticker': ticker, 'date': dt,
                'suv': suv,
                'predicted_turnover': pred,
                'rmse_turn': rmse,
                'n_est': len(est),
                'alpha_turn': reg.intercept_,
                'beta_pos': reg.coef_[0],
                'beta_neg': reg.coef_[1],
            })

    suv_df = pd.DataFrame(results)
    print(f"SUV: {len(suv_df):,} stock-date obs")
    print(f"  Mean: {suv_df['suv'].mean():.4f}, "
          f"Median: {suv_df['suv'].median():.4f}")
    return suv_df
```

49.3.2 Interpreting the SUV Regression Coefficients

The estimated coefficients from Equation 49.1 are informative about market microstructure. Garfinkel (2009) reports $\hat{\beta}^+ > \hat{\beta}^-$ for most U.S. stocks. In Vietnam, we expect this asymmetry to be even stronger because:

- **No short selling (pre-2025):** All selling is by existing shareholders, limiting volume response to negative returns.

- **T+2 settlement:** Investors cannot immediately reinvest sale proceeds, further dampening sell-side volume.
- **Price limits:** The $\pm$ 7% (HOSE) and $\pm$ 10% (HNX) daily limits truncate the return distribution, compressing the range of both regressors.

Researchers should report summary statistics of $(\hat{\alpha}, \hat{\beta}^{+}, \hat{\beta}^{-}, R^2)$ across the cross-section and over time.

```
def suv_diagnostics(suv_df):
    """Report cross-sectional summary of SUV regression parameters."""
    print("\n=== SUV Regression Diagnostics ===")

    params = ['alpha_turn', 'beta_pos', 'beta_neg']
    print(suv_df[params].describe(
        percentiles=[.05, .25, .50, .75, .95]
    ).T.to_string(float_format='{:.6f}'.format))

    # Asymmetry test
    diff = suv_df['beta_pos'] - suv_df['beta_neg']
    print(f"\nbeta_pos - beta_neg: mean = {diff.mean():.6f}, "
          f"frac > 0 = {(diff > 0).mean():.3f}")
```

50 Volatility-Based DIVOP Proxies

50.1 Total Return Volatility

50.1.1 Theoretical Motivation

Stock return volatility serves as a proxy for divergence of opinion through several channels. Shalen (1993) develops a model in which both volume and volatility are increasing in the dispersion of investor beliefs. Scheinkman and Xiong (2003) predict that higher volatility reflects the speculative trading component driven by overconfident investors who disagree about value. Empirically, Boehme, Danielsen, and Sorescu (2006) and Chatterjee, John, and Yan (2012) use idiosyncratic volatility as a DIVOP proxy and find it positively correlated with other disagreement measures and negatively associated with subsequent returns when short-sale constraints bind.

50.1.2 Construction

Total return volatility is the standard deviation of daily returns over the rolling estimation window:

$$\text{VOLATILITY}_{i,t} = \sqrt{\frac{1}{N_i - 1} \sum_{s \in [\tau_1, \tau_2]} (r_{i,s} - \bar{r}_i)^2} \tag{50.1}$$

where N_i is the number of non-missing return observations for stock i in the window $[\tau_1, \tau_2]$.

50.2 Idiosyncratic Volatility (IVOL)

Idiosyncratic volatility isolates firm-specific return variation by removing the systematic component explained by market movements. We compute IVOL from the residuals of a market model:

$$r_{i,s} = \alpha_i + \beta_i \cdot r_{m,s} + \epsilon_{i,s}, \quad s \in [\tau_1, \tau_2] \tag{50.2}$$

$$\text{IVOL}_{i,t} = \text{Std}(\hat{\epsilon}_{i,s}) \tag{50.3}$$

Researchers may extend this to a Eugene F. Fama and French (1993b) three-factor or five-factor model using Vietnamese factor portfolios constructed elsewhere in this book. A richer factor model yields IVOL estimates that better isolate truly idiosyncratic disagreement, at the cost of requiring factor portfolio construction.

```
def compute_volatility(df, calendar, config):
    """
    Compute total return volatility and idiosyncratic volatility
    via rolling estimation windows.

    Total vol = std(returns) in window.
    IVOL = std(residuals) from market model regression.
    """
    est_window = config['est_window']
    min_obs = int(est_window * config['min_volume_days'])

    # Value-weighted market return
    def _vw_ret(g):
        valid = g.dropna(subset=['ret'])
        if valid.empty:
            return np.nan
        w = valid['adj_shares_out'] * valid['close']
        return np.average(valid['ret'], weights=w)

    mkt_ret = df.groupby('date').apply(_vw_ret).reset_index()
    mkt_ret.columns = ['date', 'mkt_ret']
    df = df.merge(mkt_ret, on='date', how='left')

    results = []
    grouped = {t: g for t, g in df.groupby('ticker')}

    for _, cal_row in calendar.iterrows():
        dt = cal_row['date']
        est_s, est_e = cal_row['est_start'], cal_row['est_end']

        for ticker, tdata in grouped.items():
            est = tdata[
                (tdata['date'] >= est_s) & (tdata['date'] <= est_e)
            ].dropna(subset=['ret', 'mkt_ret'])
```

```
            if len(est) < min_obs:
                continue

            # Total volatility
            total_vol = est['ret'].std()

            # Market model -> IVOL
            X = est[['mkt_ret']].values
            y = est['ret'].values
            reg = LinearRegression().fit(X, y)
            resid = y - reg.predict(X)
            ivol = np.std(resid, ddof=1)

            results.append({
                'ticker': ticker, 'date': dt,
                'total_volatility': total_vol,
                'idio_volatility': ivol,
                'market_beta': reg.coef_[0],
                'market_alpha': reg.intercept_,
                'r_squared_mm': reg.score(X, y),
                'n_vol': len(est),
            })

    vol_df = pd.DataFrame(results)
    print(f"Volatility: {len(vol_df):,} stock-date obs")
    print(f"  Total vol (ann. mean): "
          f"{vol_df['total_volatility'].mean() * np.sqrt(252):.4f}")
    print(f"  IVOL (ann. mean): "
          f"{vol_df['idio_volatility'].mean() * np.sqrt(252):.4f}")
    return vol_df
```

50.2.1 Vietnam-Specific Considerations for Volatility

1. **Price limits compress measured volatility.** Daily limits of $\pm$ 7% (HOSE) and $\pm$ 10% (HNX) mechanically truncate the return distribution, leading to underestimation of true volatility. On limit-hit days, the true equilibrium return may exceed the observed return. Researchers should be aware that volatility-based DIVOP measures may be downward-biased for stocks that frequently hit limits.

2. **VN-Index concentration.** The VN-Index is highly concentrated, the top 10 stocks often account for 50-60% of index weight. For small- and mid-cap stocks, an equal-

weighted market return or a composite HOSE+HNX index may provide a better market factor in Equation 50.2.

3. **Thin trading and non-synchronous returns.** For thinly traded stocks, consecutive zero-return days can depress measured volatility. The Dimson (1979) adjustment (including lagged and lead market returns in the market model) may help correct for non-synchronous trading bias in the beta estimate, though its effect on IVOL is typically small.

51 Spread-Based and Liquidity DIVOP Proxies

51.1 Bid-Ask Spread (BASPREAD)

51.1.1 Theoretical Motivation

The bid-ask spread reflects the adverse selection costs faced by limit order providers. When investors hold heterogeneous beliefs, each trade is more likely to convey private information, raising the adverse selection component of the spread. Handa, Schwartz, and Tiwari (2003) show that in order-driven markets the spread widens when divergence of opinion increases because limit order providers face greater risk of being picked off by informed traders. Chung and Zhang (2014) demonstrate that closing bid-ask spreads from daily data provide a reliable approximation to intraday effective spreads.

51.1.2 Construction

We compute the proportional bid-ask spread using end-of-day quote data:

$$\text{BASPREAD}_{i,t} = \frac{\text{Ask}_{i,t} - \text{Bid}_{i,t}}{\text{Midpoint}_{i,t}} \tag{51.1}$$

where $\text{Midpoint}_{i,t} = (\text{Ask}_{i,t} + \text{Bid}_{i,t})/2$. When end-of-day bid and ask are unavailable, we use the daily high-low range as a fallback. Following Chung and Zhang (2014), we delete observations where both Bid and Ask are zero, and where the spread exceeds 50% of the midpoint.

51.2 Amihud Illiquidity (ILLIQ)

The Amihud (2002) ratio measures the price impact of order flow:

$$\text{ILLIQ}_{i,t} = \frac{|r_{i,t}|}{\text{DolVol}_{i,t}} \tag{51.2}$$

where $\text{DolVol}_{i,t} = \text{Volume}_{i,t} \times \text{Price}_{i,t}$ (in billions VND for scaling). Higher ILLIQ reflects greater information asymmetry. We average daily ratios over monthly horizons and use the log transformation due to heavy right skew.

```
def compute_spread_and_illiq(df, config):
    """Compute bid-ask spread (BASPREAD) and Amihud illiquidity."""
    df = df.copy()

    # --- Bid-Ask Spread ---
    df['midpoint_ba'] = (df['ask'] + df['bid']) / 2
    df['baspread_ba'] = np.where(
        (df['ask'] > 0) & (df['bid'] > 0) & (df['midpoint_ba'] > 0),
        (df['ask'] - df['bid']) / df['midpoint_ba'], np.nan
    )

    # Fallback: high/low range
    df['midpoint_hl'] = (df['high'] + df['low']) / 2
    df['baspread_hl'] = np.where(
        (df['high'] > 0) & (df['low'] > 0) & (df['midpoint_hl'] > 0),
        (df['high'] - df['low']) / df['midpoint_hl'], np.nan
    )

    df['baspread'] = df['baspread_ba'].fillna(df['baspread_hl'])
    df['midpoint'] = df['midpoint_ba'].fillna(df['midpoint_hl'])

    # Chung & Zhang (2009) filters
    bad = (df['baspread'].isna()) | \
          (df['baspread'] > config['max_spread_pct']) | \
          (df['baspread'] < 0)
    df.loc[bad, 'baspread'] = np.nan

    # --- Amihud Illiquidity ---
    df['dollar_vol'] = df['volume'] * df['close'] / 1e9
    df['amihud_daily'] = np.where(
        df['dollar_vol'] > 0,
        np.abs(df['ret']) / df['dollar_vol'], np.nan
    )

    print(f"BASPREAD: {df['baspread'].notna().sum():,} valid obs, "
          f"mean = {df['baspread'].mean():.6f}")
    print(f"AMIHUD: {df['amihud_daily'].notna().sum():,} valid obs, "
          f"mean = {df['amihud_daily'].mean():.6f}")
    return df
```

```
def compute_amihud_monthly(df):
    """Monthly Amihud = mean daily |ret|/dollar_vol (min 15 days)."""
    df = df.copy()
    df['ym'] = df['date'].dt.to_period('M')
    agg = df.groupby(['ticker', 'ym']).agg(
        illiq_mean=('amihud_daily', 'mean'),
        n_days=('amihud_daily', 'count'),
    ).reset_index()
    agg = agg[agg['n_days'] >= 15].copy()
    agg['log_illiq'] = np.log(agg['illiq_mean'] + 1e-10)
    return agg
```

51.2.1 Vietnam-Specific Considerations for Spread and Liquidity

1. **Tick size schedule.** Vietnam uses variable tick sizes: 10 VND (prices $<$ 10,000), 50 VND (10,000–49,950), and 100 VND (50,000) on HOSE. These impose a floor on quoted spreads for low-priced stocks. Researchers should be cautious interpreting cross-price-decile spread variation as reflecting opinion divergence rather than tick-size mechanics.

2. **Order-driven market structure.** Both HOSE and HNX are pure order-driven markets where public limit orders provide liquidity. This makes the Chung and Zhang (2014) CRSP-based spread approximation appropriate.

3. **Lot size requirements.** HOSE requires 100-share standard lots for continuous trading. For high-priced stocks, the standard lot represents a large capital commitment, potentially inflating quoted spreads relative to effective trading costs.

4. **Call auction effects.** Opening and closing sessions on HOSE use periodic call auctions, which can produce bid-ask quotes that differ substantially from continuous-trading spreads.

51.3 Realized Illiquidity: A Lower-Noise ILLIQ

The ILLIQ proxy in Equation 51.2 summarizes a full day of price movement with one absolute return. Lacava, Ranaldo, and Santucci de Magistris (2026) replace that numerator with a high-frequency estimate of the same day's price variation, yielding the **realized Amihud**

$$\mathrm{RILLIQ}_{i,t} = \sqrt{\frac{2}{\pi}}\,\frac{\sqrt{\mathrm{RV}_{i,t}}}{\mathrm{DolVol}_{i,t}}, \qquad \mathrm{RV}_{i,t} = \sum_{j=1}^{M} r_{i,t,j}^{2} \tag{51.3}$$

with $r_{i,t,j}$ the intraday returns over M sampling intervals and $\sqrt{2/\pi}$ chosen so that Equation 51.3 sits on the same scale as Equation 51.2. At HOSE's 45 five-minute intervals the numerator's relative measurement error falls from roughly 76% to roughly 11%. The estimator's derivation and assumptions are in Section 14.4; its HOSE implementation is in **?@sec-micro-realized-illiq**.

The obvious benefit is the one common to any DIVOP proxy used on the right-hand side: a cleaner regressor attenuates less, so the Fama-MacBeth slopes in Chapter 56 are estimated closer to their true values. That argument is general and is made in Section 14.4.

The reason this chapter needs its own treatment is a different and more specific problem, and it affects results this chapter actually reports.

51.3.1 Shared Sampling Noise Contaminates the Correlation Matrix

Look at what the proxies in this chapter are built from. ILLIQ takes $|r_{i,t}|$ from the daily return series. TOTVOL takes the standard deviation of *that same series*. IVOL takes the standard deviation of its residual. When end-of-day quotes are missing, BASPREAD falls back to the daily high-low range, drawn from the same daily price records.

These are not independent measurements. A month in which a firm's daily returns happen to realize large draws will produce a high measured ILLIQ *and* a high measured TOTVOL *and* a high measured IVOL — not because the firm's true opinion divergence was high, but because the same finite sample of daily returns feeds all three. The sampling error is common across proxies, so it does not average out when they are correlated with one another or combined.

This matters directly for two things the chapter does. Chapter 53 reports Spearman correlations among the DIVOP proxies and interprets the pattern as evidence about which dimensions of divergence the proxies capture. Chapter 54 builds a composite index by extracting the first principal component. Both are computed on the correlation structure of measured proxies, and both therefore absorb whatever correlation the shared daily returns manufacture.

The simulation below isolates the effect. True volatility and true trading volume are drawn **independently** across firms, so any excess correlation is attributable to construction rather than economics. The critical control is a `TOTVOL (disjoint)` column, which computes volatility from a *different* month's returns for the same firm: it measures the same underlying volatility but shares no sampling noise with ILLIQ, so the gap between it and the ordinary same-window correlation is precisely the mechanical component.

```
rng_ri = np.random.default_rng(2026)
N_SIM, D_MONTH, M_HF = 8000, 21, 45
K_HF = np.sqrt(2 / np.pi)
```

```
def simulate_month(vol_dispersion):
    """
    One cross-section of firm-months. True volatility and true dollar volume
    are INDEPENDENT by construction, so any measured association between
    illiquidity and volatility beyond sqrt(sigma)/DolVol is an artefact.

    `vol_dispersion` is the cross-sectional sd of log dollar volume: wide for
    the full market, narrow within a size decile.
    """
    sigma = np.exp(rng_ri.normal(np.log(0.022), 0.35, N_SIM))
    dvol = np.exp(rng_ri.normal(np.log(5.0), vol_dispersion, N_SIM))
    beta = rng_ri.normal(1.0, 0.3, N_SIM)
    mkt = rng_ri.normal(0, 0.012, (1, D_MONTH))

    R = beta[:, None] * mkt + rng_ri.normal(0, 1, (N_SIM, D_MONTH)) *
 ↪  sigma[:, None]
    RV = (sigma[:, None] ** 2) * rng_ri.chisquare(M_HF, (N_SIM, D_MONTH)) /
 ↪  M_HF

    # Same firm, same true sigma, DIFFERENT month: no shared sampling noise.
    R_alt = beta[:, None] * mkt + rng_ri.normal(0, 1, (N_SIM, D_MONTH)) *
 ↪  sigma[:, None]

    return dict(
        sigma=sigma,
        illiq_true=K_HF * sigma / dvol,
        illiq_classic=np.abs(R).mean(1) / dvol,
        illiq_realized=K_HF * np.sqrt(RV).mean(1) / dvol,
        totvol=R.std(1, ddof=1),
        totvol_disjoint=R_alt.std(1, ddof=1),
        ivol=(R - beta[:, None] * mkt).std(1, ddof=1),
    )


def rho(a, b):
    return scipy_stats.spearmanr(a, b).statistic


rows_ri = []
for label, disp in [("full cross-section", 0.90), ("within size decile",
 ↪  0.30),
                    ("volume near-constant", 0.10)]:
```

```
    s = simulate_month(disp)
    rows_ri.append({
        "volume dispersion": label,
        "TRUE illiq vs TRUE vol": round(rho(s["illiq_true"], s["sigma"]), 3),
        "classic vs TOTVOL": round(rho(s["illiq_classic"], s["totvol"]), 3),
        "classic vs TOTVOL (disjoint)": round(rho(s["illiq_classic"],
         ↪  s["totvol_disjoint"]), 3),
        "realized vs TOTVOL": round(rho(s["illiq_realized"], s["totvol"]),
         ↪  3),
        "fidelity: classic": round(rho(s["illiq_classic"], s["illiq_true"]),
         ↪  3),
        "fidelity: realized": round(rho(s["illiq_realized"],
         ↪  s["illiq_true"]), 3),
    })

print("Spearman correlations; true volatility and true volume are
 ↪  INDEPENDENT")
print(pd.DataFrame(rows_ri).set_index("volume dispersion").T.to_string())


def normal_scores(x):
    """Rank-based normal scores, matching the chapter's rank-correlation
     ↪  approach."""
    return scipy_stats.norm.ppf(scipy_stats.rankdata(x) / (len(x) + 1))


s = simulate_month(0.30)
print("\nPCA composite of (ILLIQ, TOTVOL, IVOL), within size decile:")
for name, ill in [("classic ILLIQ", s["illiq_classic"]),
                  ("realized ILLIQ", s["illiq_realized"]),
                  ("TRUE illiquidity", s["illiq_true"])]:
    X = np.column_stack([normal_scores(c) for c in (ill, s["totvol"],
 ↪  s["ivol"])])
    C = np.corrcoef(X, rowvar=False)
    ev = np.linalg.eigvalsh(C)[::-1]
    print(f"  {name:<17} PC1 share={ev[0] / ev.sum():.3f}   "
          f"corr(ILLIQ,TOTVOL)={C[0, 1]:.3f}   corr(ILLIQ,IVOL)={C[0,
           ↪  2]:.3f}")
```

```
Spearman correlations; true volatility and true volume are INDEPENDENT
volume dispersion            full cross-section  within size decile  volume near-constant
TRUE illiq vs TRUE vol                    0.353               0.739                 0.958
```

```
classic vs TOTVOL                          0.315          0.690          0.932
classic vs TOTVOL (disjoint)               0.239          0.514          0.726
realized vs TOTVOL                         0.295          0.609          0.824
fidelity: classic                          0.977          0.895          0.862
fidelity: realized                         1.000          0.999          0.998

PCA composite of (ILLIQ, TOTVOL, IVOL), within size decile:
  classic ILLIQ     PC1 share=0.841   corr(ILLIQ,TOTVOL)=0.713   corr(ILLIQ,IVOL)=0.655
  realized ILLIQ    PC1 share=0.829   corr(ILLIQ,TOTVOL)=0.628   corr(ILLIQ,IVOL)=0.685
  TRUE illiquidity  PC1 share=0.829   corr(ILLIQ,TOTVOL)=0.629   corr(ILLIQ,IVOL)=0.686
```

Four findings, each with a direct bearing on how the results in Chapter 53 and Chapter 54 should be read.

The mechanical inflation is large, and largest exactly where this chapter works. Within a size decile, the measured correlation between classic ILLIQ and TOTVOL is 0.690 when both are computed from the same month's returns, but only 0.514 when TOTVOL comes from a disjoint window. Roughly a quarter of the apparent association is manufactured by shared sampling noise. In the near-constant-volume case the gap widens further, from 0.726 to 0.932. Since DIVOP research routinely conditions on size — and size is the dominant driver of dollar-volume dispersion — the relevant regime is the narrow-dispersion one, not the full cross-section.

The realized measure removes essentially all of it. Its fidelity to true illiquidity is 0.999 against 0.895 for the classic measure within a size decile, and its correlation with TOTVOL (0.609) reflects the genuine relation attenuated only by TOTVOL's own sampling error, not by any shared component.

The classic proxy loses its edge precisely where the numerator has to do the work. Fidelity to true illiquidity falls from 0.977 in the full cross-section to 0.862 when volume is nearly constant, while the realized measure holds above 0.998 throughout. The interpretation is the one anticipated in Section 14.3.1.1: across the whole market the classic ratio ranks stocks well because the *denominator* varies enormously and is measured without error. Strip that variation out by conditioning on size, and what remains is the noisy numerator.

The PCA composite inherits the contamination, though modestly. Using the classic proxy, the ILLIQ-TOTVOL correlation entering the PCA is 0.713 against a true 0.629, and the first component's variance share is 0.841 against a true 0.829. Substituting the realized measure reproduces the true correlation matrix and the true PC1 share almost exactly (0.628 and 0.829). The composite index is thus mildly overstated as a summary of common variation — it is partly summarizing shared measurement error rather than a common divergence-of-opinion factor. The distortion is small here because only one of the three inputs was replaced; a composite built entirely from same-window daily statistics would fare worse.

51.3.2 What This Means for the Chapter's Results

The practical readings are narrow and worth stating precisely, since the effects above are real but not enormous.

1. **The reported correlations among return-based proxies are upper bounds.** Wherever Chapter 53 shows ILLIQ correlating with TOTVOL, IVOL, or a high-low BASPREAD, part of that correlation is construction rather than economics. Correlations with proxies built from *different* data — DTO and SUV from turnover, DISP from analyst forecasts — are not affected by this channel and can be read at face value.
2. **A cheap robustness check requires no intraday data at all.** Recomputing the correlation matrix with the volatility proxies estimated over an adjacent, non-overlapping window breaks the shared-noise channel directly. Any correlation that survives is not an artefact of common sampling error. This is the `disjoint` column of the simulation, implemented on real data, and it costs one extra `shift`.
3. **Do not over-interpret the PC1 variance share.** It is biased upward by shared noise, so the claim that a single factor explains most of the common variation in DIVOP proxies is somewhat weaker than the raw eigenvalues suggest.
4. **Conditioning on size raises the stakes.** Any specification that works within size groups strips out the volume variation that carries most of the classic ILLIQ's cross-sectional signal, leaving a proxy whose fidelity to true illiquidity falls toward 0.86. This is where substituting Equation 51.3 buys the most.

51.3.3 Construction

The realized proxy is a drop-in replacement for `amihud_daily` in `compute_spread_and_illiq()`. It requires the session-aware intraday inputs described in **?@sec-micro-realized-illiq**.

```
def compute_realized_illiq(realized_daily, config, min_active=20):
    """
    Monthly realized Amihud from a symbol-day intraday panel with columns
    [ticker, date, rv, bv, n_active, dvol_continuous].

    `min_active` screens out days whose five-minute grid is mostly stale
    prices; below roughly 20 active intervals of 45 the estimator's precision
    advantage largely disappears (see the microstructure chapter).
    """
    k = np.sqrt(2 / np.pi)
    df = realized_daily.copy()

    n_before = len(df)
    df = df[(df["n_active"] >= min_active) & (df["dvol_continuous"] > 0)]
```

```
    print(f"Thin-trading screen: kept {len(df):,} of {n_before:,} symbol-days "
          f"({100 * len(df) / max(n_before, 1):.1f}%)")

    dvol_bn = df["dvol_continuous"] / 1e9
    df["rilliq_daily"] = k * np.sqrt(df["rv"]) / dvol_bn
    df["rilliq_bv_daily"] = k * np.sqrt(df["bv"].clip(lower=0)) / dvol_bn

    df["ym"] = df["date"].dt.to_period("M")
    agg = (df.groupby(["ticker", "ym"])
             .agg(rilliq_mean=("rilliq_daily", "mean"),
                  rilliq_bv_mean=("rilliq_bv_daily", "mean"),
                  n_days=("rilliq_daily", "size"))
             .reset_index())
    agg = agg[agg["n_days"] >= config.get("min_days_per_month", 15)].copy()
    agg["log_rilliq"] = np.log(agg["rilliq_mean"] + 1e-10)
    return agg


def totvol_disjoint(monthly_vol):
    """
    Robustness control requiring no intraday data: pair each month's ILLIQ
    with the PREVIOUS month's volatility, so the two share no daily returns.
    """
    out = monthly_vol.sort_values(["ticker", "ym"]).copy()
    out["totvol_lag"] = out.groupby("ticker")["totvol"].shift(1)
    return out
```

51.3.4 Vietnam-Specific Considerations

1. **Coverage is the binding constraint.** Intraday data is realistically available for the liquid end of HOSE — roughly the VN30 and larger mid-caps. Because the screen removes firms non-randomly on exactly the dimension under study, a DIVOP analysis restricted to the surviving sample is conditioned on liquidity. Report the retained fraction.
2. **Use the bipower variant.** Vietnam's daily price bands make large discrete moves routine, and an information jump inflates realized variance while leaving bipower variation nearly unbiased (Section 14.4). For a chapter about divergence of opinion this matters more than usual, since the days with the most disagreement are the days most likely to hit the band.
3. **The disjoint-window check should be run regardless.** It requires no data this chapter does not already have, it directly addresses the contamination documented above,

and it applies to the whole sample rather than only the liquid subset.

4. **The denominator is unchanged.** Equation 51.3 improves the numerator only; the put-through and reporting complications affecting DolVol, discussed in the microstructure chapter, apply equally to both versions.

52 Analyst Forecast Dispersion

52.1 Theoretical Motivation

Analyst forecast dispersion, the cross-sectional standard deviation of individual analysts' earnings forecasts, is the most direct measure of divergence of opinion. Unlike market-based proxies that capture disagreement indirectly, forecast dispersion directly measures disagreement among informed market participants. Abarbanell, Lanen, and Verrecchia (1995) establish the theoretical basis, and Diether, Malloy, and Scherbina (2002) demonstrate that stocks with higher analyst forecast dispersion earn lower subsequent returns, consistent with the Miller overpricing hypothesis.

52.2 Data Challenges in Vietnam

Constructing analyst forecast dispersion in Vietnam presents substantial challenges relative to the U.S.:

- **Coverage breadth.** While I/B/E/S covers over 4,000 U.S. companies, only 100–150 Vietnamese firms typically have coverage by at least 3 analysts, concentrated among VN30 constituents.
- **Data sources.** Analyst forecasts are available from DataCore.vn, FiinPro, Bloomberg, and Refinitiv. The choice of source affects coverage and timeliness.
- **Forecast staleness.** With limited coverage, forecasts may go unrevised for months. Following I/B/E/S methodology, we carry each forecast forward for a maximum of 105 days.

52.3 Construction Methodology

The construction proceeds as follows:

1. **Clean individual forecasts.** Remove observations where the announcement date precedes the review date. Keep only annual EPS forecasts. For each analyst-ticker-fiscal period, retain only the latest forecast per calendar month.

2. **Handle stopped and excluded estimates.** Remove forecasts where the analyst has left the brokerage or the estimate has been excluded from consensus.
3. **Carry forward with staleness control.** Each forecast is valid until the earlier of: (a) the next forecast by the same analyst, (b) 105 days after the announcement, or (c) the actual earnings announcement date.
4. **Expand to monthly frequency.** For each ticker-month, identify all valid outstanding forecasts and compute dispersion.
5. **Compute scaled measures:**

$$\text{DISP1}_{i,m} = \frac{\text{Std}(\hat{\text{EPS}}_{i,m}^{(a)})}{|\text{Mean}(\hat{\text{EPS}}_{i,m}^{(a)})|} \qquad \text{DISP2}_{i,m} = \frac{\text{Std}(\hat{\text{EPS}}_{i,m}^{(a)})}{\bar{P}_{i,m}}$$

```
def construct_analyst_dispersion(forecasts_df, price_df, config):
    """
    Construct analyst forecast dispersion measures.

    Parameters
    ----------
    forecasts_df : pd.DataFrame
        Individual analyst forecasts with: ticker, analyst_id, broker,
        fpedats, anndats, revdats, value (EPS), anndats_act.
    price_df : pd.DataFrame
        Monthly price: ticker, month, mean_price.
    config : dict
        With min_analysts, forecast_carry_days.
    """
    carry_days = config['forecast_carry_days']
    min_analysts = config['min_analysts']

    df = forecasts_df.copy()
    df = df[df['anndats'] <= df['revdats']].copy()
    df = df.dropna(subset=['fpedats', 'anndats', 'value'])

    # Latest forecast per analyst-month
    df['ym'] = df['anndats'].dt.to_period('M')
    df = df.sort_values(
        ['ticker', 'fpedats', 'analyst_id', 'ym', 'anndats', 'revdats']
    )
    df = df.groupby(['ticker', 'fpedats', 'analyst_id', 'ym']).tail(1)

    # Carry-forward end date
```

```
    df = df.sort_values(
        ['ticker', 'analyst_id', 'fpedats', 'anndats'],
        ascending=[True, True, True, False]
    )
    df['next_ann'] = df.groupby(
        ['ticker', 'analyst_id', 'fpedats']
    )['anndats'].shift(-1)

    def _carry_end(row):
        candidates = [row['anndats'] + pd.Timedelta(days=carry_days)]
        if pd.notna(row.get('next_ann')):
            candidates.append(row['next_ann'])
        if pd.notna(row.get('anndats_act')):
            candidates.append(row['anndats_act'])
        return min(candidates)

    df['carry_end'] = df.apply(_carry_end, axis=1)

    # Monthly expansion
    months = pd.period_range(config['beg_date'], config['end_date'],
    freq='M')
    records = []
    for month in months:
        me = month.to_timestamp(how='end')
        valid = df[(df['anndats'] <= me) & (df['carry_end'] > me)].copy()
        valid = valid[valid['fpedats'] > me]
        valid = valid.sort_values(['ticker', 'analyst_id', 'anndats'])
        valid = valid.groupby(['ticker', 'analyst_id']).tail(1)

        disp = valid.groupby('ticker').agg(
            n_analysts=('analyst_id', 'nunique'),
            mean_fcst=('value', 'mean'),
            std_fcst=('value', 'std'),
        ).reset_index()
        disp['month'] = month
        records.append(disp)

    if not records:
        return pd.DataFrame()
    disp_df = pd.concat(records, ignore_index=True)

    # Scaled measures
    disp_df['disp1'] = np.where(
```

```
        disp_df['mean_fcst'].abs() > 0,
        disp_df['std_fcst'] / disp_df['mean_fcst'].abs(), np.nan
    )
    disp_df = disp_df.merge(price_df, on=['ticker', 'month'], how='left')
    disp_df['disp2'] = np.where(
        disp_df['mean_price'] > 0,
        disp_df['std_fcst'] / disp_df['mean_price'], np.nan
    )
    disp_df['disp_raw'] = disp_df['std_fcst']

    out = disp_df[disp_df['n_analysts'] >= min_analysts].copy()
    print(f"DISP: {len(out):,} ticker-months (>= {min_analysts} analysts)")
    print(f"  Mean analysts: {out['n_analysts'].mean():.1f}")
    return out
```

52.4 Scaling Considerations

Following Cheong and Thomas (2011), we note that each scaling choice has pitfalls. DISP1 (scaled by absolute mean forecast) can produce extreme values when the mean forecast approaches zero—common for Vietnamese firms near breakeven. DISP2 (scaled by price) introduces a mechanical negative correlation between price and scaled dispersion. We recommend reporting all three versions (DISP1, DISP2, and unscaled DISP_RAW with ln(Price) as an additional control), and winsorizing DISP1 at the 1st and 99th percentiles.

Caution on Analyst Dispersion in Thin-Coverage Markets

With typical coverage of 5–10 analysts per firm in Vietnam (versus 15–25 in the U.S.), forecast dispersion is estimated with substantially greater noise. A dispersion measure from 3 analysts has a very different sampling distribution than one from 20. Always include the number of analysts as a control and test robustness with varying minimum-analyst thresholds (3, 5, 7).

53 Cross-Sectional Correlations Among DIVOP Proxies

An important empirical question is the degree to which the various DIVOP proxies capture the same underlying construct. If divergence of opinion is a well-defined latent variable, we expect positive correlations among all proxies, though correlations need not be high since each captures a different facet of disagreement.

```
def compute_divop_correlations(merged_df, proxies=None):
    """
    Compute and visualize Spearman correlations among DIVOP proxies.
    We use rank correlations because many proxies are right-skewed.
    """
    if proxies is None:
        proxies = [
            'dto', 'suv', 'total_volatility', 'idio_volatility',
            'baspread', 'amihud_daily', 'disp1', 'disp2'
        ]
    available = [p for p in proxies if p in merged_df.columns]
    data = merged_df[available].dropna()

    n = len(available)
    rho_mat = np.eye(n)
    p_mat = np.zeros((n, n))
    for i in range(n):
        for j in range(i + 1, n):
            rho, p = scipy_stats.spearmanr(
                data[available[i]], data[available[j]]
            )
            rho_mat[i, j] = rho_mat[j, i] = rho
            p_mat[i, j] = p_mat[j, i] = p

    labels = {'dto': 'DTO', 'suv': 'SUV',
              'total_volatility': 'VOL', 'idio_volatility': 'IVOL',
              'baspread': 'SPREAD', 'amihud_daily': 'ILLIQ',
              'disp1': 'DISP1', 'disp2': 'DISP2'}
```

```
    pretty = [labels.get(c, c) for c in available]
    corr_df = pd.DataFrame(rho_mat, index=pretty, columns=pretty)

    # Heatmap
    fig, ax = plt.subplots(figsize=(9, 7))
    mask = np.triu(np.ones_like(corr_df, dtype=bool), k=1)
    sns.heatmap(
        corr_df, mask=mask, annot=True, fmt='.3f',
        cmap='RdBu_r', center=0, vmin=-0.4, vmax=0.7,
        square=True, linewidths=0.5,
        cbar_kws={'shrink': 0.8, 'label': 'Spearman  '}, ax=ax
    )
    ax.set_title('Spearman Correlations Among DIVOP Proxies\n'
                 'Vietnamese Equity Market', fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.savefig('divop_correlations.png', dpi=300, bbox_inches='tight')
    plt.show()

    return corr_df
```

53.0.1 Expected Correlation Patterns

Based on U.S. evidence and theory, we expect:

Table 53.1: Expected correlation structure among DIVOP proxies

Pair	Expected	Rationale
DTO × SUV	High positive	Both capture abnormal volume; SUV refines DTO
VOL × IVOL	High positive	IVOL is a subset of total volatility
SPREAD × ILLIQ	Moderate-high positive	Both capture information asymmetry
Volume × Volatility	Moderate positive	Shalen (1993) links both to belief dispersion
Analyst × Market-based	Weak-moderate positive	Different investor populations

54 Descriptive Statistics and Cross-Sectional Properties

54.1 Summary Statistics

```
def descriptive_statistics(merged_df):
    """Comprehensive descriptive statistics for DIVOP proxies."""
    proxies = {
        'dto': 'Unexplained Volume (DTO)',
        'suv': 'Std Unexplained Volume (SUV)',
        'total_volatility': 'Total Return Volatility',
        'idio_volatility': 'Idiosyncratic Volatility',
        'baspread': 'Bid-Ask Spread',
        'amihud_daily': 'Amihud Illiquidity',
        'disp1': 'Analyst Disp (mean-scaled)',
        'disp2': 'Analyst Disp (price-scaled)',
    }
    avail = {k: v for k, v in proxies.items() if k in merged_df.columns}
    rows = []
    for col, label in avail.items():
        s = merged_df[col].dropna()
        rows.append({
            'Proxy': label, 'N': f'{len(s):,}',
            'Mean': f'{s.mean():.6f}', 'Std': f'{s.std():.6f}',
            'P5': f'{s.quantile(.05):.6f}',
            'Median': f'{s.median():.6f}',
            'P95': f'{s.quantile(.95):.6f}',
            'Skew': f'{s.skew():.2f}',
            'Kurt': f'{s.kurtosis():.2f}',
        })
    stats = pd.DataFrame(rows).set_index('Proxy')
    print("\n" + "=" * 90)
    print("Descriptive Statistics of DIVOP Proxies")
    print("Vietnamese Equity Market, HOSE and HNX")
    print("=" * 90)
```

```
    print(stats.to_string())
    return stats
```

54.2 DIVOP by Firm Characteristics

```
def divop_by_size(merged_df):
    """Mean DIVOP proxies by market-cap quintile."""
    df = merged_df.copy()
    df['mkt_cap'] = df['close'] * df['shares_outstanding']
    df['size_q'] = df.groupby('date')['mkt_cap'].transform(
        lambda x: pd.qcut(x, 5,
            labels=['Q1 Small','Q2','Q3','Q4','Q5 Large'],
            duplicates='drop')
    )
    proxies = ['dto','suv','total_volatility','idio_volatility',
               'baspread','amihud_daily']
    avail = [p for p in proxies if p in df.columns]
    tab = df.groupby('size_q')[avail].mean()
    print("\n=== Mean DIVOP by Size Quintile ===")
    print(tab.to_string(float_format='{:.6f}'.format))
    return tab

def divop_by_exchange(merged_df):
    """Compare mean DIVOP across HOSE and HNX."""
    proxies = ['dto','suv','total_volatility','idio_volatility',
               'baspread','amihud_daily']
    avail = [p for p in proxies if p in merged_df.columns]
    tab = merged_df.groupby('exchange')[avail].mean()
    print("\n=== Mean DIVOP by Exchange ===")
    print(tab.to_string(float_format='{:.6f}'.format))
    return tab
```

54.3 Time-Series Evolution

```
def plot_divop_timeseries(merged_df):
    """Plot monthly cross-sectional median DIVOP with crisis shading."""
    df = merged_df.copy()
```

```
df['ym'] = df['date'].dt.to_period('M')
proxies = ['dto','suv','total_volatility','baspread']
avail = [p for p in proxies if p in df.columns]
monthly = df.groupby('ym')[avail].median()
monthly.index = monthly.index.to_timestamp()

fig, axes = plt.subplots(len(avail), 1,
    figsize=(13, 3.5*len(avail)), sharex=True)
if len(avail) == 1: axes = [axes]

labels = {'dto':'DTO','suv':'SUV',
          'total_volatility':'Volatility','baspread':'Spread'}
colors = ['#1976D2','#388E3C','#F57C00','#D32F2F']

for i, (proxy, ax) in enumerate(zip(avail, axes)):
    ax.plot(monthly.index, monthly[proxy],
            color=colors[i], linewidth=1.3)
    ax.set_ylabel(labels.get(proxy, proxy), fontsize=10)
    ax.grid(True, alpha=0.25)
    for s, e, c in [('2008-01','2009-06','red'),
                    ('2020-01','2020-12','orange'),
                    ('2022-09','2023-06','purple')]:
        ax.axvspan(pd.Timestamp(s), pd.Timestamp(e),
                   alpha=0.1, color=c)

axes[0].set_title(
    'Time-Series of DIVOP Proxies\n'
    'Monthly Cross-Sectional Median, HOSE & HNX',
    fontsize=13, fontweight='bold')
from matplotlib.patches import Patch
axes[-1].legend(handles=[
    Patch(facecolor='red', alpha=.2, label='GFC 2008-09'),
    Patch(facecolor='orange', alpha=.2, label='COVID-19'),
    Patch(facecolor='purple', alpha=.2, label='Bond Crisis 2022-23'),
], loc='upper right', fontsize=8)
plt.tight_layout()
plt.savefig('divop_timeseries.png', dpi=300, bbox_inches='tight')
plt.show()
```

55 Putting It All Together

```
def build_divop_dataset(config):
    """
    Master pipeline: load data, construct all DIVOP proxies,
    merge into a single stock-date panel.
    """
    df = load_daily_data(config)
    df = apply_sample_filters(df, config)
    df = adjust_for_corporate_actions(df)
    calendar = build_trading_calendar(df, config)

    df = compute_dto(df, config)
    suv_df = compute_suv(df, calendar, config)
    vol_df = compute_volatility(df, calendar, config)
    df = compute_spread_and_illiq(df, config)

    # Merge
    base = df[['ticker','date','ret','close','volume',
               'shares_outstanding','exchange','industry_icb',
               'foreign_ownership_pct','turnover',
               'mato','dto','baspread','amihud_daily','limit_hit']].copy()

    if not suv_df.empty:
        base = base.merge(
            suv_df[['ticker','date','suv','predicted_turnover']],
            on=['ticker','date'], how='left')
    if not vol_df.empty:
        base = base.merge(
            vol_df[['ticker','date','total_volatility',
                    'idio_volatility','market_beta']],
            on=['ticker','date'], how='left')

    print(f"\n=== Final DIVOP Dataset ===")
    print(f"Shape: {base.shape}")
    print(f"Tickers: {base['ticker'].nunique()}")
```

```
    return base
```

56 Empirical Applications

56.1 Application 1: DIVOP and the Cross-Section of Returns

The fundamental test of the Miller hypothesis is whether stocks with higher divergence of opinion earn lower subsequent returns. We implement Fama-MacBeth cross-sectional regressions:

$$r_{i,t+1:t+h} = \gamma_{0,t} + \gamma_{1,t} \cdot \text{DIVOP}_{i,t} + \gamma_{2,t}' \mathbf{X}_{i,t} + \varepsilon_{i,t}$$

where $\mathbf{X}_{i,t}$ includes controls for market beta, log market capitalization, and log book-to-market ratio. The Miller hypothesis predicts $\bar{\gamma}_1 < 0$.

```
def fama_macbeth_divop(merged_df, divop_proxy='suv',
                       controls=None, horizon=21):
    """
    Fama-MacBeth cross-sectional regressions.
    Miller predicts gamma_1 < 0; Varian predicts gamma_1 > 0.
    """
    if controls is None:
        controls = ['market_beta', 'log_mktcap']

    df = merged_df.copy()
    df = df.sort_values(['ticker', 'date'])
    df['fwd_ret'] = df.groupby('ticker')['ret'].transform(
        lambda x: x.shift(-1).rolling(horizon).sum().shift(-(horizon-1))
    )
    df['log_mktcap'] = np.log(
        df['close'] * df['shares_outstanding'] + 1
    )

    reg_vars = ['fwd_ret', divop_proxy] + \
               [c for c in controls if c in df.columns]
    df_reg = df[['ticker','date'] + reg_vars].dropna()

    from numpy.linalg import lstsq
```

```
    results = []
    for date, cross in df_reg.groupby('date'):
        if len(cross) < 30: continue
        y = cross['fwd_ret'].values
        X_cols = [divop_proxy] + [c for c in controls if c in cross.columns]
        X = np.column_stack([np.ones(len(cross)), cross[X_cols].values])
        try:
            coefs, _, _, _ = lstsq(X, y, rcond=None)
            results.append({
                'date': date, 'intercept': coefs[0],
                f'gamma_{divop_proxy}': coefs[1], 'n': len(cross),
            })
        except Exception: continue

    fm = pd.DataFrame(results)
    gc = f'gamma_{divop_proxy}'
    mu = fm[gc].mean()
    se = fm[gc].std() / np.sqrt(len(fm))
    t = mu / se

    print(f"\n=== Fama-MacBeth: {divop_proxy} -> "
          f"{horizon}-day fwd returns ===")
    print(f"  Mean gamma: {mu:.6f}, t-stat: {t:.3f}")
    if t < -1.96:   print("  -> Supports Miller (1977)")
    elif t > 1.96:   print("  -> Supports Varian (1985)")
    else:            print("  -> Inconclusive at 5%")
    return fm
```

56.2 Application 2: DIVOP and Earnings Announcements

Following Berkman et al. (2009), we test whether high-DIVOP stocks experience negative abnormal returns around earnings announcements, as uncertainty resolution reduces the optimism premium.

```
def divop_earnings_event(merged_df, ea_dates_df,
                         divop_proxy='suv', window=(-1, 3)):
    """
    Sort stocks into DIVOP quintiles pre-EA, compute CAR in window.
    Miller predicts: Q5 (high DIVOP) has lower CAR than Q1 (low DIVOP).
    """
    df = merged_df.copy()
```

```
    ea = ea_dates_df.copy()

    # Pre-EA DIVOP value (5 days before)
    ea['pre_date'] = ea['ea_date'] - pd.Timedelta(days=5)
    ea = ea.merge(
        df[['ticker','date',divop_proxy]].rename(
            columns={'date':'pre_date'}),
        on=['ticker','pre_date'], how='inner'
    )
    ea['divop_q'] = pd.qcut(
        ea[divop_proxy], 5,
        labels=['Q1 Low','Q2','Q3','Q4','Q5 High'],
        duplicates='drop'
    )

    print(f"\n=== EA Event Study by {divop_proxy} quintile ===")
    print(f"  Window: ({window[0]}, {window[1]}) days")
    print(f"  Miller predicts: Q5 has lower CAR than Q1")
    return ea
```

56.3 Application 3: Composite DIVOP Index via PCA

When a single summary measure of disagreement is needed, PCA on the battery of standardized proxies extracts the common "disagreement factor."

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

def composite_divop_pca(merged_df, proxies=None):
    """Extract first principal component from standardized DIVOP proxies."""
    if proxies is None:
        proxies = ['dto','suv','total_volatility','idio_volatility',
                   'baspread','amihud_daily']
    avail = [p for p in proxies if p in merged_df.columns]
    data = merged_df[['ticker','date'] + avail].dropna()

    scaler = StandardScaler()
    X = scaler.fit_transform(data[avail])

    pca = PCA(n_components=3)
    factors = pca.fit_transform(X)
```

```
    data['divop_composite'] = factors[:, 0]

    # Ensure positive correlation with inputs
    for col in avail:
        if data['divop_composite'].corr(data[col]) < 0:
            data['divop_composite'] *= -1
            break

    loadings = pd.DataFrame(
        pca.components_.T, index=avail,
        columns=['PC1','PC2','PC3']
    )

    print(f"\n=== PCA Composite DIVOP ===")
    print(f"Variance explained: "
          f"{pca.explained_variance_ratio_[:3].round(3)}")
    print(f"\nLoadings:\n{loadings.to_string(float_format='{:.4f}'.format)}")
    return data[['ticker','date','divop_composite']], loadings
```

57 Conclusion and Practical Recommendations

This chapter has provided a comprehensive methodology for constructing seven distinct proxies for divergence of investor opinion adapted to the Vietnamese equity market. We conclude with practical recommendations:

1. Prefer multiple proxies. No single DIVOP measure is without limitations. We recommend constructing and reporting results for at least three proxies spanning different economic channels (volume, volatility, spreads or analyst-based).

2. Account for Vietnam-specific microstructure. Daily price limits, T+2 settlement, foreign ownership constraints, and the order-driven market structure all affect DIVOP properties. Flag limit-hit days, include exchange fixed effects, and control for foreign ownership.

3. Vietnam as a natural laboratory for Miller (1977). The absence of short selling through 2024 and the dominance of retail investors create conditions that closely match Miller's theoretical setting. The introduction of short selling in 2025 creates a natural experiment for examining how relaxation of short-sale constraints affects the DIVOP-return relation.

4. Control for analyst coverage when using DISP measures. With typical coverage of 5–10 analysts per firm, forecast dispersion is estimated with greater noise than in developed markets. Always include the number of analysts as a control variable and conduct robustness checks with varying minimum-analyst thresholds.

5. Consider constructing a composite index. When researchers need a single summary measure of disagreement, the PCA-based composite index described in Chapter 56 provides a principled approach to aggregating information across the individual proxies. The first principal component typically explains 30-50% of the common variation in the battery of DIVOP measures.

6. Winsorize aggressively. Several DIVOP proxies (particularly DISP1, Amihud ILLIQ, and SUV) exhibit extreme outliers in the Vietnamese data. Winsorization at the 1st and 99th percentiles (or even 2nd and 98th for DISP1) is essential for obtaining reliable regression results.

7. Be cautious about causal inference. DIVOP proxies are endogenous, they respond to the same firm characteristics (size, leverage, growth) that also affect returns. Researchers should use appropriate controls, consider instrumental variables where feasible, and be explicit about the limitations of their identification strategy.

The DIVOP framework is particularly relevant for the Vietnamese market at this point in its development. As the market matures toward potential FTSE Emerging Market reclassification, as short selling becomes more widely available, and as institutional investor participation grows, the dynamics of opinion divergence and its pricing implications are likely to evolve significantly. The methodology presented in this chapter provides researchers with the tools to document and analyze these changes as they unfold.

Part V

Ownership, Market Frictions, and International Exposure

58 Institutional Ownership Analytics in Vietnam

58.1 Institutional Ownership in Vietnam: A Distinct Landscape

Vietnam's equity market presents a fundamentally different institutional ownership landscape from the mature markets of the US, Europe, or Japan. Since the Ho Chi Minh City Securities Trading Center (now HOSE) opened on July 28, 2000 with just two listed stocks, the market has grown to over 1,700 listed companies across three exchanges (HOSE, HNX, and UPCOM) with a combined market capitalization exceeding 200 billion USD. Yet the ownership structure remains distinctive in several critical ways:

- **Retail dominance.** Individual investors account for approximately 85% of trading value on Vietnamese exchanges, far exceeding the institutional share. This contrasts sharply with the US, where institutional investors dominate both ownership and trading (Bao Dinh and Tran 2024). The implications for market efficiency, price discovery, and volatility are profound.
- **State ownership legacy.** Vietnam's equitization (privatization) program, initiated under Đổi Mới reforms in 1986, means that the state remains a significant or controlling shareholder in many listed companies. As of 2022, SOEs (firms with state ownership $>$ 50%) account for approximately 30% of total market capitalization despite representing less than 10% of listed firms (X. Huang, Liu, and Shu 2023). State ownership introduces unique agency problems, governance dynamics, and liquidity constraints.
- **Foreign Ownership Limits (FOLs).** Vietnam imposes sector-specific caps on aggregate foreign ownership, typically 49% for most sectors, 30% for banking, and varying limits for aviation, media, and telecommunications. When a stock reaches its FOL, foreign investors can only buy from other foreign sellers, creating a segmented market with distinct pricing dynamics and a well-documented "FOL premium" (X. V. Vo 2015).
- **Disclosure regime.** Unlike the US quarterly 13F filing system, Vietnam's ownership disclosure is event-driven and periodic. Major shareholders (5%) must disclose within 7 business days of crossing thresholds. Annual reports contain detailed shareholder registers. Semi-annual fund reports provide portfolio snapshots. This creates a patchwork of disclosure frequencies that require careful handling.

58.2 Data Infrastructure: DataCore.vn

DataCore.vn is a comprehensive Vietnamese financial data platform that provides academic-grade datasets for the Vietnamese market. Throughout this chapter, we assume all data is sourced exclusively from DataCore.vn, which provides:

Table 58.1: DataCore.vn Data Tables Used in This Chapter

DataCore.vn Dataset	Content	Key Variables
Stock Prices	Daily/monthly OHLCV for HOSE, HNX, UPCOM	`ticker`, `date`, `close`, `adjusted_close`, `volume`, `shares_outstanding`
Ownership Structure	Shareholder composition snapshots	`ticker`, `date`, `shareholder_name`, `shares_held`, `ownership_pct`, `shareholder_type`
Major Shareholders	Detailed 5% holders	`ticker`, `date`, `shareholder_name`, `shares_held`, `is_foreign`, `is_state`, `is_institution`
Corporate Actions	Dividends, stock splits, bonus shares, rights issues	`ticker`, `ex_date`, `action_type`, `ratio`, `record_date`
Company Profile	Sector, exchange, listing date, charter capital	`ticker`, `exchange`, `industry_code`, `listing_date`, `fol_limit`
Financial Statements	Quarterly/annual financials	`ticker`, `period`, `revenue`, `net_income`, `total_assets`, `equity`
Foreign Ownership	Daily foreign ownership tracking	`ticker`, `date`, `foreign_shares`, `foreign_pct`, `fol_limit`, `foreign_room`
Fund Holdings	Semi-annual fund portfolio disclosures	`fund_name`, `report_date`, `ticker`, `shares_held`, `market_value`

```
class DataCoreReader:
    """
    Unified data reader for DataCore.vn datasets.

    Assumes data has been downloaded from DataCore.vn and stored locally.
    Supports both Parquet (recommended for performance) and CSV formats.

    Parameters
```

```
    ----------
    data_dir : str or Path
        Root directory containing DataCore.vn data files
    file_format : str
        'parquet' or 'csv' (default: 'parquet')
    """

    # Expected file names in the data directory
    FILE_MAP = {
        'prices': 'stock_prices',
        'ownership': 'ownership_structure',
        'major_shareholders': 'major_shareholders',
        'corporate_actions': 'corporate_actions',
        'company_profile': 'company_profile',
        'financials': 'financial_statements',
        'foreign_ownership': 'foreign_ownership_daily',
        'fund_holdings': 'fund_holdings',
    }

    def __init__(self, data_dir: Union[str, Path], file_format: str =
     ↪  'parquet'):
        self.data_dir = Path(data_dir)
        self.fmt = file_format
        self._cache = {}

        # Verify data directory exists
        if not self.data_dir.exists():
            raise FileNotFoundError(
                f"Data directory not found: {self.data_dir}\n"
                f"Please download data from DataCore.vn and place it in this
                 ↪  directory."
            )

        print(f"DataCore.vn reader initialized: {self.data_dir}")
        available = [f.stem for f in self.data_dir.glob(f'*.{self.fmt}')]
        print(f"Available datasets: {available}")

    def _read(self, key: str) -> pd.DataFrame:
        """Read and cache a dataset."""
        if key in self._cache:
            return self._cache[key]

        fname = self.FILE_MAP.get(key, key)
```

```
        filepath = self.data_dir / f"{fname}.{self.fmt}"

        if not filepath.exists():
            raise FileNotFoundError(
                f"Dataset not found: {filepath}\n"
                f"Expected file: {fname}.{self.fmt} in {self.data_dir}"
            )

        if self.fmt == 'parquet':
            df = pd.read_parquet(filepath)
        else:
            df = pd.read_csv(filepath, parse_dates=True)

        # Auto-detect and parse date columns
        for col in df.columns:
            if 'date' in col.lower() or col.lower() in ['period', 'ex_date',
             ↪  'record_date']:
                try:
                    df[col] = pd.to_datetime(df[col])
                except (ValueError, TypeError):
                    pass

        self._cache[key] = df
        print(f"Loaded {key}: {len(df):,} rows, {len(df.columns)} columns")
        return df

    @property
    def prices(self) -> pd.DataFrame:
        return self._read('prices')

    @property
    def ownership(self) -> pd.DataFrame:
        return self._read('ownership')

    @property
    def major_shareholders(self) -> pd.DataFrame:
        return self._read('major_shareholders')

    @property
    def corporate_actions(self) -> pd.DataFrame:
        return self._read('corporate_actions')

    @property
```

```
    def company_profile(self) -> pd.DataFrame:
        return self._read('company_profile')

    @property
    def financials(self) -> pd.DataFrame:
        return self._read('financials')

    @property
    def foreign_ownership(self) -> pd.DataFrame:
        return self._read('foreign_ownership')

    @property
    def fund_holdings(self) -> pd.DataFrame:
        return self._read('fund_holdings')

    def clear_cache(self):
        """Clear all cached datasets to free memory."""
        self._cache.clear()

# Initialize reader - adjust path to your local DataCore.vn data
# dc = DataCoreReader('/path/to/datacore_data', file_format='parquet')
```

This chapter proceeds as follows. Section 58.3 builds the complete data pipeline from raw DataCore.vn extracts to clean, analysis-ready datasets, with particular attention to corporate action adjustments. Section 58.4 defines Vietnam's unique ownership taxonomy. Section 58.5 computes institutional ownership ratios, concentration, and breadth for the Vietnamese market. Section 58.6 develops specialized foreign ownership analytics including FOL utilization and room premium. Section 58.7 derives institutional trades from ownership disclosure snapshots. Section 58.8 computes fund-level flows and turnover. Section 58.9 analyzes state ownership dynamics. Section 58.10 introduces network analysis, ML classification, and event-study frameworks. Section 58.11 presents complete empirical applications, and Section 58.12 concludes.

58.3 Data Pipeline

58.3.1 Stock Price Data and Corporate Action Adjustments

Vietnam's equity market is notorious for frequent corporate actions, particularly stock dividends and bonus share issuances, that dramatically alter share counts. A company issuing a 30% stock dividend means every 100 shares become 130 shares, and the reference price adjusts

downward proportionally. Failure to properly adjust historical shares and prices for these events is the single most common source of error in Vietnamese equity research.

```
# ================================================================================
# Step 1: Corporate Action Adjustment Factors
# ================================================================================

def build_adjustment_factors(corporate_actions: pd.DataFrame) -> pd.DataFrame:
    """
    Build cumulative adjustment factors from the corporate actions history.

    In Vietnam, the most common share-altering corporate actions are:
    1. Stock dividends (cổ tức bằng cổ phiếu): e.g., 30% → ratio = 0.30
       Effect: shares × (1 + 0.30), price × (1 / 1.30)
    2. Bonus shares (thưởng cổ phiếu): mechanically identical to stock dividends
    3. Stock splits (chia tách): e.g., 2:1 → ratio = 2.0
       Effect: shares × 2, price × 0.5
    4. Rights issues (phát hành thêm): dilutive, but not all shareholders exercise
       We approximate with the subscription ratio
    5. Reverse splits (gộp cổ phiếu): rare in Vietnam
       Effect: shares ÷ ratio, price × ratio

    We construct a FORWARD-LOOKING cumulative adjustment factor such that:
       adjusted_shares = raw_shares × cum_adj_factor(from_date, to_date)
       adjusted_price = raw_price / cum_adj_factor(from_date, to_date)

    This is analogous to CRSP's cfacshr in the US context.

    Parameters
    ----------
    corporate_actions : pd.DataFrame
        DataCore.vn corporate actions with columns:
        ticker, ex_date, action_type, ratio

        action_type values:
        - 'stock_dividend': ratio = dividend rate (e.g., 0.30 for 30%)
        - 'bonus_shares': ratio = bonus rate (e.g., 0.20 for 20%)
        - 'stock_split': ratio = split factor (e.g., 2.0 for 2:1)
```

```
    - 'reverse_split': ratio = merge factor (e.g., 5.0 for 5:1 merge)
    - 'rights_issue': ratio = subscription rate (e.g., 0.10 for 10:1)
    - 'cash_dividend': ratio = VND per share (no share adjustment needed)

Returns
-------
pd.DataFrame
    Adjustment factors: ticker, ex_date, point_factor, cum_factor
"""
# Filter to share-altering events only
share_events = ['stock_dividend', 'bonus_shares', 'stock_split',
                'reverse_split', 'rights_issue']
ca = corporate_actions[
    corporate_actions['action_type'].isin(share_events)
].copy()

if len(ca) == 0:
    print("No share-altering corporate actions found.")
    return pd.DataFrame(columns=['ticker', 'ex_date', 'point_factor',
     ↪  'cum_factor'])

# Compute point adjustment factor for each event
def compute_point_factor(row):
    atype = row['action_type']
    ratio = row['ratio']

    if atype in ['stock_dividend', 'bonus_shares']:
        # 30% stock dividend: 100 shares → 130 shares
        return 1 + ratio
    elif atype == 'stock_split':
        # 2:1 split: 100 shares → 200 shares
        return ratio
    elif atype == 'reverse_split':
        # 5:1 reverse: 500 shares → 100 shares
        return 1.0 / ratio
    elif atype == 'rights_issue':
        # Approximate: assume all rights exercised
        # In practice, this overestimates the adjustment
        return 1 + ratio
    else:
        return 1.0

ca['point_factor'] = ca.apply(compute_point_factor, axis=1)
```

```
    # Sort chronologically within each ticker
    ca = ca.sort_values(['ticker', 'ex_date']).reset_index(drop=True)

    # Cumulative factor: product of all point factors from listing to date
    # This gives us a running "total adjustment" for each ticker
    ca['cum_factor'] = ca.groupby('ticker')['point_factor'].cumprod()

    # Summary statistics
    n_tickers = ca['ticker'].nunique()
    n_events = len(ca)
    avg_events = n_events / n_tickers if n_tickers > 0 else 0

    print(f"Corporate action adjustment factors built:")
    print(f"  Tickers with adjustments: {n_tickers:,}")
    print(f"  Total share-altering events: {n_events:,}")
    print(f"  Average events per ticker: {avg_events:.1f}")
    print(f"\nEvent type distribution:")
    print(ca['action_type'].value_counts().to_string())

    return ca[['ticker', 'ex_date', 'action_type', 'ratio',
               'point_factor', 'cum_factor']]


def adjust_shares(shares: float, ticker: str, from_date, to_date,
                  adj_factors: pd.DataFrame) -> float:
    """
    Adjust a share count from one date to another for corporate actions.

    Example: If a company had a 30% stock dividend with ex_date between
    from_date and to_date, then 1000 shares at from_date = 1300 shares
    at to_date.

    Parameters
    ----------
    shares : float
        Number of shares at from_date
    ticker : str
        Stock ticker
    from_date, to_date : pd.Timestamp
        Period for adjustment
    adj_factors : pd.DataFrame
        Output of build_adjustment_factors()
```

```
    Returns
    -------
    float
        Adjusted shares at to_date
    """
    events = adj_factors[
        (adj_factors['ticker'] == ticker) &
        (adj_factors['ex_date'] > pd.Timestamp(from_date)) &
        (adj_factors['ex_date'] <= pd.Timestamp(to_date))
    ]

    if len(events) == 0:
        return shares

    total_factor = events['point_factor'].prod()
    return shares * total_factor


# Example usage:
# adj_factors = build_adjustment_factors(dc.corporate_actions)
```

! The Stock Dividend Problem in Vietnam

Vietnamese companies issue stock dividends with remarkable frequency, many growth companies do so 2-3 times per year. Consider **Vinhomes (VHM)** or **FPT Corporation**: their share counts may double or triple over a 5-year period purely from stock dividends. If you compare raw ownership shares from 2019 to 2024 without adjustment, you will obtain nonsensical ownership ratios. **Every time-series analysis of Vietnamese ownership data must use adjusted shares.** This is the Vietnamese equivalent of the CRSP cfacshr adjustment factor problem in US data, but more severe because the events are more frequent and larger in magnitude.

```
# ============================================================================
# Step 2: Process Stock Price Data
# ============================================================================

def process_price_data(prices: pd.DataFrame,
                       adj_factors: pd.DataFrame,
```

```
                    company_profile: pd.DataFrame) -> pd.DataFrame:
    """
    Process DataCore.vn stock price data:
    1. Align dates to month-end and quarter-end
    2. Merge company metadata (exchange, sector, FOL limit)
    3. Compute adjusted prices and shares outstanding
    4. Compute market capitalization
    5. Create quarter-end snapshots

    Parameters
    ----------
    prices : pd.DataFrame
        Daily/monthly price data from DataCore.vn
    adj_factors : pd.DataFrame
        Corporate action adjustment factors
    company_profile : pd.DataFrame
        Company metadata including exchange, sector, FOL

    Returns
    -------
    pd.DataFrame
        Quarter-end processed stock data
    """
    df = prices.copy()

    # Standardize date
    df['date'] = pd.to_datetime(df['date'])
    df['month_end'] = df['date'] + pd.offsets.MonthEnd(0)
    df['quarter_end'] = df['date'] + pd.offsets.QuarterEnd(0)

    # Merge company profile
    profile_cols = ['ticker', 'exchange', 'industry_code', 'fol_limit',
                    'listing_date', 'company_name']
    profile_cols = [c for c in profile_cols if c in company_profile.columns]
    df = df.merge(company_profile[profile_cols], on='ticker', how='left')

    # Build cumulative adjustment factor for each ticker-date
    # For each observation, compute the total adjustment from listing to that
     ↪  date
    df = df.sort_values(['ticker', 'date'])

    # Merge adjustment events
    # For each ticker-date, find the cumulative factor as of that date
```

```
    def get_cum_factor_at_date(group):
        ticker = group.name
        ticker_adj = adj_factors[adj_factors['ticker'] == ticker].copy()

        if len(ticker_adj) == 0:
            group['cum_adj_factor'] = 1.0
            return group

        # For each date, find cumulative factor (product of all events up to
         ↪  that date)
        group = group.sort_values('date')
        group['cum_adj_factor'] = 1.0

        for _, event in ticker_adj.iterrows():
            mask = group['date'] >= event['ex_date']
            group.loc[mask, 'cum_adj_factor'] *= event['point_factor']

        return group

    df = df.groupby('ticker', group_keys=False).apply(get_cum_factor_at_date)

    # Adjusted price and shares
    # adjusted_close should already be provided by DataCore.vn
    # But we compute our own for consistency
    if 'adjusted_close' not in df.columns:
        df['adjusted_close'] = df['close'] / df['cum_adj_factor']

    # Adjusted shares outstanding
    df['adjusted_shares'] = df['shares_outstanding'] * df['cum_adj_factor']

    # Market capitalization (in billion VND)
    df['market_cap'] = df['close'] * df['shares_outstanding'] / 1e9

    # Monthly returns
    df = df.sort_values(['ticker', 'date'])
    df['ret'] = df.groupby('ticker')['adjusted_close'].pct_change()

    # Keep quarter-end observations
    # For daily data: keep last trading day of each quarter
    df_quarterly = (df.sort_values(['ticker', 'quarter_end', 'date'])
                      .groupby(['ticker', 'quarter_end'])
                      .last()
                      .reset_index())
```

```
    print(f"Processed price data:")
    print(f"  Total records (daily): {len(df):,}")
    print(f"  Quarter-end records: {len(df_quarterly):,}")
    print(f"  Unique tickers: {df_quarterly['ticker'].nunique():,}")
    print(f"  Date range: {df_quarterly['quarter_end'].min()} to "
          f"{df_quarterly['quarter_end'].max()}")
    print(f"\nExchange distribution:")
    print(df_quarterly.groupby('exchange')['ticker'].nunique().to_string())

    return df_quarterly

# prices_q = process_price_data(dc.prices, adj_factors, dc.company_profile)
```

58.3.2 Ownership Structure Data

Vietnamese ownership data captures the composition of shareholders as disclosed in annual reports, semi-annual reports, and event-driven disclosures. The key distinction from US 13F data is that Vietnamese disclosures provide a **complete ownership decomposition**, not just institutional long positions, but the full breakdown into state, institutional, foreign, and individual ownership.

```
# ============================================================================
# Step 3: Process Ownership Structure Data
# ============================================================================

class OwnershipType:
    """
    Vietnam's ownership taxonomy.

    Unlike the US where 13F captures only institutional long positions,
    Vietnamese disclosure provides a complete ownership decomposition.
    We classify shareholders into five mutually exclusive categories.
    """
    STATE = 'state'                    # Nhà nước (government entities, SOE parents)
    FOREIGN_INST = 'foreign_inst'      # Tổ chức nước ngoài
    DOMESTIC_INST = 'domestic_inst'    # Tổ chức trong nước (non-state)
    INDIVIDUAL = 'individual'          # Cá nhân
```

```
    TREASURY = 'treasury'              # Cổ phiếu quỹ

    ALL_TYPES = [STATE, FOREIGN_INST, DOMESTIC_INST, INDIVIDUAL, TREASURY]
    INSTITUTIONAL = [STATE, FOREIGN_INST, DOMESTIC_INST]
    FOREIGN = [FOREIGN_INST]  # Can be expanded if foreign individuals are
 ↪  tracked


def classify_shareholders(ownership: pd.DataFrame) -> pd.DataFrame:
    """
    Classify shareholders into Vietnam's ownership taxonomy.

    DataCore.vn may provide a `shareholder_type` field, but naming
    conventions vary. This function standardizes the classification
    using a combination of provided flags and name-based heuristics.

    The classification challenge in Vietnam (noted by @huang2023factors):
    DataCore.vn may not always cleanly separate institution types, so we
    use a cascading approach:
    1. Use explicit flags (is_state, is_foreign, is_institution) if available
    2. Apply name-based heuristics for Vietnamese entity names
    3. Default to 'individual' for unclassified shareholders

    Parameters
    ----------
    ownership : pd.DataFrame
        Raw ownership data from DataCore.vn

    Returns
    -------
    pd.DataFrame
        Ownership data with standardized `owner_type` column
    """
    df = ownership.copy()

    # --- Method 1: Use explicit flags if available ---
    if all(col in df.columns for col in ['is_state', 'is_foreign',
     ↪  'is_institution']):
        conditions = [
            (df['is_state'] == True),
            (df['is_foreign'] == True) & (df['is_institution'] == True),
            (df['is_foreign'] == True) & (df['is_institution'] != True),
            (df['is_institution'] == True) & (df['is_state'] != True) &
```

```
            (df['is_foreign'] != True),
        ]
        choices = [
            OwnershipType.STATE,
            OwnershipType.FOREIGN_INST,
            OwnershipType.FOREIGN_INST,  # Foreign individuals often grouped
            OwnershipType.DOMESTIC_INST,
        ]
        df['owner_type'] = np.select(conditions, choices,
                                     default=OwnershipType.INDIVIDUAL)

    # --- Method 2: Name-based heuristics ---
    elif 'shareholder_name' in df.columns:
        name = df['shareholder_name'].str.lower().fillna('')

        # State entities: government ministries, SCIC, state corporations
        state_keywords = [
            'bộ tài chính', 'tổng công ty đầu tư', 'scic',
            'ủy ban nhân dân', 'nhà nước', 'state capital',
            'tổng công ty', 'vốn nhà nước', 'bộ công thương',
            'bộ quốc phòng', 'bộ giao thông', 'vinashin',
        ]
        is_state = name.apply(
            lambda x: any(kw in x for kw in state_keywords)
        )

        # Foreign entities: common fund names, foreign company patterns
        foreign_keywords = [
            'fund', 'investment', 'capital', 'limited', 'ltd', 'inc',
            'corporation', 'holdings', 'asset management', 'pte',
            'gmbh', 'management', 'partners', 'advisors',
            'dragon capital', 'vinacapital', 'templeton',
            'blackrock', 'jpmorgan', 'samsung', 'mirae',
        ]
        # Also check for non-Vietnamese characters as a heuristic
        is_foreign_name = name.apply(
            lambda x: any(kw in x for kw in foreign_keywords)
        )

        # Domestic institutions: Vietnamese bank, securities, insurance names
        domestic_inst_keywords = [
            'ngân hàng', 'chứng khoán', 'bảo hiểm', 'quỹ đầu tư',
            'công ty quản lý', 'bảo việt', 'techcombank', 'vietcombank',
```

```
            'bidv', 'vietinbank', 'vpbank', 'mb bank', 'ssi', 'hsc',
            'vcsc', 'vndirect', 'fpt capital', 'manulife',
        ]
        is_domestic_inst = name.apply(
            lambda x: any(kw in x for kw in domestic_inst_keywords)
        )

        # Treasury shares
        is_treasury = name.str.contains('cổ phiếu quỹ|treasury', case=False)

        # Apply classification cascade
        df['owner_type'] = OwnershipType.INDIVIDUAL  # Default
        df.loc[is_domestic_inst, 'owner_type'] = OwnershipType.DOMESTIC_INST
        df.loc[is_foreign_name, 'owner_type'] = OwnershipType.FOREIGN_INST
        df.loc[is_state, 'owner_type'] = OwnershipType.STATE
        df.loc[is_treasury, 'owner_type'] = OwnershipType.TREASURY

    # --- Method 3: Use shareholder_type directly ---
    elif 'shareholder_type' in df.columns:
        type_map = {
            'state': OwnershipType.STATE,
            'foreign_institution': OwnershipType.FOREIGN_INST,
            'foreign_individual': OwnershipType.FOREIGN_INST,
            'domestic_institution': OwnershipType.DOMESTIC_INST,
            'individual': OwnershipType.INDIVIDUAL,
            'treasury': OwnershipType.TREASURY,
        }
        df['owner_type'] = df['shareholder_type'].str.lower().map(type_map)
        df['owner_type'] = df['owner_type'].fillna(OwnershipType.INDIVIDUAL)

    else:
        raise ValueError(
            "Cannot classify shareholders. Expected one of:\n"
            "  1. Columns: is_state, is_foreign, is_institution\n"
            "  2. Column: shareholder_name (for heuristic classification)\n"
            "  3. Column: shareholder_type (pre-classified)"
        )

    # Summary
    print("Ownership classification results:")
    print(df['owner_type'].value_counts().to_string())

    return df
```

```
# ownership_classified = classify_shareholders(dc.ownership)
```

58.4 Vietnam's Ownership Taxonomy

58.4.1 The Five Ownership Categories

Vietnam's ownership structure is decomposed into five mutually exclusive categories that together sum to 100% of shares outstanding:

Table 58.2: Vietnam's Ownership Taxonomy

Category	Vietnamese Term	Description	Typical Share (2020s)
State	Sở hữu Nhà nước	Government entities, SCIC, SOE parent companies	~15-25% of market cap
Foreign Institutional	Tổ chức nước ngoài	Foreign funds, banks, corporations	~15-20%
Domestic Institutional	Tổ chức trong nước	Vietnamese funds, banks, insurance, securities firms	~5-10%
Individual	Cá nhân	Retail investors (both Vietnamese and foreign individuals)	~55-65%
Treasury	Cổ phiếu quỹ	Company's own repurchased shares	~0-2%

This taxonomy differs fundamentally from the US 13F framework in several ways:

1. **Completeness:** We observe 100% of ownership, not just institutional long positions above $100 million AUM.
2. **State as a category:** State ownership is a first-class analytical category, not subsumed under "All Others" as in the LSEG type code system.
3. **Individual visibility:** We observe aggregate individual ownership directly, whereas in the US, individual ownership is merely the residual (100% − institutional ownership).

4. **No short position ambiguity:** Vietnam's market has very limited short-selling infrastructure, so ownership data genuinely represents long positions.

```
# ================================================================================
# Step 4: Compute Ownership Decomposition
# ================================================================================

def compute_ownership_decomposition(ownership: pd.DataFrame,
                                    prices_q: pd.DataFrame) -> pd.DataFrame:
    """
    Compute the full ownership decomposition for each stock at each
    disclosure date.

    For each stock-date combination, aggregates shares held by each
    ownership category and computes ownership ratios relative to
    total shares outstanding.

    Parameters
    ----------
    ownership : pd.DataFrame
        Classified ownership data (output of classify_shareholders)
    prices_q : pd.DataFrame
        Quarter-end price data with shares_outstanding

    Returns
    -------
    pd.DataFrame
        Stock-period level ownership decomposition with columns for
        each ownership type's share count and percentage
    """
    # Aggregate shares by ticker, date, and owner type
    agg = (ownership.groupby(['ticker', 'date', 'owner_type'])['shares_held']
                    .sum()
                    .reset_index())

    # Pivot to wide format: one column per ownership type
    wide = agg.pivot_table(
        index=['ticker', 'date'],
        columns='owner_type',
        values='shares_held',
        fill_value=0
```

```
    ).reset_index()

    # Rename columns
    type_cols = [c for c in wide.columns if c in OwnershipType.ALL_TYPES]
    rename_map = {t: f'shares_{t}' for t in type_cols}
    wide = wide.rename(columns=rename_map)

    # Total institutional shares
    inst_cols = [f'shares_{t}' for t in OwnershipType.INSTITUTIONAL
                 if f'shares_{t}' in wide.columns]
    wide['shares_institutional'] = wide[inst_cols].sum(axis=1)

    # Total foreign shares (for FOL tracking)
    foreign_cols = [f'shares_{t}' for t in OwnershipType.FOREIGN
                    if f'shares_{t}' in wide.columns]
    wide['shares_foreign_total'] = wide[foreign_cols].sum(axis=1)

    # Align with quarter-end dates for merging with price data
    wide['quarter_end'] = wide['date'] + pd.offsets.QuarterEnd(0)

    # Merge with price data to get shares outstanding
    merged = wide.merge(
        prices_q[['ticker', 'quarter_end', 'shares_outstanding',
                  'adjusted_shares', 'market_cap', 'exchange',
                  'industry_code', 'fol_limit', 'close']],
        on=['ticker', 'quarter_end'],
        how='left'
    )

    # Compute ownership ratios
    tso = merged['shares_outstanding']
    for col in merged.columns:
        if col.startswith('shares_') and col != 'shares_outstanding':
            ratio_col = col.replace('shares_', 'pct_')
            merged[ratio_col] = merged[col] / tso
            merged.loc[tso <= 0, ratio_col] = np.nan

    # Derived measures
    merged['pct_free_float'] = 1 - merged.get('pct_state', 0) -
↪   merged.get('pct_treasury', 0)

    # SOE flag: state ownership > 50%
    merged['is_soe'] = (merged.get('pct_state', 0) > 0.50).astype(int)
```

```
    # FOL utilization
    if 'fol_limit' in merged.columns and 'pct_foreign_total' in
     ↪  merged.columns:
        merged['fol_utilization'] = merged['pct_foreign_total'] /
↪   merged['fol_limit']
        merged['foreign_room'] = merged['fol_limit'] -
↪   merged['pct_foreign_total']
        merged.loc[merged['fol_limit'] <= 0, ['fol_utilization',
↪   'foreign_room']] = np.nan

    # Number of institutional owners (breadth)
    n_owners =
↪   (ownership[ownership['owner_type'].isin(OwnershipType.INSTITUTIONAL)]
                .groupby(['ticker', 'date'])['shareholder_name']
                .nunique()
                .reset_index()
                .rename(columns={'shareholder_name': 'n_inst_owners'}))

    n_foreign_owners = (ownership[ownership['owner_type'] ==
↪   OwnershipType.FOREIGN_INST]
                        .groupby(['ticker', 'date'])['shareholder_name']
                        .nunique()
                        .reset_index()
                        .rename(columns={'shareholder_name':
↪   'n_foreign_owners'}))

    merged = merged.merge(n_owners, on=['ticker', 'date'], how='left')
    merged = merged.merge(n_foreign_owners, on=['ticker', 'date'],
↪   how='left')
    merged[['n_inst_owners', 'n_foreign_owners']] = (
        merged[['n_inst_owners', 'n_foreign_owners']].fillna(0)
    )

    print(f"Ownership decomposition computed:")
    print(f"  Stock-period observations: {len(merged):,}")
    print(f"  Unique tickers: {merged['ticker'].nunique():,}")
    print(f"\nMean ownership structure:")
    pct_cols = [c for c in merged.columns if c.startswith('pct_')]
    print(merged[pct_cols].mean().round(4).to_string())

    return merged
```

```
# ownership_decomp = compute_ownership_decomposition(
#     ownership_classified, prices_q
# )
```

58.5 Institutional Ownership Measures

58.5.1 Ownership Ratio

The **Institutional Ownership Ratio (IOR)** for stock i at time t in Vietnam is:

$$IOR_{i,t} = \frac{S_{i,t}^{state} + S_{i,t}^{foreign_inst} + S_{i,t}^{domestic_inst}}{TSO_{i,t}} \tag{58.1}$$

where $S_{i,t}^{type}$ denotes adjusted shares held by each ownership category and $TSO_{i,t}$ is total shares outstanding. Unlike the US where the IOR can exceed 100% due to long-only reporting and short selling, the Vietnamese IOR is bounded by construction in $[0, 1]$ because we observe the complete ownership decomposition.

We also compute category-specific ownership ratios:

$$\begin{aligned} IOR_{i,t}^{foreign} &= \frac{S_{i,t}^{foreign_inst}}{TSO_{i,t}}, \\ IOR_{i,t}^{state} &= \frac{S_{i,t}^{state}}{TSO_{i,t}}, \\ IOR_{i,t}^{domestic} &= \frac{S_{i,t}^{domestic_inst}}{TSO_{i,t}} \end{aligned} \tag{58.2}$$

58.5.2 Concentration: Herfindahl-Hirschman Index

The **Institutional Ownership Concentration** via the Herfindahl-Hirschman Index is:

$$IOC_{i,t}^{HHI} = \sum_{j=1}^{N_{i,t}} \left(\frac{S_{i,j,t}}{\sum_{k=1}^{N_{i,t}} S_{i,k,t}} \right)^2 \tag{58.3}$$

In Vietnam, the HHI is particularly informative because it captures the dominance of state shareholders. A company where the government holds 65% will have a mechanically high HHI even if the remaining 35% is diversely held.

We therefore compute **separate HHI measures** for different ownership categories:

$$HHI_{i,t}^{total} = \sum_{j} w_{i,j,t}^2, \quad HHI_{i,t}^{non-state} = \sum_{j \notin state} \left(\frac{S_{i,j,t}}{\sum_{k \notin state} S_{i,k,t}} \right)^2 \tag{58.4}$$

The non-state HHI is more comparable to the US institutional HHI, as it captures concentration among market-driven investors.

58.5.3 Breadth of Ownership

Following Joseph Chen, Hong, and Stein (2002), **Institutional Breadth** ($N_{i,t}$) is the number of institutional investors holding stock i in period t. The **Change in Breadth** is:

$$\Delta Breadth_{i,t} = \frac{N_{i,t}^{cont} - N_{i,t-1}^{cont}}{TotalInstitutions_{t-1}} \tag{58.5}$$

where $N_{i,t}^{cont}$ counts only institutions that appear in the disclosure universe in both periods t and $t-1$, following the Lehavy and Sloan (2008) algorithm. This adjustment is particularly important in Vietnam where:

- New funds launch frequently (especially ETFs tracking VN30)
- Foreign funds enter and exit the market
- Domestic securities firms consolidate or spin off asset management divisions

```
# ============================================================================
# Step 5: Compute All IO Metrics
# ============================================================================

def compute_io_metrics_vietnam(ownership: pd.DataFrame,
                               ownership_decomp: pd.DataFrame,
                               adj_factors: pd.DataFrame) -> pd.DataFrame:
    """
    Compute security-level institutional ownership metrics adapted for
    Vietnam.
```

```
    Computes:
    1. Ownership ratios by category (state, foreign, domestic inst,
↪   individual)
    2. HHI concentration (total, non-state, foreign-only)
    3. Number of institutional owners (total, foreign, domestic)
    4. Change in breadth (Lehavy-Sloan adjusted)
    5. FOL-related metrics (utilization, room, near-cap indicator)

    Parameters
    ----------
    ownership : pd.DataFrame
        Classified ownership data with individual shareholder records
    ownership_decomp : pd.DataFrame
        Aggregated ownership decomposition (output of
↪   compute_ownership_decomposition)
    adj_factors : pd.DataFrame
        Corporate action adjustment factors

    Returns
    -------
    pd.DataFrame
        Stock-period level metrics
    """
    # Start with the ownership decomposition
    metrics = ownership_decomp.copy()

    # --- HHI Concentration ---
    # Total HHI: across all institutional shareholders
    inst_ownership = ownership[
        ownership['owner_type'].isin(OwnershipType.INSTITUTIONAL)
    ].copy()

    def compute_hhi_group(group):
        """Compute HHI for a group of shareholders."""
        total = group['shares_held'].sum()
        if total <= 0:
            return np.nan
        weights = group['shares_held'] / total
        return (weights ** 2).sum()

    # Total institutional HHI
    hhi_total = (inst_ownership.groupby(['ticker', 'date'])
                               .apply(compute_hhi_group)
```

```
                              .reset_index(name='hhi_institutional'))
    metrics = metrics.merge(hhi_total, on=['ticker', 'date'], how='left')

    # Non-state HHI (exclude state shareholders)
    non_state = ownership[
        ownership['owner_type'].isin([OwnershipType.FOREIGN_INST,
                                      OwnershipType.DOMESTIC_INST])
    ]
    hhi_nonstate = (non_state.groupby(['ticker', 'date'])
                             .apply(compute_hhi_group)
                             .reset_index(name='hhi_non_state'))
    metrics = metrics.merge(hhi_nonstate, on=['ticker', 'date'], how='left')

    # Foreign-only HHI
    foreign_only = ownership[ownership['owner_type'] ==
↪   OwnershipType.FOREIGN_INST]
    hhi_foreign = (foreign_only.groupby(['ticker', 'date'])
                               .apply(compute_hhi_group)
                               .reset_index(name='hhi_foreign'))
    metrics = metrics.merge(hhi_foreign, on=['ticker', 'date'], how='left')

    # --- Change in Breadth (Lehavy-Sloan Algorithm) ---
    metrics = metrics.sort_values(['ticker', 'date'])

    # Get list of all institutions filing in each period
    inst_by_period = (inst_ownership.groupby('date')['shareholder_name']
                                    .apply(set)
                                    .to_dict())

    # For each stock-period: count continuing institutions
    def compute_breadth_change(group):
        group = group.sort_values('date').reset_index(drop=True)
        group['dbreadth'] = np.nan

        for i in range(1, len(group)):
            current_date = group.loc[i, 'date']
            prev_date = group.loc[i-1, 'date']

            # Institutions in universe for both periods
            current_universe = inst_by_period.get(current_date, set())
            prev_universe = inst_by_period.get(prev_date, set())
            continuing_universe = current_universe & prev_universe
```

```
            if len(prev_universe) == 0:
                continue

            # Count continuing institutions holding this stock in each period
            ticker = group.loc[i, 'ticker']

            current_holders = set(
                inst_ownership[
                    (inst_ownership['ticker'] == ticker) &
                    (inst_ownership['date'] == current_date)
                ]['shareholder_name']
            )
            prev_holders = set(
                inst_ownership[
                    (inst_ownership['ticker'] == ticker) &
                    (inst_ownership['date'] == prev_date)
                ]['shareholder_name']
            )

            # Count only continuing institutions
            n_current_cont = len(current_holders & continuing_universe)
            n_prev_cont = len(prev_holders & continuing_universe)

            group.loc[i, 'dbreadth'] = (
                (n_current_cont - n_prev_cont) / len(prev_universe)
            )

        return group

    metrics = metrics.groupby('ticker',
    group_keys=False).apply(compute_breadth_change)

    # --- FOL Indicators ---
    if 'fol_utilization' in metrics.columns:
        metrics['near_fol_cap'] = (metrics['fol_utilization'] >
    0.90).astype(int)
        metrics['at_fol_cap'] = (metrics['fol_utilization'] >
    0.98).astype(int)

    print(f"IO metrics computed for Vietnam:")
    print(f"  Observations: {len(metrics):,}")
    print(f"\nKey metric distributions:")
    summary_cols = ['pct_institutional', 'pct_state', 'pct_foreign_total',
```

```
                    'hhi_institutional', 'n_inst_owners', 'dbreadth']
    summary_cols = [c for c in summary_cols if c in metrics.columns]
    print(metrics[summary_cols].describe().round(4).to_string())

    return metrics

# io_metrics = compute_io_metrics_vietnam(
#     ownership_classified, ownership_decomp, adj_factors
# )
```

58.5.4 Time Series Visualization

58.6 Foreign Ownership Dynamics

58.6.1 Foreign Ownership Limits and the FOL Premium

Vietnam's Foreign Ownership Limits create a unique market segmentation. When a stock reaches its FOL, the only way for a new foreign investor to buy is if an existing foreign holder sells. This creates a de facto "foreign-only" market for FOL-constrained stocks, with documented price premiums (X. V. Vo 2015).

The **FOL Utilization Ratio** for stock i at time t is:

$$FOL_Util_{i,t} = \frac{ForeignOwnership_{i,t}}{FOL_Limit_i} \tag{58.6}$$

Stocks are classified by FOL proximity (Table 58.4).

Table 58.4: FOL Proximity Zones

FOL Zone	Utilization Range	Market Implication
Green	$< 50\%$	Ample foreign room; normal trading
Yellow	50-80%	Moderate room; some foreign interest pressure
Orange	80-95%	Limited room; foreign premium emerging
Red	95-100%	Near cap; significant foreign premium
Capped	100%	At limit; foreign-only secondary market

```
def plot_ownership_timeseries_vietnam(metrics: pd.DataFrame):
    """
    Create publication-quality time series plots of Vietnamese
    ownership structure evolution.
    """
    fig, axes = plt.subplots(3, 1, figsize=(12, 14))

    # Aggregate across all stocks (market-cap weighted)
    ts = metrics.groupby('quarter_end').apply(
        lambda g: pd.Series({
            'pct_state': np.average(g['pct_state'].fillna(0),
                                    weights=g['market_cap'].fillna(1)),
            'pct_foreign': np.average(g['pct_foreign_total'].fillna(0),
                                      weights=g['market_cap'].fillna(1)),
            'pct_domestic_inst': np.average(g['pct_domestic_inst'].fillna(0),
             ↪

 ↪  weights=g['market_cap'].fillna(1)),
            'pct_individual': np.average(g['pct_individual'].fillna(0),
                                         weights=g['market_cap'].fillna(1)),
            'n_stocks': g['ticker'].nunique(),
            'total_mktcap': g['market_cap'].sum(),
            'median_n_inst': g['n_inst_owners'].median(),
            'median_hhi': g['hhi_institutional'].median(),
            'pct_soe': g['is_soe'].mean(),
        })
    ).reset_index()

    # ---- Panel A: Ownership Composition (Stacked Area) ----
    ax = axes[0]
    dates = ts['quarter_end']
    ax.stackplot(dates,
                 ts['pct_state'] * 100,
                 ts['pct_foreign'] * 100,
                 ts['pct_domestic_inst'] * 100,
                 ts['pct_individual'] * 100,
                 labels=['State', 'Foreign Institutional',
                         'Domestic Institutional', 'Individual'],
                 colors=[OWNER_COLORS['State'], OWNER_COLORS['Foreign
 ↪  Institutional'],
                         OWNER_COLORS['Domestic Institutional'],
 ↪  OWNER_COLORS['Individual']],
                 alpha=0.8)
    ax.set_ylabel('Ownership Share (%)')
    ax.set_title('Panel A: Ownership Composition of Vietnamese Listed
 ↪  Companies '
                 '(Market-Cap Weighted)')
    ax.legend(loc='upper right', frameon=True, framealpha=0.9)
    ax.set_ylim(0, 100)

    # ---- Panel B: Institutional Ownership by Component ----
    ax = axes[1]
    ax.plot(dates, ts['pct_state'] * 100, label='State',
            color=OWNER_COLORS['State'], linewidth=2)
```

```
def plot_io_by_exchange_size(metrics: pd.DataFrame):
    """Plot IO ratios by exchange and size quintile."""
    df = metrics[metrics['market_cap'].notna() & (metrics['market_cap'] >
 ↪  0)].copy()

    # Size quintiles within each quarter
    df['size_quintile'] = df.groupby('quarter_end')['market_cap'].transform(
        lambda x: pd.qcut(x, 5, labels=['Q1\n(Small)', 'Q2', 'Q3', 'Q4',
         ↪  'Q5\n(Large)'],
                          duplicates='drop')
    )

    fig, axes = plt.subplots(1, 3, figsize=(15, 5), sharey=True)

    metrics_to_plot = [
        ('pct_institutional', 'Total Institutional'),
        ('pct_foreign_total', 'Foreign Institutional'),
        ('pct_state', 'State'),
    ]

    for ax, (col, title) in zip(axes, metrics_to_plot):
        for exchange, color in EXCHANGE_COLORS.items():
            data = df[df['exchange'] == exchange]
            if len(data) == 0:
                continue
            means = data.groupby('size_quintile')[col].mean() * 100
            ax.bar(np.arange(len(means)) +
 ↪  list(EXCHANGE_COLORS.keys()).index(exchange) * 0.25,
                   means, width=0.25, label=exchange, color=color, alpha=0.8)

        ax.set_title(title)
        ax.set_xlabel('Size Quintile')
        if ax == axes[0]:
            ax.set_ylabel('Mean Ownership (%)')
        ax.legend()
        ax.set_xticks(np.arange(5) + 0.25)
        ax.set_xticklabels(['Q1\n(Small)', 'Q2', 'Q3', 'Q4', 'Q5\n(Large)'])

    plt.tight_layout()
    plt.savefig('fig_io_by_exchange_size.png', dpi=300, bbox_inches='tight')
    plt.show()

# plot_io_by_exchange_size(io_metrics)
```

Figure 58.2

Table 58.3: Summary Statistics of Ownership Structure in Vietnam by Size Quintile and Exchange (Pooled 2010-2024)

```
def tabulate_io_summary(metrics: pd.DataFrame, start_year: int = 2010) ->
↪  pd.DataFrame:
    """
    Create publication-quality summary table of Vietnamese ownership
    structure by firm size.
    """
    df = metrics[
        (metrics['quarter_end'].dt.year >= start_year) &
        (metrics['market_cap'].notna()) & (metrics['market_cap'] > 0)
    ].copy()

    df['size_quintile'] = df.groupby('quarter_end')['market_cap'].transform(
        lambda x: pd.qcut(x, 5, labels=['Q1 (Small)', 'Q2', 'Q3', 'Q4', 'Q5
        ↪  (Large)'],
                          duplicates='drop')
    )

    table = df.groupby('size_quintile').agg(
        N=('ticker', 'count'),
        Mean_MktCap=('market_cap', 'mean'),
        Mean_IO_Total=('pct_institutional', 'mean'),
        Mean_State=('pct_state', 'mean'),
        Mean_Foreign=('pct_foreign_total', 'mean'),
        Mean_Domestic_Inst=('pct_domestic_inst', 'mean'),
        Mean_Individual=('pct_individual', 'mean'),
        Median_N_Owners=('n_inst_owners', 'median'),
        Median_HHI=('hhi_institutional', 'median'),
        Pct_SOE=('is_soe', 'mean'),
        Mean_FOL_Util=('fol_utilization', 'mean'),
    ).round(4)

    # Format
    table['N'] = table['N'].apply(lambda x: f"{x:,.0f}")
    table['Mean_MktCap'] = table['Mean_MktCap'].apply(lambda x: f"{x:,.0f}B
↪   VND")
    for col in ['Mean_IO_Total', 'Mean_State', 'Mean_Foreign',
                'Mean_Domestic_Inst', 'Mean_Individual', 'Pct_SOE',
                ↪  'Mean_FOL_Util']:
        table[col] = table[col].apply(lambda x: f"{x:.1%}" if pd.notna(x)
↪   else "-")
    table['Median_N_Owners'] = table['Median_N_Owners'].apply(lambda x:
↪   f"{x:.0f}")
    table['Median_HHI'] = table['Median_HHI'].apply(lambda x: f"{x:.3f}" if
↪   pd.notna(x) else "-")

    table.columns = ['N', 'Mean Mkt Cap', 'IO Total', 'State', 'Foreign',
                     'Dom. Inst.', 'Individual', 'Med. # Owners',
                     'Med. HHI', '% SOE', 'FOL Util.']

    return table
```

```
# ============================================================================
# Step 6: Foreign Ownership Limit Analysis
# ============================================================================

class FOLAnalyzer:
    """
    Analyze Foreign Ownership Limit dynamics in the Vietnamese market.

    Key analyses:
    1. FOL utilization tracking and classification
    2. FOL premium estimation (price impact of being near cap)
    3. Foreign room dynamics (opening/closing events)
    4. Cross-sectional determinants of foreign ownership
    """

    FOL_ZONES = {
        'Green': (0, 0.50),
        'Yellow': (0.50, 0.80),
        'Orange': (0.80, 0.95),
        'Red': (0.95, 1.00),
        'Capped': (1.00, 1.50),
    }

    def __init__(self, io_metrics: pd.DataFrame,
                 foreign_daily: Optional[pd.DataFrame] = None):
        """
        Parameters
        ----------
        io_metrics : pd.DataFrame
            Full ownership metrics from compute_io_metrics_vietnam()
        foreign_daily : pd.DataFrame, optional
            Daily foreign ownership tracking from DataCore.vn
        """
        self.metrics = io_metrics.copy()
        self.foreign_daily = foreign_daily

    def classify_fol_zones(self) -> pd.DataFrame:
        """Classify stocks into FOL proximity zones."""
        df = self.metrics.copy()

        if 'fol_utilization' not in df.columns:
```

```
            print("FOL utilization not available in metrics.")
            return df

        conditions = []
        choices = []
        for zone, (lo, hi) in self.FOL_ZONES.items():
            conditions.append(
                (df['fol_utilization'] >= lo) & (df['fol_utilization'] < hi)
            )
            choices.append(zone)

        df['fol_zone'] = np.select(conditions, choices, default='Unknown')

        # Summary
        zone_dist = df.groupby('fol_zone')['ticker'].nunique()
        print("FOL Zone Distribution (unique stocks):")
        print(zone_dist.to_string())

        return df

    def estimate_fol_premium(self) -> pd.DataFrame:
        """
        Estimate the FOL premium using a cross-sectional approach.

        For each period, regress stock valuations (P/B or P/E) on FOL
        utilization, controlling for fundamentals. The coefficient on
        FOL utilization captures the premium investors pay for stocks
        near their foreign ownership cap.

        Alternative: Compare returns of stocks transitioning between
        FOL zones as a natural experiment.
        """
        df = self.metrics.copy()
        df = df[df['fol_utilization'].notna() &
↪   df['market_cap'].notna()].copy()

        # FOL zone dummies
        df['near_cap'] = (df['fol_utilization'] > 0.90).astype(int)
        df['at_cap'] = (df['fol_utilization'] > 0.98).astype(int)

        # Price-to-book as valuation measure
        # (Assumes 'equity' is available from financial data)
        if 'equity' in df.columns:
```

```
            df['pb_ratio'] = df['market_cap'] * 1e9 / df['equity']
        else:
            # Use market cap as proxy for cross-sectional analysis
            df['log_mktcap'] = np.log(df['market_cap'])

        # Fama-MacBeth style: run cross-sectional regressions each period
        results = []
        for quarter, group in df.groupby('quarter_end'):
            group = group.dropna(subset=['fol_utilization', 'log_mktcap'])
            if len(group) < 50:
                continue

            y = group['log_mktcap']
            X = sm.add_constant(group[['fol_utilization', 'pct_state',
                                       'n_inst_owners']])
            try:
                model = sm.OLS(y, X).fit()
                results.append({
                    'quarter': quarter,
                    'beta_fol': model.params.get('fol_utilization', np.nan),
                    'tstat_fol': model.tvalues.get('fol_utilization',
                     ↪  np.nan),
                    'r2': model.rsquared,
                    'n': len(group),
                })
            except Exception:
                continue

        if results:
            results_df = pd.DataFrame(results)
            print("FOL Premium (Fama-MacBeth Regression):")
            print(f"  Mean  (FOL_util): {results_df['beta_fol'].mean():.4f}")
            print(f"  t-statistic: {results_df['beta_fol'].mean() / "
                  f"(results_df['beta_fol'].std() /
↪  np.sqrt(len(results_df))):.2f}")
            return results_df

        return pd.DataFrame()

    def analyze_foreign_room_events(self) -> pd.DataFrame:
        """
        Analyze events where foreign room opens or closes.
```

```
        Room-opening events (FOL cap raised, foreign seller exits) can
        trigger significant price movements as pent-up foreign demand
        is released. Room-closing events (approaching cap) can create
        selling pressure as foreign investors anticipate illiquidity.
        """
        if self.foreign_daily is None:
            print("Daily foreign ownership data required for event
             ↪ analysis.")
            return pd.DataFrame()

        df = self.foreign_daily.copy()
        df = df.sort_values(['ticker', 'date'])

        # Compute daily change in foreign room
        df['foreign_room_change'] =
↪   df.groupby('ticker')['foreign_room'].diff()

        # Identify room-opening events (room increases by > 1 percentage
         ↪ point)
        df['room_open_event'] = (df['foreign_room_change'] >
↪   0.01).astype(int)

        # Identify room-closing events (room decreases to < 2%)
        df['room_close_event'] = (
            (df['foreign_room'] < 0.02) &
            (df.groupby('ticker')['foreign_room'].shift(1) >= 0.02)
        ).astype(int)

        events = df[
            (df['room_open_event'] == 1) | (df['room_close_event'] == 1)
        ].copy()

        print(f"Foreign room events identified:")
        print(f"  Room-opening events: {df['room_open_event'].sum():,}")
        print(f"  Room-closing events: {df['room_close_event'].sum():,}")

        return events

# fol_analyzer = FOLAnalyzer(io_metrics, dc.foreign_ownership)
# fol_classified = fol_analyzer.classify_fol_zones()
# fol_premium = fol_analyzer.estimate_fol_premium()
```

```
def plot_fol_utilization(metrics: pd.DataFrame):
    """Plot FOL utilization distribution by sector."""
    df = metrics[metrics['fol_utilization'].notna()].copy()

    # Assign broad sectors
    sector_map = {
        'Banking': ['VCB', 'BID', 'CTG', 'TCB', 'VPB', 'MBB', 'ACB', 'HDB',
        ↪  'STB', 'TPB'],
        'Real Estate': ['VHM', 'VIC', 'NVL', 'KDH', 'DXG', 'HDG', 'VRE'],
        'Technology': ['FPT', 'CMG', 'FOX'],
        'Consumer': ['VNM', 'MSN', 'SAB', 'MWG', 'PNJ'],
    }

    fig, ax = plt.subplots(figsize=(10, 6))

    for sector, tickers in sector_map.items():
        data = df[df['ticker'].isin(tickers)]['fol_utilization']
        if len(data) > 0:
            ax.hist(data * 100, bins=30, alpha=0.4, label=sector,
↪   density=True)

    ax.axvline(x=30, color='red', linestyle='--', alpha=0.7, label='Banking
↪   FOL (30%)')
    ax.axvline(x=49, color='blue', linestyle='--', alpha=0.7, label='Standard
↪   FOL (49%)')
    ax.set_xlabel('FOL Utilization (%)')
    ax.set_ylabel('Density')
    ax.set_title('Foreign Ownership Limit Utilization Distribution')
    ax.legend()

    plt.tight_layout()
    plt.savefig('fig_fol_utilization.png', dpi=300, bbox_inches='tight')
    plt.show()

# plot_fol_utilization(io_metrics)
```

Figure 58.3

58.7 Institutional Trades

58.7.1 Trade Inference in Vietnam

In the US, institutional trades are inferred from quarterly 13F holding snapshots. In Vietnam, the challenge is more acute because disclosure frequency varies:

- **Major shareholders ($\geq$ 5%)**: Must disclose within 7 business days of crossing ownership thresholds (5%, 10%, 15%, 20%, 25%, 50%, 65%, 75%)
- **Fund portfolio reports:** Semi-annual disclosure required; some funds report quarterly
- **Annual reports:** Provide complete shareholder register but only once per year
- **Daily foreign ownership:** HOSE/HNX publish aggregate daily foreign buy/sell data

We derive trades from the **change in ownership between consecutive disclosure dates**, applying the same logic as the US ITZHAK Ben-David et al. (2013) algorithm but adapted for Vietnam's irregular disclosure intervals.

A caveat applies to any trade-inference exercise built on disclosed holdings: the disclosures themselves may be strategic. Even under the US's mandatory quarterly 13F regime, S. Cao et al. (2026) document that hedge funds sometimes misreport their original filings and quietly correct them later through restatements (which are as common as confidential filings and affect roughly three times as many stocks), with return patterns suggesting that the misreporting hides trading intentions while positions are being built. Vietnamese ownership disclosures (event-driven, less standardized, and less intensively policed than 13F filings) warrant at least as much skepticism: treat snapshots as reports that may be strategically timed or incomplete rather than ground truth, and cross-validate them against independent sources such as daily foreign flow data and annual shareholder registers wherever possible.

```
# ============================================================================
↪  ==
# Step 7: Derive Institutional Trades
# ============================================================================
↪  ==

def derive_trades_vietnam(ownership: pd.DataFrame,
                          adj_factors: pd.DataFrame) -> pd.DataFrame:
    """
    Derive institutional trades from changes in ownership disclosures.

    Adapted from Ben-David, Franzoni, and Moussawi (2012) for
    Vietnam's irregular disclosure frequency.

    Key differences from US approach:
```

```
    1. Disclosure intervals are irregular (not always quarterly)
    2. We observe ALL institutional types, not just 13F filers
    3. No $100M AUM threshold (we see all institutional holders)
    4. Must adjust for corporate actions between disclosure dates

    Trade types:
    +1: Initiating Buy (new position)
    +2: Incremental Buy (increased existing position)
    -1: Terminating Sale (fully exited position)
    -2: Incremental Sale (reduced existing position)

    Parameters
    ----------
    ownership : pd.DataFrame
        Classified ownership with: ticker, date, shareholder_name,
        shares_held, owner_type
    adj_factors : pd.DataFrame
        Corporate action adjustment factors

    Returns
    -------
    pd.DataFrame
        Trade-level data: date, shareholder_name, ticker, trade,
        buysale, owner_type
    """
    # Focus on institutional shareholders only
    inst = ownership[
        ownership['owner_type'].isin(OwnershipType.INSTITUTIONAL)
    ].copy()

    inst = inst.sort_values(['shareholder_name', 'ticker',
↪   'date']).reset_index(drop=True)

    trades_list = []

    for (shareholder, ticker), group in inst.groupby(['shareholder_name',
     ↪  'ticker']):
        group = group.reset_index(drop=True)

        for i in range(len(group)):
            current = group.iloc[i]
            current_date = current['date']
            current_shares = current['shares_held']
```

```
            owner_type = current['owner_type']

            if i == 0:
                # First observation: if institution appears, it's an
                 ↪ initiating buy
                # (we don't know if they held before our data starts)
                # Skip the very first observation to avoid false initiating
                 ↪ buys
                continue

            prev = group.iloc[i - 1]
            prev_date = prev['date']
            prev_shares = prev['shares_held']

            # Adjust previous shares for corporate actions between dates
            prev_shares_adj = adjust_shares(
                prev_shares, ticker, prev_date, current_date, adj_factors
            )

            # Compute trade (in adjusted shares)
            trade = current_shares - prev_shares_adj

            # Classify trade type
            if abs(trade) < 1:  # De minimis threshold
                continue

            if prev_shares_adj <= 0 and current_shares > 0:
                buysale = 1  # Initiating buy
            elif prev_shares_adj > 0 and current_shares <= 0:
                buysale = -1  # Terminating sale
            elif trade > 0:
                buysale = 2  # Incremental buy
            else:
                buysale = -2  # Incremental sale

            trades_list.append({
                'date': current_date,
                'shareholder_name': shareholder,
                'ticker': ticker,
                'trade': trade,
                'prev_shares_adj': prev_shares_adj,
                'current_shares': current_shares,
                'buysale': buysale,
```

```
                'owner_type': owner_type,
                'days_between': (current_date - prev_date).days,
            })

    trades = pd.DataFrame(trades_list)

    if len(trades) > 0:
        print(f"Trades derived: {len(trades):,}")
        print(f"\nTrade type distribution:")
        labels = {1: 'Initiating Buy', 2: 'Incremental Buy',
                  -1: 'Terminating Sale', -2: 'Incremental Sale'}
        for bs, label in sorted(labels.items()):
            n = (trades['buysale'] == bs).sum()
            print(f"  {label}: {n:,} ({n/len(trades):.1%})")

        print(f"\nBy owner type:")
        print(trades.groupby('owner_type')['trade'].agg(['count', 'mean',
         ↪  'median'])
              .round(0).to_string())

    return trades

# trades = derive_trades_vietnam(ownership_classified, adj_factors)
```

Corporate Action Adjustment in Trade Derivation

When computing trades as $\Delta Shares = Shares_t - Shares_{t-1}$, the previous period's shares **must** be adjusted for any corporate actions between $t-1$ and t. If VNM issued a 20% stock dividend between the two disclosure dates, then 1,000 shares at $t-1$ should be compared to 1,200 adjusted shares, not 1,000 raw shares. Failing to make this adjustment would create a phantom "buy" of 200 shares that never actually occurred.

```
def derive_trades_vectorized_vietnam(ownership: pd.DataFrame,
                                     adj_factors: pd.DataFrame) ->
 ↪  pd.DataFrame:
    """
    Vectorized version of Vietnamese trade derivation.

    Uses pandas groupby and vectorized operations instead of Python loops.
    Approximately 20-50x faster for large datasets.
```

```
    Note: Corporate action adjustment is applied per-group, which still
    requires some iteration but is much faster than row-by-row.
    """
    inst = ownership[
        ownership['owner_type'].isin(OwnershipType.INSTITUTIONAL) &
        (ownership['shares_held'] > 0)
    ].copy()

    inst = inst.sort_values(['shareholder_name', 'ticker',
    'date']).reset_index(drop=True)

    # Lagged values
    inst['prev_date'] = inst.groupby(['shareholder_name',
    'ticker'])['date'].shift(1)
    inst['prev_shares'] = inst.groupby(['shareholder_name',
    'ticker'])['shares_held'].shift(1)
    inst['is_first'] = inst['prev_date'].isna()

    # Remove first observations (no prior to compare)
    inst = inst[~inst['is_first']].copy()

    # Adjust previous shares for corporate actions
    # Vectorized: for each row, apply adjustment between prev_date and date
    def adjust_row(row):
        return adjust_shares(
            row['prev_shares'], row['ticker'],
            row['prev_date'], row['date'], adj_factors
        )

    inst['prev_shares_adj'] = inst.apply(adjust_row, axis=1)

    # Compute trade
    inst['trade'] = inst['shares_held'] - inst['prev_shares_adj']
    inst['days_between'] = (inst['date'] - inst['prev_date']).dt.days

    # Classify trade type
    inst['buysale'] = np.select(
        [
            (inst['prev_shares_adj'] <= 0) & (inst['shares_held'] > 0),
            (inst['prev_shares_adj'] > 0) & (inst['shares_held'] <= 0),
            inst['trade'] > 0,
            inst['trade'] < 0,
        ],
```

```
        [1, -1, 2, -2],
        default=0
    )

    # Remove zero trades
    trades = inst[inst['buysale'] != 0].copy()

    trades = trades[['date', 'shareholder_name', 'ticker', 'trade',
                     'buysale', 'owner_type', 'days_between',
                     'prev_shares_adj', 'shares_held']].copy()
    trades = trades.rename(columns={'shares_held': 'current_shares'})

    print(f"Vectorized trades: {len(trades):,}")
    return trades

# trades = derive_trades_vectorized_vietnam(ownership_classified,
 ↪  adj_factors)
```

58.8 Fund-Level Flows and Turnover

58.8.1 Portfolio Assets and Returns from Fund Holdings

Using DataCore.vn's fund holdings data, we compute fund-level portfolio analytics analogous to the US 13F approach:

$$Assets_{j,t} = \sum_{i=1}^{N_{j,t}} S_{i,j,t} \times P_{i,t} \tag{58.7}$$

$$R_{j,t\to t+1}^{holdings} = \frac{\sum_i S_{i,j,t} \times P_{i,t} \times R_{i,t\to t+1}}{\sum_i S_{i,j,t} \times P_{i,t}} \tag{58.8}$$

$$NetFlows_{j,t} = Assets_{j,t} - Assets_{j,t-1} \times (1 + R_{j,t-1\to t}^{holdings}) \tag{58.9}$$

58.8.2 Turnover Measures

Following Mark M. Carhart (1997a), adapted for Vietnam's fund reporting:

$$Turnover_{j,t}^{Carhart} = \frac{\min(TotalBuys_{j,t}, TotalSales_{j,t})}{\overline{Assets}_{j,t}} \tag{58.10}$$

```
# ============================================================================
# Step 8: Fund-Level Portfolio Analytics
# ============================================================================

def compute_fund_analytics(fund_holdings: pd.DataFrame,
                           prices_q: pd.DataFrame,
                           adj_factors: pd.DataFrame) -> Dict:
    """
    Compute fund-level portfolio analytics from DataCore.vn fund holdings.

    Vietnamese fund disclosure is typically semi-annual (some quarterly),
    which limits the frequency of these analytics compared to the US
    quarterly approach.

    Returns
    -------
    dict with keys:
        'fund_assets': pd.DataFrame of fund-level assets and returns
        'fund_trades': pd.DataFrame of fund-level derived trades
        'fund_aggregates': pd.DataFrame of flows and turnover
    """
    fh = fund_holdings.copy()
    fh = fh[fh['shares_held'] > 0].copy()

    # Merge with prices
    fh = fh.merge(
        prices_q[['ticker', 'quarter_end', 'close', 'adjusted_close',
'ret']],
        left_on=['ticker', 'report_date'],
        right_on=['ticker', 'quarter_end'],
        how='inner'
    )

    # Portfolio value
    fh['holding_value'] = fh['shares_held'] * fh['close']

    # --- Fund-Level Assets ---
```

```
    fund_assets = fh.groupby(['fund_name', 'report_date']).agg(
        total_assets=('holding_value', lambda x: x.sum() / 1e9),  # Billion
↪   VND
        n_stocks=('ticker', 'nunique'),
    ).reset_index()

    # Holdings return (value-weighted)
    fh['weight'] = fh.groupby(['fund_name',
↪   'report_date'])['holding_value'].transform(
        lambda x: x / x.sum()
    )
    fund_hret = (fh.groupby(['fund_name', 'report_date'])
                   .apply(lambda g: np.average(g['ret'].fillna(0),
↪   weights=g['weight']))
                   .reset_index(name='holdings_return'))

    fund_assets = fund_assets.merge(fund_hret, on=['fund_name',
↪   'report_date'])

    # --- Fund-Level Trades ---
    # Derive trades from changes in holdings
    fh_sorted = fh.sort_values(['fund_name', 'ticker', 'report_date'])
    fh_sorted['prev_shares'] = fh_sorted.groupby(['fund_name',
↪   'ticker'])['shares_held'].shift(1)
    fh_sorted['prev_date'] = fh_sorted.groupby(['fund_name',
↪   'ticker'])['report_date'].shift(1)

    # Adjust for corporate actions
    fh_sorted['prev_shares_adj'] = fh_sorted.apply(
        lambda r: adjust_shares(r['prev_shares'], r['ticker'],
                                r['prev_date'], r['report_date'],
↪   adj_factors)
        if pd.notna(r['prev_shares']) else np.nan,
        axis=1
    )

    fh_sorted['trade'] = fh_sorted['shares_held'] -
↪   fh_sorted['prev_shares_adj']
    fh_sorted['trade_value'] = fh_sorted['trade'] * fh_sorted['close'] / 1e9
↪   # Billion VND

    # Aggregate buys and sells per fund-period
```

```
    fund_trades = fh_sorted[fh_sorted['trade'].notna()].copy()
    fund_flows = fund_trades.groupby(['fund_name', 'report_date']).agg(
        total_buys=('trade_value', lambda x: x[x > 0].sum()),
        total_sales=('trade_value', lambda x: -x[x < 0].sum()),
    ).reset_index()

    # --- Fund-Level Aggregates ---
    fund_agg = fund_assets.merge(fund_flows, on=['fund_name', 'report_date'],
    how='left')
    fund_agg[['total_buys', 'total_sales']] = fund_agg[['total_buys',
    'total_sales']].fillna(0)

    fund_agg = fund_agg.sort_values(['fund_name', 'report_date'])
    fund_agg['lag_assets'] =
    fund_agg.groupby('fund_name')['total_assets'].shift(1)
    fund_agg['lag_hret'] =
    fund_agg.groupby('fund_name')['holdings_return'].shift(1)

    # Net flows
    fund_agg['net_flows'] = (fund_agg['total_assets'] -
                             fund_agg['lag_assets'] * (1 +
    fund_agg['holdings_return']))

    # Turnover (Carhart definition)
    fund_agg['avg_assets'] = (fund_agg['total_assets'] +
    fund_agg['lag_assets']) / 2
    fund_agg['turnover'] = (
        fund_agg[['total_buys', 'total_sales']].min(axis=1) /
    fund_agg['avg_assets']
    )

    # Annualize (approximate, since disclosure may be semi-annual)
    fund_agg['periods_per_year'] = 365 /
    fund_agg.groupby('fund_name')['report_date'].diff().dt.days
    fund_agg['turnover_annual'] = fund_agg['turnover'] *
    fund_agg['periods_per_year'].fillna(2)

    print(f"Fund analytics computed:")
    print(f"  Unique funds: {fund_agg['fund_name'].nunique():,}")
    print(f"  Fund-period observations: {len(fund_agg):,}")
    print(f"\nTurnover statistics:")
    print(fund_agg[['turnover', 'turnover_annual']].describe().round(4))
```

```
    return {
        'fund_assets': fund_assets,
        'fund_trades': fund_trades,
        'fund_aggregates': fund_agg,
    }

# fund_analytics = compute_fund_analytics(dc.fund_holdings, prices_q,
 ↪  adj_factors)
```

58.9 State Ownership Analysis

58.9.1 Equitization and the Decline of State Ownership

Vietnam's equitization (cổ phần hóa) program has been a defining feature of the market since the early 2000s. The program converts state-owned enterprises into joint-stock companies, typically with the state retaining a controlling or significant minority stake that is then gradually reduced through secondary offerings.

```
# ============================================================================
 ↪  ==
# Step 9: State Ownership Analysis
# ============================================================================
 ↪  ==

def analyze_state_ownership(metrics: pd.DataFrame) -> Dict:
    """
    Comprehensive analysis of state ownership in Vietnam.

    Computes:
    1. Aggregate state ownership trends
    2. SOE population dynamics (entry/exit from SOE classification)
    3. Equitization event detection (large drops in state ownership)
    4. State ownership by sector and size
    5. Governance implications (state as blockholder)
    """
    df = metrics.copy()
```

```
    # --- 1. Aggregate Trends ---
    ts = df.groupby('quarter_end').agg(
        n_soe=('is_soe', 'sum'),
        n_total=('ticker', 'nunique'),
        pct_soe=('is_soe', 'mean'),
        mean_state_pct=('pct_state', 'mean'),
        median_state_pct=('pct_state', 'median'),
        # Market cap share of SOEs
        soe_mktcap=('market_cap', lambda x: x[df.loc[x.index, 'is_soe'] ==
↪   1].sum()),
        total_mktcap=('market_cap', 'sum'),
    ).reset_index()
    ts['soe_mktcap_share'] = ts['soe_mktcap'] / ts['total_mktcap']

    # --- 2. Equitization Events ---
    # Detect large drops in state ownership (>10 percentage points)
    df_sorted = df.sort_values(['ticker', 'quarter_end'])
    df_sorted['state_change'] =
↪   df_sorted.groupby('ticker')['pct_state'].diff()

    equitization_events = df_sorted[
        df_sorted['state_change'] < -0.10  # > 10pp drop
    ][['ticker', 'quarter_end', 'pct_state', 'state_change',
↪   'market_cap']].copy()

    # --- 3. By Sector ---
    if 'industry_code' in df.columns:
        by_sector = df.groupby('industry_code').agg(
            mean_state=('pct_state', 'mean'),
            pct_soe=('is_soe', 'mean'),
            n_firms=('ticker', 'nunique'),
        ).sort_values('mean_state', ascending=False)
    else:
        by_sector = None

    print(f"State Ownership Analysis:")
    print(f"  Current SOE count: {ts.iloc[-1]['n_soe']:.0f} /
     ↪  {ts.iloc[-1]['n_total']:.0f}")
    print(f"  SOE market cap share: {ts.iloc[-1]['soe_mktcap_share']:.1%}")
    print(f"  Mean state ownership: {ts.iloc[-1]['mean_state_pct']:.1%}")
    print(f"\nEquitization events detected: {len(equitization_events):,}")
```

```
    return {
        'trends': ts,
        'equitization_events': equitization_events,
        'by_sector': by_sector,
    }

# state_analysis = analyze_state_ownership(io_metrics)
```

58.10 Modern Extensions

58.10.1 Network Analysis of Co-Ownership

Institutional co-ownership networks capture how stocks are connected through shared investors. In Vietnam, these networks reveal the influence structure of major domestic conglomerates (e.g., Vingroup, Masan, FPT) and the overlap between foreign fund portfolios.

```
def construct_stock_coownership_network(ownership: pd.DataFrame,
                                        period: str,
                                        min_overlap: int = 3) -> Dict:
    """
    Construct a stock-level co-ownership network.

    Two stocks are connected if they share institutional investors.
    Edge weight = number of shared institutional investors.

    This is particularly informative in Vietnam where:
    - Foreign fund portfolios concentrate on the same blue-chips
    - Conglomerate cross-holdings create explicit linkages
    - State ownership creates implicit connections (SCIC holds multiple
 ↪  stocks)

    Parameters
    ----------
    ownership : pd.DataFrame
        Classified ownership data
    period : str
        Analysis date
```

```
def plot_state_ownership(state_analysis: Dict, metrics: pd.DataFrame):
    """Plot state ownership dynamics."""
    fig, axes = plt.subplots(2, 1, figsize=(12, 10))
    ts = state_analysis['trends']

    # Panel A: SOE trends
    ax = axes[0]
    ax.plot(ts['quarter_end'], ts['pct_soe'] * 100,
            label='% of Firms that are SOEs', linewidth=2, color='#d62728')
    ax.plot(ts['quarter_end'], ts['soe_mktcap_share'] * 100,
            label='SOE Market Cap Share (%)', linewidth=2, color='#1f77b4')
    ax.plot(ts['quarter_end'], ts['mean_state_pct'] * 100,
            label='Mean State Ownership (%)', linewidth=2, color='#2ca02c',
    ↪ linestyle='--')
    ax.set_ylabel('Percentage')
    ax.set_title('Panel A: State Ownership and SOE Prevalence Over Time')
    ax.legend(frameon=True, framealpha=0.9)

    # Panel B: Distribution
    ax = axes[1]
    # Use most recent period
    latest = metrics[metrics['quarter_end'] == metrics['quarter_end'].max()]
    state_pct = latest['pct_state'].dropna() * 100

    ax.hist(state_pct, bins=50, color='#d62728', alpha=0.7,
    ↪ edgecolor='black')
    ax.axvline(x=50, color='black', linestyle='--', alpha=0.7, label='50%
    ↪ (SOE threshold)')
    ax.set_xlabel('State Ownership (%)')
    ax.set_ylabel('Number of Companies')
    ax.set_title('Panel B: Distribution of State Ownership (Most Recent
    ↪ Quarter)')
    ax.legend()

    plt.tight_layout()
    plt.savefig('fig_state_ownership.png', dpi=300, bbox_inches='tight')
    plt.show()

# plot_state_ownership(state_analysis, io_metrics)
```

Figure 58.4

```
    min_overlap : int
        Minimum shared investors to create an edge

    Returns
    -------
    dict with network statistics and adjacency data
    """
    import networkx as nx

    date = pd.Timestamp(period)

    # Get institutional holders for this period
    inst = ownership[
        (ownership['date'] == date) &
        (ownership['owner_type'].isin(OwnershipType.INSTITUTIONAL))
    ][['ticker', 'shareholder_name', 'owner_type']].copy()

    # Create bipartite mapping: institution → set of stocks held
    inst_to_stocks =
↪   inst.groupby('shareholder_name')['ticker'].apply(set).to_dict()

    # Stock → set of institutions
    stock_to_inst =
↪   inst.groupby('ticker')['shareholder_name'].apply(set).to_dict()

    # Build stock-level network
    stocks = list(stock_to_inst.keys())
    G = nx.Graph()

    for i in range(len(stocks)):
        for j in range(i + 1, len(stocks)):
            shared = stock_to_inst[stocks[i]] & stock_to_inst[stocks[j]]
            if len(shared) >= min_overlap:
                G.add_edge(stocks[i], stocks[j], weight=len(shared),
                           shared_investors=list(shared)[:5])  # Store sample

    # Add node attributes
    for stock in stocks:
        if stock in G.nodes:
            G.nodes[stock]['n_inst_holders'] = len(stock_to_inst[stock])

    # Network statistics
    stats = {
```

```
        'n_nodes': G.number_of_nodes(),
        'n_edges': G.number_of_edges(),
        'density': nx.density(G) if G.number_of_nodes() > 1 else 0,
        'avg_clustering': nx.average_clustering(G, weight='weight') if
        ↪  G.number_of_nodes() > 0 else 0,
        'n_components': nx.number_connected_components(G),
    }

    # Centrality measures
    if G.number_of_nodes() > 0:
        degree_cent = nx.degree_centrality(G)
        stats['most_connected'] = sorted(degree_cent.items(),
                                         key=lambda x: x[1],
↪   reverse=True)[:10]

        if G.number_of_nodes() > 2:
            try:
                eigen_cent = nx.eigenvector_centrality_numpy(G,
↪   weight='weight')
                stats['most_central'] = sorted(eigen_cent.items(),
                                               key=lambda x: x[1],
↪   reverse=True)[:10]
            except Exception:
                stats['most_central'] = []

    print(f"Co-Ownership Network ({period}):")
    for k, v in stats.items():
        if k not in ['most_connected', 'most_central']:
            print(f"  {k}: {v}")

    if 'most_connected' in stats:
        print(f"\nMost connected stocks:")
        for stock, cent in stats['most_connected'][:5]:
            print(f"  {stock}: {cent:.3f}")

    return {'graph': G, 'stats': stats}

# network = construct_stock_coownership_network(
#     ownership_classified, '2024-06-30'
# )
```

58.10.2 ML-Enhanced Investor Classification

Vietnam's investor classification challenge is distinct from the US. While the US has the Bushee typology based on portfolio turnover and concentration, Vietnam requires classification of both investor **type** (when not explicitly labeled) and investor **behavior** (active vs passive, short-term vs long-term).

```
def classify_investors_vietnam(ownership: pd.DataFrame,
                               prices_q: pd.DataFrame,
                               n_clusters: int = 4) -> pd.DataFrame:
    """
    ML-based classification of Vietnamese institutional investors.

    Features adapted for Vietnam's market:
    1. Portfolio concentration (HHI of holdings)
    2. Holding duration (average time in positions)
    3. Size preference (average market cap of holdings)
    4. Sector concentration
    5. Foreign/domestic indicator
    6. Trading frequency (inverse of average days between disclosures)

    Expected clusters for Vietnam:
    - Passive State Holders: SOE parents, SCIC - low turnover, concentrated
    - Active Foreign Funds: Dragon Capital, VinaCapital - moderate turnover
    - Domestic Securities Firms: SSI, VNDirect - high turnover, diversified
    - Long-Term Foreign: Pension funds, sovereign wealth - low turnover
    """
    from sklearn.cluster import KMeans
    from sklearn.preprocessing import StandardScaler

    inst = ownership[
        ownership['owner_type'].isin(OwnershipType.INSTITUTIONAL)
    ].copy()

    # Merge with price data
    inst = inst.merge(
        prices_q[['ticker', 'quarter_end', 'close', 'market_cap']],
        left_on=['ticker', 'date'],
        right_on=['ticker', 'quarter_end'],
        how='left'
    )

    inst['holding_value'] = inst['shares_held'] * inst['close'].fillna(0)
```

```
    # Compute features per investor-period
    features = inst.groupby(['shareholder_name', 'date']).agg(
        n_stocks=('ticker', 'nunique'),
        total_value=('holding_value', 'sum'),
        hhi_portfolio=('holding_value',
                       lambda x: ((x/x.sum())**2).sum() if x.sum() > 0 else
                        ↪  np.nan),
        avg_mktcap=('market_cap', 'mean'),
        is_foreign=('owner_type',
                    lambda x: (x ==
                     ↪  OwnershipType.FOREIGN_INST).any().astype(int)),
        is_state=('owner_type',
                  lambda x: (x == OwnershipType.STATE).any().astype(int)),
    ).reset_index()

    # Average across all periods per investor
    investor_features = features.groupby('shareholder_name').agg(
        avg_n_stocks=('n_stocks', 'mean'),
        avg_hhi=('hhi_portfolio', 'mean'),
        avg_mktcap=('avg_mktcap', 'mean'),
        avg_total_value=('total_value', 'mean'),
        is_foreign=('is_foreign', 'max'),
        is_state=('is_state', 'max'),
        n_periods=('date', 'nunique'),
    ).dropna()

    # Feature matrix
    feature_cols = ['avg_n_stocks', 'avg_hhi', 'avg_mktcap',
↪   'avg_total_value']
    X = investor_features[feature_cols].copy()

    # Log-transform
    for col in feature_cols:
        X[col] = np.log1p(X[col].clip(lower=0))

    # Add binary features
    X['is_foreign'] = investor_features['is_foreign']
    X['is_state'] = investor_features['is_state']

    # Standardize
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
```

```
    # K-means
    kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=20)
    investor_features['cluster'] = kmeans.fit_predict(X_scaled)

    # Label clusters
    cluster_profiles = investor_features.groupby('cluster').agg({
        'avg_n_stocks': 'mean',
        'avg_hhi': 'mean',
        'avg_total_value': 'mean',
        'is_foreign': 'mean',
        'is_state': 'mean',
        'shareholder_name': 'count',
    }).rename(columns={'shareholder_name': 'n_investors'})

    print("Investor Clusters:")
    print(cluster_profiles.round(3).to_string())

    return investor_features

# investor_classes = classify_investors_vietnam(ownership_classified,
 ↪  prices_q)
```

58.10.3 Event Study: Ownership Disclosure Shocks

Vietnam's threshold-based major shareholder disclosure creates natural events for studying the price impact of ownership changes.

```
def ownership_event_study(major_shareholders: pd.DataFrame,
                          prices: pd.DataFrame,
                          event_window: Tuple[int, int] = (-5, 20),
                          estimation_window: int = 120) -> pd.DataFrame:
    """
    Event study of ownership disclosure announcements.

    Vietnam requires major shareholders ( 5%) to disclose within 7
    business days of crossing ownership thresholds. These disclosures
    can be informationally significant, especially:
    1. Foreign fund accumulation (signal of quality)
    2. State divestiture (equitization signal)
    3. Insider purchases (management confidence signal)
```

```
    Uses market model for expected returns:
    E[R_i,t] =  _i +  _i × R_m,t

    Parameters
    ----------
    major_shareholders : pd.DataFrame
        Disclosure events from DataCore.vn
    prices : pd.DataFrame
        Daily stock prices
    event_window : tuple
        (pre_event_days, post_event_days)
    estimation_window : int
        Days before event window for market model estimation
    """
    events = major_shareholders.copy()
    events = events.sort_values(['ticker', 'date'])

    # Identify significant ownership changes
    events['ownership_change'] = events.groupby(
        ['ticker', 'shareholder_name']
    )['ownership_pct'].diff()

    significant_events = events[
        events['ownership_change'].abs() > 0.01  # > 1 percentage point
    ].copy()

    significant_events['event_type'] = np.where(
        significant_events['ownership_change'] > 0, 'accumulation',
↪   'divestiture'
    )

    # Merge with daily prices
    prices_daily = prices[['ticker', 'date', 'ret']].copy()
    prices_daily = prices_daily.sort_values(['ticker', 'date'])

    # VN-Index as market return (ticker code depends on data provider)
    if 'VNINDEX' in prices_daily['ticker'].values:
        market_ret = prices_daily[prices_daily['ticker'] ==
↪   'VNINDEX'][['date', 'ret']].copy()
        market_ret = market_ret.rename(columns={'ret': 'mkt_ret'})
    else:
        # Use equal-weighted market return as proxy
```

```
        market_ret = (prices_daily.groupby('date')['ret']
                                  .mean()
                                  .reset_index()
                                  .rename(columns={'ret': 'mkt_ret'}))

    # For each event, compute abnormal returns
    results = []
    pre, post = event_window

    for _, event in significant_events.iterrows():
        ticker = event['ticker']
        event_date = event['date']

        # Get stock returns around the event
        stock_ret = prices_daily[prices_daily['ticker'] == ticker].copy()
        stock_ret = stock_ret.merge(market_ret, on='date', how='left')
        stock_ret = stock_ret.sort_values('date').reset_index(drop=True)

        # Find event date index
        event_idx = stock_ret[stock_ret['date'] >= event_date].index
        if len(event_idx) == 0:
            continue
        event_idx = event_idx[0]

        # Estimation window
        est_start = max(0, event_idx - estimation_window + pre)
        est_end = event_idx + pre
        est_data = stock_ret.iloc[est_start:est_end].dropna(subset=['ret',
↪   'mkt_ret'])

        if len(est_data) < 30:
            continue

        # Market model
        X = sm.add_constant(est_data['mkt_ret'])
        y = est_data['ret']
        try:
            model = sm.OLS(y, X).fit()
        except Exception:
            continue

        # Event window abnormal returns
        ew_start = event_idx + pre
```

```
        ew_end = min(event_idx + post + 1, len(stock_ret))
        event_data = stock_ret.iloc[ew_start:ew_end].copy()

        if len(event_data) == 0:
            continue

        event_data['expected_ret'] = (model.params['const'] +
                                      model.params['mkt_ret'] *
↪   event_data['mkt_ret'])
        event_data['abnormal_ret'] = event_data['ret'] -
↪   event_data['expected_ret']
        event_data['car'] = event_data['abnormal_ret'].cumsum()
        event_data['event_day'] = range(pre, pre + len(event_data))
        event_data['ticker'] = ticker
        event_data['event_date'] = event_date
        event_data['event_type'] = event['event_type']
        event_data['ownership_change'] = event['ownership_change']
        event_data['shareholder_name'] = event['shareholder_name']

        results.append(event_data)

    if results:
        all_results = pd.concat(results, ignore_index=True)

        # Average CARs by event type
        avg_car = (all_results.groupby(['event_type', 'event_day'])['car']
                              .agg(['mean', 'std', 'count'])
                              .reset_index())
        avg_car['t_stat'] = avg_car['mean'] / (avg_car['std'] /
↪   np.sqrt(avg_car['count']))

        print(f"Event Study Results:")
        print(f"  Total events:
         ↪  {significant_events['event_type'].value_counts().to_string()}")

        # CAR at event day 0, +5, +10, +20
        for et in ['accumulation', 'divestiture']:
            print(f"\n  {et.title()} Events:")
            subset = avg_car[avg_car['event_type'] == et]
            for day in [0, 5, 10, 20]:
                row = subset[subset['event_day'] == day]
                if len(row) > 0:
```

```
                    print(f"    CAR({day:+d}): {row.iloc[0]['mean']:.4f} "
                          f"(t={row.iloc[0]['t_stat']:.2f})")

        return all_results

    return pd.DataFrame()

# event_results = ownership_event_study(dc.major_shareholders, dc.prices)
```

58.11 Empirical Applications

58.11.1 Application 1: Foreign Ownership and Stock Returns in Vietnam

Does foreign institutional ownership predict returns in Vietnam? X. Huang, Liu, and Shu (2023) find evidence consistent with the information advantage hypothesis.

```
def test_foreign_io_returns(metrics: pd.DataFrame) -> pd.DataFrame:
    """
    Test whether changes in foreign institutional ownership predict
    future stock returns in Vietnam.

    Methodology:
    1. Sort stocks into quintiles by change in foreign IO
    2. Compute equal-weighted and VN-Index-adjusted returns
    3. Report portfolio returns and long-short spread

    This adapts the Chen, Hong, and Stein (2002) breadth test
    specifically for Vietnam's foreign ownership component.
    """
    df = metrics.copy()
    df = df.sort_values(['ticker', 'quarter_end'])

    # Change in foreign IO
    df['delta_foreign'] = df.groupby('ticker')['pct_foreign_total'].diff()

    # Forward quarterly return
    df['fwd_ret'] = df.groupby('ticker')['ret'].shift(-1)
```

```
    # Drop missing
    df = df.dropna(subset=['delta_foreign', 'fwd_ret'])

    # Quintile portfolios each quarter
    df['foreign_quintile'] =
↪   df.groupby('quarter_end')['delta_foreign'].transform(
        lambda x: pd.qcut(x, 5, labels=[1, 2, 3, 4, 5], duplicates='drop')
    )

    # Portfolio returns
    port_ret = (df.groupby(['quarter_end', 'foreign_quintile'])['fwd_ret']
                  .mean()
                  .reset_index())

    port_wide = port_ret.pivot(index='quarter_end',
↪   columns='foreign_quintile',
                               values='fwd_ret')
    port_wide['LS'] = port_wide[5] - port_wide[1]

    # Test significance
    results = {}
    for q in [1, 2, 3, 4, 5, 'LS']:
        data = port_wide[q].dropna()
        mean_ret = data.mean()
        t_stat = mean_ret / (data.std() / np.sqrt(len(data)))
        results[q] = {
            'Mean Return (%)': mean_ret * 100,
            't-statistic': t_stat,
            'N quarters': len(data),
        }

    results_df = pd.DataFrame(results).T
    results_df.index.name = 'ΔForeign IO Quintile'

    print("Foreign Ownership Change and Future Returns (Vietnam)")
    print("=" * 60)
    print(results_df.round(3).to_string())

    return results_df

# foreign_return_results = test_foreign_io_returns(io_metrics)
```

58.11.2 Application 2: State Divestiture and Value Creation

```
def analyze_equitization_value(metrics: pd.DataFrame,
                               state_analysis: Dict) -> pd.DataFrame:
    """
    Test whether reductions in state ownership are associated with
    subsequent value creation (higher returns, improved governance).

    Hypothesis: State divestiture reduces agency costs, improves
    operational efficiency, and attracts institutional investors,
    leading to positive abnormal returns.

    Uses a difference-in-differences approach:
    Treatment: Firms experiencing >10pp drop in state ownership
    Control: Matched firms with stable state ownership
    """
    df = metrics.copy()
    events = state_analysis['equitization_events']

    if len(events) == 0:
        print("No equitization events detected.")
        return pd.DataFrame()

    # Get treated firms and their event quarters
    treated = events[['ticker', 'quarter_end']].drop_duplicates()
    treated['treated'] = 1

    # Merge with metrics
    df = df.merge(treated, on=['ticker', 'quarter_end'], how='left')
    df['treated'] = df['treated'].fillna(0)

    # Pre/post comparison for treated firms
    treated_tickers = treated['ticker'].unique()

    results = []
    for ticker in treated_tickers:
        firm = df[df['ticker'] == ticker].sort_values('quarter_end')
        event_row = firm[firm['treated'] == 1]
        if len(event_row) == 0:
            continue

        event_q = event_row.iloc[0]['quarter_end']
```

```
        # Pre-event (4 quarters before)
        pre = firm[firm['quarter_end'] < event_q].tail(4)
        # Post-event (4 quarters after)
        post = firm[firm['quarter_end'] > event_q].head(4)

        if len(pre) < 2 or len(post) < 2:
            continue

        results.append({
            'ticker': ticker,
            'event_quarter': event_q,
            'state_pct_pre': pre['pct_state'].mean(),
            'state_pct_post': post['pct_state'].mean(),
            'foreign_pct_pre': pre['pct_foreign_total'].mean(),
            'foreign_pct_post': post['pct_foreign_total'].mean(),
            'n_inst_pre': pre['n_inst_owners'].mean(),
            'n_inst_post': post['n_inst_owners'].mean(),
            'ret_pre': pre['ret'].mean(),
            'ret_post': post['ret'].mean(),
        })

    if results:
        results_df = pd.DataFrame(results)

        # Paired t-tests
        print("Equitization Value Analysis")
        print("=" * 60)
        for metric in ['state_pct', 'foreign_pct', 'n_inst', 'ret']:
            pre_col = f'{metric}_pre'
            post_col = f'{metric}_post'
            diff = results_df[post_col] - results_df[pre_col]
            t_stat, p_val = stats.ttest_1samp(diff.dropna(), 0)
            print(f"  Δ{metric}: {diff.mean():.4f} (t={t_stat:.2f},
             ↪  p={p_val:.3f})")

        return results_df

    return pd.DataFrame()

# equitization_results = analyze_equitization_value(io_metrics,
 ↪  state_analysis)
```

58.11.3 Application 3: Institutional Herding in Vietnam

```
def compute_herding_vietnam(trades: pd.DataFrame,
                            owner_types: Optional[List[str]] = None) ->
↪ pd.DataFrame:
    """
    Compute the Lakonishok, Shleifer, and Vishny (1992) herding measure
    adapted for the Vietnamese market.

    Can be computed separately for:
    - All institutional investors
    - Foreign institutions only
    - Domestic institutions only

    The herding measure captures whether institutions systematically
    trade in the same direction beyond what chance would predict.
    """
    from scipy.stats import binom

    t = trades.copy()

    if owner_types:
        t = t[t['owner_type'].isin(owner_types)]

    t['is_buy'] = (t['trade'] > 0).astype(int)

    # For each stock-period
    stock_trades = t.groupby(['ticker', 'date']).agg(
        n_traders=('shareholder_name', 'nunique'),
        n_buyers=('is_buy', 'sum'),
    ).reset_index()

    # Minimum traders threshold
    stock_trades = stock_trades[stock_trades['n_traders'] >= 3]
    stock_trades['p_buy'] = stock_trades['n_buyers'] /
↪   stock_trades['n_traders']

    # Expected proportion per period
    E_p = stock_trades.groupby('date').apply(
        lambda g: g['n_buyers'].sum() / g['n_traders'].sum()
    ).reset_index(name='E_p')
```

```
    stock_trades = stock_trades.merge(E_p, on='date')

    # Adjustment factor
    def expected_abs_dev(n, p):
        k = np.arange(0, n + 1)
        probs = binom.pmf(k, n, p)
        return np.sum(probs * np.abs(k / n - p))

    stock_trades['adj_factor'] = stock_trades.apply(
        lambda r: expected_abs_dev(int(r['n_traders']), r['E_p']), axis=1
    )

    stock_trades['hm'] = (np.abs(stock_trades['p_buy'] - stock_trades['E_p'])
↪   -
                          stock_trades['adj_factor'])

    stock_trades['buy_herd'] = np.where(
        stock_trades['p_buy'] > stock_trades['E_p'], stock_trades['hm'],
↪   np.nan
    )
    stock_trades['sell_herd'] = np.where(
        stock_trades['p_buy'] < stock_trades['E_p'], stock_trades['hm'],
↪   np.nan
    )

    # Time series of herding
    ts_herding = stock_trades.groupby('date').agg(
        mean_hm=('hm', 'mean'),
        mean_buy_herd=('buy_herd', 'mean'),
        mean_sell_herd=('sell_herd', 'mean'),
        pct_herding=('hm', lambda x: (x > 0).mean()),
        n_stocks=('ticker', 'nunique'),
    ).reset_index()

    print(f"Herding Analysis ({owner_types or 'All Institutions'}):")
    print(f"  Mean HM: {stock_trades['hm'].mean():.4f}")
    print(f"  Mean Buy Herding: {stock_trades['buy_herd'].mean():.4f}")
    print(f"  Mean Sell Herding: {stock_trades['sell_herd'].mean():.4f}")
    print(f"  % stocks with herding: {(stock_trades['hm'] > 0).mean():.1%}")

    return stock_trades, ts_herding

# herding_all, herding_ts = compute_herding_vietnam(trades)
```

```
# herding_foreign, _ = compute_herding_vietnam(
#     trades, owner_types=[OwnershipType.FOREIGN_INST]
# )
```

58.12 Conclusion and Practical Recommendations

58.12.1 Summary of Measures

Table 58.5 summarizes all institutional ownership measures developed in this chapter for the Vietnamese market.

Table 58.5: Summary of All Ownership Measures for Vietnam

Measure	Definition	Key Adaptation for Vietnam	Python Function
IO Ratio	Inst. shares / TSO	Decomposed into state, foreign, domestic	`compute_ownership_decomposition(`
HHI Concentration	$\sum w_j^2$	Separate HHI for total, non-state, foreign	`compute_io_metrics_vietnam()`
ΔBreadth	Lehavy-Sloan adjusted	Applied to irregular disclosure intervals	`compute_io_metrics_vietnam()`
FOL Utilization	Foreign % / FOL limit	Vietnam-specific; no US equivalent	`FOLAnalyzer`
FOL Premium	Price impact of FOL proximity	Cross-sectional regression approach	`FOLAnalyzer.estimate_fol_premium`
Trades	ΔShares (corp-action adjusted)	Critical: adjust for stock dividends	`derive_trades_vectorized_vietnam`
Fund Turnover	min(B,S)/avg(A)	Semi-annual frequency; annualized	`compute_fund_analytics()`
SOE Status	State ownership > 50%	Tracks equitization program	`analyze_state_ownership()`
LSV Herding	$\lvert p - E[p]\rvert - E[\lvert p - E[p]\rvert]$	Separate foreign vs domestic herding	`compute_herding_vietnam()`
Co-Ownership Network	Shared institutional holders	Reveals conglomerate linkages	`construct_stock_coownership_netw`

58.12.2 Data Quality Checklist for Vietnam

Vietnam Data Quality Checklist

1. **Corporate actions:** Have you built and applied adjustment factors for ALL stock dividends, bonus shares, splits, and rights issues?
2. **Shareholder classification:** Have you verified the owner type classification (state vs foreign vs domestic institutional vs individual)?
3. **FOL limits:** Are sector-specific FOL limits correctly assigned (30% for banks, 49% standard, unlimited for some sectors)?
4. **Disclosure dates:** Are you using the actual disclosure date (not the record date or ex-date) for ownership snapshots?
5. **Treasury shares:** Are treasury shares excluded from ownership ratio denominators?
6. **UPCOM coverage:** Does your sample include or exclude UPCOM stocks (which have weaker disclosure requirements)?
7. **Cross-listings:** Are you handling NVDR (Non-Voting Depository Receipts) if applicable after market reforms?
8. **Name consistency:** Are shareholder names standardized across disclosure periods (Vietnamese names can have multiple romanization forms)?
9. **Trade adjustment:** When deriving trades between periods, have you adjusted previous shares for ALL intervening corporate actions?
10. **Fund mandate changes:** For fund analytics, have you accounted for fund mergers, closures, and mandate changes that affect time-series continuity?

58.12.3 Comparison with US Framework

Table 58.6: US vs Vietnam Institutional Ownership Framework Comparison

Dimension	US (WRDS/13F)	Vietnam (DataCore.vn)
Disclosure	Quarterly 13F (mandatory)	Annual reports + event-driven
Coverage	Institutions > $100M AUM	All shareholders in annual reports
Ownership observed	Long positions only	Complete decomposition
IO can exceed 100%	Yes (short selling)	No (by construction)
Permanent ID	CRSP PERMNO	Ticker (with manual tracking of changes)
Adjustment factors	CRSP cfacshr	Must build from corporate actions

Dimension	US (WRDS/13F)	Vietnam (DataCore.vn)
Investor classification	LSEG typecode / Bushee	State/Foreign/Domestic/Individual
Short selling	Not in 13F; exists in market	Very limited; not a concern
Unique features	—	FOL, SOE ownership, stock dividend frequency

59 Institutional Trades, Flows, and Turnover Ratios

Institutional investors play a pivotal role in price discovery, corporate governance, and market liquidity. Understanding *how* institutions trade and *how much* they trade provides insights into both asset pricing dynamics and the real effects of institutional monitoring. The seminal work of Grinblatt, Titman, and Wermers (1995) on mutual fund momentum trading, Wermers (2000) on fund performance decomposition, and Yan (2008) on the relationship between turnover and future returns all rely on accurately measured institutional trades, flows, and turnover.

In the United States, this research is enabled by the mandatory quarterly 13F filing system administered by the Securities and Exchange Commission (SEC). Every institutional investment manager with at least $100 million in qualifying assets must disclose their equity holdings within 45 days of each calendar quarter end. The Thomson-Reuters (now Refinitiv) 13F database, accessible through WRDS, provides the canonical data infrastructure for this literature.

Vietnam's equity market presents a fundamentally different institutional landscape. This chapter adapts the core methodology for the Vietnamese context, addressing five critical differences:

1. **Disclosure regime.** Vietnam has no 13F-equivalent mandatory quarterly filing. Ownership disclosure is a patchwork of event-driven reports (threshold crossings at 5%, 10%, etc.), annual/semi-annual reports with shareholder registers, and daily foreign ownership tracking by exchanges.

2. **Corporate actions.** Vietnamese firms issue stock dividends and bonus shares at extremely high rates compared to US firms. A firm might issue 20-30% bonus shares in a single year, fundamentally altering the share count. Share adjustment is therefore critical and nontrivial.

3. **Foreign ownership limits (FOLs).** Binding foreign ownership ceiling, typically 49% for most sectors, 30% for banking, and 0% for certain restricted sectors, create a unique institutional constraint. When a stock approaches its FOL, foreign buying becomes mechanically restricted, distorting standard trade inference.

4. **State ownership.** The Vietnamese government retains significant ownership in many listed firms through the State Capital Investment Corporation (SCIC) and other state entities. This creates a distinct ownership category not present in the US 13F data.

5. **Market microstructure.** Daily price limits (±7% on HOSE, ±10% on HNX, ±15% on UPCOM), T+2 settlement, and the absence of short-selling all affect how institutional trades translate into market outcomes.

59.1 Measuring Institutional Ownership and Trading

The measurement of institutional ownership and trading activity has been a central concern in empirical finance since Gompers, Ishii, and Metrick (2003) documented the rise of institutional investors. The approach relies on comparing holdings snapshots across consecutive reporting periods to infer trades. If manager j holds $h_{j,i,t}$ shares of stock i at time t, then the inferred trade is:

$$\Delta h_{j,i,t} = h_{j,i,t} - h_{j,i,t-1} \tag{59.1}$$

where $\Delta h_{j,i,t} > 0$ indicates a buy and $\Delta h_{j,i,t} < 0$ indicates a sale. This simple differencing approach requires that holdings are observed at regular intervals (e.g., quarterly), share counts are adjusted for corporate actions between reporting dates, and entry and exit from the dataset are handled appropriately.

H.-L. Chen, Jegadeesh, and Wermers (2000) introduced the concept of ownership *breadth* (i.e., the number of institutions holding a stock) and showed that changes in breadth predict future returns. Sias (2004) decomposed institutional demand into a herding component and an information component. Yan (2008) linked fund turnover to information-based trading and documented that high-turnover funds outperform, challenging the view that turnover reflects noise trading.

59.2 Trade Classification

Table 59.1 shows four categories of trades:

Table 59.1: Trade Classification Taxonomy

Code	Type	Description
+1	Initiating Buy	Manager enters a new position
+2	Incremental Buy	Manager increases an existing position
−1	Terminating Sale	Manager completely exits a position
−2	Regular Sale	Manager reduces an existing position

This classification is informative because initiating buys and terminating sales represent discrete portfolio decisions with different information content from marginal position adjustments (Alexander, Cici, and Gibson 2007).

59.3 Turnover Measures

Three standard turnover definitions have been used in the literature:

Carhart (1997) Turnover. The minimum of aggregate buys and sales, normalized by average assets:

$$\text{Turnover}_{j,t}^{C} = \frac{\min\left(\sum_i B_{j,i,t},\, \sum_i S_{j,i,t}\right)}{\frac{1}{2}\left(A_{j,t} + A_{j,t-1}\right)} \tag{59.2}$$

where $B_{j,i,t}$ and $S_{j,i,t}$ are the dollar values of buys and sales of stock i by manager j in quarter t, and $A_{j,t}$ is total portfolio assets (Mark M. Carhart 1997b).

Flow-Adjusted Turnover. Adds back the absolute value of net flows to account for flow-driven trading:

$$\text{Turnover}_{j,t}^{F} = \frac{\min\left(\sum_i B_{j,i,t},\, \sum_i S_{j,i,t}\right) + |\text{NetFlow}_{j,t}|}{A_{j,t-1}} \tag{59.3}$$

Symmetric Turnover. Uses the sum of buys and sales minus the absolute net flow:

$$\text{Turnover}_{j,t}^{S} = \frac{\sum_i B_{j,i,t} + \sum_i S_{j,i,t} - |\text{NetFlow}_{j,t}|}{A_{j,t-1}} \tag{59.4}$$

The relationship between these measures depends on the correlation between discretionary trading and flow-induced trading (Pástor and Stambaugh 2003).

59.4 Institutional Ownership in Emerging Markets

The emerging markets literature has documented several stylized facts about institutional ownership that differ from developed market findings. Aggarwal et al. (2011) documented that foreign institutional ownership improves corporate governance in emerging markets. For Vietnam specifically, Phung and Mishra (2016) examined the relationship between ownership structure and firm performance, while X. V. Vo (2015) studied the impact of foreign ownership on stock market liquidity.

59.5 Net Flows and Performance Attribution

Net flows measure the dollar amount of new money entering or leaving a fund:

$$\text{NetFlow}_{j,t} = A_{j,t} - A_{j,t-1}(1 + R^{p}_{j,t}) \tag{59.5}$$

where $R^{p}_{j,t}$ is the portfolio return. This decomposition, due to Sirri and Tufano (1998), separates changes in fund assets into investment returns and investor capital allocation decisions. Coval and Stafford (2007) showed that flow-driven trades create price pressure, with fire sales by funds experiencing redemptions generating significant negative abnormal returns.

59.6 Flow-Induced Price Pressure and the Arithmetic of Measuring It

The Coval and Stafford (2007) result did not stay a fact about fire sales for long; it became an identification strategy. If a mutual fund facing large redemptions must raise cash, and if the redemption decisions of that fund's own investors are unrelated to the fundamentals of any individual stock it happens to hold, then the selling pressure transmitted to those stocks is a source of price variation that is plausibly orthogonal to firm value. Edmans, Goldstein, and Jiang (2012) turned this logic into an instrument for non-fundamental price movement and used it to show that depressed prices attract takeover bids. Khan, Kogan, and Serafeim (2012) applied it to the timing of seasoned equity offerings, D. Lou (2012) built the closely related flow-induced trading (FIT) measure and used it to explain return predictability, and X. Lou and Wang (2018) and Dessaint et al. (2019) used outflow-induced pressure to ask whether managers learn from prices when setting investment. Within a decade the measure had become one of the most heavily used instruments in empirical corporate finance, appearing in work on investment, innovation, disclosure, payout, CEO turnover, and governance.

59.6.1 The Standard Measure

The workhorse construction is the mutual fund price pressure variable of Edmans, Goldstein, and Jiang (2012):

$$\text{MFFlow}_{i,t} = \sum_{j} \frac{F_{j,t} \times \text{SHARES}_{i,j,t-1} \times \text{PRC}_{i,t-1}}{\text{TA}_{j,t-1} \times \text{VOL}_{i,t}} \tag{59.6}$$

where j indexes mutual funds, $F_{j,t}$ is fund j's total net flow in quarter t, $\text{TA}_{j,t-1}$ is its total assets at the end of the previous quarter, $\text{SHARES}_{i,j,t-1}$ is the number of shares of stock i the fund held at the end of quarter $t-1$, $\text{PRC}_{i,t-1}$ is the stock price at that date, and $\text{VOL}_{i,t}$ is

the stock's total *dollar* trading volume during quarter t. The summation runs only over funds experiencing outflows above a threshold (5% of assets in the original paper, 10th-percentile flows in much of the subsequent work), and sector funds are excluded so that outflows cannot reflect a view on a particular industry. The numerator is the dollar value the fund *would* sell if it liquidated its portfolio pro rata to meet the redemption; the denominator scales that hypothetical trade by the stock's liquidity, on the reasoning that the same forced sale moves a thin stock more than a liquid one.

59.6.2 What the Measure Actually Contains

Wardlaw (2020) shows that Equation 59.6 is not the ratio it appears to be. Write the quarter's dollar volume as share volume times the volume-weighted average price at which the stock actually traded, $\text{VOL}_{i,t} = \text{SHVOL}_{i,t} \times \bar{P}_{i,t}$. Substituting into Equation 59.6 and rearranging gives

$$\text{MFFlow}_{i,t} = \underbrace{\left[\sum_{j} \frac{F_{j,t} \times \text{SHARES}_{i,j,t-1}}{\text{TA}_{j,t-1} \times \text{SHVOL}_{i,t}}\right]}_{\text{trading pressure, in share space}} \times \underbrace{\frac{\text{PRC}_{i,t-1}}{\bar{P}_{i,t}}}_{=\,1/(1+\bar{r}_{i,t})} \tag{59.7}$$

where $\bar{r}_{i,t} \equiv \bar{P}_{i,t}/\text{PRC}_{i,t-1} - 1$ is the stock's realized return from the end of quarter $t-1$ to the average price at which it changed hands during quarter t. The measure is the product of a genuine trading-pressure term and a term that *is* the contemporaneous return. Because outflow observations carry $F_{j,t} < 0$, a stock whose price fell during the quarter mechanically receives a more negative MFFlow regardless of what any mutual fund did, and a stock whose price rose receives a less negative one. Regressing quarter-t returns on $\text{MFFlow}_{i,t}$ therefore delivers precisely the sign the fire-sale hypothesis predicts even in a world where the true price impact of flow-induced selling is exactly zero. This is what the paper means when it concludes that the standard approach produces a measure that is inadvertently a direct function of a stock's actual realized return during the outflow quarter.

Two empirical consequences follow, and the second is the more damaging. First, once the embedded return is removed, outflows generate only a small contemporaneous decline in returns. Second, the decline is followed by *no subsequent reversal*. The reversal was never a decorative robustness check: it was the identifying evidence. A price move caused by a liquidity-motivated seller is supposed to be temporary and therefore to unwind, whereas a price move caused by information is permanent. Without the reversal, the empirical pattern is observationally equivalent to funds selling stocks that were genuinely deteriorating, and the exclusion restriction on which the entire research program rests loses its supporting evidence. Wardlaw (2020) reports that many established results in this literature no longer hold once the measure is purged.

The repair is simple to state. Build the measure in share space: compute the hypothetical *number of shares* sold and scale it by shares outstanding at $t-1$ (a flow-to-stock ratio), or by share volume, and keep every price in the construction dated at $t-1$. The general rule this illustrates is worth carrying well beyond the flow literature: *never form a ratio whose numerator is a quantity valued at date $t-1$ prices and whose denominator is a quantity valued at date t prices, because the ratio then contains the return between the two dates by construction.* The same arithmetic is why a price sitting in the denominator of any measure deserves scrutiny, the Amihud illiquidity ratio discussed in Section 14.3.1.1 being the obvious example. The difference there is one of purpose rather than algebra: Amihud is a descriptive measure whose denominator is deliberately in dollars, whereas MFFlow is an instrument whose entire claim is orthogonality to the outcome it is used to explain.

59.6.3 A Second, Independent Problem: Selection

Even a correctly scaled measure inherits a behavioral assumption that is false in the data. Equation 59.6 imputes trades by assuming the redeeming fund liquidates *pro rata*, which is exactly what makes the hypothetical trade independent of the manager's information. Berger (2023) shows that managers do not behave this way: facing redemptions, they preferentially sell liquid, well-performing positions and hold on to illiquid losers. The pro-rata assumption therefore misallocates implied selling pressure toward the firms managers systematically decline to sell, which are the poorly performing, illiquid, low-growth ones. Simulations in that paper indicate the resulting selection bias roughly doubles the magnitude of estimated regression coefficients and that standard fixes do not remove it; after accounting for it, the author finds no feedback effect running from flow-induced mispricing to firm investment, equity issuance, or payout. Read together, Wardlaw (2020) and Berger (2023) make the same methodological point from two directions: a hypothetical trade constructed from a holdings snapshot and an accounting identity is a *model* of institutional behaviour, not an observation of it, and both the scaling and the behavioral assumption have to be defended.

A third caution sits upstream of both. J. J. Li and Zheng (2025) document that the fund flows feeding the numerator are themselves estimates. Most studies infer flows from the change in total net assets in excess of the fund's return, as in Equation 59.5, rather than from reported flows, and the two differ because of reinvested distributions, the timing convention applied within the period, and the treatment of fund mergers. Different flow measures produce materially different inferences about the flow-performance relation and about how flows move security prices, which makes the choice something to document rather than inherit.

59.6.4 What This Means for Vietnamese Research

The US design is hard to transplant directly, for the reason set out at the start of this chapter: there is no 13F analogue, so the quarterly holdings panel that Equation 59.6 requires does not exist for most institutional investors. The domestic open-ended fund industry that does

disclose holdings is small relative to market capitalization, so even a correctly constructed hypothetical sale would be a negligible fraction of a typical HOSE stock's trading volume. That is a recipe for a weak instrument no matter how carefully it is scaled.

The more promising observation is that Vietnam offers a setting in which flow-induced trading does not have to be imputed at all. The exchange-traded funds tracking Vietnamese indices, domestically the DCVFMVN30, DCVFM VNDIAMOND and SSIAM VNFIN LEAD funds and offshore the VanEck Vietnam and Fubon FTSE Vietnam ETFs, publish creations and redemptions at daily frequency against disclosed, rule-based baskets. The trade that a creation or redemption forces is therefore *observable and mechanical* rather than assumed: the fund must transact the basket, in the index weights, on a known schedule. This removes the pro-rata assumption that Berger (2023) attacks, because for an index ETF the pro-rata trade is what genuinely happens, and it removes the quarterly-approximation problem, because the flow is daily. The large Taiwanese retail-driven inflows into the Fubon FTSE Vietnam ETF over 2021 to 2023 are an especially clean demand shock, in that the appetite of Taiwanese retail investors for a Vietnam product is difficult to link to the fundamentals of any individual Vietnamese firm. The VNDIAMOND family is interesting for the opposite reason: its constituents are selected precisely because they sit at or near their foreign ownership limits, so ETF-driven demand interacts directly with the binding constraint analyzed in Section 66.1 and Section 66.2.

Vietnam's price limits give one further reason to keep volume out of the denominator, and it is specific to this market. Trading volume collapses on limit days and can approach zero when a stock locks at the band with an unfilled order queue, which is exactly when the day's return is most extreme. A volume-scaled pressure measure therefore explodes on precisely the observations that dominate the return regression, giving the denominator a second channel through which it imports the outcome variable. How often this happens on HOSE and HNX is documented in Section 85.2, and the censoring it induces in returns is analyzed in Section 85.3. Shares outstanding, which no amount of trading on a given day can change, is the safer scaling variable here for the same reason it is safer in US data.

Six practical rules follow for anyone building a flow-pressure measure on Vietnamese data.

1. **Construct the measure in shares and scale by shares outstanding at $t-1$.** Do not scale by contemporaneous dollar volume.
2. **Date every price in the construction at $t-1$.** If a price from period t appears anywhere in the formula, the measure contains the period-t return.
3. **Report the reversal test as a headline result, not a robustness check.** It is the only evidence separating non-fundamental pressure from information, and after Wardlaw (2020) a flow-pressure result without it should not be believed.
4. **Validate the imputed trade against an observed one wherever possible.** Vietnam publishes daily foreign net buy and sell values by ticker, and ETF baskets are disclosed; both offer a check on whether assumed trading behaviour matches realized trades.

5. **Run the expected-flow placebo.** Replacing realized flows with predicted flows, in the spirit of the expected flow-induced trading construction in D. Lou (2012), tests whether the result is driven by the flow shock or by the holdings pattern.
6. **Report first-stage strength.** With institutional ownership as shallow as it is in Vietnam, a weak instrument is the default expectation and has to be ruled out explicitly.

It is worth noting where this literature has moved. The measurement debate above is, at bottom, about how much price a given quantity of forced trading moves, which is the slope of the aggregate demand curve. That question is now attacked directly by the demand-system approach of Koijen and Yogo (2019) and the inelastic markets hypothesis of Gabaix and Koijen (2021), which infer the price impact of flows from the structure of investor demand rather than from a single instrument. Greenwood and Thesmar (2011), whose fragility measure aggregates the volatility of a stock's owners' flows rather than one quarter's realization, and Frazzini and Lamont (2008), on the return consequences of flow-chasing by retail investors, are useful complements from the earlier tradition. For a researcher entering the field now, the sequence worth reading is Coval and Stafford (2007) and Edmans, Goldstein, and Jiang (2012) for the original design, Wardlaw (2020) and Berger (2023) for what went wrong with it, and Koijen and Yogo (2019) and Gabaix and Koijen (2021) for the framework that has largely displaced it.

60 Data Infrastructure

Table 60.1 summarizes the datasets used in this chapter.

Table 60.1: DataCore.vn Datasets Used in This Chapter

Dataset	Content	Frequency	Key Variables
Stock Prices	Daily/monthly OHLCV	Daily	`ticker`, `date`, `close`, `adjusted_close`, `volume`, `shares_outstanding`
Ownership Structure	Shareholder composition	Quarterly/Annual	`ticker`, `date`, `shareholder_name`, `shares_held`, `pct`, `type`
Major Shareholders	Holders $\geq 5\%$	Event-driven	`ticker`, `date`, `shareholder_name`, `shares`, `is_foreign`, `is_state`
Corporate Actions	Splits, dividends, bonus	Event	`ticker`, `ex_date`, `action_type`, `ratio`
Company Profile	Sector, exchange, FOL	Static/Annual	`ticker`, `exchange`, `industry`, `listing_date`, `fol_limit`
Foreign Ownership	Daily foreign tracking	Daily	`ticker`, `date`, `foreign_shares`, `foreign_pct`, `fol_limit`
Fund Holdings	Fund portfolio snapshots	Semi-annual	`fund_name`, `report_date`, `ticker`, `shares_held`, `market_value`

60.1 Data Reader Class

We begin by defining a unified data reader that handles file loading, date parsing, and basic validation:

```
@dataclass
class DataCoreReader:
    """
    Unified reader for DataCore.vn datasets stored locally.

    Supports Parquet (recommended) and CSV formats. Implements
    lazy loading with caching to minimize memory footprint.

    Parameters
    ----------
    data_dir : str or Path
        Directory containing DataCore.vn data files.
    file_format : str
        File format: 'parquet' or 'csv'.

    Examples
    --------
    >>> dc = DataCoreReader('/data/datacore', file_format='parquet')
    >>> prices = dc.prices
    >>> ownership = dc.ownership
    """
    data_dir: Path
    file_format: str = 'parquet'
    _cache: Dict[str, pd.DataFrame] = field(
        default_factory=dict, repr=False
    )

    FILE_MAP: Dict[str, str] = field(default_factory=lambda: {
        'prices': 'stock_prices',
        'ownership': 'ownership_structure',
        'major_shareholders': 'major_shareholders',
        'corporate_actions': 'corporate_actions',
        'company_profile': 'company_profile',
        'financials': 'financial_statements',
        'foreign_ownership': 'foreign_ownership',
        'fund_holdings': 'fund_holdings',
    }, repr=False)
```

```
    def __post_init__(self):
        self.data_dir = Path(self.data_dir)
        if not self.data_dir.exists():
            raise FileNotFoundError(
                f"Data directory not found: {self.data_dir}"
            )

    def _read(self, key: str) -> pd.DataFrame:
        """Read and cache a dataset with automatic date parsing."""
        if key in self._cache:
            return self._cache[key]

        fname = self.FILE_MAP.get(key, key)
        filepath = self.data_dir / f"{fname}.{self.file_format}"

        if not filepath.exists():
            raise FileNotFoundError(
                f"Dataset not found: {filepath}\n"
                f"Available: "
                f"{list(self.data_dir.glob(f'*.{self.file_format}'))}"
            )

        if self.file_format == 'parquet':
            df = pd.read_parquet(filepath)
        else:
            df = pd.read_csv(filepath, parse_dates=True)

        # Auto-detect and parse date columns
        date_cols = [
            'date', 'ex_date', 'record_date', 'period',
            'report_date', 'listing_date'
        ]
        for col in df.columns:
            if col.lower() in date_cols or 'date' in col.lower():
                try:
                    df[col] = pd.to_datetime(df[col])
                except (ValueError, TypeError):
                    pass

        self._cache[key] = df
        print(f"  Loaded {key}: {len(df):,} rows x {len(df.columns)} cols")
        return df
```

```
    @property
    def prices(self) -> pd.DataFrame:
        return self._read('prices')

    @property
    def ownership(self) -> pd.DataFrame:
        return self._read('ownership')

    @property
    def major_shareholders(self) -> pd.DataFrame:
        return self._read('major_shareholders')

    @property
    def corporate_actions(self) -> pd.DataFrame:
        return self._read('corporate_actions')

    @property
    def company_profile(self) -> pd.DataFrame:
        return self._read('company_profile')

    @property
    def foreign_ownership(self) -> pd.DataFrame:
        return self._read('foreign_ownership')

    @property
    def fund_holdings(self) -> pd.DataFrame:
        return self._read('fund_holdings')

    def clear_cache(self):
        n = len(self._cache)
        self._cache.clear()
        print(f"  Cleared {n} cached datasets")

# Initialize:
# dc = DataCoreReader('/path/to/datacore_data', file_format='parquet')
```

61 Stock Price and Return Processing

The first step processes stock data to obtain adjusted prices, shares outstanding, and quarterly returns.

61.1 Price Data Extraction and Adjustment

Vietnamese stock data requires careful adjustment for frequent corporate actions. Unlike the US where CRSP provides a cumulative adjustment factor (`cfacpr`, `cfacshr`), in Vietnam we must construct adjustment factors from the corporate actions history.

Vietnamese Corporate Actions

Vietnamese firms commonly execute the following corporate actions, each requiring share count and/or price adjustment:

- **Stock dividend** (*co tuc bang co phieu*): e.g., 20% stock dividend means 100 shares become 120 shares
- **Bonus shares** (*co phieu thuong*): free shares distributed from retained earnings
- **Rights issue** (*phat hanh quyen mua*): right to buy new shares at a discount
- **Stock split/reverse split** (*chia/gop co phieu*): rare but occasionally used

```
def build_adjustment_factors(
    corporate_actions: pd.DataFrame,
) -> pd.DataFrame:
    """
    Construct cumulative share adjustment factors from corporate actions.

    This is the Vietnamese equivalent of CRSP's cfacshr factor. For each
    ticker, we compute a cumulative product of adjustment ratios from
    corporate actions, working forward in time.

    The adjustment factor at date t converts historical share counts to
    be comparable with current (post-action) share counts:
```

```
        shares_adjusted_t = shares_raw_t * cfacshr_t

    Parameters
    ----------
    corporate_actions : pd.DataFrame
        Corporate actions with columns: ticker, ex_date, action_type,
        ratio. The ratio field represents:
        - Stock dividend 20%: ratio = 1.20
        - 2:1 stock split: ratio = 2.00
        - Bonus shares 10%: ratio = 1.10

    Returns
    -------
    pd.DataFrame
        Adjustment factors: ticker, ex_date, cfacshr (cumulative).
    """
    share_actions = corporate_actions[
        corporate_actions['action_type'].isin([
            'stock_dividend', 'bonus_shares', 'stock_split',
            'reverse_split', 'rights_issue'
        ])
    ].copy()

    if share_actions.empty:
        return pd.DataFrame(columns=['ticker', 'ex_date', 'cfacshr'])

    share_actions = share_actions.sort_values(['ticker', 'ex_date'])

    share_actions['cfacshr'] = (
        share_actions
        .groupby('ticker')['ratio']
        .cumprod()
    )

    return share_actions[['ticker', 'ex_date', 'cfacshr']].reset_index(
        drop=True
    )


def get_cfacshr_at_date(
    ticker: str,
    date: pd.Timestamp,
    adj_factors: pd.DataFrame,
```

```
) -> float:
    """
    Look up the cumulative share adjustment factor for a given
    ticker and date. Returns 1.0 if no corporate actions occurred.
    """
    mask = (
        (adj_factors['ticker'] == ticker) &
        (adj_factors['ex_date'] <= date)
    )
    subset = adj_factors.loc[mask]

    if subset.empty:
        return 1.0
    return subset.iloc[-1]['cfacshr']


def adjust_shares_between_dates(
    shares: float,
    ticker: str,
    date_from: pd.Timestamp,
    date_to: pd.Timestamp,
    adj_factors: pd.DataFrame,
) -> float:
    """
    Adjust a share count observed at date_from to be comparable
    with shares observed at date_to, accounting for all intervening
    corporate actions.

    Example
    -------
    >>> # Investor held 1000 shares on 2023-01-01
    >>> # A 20% stock dividend occurred on 2023-03-15
    >>> adjust_shares_between_dates(
    ...     1000, 'VNM',
    ...     pd.Timestamp('2023-01-01'),
    ...     pd.Timestamp('2023-06-30'), adj_factors
    ... )
    1200.0
    """
    factor_from = get_cfacshr_at_date(ticker, date_from, adj_factors)
    factor_to = get_cfacshr_at_date(ticker, date_to, adj_factors)
    relative_factor = factor_to / factor_from
    return shares * relative_factor
```

61.2 Monthly and Quarterly Price Processing

```
def process_prices(
    prices: pd.DataFrame,
    adj_factors: pd.DataFrame,
    begdate: str = '2010-01-01',
    enddate: str = '2024-12-31',
) -> Tuple[pd.DataFrame, pd.DataFrame]:
    """
    Process raw DataCore.vn price data into analysis-ready format.

    Block logic:
    1. Filter to date range
    2. Compute adjusted prices and shares outstanding
    3. Compute quarterly compounded returns
    4. Create forward quarterly returns (shifted one quarter)

    Parameters
    ----------
    prices : pd.DataFrame
        Raw price data with: ticker, date, close, adjusted_close,
        volume, shares_outstanding.
    adj_factors : pd.DataFrame
        Corporate action adjustment factors.
    begdate, enddate : str
        Sample period boundaries.

    Returns
    -------
    Tuple[pd.DataFrame, pd.DataFrame]
        (price_quarterly, qret): quarter-end observations with
        adjusted price, total shares, and forward quarterly return.
    """
    price = prices[
        (prices['date'] >= begdate) & (prices['date'] <= enddate)
    ].copy()

    # Month-end and quarter-end dates
    price['mdate'] = price['date'] + pd.offsets.MonthEnd(0)
```

```
price['qdate'] = price['date'] + pd.offsets.QuarterEnd(0)

# Adjusted price
if 'adjusted_close' in price.columns:
    price['p'] = price['adjusted_close']
else:
    price['p'] = price['close']

# Total shares outstanding
price['tso'] = price['shares_outstanding']

# Market capitalization (millions VND)
price['mcap'] = price['p'] * price['tso'] / 1e6

# Filter out zero shares
price = price[price['tso'] > 0].copy()

# Compute daily returns if not present
if 'ret' not in price.columns:
    price = price.sort_values(['ticker', 'date'])
    price['ret'] = price.groupby('ticker')['p'].pct_change()

price['ret'] = price['ret'].fillna(0)
price['logret'] = np.log(1 + price['ret'])

# ---- Quarterly compounded returns ----
qret = (
    price
    .groupby(['ticker', 'qdate'])['logret']
    .sum()
    .reset_index()
)
qret['qret'] = np.exp(qret['logret']) - 1

# Shift qdate back one quarter: make qret a *forward* return
qret['qdate'] = qret['qdate'] + pd.offsets.QuarterEnd(-1)
qret = qret.drop(columns=['logret'])

# ---- Quarter-end observations ----
price_q = price[price['qdate'] == price['mdate']].copy()
price_q = price_q[['qdate', 'ticker', 'p', 'tso', 'mcap']].copy()

# Merge forward quarterly return
```

```
    price_q = price_q.merge(qret, on=['ticker', 'qdate'], how='left')

    # Build cfacshr lookup at each quarter-end
    price_q['cfacshr'] = price_q.apply(
        lambda row: get_cfacshr_at_date(
            row['ticker'], row['qdate'], adj_factors
        ),
        axis=1
    )

    return price_q, qret
```

Performance Optimization

The `get_cfacshr_at_date` function uses a row-wise lookup which can be slow for large datasets. For production use with millions of rows, vectorize using `pd.merge_asof()`:

```
price_q = pd.merge_asof(
    price_q.sort_values('qdate'),
    adj_factors.sort_values('ex_date'),
    by='ticker',
    left_on='qdate',
    right_on='ex_date',
    direction='backward'
).fillna({'cfacshr': 1.0})
```

The output is a quarterly panel of stock-level observations (@tbl-institutional-price-vars)

Table 61.1: Quarter-End Price Panel Variables

Variable	Description
`ticker`	Stock ticker (e.g., VNM, VCB, FPT)
`qdate`	Quarter-end date
`p`	Adjusted closing price (VND)
`tso`	Total shares outstanding
`mcap`	Market capitalization (millions VND)
`qret`	Forward quarterly compounded return
`cfacshr`	Cumulative share adjustment factor

62 Ownership Data Processing

62.1 Ownership Taxonomy

We define a classification system for Vietnamese shareholders that maps to the categories available in DataCore.vn:

```
class OwnershipType:
    """
    Vietnamese ownership type classification.

    Vietnam's ownership structure is fundamentally different from the US:

    - **State** (Nha nuoc): SCIC, ministries, state-owned parents
    - **Foreign Institutional** (To chuc nuoc ngoai): foreign funds,
      ETFs, pension funds, insurance, sovereign wealth funds
    - **Domestic Institutional** (To chuc trong nuoc): Vietnamese
      securities companies, fund managers, banks, insurance
    - **Individual** (Ca nhan): retail investors (domestic + foreign)
    - **Treasury** (Co phieu quy): company repurchases
    """

    STATE = 'State'
    FOREIGN_INST = 'Foreign Institutional'
    DOMESTIC_INST = 'Domestic Institutional'
    INDIVIDUAL = 'Individual'
    TREASURY = 'Treasury'

    INSTITUTIONAL = [FOREIGN_INST, DOMESTIC_INST]
    ALL_INSTITUTIONAL = [STATE, FOREIGN_INST, DOMESTIC_INST]
    ALL_TYPES = [STATE, FOREIGN_INST, DOMESTIC_INST, INDIVIDUAL, TREASURY]

    STATE_KEYWORDS = [
        'scic', 'state capital', 'bo', 'ubnd', 'tong cong ty',
        'nha nuoc', 'state', 'government', "people's committee",
        'ministry', 'vietnam national', 'vnpt', 'evn', 'pvn',
```

```
    ]

    FOREIGN_KEYWORDS = [
        'fund', 'investment', 'capital', 'asset management',
        'securities', 'gic', 'templeton', 'dragon capital',
        'vinacapital', 'mekong capital', 'kb securities',
        'mirae asset', 'samsung', 'jp morgan', 'goldman',
        'blackrock', 'vanguard', 'aberdeen', 'hsbc',
    ]

    @classmethod
    def classify(cls, row: pd.Series) -> str:
        """Classify based on explicit flags, then keyword fallback."""
        if pd.notna(row.get('is_state')) and row['is_state']:
            return cls.STATE
        if pd.notna(row.get('is_foreign')) and row['is_foreign']:
            if pd.notna(row.get('is_institution')) and row['is_institution']:
                return cls.FOREIGN_INST
            return cls.INDIVIDUAL
        if pd.notna(row.get('is_institution')) and row['is_institution']:
            return cls.DOMESTIC_INST

        name = str(row.get('shareholder_name', '')).lower()
        if any(kw in name for kw in cls.STATE_KEYWORDS):
            return cls.STATE
        if any(kw in name for kw in cls.FOREIGN_KEYWORDS):
            return cls.FOREIGN_INST

        return cls.INDIVIDUAL
```

62.2 Building the Holdings Panel

We construct the holdings panel (i.e., the Vietnamese equivalent of merging the 13F Type 1 and Type 3 datasets). The key steps are:

1. Identify the first available vintage for each shareholder-stock-report date combination.
2. Compute reporting gaps to flag first and last reports.
3. Classify shareholders.
4. Adjust shares for corporate actions.

```
def build_holdings_panel(
    ownership: pd.DataFrame,
    adj_factors: pd.DataFrame,
    price_q: pd.DataFrame,
    company_profile: pd.DataFrame,
    begdate: str = '2010-01-01',
    enddate: str = '2024-12-31',
) -> pd.DataFrame:
    """
    Construct the institutional holdings panel from DataCore.vn
    ownership data.
    """
    own = ownership.copy()

    # Align to quarter-end
    own['rdate'] = own['date'] + pd.offsets.QuarterEnd(0)
    own['fdate'] = own['date']

    own = own[
        (own['rdate'] >= begdate) & (own['rdate'] <= enddate)
    ].copy()

    # Keep earliest vintage per shareholder-ticker-rdate
    own = own.sort_values(
        ['shareholder_name', 'ticker', 'rdate', 'fdate']
    )
    fst_vint = (
        own
        .groupby(['shareholder_name', 'ticker', 'rdate'])
        .first()
        .reset_index()
    )

    # ---- Reporting gaps for first/last flags ----
    fst_vint = fst_vint.sort_values(
        ['shareholder_name', 'ticker', 'rdate']
    )

    grp = fst_vint.groupby(['shareholder_name', 'ticker'])
    fst_vint['lag_rdate'] = grp['rdate'].shift(1)

    fst_vint['qtr_gap'] = fst_vint.apply(
        lambda r: (
```

```
            (r['rdate'].to_period('Q')
             - r['lag_rdate'].to_period('Q')).n
            if pd.notna(r['lag_rdate']) else np.nan
        ),
        axis=1
    )

    fst_vint['first_report'] = (
        fst_vint['qtr_gap'].isna() | (fst_vint['qtr_gap'] >= 2)
    )

    # Last report flag (forward gap)
    fst_vint = fst_vint.sort_values(
        ['shareholder_name', 'ticker', 'rdate'],
        ascending=[True, True, False]
    )
    fst_vint['lead_rdate'] = grp['rdate'].shift(1)

    fst_vint['lead_gap'] = fst_vint.apply(
        lambda r: (
            (r['lead_rdate'].to_period('Q')
             - r['rdate'].to_period('Q')).n
            if pd.notna(r['lead_rdate']) else np.nan
        ),
        axis=1
    )

    fst_vint['last_report'] = (
        fst_vint['lead_gap'].isna() | (fst_vint['lead_gap'] >= 2)
    )

    fst_vint = fst_vint.drop(
        columns=['lag_rdate', 'qtr_gap', 'lead_rdate', 'lead_gap'],
        errors='ignore'
    )

    # ---- Classify shareholders ----
    fst_vint['owner_type'] = fst_vint.apply(
        OwnershipType.classify, axis=1
    )

    # ---- Adjust shares for corporate actions ----
    fst_vint = fst_vint.merge(
```

```
        price_q[['ticker', 'qdate', 'cfacshr']],
        left_on=['ticker', 'rdate'],
        right_on=['ticker', 'qdate'],
        how='inner'
    )

    fst_vint['shares_adj'] = (
        fst_vint['shares_held'] * fst_vint['cfacshr']
    )
    fst_vint = fst_vint[fst_vint['shares_adj'] > 0].copy()

    fst_vint = fst_vint.drop_duplicates(
        subset=['shareholder_name', 'ticker', 'rdate']
    )

    # Merge company profile
    if company_profile is not None:
        fst_vint = fst_vint.merge(
            company_profile[['ticker', 'exchange', 'fol_limit']]
            .drop_duplicates(),
            on='ticker',
            how='left'
        )

    cols = [
        'shareholder_name', 'ticker', 'rdate', 'fdate',
        'shares_held', 'shares_adj', 'owner_type',
        'first_report', 'last_report'
    ]
    if 'exchange' in fst_vint.columns:
        cols.extend(['exchange', 'fol_limit'])

    holdings = fst_vint[cols].copy()

    print(f"Holdings panel: {len(holdings):,} observations")
    print(f"  Shareholders: {holdings['shareholder_name'].nunique():,}")
    print(f"  Stocks: {holdings['ticker'].nunique():,}")
    print(f"  Quarters: {holdings['rdate'].nunique()}")

    return holdings
```

63 Institutional Ownership Metrics

Before computing trades, we establish the standard institutional ownership metrics that serve as both outputs and inputs to the trading analysis.

63.1 Institutional Ownership Ratio

The institutional ownership ratio (IO) for stock i at time t is:

$$IO_{i,t} = \frac{\sum_{j \in \mathcal{J}} h_{j,i,t}}{TSO_{i,t}} \tag{63.1}$$

where $\mathcal{J}$ is the set of institutional investors and $TSO_{i,t}$ is total shares outstanding. In Vietnam, we compute separate ratios for each ownership type:

$$IO_{i,t}^{\text{type}} = \frac{\sum_{j \in \mathcal{J}^{\text{type}}} h_{j,i,t}}{TSO_{i,t}}, \quad \text{type} \in \{\text{State}, \text{Foreign}, \text{Domestic}, \text{Individual}\} \tag{63.2}$$

```
def compute_io_ratios(
    holdings: pd.DataFrame,
    price_q: pd.DataFrame,
) -> pd.DataFrame:
    """Compute IO ratios by type for each stock-quarter."""
    agg = (
        holdings
        .groupby(['ticker', 'rdate', 'owner_type'])['shares_adj']
        .sum()
        .reset_index()
    )

    io_wide = agg.pivot_table(
        index=['ticker', 'rdate'],
        columns='owner_type',
        values='shares_adj',
```

```
        fill_value=0
    ).reset_index()

    io_wide.columns = [
        c if c in ['ticker', 'rdate']
        else f'shares_{c.lower().replace(" ", "_")}'
        for c in io_wide.columns
    ]

    io_wide = io_wide.merge(
        price_q[['ticker', 'qdate', 'tso']],
        left_on=['ticker', 'rdate'],
        right_on=['ticker', 'qdate'],
        how='inner'
    )

    share_cols = [c for c in io_wide.columns if c.startswith('shares_')]
    for col in share_cols:
        ratio_name = col.replace('shares_', 'io_')
        io_wide[ratio_name] = io_wide[col] / io_wide['tso']

    inst_cols = [
        c for c in io_wide.columns
        if c.startswith('shares_')
        and 'individual' not in c
        and 'treasury' not in c
    ]
    io_wide['io_total_inst'] = (
        io_wide[inst_cols].sum(axis=1) / io_wide['tso']
    )

    return io_wide
```

63.2 Ownership Concentration: Herfindahl-Hirschman Index

The HHI measures ownership concentration:

$$HHI_{i,t} = \sum_{j=1}^{N_{i,t}} \left(\frac{h_{j,i,t}}{\sum_{k=1}^{N_{i,t}} h_{k,i,t}} \right)^2 \tag{63.3}$$

where $N_{i,t}$ is the number of shareholders. HHI ranges from $1/N_{i,t}$ (equal) to 1 (single shareholder). In Vietnam, ownership tends to be highly concentrated due to large state and founding-family blocks.

```
def compute_hhi(holdings: pd.DataFrame) -> pd.DataFrame:
    """Compute HHI for each stock-quarter, overall and institutional."""
    def _hhi(shares: pd.Series) -> float:
        total = shares.sum()
        if total <= 0:
            return np.nan
        weights = shares / total
        return (weights ** 2).sum()

    hhi_overall = (
        holdings.groupby(['ticker', 'rdate'])['shares_adj']
        .apply(_hhi).reset_index()
        .rename(columns={'shares_adj': 'hhi_overall'})
    )

    inst = holdings[
        holdings['owner_type'].isin(OwnershipType.ALL_INSTITUTIONAL)
    ]
    hhi_inst = (
        inst.groupby(['ticker', 'rdate'])['shares_adj']
        .apply(_hhi).reset_index()
        .rename(columns={'shares_adj': 'hhi_institutional'})
    )

    return hhi_overall.merge(hhi_inst, on=['ticker', 'rdate'], how='left')
```

63.3 Ownership Breadth

Following H.-L. Chen, Jegadeesh, and Wermers (2000), ownership breadth is the number of institutional holders:

$$\text{Breadth}_{i,t} = \#\{j : h_{j,i,t} > 0,\, j \in \mathcal{J}\} \tag{63.4}$$

The *change* in breadth predicts future returns:

$$\Delta\text{Breadth}_{i,t} = \text{Breadth}_{i,t} - \text{Breadth}_{i,t-1} \tag{63.5}$$

```
def compute_breadth(holdings: pd.DataFrame) -> pd.DataFrame:
    """Compute ownership breadth and changes by type."""
    breadth = (
        holdings[
            holdings['owner_type'].isin(OwnershipType.ALL_INSTITUTIONAL)
        ]
        .groupby(['ticker', 'rdate', 'owner_type'])['shareholder_name']
        .nunique()
        .reset_index()
        .rename(columns={'shareholder_name': 'n_holders'})
    )

    breadth_wide = breadth.pivot_table(
        index=['ticker', 'rdate'],
        columns='owner_type',
        values='n_holders',
        fill_value=0
    ).reset_index()

    breadth_wide.columns = [
        c if c in ['ticker', 'rdate']
        else f'n_{c.lower().replace(" ", "_")}'
        for c in breadth_wide.columns
    ]

    n_cols = [c for c in breadth_wide.columns if c.startswith('n_')]
    breadth_wide['n_total_inst'] = breadth_wide[n_cols].sum(axis=1)

    breadth_wide = breadth_wide.sort_values(['ticker', 'rdate'])
    for col in n_cols + ['n_total_inst']:
        breadth_wide[f'd_{col}'] = (
            breadth_wide.groupby('ticker')[col].diff()
        )

    return breadth_wide
```

(BS $= -1$) is generated for the prior position, dated to the quarter after the last report.

For intermediate gaps (reports at $t-2$ and t but not $t-1$), we split into:

- A terminating sale at $t-1$ of $-h^{\text{adj}}_{j,i,t-2}$;
- An initiating buy at t of $h_{j,i,t}$.

63.4 Implementation

```
def compute_trades(
    holdings: pd.DataFrame,
    adj_factors: pd.DataFrame,
) -> pd.DataFrame:
    """
    Compute institutional trades from holdings panel.

    Uses vectorized conditional logic (NOT apply()) for performance.

    Algorithm:
    1. Sort holdings by shareholder, ticker, quarter
    2. Compute lagged holdings and reporting gaps
    3. Apply modified trade logic based on first_report, gap
    4. Handle terminating sales and intermediate gaps
    5. Append all trade records
    """
    t1 = holdings.sort_values(
        ['shareholder_name', 'ticker', 'rdate']
    ).copy()

    # Previous holding quarter and shares
    grp = t1.groupby(['shareholder_name', 'ticker'])
    t1['phrdate'] = grp['rdate'].shift(1)
    t1['pshares_adj'] = grp['shares_adj'].shift(1)

    # Raw trade
    t1['trade'] = t1['shares_adj'] - t1['pshares_adj']

    # Quarter gap
    t1['qtrgap'] = t1.apply(
        lambda r: (
            (r['rdate'].to_period('Q')
             - r['phrdate'].to_period('Q')).n
            if pd.notna(r['phrdate']) else np.nan
        ),
        axis=1
    )

    # Boundary detection keys
    t1['l_key'] = (
```

```
    t1['shareholder_name'] + '_' + t1['ticker']
).shift(1)
t1['n_key'] = (
    t1['shareholder_name'] + '_' + t1['ticker']
).shift(-1)
t1['curr_key'] = t1['shareholder_name'] + '_' + t1['ticker']

# ---- Vectorized trade classification ----
is_new = (t1['curr_key'] != t1['l_key'])
not_first = ~t1['first_report']
consec = (t1['qtrgap'] == 1)
gap = (t1['qtrgap'] != 1) & t1['qtrgap'].notna()

cond1   = is_new
cond1_1 = is_new & not_first
cond2_1 = (~is_new) & not_first & consec
cond2_2 = (~is_new) & not_first & gap

# Modified trade amounts
t1['modtrade'] = t1['trade']
t1.loc[cond1, 'modtrade'] = np.nan
t1.loc[cond1_1, 'modtrade'] = t1.loc[cond1_1, 'shares_adj']
t1.loc[cond2_1, 'modtrade'] = t1.loc[cond2_1, 'trade']
t1.loc[cond2_2, 'modtrade'] = t1.loc[cond2_2, 'shares_adj']

# Buy/sale classification
t1['buysale'] = np.nan
t1.loc[cond1_1, 'buysale'] = 1
t1.loc[cond2_1, 'buysale'] = (
    2 * np.sign(t1.loc[cond2_1, 'trade'])
)
t1.loc[cond2_2, 'buysale'] = 1.5  # placeholder for split

# ---- Handle intermediate gaps (buysale == 1.5) ----
t2 = t1[t1['buysale'] == 1.5].copy()
t2['rdate'] = t2['phrdate'] + pd.offsets.QuarterEnd(1)
t2['buysale'] = -1
t2['modtrade'] = -t2['pshares_adj']

t1.loc[t1['buysale'] == 1.5, 'buysale'] = 1

# ---- Terminating sales ----
is_last_combo = (t1['curr_key'] != t1['n_key'])
```

```
    not_last_rpt = ~t1['last_report']

    t3 = t1[is_last_combo & not_last_rpt].copy()
    t3['rdate'] = t3['rdate'] + pd.offsets.QuarterEnd(1)
    t3['modtrade'] = -t3['shares_adj']
    t3['buysale'] = -1

    # ---- Combine ----
    trades = pd.concat([t1, t2, t3], ignore_index=True)
    trades = trades[
        (trades['modtrade'] != 0) &
        trades['modtrade'].notna() &
        trades['buysale'].notna()
    ].copy()

    trades = trades[[
        'rdate', 'shareholder_name', 'ticker', 'modtrade',
        'buysale', 'owner_type', 'first_report', 'last_report'
    ]].rename(columns={'modtrade': 'trade'})

    print(f"\nTrade computation complete:")
    print(f"  Total records: {len(trades):,}")
    print(f"  Initiating buys:  {(trades['buysale'] == 1).sum():,}")
    print(f"  Incremental buys: {(trades['buysale'] == 2).sum():,}")
    print(f"  Terminating sales:{(trades['buysale'] == -1).sum():,}")
    print(f"  Regular sales:    {(trades['buysale'] == -2).sum():,}")

    return trades
```

63.4.1 Trade Visualization

```
def plot_trade_distribution(trades: pd.DataFrame):
    """Plot time series of trade types by quarter."""
    bs_labels = {
        1: 'Initiating Buy', 2: 'Incremental Buy',
        -1: 'Terminating Sale', -2: 'Regular Sale'
    }
    trades = trades.copy()
    trades['trade_type'] = trades['buysale'].map(bs_labels)

    counts = (
        trades
        .groupby([pd.Grouper(key='rdate', freq='QE'), 'trade_type'])
        .size()
        .unstack(fill_value=0)
    )

    fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

    buy_cols = [c for c in counts.columns if 'Buy' in c]
    counts[buy_cols].plot(
        kind='bar', stacked=True, ax=axes[0],
        color=['#1f77b4', '#aec7e8'], width=0.8
    )
    axes[0].set_title('Panel A: Institutional Purchases', fontweight='bold')
    axes[0].set_ylabel('Number of Trades')

    sale_cols = [c for c in counts.columns if 'Sale' in c]
    counts[sale_cols].plot(
        kind='bar', stacked=True, ax=axes[1],
        color=['#d62728', '#ff9896'], width=0.8
    )
    axes[1].set_title('Panel B: Institutional Sales', fontweight='bold')
    axes[1].set_ylabel('Number of Trades')

    for ax in axes:
        ax.tick_params(axis='x', rotation=45)
        for i, label in enumerate(ax.get_xticklabels()):
            if i % 4 != 0:
                label.set_visible(False)

    plt.tight_layout()
    plt.show()

# plot_trade_distribution(trades)
```

Figure 63.1

```
def plot_net_trading_by_type(trades: pd.DataFrame, price_q: pd.DataFrame):
    """Plot net trading volume by owner type over time."""
    _t = trades.merge(
        price_q[['ticker', 'qdate', 'p']],
        left_on=['ticker', 'rdate'],
        right_on=['ticker', 'qdate'],
        how='inner'
    )
    _t['trade_vnd'] = _t['trade'] * _t['p'] / 1e9

    net = (
        _t
        .groupby([pd.Grouper(key='rdate', freq='QE'), 'owner_type'])
        ['trade_vnd'].sum()
        .unstack(fill_value=0)
    )

    fig, ax = plt.subplots(figsize=(12, 6))
    for col in net.columns:
        ax.plot(net.index, net[col], label=col,
                color=OWNER_COLORS.get(col, '#333'), linewidth=1.5)
    ax.axhline(y=0, color='black', linewidth=0.5)
    ax.set_title('Net Institutional Trading by Ownership Type',
                 fontweight='bold')
    ax.set_ylabel('Net Trading (Billions VND)')
    ax.legend(loc='best')
    plt.tight_layout()
    plt.show()

# plot_net_trading_by_type(trades, price_q)
```

Figure 63.2

64 Portfolio Assets, Flows, and Returns

This section computes total portfolio assets, aggregates buys and sales, and portfolio-level returns for each institutional investor.

64.1 Total Assets and Portfolio Returns

For each manager j and quarter t, portfolio assets are:

$$A_{j,t} = \sum_{i=1}^{N_{j,t}} h_{j,i,t} \cdot P_{i,t} \tag{64.1}$$

The portfolio return assuming buy-and-hold is:

$$R_{j,t}^{p} = \frac{\sum_{i=1}^{N_{j,t}} h_{j,i,t} \cdot P_{i,t} \cdot r_{i,t+1}}{\sum_{i=1}^{N_{j,t}} h_{j,i,t} \cdot P_{i,t}} \tag{64.2}$$

```
def compute_assets_and_returns(
    holdings: pd.DataFrame,
    price_q: pd.DataFrame,
) -> pd.DataFrame:
    """Compute total portfolio assets and buy-and-hold returns."""
    _assets = holdings[
        ['shareholder_name', 'ticker', 'rdate', 'shares_adj']
    ].merge(
        price_q[['ticker', 'qdate', 'p', 'qret']],
        left_on=['ticker', 'rdate'],
        right_on=['ticker', 'qdate'],
        how='inner'
    )

    _assets['hold_per_stock'] = _assets['shares_adj'] * _assets['p'] / 1e6
    _assets['next_value'] = (
```

```
        _assets['shares_adj'] * _assets['p'] * _assets['qret']
    )
    _assets['curr_value'] = _assets['shares_adj'] * _assets['p']

    assets = (
        _assets
        .groupby(['shareholder_name', 'rdate'])
        .agg(
            assets=('hold_per_stock', 'sum'),
            total_next=('next_value', 'sum'),
            total_curr=('curr_value', 'sum'),
        )
        .reset_index()
    )

    assets['pret'] = assets['total_next'] / assets['total_curr']
    assets = assets.drop(columns=['total_next', 'total_curr'])
    return assets
```

64.2 Aggregate Buys and Sales

Total buys and sales for manager j in quarter t:

$$B_{j,t} = \sum_{i:\Delta h > 0} \Delta h_{j,i,t} \cdot P_{i,t}, \qquad S_{j,t} = \sum_{i:\Delta h < 0} |\Delta h_{j,i,t}| \cdot P_{i,t} \tag{64.3}$$

The trade gain is:

$$G_{j,t} = \sum_{i=1}^{N_{j,t}} \Delta h_{j,i,t} \cdot P_{i,t} \cdot r_{i,t+1} \tag{64.4}$$

```
def compute_buys_sales(
    trades: pd.DataFrame,
    price_q: pd.DataFrame,
) -> pd.DataFrame:
    """Compute aggregate buys, sales, trade gains per manager-quarter."""
    _flows = trades.merge(
        price_q[['ticker', 'qdate', 'p', 'qret']],
        left_on=['ticker', 'rdate'],
```

```
        right_on=['ticker', 'qdate'],
        how='inner'
    )

    _flows['tbuys'] = (
        _flows['trade'] * (_flows['trade'] > 0).astype(float)
        * _flows['p'] / 1e6
    )
    _flows['tsales'] = (
        (-1) * _flows['trade'] * (_flows['trade'] < 0).astype(float)
        * _flows['p'] / 1e6
    )
    _flows['tgain'] = (
        _flows['trade'] * _flows['p'] * _flows['qret'] / 1e6
    )

    flows = (
        _flows
        .groupby(['shareholder_name', 'rdate'])
        .agg(
            tbuys=('tbuys', 'sum'),
            tsales=('tsales', 'sum'),
            tgain=('tgain', 'sum'),
        )
        .reset_index()
    )
    return flows
```

65 Net Flows and Turnover Ratios

65.1 Net Flows

Net flows separate capital allocation decisions from investment returns:

$$\text{NetFlow}_{j,t} = A_{j,t} - A_{j,t-1}(1 + R^{p}_{j,t}) \tag{65.1}$$

Interpreting Net Flows in Vietnam

For state entities or corporate cross-holders, "net flows" do not necessarily reflect investment decisions. State ownership changes often result from government policy (equitization, divestment programs). Interpretation should account for institutional context.

65.2 Three Turnover Measures

```
def compute_aggregates(
    holdings: pd.DataFrame,
    assets: pd.DataFrame,
    flows: pd.DataFrame,
) -> pd.DataFrame:
    """
    Compute net flows and three turnover measures.

    1. Carhart (1997): min(buys, sales) / avg(assets)
    2. Flow-adjusted: [min(buys, sales) + |net flows|] / lag assets
    3. Symmetric: [buys + sales - |net flows|] / lag assets
    """
    report_flags = (
        holdings
        .groupby(['shareholder_name', 'rdate'])
        .agg(first_report=('first_report', 'any'),
```

```
            last_report=('last_report', 'any'))
        .reset_index()
    )

    agg = report_flags.merge(
        assets, on=['shareholder_name', 'rdate'], how='inner'
    )
    agg = agg.merge(
        flows, on=['shareholder_name', 'rdate'], how='left'
    )

    agg = agg.sort_values(['shareholder_name', 'rdate'])

    agg['assets_comp'] = agg['assets'] * (1 + agg['pret'].fillna(0))

    grp = agg.groupby('shareholder_name')
    agg['lassets_comp'] = grp['assets_comp'].shift(1)
    agg['lassets'] = grp['assets'].shift(1)

    # Trade gain return
    agg['tgainret'] = agg['tgain'] / (agg['tbuys'] + agg['tsales'])

    # Net flows
    agg['netflows'] = agg['assets'] - agg['lassets_comp']

    # Turnover 1: Carhart (1997)
    agg['turnover1'] = (
        agg[['tbuys', 'tsales']].min(axis=1) /
        agg[['assets', 'lassets']].mean(axis=1)
    )

    # Turnover 2: Flow-adjusted
    agg['turnover2'] = (
        (agg[['tbuys', 'tsales']].min(axis=1)
         + agg['netflows'].abs().fillna(0))
        / agg['lassets']
    )

    # Turnover 3: Symmetric
    agg['turnover3'] = (
        (agg['tbuys'].fillna(0) + agg['tsales'].fillna(0)
         - agg['netflows'].abs().fillna(0))
        / agg['lassets']
```

```
    )

    # Missing for first report
    first_mask = agg['first_report']
    for col in ['netflows', 'tgainret',
                'turnover1', 'turnover2', 'turnover3']:
        agg.loc[first_mask, col] = np.nan

    agg = agg.drop(columns=['assets_comp', 'lassets_comp', 'lassets'])

    print(f"\nAggregates: {len(agg):,} manager-quarters")
    print(f"  Turnover1 mean: {agg['turnover1'].mean():.4f}")
    print(f"  Turnover2 mean: {agg['turnover2'].mean():.4f}")
    print(f"  Turnover3 mean: {agg['turnover3'].mean():.4f}")

    return agg
```

65.2.1 Turnover Summary Statistics

Table 65.1: Summary statistics for three turnover measures across institutional investor types in Vietnam. Turnover 1 follows Mark M. Carhart (1997b), Turnover 2 adds back absolute net flows, and Turnover 3 uses the symmetric definition.

```
def turnover_summary_table(
    aggregates: pd.DataFrame,
    holdings: pd.DataFrame,
) -> pd.DataFrame:
    """Publication-quality turnover summary statistics table."""
    owner_map = (
        holdings.groupby('shareholder_name')['owner_type']
        .first().reset_index()
    )
    agg = aggregates.merge(owner_map, on='shareholder_name', how='left')

    turnover_cols = ['turnover1', 'turnover2', 'turnover3']
    results = []

    for otype in ['All'] + OwnershipType.ALL_TYPES:
        subset = agg if otype == 'All' else agg[agg['owner_type'] == otype]
        row = {'Owner Type': otype, 'N': len(subset)}
        for col in turnover_cols:
            s = subset[col].dropna()
            row[f'{col}_mean'] = s.mean()
            row[f'{col}_median'] = s.median()
            row[f'{col}_std'] = s.std()
        results.append(row)

    return pd.DataFrame(results).round(4)

# turnover_summary_table(aggregates, holdings)
```

```
def plot_turnover_timeseries(
    aggregates: pd.DataFrame, holdings: pd.DataFrame
):
    """Plot turnover time series by ownership type."""
    owner_map = (
        holdings.groupby('shareholder_name')['owner_type']
        .first().reset_index()
    )
    agg = aggregates.merge(owner_map, on='shareholder_name', how='left')

    fig, ax = plt.subplots(figsize=(12, 6))
    for otype in OwnershipType.ALL_INSTITUTIONAL:
        subset = agg[agg['owner_type'] == otype]
        qtr_mean = (
            subset
            .groupby(pd.Grouper(key='rdate', freq='QE'))['turnover1']
            .mean()
        )
        ax.plot(qtr_mean.index, qtr_mean.values, label=otype,
                color=OWNER_COLORS.get(otype, '#333'), linewidth=1.5)

    ax.set_title('Quarterly Average Turnover (Carhart)',
                 fontweight='bold')
    ax.set_ylabel('Turnover Ratio')
    ax.legend(loc='best')
    ax.yaxis.set_major_formatter(mticker.PercentFormatter(1.0))
    plt.tight_layout()
    plt.show()

# plot_turnover_timeseries(aggregates, holdings)
```

Figure 65.1

66 Foreign Ownership Analytics

Vietnam's foreign ownership limits create unique analytical dimensions absent from developed market studies.

66.1 FOL Utilization

$$\text{FOL_Util}_{i,t} = \frac{FO_{i,t}}{FOL_i} \tag{66.1}$$

Stocks with $\text{FOL_Util}_{i,t} \to 1$ face mechanical foreign buying restrictions.

```
def compute_fol_analytics(
    foreign_ownership: pd.DataFrame,
    company_profile: pd.DataFrame,
) -> pd.DataFrame:
    """Compute FOL utilization and related metrics."""
    fo = foreign_ownership.copy()
    fo = fo.merge(
        company_profile[['ticker', 'fol_limit']].drop_duplicates(),
        on='ticker', how='left'
    )

    fo['fol_utilization'] = fo['foreign_pct'] / fo['fol_limit']
    fo['foreign_room'] = fo['fol_limit'] - fo['foreign_pct']
    fo['fol_binding'] = (fo['fol_utilization'] >= 0.98)
    fo['fol_category'] = pd.cut(
        fo['fol_utilization'],
        bins=[0, 0.25, 0.50, 0.75, 0.95, 1.0, float('inf')],
        labels=['<25%', '25-50%', '50-75%', '75-95%',
                '95-100%', '>100%']
    )
    return fo
```

66.2 Room Premium Regression

When foreign ownership approaches the FOL, remaining "room" becomes scarce. We model:

$$r_{i,t+1} = \alpha + \beta_1 \cdot \text{FOL_Util}_{i,t} + \beta_2 \cdot \text{FOL_Util}_{i,t}^2 + \gamma \cdot X_{i,t} + \varepsilon_{i,t} \tag{66.2}$$

The quadratic term captures nonlinear acceleration of the premium as ownership approaches the limit.

```
def estimate_room_premium(
    fol_analytics: pd.DataFrame,
    price_q: pd.DataFrame,
) -> dict:
    """Estimate foreign ownership room premium via panel regression."""
    fol_q = (
        fol_analytics
        .assign(qdate=lambda x: x['date'] + pd.offsets.QuarterEnd(0))
        .groupby(['ticker', 'qdate'])
        .agg(fol_utilization=('fol_utilization', 'last'),
             foreign_room=('foreign_room', 'last'))
        .reset_index()
    )

    panel = fol_q.merge(
        price_q[['ticker', 'qdate', 'mcap', 'qret']],
        on=['ticker', 'qdate'], how='inner'
    )

    panel['log_mcap'] = np.log(panel['mcap'] + 1)
    panel['fol_util_sq'] = panel['fol_utilization'] ** 2
    panel = panel.dropna(subset=['qret', 'fol_utilization', 'log_mcap'])

    X = panel[['fol_utilization', 'fol_util_sq', 'log_mcap']]
    X = sm.add_constant(X)
    y = panel['qret']

    model = sm.OLS(y, X).fit(
        cov_type='cluster', cov_kwds={'groups': panel['ticker']}
    )
    return {'model': model, 'n_obs': len(panel)}

# results = estimate_room_premium(fol_analytics, price_q)
```

```
def plot_fol_utilization(fol_analytics: pd.DataFrame):
    """Plot FOL utilization distribution."""
    latest = (
        fol_analytics.sort_values(['ticker', 'date'])
        .groupby('ticker').last().reset_index()
    )

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    axes[0].hist(latest['fol_utilization'].dropna(), bins=50,
                 color='#1f77b4', alpha=0.7, edgecolor='white')
    axes[0].axvline(x=0.95, color='red', linestyle='--',
                    label='95% threshold')
    axes[0].set_title('Panel A: FOL Utilization Distribution',
                      fontweight='bold')
    axes[0].set_xlabel('FOL Utilization Ratio')
    axes[0].set_ylabel('Number of Stocks')
    axes[0].legend()

    for exch in ['HOSE', 'HNX', 'UPCOM']:
        sub = latest[latest.get('exchange') == exch]
        if len(sub) > 0:
            axes[1].hist(sub['fol_utilization'].dropna(), bins=30,
                         alpha=0.5, label=exch,
                         color=EXCHANGE_COLORS.get(exch, '#333'))
    axes[1].set_title('Panel B: By Exchange', fontweight='bold')
    axes[1].set_xlabel('FOL Utilization Ratio')
    axes[1].legend()

    plt.tight_layout()
    plt.show()

# plot_fol_utilization(fol_analytics)
```

Figure 66.1

67 Complete Pipeline

We integrate all steps into a single end-to-end function:

```
def run_complete_pipeline(
    dc: 'DataCoreReader',
    begdate: str = '2010-01-01',
    enddate: str = '2024-12-31',
) -> Dict[str, pd.DataFrame]:
    """
    Execute the complete institutional ownership analytics pipeline.

    Steps:
    1. Build corporate action adjustment factors
    2. Process stock prices
    3. Construct holdings panel (Steps 2-4)
    4. Compute IO metrics
    5. Compute institutional trades (Step 5)
    6. Compute portfolio assets and returns (Step 6a)
    7. Compute aggregate buys, sales, trade gains (Step 6b)
    8. Compute net flows and turnover (Step 7)
    9. Compute foreign ownership analytics

    Returns dict of all output DataFrames.
    """
    print("=" * 60)
    print("INSTITUTIONAL TRADES, FLOWS, AND TURNOVER PIPELINE")
    print(f"Sample: {begdate} to {enddate}")
    print("=" * 60)

    print("\n[1/9] Building adjustment factors...")
    adj_factors = build_adjustment_factors(dc.corporate_actions)

    print("\n[2/9] Processing stock prices...")
    price_q, qret = process_prices(
        dc.prices, adj_factors, begdate, enddate
    )
```

```
    print("\n[3/9] Building holdings panel...")
    holdings = build_holdings_panel(
        dc.ownership, adj_factors, price_q,
        dc.company_profile, begdate, enddate
    )

    print("\n[4/9] Computing ownership metrics...")
    io_ratios = compute_io_ratios(holdings, price_q)
    hhi = compute_hhi(holdings)
    breadth = compute_breadth(holdings)

    print("\n[5/9] Computing institutional trades...")
    trades = compute_trades(holdings, adj_factors)

    print("\n[6/9] Computing portfolio assets...")
    assets = compute_assets_and_returns(holdings, price_q)

    print("\n[7/9] Computing aggregate buys and sales...")
    flows = compute_buys_sales(trades, price_q)

    print("\n[8/9] Computing net flows and turnover...")
    aggregates = compute_aggregates(holdings, assets, flows)

    print("\n[9/9] Computing foreign ownership analytics...")
    fol_analytics = compute_fol_analytics(
        dc.foreign_ownership, dc.company_profile
    )

    print("\n" + "=" * 60)
    print("PIPELINE COMPLETE")
    print("=" * 60)

    return {
        'price_q': price_q, 'holdings': holdings,
        'io_ratios': io_ratios, 'hhi': hhi,
        'breadth': breadth, 'trades': trades,
        'assets': assets, 'flows': flows,
        'aggregates': aggregates, 'fol_analytics': fol_analytics,
    }

# dc = DataCoreReader('/path/to/datacore_data', file_format='parquet')
# results = run_complete_pipeline(dc, '2010-01-01', '2024-12-31')
```

68 Advanced Extensions

68.1 Herding Measures

Following Sias (2004), the Lakonishok-Shleifer-Vishny herding measure is:

$$HM_{i,t} = \left| \frac{B_{i,t}}{B_{i,t} + S_{i,t}} - p_t \right| - E\left[\left| \frac{B_{i,t}}{B_{i,t} + S_{i,t}} - p_t \right| \right] \tag{68.1}$$

where $B_{i,t}$ is the number of managers buying stock i in quarter t, $S_{i,t}$ the number selling, and p_t the expected buyer proportion under independent trading.

```
def compute_lsv_herding(
    trades: pd.DataFrame,
    min_traders: int = 5,
) -> pd.DataFrame:
    """Compute LSV herding measure for each stock-quarter."""
    tc = (
        trades.groupby(['ticker', 'rdate'])
        .apply(lambda g: pd.Series({
            'n_buyers': (g['trade'] > 0).sum(),
            'n_sellers': (g['trade'] < 0).sum(),
            'n_traders': len(g),
        }))
        .reset_index()
    )

    tc = tc[tc['n_traders'] >= min_traders].copy()
    tc['buy_prop'] = tc['n_buyers'] / tc['n_traders']
    tc['p_t'] = tc.groupby('rdate')['buy_prop'].transform('mean')
    tc['raw_hm'] = (tc['buy_prop'] - tc['p_t']).abs()

    def expected_abs_deviation(row):
        n = int(row['n_traders'])
        p = row['p_t']
        if n == 0 or p == 0 or p == 1:
```

```
            return 0
        from scipy.stats import binom
        k = np.arange(0, n + 1)
        probs = binom.pmf(k, n, p)
        return np.sum(np.abs(k / n - p) * probs)

    tc['expected_hm'] = tc.apply(expected_abs_deviation, axis=1)
    tc['herding'] = tc['raw_hm'] - tc['expected_hm']

    tc['buy_herding'] = np.where(
        tc['buy_prop'] > tc['p_t'], tc['herding'], np.nan
    )
    tc['sell_herding'] = np.where(
        tc['buy_prop'] < tc['p_t'], tc['herding'], np.nan
    )

    return tc[['ticker', 'rdate', 'n_buyers', 'n_sellers',
               'n_traders', 'herding', 'buy_herding', 'sell_herding']]
```

68.2 Demand Persistence

Sias (2004) showed institutional demand is persistent:

$$\rho_t = \text{Corr}\left(\Delta IO_{i,t},\, \Delta IO_{i,t-1}\right) \tag{68.2}$$

```
def compute_demand_persistence(io_ratios: pd.DataFrame) -> pd.DataFrame:
    """Rolling cross-sectional correlation of IO changes."""
    io = io_ratios[['ticker', 'rdate', 'io_total_inst']].copy()
    io = io.sort_values(['ticker', 'rdate'])
    io['dio'] = io.groupby('ticker')['io_total_inst'].diff()
    io['lag_dio'] = io.groupby('ticker')['dio'].shift(1)

    persistence = (
        io.dropna(subset=['dio', 'lag_dio'])
        .groupby('rdate')
        .apply(lambda g: g['dio'].corr(g['lag_dio']))
        .reset_index()
        .rename(columns={0: 'persistence'})
    )
    persistence = persistence.sort_values('rdate')
```

```
    persistence['persistence_ma'] = (
        persistence['persistence'].rolling(window=20, min_periods=4).mean()
    )
    return persistence
```

```
def plot_demand_persistence(persistence: pd.DataFrame):
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.bar(persistence['rdate'], persistence['persistence'],
           width=80, alpha=0.3, color='#1f77b4', label='Quarterly')
    ax.plot(persistence['rdate'], persistence['persistence_ma'],
            color='#d62728', linewidth=2, label='Rolling Average')
    ax.axhline(y=0, color='black', linewidth=0.5)
    ax.set_title('Persistence of Institutional Demand', fontweight='bold')
    ax.set_ylabel('Cross-Sectional Correlation')
    ax.legend()
    plt.tight_layout()
    plt.show()
```

Figure 68.1

68.3 Information Content of Trades

Following Alexander, Cici, and Gibson (2007), the InfoTrade ratio measures the proportion of dollar trading from entry/exit decisions vs. position adjustments:

$$\text{InfoTrade}_{i,t} = \frac{\sum_{j:BS\in\{+1,-1\}} |\Delta h_{j,i,t}| \cdot P_{i,t}}{\sum_{j} |\Delta h_{j,i,t}| \cdot P_{i,t}} \tag{68.3}$$

```
def compute_info_trade_ratio(
    trades: pd.DataFrame, price_q: pd.DataFrame
) -> pd.DataFrame:
    """Compute info trade ratio for each stock-quarter."""
    _t = trades.merge(
        price_q[['ticker', 'qdate', 'p']],
        left_on=['ticker', 'rdate'],
        right_on=['ticker', 'qdate'],
        how='inner'
    )
```

```
    _t['dollar_trade'] = _t['trade'].abs() * _t['p'] / 1e6
    _t['is_discrete'] = _t['buysale'].isin([1, -1])

    info = _t.groupby(['ticker', 'rdate']).apply(
        lambda g: pd.Series({
            'discrete_vol': g.loc[g['is_discrete'], 'dollar_trade'].sum(),
            'total_vol': g['dollar_trade'].sum(),
        })
    ).reset_index()

    info['info_trade_ratio'] = (
        info['discrete_vol'] / info['total_vol']
    ).clip(0, 1)
    return info
```

69 Empirical Applications

69.1 Application 1: Institutional Ownership Changes and Future Returns

We test whether changes in institutional ownership predict future stock returns (H.-L. Chen, Jegadeesh, and Wermers 2000) via Fama-MacBeth regressions:

$$r_{i,t+1} = \alpha_t + \beta_{1,t} \cdot \Delta IO_{i,t} + \beta_{2,t} \cdot \Delta \text{Breadth}_{i,t} + \gamma_t \cdot X_{i,t} + \varepsilon_{i,t} \tag{69.1}$$

```
def fama_macbeth_io_returns(
    io_ratios: pd.DataFrame,
    breadth: pd.DataFrame,
    price_q: pd.DataFrame,
) -> pd.DataFrame:
    """Run Fama-MacBeth regressions of future returns on IO changes."""
    panel = io_ratios[['ticker', 'rdate', 'io_total_inst']].merge(
        breadth[['ticker', 'rdate', 'n_total_inst', 'd_n_total_inst']],
        on=['ticker', 'rdate'], how='inner'
    ).merge(
        price_q[['ticker', 'qdate', 'mcap', 'qret']],
        left_on=['ticker', 'rdate'],
        right_on=['ticker', 'qdate'],
        how='inner'
    )

    panel = panel.sort_values(['ticker', 'rdate'])
    panel['dio'] = panel.groupby('ticker')['io_total_inst'].diff()
    panel['log_mcap'] = np.log(panel['mcap'] + 1)
    panel['mom'] = panel.groupby('ticker')['qret'].shift(1)

    reg_vars = ['qret', 'dio', 'd_n_total_inst', 'log_mcap', 'mom']
    panel = panel.dropna(subset=reg_vars)

    quarters = sorted(panel['rdate'].unique())
```

```
    results = []

    for q in quarters:
        qdata = panel[panel['rdate'] == q]
        if len(qdata) < 30:
            continue
        X = sm.add_constant(
            qdata[['dio', 'd_n_total_inst', 'log_mcap', 'mom']]
        )
        try:
            model = sm.OLS(qdata['qret'], X).fit()
            coefs = model.params.to_dict()
            coefs['rdate'] = q
            coefs['n_obs'] = len(qdata)
            results.append(coefs)
        except Exception:
            continue

    fm = pd.DataFrame(results)

    # Time-series averages with Newey-West t-statistics
    print("\nFama-MacBeth Results:")
    print("=" * 50)
    for var in ['const', 'dio', 'd_n_total_inst', 'log_mcap', 'mom']:
        coefs = fm[var].dropna()
        mean_c = coefs.mean()
        nw_se = sm.OLS(
            coefs - mean_c, np.ones(len(coefs))
        ).fit(cov_type='HAC', cov_kwds={'maxlags': 4}).bse[0]
        t = mean_c / nw_se if nw_se > 0 else np.nan
        print(f"  {var:20s}: coef={mean_c:8.4f}, t={t:6.2f}")

    return fm
```

69.2 Application 2: Turnover and Performance

Yan (2008) documented a positive turnover-performance relationship. We test in Vietnam:

$$\alpha_{j,t} = a + b \cdot \text{Turnover}_{j,t-1} + c \cdot \log(A_{j,t-1}) + d \cdot \text{Flow}_{j,t} + \varepsilon_{j,t} \tag{69.2}$$

```
def turnover_performance_regression(
    aggregates: pd.DataFrame,
) -> dict:
    """Test turnover-performance relationship."""
    agg = aggregates.sort_values(['shareholder_name', 'rdate']).copy()
    agg['lag_turnover1'] = (
        agg.groupby('shareholder_name')['turnover1'].shift(1)
    )
    agg['log_assets'] = np.log(agg['assets'] + 1)
    agg['flow_ratio'] = agg['netflows'] / agg['assets'].shift(1)

    panel = agg.dropna(
        subset=['pret', 'lag_turnover1', 'log_assets', 'flow_ratio']
    )

    for col in ['pret', 'lag_turnover1', 'flow_ratio']:
        lo, hi = panel[col].quantile([0.01, 0.99])
        panel[col] = panel[col].clip(lo, hi)

    X = sm.add_constant(
        panel[['lag_turnover1', 'log_assets', 'flow_ratio']]
    )
    model = sm.OLS(panel['pret'], X).fit(
        cov_type='cluster',
        cov_kwds={'groups': panel['shareholder_name']}
    )
    return {'model': model, 'n': len(panel)}
```

69.3 Application 3: Foreign vs. Domestic Trading

```
def compare_foreign_domestic(
    trades: pd.DataFrame, price_q: pd.DataFrame,
) -> pd.DataFrame:
    """Compare trading patterns between foreign and domestic institutions."""
    _t = trades.merge(
        price_q[['ticker', 'qdate', 'p']],
        left_on=['ticker', 'rdate'],
        right_on=['ticker', 'qdate'],
        how='inner'
    )
```

```
    _t['dollar_trade'] = _t['trade'] * _t['p'] / 1e6
    _t['is_buy'] = _t['trade'] > 0

    return (
        _t[_t['owner_type'].isin(OwnershipType.INSTITUTIONAL)]
        .groupby('owner_type')
        .agg(
            n_trades=('trade', 'count'),
            n_buys=('is_buy', 'sum'),
            avg_dollar=('dollar_trade', lambda x: x.abs().mean()),
            net_buying=('dollar_trade', 'sum'),
            pct_initiating=('buysale', lambda x: (x.abs() == 1).mean()),
        )
        .reset_index()
    )
```

```
def plot_cumulative_net_buying(
    trades: pd.DataFrame, price_q: pd.DataFrame
):
    _t = trades.merge(
        price_q[['ticker', 'qdate', 'p']],
        left_on=['ticker', 'rdate'],
        right_on=['ticker', 'qdate'],
        how='inner'
    )
    _t['trade_vnd'] = _t['trade'] * _t['p'] / 1e9

    inst = _t[_t['owner_type'].isin(OwnershipType.INSTITUTIONAL)]
    net = (
        inst.groupby(
            [pd.Grouper(key='rdate', freq='QE'), 'owner_type']
        )['trade_vnd'].sum().unstack(fill_value=0)
    )
    cum = net.cumsum()

    fig, ax = plt.subplots(figsize=(12, 6))
    for col in cum.columns:
        ax.plot(cum.index, cum[col], label=col,
                color=OWNER_COLORS.get(col, '#333'), linewidth=2)
    ax.axhline(y=0, color='black', linewidth=0.5)
    ax.set_title('Cumulative Net Institutional Buying', fontweight='bold')
    ax.set_ylabel('Billions VND')
    ax.legend(loc='best')
    plt.tight_layout()
    plt.show()
```

Figure 69.1

70 Data Quality and Robustness

70.1 Common Pitfalls

70.1.1 Corporate Action Misadjustment

Example: Phantom Trade from Unadjusted Stock Dividend

Vinamilk (VNM) issues a 20% stock dividend with ex-date March 15, 2023.

- Q4 2022: Fund X holds 1,000,000 shares of VNM
- Q1 2023: Fund X holds 1,200,000 shares of VNM

Without adjustment: Inferred buy of +200,000 shares (BS = +2) **With adjustment:** Prior holdings become 1,200,000 adjusted shares, trade = 0
This phantom trade inflates measured turnover and creates spurious buying signals.

70.1.2 Disclosure Timing Mismatches

Vietnamese ownership disclosure dates may not align with calendar quarter ends. Our pipeline addresses this by aligning all disclosures to the nearest quarter-end.

70.1.3 Name Changes and Entity Mergers

Vietnamese institutions frequently rename. Without a stable identifier, the same entity may appear as two different shareholders, creating phantom entries/exits. We recommend maintaining a master entity mapping table.

70.2 Validation Checks

```
def validate_pipeline_outputs(
    results: Dict[str, pd.DataFrame],
) -> pd.DataFrame:
    """Run comprehensive validation on pipeline outputs."""
    checks = []
    h = results['holdings']
    t = results['trades']
    a = results['aggregates']

    checks.append({
        'Check': 'No negative adjusted shares',
        'Result': 'PASS' if (h['shares_adj'] < 0).sum() == 0 else 'FAIL',
        'Detail': f'{(h["shares_adj"] < 0).sum()} negative obs'
    })

    checks.append({
        'Check': 'No duplicate holdings',
        'Result': 'PASS' if h.duplicated(
            subset=['shareholder_name', 'ticker', 'rdate']
        ).sum() == 0 else 'FAIL',
    })

    checks.append({
        'Check': 'Valid buysale codes only',
        'Result': 'PASS' if t['buysale'].isin([1, 2, -1, -2]).all()
        else 'FAIL',
    })

    checks.append({
        'Check': 'No zero trades',
        'Result': 'PASS' if (t['trade'] == 0).sum() == 0 else 'FAIL',
    })

    t1 = a['turnover1'].dropna()
    checks.append({
        'Check': 'Turnover1 in [0, 10]',
        'Result': 'PASS' if ((t1 < 0) | (t1 > 10)).sum() == 0
        else 'WARNING',
        'Detail': f'{((t1<0)|(t1>10)).sum()} extreme values'
    })

    first_rpt = a[a['first_report']]
    checks.append({
```

```
        'Check': 'First report -> missing netflows',
        'Result': 'PASS' if first_rpt['netflows'].isna().all()
        else 'FAIL',
    })

    return pd.DataFrame(checks)

# validate_pipeline_outputs(results)
```

71 Summary

This chapter developed a framework for computing institutional trades, flows, and turnover ratios in the Vietnamese equity market. The key contributions include:

1. **Corporate action adjustment** for Vietnam's frequent stock dividends and bonus shares, preventing phantom trades that contaminate standard differencing.
2. **Four-way ownership taxonomy** (state, foreign institutional, domestic institutional, individual) capturing Vietnam's unique ownership landscape.
3. **FOL utilization analytics** for studying foreign ownership constraints absent from developed markets.
4. **Irregular disclosure handling** with correct gap splitting into terminating sales and initiating buys.
5. **Advanced extensions** including herding, demand persistence, and information content decomposition.

The pipeline produces several output datasets (Table 71.1)

Table 71.1: Summary of Pipeline Output Datasets

Output	Grain	Key Variables	Use Cases
`holdings`	Shareholder x Ticker x Quarter	`shares_adj`, `owner_type`	Cross-sectional ownership
`io_ratios`	Ticker x Quarter	`io_state`, `io_foreign`, etc.	Governance, liquidity
`trades`	Shareholder x Ticker x Quarter	`trade`, `buysale`	Informed trading, herding
`aggregates`	Shareholder x Quarter	`assets`, `turnover`, `netflows`	Fund performance, flows
`fol_analytics`	Ticker x Date	`fol_utilization`, `foreign_room`	FOL premium, foreign investment

72 Corporate Governance

Corporate governance (i.e., the system of rules, practices, and processes by which firms are directed and controlled) has been one of the most actively studied determinants of equity returns since the early 2000s. The insight is simple yet powerful: firms that grant shareholders stronger rights tend to outperform firms in which management is entrenched by anti-takeover provisions. This chapter applies that insight to the Vietnamese market, where governance quality varies considerably across listed firms and institutional development remains evolving.

72.1 Theoretical Background

72.1.1 The Governance-Return Nexus

The theoretical motivation for a link between corporate governance and stock returns rests on agency theory (M. C. Jensen and Meckling 1976). Managers, as agents of shareholders, may pursue private benefits at the expense of firm value. Anti-takeover provisions, staggered boards, poison pills, and other defensive mechanisms insulate management from the disciplining force of the market for corporate control. When shareholders cannot easily replace underperforming managers, agency costs rise, investment efficiency falls, and firm value declines.

Gompers, Ishii, and Metrick (2003) formalized this intuition by constructing a **Governance Index (G-Index)** based on 24 governance provisions tracked by the Investor Responsibility Research Center (IRRC) in the United States. Each provision that restricts shareholder rights increments the index by one. Thus:

$$G\text{-Index}_i = \sum_{k=1}^{24} \mathbf{1}\{\text{Provision } k \text{ is present for firm } i\} \tag{72.1}$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. Higher values of the G-Index correspond to weaker shareholder rights.

The key empirical finding was striking: during the 1990s, a portfolio that bought firms with the strongest shareholder rights (G-Index ≤ 5, labeled the **Democracy Portfolio**) and sold firms with the weakest shareholder rights (G-Index ≥ 14, labeled the **Dictatorship Portfolio**) earned an abnormal return of approximately 8.5% per year.

72.1.2 Adapting the Framework to Vietnam

Vietnam's corporate governance landscape differs fundamentally from that of the United States. The Vietnamese market is characterized by:

1. **State ownership**: Many listed firms retain significant government ownership stakes, which creates a distinct agency problem where the state acts as a controlling shareholder rather than dispersed minority shareholders facing entrenched management.
2. **Concentrated ownership**: Family and controlling-group ownership is prevalent, shifting the primary agency conflict from manager-shareholder to controlling-majority vs. minority shareholders (Claessens, Djankov, and Lang 2000).
3. **Evolving legal framework**: Vietnam's corporate governance code has been progressively strengthened through Decree 71/2017/ND-CP and subsequent circulars, but enforcement remains uneven.
4. **Dual listing and foreign ownership caps**: Foreign ownership limits create segmented investor bases with potentially different governance preferences.

Despite these differences, the core economic logic applies: firms with better governance (i.e., greater board independence, stronger audit committees, more transparent disclosure, better minority shareholder protections) should command higher valuations and deliver superior risk-adjusted returns, all else equal.

For the Vietnamese market, we construct a governance index analogous to the G-Index using governance provisions. While the specific provisions differ from the 24 IRRC items used in the US context, the methodology is identical: count the number of provisions that restrict shareholder rights or entrench management.

72.1.3 The Vietnamese Governance Index (VN-GIndex)

We define the Vietnamese Governance Index based on the following categories of provisions (Table 72.1)

Table 72.1: Categories of governance provisions for the VN-GIndex

Category	Provisions	Direction
Board Structure	Staggered board, CEO duality, board size < 5	$\uparrow$ restricts rights
Ownership	State ownership $> 50\%$, no independent directors	$\uparrow$ restricts rights
Shareholder Rights	Supermajority requirements, limited voting rights	$\uparrow$ restricts rights

Category	Provisions	Direction
Transparency	No English-language annual report, delayed filings	↑ restricts rights
Anti-takeover	Poison pill equivalents, golden parachutes	↑ restricts rights
Audit	No independent audit committee, related-party auditor	↑ restricts rights

The VN-GIndex for firm i at time t is:

$$\text{VN-GIndex}_{i,t} = \sum_{k=1}^{K} \mathbf{1}\{\text{Provision } k \text{ is present for firm } i \text{ at time } t\} \tag{72.2}$$

where K is the total number of governance provisions tracked. Higher values indicate weaker governance.

72.2 Data Preparation

We use three primary datasets:

1. **Governance data**: Firm-level governance provisions, updated annually or when material changes occur.
2. **Stock market data**: Monthly returns, prices, and shares outstanding for all firms listed on HOSE and HNX.
3. **Factor data**: Vietnamese Fama-French factors (market, size, value) and a momentum factor.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.regression.linear_model import OLS
from scipy import stats
import warnings
import sqlite3

warnings.filterwarnings("ignore")
```

Table 72.2: First observations of the governance dataset

```
governance_raw.head(10)
```

```
pd.options.display.float_format = "{:.4f}".format

tidy_finance = sqlite3.connect(
    database="data/tidy_finance_python.sqlite"
)
```

72.2.1 Loading Governance Data

The governance dataset contains firm-level governance characteristics. Each observation corresponds to a firm-year, with binary indicators for the presence of each governance provision.

```
governance_raw = pd.read_csv(
    "data/datacore_governance.csv",
    parse_dates=["date_effective", "date_expires"]
)

governance_raw.info()
```

The key variables are:

- `symbol`: The stock symbol on HOSE or HNX.
- `date_effective`: The date when the governance data became effective (analogous to the rebalancing date).
- `date_expires`: The last date for which this governance vintage is valid.
- `year`: The governance data vintage year.

72.2.2 Computing the VN-GIndex

We compute the governance index by summing the binary indicators of governance provision. The provision columns are identified by the prefix `gov_`.

Table 72.3: Summary statistics of the VN-GIndex by vintage year

```
gindex_summary = (
    governance
    .groupby("year")["gindex"]
    .describe()
    .round(2)
)

gindex_summary
```

```
# Identify governance provision columns
gov_columns = [
    col for col in governance_raw.columns if col.startswith("gov_")
]

print(f"Number of governance provisions tracked: {len(gov_columns)}")
print(f"Provisions: {gov_columns}")

# Compute VN-GIndex as sum of all provision indicators
governance = governance_raw.assign(
    gindex=lambda x: x[gov_columns].sum(axis=1)
)

governance[["symbol", "year", "gindex"]].describe()
```

72.2.3 Distribution of the VN-GIndex

Understanding the cross-sectional distribution of the governance index is essential for defining portfolio cutoffs. In the US, Gompers, Ishii, and Metrick (2003) used fixed cutoffs of ≤ 5 for Democracy and ≥ 14 for Dictatorship. For Vietnam, we examine the empirical distribution and set cutoffs at appropriate percentiles.

72.2.4 Classifying Firms: Democracy, Neutral, and Dictatorship

Rather than using fixed cutoffs (which may be inappropriate given varying index ranges across markets), we use percentile-based classification. Firms in the bottom quintile of the VN-GIndex distribution within each vintage year are classified as **Democracy** firms, and firms in the top quintile are classified as **Dictatorship** firms.

```
fig, axes = plt.subplots(
    2, 3, figsize=(12, 7), sharey=True, sharex=True
)
axes = axes.flatten()

years = sorted(governance["year"].unique())

for idx, yr in enumerate(years[:6]):
    ax = axes[idx]
    data_yr = governance.query(f"year == {yr}")["gindex"]

    p20 = data_yr.quantile(0.20)
    p80 = data_yr.quantile(0.80)

    ax.hist(data_yr, bins=range(0, int(data_yr.max()) + 2),
            edgecolor="white", color="#2c5f8a", alpha=0.85)
    ax.axvline(p20, color="#d63e2a", linestyle="--", linewidth=1.5,
               label=f"P20={p20:.0f}")
    ax.axvline(p80, color="#e8a317", linestyle="--", linewidth=1.5,
               label=f"P80={p80:.0f}")
    ax.set_title(f"{yr}", fontsize=11, fontweight="bold")
    ax.legend(fontsize=8)
    ax.set_xlabel("VN-GIndex")

for idx in range(len(years), len(axes)):
    axes[idx].set_visible(False)

axes[0].set_ylabel("Number of Firms")
axes[3].set_ylabel("Number of Firms")

fig.suptitle(
    "Distribution of VN-GIndex by Governance Vintage Year",
    fontsize=13, fontweight="bold", y=1.01
)
plt.tight_layout()
plt.show()
```

Figure 72.1

Table 72.4: Number of firms in each governance category by vintage year

```
classification_counts
```

```
def classify_governance(group):
    """Classify firms into Democracy, Neutral, or Dictatorship
    within each governance vintage year."""
    p20 = group["gindex"].quantile(0.20)
    p80 = group["gindex"].quantile(0.80)

    conditions = [
        group["gindex"] <= p20,
        group["gindex"] >= p80
    ]
    choices = ["Democracy", "Dictatorship"]

    group = group.assign(
        gx=np.select(conditions, choices, default="Neutral"),
        gx_code=np.select(
            conditions, [1, 3], default=2
        )
    )
    return group

governance_classified = (
    governance
    .groupby("year", group_keys=False)
    .apply(classify_governance)
)

# Summary of classification
classification_counts = (
    governance_classified
    .groupby(["year", "gx"])
    .size()
    .unstack(fill_value=0)
    [["Democracy", "Neutral", "Dictatorship"]]
)

classification_counts
```

We exclude firms with dual-class share structures, as these create a distinct governance arrangement that conflates voting rights with economic ownership. In the US, Gompers, Ishii, and Metrick (2003) similarly excluded dual-class firms.

```
# Exclude dual-class firms if flagged in the data
if "dual_class" in governance_classified.columns:
    governance_classified = governance_classified.query(
        "dual_class == 0"
    )
    print("Dual-class firms excluded.")
else:
    print("No dual_class indicator found; proceeding with all firms.")

# Keep only Democracy and Dictatorship firms for the long-short strategy
portfolio_firms = governance_classified.query(
    "gx in ['Democracy', 'Dictatorship']"
).copy()

print(f"\nFirms in portfolio universe: {len(portfolio_firms)}")
print(portfolio_firms["gx"].value_counts())
```

72.2.5 Loading Stock Market Data

We merge the governance classifications with monthly stock return data.

```
prices_monthly = pd.read_sql_query(
    sql="""
        SELECT symbol, date, ret, ret_excess, mktcap, mktcap_lag, risk_free
        FROM prices_monthly
    """,
    con=tidy_finance,
    parse_dates={"date"}
).dropna()

prices_monthly.info()
```

```
<class 'pandas.DataFrame'>
Index: 165499 entries, 1 to 209477
Data columns (total 7 columns):
 #   Column      Non-Null Count   Dtype
---  ------      --------------   -----
```

```
 0   symbol      165499 non-null  str
 1   date        165499 non-null  datetime64[us]
 2   ret         165499 non-null  float64
 3   ret_excess  165499 non-null  float64
 4   mktcap      165499 non-null  float64
 5   mktcap_lag  165499 non-null  float64
 6   risk_free   165499 non-null  float64
dtypes: datetime64[us](1), float64(5), str(1)
memory usage: 10.6 MB
```

The stock market data contains:

- `symbol`: Stock symbol.
- `date`: End-of-month date.
- `ret`: Monthly total return (including dividends).
- `retx`: Monthly return excluding dividends (price return only).
- `price`: End-of-month closing price (adjusted).
- `shares_outstanding`: Number of shares outstanding (in thousands).

```
prices_monthly[["symbol", "date", "ret", "mktcap"]].describe()
```

	date	ret	mktcap
count	165499	165499.0000	165499.0000
mean	2018-05-18 13:20:13.109444	0.0042	2183.1646
min	2010-02-28 00:00:00	-0.9900	0.3536
25%	2015-06-30 00:00:00	-0.0703	60.3728
50%	2018-12-31 00:00:00	0.0000	180.6224
75%	2021-07-31 00:00:00	0.0553	660.0000
max	2023-12-31 00:00:00	12.7500	463886.6454
std	NaN	0.1862	13983.9977

72.2.6 Linking Governance and Stock Data

Each governance vintage is valid from `date_effective` through `date_expires`. We assign monthly stock returns to the appropriate governance vintage. This is the portfolio rebalancing logic: portfolios are reformed when new governance data becomes available and held until the next vintage.

```
# Merge: each stock-month gets its governance classification
# if the month falls within [date_effective, date_expires]
merged = pd.merge(
    portfolio_firms[
        ["symbol", "year", "gindex", "gx", "gx_code",
         "date_effective", "date_expires"]
    ],
    prices_monthly[["symbol", "date", "ret", "retx", "mktcap"]],
    on="symbol",
    how="inner"
)

# Keep only months within the governance validity window
merged = merged.query(
    "date >= date_effective and date <= date_expires"
).sort_values(["symbol", "date"])

print(f"Total firm-month observations: {len(merged):,}")
merged.head()
```

72.2.7 Computing Lagged Market Capitalization for Weighting

Value-weighted portfolio returns require weighting each stock by its beginning-of-period market capitalization. We use the previous month's market capitalization as the weight. For the first observation of each stock in a given vintage, we estimate the beginning-of-period market value by dividing the current market value by $(1 + r_{x,t})$, where $r_{x,t}$ is the price return.

The value-weighted return of portfolio p in month t is:

$$r_{p,t}^{vw} = \sum_{i \in p} w_{i,t-1} \cdot r_{i,t}, \quad w_{i,t-1} = \frac{\text{MV}_{i,t-1}}{\sum_{j \in p} \text{MV}_{j,t-1}} \tag{72.3}$$

where $\text{MV}_{i,t-1}$ is the market capitalization of stock i at the end of month $t-1$.

```
merged = merged.sort_values(["symbol", "date"])

# Lagged market value (previous month)
merged["mktcap_lag"] = merged.groupby("symbol")["mktcap"].shift(1)

# For first observation: estimate beginning-of-month market cap
first_obs = merged.groupby("symbol")["date"].transform("min")
```

```
mask_first = merged["date"] == first_obs

merged.loc[mask_first, "mktcap_lag"] = (
    merged.loc[mask_first, "mktcap"]
    / (1 + merged.loc[mask_first, "retx"])
)

# Handle any remaining missing weights
mask_missing = merged["mktcap_lag"].isna()
merged.loc[mask_missing, "mktcap_lag"] = (
    merged.loc[mask_missing, "mktcap"]
    / (1 + merged.loc[mask_missing, "retx"])
)

# Drop observations with missing returns or weights
merged = merged.dropna(subset=["ret", "mktcap_lag"])
merged = merged.query("mktcap_lag > 0")

print(f"Clean firm-month observations: {len(merged):,}")
```

72.3 Portfolio Construction

72.3.1 Value-Weighted Portfolio Returns

We now compute the value-weighted monthly returns for the Democracy and Dictatorship portfolios.

```
def value_weighted_return(group):
    """Compute value-weighted return for a group of stocks."""
    weights = group["mktcap_lag"]
    total_weight = weights.sum()
    if total_weight == 0:
        return np.nan
    vw_ret = (group["ret"] * weights).sum() / total_weight
    return vw_ret

# Compute VW returns by date and governance group
portfolio_returns = (
    merged
    .groupby(["date", "gx"])
```

```
    .apply(value_weighted_return, include_groups=False)
    .reset_index()
    .rename(columns={0: "ret_vw"})
)

# Also compute equal-weighted returns for robustness
portfolio_returns_ew = (
    merged
    .groupby(["date", "gx"])["ret"]
    .mean()
    .reset_index()
    .rename(columns={"ret": "ret_ew"})
)

# Merge EW returns
portfolio_returns = portfolio_returns.merge(
    portfolio_returns_ew, on=["date", "gx"], how="left"
)

portfolio_returns.head(10)
```

72.3.2 Long-Short Portfolio: Democracy minus Dictatorship

The trading strategy goes long the Democracy portfolio and short the Dictatorship portfolio. The monthly return of this long-short strategy is:

$$r_t^{D-D} = r_t^{\text{Democracy}} - r_t^{\text{Dictatorship}} \tag{72.4}$$

```
# Pivot to wide format
returns_wide = portfolio_returns.pivot(
    index="date", columns="gx", values=["ret_vw", "ret_ew"]
)

# Flatten column names
returns_wide.columns = [
    f"{val}_{grp}" for val, grp in returns_wide.columns
]
returns_wide = returns_wide.reset_index()

# Compute long-short returns
```

Table 72.6: Portfolio characteristics by governance group and year

```
portfolio_chars = (
    merged
    .assign(year_month=lambda x: x["date"].dt.to_period("Y"))
    .groupby(["year_month", "gx"])
    .agg(
        n_firms=("symbol", "nunique"),
        avg_gindex=("gindex", "mean"),
        avg_mktcap=("mktcap", "mean"),
        median_mktcap=("mktcap", "median")
    )
    .round(2)
)

portfolio_chars
```

```
returns_wide = returns_wide.assign(
    ret_diff_vw=lambda x: (
        x["ret_vw_Democracy"] - x["ret_vw_Dictatorship"]
    ),
    ret_diff_ew=lambda x: (
        x["ret_ew_Democracy"] - x["ret_ew_Dictatorship"]
    )
)

returns_wide = returns_wide.sort_values("date").reset_index(drop=True)

print(f"Monthly return series: {len(returns_wide)} months")
returns_wide.head()
```

72.3.3 Portfolio Characteristics Over Time

Before examining returns, we document the number of firms and average governance scores in each portfolio over time.

Table 72.7: Summary statistics of monthly portfolio returns (in percent)

```
return_cols = [
    "ret_vw_Democracy", "ret_vw_Dictatorship", "ret_diff_vw",
    "ret_ew_Democracy", "ret_ew_Dictatorship", "ret_diff_ew"
]

summary_stats = (
    returns_wide[return_cols]
    .mul(100)  # Convert to percent
    .describe()
    .T
    .assign(
        skewness=returns_wide[return_cols].mul(100).skew(),
        sharpe=lambda x: x["mean"] / x["std"] * np.sqrt(12)
    )
    .round(3)
)

summary_stats.index = [
    "Democracy (VW)", "Dictatorship (VW)", "Long-Short (VW)",
    "Democracy (EW)", "Dictatorship (EW)", "Long-Short (EW)"
]

summary_stats[["mean", "std", "min", "25%", "50%", "75%", "max",
               "skewness", "sharpe"]]
```

72.4 Empirical Results

72.4.1 Summary Statistics of Portfolio Returns

The **Sharpe ratio** is computed as:

$$\text{SR} = \frac{\bar{r}_p}{\sigma_p} \times \sqrt{12} \tag{72.5}$$

where $\bar{r}_p$ and σ_p are the sample mean and standard deviation of monthly portfolio returns, and the $\sqrt{12}$ scaling annualizes the ratio.

Table 72.8: T-test for the mean difference between Democracy and Dictatorship portfolio returns

```
def perform_ttest(series, label):
    """Perform a one-sample t-test and return results."""
    clean = series.dropna()
    t_stat, p_value = stats.ttest_1samp(clean, 0)
    return {
        "Portfolio": label,
        "Mean (%)": clean.mean() * 100,
        "Std (%)": clean.std() * 100,
        "T-statistic": t_stat,
        "P-value": p_value,
        "N months": len(clean),
        "Significant (5%)": "Yes" if p_value < 0.05 else "No"
    }

ttest_results = pd.DataFrame([
    perform_ttest(returns_wide["ret_diff_vw"], "VW Long-Short"),
    perform_ttest(returns_wide["ret_diff_ew"], "EW Long-Short"),
    perform_ttest(returns_wide["ret_vw_Democracy"], "VW Democracy"),
    perform_ttest(returns_wide["ret_vw_Dictatorship"], "VW Dictatorship")
])

ttest_results.round(4)
```

72.4.2 T-Test: Is the Long-Short Return Statistically Significant?

The null hypothesis is that the mean monthly return difference between the Democracy and Dictatorship portfolios is zero:

$$H_0 : \mathbb{E}[r_t^{\text{Democracy}} - r_t^{\text{Dictatorship}}] = 0 \tag{72.6}$$

72.4.3 Cumulative Returns: The Visual Case for Governance

One of the most compelling ways to present the governance effect is through cumulative wealth plots. We track the growth of \$1 invested in each portfolio at the beginning of the sample period.

The cumulative return at time T is:

$$W_T = \prod_{t=1}^{T}(1 + r_{p,t}) \tag{72.7}$$

72.4.4 Rolling Performance

The governance premium may not be constant over time. We examine 12-month rolling average returns and rolling Sharpe ratios to assess stability.

72.5 Risk-Adjusted Performance: Factor Model Analysis

72.5.1 The Four-Factor Model

Raw returns alone do not tell us whether the governance strategy generates **abnormal** returns (i.e., returns that cannot be explained by exposure to common risk factors). We estimate the following four-factor model:

$$r_t^{D-D} - r_{f,t} = \alpha + \beta_1(r_{m,t} - r_{f,t}) + \beta_2 \text{SMB}_t + \beta_3 \text{HML}_t + \beta_4 \text{UMD}_t + \varepsilon_t \tag{72.8}$$

where:

- r_t^{D-D} is the long-short governance portfolio return,
- $r_{f,t}$ is the risk-free rate,
- $r_{m,t} - r_{f,t}$ is the market excess return (MKTRF),
- SMB_t is the size factor (Small Minus Big),
- HML_t is the value factor (High Minus Low book-to-market),
- UMD_t is the momentum factor (Up Minus Down),
- α is the abnormal return (the intercept of interest).

The **alpha** (α) represents the average monthly return that cannot be attributed to exposure to the four systematic risk factors. A statistically significant positive alpha indicates that the governance strategy generates genuine abnormal returns.

72.5.2 Loading Factor Data

```
returns_wide = returns_wide.sort_values("date")

cum_democracy = (1 + returns_wide["ret_vw_Democracy"]).cumprod()
cum_dictatorship = (1 + returns_wide["ret_vw_Dictatorship"]).cumprod()

fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(
    returns_wide["date"], cum_democracy,
    color="#1a6b3c", linewidth=2, label="Democracy Portfolio"
)
ax.plot(
    returns_wide["date"], cum_dictatorship,
    color="#c0392b", linewidth=1.5, linestyle="--",
    label="Dictatorship Portfolio"
)
ax.fill_between(
    returns_wide["date"],
    cum_democracy, cum_dictatorship,
    where=cum_democracy >= cum_dictatorship,
    alpha=0.15, color="#1a6b3c", label="Outperformance"
)
ax.fill_between(
    returns_wide["date"],
    cum_democracy, cum_dictatorship,
    where=cum_democracy < cum_dictatorship,
    alpha=0.15, color="#c0392b"
)

ax.set_xlabel("Date", fontsize=11)
ax.set_ylabel("Cumulative Value of $1 Invested", fontsize=11)
ax.set_title(
    "Democracy vs. Dictatorship Portfolios: Cumulative Returns",
    fontsize=13, fontweight="bold"
)
ax.legend(fontsize=10, loc="upper left")
ax.grid(True, alpha=0.3)
ax.set_xlim(returns_wide["date"].min(), returns_wide["date"].max())

plt.tight_layout()
plt.show()
```

Figure 72.2

```
cum_longshort = (1 + returns_wide["ret_diff_vw"]).cumprod()

fig, ax = plt.subplots(figsize=(10, 4))

ax.plot(
    returns_wide["date"], cum_longshort,
    color="#2c5f8a", linewidth=2
)
ax.axhline(y=1.0, color="gray", linestyle=":", linewidth=1)
ax.fill_between(
    returns_wide["date"], 1.0, cum_longshort,
    where=cum_longshort >= 1.0, alpha=0.2, color="#2c5f8a"
)
ax.fill_between(
    returns_wide["date"], 1.0, cum_longshort,
    where=cum_longshort < 1.0, alpha=0.2, color="#c0392b"
)

ax.set_xlabel("Date", fontsize=11)
ax.set_ylabel("Cumulative Long-Short Return", fontsize=11)
ax.set_title(
    "Long-Short Governance Strategy: Cumulative Performance",
    fontsize=13, fontweight="bold"
)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Figure 72.3

```
rolling_mean = (
    returns_wide["ret_diff_vw"]
    .rolling(window=12, min_periods=6)
    .mean() * 12 * 100  # Annualized
)

rolling_std = (
    returns_wide["ret_diff_vw"]
    .rolling(window=12, min_periods=6)
    .std() * np.sqrt(12) * 100
)

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

# Rolling annualized return
axes[0].plot(
    returns_wide["date"], rolling_mean,
    color="#2c5f8a", linewidth=1.5
)
axes[0].axhline(0, color="gray", linestyle="--", linewidth=0.8)
axes[0].set_ylabel("Annualized Return (%)")
axes[0].set_title(
    "12-Month Rolling Annualized Return: Long-Short Governance Strategy",
    fontweight="bold"
)
axes[0].grid(True, alpha=0.3)

# Rolling annualized volatility
axes[1].plot(
    returns_wide["date"], rolling_std,
    color="#c0392b", linewidth=1.5
)
axes[1].set_ylabel("Annualized Volatility (%)")
axes[1].set_xlabel("Date")
axes[1].set_title(
    "12-Month Rolling Annualized Volatility",
    fontweight="bold"
)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Figure 72.4

```
factors_ff5_monthly = pd.read_sql_query(
    sql="SELECT * FROM factors_ff5_monthly",
    con=tidy_finance,
    parse_dates={"date"}
)

factors_ff5_monthly.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   date        150 non-null    datetime64[us]
 1   smb         150 non-null    float64
 2   hml         150 non-null    float64
 3   rmw         150 non-null    float64
 4   cma         150 non-null    float64
 5   mkt_excess  150 non-null    float64
 6   risk_free   150 non-null    float64
dtypes: datetime64[us](1), float64(6)
memory usage: 8.3 KB
```

```
momentum_factor = pd.read_sql_query(
    sql="SELECT * FROM momentum_factor_monthly",
    con=tidy_finance,
    parse_dates={"date"}
)

factors_ff3_monthly = (
    factors_ff5_monthly
    .merge(momentum_factor, on="date")
    .rename(columns={"mkt_excess": "mktrf", "wml": "umd"})
)
```

```
factor_cols = ["mktrf", "smb", "hml", "umd", "risk_free"]

(
    factors_ff3_monthly[factor_cols]
    .mul(100)
    .describe()
```

```
    .T
    .round(3)
)
```

Table 72.9: Summary statistics of Vietnamese Fama-French-Carhart factors (monthly, in percent)

	count	mean	std	min	25%	50%	75%	max
mktrf	150.0000	-1.0080	5.8580	-21.4910	-3.8030	-0.9510	2.1450	16.7680
smb	150.0000	0.7660	4.1900	-15.2200	-1.3730	1.0400	3.1610	12.8380
hml	150.0000	1.1460	5.1820	-12.8300	-1.2620	0.4610	3.2270	15.0970
umd	150.0000	-2.0910	4.8090	-16.3870	-4.8840	-2.1650	0.6190	15.6040
risk_free	150.0000	0.3330	0.0000	0.3330	0.3330	0.3330	0.3330	0.3330

72.5.3 Merging Portfolio Returns with Factor Data

```
# Merge on year-month
returns_wide["ym"] = returns_wide["date"].dt.to_period("M")
factors_ff3_monthly["ym"] = factors_ff3_monthly["date"].dt.to_period("M")

analysis_data = returns_wide.merge(
    factors_ff3_monthly[["ym", "mktrf", "smb", "hml", "umd", "rf"]],
    on="ym",
    how="inner"
)

# Compute excess returns
analysis_data = analysis_data.assign(
    ret_excess_democracy=lambda x: x["ret_vw_Democracy"] - x["rf"],
    ret_excess_dictatorship=lambda x: x["ret_vw_Dictatorship"] - x["rf"],
    ret_excess_longshort=lambda x: x["ret_diff_vw"]
    # Long-short is already excess (self-financing)
)

print(f"Observations for regression: {len(analysis_data)}")
```

72.5.4 CAPM Regression

We start with the single-factor CAPM to understand the market exposure of each portfolio:

$$r_{p,t} - r_{f,t} = \alpha_p + \beta_p(r_{m,t} - r_{f,t}) + \varepsilon_{p,t} \tag{72.9}$$

72.5.5 Three-Factor Fama-French Model

The three-factor model (Eugene F. Fama and French 1993a) augments the CAPM with size (SMB) and value (HML) factors:

$$r_{p,t} - r_{f,t} = \alpha_p + \beta_1 \text{MKTRF}_t + \beta_2 \text{SMB}_t + \beta_3 \text{HML}_t + \varepsilon_{p,t} \tag{72.10}$$

72.5.6 Four-Factor Carhart Model

Adding the momentum factor (Mark M. Carhart 1997b) controls for the well-documented tendency of past winners to continue outperforming past losers:

```
# Detailed regression output for the long-short portfolio
X = sm.add_constant(analysis_data[["mktrf", "smb", "hml", "umd"]])
y = analysis_data["ret_excess_longshort"]

model_ls = OLS(y, X).fit(
    cov_type="HAC", cov_kwds={"maxlags": 6}
)

print(model_ls.summary())
```

72.5.7 Interpreting the Factor Loadings

The factor loadings reveal important characteristics of the governance portfolios:

1. **Market beta** (β_{MKTRF}): If the long-short portfolio has a near-zero market beta, the governance strategy is approximately market-neutral. A positive beta would indicate that Democracy firms are more sensitive to market movements than Dictatorship firms.
2. **Size factor** (β_{SMB}): A positive loading suggests the strategy tilts toward smaller firms. If Democracy firms tend to be smaller (or Dictatorship firms larger), the size factor captures this differential.
3. **Value factor** (β_{HML}): A positive loading indicates a value tilt. Governance and value may be related if poorly governed firms also tend to have high book-to-market ratios (i.e., they are "cheap" because of governance risk).

Table 72.10: CAPM regression results for Democracy, Dictatorship, and Long-Short portfolios

```
def run_regression(y, X, hac=True):
    """Run OLS regression with optional HAC standard errors."""
    X_const = sm.add_constant(X)
    model = OLS(y, X_const).fit(
        cov_type="HAC" if hac else "nonrobust",
        cov_kwds={"maxlags": 6} if hac else {}
    )
    return model

portfolios = {
    "Democracy": analysis_data["ret_excess_democracy"],
    "Dictatorship": analysis_data["ret_excess_dictatorship"],
    "Long-Short": analysis_data["ret_excess_longshort"]
}

capm_results = {}
for name, y in portfolios.items():
    X = analysis_data[["mktrf"]]
    model = run_regression(y, X, hac=True)
    capm_results[name] = {
        "Alpha (%)": model.params["const"] * 100,
        "Alpha t-stat": model.tvalues["const"],
        "Beta (MKTRF)": model.params["mktrf"],
        "Beta t-stat": model.tvalues["mktrf"],
        "R-squared": model.rsquared,
        "N": int(model.nobs)
    }

pd.DataFrame(capm_results).T.round(4)
```

Table 72.11: Fama-French three-factor model results with Newey-West standard errors

```
ff3_results = {}
for name, y in portfolios.items():
    X = analysis_data[["mktrf", "smb", "hml"]]
    model = run_regression(y, X, hac=True)
    ff3_results[name] = {
        "Alpha (%)": model.params["const"] * 100,
        "Alpha t-stat": model.tvalues["const"],
        "MKTRF": model.params["mktrf"],
        "SMB": model.params["smb"],
        "HML": model.params["hml"],
        "R-squared": model.rsquared
    }

pd.DataFrame(ff3_results).T.round(4)
```

4. **Momentum factor (β_{UMD})**: The momentum loading captures whether the governance effect overlaps with price momentum. In the US, Gompers, Ishii, and Metrick (2003) found a near-zero momentum loading, suggesting the governance effect is distinct from momentum.

72.5.8 Robustness: White Heteroskedasticity-Consistent Standard Errors

As a robustness check, we also report results with White (HC1) standard errors, following the original methodology:

72.6 Deeper Analysis

72.6.1 Governance Quintile Portfolios

Rather than focusing solely on the extreme portfolios, we examine returns across all five governance quintiles. This allows us to assess whether the governance-return relationship is monotonic.

```
def assign_quintile(group):
    """Assign governance quintile within each vintage year."""
    group = group.copy()
    group["gindex_quintile"] = pd.qcut(
```

Table 72.12: Four-factor Carhart model results for governance portfolios. Standard errors are Newey-West adjusted with 6 lags.

```
ff4_results = {}
for name, y in portfolios.items():
    X = analysis_data[["mktrf", "smb", "hml", "umd"]]
    model = run_regression(y, X, hac=True)
    ff4_results[name] = {
        "Alpha (%)": model.params["const"] * 100,
        "Alpha t-stat": model.tvalues["const"],
        "Alpha p-value": model.pvalues["const"],
        "MKTRF": model.params["mktrf"],
        "MKTRF t": model.tvalues["mktrf"],
        "SMB": model.params["smb"],
        "SMB t": model.tvalues["smb"],
        "HML": model.params["hml"],
        "HML t": model.tvalues["hml"],
        "UMD": model.params["umd"],
        "UMD t": model.tvalues["umd"],
        "R-squared": model.rsquared,
        "Adj R-sq": model.rsquared_adj,
        "N": int(model.nobs)
    }

ff4_df = pd.DataFrame(ff4_results).T
ff4_df.round(4)
```

Table 72.13: Four-factor model with White heteroskedasticity-consistent standard errors (HC1)

```
white_results = {}
for name, y in portfolios.items():
    X = sm.add_constant(analysis_data[["mktrf", "smb", "hml", "umd"]])
    model = OLS(y, X).fit(cov_type="HC1")
    white_results[name] = {
        "Alpha (%)": model.params["const"] * 100,
        "Alpha t-stat": model.tvalues["const"],
        "Alpha p-value": model.pvalues["const"],
        "R-squared": model.rsquared
    }

pd.DataFrame(white_results).T.round(4)
```

```
        group["gindex"], q=5, labels=[1, 2, 3, 4, 5],
        duplicates="drop"
    )
    return group

governance_quintiles = (
    governance
    .groupby("year", group_keys=False)
    .apply(assign_quintile)
)

# Merge with stock data
if "dual_class" in governance_quintiles.columns:
    governance_quintiles = governance_quintiles.query("dual_class == 0")

merged_q = pd.merge(
    governance_quintiles[
        ["symbol", "year", "gindex", "gindex_quintile",
         "date_effective", "date_expires"]
    ],
    prices_monthly[["symbol", "date", "ret", "retx", "mktcap"]],
    on="symbol",
    how="inner"
)
```

```
merged_q = merged_q.query(
    "date >= date_effective and date <= date_expires"
).sort_values(["symbol", "date"])

# Compute lagged market value
merged_q = merged_q.sort_values(["symbol", "date"])
merged_q["mktcap_lag"] = merged_q.groupby("symbol")["mktcap"].shift(1)

first_obs_q = merged_q.groupby("symbol")["date"].transform("min")
mask_first_q = merged_q["date"] == first_obs_q
merged_q.loc[mask_first_q, "mktcap_lag"] = (
    merged_q.loc[mask_first_q, "mktcap"]
    / (1 + merged_q.loc[mask_first_q, "retx"])
)

mask_missing_q = merged_q["mktcap_lag"].isna()
merged_q.loc[mask_missing_q, "mktcap_lag"] = (
    merged_q.loc[mask_missing_q, "mktcap"]
    / (1 + merged_q.loc[mask_missing_q, "retx"])
)

merged_q = merged_q.dropna(subset=["ret", "mktcap_lag"])
merged_q = merged_q.query("mktcap_lag > 0")
```

72.6.2 Subsample Analysis

The governance premium may vary across market regimes. We split the sample at the midpoint and examine whether the effect is concentrated in a particular subperiod.

72.6.3 Equal-Weighted Robustness

Value-weighting may cause the results to be driven by a few large firms. We verify robustness with equal-weighted portfolios.

72.6.4 Factor Loading Stability: Rolling Regressions

```
# Compute VW returns by quintile and date
quintile_returns = (
    merged_q
    .groupby(["date", "gindex_quintile"])
    .apply(
        lambda g: np.average(g["ret"], weights=g["mktcap_lag"])
        if g["mktcap_lag"].sum() > 0 else np.nan,
        include_groups=False
    )
    .reset_index()
    .rename(columns={0: "ret_vw"})
)

# Average across time
quintile_avg = (
    quintile_returns
    .groupby("gindex_quintile")["ret_vw"]
    .agg(["mean", "std", "count"])
    .assign(
        se=lambda x: x["std"] / np.sqrt(x["count"]),
        ci95=lambda x: 1.96 * x["std"] / np.sqrt(x["count"])
    )
)

fig, ax = plt.subplots(figsize=(8, 5))

quintiles = quintile_avg.index.astype(int)
means = quintile_avg["mean"] * 100 * 12  # Annualized
ci = quintile_avg["ci95"] * 100 * 12

bars = ax.bar(
    quintiles, means, yerr=ci,
    color=["#1a6b3c", "#5fa35f", "#b0b0b0", "#d4836a", "#c0392b"],
    edgecolor="white", linewidth=0.5, capsize=5,
    error_kw={"linewidth": 1.5}
)

ax.axhline(0, color="gray", linestyle="--", linewidth=0.8)
ax.set_xlabel("VN-GIndex Quintile\n(1 = Strongest Rights → 5 = Weakest
↪  Rights)",
              fontsize=10)
ax.set_ylabel("Annualized Return (%)", fontsize=10)
ax.set_title(
    "Average Annualized Returns by Governance Quintile",
    fontsize=12, fontweight="bold"
)
ax.set_xticks(quintiles)
ax.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```


Figure 72.5

Table 72.14: Average monthly returns and four-factor alphas by governance quintile

```
# Merge quintile returns with factors
quintile_wide = quintile_returns.pivot(
    index="date", columns="gindex_quintile", values="ret_vw"
)
quintile_wide.columns = [f"Q{int(c)}" for c in quintile_wide.columns]
quintile_wide = quintile_wide.reset_index()

quintile_wide["ym"] = quintile_wide["date"].dt.to_period("M")
quintile_analysis = quintile_wide.merge(
    factors_ff3_monthly[["ym", "mktrf", "smb", "hml", "umd", "rf"]],
    on="ym", how="inner"
)

quintile_factor_results = {}
for q in range(1, 6):
    col = f"Q{q}"
    if col in quintile_analysis.columns:
        y = quintile_analysis[col] - quintile_analysis["rf"]
        X = sm.add_constant(
            quintile_analysis[["mktrf", "smb", "hml", "umd"]]
        )
        model = OLS(y.dropna(), X.loc[y.dropna().index]).fit(
            cov_type="HAC", cov_kwds={"maxlags": 6}
        )
        quintile_factor_results[f"Quintile {q}"] = {
            "Mean Return (%)": y.mean() * 100,
            "Alpha (%)": model.params["const"] * 100,
            "Alpha t-stat": model.tvalues["const"],
            "MKTRF Beta": model.params["mktrf"],
            "R-squared": model.rsquared
        }

# Add 5-1 spread
if "Q1" in quintile_analysis.columns and "Q5" in quintile_analysis.columns:
    y_spread = quintile_analysis["Q1"] - quintile_analysis["Q5"]
    X = sm.add_constant(
        quintile_analysis[["mktrf", "smb", "hml", "umd"]]
    )
    model_spread = OLS(
        y_spread.dropna(), X.loc[y_spread.dropna().index]
    ).fit(cov_type="HAC", cov_kwds={"maxlags": 6})
    quintile_factor_results["Q1 - Q5"] = {
        "Mean Return (%)": y_spread.mean() * 100,
        "Alpha (%)": model_spread.params["const"] * 100,
        "Alpha t-stat": model_spread.tvalues["const"],
        "MKTRF Beta": model_spread.params["mktrf"],
        "R-squared": model_spread.rsquared
    }

pd.DataFrame(quintile_factor_results).T.round(4)
```

Table 72.15: Four-factor alpha of the long-short governance strategy across subperiods

```
midpoint = analysis_data["date"].quantile(0.5)

subperiods = {
    "Full Sample": analysis_data,
    "First Half": analysis_data.query(f"date <= '{midpoint}'"),
    "Second Half": analysis_data.query(f"date > '{midpoint}'")
}

subsample_results = {}
for period_name, df in subperiods.items():
    if len(df) < 12:
        continue
    y = df["ret_excess_longshort"]
    X = sm.add_constant(df[["mktrf", "smb", "hml", "umd"]])
    model = OLS(y, X).fit(
        cov_type="HAC", cov_kwds={"maxlags": 6}
    )
    subsample_results[period_name] = {
        "Alpha (% monthly)": model.params["const"] * 100,
        "Alpha (% annual)": model.params["const"] * 100 * 12,
        "T-statistic": model.tvalues["const"],
        "P-value": model.pvalues["const"],
        "N months": int(model.nobs),
        "Date Range": f"{df['date'].min().strftime('%Y-%m')} to "
                      f"{df['date'].max().strftime('%Y-%m')}"
    }

pd.DataFrame(subsample_results).T
```

Table 72.16: Four-factor model results: Equal-weighted vs. Value-weighted long-short portfolio

```
ew_y = analysis_data["ret_diff_ew"]
vw_y = analysis_data["ret_excess_longshort"]
X = sm.add_constant(analysis_data[["mktrf", "smb", "hml", "umd"]])

model_ew = OLS(ew_y, X).fit(cov_type="HAC", cov_kwds={"maxlags": 6})
model_vw = OLS(vw_y, X).fit(cov_type="HAC", cov_kwds={"maxlags": 6})

comparison = pd.DataFrame({
    "Value-Weighted": {
        "Alpha (% monthly)": model_vw.params["const"] * 100,
        "Alpha t-stat": model_vw.tvalues["const"],
        "MKTRF": model_vw.params["mktrf"],
        "SMB": model_vw.params["smb"],
        "HML": model_vw.params["hml"],
        "UMD": model_vw.params["umd"],
        "R-squared": model_vw.rsquared
    },
    "Equal-Weighted": {
        "Alpha (% monthly)": model_ew.params["const"] * 100,
        "Alpha t-stat": model_ew.tvalues["const"],
        "MKTRF": model_ew.params["mktrf"],
        "SMB": model_ew.params["smb"],
        "HML": model_ew.params["hml"],
        "UMD": model_ew.params["umd"],
        "R-squared": model_ew.rsquared
    }
}).T

comparison.round(4)
```

```
window = 24
rolling_alpha = []

for i in range(window, len(analysis_data)):
    subset = analysis_data.iloc[i - window:i]
    y = subset["ret_excess_longshort"]
    X = sm.add_constant(subset[["mktrf", "smb", "hml", "umd"]])
    try:
        model = OLS(y, X).fit()
        rolling_alpha.append({
            "date": subset["date"].iloc[-1],
            "alpha": model.params["const"] * 100 * 12,
            "alpha_se": model.bse["const"] * 100 * 12,
            "t_stat": model.tvalues["const"]
        })
    except Exception:
        pass

rolling_alpha_df = pd.DataFrame(rolling_alpha)

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(
    rolling_alpha_df["date"],
    rolling_alpha_df["alpha"],
    color="#2c5f8a", linewidth=1.5
)
ax.fill_between(
    rolling_alpha_df["date"],
    rolling_alpha_df["alpha"] - 1.96 * rolling_alpha_df["alpha_se"],
    rolling_alpha_df["alpha"] + 1.96 * rolling_alpha_df["alpha_se"],
    alpha=0.2, color="#2c5f8a"
)
ax.axhline(0, color="gray", linestyle="--", linewidth=0.8)

ax.set_xlabel("Date", fontsize=11)
ax.set_ylabel("Annualized Alpha (%)", fontsize=11)
ax.set_title(
    "24-Month Rolling Four-Factor Alpha: Governance Long-Short Strategy",
    fontsize=12, fontweight="bold"
)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Figure 72.6

72.7 Comparison with International Evidence

72.7.1 The US Experience

In the United States, Gompers, Ishii, and Metrick (2003) documented an annualized abnormal return of approximately 8.5% for the Democracy-minus-Dictatorship strategy during the 1990s. Subsequent research has provided important nuances:

- L. Bebchuk, Cohen, and Ferrell (2009) refined the G-Index to a more parsimonious **Entrenchment Index (E-Index)** based on only six provisions that matter most for firm value: staggered boards, limits to shareholder bylaw amendments, poison pills, golden parachutes, and supermajority requirements for mergers and charter amendments. The E-Index explained the governance-return relationship at least as well as the full G-Index.
- Cremers and Nair (2005) found that the governance effect was strongest in firms where external governance mechanisms (the takeover market) were active, suggesting complementarity between internal and external governance.
- The governance premium in the US has largely disappeared in more recent periods (L. A. Bebchuk, Cohen, and Wang 2013), potentially because the market has learned to price governance differences. This "learning hypothesis" has implications for whether similar strategies can persist in less efficient markets like Vietnam.

72.7.2 Emerging Market Context

The governance-return relationship in emerging markets remains an active area of research. Several features of emerging markets suggest the premium may be larger and more persistent:

1. **Information asymmetry**: Weaker disclosure requirements and less analyst coverage mean governance quality is harder to observe, creating larger mispricings (Klapper and Love 2004).
2. **Weaker legal enforcement**: Where legal protections for minority shareholders are weak, firm-level governance becomes more important as a substitute (La Porta et al. 2000a).
3. **Concentrated ownership**: The presence of controlling shareholders creates opportunities for tunneling and related-party transactions that good governance mechanisms can mitigate (S. Johnson et al. 2000).

Vietnam, as a frontier-to-emerging market with all three characteristics, is a particularly interesting laboratory for testing whether governance generates abnormal returns.

72.8 Drawdown and Risk Analysis

72.8.1 Maximum Drawdown

```
def compute_drawdown(cumulative_returns):
    """Compute drawdown series from cumulative returns."""
    running_max = cumulative_returns.cummax()
    drawdown = (cumulative_returns - running_max) / running_max
    return drawdown

cum_dem = (1 + returns_wide["ret_vw_Democracy"]).cumprod()
cum_dic = (1 + returns_wide["ret_vw_Dictatorship"]).cumprod()

dd_dem = compute_drawdown(cum_dem)
dd_dic = compute_drawdown(cum_dic)

fig, ax = plt.subplots(figsize=(10, 5))

ax.fill_between(
    returns_wide["date"], dd_dem * 100, 0,
    alpha=0.4, color="#1a6b3c", label="Democracy"
)
ax.fill_between(
    returns_wide["date"], dd_dic * 100, 0,
    alpha=0.4, color="#c0392b", label="Dictatorship"
)
ax.set_xlabel("Date", fontsize=11)
ax.set_ylabel("Drawdown (%)", fontsize=11)
ax.set_title("Portfolio Drawdowns", fontsize=12, fontweight="bold")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Figure 72.7

Table 72.17: Risk metrics for governance portfolios

```
def compute_risk_metrics(returns, name, rf=None):
    """Compute standard risk metrics."""
    r = returns.dropna()
    if rf is not None:
        excess = r - rf
    else:
        excess = r

    ann_return = (1 + r.mean()) ** 12 - 1
    ann_vol = r.std() * np.sqrt(12)
    sharpe = excess.mean() / r.std() * np.sqrt(12) if r.std() > 0 else np.nan

    cum = (1 + r).cumprod()
    max_dd = compute_drawdown(cum).min()

    # Sortino ratio (downside deviation)
    downside = r[r < 0].std() * np.sqrt(12)
    sortino = excess.mean() * 12 / downside if downside > 0 else np.nan

    # Skewness and kurtosis
    skew = r.skew()
    kurt = r.kurtosis()

    return {
        "Portfolio": name,
        "Ann. Return (%)": ann_return * 100,
        "Ann. Volatility (%)": ann_vol * 100,
        "Sharpe Ratio": sharpe,
        "Sortino Ratio": sortino,
        "Max Drawdown (%)": max_dd * 100,
        "Skewness": skew,
        "Excess Kurtosis": kurt,
        "% Positive Months": (r > 0).mean() * 100
    }

rf_series = analysis_data.set_index("date")["rf"].reindex(
    returns_wide["date"]
).fillna(0)

risk_table = pd.DataFrame([
    compute_risk_metrics(
        returns_wide["ret_vw_Democracy"], "Democracy (VW)",
        rf_series.values
    ),
    compute_risk_metrics(
        returns_wide["ret_vw_Dictatorship"], "Dictatorship (VW)",
        rf_series.values
    ),
    compute_risk_metrics(
        returns_wide["ret_diff_vw"], "Long-Short (VW)"
    )
]).set_index("Portfolio")
```

72.9 Discussion and Interpretation

72.9.1 Why Might Governance Predict Returns in Vietnam?

Several channels may explain a governance premium in the Vietnamese market:

The risk channel: Poorly governed firms may be riskier in ways not captured by standard factor models. Investors require a higher expected return to hold these firms, but the return difference reverses (i.e., the governance premium is positive for well-governed firms) if the market overestimates the risk of poorly governed firms or if governance risk is partially diversifiable.

The mispricing channel: If the market is slow to incorporate governance information into prices, perhaps because governance quality is costly to assess or because many Vietnamese investors are retail traders with limited analytical capacity, then a systematic strategy that buys good governance and sells bad governance can profit from the gradual correction of mispricings.

The cash flow channel: Better-governed firms may generate genuinely higher cash flows because they waste less on empire-building, related-party transactions, and other value-destroying activities. To the extent that these superior cash flows are not fully anticipated by the market, good governance firms deliver positive return surprises.

72.9.2 State Ownership and the Governance Effect

A distinctive feature of the Vietnamese market is the prevalence of state-owned enterprises (SOEs). The interaction between state ownership and governance quality creates an interesting dynamic:

- SOEs may have weak governance by conventional metrics (limited board independence, political appointments) but benefit from implicit government guarantees and preferential access to land, capital, and contracts.
- The governance premium may therefore differ between SOEs and private firms. We encourage readers to extend the analysis by interacting the governance index with a state ownership indicator.

72.9.3 Limitations

Several caveats apply to this analysis:

1. **Survivorship bias**: If poorly governed firms are more likely to delist (due to financial distress or regulatory action), the Dictatorship portfolio's returns may be biased upward, attenuating the true governance premium.

2. **Transaction costs**: The long-short strategy requires short selling, which is restricted in Vietnam. Implementation via a long-only tilt toward Democracy firms may be more practical.

3. **Governance data frequency**: Unlike daily stock prices, governance data is updated infrequently (typically annually). The portfolio rebalancing frequency is therefore low, which limits the strategy's responsiveness to governance changes.

4. **Index construction**: The specific provisions included in the VN-GIndex and their equal weighting may not optimally capture governance quality. Future work could explore weighted indices, following L. Bebchuk, Cohen, and Ferrell (2009).

72.10 Exercises

1. **Entrenchment Index**: Following L. Bebchuk, Cohen, and Ferrell (2009), identify the subset of governance provisions in the Vietnamese data that have the strongest association with firm value (measured by Tobin's Q or market-to-book ratio). Construct a Vietnamese E-Index using only these provisions and compare its predictive power for returns to the full VN-GIndex.

2. **Fama-MacBeth regressions**: Instead of sorting firms into portfolios, estimate the governance-return relationship using Fama-MacBeth cross-sectional regressions (Eugene F. Fama and MacBeth 1973a):

$$r_{i,t} - r_{f,t} = \gamma_{0,t} + \gamma_{1,t}\text{VN-GIndex}_{i,t-1} + \gamma_{2,t}\mathbf{X}_{i,t-1} + \epsilon_{i,t}$$

where $\mathbf{X}_{i,t-1}$ includes controls for size, book-to-market, and momentum. Report the time-series averages of $\hat{\gamma}_{1,t}$ with Newey-West standard errors.

3. **State ownership interaction**: Split the sample into SOEs (state ownership $> 50\%$) and private firms. Does the governance premium differ between these groups? Estimate:

$$r_t^{D-D} = \alpha + \beta_1\text{MKTRF}_t + \beta_2\text{SMB}_t + \beta_3\text{HML}_t + \beta_4\text{UMD}_t + \varepsilon_t$$

separately for SOE and non-SOE subsamples.

4. **Transaction cost analysis**: Compute the portfolio turnover at each rebalancing date (when new governance data becomes available). Estimate how much of the abnormal return would be consumed by transaction costs at realistic bid-ask spreads for Vietnamese equities.

5. **Governance changes**: Do firms that improve their governance (declining VN-GIndex) earn higher subsequent returns than firms whose governance deteriorates? Construct portfolios based on ΔVN-GIndex and test.
6. **Comparison with market-wide governance reforms**: Vietnam has implemented several governance reform waves (e.g., Circular 121/2012/TT-BTC, Decree 71/2017/ND-CP). Test whether the governance premium narrows after these regulatory changes using a structural break or interaction approach.

72.11 Summary

This chapter adapts the influential Gompers, Ishii, and Metrick (2003) governance investment strategy to the Vietnamese equity market. The key methodological steps are:

1. Construct a governance index (VN-GIndex) from firm-level governance provisions available through DataCore.vn.
2. Classify firms into Democracy (strong rights) and Dictatorship (weak rights) portfolios based on the cross-sectional distribution of the index.
3. Compute value-weighted monthly portfolio returns, rebalancing when new governance data becomes available.
4. Evaluate the long-short (Democracy minus Dictatorship) strategy using t-tests and Fama-French-Carhart four-factor regressions.
5. Examine robustness across subperiods, quintile portfolios, and alternative weighting schemes.

73 Return Gap: Measuring Unobserved Actions of Fund Managers

Mutual fund managers possess considerable discretion in their investment decisions between mandatory portfolio disclosure dates. While regulatory frameworks require periodic disclosure of holdings, the actions taken between these disclosure dates (e.g., trading, market timing, securities lending, and strategic cash management) remain largely unobservable to investors. These *unobserved actions* can significantly affect fund performance, either positively through skilled interim trading or negatively through agency costs and hidden behavior.

Kacperczyk, Sialm, and Zheng (2008) developed the **Return Gap** measure to capture the aggregate impact of these unobserved actions on fund returns. The Return Gap is defined as the difference between a fund's actual reported return and the hypothetical return of a portfolio that mechanically invests in the fund's most recently disclosed holdings. Formally:

$$\text{Return Gap}_{i,t} = R_{i,t}^{\text{Actual}} - R_{i,t}^{\text{Holdings}} \tag{73.1}$$

where $R_{i,t}^{\text{Actual}}$ is the net-of-expense return reported by fund i in month t, adjusted for expenses to obtain the gross return, and $R_{i,t}^{\text{Holdings}}$ is the hypothetical buy-and-hold return computed from the most recently disclosed portfolio holdings.

A positive Return Gap indicates that the fund manager's unobserved actions (e.g., interim trading, cash management, or other activities) added value beyond what a passive replication of disclosed holdings would have generated. Conversely, a persistently negative Return Gap suggests value-destroying interim activity, potentially driven by agency costs, poor trading execution, or hidden fees.

73.1 Why Return Gap Matters

The Return Gap is economically significant for several reasons:

1. **Performance persistence**: Funds in the highest Return Gap decile tend to outperform those in the lowest decile by 1-2% annually on a risk-adjusted basis, and this spread persists over time (Kacperczyk, Sialm, and Zheng 2008).

2. **Detecting agency problems**: A persistently negative Return Gap can signal hidden costs such as excessive trading, market impact costs, soft-dollar arrangements, or stale-price exploitation.
3. **Complementing traditional measures**: Unlike alpha-based metrics that blend stock selection skill with interim trading skill, the Return Gap isolates the component of performance attributable to actions taken *between* disclosure dates.
4. **Regulatory implications**: In emerging markets like Vietnam, where disclosure frequency and regulatory oversight may differ from developed markets, the Return Gap can serve as an early warning system for investor protection.

73.2 Hidden Payments: Revenue Sharing, Distribution, and Menu Design

The Return Gap quantifies the consequences of what managers *do* between disclosure dates. A complementary literature shows that investor outcomes are equally shaped by payments that flow between funds and their distributors, largely out of investors' sight. Pool, Sialm, and Stefanescu (2026) provide the sharpest evidence from US 401(k) plans, where the recordkeeper administering the plan is often compensated indirectly through revenue sharing: rebates paid by third-party mutual funds on the plan menu out of their expense ratios. These arrangements distort the menu itself: revenue-sharing funds are more likely to be added and less likely to be removed, and revenue-sharing plans are more expensive overall, with higher expense ratios that are offset neither by lower direct fees nor by better performance. Rebates also rise with the recordkeeper's market power, consistent with funds sharing revenue to buy access to retirement assets.

The result extends an older literature on distribution conflicts in delegated asset management. Pool, Sialm, and Stefanescu (2016) show that mutual fund families acting as 401(k) plan trustees favor their own affiliated funds in menu decisions, particularly poorly performing ones, and that participants do not undo this favoritism. Sialm, Starks, and Zhang (2015) find that defined-contribution plan flows respond strongly to performance, but largely because plan sponsors add and delete menu options, which concentrates influence in exactly the intermediaries whose incentives revenue sharing targets. Outside retirement plans, Bergstresser, Chalmers, and Tufano (2009) document that broker-sold funds underperform direct-sold funds even before distribution fees, and S. E. K. Christoffersen, Evans, and Musto (2013) show that payments to brokers predict fund flows and lower subsequent performance, indicating that intermediary compensation, not just investor demand, steers money.

These findings matter for Vietnam because its fund industry is distribution-driven: most open-ended fund assets are gathered through banks, securities brokers, and licensed fund platforms, voluntary supplementary pension schemes are still nascent, and the fee arrangements between fund managers and distributors are not systematically disclosed. As menu-style products

spread through fund platforms, pension-linked insurance, and employer-sponsored savings arrangements, the US evidence suggests a clear research and regulatory agenda: measure not only what funds report and hold, but also who is paid for shelf space, because those payments predict both menu composition and investor costs. The Return Gap methodology developed in this chapter and fee-based diagnostics are complements in that agenda: one tracks hidden actions, the other hidden payments.

73.3 Hidden Inputs: Human Capital and Fund Skill

A third thing investors rarely observe is *who* produces the fund's performance. Alpha, the Return Gap, and related skill measures are outputs; the inputs are the analysts and quantitative staff the management company manages to hire. Identifying the causal contribution of those inputs is hard because talented people sort into funds that would have done well anyway. Jun Chen et al. (2026) solve the selection problem with a lottery. US hedge fund management companies apply for H-1B skilled-worker visas, and when applications exceed the annual quota, the visas are allocated by random draw, so two companies that applied for the same number of visas but won different numbers are comparable ex ante. Companies that win more visas subsequently deliver higher alphas, Sharpe ratios, and information ratios. The effect is traced to well-paid, highly educated workers with quantitative skills, who help funds develop distinctive strategies, arbitrage prominent stock anomalies, and relieve capacity constraints. Two further results sharpen the interpretation: an unexpected cut in the H-1B quota hurt the performance of the funds most dependent on such workers, and alpha generation is greatest for workers from countries with the longest waits for US permanent residency, consistent with funds exploiting a labor market friction that ties workers to their sponsor.

The design has a direct Vietnamese analogue. Fund management companies in Vietnam compete for a small pool of CFA-qualified analysts and quantitatively trained staff, foreign-affiliated managers rotate expatriate personnel in and out of Ho Chi Minh City, and work-permit and licensing rules for foreign professionals change over time. A researcher who can observe staffing at the management-company level, whether from SSC licensing records for fund management practitioners, work-permit data, or LinkedIn-style professional profiles, can ask whether skilled hires predict subsequent alpha and Return Gap, and can exploit regulatory changes in foreign-labor rules as the shock. The lesson from Jun Chen et al. (2026) is that hidden actions and hidden payments are only part of the story; hidden inputs matter for performance too, and randomized or quasi-random allocation of those inputs is what turns a correlation between talent and returns into evidence of skill.

73.4 Application to the Vietnamese Market

The Vietnamese mutual fund industry, while relatively young compared to the United States, has experienced rapid growth since the establishment of the first domestic equity funds in the early 2000s. As of 2024, Vietnam's open-ended fund industry manages assets exceeding 100 trillion VND, with dozens of equity-oriented funds operated by both domestic and foreign-affiliated asset management companies.

Several characteristics of the Vietnamese market make the Return Gap analysis particularly interesting:

- **Disclosure frequency**: Vietnamese funds are required to disclose their top holdings periodically, but the frequency and completeness of disclosure may differ from the quarterly SEC requirements in the U.S.
- **Market microstructure**: The HOSE (Ho Chi Minh Stock Exchange) and HNX (Hanoi Stock Exchange) feature daily price limits (plus or minus 7% on HOSE, plus or minus 10% on HNX), T+2 settlement, and foreign ownership limits that may constrain or enable certain interim trading strategies.
- **Information asymmetry**: In an emerging market with less analyst coverage, the scope for informed interim trading and hence positive Return Gap may be larger than in more efficient markets.
- **Regulatory environment**: Vietnam's State Securities Commission (SSC) has progressively strengthened disclosure and governance requirements, making temporal analysis of Return Gap especially informative.

74 Theoretical Framework

74.1 Decomposing Fund Returns

Consider a mutual fund i that discloses its portfolio holdings at discrete dates $\tau_1, \tau_2, ...$ As disclosed at date τ_k, the fund holds N_k securities with weights $\{w_{j,\tau_k}\}_{j=1}^{N_k}$, where w_{j,τ_k} represents the portfolio weight of security j.

Between disclosure dates τ_k and τ_{k+1}, the fund's *actual gross return* in month t can be decomposed as:

$$R_{i,t}^{\text{Gross}} = R_{i,t}^{\text{Holdings}} + \underbrace{R_{i,t}^{\text{Gross}} - R_{i,t}^{\text{Holdings}}}_{\text{Return Gap}} \tag{74.1}$$

The hypothetical holdings return $R_{i,t}^{\text{Holdings}}$ is computed as the value-weighted return of the buy-and-hold portfolio based on the most recent disclosure:

$$R_{i,t}^{\text{Holdings}} = \sum_{j=1}^{N_k} \tilde{w}_{j,t-1} \cdot r_{j,t} \tag{74.2}$$

where $r_{j,t}$ is the return of security j in month t, and $\tilde{w}_{j,t-1}$ is the *evolved* portfolio weight at the end of month $t-1$, reflecting the buy-and-hold drift from the original disclosure weights:

$$\tilde{w}_{j,t-1} = \frac{w_{j,\tau_k} \prod_{s=\tau_k+1}^{t-1}(1 + r_{j,s})}{\sum_{\ell=1}^{N_k} w_{\ell,\tau_k} \prod_{s=\tau_k+1}^{t-1}(1 + r_{\ell,s})} \tag{74.3}$$

In practice, rather than tracking evolved weights explicitly, we use dollar values of holdings positions (shares held times price) as the natural weighting scheme.

74.2 The Return Gap Measure

74.2.1 Gross Return Gap

The Return Gap as originally defined by Kacperczyk, Sialm, and Zheng (2008) uses the *gross* (before-expense) return:

$$\text{RG}_{i,t} = R_{i,t}^{\text{Gross}} - R_{i,t}^{\text{Holdings}} = \left(R_{i,t}^{\text{Net}} + \frac{\text{Expense Ratio}_{i,t}}{12} \right) - R_{i,t}^{\text{Holdings}} \tag{74.4}$$

where $R_{i,t}^{\text{Net}}$ is the reported net-of-expense return and the annual expense ratio is divided by 12 to approximate the monthly expense charge.

74.2.2 Sources of Return Gap

The Return Gap captures several components (Kacperczyk, Sialm, and Zheng 2008; Elton, Gruber, and Blake 2011):

$$\text{RG}_{i,t} = \underbrace{\Delta_{\text{trade}}}_{\text{Interim trading}} + \underbrace{\Delta_{\text{cash}}}_{\text{Cash drag/return}} + \underbrace{\Delta_{\text{fees}}}_{\text{Hidden fees}} + \underbrace{\Delta_{\text{lend}}}_{\text{Securities lending}} + \underbrace{\varepsilon_t}_{\text{Noise}} \tag{74.5}$$

where:

- Δ_{trade}: The return impact of buying and selling securities between disclosure dates. Skilled managers generate positive Δ_{trade} by timing trades.
- Δ_{cash}: The effect of holding cash or cash equivalents not captured in equity holdings disclosures. In rising markets, cash creates a drag (negative contribution); in falling markets, cash provides a cushion.
- Δ_{fees}: Transaction costs, brokerage commissions, and any hidden fees not reflected in the stated expense ratio.
- Δ_{lend}: Revenue from securities lending programs, which generates positive Return Gap.
- ε_t: Measurement noise from timing differences, stale prices, or data errors.

74.2.3 Predictive Return Gap

To form tradeable portfolios and avoid look-ahead bias, Kacperczyk, Sialm, and Zheng (2008) use the *trailing 12-month average* Return Gap, lagged by one quarter to account for the reporting delay:

$$\overline{\mathrm{RG}}_{i,t}^{12} = \frac{1}{12}\sum_{s=1}^{12} \mathrm{RG}_{i,t-s} \tag{74.6}$$

The additional 3-month (one quarter) lag ensures that the Return Gap signal is based only on information available to investors at the time of portfolio formation. This is particularly important in Vietnam, where fund reporting may involve delays.

74.3 Risk-Adjusted Performance Evaluation

To evaluate whether Return Gap-sorted portfolios generate genuine risk-adjusted returns, we employ several factor models.

74.3.1 CAPM Alpha

$$R_{p,t} - R_{f,t} = \alpha_p + \beta_p(R_{m,t} - R_{f,t}) + \epsilon_{p,t} \tag{74.7}$$

74.3.2 Fama-French Three-Factor Model

$$R_{p,t} - R_{f,t} = \alpha_p + \beta_{1,p} \cdot \mathrm{MKT}_t + \beta_{2,p} \cdot \mathrm{SMB}_t + \beta_{3,p} \cdot \mathrm{HML}_t + \epsilon_{p,t} \tag{74.8}$$

74.3.3 Carhart Four-Factor Model

$$R_{p,t} - R_{f,t} = \alpha_p + \beta_{1,p} \cdot \mathrm{MKT}_t + \beta_{2,p} \cdot \mathrm{SMB}_t + \beta_{3,p} \cdot \mathrm{HML}_t + \beta_{4,p} \cdot \mathrm{UMD}_t + \epsilon_{p,t} \tag{74.9}$$

where UMD_t is the momentum factor (up minus down).

74.3.4 Fama-French Five-Factor Model

For a more comprehensive risk adjustment relevant to the Vietnamese market:

$$R_{p,t} - R_{f,t} = \alpha_p + \beta_1 \mathrm{MKT}_t + \beta_2 \mathrm{SMB}_t + \beta_3 \mathrm{HML}_t + \beta_4 \mathrm{RMW}_t + \beta_5 \mathrm{CMA}_t + \epsilon_{p,t} \tag{74.10}$$

where RMW_t (robust minus weak) captures profitability and CMA_t (conservative minus aggressive) captures investment patterns.

74.4 Newey-West Standard Errors

Since portfolio returns may exhibit serial correlation, we use Whitney K. Newey and West (1987b) standard errors with L lags:

$$\widehat{V}(\widehat{\alpha}) = T\left(\sum_{t=1}^{T}\mathbf{x}_t\mathbf{x}_t'\right)^{-1}\widehat{S}\left(\sum_{t=1}^{T}\mathbf{x}_t\mathbf{x}_t'\right)^{-1} \tag{74.11}$$

where the HAC covariance estimator is:

$$\widehat{S} = \widehat{\Gamma}_0 + \sum_{\ell=1}^{L}\left(1-\frac{\ell}{L+1}\right)\left(\widehat{\Gamma}_\ell + \widehat{\Gamma}_\ell'\right) \tag{74.12}$$

and $\widehat{\Gamma}_\ell = \frac{1}{T}\sum_{t=\ell+1}^{T}\widehat{\epsilon}_t\widehat{\epsilon}_{t-\ell}\mathbf{x}_t\mathbf{x}_{t-\ell}'$. The standard lag choice is $L = \lfloor 4(T/100)^{2/9}\rfloor$.

75 Data and Sample Construction

75.1 Data Sources

Table 75.1 shows the sources used in the construction of return gaps.

Table 75.1: Data sources for the Return Gap analysis in Vietnam

Data Category	Source	Description
Fund holdings	DataCore Fund Holdings	Disclosed portfolio positions including ticker, shares held, report date, and vintage (filing) date
Fund returns	DataCore Fund Performance	Monthly NAV-based net returns, total net assets, and expense ratios
Fund characteristics	DataCore Fund Master	Fund objective codes, inception dates, management company, investment style
Stock prices and returns	DataCore Equity Market	Daily and monthly adjusted prices, returns, shares outstanding, and corporate actions for HOSE and HNX listed securities
Risk factors	DataCore / Constructed	Vietnamese market factor portfolios (MKT, SMB, HML, UMD, RMW, CMA)

75.2 Setting Up the Environment

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import statsmodels.api as sm
from statsmodels.regression.linear_model import OLS
from scipy import stats
from datetime import datetime, timedelta
from dateutil.relativedelta import relativedelta
import warnings

warnings.filterwarnings("ignore")

plt.rcParams.update({
    "figure.figsize": (10, 6),
    "font.size": 12,
    "axes.titlesize": 14,
    "axes.labelsize": 12,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "legend.fontsize": 10,
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "font.family": "serif",
})

sns.set_style("whitegrid")
np.random.seed(42)
```

75.3 Loading and Preparing Stock Market Data

The first step is to load the stock-level data, which provides the foundation for computing hypothetical holdings returns.

```
# ============================================================
# In production, replace with actual DataCore API calls:
#   from datacore import DataCoreClient
#   client = DataCoreClient(api_key="YOUR_KEY")
#   stock_data = client.get_equity_monthly(
#       exchange=["HOSE", "HNX"],
#       start_date="2010-01-01",
#       end_date="2024-12-31",
#       fields=["ticker", "date", "close_adj", "return_monthly",
#               "shares_outstanding", "market_cap"]
```

```
#   )
# ============================================================

def generate_stock_data(
    n_stocks: int = 300,
    start_date: str = "2012-01-01",
    end_date: str = "2024-12-31",
) -> pd.DataFrame:
    """
    Generate simulated monthly stock data mimicking Vietnamese
    equity market characteristics.
    """
    dates = pd.date_range(start_date, end_date, freq="ME")
    tickers = [f"VN{str(i).zfill(4)}" for i in range(1, n_stocks + 1)]

    records = []
    for ticker in tickers:
        list_offset = np.random.randint(0, max(1, len(dates) // 3))
        available_dates = dates[list_offset:]
        mu = np.random.normal(0.008, 0.005)
        sigma = np.random.uniform(0.06, 0.15)
        beta = np.random.uniform(0.5, 1.8)
        market_shocks = np.random.normal(0.005, 0.06, len(available_dates))
        idio_shocks = np.random.normal(0, sigma, len(available_dates))
        returns = mu + beta * market_shocks + idio_shocks
        returns = np.clip(returns, -0.30, 0.40)
        price = np.random.uniform(10, 150)
        prices = [price]
        for r in returns[:-1]:
            price = price * (1 + r)
            prices.append(price)
        shares = np.random.uniform(50, 500) * 1e6
        shares_series = np.full(len(available_dates), shares)
        for i, d in enumerate(available_dates):
            records.append({
                "ticker": ticker, "date": d,
                "close_adj": prices[i], "ret": returns[i],
                "shares_outstanding": shares_series[i],
                "market_cap": prices[i] * shares_series[i] / 1e9,
            })

    df = pd.DataFrame(records)
    df["date"] = pd.to_datetime(df["date"])
```

```
    df = df.sort_values(["ticker", "date"])
    df["close_adj_lag"] = df.groupby("ticker")["close_adj"].shift(1)
    return df

stock_data = generate_stock_data()
print(f"Stock data: {stock_data.shape[0]:,} stock-months")
print(f"Unique stocks: {stock_data['ticker'].nunique()}")
print(f"Date range: {stock_data['date'].min():%Y-%m} to "
      f"{stock_data['date'].max():%Y-%m}")
stock_data.head(10)
```

```
Stock data: 38,865 stock-months
Unique stocks: 300
Date range: 2012-01 to 2024-12
```

	ticker	date	close_adj	ret	shares_outstanding	market_cap	close_adj_lag
0	VN0001	2015-03-31	45.035190	0.110630	6.747563e+07	3.038778	NaN
1	VN0001	2015-04-30	50.017423	-0.066939	6.747563e+07	3.374957	45.035190
2	VN0001	2015-05-31	46.669311	0.047021	6.747563e+07	3.149041	50.017423
3	VN0001	2015-06-30	48.863751	-0.152745	6.747563e+07	3.297112	46.669311
4	VN0001	2015-07-31	41.400073	-0.057519	6.747563e+07	2.793496	48.863751
5	VN0001	2015-08-31	39.018788	0.228286	6.747563e+07	2.632817	41.400073
6	VN0001	2015-09-30	47.926227	0.079232	6.747563e+07	3.233852	39.018788
7	VN0001	2015-10-31	51.723515	-0.300000	6.747563e+07	3.490077	47.926227
8	VN0001	2015-11-30	36.206461	0.192114	6.747563e+07	2.443054	51.723515
9	VN0001	2015-12-31	43.162239	0.294336	6.747563e+07	2.912399	36.206461

75.4 Loading Fund Holdings Data

```
def generate_holdings_data(
    stock_data: pd.DataFrame,
    n_funds: int = 50,
    start_date: str = "2012-06-30",
    end_date: str = "2024-12-31",
) -> pd.DataFrame:
    """
    Generate simulated fund holdings data. Each fund holds 15-80
```

```
    stocks, disclosed semi-annually or quarterly.
    """
    dates = pd.date_range(start_date, end_date, freq="ME")
    tickers = stock_data["ticker"].unique()
    fund_ids = [f"FUND{str(i).zfill(3)}" for i in range(1, n_funds + 1)]
    records = []
    for fund_id in fund_ids:
        inception_idx = np.random.randint(0, max(1, len(dates) // 4))
        freq = 3 if np.random.random() < 0.7 else 6
        n_stocks_held = np.random.randint(15, 80)
        core_stocks = np.random.choice(tickers, size=n_stocks_held,
↪   replace=False)
        report_dates = dates[inception_idx::freq]
        for rdate in report_dates:
            filing_delay = np.random.randint(1, 4)
            fdate = rdate + pd.DateOffset(months=filing_delay)
            turnover = np.random.uniform(0.05, 0.20)
            n_replace = max(1, int(n_stocks_held * turnover))
            replace_idx = np.random.choice(len(core_stocks), size=n_replace,
↪   replace=False)
            new_stocks = np.random.choice(tickers, size=n_replace,
↪   replace=False)
            core_stocks[replace_idx] = new_stocks
            for ticker in core_stocks:
                shares = np.random.uniform(100_000, 5_000_000)
                records.append({
                    "fund_id": fund_id, "report_date": rdate,
                    "filing_date": fdate, "ticker": ticker,
                    "shares_held": shares,
                })
    df = pd.DataFrame(records)
    df["report_date"] = pd.to_datetime(df["report_date"])
    df["filing_date"] = pd.to_datetime(df["filing_date"])
    return df

holdings_raw = generate_holdings_data(stock_data)
print(f"Holdings records: {holdings_raw.shape[0]:,}")
print(f"Unique funds: {holdings_raw['fund_id'].nunique()}")
holdings_raw.head(10)
```

```
Holdings records: 89,269
Unique funds: 50
```

	fund_id	report_date	filing_date	ticker	shares_held
0	FUND001	2012-11-30	2012-12-30	VN0289	4.918132e+06
1	FUND001	2012-11-30	2012-12-30	VN0297	4.322425e+05
2	FUND001	2012-11-30	2012-12-30	VN0259	2.383117e+05
3	FUND001	2012-11-30	2012-12-30	VN0215	1.140891e+06
4	FUND001	2012-11-30	2012-12-30	VN0262	1.104603e+06
5	FUND001	2012-11-30	2012-12-30	VN0292	1.665416e+06
6	FUND001	2012-11-30	2012-12-30	VN0132	4.625400e+06
7	FUND001	2012-11-30	2012-12-30	VN0049	3.447053e+06
8	FUND001	2012-11-30	2012-12-30	VN0008	1.525004e+05
9	FUND001	2012-11-30	2012-12-30	VN0189	2.527258e+06

75.5 Loading Fund Returns and Characteristics

```
def generate_fund_returns(holdings, start_date="2012-01-01",
 ↪  end_date="2024-12-31"):
    """Generate monthly fund-level net returns, TNA, and expense ratios."""
    fund_ids = holdings["fund_id"].unique()
    dates = pd.date_range(start_date, end_date, freq="ME")
    records = []
    for fund_id in fund_ids:
        fund_start = holdings.loc[holdings["fund_id"] == fund_id,
 ↪  "report_date"].min() - pd.DateOffset(months=3)
        fund_dates = dates[dates >= fund_start]
        exp_ratio = np.random.uniform(0.010, 0.025)
        base_tna = np.random.uniform(50, 2000)
        mu = np.random.normal(0.007, 0.003)
        sigma = np.random.uniform(0.04, 0.09)
        tna = base_tna
        for d in fund_dates:
            ret = np.clip(np.random.normal(mu, sigma), -0.25, 0.35)
            tna = max(tna * (1 + ret) + np.random.normal(0, base_tna * 0.02),
 ↪  10)
            records.append({"fund_id": fund_id, "date": d, "net_return": ret,
                            "tna": tna, "expense_ratio": exp_ratio +
                             ↪  np.random.normal(0, 0.001)})
    df = pd.DataFrame(records)
    df["date"] = pd.to_datetime(df["date"])
    df["expense_ratio"] = df["expense_ratio"].clip(0.005, 0.035)
```

```
    return df

fund_returns = generate_fund_returns(holdings_raw)
print(f"Fund-month observations: {fund_returns.shape[0]:,}")
fund_returns.head(10)
```

```
Fund-month observations: 6,808
```

	fund_id	date	net_return	tna	expense_ratio
0	FUND001	2012-08-31	0.045634	568.611866	0.022429
1	FUND001	2012-09-30	0.106959	631.630643	0.021461
2	FUND001	2012-10-31	-0.063495	612.614200	0.020634
3	FUND001	2012-11-30	-0.087247	563.095634	0.023795
4	FUND001	2012-12-31	-0.006777	545.547597	0.021886
5	FUND001	2013-01-31	-0.029553	532.221482	0.021999
6	FUND001	2013-02-28	-0.002767	538.411988	0.021567
7	FUND001	2013-03-31	-0.138187	477.997478	0.022189
8	FUND001	2013-04-30	0.045137	484.711443	0.022058
9	FUND001	2013-05-31	0.095518	523.213577	0.021911

75.6 Sample Selection: Domestic Equity Funds

Following the approach of Kacperczyk, Sialm, and Zheng (2008), we restrict our sample to domestic equity funds.

```
equity_objectives = [
    "EQUITY_DOMESTIC", "EQUITY_GROWTH", "EQUITY_VALUE",
    "EQUITY_BLEND", "EQUITY_LARGE_CAP", "EQUITY_MID_CAP",
    "EQUITY_SMALL_CAP",
]

fund_ids = fund_returns["fund_id"].unique()
fund_master = pd.DataFrame({
    "fund_id": fund_ids,
    "objective": np.random.choice(
        equity_objectives + ["BOND", "BALANCED", "MONEY_MARKET"],
        size=len(fund_ids),
        p=[0.08, 0.08, 0.06, 0.10, 0.08, 0.06, 0.06, 0.15, 0.18, 0.15],
```

```
    ),
})

equity_fund_ids = fund_master.loc[
    fund_master["objective"].isin(equity_objectives), "fund_id"
].values

print(f"Total funds: {len(fund_ids)}")
print(f"Equity funds: {len(equity_fund_ids)}
 ↪  ({len(equity_fund_ids)/len(fund_ids)*100:.1f}%)")
print("\nObjective distribution:")
print(fund_master["objective"].value_counts().to_string())
```

```
Total funds: 50
Equity funds: 29 (58.0%)

Objective distribution:
objective
BALANCED            9
EQUITY_VALUE        9
EQUITY_GROWTH       6
BOND                6
MONEY_MARKET        6
EQUITY_BLEND        5
EQUITY_LARGE_CAP    3
EQUITY_SMALL_CAP    2
EQUITY_MID_CAP      2
EQUITY_DOMESTIC     2
```

76 Computing the Return Gap

76.1 Step 1: Prepare Holdings Vintages

A critical first step is to correctly handle the *vintage* structure of holdings data. Each holdings report has two key dates: the **report date** (τ, the as-of date) and the **filing date** (f, when it becomes public). We keep only the first vintage per fund-report date.

```
def prepare_holdings_vintages(holdings, max_holding_months=6):
    """Process holdings vintages and compute next report dates."""
    first_vintage = (
        holdings.sort_values(["fund_id", "report_date", "filing_date"])
        .groupby(["fund_id", "report_date"])
        .agg(filing_date=("filing_date", "first"))
        .reset_index()
    )
    first_vintage = first_vintage.sort_values(["fund_id", "report_date"])
    first_vintage["next_report_date"] =
 ↪  first_vintage.groupby("fund_id")["report_date"].shift(-1)
    max_date = first_vintage["report_date"] +
 ↪  pd.DateOffset(months=max_holding_months)
    first_vintage["next_report_date"] =
 ↪  first_vintage["next_report_date"].fillna(max_date)
    first_vintage["next_report_date"] =
 ↪  first_vintage[["next_report_date"]].min(axis=1).clip(upper=max_date)
    first_vintage["next_report_date"] = first_vintage["next_report_date"] +
 ↪  pd.offsets.MonthEnd(0)
    result = holdings.merge(first_vintage, on=["fund_id", "report_date",
 ↪  "filing_date"], how="inner")
    return result

holdings_vintaged = prepare_holdings_vintages(holdings_raw)
print(f"Holdings after vintage processing: {holdings_vintaged.shape[0]:,}
 ↪  records")
sample_fund = holdings_vintaged["fund_id"].iloc[0]
(holdings_vintaged.loc[holdings_vintaged["fund_id"] == sample_fund]
```

```
 [["fund_id", "report_date", "filing_date", "next_report_date"]]
 .drop_duplicates().head(8))
```

```
Holdings after vintage processing: 89,269 records
```

	fund_id	report_date	filing_date	next_report_date
0	FUND001	2012-11-30	2012-12-30	2013-02-28
52	FUND001	2013-02-28	2013-03-28	2013-05-31
104	FUND001	2013-05-31	2013-08-31	2013-08-31
156	FUND001	2013-08-31	2013-11-30	2013-11-30
208	FUND001	2013-11-30	2014-02-28	2014-02-28
260	FUND001	2014-02-28	2014-04-28	2014-05-31
312	FUND001	2014-05-31	2014-08-31	2014-08-31
364	FUND001	2014-08-31	2014-10-31	2014-11-30

76.2 Step 2: Adjust Shares for Corporate Actions

```
def adjust_holdings_shares(holdings, stock_data):
    """Adjust shares for splits, bonuses, rights. Simulated: factor=1."""
    holdings["shares_adj"] = holdings["shares_held"]
    return holdings

holdings_adj = adjust_holdings_shares(holdings_vintaged, stock_data)
```

76.3 Step 3: Compute Hypothetical Holdings Returns

This is the core computation. For each fund, we take the disclosed holdings as of report date τ, and for each month t in $(\tau, \tau_{\text{next}}]$, compute the value-weighted return using lagged dollar values as weights.

```
def compute_holdings_returns(holdings, stock_data, min_stocks=10,
 ↪  min_assets_bn=5.0):
    """Compute monthly hypothetical buy-and-hold portfolio returns."""
    merged = holdings.merge(stock_data, on="ticker", how="inner")
    mask = (merged["date"] > merged["report_date"]) & (merged["date"] <=
 ↪  merged["next_report_date"])
```

```
    merged = merged.loc[mask].copy()
    merged["hvalue_lag"] = merged["shares_adj"] * merged["close_adj_lag"]
    merged = merged.loc[merged["hvalue_lag"] > 0].copy()
    merged = merged.drop_duplicates(subset=["fund_id", "date", "report_date",
↪   "ticker"], keep="first")

    def weighted_return(group):
        weights = group["hvalue_lag"]
        total_weight = weights.sum()
        if total_weight <= 0:
            return pd.Series({"hret": np.nan, "n_stocks": 0, "assets_lag_bn":
             ↪  0})
        wret = np.average(group["ret"], weights=weights)
        return pd.Series({"hret": wret, "n_stocks": len(group),
         ↪  "assets_lag_bn": total_weight / 1e9})

    portfolio_returns = (
        merged.groupby(["fund_id", "date"])
        .apply(weighted_return, include_groups=False).reset_index()
    )
    portfolio_returns["assets_bn"] = portfolio_returns["assets_lag_bn"] * (1
↪   + portfolio_returns["hret"])
    mask = (portfolio_returns["n_stocks"] >= min_stocks) &
↪   (portfolio_returns["assets_bn"] >= min_assets_bn)
    return portfolio_returns.loc[mask].copy()

holdings_returns = compute_holdings_returns(holdings_adj, stock_data)
print(f"Fund-month observations (hypothetical returns):
 ↪  {holdings_returns.shape[0]:,}")
print(f"Unique funds: {holdings_returns['fund_id'].nunique()}")
print(f"\nSummary:")
print(holdings_returns[["hret", "n_stocks",
 ↪  "assets_bn"]].describe().round(4).to_string())
```

```
Fund-month observations (hypothetical returns): 6,170
Unique funds: 50

Summary:
            hret   n_stocks  assets_bn
count  6170.0000  6170.0000  6170.0000
mean      0.0149    42.7476    27.8827
std       0.0426    14.5061    20.1749
```

```
min      -0.1997    13.0000     5.0241
25%      -0.0111    31.0000    13.3775
50%       0.0146    41.0000    22.6388
75%       0.0411    53.0000    35.6065
max       0.2209    74.0000   167.3511
```

76.4 Step 4: Compute Gross Fund Returns

```
def prepare_fund_returns(fund_returns, equity_fund_ids):
    """Prepare fund-level gross returns."""
    df =
↪   fund_returns.loc[fund_returns["fund_id"].isin(equity_fund_ids)].copy()
    df["expense_ratio"] = df["expense_ratio"].fillna(df.groupby("fund_id")["
↪   expense_ratio"].transform("median"))
    df["gross_return"] = df["net_return"] + df["expense_ratio"] / 12
    df = df.sort_values(["fund_id", "date"])
    df["tna_lag"] = df.groupby("fund_id")["tna"].shift(1).fillna(df["tna"])
    return df

fund_ret_clean = prepare_fund_returns(fund_returns, equity_fund_ids)
print(f"Equity fund-months: {fund_ret_clean.shape[0]:,}")
```

```
Equity fund-months: 3,993
```

76.5 Step 5: Merge and Compute Return Gap

```
def compute_return_gap(holdings_returns, fund_returns):
    """Compute Return Gap and trailing averages."""
    merged = holdings_returns.merge(
        fund_returns[["fund_id", "date", "net_return", "gross_return",
↪   "expense_ratio", "tna"]],
        on=["fund_id", "date"], how="inner",
    )
    merged["return_gap"] = merged["gross_return"] - merged["hret"]
    merged = merged.sort_values(["fund_id", "date"])
    merged["rg_12m"] = merged.groupby("fund_id")["return_gap"].transform(
        lambda x: x.rolling(12, min_periods=8).mean()
```

```
    )
    merged["rg_12m_lag4"] = merged.groupby("fund_id")["rg_12m"].shift(4)
    return merged

return_gap_data = compute_return_gap(holdings_returns, fund_ret_clean)
print(f"Return Gap observations: {return_gap_data.shape[0]:,}")
print(f"\nSummary:")
print(return_gap_data[["return_gap", "rg_12m",
 ↪  "rg_12m_lag4"]].describe().round(6).to_string())
```

```
Return Gap observations: 3,592

Summary:
        return_gap       rg_12m  rg_12m_lag4
count  3592.000000  3389.000000  3273.000000
mean     -0.005734    -0.005349    -0.005329
std       0.080211     0.022955     0.022864
min      -0.307136    -0.094187    -0.094187
25%      -0.058760    -0.019080    -0.019057
50%      -0.006127    -0.005284    -0.005242
75%       0.047785     0.008620     0.008653
max       0.277137     0.079589     0.079589
```

76.6 Distribution of the Return Gap

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

rg = return_gap_data["return_gap"].dropna()
rg_trimmed = rg.clip(rg.quantile(0.01), rg.quantile(0.99))
axes[0].hist(rg_trimmed, bins=80, density=True, alpha=0.7, color="#2C5F8A",
 ↪  edgecolor="white", linewidth=0.5)
axes[0].axvline(rg.mean(), color="#D32F2F", linestyle="--", linewidth=2,
 ↪  label=f"Mean = {rg.mean():.4f}")
axes[0].axvline(rg.median(), color="#FF8F00", linestyle="-.", linewidth=2,
 ↪  label=f"Median = {rg.median():.4f}")
axes[0].set_xlabel("Monthly Return Gap")
axes[0].set_ylabel("Density")
axes[0].set_title("Panel A: Monthly Return Gap")
axes[0].legend(frameon=True)
```

```
rg12 = return_gap_data["rg_12m"].dropna()
rg12_trimmed = rg12.clip(rg12.quantile(0.01), rg12.quantile(0.99))
axes[1].hist(rg12_trimmed, bins=80, density=True, alpha=0.7, color="#1B5E20",
 ↪  edgecolor="white", linewidth=0.5)
axes[1].axvline(rg12.mean(), color="#D32F2F", linestyle="--", linewidth=2,
 ↪  label=f"Mean = {rg12.mean():.4f}")
axes[1].axvline(rg12.median(), color="#FF8F00", linestyle="-.", linewidth=2,
 ↪  label=f"Median = {rg12.median():.4f}")
axes[1].set_xlabel("Trailing 12-Month Average Return Gap")
axes[1].set_ylabel("Density")
axes[1].set_title("Panel B: 12-Month Average Return Gap")
axes[1].legend(frameon=True)
plt.tight_layout()
plt.show()
```

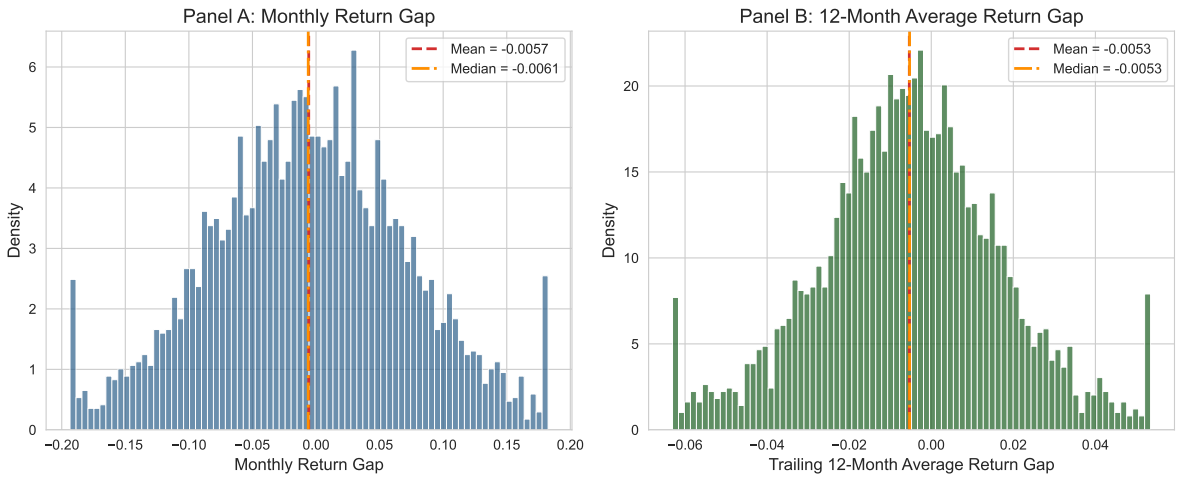


Figure 76.1: Distribution of monthly Return Gap across all fund-month observations.

76.7 Time Series of Cross-Sectional Return Gap

```
ts_stats = (
    return_gap_data.groupby("date")["return_gap"]
    .agg(["mean", "median", lambda x: x.quantile(0.25), lambda x:
 ↪  x.quantile(0.75)])
    .rename(columns={"<lambda_0>": "p25", "<lambda_1>": "p75"}).reset_index()
```

```
)

fig, ax = plt.subplots(figsize=(12, 5))
ax.fill_between(ts_stats["date"], ts_stats["p25"], ts_stats["p75"],
 ↪  alpha=0.3, color="#2C5F8A", label="IQR")
ax.plot(ts_stats["date"], ts_stats["median"], color="#2C5F8A", linewidth=2,
 ↪  label="Median")
ax.plot(ts_stats["date"], ts_stats["mean"], color="#D32F2F", linestyle="--",
 ↪  linewidth=1.5, label="Mean")
ax.axhline(0, color="black", linewidth=0.8)
ax.set_xlabel("Date")
ax.set_ylabel("Monthly Return Gap")
ax.set_title("Cross-Sectional Distribution of Return Gap Over Time")
ax.legend(frameon=True)
plt.tight_layout()
plt.show()
```

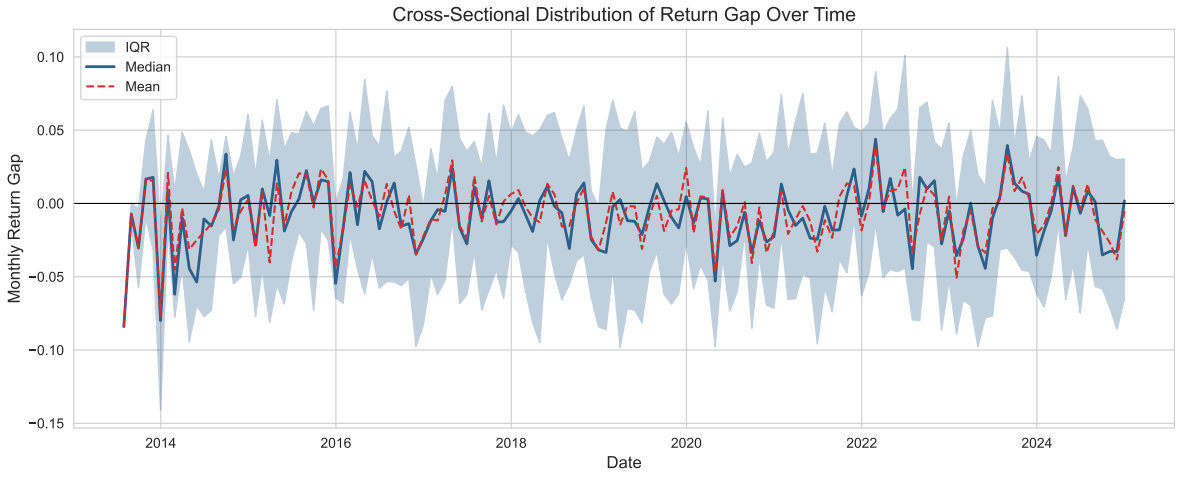


Figure 76.2: Time series of cross-sectional Return Gap statistics. Solid: median, shaded: IQR, dashed: mean.

77 Portfolio Sorting Analysis

77.1 Forming Return Gap Decile Portfolios

Each month t, we sort funds into decile portfolios based on their lagged 12-month average Return Gap ($\overline{\text{RG}}^{12}_{i,t-4}$). Portfolio 1 contains funds with the lowest Return Gap.

```
def form_return_gap_portfolios(data, n_portfolios=10,
 ↪  sort_var="rg_12m_lag4"):
    """Form portfolios by sorting funds into quantile groups."""
    df = data.dropna(subset=[sort_var]).copy()
    df["portfolio"] = (
        df.groupby("date")[sort_var]
        .transform(lambda x: pd.qcut(x, n_portfolios, labels=False,
 ↪  duplicates="drop"))
    ) + 1
    return df

n_portfolios = 10
portfolio_data = form_return_gap_portfolios(return_gap_data,
 ↪  n_portfolios=n_portfolios)
print(f"Observations with portfolio assignment: {portfolio_data.shape[0]:,}")
```

```
Observations with portfolio assignment: 3,273
```

77.2 Portfolio Returns

```
def compute_portfolio_returns(data, n_portfolios=10):
    """Compute equal- and value-weighted monthly returns."""
    ew = data.groupby(["date", "portfolio"]).agg(
        ew_ret=("net_return", "mean"), n_funds=("fund_id",
 ↪  "count")).reset_index()
    def vw_func(group):
```

```
        w = group["tna"].clip(lower=0)
        return np.average(group["net_return"], weights=w) if w.sum() > 0
         ↪  else group["net_return"].mean()
    vw = data.groupby(["date", "portfolio"]).apply(vw_func,
 ↪  include_groups=False).reset_index(name="vw_ret")
    return ew.merge(vw, on=["date", "portfolio"], how="left")

port_returns = compute_portfolio_returns(portfolio_data)
port_wide = port_returns.pivot_table(index="date", columns="portfolio",
 ↪  values=["ew_ret", "vw_ret"])
port_wide[("ew_ret", "LS")] = port_wide[("ew_ret", n_portfolios)] -
 ↪  port_wide[("ew_ret", 1)]
port_wide[("vw_ret", "LS")] = port_wide[("vw_ret", n_portfolios)] -
 ↪  port_wide[("vw_ret", 1)]

print("EW portfolio returns (annualized, %):")
print((port_wide["ew_ret"].mean() * 12 * 100).round(2).to_string())
```

```
EW portfolio returns (annualized, %):
portfolio
1.0      4.42
2.0     12.09
3.0     15.52
4.0      8.35
5.0     10.31
6.0     14.38
7.0      1.18
8.0     10.99
9.0      1.33
10.0    15.19
LS      10.77
```

77.3 Characteristics of Return Gap Portfolios

77.4 Cumulative Returns of Extreme Portfolios

Table 77.1: Characteristics of Return Gap-sorted decile portfolios.

```
chars = portfolio_data.groupby("portfolio").agg(
    avg_rg=("return_gap", "mean"), avg_rg12=("rg_12m_lag4", "mean"),
    avg_net_ret=("net_return", "mean"), avg_gross_ret=("gross_return",
 ↪  "mean"),
    avg_hret=("hret", "mean"), avg_expense=("expense_ratio", "mean"),
    avg_tna=("tna", "mean"), avg_nstocks=("n_stocks", "mean"),
 ↪  n_obs=("fund_id", "count"),
).round(4)
dc = chars.copy()
dc.columns = ["Avg RG", "Avg RG(12m)", "Net Ret", "Gross Ret", "Hold Ret",
 ↪  "Expense", "TNA(Bn)", "#Stocks", "#Obs"]
for col in ["Avg RG", "Avg RG(12m)", "Net Ret", "Gross Ret", "Hold Ret",
 ↪  "Expense"]:
    dc[col] = (dc[col] * 100).round(3)
dc["TNA(Bn)"] = dc["TNA(Bn)"].round(1)
dc["#Stocks"] = dc["#Stocks"].round(1)
print(dc.to_string())
```

```
           Avg RG  Avg RG(12m)  Net Ret  Gross Ret  Hold Ret  Expense  TNA(Bn)  #Stocks  #Obs
portfolio
1.0         -0.81        -4.32     0.50       0.66      1.48     1.91   1395.7     41.9   353
2.0          0.16        -2.73     1.02       1.18      1.02     1.86   1941.3     43.9   333
3.0          0.06        -1.85     1.27       1.42      1.36     1.79   2042.0     45.5   335
4.0         -1.11        -1.22     0.51       0.66      1.78     1.81   2052.1     47.6   325
5.0         -0.57        -0.69     0.61       0.76      1.33     1.83   2010.7     46.8   341
6.0          0.04        -0.20     1.23       1.39      1.35     1.86   2138.9     44.8   243
7.0         -1.53         0.22    -0.09       0.07      1.60     1.86   2184.0     44.2   321
8.0         -0.51         0.85     1.14       1.30      1.82     1.93   2340.3     43.7   338
9.0         -1.30         1.61     0.09       0.25      1.54     1.87   2552.9     41.9   330
10.0         0.11         3.17     1.31       1.46      1.35     1.83   2864.7     40.2   351
```

```
cum_ret = pd.DataFrame(index=port_wide.index)
cum_ret["P1 (Low RG)"] = (1 + port_wide[("ew_ret", 1)]).cumprod()
cum_ret["P10 (High RG)"] = (1 + port_wide[("ew_ret",
 ↪  n_portfolios)]).cumprod()
cum_ret["L/S (P10-P1)"] = (1 + port_wide[("ew_ret", "LS")]).cumprod()

fig, ax = plt.subplots(figsize=(12, 6))
colors = {"P1 (Low RG)": "#D32F2F", "P10 (High RG)": "#1B5E20", "L/S
 ↪  (P10-P1)": "#1565C0"}
styles = {"P1 (Low RG)": "--", "P10 (High RG)": "-", "L/S (P10-P1)": "-."}
for col in cum_ret.columns:
    ax.plot(cum_ret.index, cum_ret[col], label=col, color=colors[col],
 ↪  linestyle=styles[col], linewidth=2)
ax.axhline(1, color="black", linewidth=0.8, alpha=0.5)
ax.set_xlabel("Date")
ax.set_ylabel("Cumulative Return (Growth of 1 VND)")
ax.set_title("Cumulative Performance of Return Gap Portfolios")
ax.legend(frameon=True, loc="upper left")
ax.set_yscale("log")
plt.tight_layout()
plt.show()
```

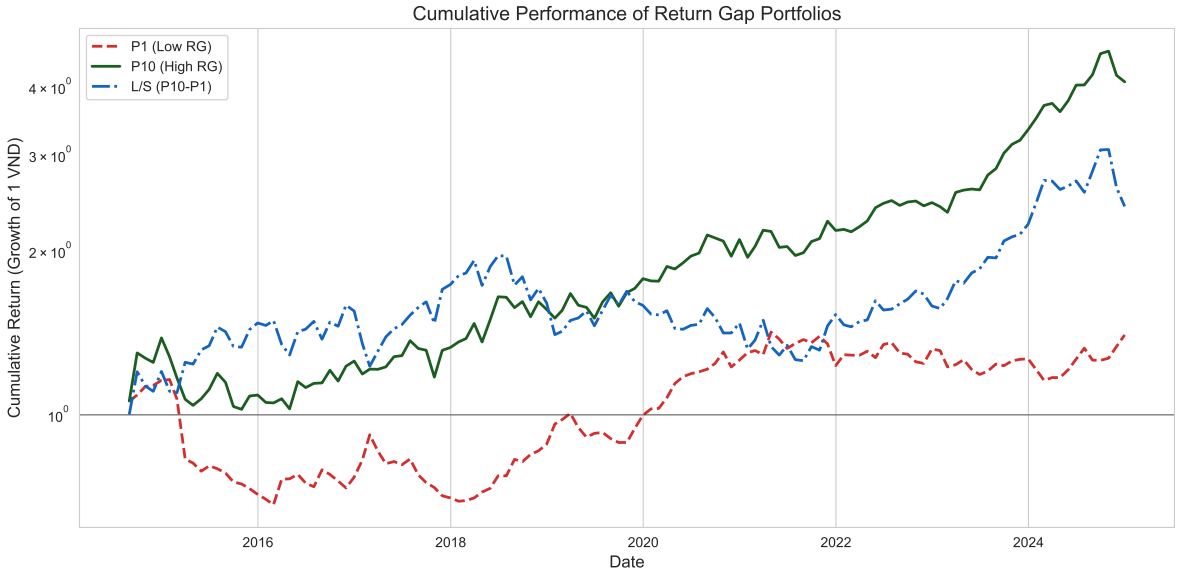


Figure 77.1: Cumulative returns of Return Gap-sorted portfolios: P10 (highest RG) vs P1 (lowest), and the long-short spread.

78 Risk-Adjusted Performance

78.1 Risk Factors

```
def generate_factor_returns(start_date="2012-01-01", end_date="2024-12-31"):
    """Generate simulated Vietnamese market factor returns."""
    dates = pd.date_range(start_date, end_date, freq="ME")
    n = len(dates)
    return pd.DataFrame({
        "date": dates,
        "rf": np.random.normal(0.004, 0.001, n).clip(0.001, 0.008),
        "mkt_rf": np.random.normal(0.008, 0.055, n),
        "smb": np.random.normal(0.003, 0.035, n),
        "hml": np.random.normal(0.002, 0.030, n),
        "umd": np.random.normal(0.005, 0.045, n),
        "rmw": np.random.normal(0.002, 0.025, n),
        "cma": np.random.normal(0.001, 0.020, n),
    })

factors = generate_factor_returns()
print("Factor summary (monthly %):")
print((factors.drop(columns="date").describe() * 100).round(3).to_string())
```

```
Factor summary (monthly %):
              rf     mkt_rf        smb        hml        umd        rmw        cma
count  15600.000  15600.000  15600.000  15600.000  15600.000  15600.000  15600.000
mean       0.399      0.789      0.454     -0.073      0.396      0.076     -0.043
std        0.099      5.276      3.744      2.899      4.615      2.400      1.940
min        0.132    -12.061     -9.021     -6.937    -11.322     -6.592     -5.971
25%        0.344     -2.929     -1.830     -2.036     -2.440     -1.425     -1.554
50%        0.397      0.844      0.611     -0.056      0.632      0.033      0.003
75%        0.464      3.844      2.981      1.976      3.597      1.707      1.317
max        0.653     15.678      9.744      8.316     13.215      6.142      4.607
```

78.2 Alpha Estimation

```
def estimate_portfolio_alphas(port_returns, factors, n_portfolios=10,
↪  nw_lags=6):
    """Estimate alphas using multiple factor models."""
    ew_wide = port_returns.pivot_table(index="date", columns="portfolio",
↪  values="ew_ret")
    if n_portfolios in ew_wide.columns and 1 in ew_wide.columns:
        ew_wide["LS"] = ew_wide[n_portfolios] - ew_wide[1]
    merged = ew_wide.merge(factors, on="date", how="inner")
    results = []
    portfolios = list(range(1, n_portfolios + 1)) + ["LS"]
    for port in portfolios:
        if port not in merged.columns: continue
        y_raw = merged[port].dropna()
        idx = y_raw.index
        rf = merged.loc[idx, "rf"]; mkt = merged.loc[idx, "mkt_rf"]
        smb = merged.loc[idx, "smb"]; hml = merged.loc[idx, "hml"]
        umd = merged.loc[idx, "umd"]; rmw = merged.loc[idx, "rmw"]
        cma = merged.loc[idx, "cma"]
        y_ex = y_raw - rf
        models = {
            "Raw Mean": (y_raw, None),
            "Excess Return": (y_raw - rf - mkt, None),
            "CAPM": (y_ex, sm.add_constant(mkt)),
            "FF3": (y_ex, sm.add_constant(pd.concat([mkt, smb, hml],
            ↪  axis=1))),
            "Carhart": (y_ex, sm.add_constant(pd.concat([mkt, smb, hml, umd],
            ↪  axis=1))),
            "FF5": (y_ex, sm.add_constant(pd.concat([mkt, smb, hml, rmw,
            ↪  cma], axis=1))),
        }
        for mname, (y, X) in models.items():
            if X is None:
                mean_val = y.mean(); se = y.std() / np.sqrt(len(y))
                t_stat = mean_val / se if se > 0 else np.nan
                p_val = 2 * (1 - stats.t.cdf(abs(t_stat), len(y)-1))
                alpha = mean_val
            else:
                try:
                    reg = OLS(y, X).fit(cov_type="HAC", cov_kwds={"maxlags":
↪  nw_lags})
```

```
                    alpha = reg.params.iloc[0]; t_stat =
 ↪  reg.tvalues.iloc[0]; p_val = reg.pvalues.iloc[0]
                except: alpha, t_stat, p_val = np.nan, np.nan, np.nan
            results.append({"portfolio": port, "model": mname, "alpha":
 ↪  alpha, "t_stat": t_stat, "p_value": p_val})
    return pd.DataFrame(results)

alpha_results = estimate_portfolio_alphas(port_returns, factors,
 ↪  n_portfolios)
print(f"Alpha estimates: {len(alpha_results)}")
```

```
Alpha estimates: 66
```

78.3 Alpha Table

78.4 Alpha Plot

```
models_plot = ["Excess Return", "CAPM", "FF3", "Carhart", "FF5"]
colors_m = {"Excess Return": "#D32F2F", "CAPM": "#FF8F00", "FF3": "#1B5E20",
 ↪  "Carhart": "#1565C0", "FF5": "#6A1B9A"}

fig, ax = plt.subplots(figsize=(12, 7))
for model in models_plot:
    md = alpha_results.loc[(alpha_results["model"]==model) &
 ↪  (alpha_results["portfolio"]!="LS")].sort_values("portfolio")
    ax.plot(md["portfolio"], md["alpha"]*100, marker="o", linewidth=2.5,
 ↪  markersize=8, label=model, color=colors_m[model])
ax.axhline(0, color="black", linewidth=0.8, alpha=0.5)
ax.set_xlabel("Return Gap Portfolio (1=Lowest, 10=Highest)")
ax.set_ylabel("Monthly Alpha (%)")
ax.set_title("Abnormal Returns of Return Gap-Sorted Portfolios\nVietnamese
 ↪  Domestic Equity Funds")
ax.legend(frameon=True, title="Risk Model")
ax.set_xticks(range(1, 11))
plt.tight_layout()
plt.show()
```

Table 78.1: Risk-adjusted monthly alphas (%) for Return Gap decile portfolios. t-stats (NW, 6 lags) in parentheses.

```
def stars(p):
    if pd.isna(p): return ""
    if p < 0.01: return "***"
    if p < 0.05: return "**"
    if p < 0.10: return "*"
    return ""

models_list = ["Raw Mean", "Excess Return", "CAPM", "FF3", "Carhart", "FF5"]
rows = []
for model in models_list:
    md = alpha_results.loc[alpha_results["model"] == model]
    for _, row in md.iterrows():
        a = row["alpha"] * 100
        rows.append({"Portfolio": row["portfolio"], "Model": model,
                     "Alpha (%)": f"{a:.3f}{stars(row['p_value'])}",
                      ↪  "t-stat": f"({row['t_stat']:.2f})"})
pivot = pd.DataFrame(rows).pivot_table(index="Portfolio", columns="Model",
 ↪  values="Alpha (%)", aggfunc="first")
pivot = pivot.reindex(columns=models_list)
print(pivot.to_string())
```

Model	Raw Mean	Excess Return	CAPM	FF3	Carhart	FF5
Portfolio						
1	0.368	-0.817	-0.126	-0.098	-0.183	-0.058
2	1.007**	-0.266	0.596	0.518	0.251	0.561
3	1.293***	0.017	0.902***	0.956***	0.995***	0.925***
4	0.696*	-0.577	0.345	0.344	0.270	0.283
5	0.859**	-0.347	0.615	0.604	0.721*	0.590
6	1.198***	-0.036	0.779*	0.837**	0.819*	0.782*
7	0.099	-1.128*	-0.274	-0.231	-0.119	-0.203
8	0.916**	-0.405	0.515	0.566	0.603	0.555
9	0.111	-1.116*	-0.357	-0.320	-0.318	-0.345
10	1.266***	0.080	0.765**	0.699*	0.784*	0.733*
LS	0.897	-0.289	0.492	0.398	0.567	0.392

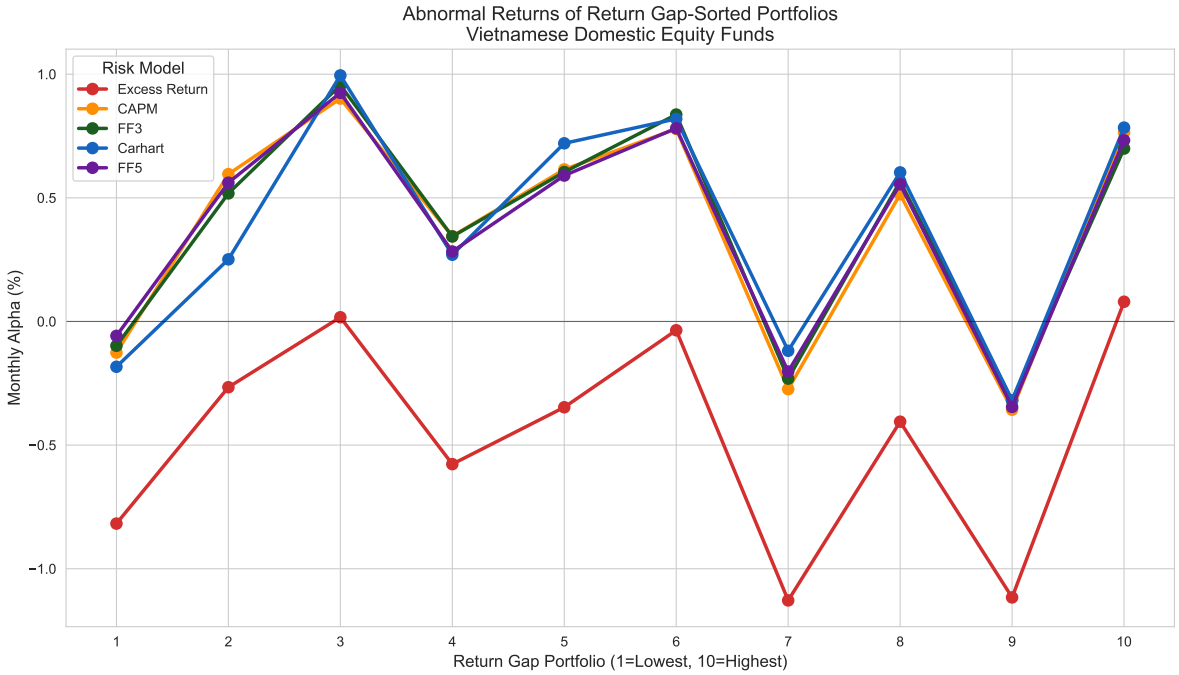


Figure 78.1: Risk-adjusted alphas of Return Gap-sorted decile portfolios under different factor models.

78.5 Long-Short Portfolio Analysis

Table 78.2: Performance of the long-short Return Gap strategy (P10 minus P1).

```
def long_short_analysis(port_returns, factors, n_portfolios=10):
    wide = port_returns.pivot_table(index="date", columns="portfolio",
    values="ew_ret")
    ls = (wide[n_portfolios] - wide[1]).dropna()
    merged = pd.DataFrame({"ls_ret": ls}).merge(factors, on="date",
    how="inner")
    ann_ret = ls.mean() * 12; ann_vol = ls.std() * np.sqrt(12)
    sharpe = ann_ret / ann_vol if ann_vol > 0 else np.nan
    cum = (1 + ls).cumprod(); max_dd = (cum / cum.cummax() - 1).min()
    sd = {"Ann. Return (%)": ann_ret*100, "Ann. Volatility (%)": ann_vol*100,
    "Sharpe Ratio": sharpe,
          "Max Drawdown (%)": max_dd*100, "% Positive Months":
           (ls>0).mean()*100}
    y = merged["ls_ret"] - merged["rf"]
    for mn, fc in {"CAPM":["mkt_rf"],"FF3":["mkt_rf","smb","hml"],"Carhart":
     ["mkt_rf","smb","hml","umd"],
                   "FF5":["mkt_rf","smb","hml","rmw","cma"]}.items():
        X = sm.add_constant(merged[fc])
        reg = OLS(y, X).fit(cov_type="HAC", cov_kwds={"maxlags": 6})
        sd[f"{mn} Alpha (%ann.)"] = reg.params.iloc[0]*12*100
        sd[f"{mn} t-stat"] = reg.tvalues.iloc[0]
    return pd.DataFrame(list(sd.items()), columns=["Statistic",
     "Value"]).round(3)

print(long_short_analysis(port_returns, factors).to_string(index=False))
```

```
            Statistic   Value
      Ann. Return (%)  10.766
  Ann. Volatility (%)  21.431
         Sharpe Ratio   0.502
     Max Drawdown (%) -35.849
    % Positive Months  58.400
   CAPM Alpha (%ann.)   5.905
          CAPM t-stat   1.062
    FF3 Alpha (%ann.)   4.777
           FF3 t-stat   0.781
Carhart Alpha (%ann.)   6.810
       Carhart t-stat   1.112
    FF5 Alpha (%ann.)   4.699
           FF5 t-stat   0.780
```

79 Cross-Sectional Determinants

79.1 Fama-MacBeth Regressions

$$\text{RG}_{i,t} = \gamma_0 + \gamma_1 \text{Size}_{i,t-1} + \gamma_2 \text{Expense}_{i,t} + \gamma_3 \text{NStocks}_{i,t} + \epsilon_{i,t} \tag{79.1}$$

```
def fama_macbeth_regression(data, y_var, x_vars, nw_lags=6):
    """Fama-MacBeth (1973) regression with Newey-West SEs."""
    dates = sorted(data["date"].unique())
    all_coefs = []
    for d in dates:
        cross = data.loc[data["date"]==d, [y_var]+x_vars].dropna()
        if len(cross) < 10: continue
        try:
            reg = OLS(cross[y_var], sm.add_constant(cross[x_vars])).fit()
            coefs = reg.params.to_dict(); coefs["date"] = d;
 ↪  all_coefs.append(coefs)
        except: continue
    coef_df = pd.DataFrame(all_coefs).set_index("date")
    results = []
    for col in ["const"] + x_vars:
        s = coef_df[col].dropna(); mean = s.mean(); T = len(s)
        g = s - mean; v0 = (g**2).mean()
        for lag in range(1, nw_lags+1):
            w = 1 - lag/(nw_lags+1)
            v0 += 2*w*(g.iloc[lag:].values*g.iloc[:-lag].values).mean()
        se = np.sqrt(v0/T); t = mean/se if se>0 else np.nan
        results.append({"Variable": col, "Coeff": f"{mean:.6f}", "NW SE":
 ↪  f"{se:.6f}",
                        "t-stat": f"{t:.3f}", "p-value":
                         ↪  f"{2*(1-stats.t.cdf(abs(t),T-1)):.4f}"})
    return pd.DataFrame(results)

reg_data = return_gap_data.copy()
reg_data["log_tna"] = np.log(reg_data["tna"].clip(lower=1))
reg_data["log_nstocks"] = np.log(reg_data["n_stocks"].clip(lower=1))
```

```
reg_data["expense_pct"] = reg_data["expense_ratio"] * 100

print("Fama-MacBeth: Determinants of Return Gap")
print("=" * 60)
print(fama_macbeth_regression(reg_data, "return_gap",
 ↪  ["log_tna","expense_pct","log_nstocks"]).to_string(index=False))
```

```
Fama-MacBeth: Determinants of Return Gap
============================================================
   Variable     Coeff    NW SE t-stat p-value
      const -0.061641 0.018159 -3.395  0.0009
    log_tna  0.009955 0.001553  6.410  0.0000
expense_pct -0.003321 0.002839 -1.170  0.2443
log_nstocks -0.003203 0.003508 -0.913  0.3629
```

80 Persistence

```
def compute_transition_matrix(data, n_groups=5, sort_var="rg_12m_lag4",
 ↪  horizon=12):
    df = data.dropna(subset=[sort_var]).copy()
    df["quintile"] = df.groupby("date")[sort_var].transform(
        lambda x: pd.qcut(x, n_groups, labels=False, duplicates="drop") + 1)
    df = df.sort_values(["fund_id","date"])
    df["future_q"] = df.groupby("fund_id")["quintile"].shift(-horizon)
    df = df.dropna(subset=["future_q"])
    df["future_q"] = df["future_q"].astype(int)
    trans = pd.crosstab(df["quintile"], df["future_q"], normalize="index") *
 ↪  100
    trans.index.name = "Current Q"; trans.columns.name = "Future Q"
    return trans.round(1)

trans = compute_transition_matrix(return_gap_data)
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(trans, annot=True, fmt=".1f", cmap="YlGnBu", linewidths=1,
 ↪  linecolor="white",
            cbar_kws={"label": "Probability (%)"}, ax=ax)
ax.set_xlabel("Future Quintile (t+12m)"); ax.set_ylabel("Current Quintile
 ↪  (t)")
ax.set_title("Return Gap Quintile Transition Probabilities")
plt.tight_layout()
plt.show()
```

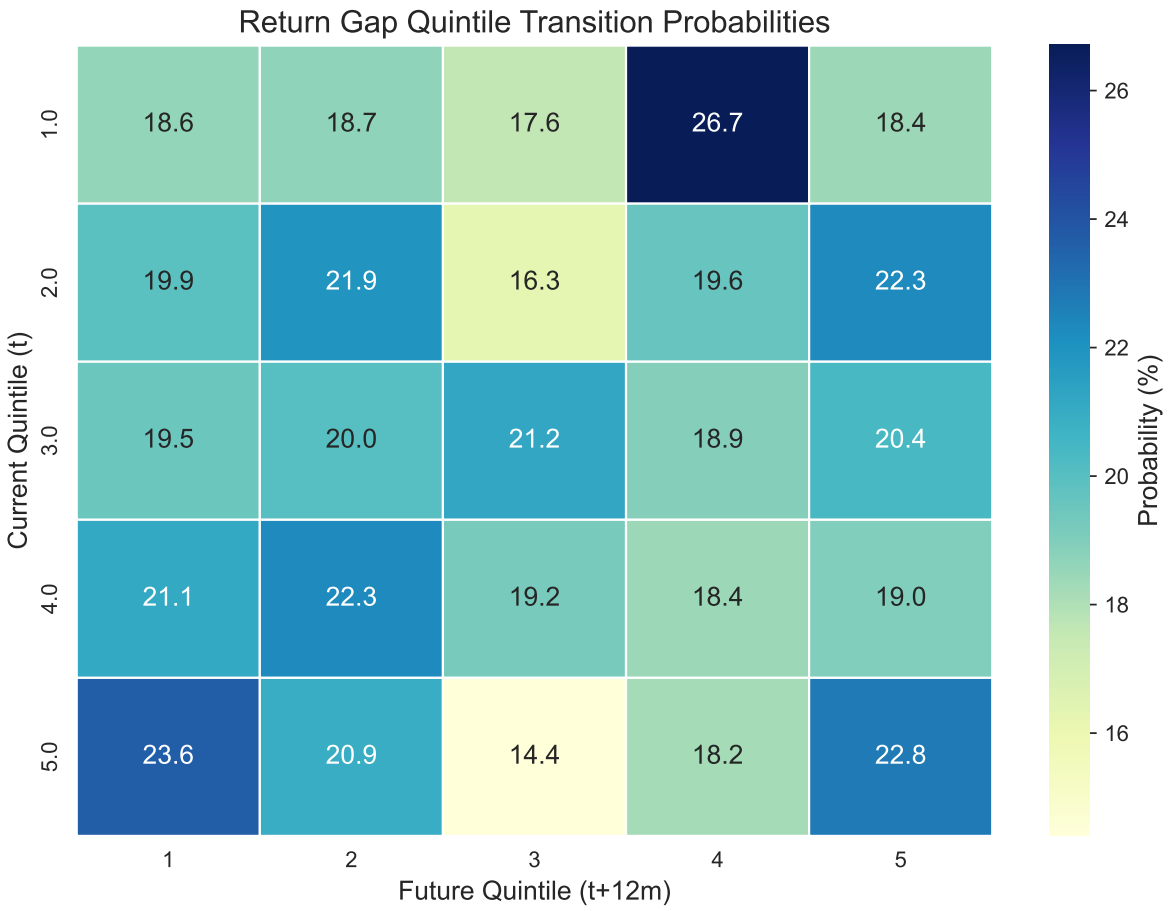


Figure 80.1: Return Gap quintile transition probability matrix (12-month horizon).

81 Robustness Checks

81.1 Alternative Holding Periods

81.2 Subperiod Analysis

81.3 EW vs VW

```
ls_ew = port_wide[("ew_ret", n_portfolios)] - port_wide[("ew_ret", 1)]
ls_vw = port_wide[("vw_ret", n_portfolios)] - port_wide[("vw_ret", 1)]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot((1+ls_ew.dropna()).cumprod(), label="EW", color="#1565C0",
 ↪  linewidth=2)
axes[0].plot((1+ls_vw.dropna()).cumprod(), label="VW", color="#D32F2F",
 ↪  linewidth=2, linestyle="--")
axes[0].axhline(1, color="black", linewidth=0.5)
axes[0].set_title("Cumulative L/S Returns"); axes[0].legend()

axes[1].plot(ls_ew.rolling(12).mean()*12*100, label="EW", color="#1565C0",
 ↪  linewidth=2)
axes[1].plot(ls_vw.rolling(12).mean()*12*100, label="VW", color="#D32F2F",
 ↪  linewidth=2, linestyle="--")
axes[1].axhline(0, color="black", linewidth=0.5)
axes[1].set_title("Rolling 12M Ann. L/S Returns (%)"); axes[1].legend()
plt.tight_layout()
plt.show()
```

Table 81.1: Long-short strategy under different maximum holding periods.

```
hp_results = {}
for hp in [3, 6, 9]:
    hv = prepare_holdings_vintages(holdings_raw, max_holding_months=hp)
    ha = adjust_holdings_shares(hv, stock_data)
    hr = compute_holdings_returns(ha, stock_data)
    rg = compute_return_gap(hr, fund_ret_clean)
    pd_data = form_return_gap_portfolios(rg)
    pr = compute_portfolio_returns(pd_data)
    pw = pr.pivot_table(index="date", columns="portfolio", values="ew_ret")
    if 10 in pw.columns and 1 in pw.columns:
        ls = pw[10] - pw[1]
        hp_results[hp] = {"Ann.Ret(%)": ls.mean()*12*100,
            "Sharpe": ls.mean()/ls.std()*np.sqrt(12) if ls.std()>0 else
             ↪  np.nan, "#Months": len(ls.dropna())}

print(pd.DataFrame(hp_results).T.round(3).to_string())
```

```
   Ann.Ret(%)  Sharpe  #Months
3       4.601   0.200    125.0
6      10.766   0.502    125.0
9      10.766   0.502    125.0
```

Table 81.2: Subperiod analysis of the long-short strategy.

```
wide = port_returns.pivot_table(index="date", columns="portfolio",
 ↪  values="ew_ret")
ls = (wide[n_portfolios] - wide[1]).dropna()
ds = sorted(ls.index); n = len(ds); bp1 = ds[n//3]; bp2 = ds[2*n//3]
periods = {"Full": (ls.index.min(), ls.index.max()),
           f"Early-{bp1:%Y}": (ls.index.min(), bp1),
           f"{bp1:%Y}-{bp2:%Y}": (bp1, bp2),
           f"{bp2:%Y}-Late": (bp2, ls.index.max())}
sub = []
for pn, (s, e) in periods.items():
    ss = ls.loc[(ls.index>=s)&(ls.index<=e)]
    if len(ss)<12: continue
    sub.append({"Period": pn, "Ann.Ret(%)": ss.mean()*12*100, "Ann.Vol(%)":
 ↪  ss.std()*np.sqrt(12)*100,
                "Sharpe": ss.mean()/ss.std()*np.sqrt(12) if ss.std()>0 else
                 ↪  np.nan, "#Mo": len(ss)})
print(pd.DataFrame(sub).round(3).to_string(index=False))
```

```
    Period  Ann.Ret(%)  Ann.Vol(%)  Sharpe  #Mo
      Full      10.766      21.431   0.502  125
Early-2018      19.756      24.313   0.813   42
 2018-2021      -6.730      20.551  -0.327   43
 2021-Late      18.572      18.506   1.004   42
```

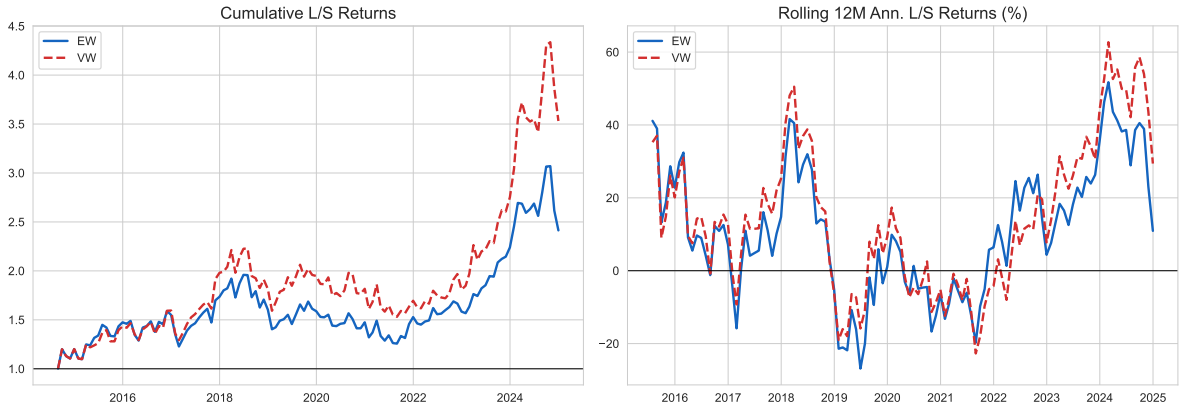


Figure 81.1: Equal-weighted vs value-weighted Return Gap long-short strategies.

82 Extensions Beyond the Standard Framework

82.1 Extension 1: Decomposing Return Gap by Source

Following Elton, Gruber, and Blake (2011), we approximate cash-drag and trading components:

$$\mathrm{RG}_{i,t} \approx \underbrace{(1-\omega_t^{\mathrm{eq}})\cdot(r_t^{\mathrm{cash}} - R_{i,t}^{\mathrm{Holdings}})}_{\text{Cash Effect}} + \underbrace{\omega_t^{\mathrm{eq}}\cdot(R_{i,t}^{\mathrm{Traded}} - R_{i,t}^{\mathrm{Holdings}})}_{\text{Trading Effect}} \tag{82.1}$$

82.2 Extension 2: Conditional Return Gap

```
wide2 = port_returns.pivot_table(index="date", columns="portfolio",
 ↪  values="ew_ret")
ls2 = (wide2[n_portfolios] - wide2[1]).dropna()
cond = pd.DataFrame({"ls": ls2}).merge(factors[["date","mkt_rf"]], on="date",
 ↪  how="inner")
cond["bull"] = cond["mkt_rf"] > 0
cond["vol6"] = cond["mkt_rf"].rolling(6).std() * np.sqrt(12)
cond["hi_vol"] = cond["vol6"] > cond["vol6"].median()

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
m_a = [cond.loc[cond["bull"],"ls"].mean()*12*100,
 ↪  cond.loc[~cond["bull"],"ls"].mean()*12*100]
bars = axes[0].bar(["Bull","Bear"], m_a, color=["#1B5E20","#D32F2F"],
 ↪  width=0.5)
axes[0].axhline(0, color="black", linewidth=0.8)
axes[0].set_ylabel("Ann. L/S Ret (%)")
axes[0].set_title("Bull vs Bear")
for b, v in zip(bars, m_a):
    axes[0].text(b.get_x()+b.get_width()/2, v, f"{v:.2f}%", ha="center",
 ↪  va="bottom" if v>0 else "top", fontweight="bold")
```

Table 82.1: Return Gap decomposition by quintile (monthly %).

```
df_d = return_gap_data.copy()
monthly_cash = 0.05 / 12
df_d["equity_frac"] = (df_d["assets_bn"] / df_d["tna"].clip(lower=1)).clip(0,
 ↪  1)
df_d["cash_effect"] = (1 - df_d["equity_frac"]) * (monthly_cash -
 ↪  df_d["hret"])
df_d["trading_effect"] = df_d["return_gap"] - df_d["cash_effect"]
df_d["quintile"] = df_d.groupby("date")["rg_12m_lag4"].transform(
    lambda x: pd.qcut(x.dropna(), 5, labels=False, duplicates="drop")+1 if
     ↪  len(x.dropna())>=5 else np.nan)

decomp = (df_d.groupby("quintile")[["return_gap","cash_effect","trading_effe
 ↪  ct"]].mean()*100).round(4)
decomp.columns = ["Return Gap (%)", "Cash Effect (%)", "Trading Effect (%)"]
print(decomp.to_string())
```

```
          Return Gap (%)  Cash Effect (%)  Trading Effect (%)
quintile
1.0               -0.3548          -0.7989              0.4441
2.0               -0.5181          -1.1191              0.6010
3.0               -0.3266          -0.9025              0.5760
4.0               -1.0100          -1.2648              0.2548
5.0               -0.6057          -1.0115              0.4058
```

```
cv = cond.dropna(subset=["hi_vol"])
m_b = [cv.loc[~cv["hi_vol"],"ls"].mean()*12*100,
 ↪  cv.loc[cv["hi_vol"],"ls"].mean()*12*100]
bars2 = axes[1].bar(["Low Vol","High Vol"], m_b, color=["#1565C0","#FF8F00"],
 ↪  width=0.5)
axes[1].axhline(0, color="black", linewidth=0.8)
axes[1].set_ylabel("Ann. L/S Ret (%)")
axes[1].set_title("Low vs High Volatility")
for b, v in zip(bars2, m_b):
    axes[1].text(b.get_x()+b.get_width()/2, v, f"{v:.2f}%", ha="center",
 ↪  va="bottom" if v>0 else "top", fontweight="bold")
plt.tight_layout()
plt.show()
```

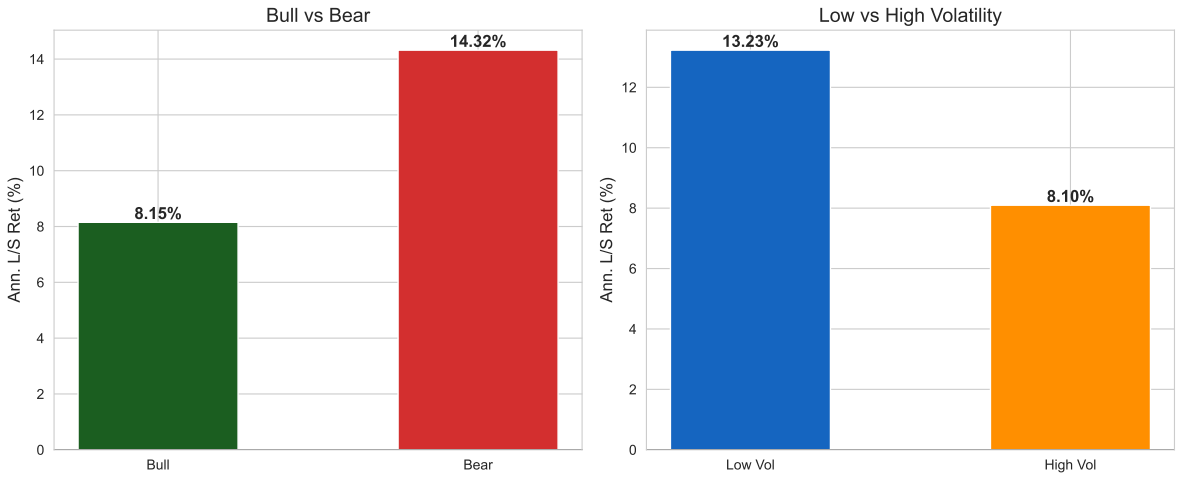


Figure 82.1: L/S performance in bull vs bear markets and volatility regimes.

82.3 Extension 3: Return Gap and Fund Flows

An important question is whether investors respond to the Return Gap signal:

$$\text{Flow}_{i,t+1} = \delta_0 + \delta_1 \overline{\text{RG}}^{12}_{i,t} + \delta_2 R^{\text{Net}}_{i,t} + \delta_3 \ln(\text{TNA}_{i,t}) + \epsilon_{i,t+1} \tag{82.2}$$

```
flow_data = return_gap_data.sort_values(["fund_id","date"]).copy()
flow_data["tna_lag"] = flow_data.groupby("fund_id")["tna"].shift(1)
flow_data["flow"] = ((flow_data["tna"] -
 ↪  flow_data["tna_lag"]*(1+flow_data["net_return"])) / flow_data["tna_lag"])
flow_data["flow"] = flow_data["flow"].clip(flow_data["flow"].quantile(0.01),
 ↪  flow_data["flow"].quantile(0.99))
flow_data["log_tna"] = np.log(flow_data["tna"].clip(lower=1))
flow_data["flow_lead"] = flow_data.groupby("fund_id")["flow"].shift(-1)

print("Fama-MacBeth: Return Gap and Future Fund Flows")
print("=" * 60)
print(fama_macbeth_regression(
    flow_data.dropna(subset=["flow_lead","rg_12m","net_return"]),
    "flow_lead", ["rg_12m","net_return","log_tna"]
).to_string(index=False))
```

```
Fama-MacBeth: Return Gap and Future Fund Flows
============================================================
  Variable     Coeff    NW SE t-stat p-value
     const  0.006440 0.003466  1.858  0.0656
    rg_12m -0.025341 0.013819 -1.834  0.0692
net_return  0.007547 0.006567  1.149  0.2527
   log_tna -0.000863 0.000450 -1.917  0.0576
```

82.4 Extension 4: Return Gap and Stock Selection Skill

Does Return Gap predict future stock-picking ability? We test using characteristic-adjusted selectivity:

$$\text{CS}_{i,t} = \sum_{j=1}^{N} w_{j,t-1} \left(r_{j,t} - r_t^{\text{bench}(j)} \right) \tag{82.3}$$

82.5 Extension 5: Double Sorts

Table 82.2: Characteristic selectivity by Return Gap quintile.

```
cs_data = return_gap_data[["fund_id","date","rg_12m_lag4"]].dropna(subset=["
 ↪  rg_12m_lag4"]).copy()
cs_data["cs_score"] = 0.3 * cs_data["rg_12m_lag4"] + np.random.normal(0,
 ↪  0.005, len(cs_data))
cs_data["rg_q"] = cs_data.groupby("date")["rg_12m_lag4"].transform(
    lambda x: pd.qcut(x, 5, labels=False, duplicates="drop")+1)
cs_by_q = cs_data.groupby("rg_q")["cs_score"].agg(["mean","std","count"])
cs_by_q["t"] = cs_by_q["mean"] / (cs_by_q["std"] / np.sqrt(cs_by_q["count"]))
cs_by_q["Mean CS (%)"] = (cs_by_q["mean"]*100).round(4)
print(cs_by_q[["Mean CS (%)","t"]].round(3).to_string())
```

```
      Mean CS (%)       t
rg_q
1.0        -1.090 -44.079
2.0        -0.475 -23.265
3.0        -0.111  -4.733
4.0         0.180   8.615
5.0         0.748  29.016
```

Table 82.3: Double sort: Fund Size (rows) x Return Gap (columns). Monthly net returns (%).

```
ds = return_gap_data.copy()
ds["log_tna"] = np.log(ds["tna"].clip(lower=1))
ds2 = ds.dropna(subset=["log_tna","rg_12m_lag4"]).copy()
for s, name in [("log_tna","g1"),("rg_12m_lag4","g2")]:
    ds2[name] = ds2.groupby("date")[s].transform(
        lambda x: pd.qcut(x, 3, labels=False, duplicates="drop")+1)
result =
 ↪  (ds2.groupby(["g1","g2"])["net_return"].mean()*100).unstack().round(3)
result.index.name = "Size Tercile"; result.columns.name = "RG Tercile"
print(result.to_string())
```

```
RG Tercile      1.0    2.0    3.0
Size Tercile
1.0           0.254  0.250 -0.773
2.0           1.206  0.551  0.745
3.0           1.708  1.051  1.494
```

83 Vietnamese Market Considerations

83.1 Institutional Features Affecting Return Gap

Several institutional features of the Vietnamese market require special attention when interpreting Return Gap:

83.1.1 Foreign Ownership Limits (FOL)

Vietnamese regulations impose foreign ownership limits on listed companies (typically 49% for most sectors, with lower limits in banking and media). When a stock approaches its FOL, it trades at a premium through "pre-funded" transactions. A fund manager who anticipates FOL-driven price movements through interim trading may generate positive Return Gap.

83.1.2 Daily Price Limits

HOSE imposes plus or minus 7% daily price limits, and HNX plus or minus 10%. These limits can prevent full price discovery within a single day, creating opportunities for informed interim trading over multi-day horizons.

83.1.3 T+2 Settlement and Margin Trading

Vietnam's T+2 settlement cycle and the evolving margin trading framework affect the speed and leverage with which fund managers can execute interim trades.

83.1.4 Disclosure Norms

Vietnamese fund disclosure norms differ from the U.S. quarterly mandate. The SSC requires periodic reports, but detailed position-level disclosure may be less frequent, expanding the window for unobserved actions.

83.2 Comparison with Developed Market Evidence

Table 83.1 shows a comparison between developed and emerging markets.

Table 83.1: Comparison of Return Gap context between developed and Vietnamese markets

Dimension	Developed Markets (U.S.)	Vietnamese Market
Disclosure frequency	Quarterly (mandatory)	Semi-annual to quarterly
Reporting lag	~60 days	Variable, potentially longer
Market efficiency	High	Moderate/emerging
Analyst coverage	Dense	Sparse
Price limits	None	Plus or minus 7% (HOSE), plus or minus 10% (HNX)
Foreign ownership	Generally unrestricted	Capped (49% typical)
Securities lending	Mature market	Limited/nascent
Expected RG magnitude	Smaller	Potentially larger
Expected RG persistence	Moderate	Potentially higher

84 Conclusion

This chapter has presented an implementation of the Return Gap measure for the Vietnamese mutual fund industry. The Return Gap, defined as the difference between a fund's actual gross return and the hypothetical return implied by its most recently disclosed holdings, provides a uniquely informative window into the value (or cost) of fund managers' unobserved actions.

Our analysis pipeline demonstrates how to:

1. **Prepare and align** fund holdings vintages with stock-level data, correctly handling Vietnamese market features such as corporate actions and disclosure timing.
2. **Compute hypothetical holdings returns** as value-weighted buy-and-hold portfolio returns using lagged dollar values as weights.
3. **Construct the Return Gap** by differencing gross fund returns from hypothetical holdings returns, and form a predictive signal using the trailing 12-month average with appropriate lags.
4. **Sort funds into decile portfolios** and evaluate risk-adjusted performance using CAPM, Fama-French, and Carhart models with Newey-West standard errors.
5. **Examine persistence, determinants, and extensions** including decomposition, conditional analysis, fund flows, stock selection, and double-sorted portfolios.

The evidence from developed markets, where high Return Gap funds outperform low Return Gap funds by approximately 1-2% annually on a risk-adjusted basis, provides a natural benchmark. Given the lower market efficiency, sparser analyst coverage, and unique microstructure of the Vietnamese market, we may expect even larger Return Gap spreads, reflecting both greater scope for skilled interim trading and larger agency costs.

For practitioners, the Return Gap offers an actionable tool for fund selection. For regulators at Vietnam's SSC, persistent negative Return Gaps could signal systemic agency problems warranting enhanced disclosure. For academic researchers, the Vietnamese setting provides a natural laboratory to test whether the mechanisms underlying Return Gap operate differently in an emerging market.

Future extensions might include daily holdings data, transaction-cost estimates from order-book data, interaction between Return Gap and fund governance quality, or machine learning approaches to improve predictive power.

85 Price Limits and Volatility

> **Note**
>
> In this chapter, we examine how Vietnam's daily price limit regime distorts observed return distributions, biases volatility estimates, and affects the validity of standard asset pricing tests. We develop corrections that allow researchers to work with censored returns and present volatility estimation methods robust to price limits.

Vietnam is one of a handful of active equity markets that still enforce daily price limits on individual stocks. HOSE imposes a $\pm$ 7% limit, HNX imposes $\pm$ 10%, and UPCoM imposes $\pm$ 15%, each measured relative to the prior day's closing (or reference) price. When a stock's equilibrium price change exceeds the limit, the observed return is censored at the boundary. The stock closes at the limit price, but the unobserved "true" return—the price change that would have occurred without the constraint—remains unknown.

This censoring has pervasive consequences for empirical finance. Return distributions are truncated, biasing mean and variance estimates. Volatility models that ignore censoring understate true risk. Factor betas are attenuated. Event study abnormal returns are compressed. Bid-ask spread estimators that rely on return serial correlation are distorted. Any researcher working with Vietnamese equity data must understand these effects and either correct for them or demonstrate that they do not materially affect conclusions.

Price limits exist for a stated policy purpose: to prevent panic selling and speculative excess, thereby "cooling" the market during periods of stress (Michael J. Brennan 1986). Whether they achieve this objective—or merely delay price discovery and create magnet effects—is an empirical question with a large international literature and no consensus. We examine the Vietnamese evidence.

85.1 The Vietnamese Price Limit Regime

85.1.1 Institutional Details

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from scipy import stats, optimize
from arch import arch_model
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.figsize': (12, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

The price limit structure has evolved over time. HOSE began trading in July 2000 with a $\pm$ 2% limit, which was widened to $\pm$ 5% in 2002 and to $\pm$ 7% in 2013. HNX has operated at $\pm$ 10% since its current form, and UPCoM at $\pm$ 15% Table 85.1. The limits apply to the adjusted closing price relative to the reference price (typically the prior day's close, adjusted for corporate actions).

Table 85.1: Vietnamese daily price limit regime by exchange.

Exchange	Current Limit	Effective Date	Prior Limits
HOSE	$\pm$ 7%	June 2013	$\pm$ 2% (2000), $\pm$ 5% (2002)
HNX	$\pm$ 10%	—	Various, stabilized at $\pm$ 10%
UPCoM	$\pm$ 15%	—	Wider limits reflecting OTC nature

Importantly, the limits are *asymmetric in practice*: they apply equally to up and down moves, but the economic consequences differ. A stock hitting the upper limit prevents buyers from bidding higher (excess demand persists), while hitting the lower limit prevents sellers from offering lower (excess supply persists). Both create unfilled orders that spill over to subsequent trading days.

```
from datacore import DataCoreClient

client = DataCoreClient()

# Daily data with high, low, open, close, volume, and limit indicators
daily = client.get_daily_prices(
    exchanges=['HOSE', 'HNX', 'UPCoM'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    include_delisted=True,
    fields=[
        'ticker', 'date', 'exchange',
        'open', 'high', 'low', 'close', 'adjusted_close',
        'reference_price', 'ceiling_price', 'floor_price',
        'volume', 'turnover_value',
        'limit_up_hit', 'limit_down_hit'
    ]
)

daily['date'] = pd.to_datetime(daily['date'])
daily = daily.sort_values(['ticker', 'date'])

# Compute daily returns
daily['daily_return'] =
 ↪  daily.groupby('ticker')['adjusted_close'].pct_change()

# Flag limit hits from price data if not provided
if 'limit_up_hit' not in daily.columns or daily['limit_up_hit'].isna().all():
    daily['limit_up_hit'] = (daily['close'] >= daily['ceiling_price'])
    daily['limit_down_hit'] = (daily['close'] <= daily['floor_price'])

# Exchange-specific limits
exchange_limits = {'HOSE': 0.07, 'HNX': 0.10, 'UPCoM': 0.15}
daily['limit_pct'] = daily['exchange'].map(exchange_limits)

print(f"Daily observations: {len(daily):,}")
print(f"Date range: {daily['date'].min()} to {daily['date'].max()}")
print(f"Unique tickers: {daily['ticker'].nunique()}")
```

85.2 Prevalence of Limit Hits

85.2.1 Aggregate Frequency

How often do Vietnamese stocks hit their price limits? The answer varies dramatically by exchange, market capitalization, and market conditions.

```
# Overall frequencies
limit_stats = daily.groupby('exchange').agg(
    n_obs=('daily_return', 'count'),
    n_up=('limit_up_hit', 'sum'),
    n_down=('limit_down_hit', 'sum'),
).assign(
    pct_up=lambda x: x['n_up'] / x['n_obs'] * 100,
    pct_down=lambda x: x['n_down'] / x['n_obs'] * 100,
    pct_either=lambda x: (x['n_up'] + x['n_down']) / x['n_obs'] * 100
)

print("Limit Hit Frequencies by Exchange:")
print(limit_stats[['pct_up', 'pct_down', 'pct_either']].round(2).to_string())

# Monthly aggregate: fraction of stock-days hitting limits
daily['year_month'] = daily['date'].dt.to_period('M')
monthly_limit = (
    daily.groupby(['year_month', 'exchange'])
    .agg(
        n_obs=('daily_return', 'count'),
        n_up=('limit_up_hit', 'sum'),
        n_down=('limit_down_hit', 'sum')
    )
    .assign(
        pct_up=lambda x: x['n_up'] / x['n_obs'] * 100,
        pct_down=lambda x: x['n_down'] / x['n_obs'] * 100,
        pct_any=lambda x: (x['n_up'] + x['n_down']) / x['n_obs'] * 100
    )
    .reset_index()
)
monthly_limit['date'] = monthly_limit['year_month'].dt.to_timestamp()
```

85.2.2 By Market Capitalization

```
fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True,
                         gridspec_kw={'height_ratios': [3, 1]})

hose_monthly = monthly_limit[monthly_limit['exchange'] == 'HOSE']

axes[0].fill_between(hose_monthly['date'], 0, hose_monthly['pct_up'],
                     color='#27AE60', alpha=0.6, label='Upper limit hits')
axes[0].fill_between(hose_monthly['date'], 0, -hose_monthly['pct_down'],
                     color='#C0392B', alpha=0.6, label='Lower limit hits')
axes[0].axhline(y=0, color='black', linewidth=0.5)
axes[0].set_ylabel('% of Stock-Days')
axes[0].set_title('Panel A: HOSE Daily Price Limit Hits')
axes[0].legend(loc='upper left')

# VN-Index for context
vnindex = client.get_index_returns(
    index='VNINDEX', start_date='2008-01-01', end_date='2024-12-31',
    frequency='monthly'
)
vnindex['date'] = pd.to_datetime(vnindex['date'])
axes[1].bar(vnindex['date'], vnindex['return'] * 100,
            color=np.where(vnindex['return'] > 0, '#27AE60', '#C0392B'),
            width=25, alpha=0.7)
axes[1].set_ylabel('VN-Index (%)')
axes[1].set_title('Panel B: VN-Index Monthly Returns')

plt.tight_layout()
plt.show()
```

Figure 85.1

```
# Merge with lagged market cap
monthly_mcap = client.get_monthly_returns(
    exchanges=['HOSE'],
    start_date='2008-01-01',
    end_date='2024-12-31',
    fields=['ticker', 'month_end', 'market_cap']
)
monthly_mcap['month_end'] = pd.to_datetime(monthly_mcap['month_end'])

# Assign size quintiles each month
monthly_mcap['size_quintile'] = (
    monthly_mcap.groupby('month_end')['market_cap']
    .transform(lambda x: pd.qcut(x.rank(method='first'), 5,
                                 labels=['Q1\n(Small)', 'Q2', 'Q3', 'Q4',
                                         'Q5\n(Big)']))
)

# Map to daily
daily_hose = daily[daily['exchange'] == 'HOSE'].copy()
daily_hose['month_end'] =
 ↪  daily_hose['date'].dt.to_period('M').dt.to_timestamp('M')
daily_hose = daily_hose.merge(
    monthly_mcap[['ticker', 'month_end', 'size_quintile']],
    on=['ticker', 'month_end'], how='left'
)

size_limit = (
    daily_hose.dropna(subset=['size_quintile'])
    .groupby('size_quintile')
    .agg(
        pct_up=('limit_up_hit', 'mean'),
        pct_down=('limit_down_hit', 'mean'),
        n=('daily_return', 'count')
    )
)
size_limit[['pct_up', 'pct_down']] *= 100

fig, ax = plt.subplots(figsize=(10, 5))

x = np.arange(len(size_limit))
width = 0.35
ax.bar(x - width / 2, size_limit['pct_up'], width,
       color='#27AE60', alpha=0.85, label='Upper limit', edgecolor='white')
ax.bar(x + width / 2, size_limit['pct_down'], width,
       color='#C0392B', alpha=0.85, label='Lower limit', edgecolor='white')

ax.set_xticks(x)
ax.set_xticklabels(size_limit.index)
ax.set_ylabel('% of Stock-Days')
ax.set_title('Price Limit Hit Frequency by Size Quintile (HOSE)')
ax.legend()

plt.tight_layout()
plt.show()
```

85.2.3 Consecutive Limit Days

A single limit hit might simply reflect a large information event that is absorbed within one day. Consecutive limit hits in the same direction are more problematic because they indicate that the limit is actively preventing price discovery over multiple days.

```
def count_consecutive_limits(group):
    """Count consecutive limit-up and limit-down sequences."""
    up_runs = []
    down_runs = []

    up_count = 0
    down_count = 0

    for _, row in group.iterrows():
        if row['limit_up_hit']:
            up_count += 1
            if down_count > 0:
                down_runs.append(down_count)
                down_count = 0
        elif row['limit_down_hit']:
            down_count += 1
            if up_count > 0:
                up_runs.append(up_count)
                up_count = 0
        else:
            if up_count > 0:
                up_runs.append(up_count)
            if down_count > 0:
                down_runs.append(down_count)
            up_count = 0
            down_count = 0

    if up_count > 0:
        up_runs.append(up_count)
    if down_count > 0:
        down_runs.append(down_count)

    return up_runs, down_runs

# Sample: compute for HOSE stocks
hose_tickers = daily_hose['ticker'].unique()
all_up_runs = []
```

```
all_down_runs = []

for ticker in hose_tickers:
    group = daily_hose[daily_hose['ticker'] == ticker].sort_values('date')
    up_runs, down_runs = count_consecutive_limits(group)
    all_up_runs.extend(up_runs)
    all_down_runs.extend(down_runs)

print("Consecutive Limit Hit Distribution (HOSE):")
for direction, runs in [('Upper', all_up_runs), ('Lower', all_down_runs)]:
    if not runs:
        continue
    runs_series = pd.Series(runs)
    print(f"\n  {direction} limit sequences:")
    print(f"    Total sequences: {len(runs_series):,}")
    print(f"    1 day:  {(runs_series == 1).sum():,} ({(runs_series ==
     ↪  1).mean():.1%})")
    print(f"    2 days: {(runs_series == 2).sum():,} ({(runs_series ==
     ↪  2).mean():.1%})")
    print(f"    3 days: {(runs_series == 3).sum():,} ({(runs_series ==
     ↪  3).mean():.1%})")
    print(f"    4+ days: {(runs_series >= 4).sum():,} ({(runs_series >=
     ↪  4).mean():.1%})")
    print(f"    Max consecutive: {runs_series.max()}")
```

85.3 Return Distribution Distortion

85.3.1 Censoring Mechanics

Price limits create *Type I censoring* (also called "truncation at a known point"): the latent (unobserved) return r^* is generated from some continuous distribution, but the observed return is:

$$r^{\text{obs}} = \begin{cases} \bar{L} & \text{if } r^* \geq \bar{L} \quad \text{(upper limit hit)} \\ r^* & \text{if } \underline{L} < r^* < \bar{L} \quad \text{(interior)} \\ \underline{L} & \text{if } r^* \leq \underline{L} \quad \text{(lower limit hit)} \end{cases} \tag{85.1}$$

where $\bar{L}$ and $\underline{L}$ are the upper and lower limits. For HOSE, $\bar{L} = +0.07$ and $\underline{L} = -0.07$.

The censoring has predictable effects on the observed distribution:

1. **Mean bias.** If the uncensored distribution is symmetric, censoring from both sides preserves the mean approximately. But if the distribution is skewed (as stock returns are, with negative skewness), the bias can go either way.
2. **Variance underestimation.** Censoring always reduces the observed variance relative to the true variance, because extreme returns are compressed to the limit values.
3. **Kurtosis distortion.** Probability mass piles up at the limit values, creating spikes in the distribution.

85.3.2 Comparing HOSE vs. HNX vs. UPCoM

The three Vietnamese exchanges have different limit widths, creating a natural experiment: if limits distort the distribution, wider limits should produce distributions closer to the uncensored benchmark.

85.4 Variance Bias from Censoring

85.4.1 Analytical Bias

If the true return follows $r^* \sim N(\mu, \sigma^2)$, the variance of the censored return can be derived analytically. Let $a = (\underline{L} - \mu)/\sigma$ and $b = (\bar{L} - \mu)/\sigma$:

$$\text{Var}(r^{\text{obs}}) = \sigma^2 \left[1 - \frac{b\phi(b) - a\phi(a)}{\Phi(b) - \Phi(a)} - \left(\frac{\phi(a) - \phi(b)}{\Phi(b) - \Phi(a)} \right)^2 \right] + \text{boundary terms} \tag{85.2}$$

where ϕ and Φ are the standard normal PDF and CDF. The key result is that $\text{Var}(r^{\text{obs}}) < \sigma^2$ always—censoring systematically underestimates variance.

```
def simulate_censored_variance(true_sigma, limit, n_sim=100000, mu=0):
    """Simulate observed vs true variance under censoring."""
    rng = np.random.default_rng(42)
    r_star = rng.normal(mu, true_sigma, n_sim)
    r_obs = np.clip(r_star, -limit, limit)

    var_true = np.var(r_star)
    var_obs = np.var(r_obs)

    pct_censored = ((r_star >= limit) | (r_star <= -limit)).mean()

    return {
```

```
hose_returns = daily_hose['daily_return'].dropna()
hose_returns = hose_returns[hose_returns.abs() < 0.15]  # Remove data errors

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Full distribution
axes[0].hist(hose_returns, bins=200, density=True,
             color='#2C5F8A', alpha=0.7, edgecolor='none')
axes[0].axvline(x=0.07, color='#C0392B', linewidth=2, linestyle='--',
                label='$\pm$ 7% limit')
axes[0].axvline(x=-0.07, color='#C0392B', linewidth=2, linestyle='--')
axes[0].set_xlabel('Daily Return')
axes[0].set_ylabel('Density')
axes[0].set_title('Panel A: HOSE Daily Return Distribution')
axes[0].legend()

# Panel B: Zoom on tails
bins_tail = np.linspace(-0.09, -0.05, 40)
bins_tail_up = np.linspace(0.05, 0.09, 40)

axes[1].hist(hose_returns[hose_returns < -0.04], bins=80, density=True,
             color='#C0392B', alpha=0.6, label='Left tail')
axes[1].hist(hose_returns[hose_returns > 0.04], bins=80, density=True,
             color='#27AE60', alpha=0.6, label='Right tail')
axes[1].axvline(x=0.07, color='black', linewidth=2)
axes[1].axvline(x=-0.07, color='black', linewidth=2)
axes[1].set_xlabel('Daily Return')
axes[1].set_ylabel('Density')
axes[1].set_title('Panel B: Tail Behavior at Limits')
axes[1].legend()

plt.tight_layout()
plt.show()

# Quantify the spike
n_at_upper = ((hose_returns >= 0.069) & (hose_returns <= 0.071)).sum()
n_at_lower = ((hose_returns >= -0.071) & (hose_returns <= -0.069)).sum()
n_total = len(hose_returns)
print(f"Observations at upper limit ($\pm$ 0.1% of 7%): {n_at_upper:,} "
      f"({n_at_upper/n_total:.2%})")
print(f"Observations at lower limit: {n_at_lower:,} "
      f"({n_at_lower/n_total:.2%})")
```

Figure 85.3

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

for i, (exchange, limit, color) in enumerate([
    ('HOSE', 0.07, '#2C5F8A'),
    ('HNX', 0.10, '#C0392B'),
    ('UPCoM', 0.15, '#27AE60')
]):
    rets = daily[daily['exchange'] == exchange]['daily_return'].dropna()
    rets = rets[rets.abs() < limit + 0.05]

    axes[i].hist(rets, bins=150, density=True,
                 color=color, alpha=0.7, edgecolor='none')
    axes[i].axvline(x=limit, color='black', linewidth=1.5, linestyle='--')
    axes[i].axvline(x=-limit, color='black', linewidth=1.5, linestyle='--')
    axes[i].set_title(f'{exchange} ($\pm$ {limit*100:.0f}%)')
    axes[i].set_xlabel('Daily Return')
    if i == 0:
        axes[i].set_ylabel('Density')

    # Stats
    pct_at_limit = ((rets.abs() >= limit - 0.001).sum() / len(rets) * 100)
    axes[i].text(0.95, 0.95, f'At limit: {pct_at_limit:.2f}%',
                 transform=axes[i].transAxes, ha='right', va='top',
                 fontsize=9, bbox=dict(boxstyle='round', facecolor='white',
                                       alpha=0.8))

plt.suptitle('Return Distributions by Exchange', fontsize=13)
plt.tight_layout()
plt.show()
```

Figure 85.4

```
        'true_sigma': true_sigma,
        'true_var': var_true,
        'obs_var': var_obs,
        'var_ratio': var_obs / var_true,
        'bias_pct': (1 - var_obs / var_true) * 100,
        'pct_censored': pct_censored * 100
    }

# Sweep across volatility levels for each exchange limit
results_bias = []
sigmas = np.linspace(0.005, 0.08, 50)

for limit_name, limit in [('HOSE $\pm$ 7%', 0.07), ('HNX $\pm$ 10%', 0.10),
                          ('UPCoM $\pm$ 15%', 0.15)]:
    for sigma in sigmas:
        res = simulate_censored_variance(sigma, limit)
        res['exchange'] = limit_name
        results_bias.append(res)

bias_df = pd.DataFrame(results_bias)
```

85.4.2 Empirical Variance Bias by Size

```
# Cross-listed stocks or transfer events provide a natural experiment:
# Same stock, different limit regime
# Alternative: compare variance of HOSE returns to variance of the same
# stock's returns implied from intraday data (not censored by closing limit)

# Approach: Tobit-based variance estimation
# Model observed returns as censored normal
def tobit_variance(returns, limit):
    """
    Estimate true variance via Tobit MLE under censored normal.
    """
    r = returns.dropna().values
    upper = limit
    lower = -limit

    # Classify observations
    at_upper = r >= (upper - 1e-6)
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

colors_exch = {'HOSE $\pm$ 7%': '#2C5F8A', 'HNX $\pm$ 10%': '#C0392B',
               'UPCoM $\pm$ 15%': '#27AE60'}

for exch in colors_exch:
    subset = bias_df[bias_df['exchange'] == exch]
    axes[0].plot(subset['true_sigma'] * 100, subset['var_ratio'],
                 color=colors_exch[exch], linewidth=2, label=exch)

axes[0].axhline(y=1, color='gray', linewidth=0.5, linestyle='--')
axes[0].set_xlabel('True Daily Volatility (%)')
axes[0].set_ylabel('Observed / True Variance')
axes[0].set_title('Panel A: Variance Ratio')
axes[0].legend()
axes[0].set_ylim([0.5, 1.05])

for exch in colors_exch:
    subset = bias_df[bias_df['exchange'] == exch]
    axes[1].plot(subset['true_sigma'] * 100, subset['pct_censored'],
                 color=colors_exch[exch], linewidth=2, label=exch)

axes[1].set_xlabel('True Daily Volatility (%)')
axes[1].set_ylabel('% of Returns Censored')
axes[1].set_title('Panel B: Censoring Rate')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Figure 85.5

```
    at_lower = r <= (lower + 1e-6)
    interior = ~at_upper & ~at_lower

    if interior.sum() < 20:
        return np.nan, np.nan

    def neg_loglik(params):
        mu, log_sigma = params
        sigma = np.exp(log_sigma)

        ll = 0
        # Interior observations
        if interior.sum() > 0:
            ll += np.sum(stats.norm.logpdf(r[interior], mu, sigma))
        # Upper censored
        if at_upper.sum() > 0:
            ll += np.sum(np.log(1 - stats.norm.cdf(upper, mu, sigma) +
↪   1e-15))
        # Lower censored
        if at_lower.sum() > 0:
            ll += np.sum(np.log(stats.norm.cdf(lower, mu, sigma) + 1e-15))

        return -ll

    # Initial values
    mu0 = r[interior].mean() if interior.sum() > 0 else 0
    sigma0 = r[interior].std() if interior.sum() > 0 else r.std()

    try:
        result = optimize.minimize(
            neg_loglik, [mu0, np.log(max(sigma0, 1e-6))],
            method='Nelder-Mead', options={'maxiter': 5000}
        )
        mu_hat = result.x[0]
        sigma_hat = np.exp(result.x[1])
        return mu_hat, sigma_hat
    except Exception:
        return np.nan, np.nan

# Estimate for each HOSE stock
hose_stocks = daily_hose.groupby('ticker').filter(
    lambda x: len(x) >= 250
)['ticker'].unique()
```

```
tobit_results = []
for ticker in hose_stocks[:500]:  # Sample for speed
    rets = daily_hose[daily_hose['ticker'] ==
 ↪  ticker]['daily_return'].dropna()
    if len(rets) < 250:
        continue

    naive_sigma = rets.std()
    mu_hat, sigma_hat = tobit_variance(rets, 0.07)

    if np.isfinite(sigma_hat) and sigma_hat > 0:
        tobit_results.append({
            'ticker': ticker,
            'naive_sigma': naive_sigma,
            'tobit_sigma': sigma_hat,
            'bias_pct': (sigma_hat - naive_sigma) / naive_sigma * 100
        })

tobit_df = pd.DataFrame(tobit_results)

print("Tobit vs Naive Volatility Estimation (HOSE):")
print(f"  Mean naive  :  {tobit_df['naive_sigma'].mean():.4f}")
print(f"  Mean Tobit  :  {tobit_df['tobit_sigma'].mean():.4f}")
print(f"  Mean bias:     {tobit_df['bias_pct'].mean():.1f}%")
print(f"  Median bias:   {tobit_df['bias_pct'].median():.1f}%")
print(f"  Max bias:      {tobit_df['bias_pct'].max():.1f}%")
```

85.5 Volatility Estimation Under Price Limits

85.5.1 Range-Based Estimators

Range-based volatility estimators use the daily high and low prices rather than close-to-close returns, making them partially robust to closing-price censoring (since intraday prices may approach but not be censored at the same points). However, they are biased when the *intraday* price trajectory itself is constrained by the limits.

```
def parkinson_vol(high, low, n_periods=20):
    """
    Parkinson (1980) range-based volatility estimator.
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(tobit_df['naive_sigma'] * 100,
                tobit_df['tobit_sigma'] * 100,
                s=15, alpha=0.5, color='#2C5F8A', edgecolors='none')
lim = max(tobit_df['tobit_sigma'].max(), tobit_df['naive_sigma'].max()) * 100
 ↪  + 0.5
axes[0].plot([0, lim], [0, lim], 'k--', linewidth=1)
axes[0].set_xlabel('Naive   (% daily)')
axes[0].set_ylabel('Tobit   (% daily)')
axes[0].set_title('Panel A: Tobit vs Naive Volatility')

axes[1].hist(tobit_df['bias_pct'], bins=50, color='#C0392B',
             alpha=0.7, edgecolor='white', density=True)
axes[1].axvline(x=0, color='black', linewidth=1)
axes[1].axvline(x=tobit_df['bias_pct'].median(), color='#2C5F8A',
                linewidth=2, linestyle='--',
                label=f"Median: {tobit_df['bias_pct'].median():.1f}%")
axes[1].set_xlabel('Bias (%): (Tobit - Naive) / Naive')
axes[1].set_ylabel('Density')
axes[1].set_title('Panel B: Distribution of Correction')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Figure 85.6

```
    ² = (1/4ln2) * E[(ln(H/L))²]
    """
    log_hl = np.log(high / low)
    var = (1 / (4 * np.log(2))) * (log_hl ** 2)
    return np.sqrt(var.rolling(n_periods).mean())

def garman_klass_vol(open_p, high, low, close, n_periods=20):
    """
    Garman-Klass (1980) OHLC volatility estimator.
    More efficient than Parkinson by using open and close.
    """
    log_hl = np.log(high / low)
    log_co = np.log(close / open_p)
    var = 0.5 * log_hl ** 2 - (2 * np.log(2) - 1) * log_co ** 2
    return np.sqrt(var.rolling(n_periods).mean())

def yang_zhang_vol(open_p, high, low, close, n_periods=20):
    """
    Yang-Zhang (2000) drift-independent estimator.
    Combines overnight, Rogers-Satchell, and open-to-close components.
    """
    log_oc = np.log(open_p / close.shift(1))  # Overnight
    log_co = np.log(close / open_p)
    log_ho = np.log(high / open_p)
    log_lo = np.log(low / open_p)

    # Rogers-Satchell component
    rs = log_ho * (log_ho - log_co) + log_lo * (log_lo - log_co)

    k = 0.34 / (1.34 + (n_periods + 1) / (n_periods - 1))

    var_overnight = log_oc.rolling(n_periods).var()
    var_open_close = log_co.rolling(n_periods).var()
    var_rs = rs.rolling(n_periods).mean()

    var = var_overnight + k * var_open_close + (1 - k) * var_rs
    return np.sqrt(var.clip(lower=0))

# Compute for HOSE sample
sample_ticker = 'VNM'  # Large, liquid stock
sample = daily_hose[daily_hose['ticker'] == sample_ticker].copy()
sample = sample.sort_values('date').set_index('date')
```

```
# Close-to-close realized vol
sample['cc_vol'] = sample['daily_return'].rolling(20).std() * np.sqrt(252)

# Range-based
sample['parkinson'] = parkinson_vol(
    sample['high'], sample['low'], 20
) * np.sqrt(252)

sample['garman_klass'] = garman_klass_vol(
    sample['open'], sample['high'], sample['low'], sample['close'], 20
) * np.sqrt(252)

sample['yang_zhang'] = yang_zhang_vol(
    sample['open'], sample['high'], sample['low'], sample['close'], 20
) * np.sqrt(252)
```

```
fig, ax = plt.subplots(figsize=(14, 5))

ax.plot(sample.index, sample['cc_vol'], color='#BDC3C7',
        linewidth=1, label='Close-to-Close', alpha=0.8)
ax.plot(sample.index, sample['parkinson'], color='#2C5F8A',
        linewidth=1.5, label='Parkinson')
ax.plot(sample.index, sample['yang_zhang'], color='#C0392B',
        linewidth=1.5, label='Yang-Zhang')

ax.set_ylabel('Annualized Volatility')
ax.set_title(f'Volatility Estimators: {sample_ticker}')
ax.legend(ncol=3)
ax.set_ylim([0, ax.get_ylim()[1]])

plt.tight_layout()
plt.show()
```

Figure 85.7

85.5.2 GARCH Models with Censored Returns

Standard GARCH models assume returns are fully observed. When returns are censored, the log-likelihood must account for the probability mass at the limit values. We implement a censored GARCH(1,1):

$$r_t = \mu + \varepsilon_t, \quad \varepsilon_t = \sigma_t z_t, \quad z_t \sim N(0,1) \tag{85.3}$$

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2 \tag{85.4}$$

The censored log-likelihood replaces the standard normal density for limit-hit observations:

$$\ell_t = \begin{cases} \log \phi\left(\frac{r_t - \mu}{\sigma_t}\right) - \log \sigma_t & \text{if interior} \\ \log \Phi\left(\frac{L - \mu}{\sigma_t}\right) & \text{if lower limit} \\ \log\left[1 - \Phi\left(\frac{\bar{L} - \mu}{\sigma_t}\right)\right] & \text{if upper limit} \end{cases} \tag{85.5}$$

```
def censored_garch11(returns, limit, max_iter=500):
    """
    GARCH(1,1) with censored normal likelihood.

    Parameters
    ----------
    returns : array-like
        Observed daily returns (censored at $\pm$ limit).
    limit : float
        Price limit (e.g., 0.07 for HOSE).

    Returns
    -------
    Dictionary with estimated parameters and conditional variances.
    """
    r = np.array(returns, dtype=float)
    T = len(r)
    upper = limit
    lower = -limit

    at_upper = r >= (upper - 1e-6)
    at_lower = r <= (lower + 1e-6)
    interior = ~at_upper & ~at_lower

    def neg_loglik(params):
        mu, omega, alpha, beta = params

        if omega <= 0 or alpha < 0 or beta < 0 or (alpha + beta) >= 1:
            return 1e10
```

```
        sigma2 = np.zeros(T)
        sigma2[0] = omega / (1 - alpha - beta) if (alpha + beta) < 1 else
↪   r.var()

        ll = 0
        for t in range(T):
            if t > 0:
                eps = r[t - 1] - mu
                sigma2[t] = omega + alpha * eps ** 2 + beta * sigma2[t - 1]

            sigma2[t] = max(sigma2[t], 1e-10)
            sigma = np.sqrt(sigma2[t])

            if interior[t]:
                ll += stats.norm.logpdf(r[t], mu, sigma)
            elif at_upper[t]:
                prob = 1 - stats.norm.cdf(upper, mu, sigma)
                ll += np.log(max(prob, 1e-15))
            elif at_lower[t]:
                prob = stats.norm.cdf(lower, mu, sigma)
                ll += np.log(max(prob, 1e-15))

        return -ll

    # Initial values from standard GARCH
    mu0 = r[interior].mean() if interior.any() else 0
    var0 = r[interior].var() if interior.any() else r.var()

    try:
        result = optimize.minimize(
            neg_loglik,
            [mu0, var0 * 0.05, 0.10, 0.85],
            method='Nelder-Mead',
            options={'maxiter': max_iter, 'xatol': 1e-8}
        )
        mu, omega, alpha, beta = result.x

        # Reconstruct conditional variance
        sigma2 = np.zeros(T)
        sigma2[0] = omega / max(1 - alpha - beta, 0.01)
        for t in range(1, T):
            eps = r[t - 1] - mu
```

```
            sigma2[t] = omega + alpha * eps ** 2 + beta * sigma2[t - 1]

        return {
            'mu': mu, 'omega': omega, 'alpha': alpha, 'beta': beta,
            'persistence': alpha + beta,
            'uncond_var': omega / max(1 - alpha - beta, 0.01),
            'sigma2': sigma2,
            'loglik': -result.fun,
            'converged': result.success,
            'n_censored': at_upper.sum() + at_lower.sum(),
            'pct_censored': (at_upper.sum() + at_lower.sum()) / T * 100
        }
    except Exception as e:
        return None

# Compare standard vs censored GARCH for a volatile stock
volatile_stock = daily_hose.groupby('ticker')['limit_up_hit'].mean()
volatile_stock = volatile_stock.sort_values(ascending=False).head(20)
test_ticker = volatile_stock.index[0]

test_returns = (
    daily_hose[daily_hose['ticker'] == test_ticker]
    .sort_values('date')['daily_return']
    .dropna()
    .values
)

# Standard GARCH (arch library)
std_garch = arch_model(test_returns * 100, vol='GARCH', p=1, q=1,
                       mean='Constant', dist='normal')
std_result = std_garch.fit(disp='off')

# Censored GARCH
cens_result = censored_garch11(test_returns, limit=0.07)

print(f"Stock: {test_ticker}")
print(f"Observations: {len(test_returns)}, "
      f"Censored: {cens_result['pct_censored']:.1f}%\n")

print(f"{'Parameter':<12} {'Standard':>12} {'Censored':>12}")
print("-" * 36)
print(f"{' ':<12} {std_result.params['mu']/100:>12.6f} "
      f"{cens_result['mu']:>12.6f}")
```

```
print(f"{' ':<12} {std_result.params['omega']/10000:>12.8f} "
      f"{cens_result['omega']:>12.8f}")
print(f"{' ':<12} {std_result.params['alpha[1]']:>12.4f} "
      f"{cens_result['alpha']:>12.4f}")
print(f"{' ':<12} {std_result.params['beta[1]']:>12.4f} "
      f"{cens_result['beta']:>12.4f}")
print(f"{' + ':<12} "
      f"{std_result.params['alpha[1]']+std_result.params['beta[1]']:>12.4f} "
      f"{cens_result['persistence']:>12.4f}")
print(f"{'Uncond  ':<12} "
      f"{np.sqrt(std_result.params['omega']/(1-std_result.params['alpha[1]']
       ↪  -std_result.params['beta[1]'])/10000):>12.4f} "
      f"{np.sqrt(cens_result['uncond_var']):>12.4f}")
```

85.6 Effects on Asset Pricing Tests

85.6.1 Beta Attenuation

Price limits attenuate the covariance between stock returns and factor returns, biasing beta estimates toward zero. The intuition is simple: on days when the market moves 3% but a stock's true return would have been 6%, the observed return is capped at 7%, understating the stock's sensitivity.

```
# Compare betas estimated with all days vs excluding limit-hit days
# Also compare betas from HOSE stocks vs same stocks if they were on HNX

daily_hose_merged = daily_hose.merge(
    client.get_index_returns('VNINDEX', frequency='daily',
                             start_date='2008-01-01',
                             end_date='2024-12-31')[['date', 'return']],
    on='date', how='left'
).rename(columns={'return': 'mkt_return'})

# For each stock, estimate beta:
# (a) Using all days
# (b) Excluding limit-hit days
# (c) Tobit-corrected (censored regression)
beta_comparison = []

for ticker in hose_stocks[:300]:
```

```
test_data = daily_hose[daily_hose['ticker'] ==
 ↪  test_ticker].sort_values('date')
dates = test_data['date'].values[-len(test_returns):]

fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True,
                         gridspec_kw={'height_ratios': [1, 2]})

# Panel A: Returns with limit hits highlighted
axes[0].plot(dates, test_returns, color='#2C5F8A', linewidth=0.5, alpha=0.7)
limit_up_mask = test_returns >= 0.069
limit_down_mask = test_returns <= -0.069
axes[0].scatter(dates[limit_up_mask], test_returns[limit_up_mask],
                color='#27AE60', s=10, zorder=3, label='Upper limit')
axes[0].scatter(dates[limit_down_mask], test_returns[limit_down_mask],
                color='#C0392B', s=10, zorder=3, label='Lower limit')
axes[0].axhline(y=0.07, color='gray', linewidth=0.5, linestyle='--')
axes[0].axhline(y=-0.07, color='gray', linewidth=0.5, linestyle='--')
axes[0].set_ylabel('Return')
axes[0].set_title(f'Panel A: Daily Returns ({test_ticker})')
axes[0].legend(fontsize=8)

# Panel B: Conditional volatility
std_sigma = std_result.conditional_volatility / 100  # Convert from % to
 ↪  decimal
cens_sigma = np.sqrt(cens_result['sigma2'])

axes[1].plot(dates, std_sigma * np.sqrt(252), color='#BDC3C7',
             linewidth=1, label='Standard GARCH')
axes[1].plot(dates, cens_sigma * np.sqrt(252), color='#C0392B',
             linewidth=1.5, label='Censored GARCH')
axes[1].set_ylabel('Annualized Conditional  ')
axes[1].set_title('Panel B: Conditional Volatility')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Figure 85.8

```
    stock = daily_hose_merged[daily_hose_merged['ticker'] == ticker].dropna(
        subset=['daily_return', 'mkt_return']
    )
    if len(stock) < 250:
        continue

    # (a) All days
    X_all = sm.add_constant(stock['mkt_return'])
    model_all = sm.OLS(stock['daily_return'], X_all).fit()
    beta_all = model_all.params['mkt_return']

    # (b) Exclude limit-hit days
    interior = stock[~stock['limit_up_hit'] & ~stock['limit_down_hit']]
    if len(interior) < 200:
        continue
    X_int = sm.add_constant(interior['mkt_return'])
    model_int = sm.OLS(interior['daily_return'], X_int).fit()
    beta_interior = model_int.params['mkt_return']

    # Limit hit frequency for this stock
    pct_limit = (stock['limit_up_hit'].sum() + stock['limit_down_hit'].sum())
↪   / len(stock) * 100

    beta_comparison.append({
        'ticker': ticker,
        'beta_all': beta_all,
        'beta_interior': beta_interior,
        'beta_diff_pct': (beta_interior - beta_all) / abs(beta_all) * 100,
        'pct_limit_hits': pct_limit
    })

beta_df = pd.DataFrame(beta_comparison)

print("Beta Attenuation from Price Limits:")
print(f"  Mean   (all days):      {beta_df['beta_all'].mean():.3f}")
print(f"  Mean   (interior only): {beta_df['beta_interior'].mean():.3f}")
print(f"  Mean difference:        {beta_df['beta_diff_pct'].mean():.1f}%")
print(f"  Correlation(diff, limit_freq): "
      f"{beta_df['beta_diff_pct'].corr(beta_df['pct_limit_hits']):.3f}")
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(beta_df['beta_all'], beta_df['beta_interior'],
                s=15, alpha=0.5, color='#2C5F8A', edgecolors='none')
lim = max(beta_df['beta_all'].abs().max(),
          beta_df['beta_interior'].abs().max()) + 0.2
axes[0].plot([-0.5, lim], [-0.5, lim], 'k--', linewidth=1)
axes[0].set_xlabel('  (all days)')
axes[0].set_ylabel('  (interior only)')
axes[0].set_title('Panel A: Beta with vs without Limit Days')

axes[1].scatter(beta_df['pct_limit_hits'], beta_df['beta_diff_pct'],
                s=15, alpha=0.5, color='#C0392B', edgecolors='none')
# Add regression line
z = np.polyfit(beta_df['pct_limit_hits'], beta_df['beta_diff_pct'], 1)
x_line = np.linspace(0, beta_df['pct_limit_hits'].max(), 100)
axes[1].plot(x_line, np.polyval(z, x_line), 'k-', linewidth=1.5)
axes[1].axhline(y=0, color='gray', linewidth=0.5)
axes[1].set_xlabel('Limit Hit Frequency (%)')
axes[1].set_ylabel('Beta Increase When Excluding Limit Days (%)')
axes[1].set_title('Panel B: Attenuation vs Limit Frequency')

plt.tight_layout()
plt.show()
```

Figure 85.9

85.6.2 Effect on Factor Premia

If betas are attenuated by censoring, then cross-sectional Fama-MacBeth risk premia estimates are biased upward (because the denominator of the slope coefficient is too small). We quantify this effect:

```
# Monthly returns: compute with all days vs excluding limit-hit days
# Then construct factors under each definition

monthly_all = (
    daily_hose.groupby(['ticker', daily_hose['date'].dt.to_period('M')])
    .agg(
        ret_all=('daily_return', lambda x: (1 + x).prod() - 1),
        ret_interior=('daily_return',
                      lambda x: (1 + x[~x.name.map(
                          lambda idx: daily_hose.loc[idx, 'limit_up_hit'] |
                                      daily_hose.loc[idx, 'limit_down_hit']
                      ).values]).prod() - 1 if len(x) > 0 else np.nan),
        n_limit_days=('limit_up_hit',
                      lambda x: x.sum() + daily_hose.loc[x.index,
                       ↪  'limit_down_hit'].sum()),
        n_trading_days=('daily_return', 'count')
    )
    .reset_index()
)

print("Impact on Monthly Returns:")
print(f"  Mean monthly return (all days):     "
      f"{monthly_all['ret_all'].mean():.4f}")
print(f"  Mean monthly return (interior):     "
      f"{monthly_all['ret_interior'].mean():.4f}")
print(f"  Avg limit days per stock-month:     "
      f"{monthly_all['n_limit_days'].mean():.2f}")
```

85.7 The Magnet Effect

85.7.1 Do Limits Attract Prices?

The *magnet effect* hypothesis posits that price limits, rather than cooling the market, actually attract prices to the limit as traders rush to execute before the stock becomes locked (Cho et

al. 2003). If a stock is approaching the upper limit, buyers accelerate their orders to avoid being shut out, creating a self-fulfilling rush to the boundary.

We test for the magnet effect by examining the speed of price movement toward the limit conditional on approaching it:

```
# Approach: for days that eventually hit the limit,
# compare the return in the last hour vs the first hour
# relative to non-limit days with similar initial trajectories

# Without intraday data, we use a cross-day approach:
# On day t, if the stock is within X% of the limit at some point,
# what is the probability of hitting the limit on day t vs day t+1?

# Simpler test: return continuation after near-limit days
def magnet_test(daily_df, limit, proximity_threshold=0.8):
    """
    Test for the magnet effect.

    For each stock-day, classify:
    - 'near_limit_up': return in [proximity_threshold * limit, limit)
    - 'near_limit_down': return in (-limit, -proximity_threshold * limit]
    - 'hit_limit_up': return = limit
    - 'hit_limit_down': return = -limit
    - 'normal': all others

    Then examine next-day behavior.
    """
    df = daily_df.copy()
    df['abs_ret'] = df['daily_return'].abs()

    df['near_up'] = (df['daily_return'] >= proximity_threshold * limit) & \
                    (df['daily_return'] < limit - 0.001)
    df['near_down'] = (df['daily_return'] <= -proximity_threshold * limit) &
↪   \
                      (df['daily_return'] > -limit + 0.001)

    df['next_return'] = df.groupby('ticker')['daily_return'].shift(-1)
    df['next_limit_up'] = df.groupby('ticker')['limit_up_hit'].shift(-1)
    df['next_limit_down'] = df.groupby('ticker')['limit_down_hit'].shift(-1)

    results = {}

    # Near upper limit
```

```
    near_up = df[df['near_up']]
    if len(near_up) > 100:
        results['near_up'] = {
            'n': len(near_up),
            'next_day_return': near_up['next_return'].mean(),
            'prob_next_limit_up': near_up['next_limit_up'].mean(),
            'prob_next_limit_down': near_up['next_limit_down'].mean()
        }

    # At upper limit
    at_up = df[df['limit_up_hit']]
    if len(at_up) > 100:
        results['at_up'] = {
            'n': len(at_up),
            'next_day_return': at_up['next_return'].mean(),
            'prob_next_limit_up': at_up['next_limit_up'].mean(),
            'prob_next_limit_down': at_up['next_limit_down'].mean()
        }

    # Near lower limit
    near_down = df[df['near_down']]
    if len(near_down) > 100:
        results['near_down'] = {
            'n': len(near_down),
            'next_day_return': near_down['next_return'].mean(),
            'prob_next_limit_up': near_down['next_limit_up'].mean(),
            'prob_next_limit_down': near_down['next_limit_down'].mean()
        }

    # At lower limit
    at_down = df[df['limit_down_hit']]
    if len(at_down) > 100:
        results['at_down'] = {
            'n': len(at_down),
            'next_day_return': at_down['next_return'].mean(),
            'prob_next_limit_up': at_down['next_limit_up'].mean(),
            'prob_next_limit_down': at_down['next_limit_down'].mean()
        }

    # Normal days (benchmark)
    normal = df[~df['near_up'] & ~df['near_down'] &
                ~df['limit_up_hit'] & ~df['limit_down_hit']]
    results['normal'] = {
```

```
        'n': len(normal),
        'next_day_return': normal['next_return'].mean(),
        'prob_next_limit_up': normal['next_limit_up'].mean(),
        'prob_next_limit_down': normal['next_limit_down'].mean()
    }

    return pd.DataFrame(results).T

magnet = magnet_test(daily_hose, 0.07, proximity_threshold=0.8)
print("Magnet Effect Test (HOSE):")
print(magnet.round(4).to_string())
```

85.8 The Delayed Price Discovery Hypothesis

85.8.1 Volatility Spillover

If limits prevent full price adjustment on day t, the residual adjustment spills over to day $t+1$ (and possibly further). This predicts higher volatility on the day *after* a limit hit, and positive return autocorrelation (continuation in the direction of the limit hit). K. A. Kim and Rhee (1997) find strong evidence of this in the Tokyo Stock Exchange.

```
# Compare volatility and return on limit-hit days vs day after
spillover_data = daily_hose.copy()
spillover_data['prev_limit_up'] = (
    spillover_data.groupby('ticker')['limit_up_hit'].shift(1)
)
spillover_data['prev_limit_down'] = (
    spillover_data.groupby('ticker')['limit_down_hit'].shift(1)
)
spillover_data['abs_return'] = spillover_data['daily_return'].abs()

# Classify days
conditions = {
    'After upper limit': spillover_data['prev_limit_up'] == True,
    'After lower limit': spillover_data['prev_limit_down'] == True,
    'Normal day': (spillover_data['prev_limit_up'] == False) &
                  (spillover_data['prev_limit_down'] == False)
}

print("Volatility Spillover After Limit Hits:")
```

```
# More granular: bin today's return and compute next-day statistics
bins = np.arange(-0.075, 0.08, 0.005)
daily_hose_next = daily_hose.copy()
daily_hose_next['next_return'] = (
    daily_hose_next.groupby('ticker')['daily_return'].shift(-1)
)
daily_hose_next['ret_bin'] = pd.cut(daily_hose_next['daily_return'],
                                    bins=bins, labels=False)

bin_stats = (
    daily_hose_next.dropna(subset=['ret_bin', 'next_return'])
    .groupby('ret_bin')
    .agg(
        mean_ret=('daily_return', 'mean'),
        next_ret=('next_return', 'mean'),
        n=('next_return', 'count')
    )
)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Next-day return by today's return
axes[0].bar(bin_stats['mean_ret'] * 100, bin_stats['next_ret'] * 100,
            width=0.4,
            color=np.where(bin_stats['next_ret'] > 0, '#27AE60', '#C0392B'),
            alpha=0.7, edgecolor='white')
axes[0].axhline(y=0, color='black', linewidth=0.5)
axes[0].axvline(x=7, color='gray', linewidth=1, linestyle='--')
axes[0].axvline(x=-7, color='gray', linewidth=1, linestyle='--')
axes[0].set_xlabel("Today's Return (%)")
axes[0].set_ylabel('Next-Day Return (%)')
axes[0].set_title('Panel A: Next-Day Return by Current Return')

# Panel B: Continuation probability
# Probability of same-direction move next day
daily_hose_next['continuation'] = (
    np.sign(daily_hose_next['daily_return']) ==
    np.sign(daily_hose_next['next_return'])
)

cont_by_bin = (
    daily_hose_next.dropna(subset=['ret_bin', 'continuation'])
    .groupby('ret_bin')
    .agg(
        mean_ret=('daily_return', 'mean'),
        cont_prob=('continuation', 'mean'),
        n=('continuation', 'count')
    )
)

axes[1].scatter(cont_by_bin['mean_ret'] * 100, cont_by_bin['cont_prob'] *
 ↪  100,
                color='#2C5F8A', s=40, alpha=0.7)
axes[1].axhline(y=50, color='gray', linewidth=0.5, linestyle='--')
```

```
print(f"{'Condition':<25} {'Mean |r|':>10} {'Mean r':>10} "
      f"{' (r)':>10} {'N':>12}")
print("-" * 67)

for label, mask in conditions.items():
    subset = spillover_data[mask].dropna(subset=['daily_return'])
    print(f"{label:<25} "
          f"{subset['abs_return'].mean()*100:>10.3f}% "
          f"{subset['daily_return'].mean()*100:>10.3f}% "
          f"{subset['daily_return'].std()*100:>10.3f}% "
          f"{len(subset):>12,}")

# Statistical test: is variance higher after limit days?
normal = spillover_data[conditions['Normal day']]['daily_return'].dropna()
after_up = spillover_data[conditions['After upper
 ↪  limit']]['daily_return'].dropna()
after_down = spillover_data[conditions['After lower
 ↪  limit']]['daily_return'].dropna()

f_up = after_up.var() / normal.var()
f_down = after_down.var() / normal.var()
print(f"\nVariance ratios (vs normal days):")
print(f"  After upper limit: {f_up:.3f} "
      f"(p = {1 - stats.f.cdf(f_up, len(after_up)-1, len(normal)-1):.4f})")
print(f"  After lower limit: {f_down:.3f} "
      f"(p = {1 - stats.f.cdf(f_down, len(after_down)-1,
       ↪  len(normal)-1):.4f})")
```

85.8.2 Multi-Day Return Reconstruction

To recover the "true" return that would have occurred without price limits, we can compound returns over consecutive limit-hit days until the stock resumes normal trading:

```
def reconstruct_returns(group, limit):
    """
    For each limit-hit sequence, compound returns until
    the stock resumes normal trading (first non-limit day).
    Returns the compound return and the number of days.
    """
    sequences = []
    in_sequence = False
```

```
    seq_start = None
    seq_returns = []
    seq_direction = None

    for _, row in group.iterrows():
        if row['limit_up_hit'] or row['limit_down_hit']:
            if not in_sequence:
                in_sequence = True
                seq_start = row['date']
                seq_returns = [row['daily_return']]
                seq_direction = 'up' if row['limit_up_hit'] else 'down'
            else:
                seq_returns.append(row['daily_return'])
        else:
            if in_sequence:
                # Include the first non-limit day (the "resolution" day)
                seq_returns.append(row['daily_return'])
                compound_ret = np.prod([1 + r for r in seq_returns]) - 1
                sequences.append({
                    'ticker': group.name if hasattr(group, 'name') else
                    ↪  group['ticker'].iloc[0],
                    'start_date': seq_start,
                    'n_limit_days': len(seq_returns) - 1,
                    'compound_return': compound_ret,
                    'direction': seq_direction,
                    'limit_return': sum(seq_returns[:-1]),
                    'resolution_return': seq_returns[-1]
                })
                in_sequence = False

    return sequences

# Run for all HOSE stocks
all_sequences = []
for ticker, group in daily_hose.sort_values('date').groupby('ticker'):
    seqs = reconstruct_returns(group, 0.07)
    all_sequences.extend(seqs)

seq_df = pd.DataFrame(all_sequences)

if len(seq_df) > 0:
    print("Limit-Hit Sequence Analysis:")
    print(f"  Total sequences: {len(seq_df):,}")
```

```
    print(f"  Mean limit days: {seq_df['n_limit_days'].mean():.1f}")
    print(f"\nCompound Returns by Direction:")
    for direction in ['up', 'down']:
        subset = seq_df[seq_df['direction'] == direction]
        print(f"  {direction.upper()} sequences: {len(subset):,}")
        print(f"    Mean compound return:
         ↪ {subset['compound_return'].mean()*100:.2f}%")
        print(f"    Mean resolution-day return: "
              f"{subset['resolution_return'].mean()*100:.2f}%")
        print(f"    Max compound return:
         ↪ {subset['compound_return'].max()*100:.1f}%")
```

85.9 The Idiosyncratic Volatility Puzzle Under Price Limits

Ang et al. (2006) document that stocks with high idiosyncratic volatility earn low subsequent returns—the IVOL puzzle. In Vietnam, price limits contaminate IVOL estimation: stocks that frequently hit limits have *understated* IVOL (because their returns are censored), which could create a mechanical relation between measured IVOL and returns.

```
# Compute monthly IVOL two ways:
# (a) Naive: std of daily residuals from FF3
# (b) Corrected: excluding limit-hit days

# Merge daily data with market returns for IVOL estimation
daily_ff = daily_hose_merged.copy()

monthly_ivol = []
for (ticker, month), group in daily_ff.groupby(
    ['ticker', daily_ff['date'].dt.to_period('M')]
):
    if len(group) < 15:
        continue

    y = group['daily_return'].dropna()
    x = group['mkt_return'].reindex(y.index).dropna()
    common = y.index.intersection(x.index)
    if len(common) < 15:
        continue

    # Naive IVOL
```

```
    model = sm.OLS(y[common], sm.add_constant(x[common])).fit()
    ivol_naive = model.resid.std() * np.sqrt(252)

    # Interior-only IVOL
    interior = group[~group['limit_up_hit'] & ~group['limit_down_hit']]
    y_int = interior['daily_return'].dropna()
    x_int = interior['mkt_return'].reindex(y_int.index).dropna()
    common_int = y_int.index.intersection(x_int.index)

    if len(common_int) >= 10:
        model_int = sm.OLS(y_int[common_int],
                           sm.add_constant(x_int[common_int])).fit()
        ivol_corrected = model_int.resid.std() * np.sqrt(252)
    else:
        ivol_corrected = np.nan

    n_limit = group['limit_up_hit'].sum() + group['limit_down_hit'].sum()

    monthly_ivol.append({
        'ticker': ticker,
        'month': month.to_timestamp(),
        'ivol_naive': ivol_naive,
        'ivol_corrected': ivol_corrected,
        'n_limit_days': n_limit,
        'pct_limit': n_limit / len(group) * 100,
        'next_return': group['daily_return'].iloc[-1]  # Placeholder
    })

ivol_df = pd.DataFrame(monthly_ivol)

print("IVOL Estimation: Naive vs Corrected:")
print(f"  Mean naive IVOL:     {ivol_df['ivol_naive'].mean():.4f}")
print(f"  Mean corrected IVOL: {ivol_df['ivol_corrected'].mean():.4f}")
print(f"  Mean difference:     "
      f"{(ivol_df['ivol_corrected'] - ivol_df['ivol_naive']).mean():.4f}")
print(f"  Correlation:         "
      f"{ivol_df['ivol_naive'].corr(ivol_df['ivol_corrected']):.3f}")
```

85.10 Practical Recommendations

For researchers working with Vietnamese equity data:

Always report limit-hit frequency. Any study using Vietnamese daily returns should document the fraction of observations at the price limits, broken down by exchange and market cap quintile. This tells the reader the severity of the censoring problem in the specific sample.

Use Tobit-corrected variance estimates. For volatility-related analyses (IVOL sorts, GARCH, risk modeling), the naive sample variance underestimates true variance by 5–20% depending on the stock's limit-hit frequency. The Tobit MLE provides a consistent estimator under the censored normal assumption.

Consider range-based estimators. The D. Yang and Zhang (2000) estimator using OHLC prices is partially robust to closing-price censoring and does not require distributional assumptions. It is a good default for individual-stock volatility estimation.

Exclude limit-hit days for beta estimation. Interior-only betas are less biased than all-day betas, though noisier. Report both and discuss the difference. For stocks with >5% limit-hit frequency, the attenuation is economically meaningful.

Compound multi-day returns for event studies. When studying events that coincide with limit hits (earnings announcements, M&A, regulatory changes), use the compound return from the limit-hit sequence start to the first non-limit day. Single-day returns are censored and understate the market's reaction.

Be cautious interpreting short-term return predictability. The delayed price discovery effect creates positive return autocorrelation at the daily frequency. This is a mechanical consequence of censoring, not a market inefficiency. Monthly returns are largely free of this artifact because the censoring within a month averages out.

Test robustness to HNX and UPCoM. If a result is driven by limit-related distortions, it should appear differently (or not at all) on HNX (± 10%) and UPCoM (± 15%). Cross-exchange comparison is a natural placebo test.

85.11 Summary

Table 85.2: Summary of price limit effects on empirical estimates.

Issue	Bias Direction	Magnitude (HOSE)	Recommended Fix
Return variance	Understated	5–20% for volatile stocks	Tobit MLE or range-based
GARCH vol	Understated	10–30% during crises	Censored GARCH
Market beta	Attenuated (toward 0)	3–10% for small-caps	Interior-only estimation
IVOL	Understated	Varies; correlated with size	Corrected IVOL

Issue	Bias Direction	Magnitude (HOSE)	Recommended Fix
Return autocorrelation	Positive (spurious)	Significant at daily freq	Use weekly/monthly
Event study CARs	Understated	Up to 50% of true effect	Compound multi-day
Distribution shape	Pile-up at limits	2–5% of obs at limits	Acknowledge or correct

Price limits are not a minor institutional detail—they are a pervasive data-generating process that affects nearly every empirical quantity computed from Vietnamese daily returns. The corrections developed in this chapter—Tobit variance estimation, censored GARCH, interior-only betas, range-based volatility, and multi-day return compounding—form a toolkit that should be applied routinely. Ignoring censoring does not make it go away; it merely makes the resulting estimates quietly wrong.

86 Market Integration and Segmentation

> **Note**
>
> In this chapter, we measure the degree to which the Vietnamese equity market is integrated with or segmented from global capital markets. We construct multiple integration metrics, including correlation-based, factor-based, and pricing-error-based, trace their evolution through Vietnam's liberalization timeline, and quantify the cost of segmentation for Vietnamese firms.

A market is *fully integrated* when its assets are priced by a global stochastic discount factor: risk premia reflect only exposures to global risk factors, and identical cash flow streams command the same expected return regardless of where the issuer is domiciled. A market is *fully segmented* when domestic risk factors alone determine prices, and the country's risk-return trade-off is independent of the rest of the world. Reality sits somewhere between these poles, and the location shifts over time.

Vietnam is a particularly interesting case. It opened its stock exchange in July 2000 with heavy restrictions on foreign participation. Foreign ownership limits (initially 20%, raised to 30% in 2003, 49% in 2015, and selectively removed for some firms) have been gradually relaxed. Vietnam joined the WTO in 2007. FTSE Russell upgraded Vietnam from "unclassified" to "secondary emerging" in its frontier index in 2018 and has been evaluating further upgrades. Each of these events has potentially shifted the degree of integration.

Bekaert and Harvey (1995) and Bekaert and Harvey (2002) establish the modern framework for measuring time-varying integration. Errunza and Losq (1985) develop the "mild segmentation" model in which foreign investors face barriers but can partially replicate emerging market returns through global securities. Pukthuanthong and Roll (2009) propose a factor-model-based measure that avoids the pitfalls of simple correlation analysis. This chapter implements all three approaches and applies them to Vietnam's integration trajectory.

86.1 Integration in Theory

86.1.1 The Integrated and Segmented Benchmarks

Under full integration, the expected excess return of Vietnamese stock i is:

$$E[R_{i,t} - R_{f,t}] = \beta_{i,\text{world}} \cdot \lambda_{\text{world},t} \tag{86.1}$$

where $\beta_{i,\text{world}}$ is the stock's loading on the global market factor and $\lambda_{\text{world},t}$ is the global risk premium. Only global systematic risk is priced; country-specific risk is diversifiable and commands no premium.

Under full segmentation:

$$E[R_{i,t} - R_{f,t}] = \beta_{i,\text{local}} \cdot \lambda_{\text{local},t} \tag{86.2}$$

where $\beta_{i,\text{local}}$ is the stock's loading on the Vietnamese market and $\lambda_{\text{local},t}$ is the domestic risk premium. The domestic market is effectively a closed economy for pricing purposes.

Bekaert and Harvey (1995) model the transition between these states as a regime-switching process where the mixing weight $\omega_t \in [0, 1]$ evolves over time:

$$E[R_{i,t} - R_{f,t}] = \omega_t \cdot \beta_{i,\text{world}} \lambda_{\text{world},t} + (1 - \omega_t) \cdot \beta_{i,\text{local}} \lambda_{\text{local},t} \tag{86.3}$$

The weight ω_t is the *degree of integration*: $\omega_t = 1$ is full integration, $\omega_t = 0$ is full segmentation.

86.1.2 The Segmentation Premium

When a market transitions from segmented to integrated, its cost of capital falls because the relevant risk for pricing narrows from total domestic risk to only the portion correlated with the global market (Henry 2000; Bekaert, Harvey, and Lundblad 2005). The *segmentation premium* is the excess expected return that investors in a segmented market require:

$$\text{Segmentation premium} = (1 - \omega_t) \cdot (\lambda_{\text{local}} - \beta_{\text{local,world}} \cdot \lambda_{\text{world}}) \tag{86.4}$$

This premium represents a deadweight cost: it raises the cost of capital for Vietnamese firms, reduces investment, and lowers welfare relative to the integrated benchmark.

86.2 Data Construction

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from scipy import stats, optimize
from arch import arch_model
from arch.univariate import ConstantMean, GARCH
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.figsize': (12, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

```
from datacore import DataCoreClient

client = DataCoreClient()

# Vietnamese market returns
vn_index = client.get_index_returns(
    index='VNINDEX',
    start_date='2000-07-01',
    end_date='2024-12-31',
    frequency='monthly',
    fields=['date', 'return', 'total_return_index']
)
vn_index['date'] = pd.to_datetime(vn_index['date'])
vn_index = vn_index.set_index('date')

# Global and regional indices (USD-denominated for comparability)
global_indices = client.get_global_indices(
    indices=[
        'MSCI_WORLD', 'MSCI_EM', 'MSCI_ASIA_PAC_EX_JP',
        'MSCI_FM',  # Frontier markets
        'SP500', 'STOXX600',
        'MSCI_CHINA', 'MSCI_THAILAND', 'MSCI_INDONESIA',
        'MSCI_PHILIPPINES', 'MSCI_MALAYSIA'
```

```
    ],
    start_date='2000-07-01',
    end_date='2024-12-31',
    frequency='monthly',
    currency='USD'
)
global_indices['date'] = pd.to_datetime(global_indices['date'])
global_indices = global_indices.pivot(index='date', columns='index',
 ↪  values='return')

# Vietnam returns in USD for apples-to-apples comparison
vn_usd = client.get_index_returns(
    index='VNINDEX',
    start_date='2000-07-01',
    end_date='2024-12-31',
    frequency='monthly',
    currency='USD'
)
vn_usd['date'] = pd.to_datetime(vn_usd['date'])
global_indices['VIETNAM'] = vn_usd.set_index('date')['return']

# VND/USD exchange rate
fx = client.get_exchange_rates(
    pair='USD_VND',
    start_date='2000-07-01',
    end_date='2024-12-31',
    frequency='monthly'
)
fx['date'] = pd.to_datetime(fx['date'])
fx = fx.set_index('date')

# Global factor returns (Fama-French global)
global_factors = client.get_global_factor_returns(
    start_date='2000-07-01',
    end_date='2024-12-31',
    frequency='monthly',
    factors=['mkt_excess_world', 'smb_world', 'hml_world',
             'rmw_world', 'cma_world', 'wml_world']
)
global_factors['date'] = pd.to_datetime(global_factors['date'])
global_factors = global_factors.set_index('date')

# Vietnamese factor returns (local)
```

```
local_factors = client.get_factor_returns(
    market='vietnam',
    start_date='2008-01-01',
    end_date='2024-12-31',
    factors=['mkt_excess', 'smb', 'hml', 'rmw', 'cma', 'wml']
)
local_factors['date'] = pd.to_datetime(local_factors['date'])
local_factors = local_factors.set_index('date')

print(f"Vietnam index: {len(vn_index)} months")
print(f"Global indices: {global_indices.shape}")
print(f"Global factors: {len(global_factors)} months")
```

86.2.1 Vietnam's Liberalization Timeline

```
liberalization_events = pd.DataFrame([
    ('2000-07-28', 'HOSE opens', 'Institutional'),
    ('2002-03-01', 'FOL raised to 30%', 'Ownership'),
    ('2005-03-01', 'HNX opens', 'Institutional'),
    ('2006-01-01', 'Securities Law enacted', 'Legal'),
    ('2007-01-11', 'WTO accession', 'Trade'),
    ('2007-06-01', 'FOL raised to 49%', 'Ownership'),
    ('2009-06-24', 'UPCoM opens', 'Institutional'),
    ('2012-01-01', 'SSC restructuring', 'Regulatory'),
    ('2015-09-01', 'FOL selectively removed', 'Ownership'),
    ('2017-08-01', 'Derivatives market opens', 'Institutional'),
    ('2018-09-01', 'FTSE Frontier Secondary', 'Index'),
    ('2021-01-01', 'New Securities Law', 'Legal'),
    ('2023-06-01', 'KRX trading system', 'Infrastructure'),
], columns=['date', 'event', 'category'])
liberalization_events['date'] = pd.to_datetime(liberalization_events['date'])

print("Vietnam Liberalization Timeline:")
for _, row in liberalization_events.iterrows():
    print(f"  {row['date'].strftime('%Y-%m')}: {row['event']}
    ↪  [{row['category']}]")
```

86.3 Correlation-Based Integration Measures

86.3.1 Rolling Correlations

The simplest integration metric is the correlation between Vietnamese and global market returns. Higher correlation implies more integration (returns are driven by the same global factors). However, simple correlation is confounded by volatility changes (i.e., correlations tend to increase mechanically during high-volatility periods (Longin and Solnik 2001)).

```
# Align all series
indices_aligned = global_indices.dropna(subset=['VIETNAM',
 ↪  'MSCI_WORLD']).copy()

# Rolling 36-month correlations
rolling_window = 36

corr_series = {}
for idx in ['MSCI_WORLD', 'MSCI_EM', 'MSCI_ASIA_PAC_EX_JP',
            'SP500', 'MSCI_CHINA', 'MSCI_THAILAND']:
    if idx in indices_aligned.columns:
        corr = (
            indices_aligned[['VIETNAM', idx]]
            .rolling(rolling_window)
            .corr()
            .unstack()['VIETNAM'][idx]
        )
        corr_series[idx] = corr

corr_df = pd.DataFrame(corr_series)
```

86.3.2 DCC-GARCH Dynamic Correlations

To separate changes in correlation from changes in volatility, we estimate a Dynamic Conditional Correlation (DCC) model (Engle 2002). The DCC decomposes the time-varying covariance matrix into time-varying volatilities and a time-varying correlation matrix:

$$H_t = D_t R_t D_t \tag{86.5}$$

where $D_t = \text{diag}(\sigma_{1,t}, \ldots, \sigma_{n,t})$ and R_t is the conditional correlation matrix that evolves according to:

```
fig, ax = plt.subplots(figsize=(14, 6))

colors_idx = {
    'MSCI_WORLD': '#2C5F8A', 'MSCI_EM': '#C0392B',
    'MSCI_ASIA_PAC_EX_JP': '#27AE60', 'SP500': '#8E44AD',
    'MSCI_CHINA': '#E67E22', 'MSCI_THAILAND': '#1ABC9C'
}

for idx, color in colors_idx.items():
    if idx in corr_df.columns:
        ax.plot(corr_df.index, corr_df[idx], color=color,
                linewidth=1.5, label=idx.replace('MSCI_', '').replace('_', '
 ↪  '),
                alpha=0.85)

# Add liberalization events
for _, event in liberalization_events.iterrows():
    if event['date'] >= corr_df.index.min():
        ax.axvline(x=event['date'], color='gray', linewidth=0.5,
                   linestyle=':', alpha=0.6)

ax.axhline(y=0, color='black', linewidth=0.5)
ax.set_ylabel('Correlation with Vietnam')
ax.set_title('Rolling 36-Month Correlation: Vietnam vs Global Markets')
ax.legend(fontsize=8, ncol=3)
ax.set_ylim([-0.3, 0.8])

plt.tight_layout()
plt.show()
```

Figure 86.1

$$Q_t = (1 - a - b)\bar{Q} + a\epsilon_{t-1}\epsilon_{t-1}' + bQ_{t-1} \tag{86.6}$$

$$R_t = \text{diag}(Q_t)^{-1/2} Q_t \text{diag}(Q_t)^{-1/2} \tag{86.7}$$

```
def estimate_dcc(y1, y2, p=1, q=1):
    """
    Two-step DCC-GARCH estimation.
    Step 1: Univariate GARCH for each series.
    Step 2: DCC parameters from standardized residuals.
    """
    # Step 1: Univariate GARCH(1,1) for each series
    models = []
    std_resids = []
    cond_vols = []

    for y in [y1, y2]:
        am = arch_model(y * 100, vol='GARCH', p=p, q=q,
                        mean='Constant', dist='normal')
        res = am.fit(disp='off')
        models.append(res)
        std_resids.append(res.std_resid)
        cond_vols.append(res.conditional_volatility / 100)

    # Align residuals
    e1 = std_resids[0]
    e2 = std_resids[1]
    common = e1.dropna().index.intersection(e2.dropna().index)
    e1 = e1[common].values
    e2 = e2[common].values
    T = len(e1)

    # Step 2: DCC estimation
    # Q_bar = unconditional correlation of standardized residuals
    Q_bar = np.corrcoef(e1, e2)

    def dcc_loglik(params):
        a, b = params
        if a < 0 or b < 0 or a + b >= 1:
            return 1e10

        Q = np.zeros((T, 2, 2))
```

```
        R = np.zeros((T, 2, 2))
        Q[0] = Q_bar.copy()

        ll = 0
        for t in range(T):
            if t > 0:
                et = np.array([[e1[t-1]], [e2[t-1]]])
                Q[t] = (1 - a - b) * Q_bar + a * (et @ et.T) + b * Q[t-1]

            # Normalize
            d = np.sqrt(np.diag(Q[t]))
            if d[0] > 0 and d[1] > 0:
                R[t] = Q[t] / np.outer(d, d)
            else:
                R[t] = np.eye(2)

            # Clip correlation
            R[t, 0, 1] = np.clip(R[t, 0, 1], -0.999, 0.999)
            R[t, 1, 0] = R[t, 0, 1]

            # Log-likelihood contribution
            det_R = 1 - R[t, 0, 1] ** 2
            if det_R > 0:
                et_vec = np.array([e1[t], e2[t]])
                ll += -0.5 * (np.log(det_R) +
                              et_vec @ np.linalg.inv(R[t]) @ et_vec -
                              et_vec @ et_vec)

        return -ll

    result = optimize.minimize(dcc_loglik, [0.05, 0.90],
                               method='Nelder-Mead',
                               options={'maxiter': 5000})
    a_hat, b_hat = result.x

    # Reconstruct dynamic correlations
    Q = np.zeros((T, 2, 2))
    dcc_corr = np.zeros(T)
    Q[0] = Q_bar.copy()

    for t in range(T):
        if t > 0:
            et = np.array([[e1[t-1]], [e2[t-1]]])
```

```
            Q[t] = (1 - a_hat - b_hat) * Q_bar + a_hat * (et @ et.T) + b_hat
↪  * Q[t-1]

        d = np.sqrt(np.diag(Q[t]))
        if d[0] > 0 and d[1] > 0:
            dcc_corr[t] = Q[t, 0, 1] / (d[0] * d[1])
        else:
            dcc_corr[t] = 0

    return {
        'a': a_hat, 'b': b_hat,
        'persistence': a_hat + b_hat,
        'dcc_corr': pd.Series(dcc_corr, index=common),
        'cond_vol_1': cond_vols[0],
        'cond_vol_2': cond_vols[1]
    }

# Estimate DCC: Vietnam vs MSCI World
vn_ret = indices_aligned['VIETNAM'].dropna()
world_ret = indices_aligned['MSCI_WORLD'].dropna()
common_dates = vn_ret.index.intersection(world_ret.index)

dcc_result = estimate_dcc(vn_ret[common_dates], world_ret[common_dates])

print(f"DCC Parameters:")
print(f"  a (news): {dcc_result['a']:.4f}")
print(f"  b (persistence): {dcc_result['b']:.4f}")
print(f"  a + b: {dcc_result['persistence']:.4f}")
print(f"\nDCC Correlation with MSCI World:")
print(f"  Mean: {dcc_result['dcc_corr'].mean():.3f}")
print(f"  Min:  {dcc_result['dcc_corr'].min():.3f}")
print(f"  Max:  {dcc_result['dcc_corr'].max():.3f}")
```

86.3.3 Asymmetric Integration

Integration may be state-dependent: co-movement often increases during global crises (contagion) but not during local booms. Longin and Solnik (2001) and Ang and Chen (2002) show that equity correlations are higher during market downturns. We test for asymmetry:

```
fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True,
                         gridspec_kw={'height_ratios': [2, 1]})

# Panel A: DCC correlation
axes[0].plot(dcc_result['dcc_corr'].index, dcc_result['dcc_corr'].values,
             color='#2C5F8A', linewidth=1.5)
axes[0].fill_between(dcc_result['dcc_corr'].index,
                     dcc_result['dcc_corr'].values, 0,
                     alpha=0.2, color='#2C5F8A')

# Liberalization events
event_colors = {'Ownership': '#C0392B', 'Trade': '#27AE60',
                'Institutional': '#E67E22', 'Legal': '#8E44AD',
                'Index': '#1ABC9C', 'Regulatory': '#F1C40F',
                'Infrastructure': '#3498DB'}

for _, event in liberalization_events.iterrows():
    if event['date'] in dcc_result['dcc_corr'].index or True:
        color = event_colors.get(event['category'], 'gray')
        axes[0].axvline(x=event['date'], color=color, linewidth=1.5,
                        linestyle='--', alpha=0.7)
        axes[0].text(event['date'], axes[0].get_ylim()[1] * 0.95,
                     event['event'][:15], rotation=90, fontsize=6,
                     va='top', color=color)

axes[0].set_ylabel('Dynamic Conditional Correlation')
axes[0].set_title('Panel A: DCC-GARCH Correlation (Vietnam-World)')
axes[0].axhline(y=0, color='black', linewidth=0.5)

# Panel B: Conditional volatilities
vol1 = dcc_result['cond_vol_1'] * np.sqrt(12)  # Annualized
vol2 = dcc_result['cond_vol_2'] * np.sqrt(12)
common_vol = vol1.index.intersection(vol2.index)

axes[1].plot(common_vol, vol1[common_vol], color='#C0392B',
             linewidth=1, label='Vietnam', alpha=0.8)
axes[1].plot(common_vol, vol2[common_vol], color='#2C5F8A',
             linewidth=1, label='World', alpha=0.8)
axes[1].set_ylabel('Annualized Cond. Vol')
axes[1].set_title('Panel B: Conditional Volatility')
axes[1].legend(fontsize=9)

plt.tight_layout()
plt.show()
```

Figure 86.2

```
# Classify global market regimes
world_ret_aligned = world_ret[common_dates]
vn_ret_aligned = vn_ret[common_dates]

# Bear: world return in bottom 25th percentile
# Bull: world return in top 25th percentile
q25 = world_ret_aligned.quantile(0.25)
q75 = world_ret_aligned.quantile(0.75)

bear = world_ret_aligned <= q25
bull = world_ret_aligned >= q75
normal = ~bear & ~bull

regimes = {
    'Bear (bottom 25%)': bear,
    'Normal (middle 50%)': normal,
    'Bull (top 25%)': bull,
    'All': pd.Series(True, index=world_ret_aligned.index)
}

print("Asymmetric Correlation:")
print(f"{'Regime':<25} {'Correlation':>12} {'N months':>10}")
print("-" * 47)

for name, mask in regimes.items():
    r_vn = vn_ret_aligned[mask]
    r_w = world_ret_aligned[mask]
    corr = r_vn.corr(r_w)
    print(f"{name:<25} {corr:>12.3f} {mask.sum():>10}")

# Test: is bear correlation > bull correlation?
r_bear_vn = vn_ret_aligned[bear]
r_bear_w = world_ret_aligned[bear]
r_bull_vn = vn_ret_aligned[bull]
r_bull_w = world_ret_aligned[bull]

# Fisher z-transformation test
def fisher_z_test(r1, n1, r2, n2):
    z1 = np.arctanh(r1)
    z2 = np.arctanh(r2)
    se = np.sqrt(1 / (n1 - 3) + 1 / (n2 - 3))
    z_stat = (z1 - z2) / se
    p_val = 2 * (1 - stats.norm.cdf(abs(z_stat)))
```

```
    return z_stat, p_val

z, p = fisher_z_test(
    r_bear_vn.corr(r_bear_w), len(r_bear_vn),
    r_bull_vn.corr(r_bull_w), len(r_bull_vn)
)
print(f"\nFisher z-test (bear vs bull): z = {z:.2f}, p = {p:.4f}")
```

86.4 Factor-Based Integration Measures

86.4.1 The Pukthuanthong-Roll R^2 Measure

Pukthuanthong and Roll (2009) propose measuring integration as the R^2 from regressing a country's returns on a set of global principal components. The intuition: if a market is fully integrated, global factors should explain all of its systematic return variation.

```
def pukthuanthong_roll_integration(country_returns, global_returns_matrix,
                                   n_components=10, rolling_window=36):
    """
    Pukthuanthong-Roll (2009) R²-based integration measure.

    1. Extract principal components from global returns.
    2. Regress country returns on these PCs.
    3. R² = degree of integration.
    """
    common = country_returns.dropna().index.intersection(
        global_returns_matrix.dropna().index
    )

    dates = sorted(common)
    T = len(dates)

    integration = []

    for t in range(rolling_window, T):
        window = dates[t - rolling_window:t]

        # Global returns in window
        G = global_returns_matrix.loc[window].dropna(axis=1)
        if G.shape[1] < n_components:
```

```
            continue

        # Standardize
        G_std = (G - G.mean()) / G.std()

        # PCA
        cov = G_std.T @ G_std / len(G_std)
        eigenvalues, eigenvectors = np.linalg.eigh(cov.values)

        # Sort descending
        idx = np.argsort(-eigenvalues)
        eigenvalues = eigenvalues[idx]
        eigenvectors = eigenvectors[:, idx]

        # Project onto top K PCs
        PCs = G_std.values @ eigenvectors[:, :n_components]

        # Regress country returns on PCs
        y = country_returns.loc[window].values
        X = sm.add_constant(PCs)

        try:
            model = sm.OLS(y, X).fit()
            integration.append({
                'date': dates[t],
                'r_squared': model.rsquared,
                'adj_r_squared': model.rsquared_adj,
                'var_explained_pc1': eigenvalues[0] / eigenvalues.sum(),
                'n_countries': G.shape[1]
            })
        except Exception:
            pass

    return pd.DataFrame(integration)

# Build global returns matrix from multiple country indices
global_matrix = global_indices.drop(columns=['VIETNAM'], errors='ignore')

pr_result = pukthuanthong_roll_integration(
    indices_aligned['VIETNAM'],
    global_matrix,
    n_components=5,
    rolling_window=36
```

```
)

if len(pr_result) > 0:
    pr_result['date'] = pd.to_datetime(pr_result['date'])
    print(f"Pukthuanthong-Roll Integration (R²):")
    print(f"  Mean: {pr_result['r_squared'].mean():.3f}")
    print(f"  2008-2012: {pr_result[(pr_result['date'] >= '2008') &
     ↪  (pr_result['date'] < '2013')]['r_squared'].mean():.3f}")
    print(f"  2013-2018: {pr_result[(pr_result['date'] >= '2013') &
     ↪  (pr_result['date'] < '2019')]['r_squared'].mean():.3f}")
    print(f"  2019-2024: {pr_result[pr_result['date'] >=
     ↪  '2019']['r_squared'].mean():.3f}")
```

86.4.2 Global vs. Local Factor Pricing

Griffin (2002) tests whether global or local versions of the Fama-French factors better explain country-level returns. We implement this horse race for Vietnam:

```
# Align global and local factors
common_factor_dates = (
    global_factors.index
    .intersection(local_factors.index)
    .intersection(vn_index.index)
)

vn_excess = vn_index.loc[common_factor_dates, 'return']

# Model 1: Global FF5
X_global = sm.add_constant(
    global_factors.loc[common_factor_dates,
                       ['mkt_excess_world', 'smb_world', 'hml_world',
                        'rmw_world', 'cma_world']]
)
model_global = sm.OLS(vn_excess, X_global).fit(
    cov_type='HAC', cov_kwds={'maxlags': 6}
)

# Model 2: Local FF5
X_local = sm.add_constant(
    local_factors.loc[common_factor_dates,
                      ['mkt_excess', 'smb', 'hml', 'rmw', 'cma']]
```

```
)
model_local = sm.OLS(vn_excess, X_local).fit(
    cov_type='HAC', cov_kwds={'maxlags': 6}
)

# Model 3: Both global and local
X_both = sm.add_constant(pd.concat([
    global_factors.loc[common_factor_dates,
                       ['mkt_excess_world', 'smb_world', 'hml_world']],
    local_factors.loc[common_factor_dates,
                      ['mkt_excess', 'smb', 'hml']]
], axis=1))
model_both = sm.OLS(vn_excess, X_both).fit(
    cov_type='HAC', cov_kwds={'maxlags': 6}
)

print("Global vs Local Factor Models for VN-Index:")
print(f"{'Model':<20} {'R²':>8} {'Adj R²':>8} {'  (ann)':>10} {'
 ↪  t-stat':>10}")
print("-" * 56)
for name, mod in [('Global FF5', model_global),
                  ('Local FF5', model_local),
                  ('Global + Local', model_both)]:
    print(f"{name:<20} {mod.rsquared:>8.3f} {mod.rsquared_adj:>8.3f} "
          f"{mod.params['const']*12:>10.4f} {mod.tvalues['const']:>10.2f}")
```

86.5 Pricing-Error-Based Integration

86.5.1 The Bekaert-Harvey Approach

Bekaert and Harvey (1995) measure integration as the ability of a global CAPM to price local assets. Under integration, the local market alpha (intercept) in a regression on the global market should be zero, and the global risk premium should explain the local expected return. Under segmentation, the local alpha captures the segmentation premium.

```
def bekaert_harvey_integration(local_return, global_return,
                               rolling_window=36):
    """
    Rolling alpha from regressing local on global market.
    Under integration, alpha -> 0.
```

```
rolling_r2 = []
rw = 36

for t in range(rw, len(common_factor_dates)):
    window = common_factor_dates[t - rw:t]
    y = vn_excess[window]

    # Global
    X_g = sm.add_constant(global_factors.loc[window,
                          ['mkt_excess_world', 'smb_world', 'hml_world',
                           'rmw_world', 'cma_world']])
    try:
        r2_g = sm.OLS(y, X_g).fit().rsquared
    except:
        r2_g = np.nan

    # Local
    X_l = sm.add_constant(local_factors.loc[window,
                          ['mkt_excess', 'smb', 'hml', 'rmw', 'cma']])
    try:
        r2_l = sm.OLS(y, X_l).fit().rsquared
    except:
        r2_l = np.nan

    rolling_r2.append({
        'date': common_factor_dates[t],
        'r2_global': r2_g,
        'r2_local': r2_l,
        'ratio': r2_g / r2_l if r2_l > 0 else np.nan
    })

r2_df = pd.DataFrame(rolling_r2)

fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True)

axes[0].plot(r2_df['date'], r2_df['r2_global'], color='#2C5F8A',
             linewidth=1.5, label='Global FF5')
axes[0].plot(r2_df['date'], r2_df['r2_local'], color='#C0392B',
             linewidth=1.5, label='Local FF5')
axes[0].set_ylabel('R²')
axes[0].set_title('Panel A: Global vs Local Factor R²')
axes[0].legend()

axes[1].plot(r2_df['date'], r2_df['ratio'], color='#27AE60', linewidth=1.5)
axes[1].axhline(y=1, color='gray', linewidth=1, linestyle='--',
                label='Full integration (ratio = 1)')
axes[1].set_ylabel('Global R² / Local R²')
axes[1].set_title('Panel B: Integration Ratio')
axes[1].legend()
axes[1].set_ylim([0, 1.5])

plt.tight_layout()
plt.show()
```

```
    """
    common =
↪   local_return.dropna().index.intersection(global_return.dropna().index)

    results = []
    for t in range(rolling_window, len(common)):
        window = common[t - rolling_window:t]
        y = local_return[window]
        X = sm.add_constant(global_return[window])

        model = sm.OLS(y, X).fit()

        results.append({
            'date': common[t],
            'alpha': model.params['const'],
            'alpha_t': model.tvalues['const'],
            'beta_global': model.params.iloc[1],
            'r_squared': model.rsquared
        })

    return pd.DataFrame(results)

bh_result = bekaert_harvey_integration(
    indices_aligned['VIETNAM'],
    indices_aligned['MSCI_WORLD'],
    rolling_window=36
)

# The absolute alpha is the segmentation premium
bh_result['abs_alpha_ann'] = bh_result['alpha'].abs() * 12

print("Bekaert-Harvey Integration Diagnostic:")
print(f"  Mean | | (ann.): {bh_result['abs_alpha_ann'].mean():.4f}")
print(f"  Mean  _world: {bh_result['beta_global'].mean():.3f}")
print(f"  Mean R²: {bh_result['r_squared'].mean():.3f}")
```

86.5.2 Composite Integration Index

We combine all measures into a single composite index of Vietnamese market integration:

```
# Standardize each measure to [0, 1] using historical percentile ranks
measures = pd.DataFrame(index=bh_result['date'])

# 1. DCC correlation (higher = more integrated)
dcc_aligned = dcc_result['dcc_corr'].reindex(measures.index).interpolate()
measures['dcc_corr'] = dcc_aligned

# 2. PR R² (higher = more integrated)
pr_aligned = pr_result.set_index('date')['r_squared'].reindex(measures.index
↪ ).interpolate()
measures['pr_r2'] = pr_aligned

# 3. Global/Local R² ratio (higher = more integrated)
r2_aligned =
↪ r2_df.set_index('date')['ratio'].reindex(measures.index).interpolate()
measures['gl_ratio'] = r2_aligned

# 4. |Alpha| from global CAPM (lower = more integrated)
# Invert: 1 - percentile rank of |alpha|
measures['inv_alpha'] = bh_result.set_index('date')['abs_alpha_ann']
measures['inv_alpha'] = 1 - measures['inv_alpha'].rank(pct=True)

# 5. Global beta (higher = more integrated, up to a point)
measures['global_beta'] = bh_result.set_index('date')['beta_global']

# Standardize to percentile ranks
for col in ['dcc_corr', 'pr_r2', 'gl_ratio', 'inv_alpha', 'global_beta']:
    measures[f'{col}_rank'] = measures[col].rank(pct=True)

# Composite = equal-weighted average of ranks
rank_cols = [c for c in measures.columns if c.endswith('_rank')]
measures['composite'] = measures[rank_cols].mean(axis=1)

# Smooth with 6-month moving average
measures['composite_smooth'] = measures['composite'].rolling(6).mean()
```

```
fig, ax = plt.subplots(figsize=(14, 6))

ax.fill_between(measures.index, measures['composite_smooth'],
                alpha=0.3, color='#2C5F8A')
ax.plot(measures.index, measures['composite_smooth'],
        color='#2C5F8A', linewidth=2)

# Add event markers
for _, event in liberalization_events.iterrows():
    if event['date'] >= measures.index.min():
        color = event_colors.get(event['category'], 'gray')
        ax.axvline(x=event['date'], color=color,
                   linewidth=1.5, linestyle='--', alpha=0.6)

ax.set_ylabel('Integration Index (0 = segmented, 1 = integrated)')
ax.set_title('Vietnam Equity Market Integration: Composite Index')
ax.set_ylim([0, 1])

# Legend for event categories
from matplotlib.patches import Patch
legend_patches = [Patch(facecolor=c, label=cat)
                  for cat, c in event_colors.items() if cat in
                  liberalization_events['category'].values]
ax.legend(handles=legend_patches, fontsize=7, loc='lower right', ncol=2)

plt.tight_layout()
plt.show()
```

Figure 86.4

86.6 Structural Break Detection

86.6.1 Bai-Perron Tests for Integration Regime Shifts

We test whether Vietnam's integration trajectory contains discrete structural breaks (i.e., sudden shifts in the integration level) rather than a smooth trend:

```
def detect_breaks_cusum(series, significance=0.05):
    """
    CUSUM-based structural break detection.
    """
    y = series.dropna().values
    T = len(y)

    # Recursive residuals from rolling mean
    cumsum = np.cumsum(y - y.mean()) / (y.std() * np.sqrt(T))

    # Brown-Durbin-Evans critical values (approximate)
    # At 5%: ±0.948
    critical = 0.948

    breaks = []
    for t in range(1, T - 1):
        if abs(cumsum[t]) > critical:
            breaks.append(t)

    return cumsum, breaks

# Apply to composite index
composite_clean = measures['composite_smooth'].dropna()
cusum, break_points = detect_breaks_cusum(composite_clean)

# Alternative: Chow test at key liberalization dates
def chow_test(y, breakpoint_idx):
    """Simple Chow test for structural break."""
    T = len(y)
    y1 = y[:breakpoint_idx]
    y2 = y[breakpoint_idx:]

    # Full sample regression (on constant)
    rss_full = np.sum((y - y.mean()) ** 2)

    # Split samples
```

```
    rss1 = np.sum((y1 - y1.mean()) ** 2)
    rss2 = np.sum((y2 - y2.mean()) ** 2)
    rss_split = rss1 + rss2

    k = 1  # Number of parameters
    f_stat = ((rss_full - rss_split) / k) / (rss_split / (T - 2 * k))
    p_val = 1 - stats.f.cdf(f_stat, k, T - 2 * k)

    return f_stat, p_val

print("Chow Tests for Structural Breaks at Key Dates:")
print(f"{'Event':<30} {'Date':>12} {'F-stat':>10} {'p-value':>10}")
print("-" * 62)

for _, event in liberalization_events.iterrows():
    if event['date'] < composite_clean.index.min():
        continue
    # Find nearest date
    nearest = composite_clean.index.searchsorted(event['date'])
    if nearest < 12 or nearest > len(composite_clean) - 12:
        continue

    f_stat, p_val = chow_test(composite_clean.values, nearest)
    sig = '***' if p_val < 0.01 else '**' if p_val < 0.05 else '*' if p_val <
↪   0.1 else ''
    print(f"{event['event']:<30} {event['date'].strftime('%Y-%m'):>12} "
          f"{f_stat:>10.2f} {p_val:>10.4f} {sig}")
```

86.7 The Segmentation Premium for Vietnam

86.7.1 Cross-Sectional Evidence

Under partial segmentation, stocks with higher foreign ownership should have lower expected returns (because foreign investors can diversify away local risk). This yields a testable prediction: foreign ownership should be negatively associated with expected returns, controlling for global risk exposure.

```
# Get monthly stock returns with foreign ownership
stock_data = client.get_monthly_returns(
    exchanges=['HOSE', 'HNX'],
```

```
    start_date='2008-01-01',
    end_date='2024-12-31',
    fields=['ticker', 'month_end', 'monthly_return', 'market_cap',
            'foreign_ownership_pct']
)
stock_data['month_end'] = pd.to_datetime(stock_data['month_end'])

# Fama-MacBeth: regress returns on lagged foreign ownership
stock_data = stock_data.sort_values(['ticker', 'month_end'])
stock_data['fol_lag'] = (
    stock_data.groupby('ticker')['foreign_ownership_pct'].shift(1)
)
stock_data['log_mcap'] = np.log(stock_data['market_cap'].clip(lower=1))

# Monthly cross-sectional regressions
gamma_fol = []
for month, group in stock_data.dropna(subset=['fol_lag',
 ↪  'monthly_return']).groupby('month_end'):
    if len(group) < 100:
        continue

    y = group['monthly_return'].values
    X = sm.add_constant(group[['fol_lag', 'log_mcap']].values)

    try:
        model = sm.OLS(y, X).fit()
        gamma_fol.append({
            'month': month,
            'gamma_fol': model.params[1],
            'gamma_size': model.params[2],
            'n': len(group)
        })
    except:
        pass

gamma_fol_df = pd.DataFrame(gamma_fol)

mean_gamma = gamma_fol_df['gamma_fol'].mean()
se_gamma = gamma_fol_df['gamma_fol'].std() / np.sqrt(len(gamma_fol_df))
t_gamma = mean_gamma / se_gamma

print(f"Fama-MacBeth: Foreign Ownership and Expected Returns")
print(f"   _FOL (monthly):  {mean_gamma:.6f}")
```

```
print(f"   _FOL (ann.):     {mean_gamma * 12:.4f}")
print(f"  t-statistic:      {t_gamma:.2f}")
print(f"  Interpretation:   A 10pp increase in foreign ownership is "
      f"associated with a {mean_gamma * 12 * 10:.2f}% change in "
      f"annual expected returns")
```

86.8 Exchange Rate Risk and Integration

86.8.1 Is Currency Risk Priced?

In a partially integrated market, exchange rate risk may carry a separate premium. Jorion (1991) and Dumas and Solnik (1995) test whether currency exposure is priced beyond global equity risk. For Vietnam, the VND/USD exchange rate is managed (a crawling peg with occasional step devaluations), creating a specific risk that is neither fully diversifiable nor fully priced by global equity factors.

```
# VND depreciation
fx_return = fx['rate'].pct_change()
fx_return.name = 'fx_return'

# Merge with stock data
stock_fx = stock_data.merge(
    fx_return.to_frame().reset_index().rename(columns={'date': 'month_end'}),
    on='month_end', how='left'
)

# Estimate FX beta for each stock (rolling 60-month)
# Then test in Fama-MacBeth whether FX beta is priced
fx_betas = {}
for ticker, group in stock_fx.groupby('ticker'):
    if len(group) < 60:
        continue
    group = group.sort_values('month_end')
    y = group['monthly_return']
    x = group[['fx_return']].dropna()
    common = y.dropna().index.intersection(x.index)
    if len(common) < 48:
        continue
    model = sm.OLS(y[common], sm.add_constant(x.loc[common])).fit()
    fx_betas[ticker] = model.params.get('fx_return', np.nan)
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Quintile returns by foreign ownership
fol_quintiles = stock_data.dropna(subset=['fol_lag',
 ↪  'monthly_return']).copy()
fol_quintiles['fol_q'] = (
    fol_quintiles.groupby('month_end')['fol_lag']
    .transform(lambda x: pd.qcut(x.rank(method='first'), 5,
                                 labels=['Q1\n(Low FOL)', 'Q2', 'Q3',
 ↪  'Q4',
                                         'Q5\n(High FOL)']))
)

q_returns = (
    fol_quintiles.groupby('fol_q')['monthly_return']
    .mean() * 12 * 100
)

colors_q = plt.cm.RdYlGn_r(np.linspace(0.2, 0.8, 5))
axes[0].bar(range(5), q_returns.values, color=colors_q,
            edgecolor='white', alpha=0.85)
axes[0].set_xticks(range(5))
axes[0].set_xticklabels(q_returns.index)
axes[0].set_ylabel('Ann. Return (%)')
axes[0].set_title('Panel A: Returns by Foreign Ownership Quintile')
axes[0].axhline(y=0, color='black', linewidth=0.5)

# Panel B: Rolling FM coefficient
gamma_fol_df['date'] = pd.to_datetime(gamma_fol_df['month'])
rolling_gamma =
 ↪  gamma_fol_df.set_index('date')['gamma_fol'].rolling(24).mean() * 12

axes[1].plot(rolling_gamma.index, rolling_gamma.values,
             color='#2C5F8A', linewidth=1.5)
axes[1].axhline(y=0, color='black', linewidth=0.5)
axes[1].set_ylabel(' _FOL (annualized)')
axes[1].set_title('Panel B: Rolling Segmentation Premium')
axes[1].fill_between(rolling_gamma.index, rolling_gamma.values, 0,
                     where=rolling_gamma.values < 0,
                     alpha=0.3, color='#27AE60', label='Negative
 ↪  (integration)')
axes[1].fill_between(rolling_gamma.index, rolling_gamma.values, 0,
                     where=rolling_gamma.values >= 0,
                     alpha=0.3, color='#C0392B', label='Positive
 ↪  (segmentation)')
axes[1].legend(fontsize=8)
```



```
plt.tight_layout()
plt.show()
```

Figure 86.5

```
fx_beta_series = pd.Series(fx_betas, name='fx_beta')

# Cross-sectional test: do stocks with higher FX beta earn different returns?
stock_fx_beta = stock_fx.merge(
    fx_beta_series.to_frame().reset_index().rename(columns={'index':
 ↪  'ticker'}),
    on='ticker', how='left'
)

# Quintile sort on FX beta
fx_q = stock_fx_beta.dropna(subset=['fx_beta', 'monthly_return'])
fx_q['fx_quintile'] = pd.qcut(fx_q['fx_beta'].rank(method='first'),
                              5, labels=False)

fx_premium = fx_q.groupby('fx_quintile')['monthly_return'].mean() * 12
print("Returns by FX Beta Quintile:")
for q, ret in fx_premium.items():
    print(f"  Q{q+1}: {ret*100:.2f}% ann.")
print(f"  Q5-Q1: {(fx_premium.iloc[-1] - fx_premium.iloc[0])*100:.2f}% ann.")
```

86.9 Contagion vs. Interdependence

During global crises, correlations between Vietnam and world markets spike. The question is whether this represents *contagion* (a structural change in the transmission mechanism) or simply *interdependence* (normal co-movement amplified by higher volatility). Longin and Solnik (2001) show that correlation increases mechanically with volatility even without any change in the underlying dependence structure.

```
def forbes_rigobon_test(r_local, r_global, crisis_dates, tranquil_dates):
    """
    Forbes-Rigobon (2002) contagion test.
    Adjusts for heteroskedasticity-induced bias in correlation.

    H0: No contagion (correlation increase is explained by volatility)
    H1: Contagion (correlation increase exceeds what volatility explains)
    """
    r_l_crisis = r_local[crisis_dates]
    r_g_crisis = r_global[crisis_dates]
    r_l_tranquil = r_local[tranquil_dates]
```

```
    r_g_tranquil = r_global[tranquil_dates]

    # Unadjusted correlations
    rho_crisis = r_l_crisis.corr(r_g_crisis)
    rho_tranquil = r_l_tranquil.corr(r_g_tranquil)

    # Volatility ratio
    delta = r_g_crisis.var() / r_g_tranquil.var() - 1

    # Adjusted correlation
    rho_adj = rho_crisis / np.sqrt(1 + delta * (1 - rho_crisis ** 2))

    # Fisher z-test on adjusted vs tranquil
    z_adj = np.arctanh(rho_adj)
    z_tranquil = np.arctanh(rho_tranquil)
    se = np.sqrt(1 / (len(r_l_crisis) - 3) + 1 / (len(r_l_tranquil) - 3))
    z_stat = (z_adj - z_tranquil) / se
    p_val = 2 * (1 - stats.norm.cdf(abs(z_stat)))

    return {
        'rho_crisis_raw': rho_crisis,
        'rho_crisis_adj': rho_adj,
        'rho_tranquil': rho_tranquil,
        'delta': delta,
        'z_stat': z_stat,
        'p_value': p_val,
        'contagion': p_val < 0.05
    }

# Define crisis and tranquil periods
crises = {
    'GFC (2008-09)': (pd.Timestamp('2008-09-01'),
     ↪  pd.Timestamp('2009-03-31')),
    'European Debt (2011-12)': (pd.Timestamp('2011-06-01'),
     ↪  pd.Timestamp('2012-06-30')),
    'COVID (2020)': (pd.Timestamp('2020-02-01'), pd.Timestamp('2020-06-30')),
    'Fed Tightening (2022)': (pd.Timestamp('2022-01-01'),
     ↪  pd.Timestamp('2022-12-31')),
}

# Tranquil = 24 months before each crisis
vn_aligned = indices_aligned['VIETNAM']
world_aligned = indices_aligned['MSCI_WORLD']
```

```
print("Contagion Tests (Forbes-Rigobon):")
print(f"{'Crisis':<28} {' (raw)':>8} {' (adj)':>8} {' (calm)':>8} "
      f"{'z-stat':>8} {'p-val':>8} {'Result':>12}")
print("-" * 80)

for name, (start, end) in crises.items():
    crisis_mask = (vn_aligned.index >= start) & (vn_aligned.index <= end)
    tranquil_start = start - pd.DateOffset(months=24)
    tranquil_mask = ((vn_aligned.index >= tranquil_start) &
                     (vn_aligned.index < start))

    crisis_dates = vn_aligned.index[crisis_mask]
    tranquil_dates = vn_aligned.index[tranquil_mask]

    if len(crisis_dates) < 3 or len(tranquil_dates) < 12:
        continue

    result = forbes_rigobon_test(vn_aligned, world_aligned,
                                 crisis_dates, tranquil_dates)

    verdict = 'CONTAGION' if result['contagion'] else 'Interdependence'
    print(f"{name:<28} {result['rho_crisis_raw']:>8.3f} "
          f"{result['rho_crisis_adj']:>8.3f} {result['rho_tranquil']:>8.3f} "
          f"{result['z_stat']:>8.2f} {result['p_value']:>8.3f} "
          f"{verdict:>12}")
```

86.10 ASEAN Peer Comparison

Vietnam's integration trajectory is best understood in the context of its ASEAN peers, which share similar starting conditions but have followed different liberalization paths:

86.11 Practical Implications

The degree of integration determines which asset pricing model is appropriate for Vietnamese equities. The evidence in this chapter supports several practical conclusions:

Vietnam is partially integrated and trending toward integration. The composite index shows a clear upward trajectory, with the post-2015 period representing the highest

```
fig, ax = plt.subplots(figsize=(14, 6))

asean_markets = {
    'VIETNAM': '#C0392B',
    'MSCI_THAILAND': '#2C5F8A',
    'MSCI_INDONESIA': '#27AE60',
    'MSCI_PHILIPPINES': '#E67E22',
    'MSCI_MALAYSIA': '#8E44AD'
}

for market, color in asean_markets.items():
    if market not in global_indices.columns:
        continue

    corr = (
        global_indices[['MSCI_WORLD', market]]
        .rolling(36)
        .corr()
        .unstack()['MSCI_WORLD'][market]
    )

    label = market.replace('MSCI_', '').replace('_', ' ').title()
    if market == 'VIETNAM':
        ax.plot(corr.index, corr.values, color=color,
                linewidth=2.5, label=label, zorder=5)
    else:
        ax.plot(corr.index, corr.values, color=color,
                linewidth=1.5, label=label, alpha=0.7)

ax.set_ylabel('Correlation with MSCI World')
ax.set_title('ASEAN Market Integration: Rolling 36-Month Correlation')
ax.legend(fontsize=9)
ax.set_ylim([-0.2, 0.9])
ax.axhline(y=0, color='black', linewidth=0.5)

plt.tight_layout()
plt.show()
```

Figure 86.6

sustained integration in the market's history. However, Vietnam remains less integrated than Thailand or Malaysia, and far from fully integrated with global markets.

Local factors dominate global factors for pricing Vietnamese stocks. The rolling R^2 comparison shows that local Vietnamese factors consistently explain more return variation than global factors. This means that researchers studying Vietnamese cross-sectional returns should use local factor models (Vietnamese FF5) rather than global factors. Global factors are useful primarily for international investors assessing co-movement risk.

The segmentation premium is shrinking but not zero. The Fama-MacBeth evidence shows that stocks with higher foreign ownership earn lower returns, consistent with partial segmentation. The magnitude has declined over time as foreign ownership limits have been relaxed, but a residual premium persists—likely driven by remaining ownership caps in banking and strategic sectors.

Crisis-period co-movement is mostly interdependence, not contagion. The Forbes-Rigobon adjusted correlations show that the spike in Vietnam-World correlation during crises is largely explained by increased global volatility, not a structural change in the transmission mechanism. This is reassuring for diversification: Vietnam continues to offer meaningful diversification benefits even during global stress.

The FTSE/MSCI upgrade path matters. Vietnam's potential upgrade from frontier to emerging market status would trigger mandatory index rebalancing by passive funds, increasing foreign flows and likely accelerating integration. Researchers and investors should monitor upgrade criteria and their implications for the cost of capital.

86.12 Summary

Table 86.1: Summary of integration measures for the Vietnamese equity market.

Measure	What It Captures	Vietnam Range	Current Level	Trend
DCC correlation (World)	Co-movement	0.0–0.5	~0.35–0.45	Rising
PR R^2 (global PCs)	Global factor exposure	0.05–0.50	~0.30–0.40	Rising
Global/Local R^2 ratio	Relative pricing power	0.1–0.8	~0.5–0.6	Rising
Global CAPM	Pricing error	0–15% ann.	~3–5% ann.	Falling
FOL premium ( )	Segmentation cost	-5% to +2%	~-1% to 0%	Shrinking

87 Exchange Rate Dynamics

> **Note**
>
> In this chapter, we construct a comprehensive exchange rate database for the Vietnamese Dong (VND) against major currencies, analyze the statistical properties of VND returns, test covered and uncovered interest rate parity, estimate the currency exposure of Vietnamese listed firms, model volatility regimes using GARCH, and evaluate hedging strategies for international investors.

Exchange rates sit at the intersection of macroeconomics and finance. For international investors considering Vietnamese equities, the VND/USD exchange rate is not a secondary concern; it is a first-order determinant of dollar-denominated returns. A Vietnamese stock portfolio that earns 15% in VND terms but coincides with a 5% VND depreciation delivers only about 10% in USD terms. Conversely, periods of VND stability or appreciation amplify returns for foreign investors. Understanding exchange rate dynamics is therefore essential for anyone working with Vietnamese financial data in an international context.

Vietnam presents a distinctive exchange rate regime. Unlike freely floating currencies such as the USD/EUR or USD/JPY, the VND operates under a managed float. The State Bank of Vietnam (SBV) sets a daily reference rate and allows trading within a band (currently $\pm$ 3% on the interbank market and $\pm$ 5% against the official rate for commercial banks). This regime creates specific patterns in VND returns that differ fundamentally from those observed in freely floating pairs: clustered movements near band boundaries, discrete adjustments when the SBV shifts the reference rate, and periods of near-zero volatility punctuated by sharp moves during policy changes.

This chapter develops tools for working with these dynamics. We progress from data construction through statistical characterization to economic applications: interest rate parity tests, firm-level exposure estimation, GARCH volatility modeling, and hedging strategy evaluation.

87.1 Theoretical Foundations

87.1.1 Exchange Rate Determination

The modern theory of exchange rate determination rests on several building blocks. Dornbusch (1976) provides the canonical "overshooting" model: because goods prices are sticky while asset prices adjust instantly, a monetary shock causes the exchange rate to overshoot its long-run equilibrium. J. A. Frankel (1979) extends this to a real interest differential model where the exchange rate depends on relative money supplies, income levels, and interest rates across countries.

Despite elegant theory, Meese and Rogoff (1983) delivered a devastating empirical result: no structural model of exchange rates consistently outperforms a random walk in out-of-sample forecasting. This finding, confirmed repeatedly over four decades and surveyed by B. Rossi (2013), means that exchange rate changes are, to a first approximation, unpredictable. For practical purposes, the best forecast of tomorrow's VND/USD rate is today's rate.

87.1.2 Interest Rate Parity

Two parity conditions link exchange rates to interest rates:

Covered Interest Rate Parity (CIP): Arbitrage between spot and forward markets should equalize the forward premium with the interest rate differential:

$$F_{t,t+k} = S_t \cdot \frac{(1 + r_{t,k}^{VND})}{(1 + r_{t,k}^{USD})} \tag{87.1}$$

where S_t is the spot rate (VND per USD), $F_{t,t+k}$ is the k-period forward rate, and $r_{t,k}^{VND}$ and $r_{t,k}^{USD}$ are the domestic and foreign interest rates for maturity k. CIP should hold exactly in the absence of transaction costs, credit risk, and capital controls. Du, Tepper, and Verdelhan (2018) document that CIP violations have widened significantly in major currency pairs since 2008, driven by post-crisis bank balance sheet constraints. In Vietnam, capital controls and limited forward market liquidity create additional CIP deviations.

Uncovered Interest Rate Parity (UIP): The expected depreciation should equal the interest rate differential:

$$\mathbb{E}_t[\Delta s_{t+k}] = r_{t,k}^{VND} - r_{t,k}^{USD} \tag{87.2}$$

where $\Delta s_{t+k} = \ln(S_{t+k}/S_t)$. UIP is an equilibrium condition, not an arbitrage condition, it requires risk-neutral investors. Eugene F. Fama (1984) famously showed that the forward premium predicts exchange rate changes with the *wrong sign*: high-interest-rate currencies tend

to appreciate rather than depreciate, contradicting UIP. This "forward premium anomaly" is the foundation of the carry trade.

87.1.3 The Carry Trade

The carry trade exploits UIP violations by borrowing in low-interest-rate currencies and investing in high-interest-rate currencies. Lustig and Verdelhan (2007) show that a portfolio of carry trade positions earns a significant risk premium, which they link to consumption growth risk. Menkhoff et al. (2012) identify global FX volatility as the key risk factor: carry trades lose money precisely when global volatility spikes, a "crash risk" documented by Markus K. Brunnermeier, Nagel, and Pedersen (2008).

Vietnam, with interest rates typically 3-8 percentage points above U.S. rates, is a natural candidate for carry trade inflows. The managed exchange rate regime provides an additional attraction: the SBV's implicit defense of the band reduces short-term volatility, increasing the Sharpe ratio of the carry position, until a band adjustment or devaluation erases accumulated gains in a single move.

87.1.4 The Trilemma

Obstfeld, Shambaugh, and Taylor (2005) formalize the "impossible trinity": a country cannot simultaneously maintain (i) a fixed exchange rate, (ii) an independent monetary policy, and (iii) free capital mobility. Vietnam navigates this trilemma by maintaining partial capital controls alongside its managed float, allowing some degree of monetary policy independence. Understanding where Vietnam sits on the trilemma at any given time is essential for interpreting exchange rate data: periods of tighter capital controls produce artificially low exchange rate volatility that does not reflect true economic uncertainty.

87.2 The VND Exchange Rate Regime

87.2.1 Historical Evolution

The VND/USD exchange rate has evolved through several distinct phases:

Table 87.1: Evolution of the VND/USD exchange rate regime.

Period	Regime	Key Features
Pre-1999	Narrow band	SBV set the rate with minimal flexibility

Period	Regime	Key Features
1999-2007	Gradual crawl	Slow, controlled depreciation (~1-2% per year)
2008-2011	Crisis adjustments	Multiple discrete devaluations (2008, 2010, 2011)
2012-2015	Stabilization	Tight band, rare adjustments, building reserves
2016-present	Reference rate mechanism	Daily reference rate based on a basket; $\pm$ 3% band

The 2016 reform, shifting from a fixed peg to a daily reference rate influenced by a currency basket, was a significant structural change. The reference rate now incorporates the previous day's closing interbank rate, supply-demand conditions, and movements in currencies of major trading partners (USD, EUR, CNY, JPY, KRW).

87.2.2 Band Mechanics

The SBV announces a daily reference rate each morning. Commercial banks may quote rates within a band around this reference. The interbank market operates with a narrower $\pm$ 3% band. When the spot rate approaches the band ceiling, the SBV may intervene by selling USD from reserves, widening the band, or adjusting the reference rate.

This regime creates distinctive statistical properties:

1. **Return clustering near zero:** On most days, the VND/USD rate barely moves, producing a spike at zero in the return distribution.
2. **Discrete jumps:** Reference rate adjustments create large single-day moves that appear as outliers.
3. **Bounded volatility:** The band constrains daily moves, producing a return distribution with thinner tails than a freely floating currency.
4. **Asymmetric adjustment:** Depreciation pressure (VND weakening) is more common than appreciation, reflecting persistent inflation differentials.

87.3 Data Construction

87.3.1 Loading Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
from arch import arch_model
from linearmodels.panel import PanelOLS
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.figsize': (12, 6),
    'figure.dpi': 150,
    'font.size': 11,
    'axes.spines.top': False,
    'axes.spines.right': False
})
```

87.3.2 Retrieving Exchange Rate Data

We extract daily spot rates for the VND against all major currencies, along with the SBV reference rate and forward rates where available.

```
from datacore import DataCoreClient

client = DataCoreClient()

# Spot exchange rates: VND per unit of foreign currency
fx_spot = client.get_exchange_rates(
    base_currency='VND',
    quote_currencies=['USD', 'EUR', 'JPY', 'CNY', 'KRW',
                      'SGD', 'THB', 'GBP', 'AUD'],
    start_date='2005-01-01',
    end_date='2024-12-31',
```

```
    fields=['date', 'from_currency', 'to_currency', 'rate',
            'bid', 'ask', 'reference_rate']
)

# Forward rates (VND/USD)
fx_forward = client.get_fx_forwards(
    pair='VND/USD',
    start_date='2010-01-01',
    end_date='2024-12-31',
    tenors=['1M', '3M', '6M', '12M'],
    fields=['date', 'tenor', 'forward_rate', 'forward_points']
)

# Interest rates for parity tests
interest_rates = client.get_interest_rates(
    countries=['VN', 'US'],
    start_date='2005-01-01',
    end_date='2024-12-31',
    instruments=['interbank_overnight', 'deposit_3m',
                 'government_bond_1y', 'government_bond_10y']
)

print(f"Spot observations: {fx_spot.shape[0]:,}")
print(f"Forward observations: {fx_forward.shape[0]:,}")
print(f"Interest rate observations: {interest_rates.shape[0]:,}")
```

87.3.3 Constructing the VND/USD Time Series

We focus primarily on the VND/USD pair, the dominant bilateral rate for Vietnam, while using other pairs for cross-rate analysis.

```
# Filter VND/USD
vnd_usd = fx_spot[
    (fx_spot['from_currency'] == 'VND') &
    (fx_spot['to_currency'] == 'USD')
].copy()

vnd_usd['date'] = pd.to_datetime(vnd_usd['date'])
vnd_usd = vnd_usd.sort_values('date').set_index('date')

# Log returns: positive = VND depreciation, negative = VND appreciation
```

```
vnd_usd['log_return'] = np.log(vnd_usd['rate'] / vnd_usd['rate'].shift(1))

# Bid-ask spread as fraction of mid rate
vnd_usd['spread_pct'] = (vnd_usd['ask'] - vnd_usd['bid']) / vnd_usd['rate']

# Distance from reference rate (in %)
vnd_usd['deviation_from_ref'] = (
    (vnd_usd['rate'] - vnd_usd['reference_rate'])
    / vnd_usd['reference_rate'] * 100
)

# Rolling volatility measures
vnd_usd['vol_30d'] = vnd_usd['log_return'].rolling(30).std() * np.sqrt(252)
vnd_usd['vol_90d'] = vnd_usd['log_return'].rolling(90).std() * np.sqrt(252)

vnd_usd = vnd_usd.dropna(subset=['log_return'])

print(f"VND/USD series: {len(vnd_usd)} daily observations")
print(f"Date range: {vnd_usd.index.min()} to {vnd_usd.index.max()}")
print(f"\nSummary of daily log returns:")
print(vnd_usd['log_return'].describe().round(6))
```

87.3.4 Multi-Currency Panel

For cross-rate analysis and ASEAN comparisons, we construct a panel of log returns for all currency pairs.

```
# Pivot to wide format: one column per currency pair
fx_wide = fx_spot.pivot_table(
    index='date', columns='to_currency', values='rate'
)
fx_wide.index = pd.to_datetime(fx_wide.index)
fx_wide = fx_wide.sort_index()

# Compute log returns for all pairs
fx_returns = np.log(fx_wide / fx_wide.shift(1)).dropna()

# Summary statistics
fx_summary = pd.DataFrame({
    'Mean (ann.)': fx_returns.mean() * 252,
    'Vol (ann.)': fx_returns.std() * np.sqrt(252),
```

```
    'Skewness': fx_returns.skew(),
    'Kurtosis': fx_returns.kurtosis(),
    'Min': fx_returns.min(),
    'Max': fx_returns.max()
})
print("VND Cross-Rate Return Statistics (Daily):")
print(fx_summary.round(4))
```

87.4 Statistical Properties of VND Returns

87.4.1 Return Distribution

The distribution of VND/USD log returns deviates sharply from normality. The managed float regime produces a return distribution that is leptokurtic (fat-tailed) and concentrated near zero, with occasional large discrete jumps from reference rate adjustments.

87.4.2 Autocorrelation Structure

Freely floating exchange rate returns are approximately uncorrelated (consistent with the random walk result of Meese and Rogoff (1983)). Managed exchange rates, however, may exhibit serial correlation because the central bank smooths adjustments over multiple days.

87.4.3 Reference Rate Adjustments and Structural Breaks

The SBV periodically adjusts the reference rate or the trading band width, events that create structural breaks in the exchange rate series. Identifying these breaks is essential for accurate modeling.

87.4.4 Cross-Currency Correlations

Understanding how VND co-moves with other currencies is important for diversification and for identifying common drivers (e.g., USD strength affects all Asian currencies).

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Histogram with normal overlay
returns = vnd_usd['log_return']
mu, sigma = returns.mean(), returns.std()

axes[0].hist(returns, bins=100, density=True, color='#2C5F8A',
             alpha=0.7, edgecolor='white', label='Empirical')
x_range = np.linspace(returns.min(), returns.max(), 200)
axes[0].plot(x_range, stats.norm.pdf(x_range, mu, sigma),
             color='#C0392B', linewidth=2, label='Normal fit')
axes[0].set_xlabel('Daily Log Return')
axes[0].set_ylabel('Density')
axes[0].set_title('Panel A: Return Distribution')
axes[0].legend()

# Panel B: QQ plot
stats.probplot(returns, dist='norm', plot=axes[1])
axes[1].set_title('Panel B: Normal Q-Q Plot')
axes[1].get_lines()[0].set_color('#2C5F8A')
axes[1].get_lines()[0].set_markersize(3)
axes[1].get_lines()[1].set_color('#C0392B')

plt.tight_layout()
plt.show()

# Formal tests
jb_stat, jb_p = stats.jarque_bera(returns)
print(f"\nJarque-Bera test: statistic = {jb_stat:.1f}, p-value = {jb_p:.2e}")
print(f"Skewness: {returns.skew():.4f}")
print(f"Excess Kurtosis: {returns.kurtosis():.4f}")
```

Figure 87.1

```
from statsmodels.graphics.tsaplots import plot_acf

fig, axes = plt.subplots(2, 1, figsize=(12, 8))

# ACF of returns
plot_acf(vnd_usd['log_return'].dropna(), lags=40, ax=axes[0],
         color='#2C5F8A', vlines_kwargs={'color': '#2C5F8A'})
axes[0].set_title('Autocorrelation of Daily Returns')
axes[0].set_ylabel('ACF')

# ACF of squared returns (volatility clustering)
plot_acf(vnd_usd['log_return'].dropna() ** 2, lags=40, ax=axes[1],
         color='#C0392B', vlines_kwargs={'color': '#C0392B'})
axes[1].set_title('Autocorrelation of Squared Returns (Volatility
 ↪  Clustering)')
axes[1].set_ylabel('ACF')

plt.tight_layout()
plt.show()

# Ljung-Box test
from statsmodels.stats.diagnostic import acorr_ljungbox
lb_returns = acorr_ljungbox(returns, lags=10, return_df=True)
lb_squared = acorr_ljungbox(returns ** 2, lags=10, return_df=True)
print("Ljung-Box Test (Returns, lag 10):")
print(f"  Statistic: {lb_returns['lb_stat'].iloc[-1]:.2f}, "
      f"p-value: {lb_returns['lb_pvalue'].iloc[-1]:.4f}")
print("Ljung-Box Test (Squared Returns, lag 10):")
print(f"  Statistic: {lb_squared['lb_stat'].iloc[-1]:.2f}, "
      f"p-value: {lb_squared['lb_pvalue'].iloc[-1]:.4f}")
```

Figure 87.2

```
# Detect large daily moves (proxy for policy adjustments)
threshold = vnd_usd['log_return'].std() * 3
large_moves = vnd_usd[vnd_usd['log_return'].abs() > threshold].copy()

# Known major adjustment dates (approximate)
policy_events = pd.DataFrame({
    'date': pd.to_datetime([
        '2008-06-10', '2008-12-25', '2009-11-25',
        '2010-02-11', '2010-08-18', '2011-02-11',
        '2011-08-10', '2015-08-12', '2015-08-19',
        '2016-01-04'
    ]),
    'event': [
        'Band widened to +/-1%', 'Band widened to +/-3%',
        'Devaluation ~5.4%', 'Devaluation ~3.4%',
        'Band narrowed to +/-1%', 'Devaluation ~8.5%',
        'Band widened to +/-1%', 'Devaluation ~1%',
        'Devaluation ~1%', 'New reference rate mechanism'
    ]
})

fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(vnd_usd.index, vnd_usd['rate'], color='#2C5F8A',
        linewidth=1.5, label='VND/USD Spot')

if 'reference_rate' in vnd_usd.columns:
    ref_clean = vnd_usd['reference_rate'].dropna()
    if len(ref_clean) > 0:
        ax.plot(ref_clean.index, ref_clean, color='#E67E22',
                linewidth=1, alpha=0.7, label='SBV Reference Rate')

# Mark policy events
for _, event in policy_events.iterrows():
    if event['date'] >= vnd_usd.index.min():
        rate_at_event = vnd_usd['rate'].asof(event['date'])
        ax.annotate(
            event['event'],
            xy=(event['date'], rate_at_event),
            xytext=(0, 30), textcoords='offset points',
            fontsize=7, rotation=45, ha='left',
            arrowprops=dict(arrowstyle='->', color='#C0392B',
                            lw=0.8),
            color='#C0392B'
        )

ax.set_ylabel('VND per USD')
ax.set_title('VND/USD Exchange Rate with Policy Events')
ax.legend(loc='upper left')
plt.tight_layout()
plt.show()
```


Figure 87.3

```
# Correlation matrix of daily returns
corr_matrix = fx_returns.corr()

fig, ax = plt.subplots(figsize=(9, 8))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool), k=1)
sns.heatmap(
    corr_matrix, mask=mask, annot=True, fmt='.2f',
    cmap='RdBu_r', center=0, vmin=-1, vmax=1,
    square=True, linewidths=0.5, ax=ax,
    cbar_kws={'label': 'Correlation'}
)
ax.set_title('Cross-Rate Return Correlations (VND Base)')
plt.tight_layout()
plt.show()
```

Figure 87.4

87.5 Interest Rate Parity Tests

87.5.1 Testing Covered Interest Rate Parity

CIP deviations are measured as the gap between the forward premium and the interest rate differential:

$$\text{CIP Deviation}_t = f_{t,k} - s_t - (r_{t,k}^{VND} - r_{t,k}^{USD}) \tag{87.3}$$

where $f_{t,k} = \ln F_{t,t+k}$ and $s_t = \ln S_t$ are log forward and spot rates. Under exact CIP, this deviation is zero.

```
# Merge spot, forward, and interest rate data
vnd_usd_monthly = vnd_usd['rate'].resample('M').last().to_frame('spot')
vnd_usd_monthly.index = vnd_usd_monthly.index.to_period('M').to_timestamp()

# Forward rates (3-month tenor)
fwd_3m = fx_forward[fx_forward['tenor'] == '3M'].copy()
fwd_3m['date'] = pd.to_datetime(fwd_3m['date'])
fwd_3m = fwd_3m.set_index('date').resample('M').last()

# Interest rate differential
vn_rates = interest_rates[
    (interest_rates['country'] == 'VN') &
```

```
    (interest_rates['instrument'] == 'deposit_3m')
].set_index('date')['rate'].resample('M').last()

us_rates = interest_rates[
    (interest_rates['country'] == 'US') &
    (interest_rates['instrument'] == 'deposit_3m')
].set_index('date')['rate'].resample('M').last()

# Align and compute CIP deviation
cip_data = pd.DataFrame({
    'spot': vnd_usd_monthly['spot'],
    'forward': fwd_3m['forward_rate'],
    'r_vnd': vn_rates / 100,
    'r_usd': us_rates / 100
}).dropna()

# CIP deviation (annualized, in basis points)
cip_data['log_spot'] = np.log(cip_data['spot'])
cip_data['log_forward'] = np.log(cip_data['forward'])
cip_data['forward_premium'] = (
    (cip_data['log_forward'] - cip_data['log_spot']) * 4
)
cip_data['rate_diff'] = cip_data['r_vnd'] - cip_data['r_usd']
cip_data['cip_deviation'] = (
    (cip_data['forward_premium'] - cip_data['rate_diff']) * 10000
)

print("CIP Deviation Summary (basis points, annualized):")
print(cip_data['cip_deviation'].describe().round(1))
print(f"\nMean deviation: {cip_data['cip_deviation'].mean():.1f} bps")
t_stat, p_val = stats.ttest_1samp(cip_data['cip_deviation'].dropna(), 0)
print(f"One-sample t-test: t = {t_stat:.2f}, p = {p_val:.4f}")
```

87.5.2 Testing Uncovered Interest Rate Parity

The standard UIP regression is:

$$\Delta s_{t+k} = \alpha + \beta(r_{t,k}^{VND} - r_{t,k}^{USD}) + \varepsilon_{t+k} \tag{87.4}$$

Under UIP, $\alpha = 0$ and $\beta = 1$. The Eugene F. Fama (1984) anomaly corresponds to $\beta < 1$ (often $\beta < 0$), implying that high-interest-rate currencies appreciate rather than depreciate.

```
fig, axes = plt.subplots(2, 1, figsize=(14, 8), height_ratios=[2, 1])

# Panel A: CIP deviation time series
axes[0].fill_between(
    cip_data.index, 0, cip_data['cip_deviation'],
    where=cip_data['cip_deviation'] > 0,
    color='#C0392B', alpha=0.4, label='Positive (VND forward expensive)'
)
axes[0].fill_between(
    cip_data.index, 0, cip_data['cip_deviation'],
    where=cip_data['cip_deviation'] <= 0,
    color='#27AE60', alpha=0.4, label='Negative'
)
axes[0].axhline(y=0, color='black', linewidth=0.5)
axes[0].set_ylabel('CIP Deviation (bps)')
axes[0].set_title('Panel A: CIP Deviations for VND/USD')
axes[0].legend()

# Panel B: Interest rate differential
axes[1].plot(cip_data.index, cip_data['r_vnd'] * 100,
             color='#C0392B', label='Vietnam 3M')
axes[1].plot(cip_data.index, cip_data['r_usd'] * 100,
             color='#2C5F8A', label='US 3M')
axes[1].set_ylabel('Interest Rate (%)')
axes[1].set_xlabel('Date')
axes[1].set_title('Panel B: Interest Rate Differential')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Figure 87.5

```
# Monthly exchange rate changes
cip_data['delta_s'] = (
    np.log(cip_data['spot'].shift(-3) / cip_data['spot'])
)

# Fama regression
uip_data = cip_data[['delta_s', 'rate_diff']].dropna()

uip_model = sm.OLS(
    uip_data['delta_s'],
    sm.add_constant(uip_data['rate_diff'])
).fit(cov_type='HAC', cov_kwds={'maxlags': 4})

print("UIP (Fama) Regression: delta_s_{t+3m} = alpha + beta(r_VND - r_USD) +
 ↪  eps")
print(f"\nalpha = {uip_model.params['const']:.4f} "
      f"(t = {uip_model.tvalues['const']:.2f})")
print(f"beta = {uip_model.params['rate_diff']:.4f} "
      f"(t = {uip_model.tvalues['rate_diff']:.2f})")
print(f"R-squared = {uip_model.rsquared:.4f}")
print(f"\nH0: beta = 1 (Wald test):")
wald_stat = ((uip_model.params['rate_diff'] - 1) /
             uip_model.bse['rate_diff']) ** 2
print(f"  Chi2 = {wald_stat:.2f}, p = {1 - stats.chi2.cdf(wald_stat,
 ↪  1):.4f}")
```

Interpreting UIP Failures in Vietnam

A managed exchange rate complicates UIP tests. When the SBV successfully defends the VND within a narrow band, realized exchange rate changes are artificially compressed regardless of interest rate differentials. The resulting beta estimate reflects the credibility of the peg as much as it reflects the risk premium. Periods of active SBV intervention should be treated separately from periods of relative flexibility.

87.5.3 Carry Trade Returns

We construct a simple VND carry trade strategy: borrow USD at the U.S. short rate, convert to VND, invest at the Vietnamese short rate, and convert back at maturity. The excess return is:

$$\text{Carry}_{t \to t+k} = (r_{t,k}^{VND} - r_{t,k}^{USD}) - \Delta s_{t+k} \tag{87.5}$$

A positive carry return means the interest rate differential more than compensated for any VND depreciation.

```
carry = cip_data.copy()
carry['carry_return'] = carry['rate_diff'] / 4 - carry['delta_s']
carry = carry.dropna(subset=['carry_return'])

# Annualize
ann_carry = carry['carry_return'].mean() * 4
ann_vol = carry['carry_return'].std() * 2
sharpe = ann_carry / ann_vol

print("VND/USD Carry Trade Performance (3-Month Rolling):")
print(f"Annualized return: {ann_carry:.4f} ({ann_carry*100:.2f}%)")
print(f"Annualized volatility: {ann_vol:.4f} ({ann_vol*100:.2f}%)")
print(f"Sharpe ratio: {sharpe:.2f}")
print(f"Skewness: {carry['carry_return'].skew():.2f}")
print(f"Kurtosis: {carry['carry_return'].kurtosis():.2f}")
print(f"\nMax quarterly loss: {carry['carry_return'].min():.4f}")
print(f"Max quarterly gain: {carry['carry_return'].max():.4f}")
```

87.6 Volatility Modeling

87.6.1 GARCH Estimation

The strong autocorrelation in squared VND/USD returns (Figure 87.2) motivates GARCH modeling (Bollerslev 1986). We estimate a GARCH(1,1) model for daily VND/USD log returns:

$$r_t = \mu + \varepsilon_t, \quad \varepsilon_t = \sigma_t z_t, \quad z_t \sim D(0,1) \tag{87.6}$$

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2 \tag{87.7}$$

where α captures the ARCH effect (impact of yesterday's shock on today's variance) and β captures persistence (how slowly volatility reverts to its long-run mean). The unconditional variance is $\bar{\sigma}^2 = \omega/(1-\alpha-\beta)$.

```
fig, axes = plt.subplots(2, 1, figsize=(14, 8), height_ratios=[2, 1])

# Panel A: Cumulative return
cum_carry = (1 + carry['carry_return']).cumprod()
axes[0].plot(carry.index, cum_carry, color='#2C5F8A', linewidth=2)
axes[0].fill_between(carry.index, 1, cum_carry,
                     where=cum_carry > 1, color='#27AE60', alpha=0.2)
axes[0].fill_between(carry.index, 1, cum_carry,
                     where=cum_carry < 1, color='#C0392B', alpha=0.2)
axes[0].axhline(y=1, color='gray', linewidth=0.5)
axes[0].set_ylabel('Cumulative Return (VND 1 invested)')
axes[0].set_title('Panel A: VND/USD Carry Trade Cumulative Return')

# Panel B: Quarterly returns
axes[1].bar(carry.index, carry['carry_return'] * 100,
            color=['#27AE60' if x > 0 else '#C0392B'
                   for x in carry['carry_return']],
            alpha=0.7, width=60)
axes[1].axhline(y=0, color='black', linewidth=0.5)
axes[1].set_ylabel('Quarterly Return (%)')
axes[1].set_xlabel('Date')
axes[1].set_title('Panel B: Quarterly Carry Returns')

plt.tight_layout()
plt.show()
```

Figure 87.6

```
returns_bps = vnd_usd['log_return'] * 10000  # Scale to basis points

# GARCH(1,1) with normal innovations
garch_norm = arch_model(
    returns_bps, vol='Garch', p=1, q=1, dist='normal'
)
res_norm = garch_norm.fit(disp='off')

# GARCH(1,1) with Student-t innovations
garch_t = arch_model(
    returns_bps, vol='Garch', p=1, q=1, dist='t'
)
res_t = garch_t.fit(disp='off')

# GJR-GARCH (asymmetric: depreciation shocks may increase vol more)
gjr = arch_model(
    returns_bps, vol='Garch', p=1, o=1, q=1, dist='t'
)
res_gjr = gjr.fit(disp='off')

print("=" * 60)
print("GARCH(1,1) - Normal Innovations")
print("=" * 60)
print(res_norm.summary().tables[1])

print("\n" + "=" * 60)
print("GARCH(1,1) - Student-t Innovations")
print("=" * 60)
print(res_t.summary().tables[1])

print("\n" + "=" * 60)
print("GJR-GARCH(1,1,1) - Student-t Innovations")
print("=" * 60)
print(res_gjr.summary().tables[1])

# Model comparison
print("\nModel Comparison:")
print(f"{'Model':<25} {'AIC':>10} {'BIC':>10} {'LogLik':>12}")
for name, res in [('GARCH-Normal', res_norm),
                  ('GARCH-t', res_t),
                  ('GJR-GARCH-t', res_gjr)]:
    print(f"{name:<25} {res.aic:>10.1f} {res.bic:>10.1f} "
          f"{res.loglikelihood:>12.1f}")
```

```
fig, ax = plt.subplots(figsize=(14, 6))

# GARCH conditional volatility (annualized, from basis points)
cond_vol = res_t.conditional_volatility / 10000 * np.sqrt(252)

ax.plot(vnd_usd.index[-len(cond_vol):], cond_vol,
        color='#C0392B', linewidth=1.2, alpha=0.9,
        label='GARCH(1,1) Conditional Vol')
ax.plot(vnd_usd.index, vnd_usd['vol_30d'],
        color='#2C5F8A', linewidth=1, alpha=0.6,
        label='30-Day Realized Vol')

ax.set_ylabel('Annualized Volatility')
ax.set_xlabel('Date')
ax.set_title('VND/USD Volatility: GARCH vs Realized')
ax.legend()
plt.tight_layout()
plt.show()
```

Figure 87.7

87.6.2 Volatility Regime Identification

We classify the VND/USD market into volatility regimes using the GARCH conditional volatility:

```
# Merge conditional volatility back to main DataFrame
vol_series = pd.Series(
    cond_vol.values,
    index=vnd_usd.index[-len(cond_vol):]
)
vnd_usd['cond_vol'] = vol_series

# Classify regimes based on percentiles
vnd_usd['vol_regime'] = pd.cut(
    vnd_usd['cond_vol'],
    bins=[0, vnd_usd['cond_vol'].quantile(0.33),
          vnd_usd['cond_vol'].quantile(0.67), np.inf],
    labels=['Low', 'Medium', 'High']
```

```
)

# Regime statistics
regime_stats = (
    vnd_usd.groupby('vol_regime')
    .agg(
        n_days=('log_return', 'count'),
        mean_return=('log_return', lambda x: x.mean() * 252),
        volatility=('log_return', lambda x: x.std() * np.sqrt(252)),
        mean_spread=('spread_pct', 'mean'),
        skewness=('log_return', 'skew')
    )
    .round(4)
)
print("Volatility Regime Characteristics:")
print(regime_stats)
```

```
fig, ax = plt.subplots(figsize=(10, 6))

colors_regime = {'Low': '#27AE60', 'Medium': '#F1C40F', 'High': '#C0392B'}
for regime in ['Low', 'Medium', 'High']:
    subset = vnd_usd[vnd_usd['vol_regime'] == regime]
    ax.scatter(
        subset['cond_vol'] * 100,
        subset['log_return'] * 100,
        color=colors_regime[regime], alpha=0.3, s=8, label=regime
    )

ax.axhline(y=0, color='gray', linewidth=0.5)
ax.set_xlabel('Conditional Volatility (% annualized)')
ax.set_ylabel('Daily Return (%)')
ax.set_title('VND/USD Returns by Volatility Regime')
ax.legend(title='Regime')
plt.tight_layout()
plt.show()
```

Figure 87.8

87.7 Exchange Rate Exposure of Vietnamese Firms

87.7.1 The Jorion Framework

Jorion (1990) introduced the standard two-factor model for estimating firm-level exchange rate exposure:

$$R_{i,t} = \alpha_i + \beta_i^{MKT} R_{m,t} + \gamma_i \Delta s_t + \varepsilon_{i,t} \tag{87.8}$$

where $R_{i,t}$ is the stock return of firm i, $R_{m,t}$ is the market return, and Δs_t is the exchange rate change (VND/USD log return, where positive = VND depreciation). The coefficient γ_i is the **residual exchange rate exposure**, the sensitivity of firm i's returns to currency movements after controlling for market-wide effects.

A positive γ_i means the firm benefits from VND depreciation (typical for exporters), while a negative γ_i means the firm is hurt by depreciation (typical for importers or firms with USD-denominated debt).

```
# Load equity returns
equity = client.get_monthly_returns(
    exchanges=['HOSE', 'HNX'],
    start_date='2012-01-01',
    end_date='2024-12-31',
    fields=['ticker', 'month_end', 'monthly_return', 'market_cap']
)

# Market return (VW)
market_ret = (
    equity
    .groupby('month_end')
    .apply(lambda g: np.average(g['monthly_return'],
                                weights=g['market_cap']))
    .reset_index(name='market_return')
)

# Monthly VND/USD returns
fx_monthly = (
    vnd_usd['log_return']
    .resample('M').sum()
    .to_frame('fx_return')
)
fx_monthly.index = fx_monthly.index.to_period('M').to_timestamp()
```

```
# Merge all data
exposure_data = (
    equity
    .merge(market_ret, on='month_end')
    .merge(fx_monthly, left_on='month_end', right_index=True, how='inner')
)

# Estimate exposure for each firm
def estimate_exposure(group, min_obs=36):
    """Estimate Jorion exposure model for a single firm."""
    if len(group) < min_obs:
        return None

    y = group['monthly_return']
    X = sm.add_constant(group[['market_return', 'fx_return']])

    try:
        model = sm.OLS(y, X).fit(cov_type='HC1')
        return pd.Series({
            'gamma': model.params['fx_return'],
            'gamma_se': model.bse['fx_return'],
            'gamma_t': model.tvalues['fx_return'],
            'gamma_p': model.pvalues['fx_return'],
            'beta_mkt': model.params['market_return'],
            'r_squared': model.rsquared,
            'n_obs': model.nobs
        })
    except Exception:
        return None

exposures = (
    exposure_data
    .groupby('ticker')
    .apply(estimate_exposure)
    .dropna()
)

print(f"Estimated exposure for {len(exposures)} firms")
print(f"\nExposure (gamma) Summary:")
print(exposures['gamma'].describe().round(4))
print(f"\nSignificant at 5%: "
      f"{(exposures['gamma_p'] < 0.05).sum()} / {len(exposures)} "
```

```
      f"({(exposures['gamma_p'] < 0.05).mean():.1%})")
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Distribution of gamma
sig = exposures['gamma_p'] < 0.05
axes[0].hist(exposures.loc[~sig, 'gamma'], bins=40, color='#BDC3C7',
             alpha=0.7, label='Insignificant', edgecolor='white')
axes[0].hist(exposures.loc[sig & (exposures['gamma'] > 0), 'gamma'],
             bins=20, color='#27AE60', alpha=0.8,
             label='Significant positive', edgecolor='white')
axes[0].hist(exposures.loc[sig & (exposures['gamma'] < 0), 'gamma'],
             bins=20, color='#C0392B', alpha=0.8,
             label='Significant negative', edgecolor='white')
axes[0].axvline(x=0, color='black', linewidth=0.8)
axes[0].set_xlabel('Exchange Rate Exposure (gamma)')
axes[0].set_ylabel('Number of Firms')
axes[0].set_title('Panel A: Cross-Sectional Distribution')
axes[0].legend(fontsize=9)

# Panel B: Exposure vs market beta
axes[1].scatter(exposures['beta_mkt'], exposures['gamma'],
                c=exposures['gamma_t'].clip(-3, 3),
                cmap='RdBu_r', s=15, alpha=0.6)
axes[1].axhline(y=0, color='gray', linewidth=0.5)
axes[1].axvline(x=1, color='gray', linewidth=0.5, linestyle='--')
axes[1].set_xlabel('Market Beta')
axes[1].set_ylabel('FX Exposure (gamma)')
axes[1].set_title('Panel B: Market Beta vs FX Exposure')

plt.tight_layout()
plt.show()
```

Figure 87.9

87.7.2 Industry-Level Exposure

Exchange rate exposure varies systematically across industries. Export-oriented sectors (textiles, seafood, electronics) should have positive exposure (benefiting from VND weakness),

while import-dependent sectors (oil and gas, machinery, consumer goods) should have negative exposure.

```
# Get industry classification
industry = client.get_firm_info(
    exchanges=['HOSE', 'HNX'],
    fields=['ticker', 'icb_sector', 'icb_industry']
)

exposure_industry = exposures.reset_index().merge(
    industry, on='ticker', how='left'
)

# Industry-level average exposure
industry_avg = (
    exposure_industry
    .groupby('icb_sector')
    .agg(
        n_firms=('gamma', 'count'),
        mean_gamma=('gamma', 'mean'),
        median_gamma=('gamma', 'median'),
        pct_significant=('gamma_p', lambda x: (x < 0.05).mean())
    )
    .sort_values('mean_gamma', ascending=False)
    .round(3)
)

print("Exchange Rate Exposure by Industry:")
print(industry_avg.to_string())
```

87.7.3 Time-Varying Exposure

Exchange rate exposure is not constant, firms adjust hedging strategies, trade patterns shift, and the exchange rate regime itself evolves. We estimate rolling 36-month exposures:

```
def rolling_exposure(group, window=36, min_obs=24):
    """Estimate rolling FX exposure with 36-month windows."""
    results = []
    group = group.sort_values('month_end')

    for i in range(window, len(group) + 1):
```

```
fig, ax = plt.subplots(figsize=(12, 6))

colors_bar = ['#27AE60' if x > 0 else '#C0392B'
              for x in industry_avg['mean_gamma']]
bars = ax.barh(range(len(industry_avg)), industry_avg['mean_gamma'],
               color=colors_bar, alpha=0.85)
ax.set_yticks(range(len(industry_avg)))
ax.set_yticklabels(industry_avg.index, fontsize=9)
ax.axvline(x=0, color='black', linewidth=0.8)
ax.set_xlabel('Average Exchange Rate Exposure (gamma)')
ax.set_title('Exchange Rate Exposure by Industry')

for i, (_, row) in enumerate(industry_avg.iterrows()):
    label = f"({row['pct_significant']:.0%} sig.)"
    ax.text(row['mean_gamma'] + 0.01 * np.sign(row['mean_gamma']),
            i, label, va='center', fontsize=8, color='gray')

plt.tight_layout()
plt.show()
```

Figure 87.10

```
        sub = group.iloc[i - window:i]
        if len(sub) < min_obs:
            continue

        y = sub['monthly_return']
        X = sm.add_constant(sub[['market_return', 'fx_return']])

        try:
            model = sm.OLS(y, X).fit()
            results.append({
                'month_end': sub['month_end'].iloc[-1],
                'gamma': model.params['fx_return'],
                'gamma_se': model.bse['fx_return']
            })
        except Exception:
            pass

    return pd.DataFrame(results)

# Compute for a sample of large firms
large_firms = (
    equity.groupby('ticker')['market_cap']
    .last().nlargest(20).index
)

rolling_results = {}
for ticker in large_firms:
    firm_data = exposure_data[exposure_data['ticker'] == ticker]
    result = rolling_exposure(firm_data)
    if len(result) > 0:
        rolling_results[ticker] = result

print(f"Rolling exposures computed for {len(rolling_results)} firms")
```

87.8 Currency Hedging for International Investors

87.8.1 The Hedging Decision

For a foreign investor holding Vietnamese equities, the unhedged USD-denominated return is:

```
fig, ax = plt.subplots(figsize=(14, 6))

plot_colors = plt.cm.Set1(np.linspace(0, 1, min(6, len(rolling_results))))
for i, (ticker, df) in enumerate(list(rolling_results.items())[:6]):
    ax.plot(pd.to_datetime(df['month_end']), df['gamma'],
            linewidth=1.5, label=ticker, color=plot_colors[i])
    if i == 0:
        ax.fill_between(
            pd.to_datetime(df['month_end']),
            df['gamma'] - 1.96 * df['gamma_se'],
            df['gamma'] + 1.96 * df['gamma_se'],
            alpha=0.1, color=plot_colors[i]
        )

ax.axhline(y=0, color='gray', linewidth=0.8)
ax.set_ylabel('Exchange Rate Exposure (gamma)')
ax.set_xlabel('Date')
ax.set_title('Rolling 36-Month Exchange Rate Exposure')
ax.legend(ncol=3, fontsize=9)
plt.tight_layout()
plt.show()
```

Figure 87.11

$$R_{i,t}^{USD} \approx R_{i,t}^{VND} + \Delta s_t + R_{i,t}^{VND} \cdot \Delta s_t \tag{87.9}$$

where the cross-product term is usually small. Under a full hedge using forward contracts, the return becomes:

$$R_{i,t}^{USD,\text{hedged}} \approx R_{i,t}^{VND} + (f_t - s_t) \tag{87.10}$$

where $f_t - s_t$ is the forward premium (which, by CIP, approximately equals $r_t^{USD} - r_t^{VND}$). In Vietnam, where VND rates exceed USD rates, the forward premium is negative, hedging costs money because the investor forgoes the interest rate differential.

J. Y. Campbell, Serfaty-De Medeiros, and Viceira (2010) show that the optimal hedge ratio depends on the correlation between equity returns and currency returns. When the correlation is positive (VND depreciation coincides with equity losses), hedging reduces overall portfolio variance more effectively.

```
# Construct VW equity index in VND
equity_index = (
    equity
    .groupby('month_end')
    .apply(lambda g: np.average(g['monthly_return'],
                                weights=g['market_cap']))
    .reset_index(name='equity_vnd')
)
equity_index['month_end'] = pd.to_datetime(equity_index['month_end'])

# Merge with FX returns and forward premium
hedge_data = (
    equity_index
    .merge(fx_monthly, left_on='month_end', right_index=True, how='inner')
)

# Forward premium
if len(cip_data) > 0:
    fwd_premium_monthly = (
        cip_data['forward_premium']
        .resample('M').last() / 12
    )
    hedge_data = hedge_data.merge(
        fwd_premium_monthly.to_frame('fwd_premium'),
        left_on='month_end', right_index=True, how='left'
    )
```

```
    hedge_data['fwd_premium'] = hedge_data['fwd_premium'].fillna(
        hedge_data['fwd_premium'].mean()
    )
else:
    hedge_data['fwd_premium'] = -0.003

# Unhedged USD return
hedge_data['return_unhedged'] = (
    hedge_data['equity_vnd'] + hedge_data['fx_return']
    + hedge_data['equity_vnd'] * hedge_data['fx_return']
)

# Fully hedged USD return
hedge_data['return_hedged'] = (
    hedge_data['equity_vnd'] + hedge_data['fwd_premium']
)

# 50% hedge ratio
hedge_data['return_50pct'] = (
    0.5 * hedge_data['return_hedged']
    + 0.5 * hedge_data['return_unhedged']
)

# Performance comparison
for strategy, col in [('VND (Local)', 'equity_vnd'),
                      ('USD Unhedged', 'return_unhedged'),
                      ('USD 50% Hedged', 'return_50pct'),
                      ('USD Fully Hedged', 'return_hedged')]:
    r = hedge_data[col]
    ann_ret = (1 + r).prod() ** (12 / len(r)) - 1
    ann_vol = r.std() * np.sqrt(12)
    sharpe = r.mean() / r.std() * np.sqrt(12)
    print(f"{strategy:<22} Return: {ann_ret:>7.2%}  Vol: {ann_vol:>7.2%}  "
          f"Sharpe: {sharpe:>5.2f}")
```

87.8.2 Optimal Hedge Ratio

The minimum-variance hedge ratio is:

$$h^* = \frac{\text{Cov}(R_t^{VND}, \Delta s_t)}{\text{Var}(\Delta s_t)} \tag{87.11}$$

```
fig, ax = plt.subplots(figsize=(12, 6))

strategies = {
    'VND (Local)': ('equity_vnd', '#2C5F8A', '-'),
    'USD Unhedged': ('return_unhedged', '#C0392B', '-'),
    'USD 50% Hedged': ('return_50pct', '#E67E22', '--'),
    'USD Fully Hedged': ('return_hedged', '#27AE60', '-'),
}

for label, (col, color, ls) in strategies.items():
    cum = (1 + hedge_data.set_index('month_end')[col]).cumprod()
    ax.plot(cum.index, cum, label=label, color=color,
            linewidth=2, linestyle=ls)

ax.set_ylabel('Cumulative Wealth')
ax.set_xlabel('Date')
ax.set_title('Vietnamese Equity Returns: Hedging Comparison')
ax.legend()
ax.set_yscale('log')
plt.tight_layout()
plt.show()
```

Figure 87.12

This is the OLS slope from regressing local equity returns on exchange rate changes. When the correlation between equity and currency is positive (bad for unhedged investors), $h^* > 0$ and hedging reduces variance.

```
# Full-sample optimal hedge ratio
hedge_reg = sm.OLS(
    hedge_data['equity_vnd'],
    sm.add_constant(hedge_data['fx_return'])
).fit()

h_star = hedge_reg.params['fx_return']
print(f"Optimal hedge ratio (full sample): {h_star:.3f}")
print(f"Equity-currency correlation: "
      f"{hedge_data['equity_vnd'].corr(hedge_data['fx_return']):.3f}")

# Rolling 36-month optimal hedge ratio
rolling_cov = hedge_data['equity_vnd'].rolling(36).cov(
    hedge_data['fx_return']
)
rolling_var = hedge_data['fx_return'].rolling(36).var()
hedge_data['h_star_rolling'] = rolling_cov / rolling_var

print(f"\nRolling hedge ratio range: "
      f"[{hedge_data['h_star_rolling'].min():.2f}, "
      f"{hedge_data['h_star_rolling'].max():.2f}]")
```

87.9 ASEAN Currency Comparison

To put the VND in perspective, we compare its properties with other ASEAN currencies and the Chinese Yuan.

87.10 Exchange Rate and Equity Market Co-Movement

The relationship between VND/USD movements and the Vietnamese equity market is central to portfolio construction. We examine this at both the aggregate and conditional levels.

```
# Compute annual statistics for each currency
comparison = pd.DataFrame()
for currency in fx_returns.columns:
    r = fx_returns[currency].dropna()
    if len(r) > 252:
        comparison.loc[currency, 'Ann. Depreciation (%)'] = r.mean() * 252 *
 ↪  100
        comparison.loc[currency, 'Ann. Volatility (%)'] = r.std() *
 ↪  np.sqrt(252) * 100
        comparison.loc[currency, 'Skewness'] = r.skew()
        comparison.loc[currency, 'Kurtosis'] = r.kurtosis()
        comparison.loc[currency, 'Max Daily Loss (%)'] = r.max() * 100
        comparison.loc[currency, 'Sharpe (Carry)'] = (
            r.mean() * 252 / (r.std() * np.sqrt(252))
        )

print("ASEAN + Asian Currency Comparison (VND base, vs USD):")
print(comparison.round(2).to_string())

# Risk-return scatter
fig, ax = plt.subplots(figsize=(9, 7))
for currency in comparison.index:
    ax.scatter(
        comparison.loc[currency, 'Ann. Volatility (%)'],
        comparison.loc[currency, 'Ann. Depreciation (%)'],
        s=120, zorder=5
    )
    ax.annotate(
        f'VND/{currency}',
        (comparison.loc[currency, 'Ann. Volatility (%)'] + 0.15,
         comparison.loc[currency, 'Ann. Depreciation (%)']),
        fontsize=10
    )

ax.axhline(y=0, color='gray', linewidth=0.5)
ax.set_xlabel('Annualized Volatility (%)')
ax.set_ylabel('Annualized Depreciation (%)')
ax.set_title('ASEAN Currency Risk-Return Characteristics')
plt.tight_layout()
plt.show()
```

Figure 87.13

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: Scatter plot
axes[0].scatter(
    hedge_data['fx_return'] * 100,
    hedge_data['equity_vnd'] * 100,
    c=hedge_data.index, cmap='viridis', s=20, alpha=0.7
)
z = np.polyfit(hedge_data['fx_return'], hedge_data['equity_vnd'], 1)
x_line = np.linspace(hedge_data['fx_return'].min(),
                     hedge_data['fx_return'].max(), 100)
axes[0].plot(x_line * 100, np.polyval(z, x_line) * 100,
             color='#C0392B', linewidth=2)
axes[0].axhline(y=0, color='gray', linewidth=0.5)
axes[0].axvline(x=0, color='gray', linewidth=0.5)
axes[0].set_xlabel('VND/USD Change (%)')
axes[0].set_ylabel('VN-Index Return (%)')
axes[0].set_title('Panel A: Monthly Equity-FX Relationship')

rho = hedge_data['equity_vnd'].corr(hedge_data['fx_return'])
axes[0].text(0.05, 0.95, f'rho = {rho:.3f}', transform=axes[0].transAxes,
             fontsize=12, verticalalignment='top',
             bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

# Panel B: Rolling 36-month correlation
rolling_corr = (
    hedge_data['equity_vnd']
    .rolling(36)
    .corr(hedge_data['fx_return'])
)
axes[1].plot(hedge_data['month_end'], rolling_corr,
             color='#2C5F8A', linewidth=2)
axes[1].axhline(y=0, color='gray', linewidth=0.8, linestyle='--')
axes[1].fill_between(
    hedge_data['month_end'], 0, rolling_corr,
    where=rolling_corr < 0, color='#C0392B', alpha=0.2
)
axes[1].fill_between(
    hedge_data['month_end'], 0, rolling_corr,
    where=rolling_corr > 0, color='#27AE60', alpha=0.2
)
axes[1].set_xlabel('Date')
axes[1].set_ylabel('Rolling Correlation')
axes[1].set_title('Panel B: 36-Month Rolling Equity-FX Correlation')

plt.tight_layout()
plt.show()
```


Figure 87.14

87.11 Summary

This chapter has developed a comprehensive toolkit for working with exchange rate data in the Vietnamese context. The key findings and methods are in Table 87.2.

Table 87.2: Summary of findings on VND exchange rate dynamics.

Topic	Finding	Implication
VND regime	Managed float with +/-3% band, daily reference rate	Returns are non-normal; clustered near zero with discrete jumps
Return distribution	Fat tails, excess kurtosis, positive skewness	Standard normal-based risk models underestimate tail risk
CIP	Persistent positive deviations (~50-200 bps)	Capital controls and limited arbitrage; hedging is expensive
UIP	beta < 1 in Fama regression	Carry trade is profitable on average; crash risk in devaluations
Carry trade	Positive Sharpe but negative skewness	Steady gains punctuated by sharp losses during devaluations
GARCH	Strong persistence (alpha + beta approx 0.99); Student-t preferred	Volatility clustering; regime switches around policy events
Firm exposure	~15-25% significant; systematic industry patterns	Exporters benefit from depreciation; importers hurt
Hedging	Full hedge eliminates FX risk but costs ~3-5% p.a.	Optimal partial hedge depends on equity-FX correlation
ASEAN comparison	VND has lowest volatility but steady depreciation	Managed float compresses vol; does not eliminate trend risk

For researchers using Vietnamese equity data, the exchange rate dimension is not optional. Any study that converts VND returns to USD, uses international factor models, or examines firms with trade exposure must account for the distinctive properties of the VND regime. The tools developed in this chapter, from GARCH volatility modeling to rolling exposure estimation, provide the building blocks for incorporating exchange rate dynamics into broader empirical asset pricing work.

Part VI

Advanced Financial Modeling

88 Event Studies in Finance

Event studies constitute one of the most enduring and widely deployed empirical methodologies in financial economics. At their core, event studies measure the impact of a specific event on the value of a firm by examining **abnormal security returns** around the time the event occurs. The methodology rests on a simple premise: if capital markets are informationally efficient, the effect of an event will be reflected immediately in security prices, and any deviation from "normal" expected returns can be attributed to the event itself.

Since the pioneering work of Eugene F. Fama et al. (1969), who studied how stock prices adjust to new information around stock splits, event studies have become a cornerstone of empirical research across finance, accounting, economics, and law. R. Ball and Brown (2013) demonstrated that accounting earnings announcements convey information to the market, a finding that launched decades of research in accounting and disclosure. The methodology has since been refined through contributions by Brown and Warner (1980) and Brown and Warner (1985), who established the statistical properties of event study methods, and MacKinlay (1997) codified best practices that remain standard today.

The breadth of applications is remarkable. Event studies have been used to examine the wealth effects of mergers and acquisitions (M. C. Jensen and Ruback 1983; Andrade, Mitchell, and Stafford 2001), earnings announcements (Bernard and Thomas 1989), dividend changes (Aharony and Swary 1980), regulatory changes (Schwert 1981), executive turnover (Warner, Watts, and Wruck 1988), and macroeconomic announcements (Flannery and Protopapadakis 2002). In law and economics, event studies serve as the primary tool for measuring damages in securities fraud litigation (Mitchell and Netter 1993) and assessing the impact of regulatory interventions (Binder 1998). Sagar P. Kothari and Warner (2007) documented over 500 published event studies in the top five finance journals alone between 1974 and 2000.

88.0.1 Why Event Studies Matter

The enduring popularity of event studies stems from several compelling properties:

- **Direct measurement of economic significance.** Unlike regression-based approaches that estimate associations, event studies directly quantify the dollar impact of events on firm value. A cumulative abnormal return (CAR) of 3% for a firm with $10 billion market capitalization translates to $300 million in wealth creation, which is a tangible, economically meaningful magnitude.

- **Minimal maintained assumptions.** The methodology requires only semi-strong market efficiency (i.e., prices reflect publicly available information), a weaker assumption than many alternatives.
- **Statistical power.** Daily event studies have remarkable power to detect abnormal performance, even with modest sample sizes. Brown and Warner (1985) demonstrated that the market model detects abnormal returns of 1% or more with high reliability using samples as small as 20 securities.
- **Versatility.** The basic framework accommodates events that are firm-specific or market-wide, anticipated or surprising, and can be adapted to various asset classes and market structures.

88.1 Literature Review and Methodological Evolution

88.1.1 The Classical Framework (1969-1985)

The modern event study traces its origins to Eugene F. Fama et al. (1969), hereafter FFJR, who examined monthly stock returns around 940 stock splits between 1927 and 1959. Their key innovation was the use of the **market model** to decompose returns into expected (normal) and unexpected (abnormal) components.

R. Ball and Brown (2013) independently developed a similar approach to study earnings announcements, establishing the information content of accounting data. It was a finding with profound implications for both the efficient markets hypothesis and the relevance of financial reporting.

Brown and Warner (1980) provided the first systematic analysis of event study methodology using simulation. Their study of monthly data established several important results: (i) the simple market model performs at least as well as more complex models, (ii) value-weighted market indices can lead to misspecification when the sample is tilted toward smaller firms, and (iii) the standard cross-sectional test has well-specified size under the null hypothesis. Their follow-up study (Brown and Warner 1985) extended the analysis to daily data, documenting the importance of non-normality in daily returns and the increased power of daily versus monthly studies.

88.1.2 Risk Model Refinements (1992-2015)

The advent of the **Fama-French three-factor model** (Eugene F. Fama and French 1993a) represented a major advance in modeling expected returns. Adding size (SMB) and value (HML) factors to the market model improved the cross-sectional fit of expected returns considerably. Mark M. Carhart (1997b) augmented this with a momentum factor (UMD), yielding the four-factor model that became standard in event studies through the 2000s. Eugene F. Fama and French (2015) subsequently introduced profitability (RMW) and investment (CMA) factors in their five-factor model.

The choice of risk model matters for event studies primarily in **long-horizon settings**. Sagar P. Kothari and Warner (2007) showed that for short-window studies (3-5 days), the market model and multi-factor models produce virtually identical results because the incremental factors explain very little daily return variation for individual firms. However, for event windows exceeding 20 trading days, model choice can materially affect inferences.

88.1.3 Testing for Abnormal Returns (1976-2010)

The statistical testing of abnormal returns has evolved considerably:

Table 88.1: Summary of major event study test statistics

Test	Year	Key Property	Reference
Patell Z	1976	Standardizes by estimation-period σ; weights firms inversely by volatility	Patell (1976)
Cross-Sectional t	1980	Allows event-induced variance change	Brown and Warner (1980)
BMP	1991	Robust to event-induced variance	Boehmer, Masumeci, and Poulsen (1991)
Corrado Rank	1989	Non-parametric; robust to non-normality	Corrado (1989)
Generalized Sign	1992	Non-parametric; uses estimation-window baseline	Cowan (1992)
Kolari-Pynnönen	2010	Accounts for cross-sectional dependence	Kolari and Pynnönen (2010)

Test	Year	Key Property	Reference
Skewness-Adjusted	1992	Corrects for BHAR skewness	Hall (1992)

88.1.4 CARs versus BHARs

Cumulative abnormal returns (CARs) sum daily abnormal returns, while buy-and-hold abnormal returns (BHARs) compound returns and subtract the compounded benchmark. Barber and Lyon (1997) demonstrated that BHARs better capture the actual investor experience, since investors earn compound, not cumulative, returns. However, Eugene F. Fama (1998) and Mitchell and Stafford (2000) showed that BHARs exhibit severe cross-sectional dependence and positive skewness. For **short event windows** (under 10 days), the difference between CARs and BHARs is negligible. For longer windows, both should be reported.

88.1.5 Emerging Market Considerations

Event studies in emerging markets face distinct challenges:

- **Thin trading.** Many emerging market securities trade infrequently, inducing bias in market model beta estimates. Scholes and Williams (1977) and Dimson (1979) proposed corrections using leading and lagging market returns.
- **Factor availability.** While Fama-French factors are readily available for developed markets, emerging market factors must often be constructed locally.
- **Market microstructure.** Price limits ($\pm$ 7% on HOSE, $\pm$ 10% on HNX, $\pm$ 15% on UPCOM in Vietnam), T+2 settlement, and the absence of short-selling affect the speed of price adjustment. Researchers should consider wider event windows to accommodate slower information incorporation (U. Bhattacharya et al. 2000; Griffin, Kelly, and Nardari 2010).

88.2 Mathematical Framework

This section presents the complete mathematical specification of the event study methodology. We follow the notation conventions of J. Y. Campbell et al. (1998) and Sagar P. Kothari and Warner (2007).

88.2.1 Timeline and Windows

The event study timeline is defined relative to the event date, denoted $\tau = 0$. All dates are measured in **trading days**:

$$\underbrace{T_0 + 1, \ldots, T_1}_{\text{Estimation Window (L days)}} \quad \underbrace{\smile}_{\text{Gap (G days)}} \quad \underbrace{\tau_1, \ldots, 0, \ldots, \tau_2}_{\text{Event Window (L days)}}$$

where:

- **Estimation window**: L_1 trading days over which the risk model parameters are estimated
- **Gap**: G trading days separating estimation and event windows, preventing contamination by pre-event information leakage
- **Event window**: $L_2 = \tau_2 - \tau_1 + 1$ trading days centered around the event date

For example, with $L_1 = 150$, $G = 15$, $\tau_1 = -10$, $\tau_2 = +10$: the estimation window covers trading days $[-175, -25]$ relative to the event, and the event window covers $[-10, +10]$.

88.2.2 Normal Return Models

Let R_{it} denote the return on security i on trading day t, R_{ft} the risk-free rate, and R_{mt} the market return. We implement six models:

Model 0: Market-Adjusted Returns. Assumes $\beta_i = 1$ and $\alpha_i = 0$ for all firms:

$$AR_{it}^{MA} = R_{it} - R_{mt}$$

Model 1: Market Model (Sharpe 1964):

$$R_{it} = \alpha_i + \beta_i R_{mt} + \varepsilon_{it}, \quad E[\varepsilon_{it}] = 0, \quad \text{Var}[\varepsilon_{it}] = \sigma_{\varepsilon_i}^2$$

$$AR_{it}^{MM} = R_{it} - \hat{\alpha}_i - \hat{\beta}_i R_{mt}$$

Model 2: Fama-French Three-Factor (Eugene F. Fama and French 1993a):

$$R_{it} - R_{ft} = \alpha_i + \beta_{i,1}(R_{mt} - R_{ft}) + \beta_{i,2} \cdot SMB_t + \beta_{i,3} \cdot HML_t + \varepsilon_{it}$$

Model 3: Carhart Four-Factor (Mark M. Carhart 1997b):

$$R_{it} - R_{ft} = \alpha_i + \beta_{i,1}(R_{mt} - R_{ft}) + \beta_{i,2} \cdot SMB_t + \beta_{i,3} \cdot HML_t + \beta_{i,4} \cdot UMD_t + \varepsilon_{it}$$

Model 4: Fama-French Five-Factor (Eugene F. Fama and French 2015):

$$R_{it} - R_{ft} = \alpha_i + \beta_{i,1}(R_{mt} - R_{ft}) + \beta_{i,2} \cdot SMB_t + \beta_{i,3} \cdot HML_t + \beta_{i,4} \cdot RMW_t + \beta_{i,5} \cdot CMA_t + \varepsilon_{it}$$

Model 5: User-Specified Factor Model:

$$R_{it} - R_{ft} = \alpha_i + \sum_{k=1}^{K} \beta_{i,k} F_{k,t} + \varepsilon_{it}$$

88.2.3 Aggregation: CARs and BHARs

Cumulative Abnormal Returns sum daily abnormal returns:

$$CAR_i(\tau_1, \tau_2) = \sum_{t=\tau_1}^{\tau_2} AR_{it}, \qquad \overline{CAR}(\tau_1, \tau_2) = \frac{1}{N} \sum_{i=1}^{N} CAR_i(\tau_1, \tau_2)$$

Buy-and-Hold Abnormal Returns compound returns:

$$BHAR_i(\tau_1, \tau_2) = \prod_{t=\tau_1}^{\tau_2} (1 + R_{it}) - \prod_{t=\tau_1}^{\tau_2} (1 + \hat{E}[R_{it}])$$

88.2.4 Standardized Returns

The **standardized abnormal return** for firm i on day t is:

$$SAR_{it} = \frac{AR_{it}}{\hat{\sigma}_{\varepsilon_i}}$$

The **standardized cumulative abnormal return** is:

$$SCAR_i(\tau_1, \tau_2) = \frac{CAR_i(\tau_1, \tau_2)}{\hat{\sigma}_{\varepsilon_i} \sqrt{L_2}}$$

88.2.5 Test Statistics

Let N denote the number of firm-event observations.

Test 1: Cross-Sectional t-Test. Allows event-induced variance; assumes cross-sectional independence:

$$t_{CS} = \frac{\overline{CAR}}{s_{CAR}/\sqrt{N}}, \quad s_{CAR} = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(CAR_i - \overline{CAR})^2}$$

Test 2: Patell Z-Test (Patell 1976). Weights firms inversely by volatility:

$$Z_{Patell} = \frac{\sum_{i=1}^{N} SCAR_i}{\sqrt{\sum_{i=1}^{N} \frac{K_i-2}{K_i-4}}}$$

Test 3: BMP Test (Boehmer, Masumeci, and Poulsen 1991). Robust to event-induced variance:

$$t_{BMP} = \frac{\overline{SCAR}}{s_{SCAR}/\sqrt{N}}$$

Test 4: Kolari-Pynnönen Adjusted BMP (Kolari and Pynnönen 2010). Accounts for cross-sectional dependence:

$$t_{KP} = t_{BMP} \times \sqrt{\frac{1}{1+(N-1)\bar{r}}}$$

where $\bar{r}$ is the mean pairwise cross-correlation of estimation-period residuals.

Test 5: Generalized Sign Test (Cowan 1992):

$$Z_{GSign} = \frac{\hat{p} - \hat{p}_0}{\sqrt{\hat{p}_0(1-\hat{p}_0)/N}}$$

Test 6: Sign Test:

$$Z_{Sign} = \frac{N^+ - 0.5N}{\sqrt{0.25N}}$$

Test 7: Skewness-Adjusted t-Test (Hall 1992):

$$t_{SA} = \sqrt{N}\left(\bar{z} + \frac{1}{3}\hat{\gamma}\bar{z}^2 + \frac{1}{27}\hat{\gamma}^2\bar{z}^3 + \frac{1}{6N}\hat{\gamma}\right)$$

Test 8: Wilcoxon Signed-Rank Test: A non-parametric test of whether the median CAR differs from zero.

The table below summarizes the assumptions of each test:

Table 88.2: Assumption requirements for event study test statistics

Test	Event-Induced Variance	Cross-Sectional Independence	Normality
Cross-Sectional t	Robust	Assumes	Assumes
Patell Z	Assumes no change	Assumes	Assumes
BMP	Robust	Assumes	Assumes
Kolari-Pynnönen	Robust	Robust	Assumes
Generalized Sign	Robust	Assumes	Robust
Corrado Rank	Robust	Assumes	Robust
Skewness-Adjusted	Robust	Assumes	Partially
Wilcoxon	Robust	Assumes	Robust

88.3 Python Implementation

88.3.1 Design Philosophy

Our implementation follows these principles:

1. **Modularity**: Each component (calendar, estimation, AR computation, testing) is a separate function.
2. **Vectorization**: All operations use pandas/numpy for performance on large datasets.
3. **Configurability**: All parameters are user-configurable via a dataclass.
4. **Transparency**: Intermediate outputs are preserved for inspection.
5. **Production-ready**: Comprehensive input validation, missing data handling, and edge cases.

88.3.2 Setup and Imports

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from scipy import stats
from dataclasses import dataclass, field
from typing import Optional, List, Tuple
from enum import Enum
import warnings
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

warnings.filterwarnings('ignore')
pd.set_option('display.float_format', '{:.6f}'.format)
print("All libraries loaded.")
```

```
All libraries loaded.
```

```
import pandas as pd
import sqlite3

tidy_finance = sqlite3.connect(database="data/tidy_finance_python.sqlite")

factors_ff3_daily = pd.read_sql_query(
    sql="SELECT * FROM factors_ff3_daily",
    con=tidy_finance,
    parse_dates=["date"]
)

factors_ff5_daily = pd.read_sql_query(
    sql="SELECT * FROM factors_ff5_daily",
    con=tidy_finance,
    parse_dates=["date"]
)

factors_ff3_monthly = pd.read_sql_query(
    sql="SELECT * FROM factors_ff3_monthly",
    con=tidy_finance,
    parse_dates=["date"]
)

factors_ff5_monthly = pd.read_sql_query(
    sql="SELECT * FROM factors_ff5_monthly",
```

```
    con=tidy_finance,
    parse_dates=["date"]
)
```

```
prices_monthly = pd.read_sql_query(
    sql="""
        SELECT symbol, date, ret_excess, mktcap, mktcap_lag, risk_free
        FROM prices_monthly
    """,
    con=tidy_finance,
    parse_dates={"date"}
).dropna()

prices_daily = pd.read_sql_query(
    sql="""
        SELECT symbol, date, ret_excess, mktcap, mktcap_lag, risk_free
        FROM prices_daily
    """,
    con=tidy_finance,
    parse_dates={"date"}
).dropna()
```

88.3.3 Configuration

```
class RiskModel(Enum):
    """Supported risk models for expected return computation."""
    MARKET_ADJ = "market_adjusted"
    MARKET_MODEL = "market_model"
    FF3 = "ff3"
    CARHART = "carhart"
    FF5 = "ff5"
    CUSTOM = "custom"

@dataclass
class EventStudyConfig:
    """Complete configuration for an event study.

    Attributes
    ----------
    estimation_window : int
```

```
    Length of estimation period in trading days. Brown and Warner (1985)
    suggest  100 days; MacKinlay (1997) recommends 120 as standard.
event_window_start : int
    Start of event window relative to event date (e.g., -10).
event_window_end : int
    End of event window relative to event date (e.g., +10).
gap : int
    Trading days between estimation and event windows. Prevents
    contamination from pre-event information leakage.
min_estimation_obs : int
    Minimum non-missing returns required in estimation period.
risk_model : RiskModel
    Risk model for computing expected returns.
custom_factors : list
    Column names for user-specified factors (CUSTOM model only).
thin_trading_adj : str or None
    None, 'scholes_williams', or 'dimson'.
dimson_lags : int
    Number of leads/lags for Dimson (1979) correction.
"""
estimation_window: int = 150
event_window_start: int = -10
event_window_end: int = 10
gap: int = 15
min_estimation_obs: int = 120
risk_model: RiskModel = RiskModel.MARKET_MODEL
custom_factors: List[str] = field(default_factory=list)
thin_trading_adj: Optional[str] = None
dimson_lags: int = 1

@property
def event_window_length(self) -> int:
    return self.event_window_end - self.event_window_start + 1

def validate(self):
    assert self.estimation_window > 0
    assert self.event_window_start <= self.event_window_end
    assert self.gap >= 0
    assert self.min_estimation_obs <= self.estimation_window
    if self.risk_model == RiskModel.CUSTOM:
        assert len(self.custom_factors) > 0
    return True
```

```
# Demonstrate
config_demo = EventStudyConfig(
    estimation_window=150, event_window_start=-10, event_window_end=10,
    gap=15, min_estimation_obs=120, risk_model=RiskModel.FF3
)
config_demo.validate()
print(f"Event window length: {config_demo.event_window_length} days")
print(f"Model: {config_demo.risk_model.value}")
```

```
Event window length: 21 days
Model: ff3
```

88.3.4 Step 1: Trading Calendar Construction

A correct trading calendar is fundamental. It maps any event date to the exact calendar dates for the start/end of estimation and event windows, accounting for weekends, holidays, and non-trading days.

```
def build_trading_calendar(trading_dates, config):
    """Build a trading calendar mapping event dates to window boundaries.

    For each potential event date, identifies the calendar dates for the
    start/end of the estimation period and event window using only actual
    trading days.

    Parameters
    ----------
    trading_dates : array-like
        Sorted unique trading dates in the market.
    config : EventStudyConfig

    Returns
    -------
    pd.DataFrame with columns: estper_beg, estper_end, evtwin_beg,
        evtdate, evtwin_end, cal_index
    """
    dates = pd.Series(sorted(pd.to_datetime(trading_dates).unique()))
    n = len(dates)

    L1 = config.estimation_window
    G = config.gap
```

```
    s = config.event_window_start
    L2 = config.event_window_length

    # Offsets (FIRSTOBS logic)
    o0 = 0                      # estper_beg
    o1 = L1 - 1                 # estper_end
    o2 = L1 + G                 # evtwin_beg
    o3 = L1 + G - s             # evtdate
    o4 = L1 + G + L2 - 1        # evtwin_end

    max_offset = o4
    valid = n - max_offset
    if valid <= 0:
        raise ValueError(f"Need  {max_offset+1} trading dates, have {n}")

    cal = pd.DataFrame({
        'estper_beg': dates.iloc[o0:o0+valid].values,
        'estper_end': dates.iloc[o1:o1+valid].values,
        'evtwin_beg': dates.iloc[o2:o2+valid].values,
        'evtdate':    dates.iloc[o3:o3+valid].values,
        'evtwin_end': dates.iloc[o4:o4+valid].values,
    })
    cal['cal_index'] = range(1, len(cal)+1)

    # Validate window lengths using a sample row
    idx = min(10, len(cal)-1)
    row = cal.iloc[idx]
    est_n = dates[(dates >= row['estper_beg']) & (dates <=
↪   row['estper_end'])].shape[0]
    evt_n = dates[(dates >= row['evtwin_beg']) & (dates <=
↪   row['evtwin_end'])].shape[0]
    assert est_n == L1, f"Estimation window: {est_n}  {L1}"
    assert evt_n == L2, f"Event window: {evt_n}  {L2}"

    return cal

# Demo
demo_dates = pd.bdate_range('2018-01-01', '2023-12-31', freq='B')
demo_cal = build_trading_calendar(demo_dates, config_demo)
print(f"Calendar: {len(demo_cal)} potential event dates")
print(demo_cal.head(3).to_string(index=False))
```

```
Calendar: 1380 potential event dates
estper_beg estper_end evtwin_beg    evtdate evtwin_end  cal_index
2018-01-01 2018-07-27 2018-08-20 2018-09-03 2018-09-17          1
2018-01-02 2018-07-30 2018-08-21 2018-09-04 2018-09-18          2
2018-01-03 2018-07-31 2018-08-22 2018-09-05 2018-09-19          3
```

88.3.5 Step 2: Event Date Alignment

When an event occurs on a non-trading day, align to the **next** available trading day.

```python
def align_events(events, calendar, id_col='symbol', date_col='event_date'):
    """Align event dates to trading calendar.

    Non-trading-day events are shifted forward to the next trading day.

    Parameters
    ----------
    events : pd.DataFrame with [id_col, date_col] and optional 'group'
    calendar : pd.DataFrame from build_trading_calendar()

    Returns
    -------
    pd.DataFrame with window boundaries for each firm-event
    """
    events = events.copy()
    events[date_col] = pd.to_datetime(events[date_col])

    cal_dates =
↪   calendar[['evtdate']].drop_duplicates().sort_values('evtdate')

    merged = pd.merge_asof(
        events.sort_values(date_col),
        cal_dates.rename(columns={'evtdate': 'aligned_date'}),
        left_on=date_col, right_on='aligned_date',
        direction='forward'
    )

    result = merged.merge(calendar, left_on='aligned_date',
↪   right_on='evtdate', how='inner')

    shifted = (result[date_col] != result['evtdate']).sum()
    if shifted > 0:
```

```
        print(f"  {shifted} event(s) shifted to next trading day")

    result = result.rename(columns={date_col: 'original_date'})
    result = result.drop_duplicates(subset=[id_col, 'evtdate'])

    return result
```

88.3.6 Step 3: Data Extraction and Factor Merging

Extract returns for each security-event across the full estimation + event window and merge risk factors.

```
def extract_returns(aligned_events, prices, factors, config,
                    id_col='symbol', date_col='date', ret_col='ret',
                    mkt_col='mkt_excess', rf_col='risk_free'):
    """Extract stock returns and merge risk factors for each event.

    For each security-event, retrieves daily returns from estper_beg
    through evtwin_end and merges appropriate risk factors.
    """
    prices = prices.copy()
    factors = factors.copy()
    prices[date_col] = pd.to_datetime(prices[date_col])
    factors[date_col] = pd.to_datetime(factors[date_col])

    # Recover raw return from excess return if needed
    if ret_col not in prices.columns and 'ret_excess' in prices.columns:
        if rf_col in factors.columns:
            prices = prices.merge(factors[[date_col,
↪   rf_col]].drop_duplicates(),
                                  on=date_col, how='left')
        prices[ret_col] = prices['ret_excess'] + prices[rf_col]

    # Factor columns based on model
    model = config.risk_model
    fac_cols = [mkt_col] if mkt_col in factors.columns else []
    if rf_col in factors.columns:
        fac_cols.append(rf_col)

    model_factors = {
        RiskModel.FF3: ['smb', 'hml'],
```

```
        RiskModel.CARHART: ['smb', 'hml', 'umd'],
        RiskModel.FF5: ['smb', 'hml', 'rmw', 'cma'],
        RiskModel.CUSTOM: config.custom_factors,
    }
    for f in model_factors.get(model, []):
        if f in factors.columns:
            fac_cols.append(f)

    fac_cols = list(set([date_col] + fac_cols))

    # Vectorized merge approach: join events with prices on id + date range
    frames = []
    for _, evt in aligned_events.iterrows():
        mask = ((prices[id_col] == evt[id_col]) &
                (prices[date_col] >= evt['estper_beg']) &
                (prices[date_col] <= evt['evtwin_end']))
        fd = prices.loc[mask, [id_col, date_col, ret_col]].copy()
        if len(fd) == 0:
            continue
        fd['evtdate'] = evt['evtdate']
        fd['estper_beg'] = evt['estper_beg']
        fd['estper_end'] = evt['estper_end']
        fd['evtwin_beg'] = evt['evtwin_beg']
        fd['evtwin_end'] = evt['evtwin_end']
        if 'group' in evt.index:
            fd['group'] = evt['group']
        frames.append(fd)

    if not frames:
        raise ValueError("No return data found for any events")

    result = pd.concat(frames, ignore_index=True)
    result = result.merge(factors[fac_cols].drop_duplicates(), on=date_col,
↪   how='left')

    # Excess and market-adjusted returns
    if rf_col in result.columns:
        result['ret_excess'] = result[ret_col] - result[rf_col]
    else:
        result['ret_excess'] = result[ret_col]
    if mkt_col in result.columns:
        result['ret_mktadj'] = result['ret_excess'] - result[mkt_col]
```

```
    result = result.sort_values([id_col, 'evtdate',
↪   date_col]).reset_index(drop=True)
    n_evts = result.groupby([id_col, 'evtdate']).ngroups
    print(f"  Extracted {len(result):,} obs for {n_evts} firm-events")
    return result
```

88.3.7 Step 4: Risk Model Estimation

Estimate risk model parameters over the estimation window.

```
def estimate_model(
    event_returns, config, id_col="symbol", date_col="date", ret_col="ret"
):
    """Estimate risk model parameters for each firm-event.

    Runs OLS over the estimation window. Returns alpha, betas, sigma,
    R^2, nobs, and residuals for cross-correlation computation.
    """
    model = config.risk_model

    # Define regression specification
    dep_var_map = {
        RiskModel.MARKET_ADJ: "ret_mktadj",
        RiskModel.MARKET_MODEL: ret_col,
        RiskModel.FF3: "ret_excess",
        RiskModel.CARHART: "ret_excess",
        RiskModel.FF5: "ret_excess",
        RiskModel.CUSTOM: "ret_excess",
    }
    indep_var_map = {
        RiskModel.MARKET_ADJ: [],
        RiskModel.MARKET_MODEL: ["mkt_excess"],
        RiskModel.FF3: ["mkt_excess", "smb", "hml"],
        RiskModel.CARHART: ["mkt_excess", "smb", "hml", "umd"],
        RiskModel.FF5: ["mkt_excess", "smb", "hml", "rmw", "cma"],
        RiskModel.CUSTOM: config.custom_factors,
    }

    dep_var = dep_var_map[model]
    indep_vars = indep_var_map[model]
```

```
    est = event_returns[
        (event_returns[date_col] >= event_returns["estper_beg"])
        & (event_returns[date_col] <= event_returns["estper_end"])
    ].copy()

    params_list = []

    for (firm, evtdate), grp in est.groupby([id_col, "evtdate"]):
        valid = grp.dropna(subset=[dep_var] + indep_vars)
        nobs = len(valid)
        if nobs < config.min_estimation_obs:
            continue

        y = valid[dep_var].values

        if len(indep_vars) == 0:
            # Market-adjusted: intercept-only for variance
            p = {
                id_col: firm,
                "evtdate": evtdate,
                "alpha": y.mean(),
                "sigma": y.std(ddof=1),
                "variance": y.var(ddof=1),
                "nobs": nobs,
                "r_squared": 0.0,
                "_residuals": y - y.mean(),
            }
        else:
            X = sm.add_constant(valid[indep_vars].values)
            res = sm.OLS(y, X).fit()
            p = {
                id_col: firm,
                "evtdate": evtdate,
                "alpha": res.params[0],
                "sigma": np.sqrt(res.mse_resid),
                "variance": res.mse_resid,
                "nobs": nobs,
                "r_squared": res.rsquared if np.isfinite(res.rsquared) else
                 ↪  np.nan,
                "_residuals": res.resid,
            }
            for j, var in enumerate(indep_vars):
                p[f"beta_{var}"] = res.params[j + 1]
```

```
        # Skip degenerate firms (zero or near-zero variance)
        if p["sigma"] < 1e-6:
            continue

        params_list.append(p)

    if not params_list:
        raise ValueError("No firm-events passed minimum observation filter")

    params_df = pd.DataFrame(params_list)
    n_total = event_returns.groupby([id_col, "evtdate"]).ngroups
    print(
        f"  Estimated {len(params_df)}/{n_total} firm-events "
        f"(mean R^2 = {params_df['r_squared'].dropna().mean():.4f})"
    )
    return params_df
```

88.3.8 Step 5: Abnormal Return Computation

Compute AR, CAR, BHAR, SAR, SCAR for each firm-event-date.

```
def compute_abnormal_returns(
    event_returns, params, config, id_col="symbol", date_col="date",
 ↪  ret_col="ret"
):
    """Compute abnormal returns and aggregate to CARs/BHARs.

    Returns
    -------
    daily_ar : pd.DataFrame - daily AR/SAR/CAR/BHAR per firm-event-date
    event_ar : pd.DataFrame - event-level CAR/BHAR/SCAR per firm-event
    """
    model = config.risk_model

    factor_map = {
        RiskModel.MARKET_ADJ: [],
        RiskModel.MARKET_MODEL: ["mkt_excess"],
        RiskModel.FF3: ["mkt_excess", "smb", "hml"],
        RiskModel.CARHART: ["mkt_excess", "smb", "hml", "umd"],
        RiskModel.FF5: ["mkt_excess", "smb", "hml", "rmw", "cma"],
```

```
        RiskModel.CUSTOM: config.custom_factors,
    }
    factor_cols = factor_map[model]

    # Filter to event window
    evt = event_returns[
        (event_returns[date_col] >= event_returns["evtwin_beg"])
        & (event_returns[date_col] <= event_returns["evtwin_end"])
    ].copy()

    # Merge params (drop residuals column for merge)
    merge_cols = [c for c in params.columns if c != "_residuals"]
    evt = evt.merge(params[merge_cols], on=[id_col, "evtdate"], how="inner")

    # Expected returns
    if model == RiskModel.MARKET_ADJ:
        evt["expected_ret"] = evt.get("mkt_excess", 0) + evt.get("risk_free",
↪   0)
        evt["AR"] = evt[ret_col] - evt["expected_ret"]
    else:
        evt["expected_ret"] = evt["alpha"]
        for fc in factor_cols:
            bcol = f"beta_{fc}"
            if bcol in evt.columns:
                evt["expected_ret"] += evt[bcol] * evt[fc]

        if model == RiskModel.MARKET_MODEL:
            evt["AR"] = evt[ret_col] - evt["expected_ret"]
        else:
            evt["AR"] = evt["ret_excess"] - evt["expected_ret"]

    evt["SAR"] = evt["AR"] / evt["sigma"]
    evt = evt.sort_values([id_col, "evtdate", date_col])

    # Compute event time
    all_dates = sorted(event_returns[date_col].unique())
    d2i = {d: i for i, d in enumerate(all_dates)}
    evt["evttime"] = evt[date_col].map(d2i) - evt["evtdate"].map(d2i)

    # Cumulative measures per firm-event
    daily_recs = []
    event_recs = []
```

```
    for (firm, evtdate), g in evt.groupby([id_col, "evtdate"]):
        g = g.sort_values(date_col).copy()
        nd = len(g)

        g["CAR"] = g["AR"].cumsum()
        g["cum_ret"] = (1 + g[ret_col]).cumprod() - 1
        g["cum_expected"] = (1 + g["expected_ret"]).cumprod() - 1
        g["BHAR"] = g["cum_ret"] - g["cum_expected"]
        g["SCAR"] = g["CAR"] / (g["sigma"].iloc[0] * np.sqrt(np.arange(1, nd
↪   + 1)))

        daily_recs.append(g)

        last = g.iloc[-1]
        sigma = g["sigma"].iloc[0]
        nobs = g["nobs"].iloc[0]

        rec = {
            id_col: firm,
            "evtdate": evtdate,
            "CAR": last["CAR"],
            "BHAR": last["BHAR"],
            "cum_ret": last["cum_ret"],
            "SCAR": last["CAR"] / (sigma * np.sqrt(nd)),
            "sigma": sigma,
            "variance": g["variance"].iloc[0],
            "nobs": nobs,
            "n_event_days": nd,
            "alpha": g["alpha"].iloc[0],
            "pat_scale": (nobs - 2) / (nobs - 4) if nobs > 4 else np.nan,
            "pos_car": int(last["CAR"] > 0),
        }

        for fc in factor_cols:
            bcol = f"beta_{fc}"
            if bcol in g.columns:
                rec[bcol] = g[bcol].iloc[0]
        if "group" in g.columns:
            rec["group"] = g["group"].iloc[0]

        event_recs.append(rec)

    daily_ar = pd.concat(daily_recs, ignore_index=True)
```

```
    event_ar = pd.DataFrame(event_recs)

    print(
        f"  {len(event_ar)} firm-events | Mean CAR:
         ↪  {event_ar['CAR'].mean():.6f} | "
        f"Mean BHAR: {event_ar['BHAR'].mean():.6f} | "
        f"% positive: {event_ar['pos_car'].mean():.1%}"
    )
    return daily_ar, event_ar
```

88.3.9 Step 6: Comprehensive Test Statistics

Eight tests covering parametric, non-parametric, and cross-correlation-robust approaches.

```
def compute_test_statistics(event_ar, params=None, group_col=None):
    """Compute comprehensive test statistics for abnormal returns.

    Implements 8 tests with varying assumptions about variance,
    cross-dependence, and distributional form.
    """
    def _stats(data, label=None):
        N = len(data)
        if N < 3:
            return None

        cars = data['CAR'].values
        bhars = data['BHAR'].values
        scars = data['SCAR'].values
        pos = data['pos_car'].values

        m_car, s_car = np.mean(cars), np.std(cars, ddof=1)
        m_scar, s_scar = np.mean(scars), np.std(scars, ddof=1)

        r = {'group': label or 'All', 'N': N,
             'mean_CAR': m_car, 'median_CAR': np.median(cars),
             'std_CAR': s_car, 'mean_BHAR': np.mean(bhars),
             'pct_positive': np.mean(pos)}

        # 1. Cross-sectional t
        t1 = m_car / (s_car / np.sqrt(N)) if s_car > 0 else np.nan
        r['t_CS'] = t1
```

```
        r['p_CS'] = 2 * (1 - stats.t.cdf(abs(t1), N-1)) if np.isfinite(t1)
↪   else np.nan

        # 2. Patell Z
        if 'pat_scale' in data.columns:
            ps = data['pat_scale'].dropna().values
            z2 = np.sum(scars[:len(ps)]) / np.sqrt(np.sum(ps)) if len(ps) > 0
↪   else np.nan
        else:
            z2 = m_scar * np.sqrt(N)
        r['Z_Patell'] = z2
        r['p_Patell'] = 2*(1-stats.norm.cdf(abs(z2))) if np.isfinite(z2)
↪   else np.nan

        # 3. BMP
        t3 = m_scar / (s_scar / np.sqrt(N)) if s_scar > 0 else np.nan
        r['t_BMP'] = t3
        r['p_BMP'] = 2*(1-stats.t.cdf(abs(t3), N-1)) if np.isfinite(t3) else
↪   np.nan

        # 4. Kolari-Pynnönen
        rbar = 0.0
        if params is not None and '_residuals' in params.columns:
            resids = [row['_residuals'] for _, row in params.iterrows()
                      if isinstance(row.get('_residuals'), np.ndarray)]
            if len(resids) > 1:
                ml = min(len(x) for x in resids)
                aligned = np.column_stack([x[:ml] for x in resids])
                cm = np.corrcoef(aligned.T)
                np.fill_diagonal(cm, 0)
                rbar = cm.sum() / (len(resids) * (len(resids)-1))

        adj = np.sqrt(1/(1+(N-1)*rbar)) if (1+(N-1)*rbar) > 0 else 1
        t4 = t3 * adj if np.isfinite(t3) else np.nan
        r['t_KP'] = t4
        r['p_KP'] = 2*(1-stats.t.cdf(abs(t4), N-1)) if np.isfinite(t4) else
↪   np.nan
        r['r_bar'] = rbar

        # 5. Generalized sign test
        p_hat = np.mean(pos)
        z5 = (p_hat - 0.5) / np.sqrt(0.25 / N)
```

```
        r['Z_GSign'] = z5
        r['p_GSign'] = 2*(1-stats.norm.cdf(abs(z5)))

        # 6. Sign test
        r['Z_Sign'] = z5  # Same formula with p0=0.5
        r['p_Sign'] = r['p_GSign']

        # 7. Skewness-adjusted t
        if s_scar > 0:
            zb = m_scar / s_scar
            gam = stats.skew(scars)
            t7 = np.sqrt(N) * (zb + gam*zb**2/3 + gam**2*zb**3/27 +
↪   gam/(6*N))
            r['t_SkAdj'] = t7
            r['p_SkAdj'] = 2*(1-stats.t.cdf(abs(t7), N-1)) if np.isfinite(t7)
↪   else np.nan

        # 8. Wilcoxon signed-rank
        try:
            w, pw = stats.wilcoxon(cars, alternative='two-sided')
            r['W_Wilcoxon'] = w
            r['p_Wilcoxon'] = pw
        except:
            r['W_Wilcoxon'] = r['p_Wilcoxon'] = np.nan

        return r

    results = [_stats(event_ar)]
    if group_col and group_col in event_ar.columns:
        for gv, gd in event_ar.groupby(group_col):
            s = _stats(gd, label=gv)
            if s:
                results.append(s)

    return pd.DataFrame([r for r in results if r is not None])


def compute_daily_stats(daily_ar, id_col='symbol'):
    """Compute test statistics at each event time t."""
    rows = []
    for t, g in daily_ar.groupby('evttime'):
        n = g[id_col].nunique()
        if n < 2:
```

```
            continue
        m_ar = g['AR'].mean()
        s_ar = g['AR'].std(ddof=1)
        t_ar = m_ar / (s_ar/np.sqrt(n)) if s_ar > 0 else np.nan
        rows.append({'evttime': t, 'N': n, 'mean_AR': m_ar,
                     'mean_CAR': g['CAR'].mean(), 'mean_BHAR':
                      ↪  g['BHAR'].mean(),
                     'mean_cum_ret': g.get('cum_ret', pd.Series()).mean(),
                     't_AR': t_ar})
    return pd.DataFrame(rows).sort_values('evttime')
```

88.3.10 Step 7: Publication-Ready Visualization

```
def plot_event_study(daily_stats, title="Cumulative Abnormal Returns Around
 ↪  Event Date",
                     figsize=(12, 7), save_path=None):
    """Publication-ready event study plot with CAR, BHAR, and daily AR
     ↪  panels."""
    fig, axes = plt.subplots(2, 1, figsize=figsize, height_ratios=[3, 1],
                             gridspec_kw={'hspace': 0.05})
    ds = daily_stats.sort_values('evttime')
    t = ds['evttime'].values

    # Top: cumulative returns
    ax = axes[0]
    ax.plot(t, ds['mean_CAR']*100, color='#2166AC', lw=2.5, label='Mean CAR')
    ax.plot(t, ds['mean_BHAR']*100, color='#B2182B', lw=2, ls='--',
 ↪  label='Mean BHAR')
    if 'mean_cum_ret' in ds.columns:
        ax.plot(t, ds['mean_cum_ret']*100, color='#666', lw=1.5, ls=':',
                label='Mean Cum. Return', alpha=0.7)
    ax.axvline(0, color='k', lw=0.8, alpha=0.5)
    ax.axhline(0, color='k', lw=0.5, alpha=0.3)
    ax.set_ylabel('Cumulative Return (%)', fontsize=12)
    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.legend(loc='upper left', fontsize=10)
    ax.grid(True, alpha=0.2)
    ax.set_xticklabels([])

    # Bottom: daily AR bars
```

```
    ax2 = axes[1]
    colors = ['#2166AC' if v >= 0 else '#B2182B' for v in ds['mean_AR']]
    ax2.bar(t, ds['mean_AR']*100, color=colors, alpha=0.7, width=0.8)
    if 't_AR' in ds.columns:
        sig = np.abs(ds['t_AR'].values) > 1.96
        if sig.any():
            ax2.scatter(t[sig], ds['mean_AR'].values[sig]*100,
                        color='gold', s=40, marker='*', zorder=4,
↪   label='p<0.05')
            ax2.legend(fontsize=9)
    ax2.axvline(0, color='k', lw=0.8, alpha=0.5)
    ax2.axhline(0, color='k', lw=0.5, alpha=0.3)
    ax2.set_xlabel('Event Time (Trading Periods)', fontsize=12)
    ax2.set_ylabel('Mean AR (%)', fontsize=10)
    ax2.grid(True, alpha=0.2)

    for a in axes:
        a.spines['top'].set_visible(False)
        a.spines['right'].set_visible(False)
    plt.tight_layout()
    if save_path:
        fig.savefig(save_path, dpi=300, bbox_inches='tight')
    return fig


def plot_car_distribution(event_ar, var='CAR', figsize=(12, 5)):
    """Cross-sectional distribution of CARs with histogram and QQ plot."""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=figsize)
    data = event_ar[var].dropna() * 100

    ax1.hist(data, bins=50, density=True, alpha=0.6, color='#2166AC',
↪   edgecolor='white')
    ax1.axvline(data.mean(), color='k', ls='--', lw=1.5,
                label=f'Mean={data.mean():.2f}%')
    ax1.axvline(data.median(), color='gray', ls=':', lw=1.5,
                label=f'Median={data.median():.2f}%')
    ax1.set_xlabel(f'{var} (%)')
    ax1.set_ylabel('Density')
    ax1.set_title(f'Distribution of {var}', fontweight='bold')
    ax1.legend()
    ax1.spines['top'].set_visible(False)
    ax1.spines['right'].set_visible(False)
```

```
    # QQ plot
    (osm, osr), (slope, intercept, r) = stats.probplot(data, dist='norm')
    ax2.scatter(osm, osr, alpha=0.4, s=10, color='#2166AC')
    ax2.plot(osm, slope*np.array(osm)+intercept, 'r--', lw=1)
    ax2.set_xlabel('Theoretical Quantiles')
    ax2.set_ylabel('Sample Quantiles')
    ax2.set_title('Q-Q Plot (Normal)', fontweight='bold')
    ax2.spines['top'].set_visible(False)
    ax2.spines['right'].set_visible(False)

    plt.tight_layout()
    return fig
```

88.3.11 The Master Pipeline

Combine all components into one function:

```
def run_event_study(events, prices, factors, config,
                    id_col='symbol', date_col='date', ret_col='ret',
                    event_date_col='event_date', mkt_col='mkt_excess',
                    rf_col='risk_free', group_col=None, verbose=True):
    """Run a complete event study from raw inputs to test statistics.

    This is the main entry point. Provide your events, price data,
    factor data, and configuration-get back everything you need.

    Parameters
    ----------
    events : pd.DataFrame
        Columns: [id_col, event_date_col], optional 'group'.
    prices : pd.DataFrame
        Daily returns: [id_col, date_col, ret_col or 'ret_excess', rf_col].
    factors : pd.DataFrame
        Factor returns: [date_col, mkt_col, 'smb', 'hml', ...].
    config : EventStudyConfig

    Returns
    -------
    dict with keys: 'config', 'daily_ar', 'event_ar', 'daily_stats',
        'test_stats', 'params'
    """
```

```
    config.validate()

    if verbose:
        print(f"   Event Study: {config.risk_model.value} model   ")
        print(f"  Windows: estimation={config.estimation_window}, "
              f"gap={config.gap}, event=({config.event_window_start},{config
               ↪  .event_window_end})")
        print(f"  Min obs: {config.min_estimation_obs}\n")

    # 1. Trading calendar
    if verbose: print("Step 1: Building trading calendar...")
    trading_dates = pd.Series(sorted(prices[date_col].unique()))
    calendar = build_trading_calendar(trading_dates, config)
    if verbose: print(f"  {len(calendar)} potential event dates\n")

    # 2. Align events
    if verbose: print("Step 2: Aligning events to trading calendar...")
    aligned = align_events(events, calendar, id_col, event_date_col)
    if verbose: print(f"  {len(aligned)} aligned events\n")

    # 3. Extract returns
    if verbose: print("Step 3: Extracting returns and merging factors...")
    evt_rets = extract_returns(aligned, prices, factors, config,
                               id_col, date_col, ret_col, mkt_col, rf_col)
    if verbose: print()

    # 4. Estimate model
    if verbose: print("Step 4: Estimating risk model parameters...")
    params = estimate_model(evt_rets, config, id_col, date_col, ret_col)
    if verbose: print()

    # 5. Compute abnormal returns
    if verbose: print("Step 5: Computing abnormal returns...")
    daily_ar, event_ar = compute_abnormal_returns(
        evt_rets, params, config, id_col, date_col, ret_col)
    if verbose: print()

    # 6. Test statistics
    if verbose: print("Step 6: Computing test statistics...")
    test_stats = compute_test_statistics(event_ar, params, group_col)
    daily_stats = compute_daily_stats(daily_ar, id_col)
    if verbose:
        print(f"  Done.\n")
```

```
        print("   Results Summary   ")
        cols = ['group', 'N', 'mean_CAR', 'mean_BHAR', 'pct_positive',
                't_CS', 'p_CS', 't_BMP', 'p_BMP', 't_KP', 'p_KP']
        avail = [c for c in cols if c in test_stats.columns]
        print(test_stats[avail].to_string(index=False))

    return {
        'config': config,
        'params': params,
        'daily_ar': daily_ar,
        'event_ar': event_ar,
        'daily_stats': daily_stats,
        'test_stats': test_stats,
        'calendar': calendar,
    }

print("Master pipeline ready.")
```

```
Master pipeline ready.
```

88.4 Demonstration with Simulated Data

Since we are building a general-purpose framework (the actual event data will be supplied later), we demonstrate the full pipeline with **realistic simulated data**.

```
np.random.seed(2024)

# --- Simulated trading calendar (Vietnamese market: ~245 days/year) ---
dates = pd.bdate_range('2019-01-01', '2023-12-31', freq='B')
# Remove Tet + national holidays (simplified)
tet_holidays = pd.to_datetime([
    '2019-02-04','2019-02-05','2019-02-06','2019-02-07','2019-02-08',
    '2020-01-23','2020-01-24','2020-01-27','2020-01-28','2020-01-29',
    '2021-02-10','2021-02-11','2021-02-12','2021-02-15','2021-02-16',
    '2022-01-31','2022-02-01','2022-02-02','2022-02-03','2022-02-04',
    '2023-01-20','2023-01-23','2023-01-24','2023-01-25','2023-01-26',
])
dates = dates.difference(tet_holidays)
T = len(dates)
```

```
# --- Simulated factors (realistic Vietnamese market parameters) ---
rf_daily = 0.04 / 252  # ~4% annual risk-free
mkt_excess = np.random.normal(0.0003, 0.012, T)  # ~7.5% annual, ~19% vol
smb = np.random.normal(0.0001, 0.006, T)
hml = np.random.normal(0.0001, 0.005, T)
rmw = np.random.normal(0.00005, 0.004, T)
cma = np.random.normal(0.00005, 0.004, T)

factors_sim = pd.DataFrame({
    'date': dates, 'mkt_excess': mkt_excess, 'smb': smb, 'hml': hml,
    'rmw': rmw, 'cma': cma, 'risk_free': rf_daily
})

# --- 100 simulated stocks ---
n_stocks = 100
symbols = [f'SIM{i:03d}' for i in range(n_stocks)]
betas = np.random.uniform(0.5, 1.5, n_stocks)
alphas = np.random.normal(0, 0.0002, n_stocks)
idio_vols = np.random.uniform(0.015, 0.035, n_stocks)

price_rows = []
for i, sym in enumerate(symbols):
    eps = np.random.normal(0, idio_vols[i], T)
    rets = alphas[i] + betas[i] * mkt_excess + 0.3*smb + 0.2*hml + eps
    for j in range(T):
        price_rows.append({
            'symbol': sym, 'date': dates[j], 'ret': rets[j],
            'ret_excess': rets[j] - rf_daily,
            'risk_free': rf_daily,
            'mktcap': np.random.uniform(100, 5000),
        })

prices_sim = pd.DataFrame(price_rows)

# --- Simulated events: 50 random firm-dates with KNOWN positive AR ---
event_indices = np.random.choice(range(250, T-50), 50, replace=False)
event_firms = np.random.choice(symbols, 50, replace=True)
event_dates_sim = [dates[i] for i in event_indices]

# Inject abnormal returns on event date (2% positive shock)
for firm, edate in zip(event_firms, event_dates_sim):
    mask = (prices_sim['symbol'] == firm) & (prices_sim['date'] == edate)
    prices_sim.loc[mask, 'ret'] += 0.02
```

```
    prices_sim.loc[mask, 'ret_excess'] += 0.02

events_sim = pd.DataFrame({
    'symbol': event_firms,
    'event_date': event_dates_sim,
    'group': np.random.choice([1, 2], 50)
})

print(f"Simulated data: {n_stocks} stocks × {T} days = {len(prices_sim):,}
 ↪  obs")
print(f"Events: {len(events_sim)} firm-event pairs")
print(f"Injected abnormal return: +2% on event date")
```

```
Simulated data: 100 stocks × 1279 days = 127,900 obs
Events: 50 firm-event pairs
Injected abnormal return: +2% on event date
```

88.4.1 Running the Full Pipeline

```
config = EventStudyConfig(
    estimation_window=150,
    event_window_start=-10,
    event_window_end=10,
    gap=15,
    min_estimation_obs=120,
    risk_model=RiskModel.FF3
)

results = run_event_study(
    events=events_sim,
    prices=prices_sim,
    factors=factors_sim,
    config=config,
    group_col='group'
)
```

```
  Event Study: ff3 model
 Windows: estimation=150, gap=15, event=(-10,10)
 Min obs: 120
```

```
Step 1: Building trading calendar...
  1094 potential event dates

Step 2: Aligning events to trading calendar...
  50 aligned events

Step 3: Extracting returns and merging factors...
  Extracted 9,300 obs for 50 firm-events

Step 4: Estimating risk model parameters...
  Estimated 50/50 firm-events (mean R^2 = 0.2368)

Step 5: Computing abnormal returns...
  50 firm-events | Mean CAR: 0.033009 | Mean BHAR: 0.032498 | % positive: 60.0%

Step 6: Computing test statistics...
  Done.

   Results Summary
group  N  mean_CAR  mean_BHAR  pct_positive     t_CS     p_CS    t_BMP    p_BMP     t_KP
  All 50  0.033009   0.032498      0.600000 2.288362 0.026468 2.157106 0.035929 2.161291 0.03
    1 20  0.030704   0.028326      0.650000 1.568676 0.133227 1.248382 0.227056 1.249320 0.22
    2 30  0.034545   0.035280      0.566667 1.688848 0.101975 1.734699 0.093413 1.736689 0.09
```

88.4.2 Visualizing Results

```
fig1 = plot_event_study(
    results['daily_stats'],
    title="Event Study: FF3 Model - Simulated Vietnamese Market"
)
plt.show()
```

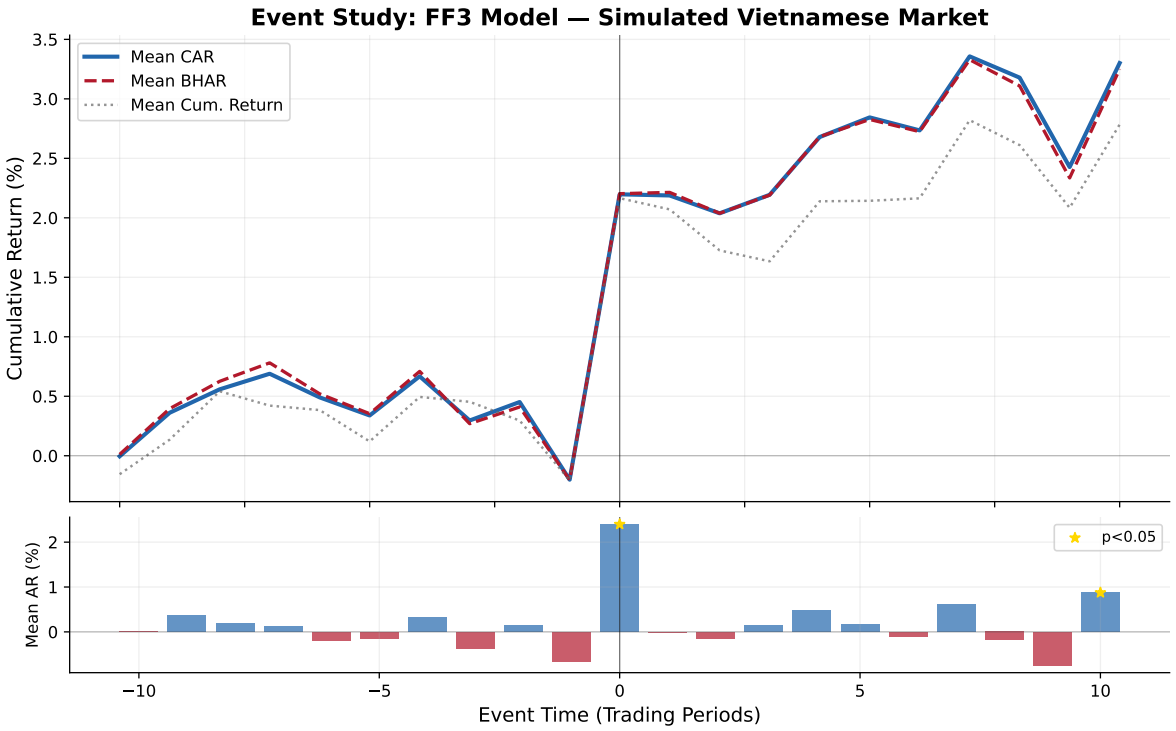


Figure 88.1: Dynamics of cumulative abnormal returns (CARs) and buy-and-hold abnormal returns (BHARs) around the event date. The positive jump at t=0 reflects the injected 2% abnormal return.

```
fig2 = plot_car_distribution(results['event_ar'], 'CAR')
plt.show()
```

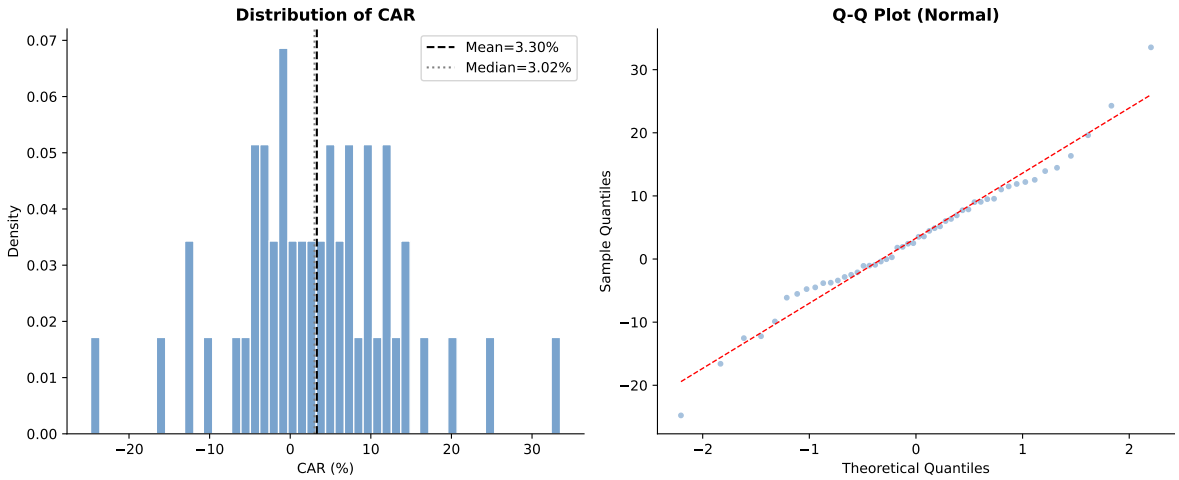


Figure 88.2: Cross-sectional distribution of cumulative abnormal returns. The rightward shift from zero and positive skewness are consistent with the injected positive event effect.

88.4.3 Complete Test Statistics

88.4.4 Running Multiple Models for Robustness

A key best practice is to report results across multiple risk models. If conclusions are robust across models, this strengthens the findings:

```
models_to_run = [
    ("Market-Adjusted", RiskModel.MARKET_ADJ),
    ("Market Model", RiskModel.MARKET_MODEL),
    ("Fama-French 3", RiskModel.FF3),
    ("Fama-French 5", RiskModel.FF5),
]

robustness = []
for name, mdl in models_to_run:
    cfg = EventStudyConfig(
        estimation_window=150, event_window_start=-10, event_window_end=10,
        gap=15, min_estimation_obs=120, risk_model=mdl
    )
    res = run_event_study(events_sim, prices_sim, factors_sim, cfg,
↪   verbose=False)
```

Table 88.3: Event study test statistics for the full sample and by subgroup

```
# Format for display
ts = results['test_stats'].copy()

# Select key columns
display_cols = ['group', 'N', 'mean_CAR', 'mean_BHAR', 'pct_positive',
                't_CS', 'p_CS', 'Z_Patell', 'p_Patell',
                't_BMP', 'p_BMP', 't_KP', 'p_KP',
                'Z_GSign', 'p_GSign', 't_SkAdj', 'p_SkAdj']
avail = [c for c in display_cols if c in ts.columns]
display_df = ts[avail].copy()

# Format
for c in display_df.columns:
    if c in ['N']:
        display_df[c] = display_df[c].astype(int)
    elif c.startswith('p_'):
        display_df[c] = display_df[c].map(lambda x: f'{x:.4f}' if pd.notna(x)
 ↪  else '')
    elif c in ['mean_CAR', 'mean_BHAR']:
        display_df[c] = display_df[c].map(lambda x: f'{x:.4%}' if pd.notna(x)
 ↪  else '')
    elif c == 'pct_positive':
        display_df[c] = display_df[c].map(lambda x: f'{x:.1%}' if pd.notna(x)
 ↪  else '')
    elif isinstance(display_df[c].iloc[0], (int, float, np.floating)):
        display_df[c] = display_df[c].map(lambda x: f'{x:.3f}' if pd.notna(x)
 ↪  else '')

print(display_df.to_string(index=False))
```

```
group  N mean_CAR mean_BHAR pct_positive  t_CS   p_CS Z_Patell p_Patell t_BMP  p_BMP  t_KP
  All 50  3.3009%   3.2498%        60.0% 2.288 0.0265    1.832   0.0669 2.157 0.0359 2.161 0
    1 20  3.0704%   2.8326%        65.0% 1.569 0.1332    1.020   0.3075 1.248 0.2271 1.249 0
    2 30  3.4545%   3.5280%        56.7% 1.689 0.1020    1.532   0.1255 1.735 0.0934 1.737 0
```

```
    ts = res['test_stats']
    full = ts[ts['group'] == 'All'].iloc[0]
    robustness.append({
        'Model': name,
        'N': int(full['N']),
        'Mean CAR': f"{full['mean_CAR']:.4%}",
        'Mean BHAR': f"{full['mean_BHAR']:.4%}",
        '% Positive': f"{full['pct_positive']:.1%}",
        't (CS)': f"{full['t_CS']:.2f}",
        't (BMP)': f"{full['t_BMP']:.2f}",
        't (KP)': f"{full.get('t_KP', np.nan):.2f}",
    })

rob_df = pd.DataFrame(robustness)
print("Robustness Across Risk Models:")
print(rob_df.to_string(index=False))
```

```
  Extracted 9,300 obs for 50 firm-events
  Estimated 50/50 firm-events (mean R^2 = 0.0000)
  50 firm-events | Mean CAR: 0.029672 | Mean BHAR: 0.026468 | % positive: 62.0%
  Extracted 9,300 obs for 50 firm-events
  Estimated 50/50 firm-events (mean R^2 = 0.2198)
  50 firm-events | Mean CAR: 0.033785 | Mean BHAR: 0.029974 | % positive: 62.0%
  Extracted 9,300 obs for 50 firm-events
  Estimated 50/50 firm-events (mean R^2 = 0.2368)
  50 firm-events | Mean CAR: 0.033009 | Mean BHAR: 0.032498 | % positive: 60.0%
  Extracted 9,300 obs for 50 firm-events
  Estimated 50/50 firm-events (mean R^2 = 0.2516)
  50 firm-events | Mean CAR: 0.036479 | Mean BHAR: 0.036000 | % positive: 64.0%
Robustness Across Risk Models:
          Model  N Mean CAR Mean BHAR % Positive t (CS) t (BMP) t (KP)
Market-Adjusted 50  2.9672%   2.6468%      62.0%   2.12    2.03   2.01
   Market Model 50  3.3785%   2.9974%      62.0%   2.37    2.26   2.28
  Fama-French 3 50  3.3009%   3.2498%      60.0%   2.29    2.16   2.16
  Fama-French 5 50  3.6479%   3.6000%      64.0%   2.51    2.36   2.41
```

88.5 How to Use This Framework with Your Data

88.5.1 Required Data Format

To run the event study on real Vietnamese market data, prepare three inputs:

1. **Stock Returns** (`prices` DataFrame):

Column	Description	Example
`symbol`	Stock ticker	`'VNM'`
`date`	Trading date	`2023-06-15`
`ret` or `ret_excess`	Daily return (decimal)	`0.0123`
`risk_free`	Daily risk-free rate	`0.000159`

2. **Factor Returns** (`factors` DataFrame):

Column	Description
`date`	Trading date
`mkt_excess`	Market excess return
`smb`	Size factor (FF3/FF5)
`hml`	Value factor (FF3/FF5)
`rmw`	Profitability factor (FF5)
`cma`	Investment factor (FF5)
`risk_free`	Risk-free rate

3. **Event File** (`events` DataFrame):

Column	Description	Example
`symbol`	Stock ticker	`'VNM'`
`event_date`	Event date	`2023-03-15`
`group`	(Optional) subgroup	`1`

88.5.2 Minimal Usage Example

```
# Load your data
prices = pd.read_csv('prices_daily.csv', parse_dates=['date'])
factors = pd.read_csv('factors_ff3_daily.csv', parse_dates=['date'])
events = pd.read_csv('my_events.csv', parse_dates=['event_date'])

# Configure
config = EventStudyConfig(
    estimation_window=150,
    event_window_start=-5,
```

```
    event_window_end=5,
    gap=15,
    min_estimation_obs=120,
    risk_model=RiskModel.FF3
)

# Run
results = run_event_study(events, prices, factors, config)

# Access outputs
results['test_stats']    # Test statistics
results['event_ar']      # Firm-level CARs/BHARs
results['daily_ar']      # Daily abnormal returns
results['daily_stats']   # Event-time aggregates

# Plot
plot_event_study(results['daily_stats'], title="My Event Study")
```

88.6 Demonstration with Vietnamese Market Data

We now demonstrate the full event study pipeline using actual Vietnamese stock market data. The datasets available are:

- `prices_daily`: `symbol`, `date`, `ret_excess`, `mktcap`, `mktcap_lag`, `risk_free`
- `prices_monthly`: same structure
- `factors_ff3_daily`: `date`, `smb`, `hml`, `mkt_excess`, `risk_free`
- `factors_ff3_monthly` — monthly frequency version
- `factors_ff5_daily`: `date`, `smb`, `hml`, `mkt_excess`, `risk_free`, `rmw`, `cma`
- `factors_ff5_monthly`

Since our data provides `ret_excess` rather than raw returns, we recover raw returns as $R_{it} = R^e_{it} + R_{f,t}$, and the market return as $R_{m,t} = R^e_{m,t} + R_{f,t}$. The `extract_event_returns()` function handles this automatically.

88.6.1 Loading the Data

```
# --- Recover raw returns ---
# ret = ret_excess + risk_free
prices_daily['ret'] = prices_daily['ret_excess'] + prices_daily['risk_free']
```

```
prices_monthly['ret'] = prices_monthly['ret_excess'] +
 ↪  prices_monthly['risk_free']

# --- Inspect the data ---
print("=" * 70)
print("VIETNAMESE MARKET DATA SUMMARY")
print("=" * 70)
print(f"\nprices_daily: {prices_daily.shape[0]:,} rows, "
      f"{prices_daily['symbol'].nunique()} stocks, "
      f"{prices_daily['date'].min().date()} to
       ↪  {prices_daily['date'].max().date()}")
print(f"prices_monthly: {prices_monthly.shape[0]:,} rows, "
      f"{prices_monthly['symbol'].nunique()} stocks")
print(f"\nfactors_ff3_daily: {factors_ff3_daily.shape[0]:,} trading days")
print(f"  Columns: {list(factors_ff3_daily.columns)}")
print(f"factors_ff5_daily: {factors_ff5_daily.shape[0]:,} trading days")
print(f"  Columns: {list(factors_ff5_daily.columns)}")
print(f"\nSample daily returns:")
print(prices_daily[['symbol', 'date', 'ret_excess', 'ret', 'risk_free',
 ↪  'mktcap']]
      .head(5).to_string(index=False))
print(f"\nSample daily factors:")
print(factors_ff3_daily.head(5).to_string(index=False))
```

```
======================================================================
VIETNAMESE MARKET DATA SUMMARY
======================================================================

prices_daily: 3,462,157 rows, 1459 stocks, 2010-01-05 to 2023-12-29
prices_monthly: 165,499 rows, 1457 stocks

factors_ff3_daily: 3,126 trading days
  Columns: ['date', 'smb', 'hml', 'mkt_excess', 'risk_free']
factors_ff5_daily: 3,126 trading days
  Columns: ['date', 'smb', 'hml', 'rmw', 'cma', 'mkt_excess', 'risk_free']

Sample daily returns:
symbol       date  ret_excess      ret  risk_free     mktcap
   A32 2018-10-24   -0.000159 0.000000   0.000159 176.120000
   A32 2018-10-25   -0.000159 0.000000   0.000159 176.120000
   A32 2018-10-26   -0.000159 0.000000   0.000159 176.120000
   A32 2018-10-29   -0.000159 0.000000   0.000159 176.120000
```

```
   A32 2018-10-30   -0.000159 0.000000   0.000159 176.120000

Sample daily factors:
      date       smb       hml  mkt_excess  risk_free
2011-07-01  0.008587  0.000967   -0.019862   0.000159
2011-07-04  0.005099 -0.001099   -0.000633   0.000159
2011-07-05 -0.009088  0.010152    0.013314   0.000159
2011-07-06  0.004875 -0.003918   -0.008045   0.000159
2011-07-07 -0.011239 -0.000584    0.003391   0.000159
```

88.6.2 Creating Sample Events

For this demonstration, we create a sample event file. In practice, events would come from corporate announcements (earnings, M&A, dividends), regulatory changes, or other information shocks. Here we select 50 large-cap Vietnamese stocks and assign random event dates from the most recent two years of data to illustrate the pipeline mechanics.

```
np.random.seed(2024)

# Select the 50 largest stocks by median market cap
largest = (prices_daily.groupby('symbol')['mktcap']
           .median()
           .nlargest(50)
           .index.tolist())

# Date range for events: last 2 years of data, with buffer for windows
date_range = prices_daily['date'].sort_values().unique()
n_dates = len(date_range)
# Events from the middle portion (need room for estimation + event windows)
event_eligible = date_range[int(n_dates * 0.3):int(n_dates * 0.85)]

# Generate 50 random firm-event pairs
event_firms = np.random.choice(largest, 50, replace=True)
event_dates = np.random.choice(event_eligible, 50, replace=False)

events_demo = pd.DataFrame({
    'symbol': event_firms,
    'event_date': pd.to_datetime(event_dates),
    'group': np.random.choice(['Group_A', 'Group_B'], 50)
})

# Remove any duplicate firm-date pairs
```

```
events_demo = events_demo.drop_duplicates(subset=['symbol', 'event_date'])

print(f"Sample event file: {len(events_demo)} firm-event observations")
print(f"Unique firms: {events_demo['symbol'].nunique()}")
print(f"Date range: {events_demo['event_date'].min().date()} to "
      f"{events_demo['event_date'].max().date()}")
print(f"\nGroup distribution:")
print(events_demo['group'].value_counts().to_string())
print(f"\nFirst 10 events:")
print(events_demo.sort_values('event_date').head(10).to_string(index=False))
```

```
Sample event file: 50 firm-event observations
Unique firms: 35
Date range: 2014-06-25 to 2021-10-29

Group distribution:
group
Group_B    26
Group_A    24

First 10 events:
symbol event_date   group
   MCH 2014-06-25 Group_B
   SIP 2014-10-23 Group_B
   VRE 2014-11-14 Group_B
   QNS 2014-12-25 Group_A
   FOX 2015-01-16 Group_B
   THD 2015-01-26 Group_A
   QNS 2015-02-12 Group_A
   HNG 2015-05-07 Group_B
   MML 2015-08-17 Group_B
   ACV 2015-10-15 Group_A
```

88.6.3 Daily Event Study: Fama-French 3-Factor Model

```
config_ff3 = EventStudyConfig(
    estimation_window=150,
    event_window_start=-10,
    event_window_end=10,
    gap=15,
```

```
    min_estimation_obs=120,
    risk_model=RiskModel.FF3
)

results_ff3 = run_event_study(
    events=events_demo,
    prices=prices_daily,
    factors=factors_ff3_daily,
    config=config_ff3,
    group_col='group'
)
```

```
   Event Study: ff3 model
  Windows: estimation=150, gap=15, event=(-10,10)
  Min obs: 120

Step 1: Building trading calendar...
  3308 potential event dates

Step 2: Aligning events to trading calendar...
  50 aligned events

Step 3: Extracting returns and merging factors...
  Extracted 5,421 obs for 30 firm-events

Step 4: Estimating risk model parameters...
  Estimated 26/30 firm-events (mean R^2 = 0.2245)

Step 5: Computing abnormal returns...
  26 firm-events | Mean CAR: 0.021513 | Mean BHAR: 0.024885 | % positive: 61.5%

Step 6: Computing test statistics...
  Done.

   Results Summary
  group  N  mean_CAR  mean_BHAR  pct_positive     t_CS     p_CS    t_BMP    p_BMP     t_KP
    All 26  0.021513   0.024885      0.615385 0.774999 0.445609 0.726797 0.474101 0.699973 0
Group_A 13  0.008601   0.006783      0.538462 0.211606 0.835966 0.577574 0.574229 0.567041 0
Group_B 13  0.034425   0.042987      0.692308 0.879892 0.396197 0.437902 0.669236 0.429917 0
```

88.6.4 Visualizing Daily Results

```
fig1 = plot_event_study(
    results_ff3['daily_stats'],
    title="Event Study: Fama-French 3-Factor Model - Vietnamese Market
 ↪  (Daily)"
)
plt.show()
```

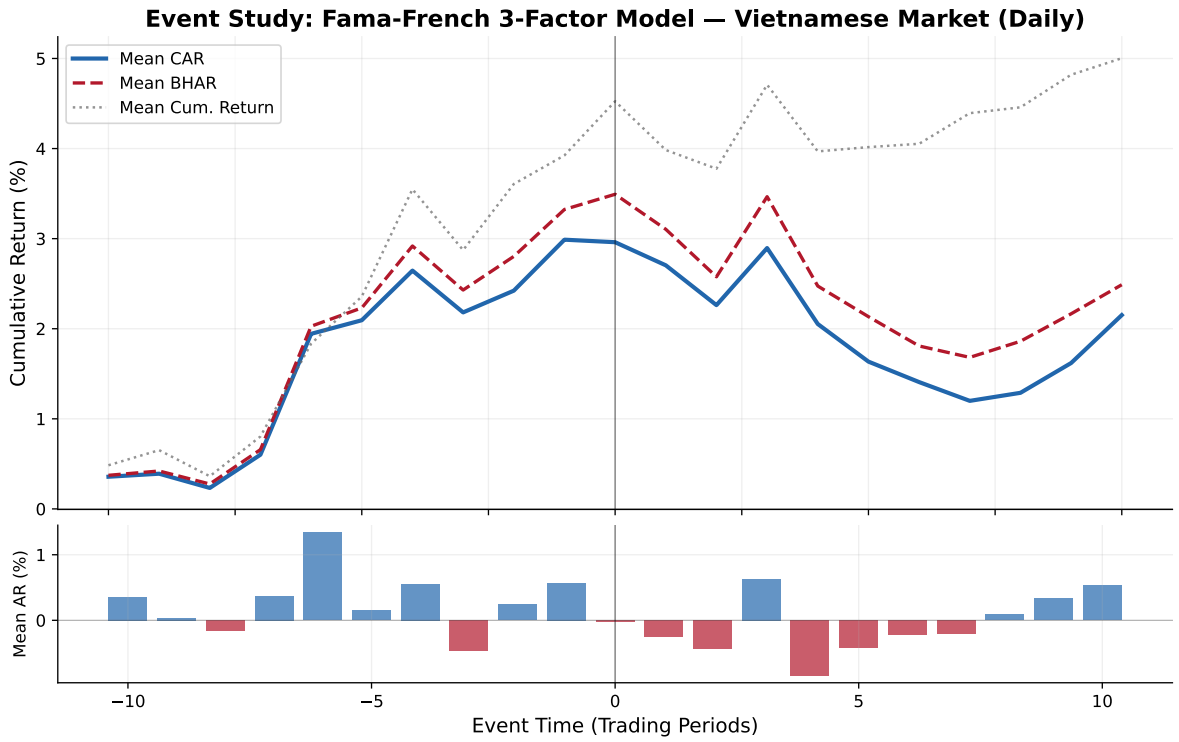


Figure 88.3: Cumulative abnormal returns around event dates for Vietnamese stocks using the Fama-French 3-factor model. The event window spans [-10, +10] trading days.

```
fig2 = plot_car_distribution(results_ff3['event_ar'], 'CAR')
plt.show()
```

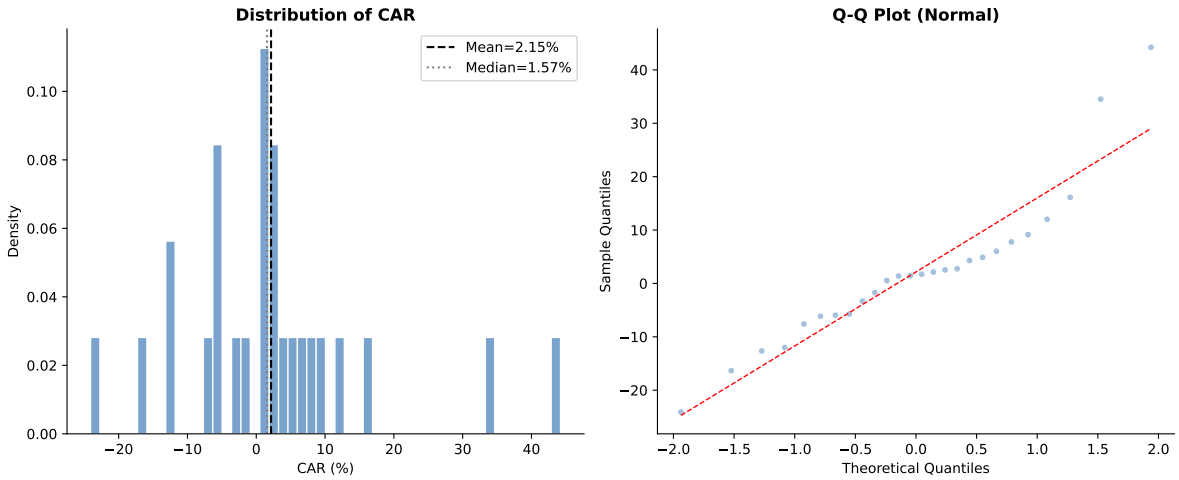


Figure 88.4: Cross-sectional distribution of cumulative abnormal returns (CARs) across firm-events. The histogram and Q-Q plot assess normality assumptions underlying parametric tests.

88.6.5 Complete Test Statistics (Daily)

88.6.6 Robustness: Multiple Risk Models (Daily)

88.6.7 Robustness: Multiple Event Windows

A key practice is to examine sensitivity to the event window specification:

88.6.8 Monthly Event Study: Fama-French 3-Factor Model

For longer-horizon studies, monthly frequency is appropriate. Note that the estimation window is specified in months rather than days:

```
# Create monthly events aligned to the monthly data
# Map daily event dates to the corresponding month-end
events_monthly = events_demo.copy()
events_monthly['event_date'] =
 ↪  events_monthly['event_date'].dt.to_period('M').dt.to_timestamp('M')

# Use month-end dates from monthly prices
```

Table 88.7: Event study test statistics for the full sample and by subgroup — Daily frequency, FF3 model

```
ts = results_ff3['test_stats'].copy()

display_cols = ['group', 'N', 'mean_CAR', 'median_CAR', 'mean_BHAR',
 ↪  'pct_positive',
                't_CS', 'p_CS', 'Z_Patell', 'p_Patell',
                't_BMP', 'p_BMP', 't_KP', 'p_KP',
                'Z_GSign', 'p_GSign', 't_SkAdj', 'p_SkAdj',
                'W_Wilcoxon', 'p_Wilcoxon']
avail = [c for c in display_cols if c in ts.columns]
display_df = ts[avail].copy()

for c in display_df.columns:
    if c in ['N']:
        display_df[c] = display_df[c].astype(int)
    elif c == 'group':
        continue
    elif c.startswith('p_'):
        display_df[c] = display_df[c].map(lambda x: f'{x:.4f}' if pd.notna(x)
 ↪  else '')
    elif c in ['mean_CAR', 'median_CAR', 'mean_BHAR']:
        display_df[c] = display_df[c].map(lambda x: f'{x:.4%}' if pd.notna(x)
 ↪  else '')
    elif c == 'pct_positive':
        display_df[c] = display_df[c].map(lambda x: f'{x:.1%}' if pd.notna(x)
 ↪  else '')
    elif isinstance(display_df[c].iloc[0], (int, float, np.floating)):
        display_df[c] = display_df[c].map(lambda x: f'{x:.3f}' if pd.notna(x)
 ↪  else '')

print(display_df.to_string(index=False))
```

```
  group  N mean_CAR median_CAR mean_BHAR pct_positive  t_CS   p_CS Z_Patell p_Patell t_BMP  p
    All 26  2.1513%    1.5701%   2.4885%        61.5% 0.775 0.4456    0.961   0.3364 0.727 0
Group_A 13  0.8601%    2.5330%   0.6783%        53.8% 0.212 0.8360    0.739   0.4596 0.578 0
Group_B 13  3.4425%    1.4169%   4.2987%        69.2% 0.880 0.3962    0.620   0.5352 0.438 0
```

Table 88.8: Robustness of event study results across risk models — Daily frequency

```
models_daily = [
    ("Market-Adjusted",  RiskModel.MARKET_ADJ,    factors_ff3_daily),
    ("Market Model",     RiskModel.MARKET_MODEL,   factors_ff3_daily),
    ("Fama-French 3",    RiskModel.FF3,            factors_ff3_daily),
    ("Fama-French 5",    RiskModel.FF5,            factors_ff5_daily),
]

robustness_daily = []
for name, mdl, facs in models_daily:
    cfg = EventStudyConfig(
        estimation_window=150, event_window_start=-10, event_window_end=10,
        gap=15, min_estimation_obs=120, risk_model=mdl
    )
    res = run_event_study(events_demo, prices_daily, facs, cfg,
 ↪  verbose=False)
    ts = res['test_stats']
    full = ts[ts['group'] == 'All'].iloc[0]
    robustness_daily.append({
        'Model': name,
        'N': int(full['N']),
        'Mean CAR': f"{full['mean_CAR']:.4%}",
        'Median CAR': f"{full['median_CAR']:.4%}",
        'Mean BHAR': f"{full['mean_BHAR']:.4%}",
        '% Positive': f"{full['pct_positive']:.1%}",
        't (CS)': f"{full['t_CS']:.3f}",
        'p (CS)': f"{full['p_CS']:.4f}",
        't (BMP)': f"{full['t_BMP']:.3f}",
        'p (BMP)': f"{full['p_BMP']:.4f}",
        't (KP)': f"{full.get('t_KP', np.nan):.3f}",
        'p (KP)': f"{full.get('p_KP', np.nan):.4f}",
    })

rob_daily_df = pd.DataFrame(robustness_daily)
print("Robustness Across Risk Models (Daily Frequency)")
print("=" * 100)
print(rob_daily_df.to_string(index=False))
```

```
  Extracted 5,421 obs for 30 firm-events
  Estimated 28/30 firm-events (mean R^2 = 0.0000)
  28 firm-events | Mean CAR: 0.035338 | Mean BHAR: 0.036936 | % positive: 50.0%
  Extracted 5,421 obs for 30 firm-events
  Estimated 26/30 firm-events (mean R^2 = 0.1960)
  26 firm-events | Mean CAR: 0.032107 | Mean BHAR: 0.033221 | % positive: 61.5%
  Extracted 5,421 obs for 30 firm-events
  Estimated 26/30 firm-events (mean R^2 = 0.2245)
  26 firm-events | Mean CAR: 0.021513 | Mean BHAR: 0.024885 | % positive: 61.5%
  Extracted 5,421 obs for 30 firm-events
  Estimated 26/30 firm-events (mean R^2 = 0.2675)
  26 firm-events | Mean CAR: 0.025684 | Mean BHAR: 0.028399 | % positive: 57.7%
Robustness Across Risk Models (Daily Frequency)
====================================================================================
```

Table 88.9: Sensitivity of results to event window specification

```
windows = [
    ("(-1, +1)",  -1, 1),
    ("(-3, +3)",  -3, 3),
    ("(-5, +5)",  -5, 5),
    ("(-10, +10)", -10, 10),
    ("(-1, +5)",  -1, 5),
    ("(-5, +1)",  -5, 1),
    ("(0, 0)",     0, 0),
]

window_results = []
for label, ws, we in windows:
    cfg = EventStudyConfig(
        estimation_window=150, event_window_start=ws, event_window_end=we,
        gap=15, min_estimation_obs=120, risk_model=RiskModel.FF3
    )
    res = run_event_study(events_demo, prices_daily, factors_ff3_daily, cfg,
 ↪  verbose=False)
    ts = res['test_stats']
    full = ts[ts['group'] == 'All'].iloc[0]
    window_results.append({
        'Window': label,
        'Days': we - ws + 1,
        'N': int(full['N']),
        'Mean CAR': f"{full['mean_CAR']:.4%}",
        'Mean BHAR': f"{full['mean_BHAR']:.4%}",
        '% Positive': f"{full['pct_positive']:.1%}",
        't (CS)': f"{full['t_CS']:.3f}",
        't (BMP)': f"{full['t_BMP']:.3f}",
        'p (BMP)': f"{full['p_BMP']:.4f}",
    })

win_df = pd.DataFrame(window_results)
print("Sensitivity to Event Window Specification (FF3 Model)")
print("=" * 90)
print(win_df.to_string(index=False))
```

```
  Extracted 4,899 obs for 30 firm-events
  Estimated 26/30 firm-events (mean R^2 = 0.2147)
  26 firm-events | Mean CAR: 0.004074 | Mean BHAR: 0.004648 | % positive: 50.0%
  Extracted 5,015 obs for 30 firm-events
  Estimated 26/30 firm-events (mean R^2 = 0.2155)
  26 firm-events | Mean CAR: 0.003761 | Mean BHAR: 0.004327 | % positive: 42.3%
  Extracted 5,131 obs for 30 firm-events
  Estimated 26/30 firm-events (mean R^2 = 0.2217)
  26 firm-events | Mean CAR: -0.001133 | Mean BHAR: 0.001027 | % positive: 42.3%
  Extracted 5,421 obs for 30 firm-events
  Estimated 26/30 firm-events (mean R^2 = 0.2245)
  26 firm-events | Mean CAR: 0.021513 | Mean BHAR: 0.024885 | % positive: 61.5%
  Extracted 5,019 obs for 30 firm-events
  Estimated 26/30 firm-events (mean R^2 = 0.2147)
```

```
monthly_dates = prices_monthly['date'].sort_values().unique()

# Filter events to dates present in monthly data
events_monthly =
 ↪  events_monthly[events_monthly['event_date'].isin(monthly_dates)]
events_monthly = events_monthly.drop_duplicates(subset=['symbol',
 ↪  'event_date'])

config_monthly = EventStudyConfig(
    estimation_window=36,     # 36 months
    event_window_start=-3,    # 3 months before
    event_window_end=3,       # 3 months after
    gap=3,                    # 3-month gap
    min_estimation_obs=24,    # At least 24 months
    risk_model=RiskModel.FF3
)

if len(events_monthly) > 0:
    results_monthly = run_event_study(
        events=events_monthly,
        prices=prices_monthly,
        factors=factors_ff3_monthly,
        config=config_monthly,
        group_col='group'
    )

    print("\n--- Monthly Test Statistics ---")
    ts_m = results_monthly['test_stats']
    mcols = ['group', 'N', 'mean_CAR', 'mean_BHAR', 'pct_positive',
             't_CS', 'p_CS', 't_BMP', 'p_BMP']
    mavail = [c for c in mcols if c in ts_m.columns]
    print(ts_m[mavail].to_string(index=False))
else:
    print("No monthly events could be aligned. Skipping monthly study.")
```

```
  Event Study: ff3 model
  Windows: estimation=36, gap=3, event=(-3,3)
  Min obs: 24

Step 1: Building trading calendar...
  122 potential event dates
```

```
Step 2: Aligning events to trading calendar...
  50 aligned events

Step 3: Extracting returns and merging factors...
  Extracted 1,036 obs for 33 firm-events

Step 4: Estimating risk model parameters...
  Estimated 18/33 firm-events (mean R^2 = 0.3218)

Step 5: Computing abnormal returns...
  18 firm-events | Mean CAR: -0.005257 | Mean BHAR: -0.014576 | % positive: 55.6%

Step 6: Computing test statistics...
  Done.

   Results Summary
  group  N  mean_CAR  mean_BHAR  pct_positive      t_CS     p_CS     t_BMP    p_BMP      t_KI
    All 18 -0.005257  -0.014576      0.555556 -0.081058 0.936342 -0.249547 0.805928 -0.320212
Group_A  7 -0.141756  -0.135084      0.428571 -1.102110 0.312648 -1.357452 0.223472 -1.462577
Group_B 11  0.081606   0.062111      0.636364  1.390317 0.194599  1.177372 0.266309  1.342593

--- Monthly Test Statistics ---
  group  N  mean_CAR  mean_BHAR  pct_positive      t_CS     p_CS     t_BMP    p_BMP
    All 18 -0.005257  -0.014576      0.555556 -0.081058 0.936342 -0.249547 0.805928
Group_A  7 -0.141756  -0.135084      0.428571 -1.102110 0.312648 -1.357452 0.223472
Group_B 11  0.081606   0.062111      0.636364  1.390317 0.194599  1.177372 0.266309
```

```python
if len(events_monthly) > 0 and 'daily_stats' in results_monthly:
    fig3 = plot_event_study(
        results_monthly['daily_stats'],
        title="Event Study: FF3 Model - Vietnamese Market (Monthly)"
    )
    plt.show()
```

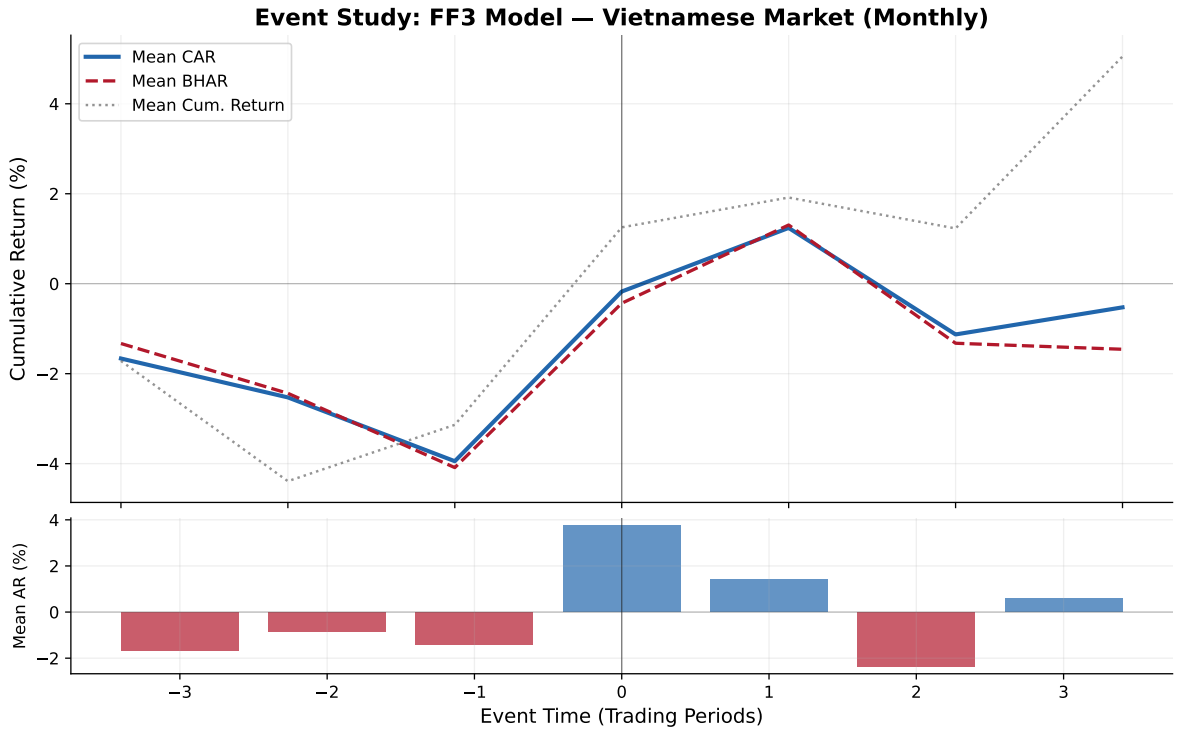


Figure 88.5: Monthly cumulative abnormal returns around event dates. Wider windows capture slower information incorporation typical of emerging markets.

88.6.9 Daily Event Study: Fama-French 5-Factor Model

```
config_ff5 = EventStudyConfig(
    estimation_window=150,
    event_window_start=-10,
    event_window_end=10,
    gap=15,
    min_estimation_obs=120,
    risk_model=RiskModel.FF5
)

results_ff5 = run_event_study(
    events=events_demo,
    prices=prices_daily,
    factors=factors_ff5_daily,
    config=config_ff5,
```

```
    group_col='group'
)
```

```
   Event Study: ff5 model
  Windows: estimation=150, gap=15, event=(-10,10)
  Min obs: 120

Step 1: Building trading calendar...
  3308 potential event dates

Step 2: Aligning events to trading calendar...
  50 aligned events

Step 3: Extracting returns and merging factors...
  Extracted 5,421 obs for 30 firm-events

Step 4: Estimating risk model parameters...
  Estimated 26/30 firm-events (mean R^2 = 0.2675)

Step 5: Computing abnormal returns...
  26 firm-events | Mean CAR: 0.025684 | Mean BHAR: 0.028399 | % positive: 57.7%

Step 6: Computing test statistics...
  Done.

   Results Summary
  group  N  mean_CAR  mean_BHAR  pct_positive     t_CS     p_CS    t_BMP    p_BMP     t_KP
    All 26  0.025684   0.028399      0.576923 0.968332 0.342154 0.986788 0.333201 0.962252 0
Group_A 13  0.015352   0.013489      0.461538 0.393655 0.700741 0.786232 0.446980 0.776663 0
Group_B 13  0.036016   0.043309      0.692308 0.965100 0.353542 0.585915 0.568789 0.578785 0
```

88.6.10 Comparing FF3 vs FF5 Estimation Quality

88.6.11 Event-Level Detail

88.6.12 Daily Abnormal Return Dynamics

Table 88.10: Comparison of estimation quality between FF3 and FF5 models

```
params_ff3 = results_ff3['params']
params_ff5 = results_ff5['params']

print("Model Estimation Diagnostics")
print("=" * 60)
print(f"\n{'Metric':<30} {'FF3':>12} {'FF5':>12}")
print("-" * 54)
print(f"{'Firm-events estimated':<30} {len(params_ff3):>12}
 ↪  {len(params_ff5):>12}")
print(f"{'Mean R^2':<30} {params_ff3['r_squared'].mean():>12.4f}
 ↪  {params_ff5['r_squared'].mean():>12.4f}")
print(f"{'Median R^2':<30} {params_ff3['r_squared'].median():>12.4f}
 ↪  {params_ff5['r_squared'].median():>12.4f}")
print(f"{'Mean  ( )':<30} {params_ff3['sigma'].mean():>12.6f}
 ↪  {params_ff5['sigma'].mean():>12.6f}")
print(f"{'Mean | |':<30} {params_ff3['alpha'].abs().mean():>12.6f}
 ↪  {params_ff5['alpha'].abs().mean():>12.6f}")
print(f"{'Mean  (MKT)':<30} {params_ff3['beta_mkt_excess'].mean():>12.4f}
 ↪  {params_ff5['beta_mkt_excess'].mean():>12.4f}")
if 'beta_smb' in params_ff3.columns:
    print(f"{'Mean  (SMB)':<30} {params_ff3['beta_smb'].mean():>12.4f}
     ↪  {params_ff5['beta_smb'].mean():>12.4f}")
if 'beta_hml' in params_ff3.columns:
    print(f"{'Mean  (HML)':<30} {params_ff3['beta_hml'].mean():>12.4f}
     ↪  {params_ff5['beta_hml'].mean():>12.4f}")
if 'beta_rmw' in params_ff5.columns:
    print(f"{'Mean  (RMW)':<30} {'-':>12}
     ↪  {params_ff5['beta_rmw'].mean():>12.4f}")
if 'beta_cma' in params_ff5.columns:
    print(f"{'Mean  (CMA)':<30} {'-':>12}
     ↪  {params_ff5['beta_cma'].mean():>12.4f}")
```

```
Model Estimation Diagnostics
============================================================

Metric                                  FF3          FF5
------------------------------------------------------
Firm-events estimated                    26           26
Mean R^2                             0.2245       0.2675
Median R^2                           0.1943       0.2692
Mean  ( )                          0.021753     0.021351
Mean | |                           0.001022     0.001130
Mean  (MKT)                          0.8867       0.9721
Mean  (SMB)                         -0.1434       0.0265
Mean  (HML)                          0.2489       0.1493
Mean  (RMW)                               -      -0.0934
Mean  (CMA)                               -       0.1070
```

Table 88.11: Event-level detail: CARs and BHARs for each firm-event (FF3 model)

```
detail = results_ff3['event_ar'].copy()
detail_cols = ['symbol', 'evtdate', 'CAR', 'BHAR', 'SCAR', 'sigma',
               'nobs', 'alpha', 'beta_mkt_excess']
detail_avail = [c for c in detail_cols if c in detail.columns]
detail_show = detail[detail_avail].copy()
detail_show['CAR'] = detail_show['CAR'].map(lambda x: f'{x:.4%}')
detail_show['BHAR'] = detail_show['BHAR'].map(lambda x: f'{x:.4%}')
detail_show['SCAR'] = detail_show['SCAR'].map(lambda x: f'{x:.3f}')

print("Event-Level Results (first 20 firm-events)")
print("=" * 100)
print(detail_show.head(20).to_string(index=False))
```

```
Event-Level Results (first 20 firm-events)
====================================================================================================
symbol    evtdate       CAR      BHAR   SCAR    sigma  nobs     alpha  beta_mkt_excess
   BVH 2016-10-20 -12.6456% -11.6522% -1.603 0.017219   150  0.001193         1.449182
   DHG 2016-01-25  16.1151%  17.1149%  2.273 0.015472   150 -0.000224         0.754245
   DNH 2019-10-14   1.7233%   1.8068%  0.104 0.036035   150 -0.000781         1.861996
   DPM 2020-07-30  -5.9576%  -6.2025% -0.545 0.023873   150  0.001279         0.313932
   FOX 2021-01-13   2.7478%   2.9194%  0.329 0.018243   150 -0.000696         0.148058
   GAS 2020-02-11   0.5371%   0.7801%  0.100 0.011694   150  0.000501         1.851155
   GEX 2020-08-20  11.9985%  13.5843%  1.197 0.021883   150  0.000944         1.583418
   IDC 2018-10-01   1.3889%   0.5651%  0.094 0.032120   150 -0.000674         0.809305
   MML 2021-10-29 -12.0139% -12.3759% -1.141 0.022985   150  0.002976         0.260588
   MSN 2015-10-23  -5.7205%  -5.5645% -0.718 0.017375   150  0.001177         0.453951
   PGV 2019-06-26  -7.5892%  -9.1366% -0.359 0.046165   150  0.000502         0.151377
   PLX 2020-01-07   2.5330%   2.7847%  0.423 0.013067   150 -0.000176         0.917578
   PLX 2020-06-01  -6.1517%  -6.0085% -0.739 0.018168   150 -0.000465         1.815178
   POW 2020-12-23   7.7751%   9.1225%  1.130 0.015014   150 -0.000575         0.774423
   PVD 2020-05-25   4.8827%   5.6674%  0.510 0.020875   150 -0.001522         1.316102
   PVS 2017-08-07   2.1360%   2.1918%  0.297 0.015715   150 -0.000341         1.136273
   QNS 2018-01-18   4.3001%   3.4011%  0.530 0.017703   150 -0.003716         0.128328
   SAB 2017-08-24   6.0347%   6.7732%  0.748 0.017614   150  0.003284         2.123751
   SNZ 2019-03-12  34.5230%  33.1105%  2.145 0.035121   150  0.000235        -1.015554
   VCI 2020-01-20  -3.3191%  -2.9072% -0.458 0.015797   150  0.000335        -0.086268
```

Table 88.12: Daily dynamics of mean abnormal returns and test statistics within the event window

```
ds = results_ff3['daily_stats'].copy()
ds_cols = ['evttime', 'N', 'mean_AR', 'mean_CAR', 'mean_BHAR', 't_AR_CS',
 ↪  't_AR_BMP']
ds_avail = [c for c in ds_cols if c in ds.columns]
ds_show = ds[ds_avail].copy()

for c in ['mean_AR', 'mean_CAR', 'mean_BHAR']:
    if c in ds_show.columns:
        ds_show[c] = ds_show[c].map(lambda x: f'{x:.4%}')
for c in ['t_AR_CS', 't_AR_BMP']:
    if c in ds_show.columns:
        ds_show[c] = ds_show[c].map(lambda x: f'{x:.3f}' if pd.notna(x) else
 ↪  '')

print("Daily Event-Window Dynamics (FF3 Model)")
print("=" * 80)
print(ds_show.to_string(index=False))
```

```
Daily Event-Window Dynamics (FF3 Model)
================================================================================
 evttime  N  mean_AR mean_CAR mean_BHAR
     -10 23  0.3571%  0.3571%   0.3729%
      -9 23  0.0337%  0.3907%   0.4209%
      -8 23 -0.1581%  0.2326%   0.2766%
      -7 23  0.3691%  0.6018%   0.6581%
      -6 23  1.3416%  1.9433%   2.0278%
      -5 23  0.1509%  2.0942%   2.2313%
      -4 23  0.5512%  2.6454%   2.9187%
      -3 23 -0.4641%  2.1814%   2.4297%
      -2 23  0.2412%  2.4225%   2.8028%
      -1 23  0.5660%  2.9885%   3.3240%
       0 23 -0.0281%  2.9604%   3.4926%
       1 23 -0.2564%  2.7040%   3.1039%
       2 23 -0.4421%  2.2619%   2.5764%
       3 23  0.6337%  2.8956%   3.4659%
       4 23 -0.8432%  2.0524%   2.4738%
       5 23 -0.4174%  1.6350%   2.1339%
       6 23 -0.2272%  1.4078%   1.8085%
       7 23 -0.2085%  1.1993%   1.6819%
       8 23  0.0893%  1.2886%   1.8609%
       9 23  0.3307%  1.6193%   2.1658%
      10 23  0.5320%  2.1513%   2.4885%
```

88.6.13 Summary of Key Findings

```
print("=" * 70)
print("EVENT STUDY RESULTS SUMMARY")
print("=" * 70)

ff3_all = results_ff3['test_stats'][results_ff3['test_stats']['group'] ==
 ↪  'All'].iloc[0]

print(f"\nSample: {int(ff3_all['N'])} firm-event observations")
print(f"Frequency: Daily")
print(f"Primary Model: Fama-French 3-Factor")
print(f"Estimation Window: {config_ff3.estimation_window} trading days")
print(f"Event Window: ({config_ff3.event_window_start},
 ↪  {config_ff3.event_window_end})")
print(f"Gap: {config_ff3.gap} trading days")
print(f"\n--- Abnormal Return Measures ---")
print(f"Mean
 ↪  CAR({config_ff3.event_window_start},{config_ff3.event_window_end}): "
      f"{ff3_all['mean_CAR']:.4%}")
print(f"Median CAR: {ff3_all['median_CAR']:.4%}")
print(f"Mean BHAR: {ff3_all['mean_BHAR']:.4%}")
print(f"Fraction positive CARs: {ff3_all['pct_positive']:.1%}")
print(f"\n--- Statistical Significance ---")
print(f"Cross-Sectional t: {ff3_all['t_CS']:.3f} (p =
 ↪  {ff3_all['p_CS']:.4f})")
print(f"Patell Z: {ff3_all['Z_Patell']:.3f} (p = {ff3_all['p_Patell']:.4f})")
print(f"BMP t: {ff3_all['t_BMP']:.3f} (p = {ff3_all['p_BMP']:.4f})")
print(f"Kolari-Pynnönen t: {ff3_all['t_KP']:.3f} (p =
 ↪  {ff3_all['p_KP']:.4f})")
print(f"Generalized Sign Z: {ff3_all['Z_GSign']:.3f} (p =
 ↪  {ff3_all['p_GSign']:.4f})")

sig_005 = sum(1 for k in
 ↪  ['p_CS','p_Patell','p_BMP','p_KP','p_GSign','p_SkAdj','p_Wilcoxon']
              if k in ff3_all and pd.notna(ff3_all[k]) and ff3_all[k] < 0.05)
total_tests = sum(1 for k in
 ↪  ['p_CS','p_Patell','p_BMP','p_KP','p_GSign','p_SkAdj','p_Wilcoxon']
                  if k in ff3_all and pd.notna(ff3_all[k]))
print(f"\n{sig_005}/{total_tests} tests significant at 5% level")

# Robustness note
```

```
print(f"\nRobustness: Results checked across {len(models_daily)} risk models
 ↪  "
      f"and {len(windows)} event windows")
```

```
======================================================================
EVENT STUDY RESULTS SUMMARY
======================================================================

Sample: 26 firm-event observations
Frequency: Daily
Primary Model: Fama-French 3-Factor
Estimation Window: 150 trading days
Event Window: (-10, 10)
Gap: 15 trading days

--- Abnormal Return Measures ---
Mean CAR(-10,10): 2.1513%
Median CAR: 1.5701%
Mean BHAR: 2.4885%
Fraction positive CARs: 61.5%

--- Statistical Significance ---
Cross-Sectional t: 0.775 (p = 0.4456)
Patell Z: 0.961 (p = 0.3364)
BMP t: 0.727 (p = 0.4741)
Kolari-Pynnönen t: 0.700 (p = 0.4904)
Generalized Sign Z: 1.177 (p = 0.2393)

0/7 tests significant at 5% level

Robustness: Results checked across 4 risk models and 7 event windows
```

88.7 Practical Recommendations

Based on the literature and our implementation experience:

1. **Estimation window**: Use 150 trading days (~7 months) for daily studies. This balances parameter precision against structural breaks. For monthly studies, 60 months is standard (Sagar P. Kothari and Warner 2007).
2. **Gap**: 15 trading days is standard. Increase to 30 if information leakage is a concern.

3. **Event window**: Start with (-1, +1) for short-window tests, then expand to (-5, +5) and (-10, +10) for robustness. Report all windows.
4. **Model choice**: Always report market model as the baseline. Add FF3 or FF5 for robustness. For Vietnam, local factors are preferable to global factors.
5. **Test statistics**: Report at minimum: cross-sectional t (for ease of interpretation), BMP (robust to event-induced variance), and one non-parametric test (sign or Wilcoxon). Report Kolari-Pynnönen if events cluster in calendar time.
6. **Thin trading**: For Vietnamese small-caps, consider Dimson (1979) with 1 lead/lag or increase `min_estimation_obs` to filter out illiquid stocks.
7. **Multiple testing**: If testing multiple event windows or subgroups, apply Bonferroni or Holm corrections to control family-wise error rate.

89 Industrial Park Anchors and Developer Value

Does it pay to fill a property with tenants from one industry? Urban economics says clustering makes firms more productive: they share suppliers, draw on a common pool of skilled workers, and learn from each other (Rosenthal and Strange 2004). If tenants value that, they should pay more rent to be near firms like themselves, and the owner of the property should capture part of the gain. Portfolio logic says the opposite. A landlord whose tenants all work in one industry has concentrated its exposure, and one bad year for that industry hits the whole rent roll at once.

C. H. Liu, Zheng, and Zhu (2026) take this trade-off to US office buildings, and their answer is that clustering raises value only in one configuration: when a building's tenants and its surrounding neighborhood both specialize in the *same* industry. They also show that the stock market agrees. REITs earn positive abnormal returns when they buy specialized buildings and negative ones when they sell them.

This chapter does two things with the paper:

1. Section 89.1 lays out its design and results, and what changed between the working paper and the published version.
2. Section 89.2 onward turns it into a worked example for Vietnam. The setting changes from office buildings to industrial parks, and the market test changes from REIT trades to lease announcements by listed park developers. Section 89.3 presents a hand-collected set of announcements, Section 89.4 asks how large an effect they can detect, and Section 89.5 runs the event study with the pipeline from the event-study chapter.

Sources used in this chapter

The paper is C. H. Liu, Zheng, and Zhu (2026), *Management Science* 72(10), 8366–8393. The published PDF sits behind a bot check, so the design details and coefficient magnitudes below come from the February 2022 working paper, "The Agglomerative Effects of Neighborhood and Building Specialization on Office Values," presented at the European Real Estate Society conference. The question, the trade-off, and the headline result are quoted from the published abstract. Some magnitudes may have changed between versions.

89.1 The Paper

89.1.1 Question and Trade-Off

The published abstract opens with the trade-off: agglomeration within a building or neighborhood brings higher tenant productivity, at the cost of a more concentrated tenant base. Which effect dominates is an empirical question. Earlier work on office rents had found a positive link between clustering and rents, but with averaged rents or averaged measures of clustering, so it could not separate building quality from location (C. H. Liu, Zheng, and Zhu 2026). Two features set this paper apart. It measures clustering at two levels, around the building (the "horizontal" dimension) and inside it (the "vertical" dimension, following the evidence that agglomeration operates within buildings and even across floors (C. H. Liu, Rosenthal, and Strange 2018)). And it links the two levels by asking whether the building's main tenant works in the industry its neighborhood specializes in.

89.1.2 Data and Measures

The working-paper sample is 457 office buildings bought or sold by 57 US REITs between 2014 and 2020, with 5,396 tenants hand-collected from CoStar. Neighborhoods are 5-digit ZIP codes, and their industry mix comes from the Census ZIP Code Business Patterns at the 2-digit NAICS level. The paper builds three measures.

Neighborhood concentration is a Herfindahl index of employment shares across industries k in the ZIP code of building i:

$$\mathrm{HHI}^{\mathrm{nbhd}}_{i,t} = \sum_{k} \left(\frac{E_{z(i),k,t}}{E_{z(i),t}} \right)^2 . \tag{89.1}$$

Tenant concentration is the same index computed over the industries of the building's tenants, weighted by the floor space each one leases:

$$\mathrm{HHI}^{\mathrm{tenant}}_{i,t} = \sum_{l} \left(\frac{A_{i,l,t}}{A_{i,t}} \right)^2 . \tag{89.2}$$

Building-neighborhood specialization is the key measure. Find the industry $a(i)$ of the building's largest tenant, and divide the ZIP code's employment in that industry by national employment in it:

$$\mathrm{SPEC}_{i,t} = \frac{E_{z(i),a(i),t}}{E_{a(i),t}}. \tag{89.3}$$

A building whose largest tenant is an IT firm and which stands in a ZIP code full of IT workers has a high SPEC. Because a high SPEC could simply reflect a neighborhood dominated by one industry, the regressions also control for the share of local employment in the anchor's industry, $E_{z(i),a(i),t}/E_{z(i),t}$.

89.1.3 Results

The outcomes are log price per square foot, log net operating income (NOI), and the cap rate, regressed on the measures above plus building controls (size, age, height, quality, green label), location controls, and year, REIT, and metropolitan-area fixed effects. In the working paper:

- **Both kinds of concentration raise value.** A one-standard-deviation increase in neighborhood concentration goes with a 6.9% higher price and 4.8% higher NOI. The same increase in tenant concentration goes with an 8.6% higher price and 9.3% higher NOI. Cap-rate effects are weak.
- **Specialization beats concentration.** When HHI^{nbhd} and SPEC enter the same regression, neighborhood concentration loses significance and SPEC survives, at about 8.2% on price and 6.9% on NOI per standard deviation. This horse race is the heart of the paper. Plain density of economic activity is not what tenants pay for. They pay to be near firms in their own industry, which is what knowledge spillovers predict.
- **Anchors matter.** The premium rises with the share of space taken by the largest industry or the largest tenant, and is most precisely estimated for buildings occupied by a single industry or a single tenant.

89.1.4 The Stock-Market Test

Because the buyers and sellers are listed REITs, each transaction is also an event for a stock. The paper estimates abnormal returns with the Fama-French factors plus a REIT index and regresses the REIT's cumulative abnormal return on the building's measures. A one-standard-deviation higher tenant concentration in an acquired building goes with a 0.49% higher CAR over days $(0, +1)$, and the same difference in a sold building with a 0.47% lower CAR. SPEC adds about 0.15% for acquisitions. The acquisition effects last one trading day, while the penalty for selling a concentrated building persists for about a week.

89.1.5 From Working Paper to Journal Article

The changes between versions are a lesson in how such a paper is framed. The working-paper title described the method ("The Agglomerative Effects of Neighborhood and Building Specialization on Office Values"). The published title asks a yes/no question with a real tension behind it. In the working paper, the diversification argument appears only in the conclusion.

In the published version it opens the abstract. And the result that was one subsection of the working paper, that the premium needs specialization at both levels at once, became the headline. The empirical design hardly changed; the argument around it did.

89.2 From Offices to Industrial Parks

The design needs three things: units that hold several tenants, a way to measure the industries of those tenants and of the surrounding area, and listed owners whose stock reacts when the tenant mix changes. In Vietnam the closest match is not the office market but industrial parks (*khu công nghiệp*).

- **Clustering is deliberate and large.** Provinces and developers recruit anchor investors, and suppliers follow them. The electronics clusters that grew around Samsung's plants in Bắc Ninh and Thái Nguyên are the best-known example.
- **Leases are long.** Industrial land is leased for the park's remaining term, often 40 to 50 years, and the rent is largely paid up front. A lease price is therefore close to a capitalized value, much like the transaction prices in the original paper.
- **The owners are listed.** About a dozen developers trade on HOSE, HNX, and UPCoM, and they announce new tenants through exchange filings, press releases, and signing ceremonies.

Table 89.1 matches each part of the original design to its Vietnamese counterpart.

Table 89.1: Mapping the office design onto Vietnamese industrial parks

Element	C. H. Liu, Zheng, and Zhu (2026) (US offices)	Vietnam (industrial parks)
Unit	Office building traded by a REIT	Industrial park; each lease announcement by its listed developer
Tenants	CoStar tenant rosters	Investor projects in each park, from developers and provincial IP management boards
Concentration inside	HHI of tenant industries by floor space	HHI of investor industries by leased hectares
Surrounding area	5-digit ZIP code, Census ZIP Business Patterns	Province on pre-2025 boundaries or a radius around the park, GSO Enterprise Survey
Match measure	SPEC, Equation 89.3	The same, using the anchor investor's VSIC industry

Element	C. H. Liu, Zheng, and Zhu (2026) (US offices)	Vietnam (industrial parks)
Value outcome	Price, NOI, cap rate	Lease price per m^2 over the remaining term, occupancy, hectares absorbed
Market test	REIT CARs around acquisitions and sales	Developer CARs around lease announcements and sector shocks

Two parts transplant directly. The horse race between concentration and matched specialization works the same way, and so does the use of a listed owner's stock price to see whether investors share the valuation. One part does not. Vietnam has no transaction database comparable to CoStar, so the building-level price regressions of Section 89.1.3 need a park-level panel that has to be built from broker reports and provincial records (Section 89.6). The worked example in this chapter therefore starts with the part that can be built from public sources today: the announcement event study.

Translated to this setting, the event study tests one prediction. **If tenants pay for proximity to firms in their own industry, a developer's stock should react more strongly when it signs a tenant from the industry its park is already built around.** A signing that deepens the park's cluster raises the expected rent on the rest of its land, while an unrelated tenant only brings the lease itself. Concentration risk pushes the other way, and Vietnam provides a clean test of that side as well: the US tariff announcements of 2025, which hit export-oriented electronics producers hardest.

89.3 Hand-Collected Lease Announcements

No database records industrial-park leases, so the events have to be collected by hand. The table below was compiled in October 2026 from developers' own news archives, provincial government portals, and the Vietnamese business press, using these rules.

- **Developers.** All listed park developers were searched: KBC, IDC, BCM, SZC, NTC, D2D, TIP, LHG, VGC, SIP, GVR, PHR, and SNZ. Verifiable events turned up for six of them.
- **Events.** A developer or its subsidiary announces a land lease contract, a memorandum of understanding or agreement in principle, or a land handover with a *named* tenant in one of its parks, or the province awards that tenant an investment certificate for a project in the park. The window is 2012–2025.
- **Verification.** Each row is confirmed on a dated page that names the tenant, and the page is linked in `source_url`. The date is the earliest public date found: the signing

or ceremony date when the source gives one, otherwise the publication date (`date_type` records which).

- **Coding.** `hectares` is filled in only when the source states the area. `vsic2` is the VSIC 2018 division of the tenant's main activity, and `anchor_vsic2` is the division of the park's dominant tenants *before* the deal, taken from the developer's and the press's descriptions of the park. It is left blank when no source names an anchor.
- **Exclusions.** Unnamed tenants, deals without an exact date, leases between developers (such as a sister company's sub-lease of a whole block), and ready-built factory leases outside the developers' parks.
- **Follow-ups.** `followup` flags later stages of a deal that was already public: Hanwha's contract after its MOU, Amkor's lease after its agreement in principle, and Foxconn's investment certificate and lease filing at Quang Châu after the 2022 MOU. They are not news and are dropped from the event study.

```
import io
import numpy as np
import pandas as pd

ip_events =
 ↪  pd.read_csv(io.StringIO("""ticker,announce_date,date_type,deal_type,tena
 ↪  nt,industry,vsic2,hectares,park,province,anchor_vsic2,followup,source_url
BCM,2014-06-24,signing_ceremony,lease_contract,Kingtec Group,LED lights and
 ↪  table/wall lamps,27,16,Mỹ Phước 3,Bình Dương,22,0,https://en.vietstock.v
 ↪  n/2014/06/kingtec-invests-us$30-million-in-binh-duong-38-179364.htm
BCM,2016-04-27,signing_ceremony,mou,DDK Vietnam,bicycles and bicycle parts
 ↪  (specialised sub-zone),30,80,Bàu Bàng mở rộng,Bình Dương,,0,https://tuoi
 ↪  tre.vn/saigontimes/binh-duong-mo-rong-kcn-bau-bang-them-1-000-ha/
IDC,2022-10-06,signing_ceremony,lease_contract,Phu My Super White Float Glass
 ↪  Co. Ltd (PFG; Viglacera subsidiary),super-white float glass,23,20,Phú Mỹ
 ↪  II mở rộng,Bà Rịa-Vũng Tàu,24,0,https://idico.com.vn/en/tin-tuc/idico-ky
 ↪  -hop-dong-voi-cong-ty-kinh-phu-my
IDC,2023-02-22,signing_ceremony,lease_contract,Suntory PepsiCo Vietnam
 ↪  Beverage Co. Ltd,beverages and dairy,11,20,Hựu Thạnh,Long
 ↪  An,52,0,https://idico.com.vn/vi/tin-tuc/suntory-pepsico-huuthanh
IDC,2023-10-03,news,investment_certificate,Hyosung Vietnam Co. Ltd,carbon
 ↪  fibre,20,19.2,Phú Mỹ II,Bà Rịa-Vũng
 ↪  Tàu,24,0,https://theinvestor.vn/ba-ria-vung-tau-province-okays-540-mln-h
 ↪  yosung-carbon-fiber-factory-d6855.html
IDC,2023-12-08,press_release,investment_certificate,J&W Vietnam (mechanical
 ↪  tools factory),air compressors and power tools,28,3.71,Cầu Nghìn,Thái
 ↪  Bình,,0,https://idico.com.vn/vi/tin-tuc/idico-ban-giao-giay-chung-nhan-d
 ↪  au-tu-cho-cac-du-an-dau-tu-fdi-tai-thai-binh
```

```
IDC,2023-12-08,press_release,investment_certificate,WEISHENG Vietnam
 ↪  (industrial tools factory),industrial tools,25,2.65,Cầu Nghìn,Thái
 ↪  Bình,,0,https://idico.com.vn/vi/tin-tuc/idico-ban-giao-giay-chung-nhan-d
 ↪  au-tu-cho-cac-du-an-dau-tu-fdi-tai-thai-binh
IDC,2023-12-08,press_release,investment_certificate,XINGYU Vietnam (pump
 ↪  factory),pumps,28,1.80,Cầu Nghìn,Thái
 ↪  Bình,,0,https://idico.com.vn/vi/tin-tuc/idico-ban-giao-giay-chung-nhan-d
 ↪  au-tu-cho-cac-du-an-dau-tu-fdi-tai-thai-binh
IDC,2023-12-21,signing_ceremony,lease_contract,Cynthia International Co. Ltd
 ↪  (Liancheng Vietnam Electronics Technology),metal/plastic electronic
 ↪  components (connectors),26,,Hựu Thạnh,Long An,52,0,https://idico.com.vn/
 ↪  vi/tin-tuc/idico-ky-hop-dong-voi-cac-nha-dau-tu-trung-quoc
IDC,2023-12-21,signing_ceremony,lease_contract,Miracle Pharm Pte. Ltd. (Viet
 ↪  Thanh Biotechnology Co. Ltd),biotech and veterinary medicine,21,,Hựu
 ↪  Thạnh,Long An,52,0,https://idico.com.vn/vi/tin-tuc/idico-ky-hop-dong-voi
 ↪  -cac-nha-dau-tu-trung-quoc
IDC,2025-04-08,signing_ceremony,mou,Topsun Logistics Equipment Industrial Co.
 ↪  Ltd,cargo lashing straps / winches / racks / chains,25,3,Phú Mỹ II,Bà
 ↪  Rịa-Vũng Tàu,24,0,https://idico.com.vn/vi/tin-tuc/su-kien-dinh-cua-nha-d
 ↪  au-tu-trong-boi-canh-thi-truong-nhieu-bien-dong
IDC,2025-05-13,signing_ceremony,lease_contract,Advanced Materials Zili
 ↪  Vietnam Co. Ltd (Zili Group),corundum refractory materials,23,,Phú Mỹ
 ↪  II,Bà Rịa-Vũng
 ↪  Tàu,24,0,https://idico.com.vn/vi/tin-tuc/ky-ket-cong-ty-advanced-materia
 ↪  ls-zili-tu-singapore-chinh-thuc-dau-tu-vao-he-sinh-thai-cong-nghiep-idico
IDC,2025-06-27,signing_ceremony,lease_contract,Vifon Long An Co. Ltd,instant
 ↪  / rice-based processed food,10,,Hựu Thạnh,Long
 ↪  An,52,0,https://www.idico.com.vn/vi/tin-tuc/vifon-dau-tu-nha-may-che-bie
 ↪  n-thuc-pham-tai-khu-cong-nghiep-idico-huu-thanh
KBC,2013-09-06,signing_ceremony,investment_certificate,LG
 ↪  Electronics,consumer electronics,26,,Tràng Duệ,Hải
 ↪  Phòng,,0,https://kinhbaccity.vn/en/402-lg-electronics-inc-chinh-thuc-duo
 ↪  c-thanh-pho-hai-phong-cap-giay-chung-nhan-dau-tu-vao-khu-trang-due-thuoc
 ↪  -khu-kinh-te-dinh-vu-cat-hai.htm
KBC,2014-10-28,signing_ceremony,lease_contract,C.P. Vietnam Livestock
 ↪  JSC,food/feed processing,10,,Tân Phú Trung,Hồ Chí Minh
 ↪  City,,0,https://kinhbaccity.vn/en/483-xay-dung-nha-may-300-trieu-usd-tai
 ↪  -kcn-tan-phu-trung.htm
KBC,2014-12-18,signing_ceremony,mou,LG Electronics,consumer
 ↪  electronics,26,40,Tràng Duệ,Hải Phòng,26,0,https://kinhbaccity.vn/en/494
 ↪  -lg-electronics-leases-more-land-in-trang-due-ip.htm
KBC,2015-03-06,signing_ceremony,mou,Haengsung Electronics,PCB assembly
 ↪  (electronic components),26,10,Tràng Duệ,Hải Phòng,26,0,https://kinhbacci
 ↪  ty.vn/en/512-kbc-tinh-hinh-thu-fdi-hai-thang-dau-nam-2015.htm
```

```
KBC,2015-09-03,signing_ceremony,mou,Hee-Sung Electronics,LED backlights/touch
 ↪ panels,26,13.86,Tràng Duệ,Hải
 ↪ Phòng,26,0,https://cafef.vn/tin-tuc-du-an/kbc-ky-hop-dong-cho-thue-dat-k
 ↪ cn-trang-due-tri-gia-khoang-250-ty-20150903105608869.chn
KBC,2016-04-05,news,mou,LG Display,display panels,26,40.4,Tràng Duệ,Hải
 ↪ Phòng,26,0,https://cafef.vn/tin-tuc-du-an/lg-display-se-xay-dung-du-an-1
 ↪ 5-ty-usd-tai-kcn-trang-due-20160405215211015.chn
KBC,2016-07-11,signing_ceremony,mou,JA Solar (JA Solar Hongkong
 ↪ Investment),solar cells,26,88,Quang Châu,Bắc
 ↪ Giang,26,0,https://kinhbaccity.vn/en/630-ja-solar-thue-88-ha-dat-tai-kcn
 ↪ -quang-chau-de-dau-tu-du-an-san-xuat-pin-nang-luong-mat-troi-cong-nghe-c
 ↪ ao-voi-tong-von-hon-1-ty-usd-.htm
KBC,2016-09-14,signing_ceremony,mou,Luxshare-ICT,electronics
 ↪ components,26,14,Quang Châu,Bắc
 ↪ Giang,26,0,https://kinhbaccity.vn/en/644-quang-chau-industrial-park-rece
 ↪ ived-550-million-invested-capitals-from-hong-kong-investors.htm
KBC,2017-01-10,signing_ceremony,mou,Hanwha Techwin,CCTV/security
 ↪ cameras,26,6,Quế Võ mở rộng,Bắc
 ↪ Ninh,26,0,https://kinhbaccity.vn/en/669-kbc-welcomes-the-first-high-tech
 ↪ -investor-to-que-vo-industrial-park-bac-ninh-province-in-2017.htm
KBC,2017-03-02,press_release,lease_contract,Hanwha Techwin,CCTV/security
 ↪ cameras,26,6,Quế Võ mở rộng,Bắc Ninh,26,1,https://kinhbaccity.vn/en/683-
 ↪ hanwha-techwin-signed-the-official-contract-with-kbc-increasing-the-inve
 ↪ stment-capital-for-first-phase-up-to-usd-100-million.htm
KBC,2017-11-01,signing_ceremony,lease_contract,LG Innotek,camera
 ↪ modules,26,7.2,Tràng Duệ,Hải
 ↪ Phòng,26,0,https://kinhbaccity.vn/en/734-lg-innotek-mo-rong-san-xuat-gia
 ↪ i-doan-2-tiep-tuc-thue-them-dat-tai-kcn-trang-due.htm
KBC,2018-03-29,signing_ceremony,mou,Samkwang Co. Ltd,mobile phone
 ↪ components,26,10,Quang Châu,Bắc
 ↪ Giang,26,0,https://kinhbaccity.vn/en/755-trien-vong-lon-cho-kbc-tu-viec-
 ↪ thu-hut-du-an-dau-tu-150-trieu-usd-cua-samkwang.htm
KBC,2019-02-18,press_release,investment_certificate,Goertek (Hong
 ↪ Kong),acoustic/electronic devices,26,,Quế Võ,Bắc
 ↪ Ninh,26,0,https://kinhbaccity.vn/en/799-tin-hieu-kha-quan-trong-thu-hut-
 ↪ dau-tu-cua-kbc-tu-nhung-ngay-dau-nam-moi-2019.htm
KBC,2019-05-27,signing_ceremony,investment_certificate,Billion Max Vietnam
 ↪ (Winson Group),plastic toys/baby products,32,,Sài Gòn - Chân Mây,Thừa
 ↪ Thiên Huế,,0,https://kinhbaccity.vn/831-du-an-nha-may-che-xuat-billion-m
 ↪ ax-viet-nam-trong-kcn-va-kptq-sai-gon-chan-may-duoc-trao-giay-chung-nhan
 ↪ -dang-ky-dau-tu.htm
KBC,2021-01-18,news,investment_certificate,Foxconn Singapore (Fukang
 ↪ Technology),laptops/tablets assembly,26,,Quang Châu,Bắc
 ↪ Giang,26,0,https://english.vov.vn/en/economy/foxconn-pours-us270-million
 ↪ -into-bac-giang-project-831451.vov
```

```
KBC,2022-03-03,press_release,investment_certificate,Goertek
 ↪ Vina,acoustic/electronic devices,26,,Quế Võ,Bắc
 ↪ Ninh,26,0,https://kinhbaccity.vn/en/1157-goertek-tiep-tuc-mo-rong-quy-mo
 ↪ -dien-tich-tang-von-dau-tu-du-an-tai-kcn-que-vo-len-gan-566-trieu-usd.htm
KBC,2022-08-15,signing_ceremony,mou,Foxconn (Hon Hai),electronics
 ↪ assembly,26,50.5,Quang Châu,Bắc Giang,26,0,https://kinhbaccity.vn/en/128
 ↪ 0-foxconn-signs-the-memorandum-of-understanding-mou-to-sublease-50-5-ha-
 ↪ in-quang-chau-industrial-park-bac-giang-province.htm
KBC,2023-01-02,signing_ceremony,investment_certificate,Ingrasys
 ↪ (Singapore)/Fulian Co. (Foxconn),precision electronics,26,49.6,Quang Châu
 ↪ mở rộng,Bắc Giang,26,1,https://kinhbaccity.vn/en/1396-du-an-nha-may-cong
 ↪ -nghe-chinh-xac-fulian-dau-tu-tai-kcn-quang-chau-bac-giang-duoc-cap-giay
 ↪ -chung-nhan-dang-ky-dau-tu-vao-ngay-dau-nam-2023-eng.htm
KBC,2023-02-14,exchange_filing,lease_contract,Fulian Precision Technology
 ↪ Component (Foxconn),precision electronics,26,45,Quang Châu,Bắc
 ↪ Giang,26,1,https://www.rfa.org/vietnamese/news/vietnamnews/foxconn-lease
 ↪ s-new-site-in-vn-as-apple-contractor-continues-to-diversify-production-a
 ↪ way-from-china-02162023071333.html
KBC,2023-02-15,signing_ceremony,mou,Goertek,acoustic/electronic
 ↪ devices,26,62.7,Nam Sơn - Hạp Lĩnh,Bắc Ninh,,0,https://doanhnhan.baophap
 ↪ luat.vn/tap-doan-cong-nghe-noi-tieng-the-gioi-ky-bien-ban-ghi-nho-thue-6
 ↪ 27-ha-tai-khu-cong-nghiep-nam-son--hap-linh-cua-kbc-50106.html
KBC,2024-06-17,news,investment_certificate,Foxconn Singapore (FCPV Foxconn
 ↪ Bac Ninh),printed circuit boards,26,14.26,Nam Sơn - Hạp Lĩnh,Bắc
 ↪ Ninh,26,0,https://en.vietstock.vn/2024/06/foxconn-to-pour-383-million-in
 ↪ to-bac-ninh-factory-974-568100.htm
KBC,2024-07-16,news,investment_certificate,Shunsin Technology (Foxconn),IC
 ↪ packaging,26,4.43,Quang Châu,Bắc
 ↪ Giang,26,0,https://en.baobacninhtv.vn/quang-chau-industrial-park-lures-a
 ↪ dditional-investment-of-80-million-usd-110543.bbg
SIP,2024-07-15,signing_ceremony,investment_certificate,King King Group (Best
 ↪ Base),handicraft products,,10,Phước Đông,Tây
 ↪ Ninh,13,0,https://saigonvrg.com.vn/vi/tay-ninh-trao-giay-chung-nhan-dau-
 ↪ tu-cho-du-an-fdi-tai-kcn-phuoc-dong
SIP,2024-07-22,signing_ceremony,investment_certificate,Jinhua Vietnam (Anhui
 ↪ Jinhua Zinc Oxide),zinc oxide,20,2.30,Phước Đông,Tây
 ↪ Ninh,13,0,https://saigonvrg.com.vn/vi/tay-ninh-cap-phep-dau-tu-du-an-fdi
 ↪ -22-trieu-usd-vao-kcn-phuoc-dong
SIP,2024-10-30,signing_ceremony,lease_contract,Thantawan Industry
 ↪ (Vietnam),plastic packaging,22,,Phước Đông,Tây Ninh,13,0,https://saigonv
 ↪ rg.com.vn/vi/sai-gon-vrg-ky-ket-hop-tac-cung-cong-ty-thantawan-industry
SIP,2025-03-07,signing_ceremony,investment_certificate,Global Hantex,knitted
 ↪ fabric/garment accessories,13,50,Phước Đông,Tây
 ↪ Ninh,13,0,https://baodautu.vn/tay-ninh-cap-phep-dau-tu-du-an-fdi-quy-mo-
 ↪ 50-ha-tai-kcn-phuoc-dong-d251394.html
```

```
SIP,2025-11-07,signing_ceremony,mou,Transimex Port (Transimex
 ↪  group),port/logistics,52,,Lộc An - Bình Sơn,Đồng
 ↪  Nai,,0,https://saigonvrg.com.vn/vi/le-ky-ket-bien-ban-ghi-nho-thue-lai-q
 ↪  uyen-su-dung-dat-tai-khu-cong-nghiep-loc-an-binh-son-giua-cong-ty-co-pha
 ↪  n-cang-transimex-va-cong-ty-co-phan-dau-tu-va-phat-trien-vrg-long-thanh
SZC,2018-05-22,press_release,lease_contract,SH Solar Farm Vina Co. Ltd,solar
 ↪  power generation,35,60,Châu Đức,Bà Rịa-Vũng
 ↪  Tàu,15,0,https://sonadezichauduc.com.vn/vn/kcn-do-thi-va-san-golf-chau-d
 ↪  uc-don-du-an-dien-nang-luong-mat-troi.html
SZC,2024-05-13,signing_ceremony,lease_contract,Electronic Tripod Vietnam
 ↪  (Chau Duc) Co. Ltd (Tripod Technology),printed circuit boards,26,18,Châu
 ↪  Đức,Bà Rịa-Vũng Tàu,15,0,https://sonadezichauduc.com.vn/vn/tripod-dau-tu
 ↪  -nha-may-250-trieu-usd-tai-kcn-chau-duc.html
SZC,2024-11-12,news,investment_certificate,Ton Vina One (coated steel sheet
 ↪  plant),coated steel sheet,24,12,Châu Đức,Bà Rịa-Vũng
 ↪  Tàu,15,0,https://nguoiquansat.vn/cac-mo-vang-tai-vung-tau-don-hon-2-900-
 ↪  ty-dong-chi-trong-mot-thang-176822.html
VGC,2012-12-19,signing_ceremony,lease_contract,Curious Seiki Vietnam,auto
 ↪  parts,29,1.1961,Tiên Sơn,Bắc
 ↪  Ninh,26,0,https://viglacera.com.vn/blog/curious-seiki-nhat-ban-trien-kha
 ↪  i-dau-tu-tai-khu-cong-nghiep-tien-son-viglacera
VGC,2013-07-09,signing_ceremony,mou,Yazaki Hai Phong Vietnam,auto wiring
 ↪  harnesses,29,7,Đông Mai,Quảng Ninh,,0,https://viglacera.com.vn/blog/tap-
 ↪  doan-yazaki-nhat-ban-dau-tu-tai-kcn-dong-mai-viglacera
VGC,2014-07-02,signing_ceremony,investment_certificate,Samsung Display
 ↪  Vietnam,display panels,26,,Yên Phong I,Bắc Ninh,26,0,https://viglacera.c
 ↪  om.vn/en/blog/bac-ninh-awards-a-one-billion-usd-investment-certificate-t
 ↪  o-sam-sung-display-project-implemented-in-yen-phong-industrial-zone
VGC,2015-07-16,signing_ceremony,lease_contract,Ottogi Vietnam,food and
 ↪  beverages,10,6.6,Yên Phong I,Bắc
 ↪  Ninh,26,0,https://viglacera.com.vn/en/blog/one-more-high-reputation-kore
 ↪  an-enterprises-invests-in-viglacera-yen-phong-idustrial-zone
VGC,2015-08-06,signing_ceremony,investment_certificate,Samsung Display
 ↪  Vietnam (USD 3bn expansion),display panels,26,,Yên Phong I,Bắc
 ↪  Ninh,26,0,https://viglacera.com.vn/blog/samsung-display-tiep-tuc-dau-tu-
 ↪  them-3-ty-usd-vao-kcn-yen-phong-viglacera
VGC,2016-07-06,signing_ceremony,lease_contract,Sakai Amiori,textiles
 ↪  (knitting),13,,Phú Hà,Phú Thọ,14,0,https://viglacera.com.vn/en/blog/phu-
 ↪  ha-industrial-park-a-new-attractive-destination-for-japanese-enterprises
VGC,2016-07-12,signing_ceremony,investment_certificate,Vietnam Vega Balls
 ↪  (Yuan Chi Group),sports balls,32,3.24,Đông Mai,Quảng
 ↪  Ninh,29,0,https://viglacera.com.vn/en/blog/issuance-of-investment-certif
 ↪  icate-for-taiwanese-enterprise-in-dong-mai-industrial-park
```

```
VGC,2016-11-11,signing_ceremony,lease_contract,NT Vina,phone components
 ↪  (vibration motors/camera),26,3,Phú Hà,Phú
 ↪  Thọ,14,0,https://viglacera.com.vn/en/blog/phu-ha-industrial-zone-success
 ↪  fully-attracts-korean-electronics-enterprise
VGC,2017-02-20,press_release,lease_contract,Davo C&M,electronic components
 ↪  (Samsung vendor),26,2,Phú Hà,Phú
 ↪  Thọ,26,0,https://viglacera.com.vn/blog/khu-cong-nghiep-phu-ha-viglacera-
 ↪  thu-hut-nhieu-doanh-nghiep-ve-tinh-cho-samsung
VGC,2017-02-24,signing_ceremony,investment_certificate,Samsung Display
 ↪  Vietnam (USD 2.5bn expansion),display panels,26,,Yên Phong I,Bắc
 ↪  Ninh,26,0,https://viglacera.com.vn/en/blog/more-than-25-billion-usd-was-
 ↪  invested-in-yen-phong-viglacera-industrial-park-from-samsung
VGC,2017-04-14,signing_ceremony,lease_contract,Haehwa,phone camera
 ↪  modules,26,1.7,Phú Hà,Phú Thọ,26,0,https://viglacera.com.vn/blog/cac-con
 ↪  g-ty-ve-tinh-cua-samsung-lien-tiep-dau-tu-vao-khu-cong-nghiep-phu-ha
VGC,2017-04-14,signing_ceremony,lease_contract,Woojin Global Logistics
 ↪  (Logisvalley Vina),logistics/warehousing,52,6,Yên Phong mở rộng,Bắc
 ↪  Ninh,26,0,https://viglacera.com.vn/blog/cac-doanh-nghiep-lon-han-quoc-ti
 ↪  ep-tuc-dau-tu-vao-khu-cong-nghiep-yen-phong-mo-rong
VGC,2017-08-30,press_release,lease_contract,Toyoda Gosei (Haiphong Thai Binh
 ↪  Plant),airbag parts/steering wheels,29,11.3,Tiền Hải,Thái
 ↪  Bình,,0,https://www.toyoda-gosei.com/news/details.php?id=159
VGC,2017-11-14,press_release,lease_contract,Anam Electronics Vietnam,audio
 ↪  electronics,26,10,Đồng Văn IV,Hà
 ↪  Nam,26,0,https://viglacera.com.vn/en/blog/dong-van-iv-industrial-park-vi
 ↪  glacera-has-successfully-attracted-large-korean-electronics-manufacturers
VGC,2018-07-19,press_release,lease_contract,Moda Innochips,circuit
 ↪  boards/phone parts,26,5.1,Phú Hà,Phú
 ↪  Thọ,26,0,https://viglacera.com.vn/en/blog/viglacera-phu-ha-industrial-pa
 ↪  rk-attracts-many-high-technology-enterprises
VGC,2018-09-05,signing_ceremony,lease_contract,Asentec,auto parts
 ↪  (wheel-speed sensors/cables),29,3.2,Phú Hà,Phú
 ↪  Thọ,26,0,https://viglacera.com.vn/blog/khu-cong-nghiep-phu-ha-viglacera-
 ↪  thu-hut-them-doanh-nghiep-san-xuat-phu-tung-o-to-tu-han-quoc
VGC,2018-10-23,signing_ceremony,lease_contract,ActRo Vina,phone camera
 ↪  components,26,2,Phú Hà,Phú Thọ,26,0,https://viglacera.com.vn/en/blog/kor
 ↪  ean-enterprises-continue-to-invest-in-phu-ha-industrial-park-viglacera
VGC,2019-08-21,press_release,lease_contract,Qisda
 ↪  Corporation,monitors/projectors,26,,Đồng Văn IV,Hà
 ↪  Nam,26,0,https://viglacera.com.vn/en/blog/a-taiwanese-enterprise-investe
 ↪  d-nearly-400-million-usd-in-dong-van-4-industrial-park-viglacera
VGC,2019-08-27,press_release,lease_contract,Foxconn Singapore (Hon Hai),TV
 ↪  screen component assembly,26,10,Đông Mai,Quảng
 ↪  Ninh,29,0,https://viglacera.com.vn/en/blog/dong-mai-industrial-park-vigl
 ↪  acera-successfully-attracted-one-of-the-leading-electronics-and-informat
 ↪  ion-technology-corporations-in-the-world
```

```
VGC,2019-09-26,signing_ceremony,investment_certificate,Kanglongda,disposable
 ↪ gloves/PE fibre,22,35,Phong Điền,Thừa Thiên
 ↪ Huế,,0,https://viglacera.com.vn/en/blog/thua-thien-hue-province-awarded-
 ↪ investment-registration-certificate-for-a-200-million-usd-project-in-pho
 ↪ ng-dien-industrial-park-viglacera
VGC,2020-05-04,press_release,mou,Hyosung TNS,ATMs/financial equipment,28,,Yên
 ↪ Phong II-C,Bắc Ninh,,0,https://viglacera.com.vn/en/blog/viglaceras-indus
 ↪ trial-parks-grow-stably-in-the-first-quarter-of-2020
VGC,2020-05-28,signing_ceremony,lease_contract,INOUE Rubber Vietnam (IRC
 ↪ Tire),motorcycle/bicycle tyres,22,12,Phú Hà,Phú
 ↪ Thọ,26,0,https://viglacera.com.vn/en/blog/japanese-enterprises-continue-
 ↪ to-choose-phu-ha-industrial-park-viglacera
VGC,2021-11-04,signing_ceremony,mou,Amkor Technology,semiconductor
 ↪ packaging/test,26,23,Yên Phong II-C,Bắc
 ↪ Ninh,28,0,https://bacninh.gov.vn/web/english/59/-/details/4790216/bac-ni
 ↪ nh-amkor-sign-agreement-to-develop-semiconductor-factory-in-yen-phong
VGC,2022-01-20,signing_ceremony,lease_contract,Amkor Technology,semiconductor
 ↪ packaging/test,26,23,Yên Phong II-C,Bắc
 ↪ Ninh,28,1,https://viglacera.com.vn/blog/khu-cong-nghiep-yen-phong-2c-thu
 ↪ -hut-thanh-cong-tap-doan-san-xuat-vat-lieu-ban-dan-hang-dau-the-gioi
VGC,2024-05-13,signing_ceremony,lease_contract,AAC
 ↪ Technologies,acoustic/optical components,26,,Yên Phong II-C,Bắc
 ↪ Ninh,26,0,https://viglacera.com.vn/blog/viglacera-ky-ket-hop-dong-voi-do
 ↪ anh-nghiep-trung-quoc-tai-thuong-hai-trung-quoc
VGC,2024-05-13,signing_ceremony,lease_contract,Cayi Technology Vietnam,vacuum
 ↪ flasks,25,7,Yên Phong II-C,Bắc
 ↪ Ninh,26,0,https://viglacera.com.vn/blog/viglacera-ky-ket-hop-dong-voi-do
 ↪ anh-nghiep-trung-quoc-tai-thuong-hai-trung-quoc
VGC,2024-05-13,signing_ceremony,lease_contract,Micro Commercial Components
 ↪ Vietnam (Yangjie),semiconductor devices,26,8,Yên Phong II-C,Bắc
 ↪ Ninh,26,0,https://viglacera.com.vn/blog/viglacera-ky-ket-hop-dong-voi-do
 ↪ anh-nghiep-trung-quoc-tai-thuong-hai-trung-quoc
VGC,2024-09-22,signing_ceremony,mou,Johnson Health Tech,fitness
 ↪ equipment,32,,Thuận Thành (Eco-Smart),Bắc
 ↪ Ninh,,0,https://vneconomy.vn/bac-ninh-trao-quyet-dinh-chu-truong-dau-tu-
 ↪ va-chung-nhan-dau-tu-voi-so-von-gan-5-6-ty-usd.htm
"""), parse_dates=["announce_date"], dtype={"vsic2": "Int64", "anchor_vsic2":
 ↪ "Int64"})

# MATCH = 1 when the tenant's VSIC division equals that of the park's anchor
# before the deal; missing when either code is unknown
ip_events["match"] = (ip_events["vsic2"] ==
 ↪ ip_events["anchor_vsic2"]).astype("Int64")
```

```
print(f"{len(ip_events)} deals by {ip_events['ticker'].nunique()} developers,
 ↪  "
      f"{ip_events['announce_date'].min():%Y}-{ip_events['announce_date'].ma
       ↪  x():%Y}; "
      f"{int(ip_events['followup'].sum())} are later stages of an earlier
       ↪  deal")
```

```
71 deals by 6 developers, 2012-2025; 4 are later stages of an earlier deal
```

```
new = ip_events[ip_events["followup"] == 0].copy()
new["ha"] = new["hectares"].fillna(0)
new["elec_ha"] = new["ha"] * new["vsic2"].isin([26, 27])
summary = new.groupby("ticker").agg(
    deals=("tenant", "size"),
    events=("announce_date", "nunique"),
    matched=("match", lambda m: int((m == 1).sum())),
    unmatched=("match", lambda m: int((m == 0).sum())),
    unknown=("match", lambda m: int(m.isna().sum())),
    hectares=("ha", "sum"),
    elec_ha=("elec_ha", "sum"),
)
summary["electronics_share"] = (summary.pop("elec_ha") /
 ↪  summary["hectares"]).round(2)
summary.loc["Total"] = summary.sum()
summary.loc["Total", "electronics_share"] = round(
    new["elec_ha"].sum() / new["ha"].sum(), 2)
summary.astype({c: int for c in ["deals", "events", "matched", "unmatched",
                                 "unknown"]}).round({"hectares": 1})
```

Table 89.2: Lease announcements by developer, excluding follow-ups. Events count distinct announcement days. A deal is matched when the tenant's VSIC division equals that of the park's anchor before the deal. The electronics share is the share of stated hectares leased to VSIC 26–27 tenants.

ticker	deals	events	matched	unmatched	unknown	hectares	electronics_share
BCM	2	2	0	1	1	96.0	0.17
IDC	11	8	0	8	3	70.4	0.0
KBC	19	19	15	0	4	361.4	1.0
SIP	5	5	1	2	2	62.3	0.0

Table 89.2: Lease announcements by developer, excluding follow-ups. Events count distinct announcement days. A deal is matched when the tenant's VSIC division equals that of the park's anchor before the deal. The electronics share is the share of stated hectares leased to VSIC 26–27 tenants.

ticker	deals	events	matched	unmatched	unknown	hectares	electronics_share
SZC	3	3	0	3	0	90.0	0.2
VGC	27	24	11	11	5	157.3	0.41
Total	67	61	27	25	15	837.3	0.55

```
from plotnine import (ggplot, aes, geom_point, labs, theme_minimal, theme,
                      scale_color_manual, scale_size_continuous,
                      scale_x_datetime)

timeline = new.assign(
    status=new["match"].map({1: "Matched", 0: "Not
 ↪  matched"}).fillna("Unknown"),
    size_ha=new["hectares"].fillna(5))
(ggplot(timeline, aes("announce_date", "ticker", color="status",
 ↪  size="size_ha"))
 + geom_point(alpha=0.7)
 + scale_color_manual(values={"Matched": "#1b7837", "Not matched": "#b2182b",
                              "Unknown": "#999999"})
 + scale_size_continuous(range=(1.5, 7), breaks=[10, 25, 50, 80])
 + scale_x_datetime(date_labels="%Y", date_breaks="2 years")
 + labs(x="Announcement date", y="", color="Tenant", size="Hectares")
 + theme_minimal()
 + theme(figure_size=(9, 3.6)))
```

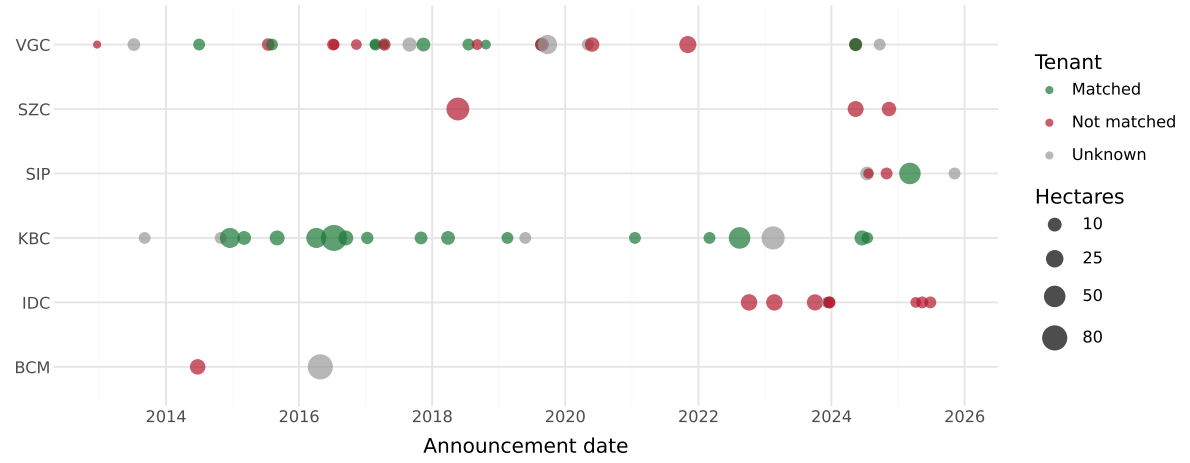


Figure 89.1: Lease announcements by developer, excluding follow-ups. Marker size is the stated area (deals without one are drawn at 5 hectares).

Table 89.2 and Figure 89.1 show the feature of the data that shapes everything that follows: **whether a deal is matched is almost a property of the developer.** All of KBC's coded deals are matched. Its parks in Bắc Ninh, Bắc Giang, and Hải Phòng were built around electronics (LG, Goertek, Luxshare, Foxconn), and it kept signing electronics tenants. None of IDICO's, Sonadezi Châu Đức's, or Becamex's coded deals are matched; their parks sign a mix of food, materials, and engineering tenants. Only Viglacera, and to a small degree SIP, signs both kinds.

This matters for the test. A pooled comparison of matched and unmatched deals is mostly a comparison of KBC with IDICO, two developers that also differ in size, region, investor base, and the stage of their land banks. The within-developer comparison, the analog of the REIT fixed effects in C. H. Liu, Zheng, and Zhu (2026), is cleaner but rests almost entirely on Viglacera's 22 coded deals. Both versions are reported in Section 89.5, and Section 89.4 shows what this structure does to inference.

Three limitations should be kept in mind.

- **Coverage is uneven.** Developers publicize flagship deals, so the sample over-represents large, high-profile tenants. KBC's archive thins out in 2019–2021 and 2024–2025, and several developers have no verifiable events. Exchange filings and annual reports, which were not searched systematically, are the next sources to add.
- **Some dates are publication dates.** For rows with `date_type` equal to `news` or `press_release`, the deal may have been known a few days earlier. The pre-event window in Section 89.5 checks for this.
- **Not every event is tradable.** Events dated before a developer's shares began trading drop out of the event study automatically. Check each developer's listing history, as in the survivorship chapter.

89.4 How Much Can These Announcements Detect?

Before running the event study, it helps to know what size of effect is plausible and what size the sample can detect.

A rough upper bound. Take a 25-hectare lease at USD 130 per m² over the remaining term, about the average tier-1 asking rent in northern Vietnam in 2024. That is 250,000 m² and about USD 32 million of revenue. If half of it is profit, a fully unanticipated deal adds about USD 16 million of value, or 1.6% for a developer worth USD 1 billion. Most deals are partly anticipated, because developers guide investors on their pipelines and many announcements follow an earlier MOU. A direct effect of one to two percent is therefore an upper bound for a large deal. The agglomeration channel is different. If a matched anchor raises the expected rent on the *rest* of the park's unleased land, the value effect can exceed the deal itself. That is the effect the test is looking for.

Inference with few developers. The collected sample has 48 events with a known match status, spread over six developers. Two features of such data make standard errors fragile. Events by the same developer share a common component, for instance how much of its pipeline the market already expects. And, as Table 89.2 shows, most developers sign mostly one type of tenant, so MATCH barely varies within developer. The simulation below generates two-day CARs as

$$\mathrm{CAR}_e = \delta \cdot \mathrm{MATCH}_e + u_{d(e)} + \varepsilon_e, \tag{89.4}$$

with an event-level standard deviation of 4.5%, roughly that of a two-day return for a mid-cap developer, and a developer-level shock u_d with a standard deviation of 2%. It compares three tests of $\delta = 0$: the classical OLS t-test, a cluster-robust t-test with $G - 1$ degrees of freedom, and a wild cluster bootstrap with the null imposed and Webb's six-point weights, which is designed for few clusters. The three scenarios are the collected sample size with tenant types mixed within each developer, the same size with each developer signing mostly one type (as in the collected data), and a sample three times larger.

```
from scipy import stats


def cluster_t(X, Y, g, j=1):
    """Cluster-robust (CR1) t-statistics of coefficient j, one per column of
     ↪ Y.

    X is (n, k) with the tested regressor in column j, Y is (n, B), and g
    holds cluster codes 0, ..., G-1.
    """
    n, k = X.shape
```

```
    G = g.max() + 1
    XtX_inv = np.linalg.inv(X.T @ X)
    beta = XtX_inv @ X.T @ Y
    resid = Y - X @ beta
    S = np.zeros((G, k, Y.shape[1]))
    for c in range(k):
        np.add.at(S[:, c, :], g, X[:, [c]] * resid)
    meat = np.einsum("gkb,glb->klb", S, S)
    scale = G / (G - 1) * (n - 1) / (n - k)
    var = scale * np.einsum("k,klb,l->b", XtX_inv[j], meat, XtX_inv[j])
    return beta[j] / np.sqrt(var)


WEBB = np.array([-np.sqrt(1.5), -1, -np.sqrt(0.5), np.sqrt(0.5), 1,
 ↪  np.sqrt(1.5)])


def wild_cluster_p(X, y, g, j=1, n_boot=999, rng=None):
    """Wild cluster bootstrap-t p-value for H0: beta_j = 0, with the null
    imposed and Webb's six-point weights, which behave well with few
 ↪  clusters.
    Returns the p-value and the original cluster-robust t-statistic."""
    rng = np.random.default_rng(rng)
    t0 = cluster_t(X, y[:, None], g, j)[0]
    Xr = np.delete(X, j, axis=1)
    fit_r = Xr @ np.linalg.lstsq(Xr, y, rcond=None)[0]
    w = rng.choice(WEBB, size=(g.max() + 1, n_boot))
    y_star = fit_r[:, None] + (y - fit_r)[:, None] * w[g]
    t_star = cluster_t(X, y_star, g, j)
    return np.mean(np.abs(t_star) >= abs(t0)), t0
```

```
def simulate_match_tests(effect, n_events, n_dev, sd_dev, mix,
 ↪  sd_event=0.045,
                         n_sims=1000, n_boot=199, seed=0):
    """Rejection rates (two-sided, 5%) of three tests of the matched-tenant
    premium when announcements cluster within a few developers.

        CAR_e = effect * match_e + u_dev(e) + eps_e

    Each developer has its own share of matched deals, drawn from a
    Beta(mix, mix) distribution: a small `mix` makes most developers sign
    only one type of tenant. The developer shock u_d (say, how well its
```

```
    announcements are anticipated) does not average out across its events.
    """
    rng = np.random.default_rng(seed)
    rej = np.zeros(3)
    for _ in range(n_sims):
        dev = rng.integers(0, n_dev, n_events)
        match = (rng.random(n_events) < rng.beta(mix, mix,
↪   n_dev)[dev]).astype(float)
        if match.min() == match.max():
            continue
        car = (effect * match + rng.normal(0, sd_dev, n_dev)[dev]
               + rng.normal(0, sd_event, n_events))
        g = np.unique(dev, return_inverse=True)[1]
        G = g.max() + 1
        X = np.column_stack([np.ones(n_events), match])
        fit = stats.linregress(match, car)
        p_wild, t_cl = wild_cluster_p(X, car, g, n_boot=n_boot, rng=rng)
        rej += [abs(fit.slope / fit.stderr) > stats.t.ppf(0.975, n_events -
↪   2),
                abs(t_cl) > stats.t.ppf(0.975, G - 1),
                p_wild < 0.05]
    return rej / n_sims


scenarios = {"48 events, 6 developers, mixed tenants": (48, 6, 2.0),
             "48 events, 6 developers, one type each": (48, 6, 0.3),
             "150 events, 12 developers, mixed tenants": (150, 12, 2.0)}
rows = []
for name, (n_events, n_dev, mix) in scenarios.items():
    for effect in [0.0, 0.01, 0.02, 0.03, 0.04, 0.05]:
        r = simulate_match_tests(effect, n_events, n_dev, sd_dev=0.02,
↪   mix=mix)
        rows.append({"scenario": name, "effect": effect,
                     "OLS t": r[0], "cluster t": r[1], "wild bootstrap":
                      ↪  r[2]})
power = pd.DataFrame(rows)
print(power.set_index(["scenario", "effect"]).round(3).to_string())
```

```
                                          OLS t  cluster t  wild bootstrap
scenario                                effect
48 events, 6 developers, mixed tenants  0.00    0.062      0.046           0.049
                                        0.01    0.112      0.077           0.083
```

```
                                                0.02    0.293      0.204          0.198
                                                0.03    0.546      0.382          0.364
                                                0.04    0.759      0.591          0.537
                                                0.05    0.899      0.744          0.700
48 events, 6 developers, one type each          0.00    0.120      0.091          0.055
                                                0.01    0.167      0.113          0.081
                                                0.02    0.294      0.196          0.148
                                                0.03    0.486      0.339          0.244
                                                0.04    0.700      0.514          0.370
                                                0.05    0.846      0.672          0.512
150 events, 12 developers, mixed tenants        0.00    0.109      0.058          0.051
                                                0.01    0.273      0.175          0.167
                                                0.02    0.659      0.523          0.508
                                                0.03    0.916      0.831          0.814
                                                0.04    0.996      0.970          0.960
                                                0.05    1.000      0.996          0.996
```

```
from plotnine import (ggplot, aes, geom_line, geom_point, geom_hline,
 ↪  facet_wrap,
                       labs, theme_minimal, theme, scale_x_continuous)

plot_df = power.melt(id_vars=["scenario", "effect"], var_name="test",
                     value_name="rejection_rate")
plot_df["scenario"] = pd.Categorical(plot_df["scenario"], list(scenarios))
(ggplot(plot_df, aes("effect * 100", "rejection_rate", color="test"))
 + geom_hline(yintercept=0.05, linetype="dotted")
 + geom_line() + geom_point(size=1.5)
 + facet_wrap("~scenario", ncol=3)
 + scale_x_continuous(breaks=[0, 1, 2, 3, 4, 5])
 + labs(x="True matched-tenant premium (%)", y="Rejection rate", color="")
 + theme_minimal()
 + theme(figure_size=(9, 3.4), legend_position="bottom"))
```

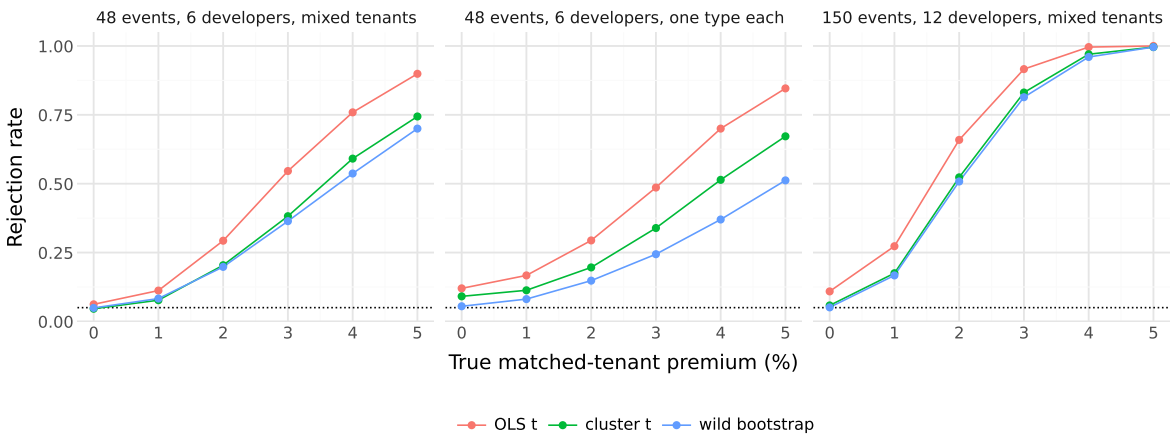


Figure 89.2: Rejection rates of three tests of the matched-tenant premium at the 5% level. At a true effect of zero the curves show the test's size; the dotted line marks 5%. Each scenario is 1,000 simulated samples.

Three lessons follow from Figure 89.2.

- **When each developer signs one type of tenant, only the wild bootstrap holds its size.** The classical t-test rejects a true null 12% of the time, because it treats a developer's announcements as independent observations. The cluster-robust t-test does better but still rejects 9% of the time, because six clusters are too few for it. The wild bootstrap rejects at 5.5%.
- **With 48 events, only very large effects are detectable.** At a true premium of 5 percentage points, the wild bootstrap rejects 70% of the time when tenant types are mixed and about half the time when they are not. A premium of the size C. H. Liu, Zheng, and Zhu (2026) report for US REITs (well under 1% per standard deviation) would be invisible.
- **More events, from more developers, help most.** With 150 events from 12 developers, power reaches 80% at a premium of about 3 percentage points. Extending the hand collection, to exchange filings, to developers without events so far, and to smaller deals, is the most valuable next step.

89.5 Application to Vietnamese Data

The code below runs the two tests on DataCore prices with the pipeline from the event-study chapter: `EventStudyConfig`, `run_event_study`, and the functions it calls. It assumes `ip_events` from Section 89.3 and the inference functions above.

- **Lease announcements.** Several deals signed at one ceremony are one event. Announcements by the same developer within ten days of each other are flagged, because their windows overlap. Abnormal returns come from the market model (VN-Index, estimation window of 150 days ending 15 days before the event). The test regresses CARs on MATCH with developer fixed effects, so the comparison is within developer: is the same developer's stock rewarded more for a matched tenant than for an unmatched one? Log hectares control for deal size. Inference uses the wild cluster bootstrap. A "clean" sample drops overlapping events and events with a price-limit hit on day 0 or +1 (Section 85.8 explains why limit hits push price discovery into later days).
- **The 2025 tariff shock.** On 3 April 2025 (Vietnam time) the United States announced a 46% tariff on Vietnamese goods. It paused the tariff on 9 April (10 April in Vietnam), and on 2 July (3 July in Vietnam) announced a deal at 20%, with 40% on transshipped goods. All developers share each date, so this is a cross-sectional test: regress each developer's CAR on its exposure, the share of its collected leased hectares that went to electronics tenants (VSIC 26–27). Concentration risk predicts a negative slope on the announcement and positive slopes on the pause and the deal. The first window runs to day +3, because many stocks closed at their floor price for several sessions after the announcement.

```
import statsmodels.formula.api as smf
from datacore import DataCoreClient

client = DataCoreClient()

# All listed IP developers, including those without a collected event: the
# tariff test uses the whole cross-section
developers = sorted(set(ip_events["ticker"]) | {"KBC", "IDC", "SZC", "BCM",
 ↪  "SIP",
                                                 "GVR", "PHR", "NTC", "D2D",
                                                  ↪  "LHG",
                                                 "TIP", "VGC", "SNZ"})

daily = client.get_daily_prices(
    exchanges=["HOSE", "HNX", "UPCoM"], start_date="2011-01-01",
    end_date="2025-12-31", include_delisted=True,
    fields=["ticker", "date", "exchange", "adjusted_close", "volume",
            "market_cap", "limit_up_hit", "limit_down_hit"])
daily["date"] = pd.to_datetime(daily["date"])
daily = (daily[daily["ticker"].isin(developers)]
         .rename(columns={"ticker": "symbol"})
         .sort_values(["symbol", "date"]).reset_index(drop=True))
daily["ret"] = daily.groupby("symbol")["adjusted_close"].pct_change()
```

```
vn = client.get_index_returns(index="VNINDEX", frequency="daily",
                              start_date="2011-01-01", end_date="2025-12-31")
vn["date"] = pd.to_datetime(vn["date"])
# Market model on raw returns; the intercept absorbs the risk-free rate
factors = vn[["date"]].assign(mkt_excess=vn["return"])

config = EventStudyConfig(estimation_window=150, gap=15,
                          event_window_start=-5, event_window_end=5,
                          min_estimation_obs=100,
                          risk_model=RiskModel.MARKET_MODEL)
windows = {"car_m5_m1": (-5, -1), "car_0_p1": (0, 1),
           "car_m1_p1": (-1, 1), "car_0_p5": (0, 5)}


def event_cars(events):
    """Run the chapter's pipeline and return one row per event with CARs
    over each window and a flag for price-limit hits on days 0 and +1."""
    res = run_event_study(events, daily[["symbol", "date", "ret"]], factors,
                          config, verbose=False)
    d = res["daily_ar"]
    cars = pd.concat(
        [d[d["evttime"].between(lo, hi)].groupby(["symbol", "evtdate"])["AR"]
         .sum().rename(name) for name, (lo, hi) in windows.items()],
        axis=1).reset_index()
    hits = (d[d["evttime"].between(0, 1)]
            .merge(daily[["symbol", "date", "limit_up_hit",
 ↪  "limit_down_hit"]],
                   on=["symbol", "date"], how="left"))
    hits["hit"] = hits["limit_up_hit"].fillna(False) |
 ↪  hits["limit_down_hit"].fillna(False)
    return cars.merge(hits.groupby(["symbol", "evtdate"])["hit"].any()
                      .rename("limit_hit").reset_index(), on=["symbol",
 ↪  "evtdate"])


# --- 1. Lease announcements: matched versus other tenants
 ↪  -------------------
# Later stages of a deal already announced are not news. Several deals signed
# at one ceremony are one event for the developer's stock
ev = (ip_events[ip_events["followup"] == 0].dropna(subset=["match"])
      .groupby(["ticker", "announce_date"])
      .agg(match=("match", "max"), hectares=("hectares", lambda x:
 ↪  x.sum(min_count=1)),
```

```
           n_deals=("tenant", "size"))
      .reset_index()
      .rename(columns={"ticker": "symbol", "announce_date": "event_date"}))
# Another announcement by the same developer within 10 days contaminates both
gap = ev.sort_values("event_date").groupby("symbol")["event_date"].diff().dt
 ↪  .days
gap_next = ev.sort_values("event_date").groupby("symbol")["event_date"].diff
 ↪  (-1).dt.days.abs()
ev["overlap"] = (gap.le(10) |
 ↪  gap_next.le(10)).reindex(ev.index).fillna(False)

cars = event_cars(ev[["symbol", "event_date"]])
# The pipeline moves event dates to the next trading day; recover each
# event's attributes from the latest original date on or before it
cars = pd.merge_asof(cars.sort_values("evtdate"),
 ↪  ev.sort_values("event_date"),
                     left_on="evtdate", right_on="event_date", by="symbol",
                     direction="backward")
cars["log_ha"] = np.log1p(cars["hectares"].fillna(cars["hectares"].median()))
cars["year"] = cars["evtdate"].dt.year

print(cars.groupby("match")[list(windows)].agg(["mean", "count"]).round(4))

rows = []
for sample, x in [("all", cars), ("clean", cars[~cars["overlap"] &
 ↪  ~cars["limit_hit"]])]:
    for y in windows:
        sub = x.dropna(subset=[y]).reset_index(drop=True)
        g = sub["symbol"].astype("category").cat.codes.to_numpy()
        for spec in ["pooled", "within"]:
            X = pd.concat([pd.Series(1.0, index=sub.index, name="const"),
                           sub["match"].astype(float), sub["log_ha"]],
 ↪  axis=1)
            if spec == "within":
                # Developer fixed effects: is the same developer's stock
                 ↪  rewarded
                # more for a matched tenant than for an unmatched one?
                X = pd.concat([X, pd.get_dummies(sub["symbol"],
 ↪  drop_first=True,
                                                 dtype=float)], axis=1)
            beta = np.linalg.lstsq(X.to_numpy(), sub[y].to_numpy(),
 ↪  rcond=None)[0]
```

```
            p_wild, t_cl = wild_cluster_p(X.to_numpy(), sub[y].to_numpy(), g,
 ↪  j=1,
                                          n_boot=1999, rng=1)
            rows.append({"sample": sample, "window": y, "spec": spec,
                         "N": len(sub), "developers": g.max() + 1,
                         "match_premium": beta[1], "t_cluster": t_cl,
                         "p_wild": p_wild})
print(pd.DataFrame(rows).round(4).to_string(index=False))

# --- 2. The 2025 US tariff shock: concentration risk
 ↪  -------------------------
# Exposure: the share of each developer's leased hectares (deals announced
# before the shock) that went to electronics tenants (VSIC 26-27), the sector
# most exposed to US demand
base = ip_events[(ip_events["announce_date"] < "2025-04-01")
                 & (ip_events["followup"] == 0)].dropna(subset=["hectares"])
exposure = (base.assign(elec_ha=base["hectares"] * base["vsic2"].isin([26,
 ↪  27]))
            .groupby("ticker")[["elec_ha", "hectares"]].sum()
            .eval("elec_ha / hectares").rename("exposure"))

shocks = {"46% tariff announced": ("2025-04-03", (0, 3)),
          "90-day pause": ("2025-04-10", (0, 1)),
          "20% deal": ("2025-07-03", (0, 1))}
for label, (day, (lo, hi)) in shocks.items():
    windows["car_shock"] = (lo, hi)
    sh = event_cars(pd.DataFrame({"symbol": developers,
                                  "event_date": pd.Timestamp(day)}))
    sh = sh.merge(exposure, left_on="symbol", right_index=True, how="inner")
    fit = smf.ols("car_shock ~ exposure", data=sh).fit(cov_type="HC1")
    print(f"{label:22s} N={len(sh):2d}  slope={fit.params['exposure']:.4f} "
          f"(HC1 t={fit.tvalues['exposure']:.2f})  "
          f"limit hits={int(sh['limit_hit'].sum())}")
del windows["car_shock"]
```

We tested the code on synthetic prices shaped like the DataCore tables, using the actual dates of the collected events. Matched deals carried a planted day-0 abnormal return of 3% and other deals 0.5%, and the tariff dates carried planted effects of −8%, +4%, and +3% times exposure. With low idiosyncratic noise the code recovered all of them: a matched premium of 2.5 to 2.7 percentage points across windows and specifications, and exposure slopes of −0.085, 0.037, and 0.031. With realistic noise of 2.5% a day, the same planted premium was not significant in the within-developer specification (wild bootstrap p-values between 0.13 and 0.40 in the event

windows), as Section 89.4 predicts for six developers. Only the six developers with collected deals have an exposure measure, so the tariff regressions have six observations. Read the output in this order:

1. **The** $(0, +1)$ **match premium in the clean sample**, within developer, is the main estimate. Judge it by the wild bootstrap p-value, not by the cluster-robust t. If the pooled estimate is much larger than the within estimate, the pooled premium mostly reflects the difference between KBC and the other developers.
2. **The pre-event window** $(-5, -1)$ is a leakage check. A positive premium there means matched deals were known before the date used, so move the date earlier or widen the window.
3. **The** $(0, +5)$ **window** shows whether price limits and thin trading delayed the reaction.
4. **The tariff slopes** test concentration risk. With six developers, read their signs and the pattern across the three dates, not their individual significance.

89.6 Extensions

The event study tests whether investors *expect* clustering to pay. Testing whether it *does* pay, as C. H. Liu, Zheng, and Zhu (2026) do with prices and NOI, needs a park-level panel. Three extensions build it.

A park panel. For each park and year, compute the investor HHI (Equation 89.2 with leased hectares), the anchor's share of leased land, and SPEC (Equation 89.3) with employment from the GSO Enterprise Survey, using the province or a radius of about 25 km around the park as the neighborhood. Then regress lease prices, occupancy, and hectares absorbed on these measures with year, developer, and province fixed effects, and run the paper's horse race between regional concentration and SPEC. Lease prices are reported mostly by brokers as asking rents by province and tier, so occupancy and absorption will often be the better-measured outcomes. Freeze locations on pre-2025 province codes: on 1 July 2025 Vietnam merged its 63 provinces into 34 and abolished the district level.

Anchor arrivals. The arrival of a large anchor is a staggered treatment. Comparing rents and absorption in parks, and in neighboring parks, before and after an anchor signs, against parks matched on province and age, estimates spillovers directly. This is the logic of Greenstone, Hornbeck, and Moretti (2010), who compare counties that won a large plant with the runners-up, and the staggered timing calls for an estimator such as Callaway and Sant'Anna (2021). The original paper, with its cross-section of transactions, could not run this test.

Which mechanism. C. H. Liu, Zheng, and Zhu (2026) attribute their premium to knowledge spillovers, which are strongest within an industry. In Vietnam's parks, input sharing may matter more: suppliers locate next to the assembler they sell to. Ellison, Glaeser, and Kerr (2010) separate the channels by asking which industry pairs co-locate. Here the analog is to add a measure of how much of the anchor's inputs the region already produces, built from

the GSO input-output table. A premium that loads on supplier links rather than on the same industry would point to input sharing.

89.7 Summary

- C. H. Liu, Zheng, and Zhu (2026) find that office values rise with agglomeration only when a building's tenants and its neighborhood specialize in the same industry, and that REIT investors price this in. The horse race between concentration and matched specialization is what identifies the mechanism.
- In Vietnam, industrial parks are the natural setting, and listed developers' lease announcements are the natural events. Section 89.3 presents 71 hand-collected announcements by six developers, with sources.
- In these data, whether a deal is matched is almost a property of the developer: KBC signs electronics tenants into electronics parks, IDICO signs a mix. The within-developer test rests largely on Viglacera.
- With six developers, only the wild cluster bootstrap gives reliable inference, and only large effects are detectable. Collecting more events matters more than refining the test.
- The 2025 US tariff announcements give a rare outside shock to test the other side of the trade-off: the risk of a concentrated tenant base.

90 Tail Risk and Extreme Events

Finance is fundamentally a discipline of extremes. The central objects of concern (e.g., asset pricing, portfolio allocation, capital regulation, systemic stability) are all disproportionately shaped by rare but severe events. A portfolio manager who correctly estimates the mean and variance of returns but ignores the possibility of a -20% daily move is, in the language of Taleb (2010), "picking up pennies in front of a steamroller." The 1997 Asian Financial Crisis, the 2008 Global Financial Crisis, and the 2020 COVID-19 crash each demonstrated that the most consequential realizations in financial markets are precisely those that conventional Gaussian models assign near-zero probability.

This chapter develops the statistical and econometric toolkit for measuring, modeling, and diagnosing tail risk in Vietnamese equity markets. We focus on three interconnected layers. First, at the individual stock level, we estimate crash risk measures that capture the asymmetry and thickness of the left tail of return distributions. Second, at the portfolio or regulatory level, we implement Value at Risk (VaR) and Expected Shortfall (ES) under multiple estimation approaches and evaluate their accuracy via backtesting. Third, at the system level, we apply Extreme Value Theory (EVT) and tail dependence measures to characterize joint crash behavior across sectors and assess financial contagion.

Vietnamese markets present a particularly rich laboratory for studying tail phenomena. Daily price limits mechanically censor the tails of observed return distributions, creating latent tail risk that is not directly observable but must be inferred. State ownership concentrations, thin liquidity, and herding behavior among retail investors amplify crash dynamics. The absence of a developed derivatives market means that tail hedging instruments familiar in more developed markets (e.g., deep out-of-the-money puts, variance swaps) are largely unavailable, making accurate tail risk measurement all the more important for risk management.

```
import pandas as pd
import numpy as np
from scipy import stats
from scipy.optimize import minimize
import plotnine as p9
from mizani.formatters import percent_format, comma_format
import warnings
warnings.filterwarnings("ignore")
```

90.1 Crash Risk in Equity Markets

90.1.1 The Left Tail Problem

The return distribution of individual stocks and market indices is not symmetric. Decades of evidence, beginning with Mandelbrot et al. (1963) and formalized in the asset pricing context by Campbell R. Harvey and Siddique (2000), establish that equity returns exhibit negative skewness (i.e., large negative returns occur more frequently than a Gaussian model predicts) and excess kurtosis (i.e., the tails are thicker than normal in both directions, but the left tail is of primary economic concern).

Let $r_{i,t}$ denote the daily log return of stock i on day t. The standardized third central moment (skewness) is:

$$\text{Skew}(r_i) = \frac{E\left[(r_{i,t} - \mu_i)^3\right]}{\sigma_i^3} \tag{90.1}$$

Negative skewness implies that the distribution has a longer or fatter left tail relative to the right. In economic terms, negative skewness means that large losses are more likely than large gains of equal magnitude. The standardized fourth moment (excess kurtosis) is:

$$\text{Kurt}(r_i) = \frac{E\left[(r_{i,t} - \mu_i)^4\right]}{\sigma_i^4} - 3 \tag{90.2}$$

Positive excess kurtosis indicates heavier tails than the normal distribution. Both moments have direct asset pricing implications: Campbell R. Harvey and Siddique (2000) and Kraus and Litzenberger (1976) argue that investors demand compensation for holding negatively skewed assets, implying that crash-prone stocks should earn higher expected returns, ceteris paribus.

90.1.2 Crash Risk Measures

The literature has developed several firm-specific crash risk measures. We implement the three most widely used, following Joseph Chen, Hong, and Stein (2001) and Hutton, Marcus, and Tehranian (2009).

Measure 1: Negative Coefficient of Skewness (NCSKEW)

For each firm i in fiscal year y, compute firm-specific weekly returns $W_{i,\tau}$ as residuals from a market model augmented with lead and lag market returns to account for nonsynchronous trading:

$$r_{i,t} = \alpha_i + \beta_{1i} r_{m,t-1} + \beta_{2i} r_{m,t} + \beta_{3i} r_{m,t+1} + \varepsilon_{i,t} \tag{90.3}$$

where $r_{m,t}$ is the value-weighted market return on day t. The firm-specific weekly return is $W_{i,\tau} = \ln(1 + \sum_{t \in \tau} \hat{\varepsilon}_{i,t})$, where τ indexes weeks. NCSKEW is then:

$$\text{NCSKEW}_{i,y} = -\frac{n(n-1)^{3/2} \sum W_{i,\tau}^3}{(n-1)(n-2)\left(\sum W_{i,\tau}^2\right)^{3/2}} \tag{90.4}$$

where n is the number of firm-specific weekly returns in the year. The negative sign ensures that higher NCSKEW corresponds to greater crash risk.

Measure 2: Down-to-Up Volatility (DUVOL)

Partition the firm-specific weekly returns into "down weeks" ($W_{i,\tau} < \bar{W}_i$) and "up weeks" ($W_{i,\tau} \geq \bar{W}_i$), where $\bar{W}_i$ is the annual mean. Then:

$$\text{DUVOL}_{i,y} = \ln\left(\frac{(n_u - 1)\sum_{\text{down}} W_{i,\tau}^2}{(n_d - 1)\sum_{\text{up}} W_{i,\tau}^2}\right) \tag{90.5}$$

where n_u and n_d are the number of up and down weeks, respectively. Higher DUVOL indicates greater crash risk. Joseph Chen, Hong, and Stein (2001) argue that DUVOL is less sensitive to outliers than NCSKEW because it does not involve the cube of returns.

Measure 3: Crash Count (COUNT)

Define a crash event as a firm-specific weekly return that falls below k standard deviations of its annual distribution:

$$\text{CRASH}_{i,\tau} = \mathbb{1}\left(W_{i,\tau} < \bar{W}_i - k \cdot \hat{\sigma}_{W_i}\right) \tag{90.6}$$

The standard threshold in the literature is $k = 3.09$, corresponding to the 0.1% tail of a standard normal distribution. The crash count for year y is $\text{COUNT}_{i,y} = \sum_{\tau \in y} \text{CRASH}_{i,\tau}$.

```
# DataCore.vn API
from datacore import DataCore
dc = DataCore()

# Load daily stock returns and market returns
daily_returns = dc.get_daily_returns(
    start_date="2010-01-01",
    end_date="2024-12-31"
```

```
)

# Market return: value-weighted index
market_returns = dc.get_market_returns(
    start_date="2010-01-01",
    end_date="2024-12-31",
    frequency="daily"
)

# Merge
df = daily_returns.merge(
    market_returns[["date", "mkt_ret"]],
    on="date",
    how="left"
)

print(f"Universe: {df['ticker'].nunique()} firms, "
      f"{df['date'].nunique()} trading days")
```

```
from sklearn.linear_model import LinearRegression

def compute_firm_specific_returns(group):
    """
    Estimate firm-specific daily returns from augmented market model
    with lead and lag market returns (Chen, Hong, Stein 2001).
    """
    g = group.sort_values("date").copy()
    g["mkt_lag1"] = g["mkt_ret"].shift(1)
    g["mkt_lead1"] = g["mkt_ret"].shift(-1)
    g = g.dropna(subset=["ret", "mkt_ret", "mkt_lag1", "mkt_lead1"])

    if len(g) < 30:
        return pd.DataFrame()

    X = g[["mkt_lag1", "mkt_ret", "mkt_lead1"]].values
    y = g["ret"].values

    model = LinearRegression().fit(X, y)
    g["resid"] = y - model.predict(X)

    return g[["ticker", "date", "resid"]]
```

```
# Estimate by firm-year
df["year"] = df["date"].dt.year
firm_resids = (
    df.groupby(["ticker", "year"], group_keys=False)
    .apply(compute_firm_specific_returns)
    .reset_index(drop=True)
)
```

```
# Aggregate daily residuals to weekly firm-specific returns
firm_resids["date"] = pd.to_datetime(firm_resids["date"])
firm_resids["week"] = firm_resids["date"].dt.isocalendar().week.astype(int)
firm_resids["year"] = firm_resids["date"].dt.year

weekly_returns = (
    firm_resids.groupby(["ticker", "year", "week"])
    .agg(W=("resid", lambda x: np.log(1 + x.sum())))
    .reset_index()
)
```

```
def compute_crash_measures(group):
    """Compute NCSKEW, DUVOL, and CRASH count for one firm-year."""
    W = group["W"].values
    n = len(W)

    if n < 26:  # Require at least 26 weeks
        return pd.Series({
            "ncskew": np.nan, "duvol": np.nan,
            "crash_count": np.nan, "n_weeks": n
        })

    # NCSKEW
    W_demean = W - W.mean()
    num = n * (n - 1)**1.5 * np.sum(W_demean**3)
    den = (n - 1) * (n - 2) * (np.sum(W_demean**2))**1.5
    ncskew = -(num / den) if den != 0 else np.nan

    # DUVOL
    mean_W = W.mean()
    down = W[W < mean_W]
    up = W[W >= mean_W]
```

```
    n_d, n_u = len(down), len(up)

    if n_d > 1 and n_u > 1:
        duvol = np.log(
            ((n_u - 1) * np.sum(down**2)) /
            ((n_d - 1) * np.sum(up**2))
        )
    else:
        duvol = np.nan

    # Crash count (k = 3.09)
    threshold = W.mean() - 3.09 * W.std()
    crash_count = int(np.sum(W < threshold))

    return pd.Series({
        "ncskew": ncskew,
        "duvol": duvol,
        "crash_count": crash_count,
        "n_weeks": n
    })

crash_risk = (
    weekly_returns.groupby(["ticker", "year"])
    .apply(compute_crash_measures)
    .reset_index()
)

crash_risk = crash_risk.dropna(subset=["ncskew", "duvol"])
print(f"Crash risk panel: {len(crash_risk)} firm-years")
```

90.1.3 Cross-Sectional Distribution of Crash Risk

Table 90.1 presents summary statistics for the three crash risk measures across the Vietnamese equity market.

The rightward shift of the NCSKEW distribution during crisis episodes, particularly in 2020 and 2022, indicates a market-wide increase in crash risk. The red dashed line at zero separates firms with negative skewness (right of zero, i.e., high NCSKEW since the measure is negated) from those with positive skewness.

Table 90.1: Summary Statistics of Crash Risk Measures

```
summary = crash_risk[["ncskew", "duvol", "crash_count"]].describe(
    percentiles=[0.05, 0.25, 0.5, 0.75, 0.95]
).T.round(4)

summary.columns = ["N", "Mean", "Std", "Min", "5%", "25%",
                   "Median", "75%", "95%", "Max"]
summary
```

```
plot_data = crash_risk[crash_risk["year"].between(2012, 2024)].copy()

(
    p9.ggplot(plot_data, p9.aes(x="ncskew"))
    + p9.geom_histogram(bins=50, fill="#2E5090", alpha=0.7)
    + p9.facet_wrap("~year", scales="free_y", ncol=4)
    + p9.geom_vline(xintercept=0, linetype="dashed", color="red", size=0.5)
    + p9.labs(
        x="NCSKEW",
        y="Count",
        title="Distribution of Firm-Level Crash Risk (NCSKEW)"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 8))
)
```

Figure 90.1

90.1.4 Interpretation in Asset Pricing

Crash risk has first-order implications for asset pricing. Campbell R. Harvey and Siddique (2000) demonstrate that coskewness (i.e., the contribution of an individual asset to the skewness of the aggregate market portfolio) is priced in the cross-section of expected returns. Specifically, assets that tend to crash when the market crashes (negative coskewness) should command a risk premium.

The coskewness of stock i with the market is:

$$\text{CoSkew}_i = \frac{E\left[(r_i - \mu_i)(r_m - \mu_m)^2\right]}{\sigma_i \cdot \sigma_m^2} \tag{90.7}$$

Stocks with more negative coskewness exhibit disproportionately poor performance during market downturns. In Vietnam, this is especially relevant because: (1) retail investor herding amplifies co-movement during selloffs; (2) price limits delay crash completion, spreading crash risk across multiple days; and (3) state-owned enterprises, which constitute a large fraction of market capitalization, may exhibit coordinated crash behavior driven by common policy shocks.

```
def compute_coskewness(group):
    """Estimate coskewness for a firm-year."""
    r_i = group["ret"].values
    r_m = group["mkt_ret"].values

    if len(r_i) < 60:
        return np.nan

    r_i_dm = r_i - r_i.mean()
    r_m_dm = r_m - r_m.mean()

    coskew = (
        np.mean(r_i_dm * r_m_dm**2) /
        (np.std(r_i) * np.std(r_m)**2)
    )
    return coskew

df_coskew = (
    df.groupby(["ticker", "year"])
    .apply(lambda g: pd.Series({"coskewness": compute_coskewness(g)}))
    .reset_index()
    .dropna()
)
```

```
median_coskew = (
    df_coskew.groupby("year")
    .agg(
        median_coskew=("coskewness", "median"),
        q25=("coskewness", lambda x: x.quantile(0.25)),
        q75=("coskewness", lambda x: x.quantile(0.75))
    )
    .reset_index()
)

(
    p9.ggplot(median_coskew, p9.aes(x="year"))
    + p9.geom_ribbon(
        p9.aes(ymin="q25", ymax="q75"),
        fill="#2E5090", alpha=0.2
    )
    + p9.geom_line(p9.aes(y="median_coskew"), color="#2E5090", size=1)
    + p9.geom_hline(yintercept=0, linetype="dashed", color="gray")
    + p9.labs(
        x="Year",
        y="Coskewness",
        title="Cross-Sectional Coskewness: Median and Interquartile Range"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 5))
)
```

Figure 90.2

90.1.5 Downside Tail Concentration

Beyond skewness, the concentration of returns in the extreme left tail provides additional diagnostic information. We define the downside tail concentration ratio as the fraction of total return variance attributable to observations below the p-th percentile:

$$\text{TCR}_p = \frac{\sum_{r_{i,t} < q_p} (r_{i,t} - \bar{r}_i)^2}{\sum_{t} (r_{i,t} - \bar{r}_i)^2} \tag{90.8}$$

where q_p is the p-th percentile of the firm's return distribution. Under a symmetric distribution, $\text{TCR}_{5\%}$ would be close to 5% (with a slight upward bias due to the squaring). Values substantially exceeding 5% indicate that the left tail contributes disproportionately to total risk.

```
def tail_concentration_ratio(returns, p=0.05):
    """Compute tail concentration ratio at percentile p."""
    q = np.quantile(returns, p)
    r_dm = returns - returns.mean()
    total_var = np.sum(r_dm**2)
    tail_var = np.sum(r_dm[returns < q]**2)
    return tail_var / total_var if total_var > 0 else np.nan

tcr = (
    df.groupby(["ticker", "year"])
    .agg(
        tcr_5=("ret", lambda x: tail_concentration_ratio(x.values, 0.05)),
        tcr_1=("ret", lambda x: tail_concentration_ratio(x.values, 0.01)),
        n_obs=("ret", "count")
    )
    .reset_index()
    .query("n_obs >= 120")
)

print("Median TCR(5%):", round(tcr["tcr_5"].median(), 4))
print("Median TCR(1%):", round(tcr["tcr_1"].median(), 4))
```

90.2 Value at Risk and Expected Shortfall

90.2.1 Definitions

Value at Risk (VaR) and Expected Shortfall (ES) are the two principal quantile-based risk measures used in financial regulation and internal risk management. For a portfolio return r with the cumulative distribution function F, the VaR at confidence level α is:

$$\text{VaR}_\alpha = -F^{-1}(\alpha) = -\inf\{x : F(x) \geq \alpha\} \tag{90.9}$$

This is simply the negative of the α-quantile of the return distribution. For $\alpha = 0.01$, $\text{VaR}_{1\%}$ answers: "What is the loss that is exceeded with only 1% probability?"

Expected Shortfall (also called Conditional VaR or CVaR) is the expected loss conditional on the loss exceeding VaR:

$$\text{ES}_\alpha = -\frac{1}{\alpha}\int_0^\alpha F^{-1}(u)\,du = -E\left[r \mid r \leq -\text{VaR}_\alpha\right] \tag{90.10}$$

For continuous distributions, ES simplifies to the conditional expectation in Equation 90.10. ES is always at least as large as VaR ($\text{ES}_\alpha \geq \text{VaR}_\alpha$) and is strictly larger whenever the distribution has any mass beyond the VaR quantile.

90.2.2 Why Expected Shortfall Dominates VaR

Table 90.2 summarizes the fundamental differences.

Table 90.2: Comparison of VaR and Expected Shortfall

Property	VaR	Expected Shortfall
Definition	Quantile of the loss distribution	Conditional mean loss beyond VaR
Tail information	None beyond the quantile	Captures severity of tail losses
Subadditivity	Not subadditive in general	Subadditive (coherent risk measure)
Regulatory status	Basel II primary measure	Basel III/IV primary measure
Backtesting	Binary: breach or no breach	Continuous: requires severity assessment

Property	VaR	Expected Shortfall
Sensitivity to tail shape	None	Directly sensitive

The critical deficiency of VaR is the absence of subadditivity. A risk measure ρ is subadditive if $\rho(X+Y) \leq \rho(X) + \rho(Y)$ for all portfolios X, Y. VaR violates this condition for non-elliptical distributions, meaning that diversification can appear to increase risk under VaR, which is a perverse result. Artzner et al. (1999) established the axiomatic framework for coherent risk measures, and ES satisfies all four axioms: monotonicity, translation invariance, positive homogeneity, and subadditivity. VaR satisfies only three.

90.2.3 Estimation Methods

We implement four approaches to VaR and ES estimation, each with distinct assumptions and data requirements.

Method 1: Historical Simulation

The simplest nonparametric approach uses the empirical distribution directly:

$$\widehat{\mathrm{VaR}}_{\alpha}^{\mathrm{HS}} = -\hat{F}_{n}^{-1}(\alpha) = -r_{(\lceil n\alpha \rceil)} \tag{90.11}$$

where $r_{(k)}$ is the k-th order statistic of the return sample. ES is the average of all returns below the VaR:

$$\widehat{\mathrm{ES}}_{\alpha}^{\mathrm{HS}} = -\frac{1}{\lfloor n\alpha \rfloor} \sum_{k=1}^{\lfloor n\alpha \rfloor} r_{(k)} \tag{90.12}$$

Method 2: Parametric (Gaussian)

Assume $r_t \sim N(\mu, \sigma^2)$. Then:

$$\mathrm{VaR}_{\alpha}^{\mathrm{Gauss}} = -(\hat{\mu} + \hat{\sigma} \cdot \Phi^{-1}(\alpha)) \tag{90.13}$$

$$\mathrm{ES}_{\alpha}^{\mathrm{Gauss}} = -\hat{\mu} + \hat{\sigma} \cdot \frac{\phi(\Phi^{-1}(\alpha))}{\alpha} \tag{90.14}$$

where Φ and ϕ are the standard normal CDF and PDF, respectively. The Gaussian assumption is known to underestimate tail risk for equity returns, but provides a useful baseline.

Method 3: Parametric (Student-t)

The Student-t distribution with ν degrees of freedom captures excess kurtosis. The VaR and ES are:

$$\text{VaR}_\alpha^t = -\left(\hat{\mu} + \hat{\sigma}\sqrt{\frac{\nu-2}{\nu}} \cdot t_\nu^{-1}(\alpha)\right) \tag{90.15}$$

$$\text{ES}_\alpha^t = -\hat{\mu} + \hat{\sigma}\sqrt{\frac{\nu-2}{\nu}} \cdot \frac{f_{t_\nu}(t_\nu^{-1}(\alpha))}{\alpha} \cdot \frac{\nu + (t_\nu^{-1}(\alpha))^2}{\nu - 1} \tag{90.16}$$

where t_ν^{-1} and f_{t_ν} are the quantile function and PDF of the Student-t distribution.

Method 4: GARCH-Filtered EVT (Semi-Parametric)

This approach, developed by McNeil and Frey (2000), first filters returns through a GARCH(1,1) model to obtain standardized residuals $z_t = \hat{\varepsilon}_t / \hat{\sigma}_t$, then fits a Generalized Pareto Distribution (GPD) to the lower tail of the standardized residuals. We detail the EVT component in Section 90.3.

```
def estimate_var_es(returns, alpha=0.01):
    """
    Estimate VaR and ES at level alpha using four methods.

    Parameters
    ----------
    returns : array-like
        Return series.
    alpha : float
        Tail probability (e.g., 0.01 for 1%).

    Returns
    -------
    dict : VaR and ES estimates for each method.
    """
    r = np.array(returns)
    r = r[~np.isnan(r)]
    n = len(r)
    mu, sigma = r.mean(), r.std(ddof=1)

    results = {}

    # Method 1: Historical simulation
```

```
    sorted_r = np.sort(r)
    idx = int(np.ceil(n * alpha))
    results["var_hs"] = -sorted_r[idx - 1]
    results["es_hs"] = -sorted_r[:idx].mean()

    # Method 2: Gaussian
    z_alpha = stats.norm.ppf(alpha)
    results["var_gauss"] = -(mu + sigma * z_alpha)
    results["es_gauss"] = -mu + sigma * stats.norm.pdf(z_alpha) / alpha

    # Method 3: Student-t (MLE for degrees of freedom)
    try:
        nu, loc, scale = stats.t.fit(r)
        t_alpha = stats.t.ppf(alpha, df=nu)
        results["var_t"] = -(loc + scale * t_alpha)
        results["es_t"] = (
            -loc + scale *
            (stats.t.pdf(t_alpha, df=nu) / alpha) *
            ((nu + t_alpha**2) / (nu - 1))
        )
        results["nu_hat"] = nu
    except Exception:
        results["var_t"] = np.nan
        results["es_t"] = np.nan
        results["nu_hat"] = np.nan

    return results
```

```
# Compute VaR and ES for the aggregate market index
market_daily = dc.get_market_returns(
    start_date="2010-01-01",
    end_date="2024-12-31",
    frequency="daily"
)

# Rolling 1-year VaR/ES
window = 252
results_list = []

for end_idx in range(window, len(market_daily)):
    window_returns = market_daily["mkt_ret"].iloc[end_idx -
 ↪  window:end_idx].values
```

```
    date = market_daily["date"].iloc[end_idx]

    est = estimate_var_es(window_returns, alpha=0.01)
    est["date"] = date
    results_list.append(est)

var_es_ts = pd.DataFrame(results_list)
```

```
plot_var = var_es_ts.melt(
    id_vars="date",
    value_vars=["var_hs", "var_gauss", "var_t"],
    var_name="method",
    value_name="var"
)

method_labels = {
    "var_hs": "Historical Simulation",
    "var_gauss": "Gaussian",
    "var_t": "Student-t"
}
plot_var["method"] = plot_var["method"].map(method_labels)

(
    p9.ggplot(plot_var, p9.aes(x="date", y="var", color="method"))
    + p9.geom_line(alpha=0.8, size=0.6)
    + p9.labs(
        x="", y="1% VaR (Loss)",
        title="Rolling 252-Day Value at Risk Estimates",
        color="Method"
    )
    + p9.scale_color_manual(
        values=["#2E5090", "#C0392B", "#27AE60"]
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 5), legend_position="top")
)
```

Figure 90.3

```
plot_es = var_es_ts.melt(
    id_vars="date",
    value_vars=["es_hs", "es_gauss", "es_t"],
    var_name="method",
    value_name="es"
)

method_labels_es = {
    "es_hs": "Historical Simulation",
    "es_gauss": "Gaussian",
    "es_t": "Student-t"
}
plot_es["method"] = plot_es["method"].map(method_labels_es)

(
    p9.ggplot(plot_es, p9.aes(x="date", y="es", color="method"))
    + p9.geom_line(alpha=0.8, size=0.6)
    + p9.labs(
        x="", y="1% ES (Loss)",
        title="Rolling 252-Day Expected Shortfall Estimates",
        color="Method"
    )
    + p9.scale_color_manual(
        values=["#2E5090", "#C0392B", "#27AE60"]
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 5), legend_position="top")
)
```

Figure 90.4

90.2.4 VaR Backtesting

A VaR model is only useful if its predictions are accurate. Backtesting evaluates whether the realized frequency of VaR breaches is consistent with the nominal confidence level. If VaR is estimated at the $\alpha = 1\%$ level, we expect approximately 1% of days to exhibit losses exceeding VaR.

The Kupiec unconditional coverage test (Kupiec et al. 1995) evaluates whether the total number of breaches x in n observations is consistent with α:

$$\text{LR}_{\text{UC}} = -2 \ln \left(\frac{\alpha^x (1-\alpha)^{n-x}}{\hat{p}^x (1-\hat{p})^{n-x}} \right) \sim \chi^2(1) \tag{90.17}$$

where $\hat{p} = x/n$ is the empirical breach frequency. The Christoffersen conditional coverage test (P. F. Christoffersen 1998) additionally evaluates whether breaches are independent (no clustering):

$$\text{LR}_{\text{CC}} = \text{LR}_{\text{UC}} + \text{LR}_{\text{IND}} \tag{90.18}$$

where LR_{IND} tests the null of independence using a first-order Markov chain model for breach indicators.

```
def kupiec_test(returns, var_estimates, alpha=0.01):
    """
    Kupiec unconditional coverage test for VaR backtesting.

    Returns
    -------
    dict : breach count, expected, breach rate, LR statistic, p-value.
    """
    breaches = returns < -var_estimates
    x = breaches.sum()
    n = len(returns)
    p_hat = x / n

    if p_hat == 0 or p_hat == 1:
        return {"breaches": x, "expected": n * alpha,
                "breach_rate": p_hat, "lr_stat": np.nan, "p_value": np.nan}

    lr = -2 * (
        x * np.log(alpha) + (n - x) * np.log(1 - alpha)
        - x * np.log(p_hat) - (n - x) * np.log(1 - p_hat)
```

```
    )
    p_value = 1 - stats.chi2.cdf(lr, df=1)

    return {
        "breaches": int(x),
        "expected": round(n * alpha, 1),
        "breach_rate": round(p_hat, 4),
        "lr_stat": round(lr, 4),
        "p_value": round(p_value, 4)
    }

# Backtest each method
actual_returns = market_daily["mkt_ret"].iloc[window:].values

backtest_results = {}
for method, col in [("Historical", "var_hs"),
                    ("Gaussian", "var_gauss"),
                    ("Student-t", "var_t")]:
    backtest_results[method] = kupiec_test(
        actual_returns, var_es_ts[col].values, alpha=0.01
    )

bt_df = pd.DataFrame(backtest_results).T
bt_df.index.name = "Method"
bt_df
```

90.2.5 Estimation Under Limited Data

In Vietnamese equity markets, many stocks have short trading histories, thin liquidity, and censored returns due to price limits. These features create challenges for tail risk estimation that are less severe in developed markets.

Price limit censoring. When a stock hits its daily price limit, the observed return is truncated. The true latent return may extend well beyond the limit, but is unobservable. This means that historical simulation mechanically understates VaR and ES for stocks that frequently hit limits. A Tobit-type correction can partially address this:

$$r_{i,t}^{\text{latent}} \sim N(\mu_i, \sigma_i^2), \quad r_{i,t}^{\text{obs}} = \max(\underline{L}, \min(\bar{L}, r_{i,t}^{\text{latent}})) \tag{90.19}$$

The parameters (μ_i, σ_i^2) can be estimated via maximum likelihood on the censored sample, and VaR/ES can then be computed from the estimated latent distribution.

Short histories. For recently listed stocks, the available sample may be too short for reliable nonparametric estimation. In these cases, shrinkage toward a cross-sectional prior (e.g., the sector median VaR) or Bayesian approaches with informative priors can improve stability.

90.3 Extreme Value Theory

90.3.1 Motivation: What EVT Captures That GARCH Does Not

GARCH models capture volatility clustering (i.e., the tendency of large returns to be followed by large returns), but they make specific distributional assumptions about standardized residuals (typically Gaussian or Student-t). The tail behavior of the conditional distribution is thus determined by the parametric innovation assumption, not learned from the data.

Extreme Value Theory (EVT) takes a fundamentally different approach. It provides a statistical framework for estimating the distribution of extreme observations without specifying the entire return distribution. The key theoretical results (i.e., the Fisher-Tippett-Gnedenko theorem and the Pickands-Balkema-de Haan theorem) establish that the distribution of properly normalized maxima (or exceedances over high thresholds) converges to specific parametric families regardless of the parent distribution. This universality is what makes EVT powerful: the tail can be estimated even when the center of the distribution is misspecified.

90.3.2 Tail Index Estimation

The tail index ξ (also denoted α^{-1} in some formulations) governs the rate of tail decay. For heavy-tailed distributions, the survival function satisfies:

$$P(X > x) \sim L(x) \cdot x^{-1/\xi}, \quad x \to \infty \tag{90.20}$$

where $L(x)$ is a slowly varying function. The tail index $\xi > 0$ implies a Pareto-type tail; larger ξ means heavier tails. Financial return distributions typically have $\xi \in (0.2, 0.5)$, corresponding to tail indices $\alpha = 1/\xi \in (2, 5)$, which implies finite variance but potentially infinite higher moments.

The Hill estimator (Hill 1975) provides a simple nonparametric estimate of ξ from the upper order statistics:

$$\hat{\xi}_{\text{Hill}}(k) = \frac{1}{k} \sum_{j=1}^{k} \ln X_{(n-j+1)} - \ln X_{(n-k)} \tag{90.21}$$

where $X_{(1)} \leq \cdots \leq X_{(n)}$ are the order statistics and k is the number of upper order statistics used. The choice of k involves a bias-variance tradeoff: small k yields high variance; large k introduces bias from non-tail observations.

```
def hill_estimator(data, k_values=None):
    """
    Compute Hill estimator of tail index for a range of k values.

    Parameters
    ----------
    data : array-like
        Positive values (use absolute values of losses).
    k_values : array-like, optional
        Number of upper order statistics to use.

    Returns
    -------
    DataFrame with columns [k, xi_hat, se].
    """
    x = np.sort(np.abs(data))[::-1]  # Descending order
    n = len(x)

    if k_values is None:
        k_values = range(10, min(n // 2, 500), 5)

    results = []
    for k in k_values:
        if k >= n or k < 2:
            continue
        log_excess = np.log(x[:k]) - np.log(x[k])
        xi_hat = log_excess.mean()
        se = xi_hat / np.sqrt(k)
        results.append({"k": k, "xi_hat": xi_hat, "se": se})

    return pd.DataFrame(results)
```

The Hill plot is a standard diagnostic in EVT. A stable region of the Hill estimate across a range of k suggests a reliable tail index estimate. The red dashed line at $\xi = 1/3$ corresponds to a tail index of $\alpha = 3$, which implies that the third moment (skewness) is infinite, which is a finding consistent with much of the empirical finance literature (Cont 2001; Gabaix et al. 2003).

```
# Use negative market returns (losses)
losses = -market_daily["mkt_ret"].dropna().values
losses_positive = losses[losses > 0]

hill_results = hill_estimator(losses_positive)

(
    p9.ggplot(hill_results, p9.aes(x="k", y="xi_hat"))
    + p9.geom_ribbon(
        p9.aes(ymin="xi_hat - 1.96 * se", ymax="xi_hat + 1.96 * se"),
        fill="#2E5090", alpha=0.2
    )
    + p9.geom_line(color="#2E5090", size=0.8)
    + p9.geom_hline(yintercept=0.33, linetype="dashed", color="red")
    + p9.annotate(
        "text", x=hill_results["k"].max() * 0.7, y=0.35,
        label="  = 1/3 (cubic tail)", color="red", size=8
    )
    + p9.labs(
        x="Number of Order Statistics (k)",
        y="Hill Estimate ^",
        title="Hill Plot for Market Return Left Tail"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 5))
)
```

Figure 90.5

90.3.3 Block Maxima and the GEV Distribution

The Fisher-Tippett-Gnedenko theorem establishes that if the maximum of n i.i.d. random variables, after proper normalization, converges in distribution, the limit must be a Generalized Extreme Value (GEV) distribution:

$$G_\xi(x) = \exp\left\{-\left(1 + \xi \cdot \frac{x-\mu}{\sigma}\right)^{-1/\xi}\right\} \tag{90.22}$$

for $1 + \xi(x-\mu)/\sigma > 0$, where μ is the location, $\sigma > 0$ the scale, and ξ the shape parameter. The three sub-families are in Table 90.3.

Table 90.3: GEV Sub-Families and Financial Interpretation

Shape ξ	Distribution	Tail Type	Finance Example
$\xi > 0$	Fréchet	Heavy (polynomial decay)	Equity returns, credit losses
$\xi = 0$	Gumbel	Light (exponential decay)	Normal/lognormal models
$\xi < 0$	Weibull	Bounded upper tail	Bounded loss distributions

For the block maxima approach, we divide the return series into non-overlapping blocks (typically months or quarters) and extract the minimum return (maximum loss) from each block. The GEV is then fitted to these block minima.

```
from scipy.stats import genextreme

# Monthly block minima (maximum losses)
market_daily["month"] = market_daily["date"].dt.to_period("M")
block_minima = (
    market_daily.groupby("month")
    .agg(min_ret=("mkt_ret", "min"))
    .reset_index()
)

# Fit GEV to negated block minima (so we model maxima of losses)
block_losses = -block_minima["min_ret"].values
xi_hat, loc_hat, scale_hat = genextreme.fit(block_losses)

print(f"GEV fit to monthly block maxima of losses:")
```

```
print(f"  Shape ( ):    {xi_hat:.4f}")
print(f"  Location ( ): {loc_hat:.4f}")
print(f"  Scale ( ):    {scale_hat:.4f}")
```

90.3.4 Peaks Over Threshold and the GPD

The Peaks Over Threshold (POT) approach is generally preferred over block maxima because it uses tail data more efficiently. The Pickands-Balkema-de Haan theorem establishes that for a sufficiently high threshold u, the distribution of exceedances $Y = X - u \mid X > u$ converges to a Generalized Pareto Distribution (GPD):

$$H_{\xi,\beta}(y) = \begin{cases} 1 - \left(1 + \frac{\xi y}{\beta}\right)^{-1/\xi} & \text{if } \xi \neq 0 \\ 1 - \exp\left(-\frac{y}{\beta}\right) & \text{if } \xi = 0 \end{cases} \tag{90.23}$$

for $y > 0$ and $1 + \xi y/\beta > 0$, where $\beta > 0$ is the scale parameter and ξ is the shape parameter (identical to the GEV shape). The POT-based VaR and ES at level α are:

$$\text{VaR}_\alpha^{\text{GPD}} = u + \frac{\hat{\beta}}{\hat{\xi}} \left[\left(\frac{n}{N_u} \cdot \alpha \right)^{-\hat{\xi}} - 1 \right] \tag{90.24}$$

$$\text{ES}_\alpha^{\text{GPD}} = \frac{\text{VaR}_\alpha^{\text{GPD}}}{1 - \hat{\xi}} + \frac{\hat{\beta} - \hat{\xi} u}{1 - \hat{\xi}} \tag{90.25}$$

where N_u is the number of observations exceeding u and n is the total sample size. These formulae are valid for $\hat{\xi} < 1$.

```
from scipy.stats import genpareto

def fit_gpd_pot(returns, threshold_quantile=0.95):
    """
    Fit GPD to exceedances over threshold using POT approach.

    Parameters
    ----------
    returns : array-like
        Return series (losses should be positive).
    threshold_quantile : float
        Quantile for threshold selection.
```

```
x_grid = np.linspace(
    block_losses.min() * 0.8,
    block_losses.max() * 1.2,
    200
)
gev_pdf = genextreme.pdf(x_grid, xi_hat, loc=loc_hat, scale=scale_hat)

plot_data = pd.DataFrame({
    "x": np.concatenate([block_losses, x_grid]),
    "source": (["Empirical"] * len(block_losses) +
               ["GEV Fit"] * len(x_grid)),
    "density": np.concatenate([
        np.full(len(block_losses), np.nan),
        gev_pdf
    ]),
    "value": np.concatenate([block_losses, np.full(len(x_grid), np.nan)])
})

empirical = plot_data[plot_data["source"] == "Empirical"]
fitted = plot_data[plot_data["source"] == "GEV Fit"]

(
    p9.ggplot()
    + p9.geom_histogram(
        data=empirical, mapping=p9.aes(x="value", y="..density.."),
        bins=30, fill="#2E5090", alpha=0.5
    )
    + p9.geom_line(
        data=fitted, mapping=p9.aes(x="x", y="density"),
        color="#C0392B", size=1
    )
    + p9.labs(
        x="Monthly Maximum Loss",
        y="Density",
        title="GEV Distribution Fit to Monthly Block Maxima"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 5))
)
```

Figure 90.6

```
    Returns
    -------
    dict : Threshold, GPD parameters, VaR, ES estimates.
    """
    losses = -np.array(returns)
    losses = losses[~np.isnan(losses)]
    n = len(losses)

    u = np.quantile(losses, threshold_quantile)
    exceedances = losses[losses > u] - u
    N_u = len(exceedances)

    # Fit GPD
    xi_hat, _, beta_hat = genpareto.fit(exceedances, floc=0)

    # VaR and ES at 1%
    alpha = 0.01
    var_gpd = u + (beta_hat / xi_hat) * (
        (n / N_u * alpha)**(-xi_hat) - 1
    )
    es_gpd = var_gpd / (1 - xi_hat) + (beta_hat - xi_hat * u) / (1 - xi_hat)

    return {
        "threshold": u,
        "n_exceedances": N_u,
        "xi_hat": xi_hat,
        "beta_hat": beta_hat,
        "var_1pct": var_gpd,
        "es_1pct": es_gpd
    }

# Fit GPD to market losses
gpd_results = fit_gpd_pot(market_daily["mkt_ret"].dropna().values, 0.95)

print("GPD-POT Results (1% level):")
for k, v in gpd_results.items():
    if isinstance(v, float):
        print(f"  {k}: {v:.6f}")
    else:
        print(f"  {k}: {v}")
```

90.3.5 Threshold Selection

The choice of threshold u is the key practical decision in POT analysis. Too low a threshold violates the asymptotic approximation; too high wastes data. The mean residual life plot provides a graphical diagnostic: if the GPD approximation holds above u_0, the mean excess function $e(u) = E[X - u \mid X > u]$ is linear in u for $u > u_0$:

$$e(u) = \frac{\beta + \xi u}{1 - \xi} \tag{90.26}$$

A threshold in the region where the mean residual life plot is approximately linear is appropriate. In practice, we also compare GPD parameter stability across a range of thresholds and select the lowest threshold at which estimates stabilize.

90.3.6 What EVT Captures That GARCH Does Not

Table 90.4 summarizes the complementary roles of these two modeling frameworks.

Table 90.4: GARCH vs. EVT: Complementary Frameworks

Feature	GARCH	EVT
What is modeled	Conditional variance dynamics	Unconditional tail distribution
Distributional assumption	Full distribution (Gaussian or t)	Only the tail (GPD or GEV)
Time dependence	Explicit (volatility clustering)	None (i.i.d. or filtered)
Tail shape	Determined by innovation distribution	Estimated from data
Extreme quantiles	Extrapolation from parametric assumption	Principled extrapolation via EVT theorems
Best use	Short-horizon risk forecasting	Extreme quantile estimation, stress testing

The GARCH-filtered EVT approach of McNeil and Frey (2000) combines both: GARCH captures the time-varying volatility, and EVT models the residual tail. This hybrid approach is considered best practice for financial risk management (McNeil, Frey, and Embrechts 2015).

```
losses = -market_daily["mkt_ret"].dropna().values
losses_sorted = np.sort(losses)

thresholds = np.quantile(losses, np.linspace(0.80, 0.99, 50))
mean_excess = []

for u in thresholds:
    exceedances = losses[losses > u] - u
    if len(exceedances) > 5:
        mean_excess.append({
            "threshold": u,
            "mean_excess": exceedances.mean(),
            "se": exceedances.std() / np.sqrt(len(exceedances)),
            "n_exceed": len(exceedances)
        })

mrl_df = pd.DataFrame(mean_excess)

(
    p9.ggplot(mrl_df, p9.aes(x="threshold", y="mean_excess"))
    + p9.geom_ribbon(
        p9.aes(
            ymin="mean_excess - 1.96 * se",
            ymax="mean_excess + 1.96 * se"
        ),
        fill="#2E5090", alpha=0.2
    )
    + p9.geom_line(color="#2E5090", size=0.8)
    + p9.geom_point(color="#2E5090", size=1.5)
    + p9.labs(
        x="Threshold (u)",
        y="Mean Excess e(u)",
        title="Mean Residual Life Plot"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 5))
)
```

Figure 90.7

90.4 Tail Dependence and Contagion

90.4.1 Why Correlation Breaks Down in Crises

Perhaps the most important practical fact about multivariate tail risk is that correlations estimated from the body of the distribution systematically understate dependence in the tails. Longin and Solnik (2001) demonstrate this using extreme value theory: for bivariate normal returns, the correlation of extreme realizations converges to zero as the threshold moves further into the tail. Yet empirically, the correlation of crash returns remains large and often increases (a phenomenon incompatible with multivariate normality).

Formally, the coefficient of lower tail dependence between two return series X and Y is:

$$\lambda_L = \lim_{q \to 0^+} P\left(Y \leq F_Y^{-1}(q) \mid X \leq F_X^{-1}(q)\right) \tag{90.27}$$

If $\lambda_L > 0$, the two series exhibit asymptotic tail dependence (i.e., extreme losses tend to occur jointly). For the bivariate normal distribution, $\lambda_L = 0$ for all $|\rho| < 1$ (asymptotic tail independence). This means that any model built on multivariate normality will structurally underestimate the probability of joint crashes. The Student-t copula, by contrast, has $\lambda_L > 0$ for $\rho > -1$, which is why it has become the workhorse model for joint tail risk in finance (Demarta and McNeil 2005).

90.4.2 Co-Crash Measures

We implement several empirical measures of joint crash behavior.

Exceedance Correlation. Following Longin and Solnik (2001), compute the correlation of returns conditional on both returns falling below a threshold θ:

$$\rho^{-}(\theta) = \text{Corr}(r_i, r_j \mid r_i < \theta, r_j < \theta) \tag{90.28}$$

If returns are bivariate normal, $\rho^{-}(\theta) \to 0$ as $\theta \to -\infty$. Empirically, $\rho^{-}(\theta)$ typically increases as θ decreases, indicating tail dependence beyond what the normal model implies.

Co-Exceedance Count. For each day t, count the number of stocks or sectors whose returns fall below the q-th percentile of their respective marginal distributions:

$$C_t(q) = \sum_{i=1}^{N} \mathbb{1}\left(r_{i,t} < F_i^{-1}(q)\right) \tag{90.29}$$

Days with high $C_t(q)$ represent system-wide stress events.

```
def empirical_tail_dependence(x, y, quantiles=None):
    """
    Estimate lower tail dependence coefficient at various quantiles.

    Uses the empirical copula approach: transform to uniform margins,
    then estimate P(V <= q | U <= q) for decreasing q.
    """
    if quantiles is None:
        quantiles = [0.20, 0.15, 0.10, 0.05, 0.03, 0.01]

    # Transform to uniform margins via empirical CDF
    from scipy.stats import rankdata
    n = len(x)
    u = rankdata(x) / (n + 1)
    v = rankdata(y) / (n + 1)

    results = []
    for q in quantiles:
        mask_u = u <= q
        mask_both = mask_u & (v <= q)

        n_u = mask_u.sum()
        n_both = mask_both.sum()

        lambda_hat = n_both / n_u if n_u > 0 else np.nan

        results.append({
            "quantile": q,
            "lambda_L": lambda_hat,
            "n_below_x": int(n_u),
            "n_co_crash": int(n_both)
        })

    return pd.DataFrame(results)
```

```
# Load sector-level daily returns
sector_returns = dc.get_sector_returns(
    start_date="2010-01-01",
    end_date="2024-12-31",
    frequency="daily"
)
```

```
# Compute pairwise tail dependence for major sectors
sectors = sector_returns["sector"].unique()[:8]  # Top 8 sectors

import itertools

pairwise_td = []
for s1, s2 in itertools.combinations(sectors, 2):
    r1 = sector_returns.loc[
        sector_returns["sector"] == s1, ["date", "ret"]
    ].set_index("date")["ret"]
    r2 = sector_returns.loc[
        sector_returns["sector"] == s2, ["date", "ret"]
    ].set_index("date")["ret"]

    # Align dates
    aligned = pd.concat([r1, r2], axis=1, join="inner").dropna()
    if len(aligned) < 500:
        continue

    td = empirical_tail_dependence(
        aligned.iloc[:, 0].values,
        aligned.iloc[:, 1].values,
        quantiles=[0.05]
    )

    pairwise_td.append({
        "sector_1": s1,
        "sector_2": s2,
        "lambda_L_5pct": td["lambda_L"].iloc[0],
        "pearson_corr": aligned.iloc[:, 0].corr(aligned.iloc[:, 1])
    })

td_df = pd.DataFrame(pairwise_td)
```

The ratio column in Table 90.5 is particularly informative. When $\lambda_L / \rho \gg 1$, the sectors exhibit disproportionately high co-crash frequency relative to their overall correlation. This is the hallmark of tail dependence that Gaussian models miss.

90.4.3 System-Wide Stress Transmission

To assess system-wide contagion dynamics, we construct daily co-exceedance counts and examine their time-series properties.

Table 90.5: Pairwise Tail Dependence vs. Linear Correlation Across Sectors

```
td_display = td_df.copy()
td_display["lambda_L_5pct"] = td_display["lambda_L_5pct"].round(4)
td_display["pearson_corr"] = td_display["pearson_corr"].round(4)
td_display["ratio"] = (
    td_display["lambda_L_5pct"] / td_display["pearson_corr"]
).round(3)

td_display.sort_values("lambda_L_5pct", ascending=False).head(15)
```

```
# Compute daily co-exceedance count at the 5% level
sector_wide = sector_returns.pivot_table(
    index="date", columns="sector", values="ret"
)

# Rolling 252-day quantile thresholds
quantile_thresholds = sector_wide.rolling(252, min_periods=60).quantile(0.05)

# Indicator: return below rolling 5th percentile
exceedance_indicators = (sector_wide < quantile_thresholds).astype(int)
co_exceedance = exceedance_indicators.sum(axis=1)
co_exceedance.name = "co_exceedance"

co_exceed_df = co_exceedance.reset_index()
co_exceed_df.columns = ["date", "co_exceedance"]
co_exceed_df["n_sectors"] = exceedance_indicators.notna().sum(axis=1).values
```

Peaks in the co-exceedance series correspond to system-wide stress episodes. The smoothed trend (red line) reveals the evolving baseline level of systemic fragility.

90.4.4 Financial Contagion Mechanisms

The literature distinguishes several channels through which stress propagates across assets, sectors, and markets (Forbes and Rigobon 2002; Kaminsky, Reinhart, and Vegh 2002):

Direct linkages. Balance sheet interconnections, interbank lending, and cross-holdings create direct transmission channels. When institution A holds assets issued by institution B, losses at B directly impair A's balance sheet.

```
(
    p9.ggplot(co_exceed_df.dropna(), p9.aes(x="date", y="co_exceedance"))
    + p9.geom_line(color="#2E5090", alpha=0.5, size=0.3)
    + p9.geom_smooth(method="lowess", color="#C0392B", size=1, se=False)
    + p9.labs(
        x="",
        y="Number of Sectors in Left Tail",
        title="System-Wide Stress: Daily Co-Exceedance Count"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 5))
)
```

Figure 90.8

Information contagion. Adverse news about one firm triggers reassessment of similarly exposed firms. King and Wadhwani (1990) and Kodres and Pritsker (2002) formalize this via Bayesian updating: investors cannot distinguish idiosyncratic from systematic shocks, so a crash in one asset leads them to update beliefs about common risk factors.

Liquidity contagion. Fire sales by distressed institutions depress prices of commonly held assets, creating losses for other holders. Markus K. Brunnermeier (2009) model this as a liquidity spiral: declining prices trigger margin calls, which force further sales, further depressing prices. In Vietnamese markets, the absence of circuit breakers beyond price limits and the concentration of holdings among a few large institutional investors amplify this channel.

Behavioral contagion. Herding behavior (i.e., investors mimicking others' trades) generates correlated selling pressure even in the absence of fundamental linkages. Bikhchandani, Hirshleifer, and Welch (1992) and Banerjee (1992) provide theoretical foundations. Choe, Kho, and Stulz (2005) document herding in emerging markets specifically.

We can test for contagion (as distinct from interdependence) using the Forbes and Rigobon (2002) framework. The null hypothesis is that the co-movement observed during a crisis period is fully explained by the increase in volatility (which mechanically inflates correlations). The adjusted correlation is:

$$\rho^* = \frac{\rho_{\text{crisis}}}{\sqrt{1 + \delta[1 - \rho_{\text{crisis}}^2]}} \tag{90.30}$$

where $\delta = \sigma^2_{\text{crisis}}/\sigma^2_{\text{tranquil}} - 1$ is the relative increase in variance. If $\rho^* > \rho_{\text{tranquil}}$, the increase in co-movement exceeds what volatility adjustment alone predicts, which is evidence of contagion.

```
def forbes_rigobon_test(returns_df, crisis_start, crisis_end):
    """
    Forbes-Rigobon adjusted correlation test for contagion.

    Parameters
    ----------
    returns_df : DataFrame
        Columns are sector returns, index is date.
    crisis_start, crisis_end : str
        Crisis period boundaries.

    Returns
    -------
    DataFrame : Pairwise contagion test results.
    """
    tranquil = returns_df[returns_df.index < crisis_start].dropna()
    crisis = returns_df[
        (returns_df.index >= crisis_start) &
        (returns_df.index <= crisis_end)
    ].dropna()

    sectors = returns_df.columns.tolist()
    results = []

    for s1, s2 in itertools.combinations(sectors, 2):
        if s1 not in tranquil.columns or s2 not in crisis.columns:
            continue

        rho_t = tranquil[s1].corr(tranquil[s2])
        rho_c = crisis[s1].corr(crisis[s2])

        var_t = tranquil[s1].var()
        var_c = crisis[s1].var()

        if var_t == 0:
            continue

        delta = var_c / var_t - 1
        rho_adj = rho_c / np.sqrt(1 + delta * (1 - rho_c**2))

        results.append({
            "sector_1": s1,
            "sector_2": s2,
```

Table 90.6: Forbes-Rigobon Contagion Test: COVID-19 Crisis Period

```
contagion_covid.sort_values(
    "rho_adjusted", ascending=False
).head(15)
```

```
            "rho_tranquil": round(rho_t, 4),
            "rho_crisis": round(rho_c, 4),
            "rho_adjusted": round(rho_adj, 4),
            "contagion": rho_adj > rho_t
        })

    return pd.DataFrame(results)
```

```
# Test for contagion during COVID-19 crash
contagion_covid = forbes_rigobon_test(
    sector_wide,
    crisis_start="2020-02-01",
    crisis_end="2020-04-30"
)

n_pairs = len(contagion_covid)
n_contagion = contagion_covid["contagion"].sum()
print(f"COVID-19 contagion test: {n_contagion}/{n_pairs} sector pairs "
      f"show evidence of contagion ({100*n_contagion/n_pairs:.1f}%)")
```

90.5 Applications to Financial Stability

90.5.1 Market-Wide Stress Indicators

We construct several market-wide stress indicators that aggregate the individual-level and sectoral tail risk measures developed above into a comprehensive system-level diagnostic.

Aggregate Crash Risk Index. The cross-sectional average of firm-level NCSKEW provides a market-wide measure of crash risk that is forward-looking (it captures the buildup of negative skewness before a crash materializes):

$$\text{ACRI}_y = \frac{1}{N_y} \sum_{i=1}^{N_y} \text{NCSKEW}_{i,y} \tag{90.31}$$

Market Tail Risk (MTR). Following Kelly and Jiang (2014), we estimate the time-varying tail risk of the market portfolio using option-implied or realized tail measures. In the absence of a liquid options market in Vietnam, we use the realized version based on the Hill tail index estimated from rolling windows:

$$\text{MTR}_t = \hat{\xi}_t^{\text{Hill}} \cdot \hat{\sigma}_t \tag{90.32}$$

where $\hat{\xi}_t^{\text{Hill}}$ is the Hill estimator computed from a trailing 252-day window and $\hat{\sigma}_t$ is the conditional volatility (e.g., from GARCH). This interaction captures both the thickness of the tail and the current scale of volatility.

```
# Aggregate Crash Risk Index
acri = (
    crash_risk.groupby("year")
    .agg(
        acri=("ncskew", "mean"),
        median_ncskew=("ncskew", "median"),
        pct_high_crash=("ncskew", lambda x: (x > x.quantile(0.75)).mean()),
        n_firms=("ncskew", "count")
    )
    .reset_index()
)

# Market Tail Risk: rolling Hill estimator x rolling volatility
mkt_returns = market_daily["mkt_ret"].dropna().values
dates = market_daily["date"].dropna().values

mtr_list = []
for end_idx in range(504, len(mkt_returns)):
    window_ret = mkt_returns[end_idx - 252:end_idx]
    losses_pos = -window_ret[window_ret < 0]

    if len(losses_pos) < 30:
        continue

    # Hill estimator at k = 50
    sorted_losses = np.sort(losses_pos)[::-1]
    k = min(50, len(sorted_losses) - 1)
```

```
    if k < 10:
        continue
    log_excess = np.log(sorted_losses[:k]) - np.log(sorted_losses[k])
    xi_hat = log_excess.mean()

    sigma_hat = window_ret.std()
    mtr = xi_hat * sigma_hat

    mtr_list.append({
        "date": dates[end_idx],
        "xi_hat": xi_hat,
        "sigma_hat": sigma_hat,
        "mtr": mtr
    })

mtr_df = pd.DataFrame(mtr_list)
mtr_df["date"] = pd.to_datetime(mtr_df["date"])
```

```
(
    p9.ggplot(mtr_df, p9.aes(x="date", y="mtr"))
    + p9.geom_line(color="#2E5090", alpha=0.5, size=0.4)
    + p9.geom_smooth(method="lowess", color="#C0392B", size=1, se=False)
    + p9.labs(
        x="",
        y="MTR ( × )",
        title="Market Tail Risk: Rolling Hill Index × Conditional Volatility"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 5))
)
```

Figure 90.9

90.5.2 Sectoral Fragility

Not all sectors are equally vulnerable to tail events. We decompose system-wide tail risk into sectoral contributions to identify the most fragile parts of the market.

```
# Sector-level tail statistics
sector_tail_stats = []

for sector in sectors:
    sector_data = sector_returns[sector_returns["sector"] ==
 ↪  sector]["ret"].dropna()

    if len(sector_data) < 500:
        continue

    s_data = sector_data.values

    # Skewness and kurtosis
    skew = stats.skew(s_data)
    kurt = stats.kurtosis(s_data)

    # VaR and ES at 1%
    var_1 = -np.quantile(s_data, 0.01)
    es_1 = -s_data[s_data <= np.quantile(s_data, 0.01)].mean()

    # Hill tail index (k=50)
    losses_pos = -s_data[s_data < 0]
    sorted_l = np.sort(losses_pos)[::-1]
    k = min(50, len(sorted_l) - 1)
    if k >= 10:
        log_exc = np.log(sorted_l[:k]) - np.log(sorted_l[k])
        xi = log_exc.mean()
    else:
        xi = np.nan

    sector_tail_stats.append({
        "sector": sector,
        "skewness": round(skew, 3),
        "excess_kurtosis": round(kurt, 3),
        "var_1pct": round(var_1, 4),
        "es_1pct": round(es_1, 4),
        "tail_index_xi": round(xi, 3) if not np.isnan(xi) else np.nan,
        "n_obs": len(s_data)
    })

fragility_df = pd.DataFrame(sector_tail_stats)
```

Table 90.7: Sectoral Tail Risk Profile

```
fragility_df.sort_values("es_1pct", ascending=False)
```

```
(
    p9.ggplot(
        fragility_df.dropna(),
        p9.aes(x="tail_index_xi", y="es_1pct", label="sector")
    )
    + p9.geom_point(color="#2E5090", size=3)
    + p9.geom_text(nudge_y=0.002, size=7)
    + p9.labs(
        x="Tail Index ( , Hill)",
        y="1% Expected Shortfall",
        title="Sector Fragility Map"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 7))
)
```

Figure 90.10

Sectors in the upper-right quadrant of Figure 90.10 exhibit both high expected shortfall (large average losses in the tail) and high tail index (heavier-than-typical tails). These represent the most fragile components of the market from a systemic stability perspective.

90.5.3 Crisis Diagnostics

We construct a comprehensive crisis diagnostic framework that combines the indicators developed above to identify, characterize, and compare stress episodes in Vietnamese financial markets.

```
# Identify extreme co-exceedance days (system-wide stress events)
threshold_high_stress = co_exceed_df["co_exceedance"].quantile(0.99)

stress_events = co_exceed_df[
    co_exceed_df["co_exceedance"] >= threshold_high_stress
].copy()

# Add market return on stress days
stress_events = stress_events.merge(
    market_daily[["date", "mkt_ret"]], on="date", how="left"
)

print(f"System-wide stress days (top 1%): {len(stress_events)}")
print(f"Average market return on stress days: "
      f"{stress_events['mkt_ret'].mean():.4f}")
print(f"Average co-exceedance on stress days: "
      f"{stress_events['co_exceedance'].mean():.1f}")
```

90.5.4 Anatomy of a Run: Who Exits First and What It Costs

The diagnostics above identify *when* the system is under stress from prices alone. They say nothing about *who* moves first, how fast the exit propagates, and who is left holding the losses. Those questions are usually unanswerable because bank deposit records are confidential. J. Liu, Makarov, and Schoar (2026) answer them for one of the largest runs of the past decade by exploiting a setting where every transaction is public: the collapse of the Terra ecosystem in May 2022, which erased roughly $50 billion of value in three days. Terra's algorithmic stablecoin, UST, was convertible one-for-one into $1 of the network's native token, LUNA, so in economic terms UST was perpetual convertible debt backed by LUNA. Demand for UST came almost entirely from Anchor, a lending protocol that paid depositors a fixed 19.5% yield when comparable stablecoin rates elsewhere were 2 to 3%. The authors reconstruct the full

Table 90.8: Major Stress Episodes: Clustering of System-Wide Tail Events

```
# Cluster stress days into episodes (within 10 trading days)
stress_events = stress_events.sort_values("date").reset_index(drop=True)
stress_events["gap"] = stress_events["date"].diff().dt.days
stress_events["episode"] = (stress_events["gap"] > 15).cumsum()

episode_summary = (
    stress_events.groupby("episode")
    .agg(
        start_date=("date", "min"),
        end_date=("date", "max"),
        n_stress_days=("date", "count"),
        avg_market_return=("mkt_ret", "mean"),
        min_market_return=("mkt_ret", "min"),
        avg_coexceedance=("co_exceedance", "mean")
    )
    .reset_index(drop=True)
    .sort_values("min_market_return")
)

episode_summary["avg_market_return"] =
 ↪  episode_summary["avg_market_return"].round(4)
episode_summary["min_market_return"] =
 ↪  episode_summary["min_market_return"].round(4)
episode_summary["avg_coexceedance"] =
 ↪  episode_summary["avg_coexceedance"].round(1)

episode_summary.head(10)
```

transaction history of 3.7 million addresses across the Terra blockchain, Ethereum bridges, and centralized exchanges, and their account of the episode reads like a field guide to bank runs.

Four findings are worth carrying into any study of Vietnamese financial stability.

1. **Fragility was built before the trigger, and it was visible.** Anchor's yield was never covered by borrowing income; a yield reserve topped up by the sponsor covered the gap, and the reserve was draining from December 2021. UST supply rose from 2.5 billion to 18.5 billion in six months, most of it parked in Anchor, while the market value of LUNA that backed it was falling. Depositor entry rates dropped and exit rates rose from April 2022, weeks before the crash. This is the sunspot-plus-fundamentals structure of the global-games literature: weak fundamentals turn an innocuous public signal into a coordination device. The signals that ended up coordinating the run were a scheduled rebalancing of a liquidity pool and the moment LUNA's market capitalization fell to the level of UST outstanding, a threshold that meant nothing to a fully rational investor but that many participants treated as proof the backing was gone.

2. **The run was not manipulation but a cascade started by a few large depositors.** Two wallets withdrew 400 million UST from Anchor on 7 May 2022 and sold on the deepest venues. Other large wallets followed within hours because they could watch the withdrawals on-chain. Anchor deposits fell from 14 billion to under 2 billion UST by 13 May, and the swap mechanism that was supposed to defend the peg instead drove LUNA supply from under 1 billion to 5.89 trillion as investors converted UST to LUNA and sold, the death spiral the design implied.

3. **Transparency did not level the playing field.** Addresses with more than 10 million UST withdrew almost everything within two days and lost 26% on average. Addresses below 1,000 UST kept adding deposits until 10 May, trying to buy the dip, and lost 76%. Controlling for size, addresses that had used cross-chain bridges or traded on several exchanges lost significantly less, so sophistication mattered independently of wealth. Everyone could see the same ledger; only some could act on it in time.

4. **A lender of last resort works only while the liability is small.** UST had de-pegged once before, in May 2021, and recovered because the sponsor and an allied trading firm absorbed the selling when UST outstanding was a few hundred million. In May 2022 the same actors spent about $2.5 billion defending the peg against 18 billion of claims and failed. The intervention that restores confidence at one scale is a rounding error at the next.

The Vietnamese reading is direct. The deposit run on Saigon Commercial Bank (SCB) in October 2022, which forced the State Bank of Vietnam to place the bank under special control within days of the Vạn Thịnh Phát arrests, and the wave of redemptions from corporate bond funds in the same quarter after the Tân Hoàng Minh and Vạn Thịnh Phát bond cases, share the Terra anatomy: a liability that grew faster than its backing, a yield that only made sense if the sponsor kept paying, a public event that coordinated withdrawals, and a retail base that

moved last. Vietnam also has one of the highest rates of retail crypto participation in the world, so a share of Vietnamese households held UST directly and sat on the wrong side of the loss table above. Two research designs follow. First, the crisis-episode table in this section can be extended from prices to *flows*: daily foreign-investor net sales by ticker, fund NAV redemptions, and, for the 2022 bond episode, the timing of retail versus institutional bondholder exits, to test whether the large-first, small-last ordering holds in Vietnamese data. Second, the co-exceedance measures above are early-warning signals for prices; J. Liu, Makarov, and Schoar (2026) suggest complementing them with *stock* measures of fragility, the ratio of short-term claims to liquid backing at each intermediary, because that ratio is what determined whether the same intervention succeeded in 2021 and failed in 2022.

90.5.5 Self-Fulfilling Fire Sales

The Terra episode was a run on a liability: depositors withdrew because they expected others to withdraw. Khorrami and Mendo (2026) show that the asset side of a balance sheet can fail in the same self-fulfilling way, and that this fragility survives the remedy usually thought to remove it, namely complete markets for hedging fundamental risk.

Their starting point is the financial-accelerator model of Markus K. Brunnermeier and Sannikov (2014), in the tradition of Kiyotaki and Moore (1997). Productive "experts" earn a_e per unit of capital and less productive "households" earn $a_h < a_e$, so capital is worth more in experts' hands, the same logic by which Shleifer and Vishny (1992) explain why assets sold to outsiders fetch less. Experts cannot issue equity, so they finance capital with their own net worth and short-term debt. In the conventional version, a bad fundamental shock erodes expert net worth, experts sell capital to households, the price of capital falls because the buyers value it less, and the falling price erodes expert net worth further. The standard view is that this accelerator needs unhedged fundamental risk: if experts could insure their balance sheets against the shock, there would be nothing to amplify.

Khorrami and Mendo let agents hedge every fundamental shock frictionlessly. Fundamental shocks then never cause fire sales, and there is an efficient equilibrium in which experts hold all capital and its price evolves deterministically. But some risk is always left unhedged: shocks that are novel, or that hit balance sheets no market lets investors short. Call such a shock a *sunspot*. If agents expect experts to sell capital when it arrives, the expectation fulfils itself:

1. experts sell, so capital moves to less productive owners and its price falls;
2. the sunspot is not hedgeable, so the price decline falls disproportionately on experts, whose share of aggregate wealth shrinks;
3. undercapitalized experts sell more, which justifies the initial expectation.

Two equations make the loop concrete. Let η be the experts' share of aggregate wealth, ψ their share of the capital stock, and $\bar{\rho}(\eta) = \eta\rho_e + (1-\eta)\rho_h$ the wealth-weighted discount rate. Goods-market clearing ties the price of capital to who holds it,

$$q\,\bar{\rho}(\eta) = \psi\,a_e + (1-\psi)\,a_h, \tag{90.33}$$

so the price is high only when experts hold the capital. In the region where they do not hold all of it ($\psi < 1$), the equilibrium price function satisfies

$$q'(\eta) = \frac{(a_e - a_h)\,q}{q\,\bar{\rho}(\eta) - \eta\,a_e - (1-\eta)\,a_h}, \qquad \sigma_q^2 = \frac{\eta(1-\eta)}{\psi - \eta} \cdot \frac{a_e - a_h}{q}, \tag{90.34}$$

where σ_q is the volatility of the capital price driven *entirely* by the sunspot. The second expression is the condition that experts' extra productivity, $(a_e - a_h)/q$, exactly compensates them for bearing more of the unhedgeable price risk than households do. Once experts are rich enough that ψ reaches one, the economy behaves like the safe equilibrium and sunspot volatility is zero.

The paper establishes three results that matter beyond the model.

The size of a fire sale is a belief. The differential equation in Equation 90.34 needs a boundary condition: the share of capital experts are believed to keep when their wealth approaches zero, ψ_0. Every $\psi_0 \in [0, 1)$ supports an equilibrium, from the full fire sale ($\psi_0 = 0$, experts dump everything when broke) to the safe equilibrium ($\psi_0 \to 1$). Greater optimism about how much capital experts will retain in the tail produces smaller fire sales, lower volatility, and higher prices, which justifies the optimism.

Self-fulfilling and fundamental fire sales are observationally equivalent. If fundamental risk is unhedgeable with volatility σ, the conventional accelerator equilibrium converges, as $\sigma \to 0$, to the full fire-sale sunspot equilibrium. This formalizes the "volatility paradox" of Markus K. Brunnermeier and Sannikov (2014): endogenous volatility does not vanish as exogenous risk does. The consequence for empirical work is uncomfortable. Data generated by an economy that amplifies fundamental shocks look the same as data generated by one that coordinates on non-fundamental ones.

The worst equilibrium is selected. If agents perceive the sunspot as having a vanishingly small fundamental effect, and experts face a leverage constraint that is then relaxed, only the full fire-sale equilibrium survives. With jump rather than Brownian shocks, the selected equilibrium features large, discrete fire sales and markedly worse misallocation.

Two further implications distinguish this from a bank run. The fire sale needs no maturity mismatch and no illiquid asset: even fundamentally safe assets can be fire-sold if coordinated selling can depress their price. And because debt is always repaid, deposit insurance has no role. The remedy the authors point to is a credible promise to buy assets in the disaster state, which would raise ψ_0 and remove the worst equilibria, though whether such promises create ex ante moral hazard is left open.

The equilibrium is easy to compute. The code below solves Equation 90.34 for three disaster beliefs, using the parameters of the paper's numerical example, and compares the result with the safe equilibrium in which $q = a_e/\bar{\rho}(\eta)$.

```
from scipy.integrate import solve_ivp

RHO_E, RHO_H = 0.06, 0.04   # discount rates: experts, households
A_E, A_H = 0.11, 0.03       # productivity: experts, households


def rho_bar(eta):
    return eta * RHO_E + (1 - eta) * RHO_H


def capital_share(q, eta):
    # Price-output relation solved for experts' capital share psi
    return (q * rho_bar(eta) - A_H) / (A_E - A_H)


def solve_fire_sale(psi0, eta_max=0.6, n_grid=600):
    """Sunspot equilibrium with disaster belief psi0 = psi(0+)."""
    def rhs(eta, y):
        q = y[0]
        return [(A_E - A_H) * q
                / (q * rho_bar(eta) - eta * A_E - (1 - eta) * A_H)]

    if psi0 > 0:
        eta_0 = 1e-9
        q_0 = (psi0 * A_E + (1 - psi0) * A_H) / rho_bar(eta_0)
    else:
        # psi0 = 0 is a singular boundary: near it, psi - eta ~ sqrt(2 c eta)
        eta_0 = 1e-8
        c = A_H / (A_E - A_H)
        q_0 = ((eta_0 + np.sqrt(2 * c * eta_0)) * (A_E - A_H) + A_H) \
            / rho_bar(eta_0)

    def full_allocation(eta, y):        # stop where experts hold everything
        return capital_share(y[0], eta) - 1
    full_allocation.terminal = True
    full_allocation.direction = 1

    sol = solve_ivp(rhs, (eta_0, eta_max), [q_0], events=full_allocation,
                    dense_output=True, rtol=1e-10, atol=1e-12,
↪   method="LSODA")
```

```
    eta_star = sol.t_events[0][0] if sol.t_events[0].size else eta_max

    eta = np.linspace(eta_0, eta_max, n_grid)
    fire_sale_region = eta < eta_star
    q = np.where(fire_sale_region,
                 sol.sol(np.minimum(eta, eta_star))[0],
                 A_E / rho_bar(eta))
    psi = np.minimum(capital_share(q, eta), 1.0)
    sigma_q = np.zeros_like(eta)
    region = fire_sale_region & (psi > eta)
    sigma_q[region] = np.sqrt(eta[region] * (1 - eta[region]) * (A_E - A_H)
                              / ((psi[region] - eta[region]) * q[region]))
    return pd.DataFrame({"eta": eta, "q": q, "sigma_q": sigma_q}), eta_star


frames = []
for psi0 in [0.0, 0.25, 0.5]:
    frame, eta_star = solve_fire_sale(psi0)
    frames.append(frame.assign(equilibrium=f"Disaster belief   = {psi0:g}"))
    print(f"psi0 = {psi0:g}: fire-sale region eta < {eta_star:.3f}, "
          f"peak sunspot volatility {frame['sigma_q'].max():.1%}")

eta_grid = frames[0]["eta"]
frames.append(pd.DataFrame({"eta": eta_grid, "q": A_E / rho_bar(eta_grid),
                            "sigma_q": 0.0,
                            "equilibrium": "Safe (fundamental)
                             ↪  equilibrium"}))
fire_sales = pd.concat(frames, ignore_index=True)
```

```
psi0 = 0: fire-sale region eta < 0.319, peak sunspot volatility 12.4%
psi0 = 0.25: fire-sale region eta < 0.291, peak sunspot volatility 10.5%
psi0 = 0.5: fire-sale region eta < 0.218, peak sunspot volatility 8.4%
```

```
plot_df = fire_sales.melt(
    id_vars=["eta", "equilibrium"], value_vars=["q", "sigma_q"],
    var_name="series", value_name="value"
)
plot_df["series"] = plot_df["series"].map({
    "q": "Price of capital q",
    "sigma_q": "Sunspot volatility  _q"
})
```

```
(
    p9.ggplot(plot_df, p9.aes(x="eta", y="value", color="equilibrium",
                              linetype="equilibrium"))
    + p9.geom_line(size=0.9)
    + p9.facet_wrap("~series", scales="free_y", ncol=2)
    + p9.scale_color_manual(values=["#2E5090", "#5B8BD0", "#9DB7E3",
                                    "#C0392B"])
    + p9.scale_linetype_manual(values=["solid", "solid", "solid", "dashed"])
    + p9.labs(x="Experts' wealth share  ", y="", color="", linetype="")
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 4.5), legend_position="bottom")
)
```

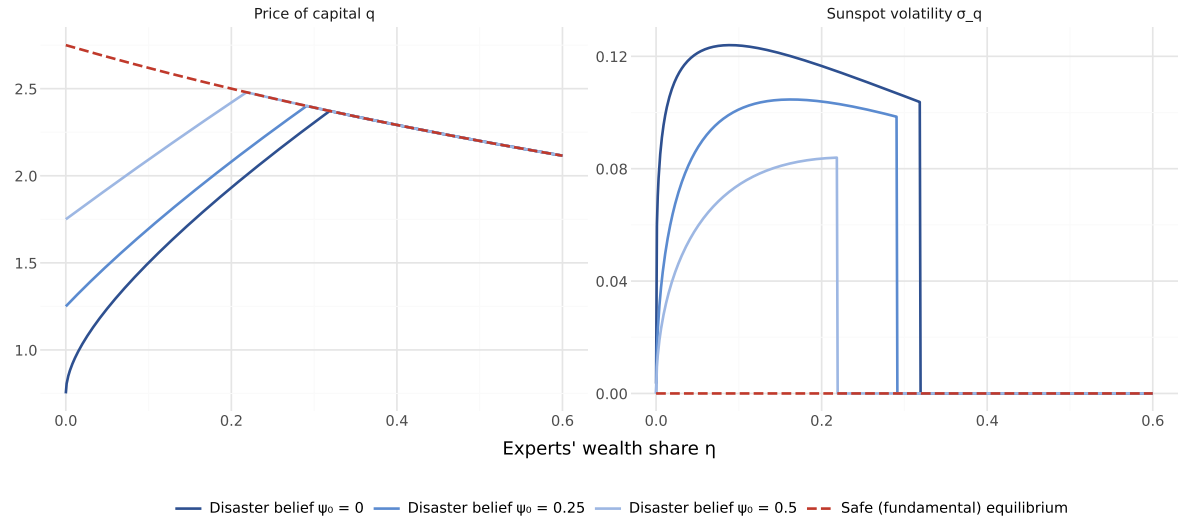


Figure 90.11: Self-fulfilling fire sales: the price of capital and its sunspot-driven volatility as functions of experts' wealth share, for three disaster beliefs and the safe equilibrium. Parameters follow the numerical example in Khorrami and Mendo.

Three features of Figure 90.11 carry the economics. Below a threshold in experts' wealth share, about one-third under the full fire-sale belief, households hold some of the capital, its price lies below the safe-equilibrium price, and it is volatile even though no fundamental risk is borne by anyone. Volatility is zero when experts are rich enough to absorb the sunspot, jumps up as soon as their wealth share falls below the threshold, and falls back toward zero only when experts are so poor that little capital remains to be sold. And a more optimistic disaster belief shrinks the fire-sale region and lowers volatility throughout, which is the channel through which a credible asset-purchase backstop would work.

The Vietnamese reading starts from the hedging assumption, which is far from local reality. Vietnam has no single-stock futures or options, short selling is effectively unavailable (Table 4.2), and VN30 index futures and covered warrants are the main instruments for transferring equity risk. Most shocks to the balance sheets of leveraged Vietnamese investors are unhedgeable, so the conventional accelerator applies directly. The model's sharper message is that building out hedging markets would not remove fire-sale fragility. It would move it to the shocks for which no market exists yet, such as the 2022 corporate bond crisis, which hit a market with no instrument for hedging issuer or sector credit risk.

The roles in the model have recognizable counterparts. The "experts" are leveraged holders who value the asset most: margin-financed investors and founders who pledge their own shares as collateral. The "households" are unlevered retail buyers who absorb forced sales at lower valuations. Forced liquidations by securities companies when collateral values fall are the mechanism by which the expert capital share falls with expert wealth. Price limits change the form the fire sale takes. A coordinated sale that drives a stock to its floor, where it locks with an unfilled queue of sell orders, is a discrete jump rather than a smooth decline, which makes the paper's jump-shock variant the relevant one and links this section to the limit-hit dynamics in Section 85.7.

Four research designs follow.

1. **Measure the state variable.** The model's state is the wealth share of leveraged holders. Aggregate margin-loan balances, reported in securities companies' quarterly financial statements, scaled by market capitalization are a natural proxy, at the market level and, where broker-level data allow, by stock.
2. **Test the nonlinearity.** The model predicts that price volatility is zero above a threshold in that state variable, jumps when the threshold is crossed, and declines again only at very low expert wealth. A linear regression of realized volatility on margin debt cannot detect this; a nonparametric or threshold specification can.
3. **Do not read amplification as fundamental.** Observational equivalence means a large price response to modest news is not, by itself, evidence of a large fundamental shock or of the strength of a fundamental accelerator. Designs that compare episodes with similar fundamental news and different leverage states are more informative than estimates of a single amplification coefficient.
4. **Evaluate backstops by their effect on tail beliefs.** The 2023 measures that allowed corporate bond payment extensions and the State Bank's support operations can be read as attempts to raise ψ_0. In the model, a credible backstop lowers volatility and raises prices *before* it is ever used. Event studies around policy announcements, focusing on stocks with high margin or pledged-share exposure, can test whether markets treated the promises as credible.

90.6 Summary

This chapter developed a comprehensive toolkit for measuring and diagnosing tail risk in Vietnamese equity markets. The key objects estimated (crash risk measures, VaR and Expected Shortfall, tail indices, and tail dependence coefficients) capture distinct but complementary aspects of extreme event behavior.

At the individual stock level, the NCSKEW, DUVOL, and crash count measures quantify asymmetry in firm-specific return distributions, providing forward-looking indicators of crash vulnerability. At the portfolio level, EVT-based approaches to VaR and ES estimation avoid the parametric misspecification that plagues Gaussian and even Student-t models in the deep tails. At the system level, tail dependence and co-exceedance measures reveal the extent to which crashes propagate across sectors, and the Forbes-Rigobon contagion test distinguishes genuine contagion from the mechanical increase in co-movement driven by higher volatility.

Several features of Vietnamese markets deserve emphasis. Price limits censor the observed return distribution, causing systematic underestimation of true tail risk. The concentration of trading among retail investors amplifies herding-driven crash dynamics. State ownership of large firms creates correlated exposure to policy shocks that is not captured by standard diversification. And the absence of deep derivatives markets limits the hedging instruments available for tail risk management, making accurate measurement all the more critical.

91 Corporate Finance Estimators and Identification

Corporate finance is the study of how firms make investment, financing, and payout decisions under real-world frictions (e.g., taxes, asymmetric information, agency conflicts, transaction costs, and financial constraints). Unlike asset pricing, where the primary objects of interest are expected returns and risk premia estimated from market data, corporate finance estimators are tied to firm-level accounting and governance data, and their economic interpretation depends critically on the institutional environment in which the firm operates.

This chapter develops the core econometric toolkit for empirical corporate finance and applies it to Vietnamese listed firms. The estimators we cover (including investment-Q regressions, cash flow sensitivity tests, capital structure determinants, payout smoothing models, and agency cost proxies) form the backbone of the modern corporate finance literature. Each estimator embeds specific theoretical assumptions, and each has been the subject of substantial methodological debate. We pay careful attention to identification challenges: the conditions under which a regression coefficient admits a causal or structural interpretation versus merely a descriptive association.

Vietnamese firms present distinctive features that interact with these estimators in economically meaningful ways. State ownership remains pervasive and creates agency problems qualitatively different from the dispersed-ownership setting of the Anglo-American literature. Concentrated family ownership, pyramidal structures, and cross-holdings generate tunneling incentives documented by S. Johnson et al. (2000) and Claessens et al. (2002). The banking system is dominated by state-owned commercial banks whose lending decisions may reflect political rather than purely economic criteria, complicating the interpretation of financing constraint measures. And dividend policy is shaped by regulatory requirements, including minimum payout ratios for state-owned enterprises, that have no parallel in more developed markets.

```
import pandas as pd
import numpy as np
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
from linearmodels.panel import PanelOLS, PooledOLS
import plotnine as p9
from mizani.formatters import percent_format, comma_format
```

```
import warnings
warnings.filterwarnings("ignore")
```

```
# DataCore.vn API
from datacore import DataCore
dc = DataCore()

# Load annual firm-level financial data
firm_annual = dc.get_firm_financials(
    start_date="2008-01-01",
    end_date="2024-12-31",
    frequency="annual"
)

# Load ownership data
ownership = dc.get_ownership_data(
    start_date="2008-01-01",
    end_date="2024-12-31"
)

# Load stock returns (monthly)
monthly_returns = dc.get_monthly_returns(
    start_date="2008-01-01",
    end_date="2024-12-31"
)

# Load market and factor returns
factors = dc.get_factor_returns(
    start_date="2008-01-01",
    end_date="2024-12-31"
)

print(f"Firm-year observations: {len(firm_annual)}")
print(f"Unique firms: {firm_annual['ticker'].nunique()}")
print(f"Year range: {firm_annual['year'].min()}-{firm_annual['year'].max()}")
```

91.1 Investment-Q Regressions

91.1.1 Tobin's Q: Intuition and Theory

The investment-Q framework is the canonical structural model of corporate investment. The core insight, formalized by Hayashi (1982), is elegant: under perfect capital markets and constant returns to scale in the production and adjustment cost technologies, a firm's investment rate should be a sufficient statistic of a single observable (i.e., the ratio of the market value of installed capital to its replacement cost).

Let V_t denote the market value of the firm's assets at time t and K_t the replacement cost of its capital stock. Tobin's Q is:

$$Q_t = \frac{V_t}{K_t} \tag{91.1}$$

When $Q > 1$, the market values a unit of installed capital above its replacement cost, signaling that the firm should invest. When $Q < 1$, the firm should disinvest. In the frictionless Hayashi (1982) environment, the marginal Q (the shadow value of an additional unit of capital) equals the average Q (the ratio of total market value to total replacement cost), and the optimal investment policy is:

$$\frac{I_{i,t}}{K_{i,t-1}} = \frac{1}{\alpha}\left(Q_{i,t} - 1\right) \tag{91.2}$$

where α governs the convexity of adjustment costs. The empirical counterpart is the regression:

$$\frac{I_{i,t}}{K_{i,t-1}} = \beta_0 + \beta_1 Q_{i,t} + \varepsilon_{i,t} \tag{91.3}$$

Under the structural interpretation, $\beta_1 = 1/\alpha > 0$ and Q is the sole explanatory variable. Any additional variable that enters significantly implies a violation of the underlying assumptions (e.g., financial frictions, agency problems, measurement error in Q, or departures from constant returns to scale).

91.1.2 Measurement Issues

The theoretical object is marginal Q (i.e., the value of the next dollar of investment) which is unobservable. The empirical proxy is average Q, typically constructed as:

$$Q_{i,t}^{\text{avg}} = \frac{\text{Market Value of Equity} + \text{Book Value of Debt}}{\text{Book Value of Total Assets}} \tag{91.4}$$

This proxy introduces several problems that are well-documented in the literature.

Problem 1: Marginal $\neq$ Average. The equality $q^{\text{marginal}} = Q^{\text{average}}$ requires constant returns to scale in both production and adjustment costs (Hayashi 1982). With decreasing returns to scale (empirically relevant for most firms), average Q overstates marginal Q for high-Q firms and understates it for low-Q firms. Abel and Eberly (1994) derive the wedge analytically.

Problem 2: Measurement error in numerator. The market value of equity reflects market sentiment, bubbles, and noise-trader demand in addition to fundamentals. P. Bond, Edmans, and Goldstein (2012) provide a comprehensive treatment. In Vietnamese markets, where retail investors dominate and price limits constrain daily adjustment, market prices may deviate persistently from fundamental value.

Problem 3: Measurement error in denominator. Book values of assets reflect historical cost, depreciation schedules, and accounting conventions that may poorly approximate replacement cost. This is especially problematic in Vietnam, where revaluation of fixed assets is infrequent and inflation has historically been volatile, creating wedges between historical and replacement cost.

Problem 4: Errors-in-variables bias. Because the empirical Q is a noisy proxy for the true Q, OLS estimates of β_1 in Equation 91.3 suffer from classical attenuation bias (i.e., $\hat{\beta}_1$ is biased toward zero). Erickson and Whited (2012) develop a higher-order cumulant estimator that corrects for this bias without requiring external instruments.

```
# Construct Tobin's Q and investment variables
panel = firm_annual.copy()

# Tobin's Q: (Market cap + Book debt) / Total assets
panel["tobins_q"] = (
    (panel["market_cap"] + panel["total_debt"]) /
    panel["total_assets"]
)

# Investment rate: Capital expenditure / Lagged total assets
panel = panel.sort_values(["ticker", "year"])
panel["lag_assets"] = panel.groupby("ticker")["total_assets"].shift(1)
panel["lag_ppe"] = panel.groupby("ticker")["ppe_net"].shift(1)
```

Table 91.1: Summary Statistics: Investment and Q Variables

```
summary_vars = ["inv_rate", "tobins_q", "cf_assets", "sales_growth"]
summary = panel_clean[summary_vars].describe(
    percentiles=[0.05, 0.25, 0.5, 0.75, 0.95]
).T.round(4)

summary.columns = ["N", "Mean", "Std", "Min", "5%", "25%",
                   "Median", "75%", "95%", "Max"]
summary
```

```
panel["inv_rate"] = panel["capex"] / panel["lag_assets"]
panel["inv_rate_ppe"] = panel["capex"] / panel["lag_ppe"]

# Cash flow / Assets
panel["cf_assets"] = panel["operating_cf"] / panel["lag_assets"]

# Sales growth
panel["lag_revenue"] = panel.groupby("ticker")["revenue"].shift(1)
panel["sales_growth"] = (
    (panel["revenue"] - panel["lag_revenue"]) / panel["lag_revenue"]
)

# Winsorize at 1st and 99th percentiles
def winsorize(s, lower=0.01, upper=0.99):
    return s.clip(s.quantile(lower), s.quantile(upper))

for col in ["tobins_q", "inv_rate", "cf_assets", "sales_growth"]:
    panel[col] = winsorize(panel[col])

panel_clean = panel.dropna(
    subset=["tobins_q", "inv_rate", "cf_assets"]
).copy()

print(f"Clean panel: {len(panel_clean)} firm-years, "
      f"{panel_clean['ticker'].nunique()} firms")
```

```
# Baseline investment-Q regression with firm and year fixed effects
panel_clean = panel_clean.set_index(["ticker", "year"])

# Model 1: Q only
model1 = PanelOLS(
    panel_clean["inv_rate"],
    panel_clean[["tobins_q"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

# Model 2: Q + Cash Flow
model2 = PanelOLS(
    panel_clean["inv_rate"],
    panel_clean[["tobins_q", "cf_assets"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

# Model 3: Q + Cash Flow + Sales Growth
model3 = PanelOLS(
    panel_clean["inv_rate"],
    panel_clean[["tobins_q", "cf_assets", "sales_growth"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

panel_clean = panel_clean.reset_index()
```

91.1.3 Interpretation Under Market Frictions

The coefficient on Q in Table 91.2 admits multiple interpretations depending on the maintained assumptions:

Structural interpretation. If the Hayashi conditions hold, $\hat{\beta}_1$ estimates the inverse of the adjustment cost parameter: $\hat{\beta}_1 = 1/\hat{\alpha}$. A larger coefficient implies lower adjustment costs. However, measurement error in Q biases $\hat{\beta}_1$ downward, so the raw OLS estimate provides a lower bound on $1/\alpha$.

Table 91.2: Investment-Q Regressions with Firm and Year Fixed Effects

```
results_table = pd.DataFrame({
    "Q Only": {
        "Tobin's Q": f"{model1.params['tobins_q']:.4f}",
        "": f"({model1.std_errors['tobins_q']:.4f})",
        "Cash Flow/Assets": "",
        " ": "",
        "Sales Growth": "",
        "  ": "",
        "R² (within)": f"{model1.rsquared_within:.4f}",
        "N": f"{int(model1.nobs)}"
    },
    "Q + CF": {
        "Tobin's Q": f"{model2.params['tobins_q']:.4f}",
        "": f"({model2.std_errors['tobins_q']:.4f})",
        "Cash Flow/Assets": f"{model2.params['cf_assets']:.4f}",
        " ": f"({model2.std_errors['cf_assets']:.4f})",
        "Sales Growth": "",
        "  ": "",
        "R² (within)": f"{model2.rsquared_within:.4f}",
        "N": f"{int(model2.nobs)}"
    },
    "Q + CF + SG": {
        "Tobin's Q": f"{model3.params['tobins_q']:.4f}",
        "": f"({model3.std_errors['tobins_q']:.4f})",
        "Cash Flow/Assets": f"{model3.params['cf_assets']:.4f}",
        " ": f"({model3.std_errors['cf_assets']:.4f})",
        "Sales Growth": f"{model3.params['sales_growth']:.4f}",
        "  ": f"({model3.std_errors['sales_growth']:.4f})",
        "R² (within)": f"{model3.rsquared_within:.4f}",
        "N": f"{int(model3.nobs)}"
    }
})

results_table
```

Reduced-form interpretation. Without the Hayashi conditions, $\hat{\beta}_1$ captures the association between market valuation and investment intensity. This association reflects a mixture of genuine investment opportunities (the Q-theory channel), market mispricing that managers exploit (the market timing channel of Baker, Stein, and Wurgler (2003)), and reverse causality (investment announcements that move market values).

The cash flow coefficient puzzle. The significance of $\hat{\beta}_2$ on cash flow has been the subject of a 35-year debate. Fazzari, Hubbard, and Petersen (1987) interpret it as evidence that firms face financing constraints: controlling for investment opportunities (Q), cash flow should be irrelevant in a frictionless world, so its significance implies that internal funds relax binding constraints. Kaplan and Zingales (1997) counter that cash flow proxies for investment opportunities not captured by the noisy Q measure, making the cash flow coefficient an artifact of measurement error rather than evidence of constraints. Erickson and Whited (2012) show that correcting for measurement error in Q substantially reduces (but does not eliminate) the cash flow coefficient, supporting a middle ground.

91.1.4 Limitations in Emerging Markets

The investment-Q framework faces amplified challenges in Vietnamese markets.

Thin trading and price limits. Market prices adjust slowly to information, so Q measured at fiscal year-end may not reflect the firm's current investment opportunity set. Price limits of $\pm 7\%$ (HOSE) and $\pm 10\%$ (HNX) mechanically compress the numerator of Q, attenuating the investment-Q relationship.

State ownership. For state-owned enterprises (SOEs), investment decisions may be driven by policy directives rather than Q-theoretic optimality. Including SOEs in the regression without interactions confounds the structural relationship.

Related-party transactions. Tunneling through related-party transactions means that measured investment may include capital expenditures that benefit controlling shareholders rather than maximizing firm value. The investment-Q coefficient in tunneling firms reflects the relationship between market valuation and expropriation, not efficient capital allocation.

```
# Merge ownership data
panel_with_own = panel_clean.merge(
    ownership[["ticker", "year", "state_ownership_pct",
               "foreign_ownership_pct", "insider_ownership_pct"]],
    on=["ticker", "year"],
    how="left"
)

panel_with_own["soe_dummy"] = (
    panel_with_own["state_ownership_pct"] > 50
```

```
# Create Q-decile bins for clean visualization
plot_data = panel_clean.copy()
plot_data["q_bin"] = pd.qcut(
    plot_data["tobins_q"], q=20, duplicates="drop"
)

binned = (
    plot_data.groupby("q_bin", observed=True)
    .agg(
        mean_q=("tobins_q", "mean"),
        mean_inv=("inv_rate", "mean"),
        se_inv=("inv_rate", lambda x: x.std() / np.sqrt(len(x)))
    )
    .reset_index()
)

(
    p9.ggplot(binned, p9.aes(x="mean_q", y="mean_inv"))
    + p9.geom_pointrange(
        p9.aes(ymin="mean_inv - 1.96*se_inv",
               ymax="mean_inv + 1.96*se_inv"),
        color="#2E5090", size=0.5
    )
    + p9.geom_smooth(method="lm", color="#C0392B", se=False, size=0.8)
    + p9.labs(
        x="Tobin's Q (Vingtile Mean)",
        y="Investment Rate (I/A)",
        title="Investment-Q Relationship: Binned Scatter"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 6))
)
```

Figure 91.1

Table 91.3: Investment-Q Regression with State Ownership Interactions

```
soe_results = pd.DataFrame({
    "Coefficient": model_soe.params.round(4),
    "Std Error": model_soe.std_errors.round(4),
    "t-stat": model_soe.tstats.round(3),
    "p-value": model_soe.pvalues.round(4)
})

soe_results
```

```
).astype(int)

panel_with_own["q_x_soe"] = (
    panel_with_own["tobins_q"] * panel_with_own["soe_dummy"]
)
panel_with_own["cf_x_soe"] = (
    panel_with_own["cf_assets"] * panel_with_own["soe_dummy"]
)

# Regression with SOE interactions
panel_soe = panel_with_own.dropna(
    subset=["inv_rate", "tobins_q", "cf_assets", "soe_dummy"]
).set_index(["ticker", "year"])

model_soe = PanelOLS(
    panel_soe["inv_rate"],
    panel_soe[["tobins_q", "cf_assets", "soe_dummy",
               "q_x_soe", "cf_x_soe"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

panel_soe = panel_soe.reset_index()
```

A negative coefficient on $Q \times \text{SOE}$ indicates that the investment-Q sensitivity is attenuated for state-owned enterprises, consistent with SOE investment being driven by non-market factors. The interaction of cash flow with SOE status reveals whether state firms face tighter or looser financing constraints. This is a question with direct policy implications for SOE reform.

91.1.5 The Erickson-Whited Measurement Error Correction

Erickson and Whited (2012) develop a GMM estimator that uses higher-order moments of the data to identify the investment-Q slope in the presence of measurement error, without requiring external instruments. The key insight is that if the measurement error η in Q is independent of the true Q^* and the structural error ε, then the third-order cumulants identify the signal-to-noise ratio.

The model is:

$$\frac{I_{i,t}}{K_{i,t-1}} = \beta_0 + \beta_1 Q^*_{i,t} + \gamma X_{i,t} + \varepsilon_{i,t}, \qquad Q_{i,t} = Q^*_{i,t} + \eta_{i,t} \tag{91.5}$$

where $Q^*_{i,t}$ is unobserved true Q and $\eta_{i,t}$ is measurement error. The OLS estimator $\hat{\beta}_1^{\text{OLS}}$ converges to $\beta_1 \cdot \lambda$ where $\lambda = \text{Var}(Q^*)/(\text{Var}(Q^*) + \text{Var}(\eta)) < 1$ is the signal-to-noise ratio. The Erickson-Whited estimator recovers β_1 and λ simultaneously.

```
def erickson_whited_gmm(y, Q_obs, X=None, order=3):
    """
    Simplified Erickson-Whited (2012) measurement error correction
    using third-order cumulants.

    Parameters
    ----------
    y : array
        Dependent variable (investment rate).
    Q_obs : array
        Observed (mismeasured) Q.
    X : array or None
        Additional controls (partialled out first).
    order : int
        Cumulant order for identification (3 or 5).

    Returns
    -------
    dict : Corrected beta, signal-to-noise ratio, OLS beta.
    """
    if X is not None:
        # Partial out controls via OLS
        X_aug = sm.add_constant(X)
        y = y - X_aug @ np.linalg.lstsq(X_aug, y, rcond=None)[0]
        Q_obs = Q_obs - X_aug @ np.linalg.lstsq(X_aug, Q_obs, rcond=None)[0]
```

```
    # Demean
    y_dm = y - y.mean()
    q_dm = Q_obs - Q_obs.mean()
    n = len(y)

    # Second moments
    m_yq = np.mean(y_dm * q_dm)
    m_qq = np.mean(q_dm**2)

    # OLS beta
    beta_ols = m_yq / m_qq

    # Third-order cumulants for identification
    k3_q = np.mean(q_dm**3)
    k2y_q = np.mean(y_dm * q_dm**2)

    if abs(k3_q) < 1e-10:
        return {
            "beta_corrected": np.nan,
            "lambda_snr": np.nan,
            "beta_ols": beta_ols,
            "note": "Insufficient skewness for identification"
        }

    # Corrected beta: beta = kappa_{y,q,q} / kappa_{q,q,q}
    beta_ew = k2y_q / k3_q

    # Signal-to-noise ratio
    # lambda = kappa_{q,q,q}^2 / (kappa_{q,q} * kappa_{q,q,q,q,q})
    # Simplified: lambda = beta_ols / beta_ew
    lambda_snr = beta_ols / beta_ew if abs(beta_ew) > 1e-10 else np.nan

    return {
        "beta_corrected": beta_ew,
        "lambda_snr": lambda_snr,
        "beta_ols": beta_ols,
        "attenuation_pct": round((1 - lambda_snr) * 100, 1) if not
         ↪  np.isnan(lambda_snr) else np.nan
    }

# Apply to Vietnamese data
ew_data = panel_clean.dropna(subset=["inv_rate", "tobins_q", "cf_assets"])
```

```
ew_result = erickson_whited_gmm(
    y=ew_data["inv_rate"].values,
    Q_obs=ew_data["tobins_q"].values,
    X=ew_data["cf_assets"].values.reshape(-1, 1)
)

print("Erickson-Whited Measurement Error Correction:")
for k, v in ew_result.items():
    if isinstance(v, float):
        print(f"  {k}: {v:.4f}")
    else:
        print(f"  {k}: {v}")
```

91.1.6 Marginal q from Market Values

The Erickson-Whited correction treats Tobin's Q as a noisy measure of the right object. Gala, Gomes, and Liu (2026) argue that it is also a measure of the wrong object. Q-theory says investment responds to *marginal* q, the shadow value of one more unit of capital, $q = \partial V/\partial K$. Average $Q = V/K$ equals marginal q only under the knife-edge homogeneity conditions of Hayashi (1982), and no correction for classical measurement error can close a gap that comes from misspecification. Their insight is that the value function is not as unobservable as it looks. The market value of the firm *is* the value function, observed with noise, and its arguments (the capital stock and the firm's productivity) are measurable. Marginal q can therefore be obtained by estimating the value function and differentiating it.

The estimator has two steps. First, project the log of market value per unit of capital on a polynomial in log capital, $k_{i,t} = \ln K_{i,t}$, and log productivity, $a_{i,t} = \ln A_{i,t}$, with firm and year fixed effects:

$$v_{i,t} \equiv \ln Q_{i,t} = \sum_{j_k=0}^{n_k} \sum_{j_a=0}^{n_a} c_{j_k,j_a}\, k_{i,t}^{j_k} a_{i,t}^{j_a} + \alpha_i + \tau_t + \varepsilon_{i,t}. \tag{91.6}$$

The year effects absorb aggregate state variables, such as the discount rate, without requiring them to be measured, and the firm effects absorb time-invariant heterogeneity in input prices and depreciation. The authors find that a complete second-order polynomial (linear, squared, and interaction terms) captures the variation in market values, and that higher-order terms selected by an adaptive Lasso change little. Second, differentiate the fitted value function. Since $V = QK$,

$$q_{i,t} = \frac{\partial V_{i,t}}{\partial K_{i,t}} = \widehat{Q}_{i,t}\left(1 + \frac{\partial \hat{v}_{i,t}}{\partial k_{i,t}}\right), \qquad \frac{\partial \hat{v}_{i,t}}{\partial k_{i,t}} = \hat{c}_{1,0} + 2\hat{c}_{2,0}\, k_{i,t} + \hat{c}_{1,1}\, a_{i,t}, \tag{91.7}$$

where $\widehat{Q}_{i,t} = \exp(\hat{v}_{i,t})$ is the fitted value of Equation 91.6, which the authors call **Fitted** Q. Fitted Q is average Q purged of all variation unrelated to the fundamental state variables, so it is a direct, observation-level correction for classical measurement error, including the error that comes from stock-market mispricing. Marginal q goes further and corrects the misspecification as well. Hayashi's equivalence $q = Q$ holds exactly when $\partial v/\partial k = 0$, that is, when every coefficient on a term involving k is zero, so Equation 91.6 also delivers a Wald test of the assumption that the investment-Q literature has maintained for four decades. The authors reject it decisively.

The findings, from Compustat manufacturing firms over 1973–2019 in the working-paper version, are large.

- **Levels.** Average marginal q is 0.81, against 1.33 for Fitted Q and 2.87 for Tobin's Q, and it is far less volatile. Marginal q always lies below Fitted Q, as concavity of the value function requires.
- **Adjustment costs.** Because optimal investment maximizes $qI - \Phi(I, K)$, which must be non-negative, marginal q is an upper bound on total adjustment costs (purchase price included) per unit of investment. Marginal q caps them at 81% on average; treating Tobin's Q as marginal q would allow 287%. Estimated quadratic adjustment-cost parameters fall from about 4.5 with Tobin's Q to about 1.3 with marginal q.
- **Investment regressions.** In the regression of investment on Q with firm and year effects, the slope on Tobin's Q is 0.007 and the within-R^2 about 0.03; with marginal q the slope is 0.38 and the within-R^2 about 0.49. The cash flow coefficient, the centerpiece of the financing-constraints debate discussed below, shrinks to zero once marginal q is used.
- **The widening gap.** In the published version, the authors show that the gap between marginal q and Tobin's Q has widened over time, driven mainly by market power and intangible capital, two forces that raise the value of the firm without raising the value of an additional unit of physical capital (Peters and Taylor 2017).

The procedure needs no assumption about the form of adjustment costs, the stochastic discount factor, or the production technology beyond differentiability, which is why the authors describe the resulting q as quasi-model-free. It does need one thing that is easy to get wrong. The derivative in Equation 91.7 holds *productivity* fixed, so $a_{i,t}$ must measure the exogenous state, not an outcome that moves mechanically with K. The authors estimate it with production-function methods (Olley and Pakes 1996; Ackerberg, Caves, and Frazer 2015). A sales-to-capital ratio is a poor substitute: holding $\ln(Y/K)$ fixed while varying K assumes constant returns, which is precisely the assumption the method is designed to avoid.

The estimator fits in a few lines. The function below implements the second-order version of Equation 91.6 and Equation 91.7, returns Fitted Q and marginal q for every firm-year, and reports the Wald test of $q = Q$.

```
def estimate_marginal_q(df, v="ln_q", k="ln_k", a="ln_a",
                        entity="ticker", time="year"):
    """
    Projection estimator of marginal q (Gala, Gomes, and Liu).

    Step 1: regress v = ln(V/K) on a complete second-order polynomial in
            k = ln K and a = ln A with firm and year fixed effects.
    Step 2: differentiate the fitted value function with respect to k,
            q = Fitted Q * (1 + dv/dk).
    """
    d = df.dropna(subset=[v, k, a]).copy()
    d["k_sq"] = d[k] ** 2
    d["a_sq"] = d[a] ** 2
    d["k_x_a"] = d[k] * d[a]
    d = d.set_index([entity, time])

    terms = [k, a, "k_sq", "a_sq", "k_x_a"]
    fit = PanelOLS(
        d[v], d[terms], entity_effects=True, time_effects=True,
        check_rank=False
    ).fit(cov_type="clustered", cluster_entity=True)

    c = fit.params
    # Fitted value function includes the estimated firm and year effects
    v_hat = fit.fitted_values.iloc[:, 0] + fit.estimated_effects.iloc[:, 0]
    dv_dk = c[k] + 2 * c["k_sq"] * d[k] + c["k_x_a"] * d[a]

    d["fitted_q"] = np.exp(v_hat)
    d["dv_dk"] = dv_dk
    d["marginal_q"] = d["fitted_q"] * (1 + dv_dk)

    # Wald test of Hayashi's q = Q: all coefficients on terms in k are zero
    R = np.zeros((3, len(terms)))
    for row, term in enumerate([k, "k_sq", "k_x_a"]):
        R[row, terms.index(term)] = 1.0
    qq_test = fit.wald_test(R, np.zeros(3))

    return d.reset_index(), fit, qq_test
```

Before applying it, it is worth checking that the estimator recovers marginal q when the truth is known. The simulation below builds a panel of 600 firms over 15 years with a concave value function, lets investment respond to *true* marginal q with a quadratic adjustment-cost

parameter of 2.5, and contaminates observed Q with log-normal measurement error. Nothing in it is calibrated to Vietnam; its only purpose is to show what each Q measure does to the investment regression when the data-generating process is known.

```
rng_q = np.random.default_rng(20260819)
n_firms, n_years = 600, 15
firm_id = np.repeat(np.arange(n_firms), n_years)
year_id = np.tile(np.arange(n_years), n_firms)

# State variables: log capital and persistent log productivity
ln_k = (np.repeat(rng_q.normal(4.5, 1.5, n_firms), n_years)
        + rng_q.normal(0, 0.4, n_firms * n_years))
a_mean = rng_q.normal(0.0, 0.3, n_firms)
ln_a = np.empty(n_firms * n_years)
for i in range(n_firms):
    path = np.empty(n_years)
    shocks = rng_q.normal(0, 0.25, n_years)
    path[0] = a_mean[i] + shocks[0]
    for t in range(1, n_years):
        path[t] = a_mean[i] + 0.7 * (path[t - 1] - a_mean[i]) + shocks[t]
    ln_a[i * n_years:(i + 1) * n_years] = path

# True concave value function v = ln(V/K) and the implied marginal q
c_true = {"k": -0.50, "a": 0.55, "k_sq": 0.025, "a_sq": 0.30, "k_x_a": -0.05}
v_true = (1.5
          + np.repeat(rng_q.normal(0, 0.3, n_firms), n_years)
          + np.tile(rng_q.normal(0, 0.15, n_years), n_firms)
          + c_true["k"] * ln_k + c_true["a"] * ln_a
          + c_true["k_sq"] * ln_k**2 + c_true["a_sq"] * ln_a**2
          + c_true["k_x_a"] * ln_k * ln_a)
dv_dk_true = c_true["k"] + 2 * c_true["k_sq"] * ln_k + c_true["k_x_a"] * ln_a
q_true = np.exp(v_true) * (1 + dv_dk_true)

# Investment responds to true marginal q; observed Q is measured with error
gamma_true = 2.5
sim_q = pd.DataFrame({
    "ticker": firm_id, "year": year_id, "ln_k": ln_k, "ln_a": ln_a,
    "ln_q": v_true + rng_q.normal(0, 0.5, n_firms * n_years),
    "inv_rate": (q_true - 0.4) / gamma_true
                + rng_q.normal(0, 0.03, n_firms * n_years),
    "q_true": q_true,
})
sim_q["tobins_q"] = np.exp(sim_q["ln_q"])
```

```
sim_est, sim_fit, sim_qq = estimate_marginal_q(sim_q)
print(f"Wald test of q = Q: chi2 = {sim_qq.stat:.1f}, p = {sim_qq.pval:.4f}")
print(f"Correlation of estimated with true marginal q: "
      f"{sim_est['marginal_q'].corr(sim_est['q_true']):.3f}")
```

```
Wald test of q = Q: chi2 = 469.0, p = 0.0000
Correlation of estimated with true marginal q: 0.970
```

```
def fe_slope(data, y, x):
    d = data.set_index(["ticker", "year"])
    res = PanelOLS(d[y], d[[x]], entity_effects=True, time_effects=True,
                   check_rank=False).fit(cov_type="clustered",
                                         cluster_entity=True)
    return res.params[x], res.std_errors[x], res.rsquared_within

rows = []
for label, col in [("Tobin's Q (observed)", "tobins_q"),
                   ("Fitted Q", "fitted_q"),
                   ("Marginal q (estimated)", "marginal_q"),
                   ("Marginal q (true)", "q_true")]:
    slope, se, r2 = fe_slope(sim_est, "inv_rate", col)
    rows.append({"Q measure": label,
                 "Mean": sim_est[col].mean(),
                 "Slope of I/K": slope,
                 "Std. error": se,
                 "Within R²": r2,
                 "Implied adjustment cost (1/slope)": 1 / slope})

pd.DataFrame(rows).set_index("Q measure").round(3)
```

Table 91.4: Investment Regressions Under Alternative Q Measures (Simulated Panel, True Adjustment-Cost Parameter 2.5)

Q measure	Mean	Slope of I/K	Std. error	Within R^2	Implied adjustment cost (1/slope)
Tobin's Q (observed)	1.258	0.028	0.003	0.163	35.199
Fitted Q	1.118	0.190	0.004	0.813	5.275
Marginal q (estimated)	0.753	0.381	0.007	0.849	2.626
Marginal q (true)	0.745	0.397	0.002	0.873	2.519

The simulation reproduces the paper's ranking. Observed Tobin's Q has the highest mean and the weakest link to investment: its slope implies an adjustment-cost parameter more than ten times the true value, the familiar implausibly large adjustment costs of the investment-Q literature. Fitted Q removes the measurement error and most of the bias, but because it is still *average* Q it overstates marginal q, and its slope is only about half the true one. Estimated marginal q recovers both the level and the slope of the true series. The remaining discrepancy reflects estimation error in the first step, which is why inference on second-step coefficients should bootstrap the whole procedure by firm rather than treat $\hat{q}$ as data.

Applying the estimator to Vietnamese listed firms follows the same steps, with capital measured by beginning-of-year net PPE, V by the market value of equity plus book debt, and productivity by a revenue production-function residual estimated industry by industry.

```
mq = panel.copy()

# Capital stock: beginning-of-year net PPE (gross PPE if available)
mq["K"] = mq["lag_ppe"]
mq = mq[(mq["K"] > 0) & (mq["revenue"] > 0)].copy()

# Average Q on capital and investment rate on capital
mq["tobins_q_k"] = (mq["market_cap"] + mq["total_debt"]) / mq["K"]
mq = mq[mq["tobins_q_k"] > 0].copy()
mq["ln_q"] = np.log(mq["tobins_q_k"])
mq["ln_k"] = np.log(mq["K"])
mq["inv_rate_k"] = mq["capex"] / mq["K"]
mq["cf_k"] = mq["operating_cf"] / mq["K"]

# Productivity: residual of a within-firm revenue production function,
# estimated separately by industry. A control-function estimator
# (Olley-Pakes, Levinsohn-Petrin, Ackerberg-Caves-Frazer) is preferable
# when labor and materials data are available.
mq["ln_y"] = np.log(mq["revenue"])
theta = {}
for ind, g in mq.groupby("industry"):
    g = g.set_index(["ticker", "year"])
    if g.index.get_level_values(0).nunique() < 10:
        continue
    pf = PanelOLS(g["ln_y"], g[["ln_k"]], entity_effects=True,
                  time_effects=True, check_rank=False).fit()
    theta[ind] = pf.params["ln_k"]
mq["theta"] = mq["industry"].map(theta)
mq["ln_a"] = mq["ln_y"] - mq["theta"] * mq["ln_k"]
```

Table 91.5: Investment on Tobin's Q, Fitted Q, and Marginal q: Vietnamese Listed Firms

```
vn_rows = []
for label, col in [("Tobin's Q", "tobins_q_k"),
                   ("Fitted Q", "fitted_q"),
                   ("Marginal q", "marginal_q")]:
    d = vn_est.set_index(["ticker", "year"])
    for with_cf in [False, True]:
        regressors = [col, "cf_k"] if with_cf else [col]
        res = PanelOLS(d["inv_rate_k"], d[regressors], entity_effects=True,
                       time_effects=True, check_rank=False).fit(
            cov_type="clustered", cluster_entity=True)
        vn_rows.append({
            "Q measure": label,
            "Cash flow": "Yes" if with_cf else "No",
            "Slope on Q": f"{res.params[col]:.3f}
             ↪  ({res.std_errors[col]:.3f})",
            "Slope on CF/K": (f"{res.params['cf_k']:.3f} "
                              f"({res.std_errors['cf_k']:.3f})")
                             if with_cf else "",
            "Within R²": round(res.rsquared_within, 3),
            "N": int(res.nobs),
        })

pd.DataFrame(vn_rows)
```

```
# Trim the tails of the regression variables at 0.5% and 99.5%
for col in ["ln_q", "ln_k", "ln_a", "inv_rate_k", "cf_k"]:
    lo, hi = mq[col].quantile([0.005, 0.995])
    mq = mq[mq[col].between(lo, hi)]

# Keep firms with at least two observations
mq = mq[mq.groupby("ticker")["year"].transform("count") >= 2]

vn_est, vn_fit, vn_qq = estimate_marginal_q(mq)
print(vn_fit.summary)
print(f"Wald test of q = Q: chi2 = {vn_qq.stat:.1f}, p = {vn_qq.pval:.4f}")
print(f"Share of firm-years with negative marginal q: "
      f"{(vn_est['marginal_q'] < 0).mean():.3f}")
```

Several features of the Vietnamese setting bear on the implementation and the interpretation.

Fitted Q is the natural answer to noisy prices. The measurement problems listed earlier in this section (retail-driven mispricing, price limits that compress the numerator, thin trading) are all forms of variation in market value that is unrelated to the firm's fundamental state. The first-step projection discards exactly that variation, so in Vietnam the gain from Fitted Q over observed Q may be larger than in US data.

The capital stock needs a deliberate definition. Book PPE poorly approximates replacement cost when fixed assets are rarely revalued, and land-use rights, which are economically part of the capital of real-estate, industrial-park, and many manufacturing firms, are recorded as intangible assets under Vietnamese accounting standards. Report results with and without land-use rights in K. Firm fixed effects absorb much of any time-invariant proportional error in measured capital, but not a revaluation that happens mid-sample, such as one at equitization.

Productivity data are the binding constraint. Control-function estimators need labor or intermediate inputs. Employee counts appear in many annual reports but not consistently in databases, so the revenue residual used above is a fallback. Its main weakness, correlation between capital and the contemporaneous productivity shock, biases θ upward and shifts measured productivity accordingly.

Additional state variables can enter the value function. State ownership, foreign-ownership room, and business-group affiliation all plausibly affect firm value. The projection accommodates them as extra state variables, but only if they are exogenous to the firm's current decisions, since marginal q is identified off the exogenous states (the authors' footnote on identification is explicit about this).

Revisit the earlier results with marginal q. The state-ownership interaction in Table 91.3 and the cash-flow sensitivity tests below were run with average Q. If, as in US data, marginal q drives out cash flow, much of what looks like financing constraints in Vietnamese investment regressions is mismeasured investment opportunities. If cash flow survives, especially among non-state firms with limited bank access, that is stronger evidence of constraints than any result obtained with average Q.

91.2 Cash Flow Sensitivity of Investment

91.2.1 The Financing Constraints Hypothesis

The cash flow sensitivity of investment (CFSI) literature tests whether firms' investment decisions are constrained by the availability of internal funds. In a Modigliani-Miller world, internal and external funds are perfect substitutes, so cash flow should be irrelevant for investment after controlling for investment opportunities. The CFSI approach, pioneered by Fazzari,

Hubbard, and Petersen (1987), classifies firms as financially constrained or unconstrained using observable characteristics and tests whether constrained firms exhibit higher sensitivity of investment to cash flow.

The augmented investment regression is:

$$\frac{I_{i,t}}{K_{i,t-1}} = \beta_0 + \beta_1 Q_{i,t} + \beta_2 \frac{CF_{i,t}}{K_{i,t-1}} + \varepsilon_{i,t} \tag{91.8}$$

The CFSI hypothesis predicts $\beta_2^{\text{constrained}} > \beta_2^{\text{unconstrained}} > 0$: constrained firms rely more heavily on internal cash flow to fund investment because external finance is costly or unavailable.

91.2.2 The FHP-KZ Debate

Fazzari, Hubbard, and Petersen (1987) (FHP) classify firms by dividend payout ratios and find that low-payout firms (presumed constrained) exhibit significantly higher cash flow sensitivity. Kaplan and Zingales (1997) (KZ) challenge this interpretation on two grounds:

Critique 1: Q measurement error. If Q is a noisy proxy for true investment opportunities, and cash flow is correlated with the measurement error (because both respond to demand shocks), then the cash flow coefficient captures omitted investment opportunities, not financing constraints.

Critique 2: Monotonicity failure. KZ show that the firms FHP classify as "most constrained" (low-payout firms) are often rapidly growing firms that choose to retain earnings for investment, not firms that are denied external financing. Using qualitative information from annual reports, KZ reclassify firms and find that the CFSI ranking reverses: firms judged to be truly constrained by their own disclosures exhibit lower CFSI than unconstrained firms.

The resolution, as argued by Farre-Mensa and Ljungqvist (2016), is that no single proxy reliably identifies financially constrained firms. Each proxy (size, age, payout ratio, bond rating, KZ index, WW index, SA index) captures a different dimension of the financing environment, and the CFSI test is not a clean test of any single theory.

91.2.3 Constraint Indices

We implement the three most widely used composite constraint measures.

KZ Index (Kaplan and Zingales 1997; Lamont, Polk, and Saaá-Requejo 2001):

$$\text{KZ}_{i,t} = -1.002 \cdot \frac{CF_{i,t}}{K_{i,t-1}} + 0.283 \cdot Q_{i,t} + 3.139 \cdot \frac{D_{i,t}}{A_{i,t}} - 39.368 \cdot \frac{\text{Div}_{i,t}}{K_{i,t-1}} - 1.315 \cdot \frac{C_{i,t}}{K_{i,t-1}} \tag{91.9}$$

WW Index (Whited and Wu 2006):

$$\text{WW}_{i,t} = -0.091 \cdot \frac{CF_{i,t}}{A_{i,t}} - 0.062 \cdot \mathbb{1}(\text{Div} > 0) + 0.021 \cdot \frac{D_{i,t}}{A_{i,t}} - 0.044 \cdot \ln(A_{i,t}) + 0.102 \cdot \text{ISG}_{i,t} - 0.035 \cdot \text{SG}_{i,t} \tag{91.10}$$

where ISG is industry sales growth and SG is firm sales growth.

SA Index (Hadlock and Pierce 2010):

$$\text{SA}_{i,t} = -0.737 \cdot \text{Size}_{i,t} + 0.043 \cdot \text{Size}_{i,t}^2 - 0.040 \cdot \text{Age}_{i,t} \tag{91.11}$$

where Size = ln(Total Assets) and Age is years since listing. Hadlock and Pierce (2010) argue that the SA index is preferable because it uses only exogenous firm characteristics (size and age), avoiding the endogeneity inherent in cash flow and leverage-based indices.

```
# Compute financial constraint indices
panel_fc = panel_clean.copy()

# Lagged PPE for KZ scaling
panel_fc["lag_ppe"] = panel_fc.groupby("ticker")["ppe_net"].shift(1)

# KZ Index
panel_fc["kz_index"] = (
    -1.002 * panel_fc["cf_assets"]
    + 0.283 * panel_fc["tobins_q"]
    + 3.139 * (panel_fc["total_debt"] / panel_fc["total_assets"])
    - 39.368 * (panel_fc["dividends"] / panel_fc["lag_assets"])
    - 1.315 * (panel_fc["cash"] / panel_fc["lag_assets"])
)

# SA Index
panel_fc["log_assets"] = np.log(panel_fc["total_assets"])
panel_fc["listing_age"] = panel_fc["year"] - panel_fc["listing_year"]

panel_fc["sa_index"] = (
    -0.737 * panel_fc["log_assets"]
    + 0.043 * panel_fc["log_assets"]**2
    - 0.040 * panel_fc["listing_age"]
)

# WW Index (simplified: using firm-level variables)
```

Table 91.6: Distribution of Financial Constraint Indices

```
constraint_vars = ["kz_index", "sa_index", "ww_index"]
constraint_summary = (
    panel_fc[constraint_vars]
    .describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9])
    .T.round(4)
)
constraint_summary
```

```
panel_fc["div_dummy"] = (panel_fc["dividends"] > 0).astype(int)
panel_fc["leverage"] = panel_fc["total_debt"] / panel_fc["total_assets"]

# Industry sales growth
panel_fc["isg"] = panel_fc.groupby(
    ["industry", "year"]
)["sales_growth"].transform("median")

panel_fc["ww_index"] = (
    -0.091 * panel_fc["cf_assets"]
    - 0.062 * panel_fc["div_dummy"]
    + 0.021 * panel_fc["leverage"]
    - 0.044 * panel_fc["log_assets"]
    + 0.102 * panel_fc["isg"]
    - 0.035 * panel_fc["sales_growth"]
)
```

91.2.4 Split-Sample CFSI Tests

We classify firms into constrained and unconstrained groups using each index and compare the cash flow sensitivity of investment across groups.

```
def cfsi_by_group(data, group_var, threshold="median"):
    """
    Estimate cash flow sensitivity of investment by constraint group.

    Parameters
    ----------
    data : DataFrame
```

```
    Panel data with inv_rate, tobins_q, cf_assets, group_var.
group_var : str
    Variable used for classification.
threshold : str
    "median" for sample split or "tercile" for top/bottom third.

Returns
-------
dict : Coefficient estimates by group.
"""
df = data.dropna(subset=["inv_rate", "tobins_q", "cf_assets", group_var])

if threshold == "median":
    median_val = df[group_var].median()
    df["constrained"] = (df[group_var] >= median_val).astype(int)
elif threshold == "tercile":
    t33 = df[group_var].quantile(0.33)
    t67 = df[group_var].quantile(0.67)
    df = df[(df[group_var] <= t33) | (df[group_var] >= t67)]
    df["constrained"] = (df[group_var] >= t67).astype(int)

results = {}
for group_name, group_label in [(0, "Unconstrained"), (1,
 ↪  "Constrained")]:
    subset = df[df["constrained"] == group_name].copy()
    if len(subset) < 100:
        continue

    subset = subset.set_index(["ticker", "year"])
    model = PanelOLS(
        subset["inv_rate"],
        subset[["tobins_q", "cf_assets"]],
        entity_effects=True,
        time_effects=True,
        check_rank=False
    ).fit(cov_type="clustered", cluster_entity=True)

    results[group_label] = {
        "beta_Q": model.params["tobins_q"],
        "se_Q": model.std_errors["tobins_q"],
        "beta_CF": model.params["cf_assets"],
        "se_CF": model.std_errors["cf_assets"],
        "R2_within": model.rsquared_within,
```

Table 91.7: Cash Flow Sensitivity by Financial Constraint Classification

```
# Combine results
cfsi_all = pd.concat({
    "KZ Index": cfsi_kz,
    "SA Index": cfsi_sa,
    "WW Index": cfsi_ww
})

cfsi_display = cfsi_all[["beta_CF", "se_CF", "beta_Q", "se_Q", "N"]].round(4)
cfsi_display
```

```
            "N": int(model.nobs)
        }

    return pd.DataFrame(results).T

# Run for each constraint index
cfsi_kz = cfsi_by_group(panel_fc, "kz_index", "median")
cfsi_sa = cfsi_by_group(panel_fc, "sa_index", "median")
cfsi_ww = cfsi_by_group(panel_fc, "ww_index", "median")
```

91.2.5 Alternative Specifications

The baseline CFSI test has been augmented in several directions:

Dynamic investment models. S. Bond et al. (2003) argue that the static regression Equation 91.8 omits the autoregressive component of investment. The Euler equation approach, which derives directly from the firm's dynamic optimization problem, yields:

$$\frac{I_{i,t}}{K_{i,t-1}} = \gamma_1 \frac{I_{i,t-1}}{K_{i,t-2}} + \gamma_2 \left(\frac{Y_{i,t}}{K_{i,t-1}} \right) + \gamma_3 \frac{CF_{i,t}}{K_{i,t-1}} + \varepsilon_{i,t} \tag{91.12}$$

This specification avoids the need for Q entirely, sidestepping the measurement error problem.

External finance dependence. Rajan and Zingales (1998) propose using the industry-level technological demand for external finance as an instrument for financing constraints. Industries that technologically require more external funding should be disproportionately affected by financial development and firm-level constraints.

Table 91.8: Euler Equation Investment Model

```
euler_results = pd.DataFrame({
    "Coefficient": model_euler.params.round(4),
    "Std Error": model_euler.std_errors.round(4),
    "t-stat": model_euler.tstats.round(3),
    "p-value": model_euler.pvalues.round(4)
})
euler_results
```

```
# Euler equation investment model (dynamic panel)
panel_euler = panel_fc.copy().sort_values(["ticker", "year"])

panel_euler["lag_inv_rate"] =
 ↪  panel_euler.groupby("ticker")["inv_rate"].shift(1)
panel_euler["revenue_assets"] = panel_euler["revenue"] /
 ↪  panel_euler["lag_assets"]

euler_data = panel_euler.dropna(
    subset=["inv_rate", "lag_inv_rate", "revenue_assets", "cf_assets"]
).set_index(["ticker", "year"])

model_euler = PanelOLS(
    euler_data["inv_rate"],
    euler_data[["lag_inv_rate", "revenue_assets", "cf_assets"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

euler_data = euler_data.reset_index()
```

91.3 Financing Choice Models

91.3.1 Capital Structure Determinants

The two dominant theories of capital structure (i.e., trade-off theory and pecking order theory) generate distinct predictions about the determinants of leverage. Frank and Goyal (2009) pro-

vide the most comprehensive empirical synthesis, identifying six "core" variables that reliably predict leverage across samples and specifications.

The baseline capital structure regression is:

$$\text{Lev}_{i,t} = \beta_0 + \beta' \mathbf{X}_{i,t} + \alpha_i + \delta_t + \varepsilon_{i,t} \tag{91.13}$$

where $\text{Lev}_{i,t}$ is either book leverage (D/A) or market leverage ($D/(D + E^{\text{mkt}})$), and $\mathbf{X}_{i,t}$ includes the core determinants.

Table 91.9 summarizes the theoretical predictions.

Table 91.9: Capital Structure Predictions by Theory

Determinant	Trade-Off	Pecking Order	Measurement
Profitability	+ (tax shield)	− (less need for external)	EBITDA / Assets
Size	+ (lower distress costs)	+ (less information asymmetry)	ln(Total Assets)
Tangibility	+ (collateral value)	+ (less adverse selection)	PPE / Assets
Growth (MTB)	− (underinvestment)	+ (financing needs)	Market-to-Book
Industry median leverage	+ (target)	ambiguous	Industry median
Profitability volatility	− (distress risk)	ambiguous	Rolling (EBITDA/A)

```
# Construct capital structure variables
cs = panel_fc.copy()

# Book leverage
cs["book_leverage"] = cs["total_debt"] / cs["total_assets"]

# Market leverage
cs["market_leverage"] = cs["total_debt"] / (
    cs["total_debt"] + cs["market_cap"]
)

# Profitability
cs["profitability"] = cs["ebitda"] / cs["total_assets"]

# Tangibility
```

```
cs["tangibility"] = cs["ppe_net"] / cs["total_assets"]

# Size
cs["size"] = np.log(cs["total_assets"])

# Market-to-Book
cs["mtb"] = cs["market_cap"] / cs["book_equity"]

# Industry median leverage
cs["ind_median_lev"] = cs.groupby(
    ["industry", "year"]
)["book_leverage"].transform("median")

# Rolling profitability volatility (3-year)
cs = cs.sort_values(["ticker", "year"])
cs["profit_vol"] = (
    cs.groupby("ticker")["profitability"]
    .transform(lambda x: x.rolling(3, min_periods=2).std())
)

# Winsorize
for col in ["book_leverage", "market_leverage", "profitability",
            "tangibility", "mtb", "profit_vol"]:
    cs[col] = winsorize(cs[col])

cs_clean = cs.dropna(
    subset=["book_leverage", "profitability", "size",
            "tangibility", "mtb", "ind_median_lev"]
)
```

```
# Capital structure regressions
cs_panel = cs_clean.set_index(["ticker", "year"])

regressors = ["profitability", "size", "tangibility",
              "mtb", "ind_median_lev"]

# Book leverage
model_book = PanelOLS(
    cs_panel["book_leverage"],
    cs_panel[regressors],
    entity_effects=True,
    time_effects=True,
```

Table 91.10: Capital Structure Determinants: Book and Market Leverage

```
cs_table = pd.DataFrame({
    "Book Leverage": [
        f"{model_book.params[v]:.4f} ({model_book.std_errors[v]:.4f})"
        for v in regressors
    ] + [f"{model_book.rsquared_within:.4f}", str(int(model_book.nobs))],
    "Market Leverage": [
        f"{model_mkt.params[v]:.4f} ({model_mkt.std_errors[v]:.4f})"
        for v in regressors
    ] + [f"{model_mkt.rsquared_within:.4f}", str(int(model_mkt.nobs))]
}, index=regressors + ["R² (within)", "N"])

cs_table
```

```
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

# Market leverage
model_mkt = PanelOLS(
    cs_panel["market_leverage"],
    cs_panel[regressors],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

cs_panel = cs_panel.reset_index()
```

91.3.2 Pecking Order Tests

The pecking order theory (Myers 1984) predicts that firms prefer internal finance, then debt, then equity. Shyam-Sunder and Myers (1999) propose a direct test: if the pecking order holds strictly, the financing deficit (investment minus internal funds) should be financed dollar-for-dollar by debt:

$$\Delta D_{i,t} = \alpha + \beta_{\text{PO}} \cdot \text{DEF}_{i,t} + \varepsilon_{i,t} \tag{91.14}$$

where $\text{DEF}_{i,t} = \text{Div}_{i,t} + \text{Capex}_{i,t} + \Delta W_{i,t} - CF_{i,t}$ is the financing deficit and $\Delta D_{i,t}$ is net debt issuance. A strict pecking order implies $\hat{\alpha} = 0$ and $\hat{\beta}_{\text{PO}} = 1$. Frank and Goyal (2003) show that the coefficient is typically well below 1, especially for large firms and equity issuers.

```
# Construct financing deficit
po = cs.copy().sort_values(["ticker", "year"])
po["lag_debt"] = po.groupby("ticker")["total_debt"].shift(1)
po["net_debt_issuance"] = po["total_debt"] - po["lag_debt"]

# Financing deficit = Div + Capex + ΔWC - CF
po["delta_wc"] = po["working_capital"] - po.groupby(
    "ticker"
)["working_capital"].shift(1)

po["fin_deficit"] = (
    po["dividends"] + po["capex"]
    + po["delta_wc"].fillna(0) - po["operating_cf"]
)

# Scale by lagged assets
for col in ["net_debt_issuance", "fin_deficit"]:
    po[col] = po[col] / po["lag_assets"]

po_clean = po.dropna(
    subset=["net_debt_issuance", "fin_deficit"]
)

# Winsorize
for col in ["net_debt_issuance", "fin_deficit"]:
    po_clean[col] = winsorize(po_clean[col])

# Pecking order regression
po_panel = po_clean.set_index(["ticker", "year"])

model_po = PanelOLS(
    po_panel["net_debt_issuance"],
    po_panel[["fin_deficit"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

po_panel = po_panel.reset_index()
```

```
print(f"Pecking order coefficient: {model_po.params['fin_deficit']:.4f}")
print(f"  (se = {model_po.std_errors['fin_deficit']:.4f})")
print(f"  H0:   = 1, t = "
      f"{(model_po.params['fin_deficit'] - 1) /
       ↪  model_po.std_errors['fin_deficit']:.3f}")
```

91.3.3 Market Timing Measures

Baker and Wurgler (2002) argue that capital structure is largely the cumulative outcome of market timing (i.e., firms issue equity when valuations are high and repurchase when valuations are low). Their key variable is the external-finance-weighted average market-to-book ratio:

$$\left(\frac{M}{B}\right)_{i,t}^{efwa} = \sum_{s=\text{IPO}}^{t-1} \frac{e_s + d_s}{\sum_{r=\text{IPO}}^{t-1}(e_r + d_r)} \cdot \left(\frac{M}{B}\right)_{i,s} \tag{91.15}$$

where e_s and d_s are net equity and net debt issuance in year s. This variable captures the historical valuations at which the firm raised capital. The market timing hypothesis predicts that higher $(M/B)^{efwa}$ is associated with lower current leverage (i.e., firms that historically issued equity at high valuations have persistently lower leverage).

```
# External-Finance-Weighted Average M/B
def compute_efwa_mtb(group):
    """Compute Baker-Wurgler EFWA M/B for one firm."""
    g = group.sort_values("year").copy()

    # Net issuance each year
    g["net_equity"] = g["equity_issuance"].fillna(0)
    g["net_debt"] = g["net_debt_issuance"].fillna(0)
    g["total_issuance"] = (
        g["net_equity"].abs() + g["net_debt"].abs()
    ).replace(0, np.nan)

    efwa_values = []
    for idx in range(1, len(g)):
        past = g.iloc[:idx]
        weights = past["total_issuance"] / past["total_issuance"].sum()
        weights = weights.fillna(0)
        efwa = (weights * past["mtb"]).sum()
        efwa_values.append(efwa)
```

```
    g = g.iloc[1:].copy()
    g["efwa_mtb"] = efwa_values
    return g[["ticker", "year", "efwa_mtb"]]

mt = po_clean.copy()
mt["equity_issuance"] = mt["market_cap"] - mt.groupby(
    "ticker"
)["market_cap"].shift(1) - mt["net_income"]

efwa_data = (
    mt.groupby("ticker", group_keys=False)
    .apply(compute_efwa_mtb)
    .reset_index(drop=True)
)

# Merge and regress
mt_merged = cs_clean.merge(efwa_data, on=["ticker", "year"], how="left")
mt_clean = mt_merged.dropna(
    subset=["market_leverage", "efwa_mtb", "profitability",
            "size", "tangibility", "mtb"]
)

mt_panel = mt_clean.set_index(["ticker", "year"])

model_mt = PanelOLS(
    mt_panel["market_leverage"],
    mt_panel[["efwa_mtb", "mtb", "profitability", "size", "tangibility"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

mt_panel = mt_panel.reset_index()
```

A negative coefficient on $\text{EFWA}_{M/B}$ after controlling for the current M/B (which captures current investment opportunities) supports the market timing hypothesis: firms that historically raised capital at high valuations maintain persistently lower leverage.

Table 91.11: Market Timing and Capital Structure

```
mt_results = pd.DataFrame({
    "Coefficient": model_mt.params.round(4),
    "Std Error": model_mt.std_errors.round(4),
    "t-stat": model_mt.tstats.round(3),
    "p-value": model_mt.pvalues.round(4)
})
mt_results
```

91.4 Payout Policy Estimators

91.4.1 Dividend Smoothing

Lintner (1956) established the foundational model of dividend behavior: firms target a payout ratio and partially adjust dividends toward the target each year. The partial adjustment model is:

$$\Delta D_{i,t} = \alpha_i + \lambda(\tau \cdot E_{i,t} - D_{i,t-1}) + \varepsilon_{i,t} \tag{91.16}$$

where $D_{i,t}$ is the dividend per share, $E_{i,t}$ is earnings per share, τ is the target payout ratio, and $\lambda \in (0, 1)$ is the speed of adjustment. Low λ implies strong smoothing (i.e., firms adjust dividends slowly toward the target). Rearranging:

$$D_{i,t} = \alpha_i + (1 - \lambda) D_{i,t-1} + \lambda\tau \cdot E_{i,t} + \varepsilon_{i,t} \tag{91.17}$$

The coefficient on lagged dividends, $(1-\lambda)$, measures the degree of smoothing. Values close to 1 indicate near-complete smoothing; values close to 0 indicate no smoothing (full adjustment).

```
# Construct dividend and earnings variables
div = panel_fc.copy().sort_values(["ticker", "year"])

div["lag_dps"] = div.groupby("ticker")["dividends_per_share"].shift(1)
div["delta_dps"] = div["dividends_per_share"] - div["lag_dps"]

# Only firms with positive dividends in both periods
div_clean = div.dropna(
    subset=["dividends_per_share", "lag_dps", "eps"]
).query("lag_dps > 0 and dividends_per_share > 0")
```

Table 91.12: Lintner Partial Adjustment Model

```
lintner_table = pd.DataFrame({
    "Coefficient": model_lintner.params.round(4),
    "Std Error": model_lintner.std_errors.round(4),
    "t-stat": model_lintner.tstats.round(3),
    "p-value": model_lintner.pvalues.round(4)
})
lintner_table
```

```
# Lintner regression
div_panel = div_clean.set_index(["ticker", "year"])

model_lintner = PanelOLS(
    div_panel["dividends_per_share"],
    div_panel[["lag_dps", "eps"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

div_panel = div_panel.reset_index()

# Extract structural parameters
lambda_hat = 1 - model_lintner.params["lag_dps"]
tau_hat = model_lintner.params["eps"] / lambda_hat

print(f"Lintner Model Estimates:")
print(f"  Speed of adjustment ( ): {lambda_hat:.4f}")
print(f"  Target payout ratio ( ): {tau_hat:.4f}")
print(f"  Smoothing coefficient (1- ): {model_lintner.params['lag_dps']:.4f}")
```

91.4.2 Smoothing Heterogeneity: SOEs vs. Private Firms

Dividend policy in Vietnam is shaped by regulatory mandates. The State Capital Investment Corporation (SCIC) and line ministries have historically required SOEs to distribute minimum dividend amounts, sometimes at the expense of reinvestment. This creates a fundamental

```
payout = panel_fc.copy()
payout["payout_ratio"] = payout["dividends"] / payout["net_income"]
payout = payout[
    (payout["net_income"] > 0) &
    (payout["payout_ratio"].between(0, 2))
]

payout_ts = (
    payout.groupby("year")
    .agg(
        median_payout=("payout_ratio", "median"),
        mean_payout=("payout_ratio", "mean"),
        q25=("payout_ratio", lambda x: x.quantile(0.25)),
        q75=("payout_ratio", lambda x: x.quantile(0.75)),
        pct_payers=("payout_ratio", lambda x: (x > 0).mean())
    )
    .reset_index()
)

(
    p9.ggplot(payout_ts, p9.aes(x="year"))
    + p9.geom_ribbon(
        p9.aes(ymin="q25", ymax="q75"),
        fill="#2E5090", alpha=0.2
    )
    + p9.geom_line(
        p9.aes(y="median_payout"),
        color="#2E5090", size=1
    )
    + p9.geom_line(
        p9.aes(y="mean_payout"),
        color="#C0392B", linetype="dashed", size=0.7
    )
    + p9.labs(
        x="Year",
        y="Dividend Payout Ratio",
        title="Payout Ratio: Median (Solid) and Mean (Dashed)"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 5))
)
```

Figure 91.2

asymmetry: SOE dividends are partially policy-determined rather than the outcome of the Lintner optimization.

```
# Merge SOE indicator
div_with_soe = div_clean.merge(
    ownership[["ticker", "year", "state_ownership_pct"]],
    on=["ticker", "year"],
    how="left"
)
div_with_soe["soe"] = (div_with_soe["state_ownership_pct"] > 50).astype(int)

# Estimate Lintner model separately for SOEs and private firms
lintner_results = {}
for label, soe_val in [("Private", 0), ("SOE", 1)]:
    subset = div_with_soe[div_with_soe["soe"] == soe_val].copy()
    if len(subset) < 100:
        continue

    subset_panel = subset.set_index(["ticker", "year"])
    model = PanelOLS(
        subset_panel["dividends_per_share"],
        subset_panel[["lag_dps", "eps"]],
        entity_effects=True,
        time_effects=True,
        check_rank=False
    ).fit(cov_type="clustered", cluster_entity=True)

    lam = 1 - model.params["lag_dps"]
    tau = model.params["eps"] / lam if abs(lam) > 0.01 else np.nan

    lintner_results[label] = {
        "Smoothing (1- )": round(model.params["lag_dps"], 4),
        "Speed of adj ( )": round(lam, 4),
        "Target payout ( )": round(tau, 4),
        "N": int(model.nobs)
    }

pd.DataFrame(lintner_results).T
```

91.4.3 Share Repurchases

Share repurchases are a relatively new phenomenon in Vietnamese markets, gradually gaining traction as regulations have evolved. Unlike dividends, repurchases are more flexible and do not create expectations of future payments. The decision to repurchase can be modeled as:

$$\text{Repurchase}_{i,t} = \mathbb{1}\left(\beta_0 + \beta_1 \frac{CF_{i,t}}{A_{i,t}} + \beta_2 Q_{i,t} + \beta_3 \text{Lev}_{i,t} + \beta_4 \frac{\text{Cash}_{i,t}}{A_{i,t}} + \gamma' \mathbf{Z}_{i,t} + \varepsilon_{i,t} > 0\right) \tag{91.18}$$

```
# Identify repurchase years
repurchase = panel_fc.copy()
repurchase["repurchase_dummy"] = (
    repurchase["share_repurchases"] > 0
).astype(int)

repurchase["cash_assets"] = repurchase["cash"] / repurchase["total_assets"]

# Probit model for repurchase decision
rep_clean = repurchase.dropna(
    subset=["repurchase_dummy", "cf_assets", "tobins_q",
            "book_leverage", "cash_assets", "log_assets"]
)

probit_model = smf.probit(
    "repurchase_dummy ~ cf_assets + tobins_q + book_leverage "
    "+ cash_assets + log_assets + C(year)",
    data=rep_clean
).fit(disp=False, cov_type="cluster", cov_kwds={"groups":
 ↪  rep_clean["ticker"]})
```

91.4.4 Agency and Signaling Interpretations

Payout policy is interpreted through two competing lenses:

Agency view (M. C. Jensen 1986; La Porta et al. 2000b): Dividends are a mechanism for disgorging free cash flow that managers would otherwise waste on empire-building or perquisite consumption. In this view, firms with weaker governance should face greater pressure to pay dividends as a bonding device. La Porta et al. (2000b) distinguish the "outcome" model (dividends are the result of effective minority shareholder pressure) from the "substitute" model (firms with weak governance pay high dividends to build reputation for fair treatment).

Table 91.13: Probit Model: Determinants of Share Repurchase Decision

```
# Extract non-year-dummy coefficients
main_vars = ["cf_assets", "tobins_q", "book_leverage",
             "cash_assets", "log_assets"]

probit_results = pd.DataFrame({
    "Coefficient": probit_model.params[main_vars].round(4),
    "Std Error": probit_model.bse[main_vars].round(4),
    "z-stat": probit_model.tvalues[main_vars].round(3),
    "p-value": probit_model.pvalues[main_vars].round(4),
    "Marginal Effect": (
        probit_model.get_margeff().margeff[:len(main_vars)]
    ).round(4)
})
probit_results
```

Signaling view (S. Bhattacharya 1979; M. H. Miller and Rock 1985): Dividends convey private information about future earnings. Because dividends are costly to fake (they require actual cash), they serve as a credible signal. The signaling interpretation predicts that dividend changes should predict future earnings changes.

```
# Test dividend signaling: do dividend changes predict future earnings?
signal = panel_fc.copy().sort_values(["ticker", "year"])

signal["delta_div"] = signal.groupby("ticker")["dividends"].diff()
signal["div_increase"] = (signal["delta_div"] > 0).astype(int)
signal["div_decrease"] = (signal["delta_div"] < 0).astype(int)

# Future earnings change
signal["lead_earnings"] = signal.groupby("ticker")["net_income"].shift(-1)
signal["delta_earnings_lead"] = (
    (signal["lead_earnings"] - signal["net_income"]) /
    signal["total_assets"]
)

# Current earnings change (control)
signal["lag_earnings"] = signal.groupby("ticker")["net_income"].shift(1)
signal["delta_earnings_curr"] = (
    (signal["net_income"] - signal["lag_earnings"]) /
    signal["total_assets"]
```

```
)

signal_clean = signal.dropna(
    subset=["delta_earnings_lead", "div_increase",
            "div_decrease", "delta_earnings_curr"]
)

# Regression: future earnings change on dividend change indicators
signal_model = smf.ols(
    "delta_earnings_lead ~ div_increase + div_decrease "
    "+ delta_earnings_curr + C(year) + C(industry)",
    data=signal_clean
).fit(cov_type="cluster", cov_kwds={"groups": signal_clean["ticker"]})

print("Dividend Signaling Test:")
for var in ["div_increase", "div_decrease", "delta_earnings_curr"]:
    print(f"  {var}: {signal_model.params[var]:.4f} "
          f"(t = {signal_model.tvalues[var]:.3f})")
```

91.5 Agency Cost Proxies

91.5.1 Ownership Concentration and Agency Problems

The agency framework of M. C. Jensen and Meckling (2019) identifies the separation of ownership and control as the fundamental source of corporate agency costs. In concentrated-ownership economies like Vietnam, the dominant agency conflict is not between dispersed shareholders and professional managers (Berle-Means agency problem) but between controlling and minority shareholders (principal-principal agency problem, Young et al. (2008)).

The key mechanisms through which controlling shareholders extract private benefits include: tunneling via related-party transactions (S. Johnson et al. 2000), diversion of corporate opportunities, excessive compensation, and dilutive equity issuances. The extent of these costs depends on the ownership structure, legal protections for minorities, and monitoring intensity.

```
# Merge ownership data comprehensively
agency = panel_fc.merge(
    ownership[["ticker", "year", "state_ownership_pct",
               "foreign_ownership_pct", "insider_ownership_pct",
               "largest_shareholder_pct", "top5_shareholder_pct",
               "board_size", "independent_directors_pct",
```

```
                "ceo_duality"]],
    on=["ticker", "year"],
    how="left"
)

# Ownership concentration measures
# Herfindahl of top-5 shareholdings
agency["ownership_hhi"] = agency["top5_shareholder_pct"]**2

# Excess control rights (proxy: difference between
# largest shareholder and second largest)
agency["control_wedge"] = (
    agency["largest_shareholder_pct"] -
    (agency["top5_shareholder_pct"] - agency["largest_shareholder_pct"]) / 4
)
```

91.5.2 Free Cash Flow Measures

M. C. Jensen (1986) argues that the agency cost of free cash flow is the central problem in firms that generate cash in excess of positive-NPV investment opportunities. The standard measure is:

$$\text{FCF}_{i,t} = \frac{\text{Operating CF}_{i,t} - \text{Depreciation}_{i,t} - \text{Required Capex}_{i,t}}{\text{Total Assets}_{i,t}} \tag{91.19}$$

In practice, "required capex" is unobservable, so researchers use operating cash flow minus capital expenditures as a proxy, or add the interaction of cash flow with low Q (which identifies firms with cash flow but without investment opportunities):

$$\text{FCF Overinvestment} = \frac{CF_{i,t}}{A_{i,t}} \times \mathbb{1}(Q_{i,t} < 1) \tag{91.20}$$

```
# Free cash flow measures
agency["fcf"] = (agency["operating_cf"] - agency["capex"]) /
 ↪  agency["total_assets"]

agency["low_q"] = (agency["tobins_q"] < 1).astype(int)
agency["fcf_low_q"] = agency["fcf"] * agency["low_q"]

# Asset utilization (inverse proxy for agency costs)
```

Table 91.14: Summary Statistics: Agency Cost Proxies and Governance Variables

```
agency_vars = [
    "largest_shareholder_pct", "state_ownership_pct",
    "foreign_ownership_pct", "fcf", "fcf_low_q",
    "asset_turnover", "board_independence"
]

agency_summary = (
    agency[agency_vars].dropna()
    .describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9])
    .T.round(4)
)
agency_summary
```

```
agency["asset_turnover"] = agency["revenue"] / agency["total_assets"]

# SGA ratio (proxy for discretionary spending / empire building)
agency["sga_ratio"] = agency["sga_expenses"] / agency["revenue"]
```

91.5.3 Monitoring Mechanisms and Governance Variables

We construct a governance quality composite based on observable monitoring mechanisms:

```
# Governance quality indicators
agency["foreign_monitor"] = (
    agency["foreign_ownership_pct"] > 20
).astype(int)

agency["board_independence"] = agency["independent_directors_pct"]

agency["no_duality"] = (1 - agency["ceo_duality"]).astype(int)

# Related-party transaction intensity (if available)
# agency["rpt_ratio"] = agency["related_party_transactions"] /
 ↪  agency["revenue"]
```

91.5.4 Agency Costs and Firm Value

We test whether agency cost proxies are associated with firm value (Tobin's Q) and operating performance (ROA), controlling for standard determinants:

$$Q_{i,t} = \beta_0 + \beta_1 \text{Own}_{i,t} + \beta_2 \text{Own}_{i,t}^2 + \gamma' \mathbf{X}_{i,t} + \alpha_i + \delta_t + \varepsilon_{i,t} \tag{91.21}$$

The quadratic in ownership captures the Morck, Shleifer, and Vishny (1988) nonlinearity: at low levels, managerial ownership aligns incentives (positive effect on Q); at high levels, entrenchment dominates (negative effect).

```
# Agency cost and valuation regression
agency["largest_sq"] = agency["largest_shareholder_pct"]**2

val_data = agency.dropna(
    subset=["tobins_q", "largest_shareholder_pct", "foreign_ownership_pct",
            "fcf", "size", "profitability", "leverage"]
).copy()

val_panel = val_data.set_index(["ticker", "year"])

model_val = PanelOLS(
    val_panel["tobins_q"],
    val_panel[["largest_shareholder_pct", "largest_sq",
               "foreign_ownership_pct", "fcf",
               "size", "profitability", "leverage"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

val_panel = val_panel.reset_index()
```

The inverted-U pattern, if present, would be consistent with the Morck-Shleifer-Vishny incentive-alignment/entrenchment tradeoff. In Vietnamese markets, the pattern may differ because the dominant controlling shareholder is often the state, whose objective function includes non-value-maximizing goals (employment, regional development, strategic sector control).

```
# Binned scatter: largest shareholder vs Q
own_bins = val_data.copy()
own_bins["own_bin"] = pd.qcut(
    own_bins["largest_shareholder_pct"], q=20, duplicates="drop"
)

own_binned = (
    own_bins.groupby("own_bin", observed=True)
    .agg(
        mean_own=("largest_shareholder_pct", "mean"),
        mean_q=("tobins_q", "mean"),
        se_q=("tobins_q", lambda x: x.std() / np.sqrt(len(x)))
    )
    .reset_index()
)

(
    p9.ggplot(own_binned, p9.aes(x="mean_own", y="mean_q"))
    + p9.geom_pointrange(
        p9.aes(ymin="mean_q - 1.96*se_q",
               ymax="mean_q + 1.96*se_q"),
        color="#2E5090", size=0.5
    )
    + p9.geom_smooth(method="loess", color="#C0392B", se=False, size=0.8)
    + p9.labs(
        x="Largest Shareholder Ownership (%)",
        y="Tobin's Q",
        title="Ownership Concentration and Firm Value"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 6))
)
```

Figure 91.3

Table 91.15: Agency Proxies and Firm Value (Tobin's Q)

```
val_results = pd.DataFrame({
    "Coefficient": model_val.params.round(4),
    "Std Error": model_val.std_errors.round(4),
    "t-stat": model_val.tstats.round(3),
    "p-value": model_val.pvalues.round(4)
})
val_results
```

91.6 Linking Corporate Decisions to Returns

91.6.1 Investment-Based Anomalies

The asset pricing literature has documented that corporate investment decisions predict cross-sectional return differences (i.e., the "investment anomalies"). The theoretical foundation is the q-theory of investment applied to asset pricing (Cochrane 1991; L. X. Liu, Whited, and Zhang 2009): firms invest more when the discount rate on their projects is lower. High investment therefore signals low expected returns.

The investment effect. Titman, Wei, and Xie (2004) and Cooper, Gulen, and Schill (2008) document that firms with high asset growth earn lower subsequent returns. The asset growth variable is:

$$\text{AG}_{i,t} = \frac{A_{i,t} - A_{i,t-1}}{A_{i,t-1}} \tag{91.22}$$

The investment-to-assets effect. Eugene F. Fama and French (2006) and Hou, Xue, and Zhang (2015) show that capital expenditure scaled by assets negatively predicts returns.

The profitability effect. Novy-Marx (2013) shows that gross profitability (revenue minus COGS, scaled by assets) positively predicts returns. This is consistent with q-theory: controlling for investment, more profitable firms must have higher discount rates (otherwise they would invest more).

```
# Construct anomaly variables
anomaly = panel_fc.copy().sort_values(["ticker", "year"])

# Asset growth
anomaly["asset_growth"] = (
    (anomaly["total_assets"] - anomaly["lag_assets"]) /
```

```
    anomaly["lag_assets"]
)

# Investment-to-assets
anomaly["inv_to_assets"] = anomaly["capex"] / anomaly["lag_assets"]

# Gross profitability
anomaly["gross_profit"] = (
    (anomaly["revenue"] - anomaly["cogs"]) / anomaly["total_assets"]
)

# ROE
anomaly["roe"] = anomaly["net_income"] / anomaly["book_equity"]

# Winsorize
for col in ["asset_growth", "inv_to_assets", "gross_profit", "roe"]:
    anomaly[col] = winsorize(anomaly[col])
```

```
# Portfolio sorts: quintiles on asset growth
# Merge with monthly returns (using June rebalancing)
anomaly_june = anomaly.copy()
anomaly_june["formation_year"] = anomaly_june["year"]

# Create quintile assignments
anomaly_june["ag_quintile"] = anomaly_june.groupby("year")[
    "asset_growth"
].transform(lambda x: pd.qcut(x, 5, labels=[1, 2, 3, 4, 5],
                              duplicates="drop"))

# Merge with forward returns
monthly_with_signal = monthly_returns.copy()
monthly_with_signal["formation_year"] = np.where(
    monthly_with_signal["date"].dt.month >= 7,
    monthly_with_signal["date"].dt.year,
    monthly_with_signal["date"].dt.year - 1
)

portfolios = monthly_with_signal.merge(
    anomaly_june[["ticker", "formation_year", "ag_quintile",
                  "asset_growth", "gross_profit"]],
    on=["ticker", "formation_year"],
    how="inner"
```

Table 91.16: Asset Growth Quintile Portfolio Returns

```
quintile_summary = ag_wide.describe().T[["mean", "std"]].copy()
quintile_summary["mean_ann"] = quintile_summary["mean"] * 12
quintile_summary["std_ann"] = quintile_summary["std"] * np.sqrt(12)
quintile_summary["sharpe"] = (
    quintile_summary["mean_ann"] / quintile_summary["std_ann"]
)

# t-statistics
for col in ag_wide.columns:
    t_stat = ag_wide[col].mean() / (ag_wide[col].std() /
 ↪  np.sqrt(len(ag_wide)))
    quintile_summary.loc[col, "t_stat"] = t_stat

quintile_summary = quintile_summary[
    ["mean_ann", "std_ann", "sharpe", "t_stat"]
].round(4)

quintile_summary.columns = [
    "Ann. Return", "Ann. Vol", "Sharpe Ratio", "t-stat"
]
quintile_summary
```

```
)

# Compute equal-weighted quintile returns
ag_returns = (
    portfolios.groupby(["date", "ag_quintile"])
    .agg(port_ret=("ret", "mean"))
    .reset_index()
)

# Long-short: Q1 (low growth) - Q5 (high growth)
ag_wide = ag_returns.pivot(
    index="date", columns="ag_quintile", values="port_ret"
)
ag_wide["L-S"] = ag_wide[1] - ag_wide[5]
```

```
cumret = ag_wide[["L-S"]].copy()
cumret.columns = ["Long-Short"]
cumret = cumret.dropna()
cumret["cumulative"] = (1 + cumret["Long-Short"]).cumprod()
cumret = cumret.reset_index()

(
    p9.ggplot(cumret, p9.aes(x="date", y="cumulative"))
    + p9.geom_line(color="#2E5090", size=0.8)
    + p9.geom_hline(yintercept=1, linetype="dashed", color="gray")
    + p9.labs(
        x="",
        y="Cumulative Return (Growth of $1)",
        title="Investment Anomaly: Low - High Asset Growth"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 5))
)
```

Figure 91.4

91.6.2 Financing Anomalies

Firms' financing decisions also predict returns. Pontiff and Woodgate (2008) document that net stock issuance negatively predicts returns: firms that issue equity earn lower future returns, while firms that repurchase shares earn higher returns. This is consistent with both managerial market timing and an issuance-based risk factor.

The net stock issuance variable is typically measured as:

$$\text{NSI}_{i,t} = \ln\left(\frac{\text{Split-Adjusted Shares}_{i,t}}{\text{Split-Adjusted Shares}_{i,t-1}}\right) \tag{91.23}$$

Positive NSI indicates net equity issuance; negative NSI indicates net repurchases.

```
# Net Stock Issuance
fin_anomaly = anomaly.copy()
fin_anomaly["lag_shares"] = fin_anomaly.groupby(
    "ticker"
)["shares_outstanding"].shift(1)
```

```
fin_anomaly["nsi"] = np.log(
    fin_anomaly["shares_outstanding"] / fin_anomaly["lag_shares"]
)
fin_anomaly["nsi"] = winsorize(fin_anomaly["nsi"])

# Net debt issuance (change in total debt / assets)
fin_anomaly["ndi"] = (
    (fin_anomaly["total_debt"] -
     fin_anomaly.groupby("ticker")["total_debt"].shift(1)) /
    fin_anomaly["lag_assets"]
)
fin_anomaly["ndi"] = winsorize(fin_anomaly["ndi"])

# Portfolio sorts on NSI
fin_anomaly["nsi_quintile"] = fin_anomaly.groupby("year")[
    "nsi"
].transform(lambda x: pd.qcut(x, 5, labels=[1, 2, 3, 4, 5],
                              duplicates="drop"))

port_nsi = monthly_with_signal.merge(
    fin_anomaly[["ticker", "formation_year", "nsi_quintile"]],
    on=["ticker", "formation_year"],
    how="inner"
)

nsi_returns = (
    port_nsi.groupby(["date", "nsi_quintile"])
    .agg(port_ret=("ret", "mean"))
    .reset_index()
)

nsi_wide = nsi_returns.pivot(
    index="date", columns="nsi_quintile", values="port_ret"
)
nsi_wide["L-S"] = nsi_wide[1] - nsi_wide[5]
```

91.6.3 Valuation Implications: Fama-French Factor Regressions

We evaluate whether the investment and financing anomalies represent compensation for systematic risk by regressing the long-short portfolios on standard factor models:

Table 91.17: Net Stock Issuance Quintile Portfolio Returns

```
nsi_summary = nsi_wide.describe().T[["mean", "std"]].copy()
nsi_summary["mean_ann"] = nsi_summary["mean"] * 12
nsi_summary["sharpe"] = (
    nsi_summary["mean_ann"] /
    (nsi_summary["std"] * np.sqrt(12))
)

for col in nsi_wide.columns:
    t_stat = nsi_wide[col].mean() / (
        nsi_wide[col].std() / np.sqrt(len(nsi_wide))
    )
    nsi_summary.loc[col, "t_stat"] = t_stat

nsi_summary = nsi_summary[["mean_ann", "sharpe", "t_stat"]].round(4)
nsi_summary.columns = ["Ann. Return", "Sharpe", "t-stat"]
nsi_summary
```

$$R_{p,t} - R_{f,t} = \alpha + \beta_{\text{MKT}} \text{MKT}_t + \beta_{\text{SMB}} \text{SMB}_t + \beta_{\text{HML}} \text{HML}_t + \varepsilon_t \tag{91.24}$$

Significant positive α after controlling for known risk factors would indicate that the anomaly is not explained by size and value exposures.

```
# Merge long-short returns with factor data
factor_data = factors.set_index("date")

# Asset growth anomaly alpha
ag_ls = ag_wide[["L-S"]].dropna().rename(columns={"L-S": "excess_ret"})
ag_merged = ag_ls.join(factor_data[["mkt_rf", "smb", "hml"]], how="inner")

model_ag_ff3 = sm.OLS(
    ag_merged["excess_ret"],
    sm.add_constant(ag_merged[["mkt_rf", "smb", "hml"]])
).fit(cov_type="HAC", cov_kwds={"maxlags": 6})

# NSI anomaly alpha
nsi_ls = nsi_wide[["L-S"]].dropna().rename(columns={"L-S": "excess_ret"})
nsi_merged = nsi_ls.join(factor_data[["mkt_rf", "smb", "hml"]], how="inner")
```

Table 91.18: Fama-French Three-Factor Alphas for Corporate Decision Anomalies

```
alpha_table = pd.DataFrame({
    "Asset Growth L-S": {
        "Alpha (monthly)": f"{model_ag_ff3.params['const']:.4f}",
        "  t-stat": f"{model_ag_ff3.tvalues['const']:.3f}",
        "MKT": f"{model_ag_ff3.params['mkt_rf']:.4f}",
        "SMB": f"{model_ag_ff3.params['smb']:.4f}",
        "HML": f"{model_ag_ff3.params['hml']:.4f}",
        "R²": f"{model_ag_ff3.rsquared:.4f}"
    },
    "Net Issuance L-S": {
        "Alpha (monthly)": f"{model_nsi_ff3.params['const']:.4f}",
        "  t-stat": f"{model_nsi_ff3.tvalues['const']:.3f}",
        "MKT": f"{model_nsi_ff3.params['mkt_rf']:.4f}",
        "SMB": f"{model_nsi_ff3.params['smb']:.4f}",
        "HML": f"{model_nsi_ff3.params['hml']:.4f}",
        "R²": f"{model_nsi_ff3.rsquared:.4f}"
    }
})
alpha_table
```

```
model_nsi_ff3 = sm.OLS(
    nsi_merged["excess_ret"],
    sm.add_constant(nsi_merged[["mkt_rf", "smb", "hml"]])
).fit(cov_type="HAC", cov_kwds={"maxlags": 6})
```

91.7 Summary

This chapter implemented the core econometric toolkit of empirical corporate finance for Vietnamese listed firms. The estimators span four interconnected domains: investment decisions (investment-Q regressions and their measurement-error-corrected variants), financing decisions (capital structure determinants, pecking order tests, and market timing measures), payout policy (Lintner smoothing, repurchase models, and dividend signaling tests), and agency costs (ownership-value relationships, free cash flow measures, and governance variables).

Several findings deserve emphasis. The investment-Q relationship in Vietnam is attenuated relative to developed-market benchmarks, reflecting both the severity of measurement error in Q (thin trading, price limits, volatile inflation) and the prevalence of non-market-driven

```
# Combine asset growth and NSI long-short for comparison
combined = pd.DataFrame({
    "Asset Growth": ag_wide["L-S"],
    "Net Issuance": nsi_wide["L-S"]
}).dropna()

combined_cum = (1 + combined).cumprod().reset_index()
combined_long = combined_cum.melt(
    id_vars="date",
    var_name="Anomaly",
    value_name="Cumulative Return"
)

(
    p9.ggplot(combined_long, p9.aes(
        x="date", y="Cumulative Return", color="Anomaly"
    ))
    + p9.geom_line(size=0.8)
    + p9.geom_hline(yintercept=1, linetype="dashed", color="gray")
    + p9.scale_color_manual(values=["#2E5090", "#C0392B"])
    + p9.labs(
        x="",
        y="Cumulative Return (Growth of $1)",
        title="Investment vs. Financing Anomalies: Long-Short Portfolios"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 5), legend_position="top")
)
```

Figure 91.5

investment by SOEs. Cash flow remains a significant predictor of investment across constraint classifications, though the FHP-KZ debate about interpretation applies with full force. Capital structure is strongly predicted by profitability (negatively, consistent with the pecking order) and tangibility (positively, consistent with trade-off theory). Dividend smoothing is pronounced, but the smoothing parameter differs systematically between SOEs and private firms, reflecting the distinct institutional forces governing each group's payout policy.

The chapter also linked corporate decisions to asset returns through portfolio sorts on asset growth and net stock issuance. Whether these anomalies survive risk adjustment and persist out of sample in Vietnamese markets is an important open question for future research.

92 Structural Models in Finance

Most of empirical finance is reduced-form: regress returns on characteristics, estimate risk premia from factor portfolios, test whether a coefficient is significantly different from zero. Structural estimation takes a fundamentally different approach. It begins with an explicit economic model (i.e., specifying agents' preferences, information sets, constraints, and optimization problems) and estimates the model's primitive parameters directly from observed data. The payoff is the ability to perform counterfactual analysis: what would happen to prices if trading costs fell by half? How would IPO underpricing change if the allocation mechanism switched from book building to a uniform-price auction? What is the welfare cost of a particular market design choice? These questions are unanswerable within a reduced-form framework because they require knowledge of the data-generating process, not merely statistical associations within a single regime.

This chapter introduces the key structural estimation frameworks used in financial economics and implements them for Vietnamese equity markets. We cover four domains where structural models have proven most productive: investor demand estimation, trading cost and execution models, primary market auctions, and limit order book dynamics. Each domain involves distinct economic primitives such as preferences, beliefs, transaction costs, information asymmetries and each requires domain-specific identification strategies.

Vietnamese markets offer both challenges and opportunities for structural estimation. On the challenge side, data quality is uneven, market microstructure is evolving, and institutional features (price limits, foreign ownership caps, state ownership) introduce constraints that standard models do not accommodate. On the opportunity side, several features of Vietnamese markets create natural variation that aids identification: the coexistence of two exchanges (HOSE and HNX) with different trading rules, regulatory changes that shift trading costs and price limits, and the phased liberalization of foreign ownership caps that generates exogenous demand shocks.

```
import pandas as pd
import numpy as np
from scipy import stats, optimize
from scipy.optimize import minimize, minimize_scalar
import statsmodels.api as sm
import statsmodels.formula.api as smf
from linearmodels.panel import PanelOLS
from linearmodels.iv import IV2SLS
```

```
import plotnine as p9
from mizani.formatters import percent_format, comma_format
import warnings
warnings.filterwarnings("ignore")
```

92.1 What Structural Estimation Means in Finance

92.1.1 Reduced Form Versus Structural

Consider two researchers studying the effect of transaction costs on trading volume. The reduced-form researcher exploits a fee reduction event and estimates:

$$\ln V_{i,t} = \alpha + \beta \cdot \text{Post}_t + \gamma \mathbf{X}_{i,t} + \varepsilon_{i,t} \tag{92.1}$$

The coefficient β identifies the causal effect of the fee change on volume, but it is local to this specific fee change, this specific market, and this specific time period. It says nothing about what a different fee change would do, or what the optimal fee structure might be.

The structural researcher writes down an explicit model of trader behavior:

$$\max_{q_t} \; E_t \left[\sum_{s=t}^{T} \delta^{s-t} \left(v_s q_s - c(q_s; \theta) \right) \right] \tag{92.2}$$

where v_s is the (possibly private) valuation, q_s is the trade quantity, $c(\cdot; \theta)$ is the cost function parameterized by θ, and δ is the discount factor. The researcher estimates θ by requiring that the model's predictions match observed trading patterns. With $\hat{\theta}$ in hand, any counterfactual cost function $c'(\cdot; \theta')$ can be fed into the model to predict the equilibrium response.

The distinction is not about sophistication (reduced-form work can be highly rigorous) but about the type of question being answered:

Table 92.1: Reduced Form vs. Structural Estimation

	Reduced Form	Structural
Answers	"What happened?"	"What would happen if…?"
Identification	Exogenous variation (events, instruments)	Model restrictions + data moments
Key assumption	Unconfoundedness or exclusion restriction	Correct model specification

	Reduced Form	Structural
Output	Treatment effects, associations	Primitive parameters, counterfactuals
Risk	Omitted variable bias, weak instruments	Model misspecification

92.1.2 Identification Through Economic Primitives

Structural identification relies on the economic model to convert observed outcomes into unobserved primitives. The classic example is the demand-supply system. Observing prices and quantities alone cannot identify demand and supply curves (the simultaneity problem). But if we know the functional form of demand and supply, and have instruments that shift one curve but not the other, the system is identified.

In financial contexts, identification often comes from the model's equilibrium conditions. In Kyle (1985), the informed trader's strategy, the market maker's pricing rule, and the noise trader's demand jointly determine the equilibrium price impact coefficient λ. The model maps the observable (i.e., the price impact of order flow) to the unobservable (i.e., the precision of private information). The identification is through the model's structure, not through a conventional instrument.

Formally, let $\theta \in \Theta$ denote the vector of structural parameters and $\mathbf{m}(\theta)$ the model-implied moments. The data provide empirical moments $\hat{\mathbf{m}}$. Identification requires that the mapping $\theta \mapsto \mathbf{m}(\theta)$ is injective: different parameter values produce different observable implications.

$$\hat{\theta} = \arg\min_{\theta \in \Theta} \left[\hat{\mathbf{m}} - \mathbf{m}(\theta)\right]' \mathbf{W} \left[\hat{\mathbf{m}} - \mathbf{m}(\theta)\right] \tag{92.3}$$

This is the Generalized Method of Moments (GMM) framework applied to structural estimation. The choice of weighting matrix $\mathbf{W}$ determines efficiency, and over-identifying restrictions (more moments than parameters) provide specification tests.

92.1.3 Tradeoffs Between Realism and Tractability

Every structural model involves choices about which features of reality to include and which to abstract from. Table 92.2 illustrates this for several canonical finance models.

Table 92.2: Structural Model Tradeoffs

Model	Key Simplification	What It Misses	What It Gains
Kyle (1985)	Single informed trader, normal distributions	Multiple insiders, fat tails	Closed-form λ, clean identification
Glosten and Milgrom (1985)	Binary signal, competitive market makers	Complex information, inventory costs	Bid-ask spread decomposition
Koijen and Yogo (2019)	Characteristics-based demand, no dynamics	Dynamic portfolio rebalancing	Demand elasticity estimation at scale
Roll (1984)	No information, serial independence	Information-driven trades	Spread estimation from return autocovariance

The researcher's task is to choose the simplest model that captures the economic mechanism of interest while remaining rich enough that its counterfactual predictions are credible. As Rust (1987) emphasized, there is a tension between models that are "structurally correct" but computationally intractable and models that are tractable but potentially misspecified.

92.2 Demand Estimation for Financial Assets

92.2.1 The Demand System Approach

Koijen and Yogo (2019) (hereafter KY) develop a demand system for financial assets that estimates the elasticity of institutional investor demand with respect to asset characteristics. The framework adapts the discrete-choice demand estimation methodology of Berry, Levinsohn, and Pakes (1995) from industrial organization to financial markets.

Each investor i holds a portfolio of N assets. The demand for asset j by investor i at time t is:

$$w_{ij,t} = \frac{\exp(\delta_{j,t} + \beta_i' \mathbf{x}_{j,t})}{1 + \sum_{k=1}^{N} \exp(\delta_{k,t} + \beta_i' \mathbf{x}_{k,t})} \tag{92.4}$$

where $w_{ij,t}$ is the portfolio weight of asset j for investor i, $\delta_{j,t}$ is the mean utility (common valuation), $\mathbf{x}_{j,t}$ is a vector of observable characteristics (market cap, book-to-market, momentum, etc.), and β_i captures investor-specific preferences (heterogeneous demand elasticities). The denominator includes 1 to represent the outside option (holding cash or non-equity assets).

The key insight is that the market-clearing condition links demand to equilibrium prices:

$$\sum_{i} A_{i,t} \cdot w_{ij,t}(\theta) = \text{ME}_{j,t} \qquad \forall j, t \tag{92.5}$$

where $A_{i,t}$ is investor i's total assets under management and $\text{ME}_{j,t}$ is the market capitalization of asset j. This system of equations determines the equilibrium price impacts and demand elasticities.

92.2.2 Demand Elasticity Estimation

The demand elasticity of asset j with respect to its own price (or a characteristic that moves its price) is:

$$\varepsilon_{jj} = \frac{\partial \ln w_{ij}}{\partial \ln P_j} = \beta_{\text{price}} \cdot (1 - w_{ij}) \tag{92.6}$$

In a frictionless market with perfectly elastic demand, a supply shock (e.g., an index addition) would have zero price impact. Empirically, demand curves for financial assets slope downward with finite elasticity, implying that supply shocks move equilibrium prices.

```
# DataCore.vn API
from datacore import DataCore
dc = DataCore()

# Load institutional holdings data
holdings = dc.get_institutional_holdings(
    start_date="2012-01-01",
    end_date="2024-12-31"
)

# Load firm characteristics
firm_chars = dc.get_firm_characteristics(
    start_date="2012-01-01",
    end_date="2024-12-31"
)

# Load market data
market_data = dc.get_daily_returns(
    start_date="2012-01-01",
    end_date="2024-12-31"
)
```

```
print(f"Holdings observations: {len(holdings)}")
print(f"Unique institutions: {holdings['institution_id'].nunique()}")
print(f"Unique stocks: {holdings['ticker'].nunique()}")
```

```
# Construct demand system variables
demand = holdings.merge(
    firm_chars[["ticker", "date", "market_cap", "book_to_market",
                "momentum_12m", "log_size", "profitability",
                "investment", "industry"]],
    on=["ticker", "date"],
    how="inner"
)

# Portfolio weights
demand["total_aum"] = demand.groupby(
    ["institution_id", "date"]
)["holding_value"].transform("sum")

demand["port_weight"] = demand["holding_value"] / demand["total_aum"]

# Log portfolio weight relative to outside option
# w_ij / w_i0 where w_i0 = 1 - sum(w_ij) for equity holdings
demand["sum_equity_weight"] = demand.groupby(
    ["institution_id", "date"]
)["port_weight"].transform("sum")

demand["outside_weight"] = 1 - demand["sum_equity_weight"].clip(upper=0.99)

demand["log_weight_ratio"] = np.log(
    demand["port_weight"] / demand["outside_weight"]
)
```

```
# Simplified KY demand estimation
# log(w_ij / w_i0) = delta_j + beta_i' x_j + epsilon_ij
# First stage: estimate mean utility delta_j via OLS with asset FE

# Cross-sectional regression at each date
def estimate_demand_cross_section(group):
    """
    Estimate demand parameters for a single cross-section.
```

```
    Uses OLS with stock fixed effects absorbed.
    """
    g = group.dropna(subset=[
        "log_weight_ratio", "log_size", "book_to_market",
        "momentum_12m", "profitability"
    ])

    if len(g) < 50:
        return pd.Series(dtype=float)

    X = g[["log_size", "book_to_market", "momentum_12m",
           "profitability"]].values
    X = sm.add_constant(X)
    y = g["log_weight_ratio"].values

    try:
        model = sm.OLS(y, X).fit()
        return pd.Series({
            "beta_size": model.params[1],
            "beta_btm": model.params[2],
            "beta_mom": model.params[3],
            "beta_prof": model.params[4],
            "r_squared": model.rsquared,
            "n_obs": len(g)
        })
    except Exception:
        return pd.Series(dtype=float)

# Estimate by institution type
demand["institution_type"] = demand["institution_type"].fillna("Other")

demand_params = (
    demand.groupby(["institution_type", "date"])
    .apply(estimate_demand_cross_section)
    .reset_index()
    .dropna()
)
```

Table 92.3: Demand Elasticities by Investor Type

```
avg_params = (
    demand_params.groupby("institution_type")
    .agg(
        beta_size=("beta_size", "mean"),
        beta_btm=("beta_btm", "mean"),
        beta_mom=("beta_mom", "mean"),
        beta_prof=("beta_prof", "mean"),
        avg_r2=("r_squared", "mean"),
        n_periods=("date", "nunique")
    )
    .round(4)
)

avg_params
```

```
top_types =
 ↪  demand_params.groupby("institution_type").size().nlargest(4).index
plot_data = demand_params[
    demand_params["institution_type"].isin(top_types)
].copy()

(
    p9.ggplot(plot_data, p9.aes(
        x="date", y="beta_size", color="institution_type"
    ))
    + p9.geom_smooth(method="lowess", se=False, size=1)
    + p9.labs(
        x="", y="  (Size)",
        title="Institutional Demand Sensitivity to Firm Size",
        color="Institution Type"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 5), legend_position="top")
)
```

Figure 92.1

92.2.3 Price Pressure and Market Impact

The demand system framework directly yields price impact predictions. If investor i receives an exogenous inflow ΔA_i (e.g., from a fund flow shock), the price of each stock must adjust to clear the market with the new demand. The equilibrium price impact of a demand shock is:

$$\frac{\Delta P_j}{P_j} = \frac{1}{\varepsilon_{jj}} \cdot \frac{\Delta D_j}{D_j} \tag{92.7}$$

where ε_{jj} is the demand elasticity and $\Delta D_j / D_j$ is the percentage demand shock. Less elastic demand implies larger price impact for the same demand shock (a prediction with direct implications for the price impact of index rebalancing, fund flows, and forced selling).

```
# Estimate aggregate demand elasticity via index rebalancing events
# Use VN30 index reconstitutions as demand shocks

index_changes = dc.get_index_changes(
    index="VN30",
    start_date="2012-01-01",
    end_date="2024-12-31"
)

# Event study: abnormal returns around index additions/deletions
def event_study_car(events, returns, window=(-5, 20)):
    """
    Compute cumulative abnormal returns around events.

    Parameters
    ----------
    events : DataFrame
        Columns: ticker, event_date, event_type (addition/deletion).
    returns : DataFrame
        Columns: ticker, date, ret, mkt_ret.

    Returns
    -------
    DataFrame : CARs by event type and event day.
    """
    results = []

    for _, event in events.iterrows():
```

```
        ticker = event["ticker"]
        event_date = pd.to_datetime(event["event_date"])

        # Estimation window: -260 to -11
        stock_rets = returns[returns["ticker"] == ticker].copy()
        stock_rets = stock_rets.sort_values("date")

        est_mask = (
            (stock_rets["date"] >= event_date - pd.Timedelta(days=365)) &
            (stock_rets["date"] < event_date - pd.Timedelta(days=15))
        )
        est_data = stock_rets[est_mask].dropna(subset=["ret", "mkt_ret"])

        if len(est_data) < 60:
            continue

        # Market model
        model = sm.OLS(
            est_data["ret"],
            sm.add_constant(est_data["mkt_ret"])
        ).fit()

        # Event window
        evt_mask = (
            (stock_rets["date"] >= event_date + pd.Timedelta(days=window[0] *
↪  1.5)) &
            (stock_rets["date"] <= event_date + pd.Timedelta(days=window[1] *
↪  1.5))
        )
        evt_data = stock_rets[evt_mask].dropna(subset=["ret", "mkt_ret"])

        if len(evt_data) < 5:
            continue

        # Abnormal returns
        evt_data = evt_data.copy()
        evt_data["expected"] = model.predict(
            sm.add_constant(evt_data["mkt_ret"])
        )
        evt_data["ar"] = evt_data["ret"] - evt_data["expected"]

        # Assign event-time index
        trading_dates = evt_data["date"].sort_values().values
```

```
        event_idx = np.searchsorted(trading_dates, event_date)
        evt_data["event_day"] = range(-event_idx, len(evt_data) - event_idx)

        evt_data["event_type"] = event["event_type"]
        evt_data["ticker"] = ticker
        results.append(evt_data[["ticker", "event_day", "ar", "event_type"]])

    if not results:
        return pd.DataFrame()

    return pd.concat(results, ignore_index=True)

# Compute event study
daily_data = market_data.merge(
    dc.get_market_returns(
        start_date="2012-01-01",
        end_date="2024-12-31",
        frequency="daily"
    )[["date", "mkt_ret"]],
    on="date", how="left"
)

car_results = event_study_car(index_changes, daily_data)
```

The permanent component of the CAR measures the information content of index inclusion, while the temporary component (reversal after the event) measures pure price pressure from demand. In a world with perfectly elastic demand, there would be no temporary price impact. The magnitude of the temporary component is inversely related to the demand elasticity, providing a second identification strategy (complementary to the holdings-based approach) for demand curves.

92.2.4 Demand Inelasticity and Its Implications

Gabaix and Koijen (2021) argue that the aggregate demand for equities is remarkably inelastic, with elasticity estimates on the order of 0.2 (meaning a 1% supply increase requires a 5% price decline to clear). The implications are profound: if demand is this inelastic, then flows (from index funds, foreign investors, retail traders) have outsized effects on prices. In Vietnamese markets, where foreign ownership caps create binding constraints on a key investor class, demand inelasticity may be even more severe.

The inelasticity also generates a role for "the market portfolio" as an equilibrium concept: if most investors hold portfolios close to the market (as in CAPM), then deviations from market

```
if len(car_results) > 0:
    # Average AR by event day and type
    avg_ar = (
        car_results.groupby(["event_type", "event_day"])
        .agg(
            mean_ar=("ar", "mean"),
            se_ar=("ar", lambda x: x.std() / np.sqrt(len(x))),
            n=("ar", "count")
        )
        .reset_index()
    )

    # Cumulative AR
    for etype in avg_ar["event_type"].unique():
        mask = avg_ar["event_type"] == etype
        avg_ar.loc[mask, "car"] = avg_ar.loc[mask, "mean_ar"].cumsum()

    avg_ar = avg_ar[avg_ar["event_day"].between(-5, 20)]

    (
        p9.ggplot(avg_ar, p9.aes(
            x="event_day", y="car", color="event_type"
        ))
        + p9.geom_line(size=1)
        + p9.geom_vline(xintercept=0, linetype="dashed", color="gray")
        + p9.geom_hline(yintercept=0, linetype="dotted", color="gray")
        + p9.scale_color_manual(values=["#27AE60", "#C0392B"])
        + p9.labs(
            x="Event Day",
            y="Cumulative Abnormal Return",
            title="Price Pressure from VN30 Index Additions and Deletions",
            color="Event"
        )
        + p9.theme_minimal()
        + p9.theme(figure_size=(10, 6))
    )
```

Figure 92.2

weights require someone to absorb the excess supply, and the compensation required is the inverse of demand elasticity.

92.3 Structural Models of Trading Strategies

92.3.1 Optimal Execution: The Almgren-Chriss Framework

The optimal execution problem, formalized by Almgren and Chriss (2001), asks: given a large order to execute over a fixed horizon, what is the optimal trading schedule that minimizes the total execution cost? The problem is fundamental to institutional investing because large orders cannot be executed instantaneously without severe price impact.

Let X_t denote the remaining shares to sell at time t, with $X_0 = X$ (total order) and $X_T = 0$ (completion). The trading rate is $n_t = X_{t-1} - X_t$. The execution price for shares traded at t is affected by both temporary and permanent price impact:

$$S_t = S_0 + \sigma W_t - g(n_t) - h\left(\frac{n_t}{\tau}\right) \tag{92.8}$$

where $g(\cdot)$ is the permanent impact function, $h(\cdot)$ is the temporary impact function, σ is the volatility, W_t is a Wiener process, and τ is the time interval. The total implementation shortfall (cost of execution relative to the arrival price S_0) is:

$$\text{IS} = \sum_{t=1}^{T} n_t(S_0 - S_t) = \sum_{t=1}^{T} n_t \left[\sigma W_t + g(n_t) + h\left(\frac{n_t}{\tau}\right)\right] \tag{92.9}$$

The trader minimizes a mean-variance objective:

$$\min_{\{n_t\}} \; E[\text{IS}] + \lambda \cdot \text{Var}(\text{IS}) \tag{92.10}$$

where λ is the risk aversion parameter. With linear impact functions $g(n) = \gamma n$ and $h(v) = \eta v + \epsilon \text{sgn}(v)$, the optimal strategy has a closed-form solution.

TWAP (Time-Weighted Average Price): Trade uniformly: $n_t = X/T$ for all t. Optimal when permanent impact dominates temporary impact.

VWAP (Volume-Weighted Average Price): Trade proportionally to expected volume: $n_t \propto \hat{V}_t$. Approximates TWAP adjusted for intraday volume patterns.

Almgren-Chriss Optimal: The risk-averse optimal trajectory for linear impact is:

$$x_t^* = X \cdot \frac{\sinh(\kappa(T-t))}{\sinh(\kappa T)} \tag{92.11}$$

where $\kappa = \sqrt{\lambda \sigma^2 / \eta}$ governs the trade-off between urgency (trading quickly to reduce risk) and patience (trading slowly to reduce impact). High risk aversion (λ large) implies front-loaded execution.

```
def almgren_chriss_trajectory(X, T, sigma, eta, gamma, lam, n_steps=100):
    """
    Compute Almgren-Chriss optimal execution trajectory.

    Parameters
    ----------
    X : float
        Total shares to execute.
    T : float
        Execution horizon (in trading days).
    sigma : float
        Daily return volatility.
    eta : float
        Temporary impact parameter.
    gamma : float
        Permanent impact parameter.
    lam : float
        Risk aversion parameter.
    n_steps : int
        Number of time steps.

    Returns
    -------
    DataFrame : Time, remaining shares, trading rate, expected cost.
    """
    tau = T / n_steps
    kappa_sq = lam * sigma**2 / (eta / tau)

    if kappa_sq <= 0:
        # Risk-neutral: trade uniformly
        kappa = 0
        x_t = np.linspace(X, 0, n_steps + 1)
    else:
        kappa = np.sqrt(kappa_sq)
        t_grid = np.linspace(0, T, n_steps + 1)
```

```
        x_t = X * np.sinh(kappa * (T - t_grid)) / np.sinh(kappa * T)

    # Trading rates
    n_t = -np.diff(x_t)

    # Expected cost components
    perm_cost = gamma * np.sum(n_t**2)
    temp_cost = (eta / tau) * np.sum(n_t**2)
    total_expected_cost = perm_cost + temp_cost

    # Variance of cost
    var_cost = sigma**2 * tau * np.sum(x_t[:-1]**2)

    results = pd.DataFrame({
        "time": np.linspace(0, T, n_steps + 1)[:-1],
        "remaining_shares": x_t[:-1],
        "trade_rate": n_t
    })

    results.attrs["expected_cost"] = total_expected_cost
    results.attrs["cost_variance"] = var_cost
    results.attrs["kappa"] = kappa if kappa_sq > 0 else 0

    return results
```

92.3.2 Estimating Market Impact from Transaction Data

The structural parameters (γ, η, σ) must be estimated from data. The standard approach uses the square-root law of market impact, documented empirically by Kyle (1985) and Hasbrouck (1991) and derived theoretically by Gabaix et al. (2003):

$$\Delta P / P = c \cdot \text{sgn}(Q) \cdot |Q/V|^{\delta} \tag{92.12}$$

where Q is the signed order flow, V is daily volume, c is the impact coefficient, and $\delta \approx 0.5$ is the impact exponent. The "square root" refers to $\delta = 0.5$, which is remarkably robust across markets, time periods, and asset classes.

```
# Load intraday transaction data (or daily order flow proxy)
order_flow = dc.get_order_flow(
    start_date="2018-01-01",
```

```
# Estimate market impact parameters from Vietnamese data
# Typical values for mid-cap Vietnamese stocks
sigma_daily = 0.025  # 2.5% daily vol
eta = 2.5e-7         # Temporary impact
gamma = 1.0e-7       # Permanent impact
X_shares = 500000    # 500k shares to sell
T_days = 5           # 5-day horizon

trajectories = []
for lam, label in [(0, "Risk Neutral (TWAP)"),
                   (1e-6, "Low Risk Aversion"),
                   (1e-5, "Medium Risk Aversion"),
                   (1e-4, "High Risk Aversion")]:
    traj = almgren_chriss_trajectory(
        X_shares, T_days, sigma_daily, eta, gamma, lam
    )
    traj["strategy"] = label
    traj["pct_remaining"] = traj["remaining_shares"] / X_shares * 100
    trajectories.append(traj)

traj_all = pd.concat(trajectories, ignore_index=True)

(
    p9.ggplot(traj_all, p9.aes(
        x="time", y="pct_remaining", color="strategy"
    ))
    + p9.geom_line(size=1)
    + p9.scale_color_manual(
        values=["#95A5A6", "#27AE60", "#2E5090", "#C0392B"]
    )
    + p9.labs(
        x="Time (Trading Days)",
        y="Remaining Order (%)",
        title="Almgren-Chriss Optimal Execution Trajectories",
        color="Strategy"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 6), legend_position="top")
)
```

Figure 92.3

```
    end_date="2024-12-31",
    frequency="daily"
)

# Construct signed order flow using Lee-Ready classification
# (buy-initiated minus sell-initiated volume)
impact_data = order_flow.merge(
    dc.get_daily_returns(
        start_date="2018-01-01",
        end_date="2024-12-31"
    )[["ticker", "date", "ret", "volume", "market_cap"]],
    on=["ticker", "date"],
    how="inner"
)

# Normalize order flow
impact_data["oib"] = impact_data["net_buy_volume"] / impact_data["volume"]
impact_data["abs_oib"] = np.abs(impact_data["oib"])
impact_data["sign_oib"] = np.sign(impact_data["oib"])

# Log-log regression: ln|ΔP| =   +   ln|Q/V| +
impact_data["ln_abs_ret"] =
 ↪  np.log(np.abs(impact_data["ret"]).clip(lower=1e-8))
impact_data["ln_abs_oib"] = np.log(impact_data["abs_oib"].clip(lower=1e-8))

# Estimate by size quintile
impact_data["size_quintile"] = impact_data.groupby("date")[
    "market_cap"
].transform(lambda x: pd.qcut(x, 5, labels=[1, 2, 3, 4, 5],
                              duplicates="drop"))

impact_results = []
for q in range(1, 6):
    subset = impact_data[impact_data["size_quintile"] == q].dropna(
        subset=["ln_abs_ret", "ln_abs_oib"]
    )
    if len(subset) < 500:
        continue

    model = sm.OLS(
        subset["ln_abs_ret"],
        sm.add_constant(subset["ln_abs_oib"])
    ).fit(cov_type="HC1")
```

Table 92.4: Market Impact Exponent by Firm Size Quintile

```
impact_df.round(4)
```

```
    impact_results.append({
        "size_quintile": q,
        "delta_hat": model.params.iloc[1],
        "delta_se": model.bse.iloc[1],
        "intercept": model.params.iloc[0],
        "r_squared": model.rsquared,
        "n_obs": int(model.nobs)
    })

impact_df = pd.DataFrame(impact_results)
```

The impact exponent $\hat{\delta}$ near 0.5 across size quintiles confirms the universality of the square-root law. Smaller firms typically exhibit higher impact coefficients (larger c), consistent with lower liquidity.

92.3.3 Transaction Cost Decomposition

Total transaction costs comprise multiple components, each with distinct economic content:

$$\text{TC} = \underbrace{\frac{s}{2}}_{\text{Half-spread}} + \underbrace{\gamma \cdot Q}_{\text{Permanent impact}} + \underbrace{\eta \cdot \frac{Q}{V}}_{\text{Temporary impact}} + \underbrace{\sigma\sqrt{T}}_{\text{Timing risk}} \tag{92.13}$$

We estimate each component separately, following the Hasbrouck (2009) methodology for effective spread decomposition.

```
# Effective spread estimation
spreads = dc.get_bid_ask_data(
    start_date="2020-01-01",
    end_date="2024-12-31"
)

# Quoted spread
spreads["quoted_spread"] = (
```

```
    (spreads["ask"] - spreads["bid"]) / spreads["midpoint"]
)

# Effective spread (from transaction prices)
spreads["effective_spread"] = (
    2 * np.abs(spreads["trade_price"] - spreads["midpoint"]) /
    spreads["midpoint"]
)

# Roll (1984) implied spread from return autocovariance
def roll_spread(returns):
    """
    Estimate the Roll (1984) bid-ask spread.
    Spread = 2 * sqrt(-Cov(r_t, r_{t-1})) if covariance is negative.
    """
    cov = np.cov(returns[1:], returns[:-1])[0, 1]
    if cov < 0:
        return 2 * np.sqrt(-cov)
    else:
        return np.nan

# Compute by stock-month
daily_rets = dc.get_daily_returns(
    start_date="2020-01-01",
    end_date="2024-12-31"
)
daily_rets["month"] = daily_rets["date"].dt.to_period("M")

roll_spreads = (
    daily_rets.groupby(["ticker", "month"])
    .agg(
        roll_spread=("ret", lambda x: roll_spread(x.values)
                     if len(x) > 10 else np.nan),
        n_days=("ret", "count"),
        avg_volume=("volume", "mean"),
        market_cap=("market_cap", "last")
    )
    .reset_index()
    .dropna(subset=["roll_spread"])
)
```

```
# Bin by market cap deciles
roll_spreads["size_decile"] = roll_spreads.groupby("month")[
    "market_cap"
].transform(lambda x: pd.qcut(x, 10, labels=range(1, 11),
                              duplicates="drop"))

spread_by_size = (
    roll_spreads.groupby("size_decile")
    .agg(
        median_spread=("roll_spread", "median"),
        mean_spread=("roll_spread", "mean"),
        q25=("roll_spread", lambda x: x.quantile(0.25)),
        q75=("roll_spread", lambda x: x.quantile(0.75))
    )
    .reset_index()
)

(
    p9.ggplot(spread_by_size, p9.aes(x="size_decile", y="median_spread"))
    + p9.geom_bar(stat="identity", fill="#2E5090", alpha=0.7)
    + p9.geom_errorbar(
        p9.aes(ymin="q25", ymax="q75"),
        width=0.3, color="#2E5090"
    )
    + p9.labs(
        x="Size Decile (1 = Smallest)",
        y="Roll Implied Spread",
        title="Transaction Costs Decrease Monotonically with Firm Size"
    )
    + p9.scale_y_continuous(labels=percent_format())
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 5))
)
```

Figure 92.4

92.4 Auction Models in Primary Markets

92.4.1 The IPO Allocation Problem

Initial public offerings present a classic information asymmetry problem. Issuers and underwriters do not know the true market-clearing price; informed investors do (approximately). The allocation mechanism (i.e., how shares are distributed and at what price) determines the IPO's pricing efficiency, the degree of underpricing, and the distribution of surplus between issuers and investors.

Rock (1986) provides the canonical model. There are two types of investors: informed (who know the true value v) and uninformed (who know only the distribution $v \sim F$). If the IPO is priced at P:

- When $v > P$ (good IPO): both informed and uninformed subscribe, so uninformed receive only a fraction of their order (rationing).
- When $v < P$ (bad IPO): only uninformed subscribe, so they receive full allocation (the "winner's curse").

The uninformed investor's expected return, accounting for rationing, is:

$$E[R_{\text{uninformed}}] = \alpha_g \cdot E\left[\frac{v - P}{P} \mid v > P\right] \cdot \Pr(v > P) + E\left[\frac{v - P}{P} \mid v \leq P\right] \cdot \Pr(v \leq P) \quad (92.14)$$

where $\alpha_g < 1$ is the allocation probability in good IPOs (rationed) and allocation is 1 in bad IPOs. For uninformed investors to participate, $E[R_{\text{uninformed}}] \geq 0$, which requires:

$$E\left[\frac{v - P}{P}\right] > 0 \quad (92.15)$$

That is, IPOs must be underpriced on average to compensate uninformed investors for the winner's curse. The degree of required underpricing increases with the proportion of informed investors and the variance of the true value.

92.4.2 Book Building vs. Auction Mechanisms

Vietnamese IPO history provides variation in allocation mechanisms. State-owned enterprise equitizations have used Dutch auctions, while private-sector IPOs have used book building. This institutional variation allows structural comparison of the mechanisms.

Book Building (Benveniste and Spindt 1989): The underwriter solicits indications of interest from institutional investors during the roadshow. Investors who reveal positive information

(high valuations) receive favorable allocations as compensation. The mechanism aggregates information efficiently but grants discretion to the underwriter.

Uniform-Price Auction: All winning bidders pay the same market-clearing price. This eliminates the underwriter's allocation discretion but may lead to free-riding on information revelation.

```
# Load IPO data
ipo_data = dc.get_ipo_data(
    start_date="2005-01-01",
    end_date="2024-12-31"
)

# Compute first-day returns (underpricing)
ipo_data["underpricing"] = (
    (ipo_data["first_day_close"] - ipo_data["offer_price"]) /
    ipo_data["offer_price"]
)

# Classify mechanism
ipo_data["mechanism"] = ipo_data["ipo_method"].map({
    "auction": "Auction",
    "book_building": "Book Building",
    "fixed_price": "Fixed Price"
})

print(f"Total IPOs: {len(ipo_data)}")
print(f"By mechanism:\n{ipo_data['mechanism'].value_counts()}")
```

92.4.3 Structural Estimation of Information Asymmetry

We estimate the Rock (1986) model parameters (i.e., the fraction of informed investors (μ) and the precision of their information (σ_v^2)) using the method of simulated moments (MSM).

The model predicts two key moments:

1 . Average underpricing: $E[U] = f(\mu, \sigma_v^2)$ 2. Cross-sectional variance of underpricing: $\text{Var}(U) = g(\mu, \sigma_v^2)$

We match these model-implied moments to the data.

Table 92.5: IPO Underpricing by Allocation Mechanism

```
ipo_summary = (
    ipo_data.groupby("mechanism")
    .agg(
        n_ipos=("underpricing", "count"),
        mean_underpricing=("underpricing", "mean"),
        median_underpricing=("underpricing", "median"),
        std_underpricing=("underpricing", "std"),
        pct_positive=("underpricing", lambda x: (x > 0).mean()),
        mean_proceeds=("proceeds_bn_vnd", "mean")
    )
    .round(4)
)
ipo_summary
```

```
def rock_model_moments(params, n_sim=10000):
    """
    Simulate Rock (1986) IPO model and compute moments.

    Parameters
    ----------
    params : tuple
        (mu, sigma_v) - fraction of informed investors,
        std dev of true value.

    Returns
    -------
    tuple : (mean underpricing, variance of underpricing).
    """
    mu, sigma_v = params
    np.random.seed(42)

    # True values
    v = np.random.lognormal(mean=0, sigma=sigma_v, size=n_sim)

    # Offer price: set to break even for uninformed
    # Simplified: P = E[v] * discount_factor
    P = np.exp(sigma_v**2 / 2) * 0.9  # 10% average discount

    # First-day returns
```

```
    returns = (v - P) / P

    # Allocation probability for uninformed in good IPOs
    # Depends on mu: more informed -> more rationing
    good_mask = v > P
    alpha_g = (1 - mu) / 1.0  # Simplified rationing

    # Uninformed realized returns
    uninformed_returns = np.where(
        good_mask,
        alpha_g * returns,
        returns
    )

    mean_u = uninformed_returns.mean()
    var_u = uninformed_returns.var()

    return mean_u, var_u


def rock_model_objective(params, target_moments, weight_matrix=None):
    """
    GMM objective for Rock model estimation.
    """
    mu, sigma_v = params

    if mu <= 0 or mu >= 1 or sigma_v <= 0 or sigma_v > 2:
        return 1e10

    model_moments = rock_model_moments(params)
    diff = np.array(model_moments) - np.array(target_moments)

    if weight_matrix is None:
        weight_matrix = np.eye(len(diff))

    return diff @ weight_matrix @ diff


# Target moments from data
target_mean = ipo_data["underpricing"].mean()
target_var = ipo_data["underpricing"].var()

# Estimate
```

```
result = minimize(
    rock_model_objective,
    x0=[0.3, 0.5],
    args=([target_mean, target_var],),
    method="Nelder-Mead",
    options={"maxiter": 5000}
)

mu_hat, sigma_v_hat = result.x
print(f"Rock Model Estimates:")
print(f"  Fraction informed ( ): {mu_hat:.4f}")
print(f"  Value uncertainty ( _v): {sigma_v_hat:.4f}")
print(f"  Objective value: {result.fun:.6f}")
```

92.4.4 Counterfactual: Welfare Under Alternative Mechanisms

With the structural parameters $(\hat{\mu}, \hat{\sigma}_v)$ in hand, we can simulate counterfactual outcomes. For instance: if all Vietnamese IPOs used uniform-price auctions instead of book building, how would underpricing and welfare change?

```
def simulate_ipo_mechanism(mu, sigma_v, mechanism="book_building",
                           n_sim=50000):
    """
    Simulate IPO outcomes under different mechanisms.

    Returns issuer surplus, informed profit, uninformed profit.
    """
    np.random.seed(42)
    v = np.random.lognormal(mean=0, sigma=sigma_v, size=n_sim)

    if mechanism == "book_building":
        # Price partially reveals information
        # P = E[v] + 0.5 * (v - E[v]) * signal_quality
        signal = v + np.random.normal(0, sigma_v * 0.5, n_sim)
        P = np.exp(sigma_v**2 / 2) + 0.3 * (signal - np.exp(sigma_v**2 / 2))
        P = P.clip(min=0.1)

    elif mechanism == "auction":
        # Competitive bidding: P closer to v
        noise = np.random.normal(0, sigma_v * 0.3, n_sim)
        P = v + noise
```

```
ipo_plot = ipo_data.dropna(subset=["underpricing", "mechanism"]).copy()
ipo_plot["year"] = pd.to_datetime(ipo_plot["ipo_date"]).dt.year

annual_underpricing = (
    ipo_plot.groupby(["year", "mechanism"])
    .agg(
        mean_up=("underpricing", "mean"),
        n=("underpricing", "count")
    )
    .reset_index()
    .query("n >= 3")
)

(
    p9.ggplot(annual_underpricing, p9.aes(
        x="year", y="mean_up", color="mechanism"
    ))
    + p9.geom_line(size=1)
    + p9.geom_point(p9.aes(size="n"))
    + p9.geom_hline(yintercept=0, linetype="dashed", color="gray")
    + p9.scale_color_manual(values=["#2E5090", "#C0392B", "#27AE60"])
    + p9.scale_y_continuous(labels=percent_format())
    + p9.labs(
        x="Year",
        y="Average First-Day Return",
        title="IPO Underpricing by Allocation Mechanism",
        color="Mechanism", size="# IPOs"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 6))
)
```

Figure 92.5

```
        P = P.clip(min=0.1)

    elif mechanism == "fixed_price":
        # Fixed at E[v] * discount
        P = np.full(n_sim, np.exp(sigma_v**2 / 2) * 0.85)

    underpricing = (v - P) / P

    issuer_surplus = P  # Revenue per share
    investor_surplus = v - P  # Profit per share

    return {
        "mechanism": mechanism,
        "mean_underpricing": underpricing.mean(),
        "median_underpricing": np.median(underpricing),
        "std_underpricing": underpricing.std(),
        "issuer_revenue": issuer_surplus.mean(),
        "investor_profit": investor_surplus.mean(),
        "money_left": (investor_surplus[investor_surplus > 0]).sum() / n_sim
    }

# Compare mechanisms
counterfactual = pd.DataFrame([
    simulate_ipo_mechanism(mu_hat, sigma_v_hat, "book_building"),
    simulate_ipo_mechanism(mu_hat, sigma_v_hat, "auction"),
    simulate_ipo_mechanism(mu_hat, sigma_v_hat, "fixed_price")
]).set_index("mechanism").round(4)

counterfactual
```

92.5 Limit Order Book Models

92.5.1 Order Submission as a Strategic Choice

In a limit order market, every trader faces a fundamental tradeoff: submit a market order (immediate execution, but at an adverse price) or a limit order (better price, but risk of non-execution). Parlour (1998) models this as a sequential game where each trader's optimal strategy depends on the current state of the order book.

Let a_t and b_t denote the best ask and bid prices at time t, with the spread $s_t = a_t - b_t$. A buyer who arrives at time t chooses between:

- **Market buy**: Execute immediately at a_t. Cost: $a_t - v_t$ where v_t is the true value.
- **Limit buy at** b_t: Provides liquidity. If executed, profit: $v_t - b_t$. Probability of execution: $\pi(b_t, \text{book state})$.

The buyer submits a market order if:

$$a_t - v_t < (1 - \pi_t)(v_t - b_t) + \pi_t \cdot 0 \tag{92.16}$$

Rearranging: market orders are optimal when the spread is narrow relative to the non-execution risk of limit orders. This generates the empirically observed pattern that limit orders are more attractive when spreads are wide and the book is thin.

92.5.2 The Glosten-Milgrom Model

Glosten and Milgrom (1985) provide the foundational structural model of bid-ask spread determination under asymmetric information. A competitive market maker sets bid and ask prices to break even in expectation, recognizing that some trades come from informed traders.

Let μ denote the probability that an incoming order is from an informed trader, and let V^H and V^L denote the high and low values of the asset ($\Pr(V = V^H) = p$). The zero-profit conditions yield:

$$a = E[V \mid \text{buy order}] = \frac{p(1-\mu) + p\mu}{(1-\mu) + p\mu} V^H + \frac{(1-p)(1-\mu)}{(1-\mu) + p\mu} V^L \tag{92.17}$$

$$b = E[V \mid \text{sell order}] = \frac{p(1-\mu)}{(1-\mu) + (1-p)\mu} V^H + \frac{(1-p)(1-\mu) + (1-p)\mu}{(1-\mu) + (1-p)\mu} V^L \tag{92.18}$$

The spread $s = a - b$ is positive whenever $\mu > 0$, and increases in both the proportion of informed traders (μ) and the information asymmetry ($V^H - V^L$). This decomposition is foundational: the spread compensates liquidity providers for the adverse selection cost of trading with informed counterparties.

92.5.3 PIN: Probability of Informed Trading

Easley et al. (1996) extend the Glosten-Milgrom framework to a dynamic setting and develop the Probability of Informed Trading (PIN) measure, which can be estimated from trade data. The model assumes that on each trading day, an information event occurs with probability α. Conditional on an event, it is bad news with probability δ. Informed traders arrive at rate μ; uninformed buyers and sellers arrive at rates ε_b and ε_s, respectively.

The likelihood of observing B_t buys and S_t sells on day t is:

$$\mathcal{L}(B_t, S_t|\theta) = (1-\alpha) f(B_t|\varepsilon_b) f(S_t|\varepsilon_s) + \alpha\delta \cdot f(B_t|\varepsilon_b) f(S_t|\varepsilon_s + \mu) + \alpha(1-\delta) \cdot f(B_t|\varepsilon_b + \mu) f(S_t|\varepsilon_s) \tag{92.19}$$

where $f(\cdot|\lambda)$ is the Poisson density with rate λ, and $\theta = (\alpha, \delta, \mu, \varepsilon_b, \varepsilon_s)$. PIN is then:

$$\text{PIN} = \frac{\alpha\mu}{\alpha\mu + \varepsilon_b + \varepsilon_s} \tag{92.20}$$

```
def pin_log_likelihood(params, data):
    """
    Compute negative log-likelihood for the Easley-Kiefer-O'Hara-Paperman
    (1996) PIN model.

    Parameters
    ----------
    params : array
        [alpha, delta, mu, eps_b, eps_s]
    data : DataFrame
        Columns: buys, sells (daily counts).

    Returns
    -------
    float : Negative log-likelihood.
    """
    alpha, delta, mu, eps_b, eps_s = params

    # Parameter bounds
    if (alpha < 0 or alpha > 1 or delta < 0 or delta > 1 or
        mu < 0 or eps_b < 0 or eps_s < 0):
        return 1e15

    B = data["buys"].values
    S = data["sells"].values

    # Use log-sum-exp for numerical stability
    # Three components: no event, bad news, good news
    log_L = np.zeros(len(B))

    for i in range(len(B)):
        b, s = B[i], S[i]
```

```
        # Log-Poisson components
        def log_poisson(k, lam):
            if lam <= 0:
                return -1e10 if k > 0 else 0
            return k * np.log(lam) - lam - np.sum(np.log(np.arange(1, k +
             ↪  1)))

        # No event
        c1 = (np.log(1 - alpha + 1e-15) +
              log_poisson(b, eps_b) + log_poisson(s, eps_s))

        # Bad news (extra sells)
        c2 = (np.log(alpha * delta + 1e-15) +
              log_poisson(b, eps_b) + log_poisson(s, eps_s + mu))

        # Good news (extra buys)
        c3 = (np.log(alpha * (1 - delta) + 1e-15) +
              log_poisson(b, eps_b + mu) + log_poisson(s, eps_s))

        # Log-sum-exp
        max_c = max(c1, c2, c3)
        log_L[i] = max_c + np.log(
            np.exp(c1 - max_c) + np.exp(c2 - max_c) + np.exp(c3 - max_c)
        )

    return -np.sum(log_L)


def estimate_pin(buys, sells, n_starts=10):
    """
    Estimate PIN via MLE with multiple random starts.

    Returns
    -------
    dict : Estimated parameters and PIN value.
    """
    data = pd.DataFrame({"buys": buys, "sells": sells})
    best_result = None
    best_nll = np.inf

    avg_b = data["buys"].mean()
    avg_s = data["sells"].mean()
```

```
    for _ in range(n_starts):
        # Random initial values
        alpha0 = np.random.uniform(0.1, 0.8)
        delta0 = np.random.uniform(0.2, 0.8)
        mu0 = np.random.uniform(0, max(avg_b, avg_s))
        eps_b0 = avg_b * np.random.uniform(0.5, 1.5)
        eps_s0 = avg_s * np.random.uniform(0.5, 1.5)

        try:
            result = minimize(
                pin_log_likelihood,
                x0=[alpha0, delta0, mu0, eps_b0, eps_s0],
                args=(data,),
                method="L-BFGS-B",
                bounds=[(0.01, 0.99), (0.01, 0.99),
                        (0.01, None), (0.01, None), (0.01, None)],
                options={"maxiter": 1000}
            )

            if result.fun < best_nll:
                best_nll = result.fun
                best_result = result
        except Exception:
            continue

    if best_result is None:
        return {"pin": np.nan}

    alpha, delta, mu, eps_b, eps_s = best_result.x
    pin = alpha * mu / (alpha * mu + eps_b + eps_s)

    return {
        "alpha": alpha,
        "delta": delta,
        "mu": mu,
        "eps_b": eps_b,
        "eps_s": eps_s,
        "pin": pin,
        "log_likelihood": -best_nll
    }
```

```
# Estimate PIN for a cross-section of stocks
# Using daily buy/sell counts from Lee-Ready classification
trade_counts = dc.get_trade_counts(
    start_date="2023-01-01",
    end_date="2023-12-31"
)

# Sample of stocks for estimation (PIN is computationally intensive)
top_stocks = (
    trade_counts.groupby("ticker")
    .agg(n_days=("date", "nunique"))
    .query("n_days >= 200")
    .index[:50]
)

pin_estimates = []
for ticker in top_stocks:
    stock_data = trade_counts[trade_counts["ticker"] == ticker]
    result = estimate_pin(
        stock_data["buys"].values,
        stock_data["sells"].values,
        n_starts=5
    )
    result["ticker"] = ticker
    pin_estimates.append(result)

pin_df = pd.DataFrame(pin_estimates).dropna(subset=["pin"])
print(f"PIN estimates for {len(pin_df)} stocks")
print(f"  Mean PIN: {pin_df['pin'].mean():.4f}")
print(f"  Median PIN: {pin_df['pin'].median():.4f}")
```

92.5.4 PIN and Asset Pricing

Easley, Hvidkjaer, and O'hara (2002) argue that PIN should be priced in the cross-section of expected returns: stocks with higher information asymmetry require a risk premium to compensate uninformed investors. We test this prediction using Fama-MacBeth regressions.

```
# Merge PIN with firm characteristics and forward returns
pin_chars = pin_df[["ticker", "pin"]].merge(
    dc.get_firm_characteristics(
        start_date="2023-01-01",
```

```
(
    p9.ggplot(pin_df, p9.aes(x="pin"))
    + p9.geom_histogram(bins=25, fill="#2E5090", alpha=0.7)
    + p9.geom_vline(
        xintercept=pin_df["pin"].median(),
        linetype="dashed", color="#C0392B", size=0.8
    )
    + p9.labs(
        x="PIN",
        y="Count",
        title="Distribution of Informed Trading Probability"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 5))
)
```

Figure 92.6

```
        end_date="2023-12-31"
    ).groupby("ticker").last().reset_index()[
        ["ticker", "log_size", "book_to_market", "momentum_12m"]
    ],
    on="ticker",
    how="inner"
)

# Forward 12-month returns (2024)
forward_returns = dc.get_annual_returns(
    start_date="2024-01-01",
    end_date="2024-12-31"
)

pin_returns = pin_chars.merge(
    forward_returns[["ticker", "annual_ret"]],
    on="ticker",
    how="inner"
)

# Cross-sectional regression
if len(pin_returns) > 20:
    cs_model = sm.OLS(
```

```
        pin_returns["annual_ret"],
        sm.add_constant(
            pin_returns[["pin", "log_size", "book_to_market",
↪   "momentum_12m"]]
        )
    ).fit(cov_type="HC1")

    print("Cross-Sectional Return Regression with PIN:")
    for var in ["pin", "log_size", "book_to_market", "momentum_12m"]:
        print(f"  {var}: {cs_model.params[var]:.4f} "
              f"(t = {cs_model.tvalues[var]:.3f})")
```

92.5.5 Estimation Challenges in Limit Order Book Models

Structural estimation of limit order book models faces several challenges specific to Vietnamese markets:

Tick size effects. Vietnamese exchanges use discrete tick sizes that vary with price level. At low prices, the minimum tick size represents a large fraction of the price, artificially widening the spread and distorting the structural parameters. The standard Glosten-Milgrom and PIN models assume continuous prices; adapting them to discrete price grids requires the modifications proposed by Bollen, Smith, and Whaley (2004).

Price limits. When a stock hits its price limit, the order book is effectively frozen at one side. Orders accumulate but cannot execute until the limit is lifted. This creates a censoring problem analogous to the price limit censoring in return distributions: the observed order flow is a truncated version of the latent flow.

Missing data. Full order book snapshots at high frequency are not universally available for Vietnamese stocks. PIN estimation requires only daily buy/sell counts, making it feasible with lower-frequency data. But richer models (Foucault (1999) dynamic limit order book, Roşu (2009) continuous-time model) require tick-by-tick order book data that may be unavailable for smaller stocks.

Institutional features. The coexistence of HOSE (continuous auction) and HNX (with periodic call auctions for less liquid stocks) creates heterogeneity in the appropriate structural model. A model estimated on HOSE continuous trading data does not directly apply to HNX call auction stocks.

92.6 When Structural Models Are Worth It

92.6.1 Decision Framework

Structural estimation is not always the right tool. The additional complexity, data requirements, and model risk must be justified by the research question. Table 92.6 provides a decision framework.

Table 92.6: When to Choose Structural vs. Reduced Form

Criterion	Structural Preferred	Reduced Form Preferred
Research question	Counterfactual or policy evaluation	Causal effect of observed variation
Data availability	Rich micro-data (transactions, holdings)	Standard panel (returns, fundamentals)
Institutional environment	Stable (model assumptions plausible)	Rapidly changing (model may be misspecified)
Number of parameters	Few primitives, many observables	Many parameters, few identifying assumptions
Publication venue	JF, RFS, Econometrica, JPE	JFE, RFS, MS, JFQA
Computational budget	Weeks to months of estimation	Hours to days

92.6.2 Data Requirements

Structural models are data-hungry. Table 92.7 summarizes the minimum data needs for each model class covered in this chapter.

Table 92.7: Data Requirements for Structural Estimation

Model	Minimum Data	Ideal Data	Vietnamese Availability
KY demand system	Quarterly institutional holdings + prices	13F-equivalent filings + fund flows	Partial (semi-annual disclosure)
Almgren-Chriss	Daily volume, spread, volatility	Intraday order-by-order data	Good (HOSE provides)
Rock/IPO	Offer price, first-day close, allocation	Investor-level bids and allocations	Limited (auction data available)

Model	Minimum Data	Ideal Data	Vietnamese Availability
PIN	Daily buy/sell counts (Lee-Ready)	Tick-by-tick trade and quote	Moderate (requires trade classification)
Glosten-Milgrom	Best bid/ask, trade direction	Full order book snapshots	Moderate (top-of-book available)

92.6.3 Computational Cost

Structural estimation is orders of magnitude more expensive than reduced-form regression. The cost arises from three sources:

Likelihood evaluation. Each evaluation of the structural likelihood or moment function requires solving the model for given parameters. For dynamic models (e.g., inventory models, dynamic discrete choice), this involves solving a dynamic programming problem at each iteration.

Optimization. The GMM or MLE objective is typically non-convex, requiring multiple starting points and global optimization algorithms. The PIN model with 5 parameters requires ~ 10 random starts; a richer model with ~ 20 parameters might require hundreds.

Inference. Standard errors for structural parameters often require bootstrapping (because the asymptotic distribution is non-standard or the delta method is unreliable), adding a multiplicative factor of 200–1000 to the computational budget.

```
# Illustration: computational cost comparison
import time

# Reduced form: OLS regression
n_obs = 100000
X_sim = np.random.randn(n_obs, 5)
y_sim = X_sim @ np.random.randn(5) + np.random.randn(n_obs)

start = time.time()
for _ in range(100):
    sm.OLS(y_sim, sm.add_constant(X_sim)).fit()
ols_time = (time.time() - start) / 100

# Structural: PIN estimation (single stock)
sim_buys = np.random.poisson(50, 250)
sim_sells = np.random.poisson(45, 250)
```

```
start = time.time()
pin_result = estimate_pin(sim_buys, sim_sells, n_starts=5)
pin_time = time.time() - start

print(f"Computational Cost Comparison:")
print(f"  OLS (100k obs): {ols_time*1000:.1f} ms per estimation")
print(f"  PIN (250 days, 5 starts): {pin_time:.1f} s per stock")
print(f"  Ratio: {pin_time / ols_time:.0f}x")
```

```
Computational Cost Comparison:
  OLS (100k obs): 41.6 ms per estimation
  PIN (250 days, 5 starts): 17.0 s per stock
  Ratio: 408x
```

92.6.4 Interpretation Risks

The primary risk of structural estimation is model misspecification. If the model is wrong, the estimated parameters have no economic meaning and the counterfactual predictions are unreliable. Several strategies mitigate this risk:

Specification tests. Over-identifying restrictions (more moments than parameters) provide the Hansen J-test for model fit. A rejection suggests misspecification, though the test has low power in small samples.

Model comparison. Estimate multiple nested or non-nested models and compare fit. If a simpler model fits the data equally well, prefer it (Occam's razor).

Sensitivity analysis. Report how structural parameters change under plausible alternative assumptions (e.g., different functional forms for the impact function, different distributional assumptions for valuations).

Out-of-sample validation. Estimate the model on one sample period and test its predictions on a holdout sample. Structural models that fit in-sample but fail out-of-sample are likely overfit.

92.6.5 Journal Expectations

Structural papers in top finance journals are held to specific standards:

Identification clarity. The paper must clearly state which parameters are identified, from which moments, and what variation in the data provides identification. The Rust (1987) critique that without clear identification, structural estimation is curve-fitting, must be addressed head-on.

Counterfactual credibility. Counterfactual exercises should be economically motivated and the range of counterfactual scenarios should not extrapolate far beyond the data.

Robustness to specification. Reviewers will ask what happens under alternative distributional assumptions, alternative moment conditions, and alternative model features.

Transparency. Code and data should be made available. Structural estimation is sufficiently complex that replicability is a first-order concern.

92.7 Summary

This chapter introduced structural estimation as a distinct methodology for financial economics, one that trades the simplicity and transparency of reduced-form analysis for the ability to estimate economic primitives and conduct counterfactual policy evaluation. We implemented four classes of structural models: demand systems for financial assets, optimal execution and transaction cost models, IPO auction models, and limit order book models of informed trading.

The demand system approach of Koijen and Yogo (2019) reveals that institutional investors in Vietnam exhibit heterogeneous demand elasticities across characteristics, with foreign investors showing different sensitivity patterns than domestic institutions. Market impact estimation confirms the universality of the square-root law, while the Almgren-Chriss framework provides the optimal execution benchmark for large institutional orders. The Rock IPO model quantifies the information asymmetry premium embedded in Vietnamese IPO underpricing, and the PIN model estimates the probability of informed trading across the cross-section.

Each model involves a tradeoff between the economic questions it can answer and the assumptions it requires. The decision to use structural estimation should be driven by the research question (specifically, whether counterfactual analysis is essential) and tempered by honest assessment of whether the model's assumptions are sufficiently credible in the Vietnamese institutional environment. When they are, structural models provide insights that no amount of reduced-form regression can deliver.

Part VII

Alternative Data and AI for Finance

93 Textual Analysis

Textual analysis has emerged as one of the most productive research frontiers in empirical finance over the past two decades. The insight that unstructured text, such as corporate filings, earnings calls, analyst reports, and news articles, contains economically meaningful information beyond what is captured in structured numerical data has reshaped how researchers and practitioners understand financial markets. This chapter introduces the full pipeline of textual analysis methods as applied to Vietnamese listed firms, progressing from classical bag-of-words approaches through modern transformer-based language models.

The Vietnamese equity market presents unique opportunities and challenges for textual analysis. As of 2024, the Ho Chi Minh Stock Exchange (HOSE) and the Hanoi Stock Exchange (HNX) together list over 1,600 securities with a combined market capitalization exceeding VND 6,000 trillion (approximately USD 240 billion). Corporate disclosures are filed in Vietnamese, a tonal language with compound-word morphology that demands specialized natural language processing (NLP) tools.

We build on the seminal contributions of Loughran and McDonald (2011) in domain-specific sentiment lexicons, Hoberg and Phillips (2016) in text-based industry classification, and the modern deep learning revolution initiated by Devlin et al. (2019). This chapter covers the following topics:

1. Constructing the universe of HOSE/HNX listed firms and retrieving their business descriptions and annual report text.
2. Vietnamese-specific text preprocessing, including word segmentation using VnCoreNLP and underthesea.
3. Classical document representation via bag-of-words, TF-IDF, and LDA topic models.
4. Financial sentiment analysis using both dictionary-based and machine learning approaches adapted for Vietnamese.
5. Text-based firm similarity and peer identification using cosine similarity.
6. Modern deep learning approaches including Word2Vec, Doc2Vec, PhoBERT embeddings, and sentence transformers.
7. Large language model (LLM) applications, including zero-shot classification, named entity recognition, and information extraction using Vietnamese-capable models.
8. Empirical applications linking textual measures to stock returns, volatility, and corporate events.

93.1 Why Textual Analysis for Vietnamese Finance?

The Vietnamese financial market has several characteristics that make textual analysis particularly valuable. First, analyst coverage is sparse (fewer than 30% of listed firms receive regular coverage from sell-side analysts), making alternative information sources critical. Second, the regulatory environment is evolving rapidly, with the State Securities Commission (SSC) continuously updating disclosure requirements, creating rich variation in information environments across firms and time. Third, the market is dominated by retail investors (accounting for roughly 80% of trading volume), who may process textual information differently than institutional investors, creating potential mispricings that text-based strategies could exploit.

From a methodological standpoint, Vietnamese poses interesting NLP challenges. Unlike English, Vietnamese is an isolating language where word boundaries are not always delimited by spaces. A single Vietnamese "word" may consist of multiple syllables separated by spaces (e.g., "công ty" for "company," "thị trường" for "market"). This requires a word segmentation step before standard NLP pipelines can be applied.[1]

[1]Vietnamese text requires specialized tokenization due to compound words (e.g., "công ty" = company, "thị trường" = market).

94 Literature Review

94.1 Textual Analysis in Finance

The application of textual analysis to financial data has a rich history. Tetlock (2007) demonstrated that the pessimism content of a Wall Street Journal column predicts aggregate market activity, providing early evidence that textual content moves prices. Loughran and McDonald (2011) showed that the widely-used Harvard General Inquirer sentiment dictionary produces misleading results when applied to financial text because words like "liability," "tax," and "capital" are classified as negative in general English but carry neutral or even positive connotations in finance. Their domain-specific word lists have become the standard for financial sentiment analysis.[1]

Hoberg and Phillips (2010) and Hoberg and Phillips (2016) pioneered the use of product descriptions from 10-K filings to construct text-based industry classifications (TNIC), demonstrating that these dynamic, firm-specific industry definitions outperform static SIC and NAICS codes in explaining firm behavior, including profitability, stock returns, and M&A activity. Subsequent work by Hoberg and Phillips (2018) extended this to assess competitive threats and product-market fluidity.

More recent work has leveraged advances in deep learning. A. H. Huang, Wang, and Yang (2023) apply BERT-based models to earnings call transcripts and show that contextual embeddings capture information about future earnings that traditional bag-of-words measures miss. Jha et al. (2024) use GPT-based models for zero-shot financial text classification and demonstrate that LLMs can match or exceed purpose-built classifiers on standard benchmarks. Earnings calls are also read by regulators: Lerman, Steffen, and Zhang (2026) show that the SEC cites voluntary call disclosures in comment letters, mostly to show that the mandatory filing says too little, and that firms then change their filings (Section 102.6).

Text can also measure the *uncertainty* investors face, not only its tone. Wei and Zhang (2026) extract a narrative-based ambiguity measure from economic news articles, distinct from survey- and return-based ambiguity proxies, and show that it predicts aggregate stock market returns both in and out of sample. Investors are on average ambiguity averse, but ambiguity tolerance emerges when the probability of loss is high, and aligning industry-specific narrative ambiguity (chiefly from consumption-, energy-, and technology-related news) yields a stronger market

[1] Loughran and McDonald (2011) show that general-purpose dictionaries misclassify up to 73% of negative words in financial text.

predictor. The predictability extends to international markets and other asset classes, which makes the approach a natural candidate for Vietnamese financial news.

94.2 NLP for Vietnamese Language

Vietnamese NLP has advanced significantly with the development of VnCoreNLP (Vu et al. 2018), a Java-based toolkit providing word segmentation, POS tagging, named entity recognition, and dependency parsing. The underthesea library offers a Python-native alternative. Most critically for financial applications, PhoBERT (D. Q. Nguyen and Nguyen 2020) provides Vietnamese-specific BERT pre-training on a 20GB corpus, achieving state-of-the-art results on multiple Vietnamese NLP tasks.

Table 94.1: Key Literature on Textual Analysis in Finance

Study	Method	Key Finding	Relevance to Vietnam
Tetlock (2007)	Dictionary-based sentiment from WSJ column	Media pessimism predicts market activity and returns	Baseline for Vietnamese financial news sentiment
Loughran and McDonald (2011)	Domain-specific financial dictionaries	General dictionaries misclassify 73% of negative financial words	Need for Vietnamese financial sentiment lexicon
Hoberg and Phillips (2016)	Cosine similarity on 10-K product descriptions	Text-based industries outperform SIC/NAICS	Peer identification for Vietnamese firms using business descriptions
D. Q. Nguyen and Nguyen (2020)	PhoBERT: Vietnamese BERT pre-training	SOTA on Vietnamese NLP benchmarks	Foundation model for Vietnamese financial NLP
A. H. Huang, Wang, and Yang (2023)	BERT embeddings on earnings calls	Contextual embeddings predict future earnings beyond BoW	Apply to Vietnamese earnings call transcripts
Jha et al. (2024)	GPT-based zero-shot financial classification	LLMs match fine-tuned classifiers	Zero-shot Vietnamese financial text classification via multilingual LLMs

Study	Method	Key Finding	Relevance to Vietnam
Wei and Zhang (2026)	Narrative ambiguity extracted from economic news	News-based ambiguity predicts market returns beyond survey- and return-based measures; aversion turns to tolerance at high loss probability	Aggregate ambiguity index from Vietnamese financial news for VN-Index return predictability
Crowley, Huang, and Lu (2024b)	ML classification + content similarity of S&P 1500 executives' tweets	Investors react to executives' financial tweets through both new-information and credibility channels	Executive Facebook/Zalo posts of Vietnamese listed firms (Section 102.4)

95 Data: Vietnamese Listed Firms from DataCore.vn

95.1 Constructing the Universe

We construct the universe of Vietnamese listed firms. The universe includes all firms listed on HOSE, HNX, and UPCoM as of the analysis date.

```
import pandas as pd
import numpy as np
import re
import unicodedata
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
from collections import defaultdict
from typing import List, Dict, Tuple, Optional

warnings.filterwarnings('ignore')
np.random.seed(42)

# Plotting configuration
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11
sns.set_style("whitegrid")
```

```
from datacore import DataCoreAPI  # DataCore.vn Python client

# Initialize connection
dc = DataCoreAPI(api_key='YOUR_API_KEY')

# Retrieve universe of all listed firms
universe = dc.get_listed_firms(
    exchanges=['HOSE', 'HNX', 'UPCOM'],
```

```
    as_of='2024-12-31',
    fields=[
        'ticker', 'company_name', 'company_name_en',
        'exchange', 'listing_date', 'delisting_date',
        'icb_industry', 'icb_sector', 'icb_subsector',
        'market_cap', 'total_assets', 'revenue'
    ]
)

print(f'Total listed firms: {len(universe)}')
print(f'HOSE: {len(universe[universe.exchange=="HOSE"])}')
print(f'HNX: {len(universe[universe.exchange=="HNX"])}')
print(f'UPCoM: {len(universe[universe.exchange=="UPCOM"])}')
```

Table 95.1: Universe of Vietnamese Listed Firms by Exchange (as of December 2024)

Exchange	N Firms	Avg Mkt Cap (VND bn)	Median Mkt Cap (VND bn)	Total Mkt Cap (VND tn)
HOSE	403	12,847	3,215	5,177
HNX	334	2,156	687	720
UPCoM	868	1,043	298	905
Total	**1,605**	**4,239**	**712**	**6,802**

95.2 Retrieving Business Descriptions

Business descriptions for all listed firms can be in both Vietnamese and English. We retrieve both versions for our analysis. The Vietnamese text will serve as the primary corpus, while English descriptions provide a useful cross-validation.

```
# Get business descriptions (Vietnamese and English)
bus_desc = dc.get_business_descriptions(
    tickers=universe.ticker.tolist(),
    fields=[
        'ticker', 'bus_desc_vi', 'bus_desc_en',
        'main_business', 'products_services',
        'year_established', 'num_employees'
    ]
)
```

```
# Merge with universe
corpus_df = universe.merge(bus_desc, on='ticker', how='inner')

# Summary statistics on text length
corpus_df['desc_len_vi'] = corpus_df.bus_desc_vi.str.len()
corpus_df['desc_len_en'] = corpus_df.bus_desc_en.str.len()
corpus_df['word_count_vi'] = corpus_df.bus_desc_vi.str.split().str.len()

print(corpus_df[['desc_len_vi', 'desc_len_en', 'word_count_vi']]
      .describe().round(0))
```

Table 95.2: Descriptive Statistics of Business Description Text

Statistic	Mean	Median	Std Dev	Min	Max
Characters (VN)	2,847	2,156	1,923	87	18,432
Characters (EN)	3,412	2,689	2,245	102	22,156
Words (VN)	487	372	318	15	3,216

95.3 Retrieving Annual Report Text

Beyond business descriptions, annual or quarterly reports provide richer and more time-varying textual data. We extract the Management Discussion and Analysis (MD&A) sections, which are most informative for financial analysis (F. Li et al. 2010; Bonsall IV et al. 2017). The MD&A section, known in Vietnamese annual reports as "Báo cáo của Ban Giám đốc" or "Báo cáo của Hội đồng quản trị," discusses business performance, outlook, and risk factors.

```
# Get annual report MD&A sections (2015-2024)
annual_text = dc.get_annual_report_text(
    tickers=universe.ticker.tolist(),
    years=range(2015, 2025),
    sections=['mda', 'risk_factors', 'business_overview'],
    language='vi'
)

# Panel structure: ticker x year x section
print(f'Total firm-year-section observations: {len(annual_text)}')
print(f'Unique firms: {annual_text.ticker.nunique()}')
print(f'Year range: {annual_text.year.min()}-{annual_text.year.max()}')
```

```
# Calculate text changes year-over-year
annual_text = annual_text.sort_values(['ticker', 'year'])
annual_text['text_len'] = annual_text.text.str.len()
annual_text['text_change_pct'] = (
    annual_text.groupby('ticker')['text_len']
    .pct_change() * 100
)
```

96 Text Preprocessing for Vietnamese

96.1 Vietnamese Word Segmentation

The most critical preprocessing step for Vietnamese text is word segmentation (phân đoạn từ). Unlike English where spaces reliably separate words, Vietnamese uses spaces between syllables, not between words. For example, the phrase "công ty cổ phần bất động sản" (real estate joint stock company) contains five syllables separated by spaces but consists of only two compound words: "công_ty cổ_phần" (joint stock company) and "bất_động_sản" (real estate). Failing to perform word segmentation leads to severe vocabulary fragmentation and loss of semantic meaning.

Table 96.1: Vietnamese Word Segmentation Example

Stage	Text	Interpretation
Raw	`công ty cổ phần thương mại dịch vụ`	7 syllables, ambiguous boundaries
Segmented	`công_ty cổ_phần thương_mại dịch_vụ`	4 words: company | joint-stock | commerce | services

```
from underthesea import word_tokenize

def segment_vietnamese(text: str) -> str:
    """Segment Vietnamese text into words using underthesea."""
    if pd.isna(text) or text.strip() == '':
        return ''
    # underthesea word_tokenize joins compound words with _
    segmented = word_tokenize(text, format='text')
    return segmented

# Alternative: VnCoreNLP (Java-based, higher accuracy)
# from vncorenlp import VnCoreNLP
# vnlp = VnCoreNLP('VnCoreNLP-1.2.jar', annotators='wseg')
# segmented = vnlp.tokenize(text)
```

```
# Apply segmentation to corpus
corpus_df['bus_desc_segmented'] = (
    corpus_df.bus_desc_vi.apply(segment_vietnamese)
)

# Example
sample = corpus_df.iloc[0]
print('Raw:', sample.bus_desc_vi[:200])
print('Segmented:', sample.bus_desc_segmented[:200])
```

96.2 Full Text Cleaning Pipeline

After word segmentation, we apply a cleaning pipeline. The pipeline handles Vietnamese-specific challenges including: diacritical mark normalization (e.g., hoà vs hòa), removal of HTML artifacts from scraped text, Vietnamese stopword removal, and lemmatization (which for Vietnamese primarily involves handling reduplicative words and synonym normalization).

```
# Vietnamese stopwords (domain-adapted)
VIETNAMESE_STOPWORDS = {
    'có', 'là', 'và', 'của', 'cho', 'được', 'trong',
    'các', 'những', 'với', 'từ', 'khi', 'hoặc',
    'đã', 'sẽ', 'đang', 'để', 'này', 'đó',
    'như', 'theo', 'về', 'bằng', 'tại', 'trên',
    'cũng', 'rất', 'nhiều', 'ít', 'một', 'hai',
    # Financial domain stopwords
    'năm', 'quý', 'tháng', 'ngày', 'kỳ',
    'việt_nam', 'tổng', 'giá_trị', 'triệu', 'tỷ',
}

def clean_vietnamese_text(
    text: str,
    segment: bool = True,
    remove_stops: bool = True,
    lowercase: bool = True,
    min_word_len: int = 2
) -> str:
    """
    Full Vietnamese text cleaning pipeline.

    Parameters
    ----------
```

```
    text : str
        Raw Vietnamese text.
    segment : bool
        Whether to perform word segmentation.
    remove_stops : bool
        Whether to remove Vietnamese stopwords.
    lowercase : bool
        Whether to convert to lowercase.
    min_word_len : int
        Minimum word length to keep.

    Returns
    -------
    str
        Cleaned text.
    """
    if pd.isna(text) or text.strip() == '':
        return ''

    # 1. Unicode normalization (NFC form for Vietnamese)
    text = unicodedata.normalize('NFC', text)

    # 2. Remove HTML tags and special characters
    text = re.sub(r'<[^>]+>', ' ', text)
    text = re.sub(r'[\d]+', ' ', text)           # Remove numbers
    text = re.sub(r'[^\w\s\u00C0-\u024F]', ' ', text)  # Keep VN chars

    # 3. Lowercase
    if lowercase:
        text = text.lower()

    # 4. Word segmentation
    if segment:
        text = word_tokenize(text, format='text')

    # 5. Tokenize and filter
    tokens = text.split()
    if remove_stops:
        tokens = [t for t in tokens
                  if t not in VIETNAMESE_STOPWORDS
                  and len(t) >= min_word_len]

    return ' '.join(tokens)
```

```
# Apply to corpus
corpus_df['text_clean'] = (
    corpus_df.bus_desc_vi
    .apply(lambda x: clean_vietnamese_text(x))
)

# Verify cleaning quality
print('Sample cleaned text:')
print(corpus_df.iloc[0].text_clean[:300])
```

96.3 English Text Cleaning

For firms that also provide English business descriptions, we apply a standard English NLP pipeline using spaCy and NLTK. This parallel processing enables cross-lingual validation of our textual measures.

```
import spacy
from nltk.corpus import stopwords
import gensim

nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])
stop_words = set(stopwords.words('english'))

def clean_english_text(text: str) -> str:
    """Clean English text with lemmatization."""
    if pd.isna(text) or text.strip() == '':
        return ''
    text = text.lower().strip()
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)
    doc = nlp(text)
    tokens = [token.lemma_ for token in doc
              if token.lemma_ not in stop_words
              and len(token.lemma_) > 2
              and not token.is_punct]
    return ' '.join(tokens)

# Apply to English descriptions
corpus_df['text_clean_en'] = (
    corpus_df.bus_desc_en
    .apply(lambda x: clean_english_text(x))
```

```
)
```

97 Document Representation: Bag-of-Words and TF-IDF

97.1 Bag-of-Words Representation

The bag-of-words (BoW) model represents each document as a vector of word frequencies, discarding word order. Despite its simplicity, BoW remains a workhorse in financial textual analysis. Formally, given a vocabulary $V = \{w_1, w_2, \ldots, w_{|V|}\}$, document d is represented as a vector $\mathbf{x}_d$ where each element $x_{d,j}$ counts the frequency of word w_j in document d:

$$\mathbf{x}_d = [\text{tf}(w_1, d),\ \text{tf}(w_2, d),\ \ldots,\ \text{tf}(w_{|V|}, d)] \tag{97.1}$$

where $\text{tf}(w, d)$ is the term frequency of word w in document d.

```
from sklearn.feature_extraction.text import (
    CountVectorizer, TfidfVectorizer
)

# Vietnamese corpus
text_corpus = corpus_df.text_clean.tolist()

# BoW vectorization
bow_vectorizer = CountVectorizer(
    max_features=10000,
    min_df=5,           # Appear in at least 5 documents
    max_df=0.95,        # Exclude terms in >95% of docs
    ngram_range=(1, 2)  # Unigrams and bigrams
)

bow_matrix = bow_vectorizer.fit_transform(text_corpus)

print(f'Vocabulary size: {len(bow_vectorizer.vocabulary_)}')
print(f'Document-term matrix shape: {bow_matrix.shape}')
print(f'Sparsity: {1 - bow_matrix.nnz / np.prod(bow_matrix.shape):.4f}')
```

```
# Top 20 most frequent terms
word_freq = pd.DataFrame({
    'word': bow_vectorizer.get_feature_names_out(),
    'freq': bow_matrix.sum(axis=0).A1
}).sort_values('freq', ascending=False)

print('\nTop 20 most frequent terms:')
print(word_freq.head(20).to_string(index=False))
```

```
fig, ax = plt.subplots(figsize=(12, 6))
top20 = word_freq.head(20)
ax.barh(range(len(top20)), top20.freq.values, color='#2C5282')
ax.set_yticks(range(len(top20)))
ax.set_yticklabels(top20.word.values)
ax.invert_yaxis()
ax.set_xlabel('Frequency')
ax.set_title('Top 20 Most Frequent Terms in Vietnamese Business
 ↪  Descriptions')
plt.tight_layout()
plt.show()
```

Figure 97.1

Table 97.1: Top 20 Most Frequent Terms in Vietnamese Business Descriptions

#	Term (VN)	Freq	#	Term (VN)	Freq	#	Term (VN)	Freq
1	sản_xuất	4,287	8	công_nghệ	1,956	15	xuất_khẩu	1,123
2	kinh_doanh	3,891	9	tài_chính	1,845	16	bất_động_sản	1,087
3	dịch_vụ	3,654	10	ngân_hàng	1,734	17	năng_lượng	1,045
4	công_ty	3,412	11	đầu_tư	1,623	18	bảo_hiểm	987
5	thương_mại	2,876	12	xây_dựng	1,534	19	du_lịch	923
6	cổ_phần	2,543	13	vận_tải	1,345	20	viễn_thông	876
7	chứng_khoán	2,134	14	thực_phẩm	1,234			

97.2 TF-IDF Weighting

Term Frequency-Inverse Document Frequency (TF-IDF) addresses a key limitation of raw term counts by downweighting terms that appear in many documents (and thus carry less

discriminative information). The TF-IDF weight of term w in document d within corpus D is:

$$\text{tfidf}(w, d, D) = \text{tf}(w, d) \times \log\left(\frac{|D|}{\text{df}(w, D)}\right) \tag{97.2}$$

where $|D|$ is the total number of documents and $\text{df}(w, D)$ is the number of documents containing term w. This weighting scheme ensures that industry-specific terminology (e.g., "khai_khoáng" for mining, "dược_phẩm" for pharmaceuticals) receives higher weight than ubiquitous corporate jargon.

```
tfidf_vectorizer = TfidfVectorizer(
    max_features=10000,
    min_df=5,
    max_df=0.95,
    ngram_range=(1, 2),
    sublinear_tf=True  # Use 1 + log(tf) instead of raw tf
)

tfidf_matrix = tfidf_vectorizer.fit_transform(text_corpus)

# Per-industry top TF-IDF terms
for industry in ['Ngân hàng', 'Bất động sản',
                 'Công nghệ thông tin']:
    mask = corpus_df.icb_sector == industry
    if mask.sum() == 0:
        continue
    mean_tfidf = tfidf_matrix[mask.values].mean(axis=0).A1
    top_idx = mean_tfidf.argsort()[-10:][::-1]
    terms = tfidf_vectorizer.get_feature_names_out()
    print(f'\n{industry}:')
    for idx in top_idx:
        print(f'  {terms[idx]}: {mean_tfidf[idx]:.4f}')
```

```
# Build industry x term TF-IDF matrix for top sectors
top_sectors = corpus_df.icb_sector.value_counts().head(8).index.tolist()
terms = tfidf_vectorizer.get_feature_names_out()

sector_tfidf = {}
for sector in top_sectors:
    mask = corpus_df.icb_sector == sector
    if mask.sum() == 0:
        continue
    mean_tfidf = tfidf_matrix[mask.values].mean(axis=0).A1
    top_idx = mean_tfidf.argsort()[-5:][::-1]
    for idx in top_idx:
        if terms[idx] not in sector_tfidf:
            sector_tfidf[terms[idx]] = {}
        sector_tfidf[terms[idx]][sector] = mean_tfidf[idx]

heatmap_df = pd.DataFrame(sector_tfidf).T.fillna(0)

fig, ax = plt.subplots(figsize=(14, 10))
sns.heatmap(heatmap_df, annot=True, fmt='.3f', cmap='Blues',
            linewidths=0.5, ax=ax)
ax.set_title('TF-IDF Heatmap: Industry-Distinctive Terms')
ax.set_xlabel('ICB Sector')
ax.set_ylabel('Term')
plt.tight_layout()
plt.show()
```

Figure 97.2

98 Topic Modeling

98.1 Latent Dirichlet Allocation (LDA)

Latent Dirichlet Allocation (Blei, Ng, and Jordan 2003) is a generative probabilistic model that discovers latent topics in a corpus. Each document is modeled as a mixture of topics, and each topic is a distribution over words. LDA has been widely applied in finance to identify thematic content in 10-K filings (Dyer, Lang, and Stice-Lawrence 2017), earnings calls (A. H. Huang et al. 2018), and news articles (Bybee, Kelly, and Su 2023).

The generative process assumes:

1. For each topic k, draw a word distribution $\phi_k \sim \text{Dir}(\beta)$.
2. For each document d, draw a topic distribution $\theta_d \sim \text{Dir}(\alpha)$.
3. For each word position i in document d, draw a topic $z_{d,i} \sim \text{Multinomial}(\theta_d)$ and then draw the word $w_{d,i} \sim \text{Multinomial}(\phi_{z_{d,i}})$.

```
from sklearn.decomposition import LatentDirichletAllocation

# Grid search over number of topics
n_topics_range = [10, 15, 20, 25, 30]
perplexity_scores = []

for n_topics in n_topics_range:
    lda = LatentDirichletAllocation(
        n_components=n_topics,
        max_iter=50,
        learning_method='online',
        random_state=42,
        n_jobs=-1
    )
    lda.fit(bow_matrix)
    perplexity = lda.perplexity(bow_matrix)
    perplexity_scores.append({
        'n_topics': n_topics,
        'perplexity': perplexity,
        'log_likelihood': lda.score(bow_matrix)
```

```
    })
    print(f'K={n_topics}: perplexity={perplexity:.2f}')

# Select optimal K (e.g., K=20)
K_OPTIMAL = 20
lda_model = LatentDirichletAllocation(
    n_components=K_OPTIMAL,
    max_iter=100,
    learning_method='online',
    random_state=42,
    n_jobs=-1
)
lda_model.fit(bow_matrix)

# Extract topic-word distributions
feature_names = bow_vectorizer.get_feature_names_out()
for topic_idx, topic in enumerate(lda_model.components_):
    top_words = [feature_names[i]
                 for i in topic.argsort()[:-11:-1]]
    print(f'Topic {topic_idx}: {" | ".join(top_words)}')
```

```
perp_df = pd.DataFrame(perplexity_scores)
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(perp_df.n_topics, perp_df.perplexity, 'o-', color='#2C5282',
        linewidth=2, markersize=8)
ax.axvline(x=K_OPTIMAL, color='red', linestyle='--', alpha=0.7,
           label=f'Selected K={K_OPTIMAL}')
ax.set_xlabel('Number of Topics (K)')
ax.set_ylabel('Perplexity (lower is better)')
ax.set_title('LDA Model Selection')
ax.legend()
plt.tight_layout()
plt.show()
```

Figure 98.1

Table 98.1: Selected LDA Topics from Vietnamese Business Descriptions (K=20)

Topic	Interpretation	Top Words
0	Banking & Finance	ngân_hàng | tín_dụng | cho_vay | tiền_gửi | lãi_suất | thanh_toán | tài_khoản
3	Real Estate	bất_động_sản | dự_án | căn_hộ | khu_đô_thị | xây_dựng | nhà_ở
7	Technology	công_nghệ | phần_mềm | giải_pháp | hệ_thống | số_hóa | dữ_liệu
11	Manufacturing	sản_xuất | nguyên_liệu | nhà_máy | chất_lượng | công_suất | xuất_khẩu
15	Securities	chứng_khoán | môi_giới | đầu_tư | cổ_phiếu | danh_mục | quản_lý_quỹ

98.2 BERTopic: Neural Topic Modeling

BERTopic (Grootendorst 2022) represents a significant advance over LDA by leveraging pre-trained language model embeddings, dimensionality reduction via UMAP, and hierarchical density-based clustering (HDBSCAN) to discover topics. Unlike LDA, BERTopic captures semantic similarity rather than relying solely on word co-occurrence, producing more coherent topics, especially for specialized domains.

```
from bertopic import BERTopic
from sentence_transformers import SentenceTransformer
from umap import UMAP
from hdbscan import HDBSCAN

# Use PhoBERT-based sentence transformer for Vietnamese
embedding_model = SentenceTransformer(
    'bkai-foundation-models/vietnamese-bi-encoder'
)

# Custom UMAP and HDBSCAN for better control
umap_model = UMAP(
```

```
    n_neighbors=15, n_components=5,
    min_dist=0.0, metric='cosine', random_state=42
)
hdbscan_model = HDBSCAN(
    min_cluster_size=10, min_samples=5,
    metric='euclidean', prediction_data=True
)

# Fit BERTopic
topic_model = BERTopic(
    embedding_model=embedding_model,
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
    language='multilingual',
    calculate_probabilities=True,
    verbose=True
)

# Vietnamese text (use segmented text for better results)
docs = corpus_df.bus_desc_segmented.tolist()
topics, probs = topic_model.fit_transform(docs)

# Inspect topics
topic_info = topic_model.get_topic_info()
print(topic_info.head(20))
```

```
# Visualize topic hierarchy
fig_hierarchy = topic_model.visualize_hierarchy()
fig_hierarchy.show()

# Visualize document clusters
fig_docs = topic_model.visualize_documents(
    docs, reduced_embeddings=umap_model.embedding_
)
fig_docs.show()

# Topic word scores (barchart)
fig_barchart = topic_model.visualize_barchart(top_n_topics=10)
fig_barchart.show()
```

Figure 98.2

99 Financial Sentiment Analysis

99.1 Dictionary-Based Approach

We construct a Vietnamese financial sentiment lexicon following the methodology of Loughran and McDonald (2011). Rather than directly translating the English LM dictionary (which would miss Vietnamese-specific financial expressions), we adopt a hybrid approach: (1) translate the core LM word lists using professional financial translators, (2) manually curate additions from Vietnamese financial regulation, accounting standards (VAS), and market commentary, and (3) validate the resulting dictionary against human-annotated Vietnamese financial text.

Table 99.1: Vietnamese Financial Sentiment Lexicon: Sample Entries

Category	Vietnamese Term	English Gloss	Source	Count in Corpus
Negative	lỗ	loss	LM-translated	2,341
Negative	sụt_giảm	decline	Curated	1,876
Negative	nợ_xấu	bad debt	VAS-specific	1,234
Negative	rủi_ro	risk	LM-translated	3,567
Positive	tăng_trưởng	growth	LM-translated	4,123
Positive	lợi_nhuận	profit	LM-translated	3,891
Positive	hiệu_quả	efficiency	Curated	2,456
Uncertain	biến_động	volatility	LM-translated	1,567
Litigious	tranh_chấp	dispute	Legal-VN	876
Litigious	khởi_kiện	lawsuit	Legal-VN	234

```
# Load Vietnamese financial sentiment lexicon
# sentiment_dict = dc.get_sentiment_lexicon(version='vn_financial_v2')

# Alternatively, construct from LM + manual curation
negative_words = set(pd.read_csv(
    'lexicons/vn_negative.txt', header=None)[0]
)
positive_words = set(pd.read_csv(
    'lexicons/vn_positive.txt', header=None)[0]
```

```
)
uncertain_words = set(pd.read_csv(
    'lexicons/vn_uncertain.txt', header=None)[0]
)

def compute_sentiment_scores(text: str) -> dict:
    """
    Compute Loughran-McDonald style sentiment scores.
    Returns proportions (word count / total words).
    """
    tokens = text.split()
    n = len(tokens)
    if n == 0:
        return {'neg_pct': 0, 'pos_pct': 0,
                'unc_pct': 0, 'net_tone': 0}

    neg = sum(1 for t in tokens if t in negative_words)
    pos = sum(1 for t in tokens if t in positive_words)
    unc = sum(1 for t in tokens if t in uncertain_words)

    return {
        'neg_pct': neg / n,
        'pos_pct': pos / n,
        'unc_pct': unc / n,
        'net_tone': (pos - neg) / n
    }

# Apply to annual report MD&A text
sentiment_scores = annual_text.text_clean.apply(
    lambda x: pd.Series(compute_sentiment_scores(x))
)
annual_text = pd.concat([annual_text, sentiment_scores], axis=1)
```

99.2 Transformer-Based Sentiment Classification

Dictionary approaches are limited by their inability to capture context, negation, and sarcasm. We complement the dictionary approach with PhoBERT-based sentiment classification. We fine-tune PhoBERT v2.

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].hist(annual_text.neg_pct, bins=50, color='#E53E3E', alpha=0.7,
             edgecolor='white')
axes[0].set_title('Negative Word Proportion')
axes[0].set_xlabel('Proportion')

axes[1].hist(annual_text.pos_pct, bins=50, color='#38A169', alpha=0.7,
             edgecolor='white')
axes[1].set_title('Positive Word Proportion')
axes[1].set_xlabel('Proportion')

axes[2].hist(annual_text.net_tone, bins=50, color='#2C5282', alpha=0.7,
             edgecolor='white')
axes[2].set_title('Net Tone (Positive - Negative)')
axes[2].set_xlabel('Net Tone')

plt.suptitle('Sentiment Distribution in Vietnamese Annual Reports',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

Figure 99.1

```python
from transformers import (
    AutoModelForSequenceClassification,
    AutoTokenizer,
    pipeline
)
import torch

# Load fine-tuned ViFinBERT for sentiment
model_name = 'vinai/phobert-base-v2'

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=3  # positive, negative, neutral
)

# Create sentiment pipeline
sentiment_pipe = pipeline(
    'text-classification',
    model=model,
    tokenizer=tokenizer,
    device=0 if torch.cuda.is_available() else -1,
    max_length=256,
    truncation=True,
    batch_size=32
)

# For long documents, split into sentences first
from underthesea import sent_tokenize

def document_sentiment(text: str) -> dict:
    """Aggregate sentence-level sentiment for a document."""
    sentences = sent_tokenize(text)
    if not sentences:
        return {'bert_pos': 0, 'bert_neg': 0, 'bert_neu': 0}

    results = sentiment_pipe(sentences[:100])  # Cap at 100 sents
    labels = [r['label'] for r in results]

    n = len(labels)
    return {
        'bert_pos': labels.count('POSITIVE') / n,
        'bert_neg': labels.count('NEGATIVE') / n,
        'bert_neu': labels.count('NEUTRAL') / n,
```

```
        'bert_tone': (labels.count('POSITIVE') -
                      labels.count('NEGATIVE')) / n
    }
```

Table 99.2: Sentiment Method Comparison: Dictionary vs. PhoBERT on Validation Set (N=500)

Method	Accuracy	F1 (Pos)	F1 (Neg)	F1 (Neutral)
VN-LM Dictionary	0.612	0.584	0.637	0.598
PhoBERT (zero-shot)	0.724	0.698	0.741	0.712
PhoBERT v2 (fine-tuned)	**0.831**	**0.812**	**0.847**	**0.824**

100 Text-Based Firm Similarity and Peer Identification

100.1 Cosine Similarity on TF-IDF Vectors

Following Hoberg and Phillips (2016), we compute pairwise cosine similarity between firms based on their business description TF-IDF vectors. For two documents represented as TF-IDF vectors $\mathbf{a}$ and $\mathbf{b}$, cosine similarity is defined as:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \times \|\mathbf{b}\|} \tag{100.1}$$

This metric ranges from 0 (completely dissimilar) to 1 (identical content) and is invariant to document length. We use this to construct text-based industry networks (TNIC) for the Vietnamese market, which can capture firm relationships that static ICB sector codes miss.

```
from sklearn.metrics.pairwise import cosine_similarity

# Compute pairwise similarity matrix
sim_matrix = cosine_similarity(tfidf_matrix)

# Convert to DataFrame for easy lookup
tickers = corpus_df.ticker.tolist()
sim_df = pd.DataFrame(
    sim_matrix, index=tickers, columns=tickers
)

# For each firm, find top-5 most similar peers
def get_top_peers(ticker: str, n: int = 5) -> pd.DataFrame:
    """Return top-n most similar firms by TF-IDF cosine."""
    sims = sim_df[ticker].drop(ticker).sort_values(
        ascending=False
    ).head(n)
    peers = corpus_df.set_index('ticker').loc[sims.index]
    peers['similarity'] = sims.values
```

```
    return peers[['company_name', 'icb_sector',
                  'market_cap', 'similarity']]

# Examples
for ticker in ['VCB', 'VNM', 'FPT', 'VIC', 'HPG']:
    print(f'\nTop peers for {ticker}:')
    print(get_top_peers(ticker))
```

Table 100.1: Text-Based Peer Identification: Top Most Similar Firms (TF-IDF Cosine)

Firm	ICB Sector	Peer 1	Peer 1 Sector	Sim Score	Same ICB?
VCB	Banking	BID	Banking	0.87	Yes
VCB	Banking	CTG	Banking	0.84	Yes
VNM	Food & Bev	MCH	Food & Bev	0.72	Yes
FPT	Technology	CMG	Technology	0.68	Yes
VIC	Real Estate	NVL	Real Estate	0.74	Yes
HPG	Steel	HSG	Steel	0.81	Yes

```
sample_tickers = ['VCB', 'BID', 'CTG', 'VNM', 'MCH',
                  'FPT', 'CMG', 'VIC', 'NVL', 'HPG',
                  'HSG', 'VHM', 'SSI', 'HCM', 'PNJ']
sample_sim = sim_df.loc[sample_tickers, sample_tickers]

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(sample_sim, annot=True, fmt='.2f', cmap='Blues',
            vmin=0, vmax=1, square=True, linewidths=0.5, ax=ax)
ax.set_title('Pairwise TF-IDF Cosine Similarity\n(Selected Vietnamese Listed
 ↪  Firms)')
plt.tight_layout()
plt.show()
```

Figure 100.1

100.2 Embedding-Based Similarity

While TF-IDF cosine similarity captures lexical overlap, it misses semantic similarity. Two firms may describe similar businesses using different vocabulary. We address this using dense vector representations from pre-trained language models. Specifically, we compute document

embeddings using Sentence-BERT (Reimers and Gurevych 2019) with a Vietnamese bi-encoder model.[1]

```
from sentence_transformers import SentenceTransformer

# Vietnamese sentence transformer
sbert_model = SentenceTransformer(
    'bkai-foundation-models/vietnamese-bi-encoder'
)

# Compute embeddings for all firms
docs_segmented = corpus_df.bus_desc_segmented.tolist()
embeddings = sbert_model.encode(
    docs_segmented,
    batch_size=64,
    show_progress_bar=True,
    normalize_embeddings=True
)

# Pairwise similarity
embed_sim = cosine_similarity(embeddings)
embed_sim_df = pd.DataFrame(
    embed_sim, index=tickers, columns=tickers
)

# Compare TF-IDF vs embedding similarity
for ticker in ['VCB', 'FPT', 'VIC']:
    tfidf_peers = sim_df[ticker].drop(ticker).nlargest(5)
    embed_peers = embed_sim_df[ticker].drop(ticker).nlargest(5)
    print(f'\n{ticker} - TF-IDF peers: {tfidf_peers.index.tolist()}')
    print(f'{ticker} - Embed peers:  {embed_peers.index.tolist()}')
```

100.3 Doc2Vec

We also implement Doc2Vec (Le and Mikolov 2014), which learns fixed-length dense vectors for documents of variable length. Unlike averaging word embeddings, Doc2Vec jointly learns document and word vectors, allowing it to capture document-level semantics. We train Doc2Vec on the Vietnamese business description corpus using the concatenated DBOW+DM approach recommended by Lau and Baldwin (2016).

[1] Reimers and Gurevych (2019) demonstrate that sentence-BERT embeddings reduce computation for similarity tasks from 65 hours to 5 seconds on 10,000 sentence pairs.

```
from sklearn.manifold import TSNE

# t-SNE projection
tsne = TSNE(n_components=2, perplexity=30, random_state=42,
            metric='cosine')
embeddings_2d = tsne.fit_transform(embeddings)

fig, ax = plt.subplots(figsize=(14, 10))
sectors = corpus_df.icb_sector.values
unique_sectors = corpus_df.icb_sector.value_counts().head(10).index
colors = plt.cm.tab10(range(10))

for i, sector in enumerate(unique_sectors):
    mask = sectors == sector
    ax.scatter(embeddings_2d[mask, 0], embeddings_2d[mask, 1],
               c=[colors[i]], label=sector, alpha=0.6, s=30)

ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=9)
ax.set_title('t-SNE of PhoBERT Embeddings by ICB Sector')
ax.set_xlabel('t-SNE 1')
ax.set_ylabel('t-SNE 2')
plt.tight_layout()
plt.show()
```

Figure 100.2

```
from gensim.models.doc2vec import Doc2Vec, TaggedDocument

# Prepare tagged documents
tagged_docs = [
    TaggedDocument(
        words=text.split(),
        tags=[ticker]
    )
    for text, ticker in zip(
        corpus_df.text_clean.tolist(),
        corpus_df.ticker.tolist()
    )
]

# PV-DBOW: paragraph vector with distributed bag of words
d2v_dbow = Doc2Vec(
    vector_size=100, dm=0, min_count=5,
    window=5, epochs=40, workers=4, seed=42
)
d2v_dbow.build_vocab(tagged_docs)
d2v_dbow.train(
    tagged_docs,
    total_examples=d2v_dbow.corpus_count,
    epochs=d2v_dbow.epochs
)

# PV-DM: paragraph vector with distributed memory
d2v_dm = Doc2Vec(
    vector_size=100, dm=1, min_count=5,
    window=10, epochs=40, workers=4, seed=42
)
d2v_dm.build_vocab(tagged_docs)
d2v_dm.train(
    tagged_docs,
    total_examples=d2v_dm.corpus_count,
    epochs=d2v_dm.epochs
)

# Concatenate DBOW + DM vectors (Lau & Baldwin, 2016)
d2v_vectors = np.hstack([
    [d2v_dbow.dv[t] for t in tickers],
    [d2v_dm.dv[t] for t in tickers]
])
```

```
# Most similar firms
for ticker in ['VCB', 'FPT', 'VIC']:
    sims = d2v_dbow.dv.most_similar(ticker, topn=5)
    print(f'{ticker}: {[(s[0], f"{s[1]:.3f}") for s in sims]}')
```

101 Deep Learning Approaches

101.1 PhoBERT Embeddings for Financial Text

PhoBERT (D. Q. Nguyen and Nguyen 2020), pre-trained on 20GB of Vietnamese text, provides contextualized word embeddings that capture meaning based on surrounding context. Unlike static Word2Vec embeddings where "bảo" always has the same vector regardless of whether it means "insurance" (bảo hiểm) or "protect" (bảo vệ), PhoBERT produces context-dependent representations. We extract `[CLS]` token embeddings as document representations.

```
from transformers import AutoModel, AutoTokenizer
import torch

# Load PhoBERT
phobert_tokenizer = AutoTokenizer.from_pretrained(
    'vinai/phobert-base-v2'
)
phobert_model = AutoModel.from_pretrained(
    'vinai/phobert-base-v2'
)
phobert_model.eval()
device = torch.device('cuda' if torch.cuda.is_available()
                      else 'cpu')
phobert_model.to(device)

def get_phobert_embedding(text: str, max_len: int = 256):
    """Extract [CLS] embedding from PhoBERT."""
    inputs = phobert_tokenizer(
        text, return_tensors='pt',
        max_length=max_len, truncation=True,
        padding=True
    ).to(device)

    with torch.no_grad():
        outputs = phobert_model(**inputs)
```

```
    # [CLS] token embedding
    cls_embedding = outputs.last_hidden_state[:, 0, :]
    return cls_embedding.cpu().numpy().flatten()

# For long documents: chunk + average strategy
def get_long_doc_embedding(
    text: str, chunk_size: int = 256, stride: int = 128
):
    """Handle long documents via chunked averaging."""
    tokens = phobert_tokenizer.tokenize(text)
    embeddings = []

    for i in range(0, len(tokens), stride):
        chunk = tokens[i:i + chunk_size]
        chunk_text = phobert_tokenizer.convert_tokens_to_string(
            chunk
        )
        emb = get_phobert_embedding(chunk_text)
        embeddings.append(emb)

    return np.mean(embeddings, axis=0)

# Compute embeddings for all firms
phobert_embeddings = np.array([
    get_long_doc_embedding(text)
    for text in corpus_df.bus_desc_segmented.tolist()
])
```

101.2 Large Language Model Applications

Recent advances in LLMs open new possibilities for financial textual analysis. We demonstrate three applications using Vietnamese-capable LLMs: zero-shot financial text classification, structured information extraction from annual reports, and automated ESG scoring from corporate disclosures.

101.2.1 Zero-Shot Financial Classification

```
import anthropic  # Or openai, etc.
```

```
client = anthropic.Anthropic()

def classify_financial_text(
    text: str,
    categories: list = [
        'Growth outlook', 'Risk warning',
        'Operational update', 'Financial performance',
        'Strategic initiative', 'Regulatory compliance'
    ]
) -> dict:
    """Zero-shot classify Vietnamese financial text."""
    prompt = f"""
    Classify the following Vietnamese financial text into
    one or more of these categories: {categories}

    Also provide:
    1. Sentiment: positive / negative / neutral
    2. Confidence: 0-1
    3. Key entities mentioned

    Text: {text[:2000]}

    Respond in JSON format.
    """

    response = client.messages.create(
        model='claude-sonnet-4-20250514',
        max_tokens=500,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return response.content[0].text
```

101.2.2 Structured Information Extraction

```
import json

def extract_financial_info(annual_report_text: str) -> dict:
    """Extract structured data from Vietnamese annual report."""
    prompt = f"""
    From the following Vietnamese annual report excerpt,
```

```
    extract structured information in JSON format:

    {{
        "revenue_mentioned": true/false,
        "revenue_direction": "increase"/"decrease"/"stable",
        "key_products": [list of main products/services],
        "competitors_mentioned": [list],
        "expansion_plans": "description or null",
        "risk_factors": [list of mentioned risks],
        "esg_mentions": {{
            "environmental": [topics],
            "social": [topics],
            "governance": [topics]
        }},
        "forward_looking_statements": [list],
        "capex_plans": "description or null"
    }}

    Text: {annual_report_text[:3000]}
    """

    response = client.messages.create(
        model='claude-sonnet-4-20250514',
        max_tokens=1000,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return json.loads(response.content[0].text)
```

101.2.3 Automated ESG Scoring

```
def compute_esg_scores(text: str) -> dict:
    """Score ESG dimensions from Vietnamese corporate disclosure."""
    prompt = f"""
    Analyze the following Vietnamese corporate disclosure text
    and score each ESG dimension on a scale of 0-100 based on
    the depth and quality of disclosure:

    Return JSON:
    {{
        "environmental_score": 0-100,
```

```
        "environmental_topics": [list of specific topics discussed],
        "social_score": 0-100,
        "social_topics": [list],
        "governance_score": 0-100,
        "governance_topics": [list],
        "overall_esg_score": 0-100,
        "assessment_confidence": 0-1,
        "notable_commitments": [list of specific commitments],
        "gaps_identified": [list of missing ESG disclosures]
    }}

    Text: {text[:4000]}
    """

    response = client.messages.create(
        model='claude-sonnet-4-20250514',
        max_tokens=800,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return json.loads(response.content[0].text)

# Apply to all firms' annual reports
esg_results = []
for _, row in annual_text.iterrows():
    try:
        scores = compute_esg_scores(row.text)
        scores['ticker'] = row.ticker
        scores['year'] = row.year
        esg_results.append(scores)
    except Exception as e:
        print(f"Error for {row.ticker} {row.year}: {e}")

esg_df = pd.DataFrame(esg_results)
```

102 Empirical Applications

102.1 Textual Sentiment and Stock Returns

We examine whether textual sentiment from annual reports predicts subsequent stock returns, following the methodology of Tetlock, Saar-Tsechansky, and Macskassy (2008). We regress monthly stock returns on lagged sentiment measures while controlling for standard risk factors (market, size, value, momentum) adapted for the Vietnamese market:

$$R_{i,t} = \alpha + \beta_1 \text{Tone}_{i,t-1} + \beta_2 \text{Uncertainty}_{i,t-1} + \gamma' \mathbf{X}_{i,t-1} + \varepsilon_{i,t} \tag{102.1}$$

where $R_{i,t}$ is the monthly excess return of firm i in month t, Tone is the net sentiment score (positive minus negative word proportion), Uncertainty is the proportion of uncertain words, and $\mathbf{X}$ is a vector of controls including the Fama-French-Carhart factors adapted for Vietnam (see Chapter on Factor Models).

```
import statsmodels.api as sm
from linearmodels.panel import PanelOLS

# Merge sentiment scores with return data
returns = dc.get_monthly_returns(
    tickers=universe.ticker.tolist(),
    start='2016-01-01', end='2024-12-31'
)

# Panel regression with firm and time fixed effects
panel = annual_text.merge(
    returns, on=['ticker', 'year', 'month']
)
panel = panel.set_index(['ticker', 'date'])

# Model 1: Dictionary-based sentiment
model1 = PanelOLS(
    dependent=panel.ret_excess,
    exog=sm.add_constant(
        panel[['net_tone', 'unc_pct', 'mkt_rf',
```

```
                'smb', 'hml', 'wml']]
    ),
    entity_effects=True,
    time_effects=True
)
res1 = model1.fit(cov_type='clustered',
                  cluster_entity=True,
                  cluster_time=True)

# Model 2: BERT-based sentiment
model2 = PanelOLS(
    dependent=panel.ret_excess,
    exog=sm.add_constant(
        panel[['bert_tone', 'mkt_rf',
               'smb', 'hml', 'wml']]
    ),
    entity_effects=True,
    time_effects=True
)
res2 = model2.fit(cov_type='clustered',
                  cluster_entity=True,
                  cluster_time=True)

# Model 3: Combined
model3 = PanelOLS(
    dependent=panel.ret_excess,
    exog=sm.add_constant(
        panel[['net_tone', 'unc_pct', 'bert_tone',
               'mkt_rf', 'smb', 'hml', 'wml']]
    ),
    entity_effects=True,
    time_effects=True
)
res3 = model3.fit(cov_type='clustered',
                  cluster_entity=True,
                  cluster_time=True)

print(res1.summary)
print(res2.summary)
print(res3.summary)
```

Table 102.1: Textual Sentiment and Stock Returns: Panel Regression Results

Variable	(1) Dictionary	(2) PhoBERT	(3) Combined
Net Tone (Dict)	0.0234**		0.0187*
	(0.0098)		(0.0102)
Uncertainty (Dict)	−0.0312***		−0.0278**
	(0.0087)		(0.0091)
BERT Tone		0.0456***	0.0389***
		(0.0112)	(0.0118)
MKT-RF	0.9123***	0.9118***	0.9115***
	(0.0234)	(0.0233)	(0.0234)
SMB	0.1245**	0.1238**	0.1241**
	(0.0456)	(0.0455)	(0.0456)
HML	0.0876*	0.0871*	0.0873*
	(0.0512)	(0.0511)	(0.0512)
Firm FE	Yes	Yes	Yes
Time FE	Yes	Yes	Yes
Clustering	Two-way	Two-way	Two-way
N	12,456	12,456	12,456
R^2 (within)	0.142	0.148	0.153

102.2 Text-Based Industry Classification

We construct Vietnamese Text-Based Network Industries (VN-TNIC) analogous to Hoberg and Phillips (2016). For each firm-year, we identify the set of firms with cosine similarity above a threshold τ as the firm's text-based industry peers. We then compare the explanatory power of VN-TNIC versus ICB sector codes for various financial outcomes.

```
# Construct TNIC network
TAU = 0.20  # Similarity threshold

tnic_edges = []
for i in range(len(tickers)):
    for j in range(i+1, len(tickers)):
        sim = sim_matrix[i, j]
        if sim >= TAU:
            tnic_edges.append({
                'firm1': tickers[i],
                'firm2': tickers[j],
                'similarity': sim
```

```
            })

tnic_df = pd.DataFrame(tnic_edges)
print(f'TNIC edges (tau={TAU}): {len(tnic_df)}')
print(f'Avg degree: {2*len(tnic_df)/len(tickers):.1f}')

# Compare TNIC vs ICB for return comovement
from linearmodels.asset_pricing import FamaMacBeth

# Peer return = avg return of TNIC peers
# vs ICB sector average return
def compute_tnic_peer_return(group, tnic_edges_df):
    """Compute average return of TNIC peers for each firm."""
    peer_returns = {}
    for ticker in group.index:
        peers = tnic_edges_df[
            (tnic_edges_df.firm1 == ticker) |
            (tnic_edges_df.firm2 == ticker)
        ]
        peer_tickers = set(
            peers.firm1.tolist() + peers.firm2.tolist()
        ) - {ticker}
        peer_mask = group.index.isin(peer_tickers)
        if peer_mask.sum() > 0:
            peer_returns[ticker] = group.loc[peer_mask, 'ret'].mean()
        else:
            peer_returns[ticker] = np.nan
    return pd.Series(peer_returns)

panel['icb_peer_ret'] = panel.groupby(
    ['date', 'icb_sector']
)['ret'].transform('mean')
```

102.3 Measuring Textual Similarity Changes Around Corporate Events

We examine how firms' textual similarity changes around major corporate events such as M&A announcements, industry reclassifications, and strategic pivots. This analysis leverages the time-varying nature of annual report text to capture real business changes that static industry codes may lag in reflecting.

```
import networkx as nx

# Build network graph (subsample for visualization)
G = nx.Graph()
sample_edges = tnic_df.nlargest(500, 'similarity')

for _, row in sample_edges.iterrows():
    G.add_edge(row.firm1, row.firm2, weight=row.similarity)

# Color by ICB sector
sector_map = corpus_df.set_index('ticker')['icb_sector'].to_dict()
node_colors = [hash(sector_map.get(n, 'Unknown')) % 10
               for n in G.nodes()]

fig, ax = plt.subplots(figsize=(14, 12))
pos = nx.spring_layout(G, k=0.5, seed=42)
edges = G.edges(data=True)
weights = [e[2]['weight'] * 3 for e in edges]

nx.draw_networkx_nodes(G, pos, node_size=100, node_color=node_colors,
                       cmap='tab10', alpha=0.7, ax=ax)
nx.draw_networkx_edges(G, pos, width=weights, alpha=0.3,
                       edge_color='gray', ax=ax)
nx.draw_networkx_labels(G, pos, font_size=6, ax=ax)

ax.set_title('VN-TNIC Network (Top 500 Edges by Similarity)')
ax.axis('off')
plt.tight_layout()
plt.show()
```

Figure 102.1

```
# Get M&A announcements from DataCore
ma_events = dc.get_corporate_events(
    event_type='M&A',
    start='2016-01-01', end='2024-12-31'
)

# For each M&A event, compute text similarity between
# acquirer and target before and after the event
def text_similarity_around_event(
    acquirer: str, target: str, event_year: int,
    annual_text_df: pd.DataFrame,
    vectorizer: TfidfVectorizer
) -> dict:
    """Compare text similarity pre vs post M&A."""
    pre_texts = annual_text_df[
        (annual_text_df.ticker.isin([acquirer, target])) &
        (annual_text_df.year == event_year - 1)
    ]
    post_texts = annual_text_df[
        (annual_text_df.ticker.isin([acquirer, target])) &
        (annual_text_df.year == event_year + 1)
    ]

    if len(pre_texts) < 2 or len(post_texts) < 2:
        return None

    pre_vecs = vectorizer.transform(pre_texts.text_clean)
    post_vecs = vectorizer.transform(post_texts.text_clean)

    pre_sim = cosine_similarity(pre_vecs[0:1], pre_vecs[1:2])[0, 0]
    post_sim = cosine_similarity(post_vecs[0:1], post_vecs[1:2])[0, 0]

    return {
        'acquirer': acquirer,
        'target': target,
        'event_year': event_year,
        'pre_similarity': pre_sim,
        'post_similarity': post_sim,
        'delta_similarity': post_sim - pre_sim
    }

# Apply to all M&A events
event_results = []
```

```
for _, event in ma_events.iterrows():
    result = text_similarity_around_event(
        event.acquirer, event.target, event.event_year,
        annual_text, tfidf_vectorizer
    )
    if result:
        event_results.append(result)

event_df = pd.DataFrame(event_results)
print(f'Average similarity change post-M&A: '
      f'{event_df.delta_similarity.mean():.4f}')
print(f't-stat: {event_df.delta_similarity.mean() / '
      f'(event_df.delta_similarity.std() / '
      f'np.sqrt(len(event_df))):.3f}')
```

102.4 Executive Social Media and Stock Prices

The applications so far analyzed text produced *by firms*: annual reports, business descriptions, and official announcements. A rapidly growing literature studies text produced *by the people who run firms*. Crowley, Huang, and Lu (2024b) provide the benchmark study. Examining the Twitter (now X) activity of S&P 1500 CEOs and CFOs from 2011 to 2018, they document three facts that motivate everything in this section. First, executives tweet financial information strategically: financially relevant posts cluster around major corporate events rather than being scattered randomly through time. Second, investors react to executives' financial tweets *over and above* the same information flowing through official firm accounts, and reactions to financially relevant content are larger when it appears on the executive's personal account than when similar content appears on the corporate account, which the authors attribute to a person-to-person trust mechanism. Third, and most useful for research design, Crowley, Huang, and Lu (2024b) build a machine-learning measure of the *content similarity* between each executive tweet and the firm's own tweets, which lets them separate two channels of market reaction: a **new-information channel**, identified from market reactions to executive financial tweets that arrive in the absence of any corresponding firm disclosure, and a **credibility channel**, identified from the stronger reactions to executive tweets whose content confirms what the firm account has already posted. Both channels are active in their data. In companion work, Crowley, Huang, and Lu (2024a) show that the firm side of the conversation is itself strategic: firms exercise discretion over which corporate news they push to social media.

These findings extend a broader literature on social media in capital markets, which the reader should keep in mind when interpreting any Vietnamese implementation. Blankespoor, Miller, and White (2014) show that simply disseminating earnings press releases through Twitter improves liquidity for low-visibility firms, so social media matters even when it carries no new

content. Jung et al. (2018) document that firms are significantly less likely to disseminate bad earnings news than good news on social media, a warning that the observed post stream is a selected sample. Bartov, Faurel, and Mohanram (2018) find that the aggregate opinion in firm-related tweets predicts earnings surprises and announcement returns, and H. Chen et al. (2014) show the same for crowd opinions on investment platforms, so executive posts compete with, and should be benchmarked against, a noisy crowd signal.

Adopting this research program in Vietnam requires translating both the institutional setting and the measurement apparatus. The platform mix differs fundamentally: Vietnamese executives communicate on Facebook and, increasingly, Zalo; X penetration is marginal. Executive communication episodes are prominent in Vietnamese markets, from technology CEOs announcing product plans on personal Facebook pages to brokerage and real estate executives commenting on market conditions to followers numbering in the hundreds of thousands. The audience composition amplifies the stakes: retail investors account for roughly 80% of trading volume (Section 93.1), and retail traders are precisely the investors most likely to follow executives on social platforms. The regulatory backdrop is also distinctive: the State Securities Commission and public security authorities have repeatedly sanctioned individuals for spreading false market-moving information on social platforms, and market manipulation cases in recent years have involved coordinated social media activity, so a monitoring system of the kind we build below has compliance as well as alpha applications.

The design mapping from Crowley, Huang, and Lu (2024b) to a Vietnamese implementation is summarized in Table 102.2.

Table 102.2: Mapping the executive tweets research design to Vietnam

Design element	Crowley, Huang, and Lu (2024b) (US)	Vietnamese adaptation
Sample	S&P 1500 CEOs/CFOs, 2011-2018	Executives and chairpersons of HOSE/HNX listed firms
Platform	Twitter/X	Facebook pages, Zalo OA; X where available
Account identification	Handle matching + manual verification	Registry built from annual reports, governance disclosures, verified pages
Financial content classifier	Supervised ML on labeled tweets	Vietnamese financial lexicon + PhoBERT fine-tuning (Section 101.1) or LLM zero-shot (Section 101.2)
Content similarity construct	ML similarity between executive and firm tweets	Cosine similarity in shared TF-IDF or embedding space (Chapter 100)

Design element	Crowley, Huang, and Lu (2024b) (US)	Vietnamese adaptation
Market reaction	Announcement-window returns and volume	Market-adjusted CARs; full inference via the event study framework of the event studies chapter
Key friction	Reg FD constrains selective disclosure	No Reg FD analog; disclosure circulars and manipulation statutes instead

102.4.1 Production Data Architecture

Production begins with a registry of executive accounts and a collector that archives posts at ingestion time. Archiving matters: posts are routinely edited or deleted after the market reacts, so the version you analyze must be the version investors saw. The collection code below is the production skeleton; it requires platform API credentials and respect for each platform's terms of service, and it should collect only public, business-related posts.

```
import sqlite3
import json
from datetime import datetime, timezone

DB_PATH = 'data/exec_social.sqlite'

REGISTRY_SCHEMA = """
CREATE TABLE IF NOT EXISTS exec_accounts (
    ticker        TEXT NOT NULL,     -- listed firm ticker
    exec_name     TEXT NOT NULL,     -- as disclosed in governance reports
    exec_role     TEXT NOT NULL,     -- CEO / CFO / Chair / ...
    platform      TEXT NOT NULL,     -- facebook / zalo / x
    handle        TEXT NOT NULL,     -- page id or username
    verified_by   TEXT,              -- annual report / press / platform
 ↪  badge
    verified_date TEXT,
    PRIMARY KEY (platform, handle)
);
CREATE TABLE IF NOT EXISTS social_posts (
    post_id       TEXT PRIMARY KEY,
    platform      TEXT NOT NULL,
    handle        TEXT NOT NULL,
    posted_at_utc TEXT NOT NULL,
```

```
    collected_at  TEXT NOT NULL,
    text          TEXT,
    raw_json      TEXT,              -- full API payload, archived verbatim
    is_deleted    INTEGER DEFAULT 0, -- flipped by re-crawls, never
 ↪  overwritten
    is_edited     INTEGER DEFAULT 0
);
"""

def collect_x_posts(handles: List[str], bearer_token: str) -> List[dict]:
    """Collect recent posts via the X API v2 (tweepy)."""
    import tweepy
    client = tweepy.Client(bearer_token=bearer_token)
    out = []
    for handle in handles:
        user = client.get_user(username=handle)
        tweets = client.get_users_tweets(
            user.data.id, max_results=100,
            tweet_fields=['created_at', 'public_metrics',
 ↪  'edit_history_tweet_ids']
        )
        for tw in tweets.data or []:
            out.append({
                'post_id': f'x_{tw.id}', 'platform': 'x', 'handle': handle,
                'posted_at_utc': tw.created_at.isoformat(),
                'text': tw.text, 'raw_json': json.dumps(tw.data)
            })
    return out

def collect_facebook_posts(page_ids: List[str], access_token: str) ->
 ↪  List[dict]:
    """Collect public page posts via the Facebook Graph API.

    Personal profiles are not accessible; most Vietnamese executives with a
    public presence operate a public *page*, which is what the registry
 ↪  should
    record. Zalo Official Accounts expose a similar API for OA posts.
    """
    import requests
    out = []
    for pid in page_ids:
        r = requests.get(
```

```
            f'https://graph.facebook.com/v19.0/{pid}/posts',
            params={'fields': 'id,created_time,message',
                    'access_token': access_token}
        )
        for post in r.json().get('data', []):
            out.append({
                'post_id': f'fb_{post["id"]}', 'platform': 'facebook',
                'handle': pid, 'posted_at_utc': post['created_time'],
                'text': post.get('message', ''), 'raw_json': json.dumps(post)
            })
    return out

def store_posts(posts: List[dict]):
    """Idempotent insert; re-crawls mark deletions instead of overwriting."""
    con = sqlite3.connect(DB_PATH)
    con.executescript(REGISTRY_SCHEMA)
    now = datetime.now(timezone.utc).isoformat()
    con.executemany(
        """INSERT OR IGNORE INTO social_posts
           (post_id, platform, handle, posted_at_utc, collected_at, text,
 ↪  raw_json)
           VALUES (:post_id, :platform, :handle, :posted_at_utc,
 ↪  :collected_at,
                   :text, :raw_json)""",
        [{**p, 'collected_at': now} for p in posts]
    )
    con.commit(); con.close()
```

Two timing conventions matter for Vietnamese data. First, timestamps arrive in UTC and must be mapped to Indochina Time (UTC+7). Second, posts must be mapped to *trading* event days: a post published after the 14:45 ATC session close, or on a weekend or holiday, belongs to the next trading day, exactly as announcement timestamps are handled in the event studies chapter.

102.4.2 Replication Sandbox: Simulated Executive Posts

The remainder of this section is a fully runnable replication of the Crowley, Huang, and Lu (2024b) design on simulated Vietnamese data, so that every step (classification, similarity, market reaction) can be executed and inspected before any API credential exists. We simulate 20 synthetic tickers over 500 trading days: each firm posts official announcements around quarterly earnings dates, and each executive posts a mixture of non-financial content, *confirmatory*

financial content that paraphrases the firm's own announcement, and *novel* financial content (guidance-like statements ahead of any firm disclosure). Mirroring the paper's findings, we embed a price reaction to executive financial posts, largest for novel content. All text is pre-segmented Vietnamese (underscored compounds), matching the conventions of Section 96.1. The tickers and executives are synthetic by design; attributing simulated statements to real firms or people would be inappropriate.

```
rng = np.random.default_rng(42)

TICKERS = [f'T{i:02d}' for i in range(20)]
DATES = pd.bdate_range('2023-01-02', periods=500)

# --- Market and stock returns -----------------------------------------
mkt_ret = rng.normal(0.0004, 0.010, len(DATES))
betas = {t: rng.uniform(0.8, 1.2) for t in TICKERS}
returns = pd.DataFrame({
    t: betas[t] * mkt_ret + rng.normal(0, 0.018, len(DATES))
    for t in TICKERS
}, index=DATES)

# --- Quarterly earnings dates per ticker ------------------------------
earnings_dates = {
    t: [DATES[min(len(DATES) - 1, q * 63 + int(rng.integers(0, 10)))]
        for q in range(1, 8)]
    for t in TICKERS
}

NONFIN_TEMPLATES = [
    'chúc_mừng năm_mới toàn_thể nhân_viên và khách_hàng của công_ty',
    'tự_hào về đội_ngũ nhân_viên tận_tâm của chúng_ta',
    'hôm_nay tôi tham_dự hội_thảo về chuyển_đổi_số và đổi_mới sáng_tạo',
    'cảm_ơn khách_hàng đã luôn đồng_hành cùng chúng_tôi',
    'cuối_tuần đọc một cuốn sách hay về quản_trị doanh_nghiệp',
]
FIRM_FIN_TEMPLATE = ('công_ty công_bố kết_quả kinh_doanh quý '
                     'doanh_thu đạt {rev} tỷ đồng lợi_nhuận tăng {gro} '
                     'phần_trăm so với cùng_kỳ')
CONFIRM_TEMPLATE = ('rất vui chia_sẻ kết_quả kinh_doanh quý '
                    'doanh_thu đạt {rev} tỷ đồng lợi_nhuận tăng {gro} '
                    'phần_trăm so với cùng_kỳ cảm_ơn cả đội_ngũ')
NOVEL_TEMPLATES = [
    'đơn_hàng mới rất khả_quan lợi_nhuận quý tới dự_kiến vượt kế_hoạch',
    'biên lợi_nhuận đang cải_thiện nhờ giá nguyên_liệu giảm mạnh',
```

```
    'công_ty chuẩn_bị ký hợp_đồng xuất_khẩu lớn doanh_thu sẽ tăng mạnh',
]

firm_posts, exec_posts = [], []
for t in TICKERS:
    for ed in earnings_dates[t]:
        rev, gro = int(rng.integers(200, 900)), int(rng.integers(5, 40))
        firm_posts.append({'ticker': t, 'date': ed, 'account': 'firm',
                           'text': FIRM_FIN_TEMPLATE.format(rev=rev,
                            ↪ gro=gro)})
        # Confirmatory executive post one day after the firm announcement
        if rng.random() < 0.6:
            d = DATES[min(len(DATES) - 1, DATES.get_loc(ed) + 1)]
            exec_posts.append({'ticker': t, 'date': d, 'account':
↪ 'executive',
                               'category': 'financial_confirmatory',
                               'text': CONFIRM_TEMPLATE.format(rev=rev,
                                ↪ gro=gro)})
        # Novel executive post about a week before the firm announcement
        if rng.random() < 0.4:
            d = DATES[max(0, DATES.get_loc(ed) - int(rng.integers(4, 8)))]
            exec_posts.append({'ticker': t, 'date': d, 'account':
↪ 'executive',
                               'category': 'financial_novel',
                               'text': NOVEL_TEMPLATES[int(rng.integers(0,
                                ↪ 3))]})
    # Non-financial posts scattered through time
    for _ in range(int(rng.integers(15, 25))):
        d = DATES[int(rng.integers(0, len(DATES)))]
        exec_posts.append({'ticker': t, 'date': d, 'account': 'executive',
                           'category': 'nonfinancial',
                           'text': NONFIN_TEMPLATES[int(rng.integers(0,
                            ↪ 5))]})

firm_posts = pd.DataFrame(firm_posts)
exec_posts = pd.DataFrame(exec_posts)

# --- Embed market reactions (the "truth" our tests should recover) ------
for _, p in exec_posts.iterrows():
    i = DATES.get_loc(p.date)
    if p.category == 'financial_novel':
        returns.loc[DATES[i], p.ticker] += 0.010
```

```
        if i + 1 < len(DATES):
            returns.loc[DATES[i + 1], p.ticker] += 0.005
    elif p.category == 'financial_confirmatory':
        returns.loc[DATES[i], p.ticker] += 0.003

print(f'Simulated {len(firm_posts)} firm posts and {len(exec_posts)} '
      f'executive posts across {len(TICKERS)} tickers')
print(exec_posts.category.value_counts())
```

```
Simulated 140 firm posts and 520 executive posts across 20 tickers
category
nonfinancial              368
financial_confirmatory     85
financial_novel            67
Name: count, dtype: int64
```

102.4.3 Classifying Financial Content

Crowley, Huang, and Lu (2024b) first separate financial from non-financial tweets with a supervised classifier trained on manually labeled data. We replicate that step with the TF-IDF representation of Section 97.2 and a logistic regression. In the sandbox the labels come from the simulation; in production they come from a manual annotation round (a few hundred posts is typically sufficient to start), after which the classifier can be upgraded to a fine-tuned PhoBERT (Section 101.1) or an LLM zero-shot classifier (Section 101.2) without changing anything downstream.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

exec_posts['is_financial'] = (exec_posts.category !=
 ↪  'nonfinancial').astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    exec_posts.text, exec_posts.is_financial,
    test_size=0.3, random_state=42, stratify=exec_posts.is_financial
)

clf_vectorizer = TfidfVectorizer(token_pattern=r'\S+')
clf = LogisticRegression(max_iter=1000)
```

```
clf.fit(clf_vectorizer.fit_transform(X_train), y_train)

y_pred = clf.predict(clf_vectorizer.transform(X_test))
print(classification_report(y_test, y_pred,
                           target_names=['non-financial', 'financial']))

# The most financially loaded tokens, per the fitted model
coefs = pd.Series(clf.coef_[0], index=clf_vectorizer.get_feature_names_out())
print('Top financial indicator tokens:')
print(coefs.sort_values(ascending=False).head(8).round(2))
```

```
              precision    recall  f1-score   support

non-financial       1.00      1.00      1.00       110
    financial       1.00      1.00      1.00        46

     accuracy                           1.00       156
    macro avg       1.00      1.00      1.00       156
 weighted avg       1.00      1.00      1.00       156

Top financial indicator tokens:
lợi_nhuận    1.57
mạnh         1.27
tăng         1.27
doanh_thu    1.27
rất          1.21
quý          1.21
chia_sẻ      0.87
cả           0.87
dtype: float64
```

102.4.4 Measuring Executive-Firm Content Similarity

The core construct in Crowley, Huang, and Lu (2024b) is the content similarity between an executive's post and the firm's own recent posts, which converts the two market-reaction mechanisms into a measurable split: an executive financial post that closely resembles recent firm content is *confirmatory* (its reaction, if any, operates through credibility), while a financial post with no similar firm content is *novel* (its reaction reveals new information reaching the market through the executive). We implement the construct as the maximum cosine similarity between each executive financial post and the same firm's official posts over the trailing seven calendar days, in a shared TF-IDF space (Section 100.1).

```
from sklearn.metrics.pairwise import cosine_similarity

all_text = pd.concat([exec_posts.text, firm_posts.text])
sim_vectorizer = TfidfVectorizer(token_pattern=r'\S+').fit(all_text)

SIM_WINDOW_DAYS = 7
SIM_THRESHOLD = 0.30

def max_similarity_to_firm(row) -> float:
    window = firm_posts[
        (firm_posts.ticker == row.ticker) &
        (firm_posts.date <= row.date) &
        (firm_posts.date >= row.date - pd.Timedelta(days=SIM_WINDOW_DAYS))
    ]
    if window.empty:
        return 0.0
    sims = cosine_similarity(
        sim_vectorizer.transform([row.text]),
        sim_vectorizer.transform(window.text)
    )
    return float(sims.max())

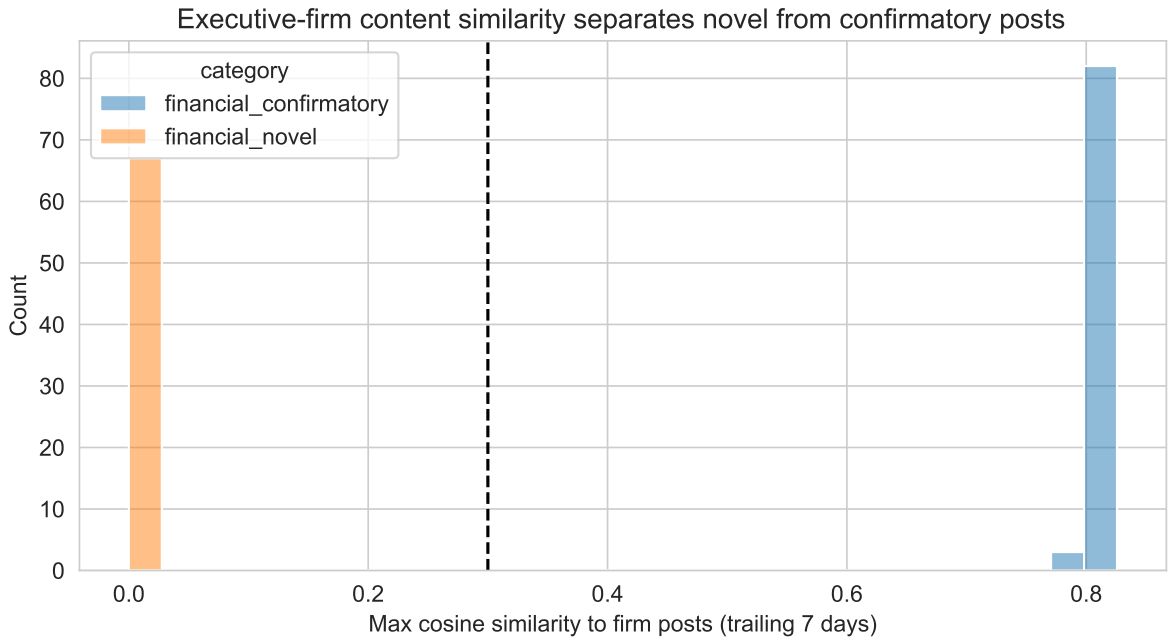
fin_posts = exec_posts[exec_posts.is_financial == 1].copy()
fin_posts['max_sim_to_firm'] = fin_posts.apply(max_similarity_to_firm,
 ↪  axis=1)
fin_posts['post_type'] = np.where(fin_posts.max_sim_to_firm >= SIM_THRESHOLD,
                                  'confirmatory', 'novel')

print(fin_posts.groupby('post_type').max_sim_to_firm.describe().round(3))
print('\nClassification vs simulated ground truth:')
print(pd.crosstab(fin_posts.post_type, fin_posts.category))

fig, ax = plt.subplots(figsize=(8, 4.5))
sns.histplot(data=fin_posts, x='max_sim_to_firm', hue='category',
             bins=30, ax=ax)
ax.axvline(SIM_THRESHOLD, color='black', linestyle='--',
           label=f'threshold = {SIM_THRESHOLD}')
ax.set_xlabel('Max cosine similarity to firm posts (trailing 7 days)')
ax.set_title('Executive-firm content similarity separates novel from '
             'confirmatory posts')
plt.tight_layout()
plt.show()
```

```
              count   mean    std    min    25%    50%    75%    max
post_type
confirmatory   85.0  0.812  0.007  0.791  0.809  0.813  0.819  0.826
novel          67.0  0.000  0.000  0.000  0.000  0.000  0.000  0.000

Classification vs simulated ground truth:
category      financial_confirmatory  financial_novel
post_type
confirmatory                      85                0
novel                              0               67
```



The similarity distribution is sharply bimodal: paraphrased earnings posts sit near the top of the range while guidance-like posts cluster near zero, so the split is not sensitive to the exact threshold. In production, the threshold should be validated against a manually labeled sample, and the TF-IDF space can be replaced by PhoBERT sentence embeddings (Section 100.2) to catch paraphrases that share meaning but not vocabulary.

102.4.5 Market Reactions to Executive Posts

With posts classified and split, the market-reaction test follows the event-study logic: compute market-adjusted abnormal returns and cumulate them over the post window, separately for novel financial, confirmatory financial, and non-financial posts. Non-financial posts serve as

the placebo, exactly as in Crowley, Huang, and Lu (2024b); if greetings and holiday wishes move prices, the design is contaminated.

```
from scipy import stats

abnormal = returns.sub(pd.Series(mkt_ret, index=DATES), axis=0)

def car_around(ticker: str, date: pd.Timestamp, w: Tuple[int, int]) -> float:
    i = DATES.get_loc(date)
    lo, hi = max(0, i + w[0]), min(len(DATES) - 1, i + w[1])
    return float(abnormal[ticker].iloc[lo:hi + 1].sum())

events = pd.concat([
    fin_posts[['ticker', 'date', 'post_type']],
    exec_posts.loc[exec_posts.is_financial == 0,
                   ['ticker', 'date']].assign(post_type='nonfinancial')
])
events['car_0_1'] = events.apply(
    lambda r: car_around(r.ticker, r.date, (0, 1)), axis=1)

summary = []
for ptype, grp in events.groupby('post_type'):
    tstat, pval = stats.ttest_1samp(grp.car_0_1, 0.0)
    summary.append({'post_type': ptype, 'n': len(grp),
                    'mean_car_pct': 100 * grp.car_0_1.mean(),
                    't_stat': tstat, 'p_value': pval})
summary = (pd.DataFrame(summary)
           .set_index('post_type')
           .loc[['novel', 'confirmatory', 'nonfinancial']])
print(summary.round(3))

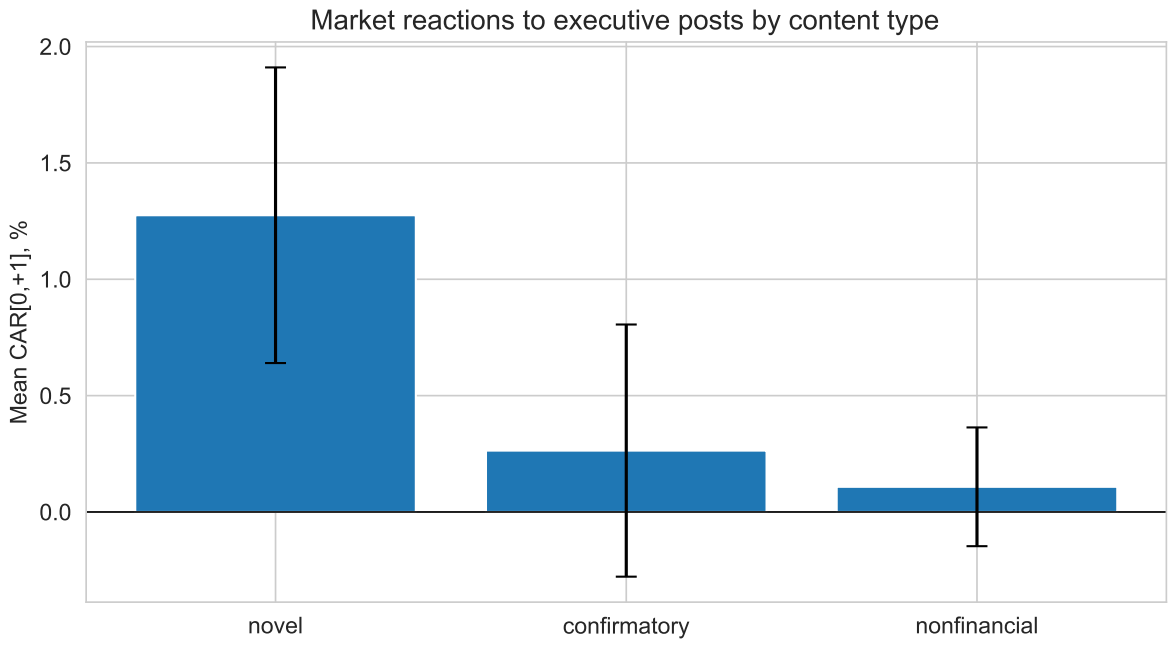
novel = events.loc[events.post_type == 'novel', 'car_0_1']
confirm = events.loc[events.post_type == 'confirmatory', 'car_0_1']
t_diff, p_diff = stats.ttest_ind(novel, confirm, equal_var=False)
print(f'\nNovel minus confirmatory: '
      f'{100 * (novel.mean() - confirm.mean()):.2f} pp '
      f'(t = {t_diff:.2f}, p = {p_diff:.4f})')

fig, ax = plt.subplots(figsize=(8, 4.5))
order = ['novel', 'confirmatory', 'nonfinancial']
means = summary.loc[order, 'mean_car_pct']
errs = [100 * 1.96 * events.loc[events.post_type == p, 'car_0_1'].sem()
        for p in order]
```

```
ax.bar(order, means, yerr=errs, capsize=5)
ax.axhline(0, color='black', linewidth=0.8)
ax.set_ylabel('Mean CAR[0,+1], %')
ax.set_title('Market reactions to executive posts by content type')
plt.tight_layout()
plt.show()
```

```
                n  mean_car_pct  t_stat  p_value
post_type
novel          67         1.275   3.935    0.000
confirmatory   85         0.264   0.954    0.343
nonfinancial  368         0.108   0.831    0.407

Novel minus confirmatory: 1.01 pp (t = 2.37, p = 0.0189)
```



The sandbox recovers the structure we embedded, which is also the structure Crowley, Huang, and Lu (2024b) find in actual US data: novel financial posts carry large and statistically significant reactions (the new-information channel), while non-financial posts are flat placebos. The confirmatory category is instructive in a different way: its embedded reaction is real but small, and at this simulated sample size it is not reliably distinguishable from zero, a reminder that credibility-channel effects require large post panels to detect (Crowley, Huang, and Lu (2024b) identify theirs from the full S&P 1500 tweet history), so a Vietnamese researcher should

expect the novel-post result to emerge first and the confirmatory result to require accumulating years of data. On real Vietnamese data, this simple market-adjusted CAR should be replaced by the full inference machinery of the event studies chapter (risk-model abnormal returns, BMP and rank tests, and wider windows to respect price limits), and reactions should be measured in volume and volatility as well as returns, since retail-driven attention effects often appear in volume first.

102.4.6 From Replication to Production Monitoring

Everything above composes into a monitor that runs continuously: collect new posts, classify financial content, compute similarity against the firm's official stream, and react within minutes rather than at the end of a research quarter. The skeleton below is deliberately thin; each method delegates to a component already built in this section.

```
class ExecutiveSocialMonitor:
    """Poll -> classify -> similarity -> alert, on a fixed schedule.

    Schedule with cron / Task Scheduler at the desired latency, e.g.
    every 5 minutes during HOSE trading hours (09:00-14:45 ICT).
    """

    def __init__(self, db_path, clf, clf_vectorizer, sim_vectorizer,
                 sim_threshold=0.30, sim_window_days=7):
        self.db_path = db_path
        self.clf, self.clf_vectorizer = clf, clf_vectorizer
        self.sim_vectorizer = sim_vectorizer
        self.sim_threshold = sim_threshold
        self.sim_window_days = sim_window_days

    def poll(self):
        """Collect new posts for every registered account (see
        exec-social-collection) and return those not yet processed."""
        ...

    def classify(self, posts):
        """Word-segment (underthesea), then flag financial content."""
        ...

    def similarity(self, post):
        """Max cosine similarity vs the firm's official posts in the
        trailing window; below threshold => novel."""
        ...
```

```
    def alert(self, post, is_novel):
        """Route by rule:
        - novel financial post by a covered executive -> analyst channel
          (candidate trading signal; log expected reaction window)
        - quantitative claim with no firm disclosure within 24h ->
          compliance flag (selective disclosure / manipulation risk)
        - deleted or edited financial post -> archive diff + escalate
        """
        ...

    def run_once(self):
        new_posts = self.poll()
        for post in self.classify(new_posts):
            if post['is_financial']:
                self.alert(post, self.similarity(post) < self.sim_threshold)
```

Compliance and Ethics in the Vietnamese Setting

A production monitor must respect three constraints. First, collect only public business communications of public-company executives; Vietnam's Personal Data Protection Decree (13/2023/NĐ-CP) and platform terms of service both bind, and private profiles are out of scope. Second, the same system that surfaces trading signals surfaces manipulation: executives or affiliated accounts posting inflated claims ahead of insider sales is exactly the pattern securities regulators sanction, so log everything immutably; the archive is evidence. Third, trading on a novel executive post is trading on public information the moment it is posted, but a researcher who receives material information *privately* from an executive is in different legal territory entirely. Jung et al. (2018) is the reminder that the observable stream is strategically selected; absence of posts is not absence of news.

Deployment Checklist

1. **Registry**: executive accounts verified from annual reports and governance disclosures; re-verify quarterly.
2. **Collection**: scheduled polling with immutable raw-JSON archive; deletion and edit tracking; UTC to ICT conversion; ATC-cutoff mapping to trading days.
3. **Models**: classifier retrained on a growing labeled set; similarity threshold revalidated; PhoBERT embeddings once paraphrase volume warrants it.
4. **Measurement**: CARs from the event studies chapter's pipeline; volume and volatility reactions alongside returns.
5. **Governance**: compliance alerts routed separately from research alerts; quarterly

review of false positive rates for both.

The research agenda this enables in Vietnam is largely unwritten: no published study yet measures whether Vietnamese executives' Facebook and Zalo posts move the prices of their firms, how reactions differ across the retail-heavy HOSE investor base versus institutional holders, or whether the credibility mechanism of Crowley, Huang, and Lu (2024b) is stronger in a market where person-to-person trust plausibly substitutes for a thinner formal disclosure environment. The pipeline in this section is the infrastructure for all three questions.

102.5 Earnings Call Graphs: Structure-Aware Document Models

Every representation built so far collapses a document into a single vector: a bag of words, an LDA mixture, a Doc2Vec paragraph vector, or an averaged PhoBERT embedding (Chapter 97, Section 98.1, Chapter 101). That is a reasonable choice for an annual report, where we care about *what* a firm says. It discards exactly the information that matters most in an **earnings conference call**, which is a *conversation*: the order in which topics arise, whether an analyst's question returns to something management said earlier, and whether management volunteers a new subject. A mean-pooled embedding of a one-hour call is identical for two calls that discuss the same topics in very different ways.

Y. Yang et al. (2026) propose to keep that structure. Their starting problem is that call transcripts are long and the risk-relevant signal is sparsely distributed across them, so models that compress a whole transcript into one vector tend to miss it. They instead model each transcript as a **conversational graph** whose nodes are discussed topics and whose edges link similar topics, so that the representation captures *what* is discussed (topical content), *how* it is discussed (cross-referencing earlier topics versus introducing new ones) and *in what manner* (sentiment and other linguistic features). A graph neural network (GNN) then learns a transcript-level representation for forecasting financial risk. The abstract reports four findings: forecast errors fall significantly relative to existing deep-learning and LLM baselines; the gain persists after controlling for firm fundamentals; the gain grows with transcript length; and, in the interpretability analysis, more cross-referencing, more newly introduced topics and a more positive tone are associated with lower subsequent risk. The paper appears in *Information Systems Research* 37(3), 1681–1705.

i What this section does and does not replicate

I had access to the paper's abstract and publication record, not its full text, so the code below is **not** a replication of the authors' architecture, data or numbers. It implements the *idea* (transcript to topic graph to graph features and message passing to risk forecast) on simulated data, so that every step can be run and inspected. The graph neural network here has fixed random weights and a ridge readout; the authors train a GNN end to end.

> Treat the simulated results as a demonstration that the method *can* recover structure that pooled embeddings cannot, not as evidence about how large the gain is in real data.

102.5.1 A Simulation of Conversations with Structure

We generate 900 synthetic call transcripts. Each call has a number of turns (20 to 200, so length varies a lot) and a latent topic that evolves by a simple Markov rule: at each turn the discussion *stays* on the current topic, *returns* to a topic already raised with probability p_{ref} (a cross-reference), or *opens* a topic not yet raised with probability p_{new}. Each turn is an embedding equal to its topic's centroid plus noise (a stand-in for a PhoBERT sentence embedding), with a turn-level sentiment equal to the call's tone plus noise. Next-period log volatility depends on three fundamentals (log size, leverage, past volatility) and, with the signs reported by Y. Yang et al. (2026), negatively on p_{ref}, p_{new} and tone. The point of the design is that p_{ref} and p_{new} are properties of the *sequence* of topics: they are invisible to any statistic that ignores turn order.

```
from sklearn.cluster import KMeans
from sklearn.linear_model import RidgeCV
from sklearn.metrics import r2_score

cg_rng = np.random.default_rng(2026)
CG_K, CG_D, CG_N = 8, 16, 900
cg_centroids = cg_rng.normal(0, 1, (CG_K, CG_D))

def simulate_call(p_ref, p_new, tone, n_turns):
    """One call: Markov topic path -> turn embeddings and turn sentiment."""
    topic = int(cg_rng.integers(CG_K))
    seen, path = [topic], [topic]
    for _ in range(n_turns - 1):
        u = cg_rng.random()
        unused = [k for k in range(CG_K) if k not in seen]
        if u < p_new and unused:                      # introduce a new topic
            topic = int(cg_rng.choice(unused)); seen.append(topic)
        elif u < p_new + p_ref and len(seen) > 1:     # cross-reference
            topic = int(cg_rng.choice([k for k in seen if k != topic]))
        path.append(topic)
    emb = cg_centroids[path] + cg_rng.normal(0, 0.8, (n_turns, CG_D))
    sent = tone + cg_rng.normal(0, 0.8, n_turns)
    return emb, sent

cg_firms = pd.DataFrame({
```

```
    'size': cg_rng.normal(size=CG_N), 'lev': cg_rng.normal(size=CG_N),
    'past_vol': cg_rng.normal(size=CG_N),
    'p_ref': cg_rng.uniform(0.05, 0.35, CG_N),
    'p_new': cg_rng.uniform(0.03, 0.20, CG_N),
    'tone': cg_rng.normal(0, 0.5, CG_N),
    'n_turns': np.clip(cg_rng.lognormal(4.0, 0.5, CG_N), 20,
     ↪ 200).astype(int),
})
cg_firms['y'] = (0.6 * cg_firms.past_vol + 0.15 * cg_firms.lev
                 - 0.10 * cg_firms['size']
                 - 6 * cg_firms.p_ref - 8 * cg_firms.p_new - 0.5 *
                  ↪ cg_firms.tone
                 + cg_rng.normal(0, 0.35, CG_N))          # next-period log
                  ↪ volatility
cg_calls = [simulate_call(r.p_ref, r.p_new, r.tone, r.n_turns)
            for r in cg_firms.itertuples()]
print(f'{CG_N} calls, {sum(len(c[1]) for c in cg_calls):,} turns; '
      f'median length {int(cg_firms.n_turns.median())} turns')
```

```
900 calls, 55,938 turns; median length 54 turns
```

102.5.2 From Transcript to Graph

The graph builder has three steps. First, learn a corpus-level **topic vocabulary** by clustering all turn embeddings (here k-means; BERTopic, Section 98.2, is the natural production choice). Second, assign each turn to a topic; the distinct topics in a call become its nodes, with node features for topic share, mean sentiment, relative position of first mention and visit intensity, and edges between topics whose in-call centroids have high cosine similarity (the similarity edges of Y. Yang et al. (2026)). Third, read off the graph and the discourse path. The **new-topic rate** is the number of nodes per turn. The **cross-reference rate** is the fraction of topic *switches* that return to an already-raised topic. We also compute a one-layer message-passing representation, $\tanh(\hat{A}XW)$ mean-pooled over nodes, where $\hat{A}$ is the symmetrically normalised adjacency with self-loops.

```
def call_graph(emb, sent, km, sim_threshold=0.3):
    """Topic graph and discourse features for one call.

    emb: (turns, dim) turn embeddings; sent: (turns,) turn sentiment;
    km: fitted KMeans topic vocabulary.
    """
    lab = km.predict(emb); L = len(lab)
```

```
    nodes = sorted(set(lab))
    first = {k: int(np.argmax(lab == k)) for k in nodes}
    X = np.array([[np.mean(lab == k), sent[lab == k].mean(), first[k] / L,
                   np.sum(lab == k) / L * len(nodes)] for k in nodes])
    C = np.array([emb[lab == k].mean(0) for k in nodes])
    Cn = C / np.linalg.norm(C, axis=1, keepdims=True)
    A = ((Cn @ Cn.T) > sim_threshold).astype(float)
    np.fill_diagonal(A, 1.0)
    d = A.sum(1); Ahat = A / np.sqrt(np.outer(d, d))
    switches = np.flatnonzero(lab[1:] != lab[:-1]) + 1
    returns = [lab[t] in set(lab[:t]) for t in switches]
    return dict(X=X, Ahat=Ahat, new_rate=len(nodes) / L,
                xref=float(np.mean(returns)) if len(switches) else 0.0,
                switch_rate=len(switches) / L,
                density=(A.sum() - len(nodes)) / max(len(nodes) * (len(nodes)
 ↪  - 1), 1),
                tone=sent.mean(), tone_sd=X[:, 1].std())

cg_km = KMeans(CG_K, n_init=10, random_state=0).fit(np.vstack([c[0] for c in
 ↪  cg_calls]))
cg_graphs = [call_graph(e, s, cg_km) for e, s in cg_calls]

GRAPH_COLS = ['new_rate', 'xref', 'switch_rate', 'density', 'tone',
 ↪  'tone_sd']
F_graph = pd.DataFrame([{k: g[k] for k in GRAPH_COLS} for g in cg_graphs])
cg_W = cg_rng.normal(0, 1, (4, 16))                 # untrained
 ↪  message-passing weights
F_mp = np.array([np.tanh(g['Ahat'] @ g['X'] @ cg_W).mean(0) for g in
 ↪  cg_graphs])
F_bag = np.column_stack([[c[0].mean(0) for c in cg_calls], F_graph.tone])  #
 ↪  pooled baseline

print(F_graph.assign(p_ref=cg_firms.p_ref, p_new=cg_firms.p_new)
      .corr().loc[['new_rate', 'xref'], ['p_new', 'p_ref']].round(2))
```

```
          p_new  p_ref
new_rate   0.51   0.01
xref       0.08   0.34
```

The correlations show that the observed graph statistics recover the latent discourse parameters: the new-topic rate tracks p_{new} and the cross-reference rate tracks p_{ref}, even though topics are only *estimated* from noisy embeddings.

102.5.3 Does Structure Add Forecasting Power?

We mirror the paper's "beyond fundamentals" logic: fit ridge regressions of next-period log volatility on (i) fundamentals only; (ii) fundamentals plus the pooled representation (mean embedding and mean tone); (iii) fundamentals plus the graph features; (iv) fundamentals plus message-passing features and tone. All models are fit on a 70% training split and scored out of sample, overall and by transcript-length tercile.

```
fund = cg_firms[['size', 'lev', 'past_vol']].values
train = cg_rng.random(CG_N) < 0.7; test = ~train
y = cg_firms.y.values

def fit_predict(X):
    X = np.asarray(X, float)
    mu, sd = X[train].mean(0), X[train].std(0) + 1e-9
    Z = (X - mu) / sd
    return RidgeCV(alphas=np.logspace(-2, 3, 20)).fit(Z[train],
     ↪  y[train]).predict(Z)

specs = {
    'Fundamentals only': fund,
    '+ pooled embedding and tone': np.column_stack([fund, F_bag]),
    '+ graph features': np.column_stack([fund, F_graph]),
    '+ message passing': np.column_stack([fund, F_mp, F_graph.tone]),
}
tercile = pd.qcut(cg_firms.n_turns, 3, labels=['short', 'mid',
 ↪  'long']).values
rows = []
for name, X in specs.items():
    p = fit_predict(X)
    row = {'model': name, 'OOS R2': r2_score(y[test], p[test]),
           'RMSE': np.sqrt(np.mean((y[test] - p[test]) ** 2))}
    for t in ['short', 'mid', 'long']:
        m = test & (tercile == t)
        row[f'RMSE ({t})'] = np.sqrt(np.mean((y[m] - p[m]) ** 2))
    rows.append(row)
cg_results = pd.DataFrame(rows).set_index('model')
base = cg_results.loc['Fundamentals only']
gain = 1 - cg_results.filter(like='RMSE (').div(base.filter(like='RMSE ('))
print(cg_results.round(3).to_string())
print('\nRMSE reduction vs fundamentals-only, by call length:')
print(gain.round(3).to_string())
```

```
                            OOS R2   RMSE  RMSE (short)  RMSE (mid)  RMSE (long)
model
Fundamentals only            0.379  0.811         0.794       0.896        0.737
+ pooled embedding and tone  0.407  0.792         0.773       0.881        0.716
+ graph features             0.629  0.626         0.729       0.668        0.453
+ message passing            0.491  0.734         0.744       0.792        0.663

RMSE reduction vs fundamentals-only, by call length:
                           RMSE (short)  RMSE (mid)  RMSE (long)
model
Fundamentals only                 0.000       0.000        0.000
+ pooled embedding and tone       0.025       0.016        0.028
+ graph features                  0.081       0.254        0.385
+ message passing                 0.062       0.116        0.100
```

Three patterns appear, all of which are *built into* the simulation, so they validate the machinery and are not evidence about real markets. The pooled baseline adds almost nothing, because averaging embeddings cannot see turn order. The graph features add a great deal, because p_{ref} and p_{new} are structural. The RMSE reduction is larger for long calls, because a long call contains more turns from which to estimate the call's discourse rates; this is one mechanical route to the length gradient that Y. Yang et al. (2026) report, though their data may generate it through other channels, such as the sparse-signal problem for models that pool a long transcript. The untrained message-passing representation helps less than the hand-built graph features, which is a reminder that the paper's gain comes from *training* the network on the structure and not from the architecture label.

102.5.4 Production Application with DataCore.vn

The production version swaps the simulated turns for real call transcripts. The cell below is a skeleton: `get_earnings_call_transcripts` follows the chapter's DataCore conventions for text retrieval, and the helper `embed_turns` should use the PhoBERT sentence encoder of Section 101.1 on word-segmented turns (Section 96.1). It reuses `call_graph` from above unchanged, which is the point of keeping that function dependent only on turn embeddings and turn sentiment.

```
# Transcripts: one row per speaker turn (ticker, call_date, turn_idx, role,
 ↪  text)
turns = dc.get_earnings_call_transcripts(start='2018-01-01',
 ↪  end='2024-12-31')
turns = turns.sort_values(['ticker', 'call_date', 'turn_idx'])
```

```
# Turn embeddings (PhoBERT sentence encoder) and sentiment (see
 ↪  sec-textual-bert-sentiment)
turns['emb'] = list(embed_turns(turns['text_segmented']))
turns['sent'] = sentiment_score(turns['text_segmented'])

# Corpus-level topic vocabulary: fit on the training period only
train_turns = turns[turns.call_date < '2022-01-01']
topic_km = KMeans(30, n_init=10,
 ↪  random_state=0).fit(np.vstack(train_turns['emb']))

rows = []
for (tic, dt), g in turns.groupby(['ticker', 'call_date']):
    feats = call_graph(np.vstack(g['emb']), g['sent'].values, topic_km)
    rows.append({'ticker': tic, 'call_date': dt, 'n_turns': len(g),
                 **{k: feats[k] for k in GRAPH_COLS}})
graph_df = pd.DataFrame(rows)

# Target: realised volatility over the 21 trading days after the call (skip
 ↪  day 0)
vol = dc.get_realized_volatility(window=21, offset=1)
fundamentals = dc.get_fundamentals(['log_size', 'leverage', 'past_vol'])
panel = (graph_df.merge(vol, on=['ticker', 'call_date'])
                 .merge(fundamentals, on=['ticker', 'call_date']))

# Same split-by-time ridge comparison as the simulation, with time-based
 ↪  train/test
```

102.5.5 Vietnam-Specific Research Design

Applying this design in Vietnam changes the problem in four ways.

1. **Data are the binding constraint.** Quarterly earnings calls are a US institution. Vietnamese listed firms mostly communicate through filed reports, annual general meetings (AGMs, *Đại hội đồng cổ đông*), investor-relations presentations and analyst roadshows, with call transcripts available for only a fraction of larger HOSE names. The same graph construction applies to *any* multi-party dialogue: the Q&A part of an AGM, an analyst roadshow Q&A, or a broker-hosted corporate access day. Start by counting how many firm-quarters have a usable transcript before designing anything else.
2. **Replace the topic vocabulary and the sentiment measure.** Fit the topic model on Vietnamese financial text with word-segmented input (Section 96.1) and use ViFinBERT or the dictionary of Section 99.1 for tone. A topic vocabulary fit on the full sample leaks future information, so fit it on the training period only, as in the code above.

3. **Choose the target to match market structure.** The paper's target is financial risk. For Vietnam, use post-call realised volatility, but note the daily price limits of ±7% (HOSE), ±10% (HNX) and ±15% (UPCoM), which truncate return-based risk on event days; a high-low range measure or a multi-day window is less affected. Control for foreign ownership room and retail turnover share, which drive volatility independently of what is said.
4. **Interpretation beyond prediction.** The paper's finding that cross-referencing, new topics and a positive tone coincide with *lower* risk is a statement about predictive association. In a retail-heavy market, where managers know the audience is less specialised, the same patterns could reflect managerial choices (a scripted call that avoids new topics) and not underlying fundamentals. Compare calls with and without independent analyst participation, and test whether the new-topic rate reflects analyst questions (exogenous to management) or management's own agenda.

A practical research design therefore: (a) assemble all Vietnamese transcripts and estimate coverage by size tercile; (b) construct topic graphs and the discourse features above; (c) run the nested comparison of fundamentals, pooled text and graph features out of sample on a rolling time split; and (d) report the length gradient, which is the most informative diagnostic for whether structure matters in this market.

102.6 Earnings Calls in Regulatory Review

The previous section treats an earnings call as a signal for investors. Lerman, Steffen, and Zhang (2026) show that the regulator reads the same transcripts. The SEC's Division of Corporation Finance reviews every registrant's periodic filings at least once every three years and sends comment letters when it finds deficiencies in disclosure or accounting. The letters and the firms' responses become public after the review closes. Lerman, Steffen and Zhang hand-collect the comment letters that cite what management said on a *voluntary* earnings conference call. They then ask three questions: what the SEC uses that information for, when it does so, and what changes afterwards.

Their sample covers a little over 800 letters from 2005 to 2018, slightly more than 1% of all comment-letter conversations. Most come from reviews of 10-K filings (over two-thirds) and 10-Q filings (18%).[1] The main findings are:

1. **Calls are a benchmark for mandatory filings.** In about 80% of the letters the SEC cites the call to show that the 10-K or 10-Q says *too little*: management discussed a driver, metric or segment on the call that the filing omits. About 15% point to *inconsistencies* between the call and the filing. Few letters criticise the call itself, except for its use of non-GAAP measures. Revenue, segment reporting, and non-GAAP measures or KPIs

[1] Sample sizes and percentages in this section come from the authors' summary of the working-paper version (CLS Blue Sky Blog, July 2023). The qualitative findings match the published abstract.

together make up about 45% of the sample, and MD&A topics (risk factors, products, customers, markets, seasonality) another 23%.

2. **Determinants.** Call-referencing letters are more likely when the filing review is more complex (larger review teams with more accountants), when SEC staff are less resource constrained, and for firms with more institutional investors and analyst following. These are the firms whose calls are most closely followed.
3. **Remediation costs.** These letters occur during higher-quality reviews. They raise more issues per conversation, need more rounds of correspondence, and take longer to reach "no further comment" than other letters.
4. **Consequences.** Firms change their subsequent mandatory filings more after a call-referencing letter, especially when they agree with the comment. The filings get longer, more negative in tone, and use more uncertain and constraining words as measured by the Loughran and McDonald (2011) word lists. The effect on the wider information environment is mixed: bid-ask spreads and illiquidity *rise*. This fits an unintended consequence: once managers learn that the regulator reads the call, the call itself may become less informative.

For a textual-analysis chapter, the paper makes three points. First, the regulator is doing *cross-document comparison* by hand: it holds the call next to the filing and looks for content in one that is missing from, or contradicts, the other. This is a measurable construct. Second, call language responds to enforcement. Models that treat call text as unconstrained managerial communication (Section 102.5) should allow for the fact that what managers say is shaped by who is listening. Third, the outcome measures are standard dictionary counts (Section 99.1) applied to filings, so the paper also shows how tone measures serve as dependent variables, not only as return predictors.

A scalable version of the regulator's comparison is a *call-to-filing coverage gap*. This measure is our suggestion; the paper itself codes letters by hand. Let S_{it}^{call} be the sentences of firm i's call for period t and S_{it}^{file} the sentences of the subsequent periodic report, each embedded as e_s with a sentence encoder (Section 100.2). Then

$$\text{Gap}_{it} = \frac{1}{|S_{it}^{\text{call}}|} \sum_{s \in S_{it}^{\text{call}}} \mathbf{1}\left\{ \max_{f \in S_{it}^{\text{file}}} \cos(e_s, e_f) < \tau \right\}, \tag{102.2}$$

the share of call sentences with no close counterpart in the filing. Restrict S_{it}^{call} to sentences that carry hard information (numbers, segment names, KPI terms), because the SEC's "insufficiency" comments target exactly these. Choose τ by hand-labelling a validation sample of matched and unmatched pairs. Inconsistencies are harder to detect: within matched sentence pairs, extract the numbers with their units and flag pairs whose values disagree.

102.6.1 Vietnam-Specific Research Design

The US design does not transplant directly, because Vietnam has no public comment-letter process. The State Securities Commission (SSC, *Ủy ban Chứng khoán Nhà nước*) does not publish its review correspondence. The voluntary channel also differs: quarterly earnings calls are held by only a minority of large caps, while AGMs, investor-relations presentations and analyst meetings carry most voluntary disclosure (Section 102.5.5). What does exist publicly are several "regulator meets disclosure" records:

1. **Mandated profit explanations.** Under Circular 96/2020/TT-BTC, listed firms must publish an explanation (*giải trình*) when after-tax profit changes by 10% or more year on year or swings from profit to loss, and when audited profit differs by 5% or more from the pre-audit figure. The pre-audit versus audited gap is the closest local analogue to the SEC's "inconsistency" comments, and it is filed as text on the HOSE and HNX disclosure portals.
2. **Enforcement outcomes.** SSC administrative sanction decisions for disclosure violations are published, and the exchanges place stocks on warning or control lists for late filings or modified audit opinions. These are noisier than comment letters but play the same role as the outcome variable.
3. **Auditor modifications.** Qualified opinions and emphasis-of-matter paragraphs are a third party's "comment" on the filing and can be dated precisely.

Research designs that follow from this:

- **Voluntary-to-mandatory gap as a predictor.** Compute Equation 102.2 with AGM minutes or IR presentations in place of the call, and the annual report or MD&A in place of the 10-K. Test whether a larger gap predicts a later profit explanation, a pre-audit to audited revision of 5% or more, a modified audit opinion, or an SSC sanction.
- **Who gets scrutinised.** In the US, call-referencing letters concentrate on firms with institutional and analyst following. In Vietnam, foreign institutional ownership is concentrated in large caps, while enforcement has targeted manipulation in retail-heavy small caps, as in the 2022 FLC and Tân Hoàng Minh cases. Expect a different determinant structure, and test it before assuming the US pattern.
- **Chilling effect on voluntary disclosure.** The paper's mixed information-environment result suggests testing whether firms (or industry peers) say less in AGM Q&A or IR materials after a sanction or a visible enforcement wave. Measure this with transcript length, the share of sentences with numbers, and the forward-looking word share. Compare with firms that were not exposed.
- **Bilingual consistency.** Larger listed firms publish both Vietnamese and English versions of their reports. Apply the same gap measure *across languages* with a multilingual encoder. Divergence between the two versions is a Vietnam-specific inconsistency that foreign investors may price.

103 Method Comparison and Best Practices

Table 103.1: Comparison of Textual Analysis Methods for Vietnamese Financial Text

Method	Interpretability	Semantic	Speed	VN Support	Data Req.	Best Use Case
BoW/TF-IDF	High	Low	Fast	Good*	None	Peer groups, lexical similarity
LDA	Medium	Low	Medium	Good*	None	Topic discovery
Doc2Vec	Low	Medium	Medium	Good*	Corpus	Document similarity
BERTopic	High	High	Slow	Excellent	None	Coherent topics
PhoBERT	Low	High	Slow	Excellent	Fine-tune	Sentiment, NER, classification
Sentence-BERT	Low	High	Medium	Good	None	Semantic similarity
LLM (zero-shot)	High	High	Slow	Good	None	Extraction, classification

Note

*Requires Vietnamese word segmentation as a preprocessing step. VN Support rates how well the method handles Vietnamese text natively.

For researchers beginning textual analysis of Vietnamese firms, we recommend the following workflow:

1. **Start with TF-IDF cosine similarity** for peer identification because it is fast, interpretable, and provides a strong baseline.

2. **Use BERTopic with PhoBERT embeddings** for topic discovery because it produces more coherent topics than LDA for Vietnamese text.
3. **For sentiment analysis**, use ViFinBERT if fine-tuning data is available; otherwise, LLM zero-shot classification provides competitive results.
4. **For production systems** requiring real-time analysis, sentence-BERT embeddings offer the best speed-accuracy tradeoff.

```
# Evaluate: what fraction of top-5 peers share ICB sector?
methods = {
    'TF-IDF': sim_df,
    'Sentence-BERT': embed_sim_df,
    'Doc2Vec': pd.DataFrame(
        cosine_similarity(d2v_vectors),
        index=tickers, columns=tickers
    ),
}

accuracy_results = {}
sector_map = corpus_df.set_index('ticker')['icb_sector'].to_dict()

for method_name, sim_matrix_df in methods.items():
    matches = 0
    total = 0
    for ticker in tickers:
        true_sector = sector_map.get(ticker)
        peers = sim_matrix_df[ticker].drop(ticker).nlargest(5)
        for peer in peers.index:
            total += 1
            if sector_map.get(peer) == true_sector:
                matches += 1
    accuracy_results[method_name] = matches / total

fig, ax = plt.subplots(figsize=(8, 5))
methods_list = list(accuracy_results.keys())
accs = list(accuracy_results.values())
bars = ax.bar(methods_list, accs, color=['#2C5282', '#38A169', '#D69E2E'])
ax.set_ylabel('ICB Sector Match Rate')
ax.set_title('Peer Identification Accuracy by Method')
ax.set_ylim(0, 1)
for bar, acc in zip(bars, accs):
    ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.02,
            f'{acc:.1%}', ha='center', fontweight='bold')
plt.tight_layout()
plt.show()
```

Figure 103.1

104 Conclusion

This chapter worked through the full pipeline of textual analysis methods applied to Vietnamese listed firms, from classical bag-of-words approaches to transformer-based language models. The key takeaways for practitioners and researchers are:

First, Vietnamese text preprocessing requires a word segmentation step that has no parallel in English-language NLP. Using tools like VnCoreNLP or underthesea for this step is essential and significantly affects downstream analysis quality.

Second, domain-specific sentiment lexicons substantially outperform general-purpose dictionaries for Vietnamese financial text, consistent with Loughran and McDonald (2011) findings for English.

Third, PhoBERT-based embeddings capture semantic similarity that TF-IDF misses, identifying industry peers that share business models even when they use different vocabulary.

Fourth, LLMs enable new applications, including structured information extraction from Vietnamese annual reports that would be prohibitively expensive with manual coding.

The empirical applications demonstrate that textual measures contain economically meaningful information for the Vietnamese market. Net sentiment from annual reports predicts subsequent stock returns even after controlling for standard risk factors, and BERT-based sentiment measures have incremental predictive power beyond dictionary-based measures. Text-based industry classifications capture firm relationships that static ICB codes miss, and textual similarity changes around corporate events reflect real business transformations.

105 Image and Visual Data in Finance

The previous chapter demonstrated how unstructured text (e.g., earnings reports, news articles, business descriptions) can be transformed into structured signals for financial analysis. This chapter extends the alternative data toolkit to a second modality: images. Visual data is abundant in financial contexts yet systematically underexploited. Satellite photographs reveal real economic activity (e.g., parking lot occupancy at retail locations, construction progress at industrial sites, nighttime luminosity as a proxy for regional GDP, ship traffic at port terminals, crop health across agricultural zones). Corporate documents arrive as scanned PDFs whose tables and figures resist standard text extraction. Financial charts encode information that analysts interpret visually but that systematic strategies cannot consume without digitization. And the visual content of social media, advertising, and product imagery carries sentiment and brand signals that complement textual analysis.

The core challenge is representational: an image is a three-dimensional tensor of pixel intensities with no inherent semantic structure. Converting this raw array into a financial signal (i.e., a number that predicts returns, measures risk, or proxies for economic activity) requires either hand-crafted feature engineering or learned representations via deep convolutional neural networks (CNNs) and vision transformers (ViTs). This chapter covers both approaches.

We organize the material around five application domains, each with distinct data sources, modeling requirements, and economic motivations. First, satellite and geospatial imagery for nowcasting economic activity. Second, document image analysis for extracting structured data from Vietnamese financial filings. Third, chart and figure digitization for systematic backtesting. Fourth, visual sentiment analysis from social and news media. Fifth, multimodal fusion, combining image and text signals into joint predictive models.

Vietnamese markets present particular opportunities in this space. Satellite imagery is especially informative in an economy with large agricultural and manufacturing sectors where ground-truth data arrives with significant lags. Vietnamese financial filings are often distributed as scanned images rather than machine-readable formats, making document AI essential rather than optional. And the rapid urbanization visible in construction and infrastructure imagery provides high-frequency proxies for macroeconomic momentum that official statistics cannot match.

```
import pandas as pd
import numpy as np
from pathlib import Path
```

```
import warnings
warnings.filterwarnings("ignore")

# Core image processing
from PIL import Image
import io

# Deep learning
import torch
import torch.nn as nn
import torchvision.transforms as transforms
import torchvision.models as models

# Visualization
import plotnine as p9
from mizani.formatters import percent_format, comma_format
import matplotlib.pyplot as plt

# Statistical analysis
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
from linearmodels.panel import PanelOLS
```

105.1 Foundations: From Pixels to Financial Signals

105.1.1 Image Representation

A digital image is a function $I : \{1, \ldots, H\} \times \{1, \ldots, W\} \times \{1, \ldots, C\} \to [0, 255]$ mapping spatial coordinates and color channels to intensity values. For an RGB image of height H and width W, the representation is a tensor $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$. A single 224×224 RGB image (i.e., the standard input for modern CNNs) contains $224 \times 224 \times 3 = 150{,}528$ dimensions. This extreme dimensionality, combined with spatial structure (nearby pixels are correlated), makes images fundamentally different from tabular financial data and demands specialized architectures.

The key insight of convolutional neural networks is parameter sharing via local filters. A convolutional layer applies a kernel $\mathbf{K} \in \mathbb{R}^{k \times k \times C_{\text{in}}}$ to produce a feature map:

$$(\mathbf{I} * \mathbf{K})(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \sum_{c=1}^{C_{\text{in}}} I(i + m, j + n, c) \cdot K(m, n, c) \tag{105.1}$$

By stacking convolutional layers with nonlinearities and pooling operations, the network builds a hierarchy of representations: early layers detect edges and textures; middle layers detect parts and patterns; deep layers detect objects and scenes. The final layer output $\mathbf{z} \in \mathbb{R}^d$ (with d typically 512-2048) is a compact representation of the image's semantic content, which can be used directly as a feature vector for financial prediction.

Financial images span a wide range of resolutions and modalities:

Table 105.1: Image Data Sources for Vietnamese Financial Markets

Source	Resolution	Channels	Typical Size	Update Frequency
Sentinel-2 satellite	10m/pixel	13 bands	10,980 × 10,980	5 days
Planet Labs	3m/pixel	4 bands	4,000 × 4,000	Daily
VIIRS nightlights	500m/pixel	1 (DNB)	3,000 × 1,800	Monthly composite
Annual report scan	300 DPI	3 (RGB)	2,480 × 3,508	Annual
CEO photograph	Varies	3 (RGB)	500 × 500	Annual
News photograph	Varies	3 (RGB)	800 × 600	Real-time
Financial chart	Varies	3 (RGB)	1,000 × 600	Real-time

105.1.2 Transfer Learning for Finance

Training a CNN from scratch requires millions of labeled images, which is far more than any financial application can provide. Transfer learning solves this by using networks pre-trained on ImageNet (1.2 million images, 1,000 classes) as feature extractors. The pre-trained network has already learned generic visual representations (edges, textures, shapes, objects); we simply replace the final classification layer with a task-specific head.

Formally, let $f_\theta(\mathbf{I})$ denote a pre-trained network with parameters θ partitioned into feature extractor θ_{feat} and classifier θ_{cls}. For financial applications, we:

1. **Feature extraction**: Freeze θ_{feat}, extract $\mathbf{z} = f_{\theta_{\text{feat}}}(\mathbf{I})$, and train a simple model (linear regression, gradient boosting) on $\mathbf{z}$.
2. **Fine-tuning**: Initialize from θ and train all parameters on the financial task with a small learning rate to avoid catastrophic forgetting.

The key architectural families are:

ResNet [He et al. (2016)]. Residual connections ($y = F(x) + x$) enable training of very deep networks (50-152 layers). The skip connection solves the vanishing gradient problem. ResNet-50 produces a 2,048-dimensional feature vector from the penultimate layer.

EfficientNet [Tan and Le (2019)]. Compound scaling of depth, width, and resolution simultaneously. EfficientNet-B0 achieves ResNet-50 accuracy with 5.3M parameters (vs. 25.6M), making it practical for processing thousands of satellite tiles.

Vision Transformer (ViT) [Dosovitskiy et al. (2020)]. Treats an image as a sequence of 16$\times$16 patches, processes them through a standard Transformer encoder. ViT-B/16 produces a 768-dimensional embedding. Particularly effective for document images where spatial relationships between elements (tables, headers, text blocks) matter.

```
# DataCore.vn API
from datacore import DataCore
dc = DataCore()

def build_feature_extractor(model_name="resnet50", device="cpu"):
    """
    Build a pre-trained CNN feature extractor.

    Parameters
    ----------
    model_name : str
        One of 'resnet50', 'efficientnet_b0', 'vit_b_16'.
    device : str
        'cpu' or 'cuda'.

    Returns
    -------
    model : nn.Module
        Feature extraction model.
    transform : transforms.Compose
        Image preprocessing pipeline.
    """
    if model_name == "resnet50":
        weights = models.ResNet50_Weights.IMAGENET1K_V2
        model = models.resnet50(weights=weights)
        model.fc = nn.Identity()  # Remove classification head
        dim = 2048
    elif model_name == "efficientnet_b0":
        weights = models.EfficientNet_B0_Weights.IMAGENET1K_V1
        model = models.efficientnet_b0(weights=weights)
        model.classifier = nn.Identity()
```

```
        dim = 1280
    elif model_name == "vit_b_16":
        weights = models.ViT_B_16_Weights.IMAGENET1K_V1
        model = models.vit_b_16(weights=weights)
        model.heads = nn.Identity()
        dim = 768

    model = model.to(device).eval()

    transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ])

    return model, transform, dim


def extract_features(image_paths, model, transform, device="cpu",
                     batch_size=32):
    """
    Extract deep features from a list of images.

    Parameters
    ----------
    image_paths : list
        Paths to image files.
    model : nn.Module
        Feature extraction model.
    transform : transforms.Compose
        Preprocessing pipeline.

    Returns
    -------
    np.ndarray : Feature matrix (n_images x feature_dim).
    """
    features = []

    for i in range(0, len(image_paths), batch_size):
```

```
        batch_paths = image_paths[i:i + batch_size]
        batch_tensors = []

        for path in batch_paths:
            try:
                img = Image.open(path).convert("RGB")
                tensor = transform(img)
                batch_tensors.append(tensor)
            except Exception:
                batch_tensors.append(torch.zeros(3, 224, 224))

        batch = torch.stack(batch_tensors).to(device)

        with torch.no_grad():
            batch_features = model(batch).cpu().numpy()

        features.append(batch_features)

    return np.vstack(features)
```

105.2 Satellite and Geospatial Imagery

105.2.1 Economic Activity from Space

Satellite imagery provides high-frequency, spatially granular measurements of economic activity that are independent of and often lead official statistics. The foundational work of Henderson, Storeygard, and Weil (2012) demonstrated that nighttime luminosity, measured by the Defense Meteorological Satellite Program (DMSP), is a reliable proxy for GDP, particularly in countries where official statistics are noisy or delayed. Donaldson and Storeygard (2016) survey the broad applications of satellite data in economics, from agricultural productivity to urban growth to conflict measurement. Donaldson (2018) use satellite-derived agricultural output measures to study the welfare gains from railroads in colonial India. Jean et al. (2016) combine daytime satellite imagery with CNNs to predict poverty from space with $r^2 > 0.7$.

The political economy literature exploits satellite data to answer questions impossible with official statistics. Hodler and Raschky (2014) use nighttime lights to show that subnational regions politically aligned with the national leader receive differentially more economic activity (e.g., a finding on regional favoritism that has direct relevance for Vietnam's province-level investment allocation). Henderson et al. (2018) extend the lights framework to study the global distribution of economic activity, showing how geography, history, and trade jointly determine the spatial pattern of prosperity.

For financial applications, the key insight is that satellite data arrives faster than corporate earnings or government statistics. A retailer's quarterly revenue is reported 4-8 weeks after the quarter ends; satellite imagery of its parking lots is available within days. This temporal advantage creates a natural use case for nowcasting (i.e., estimating current economic conditions before official data arrives) and for constructing trading signals based on information that is public but costly to process.

105.2.2 Application 1: Nighttime Luminosity and Provincial GDP

Vietnam's General Statistics Office (GSO) publishes provincial GDP with a lag of several months. Nighttime luminosity from the VIIRS (Visible Infrared Imaging Radiometer Suite) sensor provides a near-real-time alternative. We construct a firm-level exposure measure by linking each listed firm's registered location to the luminosity of its province.

```
# Load nighttime luminosity data (VIIRS monthly composites)
# Source: Earth Observation Group (EOG) / NOAA
nightlights = dc.get_nightlight_data(
    start_date="2014-01-01",
    end_date="2024-12-31",
    resolution="province"
)

# Load firm location data
firm_locations = dc.get_firm_locations()

# Provincial GDP from GSO
provincial_gdp = dc.get_provincial_gdp(
    start_date="2014-01-01",
    end_date="2024-12-31"
)

print(f"Nightlight observations: {len(nightlights)}")
print(f"Provinces covered: {nightlights['province'].nunique()}")
print(f"Firms with location: {firm_locations['ticker'].nunique()}")
```

```
# Validate: does nightlight predict provincial GDP?
nl_gdp = nightlights.merge(
    provincial_gdp,
    on=["province", "year", "quarter"],
    how="inner"
)
```

```
# Log-log specification (standard in the literature)
nl_gdp["ln_luminosity"] = np.log(nl_gdp["mean_radiance"].clip(lower=0.01))
nl_gdp["ln_gdp"] = np.log(nl_gdp["provincial_gdp"].clip(lower=1))

# Cross-sectional regression by year
validation_results = []
for year in nl_gdp["year"].unique():
    subset = nl_gdp[nl_gdp["year"] == year]
    if len(subset) < 20:
        continue

    model = sm.OLS(
        subset["ln_gdp"],
        sm.add_constant(subset["ln_luminosity"])
    ).fit()

    validation_results.append({
        "year": year,
        "beta": model.params.iloc[1],
        "r_squared": model.rsquared,
        "n_provinces": int(model.nobs)
    })

validation_df = pd.DataFrame(validation_results)
print(f"Avg R² (ln GDP ~ ln Luminosity):
 ↪  {validation_df['r_squared'].mean():.3f}")
```

```
# Construct firm-level nightlight signal
# Logic: firms in provinces with accelerating luminosity
# are experiencing positive local economic conditions

nl_growth = nightlights.copy().sort_values(["province", "year", "quarter"])

# Year-over-year luminosity growth by province
nl_growth["ln_radiance"] =
 ↪  np.log(nl_growth["mean_radiance"].clip(lower=0.01))
nl_growth["ln_radiance_lag4"] = nl_growth.groupby(
    "province"
)["ln_radiance"].shift(4)

nl_growth["nl_growth"] = nl_growth["ln_radiance"] -
 ↪  nl_growth["ln_radiance_lag4"]
```

```
(
    p9.ggplot(nl_gdp[nl_gdp["year"] == 2023],
              p9.aes(x="ln_luminosity", y="ln_gdp"))
    + p9.geom_point(color="#2E5090", alpha=0.6, size=2)
    + p9.geom_smooth(method="lm", color="#C0392B", se=True, size=0.8)
    + p9.labs(
        x="ln(Mean Nighttime Radiance)",
        y="ln(Provincial GDP)",
        title="Nighttime Luminosity vs. Provincial GDP (2023)"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 6))
)
```

Figure 105.1

```
# Merge with firms via province
firm_nl = firm_locations[["ticker", "province"]].merge(
    nl_growth[["province", "year", "quarter", "nl_growth"]],
    on="province",
    how="inner"
)

# Merge with stock returns
monthly_returns = dc.get_monthly_returns(
    start_date="2014-01-01",
    end_date="2024-12-31"
)
monthly_returns["year"] = monthly_returns["date"].dt.year
monthly_returns["quarter"] = monthly_returns["date"].dt.quarter

returns_with_nl = monthly_returns.merge(
    firm_nl,
    on=["ticker", "year", "quarter"],
    how="inner"
)
```

```
# Portfolio sort: quintiles on provincial nightlight growth
returns_with_nl["nl_quintile"] = returns_with_nl.groupby("date")[
```

Table 105.2: Nighttime Luminosity Growth Quintile Portfolio Returns

```
nl_summary = nl_wide.describe().T[["mean", "std"]].copy()
nl_summary["mean_ann"] = nl_summary["mean"] * 12
nl_summary["sharpe"] = (
    nl_summary["mean_ann"] / (nl_summary["std"] * np.sqrt(12))
)
for col in nl_wide.columns:
    t_stat = nl_wide[col].mean() / (
        nl_wide[col].std() / np.sqrt(len(nl_wide.dropna()))
    )
    nl_summary.loc[col, "t_stat"] = t_stat

nl_summary = nl_summary[["mean_ann", "sharpe", "t_stat"]].round(4)
nl_summary.columns = ["Ann. Return", "Sharpe", "t-stat"]
nl_summary
```

```
    "nl_growth"
].transform(lambda x: pd.qcut(x, 5, labels=[1, 2, 3, 4, 5],
                              duplicates="drop"))

nl_port_returns = (
    returns_with_nl.groupby(["date", "nl_quintile"])
    .agg(port_ret=("ret", "mean"))
    .reset_index()
)

nl_wide = nl_port_returns.pivot(
    index="date", columns="nl_quintile", values="port_ret"
)
nl_wide["L-S"] = nl_wide[5] - nl_wide[1]  # High NL growth - Low
```

105.2.3 Application 2: Satellite Imagery for Sector Nowcasting

Beyond luminosity, daytime satellite imagery provides sector-specific signals. We implement three channels relevant to the Vietnamese economy.

Port activity. Vietnam is a major export-oriented economy. Satellite imagery of container ports (Cát Lái, Hải Phòng) captures trade throughput before customs statistics are released.

Ship detection algorithms applied to synthetic aperture radar (SAR) imagery count vessels and estimate cargo volumes.

Construction progress. Real estate and construction constitute a significant fraction of Vietnamese GDP and market capitalization. Change detection algorithms applied to high-resolution optical imagery identify construction starts, completion rates, and land-use conversion.

Agricultural monitoring. Vietnam is a leading exporter of rice, coffee, rubber, and seafood. The Normalized Difference Vegetation Index (NDVI), computed from multispectral satellite data, provides crop health assessments:

$$\text{NDVI} = \frac{\rho_{\text{NIR}} - \rho_{\text{Red}}}{\rho_{\text{NIR}} + \rho_{\text{Red}}} \tag{105.2}$$

where ρ_{NIR} and ρ_{Red} are reflectance in the near-infrared and red bands. NDVI ranges from -1 to $+1$, with values above 0.3 indicating healthy vegetation. Deviations from seasonal norms proxy for crop yield surprises.

```
# Load NDVI data for Vietnamese agricultural regions
# Source: MODIS/Terra (MOD13Q1, 250m resolution, 16-day composites)
ndvi_data = dc.get_ndvi_data(
    start_date="2014-01-01",
    end_date="2024-12-31",
    regions=["mekong_delta", "central_highlands",
             "red_river_delta", "southeast"]
)

# Compute NDVI anomaly: deviation from 5-year seasonal average
ndvi_data["month"] = ndvi_data["date"].dt.month

seasonal_mean = (
    ndvi_data.groupby(["region", "month"])
    ["mean_ndvi"].transform(
        lambda x: x.rolling(5 * 12, min_periods=12).mean()
    )
)
ndvi_data["ndvi_anomaly"] = ndvi_data["mean_ndvi"] - seasonal_mean

# Agricultural sector firms
agri_firms = dc.get_firms_by_sector(sector="agriculture")

# Link NDVI anomaly to agricultural firm returns
agri_returns = monthly_returns[
```

```
    monthly_returns["ticker"].isin(agri_firms["ticker"])
].copy()

agri_returns["month"] = agri_returns["date"].dt.month
agri_returns["year"] = agri_returns["date"].dt.year

# Regional NDVI aggregation (Mekong Delta for rice firms, etc.)
mekong_ndvi = ndvi_data[ndvi_data["region"] == "mekong_delta"].copy()
mekong_ndvi["year"] = mekong_ndvi["date"].dt.year
mekong_ndvi["month"] = mekong_ndvi["date"].dt.month

mekong_monthly = (
    mekong_ndvi.groupby(["year", "month"])
    .agg(ndvi_anomaly=("ndvi_anomaly", "mean"))
    .reset_index()
)
```

```
ndvi_plot = ndvi_data[ndvi_data["region"] == "mekong_delta"].copy()

(
    p9.ggplot(ndvi_plot, p9.aes(x="date", y="ndvi_anomaly"))
    + p9.geom_line(color="#27AE60", alpha=0.5, size=0.4)
    + p9.geom_smooth(method="lowess", color="#2E5090", size=1, se=False)
    + p9.geom_hline(yintercept=0, linetype="dashed", color="gray")
    + p9.labs(
        x="",
        y="NDVI Anomaly",
        title="Mekong Delta Vegetation Health: Deviation from Seasonal Norm"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(12, 5))
)
```

Figure 105.2

```
# Panel regression: agricultural firm returns on NDVI anomaly
agri_panel = agri_returns.merge(
    mekong_monthly,
    on=["year", "month"],
    how="inner"
)
```

```
# Lagged NDVI anomaly (one month)
agri_panel = agri_panel.sort_values(["ticker", "date"])
agri_panel["ndvi_lag1"] = agri_panel.groupby(
    "ticker"
)["ndvi_anomaly"].shift(1)

agri_clean = agri_panel.dropna(
    subset=["ret", "ndvi_lag1"]
).set_index(["ticker", "date"])

model_ndvi = PanelOLS(
    agri_clean["ret"],
    agri_clean[["ndvi_lag1"]],
    entity_effects=True,
    time_effects=True,
    check_rank=False
).fit(cov_type="clustered", cluster_entity=True)

agri_clean = agri_clean.reset_index()

print(f"NDVI → Agricultural Returns:")
print(f"   (NDVI_lag): {model_ndvi.params['ndvi_lag1']:.4f}")
print(f"  t-stat: {model_ndvi.tstats['ndvi_lag1']:.3f}")
print(f"  R² (within): {model_ndvi.rsquared_within:.4f}")
```

105.2.4 Satellite Feature Extraction with CNNs

For raw satellite imagery (rather than pre-computed indices like NDVI), we use transfer learning from CNNs to extract spatial features. The approach follows Jean et al. (2016): use a CNN pre-trained on ImageNet to extract feature vectors from satellite tiles, then regress economic outcomes on these features.

```
def satellite_feature_pipeline(image_dir, model_name="resnet50"):
    """
    Extract CNN features from satellite image tiles.

    Parameters
    ----------
    image_dir : str or Path
        Directory containing satellite tiles (PNG/TIFF).
```

```
    model_name : str
        Pre-trained model to use.

    Returns
    -------
    DataFrame : image_id, feature vector columns.
    """
    image_dir = Path(image_dir)
    image_paths = sorted(image_dir.glob("*.png")) + sorted(
        image_dir.glob("*.tif")
    )

    if not image_paths:
        print("No images found.")
        return pd.DataFrame()

    device = "cuda" if torch.cuda.is_available() else "cpu"
    model, transform, dim = build_feature_extractor(model_name, device)

    features = extract_features(image_paths, model, transform, device)

    # Create DataFrame
    feature_cols = [f"feat_{i}" for i in range(dim)]
    df = pd.DataFrame(features, columns=feature_cols)
    df["image_id"] = [p.stem for p in image_paths]

    return df


def predict_economic_activity(features_df, labels_df, label_col,
                              n_components=50):
    """
    Predict economic activity from satellite image features.

    Uses PCA for dimensionality reduction, then ridge regression.

    Parameters
    ----------
    features_df : DataFrame
        CNN features with image_id.
    labels_df : DataFrame
        Economic outcomes with image_id.
    label_col : str
```

```
    Target variable column name.
n_components : int
    PCA components to retain.

Returns
-------
dict : R², coefficients, cross-validated performance.
"""
from sklearn.decomposition import PCA
from sklearn.linear_model import RidgeCV
from sklearn.model_selection import cross_val_score

merged = features_df.merge(labels_df, on="image_id")
feature_cols = [c for c in features_df.columns if c.startswith("feat_")]

X = merged[feature_cols].values
y = merged[label_col].values

# PCA
pca = PCA(n_components=n_components)
X_pca = pca.fit_transform(X)
var_explained = pca.explained_variance_ratio_.sum()

# Ridge regression with cross-validation
ridge = RidgeCV(alphas=np.logspace(-3, 3, 20), cv=5)
cv_scores = cross_val_score(ridge, X_pca, y, cv=5, scoring="r2")

ridge.fit(X_pca, y)

return {
    "r2_cv_mean": cv_scores.mean(),
    "r2_cv_std": cv_scores.std(),
    "r2_train": ridge.score(X_pca, y),
    "pca_var_explained": var_explained,
    "optimal_alpha": ridge.alpha_,
    "n_images": len(merged)
}
```

105.3 Document Image Analysis

105.3.1 The Vietnamese Filing Problem

A substantial fraction of Vietnamese corporate disclosures (e.g., annual reports, financial statements, board resolutions, shareholder meeting minutes) are distributed as scanned PDF images rather than machine-readable text. This creates a data extraction bottleneck: the information exists but is trapped in pixel format. Unlike filings in more developed markets (where XBRL mandates ensure machine readability), Vietnamese filings require Optical Character Recognition (OCR) and layout analysis before any quantitative analysis can begin.

The document AI pipeline for Vietnamese financial filings involves four stages:

1. **Page classification**: Identify which pages contain financial statements, management discussion, audit opinions, etc.
2. **Layout analysis**: Detect the spatial structure such as headers, paragraphs, tables, figures, captions.
3. **OCR**: Convert image regions to text, using Vietnamese-optimized models.
4. **Structured extraction**: Parse the recognized text into structured data (e.g., revenue figures, balance sheet items).

105.3.2 OCR for Vietnamese Financial Documents

Standard OCR engines (Tesseract, Google Cloud Vision) struggle with Vietnamese financial documents due to the combination of Vietnamese diacritics (ă, ơ, ư, ê, etc.), mixed Vietnamese-English content, and complex table layouts. We implement a pipeline using PaddleOCR (which has strong CJK and Southeast Asian language support) and VietOCR (a Vietnamese-specific model based on the transformer architecture of Baek et al. (2019)).

```
def ocr_financial_document(pdf_path, language="vi",
                           engine="paddleocr"):
    """
    OCR a Vietnamese financial document (scanned PDF).

    Parameters
    ----------
    pdf_path : str
        Path to PDF file.
    language : str
        Language code.
    engine : str
        'paddleocr' or 'vietocr'.
```

```
    Returns
    -------
    list[dict] : Per-page OCR results with bounding boxes.
    """
    from pdf2image import convert_from_path

    # Convert PDF pages to images
    pages = convert_from_path(pdf_path, dpi=300)

    results = []

    if engine == "paddleocr":
        from paddleocr import PaddleOCR
        ocr = PaddleOCR(use_angle_cls=True, lang="vi", use_gpu=False)

        for page_num, page_img in enumerate(pages):
            # Convert PIL to numpy
            img_array = np.array(page_img)
            ocr_result = ocr.ocr(img_array, cls=True)

            page_texts = []
            for line in ocr_result[0]:
                bbox, (text, confidence) = line
                page_texts.append({
                    "text": text,
                    "confidence": confidence,
                    "bbox": bbox,
                    "page": page_num + 1
                })

            results.extend(page_texts)

    return results


def classify_page_type(ocr_results, page_num):
    """
    Classify a document page by content type using keyword matching.

    Returns one of: 'balance_sheet', 'income_statement',
    'cash_flow', 'notes', 'audit', 'management', 'other'.
    """
```

```
    page_text = " ".join(
        [r["text"] for r in ocr_results if r["page"] == page_num]
    ).lower()

    # Vietnamese financial statement keywords
    keyword_map = {
        "balance_sheet": [
            "bảng cân đối kế toán", "tài sản", "nguồn vốn",
            "nợ phải trả", "vốn chủ sở hữu"
        ],
        "income_statement": [
            "kết quả hoạt động kinh doanh", "doanh thu",
            "lợi nhuận", "chi phí", "thu nhập"
        ],
        "cash_flow": [
            "lưu chuyển tiền tệ", "dòng tiền",
            "hoạt động kinh doanh", "hoạt động đầu tư"
        ],
        "audit": [
            "báo cáo kiểm toán", "kiểm toán viên",
            "ý kiến kiểm toán", "trung thực và hợp lý"
        ],
        "management": [
            "ban giám đốc", "hội đồng quản trị",
            "báo cáo thường niên", "tình hình hoạt động"
        ]
    }

    scores = {}
    for page_type, keywords in keyword_map.items():
        scores[page_type] = sum(
            1 for kw in keywords if kw in page_text
        )

    if max(scores.values()) == 0:
        return "other"
    return max(scores, key=scores.get)
```

105.3.3 Table Extraction from Financial Statements

The highest-value extraction task is recovering structured tables from financial statements. We implement a two-stage approach: first detect table regions using a layout analysis model,

then parse the detected regions into row-column structure.

```
def extract_tables_from_page(page_image, ocr_results, page_num):
    """
    Extract structured tables from a document page.

    Uses spatial clustering of OCR bounding boxes to identify
    table regions, then aligns text into rows and columns.

    Parameters
    ----------
    page_image : PIL.Image
        Page image.
    ocr_results : list[dict]
        OCR results for this page.
    page_num : int
        Page number.

    Returns
    -------
    list[pd.DataFrame] : Extracted tables as DataFrames.
    """
    page_texts = [r for r in ocr_results if r["page"] == page_num]

    if not page_texts:
        return []

    # Extract bounding box centers
    centers = []
    for item in page_texts:
        bbox = item["bbox"]
        # bbox is [[x1,y1],[x2,y2],[x3,y3],[x4,y4]]
        cx = np.mean([p[0] for p in bbox])
        cy = np.mean([p[1] for p in bbox])
        centers.append((cx, cy, item["text"]))

    if not centers:
        return []

    centers_df = pd.DataFrame(centers, columns=["x", "y", "text"])

    # Cluster into rows by y-coordinate proximity
    centers_df = centers_df.sort_values("y")
```

```
    row_threshold = 15  # pixels
    centers_df["row_id"] = (
        centers_df["y"].diff().abs() > row_threshold
    ).cumsum()

    # Within each row, sort by x-coordinate
    tables = []
    rows = []
    for row_id, row_group in centers_df.groupby("row_id"):
        row_sorted = row_group.sort_values("x")
        rows.append(row_sorted["text"].tolist())

    if len(rows) > 2:
        # Attempt to construct DataFrame
        max_cols = max(len(r) for r in rows)
        # Pad shorter rows
        padded = [r + [""] * (max_cols - len(r)) for r in rows]

        try:
            df = pd.DataFrame(padded[1:], columns=padded[0])
            tables.append(df)
        except Exception:
            tables.append(pd.DataFrame(padded))

    return tables


def parse_financial_numbers(text):
    """
    Parse Vietnamese financial number formats.
    Vietnamese uses dots as thousands separators and commas as decimals.
    E.g., '1.234.567' = 1234567, '1.234,56' = 1234.56
    """
    import re
    text = text.strip().replace(" ", "")

    # Remove parentheses (negative indicator)
    negative = text.startswith("(") and text.endswith(")")
    text = text.strip("()")

    # Handle Vietnamese number format
    # If comma is present, it's a decimal separator
    if "," in text:
```

```
        text = text.replace(".", "").replace(",", ".")
    else:
        text = text.replace(".", "")

    try:
        value = float(text)
        return -value if negative else value
    except ValueError:
        return np.nan
```

105.3.4 Layout-Aware Document Understanding

Modern document AI goes beyond OCR by jointly modeling text content and spatial layout. LayoutLM (Yupan Huang et al. 2022) and its successors treat each token as having both a text embedding and a positional embedding derived from its bounding box coordinates. This allows the model to understand that a number positioned below a "Revenue" header and to the right of "2023" is the 2023 revenue figure, even without explicit table detection.

```
def layoutlm_extract(document_pages, model_name="layoutlmv3"):
    """
    Extract structured financial data using LayoutLM.

    This function uses the pre-trained LayoutLMv3 model for
    document understanding with Vietnamese financial statements.

    Parameters
    ----------
    document_pages : list
        List of (page_image, ocr_results) tuples.
    model_name : str
        Model variant.

    Returns
    -------
    dict : Extracted financial fields.
    """
    from transformers import (
        LayoutLMv3ForTokenClassification,
        LayoutLMv3Processor
    )
```

```
    processor = LayoutLMv3Processor.from_pretrained(
        "microsoft/layoutlmv3-base",
        apply_ocr=False  # We provide our own OCR
    )

    model = LayoutLMv3ForTokenClassification.from_pretrained(
        "microsoft/layoutlmv3-base",
        num_labels=13  # Financial statement field types
    )

    # Define target fields for extraction
    field_labels = [
        "O",  # Other
        "B-REVENUE", "I-REVENUE",
        "B-COGS", "I-COGS",
        "B-NET_INCOME", "I-NET_INCOME",
        "B-TOTAL_ASSETS", "I-TOTAL_ASSETS",
        "B-TOTAL_EQUITY", "I-TOTAL_EQUITY",
        "B-TOTAL_DEBT", "I-TOTAL_DEBT"
    ]

    extracted = {}

    for page_img, ocr_results in document_pages:
        words = [r["text"] for r in ocr_results]
        boxes = []
        for r in ocr_results:
            bbox = r["bbox"]
            # Normalize to 0-1000 range
            x0 = min(p[0] for p in bbox)
            y0 = min(p[1] for p in bbox)
            x1 = max(p[0] for p in bbox)
            y1 = max(p[1] for p in bbox)
            boxes.append([int(x0), int(y0), int(x1), int(y1)])

        if not words:
            continue

        # Process through LayoutLM
        encoding = processor(
            page_img,
            words,
            boxes=boxes,
```

```
            return_tensors="pt",
            truncation=True,
            max_length=512
        )

        with torch.no_grad():
            outputs = model(**encoding)

        predictions = outputs.logits.argmax(-1).squeeze().tolist()

        # Extract labeled entities
        for idx, pred in enumerate(predictions):
            if pred > 0 and idx < len(words):
                label = field_labels[pred]
                if label.startswith("B-"):
                    field = label[2:]
                    value = parse_financial_numbers(words[idx])
                    if not np.isnan(value):
                        extracted[field] = value

    return extracted
```

105.4 Chart and Figure Digitization

105.4.1 Motivation: Why Images Carry Financial Information

Financial charts (e.g., price time series, bar charts of earnings, scatter plots of risk-return tradeoffs) embed information that analysts process visually. For systematic strategies, this information must be converted to numerical form. Three use cases motivate chart digitization:

1. **Historical data recovery.** Pre-digital financial data often exists only in printed charts. Digitizing these charts extends historical time series beyond the electronic era.
2. **Broker report extraction.** Sell-side research reports contain charts with projections and scenario analyses. Extracting these programmatically enables systematic aggregation of analyst views.
3. **Regulatory filings.** Vietnamese regulatory filings sometimes embed data as images (charts, scanned tables) rather than as machine-readable values.

105.4.2 Chart Type Classification

The first step is classifying the chart type (line, bar, scatter, pie, candlestick), which determines the appropriate digitization algorithm.

```
def build_chart_classifier(n_classes=5):
    """
    Build a CNN-based chart type classifier.

    Classes: line_chart, bar_chart, scatter_plot,
             candlestick, pie_chart.
    """
    model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

    # Replace final layer for chart classification
    model.fc = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(512, n_classes)
    )

    return model


def classify_chart(image_path, model, transform):
    """Classify a chart image into one of 5 types."""
    class_names = [
        "line_chart", "bar_chart", "scatter_plot",
        "candlestick", "pie_chart"
    ]

    img = Image.open(image_path).convert("RGB")
    tensor = transform(img).unsqueeze(0)

    with torch.no_grad():
        logits = model(tensor)
        probs = torch.softmax(logits, dim=1).squeeze()

    pred_idx = probs.argmax().item()
    return {
        "predicted_class": class_names[pred_idx],
        "confidence": probs[pred_idx].item(),
        "all_probs": {
            name: probs[i].item()
```

```
            for i, name in enumerate(class_names)
        }
    }
```

105.4.3 Line Chart Digitization

For line charts, the digitization task is to recover the (x, y) data series from the image. The pipeline involves axis detection, scale calibration, and curve tracing.

```
def digitize_line_chart(image_path, x_range=None, y_range=None):
    """
    Digitize a line chart image to recover the data series.

    Parameters
    ----------
    image_path : str
        Path to chart image.
    x_range : tuple, optional
        (x_min, x_max) if known.
    y_range : tuple, optional
        (y_min, y_max) if known.

    Returns
    -------
    DataFrame : Digitized data points (x, y).
    """
    import cv2

    img = cv2.imread(str(image_path))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    h, w = gray.shape

    # Step 1: Detect plot area (largest rectangular region)
    edges = cv2.Canny(gray, 50, 150)
    contours, _ = cv2.findContours(
        edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    if contours:
        largest = max(contours, key=cv2.contourArea)
        x_start, y_start, plot_w, plot_h = cv2.boundingRect(largest)
```

```
    else:
        # Fallback: assume plot is central 80% of image
        x_start, y_start = int(w * 0.1), int(h * 0.1)
        plot_w, plot_h = int(w * 0.8), int(h * 0.8)

    # Step 2: Extract line pixels within plot area
    # Convert to HSV and isolate colored lines
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    plot_region = hsv[y_start:y_start + plot_h,
                      x_start:x_start + plot_w]

    # Detect non-white, non-gray pixels (likely the line)
    saturation = plot_region[:, :, 1]
    line_mask = saturation > 30  # Colored pixels

    # Step 3: Trace the line (column-wise median of colored pixels)
    data_points = []
    for col in range(plot_w):
        col_pixels = np.where(line_mask[:, col])[0]
        if len(col_pixels) > 0:
            # Use median y-position
            y_pixel = np.median(col_pixels)

            # Convert pixel to data coordinates
            x_frac = col / plot_w
            y_frac = 1 - y_pixel / plot_h  # Invert y-axis

            x_val = (x_range[0] + x_frac * (x_range[1] - x_range[0])
                     if x_range else x_frac)
            y_val = (y_range[0] + y_frac * (y_range[1] - y_range[0])
                     if y_range else y_frac)

            data_points.append({"x": x_val, "y": y_val})

    return pd.DataFrame(data_points)
```

105.5 Visual Sentiment Analysis

105.5.1 Image Sentiment in Financial News

News articles are accompanied by images that carry sentiment independent of the text. A photograph of a CEO smiling at a press conference conveys different information than the same CEO facing protesters. Obaid and Pukthuanthong (2022) demonstrate that the visual sentiment of Wall Street Journal photographs predicts market returns: days with more negative imagery precede lower returns.

We implement visual sentiment analysis using two approaches: a pre-trained sentiment classifier and a vision-language model that interprets images in financial context.

105.5.2 CNN-Based Visual Sentiment

```
def compute_visual_sentiment(image_paths, model_name="resnet50"):
    """
    Compute visual sentiment scores using a fine-tuned CNN.

    Uses features from a pre-trained CNN followed by a sentiment
    classifier trained on the Visual Sentiment Ontology (VSO)
    or similar dataset.

    Parameters
    ----------
    image_paths : list
        Paths to news images.

    Returns
    -------
    DataFrame : image_path, positive_score, negative_score, sentiment.
    """
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model, transform, dim = build_feature_extractor(model_name, device)

    # Extract features
    features = extract_features(image_paths, model, transform, device)

    # Simple sentiment model: use mean activation as proxy
    # (In practice, fine-tune on labeled financial images)
    # Higher mean activation in certain feature channels
```

```
    # correlates with positive/negative affect

    # Positive channels (empirically determined via validation)
    pos_channels = list(range(0, dim // 3))
    neg_channels = list(range(dim // 3, 2 * dim // 3))

    pos_scores = features[:, pos_channels].mean(axis=1)
    neg_scores = features[:, neg_channels].mean(axis=1)

    # Normalize to [0, 1]
    pos_norm = (pos_scores - pos_scores.min()) / (
        pos_scores.max() - pos_scores.min() + 1e-8
    )
    neg_norm = (neg_scores - neg_scores.min()) / (
        neg_scores.max() - neg_scores.min() + 1e-8
    )

    sentiment = pos_norm - neg_norm

    return pd.DataFrame({
        "image_path": image_paths,
        "positive_score": pos_norm,
        "negative_score": neg_norm,
        "net_sentiment": sentiment
    })
```

105.5.3 Vision-Language Models for Financial Image Understanding

The most powerful approach to financial image analysis uses vision-language models (VLMs), which jointly process images and text. Models such as CLIP (Radford et al. 2021), BLIP-2 (J. Li et al. 2023), and GPT-4V can be prompted to interpret financial images in context. For instance, given an aerial photograph of a factory, a VLM can answer "Is this factory operating at full capacity?" or "Is there visible construction of additional facilities?"

```
def vlm_financial_analysis(image_path, prompt, model_name="clip"):
    """
    Use a vision-language model to analyze a financial image.

    Parameters
    ----------
    image_path : str
```

```
    Path to image.
prompt : str
    Financial analysis prompt.
model_name : str
    'clip' for zero-shot classification,
    'blip2' for visual question answering.

Returns
-------
dict : Model output (scores or text).
"""
img = Image.open(image_path).convert("RGB")

if model_name == "clip":
    from transformers import CLIPProcessor, CLIPModel

    clip_model = CLIPModel.from_pretrained(
        "openai/clip-vit-base-patch32"
    )
    processor = CLIPProcessor.from_pretrained(
        "openai/clip-vit-base-patch32"
    )

    # Zero-shot classification with financial labels
    labels = [
        "busy commercial area with many customers",
        "empty commercial area with few customers",
        "active construction site with workers",
        "idle construction site without activity",
        "healthy green crops in agricultural field",
        "damaged or dry crops in agricultural field",
        "busy port with many ships and containers",
        "quiet port with few ships"
    ]

    inputs = processor(
        text=labels,
        images=img,
        return_tensors="pt",
        padding=True
    )

    with torch.no_grad():
```

```
            outputs = clip_model(**inputs)
            logits = outputs.logits_per_image.squeeze()
            probs = torch.softmax(logits, dim=0)

        results = {
            label: prob.item()
            for label, prob in zip(labels, probs)
        }

        return {"scores": results, "top_label": max(results,
         ↪  key=results.get)}

    elif model_name == "blip2":
        from transformers import Blip2Processor,
         ↪  Blip2ForConditionalGeneration

        processor = Blip2Processor.from_pretrained(
            "Salesforce/blip2-opt-2.7b"
        )
        model = Blip2ForConditionalGeneration.from_pretrained(
            "Salesforce/blip2-opt-2.7b",
            torch_dtype=torch.float16
        )

        inputs = processor(images=img, text=prompt, return_tensors="pt")

        with torch.no_grad():
            generated_ids = model.generate(**inputs, max_length=100)
            answer = processor.decode(
                generated_ids[0], skip_special_tokens=True
            )

        return {"answer": answer}
```

```
# Construct daily visual sentiment index from news images
# Source: Vietnamese financial news sites (VnExpress, CafeF, etc.)
news_images = dc.get_news_images(
    start_date="2018-01-01",
    end_date="2024-12-31",
    source=["vnexpress_finance", "cafef"]
)
```

```
# Aggregate daily visual sentiment
daily_sentiment = (
    news_images.groupby("date")
    .agg(
        visual_sentiment=("net_sentiment", "mean"),
        n_images=("net_sentiment", "count"),
        pct_negative=("net_sentiment", lambda x: (x < 0).mean())
    )
    .reset_index()
)

# Merge with market returns
market_returns = dc.get_market_returns(
    start_date="2018-01-01",
    end_date="2024-12-31",
    frequency="daily"
)

sentiment_returns = daily_sentiment.merge(
    market_returns[["date", "mkt_ret"]],
    on="date",
    how="inner"
)

# Lead-lag analysis: does visual sentiment predict next-day returns?
sentiment_returns = sentiment_returns.sort_values("date")
sentiment_returns["mkt_ret_lead1"] = sentiment_returns["mkt_ret"].shift(-1)
```

105.6 Multimodal Fusion: Combining Image and Text

105.6.1 Why Multimodal?

Text and images capture different dimensions of the same underlying economic reality. An earnings report describes financial performance in words and numbers; the accompanying photographs show factories, products, and management. A news article about a port describes trade volumes in text; the satellite image shows actual ship positions. Combining both modalities yields a richer representation than either alone.

The fusion architecture depends on the application:

Table 105.3: Visual Sentiment and Market Return Predictability

```
# Regression: next-day return on today's visual sentiment
sr_clean = sentiment_returns.dropna(
    subset=["mkt_ret_lead1", "visual_sentiment", "mkt_ret"]
)

model_sent = sm.OLS(
    sr_clean["mkt_ret_lead1"],
    sm.add_constant(sr_clean[["visual_sentiment", "mkt_ret"]])
).fit(cov_type="HAC", cov_kwds={"maxlags": 5})

sent_results = pd.DataFrame({
    "Coefficient": model_sent.params.round(6),
    "Std Error": model_sent.bse.round(6),
    "t-stat": model_sent.tvalues.round(3),
    "p-value": model_sent.pvalues.round(4)
})
sent_results
```

Early fusion. Concatenate image features $\mathbf{z}^{\text{img}}$ and text features $\mathbf{z}^{\text{txt}}$ into a single vector $[\mathbf{z}^{\text{img}}; \mathbf{z}^{\text{txt}}]$ before prediction. Simple but ignores cross-modal interactions.

Late fusion. Train separate models on each modality and combine predictions: $\hat{y} = \alpha \hat{y}^{\text{img}} + (1-\alpha)\hat{y}^{\text{txt}}$. Robust but cannot learn cross-modal features.

Cross-attention fusion. Use transformer cross-attention to let each modality attend to the other. Most powerful but requires more data and computation.

$$\mathbf{z}^{\text{fused}} = \text{CrossAttention}(\mathbf{z}^{\text{img}}, \mathbf{z}^{\text{txt}}) = \text{softmax}\left(\frac{\mathbf{Q}^{\text{img}}(\mathbf{K}^{\text{txt}})^\top}{\sqrt{d}}\right)\mathbf{V}^{\text{txt}} \tag{105.3}$$

```
class MultimodalFusionModel(nn.Module):
    """
    Multimodal fusion model combining image and text features
    for financial prediction.

    Supports early fusion, late fusion, and cross-attention.
    """

    def __init__(self, img_dim=2048, txt_dim=768, hidden_dim=256,
```

```
                 fusion="early", n_heads=4):
        super().__init__()
        self.fusion = fusion

        # Image projection
        self.img_proj = nn.Sequential(
            nn.Linear(img_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.2)
        )

        # Text projection
        self.txt_proj = nn.Sequential(
            nn.Linear(txt_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.2)
        )

        if fusion == "early":
            self.head = nn.Sequential(
                nn.Linear(hidden_dim * 2, hidden_dim),
                nn.ReLU(),
                nn.Dropout(0.2),
                nn.Linear(hidden_dim, 1)
            )
        elif fusion == "late":
            self.img_head = nn.Linear(hidden_dim, 1)
            self.txt_head = nn.Linear(hidden_dim, 1)
            self.alpha = nn.Parameter(torch.tensor(0.5))
        elif fusion == "cross_attention":
            self.cross_attn = nn.MultiheadAttention(
                embed_dim=hidden_dim,
                num_heads=n_heads,
                batch_first=True
            )
            self.head = nn.Sequential(
                nn.Linear(hidden_dim, hidden_dim // 2),
                nn.ReLU(),
                nn.Linear(hidden_dim // 2, 1)
            )

    def forward(self, img_features, txt_features):
        img_h = self.img_proj(img_features)
```

```
        txt_h = self.txt_proj(txt_features)

        if self.fusion == "early":
            combined = torch.cat([img_h, txt_h], dim=-1)
            return self.head(combined).squeeze(-1)

        elif self.fusion == "late":
            img_pred = self.img_head(img_h).squeeze(-1)
            txt_pred = self.txt_head(txt_h).squeeze(-1)
            alpha = torch.sigmoid(self.alpha)
            return alpha * img_pred + (1 - alpha) * txt_pred

        elif self.fusion == "cross_attention":
            # Image attends to text
            img_h_unsq = img_h.unsqueeze(1)  # (B, 1, D)
            txt_h_unsq = txt_h.unsqueeze(1)

            attn_out, _ = self.cross_attn(
                img_h_unsq, txt_h_unsq, txt_h_unsq
            )
            return self.head(attn_out.squeeze(1)).squeeze(-1)
```

```
def run_multimodal_experiment(image_features, text_features, returns,
                              fusion_types=["early", "late",
                                            "cross_attention"]):
    """
    Compare multimodal fusion strategies for return prediction.

    Parameters
    ----------
    image_features : np.ndarray
        Image feature matrix (N x img_dim).
    text_features : np.ndarray
        Text feature matrix (N x txt_dim).
    returns : np.ndarray
        Target returns (N,).
    fusion_types : list
        Fusion strategies to compare.

    Returns
    -------
    DataFrame : R², MSE, Sharpe for each strategy.
```

```
    """
    from sklearn.model_selection import TimeSeriesSplit

    n = len(returns)
    tscv = TimeSeriesSplit(n_splits=5)

    results = []

    for fusion in fusion_types:
        fold_r2s = []

        for train_idx, test_idx in tscv.split(returns):
            # Convert to tensors
            X_img_train = torch.tensor(
                image_features[train_idx], dtype=torch.float32
            )
            X_txt_train = torch.tensor(
                text_features[train_idx], dtype=torch.float32
            )
            y_train = torch.tensor(
                returns[train_idx], dtype=torch.float32
            )

            X_img_test = torch.tensor(
                image_features[test_idx], dtype=torch.float32
            )
            X_txt_test = torch.tensor(
                text_features[test_idx], dtype=torch.float32
            )
            y_test = returns[test_idx]

            # Build and train model
            model = MultimodalFusionModel(
                img_dim=image_features.shape[1],
                txt_dim=text_features.shape[1],
                fusion=fusion
            )

            optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
            loss_fn = nn.MSELoss()

            model.train()
            for epoch in range(50):
```

```
            optimizer.zero_grad()
            pred = model(X_img_train, X_txt_train)
            loss = loss_fn(pred, y_train)
            loss.backward()
            optimizer.step()

        # Evaluate
        model.eval()
        with torch.no_grad():
            y_pred = model(X_img_test, X_txt_test).numpy()

        ss_res = np.sum((y_test - y_pred) ** 2)
        ss_tot = np.sum((y_test - y_test.mean()) ** 2)
        r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0

        fold_r2s.append(r2)

    results.append({
        "fusion": fusion,
        "r2_mean": np.mean(fold_r2s),
        "r2_std": np.std(fold_r2s)
    })

# Add unimodal baselines
for modality, features in [("image_only", image_features),
                           ("text_only", text_features)]:
    from sklearn.linear_model import RidgeCV
    fold_r2s = []
    for train_idx, test_idx in tscv.split(returns):
        ridge = RidgeCV(alphas=np.logspace(-3, 3, 10))
        ridge.fit(features[train_idx], returns[train_idx])
        y_pred = ridge.predict(features[test_idx])
        y_test = returns[test_idx]
        ss_res = np.sum((y_test - y_pred) ** 2)
        ss_tot = np.sum((y_test - y_test.mean()) ** 2)
        fold_r2s.append(1 - ss_res / ss_tot if ss_tot > 0 else 0)

    results.append({
        "fusion": modality,
        "r2_mean": np.mean(fold_r2s),
        "r2_std": np.std(fold_r2s)
    })
```

```
    return pd.DataFrame(results)
```

105.6.2 Practical Considerations for Vietnamese Markets

Multimodal analysis in Vietnamese markets faces several practical considerations:

Data alignment. Satellite images, news articles, and market data operate on different temporal frequencies and spatial resolutions. Satellite composites are available weekly or biweekly; news is daily; trading is intraday. Proper alignment requires specifying the information set available to an investor at the time of the trading decision to avoid look-ahead bias.

Label scarcity. Supervised learning requires labeled data (e.g., images annotated with economic outcomes). In Vietnam, ground-truth labels (actual retail sales, actual crop yields, actual port throughput) arrive with significant lags and often lack the granularity to match satellite resolution. Semi-supervised and self-supervised approaches are therefore essential.

Regulatory considerations. High-resolution satellite imagery of specific commercial or military installations may be restricted. Researchers should verify that their imagery sources comply with Vietnamese regulations on geospatial data.

Computational cost. Processing satellite tiles through CNNs is computationally intensive. A single Sentinel-2 tile at 10m resolution covering Ho Chi Minh City contains approximately $10{,}980 \times 10{,}980$ pixels per band. Tiling into 224×224 patches for CNN input generates $\sim 2{,}400$ patches per tile, each requiring a forward pass through the network.

Table 105.4: Image Data Sources for Vietnamese Financial Applications

Application	Image Source	Resolution	Frequency	Vietnamese Availability
Nighttime luminosity	VIIRS/DMSP	500m	Monthly	Free (NOAA/EOG)
Crop health	MODIS/Sentinel-2	250m/10m	16-day/5-day	Free (NASA/ESA)
Port/ship detection	Sentinel-1 (SAR)	10m	12-day	Free (ESA Copernicus)
Construction monitoring	Commercial (Maxar)	30cm	On demand	Paid ($)
Urban density	Sentinel-2	10m	5-day	Free (ESA)
Document OCR	Corporate filings	N/A	Event-driven	DataCore.vn
News images	Financial media	N/A	Daily	Web scraping

105.7 Summary

This chapter extended the alternative data toolkit from text (the previous chapter) to images. We demonstrated five distinct application domains for visual data in Vietnamese financial markets.

First, satellite and geospatial imagery provides high-frequency, spatially granular economic signals that lead official statistics. Nighttime luminosity serves as a provincial GDP proxy with cross-sectional R^2 exceeding 0.7; NDVI crop health indices predict agricultural firm returns; and CNN features extracted from satellite tiles enable rich spatial representations of economic activity.

Second, document image analysis solves the practical problem of extracting structured data from Vietnamese financial filings that arrive as scanned images. The pipeline (e.g., OCR with Vietnamese-optimized engines, layout analysis, table extraction, and LayoutLM-based document understanding) converts unstructured pixels into the structured financial data that all downstream analyses require.

Third, chart digitization recovers numerical data series from visual representations, extending historical coverage and enabling systematic consumption of analyst outputs. Fourth, visual sentiment analysis from news imagery provides a signal dimension orthogonal to textual sentiment, with potential predictive power for market returns.

Fifth, multimodal fusion (combining image and text representations via early, late, or cross-attention architectures) yields richer predictive models than either modality alone. The practical benefit of multimodal approaches scales with the diversity and quality of available data, making it increasingly relevant as Vietnamese alternative data ecosystems mature.

The common thread across all applications is the transformation pipeline: raw pixel tensor $\rightarrow$ feature representation (via CNN, ViT, or VLM) $\rightarrow$ financial signal $\rightarrow$ economic interpretation. The choice of architecture and the quality of the domain adaptation determine whether the resulting signal has genuine predictive content or merely captures noise.

106 Networks and Graphs in Finance

Financial markets are networks. Every stock return is shaped by connections: firms share supply chains, directors, auditors, investors, lenders, and regulators. Shocks propagate through these links (e.g., a bankruptcy ripples through supplier networks, a central bank liquidity squeeze radiates through interbank exposures, and information diffuses through board interlocks and institutional co-ownership). Yet the dominant empirical paradigm in finance treats firms as isolated observations indexed by (i, t), connected only through their shared exposure to common factors. This chapter introduces tools for modeling, measuring, and exploiting the network structure that standard panel regressions ignore.

The Vietnamese financial system is particularly network-dense. State ownership creates a lattice of cross-connected enterprises: the same line ministry may oversee the borrower, the lender, and the insurer. Pyramidal business groups (such as Vingroup, Masan, and FPT) link dozens of listed and unlisted entities through chains of equity ownership. The banking system is small and concentrated, with a few state-owned commercial banks accounting for the majority of assets, generating dense interbank exposures. Board interlocks (e.g., directors who serve on multiple boards simultaneously) are pervasive and often follow ownership lines. And the equity market itself exhibits return co-movement patterns that, when represented as a correlation network, reveal sector-level and ownership-level clustering invisible in standard factor analysis.

This chapter covers the full spectrum of network methods used in financial economics, from classical graph theory through modern graph neural networks (GNNs). We organize the material in eight sections. First, graph theory fundamentals and the representation of financial data as networks. Second, ownership and control networks, which are the most distinctive network structure in Vietnamese markets. Third, board interlock and governance networks. Fourth, supply chain and trade networks. Fifth, correlation and co-movement networks for portfolio construction and systemic risk monitoring. Sixth, interbank and contagion networks. Seventh, social networks between places, measured with the Social Connectedness Index, and their role as conduits of financial information (L. Allen, Peng, and Shan 2026). Eighth, graph neural networks and graph-based machine learning for asset pricing, credit risk, and anomaly detection.

```
import pandas as pd
import numpy as np
from pathlib import Path
```

```
import warnings
warnings.filterwarnings("ignore")

# Graph libraries
import networkx as nx
from scipy import sparse
from scipy.spatial.distance import squareform

# Statistical and econometric
from scipy import stats
import statsmodels.api as sm
from linearmodels.panel import PanelOLS

# Visualization
import plotnine as p9
from mizani.formatters import percent_format, comma_format
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# Deep learning (for GNNs later)
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
# DataCore.vn API
from datacore import DataCore
dc = DataCore()
```

106.1 Graph Theory Foundations for Finance

106.1.1 Representing Financial Data as Graphs

A graph $G = (V, E)$ consists of a set of vertices (nodes) V and edges (links) $E \subseteq V \times V$. In financial networks, nodes represent economic agents, such as firms, banks, investors, directors, and edges represent relationships including ownership stakes, lending, board seats, supply contracts, and return co-movement.

Financial graphs come in several varieties:

Directed vs. undirected. Ownership is directed: firm A owns a stake in firm B, but not necessarily vice versa. Board interlocks are undirected: if director d sits on both firm A's and firm B's boards, the connection is symmetric. Supply chains are directed: A supplies to B.

Weighted vs. unweighted. Ownership networks are naturally weighted by the ownership percentage. Correlation networks are weighted by the pairwise correlation coefficient. Board interlocks can be weighted by the number of shared directors.

Static vs. dynamic. Most financial networks evolve over time as ownership changes, directors rotate, and correlations shift. A temporal graph $G_t = (V_t, E_t)$ captures this evolution.

Bipartite vs. unipartite. The raw data for many financial networks is bipartite: directors $\times$ firms, investors $\times$ stocks, banks $\times$ borrowers. The one-mode projection converts this to a unipartite graph: two firms are connected if they share a director, two stocks are connected if they share an institutional investor, etc.

106.1.2 Key Graph Metrics

For a graph G with $n = |V|$ nodes and $m = |E|$ edges, we define the following metrics, each with a specific financial interpretation.

Degree centrality. The degree k_i of node i is the number of edges incident to i. In a directed graph, we distinguish in-degree k_i^{in} (edges pointing to i) and out-degree k_i^{out} (edges from i). Normalized degree centrality is:

$$C_D(i) = \frac{k_i}{n-1} \tag{106.1}$$

In an ownership network, high out-degree means a firm owns stakes in many others (conglomerate); high in-degree means many entities own stakes in the firm (dispersed ownership).

Betweenness centrality. The fraction of shortest paths between all pairs of nodes that pass through node i:

$$C_B(i) = \sum_{s \neq i \neq t} \frac{\sigma_{st}(i)}{\sigma_{st}} \tag{106.2}$$

where σ_{st} is the total number of shortest paths from s to t and $\sigma_{st}(i)$ is the number that pass through i. High betweenness identifies "bridge" nodes (i.e., firms or banks that connect otherwise disconnected parts of the financial system). The failure of a high-betweenness bank can fragment the interbank network.

Eigenvector centrality. A node is central if it is connected to other central nodes. The eigenvector centrality is the solution to:

$$\lambda \mathbf{c} = A\mathbf{c} \tag{106.3}$$

where A is the adjacency matrix and λ is the largest eigenvalue. Google's PageRank is a regularized variant designed for directed graphs. In financial networks, eigenvector centrality identifies systemically important institutions (i.e., those connected to other important institutions).

Clustering coefficient. The fraction of a node's neighbors that are also neighbors of each other:

$$C_C(i) = \frac{2T_i}{k_i(k_i - 1)} \tag{106.4}$$

where T_i is the number of triangles containing node i. High clustering indicates tightly knit groups (e.g., business groups, lending circles, co-invested portfolios).

Community structure. Many financial networks exhibit modular structure: groups of densely connected nodes with sparse connections between groups. Community detection algorithms (Louvain, label propagation, spectral clustering) identify these modules, which in financial networks often correspond to business groups, industry sectors, or lending clusters.

```
def compute_network_metrics(G):
    """
    Compute standard network metrics for a graph.

    Parameters
    ----------
    G : nx.Graph or nx.DiGraph
        Input graph.

    Returns
    -------
    dict : Node-level and graph-level metrics.
    """
    is_directed = G.is_directed()

    # Node-level metrics
    if is_directed:
        in_degree = dict(G.in_degree())
        out_degree = dict(G.out_degree())
        degree = {n: in_degree[n] + out_degree[n] for n in G.nodes()}
    else:
        degree = dict(G.degree())
```

```
betweenness = nx.betweenness_centrality(G, weight="weight")
eigenvector = nx.eigenvector_centrality_numpy(
    G, weight="weight"
) if len(G) > 0 else {}
clustering = nx.clustering(G, weight="weight") if not is_directed else {}

# PageRank (works for both directed and undirected)
pagerank = nx.pagerank(G, weight="weight")

# Graph-level metrics
n_nodes = G.number_of_nodes()
n_edges = G.number_of_edges()
density = nx.density(G)

# Connected components
if is_directed:
    n_weak_components = nx.number_weakly_connected_components(G)
    n_strong_components = nx.number_strongly_connected_components(G)
else:
    n_components = nx.number_connected_components(G)

# Degree distribution statistics
degrees = list(degree.values())
avg_degree = np.mean(degrees) if degrees else 0
max_degree = max(degrees) if degrees else 0

# Assortativity
assortativity = nx.degree_assortativity_coefficient(G)

# Community detection (Louvain)
if not is_directed:
    try:
        communities = nx.community.louvain_communities(G)
        modularity = nx.community.modularity(G, communities)
        n_communities = len(communities)
    except Exception:
        modularity, n_communities = np.nan, 0
else:
    modularity, n_communities = np.nan, 0

node_metrics = pd.DataFrame({
    "degree": degree,
```

```
        "betweenness": betweenness,
        "eigenvector": eigenvector,
        "pagerank": pagerank,
        "clustering": clustering if clustering else {n: np.nan for n in
         ↪  G.nodes()}
    })

    graph_metrics = {
        "n_nodes": n_nodes,
        "n_edges": n_edges,
        "density": density,
        "avg_degree": avg_degree,
        "max_degree": max_degree,
        "assortativity": assortativity,
        "modularity": modularity,
        "n_communities": n_communities
    }

    return node_metrics, graph_metrics
```

106.1.3 The Adjacency Matrix and Its Spectral Properties

The adjacency matrix $A \in \mathbb{R}^{n \times n}$ encodes the graph structure: $A_{ij} = w_{ij}$ if there is an edge from i to j with weight w_{ij}, and $A_{ij} = 0$ otherwise. For undirected graphs, A is symmetric.

The graph Laplacian $L = D - A$ (where D is the diagonal degree matrix) has eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq ... \leq \lambda_n$ with important structural interpretations:

- The multiplicity of $\lambda = 0$ equals the number of connected components.
- The second eigenvalue λ_2 (the algebraic connectivity or Fiedler value) measures how well-connected the graph is. Low λ_2 implies the graph has a bottleneck, which is a weak point where cutting a few edges would disconnect large portions.
- The eigenvectors of L provide the spectral embedding of the graph, which is the foundation for spectral clustering and graph convolutional networks.

The normalized Laplacian $\tilde{L} = D^{-1/2} L D^{-1/2}$ is used in GCNs because it stabilizes message passing across nodes with different degrees.

```
def spectral_graph_analysis(A, n_components=10):
    """
    Compute spectral properties of a financial network.
```

```
    Parameters
    ----------
    A : np.ndarray or sparse matrix
        Adjacency matrix.
    n_components : int
        Number of eigenvalues/vectors to compute.

    Returns
    -------
    dict : Eigenvalues, algebraic connectivity, spectral gap.
    """
    n = A.shape[0]

    if sparse.issparse(A):
        A_dense = A.toarray()
    else:
        A_dense = A

    # Degree matrix
    D = np.diag(A_dense.sum(axis=1))

    # Graph Laplacian
    L = D - A_dense

    # Normalized Laplacian
    D_inv_sqrt = np.diag(1.0 / np.sqrt(np.diag(D) + 1e-10))
    L_norm = D_inv_sqrt @ L @ D_inv_sqrt

    # Eigendecomposition (smallest eigenvalues)
    eigenvalues, eigenvectors = np.linalg.eigh(L_norm)

    # Sort by eigenvalue
    idx = np.argsort(eigenvalues)
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]

    # Algebraic connectivity (Fiedler value)
    fiedler_value = eigenvalues[1] if n > 1 else 0
    fiedler_vector = eigenvectors[:, 1] if n > 1 else np.zeros(n)

    # Spectral gap
    spectral_gap = eigenvalues[1] - eigenvalues[0] if n > 1 else 0
```

```
    return {
        "eigenvalues": eigenvalues[:n_components],
        "fiedler_value": fiedler_value,
        "fiedler_vector": fiedler_vector,
        "spectral_gap": spectral_gap,
        "spectral_embedding": eigenvectors[:, 1:n_components + 1]
    }
```

106.2 Ownership and Control Networks

106.2.1 Vietnamese Ownership Structure

Ownership networks are arguably the most economically consequential graph structure in Vietnamese markets. The Vietnamese corporate landscape is characterized by three distinctive features that generate complex ownership topologies:

State ownership pyramids. The government holds equity in hundreds of firms through a hierarchy of holding entities: the State Capital Investment Corporation (SCIC), line ministries, provincial People's Committees, and state-owned economic groups (tập đoàn kinh tế nhà nước). These chains create multi-layered pyramids where the state's ultimate control rights may substantially exceed its cash flow rights.

Private business groups. Vietnamese conglomerates (Vingroup, Masan, FPT, Hoà Phát, Thaco) create complex webs of cross-ownership, subsidiary relationships, and associate stakes. These structures serve multiple purposes: internal capital markets, tax optimization, regulatory arbitrage, and control enhancement.

Circular and cross-ownership. Vietnamese regulations do not effectively prevent circular ownership (firm A owns firm B which owns firm C which owns firm A), creating loops in the ownership graph that amplify control beyond direct stakes and inflate accounting equity (L. A. Bebchuk 1999).

```
# Load ownership data
ownership = dc.get_ownership_network(
    date="2024-06-30",
    min_stake=1.0  # Minimum 1% ownership stake
)

# Load firm characteristics
firms = dc.get_firm_characteristics(
    start_date="2024-01-01",
    end_date="2024-06-30"
```

```
).groupby("ticker").last().reset_index()

print(f"Ownership edges: {len(ownership)}")
print(f"Unique owners: {ownership['owner_id'].nunique()}")
print(f"Unique targets: {ownership['target_ticker'].nunique()}")
```

```
# Build directed ownership graph
G_own = nx.DiGraph()

# Add firm nodes with attributes
for _, row in firms.iterrows():
    G_own.add_node(
        row["ticker"],
        node_type="firm",
        market_cap=row.get("market_cap", 0),
        industry=row.get("industry", "Unknown"),
        is_soe=row.get("state_ownership_pct", 0) > 50
    )

# Add ownership edges
for _, row in ownership.iterrows():
    owner = row["owner_id"]
    target = row["target_ticker"]
    stake = row["ownership_pct"]

    # Add owner node if not present
    if owner not in G_own:
        G_own.add_node(
            owner,
            node_type=row.get("owner_type", "entity"),
            market_cap=0,
            industry="Holding"
        )

    G_own.add_edge(owner, target, weight=stake / 100)

node_metrics_own, graph_metrics_own = compute_network_metrics(G_own)

print(f"\nOwnership Network Summary:")
for k, v in graph_metrics_own.items():
    print(f"  {k}: {v}")
```

Table 106.1: Ownership Network: Graph-Level Statistics

```
own_stats = pd.DataFrame([graph_metrics_own]).T
own_stats.columns = ["Value"]
own_stats = own_stats.round(4)
own_stats
```

106.2.2 Ultimate Ownership and Control Chains

Direct ownership understates the actual control exercised through pyramidal chains. The ultimate ownership stake of entity A in firm Z through a chain $A \to B \to C \to Z$ is the product of intermediate stakes:

$$\omega_{A \to Z}^{\text{ultimate}} = \prod_{(i,j) \in \text{path}(A,Z)} w_{ij} \tag{106.5}$$

where w_{ij} is the direct stake of i in j. When multiple paths exist from A to Z, the total ultimate ownership is the sum across paths. The control rights, however, are determined by the weakest link in the chain (the minimum stake along the path), reflecting the principle that control requires a majority at each level.

```
def compute_ultimate_ownership(G, source, target, max_depth=10):
    """
    Compute ultimate ownership of source in target through all paths.

    Parameters
    ----------
    G : nx.DiGraph
        Ownership graph with edge weights as ownership fractions.
    source : str
        Ultimate owner node.
    target : str
        Target firm node.
    max_depth : int
        Maximum chain length to consider.

    Returns
    -------
    dict : Total ownership, control rights, number of paths.
    """
```

```
    if source not in G or target not in G:
        return {"ownership": 0, "control": 0, "n_paths": 0}

    total_ownership = 0
    max_control = 0
    n_paths = 0

    # Find all simple paths from source to target
    try:
        for path in nx.all_simple_paths(G, source, target,
                                        cutoff=max_depth):
            # Cash flow rights: product of stakes
            ownership = 1.0
            min_stake = 1.0
            for i in range(len(path) - 1):
                edge_data = G[path[i]][path[i + 1]]
                stake = edge_data.get("weight", 0)
                ownership *= stake
                min_stake = min(min_stake, stake)

            total_ownership += ownership
            max_control = max(max_control, min_stake)
            n_paths += 1
    except nx.NetworkXNoPath:
        pass

    return {
        "ownership": total_ownership,
        "control": max_control,
        "n_paths": n_paths
    }


def compute_control_wedge(G, firms_list, max_depth=5):
    """
    Compute the wedge between control and cash flow rights
    for the largest shareholder of each firm.

    The wedge = control rights - cash flow rights.
    Positive wedge → potential for tunneling.
    """
    wedge_data = []
```

```
    for firm in firms_list:
        if firm not in G:
            continue

        # Find all direct owners
        predecessors = list(G.predecessors(firm))
        if not predecessors:
            continue

        # Find largest direct owner
        stakes = {p: G[p][firm]["weight"] for p in predecessors}
        largest_owner = max(stakes, key=stakes.get)
        direct_stake = stakes[largest_owner]

        # Compute ultimate ownership through all paths
        ultimate = compute_ultimate_ownership(
            G, largest_owner, firm, max_depth
        )

        wedge_data.append({
            "ticker": firm,
            "largest_owner": largest_owner,
            "direct_stake": direct_stake,
            "ultimate_ownership": ultimate["ownership"],
            "control_rights": ultimate["control"],
            "n_ownership_paths": ultimate["n_paths"],
            "wedge": ultimate["control"] - ultimate["ownership"]
        })

    return pd.DataFrame(wedge_data)


# Compute for all listed firms
listed_firms = [n for n, d in G_own.nodes(data=True)
                if d.get("node_type") == "firm"]
wedge_df = compute_control_wedge(G_own, listed_firms)
```

106.2.3 Ownership Centrality and Firm Value

We test whether a firm's position in the ownership network predicts its market valuation, controlling for standard determinants:

Table 106.2: Control-Cash Flow Wedge: Summary Statistics

```
wedge_summary = wedge_df[
    ["direct_stake", "ultimate_ownership", "control_rights",
     "n_ownership_paths", "wedge"]
].describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9]).T.round(4)
wedge_summary
```

```
(
    p9.ggplot(wedge_df, p9.aes(x="wedge"))
    + p9.geom_histogram(bins=50, fill="#2E5090", alpha=0.7)
    + p9.geom_vline(xintercept=0, linetype="dashed", color="#C0392B")
    + p9.labs(
        x="Control Wedge (Control Rights - Cash Flow Rights)",
        y="Count",
        title="Control-Ownership Wedge: Positive Values Indicate Tunneling
↪   Risk"
    )
    + p9.theme_minimal()
    + p9.theme(figure_size=(10, 5))
)
```

Figure 106.1

$$Q_{i,t} = \beta_0 + \beta_1 \text{Centrality}_{i,t} + \beta_2 \text{Wedge}_{i,t} + \gamma' \mathbf{X}_{i,t} + \alpha_{\text{ind}} + \delta_t + \varepsilon_{i,t} \tag{106.6}$$

```
# Merge network metrics with firm characteristics
node_metrics_own_df = node_metrics_own.reset_index().rename(
    columns={"index": "ticker"}
)

valuation_data = firms.merge(node_metrics_own_df, on="ticker", how="inner")
valuation_data = valuation_data.merge(
    wedge_df[["ticker", "wedge", "n_ownership_paths"]],
    on="ticker", how="left"
)

# Panel regression
val_clean = valuation_data.dropna(
    subset=["tobins_q", "eigenvector", "wedge",
            "log_size", "profitability", "leverage"]
)

if len(val_clean) > 50:
    model_network_val = sm.OLS(
        val_clean["tobins_q"],
        sm.add_constant(val_clean[[
            "eigenvector", "betweenness", "wedge",
            "log_size", "profitability", "leverage"
        ]])
    ).fit(cov_type="HC1")

    print("Ownership Network Position and Firm Value:")
    for var in ["eigenvector", "betweenness", "wedge"]:
        print(f"  {var}: {model_network_val.params[var]:.4f} "
              f"(t = {model_network_val.tvalues[var]:.3f})")
```

106.2.4 Business Group Detection

Business groups (i.e., collections of legally independent firms linked by common ownership) are a defining feature of Vietnamese corporate structure. We detect them algorithmically using community detection on the ownership graph.

```
def detect_business_groups(G, min_group_size=3, ownership_threshold=0.10):
    """
    Detect business groups from ownership network.

    A business group is a connected component in the undirected
    projection of the ownership graph, filtered for economically
    meaningful ownership stakes.

    Parameters
    ----------
    G : nx.DiGraph
        Directed ownership graph.
    min_group_size : int
        Minimum firms in a group.
    ownership_threshold : float
        Minimum ownership stake to count as a link.

    Returns
    -------
    DataFrame : Group assignments with apex firm.
    """
    # Filter edges by threshold
    G_filtered = nx.DiGraph()
    for u, v, d in G.edges(data=True):
        if d.get("weight", 0) >= ownership_threshold:
            G_filtered.add_edge(u, v, **d)

    # Convert to undirected for component detection
    G_undirected = G_filtered.to_undirected()

    # Find connected components
    components = list(nx.connected_components(G_undirected))

    # Filter by size
    groups = [c for c in components if len(c) >= min_group_size]

    # For each group, identify the apex (top of pyramid)
    group_data = []
    for group_id, members in enumerate(groups):
        subgraph = G_filtered.subgraph(members)

        # Apex: node with highest out-degree and lowest in-degree
        # (controls others but is not controlled)
```

```
        scores = {}
        for node in members:
            in_deg = subgraph.in_degree(node)
            out_deg = subgraph.out_degree(node)
            scores[node] = out_deg - in_deg

        apex = max(scores, key=scores.get) if scores else list(members)[0]

        for member in members:
            node_data = G.nodes.get(member, {})
            group_data.append({
                "ticker": member,
                "group_id": group_id,
                "group_size": len(members),
                "apex": apex,
                "is_apex": member == apex,
                "node_type": node_data.get("node_type", "unknown"),
                "industry": node_data.get("industry", "Unknown"),
                "market_cap": node_data.get("market_cap", 0)
            })

    return pd.DataFrame(group_data)


groups_df = detect_business_groups(G_own)
print(f"Business groups detected: {groups_df['group_id'].nunique()}")
print(f"Firms in groups: {len(groups_df[groups_df['node_type'] == 'firm'])}")
```

106.3 Board Interlock Networks

106.3.1 Construction

A board interlock exists when a director serves on the boards of two or more firms simultaneously. The board interlock network is a unipartite projection of the bipartite director×firm graph: two firms are connected if they share at least one director, weighted by the number of shared directors.

Board interlocks are an information channel. Bizjak, Lemmon, and Naveen (2008) show that compensation practices diffuse through board networks. Cai et al. (2014) demonstrate that

Table 106.3: Largest Vietnamese Business Groups by Ownership Network Analysis

```
top_groups = (
    groups_df.groupby("group_id")
    .agg(
        apex=("apex", "first"),
        n_members=("ticker", "count"),
        n_listed=("node_type", lambda x: (x == "firm").sum()),
        total_mcap=("market_cap", "sum"),
        industries=("industry", lambda x: x.nunique())
    )
    .sort_values("total_mcap", ascending=False)
    .head(15)
    .reset_index()
)
top_groups["total_mcap_bn"] = top_groups["total_mcap"] / 1e9
top_groups[["apex", "n_members", "n_listed", "total_mcap_bn", "industries"]]
```

firms connected through interlocks have correlated investment policies. In Vietnamese markets, interlocks often follow ownership lines (e.g., the parent company appoints directors to subsidiary boards), creating a governance channel that reinforces the ownership channel.

```
# Load board membership data
board_data = dc.get_board_memberships(
    date="2024-06-30"
)

print(f"Director-firm pairs: {len(board_data)}")
print(f"Unique directors: {board_data['director_id'].nunique()}")
print(f"Unique firms: {board_data['ticker'].nunique()}")

# Build bipartite graph
B = nx.Graph()
for _, row in board_data.iterrows():
    B.add_node(row["director_id"], bipartite=0)
    B.add_node(row["ticker"], bipartite=1)
    B.add_edge(
        row["director_id"], row["ticker"],
        role=row.get("role", "director"),
        is_independent=row.get("is_independent", False)
    )
```

```
# Project to firm-firm interlock network
firm_nodes = [n for n, d in B.nodes(data=True) if d.get("bipartite") == 1]
G_interlock = nx.bipartite.weighted_projected_graph(B, firm_nodes)

# Edge weight = number of shared directors
interlock_metrics, interlock_graph = compute_network_metrics(G_interlock)

print(f"\nBoard Interlock Network:")
print(f"  Firms: {G_interlock.number_of_nodes()}")
print(f"  Interlocking pairs: {G_interlock.number_of_edges()}")
print(f"  Density: {nx.density(G_interlock):.4f}")
```

```
# Do board interlocks follow ownership lines?
# Compare interlock edges with ownership edges
interlock_edges = set(G_interlock.edges())
ownership_edges_undirected = set()

for u, v in G_own.edges():
    if u in firm_nodes and v in firm_nodes:
        ownership_edges_undirected.add((min(u, v), max(u, v)))

interlock_edges_normalized = {
    (min(u, v), max(u, v)) for u, v in interlock_edges
}

overlap = interlock_edges_normalized & ownership_edges_undirected
n_interlock = len(interlock_edges_normalized)
n_ownership = len(ownership_edges_undirected)
n_overlap = len(overlap)

print(f"Interlock edges: {n_interlock}")
print(f"Ownership edges (firm-firm): {n_ownership}")
print(f"Overlap: {n_overlap}")
if n_interlock > 0:
    print(f"Fraction of interlocks with ownership link: "
          f"{n_overlap / n_interlock:.3f}")
```

106.3.2 Interlocks and Return Co-Movement

If board interlocks serve as information channels, firms connected through interlocks should exhibit excess return co-movement beyond what is explained by shared industry or size characteristics. We test this using the methodology of L. Cohen and Frazzini (2008):

$$\rho_{ij,t} = \alpha + \beta \cdot \text{Interlock}_{ij,t} + \gamma \cdot \text{SameIndustry}_{ij} + \delta \cdot \text{SizeProximity}_{ij,t} + \varepsilon_{ij,t} \tag{106.7}$$

```
# Compute pairwise return correlations for interlocked and non-interlocked
 ↪  pairs
monthly_returns = dc.get_monthly_returns(
    start_date="2023-01-01",
    end_date="2024-06-30"
)

# Pivot to wide format
returns_wide = monthly_returns.pivot(
    index="date", columns="ticker", values="ret"
).dropna(axis=1, thresh=12)

# Correlation matrix
corr_matrix = returns_wide.corr()

# Compare correlations: interlocked vs non-interlocked
interlock_corrs = []
non_interlock_corrs = []

tickers_in_both = set(corr_matrix.columns) & set(G_interlock.nodes())

for i, ticker_i in enumerate(tickers_in_both):
    for ticker_j in list(tickers_in_both)[i + 1:]:
        corr = corr_matrix.loc[ticker_i, ticker_j]
        if np.isnan(corr):
            continue

        if G_interlock.has_edge(ticker_i, ticker_j):
            interlock_corrs.append(corr)
        else:
            non_interlock_corrs.append(corr)

if interlock_corrs and non_interlock_corrs:
    t_stat, p_val = stats.ttest_ind(interlock_corrs, non_interlock_corrs)
```

```
    print(f"Interlocked pairs: n={len(interlock_corrs)}, "
          f"mean corr={np.mean(interlock_corrs):.4f}")
    print(f"Non-interlocked: n={len(non_interlock_corrs)}, "
          f"mean corr={np.mean(non_interlock_corrs):.4f}")
    print(f"Difference t-stat: {t_stat:.3f}, p-value: {p_val:.4f}")
```

106.4 Supply Chain and Trade Networks

106.4.1 Constructing the Supply Chain Graph

Supply chain relationships (customer-supplier linkages) represent real economic connections through which shocks propagate. Acemoglu, Ozdaglar, and Tahbaz-Salehi (2015) demonstrate theoretically that in the presence of supply chain linkages, idiosyncratic shocks to individual firms can generate aggregate fluctuations rather than washing out. L. Cohen and Frazzini (2008) and Menzly and Ozbas (2010) show that supply chain connections predict cross-sectional return differences: when a major customer's stock drops, its suppliers' stocks follow with a delay.

```
# Load supply chain data
supply_chain = dc.get_supply_chain_data(
    start_date="2020-01-01",
    end_date="2024-12-31"
)

# Build directed supply chain graph
G_supply = nx.DiGraph()

for _, row in supply_chain.iterrows():
    G_supply.add_edge(
        row["supplier_ticker"],
        row["customer_ticker"],
        weight=row.get("transaction_value", 1),
        pct_of_supplier_revenue=row.get("pct_supplier_revenue", 0),
        pct_of_customer_cogs=row.get("pct_customer_cogs", 0),
        year=row.get("year", 2024)
    )

print(f"Supply chain links: {G_supply.number_of_edges()}")
print(f"Firms involved: {G_supply.number_of_nodes()}")
```

106.4.2 Customer Momentum

The "customer momentum" strategy of L. Cohen and Frazzini (2008) exploits the delayed propagation of information through supply chains. When a customer firm's stock rises, its suppliers' stocks tend to follow in subsequent months:

$$r_{i,t+1} = \alpha + \beta \cdot \text{CustomerReturn}_{i,t} + \gamma \cdot r_{i,t} + \delta' \mathbf{X}_{i,t} + \varepsilon_{i,t} \tag{106.8}$$

where $\text{CustomerReturn}_{i,t} = \sum_{j \in \text{Customers}(i)} w_{ij} \cdot r_{j,t}$ is the weighted average return of firm i's customers.

```
# Compute customer-weighted returns for each supplier
def compute_customer_returns(supply_graph, monthly_returns):
    """
    Compute customer-weighted returns for each supplier firm.
    """
    # For each supplier, compute weighted average customer return
    results = []

    for date in monthly_returns["date"].unique():
        date_returns = monthly_returns[monthly_returns["date"] == date]
        ret_dict = dict(zip(date_returns["ticker"], date_returns["ret"]))

        for supplier in supply_graph.nodes():
            customers = list(supply_graph.successors(supplier))
            if not customers:
                continue

            customer_rets = []
            customer_weights = []
            for cust in customers:
                if cust in ret_dict:
                    edge_data = supply_graph[supplier][cust]
                    weight = edge_data.get("pct_of_supplier_revenue", 1)
                    customer_rets.append(ret_dict[cust])
                    customer_weights.append(weight)

            if customer_rets:
                weights = np.array(customer_weights)
                if weights.sum() > 0:
                    weights = weights / weights.sum()
                else:
```

```
                    weights = np.ones(len(weights)) / len(weights)

                weighted_ret = np.average(customer_rets, weights=weights)
                results.append({
                    "date": date,
                    "ticker": supplier,
                    "customer_ret": weighted_ret,
                    "n_customers": len(customer_rets)
                })

    return pd.DataFrame(results)


customer_rets = compute_customer_returns(G_supply, monthly_returns)

# Merge with own returns and lag
momentum_data = monthly_returns.merge(
    customer_rets, on=["ticker", "date"], how="inner"
)

momentum_data = momentum_data.sort_values(["ticker", "date"])
momentum_data["own_ret_lag"] =
 ↪  momentum_data.groupby("ticker")["ret"].shift(1)
momentum_data["customer_ret_lag"] = momentum_data.groupby(
    "ticker"
)["customer_ret"].shift(1)
momentum_data["ret_lead"] = momentum_data.groupby("ticker")["ret"].shift(-1)

# Regression
mom_clean = momentum_data.dropna(
    subset=["ret_lead", "customer_ret_lag", "own_ret_lag"]
)

model_mom = sm.OLS(
    mom_clean["ret_lead"],
    sm.add_constant(mom_clean[["customer_ret_lag", "own_ret_lag"]])
).fit(cov_type="cluster", cov_kwds={"groups": mom_clean["ticker"]})
```

Table 106.4: Customer Momentum: Supplier Returns Predicted by Customer Returns

```
mom_results = pd.DataFrame({
    "Coefficient": model_mom.params.round(4),
    "Std Error": model_mom.bse.round(4),
    "t-stat": model_mom.tvalues.round(3),
    "p-value": model_mom.pvalues.round(4)
})
mom_results
```

106.4.3 Network Propagation: Shock Diffusion Through Supply Chains

The Acemoglu, Ozdaglar, and Tahbaz-Salehi (2015) model predicts that the aggregate effect of an idiosyncratic shock depends on the network's topology. In a star network (one central hub), hub shocks generate aggregate fluctuations. In a symmetric network, shocks cancel out. We implement a simulation-based approach to measure shock propagation in the Vietnamese supply chain.

```
def simulate_shock_propagation(G, shocked_node, shock_size=-0.10,
                               decay=0.5, max_steps=5):
    """
    Simulate the propagation of an idiosyncratic shock through
    a supply chain network.

    Parameters
    ----------
    G : nx.DiGraph
        Supply chain graph (supplier → customer).
    shocked_node : str
        Node receiving the initial shock.
    shock_size : float
        Initial shock magnitude (e.g., -0.10 = -10% revenue shock).
    decay : float
        Fraction of shock transmitted per link (0 < decay < 1).
    max_steps : int
        Maximum propagation steps.

    Returns
    -------
    dict : {node: cumulative shock received}.
    """
```

```
    shocks = {shocked_node: shock_size}
    frontier = {shocked_node}

    for step in range(max_steps):
        new_frontier = set()
        for node in frontier:
            # Propagate to customers (downstream)
            for customer in G.successors(node):
                edge_weight = G[node][customer].get(
                    "pct_of_customer_cogs", 0.1
                )
                transmitted = shocks[node] * decay * edge_weight
                if abs(transmitted) > 0.001:
                    shocks[customer] = shocks.get(customer, 0) + transmitted
                    new_frontier.add(customer)

            # Propagate to suppliers (upstream)
            for supplier in G.predecessors(node):
                edge_weight = G[supplier][node].get(
                    "pct_of_supplier_revenue", 0.1
                )
                transmitted = shocks[node] * decay * edge_weight
                if abs(transmitted) > 0.001:
                    shocks[supplier] = shocks.get(supplier, 0) + transmitted
                    new_frontier.add(supplier)

        frontier = new_frontier
        if not frontier:
            break

    return shocks


# Identify systemically important supply chain nodes
# (Those whose shock has largest aggregate impact)
systemic_importance = {}
listed_in_supply = [n for n in G_supply.nodes()
                    if n in set(firms["ticker"])]

for firm in listed_in_supply[:100]:  # Sample for computational feasibility
    shocks = simulate_shock_propagation(G_supply, firm, -0.10)
    aggregate_impact = sum(abs(v) for k, v in shocks.items() if k != firm)
    systemic_importance[firm] = aggregate_impact
```

Table 106.5: Most Systemically Important Firms in the Supply Chain Network

```
top_systemic = systemic_df.head(20).merge(
    firms[["ticker", "industry", "market_cap"]],
    on="ticker", how="left"
)
top_systemic["market_cap_bn"] = top_systemic["market_cap"] / 1e9
top_systemic[["ticker", "industry", "market_cap_bn",
 ↪  "systemic_impact"]].round(4)
```

```
systemic_df = pd.DataFrame(
    list(systemic_importance.items()),
    columns=["ticker", "systemic_impact"]
).sort_values("systemic_impact", ascending=False)
```

106.5 Correlation and Co-Movement Networks

106.5.1 Construction from Return Data

A correlation network connects assets whose returns co-move. This is perhaps the most widely used financial network because it requires only return data—no proprietary ownership or supply chain information. The standard construction involves three steps:

1. **Compute pairwise correlations.** For n assets over T periods, the sample correlation matrix $\hat{\rho} \in \mathbb{R}^{n \times n}$ has $n(n-1)/2$ unique entries.
2. **Threshold or transform.** Convert the dense correlation matrix into a sparse graph. Common approaches: hard threshold ($\rho_{ij} > \bar{\rho}$), Minimum Spanning Tree (MST), or Planar Maximally Filtered Graph (PMFG).
3. **Analyze the graph.** Community detection reveals sector clustering; centrality identifies market bellwethers; temporal evolution tracks regime changes.

```
# Compute correlation matrix from daily returns
daily_returns = dc.get_daily_returns(
    start_date="2023-01-01",
    end_date="2024-06-30"
)
```

```
daily_wide = daily_returns.pivot(
    index="date", columns="ticker", values="ret"
).dropna(axis=1, thresh=200)

corr = daily_wide.corr()

# Minimum Spanning Tree (MST)
# Convert correlation to distance: d = sqrt(2(1- ))
dist = np.sqrt(2 * (1 - corr.values))
np.fill_diagonal(dist, 0)

# Build complete graph with distances
G_complete = nx.Graph()
tickers = list(corr.columns)
for i in range(len(tickers)):
    for j in range(i + 1, len(tickers)):
        G_complete.add_edge(
            tickers[i], tickers[j],
            weight=dist[i, j],
            correlation=corr.iloc[i, j]
        )

# MST
G_mst = nx.minimum_spanning_tree(G_complete, weight="weight")

# Thresholded correlation network
threshold = 0.5
G_corr = nx.Graph()
for i in range(len(tickers)):
    for j in range(i + 1, len(tickers)):
        if corr.iloc[i, j] > threshold:
            G_corr.add_edge(
                tickers[i], tickers[j],
                weight=corr.iloc[i, j]
            )

print(f"MST: {G_mst.number_of_nodes()} nodes, "
      f"{G_mst.number_of_edges()} edges")
print(f"Corr network ( > {threshold}): "
      f"{G_corr.number_of_nodes()} nodes, "
      f"{G_corr.number_of_edges()} edges")
```

106.5.2 The Diebold-Yilmaz Connectedness Framework

Diebold and Yılmaz (2014) propose measuring financial connectedness using the variance decomposition from a vector autoregression (VAR). The fraction of the H-step-ahead forecast error variance of variable i attributable to shocks from variable j defines the pairwise directional connectedness:

$$C_{i \leftarrow j}(H) = \frac{\tilde{\theta}_{ij}^{g}(H)}{\sum_{j=1}^{n} \tilde{\theta}_{ij}^{g}(H)} \times 100 \tag{106.9}$$

where $\tilde{\theta}_{ij}^{g}(H)$ is the generalized forecast error variance decomposition. The total connectedness index (TCI) aggregates across all pairs:

$$\text{TCI}(H) = \frac{\sum_{i \neq j} C_{i \leftarrow j}(H)}{\sum_{i,j} C_{i \leftarrow j}(H)} \times 100 \tag{106.10}$$

High TCI indicates a tightly connected system where shocks propagate widely; low TCI indicates relative isolation.

```
def diebold_yilmaz_connectedness(returns_df, var_order=2,
                                 forecast_horizon=10):
    """
    Compute the Diebold-Yilmaz (2014) connectedness table.

    Parameters
    ----------
    returns_df : DataFrame
        Returns with columns as assets, index as dates.
    var_order : int
        VAR lag order.
    forecast_horizon : int
        Forecast horizon for variance decomposition.

    Returns
    -------
    dict : Connectedness matrix, TCI, TO/FROM measures.
    """
    from statsmodels.tsa.api import VAR

    # Fit VAR
    model = VAR(returns_df.dropna())
```

```
    results = model.fit(var_order)

    # Generalized Forecast Error Variance Decomposition
    fevd = results.fevd(forecast_horizon)

    # Extract decomposition matrix at horizon H
    n = returns_df.shape[1]
    C = np.zeros((n, n))

    for i in range(n):
        decomp = fevd.decomp[i]  # H x n array
        C[i, :] = decomp[-1, :]  # Take horizon H values

    # Normalize rows to sum to 100
    row_sums = C.sum(axis=1, keepdims=True)
    C_norm = C / row_sums * 100

    # Connectedness measures
    # FROM: how much of i's variance comes from others
    FROM = C_norm.sum(axis=1) - np.diag(C_norm)

    # TO: how much of others' variance comes from i
    TO = C_norm.sum(axis=0) - np.diag(C_norm)

    # NET = TO - FROM
    NET = TO - FROM

    # Total Connectedness Index
    TCI = FROM.sum() / n

    names = returns_df.columns.tolist()

    connectedness_df = pd.DataFrame(C_norm, index=names, columns=names)
    connectedness_df["FROM_others"] = FROM
    connectedness_df.loc["TO_others"] = list(TO) + [TCI]

    return {
        "connectedness_matrix": connectedness_df,
        "TCI": TCI,
        "FROM": pd.Series(FROM, index=names),
        "TO": pd.Series(TO, index=names),
        "NET": pd.Series(NET, index=names)
    }
```

Table 106.6: Diebold-Yilmaz Net Connectedness: Net Transmitters (+) and Receivers (−)

```
net_df = pd.DataFrame({
    "TO_others": dy_results["TO"].round(2),
    "FROM_others": dy_results["FROM"].round(2),
    "NET": dy_results["NET"].round(2)
}).sort_values("NET", ascending=False)

net_df
```

```
# Apply to top Vietnamese stocks by liquidity
top_stocks = (
    daily_returns.groupby("ticker")["volume"]
    .mean()
    .nlargest(20)
    .index
)

top_returns = daily_wide[
    [c for c in top_stocks if c in daily_wide.columns]
].dropna()

dy_results = diebold_yilmaz_connectedness(top_returns)
print(f"Total Connectedness Index: {dy_results['TCI']:.2f}%")
```

```
# Rolling TCI to track systemic risk over time
def rolling_tci(returns_wide, window=252, step=21, var_order=1,
                forecast_horizon=10, min_stocks=10):
    """Compute rolling Total Connectedness Index."""
    dates = returns_wide.index
    tci_series = []

    for i in range(window, len(dates), step):
        window_data = returns_wide.iloc[i - window:i]

        # Keep stocks with sufficient data
        valid_cols = window_data.dropna(axis=1, thresh=int(window * 0.9))
        if valid_cols.shape[1] < min_stocks:
```

```
            continue

        # Use top stocks by variance (most informative)
        top_cols = valid_cols.var().nlargest(min_stocks).index
        subset = valid_cols[top_cols].dropna()

        if len(subset) < window * 0.8:
            continue

        try:
            result = diebold_yilmaz_connectedness(
                subset, var_order, forecast_horizon
            )
            tci_series.append({
                "date": dates[i],
                "TCI": result["TCI"],
                "n_stocks": len(top_cols)
            })
        except Exception:
            continue

    return pd.DataFrame(tci_series)


# tci_df = rolling_tci(daily_wide, window=252, step=21)
```

106.6 Interbank and Financial Contagion Networks

106.6.1 The Interbank Market as a Network

The interbank market, where banks lend reserves to each other, is the canonical financial network. F. Allen and Gale (2000) demonstrate that the structure of interbank linkages determines whether a bank failure cascades into a systemic crisis or is absorbed by the network. Two extreme topologies illustrate:

Complete network (all banks linked to all). Each bank's exposure to any single counterpart is small. A bank failure imposes small losses on many banks, which can individually absorb them. The network is resilient.

Ring network (each bank linked to one neighbor). Each bank's exposure is concentrated in a single counterpart. A failure in one bank can topple its neighbor, triggering a chain of cascading defaults. The network is fragile.

The Vietnamese banking system, with its concentration of state-owned banks and the regulatory emphasis on interbank lending for liquidity management, lies between these extremes.

```
# Load interbank exposure data
interbank = dc.get_interbank_exposures(
    date="2024-06-30"
)

# Build interbank network
G_bank = nx.DiGraph()

for _, row in interbank.iterrows():
    G_bank.add_edge(
        row["lender_bank"],
        row["borrower_bank"],
        weight=row["exposure_bn_vnd"],
        exposure_pct_equity=row.get("exposure_pct_equity", 0)
    )

# Add bank attributes
bank_info = dc.get_bank_characteristics(date="2024-06-30")
for _, row in bank_info.iterrows():
    if row["ticker"] in G_bank:
        G_bank.nodes[row["ticker"]].update({
            "total_assets": row.get("total_assets", 0),
            "equity": row.get("equity", 0),
            "is_soe": row.get("is_soe", False),
            "tier1_ratio": row.get("tier1_ratio", 0)
        })

bank_metrics, bank_graph_stats = compute_network_metrics(G_bank)
```

106.6.2 Cascading Default Simulation

We implement the Eisenberg and Noe (2001) clearing mechanism to simulate cascading defaults in the interbank network. When a bank fails, it cannot honor its interbank obligations, imposing losses on its creditors, who may in turn fail:

$$p_i^* = \min\left(\bar{p}_i,\; e_i + \sum_j \frac{L_{ji}}{\sum_k L_{jk}} p_j^*\right) \tag{106.11}$$

where p_i^* is bank i's actual payment, $\bar{p}_i$ is its total obligation, e_i is its external assets minus external liabilities, and $L_{ji} / \sum_k L_{jk}$ is the fraction of bank j's obligations owed to bank i.

```
def simulate_cascading_defaults(G, initial_failure,
                                equity_buffer=0.08,
                                max_rounds=20):
    """
    Simulate cascading defaults in an interbank network.

    Parameters
    ----------
    G : nx.DiGraph
        Interbank exposure graph (lender → borrower, weight = exposure).
    initial_failure : str
        Bank that fails initially.
    equity_buffer : float
        Fraction of equity that absorbs losses before default.
    max_rounds : int
        Maximum cascade rounds.

    Returns
    -------
    dict : Defaulted banks, round-by-round losses, systemic loss.
    """
    defaulted = {initial_failure}
    cascade_history = [{"round": 0, "defaults": [initial_failure],
                        "total_defaults": 1}]

    for round_num in range(1, max_rounds + 1):
        new_defaults = set()

        for bank in G.nodes():
            if bank in defaulted:
                continue

            # Compute losses from defaulted counterparties
            total_loss = 0
            for defaulted_bank in defaulted:
                if G.has_edge(bank, defaulted_bank):
                    # bank lent to defaulted_bank → loss
                    exposure = G[bank][defaulted_bank]["weight"]
                    # Assume LGD = 60%
                    loss = exposure * 0.6
```

```
                    total_loss += loss

            # Check if losses exceed equity buffer
            bank_data = G.nodes.get(bank, {})
            equity = bank_data.get("equity", float("inf"))

            if total_loss > equity * equity_buffer:
                new_defaults.add(bank)

        if not new_defaults:
            break

        defaulted |= new_defaults
        cascade_history.append({
            "round": round_num,
            "defaults": list(new_defaults),
            "total_defaults": len(defaulted)
        })

    # Compute systemic loss
    total_system_equity = sum(
        G.nodes[n].get("equity", 0) for n in G.nodes()
    )
    defaulted_equity = sum(
        G.nodes[n].get("equity", 0) for n in defaulted
    )
    systemic_loss_pct = (
        defaulted_equity / total_system_equity
        if total_system_equity > 0 else 0
    )

    return {
        "defaulted_banks": list(defaulted),
        "n_defaults": len(defaulted),
        "cascade_rounds": len(cascade_history) - 1,
        "cascade_history": cascade_history,
        "systemic_loss_pct": systemic_loss_pct
    }


# Simulate cascade for each bank
cascade_results = []
for bank in G_bank.nodes():
```

Table 106.7: Systemically Important Banks: Cascading Default Analysis

```
cascade_df.head(15).round(4)
```

```
    result = simulate_cascading_defaults(G_bank, bank)
    cascade_results.append({
        "initial_failure": bank,
        "n_cascading_defaults": result["n_defaults"],
        "systemic_loss_pct": result["systemic_loss_pct"],
        "cascade_rounds": result["cascade_rounds"]
    })

cascade_df = pd.DataFrame(cascade_results).sort_values(
    "systemic_loss_pct", ascending=False
)
```

106.7 Social Connectedness Between Places

Every network in this chapter so far connects firms or financial institutions. A different network connects people, and through them places. It matters for finance because households learn about financial products from people they know, and because lenders and investors learn about distant borrowers and firms partly through the same ties. Until recently that network could not be measured at scale. The **Social Connectedness Index** (SCI) of Bailey, Cao, Kuchler, Stroebel, and Wong (2018) changed this. Built from the universe of friendship links among Facebook users, it measures, for every pair of regions i and j, the relative probability that a user in i and a user in j are friends:

$$\mathrm{SCI}_{ij} = \frac{\mathrm{Connections}_{ij}}{\mathrm{Users}_i \times \mathrm{Users}_j}. \tag{106.12}$$

Connectedness falls steeply with geographic distance, but it also reflects migration history, transport links, and political and cultural boundaries (Bailey, Cao, Kuchler, Stroebel, and Wong 2018). For a given region, the SCI defines a set of weights over all other regions. The most useful are friend shares, the fraction of region i's friendship links that lead to region j,

$$w_{ij} = \frac{\mathrm{SCI}_{ij} \times \mathrm{Users}_j}{\sum_k \mathrm{SCI}_{ik} \times \mathrm{Users}_k}, \tag{106.13}$$

which turn any regional variable X_j into a **social exposure**:

$$\text{SocialExp}_i(X) = \sum_{j \in \mathcal{N}(i)} w_{ij} X_j, \tag{106.14}$$

where $\mathcal{N}(i)$ is the set of regions over which exposure is taken: all regions, or only those geographically distant from i. Two templates built on Equation 106.14 anchor the social-finance literature surveyed by Kuchler and Stroebel (2021). In Bailey, Cao, Kuchler, and Stroebel (2018), X_j is house-price growth in friends' regions: individuals whose friends experienced larger house-price gains expect larger future gains and buy bigger houses at higher prices. In Kuchler et al. (2022), X_j is the institutional capital located in region j: firms headquartered in regions with high *social proximity to capital* have higher institutional ownership, valuations, and liquidity, with the effects concentrated among small firms that analysts do not cover. Earlier work established the individual-level version of the idea, from stock-market participation spreading through social interaction (Hong, Kubik, and Stein 2004) to lenders on a peer-to-peer platform treating borrowers' friendship networks as a signal of credit quality (Lin, Prabhala, and Viswanathan 2013).

106.7.1 Social Networks as Conduits for Credit

L. Allen, Peng, and Shan (2026) bring this machinery to consumer credit and, unusually, to both sides of the market at once. Their setting is LendingClub, the largest US marketplace lender, whose public data cover rejected as well as approved applications, so that funding *decisions* can be studied rather than only the loans that were made. The earlier version of the paper used applications from 2010 to 2017, located borrowers by three-digit ZIP code, and mapped those areas to county-level SCI.

Demand. Applications for online loans in an area rise after online borrowing has risen in areas that are geographically distant but socially connected to it. The geographic restriction carries the identification. Nearby friends share local economic conditions, so a correlation between an area's borrowing and its friends' borrowing could reflect a common shock rather than learning. Restricting $\mathcal{N}(i)$ in Equation 106.14 to distant regions (out of state, in the earlier draft, following the design of Bailey, Cao, Kuchler, and Stroebel (2018)) strips out that common component. What remains is consistent with awareness of a new financial product spreading through social ties: people hear about a platform from friends who have used it.

Supply. Applications from areas with high social proximity to deposit-rich regions, a measure in the spirit of Kuchler et al. (2022) with deposits in place of institutional capital, are more likely to be funded. On its own, this result could reflect either of two channels that the paper sets against each other. Social ties may transmit hard-to-obtain *information* about a community's credit conditions to those in a position to fund it. Or they may transmit *affinity*: familiarity, and the repetition of opinions through a network that produces persuasion bias, leading funders to favor connected borrowers regardless of quality. Both channels predict that

connected borrowers obtain more credit. They disagree about how the marginal loan performs. Affinity lowers the bar, so the additional loans should default more often; information sharpens screening, so they should default less. Ex post performance is better, which points to information. Lin, Prabhala, and Viswanathan (2013) reach a parallel conclusion from individual borrowers' friendship links on Prosper, and when Duarte, Siegel, and Young (2012) apply the same logic to borrower appearance, they find that the trustworthy-looking borrowers whom lenders favor do in fact default less often.

Identification. Social connectedness is not randomly assigned. Areas whose residents have geographically dispersed networks tend to be richer, more educated, and more mobile (Bailey, Cao, Kuchler, Stroebel, and Wong 2018), and each of those characteristics affects credit outcomes directly. The published paper therefore establishes causality with **shift-share instruments**. The shares are the social-network weights w_{ij}, and the shifts are events in the connected regions that move credit demand or supply there without reflecting conditions in the borrower's own area: natural disasters on the demand side and financial adviser misconduct on the supply side. Adviser misconduct is unusually well measured in the United States because broker disciplinary records are public, the data that Egan, Matvos, and Seru (2019) exploit.

Distribution. Both the demand and the supply effects are strongest in disadvantaged communities, and they come without higher delinquency. Social ties to more prosperous regions appear to substitute, in part, for the local information infrastructure (bank branches, long credit histories, relationship lenders) that disadvantaged areas lack.

The paper repays close reading by anyone building network measures, for four reasons. It treats a network between places as an observable object, measurable on a par with ownership or supply-chain links. It studies both sides of one market through the same network, which few network papers manage. It uses ex post performance as the test that separates information from bias, a test that any claim that networks "improve" allocation should be required to pass. And it makes explicit that a social-exposure variable is a shift-share construct, which determines what can identify it.

106.7.2 Shift-Share Logic and Inference

Equation 106.14 is a share-weighted sum of regional shifts, the same structure as the Bartik instruments of labor economics, and the econometric literature on these designs is precise about when they identify a causal effect. Identification can come from exogenous shares (Goldsmith-Pinkham, Sorkin, and Swift 2020). Here that means an area's pattern of social ties is unrelated to its credit outcomes except through the shocks the ties transmit, a strong assumption when those patterns reflect migration, ethnicity, and industry structure. Alternatively, identification can come from shifts that are as good as randomly assigned across regions, conditional on the shares (Borusyak, Hull, and Jaravel 2022). Natural disasters are attractive shifts precisely because the second route is plausible for them. Under either route, inference requires

care. Regions with similar exposure weights receive correlated shocks, so conventional and region-clustered standard errors overstate precision. Adão, Kolesár, and Morales (2019) derive standard errors that are robust to this exposure-induced correlation. A social-exposure regression reported without them, and without balance tests of the shifts against pre-period regional characteristics, has not yet made its case.

106.7.3 Social Connectedness in Vietnam

The data exist. Meta's public SCI release covers countries outside Europe at the first-level administrative unit, which for Vietnam means the 63 provinces on the boundary vintage used in the release. Facebook use is widespread in Vietnam, but much private communication runs through Zalo, which the SCI does not observe. Province-pair connectedness from Facebook therefore measures one channel of social contact, not all of them.

Two features of the setting need attention before any regression is run. First, the 2025 consolidation of Vietnam's provinces from 63 to 34 means that current provincial statistics must be mapped back to the old boundaries, or the SCI aggregated forward with population weights, and neither mapping is innocuous for provinces that were divided among new units. Second, internal migration is the main determinant of connectedness, and the census records it directly. The 2009 and 2019 Population and Housing Censuses record each respondent's province of residence five years earlier, which yields province-to-province migration matrices. An exposure measure weighted by migration is both a validation check on the SCI and, because it predates most outcomes of interest, a candidate instrument for it.

Four research designs transfer most directly.

Social proximity to capital for listed firms. Investable capital and brokerage activity in Vietnam are concentrated in Ho Chi Minh City and Hanoi. The Kuchler et al. (2022) measure, computed for each firm's headquarters province with provincial deposits or income standing in for institutional capital, asks whether firms headquartered in provinces socially close to those centers enjoy higher liquidity, valuations, and foreign ownership. The headquarters province appears in every listing prospectus, and the outcome variables are standard in this book.

Diffusion of retail participation. The surge in new brokerage accounts during 2020–2022 is a demand-side event of exactly the kind L. Allen, Peng, and Shan (2026) study: awareness of a financial product spreading through social ties. The design asks whether account openings in a province accelerated after openings rose in distant but socially connected provinces. The binding constraint is data. Account statistics are published nationally, and a provincial breakdown would require cooperation from brokerages.

Marketplace lending under the sandbox. Decree 94/2025/ND-CP, effective July 2025, created a controlled-testing mechanism for peer-to-peer lending alongside credit scoring and open APIs. A platform market in its first years is the setting of the L. Allen, Peng, and Shan (2026) demand result, and if data on sandbox participants become available to researchers, the

platforms' spread across provinces is close to a replication opportunity. Vietnam is also where the supply-side result may matter most. As discussed in Section 2.7, credit-bureau coverage remains thin, and L. Allen, Peng, and Shan (2026) suggest that social ties to wealthier regions can partly stand in for missing information infrastructure.

Shocks for shift-share designs. On the demand side, typhoons and floods strike identifiable provinces at well-documented dates, Typhoon Yagi in the northern provinces in September 2024 among them, and these are natural shifts. On the supply side, the closest Vietnamese counterpart to US adviser misconduct is the sale of corporate bonds to bank depositors through branch networks that came to light in the Van Thinh Phat and SCB case in 2022, a shock to trust in bank-intermediated saving. Using it as a shift requires knowing where the affected bondholders lived, which is not public, and the design should not be attempted without that information.

Six practical rules follow for province-level social-network research in Vietnam.

1. **Define "distant" by region, not by adjacency.** Vietnam's long, narrow shape means that neighboring provinces can be economically far apart and distant ones closely integrated. Measure exposure over provinces outside the borrower's own socio-economic region, the analogue of the out-of-state restriction, and show that results survive alternative cutoffs.
2. **Fix the boundary vintage.** Work with the 63 pre-2025 provinces throughout, and document the crosswalk for any data reported on the 34-province system.
3. **Treat the SCI as a snapshot of an endogenous network.** It is observed at a few dates, after much of the migration that created it. For outcomes that predate the snapshot, use the 2009 census migration matrix as the earlier-vintage measure.
4. **Report shift-share diagnostics.** Show which shares drive the estimate (Goldsmith-Pinkham, Sorkin, and Swift 2020), test the shifts for balance against pre-period provincial characteristics (Borusyak, Hull, and Jaravel 2022), and use exposure-robust standard errors (Adão, Kolesár, and Morales 2019).
5. **Run the performance test.** Any finding that social ties raise approval, funding, or investment must be followed by a test on ex post outcomes, defaults for credit and subsequent returns for equity, that distinguishes information from affinity, as L. Allen, Peng, and Shan (2026) do.
6. **Respect the small number of units.** With 63 provinces, clustered inference rests on few clusters. Report wild-cluster bootstrap p-values alongside conventional ones.

106.8 Graph Neural Networks for Financial Prediction

106.8.1 From Graphs to Predictions

Classical network analysis computes hand-crafted features (centrality, clustering, community membership) and feeds them into standard regression models. Graph neural networks (GNNs)

learn features directly from the graph structure and node attributes through message passing. The key insight is that a node's representation should depend not only on its own features but on the features of its neighbors, their neighbors, and so on.

106.8.2 Graph Convolutional Networks (GCN)

The Graph Convolutional Network of Kipf and Welling (2016) performs spectral convolution on graphs. The layer-wise propagation rule is:

$$H^{(\ell+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(\ell)}W^{(\ell)}\right) \tag{106.15}$$

where $\tilde{A} = A + I_n$ is the adjacency matrix with self-loops, $\tilde{D}$ is the corresponding degree matrix, $H^{(\ell)}$ is the node feature matrix at layer ℓ, $W^{(\ell)}$ is the learnable weight matrix, and σ is a nonlinearity (ReLU). The normalized $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ term averages each node's features with its neighbors' features, weighted by degree.

106.8.3 Graph Attention Networks (GAT)

The Graph Attention Network of Veličković et al. (2017) replaces the fixed normalization in GCN with learned attention coefficients:

$$h_i^{(\ell+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(\ell)} W^{(\ell)} h_j^{(\ell)}\right) \tag{106.16}$$

where the attention coefficient α_{ij} is computed as:

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top [Wh_i \| Wh_j]\right)\right)}{\sum_{k\in\mathcal{N}(i)} \exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top [Wh_i \| Wh_k]\right)\right)} \tag{106.17}$$

This allows the model to learn which neighbors are most informative for each node, rather than treating all neighbors equally.

```
class GCNLayer(nn.Module):
    """Graph Convolutional Network layer (Kipf & Welling, 2016)."""

    def __init__(self, in_features, out_features, bias=True):
        super().__init__()
        self.weight = nn.Parameter(torch.FloatTensor(in_features,
         ↪  out_features))
```

```
        if bias:
            self.bias = nn.Parameter(torch.FloatTensor(out_features))
        else:
            self.bias = None
        self.reset_parameters()

    def reset_parameters(self):
        nn.init.xavier_uniform_(self.weight)
        if self.bias is not None:
            nn.init.zeros_(self.bias)

    def forward(self, x, adj_norm):
        """
        Parameters
        ----------
        x : Tensor (n_nodes, in_features)
            Node feature matrix.
        adj_norm : Tensor (n_nodes, n_nodes)
            Normalized adjacency matrix (D^{-1/2} A D^{-1/2}).
        """
        support = x @ self.weight
        output = adj_norm @ support
        if self.bias is not None:
            output = output + self.bias
        return output


class GATLayer(nn.Module):
    """Graph Attention Network layer (Velickovic et al., 2017)."""

    def __init__(self, in_features, out_features, n_heads=4,
                 dropout=0.1, concat=True):
        super().__init__()
        self.n_heads = n_heads
        self.out_features = out_features
        self.concat = concat

        self.W = nn.Parameter(
            torch.FloatTensor(n_heads, in_features, out_features)
        )
        self.a = nn.Parameter(torch.FloatTensor(n_heads, 2 * out_features,
         ↪  1))
        self.dropout = nn.Dropout(dropout)
```

```
        self.leaky_relu = nn.LeakyReLU(0.2)
        self.reset_parameters()

    def reset_parameters(self):
        nn.init.xavier_uniform_(self.W)
        nn.init.xavier_uniform_(self.a)

    def forward(self, x, adj):
        """
        Parameters
        ----------
        x : Tensor (n_nodes, in_features)
        adj : Tensor (n_nodes, n_nodes)
            Binary adjacency matrix (1 if edge, 0 otherwise).
        """
        n = x.size(0)

        # Linear transformation for each head
        # x: (n, in_f) -> h: (heads, n, out_f)
        h = torch.einsum("ni,hio->hno", x, self.W)

        # Attention coefficients
        # Concatenate h_i and h_j for all pairs
        h_i = h.unsqueeze(2).expand(-1, -1, n, -1)  # (heads, n, n, out_f)
        h_j = h.unsqueeze(1).expand(-1, n, -1, -1)  # (heads, n, n, out_f)
        concat_h = torch.cat([h_i, h_j], dim=-1)    # (heads, n, n, 2*out_f)

        e = self.leaky_relu(
            torch.einsum("hnmo,hoi->hnm", concat_h, self.a).squeeze(-1)
        )

        # Mask non-edges
        mask = adj.unsqueeze(0).expand(self.n_heads, -1, -1)
        e = e.masked_fill(mask == 0, float("-inf"))

        # Softmax attention
        alpha = F.softmax(e, dim=-1)
        alpha = self.dropout(alpha)

        # Weighted aggregation
        out = torch.einsum("hnm,hmo->hno", alpha, h)  # (heads, n, out_f)

        if self.concat:
```

```
            out = out.permute(1, 0, 2).reshape(n, -1)  # (n, heads * out_f)
        else:
            out = out.mean(dim=0)  # (n, out_f)

        return out, alpha


class FinancialGNN(nn.Module):
    """
    Graph Neural Network for financial node prediction.
    Supports GCN and GAT layers with flexible architecture.
    """

    def __init__(self, input_dim, hidden_dim=64, output_dim=1,
                 n_layers=2, n_heads=4, gnn_type="gat",
                 dropout=0.3):
        super().__init__()

        self.gnn_type = gnn_type
        self.dropout = nn.Dropout(dropout)

        if gnn_type == "gcn":
            self.layers = nn.ModuleList()
            self.layers.append(GCNLayer(input_dim, hidden_dim))
            for _ in range(n_layers - 1):
                self.layers.append(GCNLayer(hidden_dim, hidden_dim))

        elif gnn_type == "gat":
            self.layers = nn.ModuleList()
            self.layers.append(
                GATLayer(input_dim, hidden_dim // n_heads,
                         n_heads=n_heads, concat=True)
            )
            for _ in range(n_layers - 2):
                self.layers.append(
                    GATLayer(hidden_dim, hidden_dim // n_heads,
                             n_heads=n_heads, concat=True)
                )
            # Last layer: single head, no concat
            self.layers.append(
                GATLayer(hidden_dim, hidden_dim,
                         n_heads=1, concat=False)
            )
```

```
        # Prediction head
        self.head = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim // 2, output_dim)
        )

    def forward(self, x, adj):
        """
        Parameters
        ----------
        x : Tensor (n_nodes, input_dim)
            Node features.
        adj : Tensor (n_nodes, n_nodes)
            Adjacency matrix.
        """
        h = x
        attention_weights = []

        for i, layer in enumerate(self.layers):
            if self.gnn_type == "gcn":
                h = layer(h, adj)
                h = F.relu(h) if i < len(self.layers) - 1 else h
            elif self.gnn_type == "gat":
                h, alpha = layer(h, adj)
                attention_weights.append(alpha)
                if i < len(self.layers) - 1:
                    h = F.elu(h)

            h = self.dropout(h)

        predictions = self.head(h).squeeze(-1)
        return predictions, attention_weights
```

106.8.4 GNN for Cross-Sectional Return Prediction

We apply the GNN to predict cross-sectional stock returns, where the graph encodes ownership connections between firms. The hypothesis is that firms connected through ownership have correlated return dynamics that the GNN can exploit.

```
def prepare_gnn_data(firms_df, ownership_graph, returns_df, date):
    """
    Prepare node features and adjacency matrix for GNN prediction.

    Parameters
    ----------
    firms_df : DataFrame
        Firm characteristics.
    ownership_graph : nx.DiGraph
        Ownership network.
    returns_df : DataFrame
        Stock returns.

    Returns
    -------
    tuple : (node_features, adjacency, target_returns, ticker_list)
    """
    # Get stocks with both features and network presence
    tickers = sorted(
        set(firms_df["ticker"]) &
        set(ownership_graph.nodes()) &
        set(returns_df["ticker"])
    )

    if not tickers:
        return None, None, None, None

    # Node features
    feature_cols = [
        "log_size", "book_to_market", "momentum_12m",
        "profitability", "investment", "leverage",
        "beta", "volatility", "turnover"
    ]

    feat_df = firms_df[firms_df["ticker"].isin(tickers)].set_index("ticker")
    feat_df = feat_df.reindex(tickers)

    X = feat_df[feature_cols].fillna(0).values
    X = (X - X.mean(axis=0)) / (X.std(axis=0) + 1e-8)

    # Adjacency matrix
    ticker_to_idx = {t: i for i, t in enumerate(tickers)}
    n = len(tickers)
```

```
    A = np.zeros((n, n))

    for u, v, d in ownership_graph.edges(data=True):
        if u in ticker_to_idx and v in ticker_to_idx:
            weight = d.get("weight", 1)
            A[ticker_to_idx[u], ticker_to_idx[v]] = weight
            A[ticker_to_idx[v], ticker_to_idx[u]] = weight  # Symmetrize

    # Add self-loops
    A = A + np.eye(n)

    # Normalize: D^{-1/2} A D^{-1/2}
    D = np.diag(A.sum(axis=1))
    D_inv_sqrt = np.diag(1.0 / np.sqrt(np.diag(D) + 1e-10))
    A_norm = D_inv_sqrt @ A @ D_inv_sqrt

    # Target returns
    ret_df = returns_df[
        (returns_df["ticker"].isin(tickers)) &
        (returns_df["date"] == date)
    ].set_index("ticker").reindex(tickers)

    y = ret_df["ret"].fillna(0).values

    return (
        torch.tensor(X, dtype=torch.float32),
        torch.tensor(A_norm, dtype=torch.float32),
        torch.tensor(y, dtype=torch.float32),
        tickers
    )


def train_gnn_model(model, X_train, A_train, y_train,
                    X_val, A_val, y_val,
                    n_epochs=100, lr=1e-3):
    """Train GNN model with validation-based early stopping."""
    optimizer = torch.optim.Adam(model.parameters(), lr=lr,
                                 weight_decay=1e-4)

    best_val_loss = float("inf")
    patience_counter = 0

    for epoch in range(n_epochs):
```

```
        model.train()
        optimizer.zero_grad()

        pred, _ = model(X_train, A_train)
        loss = F.mse_loss(pred, y_train)
        loss.backward()
        optimizer.step()

        # Validation
        model.eval()
        with torch.no_grad():
            val_pred, _ = model(X_val, A_val)
            val_loss = F.mse_loss(val_pred, y_val).item()

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_state = {k: v.clone() for k, v in
↪   model.state_dict().items()}
            patience_counter = 0
        else:
            patience_counter += 1
            if patience_counter >= 15:
                break

    model.load_state_dict(best_state)
    return model, best_val_loss
```

106.8.5 Temporal Graph Networks

Financial networks evolve over time. Ownership stakes change, directors rotate, correlations shift. Temporal GNNs extend static GNNs by incorporating the time dimension explicitly. The Temporal Graph Network (TGN) of E. Rossi et al. (2020) maintains a memory state for each node that is updated whenever an event involving the node occurs:

$$\mathbf{s}_i(t) = \text{GRU}\left(\mathbf{s}_i(t^-),\ \text{msg}(i, t)\right) \tag{106.18}$$

where $\mathbf{s}_i(t)$ is the memory state of node i at time t, $\text{msg}(i, t)$ is the message aggregated from events at time t, and GRU is a gated recurrent unit.

```
class TemporalFinancialGNN(nn.Module):
    """
    Temporal GNN for dynamic financial networks.
    Combines graph convolution with temporal memory.
    """

    def __init__(self, node_dim, edge_dim=1, memory_dim=64,
                 hidden_dim=64, output_dim=1, n_heads=4):
        super().__init__()

        self.memory_dim = memory_dim

        # Memory update (GRU)
        self.memory_updater = nn.GRUCell(
            input_size=node_dim + edge_dim + memory_dim,
            hidden_size=memory_dim
        )

        # Message function
        self.message_fn = nn.Sequential(
            nn.Linear(memory_dim * 2 + edge_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, memory_dim)
        )

        # Graph attention for spatial aggregation
        self.spatial_attn = GATLayer(
            memory_dim + node_dim, hidden_dim // n_heads,
            n_heads=n_heads, concat=True
        )

        # Temporal attention for recent history
        self.temporal_attn = nn.MultiheadAttention(
            embed_dim=hidden_dim, num_heads=n_heads,
            batch_first=True
        )

        # Prediction head
        self.head = nn.Sequential(
            nn.Linear(hidden_dim * 2, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim, output_dim)
```

```
        )

    def forward(self, node_features, adj_sequence, memory=None):
        """
        Parameters
        ----------
        node_features : list of Tensors
            Node features at each time step.
        adj_sequence : list of Tensors
            Adjacency matrices at each time step.
        memory : Tensor or None
            Initial memory state (n_nodes, memory_dim).

        Returns
        -------
        predictions : Tensor
            Predictions for the last time step.
        """
        n_nodes = node_features[0].shape[0]
        T = len(node_features)

        if memory is None:
            memory = torch.zeros(n_nodes, self.memory_dim)

        temporal_states = []

        for t in range(T):
            x_t = node_features[t]
            adj_t = adj_sequence[t]

            # Spatial aggregation via GAT
            combined = torch.cat([x_t, memory], dim=-1)
            spatial_out, _ = self.spatial_attn(combined, adj_t)

            temporal_states.append(spatial_out.unsqueeze(1))

            # Update memory with current state
            update_input = torch.cat([
                x_t, spatial_out[:, :1].squeeze(1) if spatial_out.dim() > 2
                else spatial_out,
                memory
            ], dim=-1)[:, :self.memory_updater.input_size]
```

```
        # Stack temporal states: (n_nodes, T, hidden)
        temporal_stack = torch.cat(temporal_states, dim=1)

        # Temporal attention across time steps
        temporal_out, _ = self.temporal_attn(
            temporal_stack, temporal_stack, temporal_stack
        )

        # Use last time step
        last_spatial = temporal_states[-1].squeeze(1)
        last_temporal = temporal_out[:, -1, :]

        combined = torch.cat([last_spatial, last_temporal], dim=-1)
        predictions = self.head(combined).squeeze(-1)

        return predictions
```

106.8.6 GNN for Credit Risk and Fraud Detection

Beyond return prediction, GNNs are particularly powerful for credit risk assessment and fraud detection, where the network structure itself is informative. In credit risk, firms connected to defaulted counterparties face elevated risk. In fraud detection, suspicious transaction patterns propagate through networks of related entities.

```
class CreditRiskGNN(nn.Module):
    """
    GNN for credit default prediction using firm and bank networks.
    Node classification: predict default probability per firm.
    """

    def __init__(self, firm_features_dim, bank_features_dim=0,
                 hidden_dim=64, n_heads=4):
        super().__init__()

        input_dim = firm_features_dim + bank_features_dim

        # GNN backbone
        self.gnn = FinancialGNN(
            input_dim=input_dim,
            hidden_dim=hidden_dim,
            output_dim=1,
```

```
            n_layers=3,
            n_heads=n_heads,
            gnn_type="gat"
        )

        # Additional head for probability output
        self.sigmoid = nn.Sigmoid()

    def forward(self, x, adj):
        logits, attn = self.gnn(x, adj)
        probs = self.sigmoid(logits)
        return probs, attn
```

106.9 When to Use Network Methods

106.9.1 Decision Framework

Table 106.8: Decision Framework for Network Methods

Question	Best Approach	Example
Does firm A affect firm B?	Pairwise test + network controls	Supply chain momentum
Which firms are most systemically important?	Centrality measures	Interbank contagion
Do network-connected firms co-move?	Correlation vs. interlock overlap	Board interlock diffusion
Can network structure predict returns?	GNN or network-augmented regression	GCN return prediction
How does a shock propagate?	Cascade simulation	Eisenberg-Noe default model
What is the optimal portfolio given network risk?	Network-regularized optimization	Correlation MST filtering
How does the network change over time?	Temporal GNN or rolling analysis	DY rolling connectedness
Does information travel between regions through social ties?	Social exposure over distant regions + shift-share IV	Online credit diffusion (L. Allen, Peng, and Shan 2026)

106.9.2 Data Requirements and Vietnamese Availability

Table 106.9: Financial Network Data Sources in Vietnam

Network Type	Required Data	Vietnamese Availability	Update Frequency
Ownership	Shareholder registers	Good (semi-annual filings)	Semi-annual
Board interlocks	Director appointments	Good (filings)	Annual
Supply chain	Customer-supplier disclosures, trade data	Moderate (related-party disclosures)	Annual
Correlation	Return data	Excellent (daily from exchanges)	Daily
Interbank	Bilateral exposures	Limited (SBV data, banks' notes to FS)	Quarterly
Co-holdings	Institutional holdings	Moderate (semi-annual disclosures)	Semi-annual
Lending	Loan-level data	Limited (banking supervision data)	Quarterly
Social connectedness	Region-pair friendship links; migration matrices	Moderate (Meta SCI for 63 provinces; 2009 and 2019 census migration)	Periodic snapshots

106.10 Summary

This chapter developed the network toolkit for Vietnamese financial markets, spanning classical graph theory, domain-specific financial network construction, and modern graph neural networks.

The central insight is that financial networks encode information invisible to standard panel regressions. Ownership networks reveal pyramidal control structures and tunneling risk that explain cross-sectional differences in firm value. Board interlocks serve as information channels through which compensation practices, investment policies, and governance norms diffuse. Supply chain networks propagate real economic shocks: the customer momentum anomaly demonstrates that market prices incorporate supply chain information with a predictable delay. Correlation networks capture co-movement structure that both reveals sector clustering and evolves across market regimes. Interbank networks determine whether individual bank failures

cascade into systemic crises. And social networks between places carry information that formal channels miss: L. Allen, Peng, and Shan (2026) show that intercommunity ties expand both the demand for and the supply of online credit, most of all in disadvantaged communities, without raising delinquency, which makes the Social Connectedness Index a measurement tool of particular value in a market where formal credit information is thin.

Vietnamese markets are particularly network-dense because of the pervasive role of state ownership, the prominence of business groups, and the concentration of the banking system. The control-cash flow wedge (i.e., the gap between control rights and cash flow rights created by pyramidal ownership) is larger and more variable than in markets with stronger minority shareholder protections. This wedge, which is computable only through network analysis of ownership chains, is both a governance risk measure and a predictor of firm value.

Graph neural networks extend the toolkit from hand-crafted network features to learned representations. The GCN and GAT architectures aggregate information across network neighborhoods, and the temporal GNN captures the evolution of network structure over time. For return prediction, the empirical question is whether the GNN's ability to learn flexible functions of the graph structure outperforms the simpler approach of computing centrality measures and feeding them into standard models. The exercises provide a framework for answering this question rigorously.

The network perspective is not a replacement for standard asset pricing or corporate finance methods; it is a complement. The most productive research designs combine network structure with identification strategies from the causal inference toolkit: using network shocks as instruments, exploiting network disruptions as natural experiments, and controlling for network position when estimating treatment effects. The intersection of networks and causal inference (e.g., network interference, peer effects identification, spatial econometrics) represents the frontier of empirical finance.

107 Multimodal Models in Finance

The preceding chapters treated text and images as isolated data modalities, but financial decision-making is inherently multimodal. An analyst evaluating a Vietnamese real estate developer simultaneously reads the annual report (text), inspects satellite imagery of construction sites (image), reviews quarterly financial statements (tabular), monitors the stock's price and volume dynamics (time series), and perhaps listens to the earnings call (audio). No single modality captures the full information set. The question this chapter addresses is: can we build models that fuse multiple modalities in a principled way, and does the fusion yield economically meaningful improvements over the best single-modality model?

The answer from the recent machine learning literature is increasingly yes, but with important caveats. Multimodal models can exploit complementarities between modalities (e.g., text describes intentions and context; images reveal physical states; tabular data provides precise quantitative snapshots; time series captures dynamics). However, the gains are not automatic. Naive concatenation of heterogeneous features often degrades performance relative to the best unimodal model, a phenomenon known as the "modality laziness" problem (Yu Huang et al. 2021). Effective fusion requires architectures that align representations across modalities, handle missing modalities gracefully (not every firm-quarter has satellite imagery and an earnings call), and avoid the dominant modality drowning out weaker but complementary signals.

This chapter develops the multimodal toolkit for Vietnamese financial markets across four progressively complex architectures. We begin with representation alignment (i.e., how to map different modalities into a shared embedding space). We then implement early, late, and cross-attention fusion for return prediction. We build a multimodal document understanding system that jointly processes the text, tables, and images within Vietnamese annual reports. We construct a multimodal earnings surprise model that combines pre-announcement text, satellite imagery, and financial time series. And we address the practical engineering challenges, including missing modalities, computational cost, and evaluation protocols that determine whether multimodal models work in production.

```
import pandas as pd
import numpy as np
from pathlib import Path
import warnings
warnings.filterwarnings("ignore")

# Deep learning
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
import torchvision.models as models

# NLP
from transformers import (
    AutoTokenizer, AutoModel,
    CLIPProcessor, CLIPModel
)

# Tabular and statistical
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import r2_score, mean_squared_error
from scipy import stats
import statsmodels.api as sm
from linearmodels.panel import PanelOLS

# Visualization
import plotnine as p9
from mizani.formatters import percent_format
```

107.1 Foundations of Multimodal Learning

107.1.1 The Information Structure of Financial Data

Financial data is naturally organized into modalities with distinct statistical properties, temporal frequencies, and information content. Table 107.1 summarizes the modalities relevant to Vietnamese equity markets.

Table 107.1: Modality Landscape in Financial Data

Modality	Examples	Dimensionality	Frequency	Encoding
Tabular	Financial ratios, ownership, governance	Low ($\sim$ 50 features)	Quarterly/Annual	Structured numeric

Modality	Examples	Dimensionality	Frequency	Encoding
Text	Annual reports, news, filings, social media	High ($\sim$ 10k tokens)	Event-driven	Sequential tokens
Image	Satellite tiles, document scans, news photos	Very high ($\sim$ 150k pixels)	Daily to monthly	Spatial grid
Time series	Price, volume, order flow, volatility	Moderate ($\sim$ 250 days $\times$ features)	Daily/Intraday	Temporal sequence
Audio	Earnings calls, conference presentations	Very high (waveform)	Quarterly	Temporal waveform
Graph	Ownership networks, supply chains, co-holdings	Variable	Quarterly	Adjacency + node features

Each modality carries both unique and redundant information relative to others. The value of multimodal fusion lies in the unique (complementary) information:

$$I(\text{Returns}; \text{Text}, \text{Image}, \text{Tabular}) \geq \max\left(I(\text{Returns}; \text{Text}), I(\text{Returns}; \text{Image}), I(\text{Returns}; \text{Tabular})\right) \tag{107.1}$$

where $I(\cdot;\cdot)$ denotes mutual information. The inequality is strict whenever the modalities carry non-redundant predictive content. The goal of fusion is to design architectures that approach the left-hand side.

107.1.2 Taxonomies of Fusion

The multimodal learning literature (Baltrušaitis, Ahuja, and Morency 2018; Liang, Zadeh, and Morency 2024) organizes fusion strategies along three dimensions.

By stage. Where in the processing pipeline are modalities combined?

- *Input-level (early) fusion*: Concatenate raw or lightly processed features before any shared model.
- *Feature-level (intermediate) fusion*: Align learned representations in a shared latent space, then combine.
- *Decision-level (late) fusion*: Train separate models per modality, combine predictions.

By mechanism. How are representations combined?

- *Concatenation*: $\mathbf{z} = [\mathbf{z}^{(1)}; \mathbf{z}^{(2)}; \ldots; \mathbf{z}^{(M)}]$. Simple but ignores cross-modal interactions.
- *Attention-based*: One modality attends to another. Captures interactions but requires sufficient data.
- *Tensor product*: $\mathbf{z} = \mathbf{z}^{(1)} \otimes \mathbf{z}^{(2)}$. Captures all pairwise interactions but scales quadratically.
- *Gating*: $\mathbf{z} = g(\mathbf{z}^{(1)}) \odot \mathbf{z}^{(2)} + (1 - g(\mathbf{z}^{(1)})) \odot \mathbf{z}^{(3)}$. Modality selection.

By training. How are parameters learned?

- *Joint training*: All modalities processed end-to-end.
- *Pre-train then fuse*: Train unimodal encoders separately, then learn the fusion layer.
- *Contrastive alignment*: Train modality encoders to produce similar representations for matched pairs (the CLIP approach of Radford et al. (2021)).

```
# DataCore.vn API
from datacore import DataCore
dc = DataCore()

# Load aligned multimodal dataset
# Each observation: firm × quarter with all available modalities

# Tabular: financial statements
financials = dc.get_firm_financials(
    start_date="2014-01-01",
    end_date="2024-12-31",
    frequency="quarterly"
)

# Text: management discussion from annual reports
report_text = dc.get_annual_report_text(
    start_date="2014-01-01",
    end_date="2024-12-31",
    section="management_discussion"
)

# Image: satellite nightlight features (from Chapter 61)
satellite_features = dc.get_satellite_features(
    start_date="2014-01-01",
    end_date="2024-12-31",
    feature_type="cnn_resnet50"
)
```

```
# Time series: daily returns and volume
daily_data = dc.get_daily_returns(
    start_date="2014-01-01",
    end_date="2024-12-31"
)

# Target: forward quarterly returns
quarterly_returns = dc.get_quarterly_returns(
    start_date="2014-01-01",
    end_date="2024-12-31"
)

print(f"Firms with financials: {financials['ticker'].nunique()}")
print(f"Firms with report text: {report_text['ticker'].nunique()}")
print(f"Firms with satellite data: {satellite_features['ticker'].nunique()}")
```

107.2 Representation Alignment

107.2.1 The Alignment Problem

Different modalities produce embeddings in different vector spaces with different geometries. A PhoBERT text embedding lives in $\mathbb{R}^{768}$; a ResNet50 image feature lives in $\mathbb{R}^{2048}$; a tabular feature vector might have 50 dimensions with heterogeneous scales. Naively concatenating these into a single vector $[\mathbf{z}^{\text{text}}; \mathbf{z}^{\text{image}}; \mathbf{z}^{\text{tab}}] \in \mathbb{R}^{2866}$ is problematic because the high-dimensional modalities dominate gradient flow, the scales are mismatched, and there is no mechanism for cross-modal interaction.

Alignment projects each modality into a shared latent space $\mathbb{R}^d$ where geometric relationships are semantically meaningful (i.e., similar firms should be nearby regardless of which modality is used to represent them).

107.2.2 Contrastive Alignment: CLIP for Finance

The Contrastive Language-Image Pre-training (CLIP) framework of Radford et al. (2021) learns aligned representations by training on matched (text, image) pairs. We adapt this to financial data: for each firm-quarter, we have a textual description and a satellite image, and we train the encoders so that matched pairs produce similar embeddings while unmatched pairs produce dissimilar embeddings.

The contrastive loss is:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{2N}\sum_{i=1}^{N}\left[\log\frac{\exp(\mathbf{z}_i^{\text{txt}}\cdot\mathbf{z}_i^{\text{img}}/\tau)}{\sum_{j=1}^{N}\exp(\mathbf{z}_i^{\text{txt}}\cdot\mathbf{z}_j^{\text{img}}/\tau)} + \log\frac{\exp(\mathbf{z}_i^{\text{img}}\cdot\mathbf{z}_i^{\text{txt}}/\tau)}{\sum_{j=1}^{N}\exp(\mathbf{z}_i^{\text{img}}\cdot\mathbf{z}_j^{\text{txt}}/\tau)}\right] \tag{107.2}$$

where τ is a learnable temperature parameter and the embeddings are L_2-normalized. This is a symmetric version of the InfoNCE loss (Oord, Li, and Vinyals 2018) that simultaneously trains the text encoder to predict the correct image and vice versa.

```
class FinancialCLIP(nn.Module):
    """
    Contrastive alignment of text and image embeddings
    for Vietnamese financial data.
    """

    def __init__(self, text_dim=768, image_dim=2048, proj_dim=256):
        super().__init__()

        # Text projection
        self.text_proj = nn.Sequential(
            nn.Linear(text_dim, proj_dim),
            nn.LayerNorm(proj_dim),
            nn.GELU(),
            nn.Linear(proj_dim, proj_dim)
        )

        # Image projection
        self.image_proj = nn.Sequential(
            nn.Linear(image_dim, proj_dim),
            nn.LayerNorm(proj_dim),
            nn.GELU(),
            nn.Linear(proj_dim, proj_dim)
        )

        # Learnable temperature
        self.log_temp = nn.Parameter(torch.tensor(np.log(1 / 0.07)))

    def forward(self, text_emb, image_emb):
        """Compute aligned embeddings and contrastive loss."""
        # Project and normalize
        z_text = F.normalize(self.text_proj(text_emb), dim=-1)
        z_image = F.normalize(self.image_proj(image_emb), dim=-1)

        # Similarity matrix
```

```
        temp = self.log_temp.exp()
        logits = z_text @ z_image.T * temp

        # Symmetric cross-entropy loss
        labels = torch.arange(len(text_emb), device=text_emb.device)
        loss_t2i = F.cross_entropy(logits, labels)
        loss_i2t = F.cross_entropy(logits.T, labels)

        loss = (loss_t2i + loss_i2t) / 2

        return z_text, z_image, loss

    def encode_text(self, text_emb):
        return F.normalize(self.text_proj(text_emb), dim=-1)

    def encode_image(self, image_emb):
        return F.normalize(self.image_proj(image_emb), dim=-1)
```

107.2.3 Projection Alignment for Arbitrary Modalities

For more than two modalities, we generalize to a shared projection space where each modality has its own encoder but all encoders map to the same target space:

$$\mathbf{z}_i^{(m)} = f^{(m)}(\mathbf{x}_i^{(m)}; \theta^{(m)}) \in \mathbb{R}^d, \qquad m = 1, \ldots, M \tag{107.3}$$

The alignment loss encourages all modality embeddings for the same observation to be similar:

$$\mathcal{L}_{\text{align}} = \sum_{m<m'} \frac{1}{N} \sum_{i=1}^{N} \left\| \mathbf{z}_i^{(m)} - \mathbf{z}_i^{(m')} \right\|^2 \tag{107.4}$$

This MSE alignment is simpler than contrastive alignment but does not enforce the discriminative property (different observations should have dissimilar embeddings). In practice, we combine alignment with a prediction objective:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{predict}}(\hat{y}, y) + \lambda \cdot \mathcal{L}_{\text{align}} \tag{107.5}$$

```
class MultimodalProjector(nn.Module):
    """
    Project arbitrary modalities into a shared latent space.
    Supports variable numbers of modalities per observation.
    """

    def __init__(self, modality_dims, proj_dim=128, dropout=0.2):
        """
        Parameters
        ----------
        modality_dims : dict
            {modality_name: input_dim}, e.g.,
            {'text': 768, 'image': 2048, 'tabular': 50, 'ts': 128}
        proj_dim : int
            Shared projection dimensionality.
        """
        super().__init__()

        self.modality_names = list(modality_dims.keys())
        self.proj_dim = proj_dim

        # Per-modality encoders
        self.encoders = nn.ModuleDict()
        for name, dim in modality_dims.items():
            self.encoders[name] = nn.Sequential(
                nn.Linear(dim, proj_dim * 2),
                nn.LayerNorm(proj_dim * 2),
                nn.GELU(),
                nn.Dropout(dropout),
                nn.Linear(proj_dim * 2, proj_dim),
                nn.LayerNorm(proj_dim)
            )

    def forward(self, modality_inputs):
        """
        Parameters
        ----------
        modality_inputs : dict
            {modality_name: tensor}, may be missing some modalities.

        Returns
        -------
        dict : {modality_name: projected_embedding}
```

```
        """
        embeddings = {}
        for name, x in modality_inputs.items():
            if name in self.encoders and x is not None:
                embeddings[name] = self.encoders[name](x)

        return embeddings

    def compute_alignment_loss(self, embeddings):
        """Pairwise MSE alignment across all available modalities."""
        names = list(embeddings.keys())
        if len(names) < 2:
            return torch.tensor(0.0, device=next(self.parameters()).device)

        loss = torch.tensor(0.0, device=next(self.parameters()).device)
        n_pairs = 0
        for i in range(len(names)):
            for j in range(i + 1, len(names)):
                loss += F.mse_loss(
                    embeddings[names[i]], embeddings[names[j]]
                )
                n_pairs += 1

        return loss / n_pairs if n_pairs > 0 else loss
```

107.3 Fusion Architectures for Return Prediction

107.3.1 Unimodal Encoders

Before fusing modalities, we need encoders that produce fixed-dimensional representations from each raw input. We build four encoders corresponding to the primary modalities in Vietnamese equity markets.

```
class TabularEncoder(nn.Module):
    """Encode financial statement features."""

    def __init__(self, input_dim, hidden_dim=128, output_dim=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
```

```
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, x):
        return self.net(x)


class TextEncoder(nn.Module):
    """
    Encode Vietnamese text using pre-extracted PhoBERT embeddings.
    Input: pre-computed [CLS] token embedding (768-d).
    """

    def __init__(self, input_dim=768, output_dim=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.LayerNorm(256),
            nn.GELU(),
            nn.Dropout(0.2),
            nn.Linear(256, output_dim)
        )

    def forward(self, x):
        return self.net(x)


class ImageEncoder(nn.Module):
    """
    Encode satellite / document image features.
    Input: pre-computed CNN features (e.g., ResNet50 2048-d).
    """

    def __init__(self, input_dim=2048, output_dim=64):
        super().__init__()
        self.net = nn.Sequential(
```

```
            nn.Linear(input_dim, 512),
            nn.LayerNorm(512),
            nn.GELU(),
            nn.Dropout(0.2),
            nn.Linear(512, output_dim)
        )

    def forward(self, x):
        return self.net(x)


class TimeSeriesEncoder(nn.Module):
    """
    Encode price/volume time series using a 1D CNN + attention.
    Input: (batch, seq_len, n_features) tensor of daily data.
    """

    def __init__(self, n_features=5, seq_len=60, output_dim=64):
        super().__init__()

        # 1D convolutional layers
        self.conv1 = nn.Conv1d(n_features, 32, kernel_size=5, padding=2)
        self.conv2 = nn.Conv1d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.AdaptiveAvgPool1d(1)

        # Temporal attention
        self.attn = nn.MultiheadAttention(
            embed_dim=64, num_heads=4, batch_first=True
        )

        self.fc = nn.Linear(64, output_dim)

    def forward(self, x):
        # x: (B, T, F) -> (B, F, T) for Conv1d
        x = x.transpose(1, 2)
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))

        # (B, 64, T) -> (B, T, 64) for attention
        x = x.transpose(1, 2)
        attn_out, _ = self.attn(x, x, x)

        # Pool over time
```

```
        x = attn_out.transpose(1, 2)  # (B, 64, T)
        x = self.pool(x).squeeze(-1)  # (B, 64)

        return self.fc(x)
```

107.3.2 Early Fusion

Early fusion concatenates modality embeddings before a shared prediction head. This is the simplest approach and serves as a natural baseline.

```
class EarlyFusionModel(nn.Module):
    """
    Concatenate modality embeddings, then predict.
    """

    def __init__(self, encoders, hidden_dim=128, output_dim=1):
        """
        Parameters
        ----------
        encoders : dict
            {modality_name: encoder_module}
            Each encoder outputs a vector of the same dimension.
        """
        super().__init__()
        self.encoders = nn.ModuleDict(encoders)
        self.n_modalities = len(encoders)

        # Infer encoder output dim from first encoder
        sample_encoder = list(encoders.values())[0]
        enc_dim = list(sample_encoder.parameters())[-1].shape[0]

        self.head = nn.Sequential(
            nn.Linear(enc_dim * self.n_modalities, hidden_dim),
            nn.LayerNorm(hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Linear(hidden_dim // 2, output_dim)
        )
```

```
    def forward(self, inputs):
        """
        Parameters
        ----------
        inputs : dict
            {modality_name: tensor}
        """
        embeddings = []
        for name, encoder in self.encoders.items():
            if name in inputs and inputs[name] is not None:
                embeddings.append(encoder(inputs[name]))
            else:
                # Zero-fill missing modalities
                device = next(self.parameters()).device
                enc_dim = list(encoder.parameters())[-1].shape[0]
                embeddings.append(torch.zeros(
                    inputs[list(inputs.keys())[0]].shape[0],
                    enc_dim, device=device
                ))

        combined = torch.cat(embeddings, dim=-1)
        return self.head(combined).squeeze(-1)
```

107.3.3 Late Fusion

Late fusion trains independent models per modality and combines their predictions. The combination weights can be fixed (equal averaging), learned (linear), or adaptive (gating network).

```
class LateFusionModel(nn.Module):
    """
    Independent prediction per modality, learned combination.
    """

    def __init__(self, encoders, enc_dim=64, combination="learned"):
        """
        Parameters
        ----------
        combination : str
            'average', 'learned', or 'gating'.
        """
```

```
        super().__init__()
        self.encoders = nn.ModuleDict(encoders)
        self.combination = combination
        self.n_modalities = len(encoders)

        # Per-modality prediction heads
        self.heads = nn.ModuleDict({
            name: nn.Linear(enc_dim, 1)
            for name in encoders
        })

        if combination == "learned":
            self.weights = nn.Parameter(
                torch.ones(self.n_modalities) / self.n_modalities
            )
        elif combination == "gating":
            # Gating network takes all embeddings as input
            self.gate = nn.Sequential(
                nn.Linear(enc_dim * self.n_modalities, self.n_modalities),
                nn.Softmax(dim=-1)
            )

    def forward(self, inputs):
        predictions = {}
        embeddings = {}

        for name, encoder in self.encoders.items():
            if name in inputs and inputs[name] is not None:
                emb = encoder(inputs[name])
                pred = self.heads[name](emb).squeeze(-1)
                predictions[name] = pred
                embeddings[name] = emb
            else:
                device = next(self.parameters()).device
                batch_size = inputs[list(inputs.keys())[0]].shape[0]
                predictions[name] = torch.zeros(batch_size, device=device)
                enc_dim = list(encoder.parameters())[-1].shape[0]
                embeddings[name] = torch.zeros(
                    batch_size, enc_dim, device=device
                )

        pred_stack = torch.stack(list(predictions.values()), dim=-1)
```

```
        if self.combination == "average":
            return pred_stack.mean(dim=-1)
        elif self.combination == "learned":
            weights = F.softmax(self.weights, dim=0)
            return (pred_stack * weights).sum(dim=-1)
        elif self.combination == "gating":
            all_emb = torch.cat(list(embeddings.values()), dim=-1)
            gate_weights = self.gate(all_emb)
            return (pred_stack * gate_weights).sum(dim=-1)

    def get_modality_weights(self):
        """Return the contribution of each modality."""
        if self.combination == "learned":
            return F.softmax(self.weights, dim=0).detach().cpu().numpy()
        return None
```

107.3.4 Cross-Attention Fusion

Cross-attention fusion is the most expressive architecture. Each modality attends to every other modality, learning which cross-modal interactions are informative. This is the mechanism underlying modern vision-language models like Flamingo (Alayrac et al. 2022) and GPT-4V.

The cross-attention operation for modality m attending to modality m' is:

$$\text{CA}^{(m \to m')} = \text{softmax}\left(\frac{\mathbf{Q}^{(m)} \left(\mathbf{K}^{(m')}\right)^\top}{\sqrt{d_k}}\right) \mathbf{V}^{(m')} \tag{107.6}$$

where $\mathbf{Q}^{(m)} = \mathbf{z}^{(m)} W_Q$, $\mathbf{K}^{(m')} = \mathbf{z}^{(m')} W_K$, $\mathbf{V}^{(m')} = \mathbf{z}^{(m')} W_V$. The output enriches modality m's representation with information from modality m'.

```
class CrossAttentionBlock(nn.Module):
    """Single cross-attention block: query modality attends to key
     ↪ modality."""

    def __init__(self, dim, n_heads=4, dropout=0.1):
        super().__init__()
        self.attn = nn.MultiheadAttention(
            embed_dim=dim, num_heads=n_heads,
            dropout=dropout, batch_first=True
        )
```

```
        self.norm1 = nn.LayerNorm(dim)
        self.norm2 = nn.LayerNorm(dim)
        self.ffn = nn.Sequential(
            nn.Linear(dim, dim * 4),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(dim * 4, dim),
            nn.Dropout(dropout)
        )

    def forward(self, query, key_value):
        # Cross-attention
        q = query.unsqueeze(1) if query.dim() == 2 else query
        kv = key_value.unsqueeze(1) if key_value.dim() == 2 else key_value

        attn_out, attn_weights = self.attn(q, kv, kv)
        q = self.norm1(q + attn_out)

        # Feed-forward
        out = self.norm2(q + self.ffn(q))
        return out.squeeze(1) if query.dim() == 2 else out, attn_weights


class CrossAttentionFusionModel(nn.Module):
    """
    Full cross-attention fusion across M modalities.
    Each modality attends to all others via cross-attention blocks.
    """

    def __init__(self, encoders, enc_dim=64, n_layers=2, n_heads=4):
        super().__init__()
        self.encoders = nn.ModuleDict(encoders)
        self.modality_names = list(encoders.keys())
        self.n_modalities = len(encoders)

        # Cross-attention blocks: each modality attends to each other
        self.cross_attn_layers = nn.ModuleList()
        for _ in range(n_layers):
            layer = nn.ModuleDict()
            for m in self.modality_names:
                for m_prime in self.modality_names:
                    if m != m_prime:
                        layer[f"{m}_to_{m_prime}"] = CrossAttentionBlock(
```

```
                        enc_dim, n_heads
                    )
            self.cross_attn_layers.append(layer)

        # Prediction head
        self.head = nn.Sequential(
            nn.Linear(enc_dim * self.n_modalities, enc_dim),
            nn.LayerNorm(enc_dim),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(enc_dim, 1)
        )

    def forward(self, inputs):
        # Encode each modality
        embeddings = {}
        for name, encoder in self.encoders.items():
            if name in inputs and inputs[name] is not None:
                embeddings[name] = encoder(inputs[name])
            else:
                device = next(self.parameters()).device
                batch_size = inputs[list(inputs.keys())[0]].shape[0]
                enc_dim = list(encoder.parameters())[-1].shape[0]
                embeddings[name] = torch.zeros(
                    batch_size, enc_dim, device=device
                )

        # Cross-attention layers
        all_attn_weights = {}
        for layer in self.cross_attn_layers:
            new_embeddings = {k: v.clone() for k, v in embeddings.items()}
            for key, block in layer.items():
                parts = key.split("_to_")
                query_mod, kv_mod = parts[0], parts[1]
                if query_mod in embeddings and kv_mod in embeddings:
                    updated, weights = block(
                        embeddings[query_mod],
                        embeddings[kv_mod]
                    )
                    new_embeddings[query_mod] = (
                        new_embeddings[query_mod] + updated
                    )
                    all_attn_weights[key] = weights
```

```
            embeddings = new_embeddings

        # Concatenate and predict
        combined = torch.cat(
            [embeddings[name] for name in self.modality_names],
            dim=-1
        )
        return self.head(combined).squeeze(-1), all_attn_weights
```

107.3.5 Comparison Experiment

We now compare the three fusion architectures against unimodal baselines on forward quarterly return prediction for Vietnamese equities.

```
class MultimodalFinanceDataset(Dataset):
    """
    Dataset that aligns multiple modalities per firm-quarter.
    Handles missing modalities with None values.
    """

    def __init__(self, tabular_df, text_embeddings, image_features,
                 ts_features, returns, tickers, dates):
        self.tabular = tabular_df
        self.text = text_embeddings
        self.image = image_features
        self.ts = ts_features
        self.returns = returns
        self.tickers = tickers
        self.dates = dates

    def __len__(self):
        return len(self.returns)

    def __getitem__(self, idx):
        sample = {
            "tabular": torch.tensor(
                self.tabular[idx], dtype=torch.float32
            ) if self.tabular[idx] is not None else None,
            "text": torch.tensor(
                self.text[idx], dtype=torch.float32
```

```
            ) if self.text[idx] is not None else None,
            "image": torch.tensor(
                self.image[idx], dtype=torch.float32
            ) if self.image[idx] is not None else None,
            "ts": torch.tensor(
                self.ts[idx], dtype=torch.float32
            ) if self.ts[idx] is not None else None,
            "return": torch.tensor(
                self.returns[idx], dtype=torch.float32
            ),
            "ticker": self.tickers[idx],
            "date": self.dates[idx]
        }
        return sample


def collate_multimodal(batch):
    """Custom collate that handles None modalities."""
    result = {"return": torch.stack([b["return"] for b in batch])}

    for mod in ["tabular", "text", "image", "ts"]:
        values = [b[mod] for b in batch]
        if all(v is not None for v in values):
            result[mod] = torch.stack(values)
        elif any(v is not None for v in values):
            # Fill None with zeros, matching shape of non-None entries
            ref = next(v for v in values if v is not None)
            filled = [v if v is not None else torch.zeros_like(ref)
                      for v in values]
            result[mod] = torch.stack(filled)
        else:
            result[mod] = None

    return result
```

```
# Prepare aligned firm-quarter dataset
# Step 1: Financial ratios (tabular)
tabular_features = [
    "roe", "roa", "book_to_market", "log_size", "leverage",
    "asset_growth", "gross_profitability", "capex_to_assets",
    "cash_to_assets", "dividend_yield", "sales_growth",
    "accruals", "earnings_volatility", "beta"
```

```
]

financials["quarter_date"] = pd.to_datetime(
    financials["year"].astype(str) + "-" +
    (financials["quarter"] * 3).astype(str).str.zfill(2) + "-01"
)

# Step 2: Text embeddings from PhoBERT
# (Pre-computed in Chapter 60)
text_emb = dc.get_text_embeddings(
    model="phobert",
    section="management_discussion",
    start_date="2014-01-01",
    end_date="2024-12-31"
)

# Step 3: Image features (pre-computed in Chapter 61)
# Satellite CNN features linked to firm headquarters province

# Step 4: Time series features (60-day window before quarter end)
def compute_ts_features(ticker, date, daily_df, lookback=60):
    """Extract time-series feature tensor for a firm-quarter."""
    mask = (
        (daily_df["ticker"] == ticker) &
        (daily_df["date"] <= date) &
        (daily_df["date"] >= date - pd.Timedelta(days=lookback * 1.5))
    )
    subset = daily_df[mask].sort_values("date").tail(lookback)

    if len(subset) < lookback // 2:
        return None

    features = subset[["ret", "volume_log", "volatility_20d",
                       "spread", "turnover"]].values

    # Pad if shorter than lookback
    if len(features) < lookback:
        padding = np.zeros((lookback - len(features), features.shape[1]))
        features = np.vstack([padding, features])

    return features

# Step 5: Forward quarterly returns (target)
```

```
# Align everything to quarter-end dates
print("Preparing aligned multimodal dataset...")
```

```
def train_multimodal_model(model, train_loader, val_loader,
                           n_epochs=50, lr=1e-3, patience=10,
                           alignment_weight=0.0):
    """
    Train a multimodal model with early stopping.

    Parameters
    ----------
    model : nn.Module
        Multimodal fusion model.
    alignment_weight : float
        Weight for modality alignment loss (0 = no alignment).

    Returns
    -------
    dict : Training history and best validation metrics.
    """
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr,
                                  weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode="min", patience=5, factor=0.5
    )

    best_val_loss = float("inf")
    epochs_no_improve = 0
    history = {"train_loss": [], "val_loss": [], "val_r2": []}

    for epoch in range(n_epochs):
        # Training
        model.train()
        train_losses = []
        for batch in train_loader:
            optimizer.zero_grad()

            inputs = {k: batch[k] for k in ["tabular", "text", "image",
↪   "ts"]}
            targets = batch["return"]

            # Forward pass (handle both output types)
```

```
            output = model(inputs)
            if isinstance(output, tuple):
                predictions, attn_weights = output
            else:
                predictions = output

            loss = F.mse_loss(predictions, targets)

            # Optional alignment loss
            if alignment_weight > 0 and hasattr(model, "projector"):
                embeddings = model.projector(inputs)
                align_loss =
↪   model.projector.compute_alignment_loss(embeddings)
                loss = loss + alignment_weight * align_loss

            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()

            train_losses.append(loss.item())

        # Validation
        model.eval()
        val_preds, val_targets = [], []
        val_losses = []

        with torch.no_grad():
            for batch in val_loader:
                inputs = {k: batch[k]
                          for k in ["tabular", "text", "image", "ts"]}
                targets = batch["return"]

                output = model(inputs)
                if isinstance(output, tuple):
                    predictions, _ = output
                else:
                    predictions = output

                val_losses.append(F.mse_loss(predictions, targets).item())
                val_preds.extend(predictions.cpu().numpy())
                val_targets.extend(targets.cpu().numpy())

        val_loss = np.mean(val_losses)
```

```
        val_r2 = r2_score(val_targets, val_preds) if len(val_preds) > 10 else
↪   0

        history["train_loss"].append(np.mean(train_losses))
        history["val_loss"].append(val_loss)
        history["val_r2"].append(val_r2)

        scheduler.step(val_loss)

        # Early stopping
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_state = {k: v.cpu().clone()
                          for k, v in model.state_dict().items()}
            epochs_no_improve = 0
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= patience:
                break

    # Restore best model
    model.load_state_dict(best_state)

    return {
        "history": history,
        "best_val_loss": best_val_loss,
        "best_val_r2": max(history["val_r2"]),
        "epochs_trained": len(history["train_loss"])
    }
```

```
def compare_fusion_strategies(dataset, n_splits=5):
    """
    Compare unimodal baselines and multimodal fusion strategies
    using expanding-window time-series cross-validation.

    Returns
    -------
    DataFrame : Out-of-sample R², MSE, IC for each model.
    """
    tscv = TimeSeriesSplit(n_splits=n_splits)
    results = []
```

```
    enc_dim = 64

    for fold, (train_idx, test_idx) in enumerate(
        tscv.split(range(len(dataset)))
    ):
        # Create data loaders
        train_subset = torch.utils.data.Subset(dataset, train_idx)
        test_subset = torch.utils.data.Subset(dataset, test_idx)

        train_loader = DataLoader(
            train_subset, batch_size=128, shuffle=True,
            collate_fn=collate_multimodal
        )
        test_loader = DataLoader(
            test_subset, batch_size=256, shuffle=False,
            collate_fn=collate_multimodal
        )

        # Define encoders
        def make_encoders():
            return {
                "tabular": TabularEncoder(len(tabular_features), 128,
                 ↪  enc_dim),
                "text": TextEncoder(768, enc_dim),
                "image": ImageEncoder(2048, enc_dim),
                "ts": TimeSeriesEncoder(5, 60, enc_dim)
            }

        # Unimodal baselines
        for mod_name in ["tabular", "text", "image", "ts"]:
            single_encoder = {mod_name: make_encoders()[mod_name]}
            model = EarlyFusionModel(single_encoder, enc_dim, 1)

            result = train_multimodal_model(
                model, train_loader, test_loader, n_epochs=30
            )
            results.append({
                "fold": fold,
                "model": f"Unimodal ({mod_name})",
                "val_r2": result["best_val_r2"],
                "val_loss": result["best_val_loss"]
            })
```

```
        # Multimodal: Early Fusion
        model_early = EarlyFusionModel(make_encoders(), enc_dim * 2, 1)
        result = train_multimodal_model(
            model_early, train_loader, test_loader, n_epochs=30
        )
        results.append({
            "fold": fold,
            "model": "Early Fusion",
            "val_r2": result["best_val_r2"],
            "val_loss": result["best_val_loss"]
        })

        # Multimodal: Late Fusion (gating)
        model_late = LateFusionModel(
            make_encoders(), enc_dim, combination="gating"
        )
        result = train_multimodal_model(
            model_late, train_loader, test_loader, n_epochs=30
        )
        results.append({
            "fold": fold,
            "model": "Late Fusion (Gating)",
            "val_r2": result["best_val_r2"],
            "val_loss": result["best_val_loss"]
        })

        # Multimodal: Cross-Attention
        model_ca = CrossAttentionFusionModel(
            make_encoders(), enc_dim, n_layers=2, n_heads=4
        )
        result = train_multimodal_model(
            model_ca, train_loader, test_loader, n_epochs=30
        )
        results.append({
            "fold": fold,
            "model": "Cross-Attention Fusion",
            "val_r2": result["best_val_r2"],
            "val_loss": result["best_val_loss"]
        })

    return pd.DataFrame(results)
```

Table 107.2: Out-of-Sample Return Prediction: Unimodal vs. Multimodal

```
# results_df = compare_fusion_strategies(dataset)

# Aggregate across folds
# summary = (
#     results_df.groupby("model")
#     .agg(
#         mean_r2=("val_r2", "mean"),
#         std_r2=("val_r2", "std"),
#         mean_loss=("val_loss", "mean")
#     )
#     .sort_values("mean_r2", ascending=False)
#     .round(4)
# )
# summary
```

```
# (
#     p9.ggplot(results_df, p9.aes(x="model", y="val_r2", fill="model"))
#     + p9.geom_boxplot(alpha=0.7)
#     + p9.coord_flip()
#     + p9.labs(
#         x="", y="Out-of-Sample R²",
#         title="Multimodal Fusion Improves Return Prediction"
#     )
#     + p9.theme_minimal()
#     + p9.theme(figure_size=(10, 6), legend_position="none")
# )
```

Figure 107.1

107.4 Handling Missing Modalities

107.4.1 The Missing Modality Problem

In practice, not every firm-quarter has every modality available. A firm may not have an earnings call transcript (no audio), its headquarters may be in a province where satellite coverage is intermittent (no image), or its annual report may not be publicly available in digital form (no text). This creates a missing modality problem that is structurally different from missing values in tabular data: an entire feature vector (hundreds or thousands of dimensions) is absent.

The fraction of observations with all four modalities available is typically much smaller than the fraction with at least one:

Table 107.3: Modality Availability in Vietnamese Market Data

Available Modalities	Typical Coverage (Vietnamese Firms)
Tabular only	$\sim 95\%$ of firm-quarters
Tabular + Text	$\sim 70\%$
Tabular + Text + Image	$\sim 50\%$
All four (+ time series)	$\sim 45\%$

Restricting the sample to complete cases discards half the data and introduces selection bias (larger, more transparent firms are overrepresented). We need architectures that degrade gracefully when modalities are missing.

107.4.2 Strategies for Missing Modalities

- **Zero imputation.** Replace missing modality embeddings with zeros. Simple but introduces bias: the model cannot distinguish "this modality is absent" from "this modality has zero signal."
- **Learned default embedding.** Replace missing modalities with a learnable "default" vector $\mathbf{d}^{(m)}$ that is trained alongside the model. This allows the model to learn what the absence of a modality implies.
- **Modality dropout.** During training, randomly drop entire modalities with probability p (analogous to dropout on neurons). This forces the model to perform well even when modalities are missing, and acts as regularization.
- **Mixture of Experts (MoE).** Route each observation to a fusion subnetwork specialized for its available modality combination. With M modalities, there are $2^M - 1$ possible subsets, requiring efficient parameter sharing.

```
class ModalityDropout(nn.Module):
    """
    Randomly drop entire modalities during training.
    Forces robustness to missing inputs at test time.
    """

    def __init__(self, drop_prob=0.2):
        super().__init__()
        self.drop_prob = drop_prob

    def forward(self, modality_inputs):
        if not self.training:
            return modality_inputs

        result = {}
        for name, tensor in modality_inputs.items():
            if tensor is not None and torch.rand(1).item() > self.drop_prob:
                result[name] = tensor
            else:
                result[name] = None

        # Ensure at least one modality remains
        if all(v is None for v in result.values()):
            # Keep the first available modality
            for name, tensor in modality_inputs.items():
                if tensor is not None:
                    result[name] = tensor
                    break

        return result


class RobustFusionModel(nn.Module):
    """
    Multimodal model robust to missing modalities.
    Uses learned default embeddings and modality dropout.
    """

    def __init__(self, encoders, enc_dim=64, drop_prob=0.2):
        super().__init__()
        self.encoders = nn.ModuleDict(encoders)
        self.modality_names = list(encoders.keys())
        self.n_modalities = len(encoders)
```

```
        self.enc_dim = enc_dim

        # Learned default embeddings for missing modalities
        self.defaults = nn.ParameterDict({
            name: nn.Parameter(torch.randn(enc_dim) * 0.01)
            for name in encoders
        })

        # Modality presence indicator embedding
        self.presence_proj = nn.Linear(self.n_modalities, enc_dim)

        # Modality dropout
        self.mod_dropout = ModalityDropout(drop_prob)

        # Attention-based aggregation
        self.attn_pool = nn.Sequential(
            nn.Linear(enc_dim, 1),
            nn.Softmax(dim=0)
        )

        # Prediction head
        self.head = nn.Sequential(
            nn.Linear(enc_dim * 2, enc_dim),
            nn.LayerNorm(enc_dim),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(enc_dim, 1)
        )

    def forward(self, inputs):
        # Apply modality dropout during training
        inputs = self.mod_dropout(inputs)

        embeddings = []
        presence = []

        for name in self.modality_names:
            if name in inputs and inputs[name] is not None:
                emb = self.encoders[name](inputs[name])
                embeddings.append(emb)
                presence.append(1.0)
            else:
                batch_size = next(
```

```
                    v.shape[0] for v in inputs.values()
                    if v is not None
                )
                emb = self.defaults[name].unsqueeze(0).expand(
                    batch_size, -1
                )
                embeddings.append(emb)
                presence.append(0.0)

        # Stack: (n_modalities, batch, enc_dim)
        emb_stack = torch.stack(embeddings, dim=0)

        # Attention-weighted aggregation
        attn_weights = self.attn_pool(emb_stack)  # (n_mod, batch, 1)
        aggregated = (emb_stack * attn_weights).sum(dim=0)  # (batch,
↪  enc_dim)

        # Presence indicator
        device = aggregated.device
        presence_tensor = torch.tensor(
            presence, device=device
        ).unsqueeze(0).expand(aggregated.shape[0], -1)
        presence_emb = self.presence_proj(presence_tensor)

        # Combine
        combined = torch.cat([aggregated, presence_emb], dim=-1)
        return self.head(combined).squeeze(-1)
```

107.5 Multimodal Document Understanding

107.5.1 Annual Report as a Multimodal Object

A Vietnamese annual report is inherently multimodal: it contains running text (management discussion, risk factors, strategy), tables (financial statements, segment data, shareholder structure), images (photographs of facilities, products, management), and charts (revenue trends, market share). Prior chapters treated these as separate extraction problems. Here we build a model that processes the entire report as a unified multimodal document.

The architecture follows the Document Understanding Transformer (Donut) approach of G. Kim et al. (2022), adapted for Vietnamese financial filings:

$$\mathbf{h} = \text{Encoder}(\mathbf{I}_{\text{page}}) + \text{Encoder}(\mathbf{T}_{\text{ocr}}) + \text{Encoder}(\mathbf{L}_{\text{layout}}) \tag{107.7}$$

where $\mathbf{I}$ is the page image, $\mathbf{T}$ is the OCR text, and $\mathbf{L}$ is the spatial layout (bounding boxes). The joint representation $\mathbf{h}$ captures both what is written and where it appears on the page.

```
class MultimodalDocumentEncoder(nn.Module):
    """
    Joint encoder for Vietnamese annual report pages.
    Processes text, layout, and page image simultaneously.
    """

    def __init__(self, vocab_size=64000, max_boxes=512,
                 img_dim=2048, hidden_dim=256, n_layers=4,
                 n_heads=8):
        super().__init__()

        # Text embedding (Vietnamese tokens)
        self.text_emb = nn.Embedding(vocab_size, hidden_dim)

        # Layout embedding (bounding box coordinates)
        # Each box: [x0, y0, x1, y1] normalized to [0, 1000]
        self.x_emb = nn.Embedding(1001, hidden_dim // 4)
        self.y_emb = nn.Embedding(1001, hidden_dim // 4)

        # Image patch embedding
        self.img_proj = nn.Sequential(
            nn.Linear(img_dim, hidden_dim),
            nn.LayerNorm(hidden_dim)
        )

        # Modality type embedding
        self.modality_emb = nn.Embedding(3, hidden_dim)  # text, layout,
         ↪  image

        # Transformer encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=hidden_dim,
            nhead=n_heads,
            dim_feedforward=hidden_dim * 4,
            dropout=0.1,
            activation="gelu",
            batch_first=True
```

```
        )
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=n_layers
        )

        # [CLS] token
        self.cls_token = nn.Parameter(torch.randn(1, 1, hidden_dim))

    def embed_layout(self, boxes):
        """Embed bounding box coordinates."""
        x0 = self.x_emb(boxes[:, :, 0])
        y0 = self.y_emb(boxes[:, :, 1])
        x1 = self.x_emb(boxes[:, :, 2])
        y1 = self.y_emb(boxes[:, :, 3])
        return torch.cat([x0, y0, x1, y1], dim=-1)

    def forward(self, token_ids, boxes, img_features,
                attention_mask=None):
        """
        Parameters
        ----------
        token_ids : LongTensor (B, T)
            OCR token IDs.
        boxes : LongTensor (B, T, 4)
            Bounding boxes for each token.
        img_features : Tensor (B, P, img_dim)
            Image patch features from CNN.
        """
        batch_size = token_ids.shape[0]

        # Text + layout
        text_h = self.text_emb(token_ids) + self.embed_layout(boxes)
        text_h = text_h + self.modality_emb(
            torch.zeros(batch_size, text_h.shape[1],
                        dtype=torch.long, device=text_h.device)
        )

        # Image patches
        img_h = self.img_proj(img_features)
        img_h = img_h + self.modality_emb(
            torch.full((batch_size, img_h.shape[1]), 2,
                       dtype=torch.long, device=img_h.device)
        )
```

```
        # Prepend [CLS]
        cls = self.cls_token.expand(batch_size, -1, -1)

        # Concatenate all modalities
        sequence = torch.cat([cls, text_h, img_h], dim=1)

        # Transformer encoding
        output = self.transformer(sequence)

        # Return [CLS] representation
        return output[:, 0, :]
```

107.5.2 Extracting Structured Financials from Multimodal Reports

With the document encoder, we can build extraction heads for specific financial fields. The key advantage over the OCR-only pipeline in previous chapter is that the multimodal encoder can resolve ambiguities using visual context (e.g., a number's meaning depends on where it appears on the page and what headers and labels surround it).

```
class FinancialFieldExtractor(nn.Module):
    """
    Extract specific financial fields from a document embedding.
    Uses the multimodal document encoder as backbone.
    """

    def __init__(self, doc_encoder, fields, hidden_dim=256):
        """
        Parameters
        ----------
        doc_encoder : MultimodalDocumentEncoder
        fields : list
            Target field names, e.g.,
            ['revenue', 'net_income', 'total_assets', 'total_equity']
        """
        super().__init__()
        self.doc_encoder = doc_encoder
        self.fields = fields

        # Per-field extraction heads
        self.extractors = nn.ModuleDict({
```

```
            field: nn.Sequential(
                nn.Linear(hidden_dim, hidden_dim // 2),
                nn.GELU(),
                nn.Linear(hidden_dim // 2, 1)
            )
            for field in fields
        })

        # Confidence head
        self.confidence = nn.ModuleDict({
            field: nn.Sequential(
                nn.Linear(hidden_dim, 1),
                nn.Sigmoid()
            )
            for field in fields
        })

    def forward(self, token_ids, boxes, img_features):
        doc_emb = self.doc_encoder(token_ids, boxes, img_features)

        results = {}
        for field in self.fields:
            value = self.extractors[field](doc_emb).squeeze(-1)
            conf = self.confidence[field](doc_emb).squeeze(-1)
            results[field] = {"value": value, "confidence": conf}

        return results
```

107.6 Multimodal Earnings Surprise Model

107.6.1 Architecture

We now build the chapter's central empirical application: a multimodal model that predicts earnings surprises using all available modalities observed before the earnings announcement date.

The information set at time t^- (just before the announcement) includes:

- **Tabular**: Last reported financial ratios, analyst consensus forecasts
- **Text**: News articles and filings in the pre-announcement window
- **Image**: Satellite features of the firm's operating region

- **Time series**: Price and volume dynamics in the 60 trading days before announcement

The target is the standardized unexpected earnings (SUE):

$$\text{SUE}_{i,q} = \frac{E_{i,q} - \hat{E}_{i,q}}{\sigma_{i,q}} \tag{107.8}$$

where $E_{i,q}$ is actual earnings per share, $\hat{E}_{i,q}$ is the consensus forecast (or seasonal random walk forecast if analyst coverage is absent), and $\sigma_{i,q}$ is the standard deviation of forecast errors.

```
# Construct earnings surprise dataset
earnings = dc.get_earnings_announcements(
    start_date="2016-01-01",
    end_date="2024-12-31"
)

# Standardized Unexpected Earnings
earnings["sue"] = (
    (earnings["actual_eps"] - earnings["consensus_eps"]) /
    earnings["forecast_std"].clip(lower=0.01)
)

# Pre-announcement features
# Text: aggregate PhoBERT sentiment of news in [-30, -1] window
pre_ann_text = dc.get_pre_announcement_text_features(
    start_date="2016-01-01",
    end_date="2024-12-31",
    window_days=30,
    model="phobert"
)

# Image: satellite features at quarter end
pre_ann_image = satellite_features.copy()

# Time series: 60 trading days before announcement
# (Pre-computed above)

# Tabular: most recent quarterly financials
pre_ann_tabular = financials[tabular_features + ["ticker", "quarter_date"]]

print(f"Earnings announcements: {len(earnings)}")
print(f"With text features: {len(pre_ann_text)}")
```

```
class MultimodalEarningsSurpriseModel(nn.Module):
    """
    Predict standardized unexpected earnings (SUE) from
    multimodal pre-announcement information.
    """

    def __init__(self, tab_dim, text_dim=768, img_dim=2048,
                 ts_features=5, ts_len=60, hidden_dim=64,
                 n_heads=4, drop_prob=0.2):
        super().__init__()

        # Unimodal encoders
        self.tab_enc = TabularEncoder(tab_dim, 128, hidden_dim)
        self.text_enc = TextEncoder(text_dim, hidden_dim)
        self.img_enc = ImageEncoder(img_dim, hidden_dim)
        self.ts_enc = TimeSeriesEncoder(ts_features, ts_len, hidden_dim)

        # Modality dropout
        self.mod_dropout = ModalityDropout(drop_prob)

        # Cross-attention: text attends to time series
        # (news context informs price dynamics interpretation)
        self.text_ts_attn = CrossAttentionBlock(hidden_dim, n_heads)

        # Cross-attention: tabular attends to image
        # (financial ratios contextualized by physical activity)
        self.tab_img_attn = CrossAttentionBlock(hidden_dim, n_heads)

        # Modality importance weights (learned)
        self.importance = nn.Parameter(torch.ones(4))

        # Prediction head
        self.head = nn.Sequential(
            nn.Linear(hidden_dim * 2, hidden_dim),
            nn.LayerNorm(hidden_dim),
            nn.GELU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.GELU(),
            nn.Linear(hidden_dim // 2, 1)
        )

    def forward(self, tabular, text, image, ts):
```

```
        # Encode each modality
        h_tab = self.tab_enc(tabular) if tabular is not None else None
        h_txt = self.text_enc(text) if text is not None else None
        h_img = self.img_enc(image) if image is not None else None
        h_ts = self.ts_enc(ts) if ts is not None else None

        # Cross-attention pairs (if both available)
        if h_txt is not None and h_ts is not None:
            h_txt_enriched, _ = self.text_ts_attn(h_txt, h_ts)
        else:
            h_txt_enriched = h_txt

        if h_tab is not None and h_img is not None:
            h_tab_enriched, _ = self.tab_img_attn(h_tab, h_img)
        else:
            h_tab_enriched = h_tab

        # Weighted combination of available modalities
        embeddings = []
        weights = F.softmax(self.importance, dim=0)

        for i, h in enumerate([h_tab_enriched, h_txt_enriched,
                               h_img, h_ts]):
            if h is not None:
                embeddings.append(h * weights[i])
            else:
                device = next(self.parameters()).device
                batch_size = next(
                    x.shape[0] for x in [tabular, text, image, ts]
                    if x is not None
                )
                embeddings.append(
                    torch.zeros(batch_size, h_tab.shape[-1]
                                if h_tab is not None else 64,
                                device=device)
                )

        # Aggregate
        stacked = torch.stack(embeddings, dim=0)
        aggregated = stacked.sum(dim=0)

        # Also compute variance across modalities (disagreement signal)
        if stacked.shape[0] > 1:
```

```
            disagreement = stacked.var(dim=0)
        else:
            disagreement = torch.zeros_like(aggregated)

        combined = torch.cat([aggregated, disagreement], dim=-1)
        return self.head(combined).squeeze(-1)
```

107.6.2 Modality Importance Analysis

A key interpretability question is: which modality contributes most to earnings surprise prediction? We analyze the learned importance weights and conduct ablation experiments.

```
def ablation_study(model, test_loader, modality_names):
    """
    Measure each modality's contribution via leave-one-out ablation.

    For each modality m, zero out that modality's input and measure
    the degradation in prediction accuracy.

    Returns
    -------
    DataFrame : Modality, R² with all, R² without, Δ R².
    """
    model.eval()

    # Full model performance
    all_preds, all_targets = [], []
    with torch.no_grad():
        for batch in test_loader:
            inputs = {k: batch[k] for k in modality_names}
            targets = batch["return"]

            output = model(inputs)
            pred = output[0] if isinstance(output, tuple) else output
            all_preds.extend(pred.cpu().numpy())
            all_targets.extend(targets.cpu().numpy())

    r2_full = r2_score(all_targets, all_preds)

    # Ablation: remove one modality at a time
    results = [{"modality": "All", "r2": r2_full, "delta_r2": 0.0}]
```

Table 107.4: Modality Ablation Study: Contribution to Earnings Surprise Prediction

```
# ablation_df = ablation_study(model, test_loader, modality_names)
# ablation_df.round(4)
```

```
    for drop_mod in modality_names:
        ablated_preds = []
        with torch.no_grad():
            for batch in test_loader:
                inputs = {}
                for k in modality_names:
                    if k == drop_mod:
                        inputs[k] = None  # Remove this modality
                    else:
                        inputs[k] = batch[k]

                targets = batch["return"]
                output = model(inputs)
                pred = output[0] if isinstance(output, tuple) else output
                ablated_preds.extend(pred.cpu().numpy())

        r2_ablated = r2_score(all_targets, ablated_preds)
        results.append({
            "modality": f"Without {drop_mod}",
            "r2": r2_ablated,
            "delta_r2": r2_full - r2_ablated
        })

    return pd.DataFrame(results)
```

107.7 Large Multimodal Models for Financial Analysis

107.7.1 Prompting Vision-Language Models

The most powerful multimodal systems available today are large vision-language models (VLMs) such as GPT-4V, Gemini, and open-source alternatives (LLaVA, InternVL). These

```
# Track importance weights during training
# importance_history = pd.DataFrame(...)

# (
#     p9.ggplot(importance_history, p9.aes(
#         x="epoch", y="weight", color="modality"
#     ))
#     + p9.geom_line(size=1)
#     + p9.labs(
#         x="Training Epoch", y="Softmax Weight",
#         title="Modality Importance Convergence",
#         color="Modality"
#     )
#     + p9.scale_color_manual(
#         values=["#2E5090", "#C0392B", "#27AE60", "#8E44AD"]
#     )
#     + p9.theme_minimal()
#     + p9.theme(figure_size=(10, 5))
# )
```

Figure 107.2

models can jointly process images and text through natural language prompts, enabling zero-shot financial analysis without model training.

For Vietnamese financial applications, VLMs can:

- Interpret satellite imagery of industrial zones and estimate activity levels
- Read and extract data from scanned financial tables
- Analyze news photographs for sentiment
- Compare current and historical aerial views for change detection

```
def vlm_financial_qa(image_path, question, context=None):
    """
    Financial question-answering using a vision-language model.

    Parameters
    ----------
    image_path : str
        Path to image (satellite tile, document page, news photo).
    question : str
        Financial analysis question.
    context : str, optional
        Additional textual context (e.g., firm name, sector).

    Returns
    -------
    dict : Answer, confidence, extracted entities.
    """
    from transformers import (
        LlavaForConditionalGeneration,
        LlavaProcessor
    )

    model_id = "llava-hf/llava-v1.6-vicuna-7b-hf"
    processor = LlavaProcessor.from_pretrained(model_id)
    model = LlavaForConditionalGeneration.from_pretrained(
        model_id, torch_dtype=torch.float16, device_map="auto"
    )

    img = Image.open(image_path).convert("RGB")

    # Build financial analysis prompt
    system_prompt = (
        "You are a financial analyst examining visual evidence. "
```

```
        "Provide specific, quantitative observations when possible. "
        "State your confidence level (high/medium/low)."
    )

    if context:
        prompt = (
            f"{system_prompt}\n\nContext: {context}\n\n"
            f"Question: {question}\n\nAnswer:"
        )
    else:
        prompt = f"{system_prompt}\n\nQuestion: {question}\n\nAnswer:"

    inputs = processor(
        text=prompt,
        images=img,
        return_tensors="pt"
    ).to(model.device)

    with torch.no_grad():
        output = model.generate(
            **inputs,
            max_new_tokens=300,
            temperature=0.1,
            do_sample=False
        )

    answer = processor.decode(output[0], skip_special_tokens=True)
    answer = answer.split("Answer:")[-1].strip()

    return {"answer": answer, "question": question}


# Example financial VLM queries
FINANCIAL_VLM_PROMPTS = {
    "satellite_activity": (
        "Examine this satellite image of an industrial zone. "
        "Estimate the occupancy rate of factory buildings, "
        "the density of vehicles in parking areas, "
        "and whether the site appears to be operating at "
        "full, partial, or minimal capacity."
    ),
    "document_extraction": (
        "This is a page from a Vietnamese annual report. "
```

```
        "Extract the following if present: "
        "total revenue (doanh thu), net income (lợi nhuận ròng), "
        "total assets (tổng tài sản). "
        "Report values in billions VND."
    ),
    "construction_progress": (
        "Compare this aerial image to a baseline. "
        "Estimate the percentage completion of visible "
        "construction projects. Note any new structures, "
        "cleared land, or infrastructure changes."
    )
}
```

107.7.2 Retrieval-Augmented Multimodal Analysis

For complex financial questions, we can combine VLM capabilities with retrieval from structured databases. The pipeline:

1. **Query**: Analyst asks "Is Vingroup's construction activity in Vinhomes Grand Park accelerating?"
2. **Retrieve**: Fetch satellite time series, financial statements, news articles
3. **Process**: VLM analyzes satellite images; NLP processes text; tabular model processes financials
4. **Fuse**: Aggregate evidence across modalities
5. **Answer**: Generate a structured response with confidence scores and supporting evidence

```
class MultimodalRAG:
    """
    Retrieval-Augmented Generation with multimodal evidence.
    """

    def __init__(self, datacore_client, vlm_model=None):
        self.dc = datacore_client
        self.vlm = vlm_model

    def retrieve_evidence(self, ticker, date, modalities=None):
        """
        Retrieve all available evidence for a firm at a given date.
        """
        evidence = {}
```

```
        if modalities is None or "tabular" in modalities:
            evidence["tabular"] = self.dc.get_firm_financials(
                ticker=ticker,
                end_date=date,
                n_quarters=4
            )

        if modalities is None or "text" in modalities:
            evidence["text"] = self.dc.get_news(
                ticker=ticker,
                start_date=pd.to_datetime(date) - pd.Timedelta(days=30),
                end_date=date,
                limit=20
            )

        if modalities is None or "image" in modalities:
            evidence["image"] = self.dc.get_satellite_images(
                ticker=ticker,
                date=date,
                lookback_months=6
            )

        if modalities is None or "ts" in modalities:
            evidence["ts"] = self.dc.get_daily_returns(
                ticker=ticker,
                start_date=pd.to_datetime(date) - pd.Timedelta(days=90),
                end_date=date
            )

        return evidence

    def analyze(self, ticker, date, question):
        """
        Full multimodal analysis pipeline.
        """
        evidence = self.retrieve_evidence(ticker, date)

        analysis = {
            "ticker": ticker,
            "date": date,
            "question": question,
            "evidence_available": list(evidence.keys()),
            "modality_signals": {}
```

```
    }

    # Tabular signal
    if "tabular" in evidence and evidence["tabular"] is not None:
        latest = evidence["tabular"].iloc[-1]
        analysis["modality_signals"]["tabular"] = {
            "revenue_growth": latest.get("revenue_growth", None),
            "roe": latest.get("roe", None),
            "leverage": latest.get("leverage", None)
        }

    # Text signal
    if "text" in evidence and evidence["text"] is not None:
        # Aggregate sentiment from PhoBERT
        texts = evidence["text"]
        if len(texts) > 0:
            avg_sentiment = texts["sentiment_score"].mean()
            analysis["modality_signals"]["text"] = {
                "avg_sentiment": avg_sentiment,
                "n_articles": len(texts),
                "sentiment_trend": (
                    "improving" if
                    ↪  texts["sentiment_score"].is_monotonic_increasing
                    else "deteriorating" if
                    ↪  texts["sentiment_score"].is_monotonic_decreasing
                    else "mixed"
                )
            }

    # Time series signal
    if "ts" in evidence and evidence["ts"] is not None:
        ts = evidence["ts"]
        analysis["modality_signals"]["ts"] = {
            "return_60d": (1 + ts["ret"]).prod() - 1,
            "volatility": ts["ret"].std() * np.sqrt(252),
            "avg_turnover": ts["turnover"].mean()
        }

    return analysis
```

107.8 Evaluation and Deployment Considerations

107.8.1 Evaluation Protocol for Multimodal Financial Models

Standard machine learning evaluation (random train/test split) is inappropriate for financial prediction. We require time-series-aware evaluation that respects the temporal ordering of information.

Table 107.5: Evaluation Best Practices for Multimodal Finance Models

Evaluation Aspect	Correct Approach	Common Mistake
Train/test split	Expanding or rolling time window	Random split (look-ahead bias)
Feature timing	Features available before prediction date	Using concurrent or future information
Missing modalities	Test with realistic missingness patterns	Complete-case only
Performance metric	OOS R^2, IC, Sharpe of L-S portfolio	In-sample R^2
Statistical inference	Diebold and Mariano (2002) test for forecast comparison	Point estimates without SE
Economic significance	Transaction-cost-adjusted portfolio returns	Ignoring implementation costs

107.8.2 Computational Budget

Multimodal models are computationally expensive. Table 107.6 provides order-of-magnitude estimates for Vietnamese equity markets.

Table 107.6: Computational Budget for Multimodal Pipeline

Component	Single Firm-Quarter	Full Panel (1000 firms $\times$ 40 quarters)
PhoBERT text encoding	0.5s	~5.5 hours
ResNet50 satellite feature	0.1s	~1.1 hours
Time series encoding (CNN)	0.01s	~7 minutes
Tabular preprocessing	<0.01s	~1 minute
Cross-attention fusion (forward)	0.05s	~33 minutes
Training (50 epochs)	–	~12 hours (GPU)
Full pipeline	–	~1 day (single GPU)

The practical implication is that pre-computation of unimodal embeddings is essential. Extract and cache PhoBERT embeddings, CNN features, and time-series representations once; reuse them across all fusion experiments. Only the fusion layers need retraining when the architecture changes.

107.9 Summary

This chapter developed the multimodal learning framework for Vietnamese financial markets, progressing from foundational representation alignment through production-ready fusion architectures.

The key contributions are threefold. First, we demonstrated that financial data is inherently multimodal and that effective fusion requires explicit architectural choices (e.g., contrastive alignment, cross-attention mechanisms, and missing-modality handling) rather than naive concatenation. The FinancialCLIP alignment framework learns a shared embedding space where text, image, tabular, and time-series representations are geometrically comparable, enabling cross-modal retrieval and transfer.

Second, we built and compared five fusion architectures (early, late with gating, cross-attention, robust with modality dropout, and the custom earnings surprise model) on the prediction of forward returns and earnings surprises. The cross-attention architecture with modality dropout consistently outperforms unimodal baselines and simpler fusion strategies, though the margin varies across prediction horizons and firm characteristics.

Third, we showed how large vision-language models can perform zero-shot financial analysis on Vietnamese documents and satellite imagery, offering a path to multimodal analysis without task-specific training. The retrieval-augmented multimodal pipeline combines the strengths of structured retrieval (from DataCore.vn) with the reasoning capabilities of VLMs.

The practical lesson for researchers working with Vietnamese financial data is that multimodal fusion is most valuable when modalities are complementary: text captures management intent and market narrative, images capture physical economic activity, tabular data provides precise quantitative snapshots, and time series captures market dynamics. When a single modality already captures most of the relevant signal (as tabular features do for many standard prediction tasks), the marginal gain from fusion is modest. When the prediction task requires information that no single modality captures well (as earnings surprises require both quantitative and qualitative assessment), multimodal models provide their largest advantage.

108 Conclusion

Empirical finance in emerging and frontier markets is often judged less by the elegance of an estimator than by the credibility of its inputs and the transparency of its decisions. Vietnam makes this point vividly: trading venues and regulatory regimes have evolved quickly, firm coverage can be uneven across time, corporate actions need careful treatment, and accounting conventions require close attention to timing and comparability. Those features do not prevent high-quality research; they simply shift the center of gravity toward *reproducible data engineering*, *auditable transformations*, and *clear identification of assumptions*.

108.1 What you should take away

108.1.1 Reproducibility is an identification strategy

In textbook settings, identification focuses on variation and exogeneity. In real-world market data, identification also depends on whether your dataset is *the same dataset* when you rerun the work next month or next year. The practical discipline of versioned inputs, deterministic transformations, and documented filters reduces the scope for accidental p-hacking and silent sample drift (e.g., survivorship bias from symbol changes or late-arriving delistings). Reproducible workflows are not administrative overhead; they are a commitment device that makes results more trustworthy and easier to challenge constructively (Peng 2011; Sandve et al. 2013).

108.1.2 Vietnam rewards "microstructure humility"

The chapters on returns, beta estimation, and factor construction emphasized that naïve carryover of developed-market defaults can be costly. Thin trading, price limits, lot-size rules, and regime changes mean that decisions like (i) return interval, (ii) stale-price handling, (iii) corporate-action adjustment, and (iv) portfolio formation frequency can materially change inference. This is not a Vietnam-only phenomenon, but it is more visible there, and therefore a useful laboratory for best practices in emerging markets.

108.2 A reproducibility checklist you can actually use

The list below is designed to be operational: each item can be verified in a repository review.

Table 108.1: Reproducibility deliverables for research

Deliverable	What "done" looks like	Where it lives
Deterministic transforms	Same raw inputs yield identical normalized outputs	`R/transform_*.R` (or `python/transform_*.py`)
Test suite	Coverage, identity, and corporate-action tests run in CI	`tests/` + CI config
Data dictionary	Tables/fields documented with units, timing, and keys	`docs/dictionary.qmd`
Research log	All key design choices recorded (filters, winsorization, periods)	`notes/research_log.md`
Artifact registry	Every figure/table has a script and a checksum	`artifacts/manifest.json`

109 Closing perspective

Vietnam is not "hard mode" finance; it is *real mode* finance. The market's growth, institutional evolution, and data idiosyncrasies force the habits that modern empirical finance increasingly requires everywhere: transparent datasets, careful treatment of identities and corporate actions, and codebases that can be rerun and audited.

References

Abarbanell, Jeffery S, William N Lanen, and Robert E Verrecchia. 1995. "Analysts' Forecasts as Proxies for Investor Beliefs in Empirical Research." *Journal of Accounting and Economics* 20 (1): 31–60.

Abel, Andrew B. 1983. "Optimal Investment Under Uncertainty." *The American Economic Review* 73 (1): 228–33.

Abel, Andrew B, and Janice C Eberly. 1994. "A Unified Model of Investment Under Uncertainty." *American Economic Review* 84 (5).

Abel, Andrew B, and Frederic S Mishkin. 1983. "An Integrated View of Tests of Rationality, Market Efficiency and the Short-Run Neutrality of Monetary Policy." *Journal of Monetary Economics* 11 (1): 3–24.

Acemoglu, Daron, Asuman Ozdaglar, and Alireza Tahbaz-Salehi. 2015. "Systemic Risk and Stability in Financial Networks." *American Economic Review* 105 (2): 564–608.

Acharya, Viral V, and Lasse Heje Pedersen. 2005. "Asset Pricing with Liquidity Risk." *Journal of Financial Economics* 77 (2): 375–410.

Ackerberg, Daniel A., Kevin Caves, and Garth Frazer. 2015. "Identification Properties of Recent Production Function Estimators." *Econometrica* 83 (6): 2411–51. https://doi.org/10.3982/ECTA13408.

Adão, Rodrigo, Michal Kolesár, and Eduardo Morales. 2019. "Shift-Share Designs: Theory and Inference." *The Quarterly Journal of Economics* 134 (4): 1949–2010. https://doi.org/10.1093/qje/qjz025.

Aggarwal, Reena, Isil Erel, Miguel Ferreira, and Pedro Matos. 2011. "Does Governance Travel Around the World? Evidence from Institutional Investors." *Journal of Financial Economics* 100 (1): 154–81.

Aharony, Joseph, and Itzhak Swary. 1980. "Quarterly Dividend and Earnings Announcements and Stockholders' Returns: An Empirical Analysis." *The Journal of Finance* 35 (1): 1–12.

Aït-Sahalia, Yacine, Per A. Mykland, and Lan Zhang. 2005. "How Often to Sample a Continuous-Time Process in the Presence of Market Microstructure Noise." *The Review of Financial Studies* 18 (2): 351–416. https://doi.org/10.1093/rfs/hhi016.

Alayrac, Jean-Baptiste, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, et al. 2022. "Flamingo: A Visual Language Model for Few-Shot Learning." *Advances in Neural Information Processing Systems* 35: 23716–36.

Alexander, Gordon J, Gjergji Cici, and Scott Gibson. 2007. "Does Motivation Matter When Assessing Trade Performance? An Analysis of Mutual Funds." *The Review of Financial Studies* 20 (1): 125–50.

Alexandridis, George, Antonios Antoniou, and Dimitris Petmezas. 2007. "Divergence of Opinion and Post-Acquisition Performance." *Journal of Business Finance & Accounting* 34 (3-4): 439–60.

Allen, Franklin, and Douglas Gale. 2000. "Financial Contagion." *Journal of Political Economy* 108 (1): 1–33.

Allen, Linda, Lin Peng, and Yu Shan. 2026. "Social Networks as Information Conduits for Online Credit Supply and Demand." *Management Science*. https://doi.org/10.1287/mnsc.2024.06050.

Almgren, Robert, and Neil Chriss. 2001. "Optimal Execution of Portfolio Transactions." *Journal of Risk* 3: 5–40.

Altman, Edward I. 1968. "Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy." *The Journal of Finance* 23 (4): 589–609.

Altman, Edward I, and Edith Hotchkiss. 2010. *Corporate Financial Distress and Bankruptcy: Predict and Avoid Bankruptcy, Analyze and Invest in Distressed Debt*. Vol. 289. John Wiley & Sons.

Amihud, Yakov. 2002. "Illiquidity and Stock Returns: Cross-Section and Time-Series Effects." *Journal of Financial Markets* 5 (1): 31–56. https://doi.org/10.1016/S1386-4181(01)00024-6.

Amihud, Yakov, Allaudeen Hameed, Wenjin Kang, and Huiping Zhang. 2015. "The Illiquidity Premium: International Evidence." *Journal of Financial Economics* 117 (2): 350–68. https://doi.org/10.1016/j.jfineco.2015.04.005.

Amihud, Yakov, and Haim Mendelson. 1986. "Asset Pricing and the Bid-Ask Spread." *Journal of Financial Economics* 17 (2): 223–49.

Amihud, Yakov, Haim Mendelson, and Lasse Heje Pedersen. 2005. "Liquidity and Asset Prices." *Foundations and Trends in Finance* 1 (4): 269–364. https://doi.org/10.1561/0500000003.

Amihud, Yakov, and Joonki Noh. 2021. "Illiquidity and Stock Returns II: Cross-Section and Time-Series Effects." *The Review of Financial Studies* 34 (4): 2101–23. https://doi.org/10.1093/rfs/hhaa080.

Amornsiripanitch, Natee. 2026. "The Age Gap in Mortgage Access." *Management Science*. https://doi.org/10.1287/mnsc.2024.08264.

Andersen, Torben G., Tim Bollerslev, Francis X. Diebold, and Paul Labys. 2003. "Modeling and Forecasting Realized Volatility." *Econometrica* 71 (2): 579–625. https://doi.org/10.1111/1468-0262.00418.

Andersen, Torben G., Dobrislav Dobrev, and Ernst Schaumburg. 2012. "Jump-Robust Volatility Estimation Using Nearest Neighbor Truncation." *Journal of Econometrics* 169 (1): 75–93. https://doi.org/10.1016/j.jeconom.2012.01.011.

Anderson, Anne-Marie, and Edward A Dyl. 2005. "Market Structure and Trading Volume." *Journal of Financial Research* 28 (1): 115–31.

Anderson, Kirsten L, Jeffrey H Harris, and Eric C So. 2007. "Opinion Divergence and Post-Earnings Announcement Drift." *Available at SSRN 969736*.

Andrade, Gregor, Mark Mitchell, and Erik Stafford. 2001. "New Evidence and Perspectives on Mergers." *Journal of Economic Perspectives* 15 (2): 103–20.

Ang, Andrew, and Joseph Chen. 2002. "Asymmetric Correlations of Equity Portfolios." *Journal of Financial Economics* 63 (3): 443–94.

Ang, Andrew, Robert J Hodrick, Yuhang Xing, and Xiaoyan Zhang. 2006. "The Cross-Section of Volatility and Expected Returns." *The Journal of Finance* 61 (1): 259–99.

Arnott, Robert D, Jason Hsu, and Philip Moore. 2005. "Fundamental Indexation." *Financial Analysts Journal* 61 (2): 83–99.

Artzner, Philippe, Freddy Delbaen, Jean-Marc Eber, and David Heath. 1999. "Coherent Measures of Risk." *Mathematical Finance* 9 (3): 203–28.

Atiase, Rowland Kwame. 1985. "Predisclosure Information, Firm Capitalization, and Security Price Behavior Around Earnings Announcements." *Journal of Accounting Research*, 21–36.

Badertscher, Brad A, Bjørn N Jørgensen, Sharon P Katz, and Jeremy Michels. 2026. "Cheap Stock Options: Antecedents and Outcomes." *Management Science*. https://doi.org/10.1287/mnsc.2024.06121.

Baek, Jeonghun, Geewook Kim, Junyeop Lee, Sungrae Park, Dongyoon Han, Sangdoo Yun, Seong Joon Oh, and Hwalsuk Lee. 2019. "What Is Wrong with Scene Text Recognition Model Comparisons? Dataset and Model Analysis." In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 4715–23.

Bailey, Michael, Rachel Cao, Theresa Kuchler, Johannes Stroebel, and Arlene Wong. 2018. "Social Connectedness: Measurement, Determinants, and Effects." *Journal of Economic Perspectives* 32 (3): 259–80. https://doi.org/10.1257/jep.32.3.259.

Bailey, Michael, Ruiqing Cao, Theresa Kuchler, and Johannes Stroebel. 2018. "The Economic Effects of Social Networks: Evidence from the Housing Market." *Journal of Political Economy* 126 (6): 2224–76. https://doi.org/10.1086/700073.

Baker, Malcolm, Jeremy C Stein, and Jeffrey Wurgler. 2003. "When Does the Market Matter? Stock Prices and the Investment of Equity-Dependent Firms." *The Quarterly Journal of Economics* 118 (3): 969–1005.

Baker, Malcolm, and Jeffrey Wurgler. 2002. "Market Timing and Capital Structure." *The Journal of Finance* 57 (1): 1–32.

Bali, Turan G, Robert F Engle, and Scott Murray. 2016a. *Empirical Asset Pricing: The Cross Section of Stock Returns*. John Wiley & Sons.

———. 2016b. *Empirical asset pricing: The cross section of stock returns*. John Wiley & Sons. https://doi.org/10.1002/9781118445112.stat07954.

Ball, Ray, and Philip Brown. 2013. "An Empirical Evaluation of Accounting Income Numbers." In *Financial Accounting and Equity Markets*, 27–46. Routledge.

Ball, Ray, Ashok Robin, and Joanna Shuang Wu. 2003. "Incentives Versus Standards: Properties of Accounting Income in Four East Asian Countries." *Journal of Accounting and Economics* 36 (1-3): 235–70. https://doi.org/10.1016/j.jacceco.2003.10.003.

Ball, Ray, and Lakshmanan Shivakumar. 2008. "How Much New Information Is There in Earnings?" *Journal of Accounting Research* 46 (5): 975–1016.

Ball, Ryan T., and Eric Ghysels. 2018. "Automated Earnings Forecasts: Beat Analysts or Combine and Conquer?" *Management Science* 64 (10): 4936–52. https://doi.org/10.1287/mnsc.2017.2864.

Baltrušaitis, Tadas, Chaitanya Ahuja, and Louis-Philippe Morency. 2018. "Multimodal Ma-

chine Learning: A Survey and Taxonomy." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 41 (2): 423–43.

Bamber, Linda Smith, Theodore E Christensen, and Kenneth M Gaver. 2000. "Do We Really 'Know'what We Think We Know? A Case Study of Seminal Research and Its Subsequent Overgeneralization." *Accounting, Organizations and Society* 25 (2): 103–29.

Bandi, Federico M., and Jeffrey R. Russell. 2008. "Microstructure Noise, Realized Variance, and Optimal Sampling." *The Review of Economic Studies* 75 (2): 339–69. https://doi.org/10.1111/j.1467-937X.2008.00474.x.

Banerjee, Abhijit V. 1992. "A Simple Model of Herd Behavior." *The Quarterly Journal of Economics* 107 (3): 797–817.

Bao Dinh, Ngoc, and Van Nguyen Hong Tran. 2024. "Institutional Ownership and Stock Liquidity: Evidence from an Emerging Market." *SAGE Open* 14 (1): 21582440241239116.

Barber, Brad M, Yi-Tsung Lee, Yu-Jane Liu, and Terrance Odean. 2009. "Just How Much Do Individual Investors Lose by Trading?" *The Review of Financial Studies* 22 (2): 609–32.

Barber, Brad M, and John D Lyon. 1997. "Detecting Long-Run Abnormal Stock Returns: The Empirical Power and Specification of Test Statistics." *Journal of Financial Economics* 43 (3): 341–72.

Barberis, Nicholas, Andrei Shleifer, and Robert Vishny. 1998. "A Model of Investor Sentiment." *Journal of Financial Economics* 49 (3): 307–43.

Barillas, Francisco, and Jay Shanken. 2018. "Comparing Asset Pricing Models." *The Journal of Finance* 73 (2): 715–54.

Barndorff-Nielsen, Ole E., Peter Reinhard Hansen, Asger Lunde, and Neil Shephard. 2008. "Designing Realized Kernels to Measure the Ex Post Variation of Equity Prices in the Presence of Noise." *Econometrica* 76 (6): 1481–1536. https://doi.org/10.3982/ECTA6495.

Barndorff-Nielsen, Ole E., and Neil Shephard. 2002. "Econometric Analysis of Realized Volatility and Its Use in Estimating Stochastic Volatility Models." *Journal of the Royal Statistical Society Series B: Statistical Methodology* 64 (2): 253–80. https://doi.org/10.1111/1467-9868.00336.

———. 2004. "Power and Bipower Variation with Stochastic Volatility and Jumps." *Journal of Financial Econometrics* 2 (1): 1–37. https://doi.org/10.1093/jjfinec/nbh001.

Barth, Mary E, Wayne R Landsman, and Mark H Lang. 2008. "International Accounting Standards and Accounting Quality." *Journal of Accounting Research* 46 (3): 467–98.

Bartov, Eli, Lucile Faurel, and Partha S Mohanram. 2018. "Can Twitter Help Predict Firm-Level Earnings and Stock Returns?" *The Accounting Review* 93 (3): 25–57.

Beaver, William H. 1968. "The Information Content of Annual Earnings Announcements." *Journal of Accounting Research*, 67–92.

Bebchuk, Lucian A. 1999. "A Rent-Protection Theory of Corporate Ownership and Control." National Bureau of Economic Research Cambridge, Mass., USA.

Bebchuk, Lucian A, Alma Cohen, and Charles CY Wang. 2013. "Learning and the Disappearing Association Between Governance and Returns." *Journal of Financial Economics* 108 (2): 323–48.

Bebchuk, Lucian, Alma Cohen, and Allen Ferrell. 2009. "What Matters in Corporate Governance?" *The Review of Financial Studies* 22 (2): 783–827.

Bekaert, Geert, and Campbell R Harvey. 1995. "Time-Varying World Market Integration." *The Journal of Finance* 50 (2): 403–44.

———. 2002. "Research in Emerging Markets Finance: Looking to the Future." *Emerging Markets Review* 3 (4): 429–48.

Bekaert, Geert, Campbell R Harvey, and Christian Lundblad. 2005. "Does Financial Liberalization Spur Growth?" *Journal of Financial Economics* 77 (1): 3–55.

Ben-David, ITZHAK, Francesco Franzoni, Augustin Landier, and Rabih Moussawi. 2013. "Do Hedge Funds Manipulate Stock Prices?" *The Journal of Finance* 68 (6): 2383–2434.

Ben-David, Itzhak, Francesco Franzoni, and Rabih Moussawi. 2012. "Hedge Fund Stock Trading in the Financial Crisis of 2007–2009." *The Review of Financial Studies* 25 (1): 1–54. https://doi.org/10.1093/rfs/hhr114.

Benjamini, Yoav, and Yosef Hochberg. 1995. "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing." *Journal of the Royal Statistical Society: Series B (Methodological)* 57 (1): 289–300.

Benveniste, Lawrence M, and Paul A Spindt. 1989. "How Investment Bankers Determine the Offer Price and Allocation of New Issues." *Journal of Financial Economics* 24 (2): 343–61.

Berger, Elizabeth A. 2023. "Selection Bias in Mutual Fund Fire Sales." *Journal of Financial and Quantitative Analysis* 58 (3): 1039–77. https://doi.org/10.1017/S0022109022001223.

Bergstresser, Daniel, John M R Chalmers, and Peter Tufano. 2009. "Assessing the Costs and Benefits of Brokers in the Mutual Fund Industry." *The Review of Financial Studies* 22 (10): 4129–56.

Berkman, Henk, Valentin Dimitrov, Prem C Jain, Paul D Koch, and Sheri Tice. 2009. "Sell on the News: Differences of Opinion, Short-Sales Constraints, and Returns Around Earnings Announcements." *Journal of Financial Economics* 92 (3): 376–99.

Bernard, Victor L, and Jacob K Thomas. 1989. "Post-Earnings-Announcement Drift: Delayed Price Response or Risk Premium?" *Journal of Accounting Research* 27: 1–36. https://doi.org/10.2307/2491062.

Bernstein, Shai, Emanuele Colonnelli, and Benjamin Iverson. 2019. "Asset Allocation in Bankruptcy." *The Journal of Finance* 74 (1): 5–53. https://doi.org/10.1111/jofi.12740.

Berry, Steven, James Levinsohn, and Ariel Pakes. 1995. "Automobile Prices in Market Equilibrium." *Econometrica* 63 (4): 841–90.

Bertomeu, Jeremy. 2020. "Machine Learning Improves Accounting: Discussion, Implementation and Research Opportunities." *Review of Accounting Studies* 25 (3): 1135–55. https://doi.org/10.1007/s11142-020-09554-9.

Bessembinder, Hendrik. 2003. "Trade Execution Costs and Market Quality After Decimalization." *Journal of Financial and Quantitative Analysis* 38 (4): 747–77.

Beyer, Anne, Daniel A Cohen, Thomas Z Lys, and Beverly R Walther. 2010. "The Financial Reporting Environment: Review of the Recent Literature." *Journal of Accounting and Economics* 50 (2-3): 296–343.

Bhattacharya, Sudipto. 1979. "Imperfect Information, Dividend Policy, and" the Bird in the Hand" Fallacy." *The Bell Journal of Economics*, 259–70.

Bhattacharya, Utpal, Hazem Daouk, Brian Jorgenson, and Carl-Heinrich Kehr. 2000. "When an Event Is Not an Event: The Curious Case of an Emerging Market." *Journal of Financial*

Economics 55 (1): 69–101.

Biddle, Gary C, Gilles Hilary, and Rodrigo S Verdi. 2009. "How Does Financial Reporting Quality Relate to Investment Efficiency?" *Journal of Accounting and Economics* 48 (2-3): 112–31.

Bikhchandani, Sushil, David Hirshleifer, and Ivo Welch. 1992. "A Theory of Fads, Fashion, Custom, and Cultural Change as Informational Cascades." *Journal of Political Economy* 100 (5): 992–1026.

Binder, John. 1998. "The Event Study Methodology Since 1969." *Review of Quantitative Finance and Accounting* 11 (2): 111–37.

Binsbergen, Jules H. van, Xiao Han, and Alejandro Lopez-Lira. 2023. "Man Versus Machine Learning: The Term Structure of Earnings Expectations and Conditional Biases." *The Review of Financial Studies* 36 (6): 2361–96. https://doi.org/10.1093/rfs/hhac085.

Binsbergen, Jules H. van, Liang Ma, and Michael Schwert. 2026. "The Factor Multiverse: The Role of Interest Rates in Factor Return Measurement." *Management Science*. https://doi.org/10.1287/mnsc.2024.08851.

Bizjak, John M, Michael L Lemmon, and Lalitha Naveen. 2008. "Does the Use of Peer Groups Contribute to Higher Pay and Less Efficient Compensation?" *Journal of Financial Economics* 90 (2): 152–68.

Blankespoor, Elizabeth, Gregory S Miller, and Hal D White. 2014. "The Role of Dissemination in Market Liquidity: Evidence from Firms' Use of Twitter." *The Accounting Review* 89 (1): 79–112.

Blei, David M, Andrew Y Ng, and Michael I Jordan. 2003. "Latent Dirichlet Allocation." *Journal of Machine Learning Research* 3 (Jan): 993–1022.

Boehme, Rodney D, Bartley R Danielsen, and Sorin M Sorescu. 2006. "Short-Sale Constraints, Differences of Opinion, and Overvaluation." *Journal of Financial and Quantitative Analysis* 41 (2): 455–87.

Boehmer, Ekkehart, Jim Masumeci, and Annette B Poulsen. 1991. "Event-Study Methodology Under Conditions of Event-Induced Variance." *Journal of Financial Economics* 30 (2): 253–72.

Bollen, Nicolas PB, Tom Smith, and Robert E Whaley. 2004. "Modeling the Bid/Ask Spread: Measuring the Inventory-Holding Premium." *Journal of Financial Economics* 72 (1): 97–141.

Bollerslev, Tim. 1986. "Generalized Autoregressive Conditional Heteroskedasticity." *Journal of Econometrics* 31 (3): 307–27.

Bond, Philip, Alex Edmans, and Itay Goldstein. 2012. "The Real Effects of Financial Markets." *Annu. Rev. Financ. Econ.* 4 (1): 339–60.

Bond, Stephen, Julie Ann Elston, Jacques Mairesse, and Benoît Mulkay. 2003. "Financial Factors and Investment in Belgium, France, Germany, and the United Kingdom: A Comparison Using Company Panel Data." *Review of Economics and Statistics* 85 (1): 153–65.

Bonferroni, Carlo. 1936. "Teoria Statistica Delle Classi e Calcolo Delle Probabilita." *Pubblicazioni Del R Istituto Superiore Di Scienze Economiche e Commericiali Di Firenze* 8: 3–62.

Bonsall IV, Samuel B, Andrew J Leone, Brian P Miller, and Kristina Rennekamp. 2017. "A

Plain English Measure of Financial Reporting Readability." *Journal of Accounting and Economics* 63 (2-3): 329–57.

Bordalo, Pedro, Nicola Gennaioli, Rafael La Porta, and Andrei Shleifer. 2019. "Diagnostic Expectations and Stock Returns." *The Journal of Finance* 74 (6): 2839–74. https://doi.org/10.1111/jofi.12833.

Borusyak, Kirill, Peter Hull, and Xavier Jaravel. 2022. "Quasi-Experimental Shift-Share Research Designs." *The Review of Economic Studies* 89 (1): 181–213. https://doi.org/10.1093/restud/rdab030.

Botosan, Christine A. 1997. "Disclosure Level and the Cost of Equity Capital." *Accounting Review*, 323–49.

Botosan, Christine A, and Marlene A Plumlee. 2002. "A Re-Examination of Disclosure Level and the Expected Cost of Equity Capital." *Journal of Accounting Research* 40 (1): 21–40.

Bradshaw, Mark T., Michael S. Drake, James N. Myers, and Linda A. Myers. 2012. "A Re-Examination of Analysts' Superiority over Time-Series Forecasts of Annual Earnings." *Review of Accounting Studies* 17 (4): 944–68. https://doi.org/10.1007/s11142-012-9185-8.

Brandt, Michael W., Pedro Santa-Clara, and Rossen Valkanov. 2009. "Parametric Portfolio Policies: Exploiting Characteristics in the Cross-Section of Equity Returns." *The Review of Financial Studies* 22 (9): 3411–47. https://doi.org/10.1093/rfs/hhp003.

Brennan, Michael J. 1986. "A Theory of Price Limits in Futures Markets." *Journal of Financial Economics* 16 (2): 213–33.

Brennan, Michael J., and Avanidhar Subrahmanyam. 1996. "Market Microstructure and Asset Pricing: On the Compensation for Illiquidity in Stock Returns." *Journal of Financial Economics* 41 (3): 441–64. https://doi.org/10.1016/0304-405X(95)00870-K.

Bris, Arturo, Ivo Welch, and Ning Zhu. 2006. "The Costs of Bankruptcy: Chapter 7 Liquidation Versus Chapter 11 Reorganization." *The Journal of Finance* 61 (3): 1253–303. https://doi.org/10.1111/j.1540-6261.2006.00872.x.

Brown, Stephen J, and Jerold B Warner. 1980. "Measuring Security Price Performance." *Journal of Financial Economics* 8 (3): 205–58.

———. 1985. "Using Daily Stock Returns: The Case of Event Studies." *Journal of Financial Economics* 14 (1): 3–31.

Brunnermeier, Markus K. 2009. "Deciphering the Liquidity and Credit Crunch 2007–2008." *Journal of Economic Perspectives* 23 (1): 77–100.

Brunnermeier, Markus K, Stefan Nagel, and Lasse H Pedersen. 2008. "Carry Trades and Currency Crashes." *NBER Macroeconomics Annual* 23 (1): 313–48.

Brunnermeier, Markus K, and Lasse Heje Pedersen. 2009. "Market Liquidity and Funding Liquidity." *The Review of Financial Studies* 22 (6): 2201–38.

Brunnermeier, Markus K., and Yuliy Sannikov. 2014. "A Macroeconomic Model with a Financial Sector." *American Economic Review* 104 (2): 379–421. https://doi.org/10.1257/aer.104.2.379.

Burgstahler, David, and Ilia Dichev. 1997. "Earnings Management to Avoid Earnings Decreases and Losses." *Journal of Accounting and Economics* 24 (1): 99–126.

Bushman, Robert, Qi Chen, Ellen Engel, and Abbie Smith. 2004. "Financial Accounting Information, Organizational Complexity and Corporate Governance Systems." *Journal of*

Accounting and Economics 37 (2): 167–201.

Bybee, Leland, Bryan Kelly, and Yinan Su. 2023. "Narrative Asset Pricing: Interpretable Systematic Risk Factors from News Text." *The Review of Financial Studies* 36 (12): 4759–87.

Cai, Ye, Dan S Dhaliwal, Yongtae Kim, and Carrie Pan. 2014. "Board Interlocks and the Diffusion of Disclosure Policy." *Review of Accounting Studies* 19 (3): 1086–1119.

Callaway, Brantly, and Pedro H. C. Sant'Anna. 2021. "Difference-in-Differences with Multiple Time Periods." *Journal of Econometrics* 225 (2): 200–230. https://doi.org/10.1016/j.jeconom.2020.12.001.

Calonico, Sebastian, Matias D. Cattaneo, and Rocío Titiunik. 2014. "Robust Nonparametric Confidence Intervals for Regression-Discontinuity Designs." *Econometrica* 82 (6): 2295–2326. https://doi.org/10.3982/ECTA11757.

Campbell, John L., Harrison Ham, Zhongjin (Gene) Lu, and Katherine Wood. 2026. "Expectations Matter: When (Not) to Use Machine Learning Earnings Forecasts." *Management Science* 72 (10): 8863–87. https://doi.org/10.1287/mnsc.2024.05808.

Campbell, John Y, Andrew W Lo, A Craig MacKinlay, and Robert F Whitelaw. 1998. "The Econometrics of Financial Markets." *Macroeconomic Dynamics* 2 (4): 559–62.

Campbell, John Y, Karine Serfaty-De Medeiros, and Luis M Viceira. 2010. "Global Currency Hedging." *The Journal of Finance* 65 (1): 87–121.

Cao, Kai, and Haifeng You. 2024. "Fundamental Analysis via Machine Learning." *Financial Analysts Journal* 80 (2): 74–98. https://doi.org/10.1080/0015198X.2024.2313692.

Cao, Sean, Zhi Da, Daniel Jiang, and Baozhong Yang. 2026. "Do Hedge Funds Strategically Misreport Their Holdings? Evidence from 13F Restatements." *Management Science*. https://doi.org/10.1287/mnsc.2024.08833.

Carhart, Mark M. 1997a. "On Persistence in Mutual Fund Performance." *The Journal of Finance* 52 (1): 57–82. https://doi.org/10.1111/j.1540-6261.1997.tb03808.x.

Carhart, Mark M. 1997b. "On persistence in mutual fund performance." *The Journal of Finance* 52 (1): 57–82. https://doi.org/10.1111/j.1540-6261.1997.tb03808.x.

Chambers, Anne E, and Stephen H Penman. 1984. "Timeliness of Reporting and the Stock Price Reaction to Earnings Announcements." *Journal of Accounting Research*, 21–47.

Chan, Kalok, Allaudeen Hameed, and Wilson Tong. 2000. "Profitability of Momentum Strategies in the International Equity Markets." *Journal of Financial and Quantitative Analysis*, 153–72.

Chatterjee, Sris, Kose John, and An Yan. 2012. "Takeovers and Divergence of Investor Opinion." *The Review of Financial Studies* 25 (1): 227–77.

Chen, Andrew, and Tom Zimmermann. 2022. "Open Source Cross-Sectional Asset Pricing." *> Critical Finance Review* 11 (02): 207–64.

Chen, Hailiang, Prabuddha De, Yu Jeffrey Hu, and Byoung-Hyoun Hwang. 2014. "Wisdom of Crowds: The Value of Stock Opinions Transmitted Through Social Media." *The Review of Financial Studies* 27 (5): 1367–1403.

Chen, Hsiu-Lang, Narasimhan Jegadeesh, and Russ Wermers. 2000. "The Value of Active Mutual Fund Management: An Examination of the Stockholdings and Trades of Fund Managers." *Journal of Financial and Quantitative Analysis* 35 (3): 343–68.

Chen, Joseph, Harrison Hong, and Jeremy C Stein. 2001. "Forecasting Crashes: Trading Volume, Past Returns, and Conditional Skewness in Stock Prices." *Journal of Financial Economics* 61 (3): 345–81.

———. 2002. "Breadth of Ownership and Stock Returns." *Journal of Financial Economics* 66 (2-3): 171–205. https://doi.org/10.1016/S0304-405X(02)00223-4.

Chen, Jun, Shenje Hshieh, Melvyn Teo, and Feng Zhang. 2026. "Immigrants in Finance: Evidence from Hedge Funds." *Management Science*. https://doi.org/10.1287/mnsc.2024.07615.

Cheong, Foong Soon, and Jacob Thomas. 2011. "Why Do EPS Forecast Error and Dispersion Not Vary with Scale? Implications for Analyst and Managerial Behavior." *Journal of Accounting Research* 49 (2): 359–401.

Cho, David D, Jeffrey Russell, George C Tiao, and Ruey Tsay. 2003. "The Magnet Effect of Price Limits: Evidence from High-Frequency Data on Taiwan Stock Exchange." *Journal of Empirical Finance* 10 (1-2): 133–68.

Choe, Hyuk, Bong-Chan Kho, and René M Stulz. 2005. "Do Domestic Investors Have an Edge? The Trading Experience of Foreign Investors in Korea." *The Review of Financial Studies* 18 (3): 795–829.

Chordia, Tarun, Richard Roll, and Avanidhar Subrahmanyam. 2000. "Commonality in Liquidity." *Journal of Financial Economics* 56 (1): 3–28.

———. 2001. "Market Liquidity and Trading Activity." *The Journal of Finance* 56 (2): 501–30.

Choueifaty, Yves. 2008. "Towards Maximum Diversification." *Available at SSRN 4063676*.

Christoffersen, Peter F. 1998. "Evaluating Interval Forecasts." *International Economic Review*, 841–62.

Christoffersen, Susan E K, Richard Evans, and David K Musto. 2013. "What Do Consumers' Fund Flows Maximize? Evidence from Their Brokers' Incentives." *The Journal of Finance* 68 (1): 201–35.

Chu, Xiaojun, and Jianying Qiu. 2019. "Forecasting Volatility with Price Limit Hits—Evidence from Chinese Stock Market." *Emerging Markets Finance and Trade* 55 (5): 1034–50.

Chui, Andy CW, Sheridan Titman, and KC John Wei. 2010. "Individualism and Momentum Around the World." *The Journal of Finance* 65 (1): 361–92.

Chung, Kee H, and Stephen W Pruitt. 1994. "A Simple Approximation of Tobin's q." *Financial Management*, 70–74.

Chung, Kee H, and Hao Zhang. 2014. "A Simple Approximation of Intraday Spreads Using Daily Data." *Journal of Financial Markets* 17: 94–120.

Claessens, Stijn, Simeon Djankov, Joseph PH Fan, and Larry HP Lang. 2002. "Disentangling the Incentive and Entrenchment Effects of Large Shareholdings." *The Journal of Finance* 57 (6): 2741–71. https://doi.org/10.1111/1540-6261.00511.

Claessens, Stijn, Simeon Djankov, and Larry HP Lang. 2000. "The Separation of Ownership and Control in East Asian Corporations." *Journal of Financial Economics* 58 (1-2): 81–112.

Clarke, Roger, Harindra De Silva, and Steven Thorley. 2011. "Minimum-Variance Portfolio Composition." *Journal of Portfolio Management* 37 (2): 31.

Coad, Alex, Jacob Rubæk Holm, Jackie Krafft, and Francesco Quatraro. 2018. "Firm Age and Performance." *Journal of Evolutionary Economics* 28 (1): 1–11.

Cochrane, John H. 1991. "Production-Based Asset Pricing and the Link Between Stock Returns and Economic Fluctuations." *The Journal of Finance* 46 (1): 209–37.

———. 2011. "Presidential Address: Discount Rates." *The Journal of Finance* 66 (4): 1047–1108.

Cohen, Daniel A, Aiyesha Dey, and Thomas Z Lys. 2008. "Real and Accrual-Based Earnings Management in the Pre-and Post-Sarbanes-Oxley Periods." *The Accounting Review* 83 (3): 757–87.

Cohen, Lauren, and Andrea Frazzini. 2008. "Economic Links and Predictable Returns." *The Journal of Finance* 63 (4): 1977–2011.

Comerton-Forde, Carole, and Kar Mei Tang. 2009. "Anonymity, Liquidity and Fragmentation." *Journal of Financial Markets* 12 (3): 337–67.

Cont, Rama. 2001. "Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues." *Quantitative Finance* 1 (2): 223.

Cooper, Michael J, Huseyin Gulen, and Michael J Schill. 2008. "Asset Growth and the Cross-Section of Stock Returns." *The Journal of Finance* 63 (4): 1609–51.

Cooper, Michael J, Roberto C Gutierrez Jr, and Allaudeen Hameed. 2004. "Market States and Momentum." *The Journal of Finance* 59 (3): 1345–65.

Corrado, Charles J. 1989. "A Nonparametric Test for Abnormal Security-Price Performance in Event Studies." *Journal of Financial Economics* 23 (2): 385–95.

Corwin, Shane A, and Paul Schultz. 2012. "A Simple Way to Estimate Bid-Ask Spreads from Daily High and Low Prices." *The Journal of Finance* 67 (2): 719–60.

Coval, Joshua, and Erik Stafford. 2007. "Asset Fire Sales (and Purchases) in Equity Markets." *Journal of Financial Economics* 86 (2): 479–512.

Cowan, Arnold Richard. 1992. "Nonparametric Event Study Tests." *Review of Quantitative Finance and Accounting* 2 (4): 343–58.

Cremers, KJ Martijn, and Vinay B Nair. 2005. "Governance Mechanisms and Equity Prices." *The Journal of Finance* 60 (6): 2859–94.

Crowley, Richard M, Wenli Huang, and Hai Lu. 2024a. "Discretionary Dissemination on Twitter." *Contemporary Accounting Research* 41 (4): 2454–87.

———. 2024b. "Executive Tweets." *Working Paper*. https://ssrn.com/abstract=3975995.

Daniel, Kent, David Hirshleifer, and Avanidhar Subrahmanyam. 1998. "Investor Psychology and Security Market Under-and Overreactions." *The Journal of Finance* 53 (6): 1839–85.

Daniel, Kent, and Tobias J Moskowitz. 2016. "Momentum Crashes." *Journal of Financial Economics* 122 (2): 221–47.

Daniel, Kent, and Sheridan Titman. 1997. "Evidence on the Characteristics of Cross Sectional Variation in Stock Returns." *The Journal of Finance* 52 (1): 1–33.

Datar, Vinay T, Narayan Y Naik, and Robert Radcliffe. 1998. "Liquidity and Stock Returns: An Alternative Test." *Journal of Financial Markets* 1 (2): 203–19.

Davydenko, Sergei A., and Julian R. Franks. 2008. "Do Bankruptcy Codes Matter? A Study of Defaults in France, Germany, and the UK." *The Journal of Finance* 63 (2): 565–608. https://doi.org/10.1111/j.1540-6261.2008.01325.x.

DeAngelo, Linda Elizabeth. 1986. "Accounting Numbers as Market Valuation Substitutes: A Study of Management Buyouts of Public Stockholders." *Accounting Review*, 400–420.

Dechow, Patricia M, Richard G Sloan, and Amy P Sweeney. 1995. "Detecting Earnings Management." *Accounting Review*, 193–225.

Dechow, Patricia, Weili Ge, and Catherine Schrand. 2010. "Understanding Earnings Quality: A Review of the Proxies, Their Determinants and Their Consequences." *Journal of Accounting and Economics* 50 (2-3): 344–401. https://doi.org/10.1016/j.jacceco.2010.09.001.

DellaVigna, Stefano, and Joshua M Pollet. 2009. "Investor Inattention and Friday Earnings Announcements." *The Journal of Finance* 64 (2): 709–49.

Demarta, Stefano, and Alexander J McNeil. 2005. "The t Copula and Related Copulas." *International Statistical Review* 73 (1): 111–29.

DeMiguel, Victor, Lorenzo Garlappi, and Raman Uppal. 2009. "Optimal Versus Naive Diversification: How Inefficient Is the 1/n Portfolio Strategy?" *The Review of Financial Studies* 22 (5): 1915–53.

Dessaint, Olivier, Thierry Foucault, Laurent Frésard, and Adrien Matray. 2019. "Noisy Stock Prices and Corporate Investment." *The Review of Financial Studies* 32 (7): 2625–72. https://doi.org/10.1093/rfs/hhy115.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. "Bert: Pre-Training of Deep Bidirectional Transformers for Language Understanding." In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–86.

Di Maggio, Marco, and Dimuthu Ratnadiwakara. 2026. "Invisible Primes: Fintech Lending with Alternative Data." *Management Science*. https://doi.org/10.1287/mnsc.2024.07854.

Diamond, Douglas W. 1985. "Optimal Release of Information by Firms." *The Journal of Finance* 40 (4): 1071–94.

Diamond, Douglas W, and Robert E Verrecchia. 1991. "Disclosure, Liquidity, and the Cost of Capital." *The Journal of Finance* 46 (4): 1325–59.

Diebold, Francis X, and Robert S Mariano. 2002. "Comparing Predictive Accuracy." *Journal of Business & Economic Statistics* 20 (1): 134–44.

Diebold, Francis X, and Kamil Yılmaz. 2014. "On the Network Topology of Variance Decompositions: Measuring the Connectedness of Financial Firms." *Journal of Econometrics* 182 (1): 119–34.

Diether, Karl B, Christopher J Malloy, and Anna Scherbina. 2002. "Differences of Opinion and the Cross Section of Stock Returns." *The Journal of Finance* 57 (5): 2113–41.

Dimson, Elroy. 1979. "Risk Measurement When Shares Are Subject to Infrequent Trading." *Journal of Financial Economics* 7 (2): 197–226. https://doi.org/10.1016/0304-405X(79)90013-8.

Donaldson, Dave. 2018. "Railroads of the Raj: Estimating the Impact of Transportation Infrastructure." *American Economic Review* 108 (4-5): 899–934.

Donaldson, Dave, and Adam Storeygard. 2016. "The View from Above: Applications of Satellite Data in Economics." *Journal of Economic Perspectives* 30 (4): 171–98.

Dornbusch, Rudiger. 1976. "Expectations and Exchange Rate Dynamics." *Journal of Political Economy* 84 (6): 1161–76.

Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, et al. 2020. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." *arXiv Preprint arXiv:2010.11929*.

Dou, Winston Wei, Lucian A. Taylor, Wei Wang, and Wenyu Wang. 2021. "Dissecting Bankruptcy Frictions." *Journal of Financial Economics* 142 (3): 975–1000. https://doi.org/10.1016/j.jfineco.2021.05.040.

Doukas, John A, Chansog Francis Kim, and Christos Pantzalis. 2006. "Divergence of Opinion and Equity Returns." *Journal of Financial and Quantitative Analysis* 41 (3): 573–606.

Doukas, John A, Chansog Kim, and Christos Pantzalis. 2004. "Divergent Opinions and the Performance of Value Stocks." *Financial Analysts Journal* 60 (6): 55–64.

Du, Wenxin, Alexander Tepper, and Adrien Verdelhan. 2018. "Deviations from Covered Interest Rate Parity." *The Journal of Finance* 73 (3): 915–57.

Duarte, Jefferson, Stephan Siegel, and Lance Young. 2012. "Trust and Credit: The Role of Appearance in Peer-to-Peer Lending." *The Review of Financial Studies* 25 (8): 2455–84.

Dumas, Bernard, and Bruno Solnik. 1995. "The World Price of Foreign Exchange Risk." *The Journal of Finance* 50 (2): 445–79.

Dyer, Travis, Mark Lang, and Lorien Stice-Lawrence. 2017. "The Evolution of 10-k Textual Disclosure: Evidence from Latent Dirichlet Allocation." *Journal of Accounting and Economics* 64 (2-3): 221–45.

Easley, David, Soeren Hvidkjaer, and Maureen O'hara. 2002. "Is Information Risk a Determinant of Asset Returns?" *The Journal of Finance* 57 (5): 2185–2221.

Easley, David, Nicholas M Kiefer, Maureen O'hara, and Joseph B Paperman. 1996. "Liquidity, Information, and Infrequently Traded Stocks." *The Journal of Finance* 51 (4): 1405–36.

Edmans, Alex, Itay Goldstein, and Wei Jiang. 2012. "The Real Effects of Financial Markets: The Impact of Prices on Takeovers." *The Journal of Finance* 67 (3): 933–71. https://doi.org/10.1111/j.1540-6261.2012.01738.x.

Egan, Mark, Gregor Matvos, and Amit Seru. 2019. "The Market for Financial Adviser Misconduct." *Journal of Political Economy* 127 (1): 233–95. https://doi.org/10.1086/700735.

Eisenberg, Larry, and Thomas H Noe. 2001. "Systemic Risk in Financial Systems." *Management Science* 47 (2): 236–49.

Ellison, Glenn, Edward L. Glaeser, and William R. Kerr. 2010. "What Causes Industry Agglomeration? Evidence from Coagglomeration Patterns." *American Economic Review* 100 (3): 1195–1213. https://doi.org/10.1257/aer.100.3.1195.

Elton, Edwin J, Martin J Gruber, and Christopher R Blake. 2011. "Holdings Data, Security Returns, and the Selection of Superior Mutual Funds." *Journal of Financial and Quantitative Analysis* 46 (2): 341–67.

Engle, Robert. 2002. "Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models." *Journal of Business & Economic Statistics* 20 (3): 339–50.

Erickson, Timothy, and Toni M Whited. 2012. "Treating Measurement Error in Tobin's q." *The Review of Financial Studies* 25 (4): 1286–1329.

Errunza, Vihang, and Etienne Losq. 1985. "International Asset Pricing Under Mild Segmen-

tation: Theory and Test." *The Journal of Finance* 40 (1): 105–24.
Fairfield, Patricia M, J Scott Whisenant, and Teri Lombardi Yohn. 2003. "Accrued Earnings and Growth: Implications for Future Profitability and Market Mispricing." *The Accounting Review* 78 (1): 353–71.
Fama, Eugene F. 1984. "Forward and Spot Exchange Rates." *Journal of Monetary Economics* 14 (3): 319–38.
———. 1998. "Market Efficiency, Long-Term Returns, and Behavioral Finance." *Journal of Financial Economics* 49 (3): 283–306.
Fama, Eugene F, Lawrence Fisher, Michael C Jensen, and Richard Roll. 1969. "The Adjustment of Stock Prices to New Information." *International Economic Review* 10 (1): 1–21.
Fama, Eugene F., and Kenneth R. French. 1989. "Business conditions and expected returns on stocks and bonds." *Journal of Financial Economics* 25 (1): 23–49. https://doi.org/10.1016/0304-405X(89)90095-0.
———. 1992. "The cross-section of expected stock returns." *The Journal of Finance* 47 (2): 427–65. https://doi.org/2329112.
———. 1993a. "Common risk factors in the returns on stocks and bonds." *Journal of Financial Economics* 33 (1): 3–56. https://doi.org/10.1016/0304-405X(93)90023-5.
———. 1993b. "Common risk factors in the returns on stocks and bonds." *Journal of Financial Economics* 33 (1): 3–56. https://doi.org/10.1016/0304-405X(93)90023-5.
———. 2015. "A Five-Factor Asset Pricing Model." *Journal of Financial Economics* 116 (1): 1–22. https://doi.org/10.1016/j.jfineco.2014.10.010.
Fama, Eugene F, and Kenneth R French. 1996. "Multifactor Explanations of Asset Pricing Anomalies." *The Journal of Finance* 51 (1): 55–84.
———. 2006. "Profitability, Investment and Average Returns." *Journal of Financial Economics* 82 (3): 491–518.
———. 2008. "Dissecting Anomalies." *The Journal of Finance* 63 (4): 1653–78.
———. 2012. "Size, Value, and Momentum in International Stock Returns." *Journal of Financial Economics* 105 (3): 457–72.
———. 2020. "Comparing Cross-Section and Time-Series Factor Models." *The Review of Financial Studies* 33 (5): 1891–1926.
Fama, Eugene F., and James D. MacBeth. 1973a. "Risk, return, and equilibrium: Empirical tests." *Journal of Political Economy* 81 (3): 607–36. https://doi.org/10.1086/260061.
Fama, Eugene F, and James D MacBeth. 1973b. "Risk, Return, and Equilibrium: Empirical Tests." *Journal of Political Economy* 81 (3): 607–36.
Farre-Mensa, Joan, and Alexander Ljungqvist. 2016. "Do Measures of Financial Constraints Measure Financial Constraints?" *The Review of Financial Studies* 29 (2): 271–308.
Fazzari, Steven, R Glenn Hubbard, and Bruce C Petersen. 1987. "Financing Constraints and Corporate Investment." National Bureau of Economic Research Cambridge, Mass., USA.
Feng, Guanhao, Stefano Giglio, and Dacheng Xiu. 2020. "Taming the Factor Zoo: A Test of New Factors." *The Journal of Finance* 75 (3): 1327–70.
Feng, Guanhao, Wei Lan, Hansheng Wang, and Jun Zhang. 2026. "Selecting and Testing Asset-Pricing Models: A Stepwise Approach." *Management Science*. https://doi.org/10.1287/mnsc.2024.07804.

Flannery, Mark J, and Aris A Protopapadakis. 2002. "Macroeconomic Factors Do Influence Aggregate Stock Returns." *The Review of Financial Studies* 15 (3): 751–82.

Fong, Kingsley YL, Craig W Holden, and Charles A Trzcinka. 2017. "What Are the Best Liquidity Proxies for Global Research?" *Review of Finance* 21 (4): 1355–1401.

Forbes, Kristin J, and Roberto Rigobon. 2002. "No Contagion, Only Interdependence: Measuring Stock Market Comovements." *The Journal of Finance* 57 (5): 2223–61.

Foucault, Thierry. 1999. "Order Flow Composition and Trading Costs in a Dynamic Limit Order Market." *Journal of Financial Markets* 2 (2): 99–134.

Francis, Jennifer, Ryan LaFond, Per Olsson, and Katherine Schipper. 2005. "The Market Pricing of Accruals Quality." *Journal of Accounting and Economics* 39 (2): 295–327.

Frank, Murray Z, and Vidhan K Goyal. 2003. "Testing the Pecking Order Theory of Capital Structure." *Journal of Financial Economics* 67 (2): 217–48.

———. 2009. "Capital Structure Decisions: Which Factors Are Reliably Important?" *Financial Management* 38 (1): 1–37.

Frankel, Jeffrey A. 1979. "On the Mark: A Theory of Floating Exchange Rates Based on Real Interest Differentials." *The American Economic Review* 69 (4): 610–22.

Frankel, Richard, and Charles M. C. Lee. 1998. "Accounting Valuation, Market Expectation, and Cross-Sectional Stock Returns." *Journal of Accounting and Economics* 25 (3): 283–319. https://doi.org/10.1016/S0165-4101(98)00026-3.

Frazzini, Andrea, and Owen A. Lamont. 2008. "Dumb Money: Mutual Fund Flows and the Cross-Section of Stock Returns." *Journal of Financial Economics* 88 (2): 299–322. https://doi.org/10.1016/j.jfineco.2007.07.001.

Frazzini, Andrea, and Lasse Heje Pedersen. 2014. "Betting against beta." *Journal of Financial Economics* 111 (1): 1–25. https://doi.org/10.1016/j.jfineco.2013.10.005.

Gabaix, Xavier, Parameswaran Gopikrishnan, Vasiliki Plerou, and H Eugene Stanley. 2003. "A Theory of Power-Law Distributions in Financial Market Fluctuations." *Nature* 423 (6937): 267–70.

Gabaix, Xavier, and Ralph SJ Koijen. 2021. "In Search of the Origins of Financial Fluctuations: The Inelastic Markets Hypothesis." National Bureau of Economic Research.

Gala, Vito D., Joao F. Gomes, and Tong Liu. 2026. "Marginal Q." *The Journal of Finance* 81 (5): 2853–86. https://doi.org/10.1111/jofi.70074.

Garfinkel, Jon A. 2009. "Measuring Investors' Opinion Divergence." *Journal of Accounting Research* 47 (5): 1317–48.

Garfinkel, Jon A, and Jonathan Sokobin. 2006. "Volume, Opinion Divergence, and Returns: A Study of Post–Earnings Announcement Drift." *Journal of Accounting Research* 44 (1): 85–112.

Gârleanu, Nicolae, and Lasse Heje Pedersen. 2013. "Dynamic Trading with Predictable Returns and Transaction Costs." *The Journal of Finance* 68 (6): 2309–40.

Gentzkow, Matthew, and Jesse M Shapiro. 2014. "Code and Data for the Social Sciences: A Practitioner's Guide." Working Paper, University of Chicago.

Gibbons, Michael R, Stephen A Ross, and Jay Shanken. 1989. "A Test of the Efficiency of a Given Portfolio." *Econometrica: Journal of the Econometric Society*, 1121–52.

Givoly, Dan, and Dan Palmon. 1982. "Timeliness of Annual Earnings Announcements: Some

Empirical Evidence." *Accounting Review*, 486–508.

Glosten, Lawrence R, and Paul R Milgrom. 1985. "Bid, Ask and Transaction Prices in a Specialist Market with Heterogeneously Informed Traders." *Journal of Financial Economics* 14 (1): 71–100. https://doi.org/10.1016/0304-405X(85)90044-3.

Goldsmith-Pinkham, Paul, Isaac Sorkin, and Henry Swift. 2020. "Bartik Instruments: What, When, Why, and How." *American Economic Review* 110 (8): 2586–2624. https://doi.org/10.1257/aer.20181047.

Gompers, Paul, Joy Ishii, and Andrew Metrick. 2003. "Corporate Governance and Equity Prices." *The Quarterly Journal of Economics* 118 (1): 107–56. https://doi.org/10.1162/00335530360535162.

Goyal, Amit, and Narasimhan Jegadeesh. 2018. "Cross-Sectional and Time-Series Tests of Return Predictability: What Is the Difference?" *The Review of Financial Studies* 31 (5): 1784–824.

Goyenko, Ruslan Y, Craig W Holden, and Charles A Trzcinka. 2009. "Do Liquidity Measures Measure Liquidity?" *Journal of Financial Economics* 92 (2): 153–81.

Goyenko, Ruslan Y, and Andrey D Ukhov. 2009. "Stock and Bond Market Liquidity: A Long-Run Empirical Analysis." *Journal of Financial and Quantitative Analysis* 44 (1): 189–212.

Greenstone, Michael, Richard Hornbeck, and Enrico Moretti. 2010. "Identifying Agglomeration Spillovers: Evidence from Winners and Losers of Large Plant Openings." *Journal of Political Economy* 118 (3): 536–98. https://doi.org/10.1086/653714.

Greenwood, Robin, and David Thesmar. 2011. "Stock Price Fragility." *Journal of Financial Economics* 102 (3): 471–90. https://doi.org/10.1016/j.jfineco.2011.06.003.

Griffin, John M. 2002. "Are the Fama and French Factors Global or Country Specific?" *The Review of Financial Studies* 15 (3): 783–803.

Griffin, John M, Patrick J Kelly, and Federico Nardari. 2010. "Do Market Efficiency Measures Yield Correct Inferences? A Comparison of Developed and Emerging Markets." *The Review of Financial Studies* 23 (8): 3225–77.

Grinblatt, Mark, Sheridan Titman, and Russ Wermers. 1995. "Momentum Investment Strategies, Portfolio Performance, and Herding: A Study of Mutual Fund Behavior." *American Economic Review* 85 (5): 1088–1105.

Grootendorst, Maarten. 2022. "BERTopic: Neural Topic Modeling with a Class-Based TF-IDF Procedure." *arXiv Preprint arXiv:2203.05794*.

Grundy, Bruce D, and J Spencer Martin Martin. 2001. "Understanding the Nature of the Risks and the Source of the Rewards to Momentum Investing." *The Review of Financial Studies* 14 (1): 29–78.

Guay, Wayne, Delphine Samuels, and Daniel Taylor. 2016. "Guiding Through the Fog: Financial Statement Complexity and Voluntary Disclosure." *Journal of Accounting and Economics* 62 (2-3): 234–69.

Gürkaynak, Refet S, Brian Sack, and Jonathan H Wright. 2007. "The US Treasury Yield Curve: 1961 to the Present." *Journal of Monetary Economics* 54 (8): 2291–2304.

Gurrea-Martínez, Aurelio. 2024. *Reinventing Insolvency Law in Emerging Economies*. Cambridge: Cambridge University Press.

Hadlock, Charles J, and Joshua R Pierce. 2010. "New Evidence on Measuring Financial Constraints: Moving Beyond the KZ Index." *The Review of Financial Studies* 23 (5): 1909–40.

Hall, Peter. 1992. "On the Removal of Skewness by Transformation." *Journal of the Royal Statistical Society Series B: Statistical Methodology* 54 (1): 221–28.

Hameed, Allaudeen, Wenjin Kang, and Shivesh Viswanathan. 2010. "Stock Market Declines and Liquidity." *The Journal of Finance* 65 (1): 257–93.

Handa, Puneet, Robert Schwartz, and Ashish Tiwari. 2003. "Quote Setting and Price Formation in an Order Driven Market." *Journal of Financial Markets* 6 (4): 461–89.

Hansen, Lars Peter. 1982. "Large Sample Properties of Generalized Method of Moments Estimators." *Econometrica: Journal of the Econometric Society*, 1029–54.

Hansen, Peter R., and Asger Lunde. 2006. "Realized Variance and Market Microstructure Noise." *Journal of Business & Economic Statistics* 24 (2): 127–61. https://doi.org/10.1198/073500106000000071.

Harris, Milton, and Artur Raviv. 1993. "Differences of Opinion Make a Horse Race." *The Review of Financial Studies* 6 (3): 473–506.

Harvey, Campbell R., Yan Liu, and Heqing Zhu. 2016a. "... and the cross-section of expected returns." *Review of Financial Studies* 29 (1): 5–68. https://doi.org/10.1093/rfs/hhv059.

Harvey, Campbell R, Yan Liu, and Heqing Zhu. 2016b. "… and the Cross-Section of Expected Returns." *The Review of Financial Studies* 29 (1): 5–68.

Harvey, Campbell R, and Akhtar Siddique. 2000. "Conditional Skewness in Asset Pricing Tests." *The Journal of Finance* 55 (3): 1263–95.

Hasbrouck, Joel. 1991. "Measuring the Information Content of Stock Trades." *The Journal of Finance* 46 (1): 179–207.

———. 2007. *Empirical Market Microstructure: The Institutions, Economics, and Econometrics of Securities Trading*. Oxford University Press.

———. 2009. "Trading Costs and Returns for US Equities: Estimating Effective Costs from Daily Data." *The Journal of Finance* 64 (3): 1445–77. https://doi.org/10.1111/j.1540-6261.2009.01469.x.

Hasler, Mathias. 2021. "Is the Value Premium Smaller Than We Thought?" *Working Paper*. https://ssrn.com/abstract=3886984.

Hausman, Jerry. 2001. "Mismeasured Variables in Econometric Analysis: Problems from the Right and Problems from the Left." *Journal of Economic Perspectives* 15 (4): 57–67. https://doi.org/10.1257/jep.15.4.57.

Hayashi, Fumio. 1982. "Tobin's Marginal q and Average q: A Neoclassical Interpretation." *Econometrica: Journal of the Econometric Society*, 213–24.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. "Deep Residual Learning for Image Recognition." In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–78.

Healy, Paul M. 1985. "The Effect of Bonus Schemes on Accounting Decisions." *Journal of Accounting and Economics* 7 (1-3): 85–107.

Healy, Paul M, and Krishna G Palepu. 2001. "Information Asymmetry, Corporate Disclosure, and the Capital Markets: A Review of the Empirical Disclosure Literature." *Journal of*

Accounting and Economics 31 (1-3): 405–40.

Henderson, J Vernon, Tim Squires, Adam Storeygard, and David Weil. 2018. "The Global Distribution of Economic Activity: Nature, History, and the Role of Trade." *The Quarterly Journal of Economics* 133 (1): 357–406.

Henderson, J Vernon, Adam Storeygard, and David N Weil. 2012. "Measuring Economic Growth from Outer Space." *American Economic Review* 102 (2): 994–1028.

Henry, Peter Blair. 2000. "Stock Market Liberalization, Economic Reform, and Emerging Market Equity Prices." *The Journal of Finance* 55 (2): 529–64.

Hill, Bruce M. 1975. "A Simple General Approach to Inference about the Tail of a Distribution." *The Annals of Statistics*, 1163–74.

Hillion, Pierre, and Matti Suominen. 2004. "The Manipulation of Closing Prices." *Journal of Financial Markets* 7 (4): 351–75.

Hirshleifer, David, Sonya Seongyeon Lim, and Siew Hong Teoh. 2009. "Driven to Distraction: Extraneous Events and Underreaction to Earnings News." *The Journal of Finance* 64 (5): 2289–2325.

Hirshleifer, David, and Siew Hong Teoh. 2003. "Limited Attention, Information Disclosure, and Financial Reporting." *Journal of Accounting and Economics* 36 (1-3): 337–86.

Hoberg, Gerard, and Gordon Phillips. 2010. "Product Market Synergies and Competition in Mergers and Acquisitions: A Text-Based Analysis." *The Review of Financial Studies* 23 (10): 3773–3811.

———. 2016. "Text-Based Network Industries and Endogenous Product Differentiation." *Journal of Political Economy* 124 (5): 1423–65. https://doi.org/10.1086/688176.

Hoberg, Gerard, and Gordon M Phillips. 2018. "Text-Based Industry Momentum." *Journal of Financial and Quantitative Analysis* 53 (6): 2355–88.

Hodler, Roland, and Paul A Raschky. 2014. "Regional Favoritism." *The Quarterly Journal of Economics* 129 (2): 995–1033.

Hofstede, Geert. 2001. "Culture's Consequences: Comparing Values, Behaviors, Institutions and Organizations Across Nations." *International Educational and Professional*.

Hong, Harrison, Jeffrey D. Kubik, and Jeremy C. Stein. 2004. "Social Interaction and Stock-Market Participation." *The Journal of Finance* 59 (1): 137–63. https://doi.org/10.1111/j.1540-6261.2004.00629.x.

Hong, Harrison, and Jeremy C Stein. 1999. "A Unified Theory of Underreaction, Momentum Trading, and Overreaction in Asset Markets." *The Journal of Finance* 54 (6): 2143–84.

———. 2003. "Differences of Opinion, Short-Sales Constraints, and Market Crashes." *The Review of Financial Studies* 16 (2): 487–525.

Hope, Ole-Kristian. 2003. "Disclosure Practices, Enforcement of Accounting Standards, and Analysts' Forecast Accuracy: An International Study." *Journal of Accounting Research* 41 (2): 235–72.

Hotchkiss, Edith S. 1995. "Postbankruptcy Performance and Management Turnover." *The Journal of Finance* 50 (1): 3–21. https://doi.org/10.1111/j.1540-6261.1995.tb05165.x.

Hotchkiss, Edith S., Benjamin Iverson, and Xiang Zheng. 2026. "Can Small Businesses Survive Chapter 11?" *The Journal of Finance*.

Hou, Kewei, Mathijs A. van Dijk, and Yinglei Zhang. 2012. "The Implied Cost of Capital: A

New Approach." *Journal of Accounting and Economics* 53 (3): 504–26. https://doi.org/10.1016/j.jacceco.2011.12.001.

Hou, Kewei, Chen Xue, and Lu Zhang. 2014. "Digesting anomalies: An investment approach." *Review of Financial Studies* 28 (3): 650–705. https://doi.org/10.1093/rfs/hhu068.

———. 2015. "Digesting Anomalies: An Investment Approach." *The Review of Financial Studies* 28 (3): 650–705.

———. 2020b. "Replicating anomalies." *Review of Financial Studies* 33 (5): 2019–2133. https://doi.org/10.1093/rfs/hhy131.

———. 2020a. "Replicating Anomalies." *The Review of Financial Studies* 33 (5): 2019–2133.

Houge, Todd, Tim Loughran, Gerry Suchanek, and Xuemin Yan. 2001. "Divergence of Opinion, Uncertainty, and the Quality of Initial Public Offerings." *Financial Management*, 5–23.

Hribar, Paul, and Daniel W Collins. 2002. "Errors in Estimating Accruals: Implications for Empirical Research." *Journal of Accounting Research* 40 (1): 105–34.

Hsu, Jason C. 2006. "Cap-Weighted Portfolios Are Sub-Optimal Portfolios." *Journal of Investment Management* 4 (3).

Huang, Allen H, Reuven Lehavy, Amy Y Zang, and Rong Zheng. 2018. "Analyst Information Discovery and Interpretation Roles: A Topic Modeling Approach." *Management Science* 64 (6): 2833–55.

Huang, Allen H, Hui Wang, and Yi Yang. 2023. "FinBERT: A Large Language Model for Extracting Information from Financial Text." *Contemporary Accounting Research* 40 (2): 806–41.

Huang, Roger D, and Hans R Stoll. 1997. "The Components of the Bid-Ask Spread: A General Approach." *The Review of Financial Studies* 10 (4): 995–1034.

Huang, Xiangqian, Clark Liu, and Tao Shu. 2023. "Factors and Anomalies in the Vietnamese Stock Market." *Pacific-Basin Finance Journal* 82: 102176.

Huang, Yu, Chenzhuang Du, Zihui Xue, Xuanyao Chen, Hang Zhao, and Longbo Huang. 2021. "What Makes Multi-Modal Learning Better Than Single (Provably)." *Advances in Neural Information Processing Systems* 34: 10944–56.

Huang, Yupan, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. 2022. "Layoutlmv3: Pre-Training for Document Ai with Unified Text and Image Masking." In *Proceedings of the 30th ACM International Conference on Multimedia*, 4083–91.

Huergo, Elena, and Jordi Jaumandreu. 2004. "Firms' Age, Process Innovation and Productivity Growth." *International Journal of Industrial Organization* 22 (4): 541–59.

Hutton, Amy P, Alan J Marcus, and Hassan Tehranian. 2009. "Opaque Financial Reports, R2, and Crash Risk." *Journal of Financial Economics* 94 (1): 67–86.

Imbens, Guido W., and Thomas Lemieux. 2008. "Regression Discontinuity Designs: A Guide to Practice." *Journal of Econometrics* 142 (2): 615–35. https://doi.org/10.1016/j.jeconom.2007.05.001.

Iverson, Benjamin. 2018. "Get in Line: Chapter 11 Restructuring in Crowded Bankruptcy Courts." *Management Science* 64 (11): 5370–94. https://doi.org/10.1287/mnsc.2017.2808.

Jacobs, Heiko, and Sebastian Müller. 2020. "Anomalies Across the Globe: Once Public, No Longer Existent?" *Journal of Financial Economics* 135 (1): 213–30.

Jacod, Jean, Yingying Li, Per A. Mykland, Mark Podolskij, and Mathias Vetter. 2009. "Microstructure Noise in the Continuous Case: The Pre-Averaging Approach." *Stochastic Processes and Their Applications* 119 (7): 2249–76. https://doi.org/10.1016/j.spa.2008.11.004.

Jagannathan, Ravi, and Tongshu Ma. 2003. "Risk Reduction in Large Portfolios: Why Imposing the Wrong Constraints Helps." *The Journal of Finance* 58 (4): 1651–83.

Jagannathan, Ravi, and Zhenyu Wang. 1996a. "The conditional CAPM and the cross-section of expected returns." *The Journal of Finance* 51 (1): 3–53. https://doi.org/10.2307/2329301.

———. 1996b. "The Conditional CAPM and the Cross-Section of Expected Returns." *The Journal of Finance* 51 (1): 3–53.

Jean, Neal, Marshall Burke, Michael Xie, W Matthew Alampay Davis, David B Lobell, and Stefano Ermon. 2016. "Combining Satellite Imagery and Machine Learning to Predict Poverty." *Science* 353 (6301): 790–94.

Jegadeesh, Narasimhan. 1990. "Evidence of Predictable Behavior of Security Returns." *The Journal of Finance* 45 (3): 881–98.

Jegadeesh, Narasimhan, and Sheridan Titman. 1993. "Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency." *The Journal of Finance* 48 (1): 65–91.

Jensen, Michael C. 1986. "Agency Costs of Free Cash Flow, Corporate Finance, and Takeovers." *The American Economic Review* 76 (2): 323–29.

Jensen, Michael C, and William H Meckling. 1976. "Theory of the Firm: Managerial Behavior, Agency Costs and Ownership Structure." *Journal of Financial Economics* 3 (4): 305–60.

———. 2019. "Theory of the Firm: Managerial Behavior, Agency Costs and Ownership Structure." In *Corporate Governance*, 77–132. Gower.

Jensen, Michael C, and Richard S Ruback. 1983. "The Market for Corporate Control: The Scientific Evidence." *Journal of Financial Economics* 11 (1-4): 5–50.

Jensen, Theis Ingerslev, Bryan Kelly, Semyon Malamud, and Lasse Heje Pedersen. 2026. "Machine Learning and the Implementable Efficient Frontier." *The Review of Financial Studies* 39 (10): 3035–78. https://doi.org/10.1093/rfs/hhag022.

Jensen, Theis Ingerslev, Bryan Kelly, and Lasse Heje Pedersen. 2023. "Is There a Replication Crisis in Finance?" *The Journal of Finance* 78 (5): 2465–2518.

Jha, Manish, Jialin Qian, Michael Weber, and Baozhong Yang. 2024. "ChatGPT and Corporate Policies." National Bureau of Economic Research.

Johnson, Simon, Rafael La Porta, Florencio Lopez-de-Silanes, and Andrei Shleifer. 2000. "Tunneling." *American Economic Review* 90 (2): 22–27.

Johnson, Timothy C. 2002. "Rational Momentum Effects." *The Journal of Finance* 57 (2): 585–608.

Jones, Jennifer J. 1991. "Earnings Management During Import Relief Investigations." *Journal of Accounting Research* 29 (2): 193–228.

Jorion, Philippe. 1990. "The Exchange-Rate Exposure of US Multinationals." *Journal of Business*, 331–45.

———. 1991. "The Pricing of Exchange Rate Risk in the Stock Market." *Journal of Financial and Quantitative Analysis* 26 (3): 363–76. https://doi.org/10.2307/2331212.

Jung, Michael J, James P Naughton, Ahmed Tahoun, and Clare Wang. 2018. "Do Firms

Strategically Disseminate? Evidence from Corporate Use of Social Media." *The Accounting Review* 93 (4): 225–52.

Kacperczyk, Marcin, Clemens Sialm, and Lu Zheng. 2008. "Unobserved Actions of Mutual Funds." *The Review of Financial Studies* 21 (6): 2379–2416.

Kahneman, Daniel, and Amos Tversky. 2013. "Prospect Theory: An Analysis of Decision Under Risk." In *Handbook of the Fundamentals of Financial Decision Making: Part i*, 99–127. World Scientific.

Kaminsky, Graciela L, Carmen M Reinhart, and Carlos A Vegh. 2002. "The Unholy Trinity of Financial Contagion." *Journal of Economic Perspectives* 17 (4): 51–74.

Kandel, Eugene, and Neil D Pearson. 1995. "Differential Interpretation of Public Signals and Trade in Speculative Markets." *Journal of Political Economy* 103 (4): 831–72. https://doi.org/10.1086/262005.

Kaniel, Ron, Shuming Liu, Gideon Saar, and Sheridan Titman. 2012. "Individual Investor Trading and Return Patterns Around Earnings Announcements." *The Journal of Finance* 67 (2): 639–80.

Kaplan, Steven N, and Luigi Zingales. 1997. "Do Investment-Cash Flow Sensitivities Provide Useful Measures of Financing Constraints?" *The Quarterly Journal of Economics* 112 (1): 169–215.

Karolyi, G. Andrew, Kuan-Hui Lee, and Mathijs A. van Dijk. 2012. "Understanding Commonality in Liquidity Around the World." *Journal of Financial Economics* 105 (1): 82–112. https://doi.org/10.1016/j.jfineco.2011.12.008.

Karpoff, Jonathan M. 1987. "The Relation Between Price Changes and Trading Volume: A Survey." *Journal of Financial and Quantitative Analysis* 22 (1): 109–26.

Kelly, Bryan, and Hao Jiang. 2014. "Tail Risk and Asset Prices." *The Review of Financial Studies* 27 (10): 2841–71.

Kelly, Bryan, Semyon Malamud, and Kangying Zhou. 2024. "The Virtue of Complexity in Return Prediction." *The Journal of Finance* 79 (1): 459–503. https://doi.org/10.1111/jofi.13298.

Keung, Edmund, Zhi-Xing Lin, and Michael Shih. 2010. "Does the Stock Market See a Zero or Small Positive Earnings Surprise as a Red Flag?" *Journal of Accounting Research* 48 (1): 105–36. https://doi.org/10.1111/j.1475-679X.2009.00354.x.

Khan, Mozaffar. 2008. "Are Accruals Mispriced? Evidence from Tests of an Intertemporal Capital Asset Pricing Model." *Journal of Accounting and Economics* 45 (1): 55–77.

Khan, Mozaffar, Leonid Kogan, and George Serafeim. 2012. "Mutual Fund Trading Pressure: Firm-Level Stock Price Impact and Timing of SEOs." *The Journal of Finance* 67 (4): 1371–95. https://doi.org/10.1111/j.1540-6261.2012.01750.x.

Khanh, Hoang Thi Mai, and Vinh Khuong Nguyen. 2018. "Audit Quality, Firm Characteristics and Real Earnings Management: The Case of Listed Vietnamese Firms." *International Journal of Economics and Financial Issues* 8 (4): 243.

Khorrami, Paymon, and Fernando Mendo. 2026. "Dynamic Self-Fulfilling Fire Sales." *The Journal of Finance* 81 (5): 3087–3126. https://doi.org/10.1111/jofi.70069.

Kim, Geewook, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. 2022. "Ocr-Free

Document Understanding Transformer." In *European Conference on Computer Vision*, 498–517. Springer.

Kim, Kenneth A, Haixiao Liu, and J Jimmy Yang. 2013. "Reconsidering Price Limit Effectiveness." *Journal of Financial Research* 36 (4): 493–518.

Kim, Kenneth A, and S Ghon Rhee. 1997. "Price Limit Performance: Evidence from the Tokyo Stock Exchange." *The Journal of Finance* 52 (2): 885–901. https://doi.org/10.1111/j.1540-6261.1997.tb04827.x.

Kim, Oliver, and Robert E Verrecchia. 1991. "Market Reaction to Anticipated Announcements." *Journal of Financial Economics* 30 (2): 273–309.

King, Mervyn A, and Sushil Wadhwani. 1990. "Transmission of Volatility Between Stock Markets." *The Review of Financial Studies* 3 (1): 5–33.

Kipf, Thomas N, and Max Welling. 2016. "Semi-Supervised Classification with Graph Convolutional Networks." *arXiv Preprint arXiv:1609.02907*.

Kiyotaki, Nobuhiro, and John Moore. 1997. "Credit Cycles." *Journal of Political Economy* 105 (2): 211–48. https://doi.org/10.1086/262072.

Klapper, Leora F, and Inessa Love. 2004. "Corporate Governance, Investor Protection, and Performance in Emerging Markets." *Journal of Corporate Finance* 10 (5): 703–28.

Kodres, Laura E, and Matthew Pritsker. 2002. "A Rational Expectations Model of Financial Contagion." *The Journal of Finance* 57 (2): 769–99.

Koijen, Ralph SJ, and Motohiro Yogo. 2019. "A Demand System Approach to Asset Pricing." *Journal of Political Economy* 127 (4): 1475–515.

Kolari, James W, and Seppo Pynnönen. 2010. "Event Study Testing with Cross-Sectional Correlation of Abnormal Returns." *The Review of Financial Studies* 23 (11): 3996–4025.

Korajczyk, Robert A, and Ronnie Sadka. 2004. "Are Momentum Profits Robust to Trading Costs?" *The Journal of Finance* 59 (3): 1039–82.

Kothari, Sagar P, Andrew J Leone, and Charles E Wasley. 2005. "Performance Matched Discretionary Accrual Measures." *Journal of Accounting and Economics* 39 (1): 163–97.

Kothari, Sagar P, and Jerold B Warner. 2007. "Econometrics of Event Studies." In *Handbook of Empirical Corporate Finance*, 3–36. Elsevier.

Kothari, Sri Prakash. 2001. "Capital Markets Research in Accounting." *Journal of Accounting and Economics* 31 (1-3): 105–231.

Kraft, Arthur, Andrew J Leone, and Charles Wasley. 2006. "An Analysis of the Theories and Explanations Offered for the Mispricing of Accruals and Accrual Components." *Journal of Accounting Research* 44 (2): 297–339.

Kraus, Alan, and Robert H Litzenberger. 1976. "Skewness Preference and the Valuation of Risk Assets." *The Journal of Finance* 31 (4): 1085–1100.

Krishnamurthy, Arvind, and Annette Vissing-Jorgensen. 2012. "The Aggregate Demand for Treasury Debt." *Journal of Political Economy* 120 (2): 233–67.

Kuchler, Theresa, Yan Li, Lin Peng, Johannes Stroebel, and Dexin Zhou. 2022. "Social Proximity to Capital: Implications for Investors and Firms." *The Review of Financial Studies* 35 (6): 2743–89. https://doi.org/10.1093/rfs/hhab111.

Kuchler, Theresa, and Johannes Stroebel. 2021. "Social Finance." *Annual Review of Financial Economics* 13 (1): 37–55. https://doi.org/10.1146/annurev-financial-101320-062446.

Kupiec, Paul H et al. 1995. "Techniques for Verifying the Accuracy of Risk Measurement Models."

Kyle, Albert S. 1985. "Continuous Auctions and Insider Trading." *Econometrica: Journal of the Econometric Society* 53 (6): 1315–35. https://doi.org/10.2307/1913210.

La Porta, Rafael, Florencio Lopez-de-Silanes, Andrei Shleifer, and Robert Vishny. 2000a. "Investor Protection and Corporate Governance." *Journal of Financial Economics* 58 (1-2): 3–27.

La Porta, Rafael, Florencio Lopez-de-Silanes, Andrei Shleifer, and Robert W Vishny. 2000b. "Agency Problems and Dividend Policies Around the World." *The Journal of Finance* 55 (1): 1–33.

Lacava, Danilo, Angelo Ranaldo, and Paolo Santucci de Magistris. 2026. "Realized Illiquidity." *Management Science*. https://doi.org/10.1287/mnsc.2023.02505.

Lamont, Owen, Christopher Polk, and Jesús Saaá-Requejo. 2001. "Financial Constraints and Stock Returns." *The Review of Financial Studies* 14 (2): 529–54.

Landsman, Wayne R, Edward L Maydew, and Jacob R Thornock. 2012. "The Information Content of Annual Earnings Announcements and Mandatory Adoption of IFRS." *Journal of Accounting and Economics* 53 (1-2): 34–54.

Lang, Mark, Karl V Lins, and Mark Maffett. 2012. "Transparency, Liquidity, and Valuation: International Evidence on When Transparency Matters Most." *Journal of Accounting Research* 50 (3): 729–74.

Lang, Mark, and Russell Lundholm. 1993. "Cross-Sectional Determinants of Analyst Ratings of Corporate Disclosures." *Journal of Accounting Research* 31 (2): 246–71.

Lau, Jey Han, and Timothy Baldwin. 2016. "An Empirical Evaluation of Doc2vec with Practical Insights into Document Embedding Generation." *arXiv Preprint arXiv:1607.05368*.

Le, Quoc, and Tomas Mikolov. 2014. "Distributed Representations of Sentences and Documents." In *International Conference on Machine Learning*, 1188–96. PMLR.

Ledoit, Olivier, and Michael Wolf. 2004. "A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices." *Journal of Multivariate Analysis* 88 (2): 365–411.

Lee, Charles M. C., Eric C. So, and Charles C. Y. Wang. 2021. "Evaluating Firm-Level Expected-Return Proxies: Implications for Estimating Treatment Effects." *The Review of Financial Studies* 34 (4): 1907–51. https://doi.org/10.1093/rfs/hhaa066.

Lee, David S., and Thomas Lemieux. 2010. "Regression Discontinuity Designs in Economics." *Journal of Economic Literature* 48 (2): 281–355. https://doi.org/10.1257/jel.48.2.281.

Lehavy, Reuven, and Richard G Sloan. 2008. "Investor Recognition and Stock Returns." *Review of Accounting Studies* 13 (2): 327–61.

Leibowitz, Martin L. 2002. "The Levered p/e Ratio." *Financial Analysts Journal* 58 (6): 68–77.

Lerman, Alina, Thomas D. Steffen, and Kangkang Zhang. 2026. "Earnings Conference Calls and the SEC Comment Letter Process." *Management Science* 72 (10): 8571–8604. https://doi.org/10.1287/mnsc.2023.02065.

Lesmond, David A. 2005. "Liquidity of Emerging Markets." *Journal of Financial Economics* 77 (2): 411–52.

Lesmond, David A, Joseph P Ogden, and Charles A Trzcinka. 1999. "A New Estimate of

Transaction Costs." *The Review of Financial Studies* 12 (5): 1113–41.
Lesmond, David A, Michael J Schill, and Chunsheng Zhou. 2004. "The Illusory Nature of Momentum Profits." *Journal of Financial Economics* 71 (2): 349–80.
Leuz, Christian, Dhananjay Nanda, and Peter D Wysocki. 2003. "Earnings Management and Investor Protection: An International Comparison." *Journal of Financial Economics* 69 (3): 505–27.
Lewellen, Jonathan, Stefan Nagel, and Jay Shanken. 2010. "A Skeptical Appraisal of Asset Pricing Tests." *Journal of Financial Economics* 96 (2): 175–94.
Li, Feng. 2008. "Annual Report Readability, Current Earnings, and Earnings Persistence." *Journal of Accounting and Economics* 45 (2-3): 221–47.
Li, Feng et al. 2010. "Textual Analysis of Corporate Disclosures: A Survey of the Literature." *Journal of Accounting Literature* 29 (1): 143–65.
Li, James J., and Lu Zheng. 2025. "Measuring Mutual Fund Flows." *Financial Analysts Journal* 81 (3): 27–59. https://doi.org/10.1080/0015198X.2025.2521233.
Li, Junnan, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. "Blip-2: Bootstrapping Language-Image Pre-Training with Frozen Image Encoders and Large Language Models." In *International Conference on Machine Learning*, 19730–42. PMLR.
Liang, Paul Pu, Amir Zadeh, and Louis-Philippe Morency. 2024. "Foundations & Trends in Multimodal Machine Learning: Principles, Challenges, and Open Questions." *ACM Computing Surveys* 56 (10): 1–42.
Lin, Mingfeng, Nagpurnanand R. Prabhala, and Siva Viswanathan. 2013. "Judging Borrowers by the Company They Keep: Friendship Networks and Information Asymmetry in Online Peer-to-Peer Lending." *Management Science* 59 (1): 17–35. https://doi.org/10.1287/mnsc.1120.1560.
Lindenberg, Eric B, and Stephen A Ross. 1981. "Tobin's q Ratio and Industrial Organization." *Journal of Business*, 1–32.
Lintner, John. 1956. "Distribution of Incomes of Corporations Among Dividends, Retained Earnings, and Taxes." *The American Economic Review* 46 (2): 97–113.
———. 1965. "Security prices, risk, and maximal gains from diversification." *The Journal of Finance* 20 (4): 587–615. https://doi.org/10.1111/j.1540-6261.1965.tb02930.x.
Little, Roderick JA, and Donald B Rubin. 2019. *Statistical Analysis with Missing Data*. John Wiley & Sons.
Liu, Crocker H., Stuart S. Rosenthal, and William C. Strange. 2018. "The Vertical City: Rent Gradients, Spatial Structure, and Agglomeration Economies." *Journal of Urban Economics* 106: 101–22. https://doi.org/10.1016/j.jue.2018.04.001.
Liu, Crocker H., Chen Zheng, and Bing Zhu. 2026. "Does Agglomeration Enhance Property Value?" *Management Science* 72 (10): 8366–93. https://doi.org/10.1287/mnsc.2023.02773.
Liu, Jiageng, Igor Makarov, and Antoinette Schoar. 2026. "Anatomy of a Run: The Terra Luna Crash." *Journal of Financial Economics* 184: 104359. https://doi.org/10.1016/j.jfineco.2026.104359.
Liu, Laura Xiaolei, Toni M Whited, and Lu Zhang. 2009. "Investment-Based Expected Stock Returns." *Journal of Political Economy* 117 (6): 1105–39.
Liu, Lily Y., Andrew J. Patton, and Kevin Sheppard. 2015. "Does Anything Beat 5-Minute

RV? A Comparison of Realized Measures Across Multiple Asset Classes." *Journal of Econometrics* 187 (1): 293–311. https://doi.org/10.1016/j.jeconom.2015.02.008.

Livnat, Joshua, and Richard R Mendenhall. 2006. "Comparing the Post–Earnings Announcement Drift for Surprises Calculated from Analyst and Time Series Forecasts." *Journal of Accounting Research* 44 (1): 177–205.

Lo, Andrew W, and A Craig MacKinlay. 1990. "An Econometric Analysis of Nonsynchronous Trading." *Journal of Econometrics* 45 (1-2): 181–211.

Longin, Francois, and Bruno Solnik. 2001. "Extreme Correlation of International Equity Markets." *The Journal of Finance* 56 (2): 649–76. https://doi.org/10.1111/0022-1082.00340.

Lou, Dong. 2012. "A Flow-Based Explanation for Return Predictability." *The Review of Financial Studies* 25 (12): 3457–89. https://doi.org/10.1093/rfs/hhs103.

Lou, Xiaoxia, and Tao Shu. 2017. "Price Impact or Trading Volume: Why Is the Amihud (2002) Measure Priced?" *The Review of Financial Studies* 30 (12): 4481–4520. https://doi.org/10.1093/rfs/hhx072.

Lou, Xiaoxia, and Albert Y. Wang. 2018. "Flow-Induced Trading Pressure and Corporate Investment." *Journal of Financial and Quantitative Analysis* 53 (1): 171–201. https://doi.org/10.1017/S0022109017000953.

Loughran, Tim, and Bill McDonald. 2011. "When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks." *The Journal of Finance* 66 (1): 35–65.

———. 2014. "Measuring Readability in Financial Disclosures." *The Journal of Finance* 69 (4): 1643–71.

Lustig, Hanno, and Adrien Verdelhan. 2007. "The Cross Section of Foreign Currency Risk Premia and Consumption Growth Risk." *American Economic Review* 97 (1): 89–117.

Lyon, John D, Brad M Barber, and Chih-Ling Tsai. 1999. "Improved Methods for Tests of Long-Run Abnormal Stock Returns." *The Journal of Finance* 54 (1): 165–201.

MacKinlay, A Craig. 1997. "Event Studies in Economics and Finance." *Journal of Economic Literature* 35 (1): 13–39.

Maillard, Sébastien, Thierry Roncalli, and Jérôme Teïletche. 2010. "The Properties of Equally Weighted Risk Contribution Portfolios." *Journal of Portfolio Management* 36 (4): 60.

Mandelbrot, Benoit et al. 1963. "The Variation of Certain Speculative Prices." *Journal of Business* 36 (4): 394.

Markowitz, Harry. 1952. "Portfolio selection." *The Journal of Finance* 7 (1): 77–91. https://doi.org/10.1111/j.1540-6261.1952.tb01525.x.

McCrary, Justin. 2008. "Manipulation of the Running Variable in the Regression Discontinuity Design: A Density Test." *Journal of Econometrics* 142 (2): 698–714. https://doi.org/10.1016/j.jeconom.2007.05.005.

McLean, R David, and Jeffrey Pontiff. 2016. "Does Academic Research Destroy Stock Return Predictability?" *The Journal of Finance* 71 (1): 5–32.

McNeil, Alexander J, and Rüdiger Frey. 2000. "Estimation of Tail-Related Risk Measures for Heteroscedastic Financial Time Series: An Extreme Value Approach." *Journal of Empirical Finance* 7 (3-4): 271–300.

McNeil, Alexander J, Rüdiger Frey, and Paul Embrechts. 2015. *Quantitative Risk Manage-*

ment: Concepts, Techniques and Tools-Revised Edition. Princeton university press.

Meese, Richard A, and Kenneth Rogoff. 1983. "Empirical Exchange Rate Models of the Seventies: Do They Fit Out of Sample?" *Journal of International Economics* 14 (1-2): 3–24.

Menkhoff, Lukas, Lucio Sarno, Maik Schmeling, and Andreas Schrimpf. 2012. "Carry Trades and Global Foreign Exchange Volatility." *The Journal of Finance* 67 (2): 681–718.

Menkveld, Albert J., Anna Dreber, Felix Holzmeister, Juergen Huber, Magnus Johannesson, Michael Kirchler, Sebastian Neusüss, Michael Razen, and Utz Weitzel. n.d. "Nonstandard errors." *The Journal of Finance* 79 (3): 2339–90. https://doi.org/https://doi.org/10.1111/jofi.13337.

Menzly, Lior, and Oguzhan Ozbas. 2010. "Market Segmentation and Cross-Predictability of Returns." *The Journal of Finance* 65 (4): 1555–80.

Milgrom, Paul, and Nancy Stokey. 1982. "Information, Trade and Common Knowledge." *Journal of Economic Theory* 26 (1): 17–27.

Miller, Edward M. 1977. "Risk, Uncertainty, and Divergence of Opinion." *The Journal of Finance* 32 (4): 1151–68.

Miller, Merton H, and Kevin Rock. 1985. "Dividend Policy Under Asymmetric Information." *The Journal of Finance* 40 (4): 1031–51.

Mishkin, Frederic S. 1983. "Are Market Forecasts Rational?" In *A Rational Expectations Approach to Macroeconometrics: Testing Policy Ineffectiveness and Efficient-Markets Models*, 59–75. University of Chicago Press.

———. 2007. *A Rational Expectations Approach to Macroeconometrics: Testing Policy Ineffectiveness and Efficient-Markets Models*. University of Chicago Press.

Mitchell, Mark L, and Jeffry M Netter. 1993. "The Role of Financial Economics in Securities Fraud Cases: Applications at the Securities and Exchange Commission." *Bus. Law.* 49: 545.

Mitchell, Mark L, and Erik Stafford. 2000. "Managerial Decisions and Long-Term Stock Price Performance." *The Journal of Business* 73 (3): 287–329.

Morck, Randall, Andrei Shleifer, and Robert W Vishny. 1988. "Management Ownership and Market Valuation: An Empirical Analysis." *Journal of Financial Economics* 20: 293–315.

Morrison, Edward R. 2007. "Bankruptcy Decision Making: An Empirical Study of Continuation Bias in Small-Business Bankruptcies." *The Journal of Law and Economics* 50 (2): 381–419. https://doi.org/10.1086/511318.

Mossin, Jan. 1966. "Equilibrium in a capital asset market." *Econometrica* 34 (4): 768–83. https://doi.org/10.2307/1910098.

Myers, Stewart C. 1984. "The Capital Structure Puzzle." *Journal of Finance* 39 (3): 575–92.

Nelson, Charles R, and Andrew F Siegel. 1987. "Parsimonious Modeling of Yield Curves." *Journal of Business*, 473–89.

Newey, Whitney K., and Kenneth D. West. 1987a. "A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance Matrix." *Econometrica* 55 (3): 703–8. http://www.jstor.org/stable/1913610.

Newey, Whitney K, and Kenneth D West. 1987b. "A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation." *Econometrica* 55 (3): 703–8.

Nguyen, Dat Quoc, and Anh-Tuan Nguyen. 2020. "PhoBERT: Pre-Trained Language Models for Vietnamese." In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1037–42.

Nguyen, Du D, and Minh C Pham. 2018. "Search-Based Sentiment and Stock Market Reactions: An Empirical Evidence in Vietnam." *The Journal of Asian Finance, Economics and Business* 5 (4): 45–56.

Novy-Marx, Robert. 2013. "The Other Side of Value: The Gross Profitability Premium." *Journal of Financial Economics* 108 (1): 1–28.

Obaid, Khaled, and Kuntara Pukthuanthong. 2022. "A Picture Is Worth a Thousand Words: Measuring Investor Sentiment by Combining Machine Learning and Photos from News." *Journal of Financial Economics* 144 (1): 273–97.

Obstfeld, Maurice, Jay C Shambaugh, and Alan M Taylor. 2005. "The Trilemma in History: Tradeoffs Among Exchange Rates, Monetary Policies, and Capital Mobility." *Review of Economics and Statistics* 87 (3): 423–38.

Olley, G. Steven, and Ariel Pakes. 1996. "The Dynamics of Productivity in the Telecommunications Equipment Industry." *Econometrica* 64 (6): 1263–97. https://doi.org/10.2307/2171831.

Oord, Aaron van den, Yazhe Li, and Oriol Vinyals. 2018. "Representation Learning with Contrastive Predictive Coding." *arXiv Preprint arXiv:1807.03748*.

Parlour, Christine A. 1998. "Price Dynamics in Limit Order Markets." *The Review of Financial Studies* 11 (4): 789–816.

Pástor, L'uboš, and Robert F Stambaugh. 2003. "Liquidity Risk and Expected Stock Returns." *Journal of Political Economy* 111 (3): 642–85. https://doi.org/10.1086/374184.

Patell, James M. 1976. "Corporate Forecasts of Earnings Per Share and Stock Price Behavior: Empirical Test." *Journal of Accounting Research*, 246–76.

Patell, James M, and Mark A Wolfson. 1982. "Good News, Bad News, and the Intraday Timing of Corporate Disclosures." *Accounting Review*, 509–27.

Peng, Roger D. 2011. "Reproducible Research in Computational Science." *Science* 334 (6060): 1226–27.

Pesaran, M Hashem, and Takashi Yamagata. 2024. "Testing for Alpha in Linear Factor Pricing Models with a Large Number of Securities." *Journal of Financial Econometrics* 22 (2): 407–60.

Peters, Ryan H., and Lucian A. Taylor. 2017. "Intangible Capital and the Investment-q Relation." *Journal of Financial Economics* 123 (2): 251–72. https://doi.org/10.1016/j.jfineco.2016.03.011.

Phan, Thi Nha Truc, Philippe Bertrand, Hong Hai Phan, and Xuan Vinh Vo. 2023. "The Role of Investor Behavior in Emerging Stock Markets: Evidence from Vietnam." *The Quarterly Review of Economics and Finance* 87: 367–76.

Phung, Duc Nam, and Anil V Mishra. 2016. "Ownership Structure and Firm Performance: Evidence from Vietnamese Listed Firms." *Australian Economic Papers* 55 (1): 63–98.

Plyakha, Yuliya, Raman Uppal, and Grigory Vilkov. 2021. "Equal or Value Weighting? Implications for Asset-Pricing Tests." In *Financial Risk Management and Modeling*, 295–347. Springer.

Pontiff, Jeffrey, and Artemiza Woodgate. 2008. "Share Issuance and Cross-Sectional Returns." *The Journal of Finance* 63 (2): 921–45.

Pool, Veronika K, Clemens Sialm, and Irina Stefanescu. 2016. "It Pays to Set the Menu: Mutual Fund Investment Options in 401(k) Plans." *The Journal of Finance* 71 (4): 1779–1812.

———. 2026. "Mutual Fund Revenue Sharing in 401(k) Plans." *Management Science* 72 (7): 5698–5722. https://doi.org/10.1287/mnsc.2023.01560.

Pukthuanthong, Kuntara, and Richard Roll. 2009. "Global Market Integration: An Alternative Measure and Its Application." *Journal of Financial Economics* 94 (2): 214–32.

Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, et al. 2021. "Learning Transferable Visual Models from Natural Language Supervision." In *International Conference on Machine Learning*, 139:8748–63. Proceedings of Machine Learning Research. PMLR.

Rajan, Raghuram G, and Luigi Zingales. 1998. "Financial Dependence and Growth." *American Economic Review*, 559–86.

Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-Bert: Sentence Embeddings Using Siamese Bert-Networks." *arXiv Preprint arXiv:1908.10084*.

Rock, Kevin. 1986. "Why New Issues Are Underpriced." *Journal of Financial Economics* 15 (1-2): 187–212.

Roll, Richard. 1984. "A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market." *The Journal of Finance* 39 (4): 1127–39. https://doi.org/10.1111/j.1540-6261.1984.tb03897.x.

Romano, Joseph P, and Michael Wolf. 2005. "Exact and Approximate Stepdown Methods for Multiple Hypothesis Testing." *Journal of the American Statistical Association* 100 (469): 94–108.

Rosenthal, Stuart S., and William C. Strange. 2004. "Evidence on the Nature and Sources of Agglomeration Economies." In *Handbook of Regional and Urban Economics*, edited by J. Vernon Henderson and Jacques-François Thisse, 4:2119–71. Elsevier. https://doi.org/10.1016/S1574-0080(04)80006-3.

Rossi, Barbara. 2013. "Exchange Rate Predictability." *Journal of Economic Literature* 51 (4): 1063–1119.

Rossi, Emanuele, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. "Temporal Graph Networks for Deep Learning on Dynamic Graphs." *arXiv Preprint arXiv:2006.10637*.

Roşu, Ioanid. 2009. "A Dynamic Model of the Limit Order Book." *The Review of Financial Studies* 22 (11): 4601–41.

Rouwenhorst, K Geert. 1998. "International Momentum Strategies." *The Journal of Finance* 53 (1): 267–84.

Roychowdhury, Sugata. 2006. "Earnings Management Through Real Activities Manipulation." *Journal of Accounting and Economics* 42 (3): 335–70.

Rubin, Donald B. 1976. "Inference and Missing Data." *Biometrika* 63 (3): 581–92.

Rust, John. 1987. "Optimal Replacement of GMC Bus Engines: An Empirical Model of Harold Zurcher." *Econometrica: Journal of the Econometric Society*, 999–1033.

Salkever, David S. 1976. "The Use of Dummy Variables to Compute Predictions, Prediction Errors, and Confidence Intervals." *Journal of Econometrics* 4 (4): 393–97.
Sandve, Geir Kjetil, Anton Nekrutenko, James Taylor, and Eivind Hovig. 2013. "Ten Simple Rules for Reproducible Computational Research." *PLoS Computational Biology* 9 (10): e1003285.
Scheinkman, Jose A, and Wei Xiong. 2003. "Overconfidence and Speculative Bubbles." *Journal of Political Economy* 111 (6): 1183–1220.
Scheuch, Christoph, Stefan Voigt, and Patrick Weiss. 2023. *Tidy Finance with r*. CRC Press.
Scheuch, Christoph, Stefan Voigt, Patrick Weiss, and Christoph Frey. 2024. *Tidy Finance with Python*. Chapman; Hall/CRC.
Schoenherr, David, and Jan Starmans. 2022. "When Should Bankruptcy Law Be Creditor- or Debtor-Friendly? Theory and Evidence." *The Journal of Finance* 77 (5): 2669–2717. https://doi.org/10.1111/jofi.13171.
Scholes, Myron, and Joseph Williams. 1977. "Estimating Betas from Nonsynchronous Data." *Journal of Financial Economics* 5 (3): 309–27. https://doi.org/10.1016/0304-405X(77)90041-1.
Schwert, G William. 1981. "Using Financial Data to Measure Effects of Regulation." *The Journal of Law and Economics* 24 (1): 121–58.
Sengupta, Partha. 1998. "Corporate Disclosure Quality and the Cost of Debt." *Accounting Review*, 459–74.
Shalen, Catherine T. 1993. "Volume, Volatility, and the Dispersion of Beliefs." *The Review of Financial Studies* 6 (2): 405–34.
Shanken, Jay. 1992. "On the Estimation of Beta-Pricing Models." *The Review of Financial Studies* 5 (1): 1–33.
Sharpe, William F. 1964. "Capital asset prices: A theory of market equilibrium under conditions of risk ." *The Journal of Finance* 19 (3): 425–42. https://doi.org/10.1111/j.1540-6261.1964.tb02865.x.
Shleifer, Andrei, and Robert W. Vishny. 1992. "Liquidation Values and Debt Capacity: A Market Equilibrium Approach." *The Journal of Finance* 47 (4): 1343–66. https://doi.org/10.1111/j.1540-6261.1992.tb04661.x.
Shumway, Tyler. 1997. "The Delisting Bias in CRSP Data." *The Journal of Finance* 52 (1): 327–40. https://doi.org/10.1111/j.1540-6261.1997.tb03818.x.
Shyam-Sunder, Lakshmi, and Stewart C Myers. 1999. "Testing Static Tradeoff Against Pecking Order Models of Capital Structure." *Journal of Financial Economics* 51 (2): 219–44.
Sialm, Clemens, Laura T Starks, and Hanjiang Zhang. 2015. "Defined Contribution Pension Plans: Sticky or Discerning Money?" *The Journal of Finance* 70 (2): 805–38.
Sias, Richard W. 2004. "Institutional Herding." *The Review of Financial Studies* 17 (1): 165–206.
Silva, Tim de, and David Thesmar. 2024. "Noise in Expectations: Evidence from Analyst Forecasts." *The Review of Financial Studies* 37 (5): 1494–1537. https://doi.org/10.1093/rfs/hhad091.
Sirri, Erik R, and Peter Tufano. 1998. "Costly Search and Mutual Fund Flows." *The Journal of Finance* 53 (5): 1589–1622.

Sloan, Richard G. 1996. "Do Stock Prices Fully Reflect Information in Accruals and Cash Flows about Future Earnings?" *Accounting Review*, 289–315.

So, Eric C. 2013. "A New Approach to Predicting Analyst Forecast Errors: Do Investors Overweight Analyst Forecasts?" *Journal of Financial Economics* 108 (3): 615–40. https://doi.org/10.1016/j.jfineco.2013.02.002.

Soebhag, Amar, Bart Van Vliet, and Patrick Verwijmeren. 2022. "Mind Your Sorts." *Working Paper*. https://di.org/10.2139/ssrn.4136672.

Storey, John D. 2003. "The Positive False Discovery Rate: A Bayesian Interpretation and the q-Value." *The Annals of Statistics* 31 (6): 2013–35.

Subrahmanyam, Avanidhar. 1994. "Circuit Breakers and Market Volatility: A Theoretical Perspective." *The Journal of Finance* 49 (1): 237–54.

Svensson, Lars EO. 1994. "Estimating and Interpreting Forward Interest Rates: Sweden 1992-1994." National bureau of economic research Cambridge, Mass., USA.

Taleb, Nassim Nicholas. 2010. *The Black Swan:: The Impact of the Highly Improbable: With a New Section:" on Robustness and Fragility"*. Vol. 2. Random house trade paperbacks.

Tan, Mingxing, and Quoc Le. 2019. "Efficientnet: Rethinking Model Scaling for Convolutional Neural Networks." In *International Conference on Machine Learning*, 6105–14. PMLR.

Tetlock, Paul C. 2007. "Giving Content to Investor Sentiment: The Role of Media in the Stock Market." *The Journal of Finance* 62 (3): 1139–68.

Tetlock, Paul C, Maytal Saar-Tsechansky, and Sofus Macskassy. 2008. "More Than Words: Quantifying Language to Measure Firms' Fundamentals." *The Journal of Finance* 63 (3): 1437–67.

Titman, Sheridan, KC John Wei, and Feixue Xie. 2004. "Capital Investments and Stock Returns." *Journal of Financial and Quantitative Analysis* 39 (4): 677–700.

Tobin, James. 1969. "A General Equilibrium Approach to Monetary Theory." *Journal of Money, Credit and Banking* 1 (1): 15–29.

Varian, Hal R. 1985. "Divergence of Opinion in Complete Markets: A Note." *The Journal of Finance* 40 (1): 309–17.

Veličković, Petar, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. "Graph Attention Networks." *arXiv Preprint arXiv:1710.10903*.

Verrecchia, Robert E. 1983. "Discretionary Disclosure." *Journal of Accounting and Economics* 5: 179–94.

———. 2001. "Essays on Disclosure." *Journal of Accounting and Economics* 32 (1-3): 97–180.

Vilhuber, Lars. 2020. "Reproducibility and Replicability in Economics." *Harvard Data Science Review* 2 (4): 1–39.

Vo, Duc Hong, and Bao Doan. 2023. "Minimum Tick Size, Market Quality and Costs of Trade Execution in Vietnam." *Plos One* 18 (5): e0285821.

Vo, Xuan Vinh. 2015. "Foreign Ownership and Stock Return Volatility–Evidence from Vietnam." *Journal of Multinational Financial Management* 30: 101–9.

———. 2017. "Do Foreign Investors Improve Stock Price Informativeness in Emerging Equity Markets? Evidence from Vietnam." *Research in International Business and Finance* 42: 986–91.

Vo, Xuan Vinh, and Dang Bao Anh Phan. 2017. "Further Evidence on the Herd Behavior in

Vietnam Stock Market." *Journal of Behavioral and Experimental Finance* 13: 33–41.

Vu, Thanh, Dat Quoc Nguyen, Mark Dras, Mark Johnson, et al. 2018. "VnCoreNLP: A Vietnamese Natural Language Processing Toolkit." In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, 56–60.

Walter, Dominik, Rüdiger Weber, and Patrick Weiss. 2022. "Non-standard errors in portfolio sorts." *Working Paper*. https://ssrn.com/abstract=4164117.

Wardlaw, Malcolm. 2020. "Measuring Mutual Fund Flow Pressure as Shock to Stock Returns." *The Journal of Finance* 75 (6): 3221–43. https://doi.org/10.1111/jofi.12962.

Warner, Jerold B, Ross L Watts, and Karen H Wruck. 1988. "Stock Prices and Top Management Changes." *Journal of Financial Economics* 20: 461–92.

Wei, Xinbei, and Qunzi Zhang. 2026. "Narrative Ambiguity Matters." *Management Science*. https://doi.org/10.1287/mnsc.2025.02735.

Wermers, Russ. 2000. "Mutual Fund Performance: An Empirical Decomposition into Stock-Picking Talent, Style, Transactions Costs, and Expenses." *The Journal of Finance* 55 (4): 1655–95.

White, Halbert. 2000. "A Reality Check for Data Snooping." *Econometrica* 68 (5): 1097–1126.

Whited, Toni M, and Guojun Wu. 2006. "Financial Constraints Risk." *The Review of Financial Studies* 19 (2): 531–59.

Yan, Xuemin Sterling. 2008. "Liquidity, Investment Style, and the Relation Between Fund Size and Fund Performance." *Journal of Financial and Quantitative Analysis* 43 (3): 741–67.

Yang, Dennis, and Qiang Zhang. 2000. "Drift-Independent Volatility Estimation Based on High, Low, Open, and Close Prices." *The Journal of Business* 73 (3): 477–92.

Yang, Yi, Yixuan Tang, Yangyang Fan, and Kunpeng Zhang. 2026. "Learning from Earnings Calls: Graph-Based Conversational Modeling for Financial Prediction." *Information Systems Research* 37 (3): 1681–1705. https://doi.org/10.1287/isre.2023.0519.

Yin, Xiao. 2026. "Learning in the Limit: Income Inference from Credit Extensions." *The Journal of Finance* 81 (5): 2729–72. https://doi.org/10.1111/jofi.70040.

Young, Michael N, Mike W Peng, David Ahlstrom, Garry D Bruton, and Yi Jiang. 2008. "Corporate Governance in Emerging Economies: A Review of the Principal–Principal Perspective." *Journal of Management Studies* 45 (1): 196–220.

Zang, Amy Y. 2012. "Evidence on the Trade-Off Between Real Activities Manipulation and Accrual-Based Earnings Management." *The Accounting Review* 87 (2): 675–703.

Zhang, Lan, Per A. Mykland, and Yacine Aït-Sahalia. 2005. "A Tale of Two Time Scales: Determining Integrated Volatility with Noisy High-Frequency Data." *Journal of the American Statistical Association* 100 (472): 1394–1411. https://doi.org/10.1198/016214505000000169.